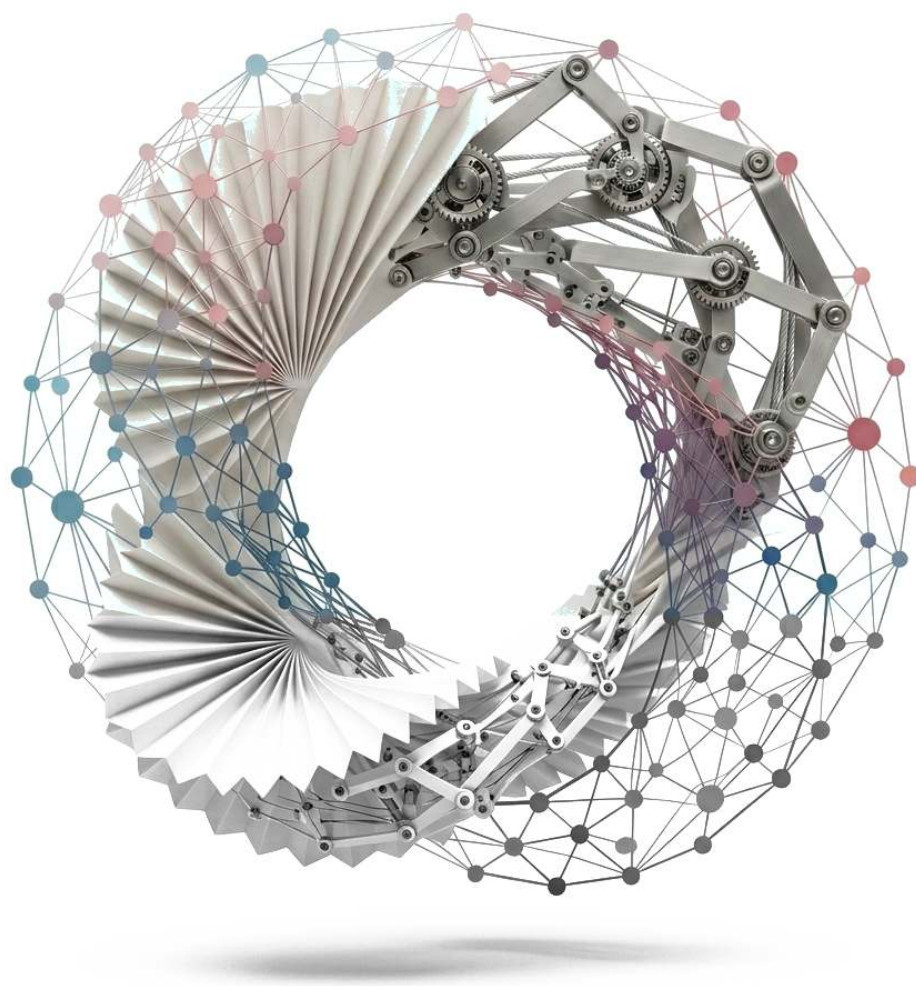




Physical AI

Machine Learning Systems

Vijay
Janapa Reddi

Physical AI

Machine Learning Systems That Sense and Act

DRAFT

Physical AI

Machine Learning Systems That Sense and Act

Vijay Janapa Reddi

Harvard University

DRAFT

The MIT Press
Cambridge, Massachusetts
London, England

Contents

Preface	xiii
Dedication	xix
Author’s Note	xxi
About the Author	xxiii
Why I Am Writing This Book: The First-Principles Systems Stance	xxiii
Intellectual Lineage and Systems Trajectory	xxiv
The Kit and Laboratory Development	xxiv
Reader Guide and Prerequisites	xxv
The Technical Baseline	xxv
Disciplinary Reading Pathways	xxv
What the Book Introduces	xxvi
How to Read the Numbers	xxvi
What We Deliberately Omit	xxvii
Acknowledgments	xxix
A Note on AI Assistance	xxxi
Notation	xxxiii
Physical AI and Embodied Systems Notation	xxxvii
1. Physical Plant State, Actuation, and Kinematic Topology	xlii
2. Rigid-Body Contact Mechanics, Friction Cones, and Centroidal Dynamics	xlii
3. Geometry, Spatial Transformations, and Perception Projections	xliii
4. Classical and Modern Control (State-Space, Observers, CBF-QPs)	xliv
5. Deep Learning for Robotics (Diffusion Policies, ACT, VLAs)	xlvi
6. Embedded Systems, Silicon Real-Time, Thermal, and Hardware Safety	xlvi

Main Text

Chapter 1 The Causal Boundary	3
1.1 Artificial Intelligence Beyond Glass	4
1.2 The Causal Boundary	6
1.3 Three Machine Classes	10
1.4 The Physical AI Scope Test	13
1.5 The Machine in Five Levels	16
1.6 When the Boundary Fails	18
1.7 The Four Bedrock Laws	21
1.8 Safety Convergence	22
1.9 The Architecture of This Book	23
1.10 Fallacies and Pitfalls	23

1.11 Summary	24
------------------------	----

Part I: The Machine Anatomy

The Machine Anatomy	29
----------------------------	-----------

Chapter 2 The Physical Body	31
------------------------------------	-----------

2.1 The Five Physical Budgets	32
2.2 Measurement Freshness	33
2.3 Kinetic Momentum	34
2.4 Actuator Transmission Limits	39
2.5 Thermal Duty Cycles	43
2.6 Electrical Power Integrity	46
2.7 Measuring a Machine's Own Limits	50
2.8 Which Budget Binds First	53
2.9 Fallacies and Pitfalls	54
2.10 Summary	55

Chapter 3 The Cognitive Brain	57
--------------------------------------	-----------

3.1 The Embodied Brain	58
3.2 The Classical Wall	59
3.3 Learned Representations	61
3.4 Supply, Freshness, and the Memory Wall	65
3.5 Cognitive Vulnerabilities	71
3.6 Fallacies and Pitfalls	73
3.7 Summary	74

Chapter 4 The Nervous System	77
-------------------------------------	-----------

4.1 The Multi-Rate Bridge	78
4.2 Where the Permission Path Runs	79
4.3 Multi-Rate Cadences	80
4.4 Moving Commands on Time	85
4.5 Actuation Authority	88
4.6 Hardware Isolation	90
4.7 Invariant Checking	92
4.8 The Handoffs Matrix	92
4.9 Fallacies and Pitfalls	96
4.10 Summary	97

Part II: Teaching the Machine

Teaching the Machine	101
-----------------------------	------------

Chapter 5 Physical Data	103
--------------------------------	------------

5.1 Endogenous Experience	104
5.2 Where Physical Data Comes From	105
5.3 Which Action Becomes the Label	105
5.4 Teleoperation Demonstration Costs	107
5.5 Collection Policy Coverage	111
5.6 Intervention and Recovery Data	113
5.7 Cross-Embodiment Transfer	115
5.8 How Datasets Go Wrong	117
5.9 The Dataset Schema	120
5.10 Fallacies and Pitfalls	122
5.11 Summary	123

Chapter 6 Policy Training	125
6.1 Policy Synthesis	126
6.2 Policy Synthesis Regimes	126
6.3 Learning by Trial	130
6.4 The Simulator as a System Component	132
6.5 What the Gap Is Made Of	136
6.6 Fine-Tuning on the Real Machine	139
6.7 How Training Choices Go Wrong	140
6.8 The Policy Manifest	141
6.9 Fallacies and Pitfalls	144
6.10 Summary	145
Chapter 7 Closed-Loop Evaluation	147
7.1 What Twenty Clean Runs Establish	148
7.2 What a Success Rate Measures	149
7.3 Closed-Loop Dynamics	151
7.4 Evidence Regimes	153
7.5 Physical Trial Limits	156
7.6 What Evaluation Cannot Establish	160
7.7 How Evaluation Goes Wrong	161
7.8 Evaluation Logs	164
7.9 Fallacies and Pitfalls	167
7.10 Summary	168
Part III: Running the Machine	
Running the Machine	173
Chapter 8 Sensor Perception	175
8.1 Photons to Spatial Claims	176
8.2 What Perception Hands Over	177
8.3 Sensor Transduction and Calibration	179
8.4 The Cost of Ingestion	183
8.5 Spatial Error and Clearance Bounds	184
8.6 How Observations Go Wrong	187
8.7 The Observation Contract	189
8.8 Fallacies and Pitfalls	191
8.9 Summary	192
Chapter 9 Spatial Memory	195
9.1 Preserving Spatial Belief	196
9.2 What Memory Hands Over	196
9.3 A State Has an Epoch	199
9.4 How a Transform Goes Stale	199
9.5 What Scene Memory Can Certify	201
9.6 Belief Through Occlusion	204
9.7 When the Estimator Cannot Know	207
9.8 How Belief Goes Wrong	208
9.9 The Spatial State Schema	210
9.10 Fallacies and Pitfalls	212
9.11 Summary	212
Chapter 10 Grounded Intent	215
10.1 Grounded Affordances	216
10.2 What Intent Hands Over	217
10.3 From Request to Expiring Geometric Proposal	219

10.4	Goals That Expire	223
10.5	Admissible Goal Sets	224
10.6	How Intent Goes Wrong	225
10.7	The Intent Lease	227
10.8	Fallacies and Pitfalls	229
10.9	Summary	230
Chapter 11	Trajectory Planning	233
11.1	Continuous Trajectories	234
11.2	What Planning Hands Over	235
11.3	From Goal to Trajectory	236
11.4	Kinodynamic Feasibility	239
11.5	Behavior at the Seam	242
11.6	How Plans Go Wrong	245
11.7	The Trajectory Contract	246
11.8	Fallacies and Pitfalls	247
11.9	Summary	248
Chapter 12	Safety Enforcement	251
12.1	The Deterministic Gatekeeper	252
12.2	What the Tracker Hands Over	252
12.3	The Authority to Refuse	254
12.4	Where the Enforcer Sits	255
12.5	Stopping Envelopes	256
12.6	When the Enforcer Runs Out of Authority	260
12.7	Safe Sets as Conditional Permission	261
12.8	Minimal Intervention	264
12.9	The Fallback Ladder	266
12.10	How Enforcement Goes Wrong	271
12.11	The Enforcement Record	271
12.12	Fallacies and Pitfalls	274
12.13	Summary	274
Chapter 13	Silicon Placement	277
13.1	The Silicon Substrate	278
13.2	Two Paths on One Die	280
13.3	Contention for Shared Resources	282
13.4	The Boundary Contract	283
13.5	Thermal and Timing Coupling	285
13.6	Testing the Separation	287
13.7	Silicon Isolation Decisions	289
13.8	Hardware Allocation	291
13.9	Fallacies and Pitfalls	293
13.10	Summary	294

Part IV: Governing the Machine

Governing the Machine	299
Chapter 14 Supervisory Intervention	301
14.1 Shared Authority	302
14.2 Human Authority Over Actuation	302
14.3 Authority Transitions	306
14.4 Transfer Without Discontinuity	310
14.5 Authenticating an Intervention	312
14.6 How Handover Goes Wrong	313

14.7	Trustworthy Operational Logs	315
14.8	The Authority Log	317
14.9	Fallacies and Pitfalls	318
14.10	Summary	319
Chapter 15	Adversarial Verification	323
15.1	Why Operation Is Not Evidence	324
15.2	The Operating Envelope	325
15.3	Deriving the Fault List	327
15.4	The Qualification Ladder	330
15.5	Fault Oracles and Acceptance	334
15.6	Fault Injection on Hardware	335
15.7	The Fault Manifest	338
15.8	Fallacies and Pitfalls	340
15.9	Summary	341
Chapter 16	Deployment Release	343
16.1	The Decision to Deploy	344
16.2	The Release Question	344
16.3	Safety Cases and Claims	347
16.4	The Three Verdicts	350
16.5	The Verdict's Standing Conditions	352
16.6	A Case Worked in Full	353
16.7	Adversarial Review	355
16.8	The Release Manifest	358
16.9	Fallacies and Pitfalls	360
16.10	Summary	361
Conclusion		
Chapter 17	The Epistemic Frontier	365
17.1	Epistemic Limits	366
17.2	What the Method Cannot Establish	367
17.3	A Residual-Claims Register	369
17.4	Failures Without Detectors	371
17.5	What Would Change These Answers	374
17.6	Generalizing to Unfamiliar Systems	376
17.7	What the Four Laws Leave Open	377
17.8	Fallacies and Pitfalls	379
17.9	Summary	380
Back Matter		
Appendices		385
References		385
Appendix A Machine Learning Background		391
Appendix B Control and Dynamics		413
Appendix C Systems and Hardware		429

Appendix D S·P·A Diagnostic Playbook	457
Glossary	461
Index	469

DRAFT

Preface

Who this book is for

This textbook is for computer science and engineering students, machine learning researchers, robotics engineers, and embedded systems architects who want to build embodied intelligent systems that safely sense, reason, and act in the physical world.

An advisory model ends at a display or a human decision, so its computation can be retried or canceled before anything outside the computer acts on the result. Its errors can still cause harm, but only through whoever or whatever responds to them.

In physical AI, software directs matter, momentum, and energy. When a vision-language model proposes motion for a mobile base or an articulated robotic arm, execution crosses the causal boundary into continuous Newtonian physics. If an actuation command exceeds the yield strength of a gearbox or the thermal dissipation limit of an inverter, physical hardware breaks. Once kinetic energy is released or material yields, software cannot undo the state transition.

Building systems that survive this reality requires a shared systems engineering discipline, and readers arrive at it from three directions:

- **Machine Learning Practitioners:** Moving beyond static test-set accuracy evaluated on stored datasets. In physical AI, actions alter future sensory observations through closed-loop feedback ($s_{t+1} \sim P(s \mid s_t, a_t)$), so a policy drifts off the states it was trained on, and the worst-case cost of that drift grows with the square of the horizon (section 1.7). Practitioners learn how to formulate continuous action policies, evaluate closed-loop survival, and submit unprivileged proposals to an independent permission path.
- **Robotics and Control Engineers:** Augmenting classical feedback loops with high-capacity neural foundation models. Modern vision-language-action models provide open-vocabulary semantic scene understanding and multimodal generalization, but they are learned proposers that run at tens of hertz with variable latency, so an independent permission path whose rate follows the plant's stopping budget must check what they propose. Control engineers learn how to build that path, including safety filters such as Control Barrier Functions, on real-time silicon.
- **Embedded Systems and Silicon Architects:** Managing multi-rate execution across heterogeneous system-on-chip (SoC) platforms. Architects learn how to isolate high-throughput neural accelerators from the safety microcontroller that runs the permission path, preventing memory bus crossbar contention, direct memory access (DMA) starvation, and supply voltage droop from breaching hard physical deadlines.

Why a physical AI systems textbook

This book approaches physical AI from a first-principles systems perspective. Its scope is clearest beside the two neighboring kinds of book that it is not.

- **Not a classical control theory book:** We do not develop continuous-time Laplace transforms, contour integrals, or Lyapunov stability proofs at length. Formal continuous-control proofs and reachability derivations are collected in the volume appendices for optional depth.

- **Not a traditional robotics kinematics book:** We do not work through Denavit-Hartenberg parameter matrices, forward-kinematics linkage catalogs, or specialized mechanical design heuristics.

Instead, we reason from physical principles (conservation of energy and momentum, stopping kinematics, thermal dissipation, and Coulomb friction) and from computational systems architecture (multi-rate execution loops, memory bus contention, asynchronous message queues, and permission paths that check learned proposals).

Mechanisms taught to the depth that changes a budget

This book does not treat learned models or control blocks as black boxes, and it does not survey their internals either. A mechanism enters the main text at the depth at which it changes a number the machine must budget: a latency, a memory footprint, a clearance, or the evidence a claim can rest on. An action-chunk decoder appears through the horizon it supplies and the time a new chunk takes. A scene representation appears through the memory it occupies and the age of the evidence it holds. A barrier filter appears through the premises its guarantee needs and the deadline it must meet. Internals that move no such number, including the arithmetic of patch tokenization and attention, decoder loss functions, and surface-reconstruction methods, are in the machine learning and control appendices for readers who want them.

Delay is distance

A machine keeps moving while it senses, decides, and waits for its brakes. Every millisecond between an observation and the moment braking force builds is spent as travel toward whatever the machine must not reach, so operating speed, delay, clearance, and braking capacity form one budget rather than four separate specifications. The book builds that budget for one machine, adding each term in the chapter that owns it, beginning with the body's stopping distance in section 2.3. A learned model's inference time enters the budget only through the age of the evidence it hands forward, because the permission path, not the model, decides each tick whether motion may continue.

What this book promises

This book does not promise a machine that is provably safe. No finite test campaign, runtime filter, or certification scheme delivers that for a learned system in an open world, and the chapters say so wherever it matters. The book teaches something narrower. It shows how to build a machine whose learned proposals cannot authorize their own motion, whose permission path decides on evidence of known age within the margin that delay has not yet consumed, and whose every guarantee is written down with the premises it rests on. A barrier filter protects the plant only while its model, its state estimate, its actuator authority, and its deadline hold. A release is justified only for the conditions its evidence covers. A premise that evidence cannot settle restricts what the machine may do instead of being assumed away. The final chapter returns to the opening question with that answer. The guarantee is conditional, and the engineering lies in knowing its conditions and recording them.

The machine in five levels

The machine in this book has five levels. The Brain holds the learned proposers, which revise goals at a few hertz and emit short action sequences at tens of hertz. Below the proposal boundary, the Nervous System holds the permission path, which admits, modifies, or refuses every proposal on fresh state and owns the fallback. The Body is the plant under its drives, the Boundary is where commands become current and the world becomes measurement, and the World is everything the machine does not own. Governance surrounds all five as a lifecycle envelope, deciding who may command the machine and what evidence licenses it to operate. The Brain proposes; only the permission path permits. Section 1.5 gives the full model and its rates.

The three machine classes

To keep systems concepts concrete, this textbook grounds its principles in the three machine classes of section 1.3:

1. **Class 1 • Mass-Dominated Mobility (AMRs and Quadrupeds):** Planar transport bases and legged platforms (such as Boston Dynamics' Spot) navigating open facilities and rugged terrain. Governed by vehicle momentum, tire-road and foot-ground Coulomb friction, kinematic stopping distance envelopes, and multi-camera / lidar perception pipelines under rolling-shutter skew.
2. **Class 2 • Contact-Dominated Manipulation:** Multi-link robotic arms performing dexterous contact tasks. Governed by non-linear arm dynamics, reflected rotor inertia ($J_{\text{ref}} = N^2 J_{\text{rotor}}$), cycloidal gearbox compliance, and contact force containment.
3. **Class 3 • Flow-Dominated Process and Energy Systems:** High-pressure fluid loops, chemical batch reactors, and thermal plants. Governed by enthalpy exchange, pipe fluid dynamics, pressure transients, and irreversible thermodynamic phase changes.

One machine runs through the book, the warehouse mobile manipulator of section 1.3, and each chapter adds what it owns to that machine's budgets. Each part closes by testing its law on a humanoid, which couples all three classes on one chassis.

How to read this book: Progressive arc vs. modular drop-in

This textbook supports two reading patterns:

For sequential readers (the cumulative curriculum arc)

Readers moving cover-to-cover follow an introduction, four parts, and a conclusion, and each part is led by one of the book's four laws. Section 1.9 traces the chain of records the chapters hand forward.

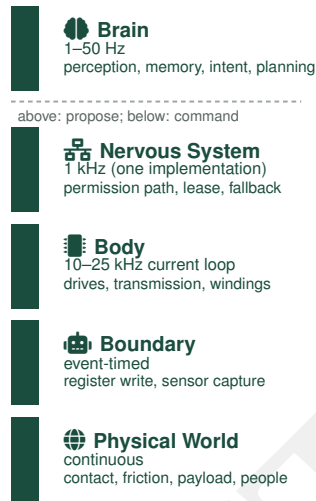
- **Introduction** (chapter 1) asks what a machine must guarantee before a learned model's proposal becomes physical work.
- **Part I: The Machine Anatomy** turns the first law, irreversibility, into budgets of time, distance, torque, heat, and voltage.
- **Part II: Teaching the Machine** asks what a learned proposer's competence rests on under the second law, endogenous data, when its own actions choose what it observes next.
- **Part III: Running the Machine** builds the runtime around the third law, proposal is not permission.
- **Part IV: Governing the Machine** applies the fourth law, evidence bounds authority, to decide who may command the machine and under which conditions it may operate.
- **Conclusion** (chapter 17) answers the opening question with what the evidence supports.

For drop-in readers and modular course instructors

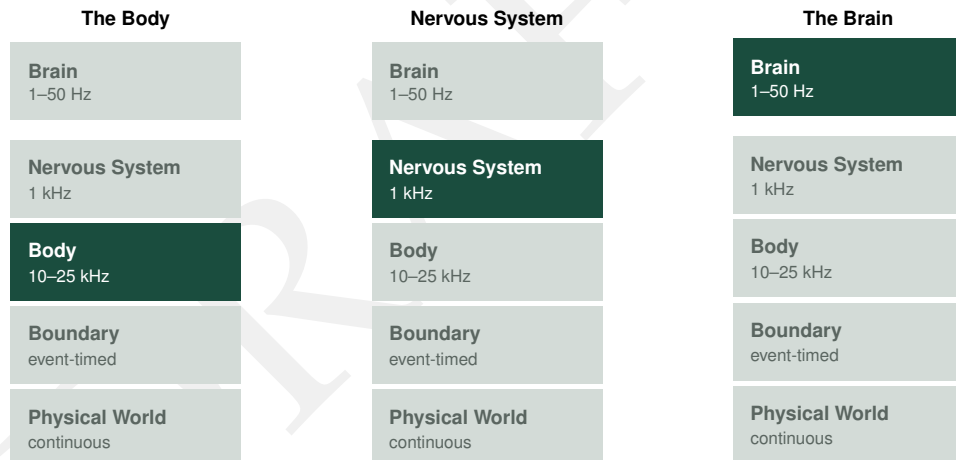
Instructors and practitioners often need to study one topic in isolation, such as sensor DMA pipelines, trajectory chunking, or Control Barrier Functions. To support modular use without prerequisite confusion:

- **Orientation at each chapter opening:** Every chapter begins with an orientation section that frames the core problem and states its physical stakes, and the margin stack at each chapter's opening marks the level the chapter works on.
- **Explicit Foundational Context:** Whenever a chapter draws upon a core concept established earlier (such as the proposal boundary, the stopping budget, or the three machine classes), the text provides an immediate inline orientation gloss and a precise section cross-reference.
- **Self-Contained Worked Examples:** Each chapter contains standalone analytical derivations, executable Python calculations, and industrial case studies that can be studied independently.

The margin stack is ordered by control rate and authority rather than by software dependency alone.



A stack with no level highlighted marks the governance chapters, because governance is an envelope around all five levels rather than one of them.

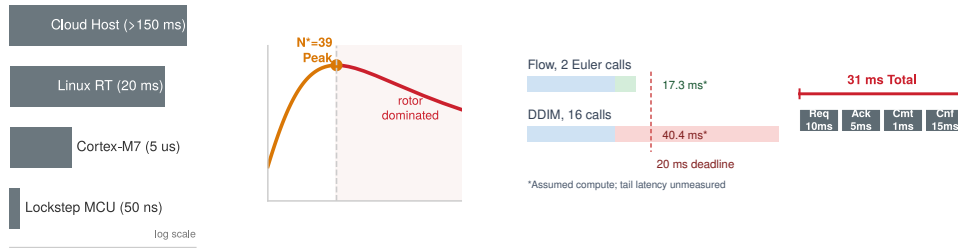


These three stacks are those of chapter 2, chapter 4, and chapter 3.

Margin reading aids

The margins carry small, unnumbered visual notes beside the passage where a relationship, a gap in scale, or a threshold is easiest to see. They are reading aids rather than figures to cite, and they come in five forms.

- **Ladders** span orders of magnitude (such as execution rates from inverter switching down to intent-model revision, or sensor ingestion bandwidths).
- **Threshold knees** mark the point beyond which a trend stops holding (such as reflected rotor inertia limiting joint acceleration, force rising with contact stiffness, or tail latency rising with memory bus contention).
- **Budget envelopes** set competing demands against a deadline (such as diffusion denoising times against control periods, or fieldbus cycle budgets).
- **Sequence strips** show multi-phase timing (such as authority handshakes, receding-horizon chunk windows, or the steps that seal a record).
- **Causal chains** trace how a fault propagates and how far it reaches (such as from an unmodeled simulator artifact to a damaged part).



The golden thread: An end-to-end forensic trace

To anchor abstract concepts across all seventeen chapters into a cohesive engineering reality, this book weaves a recurring real-world scenario—the **Golden Thread**—through the four parts and the five levels of the machine:

- **Part I · The Physical Challenge (The Plant Hazard):** An autonomous mobile robot (AMR) navigates an industrial aisle at $v = 1.3$ m/s. Without warning, an obstacle enters its path with limited clearance. The plant's physical braking capacity, tire-floor Coulomb friction, and actuator response delay dictate a hard stopping budget ($d_{\text{stop}} \leq D_{\text{clear}}$) that no software optimization can bypass.
- **Part II · The Learning Gap (The Policy Failure):** The onboard vision-language-action (VLA) policy, trained on nominal demonstrations without sudden intrusions, encounters acute out-of-distribution covariate shift ($\mathcal{O}(T^2\epsilon)$). Instead of braking, the generative action chunk decoder proposes an aggressive swerve that violates kinematic turning limits and would destabilize the chassis.
- **Part III · The Runtime Execution (The Silicon Bottleneck):** Camera exposure + SerDes transport + neural inference consumes critical milliseconds of unguided motion before an action proposal is emitted. When the proposal reaches the shared-memory mailbox, heterogeneous memory bus contention on the system-on-chip threatens the real-time deadline.
- **Part IV · The Safety Shield (The Deterministic Intervention):** Operating on an isolated real-time safety microcontroller, the high-rate reflex path evaluates the Control Barrier Function ($h(x) \geq 0$). Calculating that the proposed swerve will breach the admissible stopping envelope, the quadratic program minimally projects the proposal into an admissible braking clamp, arresting the chassis safely before contact without relying on software optimism.

Related books in the machine learning systems curriculum

This textbook is part of the *Machine Learning Systems* curriculum. *Introduction to Machine Learning Systems* examines how models use computation and memory on a single machine. *Scaling Machine Learning Systems* extends that engineering scope to coordinating distributed training and inference across cluster fleets.

The curriculum also addresses systems that take autonomous actions. *Agentic Machine Learning Systems* focuses on stateful goal-directed trajectories in digital software environments, while *Physical AI Systems* examines actions that produce force, motion, and thermodynamic change in the physical world. Each text offers a complementary perspective, and this book is self-contained, developing the physical, computational, and architectural principles required to engineer dependable embodied machines.

DRAFT

Dedication

*For my students,
who ask the question
I have not thought of yet,
and for the researchers and systems engineers
who take up the challenge of giving machines
safe and capable hands in an open physical world.*

Author's Note

WHEN A SERVER RETURNS AN ERROR FOR A PAGE REQUEST, the reader can try again. A robot approaching an obstacle keeps moving while its computer decides whether to brake. Restarting the computation cannot recover the distance already traveled.

In traditional computing, software operates behind glass: an unhandled exception or late result produces a log entry, a dropped frame, or a retry button. In Physical AI, software commands matter, momentum, and energy. The unit of engineering is the action crossing the causal boundary. Delay is paid in millimeters, and failure results in cracked composite, burned motor coils, or physical harm.

To build machines that survive this reality, we must give up the idea that a high-capacity network can drive a physical plant directly. In the machine this book builds, the learned model proposes; only the permission path permits, and only for as long as its evidence holds.

I write these chapters as an active systems researcher exploring how the laws of computing intersect with the laws of Newtonian mechanics, thermal dissipation, and functional safety. Concrete systems tasks, hardware measurements, and first-principles napkin math guide our progression; formal continuous-control derivations and proof machinery provide optional depth in the appendices.

This book is an open, working notebook. Step across the causal boundary with me, and let us explore how to engineer machines that sense, act, and survive in the physical world.

— Vijay Janapa Reddi
Cambridge, Massachusetts
2026

About the Author

Vijay Janapa Reddi is the Gordon McKay Professor of Electrical Engineering and Computer Engineering at Harvard University and a Visiting Professor at ETH Zurich in the Integrated Systems Laboratory (IIS), D-ITET. He is a founding member and Vice President of MLCommons and the author of the open-source reference text *Machine Learning Systems*.

His research sits at the intersection of computer architecture, machine learning systems, and edge robotics, focusing on how computing hardware and learned software co-design to interact with the physical world. Over the past decade, his work has concentrated on democratizing applied systems through open-source education and reproducible benchmarks, connecting first-principles silicon constraints directly to accessible, bench-verified learning.

Why I Am Writing This Book: The First-Principles Systems Stance

Every textbook reflects the intellectual lineage, biases, and lens of its author. I want to make the viewpoint behind this book explicit.

I am not a classical control theorist, nor a mechanical kinematics specialist, and I do not pretend to be one. I bring the perspective of a computer systems architect and machine learning systems (MLSys) researcher going after first principles.

My intuition for this domain dates back to my undergraduate mechatronics capstone. As an electrical and computer engineering student, that capstone was the first time I had to integrate everything—analogue circuits, sensor conditioning, microcontrollers, motor drivers, mechanical linkages, and embedded software—into a single working machine. It was the moment I realized that software cannot be designed in isolation from physical reality: motor coils heat up, power rails droop under sudden acceleration, and sensor measurements age the instant physical transduction occurs.

For over twenty years—from mobile System-on-Chip performance architecture and edge AI silicon at Google, to co-founding MLCommons/MLPerf to establish empirical benchmarking standards for AI accelerators, to pioneering the TinyML movement to compress neural networks onto milliwatt microcontrollers—my work has lived at the physical boundary where software bits meet physical silicon and energy constraints.

Physical AI brings me full circle back to that mechatronics capstone, but with a profound twist. In classical mechatronics, we closed the loop using hand-crafted, deterministic logic: PID controllers, state machines, and analytical models. In Physical AI, we are embedding high-capacity, stochastic neural networks—foundation models, diffusion policies, and learned decoders—into that exact same unforgiving loop. The engineering challenge is no longer just making the machine move; it is figuring out how to safely close the loop around an uncalibrated, stochastic brain using deterministic systems and safety principles.

That background shapes what I want students to understand. A model's proposed action must pass through a computing system before the physical machine can carry it out. I want readers to trace that path and ask what could prevent an action from being carried out as intended. The timing of computation, the body's capabilities, and the assumptions behind command checks all matter to that assessment. The explanations should make these interactions understandable before specialized mathematics or implementation detail is needed.

This textbook is my attempt to share that foundational systems perspective with the next generation of engineers, roboticists, and computer scientists. It is the book I wish my own students had when bridging machine learning systems to the physical world.

Intellectual Lineage and Systems Trajectory

This first-principles systems trajectory spans three major eras:

- **Silicon Reality & Mobile Architecture:** Grounded in computer architecture, low-power microarchitectures, and mobile System-on-Chip (SoC) design (including edge AI silicon performance architecture at Google). This work addressed the physical realities of computing: memory bus contention under high-bandwidth camera streams, thermal throttling under strict dissipated wattage envelopes, and interrupt latencies under hard real-time deadlines.
- **ML Systems & The TinyML Movement:** Bridging algorithmic machine learning abstractions to silicon execution. He co-founded MLPerf to establish the global standard for benchmarking ML hardware and software, and pioneered the TinyML movement, developing the Harvard CS249r curriculum and the HarvardX TinyML program (reaching over 100,000 learners globally) to bring deep neural models onto milliwatt microcontrollers.
- **Closing the Sense-and-Act Loop in Physical AI:** Extending this systems perspective to machines whose learned models propose physical actions. The focus is on how sensing, computation, and the body's response interact, and on the conditions under which a proposed action can be executed.

The Kit and Laboratory Development

Development of the hands-on bench laboratories and the reference hardware platform (the Physical AI Kit on Arduino UNO Q Dual-Brain) is a collaboration with Dr. Andrea Mattia Garavagno, postdoctoral researcher in the Integrated Systems Laboratory (IIS), D-ITET at ETH Zurich.

This work aims to translate the systems ideas in the book into firmware and software exercises that students and practitioners can reproduce and examine on the bench.

Reader Guide and Prerequisites

Physical AI Systems is written as an independent, self-contained textbook for advanced undergraduate students, graduate researchers, and practicing systems engineers who wish to build machines that safely couple learned intelligence to physical matter and energy.

Readers will bring different strengths from machine learning, embedded systems, or mechanics. The book introduces the concepts needed to reason about their interactions without requiring specialist training in every contributing discipline. This guide distinguishes the shared starting background from material introduced in the text and suggests where readers with different backgrounds can concentrate their attention.

The Technical Baseline

The main text assumes a shared foundation in programming, machine learning, and basic mathematics. The following preparation supports reading the explanations and following the short calculations:

- **Programming & Systems:** Readers should be able to read and write Python functions, follow data through a program, and recognize that execution takes time and uses memory. Familiarity with C or C++, pointers, and memory allocation is useful for optional implementation work.
- **Machine Learning Intuition:** Readers should recognize the difference between training a model and using it to make predictions, and be familiar with arrays or tensors as model inputs and outputs. Experience with a particular framework is unnecessary.
- **Foundational Physics & Mathematics:** Readers should be comfortable with basic algebra, vectors and matrices, and single-variable calculus (rates of change). Physical AI relies directly on freshman Newtonian mechanics:
 - Newton’s second law for linear and rotational systems ($F = ma, \tau = I\alpha$).
 - Kinetic energy and stopping distance napkin math ($E_k = \frac{1}{2}mv^2, d_{\text{stop}} \approx v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}}$).
 - Basic electrical power and heating ($P = I_{\text{phase}}^2 R_{\text{phase}}$). Advanced differential geometry, Lie group continuous control theory, and formal reachability analysis are deliberately not required; their mathematical treatments are collected in the volume appendices.

This preparation supports reasoning about the complete system. Specialized control theory, detailed robotics mathematics, and low-level implementation expertise are not starting requirements for the main explanation.

Disciplinary Reading Pathways

Readers approach physical AI from different engineering traditions. The published sequence develops a shared systems perspective; readers can give additional attention to the interactions least familiar from their own background:

- **For Machine Learning Practitioners:** Focus on the physical consequences of computation. A machine continues moving while its model processes an observation. Pay close attention to chapter 2 for motion and braking constraints, chapter 5 and chapter 6 for endogenous feedback and compounding drift ($\mathcal{O}(T^2\epsilon)$), chapter 7 for closed-loop reality, chapter 12 for checking proposed actions before execution, and chapter 13 for computing resources that affect timing.

- **For Embedded Systems & Silicon Architects:** Focus on how learned computation contributes to decisions that the machine must carry out on time. Pay close attention to chapter 1 for the proposal boundary, chapter 8 for how sensory observations inform decisions, chapter 9 for spatial belief representations, chapter 10 and chapter 11 for trajectory synthesis, and chapter 13 for system-on-chip (SoC) memory bus contention and thermal limits.
- **For Roboticists & Control Engineers:** Focus on closing the physical loop around learned models. Pay close attention to chapter 1 for limits on what a model may command, chapter 4 for timing and communication between components, chapter 14 for human authority handshakes and bumpless transfer, chapter 15 for hardware-in-the-loop fault injection, and chapter 16 for the structured evidence needed to judge readiness for release.

These points of emphasis supplement a sequential reading rather than replace it, because the book's argument lies in how each discipline connects to the rest of the machine.

What the Book Introduces

The book starts from the physical and computational relationships needed to explain a system's behavior. Specialized concepts enter through the problems they help solve, with their assumptions and limits made explicit. The main text develops the following ideas from that shared foundation:

- **Proposals and permission to act:** The text explains how a learned model can propose an action while a separate component determines whether that action may reach the body.
- **Checks under physical limits:** The explanation connects proposed actions to available space and machine motion, stating the assumptions under which a check can help.
- **Timing across components:** The text develops how components that operate at different rates exchange results and coordinate their work, and why delays matter to the physical response.
- **Observations and motion:** The explanation relates positions and directions described by sensors and models to movements of the body, including how sudden changes in commanded motion affect the mechanical response.
- **Evidence for release:** The text explains how to connect a claim about system behavior to supporting evidence and identify what that evidence leaves unresolved.

Detailed proofs, specialized notation, and implementation layouts provide optional depth. The main explanation retains the mechanism, assumptions, and engineering consequence so that following the systems argument does not depend on pursuing that detail.

How to Read the Numbers

Every number in this book carries one of five labels, four categories of evidence and one for design choices, and its label decides what it may be used for.

- **Measured:** A calibrated observation of a particular machine. It names its instrument, its operating conditions, its sample count, and, for a distribution, the percentile it reports. Only a measured number, or a number derived from measured ones, can support a safety margin.
- **Derived:** Computed from measured numbers through a stated physical or statistical model. It carries that model's assumptions and the propagated uncertainty of its inputs.
- **Datasheet:** A manufacturer's rating under the manufacturer's conditions. It says what to expect and what to test, and it is not evidence about the unit on the bench.
- **Illustrative:** Selected to show arithmetic or scaling. It supports no margin, timeout, or release decision.

A budget or requirement, such as a lease duration or a clearance margin, belongs to none of the four. It is a design decision, labeled **chosen**, and it means something only when it is checked against the measured or derived number it must fit, as a chosen lease is checked against the derived ceiling on pre-brake delay.

A worked design in this book shows a decision procedure. When its inputs are illustrative, the decision it reaches is illustrative too. It shows how the machine's speed, lease, or verdict would be set, and it would bind a real machine only after the inputs it names are measured.

When a number decides something, such as a speed limit, a timeout, or a release, the book keeps it as a **limit record** with six elements: the value or distribution; the operating conditions under which it holds (payload, temperature, supply voltage, surface, concurrent workload, firmware); the measurement uncertainty; the engineering margin between the operating target and the breakdown threshold; the detector that catches an excursion while the machine runs; and its provenance, meaning its label and the instrument or model that produced it. A limit holds only under its stated conditions. When wear, heat, a new payload, or a new build changes them, the limit must be established again before it is reused.

Worked examples label each number's category once, at first use. A result holds under the assumptions stated with its example and nowhere else. A bound from finitely many trials holds under its sampling assumptions and proves nothing beyond them, and a measured percentile describes the tested workload, not a worst case. In a release argument each premise also has a status: established (measured or derived), assumed, decided by accountable judgment, unsupported, or outside the threat model. Chapter 16 and chapter 17 use these statuses.

What We Deliberately Omit

The book focuses on how learned computation, physical mechanisms, and execution systems work together. That focus limits the depth of its treatment in neighboring fields:

- **Exhaustive Robotic Kinematic Derivations:** The main text does not develop long derivations for every manipulator configuration. The relevant systems concern is what movements a machine can make and which physical limits constrain them.
- **Deep Learning Theory:** The book does not provide a comprehensive mathematical treatment of loss landscapes and optimization. Learning mechanisms matter here through the behavior they produce and the demands they place on the physical system.
- **Framework and Tool Tutorials:** The book does not teach particular robotics middleware, simulators, or training frameworks. Software matters here through its effect on timing, memory, and evidence.

These boundaries leave the essential mechanisms and their consequences in the systems argument. They limit exhaustive specialist treatment without making familiarity with that treatment a prerequisite for following the book.

Acknowledgments

PLACEHOLDER. This book's acknowledgments are still to be written, recording the people whose expertise, interviews, and industry collaboration shaped its development.

A Note on AI Assistance

Every chapter of this book was researched, written, and rewritten by me. I also used AI tools throughout. What follows is more than the disclosure MIT Press requires of authors who use AI. It is an honest statement of how I used these tools, a model for how students and practitioners can engage with them responsibly, and a claim about what authorship means when machine learning acts as both force multiplier and sounding board.

In my previous volumes, systems and computing infrastructure was my native intellectual home. But *Physical AI* pushed me across the causal boundary into unfamiliar terrain: classical kinematics, nonlinear control barrier functions, motor winding thermodynamics, and high-frequency real-time bus contention. The most difficult challenge in writing this book was not drafting prose; it was conducting an exhaustive cross-disciplinary literature survey and synthesizing fields that rarely speak to one another. I used large language models extensively as interactive research partners to survey disparate literature, explore unfamiliar dynamics, brainstorm structural framings, and distill foundational first principles. These models excel at first-principles distillation—which is precisely how I naturally think about computer systems.

Yet distillation is only an initial proposal. Because these concepts live in the physical world where errors carry kinetic consequences, I relied heavily on human experts—both academic researchers in robotics and control theory, and practicing engineers at autonomous systems companies—to critique, validate, and challenge the architectures developed here. This book does not claim to be a final, closed work; it is an active, open iteration that evolves alongside a rapidly developing discipline.

The standard remains uncompromising traceability. Every derived metric, timing budget, and energy calculation in these chapters is computed directly by executable Python cells whose source code ships with the book. Quantitative system simulations are backed by the MLSysIM infrastructure modeling engine. A continuous pre-commit suite verifies dimensional units, citation integrity, and reference anchors on every change. AI tools made navigating a steep cross-disciplinary learning curve mechanically tractable; our verification infrastructure makes that work reproducible and auditable.

Authorship is thinking. Deciding which concepts belong in this curriculum and which do not; sequencing the chapters so that each abstraction earns its keep; determining where students will get stuck because I have watched them get stuck in my own classroom—these are judgment calls that no tool can make. The ideas and architectural synthesis are mine; the initial prose was often drafted, expanded, or restructured by AI before being aggressively rewritten and verified.

Machine learning systems cannot be certified the way traditional deterministic logic can. That verification gap applies to AI-assisted authoring with equal force. The standard I held this book to is the standard we must demand of every embodied machine we deploy: the tool can propose, and the tool can tutor, but only a human engineer can ratify and take responsibility.

Notation

Machine Learning Systems spans *machine learning* (computer science and statistics) and *systems* (computer architecture and hardware). Each field developed its notation independently, and many symbols mean different things across communities and publications. This collision creates real confusion when the disciplines merge. The conventions below establish a single notation that eliminates this ambiguity.

Consider a simple statement: “Increasing B improves throughput.” To an ML researcher, B means batch size. To a hardware engineer, B means bandwidth. Both interpretations are correct in their respective fields, but in ML Systems we need both concepts in the same equation—hence the need for a single, consistent convention.

The Iron Law of ML Systems

For serialized execution phases, the iron-law performance decomposition is:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}}$$

Each variable was chosen deliberately to avoid collision with standard ML terminology.

Symbol	Definition	Unit	Why This Symbol?
T	Time	seconds	Unambiguous. Wall-clock time for an operation.
D_{vol}	Data Volume	bytes	Avoids collision with D (Dataset Size). In scaling laws, D means training tokens. Here we need bytes moved through memory. The subscript disambiguates.
BW	Bandwidth	bytes/s	Avoids collision with B (Batch Size). Systems literature often uses B for bandwidth, while ML literature often uses B for batch size. The notation preserves the ML convention.
O	Operations	FLOPs	Total floating-point operations. Clean in equations (vs. “Ops”).
R_{peak}	Peak Rate	FLOP/s	Avoids collision with P (Parameters). Roofline presentations often use P for performance, while ML literature commonly uses P for parameter count. The notation preserves the ML convention.
η_{hw}	Efficiency	—	Hardware utilization ($0 \leq \eta_{\text{hw}} \leq 1$). Avoids collision with learning rate (η).
L_{lat}	Latency	seconds	Avoids collision with $\mathcal{L}$ (Loss). Fixed overhead time (kernel launch, network RTT). The subscript distinguishes from the loss function.

Why these choices matter

Without careful notation, sentences become ambiguous:

“Reducing D improves performance.”

The sentence has two possible readings:

- Reducing **dataset size** (fewer training samples) can speed training but may reduce accuracy.
- Reducing **data volume moved** (for example, through compression or quantization) can speed inference, with accuracy effects that depend on the technique and calibration.

With our notation, we can write precisely:

“Reducing D_{vol} through FP32-to-INT8 quantization cuts parameter memory traffic to one quarter while D (training data) remains unchanged.”

Our notation eliminates this ambiguity: “*BW limits throughput*” has only one reading.

Subscripted variants

An upright root names the quantity, and the subscript names which instance of it is meant. The convention holds closely related measurements apart instead of collapsing them into a single symbol:

- $BW_{\text{disk}}, BW_{\text{network}}, BW_{\text{accelerator}}$: bandwidth at three points on the data path
- $D_{\text{vol}}, R_{\text{peak}}, L_{\text{lat}}, \eta_{\text{hw}}$: iron-law terms, each held clear of a common ML symbol
- $E_{\text{move}}, E_{\text{compute}}, E_{\text{total}}$: one energy budget split into tradeable parts

The subscript therefore carries part of the claim. Disk and accelerator bandwidth differ by orders of magnitude, so a figure is reproducible only when the subscript says where it was measured.

The Degradation Equation

The degradation equation is a fitted local model. Divergence alone does not determine the direction of accuracy change, so estimate λ from labeled outcomes. Some symbols below, such as τ , appear in chapter prose rather than in the equation.

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

Symbol	Definition	Unit/Type	Notes
$\text{Accuracy}(t)$	Accuracy at Time t	Scalar	Model accuracy after the model has been deployed for time t .
Accuracy_0	Initial Accuracy	Scalar	Model accuracy at deployment time.
λ	Fitted Sensitivity	Scalar	Local coefficient estimated from outcomes over a stated range; not a universal constant. (<i>Not wavelength.</i>)
P_t	Current Distribution	Distribution	The data distribution at time t . (<i>Not parameters—use P for parameter count.</i>)
P_0	Training Distribution	Distribution	The data distribution at training time.
$\mathcal{D}(P_t \ P_0)$	Statistical Divergence	Scalar ≥ 0	Measures how far P_t has drifted from P_0 . Common choices: KL divergence, total variation, Wasserstein. (<i>Calligraphic to avoid collision with D = dataset size.</i>)
τ	Response Threshold	Scalar > 0	Operational threshold for investigation; it does not automatically trigger retraining.

The Energy Corollary

With effective per-byte and per-operation costs, workload energy is approximated as:

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

Symbol	Definition	Unit	Notes
E_{total}	Total Energy	joules	Total energy consumed by an ML workload, decomposed into data-movement and compute terms.
E_{move}	Energy per Byte Moved	joules/byte	Effective movement cost; total energy also depends on bytes moved and operations executed.
E_{compute}	Energy per Operation	joules/FLOP	Energy cost of a single arithmetic operation.

Deep Learning Notation

This book follows standard deep learning conventions (Goodfellow et al. 2016) with explicit disambiguation for systems variables.

Symbol	Definition	Dimensions/Type
B	Batch Size	Integer. The number of samples processed in parallel. (<i>Never bandwidth.</i>)
P	Parameters	Integer. The total count of scalar model parameters, including weights and biases. (<i>Never peak FLOP/s.</i>)
D	Dataset Size	Integer. Number of training samples or tokens. (<i>Never data volume in bytes—use D_{vol}.</i>)
S	Sequence Length	Integer. Number of tokens or time steps.
d	Hidden Dimension	Integer. Size of the hidden state vector.
d_{head}	Attention Head Dimension	Integer. Per-head hidden dimension in attention layers.
N_L	Number of Layers	Integer. Total number of layers in a network.
N_{heads}	Number of Attention Heads	Integer. Number of attention heads in a multi-head attention layer.
H_{KV}	Number of Key-Value Heads	Integer. Number of key-value heads in grouped-query or multi-query attention.
ℓ	Layer Index	Integer. Index for a layer. Use instead of bare L when indexing layers, since L collides with loss and latency.
$\mathcal{L}$	Loss Function	Scalar. The objective function minimized during training.
η	Learning Rate	Scalar. Step size for the optimizer. (<i>Never bare for hardware efficiency—use η_{hw}.</i>)
θ	Model Parameters	Parameter collection, often treated as a vector. The set of all learnable parameters.
$p(x)$	Distribution	Probability mass/density for a random variable. Use lowercase $p(\cdot)$ for generic distributions to avoid collision with P (parameter count).
$p(y x)$	Conditional Distribution	Conditional probability/density. Use this form for generic label relationships; reserve P_0 and P_t for the degradation equation’s training/current distributions.
$\Pr(E)$	Event Probability	Probability of an event E . Use for event statements such as $\Pr(\text{batch} = 0)$; use $p(x)$ and $p(y x)$ for distributions.

Latin matrices and vectors are set in bold ($\mathbf{W}$, $\mathbf{x}$); scalars, dimensions, indices, and individual matrix entries stay italic (N , d , i , W_{ij}). The generic parameter symbol θ is the one conventional exception and stays italic. Local matrix algebra follows the same bold rule: matrix operands are bold ($\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$), as in general matrix multiply (GEMM) $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$. The bold form marks a matrix operand and is distinct from the italic scalar batch size B , which remains batch size in scalar contexts.

Performance, Serving, and Memory Notation

Reusable performance and serving quantities follow the same collision-avoidance rule as the iron law. Rates use R or descriptive Greek symbols; request counts use Q_{req} rather than overloading N ; queue utilization uses a subscripted ρ so bare ρ remains available for other ratio models.

Symbol	Definition	Unit/Type	Notes
I	Arithmetic Intensity	FLOP/byte	Workload FLOPs per byte moved. The Roofline Model uses I as the independent variable.
I_{ridge}	Roofline Ridge Point	FLOP/byte	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$. Prefer this explicit form over starred shorthand so the meaning remains clear in prose.
R_{attain}	Attainable Compute Rate	FLOP/s	Roofline bound: $R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$. Uses R , not T , because this quantity is a rate.
MFU	Model FLOPs Utilization	Dimensionless	Useful model FLOP/s divided by available peak FLOP/s. Text acronym avoids overloading η .
r_{comp}	Compression Ratio	Dimensionless	Uncompressed size divided by compressed size. A compressed payload has $1/r_{\text{comp}}$ of the uncompressed size; the subscript avoids bare C collisions.

Symbol	Definition	Unit/Type	Notes
Q_{req}	Request Concurrency	requests	Average in-flight requests in a stable serving system ($Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}}$ via Little's Law). Avoids collision with N as device count in distributed settings.
λ_{arr}	Arrival Rate	requests/s	Request arrival rate. The subscript avoids collision with λ as degradation sensitivity or failure rate.
T_{lat}	Request Time in System	seconds	End-to-end queueing/serving latency for Little's Law. Distinct from L_{lat} , the fixed-latency term in the iron law.
$T_{\text{svc}}(B)$	Batch Service Time	seconds	Time to serve a batch of size B .
$\mu_{\text{eff}}(B)$	Effective Service Rate	requests/s	Batched service rate, typically $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$.
ρ_{serv}	Serving Utilization	Dimensionless	Queue/server utilization. Use instead of bare ρ , which is reserved for communication-computation ratio in distributed contexts.
M_{total}	Total Memory Footprint	bytes	Sum of explicit memory components; avoids bare M ambiguity.
M_{weights}	Weight Memory	bytes	Memory occupied by model parameters.
$M_{\text{gradients}}$	Gradient Memory	bytes	Memory occupied by stored gradients.
$M_{\text{optimizer}}$	Optimizer-State Memory	bytes	Momentum, variance, master weights, and related optimizer buffers.
$M_{\text{activations}}$	Activation Memory	bytes	Retained activations for the backward pass or serving intermediates.
s_{elem}	Element Storage Size	bytes/element	Bytes per stored tensor element.

Units and Precision

- **Physical units:** This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, the book converts to SI and notes the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).
- **Data and memory:** This book uses **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6 , GB = 10^9 , TB = 10^{12} . Binary-prefixed units do not appear in prose; all capacities, throughputs, and model sizes are reported in decimal units (for example, 80 GB, 2 TB/s, 102 MB).
- **Compute:** The notation distinguishes operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
 - 1 TFLOP = 10^{12} FLOPs
 - 1 TFLOP/s = 10^{12} FLOPs per second
 - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.
- **Currency:** Dollar amounts use the dollar sign (\$); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- **Precision:**
 - **FP64:** Double precision (8 bytes)
 - **FP32:** Single precision (4 bytes)
 - **TF32:** TensorFloat-32 (19-bit Tensor Core compute mode; inputs and storage remain FP32)
 - **FP16:** Half precision (2 bytes, standard range)
 - **BF16:** Brain float (2 bytes, wide dynamic range)
 - **FP8:** Quarter precision (1 byte, E4M3 or E5M2 format)
 - **FP4:** 4-bit floating-point format
 - **INT8:** 8-bit integer (1 byte)
 - **INT4:** 4-bit integer; lower integer precisions follow the same uppercase INTn pattern (for example, INT3, INT2)

Quick Reference: Resolving Collisions

Common collision points in ML Systems literature include:

Symbol	ML Meaning	Systems Meaning	Book Convention
B	Batch Size	Bandwidth	Batch Size. Use BW for bandwidth.
P	Parameters	Peak FLOP/s	Parameters. Use R_{peak} for peak rate.
D	Dataset Size	Data Volume	Dataset Size. Use D_{vol} for bytes moved.
L	Loss	Latency	Loss ($\mathcal{L}$). Use L_{lat} for latency.
η	Learning Rate	Efficiency	Learning Rate. Use η_{hw} for efficiency.

As a general principle, ML conventions take precedence for single letters, while systems concepts get subscripts or multi-letter symbols. This reflects the primary audience (ML practitioners learning systems) and preserves compatibility with the vast ML literature.

Physical AI and Embodied Systems Notation

Physical AI and embodied machine learning systems merge three distinct technical traditions:

1. **Machine Learning and Deep Learning** (statistical optimization, transformers, diffusion policies, imitation learning, autoregression, Action Chunking with Transformers).
2. **Robotics, Mechanics, and Classical Control** (multibody dynamics, $SE(3)$ Lie groups, rigid-body kinematics, Lyapunov stability, Control Barrier Functions, contact mechanics, actuator drive physics).
3. **Embedded Systems and Real-Time Architectures** (system-on-chip interconnects, cache and memory bus contention, hardware interrupt latency, seqlocks, CAN-FD/EtherCAT, intent leases, fail-safe interlocks).

Because each discipline developed its mathematical notation independently over decades, combining them creates severe symbol collisions—the “traditional warts” of cyber-physical engineering. For example, writing $\nabla_{\theta}\pi_{\theta}(\mathbf{a} \mid \mathbf{q})$ when θ also denotes joint angles, or writing $\mathbf{a}_t$ for an action when $\mathbf{a}$ denotes Cartesian acceleration or barrier constraints, confuses spatial mechanics with parameter optimization and control execution authority.

Below, we formalize the cross-discipline disambiguation contract, state the governing relationships of embodied systems, and tabulate the canonical symbols used throughout this book.

Cross-Discipline Disambiguation and Traditional Warts

When physical plant dynamics meet deep neural networks and real-time embedded hardware, common mathematical shorthand breaks down. The conventions below resolve these cross-domain collisions systematically:

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
θ	Neural policy network weights π_{θ}	Joint angle θ_i , trajectory polynomial $\theta(t)$, thermal resistance θ_{JA} , or heading θ	Reserve θ strictly for neural parameters. Robot joint configurations are always generalized coordinates $\mathbf{q} \in \mathbb{R}^n$ and q_i . Trajectory polynomials are $q(t) = \sum c_k t^k$ (never $\theta(t)$). Thermal resistance is R_{th} or $R_{\text{th},JA}$ (in K/W). Heading or spatial angles use ϕ, ψ or rotation matrices.
$\mathbf{a}$ vs. $\hat{\mathbf{u}}$ vs. $\mathbf{u}^*$ vs. τ	Four-stage authority chain ($\mathbf{a}_t \rightarrow \hat{\mathbf{u}}_t \rightarrow \mathbf{u}_t^* \rightarrow \tau_t$)	Generic “action” symbol colliding with deceleration a_{brake} or barrier normal $\mathbf{a}(\mathbf{x})$	Disambiguate execution authority: $\mathbf{a}_t \in \mathcal{A}$ is the unprivileged policy action (dimensionless/normalized); $\hat{\mathbf{u}}_t \in \mathcal{U}$ is the proposed plant control setpoint; $\mathbf{u}_t^* \in \mathcal{U}$ is the certified, CBF-filtered control command; $\tau_t \in \mathbb{R}^n$ is realized physical actuator torque. Credible linear deceleration is a_{brake} (m/s^2); barrier constraints use $\mathbf{A}_{\text{cbf}}(\mathbf{x})\mathbf{u} \leq \mathbf{b}_{\text{cbf}}(\mathbf{x})$.

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
τ vs. τ	Actuator torque vector $\tau \in \mathbb{R}^n$ (N · m or N)	Time constants (τ_{th}), latencies (τ_{ZOH} , τ_{lag}), intent lease duration (τ), or response threshold (τ)	Realized torque is always bold vector $\tau \in \mathbb{R}^n$ or scalar joint torque τ_i / component torque (τ_{motor}, τ_{hold}). Time intervals and latencies use Latin T or lowercase t (T_{lease} , t_{age} , t_{ZOH} , T_{blend}). Thermal time constants use explicitly subscripted $\tau_{th} = R_{th}C_{th}$. Operational thresholds use τ_{resp} . The total pre-brake delay τ_{delay} keeps its Greek letter, the control-theory convention for a transport delay, and its subscript keeps it distinct from torque.
$\mathbf{x}$ vs. s vs. $\mathbf{p}$ vs. $\mathbf{x}_p$	Mechanical plant state $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T \in \mathbb{R}^{2n}$	World state $s \in \mathcal{S}$, 3D position $\mathbf{p} = [x, y, z]^T$, or ViT image patch $\mathbf{x}_p$	Continuous mechanical plant state is bold vector $\mathbf{x} \in \mathbb{R}^{2n}$. Ground-truth latent world state is $s \in \mathcal{S}$. 3D Cartesian position is vector $\mathbf{p} \in \mathbb{R}^3$ or camera-frame point $\mathbf{P}_c$. Vision transformer patch embeddings use explicitly subscripted $\mathbf{x}_p$ or latent feature $\mathbf{z}$.
P	Model parameter count (P or P_{params})	Power (P_{elec} , P_{cont}), Riccati error covariance $\mathbf{P}(t)$, failure rate p_{fail} , or 3D point $\mathbf{p}$	Use P for parameter count. Electrical/mechanical power is P_{elec} , P_{cont} , P_{peak} (in Watts); continuous Riccati/Kalman error covariance is bold matrix $\mathbf{P}(t)$; statistical failure probability or rate is lowercase italic p (p_{target} , p_{emp} , p_{fail}); 3D Cartesian position is bold lowercase $\mathbf{p} = [x, y, z]^T$.
B	Batch size (B or B_{tokens})	Bandwidth (BW), Body frame ($\mathcal{F}_B$), bus burst size (B_k), or input matrix $\mathbf{B}$	Batch size is B ; sustained bandwidth is BW (per Iron Law); coordinate reference frames use calligraphic script ($\mathcal{F}_W$, $\mathcal{F}_B$, $\mathcal{F}_E$); bus transaction sizes use $D_{trans, k}$ or B_k ; state-space control input matrix is bold capital $\mathbf{B} \in \mathbb{R}^{n \times m}$.
D	Dataset size (D in tokens or trajectories)	Data volume (D_{vol}), clear distance (D_{clear}), duty cycle (D_{cycle}), TSDF field $D_t(\mathbf{v})$, or damping matrix $\mathbf{D}_{virt}$	Training dataset size is D ; bytes moved across buses is D_{vol} ; stopping distance is lowercase d_{stop} , and the clear distance to a boundary is D_{clear} , whose subscript keeps it distinct from dataset size; actuator duty cycle uses subscripted $D_{cycle} \in [0, 1]$; voxel TSDF fields use $D_t(\mathbf{v})$; virtual damping matrices use bold $\mathbf{D}_{virt}$.
α	Bumpless transfer blending function $\alpha(t) \in [0, 1]$	CBF decay (γ_{cbf}), significance (α_{sig}), angular acceleration (α), copper tempco (α_{Cu}), or noise schedule ($\bar{\alpha}_k$)	Authority transfer uses scalar function $\alpha(t) \in [0, 1]$; barrier linear decay uses $\gamma_{cbf} > 0$; statistical confidence significance uses $\alpha_{sig} = 1 - \text{CL}$; angular acceleration is bold vector $\alpha \in \mathbb{R}^3$ (rad/s ²); copper resistance tempco is α_{Cu} ; diffusion noise schedules use indexed $\bar{\alpha}_k$.
$\mathbf{C}$	Coriolis matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$	Safe set $\mathcal{C}$, state-space output matrix $\mathbf{C}$, capacitance (C_{th} , C_{bus}), CoM $\mathbf{c}$, or shield coverage c_{shield}	Coriolis/centrifugal forces use matrix function $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$; forward-invariant safe sets use calligraphic $\mathcal{C} \subset \mathcal{S}$; state-space output mappings use bold $\mathbf{C} \in \mathbb{R}^{p \times n}$; electrical/thermal capacitance uses italic C ; center of mass is bold vector $\mathbf{c} \in \mathbb{R}^3$; safety shield diagnostic coverage uses $c_{shield} \in [0, 1]$.
$\mathbf{K}$	Camera intrinsic calibration matrix $\mathbf{K} \in \mathbb{R}^{3 \times 3}$	Contact stiffness ($k_{contact}$), control gains (K_p , K_d), motor constants (K_t , K_e), ensembling window K , or diffusion step k	Camera intrinsics use bold matrix $\mathbf{K}$; contact stiffness is lowercase scalar k or $k_{contact}$ (N/m); feedback control gains use capitalized scalars K_p , K_d ; motor torque/back-EMF constants use K_t , K_e ; temporal ensembling window length uses integer K ; diffusion noise steps use lowercase index k .
$\mathbf{M}$	Articulated inertia matrix $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$	Mass (m , m_{eff}), memory footprint ($M_{weights}$, M_{KV}), or fleet size (N_{fleet})	Articulated generalized mass/inertia uses bold matrix $\mathbf{M}(\mathbf{q})$; physical body mass is lowercase italic m (kg); effective impact mass is m_{eff} ; hardware memory storage uses capital M with descriptive subscript (in bytes); robot fleet size uses N_{fleet} or M_{fleet} .

Governing Relationships

Embodied intelligence operates subject to thermodynamic, mechanical, kinematic, and communications constraints. The relationships below state those constraints in the symbols the chapters use.

1. Sense-to-actuation delay budget

Delay converts directly into travel before any retarding force begins. The total pre-brake delay τ_{delay} runs from detection to retarding force, and the stop it leaves must fit inside the clear distance:

$$\tau_{\text{delay}} = t_{\text{age}} + T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}$$

$$v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{eff}}} + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \leq D_{\text{clear}}$$

Inference time is spent on the proposal side of the proposal boundary. It ages the evidence behind a proposal, which the permission path checks against the proposal’s expiry, but it is neither a term of τ_{delay} nor part of the permission loop’s period T_{tick} , because the fallback does not wait for the proposer.

Symbol	Definition	Unit	Notes
τ_{delay}	Total Pre-Brake Delay	ms	Time from detection of a hazard to the onset of retarding force; the machine travels $v\tau_{\text{delay}}$ before braking begins.
t_{age}	Observation Age	ms	Age of the state the permission path acts on, from the exposure midpoint at the sensor to the decision, which enters τ_{delay} .
T_{lease}	Permission-Path Lease	ms	Longest interval a proposal may keep driving the actuators without renewal, which enters τ_{delay} ; chosen, then checked against the ceiling the stopping budget derives.
T_{tick}	Permission-Loop Period	ms	Period of the permission loop, which enters τ_{delay} because a hazard may wait up to one period before it is checked.
T_{bus}	Command Transport Time	ms	Fieldbus delivery time from the permission path to the drive, which enters τ_{delay} .
T_{act}	Actuator Response Time	ms	Time from a delivered stop command to retarding force, including gate-drive and current rise, which enters τ_{delay} .
t_{sense}	Sensor Integration Time	ms	Half the exposure duration (its integration midpoint), ADC conversion, and sensor readout latency; contributes to t_{age} .
t_{xfer}	Interconnect Transfer Time	ms	DMA bus streaming, PCIe handoff, and serializer-deserializer (SerDes) link latency; contributes to t_{age} .
t_{infer}	Proposer Inference Time	ms	Forward-pass time of the learned proposer; spent above the proposal boundary and outside the permission loop’s period.

2. Action-chunk streaming cadence

Cognitive neural policies cannot execute at kilohertz frequencies due to memory bandwidth limits. Action chunking predicts an action trajectory of horizon H at a lower cognitive frequency f_{policy} , while the chunk’s waypoints are consumed at f_{exec} . The memory streaming latency must remain bounded by the chunk execution window:

$$t_{\text{stream}} = \frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}} \leq \frac{H}{f_{\text{exec}}} - t_{\text{compute}}$$

Symbol	Definition	Unit	Notes
H	Action Chunk Horizon	Time steps	Number of contiguous future action steps generated in a single policy forward pass ($H = 16$ on the mobile manipulator; 16 to 64 steps in the literature).
M_{weights}	Model Parameter Footprint	bytes	Total memory required to store policy weights ($P \times \text{bytes-per-parameter}$).

Symbol	Definition	Unit	Notes
BW_{mem}	Sustained Memory Bandwidth	bytes/s	Achievable memory throughput of the SoC or accelerator memory subsystem (e.g., LPDDR5, HBM).
t_{stream}	Parameter Streaming Time	ms	Time required to stream non-resident model weights into execution registers.
f_{exec}	Action Execution Rate	Hz	Temporal consumption rate of chunked waypoints along the planned trajectory (100 Hz to 1 kHz).

3. The dynamic stopping, braking, and safe-set invariants

Forward invariance of the safe set $\mathcal{C}$ requires that the machine can arrest all motion before it reaches the boundary. The stopping distance combines the travel during the pre-brake delay with the braking distance, and its margins are added in the inequality of the delay budget above:

$$d_{\text{stop}}(v) = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{eff}}}$$

Symbol	Definition	Unit	Notes
d_{stop}	Stopping Distance	m	Distance required to bring the plant to rest from speed v .
D_{clear}	Clear Distance	m	Clear distance from the machine to the boundary it must not cross; the subscript keeps it distinct from dataset size D .
v	Speed at Detection	m/s	Speed of the machine at the moment the hazard is detected.
a_{brake}	Credible Deceleration	m/s ²	Validated low tail of the deceleration the machine achieves under its stated conditions, not the datasheet peak.
a_{eff}	Effective Deceleration	m/s ²	Deceleration used in the braking term: a_{brake} for a constant-deceleration stop, $a_{\text{brake}}/1.5$ when the resident stop is the C^2 stopping suffix of equation 11.5.
δ_{loc}	Localization Bound	m	Bound on localization and state-estimation error.
δ_{margin}	Protective Clearance	m	Fixed, nonzero protective clearance between the machine envelope and the boundary.
ϵ_{track}	Tracking-Error Bound	m	Bound on the deviation between the commanded and the executed trajectory, stated by the enforcement record.
v_h	Human Walking-Speed Bound	m/s	Walking-speed bound for a person approaching the machine, used in the form its safety standard states.
j_{max}	Jerk Limit	m/s ³	Maximum rate of change of deceleration; appears only in jerk-limited refinements of the braking term, which the book's stopping equations omit.

4. The digital sampling zero-order hold (ZOH) phase margin

Holding discrete control commands constant over sampling period $T_s = 1/f_s$ introduces an effective transport delay of half the sample period, eroding feedback stability margins:

$$\tau_{\text{ZOH}} = \frac{T_s}{2}, \quad \Delta\Phi_{\text{total}} = \omega_c \left(\tau_{\text{comp}} + \tau_{\text{bus}} + \frac{T_s}{2} \right) \leq \text{PM}_{\text{margin}}$$

Symbol	Definition	Unit	Notes
τ_{ZOH}	ZOH Transport Delay	s	Equivalent continuous delay introduced by discrete Zero-Order Hold reconstruction ($T_s/2$).
T_s, f_s	Sampling Period and Rate	s, Hz	Discrete controller execution period and frequency ($f_s = 1/T_s$).
ω_c	Open-Loop Crossover Frequency	rad/s	Frequency at which open-loop gain $ L(j\omega_c) = 1$ (0 dB).

Symbol	Definition	Unit	Notes
$\Delta\Phi$	Phase Lag Degradation	rad / deg	Phase erosion ($\Delta\Phi \approx 28.65^\circ \cdot (\omega_c/f_s)$) depleting closed-loop stability margin.
PM_{net}	Net Phase Margin	deg	Residual stability margin ($\text{PM}_{\text{plant}} - \Delta\Phi_{\text{total}} > 0$) preventing contact chatter.

5. Actuator electromechanical and thermal rise invariants

Actuators are constrained by copper winding heating, DC bus rail droop, and magnetic saturation. Sustainable torque output is governed by the coupled electromechanical balance:

$$V_{\text{bus}} = L \frac{di}{dt} + R(T)i + K_e \omega_m, \quad C_{\text{th}} \frac{dT}{dt} = I^2 R(T) - \frac{T - T_{\text{amb}}}{R_{\text{th}}}, \quad \tau = \eta N_{\text{gear}} K_t I$$

Symbol	Definition	Unit	Notes
K_t, K_e	Torque and Back-EMF Constants	N · m/A, V · s/rad	Motor electromechanical coupling constants ($K_t \approx K_e$ in SI units).
$R(T)$	Temperature-Dependent Resistance	Ω	Stator copper resistance scaling: $R(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$ with $\alpha_{\text{Cu}} = 0.00393 \text{ K}^{-1}$.
R_{th}	Thermal Resistance (θ_{JA})	K/W	Lumped thermal resistance from stator windings to ambient environment.
C_{th}	Stator Thermal Capacitance	J/K	Lumped heat capacity of the motor core, windings, and potting compound.
τ_{th}	Thermal Time Constant	s	Actuator heating characteristic time constant ($\tau_{\text{th}} = R_{\text{th}} C_{\text{th}}$, typically 60–300 s).
D_{cycle}	Actuator Duty Cycle	[0, 1]	Ratio of active pulse duration to cycle period ($t_{\text{on}}/(t_{\text{on}} + t_{\text{off}})$) preventing thermal runaway.
ΔV_{droop}	DC Bus Dynamic Rail Droop	V	Transient voltage collapse ($\Delta I R_{\text{bus}} + L_{\text{bus}} \frac{dI}{dt} + \frac{\Delta I \Delta t}{2C_{\text{bus}}}$) under step motor current loads.

6. Shielded hazard architecture and statistical exposure bound

Catastrophic failure rates in embodied systems cannot be certified through stochastic testing alone. A deterministic safety shield bounds overall system risk, while zero-failure trial counts follow the Rule of Three:

$$p_{\text{system}} = p_{\text{brain}}(1 - c_{\text{shield}}) \leq p_{\text{target}}, \quad n \geq \frac{-\ln \alpha_{\text{sig}}}{p}$$

Symbol	Definition	Unit	Notes
p_{system}	Overall System Hazard Rate	failures/hour	Certified dangerous failure rate of the integrated cyber-physical machine.
p_{brain}	Neural Policy Defect Rate	failures/hour	Empirical rate of unshielded hallucinations, distribution drift, or policy blunders.
c_{shield}	Shield Diagnostic Coverage	Dimensionless	Fraction of hazardous neural commands successfully intercepted by deterministic QP/interlocks ($c \geq 0.9999$).
p_{target}	Target Risk Tolerance	failures/hour	Statutory regulatory safety requirement (e.g., 10^{-9} failures/h for SIL 4 / DAL A).
α_{sig}	Statistical Significance Level	Dimensionless	Type I error threshold ($\alpha_{\text{sig}} = 1 - \text{CL}$, e.g., $\alpha = 0.05$ for 95% confidence where $-\ln(0.05) \approx 3.0$).
n	Zero-Failure Sample Exposure	Hours or Cycles	Required zero-incident validation exposure ($n \approx 3/p$) to substantiate failure rate p .

1. Physical Plant State, Actuation, and Kinematic Topology

Symbol	Definition	Unit	Notes
$s_t \in \mathcal{S}$	Physical World State	Mixed	Ground-truth continuous state of the machine, environment, and physical contacts at time t .
$\mathbf{x}$	Mechanical Plant State	rad, rad/s	Generalized state vector $\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top \in \mathbb{R}^{2n}$ containing positions and velocities.
$\mathbf{q} \in \mathbb{R}^n$	Generalized Coordinates	rad or m	Degrees of freedom describing mechanical articulation (joint angles and prismatic link extensions).
$\dot{\mathbf{q}} \in \mathbb{R}^n$	Generalized Velocities	rad/s or m/s	First time derivative of generalized coordinates.
$\ddot{\mathbf{q}} \in \mathbb{R}^n$	Generalized Accelerations	rad/s ² or m/s ²	Second time derivative of generalized coordinates.
$\boldsymbol{\tau} \in \mathbb{R}^n$	Actuator Torque Vector	N · m or N	Generalized forces and torques delivered by actuators at mechanical joints.
$\mathbf{M}(\mathbf{q})$	Generalized Inertia Matrix	kg · m ²	Symmetric positive-definite mass/inertia matrix of the articulated multibody plant.
$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$	Coriolis / Centrifugal Matrix	N · m · s/rad	Matrix satisfying $\dot{\mathbf{M}} - 2\mathbf{C}$ skew-symmetry (passivity property).
$\mathbf{g}(\mathbf{q})$	Gravitational Torque Vector	N · m	Generalized gravitational load acting on links as a function of configuration.
$\mathbf{J}(\mathbf{q})$	Manipulator Jacobian	Mixed	Linear mapping $\mathbf{v}_e = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$ from joint rates to operational task-space velocity.
$w(\mathbf{q})$	Yoshikawa Manipulability	Dimensionless	Singularity proximity metric $w(\mathbf{q}) = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)}$ vanishing at kinematic singular configurations.
K_t, K_e	Motor Torque / Back-EMF	N · m/A, V · s/rad	Electromechanical coupling constants linking winding current to rotor shaft torque and back-EMF voltage.
e_{bemf}	Back-EMF Voltage	V	Opposing voltage $e_{\text{bemf}} = K_e \omega_m$ developed across stator coils by rotor rotation.
$J_{\text{rotor}}, J_{\text{load}}$	Rotor and Load Inertia	kg · m ²	Moment of inertia of motor rotor shaft and coupled mechanical load.
J_{ref}	Reflected Rotor Inertia	kg · m ²	Effective inertia $J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$ amplified quadratically by gearbox ratio.
N_{gear}	Gear Reduction Ratio	Dimensionless	Transmission reduction ratio ($N_{\text{gear}} \gg 1$) between motor rotor and joint output shaft.
N^*	Optimal Inertia-Matching Ratio	Dimensionless	Ratio $N^* = \sqrt{J_{\text{load}}/J_{\text{rotor}}}$ maximizing load angular acceleration under peak torque.
η_{gear}	Transmission Efficiency	Dimensionless	Mechanical power transmission efficiency ($0 < \eta_{\text{gear}} \leq 1$).
$\tau_{\text{hold}}, \tau_{\text{peak}}$	Continuous / Peak Torque	N · m	Thermally continuous holding torque and transient peak acceleration torque.
$I_{\text{cont}}, I_{\text{peak}}$	Continuous / Peak Current	A	RMS current ratings bounded by winding insulation class and inverter MOSFET dissipation.

2. Rigid-Body Contact Mechanics, Friction Cones, and Centroidal Dynamics

Symbol	Definition	Unit	Notes
$\mathbf{S} \in \mathbb{R}^{n \times (n+6)}$	Actuation Selection Matrix	Matrix	Matrix $[\mathbf{0}_{n \times 6}, \mathbf{I}_{n \times n}]$ projecting generalized forces onto actuated joints in floating-base systems.
$\mathbf{f}_k \in \mathbb{R}^3$	Contact Force Vector	N	Physical contact force exerted by environment on contact patch k ($\mathbf{f}_k = [\mathbf{f}_{k,\parallel}^\top, f_{k,\perp}]^\top$).

Symbol	Definition	Unit	Notes
$\mathbf{J}_k(\mathbf{q})$	Contact Point Jacobian	Mixed	Mapping joint velocities to contact patch Cartesian linear velocity: $\dot{\mathbf{p}}_k = \mathbf{J}_k(\mathbf{q})\dot{\mathbf{q}}$.
$f_{k,\perp}$	Normal Contact Force	N	Unilateral compressive force ($f_{k,\perp} = \mathbf{f}_k^\top \hat{\mathbf{n}}_k \geq 0$) perpendicular to contact plane.
$\mathbf{f}_{k,\parallel}$	Tangential Contact Shear	N	Friction traction force tangential to contact plane ($\mathbf{f}_{k,\parallel} = \mathbf{f}_k - f_{k,\perp} \hat{\mathbf{n}}_k$).
μ	Coulomb Friction Coefficient	Dimensionless	Static/kinetic coefficient bounding friction cone: $\ \mathbf{f}_{k,\parallel}\ \leq \mu f_{k,\perp}$.
$\mathbf{c} \in \mathbb{R}^3$	Center of Mass (CoM)	m	Spatial coordinates of the total articulated body mass centroid ($\mathbf{c} = \frac{1}{m} \sum m_i \mathbf{p}_i$).
$\mathbf{h}_{\text{lin}} \in \mathbb{R}^3$	Centroidal Linear Momentum	kg · m/s	Total plant linear momentum ($\mathbf{h}_{\text{lin}} = m\dot{\mathbf{c}}$), governed by $\dot{\mathbf{h}}_{\text{lin}} = \sum \mathbf{f}_k + m\mathbf{g}$.
$\mathbf{h}_{\text{ang}} \in \mathbb{R}^3$	Centroidal Angular Momentum	kg · m ² /s	Angular momentum about CoM: $\dot{\mathbf{h}}_{\text{ang}} = \sum (\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k + \tau_k$.
$\mathbf{p}_{\text{ZMP}} \in \mathbb{R}^2$	Zero Moment Point (ZMP)	m	Ground-plane point where net horizontal tipping moment vanishes; must reside in $\text{Conv}(\{\mathbf{p}_k\})$.
$e \in [0, 1]$	Coefficient of Restitution	Dimensionless	Kinematic velocity rebound ratio during non-smooth impact ($\dot{q}^+ = -e\dot{q}^-$).
k, k_{tissue}	Contact Stiffness	N/m	Elastic stiffness of structural environment or human biomechanical tissue.
m_{eff}	Effective Impact Mass	kg	Apparent mass of moving kinematic links along collision trajectory ($m_{\text{eff}} = (\mathbf{u}_{\text{dir}}^\top \mathbf{M}^{-1} \mathbf{u}_{\text{dir}})^{-1}$).
F_{unbraked}	Unbraked Impact Force	N	Transient collision force peak ($F_{\text{unbraked}} = v_{\text{max}} \sqrt{k_{\text{tissue}} m_{\text{eff}}}$) governed by ISO/TS 15066.
$E_{k,\text{allow}}$	Allowable Collision Energy	Joules (J)	Statutory kinetic energy limit ($\frac{1}{2} m_{\text{eff}} v^2 \leq E_{k,\text{allow}}$) for transient collaborative contact.

3. Geometry, Spatial Transformations, and Perception Projections

Symbol	Definition	Unit	Notes
$SE(3)$	Special Euclidean Group	Lie Group	Group of 3D rigid-body spatial transformations (translations and rotations: $\mathbb{R}^3 \rtimes SO(3)$).
$SO(3)$	Special Orthogonal Group	Lie Group	Group of 3×3 proper orthogonal rotation matrices ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det(\mathbf{R}) = +1$).
$\mathcal{F}_W, \mathcal{F}_B, \mathcal{F}_E$	Coordinate Reference Frames	Reference	Fixed World frame ($\mathcal{F}_W$), mobile robot Base/body frame ($\mathcal{F}_B$), and End-effector frame ($\mathcal{F}_E$).
$\mathbf{p} \in \mathbb{R}^3$	Cartesian 3D Translation	m	Translation vector $[x, y, z]^\top$ expressing relative point coordinates.
$\mathbf{R} \in SO(3)$	Directional Cosine Matrix	Matrix	3×3 orthonormal matrix parameterizing spatial attitude and orientation.
$\mathbf{q}_{\text{quat}} \in \mathbb{H}$	Unit Quaternion	Dimensionless	Hypercomplex orientation vector $[w, x, y, z]^\top$ satisfying $\ \mathbf{q}_{\text{quat}}\ = 1$.
$\mathbf{T}_{AB} \in SE(3)$	Homogeneous Transform	Matrix	4×4 matrix $\begin{bmatrix} \mathbf{R}_{AB} & \mathbf{p}_{AB} \\ \mathbf{0}^\top & 1 \end{bmatrix}$ mapping coordinates from frame B into frame A .
$\mathbf{Ad}_T$	Adjoint Representation	Matrix	6×6 transformation matrix mapping spatial twists and wrenches between coordinate frames.
$\omega, \alpha \in \mathbb{R}^3$	Angular Velocity & Acceleration	rad/s, rad/s ²	Spatial angular velocity and angular acceleration vectors of a rigid body.

Symbol	Definition	Unit	Notes
$\mathbf{r}_{P/O} \in \mathbb{R}^3$	Sensor Lever-Arm Vector	m	Displacement vector from body rotation origin O to displaced sensor location P .
$\mathbf{K} \in \mathbb{R}^{3 \times 3}$	Pinhole Camera Intrinsic	Pixels	Calibration matrix $\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ parameterizing focal lengths and principal center.
$\mathbf{P}_c = [\tilde{X}, Y, Z]^\top$	Camera-Frame 3D Coordinate	m	Metric coordinates of a spatial point in the optical camera frame (Z_c is optical depth).
$[u, v]^\top$	Image Plane Pixel Coordinates	Pixels	Discrete sensor pixel coordinates ($u \sim f_x \frac{X}{Z} + c_x$, $v \sim f_y \frac{Y}{Z} + c_y$).
t_{row}	Rolling Shutter Row Readout	μs	Exposure time delta between adjacent CMOS sensor scanlines.
$\Delta \mathbf{x}(y)$	Rolling Shutter Distortion	m	Kinematic skew displacement $\Delta \mathbf{x} = \mathbf{v} \cdot y \cdot t_{\text{row}}$ induced by high-velocity platform motion.
$\text{tsdf}(\mathbf{v})$	Truncated Signed Distance	$[-1, +1]$	Normalized volumetric distance $\text{tsdf}(\mathbf{v}) = \text{clamp}(d(\mathbf{v})/\mu_{\text{tsdf}}, -1, 1)$ of voxel $\mathbf{v}$ to nearest surface.
μ_{tsdf}	TSDF Truncation Band	m	Metric thickness of the spatial surface interface band stored in voxel memory.
$D_t(\mathbf{v}), W_t(\mathbf{v})$	TSDF Accumulated Volume	Mixed	Running weighted signed distance and accumulated visibility weight at voxel $\mathbf{v}$.
Σ_{depth}	Perception Depth Covariance	m^2	Anisotropic 3D uncertainty covariance ellipsoid associated with stereo/lidar/depth lifting.
t_{age}	Observation Age	ms	The observation age of the delay budget, split into its perception-side terms: the exposure midpoint $\frac{1}{2} t_{\text{exp}}$, sensor readout, transfer, and perception processing up to the decision.

4. Classical and Modern Control (State-Space, Observers, CBF-QPs)

Symbol	Definition	Unit	Notes
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Linear State-Space Matrices	Matrix	Continuous LTI plant: $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$, observed output $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{v}$.
$\mathbf{w}(t), \mathbf{v}(t)$	Process and Measurement Noise	Vector	Zero-mean Gaussian noise vectors with covariance matrices $\mathbf{Q} \succeq 0$ and $\mathbf{R} \succ 0$.
$\mathbf{P}(t) \in \mathbb{R}^{n \times n}$	Estimation Error Covariance	Matrix	Symmetric positive-definite matrix $\mathbb{E}[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^\top]$ evolving via matrix Riccati equation.
$\mathcal{O}, \mathcal{O}_{\text{NL}}$	Observability Matrices	Matrix	Kalman observability matrix $[\mathbf{C}^\top, (\mathbf{C}\mathbf{A})^\top, \dots]^\top$ and nonlinear Lie matrix $[d\mathbf{h}, dL_f\mathbf{h}, \dots]^\top$.
$\mathcal{C} \subset \mathcal{S}$	Forward-Invariant Safe Set	Set	Sublevel/superlevel set $\mathcal{C} = \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}$ where safety invariants are strictly maintained.
$\partial \mathcal{C}$	Safety Boundary Manifold	Boundary	Zero level set $\{\mathbf{x} \mid h(\mathbf{x}) = 0\}$ separating nominal operation from irreversible hazard.
$h(\mathbf{x}) \geq 0$	Control Barrier Function (CBF)	Scalar	Continuously differentiable function whose superlevel set defines forward-invariant set $\mathcal{C}$.
$L_f h(\mathbf{x})$	Lie Derivative along Drift	s^{-1}	Directional gradient derivative $\nabla h(\mathbf{x})\mathbf{f}(\mathbf{x})$ along unforced plant dynamics.
$L_g h(\mathbf{x})$	Lie Derivative along Control	Vector	Directional control effectiveness gradient $\nabla h(\mathbf{x})\mathbf{g}(\mathbf{x})$.
r	Barrier Relative Degree	Integer	Number of differentiations required before control input $\mathbf{u}$ appears ($L_g L_f^{r-1} h \neq 0$).

Symbol	Definition	Unit	Notes
$h_{\text{kinetic}}(\mathbf{x})$	Kinetic Inset Barrier Function	Scalar	Relative-degree-1 barrier: $h_{\text{kinetic}}(\mathbf{x}) = h(\mathbf{x}) - v^2 / (2a_{\text{brake}})$ incorporating stopping distance.
γ_{cbf}	Linear Barrier Decay Rate	s^{-1}	Extended class- $\mathcal{K}$ linear decay coefficient in barrier condition $h + \gamma_{\text{cbf}}h \geq 0$.
$\mathbf{u}_{\text{nom}} \in \mathcal{U}$	Nominal Control Command	$\text{N} \cdot \text{m} / \text{rad}$	Unverified control candidate proposed across the causal boundary by learned policy.
$\mathbf{u}^* \in \mathcal{U}$	Certified Control Action	$\text{N} \cdot \text{m} / \text{rad}$	Privileged, safety-filtered control command resolved by the real-time active-set CBF-QP.
$\alpha(\hat{t})$	Bumpless Transfer Blend	$[0, 1]$	C^2 quintic polynomial $6\hat{t}^5 - 15\hat{t}^4 + 10\hat{t}^3$ ramping between prior and candidate controllers.
$\tau_{\text{blend}}, T_{\text{blend}}$	Blending Transition Window	s	Duration of bumpless transfer; bounded by $\tau_{\text{blend}, \min} = 1.875 \ \Delta \mathbf{u}\ / j_{\text{allowable}}$.
$j(t) = \dot{\mathbf{u}}(t)$	Control Setpoint Jerk	$\text{m}/\text{s}^3, \text{N} \cdot \text{m}/\text{s}$	Time derivative of commanded plant acceleration or actuator torque during mode transitions.
K_p, K_d, K_i	PID / Feedback Gains	Mixed	Proportional, derivative, and integral gains for joint-level or operational-space error stabilization.
$\mathbf{D}_{\text{virt}}, \mathbf{K}_{\text{virt}}$	Virtual Damping and Stiffness	Matrix	Target impedance/admittance matrices in compliant interaction: $\tau_{\text{safe}} = -\mathbf{D}_{\text{virt}}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$.

5. Deep Learning for Robotics (Diffusion Policies, ACT, VLAs)

Symbol	Definition	Unit	Notes
$\pi_{\theta}(\mathbf{a} \mid \mathbf{o})$	Parameterized Neural Policy	Distribution	Deep policy network parameterized by weights θ , mapping observation tensors to action distributions.
$\mathbf{o}_t \in \mathcal{O}$	Multimodal Observation	Mixed	Synchronized tokenized sensor packet (RGB images, depth latents, joint encoders, tactile arrays).
$\mathbf{a}_t \in \mathcal{A}$	Cognitive Policy Action	Normalized	Unprivileged action emitted by neural policy π_{θ} (normalized joint delta, target pose, or gripper command).
$\mathbf{A}_{t:t+H}$	Action Chunk Trajectory	Tensor	Contiguous horizon sequence $[\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times d_a}$ predicted in a single forward pass.
H	Chunk Prediction Horizon	Time steps	Number of future action steps emitted per inference cycle (16 to 64 steps in the literature).
K	Ensembling Window Length	Time steps	Number of overlapping past chunks blended in receding horizon execution ($K \leq H$).
w_i	Temporal Ensembling Weight	Scalar	Exponential decay weighting factor $w_i = \exp(-m \cdot i)$ applied to chunk predictions at offset i .
$\mathbf{z} \in \mathbb{R}^d$	Latent Style / State Embedding	Dimensionless	Compressed semantic representation or CVAE latent style variable in ACT policies.
$\bar{\alpha}_k$	Diffusion Schedule Parameter	$[0, 1]$	Cumulative noise variance schedule parameter $\bar{\alpha}_k = \prod_{s=1}^k (1 - \beta_s)$ at diffusion step k .
$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	Diffusion Gaussian Noise	Tensor	Standard normal Gaussian noise corrupted onto clean action trajectories.
$\epsilon_{\theta}(\mathbf{o}, \mathbf{A}^k, k)$	Noise Prediction Network	Tensor	Neural network predicting added noise in Denoising Diffusion Probabilistic Models (DDPM).
$\mathcal{L}_{\text{diff}}(\theta)$	Diffusion Matching Loss	Scalar	Mean-squared error objective: $\mathbb{E}[\ \epsilon - \epsilon_{\theta}(\mathbf{o}_t, \mathbf{A}_t^k, k)\ ^2]$ optimized across denoising steps.
$\mathcal{L}_{\text{ACT}}(\theta, \phi)$	ACT Variational Objective	Scalar	CVAE objective combining trajectory reconstruction $\mathcal{L}_{\text{recon}}$ with latent KL-regularization βD_{KL} .
$q_{\phi}(\mathbf{z} \mid \mathbf{o}, \mathbf{A})$	CVAE Recognition Encoder	Distribution	Variational encoder in Action Chunking with Transformers parameterizing latent distribution $\mathcal{N}(\mu, \Sigma)$.

Symbol	Definition	Unit	Notes
ϵ_{BC}	Single-Step Imitation Error	$[0, 1]$	Bounded probability of an unforced policy error during an individual execution step.
$O(T^2 \epsilon_{\text{BC}})$	Compounding Covariate Drift	Error bound	Quadratic compounding trajectory drift under open-loop behavioral cloning rollouts of horizon T .
$\mathbf{W}_{\text{patch}}, \mathbf{E}_{\text{pos}}$	ViT Projection & Positional Bias	Matrix	Convolutional/linear patch projection matrix and spatial coordinate position embedding tensors.
$\mathbf{Z}$	Multimodal Fused Token Stream	Tensor	Concatenated token sequence $[\mathbf{z}_{\text{vision}}, \mathbf{z}_{\text{text}}, \mathbf{z}_{\text{proprio}}]$ ingested by embodied transformer backbones.
M_{KV}	Transformer KV-Cache Footprint	bytes	Autoregressive key-value cache memory footprint $(2 \times N_L \times d \times S \times s_{\text{elem}})$.
Λ_{intent}	Expiring Intent Lease	Permit	Cryptographically signed, bounded time-to-live operational lease granted to learned policies.

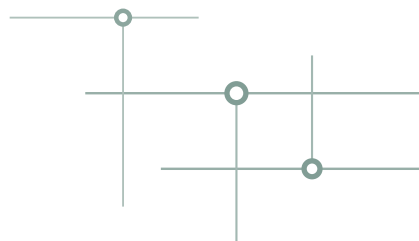
6. Embedded Systems, Silicon Real-Time, Thermal, and Hardware Safety

Symbol	Definition	Unit	Notes
I_{phase}	Motor Phase RMS Current	Amperes (A)	Electrical phase current driving stator windings in brushless PM motors ($I = \tau / (\eta N K_t)$).
$R_{\text{phase}}, R(T)$	Phase Winding Resistance	Ω	Stator copper DC resistance at temperature T ($R(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$).
$R_{\text{th}}, \theta_{JA}$	Lumped Thermal Resistance	K/W	Thermal impedance between internal hot spots and heatsink/ambient casing.
C_{th}	Stator Thermal Capacitance	J/K	Stator core heat capacity governing temperature rise dynamics ($\tau_{\text{th}} = R_{\text{th}} C_{\text{th}}$).
ΔT	Joulean Temperature Rise	Kelvin (K)	Steady-state or transient thermal rise ($\Delta T = P_{\text{cont}} R_{\text{th}}$).
$L_{\text{bus}}, R_{\text{bus}}$	Power Rail Inductance / ESR	$\mu\text{H}, \text{m}\Omega$	Parasitic power wiring harness inductance and Equivalent Series Resistance feeding inverters.
C_{bus}	DC Bus Bulk Capacitance	μF	Inverter decoupling capacitor bank buffering transient inductive voltage collapse.
ΔV_{droop}	Dynamic Rail Voltage Droop	Volts (V)	Transient voltage dip ($\Delta I R_{\text{bus}} + L_{\text{bus}} \frac{dI}{dt} + \frac{\Delta I \Delta t}{2 C_{\text{bus}}}$) triggering undervoltage reset.
$t_{\text{vulnerable}}$	Seqlock Vulnerable Interval	μs	Race window ($t_{\text{write}} + t_{\text{read}}$) during lock-free inter-processor shared SRAM exchanges.
$P_{\text{collision}}$	Seqlock Retry Probability	Ratio	Probability $t_{\text{vulnerable}}/T_{\text{writer}}$ that reader thread observes torn telemetry and must retry.
t_{queue}	Memory Bus Queue Delay	ms	Contention latency $\sum B_k / \text{BW}_{\text{peak}} + N_{\text{conflict}} t_{\text{penalty}}$ across shared SoC crossbars.
N_{conflict}	Interconnect Port Conflicts	Count	Number of simultaneous DMA / GPU / CPU memory channel access collisions.
Δt_{fresh}	Telemetry Freshness Deadline	ms	Maximum allowable age of sensor telemetry before an interlock trips: $\Delta t_{\text{fresh}} \leq e_{\text{max}} / v_{\text{max}}$.
t_{watchdog}	Hardware Watchdog Timeout	ms	Hardware timer period triggering fail-safe dynamic braking upon lost processor heartbeats.
t_{NMI}	Non-Maskable Interrupt Time	μs	Deterministic hardware interrupt response time for motor overcurrent or gate-drive faults.
δ_{skew}	Clock Synchronization Skew	ppm, $\mu\text{s/s}$	Hardware crystal drift between the application processor and safety microcontroller clocks.
Δt_{peak}	Peak Inter-Node Phase Jitter	μs	Cumulative timing uncertainty $\delta_{\text{skew}} \Delta T_{\text{blackout}} + \sigma_{\text{jitter}}$ during bus sync blackouts.

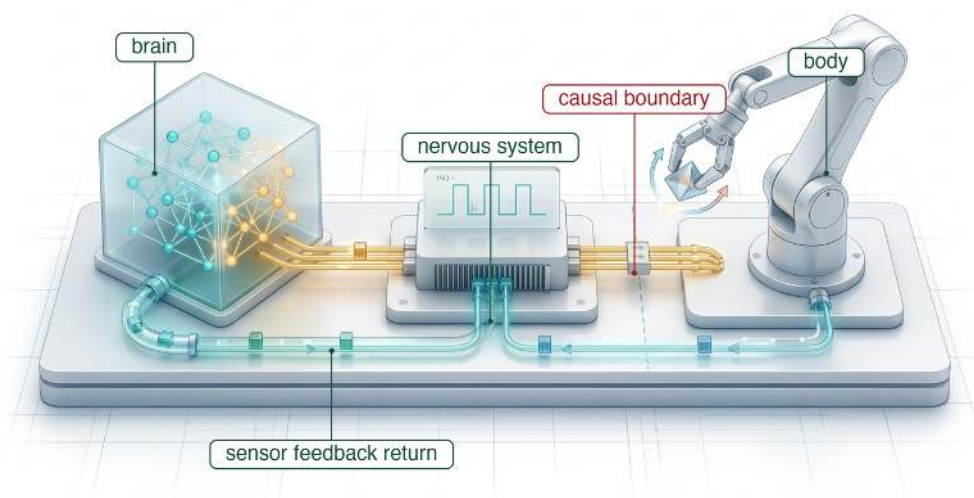
Symbol	Definition	Unit	Notes
MTBUF	Mean Time Between Unsafe Failure	Hours	Operating hours between undetected hazardous safety violations: $MTBUF = \frac{1}{\lambda_{\text{raw}}(1 - c_{\text{shield}})}$.
$\mathcal{T}_{\text{trichotomy}}$	Trichotomy Monitor State	Enum	Three-valued real-time monitor evaluation: {Known-True, Known-False, Unknown}.

DRAFT

MAIN TEXT



The Causal Boundary



- 1.1 Artificial Intelligence Beyond Glass
- 1.2 The Causal Boundary
- 1.3 Three Machine Classes
- 1.4 The Physical AI Scope Test
- 1.5 The Machine in Five Levels
- 1.6 When the Boundary Fails
- 1.7 The Four Bedrock Laws
- 1.8 Safety Convergence
- 1.9 The Architecture of This Book
- 1.10 Fallacies and Pitfalls
- 1.11 Summary

Purpose

What must a machine guarantee before a learned model's proposal becomes physical work?

A learned policy may select actions the machine cannot safely execute. Once a command reaches a motor, current generates torque and moving mass carries momentum. Canceling the computation cannot undo that motion. As a robot's software processes observations, its physical chassis continues coasting toward an obstacle. Every millisecond of computational delay consumes physical space, irreversibly shrinking the available braking envelope. Prediction accuracy alone cannot determine if an action should proceed; a policy with 99.9 percent benchmark precision still commands catastrophic collisions when scaled across thousands of operating hours in unconstrained physical environments.

Physical artificial intelligence resolves this tension by structuring the machine across five distinct operating levels. High-capacity learned models residing in the Brain propose candidate trajectories, but between statistical proposals and physical execution sits the causal boundary: the definitive interface where digital bits command motor currents. The Brain holds proposal authority, while the deterministic Nervous System retains hard real-time execution authority to verify commands against the Body's measured kinematic and thermal limits. When a learned policy experiences tail latency, memory exhaustion, or semantic error, the Nervous System arrests motion via a verified stopping suffix before the machine exhausts its clearance budget.

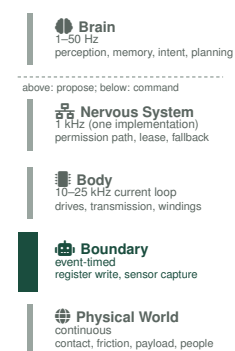


Figure 1.1: The five-level machine stack with the Boundary level highlighted.

Downstream: The plant limits behind the causal boundary are measured and recorded in section 2.7.



Learning Objectives

- Explain why a command that crosses the causal boundary cannot be recalled by software
- Calculate the blind travel of a pre-brake delay and compare it with the braking distance
- Classify physical machines by the budget that exhausts first: clearance, contact force, or a thermal or phase margin
- Apply the physical AI scope test to distinguish advisory models from autonomous physical systems
- Locate the proposal boundary in the five-level machine and explain why only the permission path may command the actuators
- Diagnose how timing and authority failures combine in an autonomous machine incident
- Evaluate how irreversibility, closed-loop data, and limited evidence each make an independent permission path necessary
- Explain why physical authority depends on state, so that safety, security, and accountability meet at the permission path

1.1 Artificial Intelligence Beyond Glass

A robot approaching an open doorway must do more than recognize the portal in its camera stream. It must evaluate whether its physical chassis fits through the opening, select a viable trajectory, and continuously adjust that path as momentum carries it forward. If a sudden obstacle appears, canceling the software thread or throwing an unhandled exception cannot stop the machine's momentum. Designing this behavior requires tracing a decision all the way from sensory evidence to electromagnetic torque and mechanical work. That complete closed loop is the domain of physical artificial intelligence systems engineering.

In a physical machine, digital bits can command electrical currents across power inverters, magnetic flux generates mechanical torque, and moving mass accumulates momentum. An erroneous command can therefore produce a sheared gearbox, a collapsed battery voltage rail, or a collision before software can correct it.

Moravec's paradox¹ (Moravec 1988) explains why learned models are entering this loop at all: tasks that require intense human conscious effort, such as playing chess or proving theorems, require modest computation, whereas sensorimotor skills executed effortlessly (such as walking across uneven gravel or grasping an egg without crushing it) demand vast computational and control resources. As high-capacity neural networks are deployed to master these sensorimotor tasks, engineering teams encounter the exposure wall, which section 7.5 derives. Offline benchmark accuracy cannot predict closed-loop stability in the open world, and no feasible amount of failure-free testing can by itself demonstrate the rare-failure rates that growing physical deployment requires, so safety arguments must cover cumulative exposure, not only controlled demonstrations.

When digital models command physical actuators, they enter the closed causal feedback loops first formalized by Norbert Wiener in cybernetics² (Wiener 1948). Information ceases to be an open-loop stream terminating at a display. Instead, every physical motion repositions the machine's sensors, reshaping subsequent observations under the deterministic laws of classical mechanics. Building such a machine draws on four technical cultures (machine learning, computer systems, classical robotics, and embedded silicon and safety), and teams that bridge them find each culture framing the problem in its own terms.

The machine learning perspective focuses on scaling parameter counts, transformer backbones, and diffusion policies across millions of simulation steps, treating real-world deployment as a challenge of broader demonstration datasets and dataset coverage. The computer systems perspective focuses on memory bus bandwidth, cache hierarchies, and tail-latency percentiles, recognizing that streaming multi-gigabyte neural weights at high frequencies threatens to saturate memory interconnects and starve camera direct memory access pipelines (Williams et al. 2009). The classical robotics perspective relies on Newton-Euler dynamics to manage mechanical forces and motion. If a neural network commands rapid changes in motion (high-frequency jerk), it can shear gearboxes or trigger structural resonance, making classical stability margins critical. Finally, the embedded

¹ | **Moravec's Paradox:** The observation in artificial intelligence (Moravec 1988) that high-level reasoning requires comparatively little computation, whereas low-level sensorimotor coordination demands enormous perceptual and computational resources.

² | **Cybernetic Feedback Loops:** In Norbert Wiener's cybernetic framework (Wiener 1948), feedback denotes adjusting future action using past performance.

silicon and safety perspective focuses on bare-metal interrupt service routines, hardware trip zones, and functional safety standards (such as ISO 26262 and UL 4600), ensuring that unverified statistical models never hold direct write access to motor power stages.

Each perspective offers a critical insight. Gradient descent does not repair a fractured transmission or prevent a supply-voltage collapse, while a hand-tuned classical controller cannot provide open-world semantic scene understanding. A viable physical AI systems architecture must unify their responsibilities without asking any single discipline to shoulder the entire problem.

Downstream: The separation of learned proposals from actuation authority is mapped to isolated silicon domains in section 4.2.

DRAFT

Definition 1.1: Physical artificial intelligence

Physical artificial intelligence is the engineering of closed-loop physical machines in which high-capacity learned models propose actions that classical mechanics, thermodynamics, and hard real-time silicon constrain.

1. **Significance:** Advisory AI outputs lack direct actuator authority, although people or downstream systems may act on them. Physical AI crosses into a continuous plant, where digital bits command motor currents, kinetic momentum accumulates, and software stalls can contribute to mechanical fractures, overheating, or collisions.
2. **Distinction:** Unlike classical robotics (which relies on hand-engineered state machines and explicit kinematic models) or internet-scale machine learning (which operates on static offline datasets), physical AI couples high-capacity statistical generalization with deterministic hardware safety filters across an endogenous closed causal loop.
3. **Common pitfall:** Treating physical embodiments as peripheral I/O devices for foundation models. Scaling laws and benchmark accuracies do not prevent supply-voltage collapse, gearbox shear, or actuator force saturation; physical safety requires an asymmetric architecture in which an independent real-time check can veto any learned proposal.

The chapter follows a command from computation to physical work, compares the budgets of three machine classes, applies a scope test to decide which machines fall within the discipline, orders the machine into five levels, and uses the Tempe collision to test the boundary between learned proposals and actuation permission. The final sections turn that failure analysis into four laws, trace where safety, security, and accountability meet, and set out the architecture of the book.

1.2 The Causal Boundary

An advisory model that only displays a recommendation has no direct path to a motor or valve. A user may dismiss the recommendation, and a database transaction may be rolled back before an external action commits.³ Neither property makes digital errors harmless: a person may follow bad advice, and an automated trading system may execute a consequential order. The relevant distinction is whether a model's output can automatically reach a physical actuator.

Physical AI removes the gap between a model's output and an actuator. In a physical machine, the final instruction of a control loop does not write to a screen or populate a web response. For a machine learning systems engineer accustomed to writing `action = policy(observation)` in PyTorch, the causal boundary is where floating-point tensors meet physical silicon. The policy's output tensor, typically an action chunk predicting a sequence of upcoming waypoints (joint positions, velocities, or 6-DOF Cartesian poses),⁴ cannot execute directly on physical motors.

An independent check must stand between a learned proposal and the actuator command. If the check admits the proposal, its setpoints pass through a deterministic cascade of drive loops that turns them into motor current, and the last software act in that cascade is a value written to a hardware register. Depending on the machine architecture, that hardware register acts as a digital-to-analog converter latch holding a commanded reference voltage, a pulse-width modulation (PWM) compare register setting inverter duty cycle, or a serial control word configuring a gate driver. At that exact memory address, informational abstraction ends. Modulating the gate voltages across power MOSFETs draws current from the high-voltage DC bus into the stator windings, generating Lorentz forces that accelerate the rotor against its reflected load inertia.

Definition 1.2: Causal boundary

Causal boundary is the physical and electrical interface where digital computational bits convert into physical forces, currents, or flows.

³ | **Idempotency:** Repeating an idempotent operation has the same specified result as executing it once. A motor setpoint command can have idempotent command semantics, but repeated execution may still dissipate energy. Idempotency does not imply reversibility or harmlessness.

⁴ | **Action Chunk Representations:** In robot learning literature (Zhao et al. 2023), action chunks are typically tensors of shape $[B, H, d_a]$ where B is batch size, H is the planning prediction horizon, and d_a is action dimension (such as target joint positions $\hat{\mathbf{q}} \in \mathbb{R}^d$ or 3D Cartesian position and orientation deltas). The formal Lie group formulation ($SE(3)$) is deferred to section B.4.3; chapter 3 defines the chunk and its horizon.

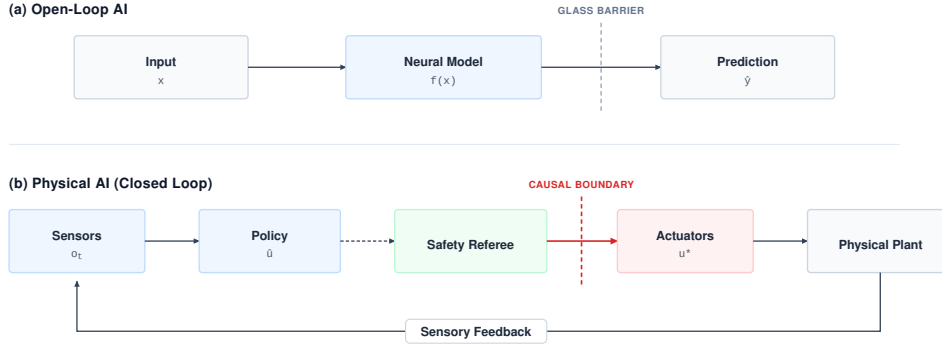


Figure 1.2: The closed causal loop: Advisory output (top) stops at an informational interface. In physical AI (bottom), permitted model proposals reach hardware registers, where electrical current produces force and motion. Those actions alter the environment and subsequent sensor observations.

1. **Significance:** Before actuation, a proposal can be rejected without applying physical work. After actuation, actions can alter the environment, dissipate heat (I^2R resistive heating, kinetic friction), and reshape subsequent sensory observations.
2. **Distinction:** Unlike a staged digital proposal, which can be rejected before it commits, an applied force cannot be canceled, and the physical work it has already performed remains in the plant.
3. **Common pitfall:** Assuming software exceptions or thread cancellations can arrest physical momentum. Once a PWM compare register latches, current flows and mass accelerates; arresting that motion requires dedicated stopping distance (d_{stop}) and friction braking.

Transitioning from digital state to physical force breaks the assumption, central to offline machine learning, that the data do not depend on the model's outputs. In offline vision, natural language processing, and tabular prediction, data collection is exogenous: the world generates a distribution of inputs, and the model processes them without altering the mechanism that created them. In a physical system, the environment evolves in a closed causal loop: the next physical state and sensor observation depend directly on the action produced by the prior model inference.

The sensory input stream is endogenous, generated by the machine's own past actions. When a policy produces an erroneous torque command, the joint rotates, the platform shifts, and mounted cameras or lidar units swing into new viewpoints. The machine drives itself into an unfamiliar region of the state space where training data is sparse, increasing the probability of subsequent inference errors and compounding the divergence from nominal operation (figure 1.2).

Because physical mass possesses inertia and computation takes finite time, latency in physical AI translates directly into physical displacement. As shown on the autonomous testbed in figure 1.3, real-world robotic systems rely on dense multi-modal sensor suites (spinning lidar scanners, camera arrays, and radar units) that continuously stream high-bandwidth telemetry to onboard processors. Every stage in this pipeline, from photodiode exposure and sensor bus serialization,⁵ to neural inference on accelerator silicon and packet transmission over deterministic fieldbuses, consumes milliseconds.

As a concrete example, consider the wheeled base of a warehouse mobile manipulator (introduced in full in section 1.3), cruising down an aisle at its 1.5 m/s drive limit (illustrative; see the Reader Guide). Its pre-brake delay, which later chapters assemble term by term, is at most 133.6 ms, from the capture of the camera frame that shows a person stepping out from a rack end to the onset of braking force. Over that interval the base travels 200 mm with no response to anything the frame recorded. A person who steps out closer than that is reached before braking begins, and no amount of model capacity can alter that outcome.

⁵ | **Automotive Serial-izer/Deserializer (SerDes):** High-speed physical-layer links (such as FPD-Link or GMSL) that transport uncompressed, low-latency multi-gigabit video streams over lightweight coaxial or shielded twisted-pair cables. Combined with dedicated hardware trigger lines, SerDes interfaces ensure microsecond-synchronized shutter capture across the sensor ring. For bus specifications, see Appendix C.

This trade-off between computational delay and physical clearance is formalized by the **kinematic roofline**. In classical computer systems architecture, the Roofline Model (Williams et al. 2009) bounds achievable throughput by the lower of two ceilings, peak memory bandwidth times arithmetic intensity (the ratio of arithmetic operations to bytes moved, in FLOPs/byte) and peak compute. In physical AI, the kinematic roofline couples the delay before braking τ_{delay} and platform velocity v to bound physical stopping clearance against the credible braking deceleration a_{brake} , but it adds its two terms rather than capping one by the other.

Under emergency braking, the total stopping distance d_{stop} is the sum of the blind travel during the delay, which grows linearly with the delay, and the braking distance, which grows with the square of speed. Section 2.3 derives the relation. The blind travel is non-negotiable. During the delay, no braking force can be applied, because the sensor exposure has not yet propagated through perception and the fieldbus to the brakes. As table 1.1 shows across representative embodied platforms, the delay’s share of the stop is largest for slow, agile platforms, while its absolute travel is largest for fast vehicles.

Table 1.1: Kinematic roofline nomogram: Stopping clearance trade-offs across embodied platforms. The warehouse mobile manipulator’s row uses its drive limit and credible deceleration. The three delay columns are classes of pre-brake delay, not the delay of any one platform. Stopping distances are rounded to three significant figures, and the added blind travel is given in centimeters to the nearest millimeter. At 200 ms of delay, an autonomous passenger car travels an additional 4 m, an entire vehicle length, before its brakes engage.

Physical Platform & Velocity v	Braking Decel a_{brake}	Me- chanical Braking d_{braking}	$\tau_{\text{delay}} = 10$ ms (Reflex Class)	$\tau_{\text{delay}} = 50$ ms (Short Class)	$\tau_{\text{delay}} = 200$ ms (Long Class)	Latency Fraction at 200 ms
Humanoid / Quadruped (1 m/s = 3.6 km/h)	4 m/s ² (0.41 g)	0.125 m	0.135 m (+1 cm)	0.175 m (+5 cm)	0.325 m (+20 cm)	61.5% of total stop
Warehouse Mobile Manipulator, Base (1.5 m/s = 5.4 km/h)	2 m/s ² (0.20 g)	0.563 m	0.578 m (+1.5 cm)	0.638 m (+7.5 cm)	0.863 m (+30 cm)	34.8% of total stop
Urban Delivery Bot (6 m/s = 21.6 km/h)	4.5 m/s ² (0.46 g)	4 m	4.06 m (+6 cm)	4.30 m (+30 cm)	5.20 m (+120 cm)	23.1% of total stop
Autonomous Vehicle (20 m/s = 72 km/h)	7 m/s ² (0.71 g)	28.6 m	28.8 m (+20 cm)	29.6 m (+100 cm)	32.6 m (+400 cm)	12.3% of total stop

Just as an architect cannot resolve a memory-bound arithmetic pipeline by adding more ALU execution units, an autonomous vehicle systems engineer cannot resolve an emergency stopping clearance violation by adding model parameters if doing so lengthens the delay τ_{delay} before braking. A heavier foundation model that adds 150 ms of computation consumes an extra 3 m of physical travel at highway speeds, distance that no friction brake or collision-avoidance logic can recover.

The physical reality of this trade-off is captured longitudinally across five decades of autonomous vehicle systems (figure ??). The safe operating speed of any physical AI platform is strictly bounded by the **kinematic stopping roofline**:

$$d_{\text{stop}} = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{max}}} \leq R_{\text{sensor}}$$

where v is vehicle velocity, τ_{delay} is the end-to-end sense-to-act loop latency, a_{max} is emergency braking deceleration ($0.7g \approx 6.86\text{ m/s}^2$ on dry asphalt), and R_{sensor} is the effective sensor perception horizon.

In the early deliberative era (1970–1980), SRI’s Shakey and Moravec’s Stanford Cart operated with loop latencies between 15 and 30 seconds due to offboard mainframe compute bottlenecks, capping safe crawl speeds below 0.2 km/h. By the mid-1980s and 1990s, Ernst Dickmanns’ VaMoRs demonstrated that running custom 4D dynamic vision algorithms at 25 Hz ($\tau = 40$ ms) unlocked 96 km/h highway speeds on the German Autobahn, proving that control loop cadence—rather than raw cognitive deliberation depth—governs safe mobility. During the DARPA Grand Challenges (2004–2007), Stanford Stanley and CMU Boss operated at 10–12 Hz (85–100 ms) to navigate desert



Figure 1.3: Lidar sensor ingress: A 64-beam spinning optical lidar assembly mounted aboard an autonomous research testbed. In physical AI, sensor streams delivering over 1.3 million points per second must be serialized over real-time hardware interfaces, synchronized to microsecond timestamps, and processed through neural perception backbones within strict latency deadlines. (Source: Steve Jurvetson, CC BY 2.0).

trails and urban intersections at 48–60 km/h. Today, production robotaxis such as the Waymo Driver maintain ~ 80 ms loop latencies to ensure stopping envelopes remain well within their 200 m sensor horizons at highway velocities (110 km/h), while autonomous racing vehicles (such as the Indy Autonomous Challenge PoliMOVE team) push loop latencies down to 28 ms (> 35 Hz) to operate safely at speeds exceeding 309 km/h.

Once digital commands transition into mechanical work, the state change becomes thermodynamically irreversible. The physical world provides no rollback mechanism.⁶ When an actuator controller applies current to a motor, electrical energy transfers from the battery into kinetic energy in mechanical linkages or thermal energy dissipated through resistive Joule heating (I^2R) in the stator windings. If an incorrect joint torque drives a manipulator into a rigid steel fixture, the kinetic energy converts into plastic deformation of the tool, mechanical strain on the gearbox teeth, and heat. No software interrupt can pull that kinetic energy out of the crushed structure or restore sheared gear teeth to their original geometry.

Physical AI inherits the learned model of advisory machine learning and the actuator of classical control, and with them a combination of failure modes that neither field faces alone (table 1.2).

Table 1.2: Comparison matrix of computing paradigms across the causal boundary: Advisory digital machine learning (left), classical robotics and control theory (middle), and physical artificial intelligence (right) compared by operational substrate, model formulation, feedback dynamics, execution cadence, actuation authority, failure semantics, and safety certification primitives.

Architectural Dimension	Digital Machine Learning (Advisory)	Classical Robotics & Control	Physical Artificial Intelligence (Closed-Loop)
Operational Substrate	Decoupled virtual memory buffers (GPU VRAM, DRAM, web endpoints)	Continuous analog state space governed by differential equations	Hybrid: high-dimensional neural representations mapped to analog physical actuators
Model Formulation	High-capacity statistical approximators (LLMs, ViTs, diffusion policies)	Analytically specified transfer functions and deterministic state-space matrices	Foundation policies paired with deterministic runtime safety filters

⁶ | **Thermodynamic Limits of Computation:** At a fundamental level, Landauer (1961) established that erasing logical bits dissipates at least $k_B T \ln 2$ of heat per erased bit. In physical AI systems, however, the dominant irreversibility is macroscopic: motor inverters draw tens of Amperes from battery packs, dissipating kilowatt-scale Joule heat (I^2R) and imparting momentum to moving linkages.

Downstream: The non-smooth contact mechanics and torque saturation limits governing motor irreversibility are derived in section 2.4.

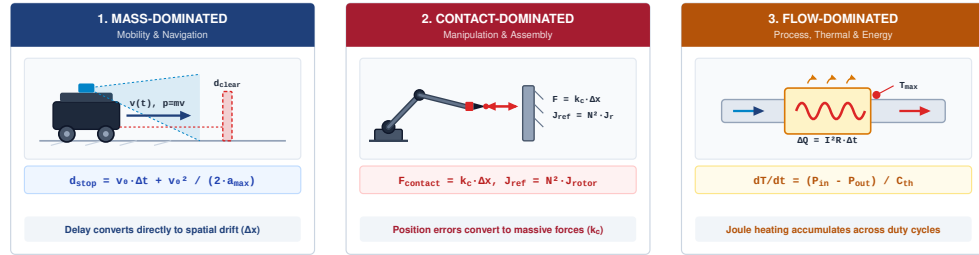


Figure 1.4: Three machine classes: The three foundational physical embodiments classified by their primary binding conservation laws: (a) mass-dominated mobility (Class 1) governed by momentum and spatial clearance; (b) contact-dominated manipulation (Class 2) governed by mechanical impedance and structural yield stress; and (c) flow-dominated process plants (Class 3) governed by thermal capacitance and transport lag.

Architectural Dimension	Digital Machine Learning (Advisory)	Classical Robotics & Control	Physical Artificial Intelligence (Closed-Loop)
Feedback Dynamics	Exogenous and static; independent and identically distributed samples	Continuous deterministic state feedback with Gaussian sensor noise	Endogenous, non-stationary closed loop; actions shape future observations
Execution Cadence	Throughput-optimized batching; soft or unbounded latency budgets	Hard real-time single-rate loops on microcontrollers (100 Hz to 1 kHz)	Multi-rate: an intent model (1–5 Hz) and a chunk policy (10–50 Hz) propose; a permission loop, 1 kHz in one implementation, admits
Actuation Authority	Advisory; sandboxed APIs, display buffers, or human-in-the-loop validation	Hardcoded feedback directly driving power inverter timers and PWM registers	Learned proposals checked by an independent path before any reaches an actuator
Failure Semantics	Outputs may be dismissed or transactions reversed before external action; downstream harm remains possible	Actuator saturation, tracking error, or loss of stability	Irreversible thermodynamic work, plastic deformation, kinetic collision, or stator overheating
Safety Assurance	Offline statistical loss minimization on validation splits	Formal mathematical stability proofs (Lyapunov functions, gain/phase margins)	Layered runtime invariant enforcement, forward-invariant barrier filters, and fault injection

The failure-semantics and actuation-authority rows of the right-hand column trace to irreversibility. A permitted command delivers work that no later software action can recall, so the check on it has to finish first. What that work damages first, and how quickly, depends on what the machine moves, and that dependence sets the deadline the check must meet.

1.3 Three Machine Classes

An autonomous delivery drone navigating turbulent wind gusts seems to share nothing with a six-axis robotic arm assembling an automotive transmission, and neither appears related to a containerized battery management plant regulating liquid coolant. One curriculum can cover them all, without becoming a collection of ad hoc recipes, only if it classifies them by something deeper than their shape.

The key is to look past cosmetic form factors to the underlying physical conservation laws. Every embodied AI system is governed by one or more of three foundational physical classes, each defined by the conservation law that dominates its actuators (figure 1.4, figure 1.5): moving mass across space, applying force through physical contact, or regulating continuous fluid and thermal flows. A humanoid couples all three on one chassis. Classifying a machine this way allows system architects to determine which resource budget exhausts first during a software stall or anomaly and to size the machine’s safety response to it. Table 1.3 sets those budgets beside the failure each one ends in and the gate that has to catch it.



Figure 1.5: Three machine classes in the field: Field deployments representing the three foundational physical conservation regimes: (a) mass-dominated mobility exemplified by Boston Dynamics’ Spot quadruped negotiating unstructured terrain; (b) contact-dominated manipulation exemplified by the Franka Emika Panda collaborative arm regulating fine assembly contact forces; and (c) flow-dominated process plants exemplified by containerized utility-scale battery energy storage systems (Tesvolt BESS) governing high-capacity thermal dissipation and liquid cooling loops. (Attribution: Wikimedia Commons, CC BY-SA 4.0).

Table 1.3: Binding budget by class: Each class exhausts a different resource first when software stalls, and that resource determines the enforcement mechanism the machine must carry. The gates are design responses to the failure mode in the same row, not certified performance claims.

Class	Exemplars	First budget to exhaust	Failure mode	Safety gate
Mass-dominated	AMRs, AVs, drones, mobile bases	Stopping distance and pre-brake delay (τ_{delay})	Kinetic collision; momentum breach	Independent braking check at a rate set by the stopping budget (1 kHz in one implementation)
Contact-dominated	Manipulators, grippers, presses	Actuator current and gearbox stress	Gear tooth shear; workpiece damage	Torque-derivative clamp and C^2 -smooth trajectory limit (continuous acceleration)
Flow-dominated	Thermal loops, inverters, bioreactors	Thermal or phase margin	Thermal runaway; insulation blowout	Hardware thermal observer and interlock

This book follows one machine from its first chapter to its last. The warehouse mobile manipulator is a wheeled base that carries a seven-joint arm, a navigation camera and lidar, a wrist camera, and two computers. An application processor runs its learned models, and a separate safety microcontroller decides, on a fixed short period, whether a proposed motion may reach the motors. Its chunk policy is a whole-body action-chunking model that proposes base velocity and arm joint targets together, as mobile manipulation systems of this class do (Fu et al. 2024). The base is a mass-dominated Class 1 machine that must stop before it reaches a person who steps out from the end of a rack. The arm is a contact-dominated Class 2 machine that must open a spring-latched cage door, lift an unfamiliar mug from a moving conveyor, and hand the mug to a coworker at the packing station, each without exceeding a contact force. The chapters that own a term of the machine’s stopping budget add that term in turn, so the value one chapter computes is the value the next one uses. The remaining chapters treat the proposer that spends that budget and the evidence that decides how much of it the proposer may spend. A Class 3 process plant appears once in each part as a contrast.

1.3.1 Class 1: Mobility systems

Class 1 machines move mass across space, bound by momentum conservation and kinetic energy dissipation (figure 1.4 a, figure 1.5 a). In autonomous transport platforms, quadruped robots navigating rugged terrain (such as Boston Dynamics’ Spot), mobile manipulators, and aerial drones, the actuator bridge applies force to translate or rotate a body with mass m at velocity v . Because kinetic energy $E_k = \frac{1}{2}mv^2$ cannot be instantaneously erased once delivered to the chassis (it must be scrubbed off through friction, aerodynamic drag, or regenerative braking), safety is primarily measured as remaining spatial clearance. The stopping distance of table 1.1 is this class’s budget, and the first boundary breached during a software stall or model misclassification is the physical distance to an external obstacle.

To see how the two terms of that budget scale, return to the warehouse base cruising at its 1.5 m/s drive limit toward a rack end where a person has stepped out. Loaded with its tote rack, its mass is at most 368 kg, and it carries 414 J of kinetic energy, which braking must shed through the grip of its wheels on the floor. At a credible deceleration of 2 m/s², braking alone takes 562.5 mm of floor. The 200 mm of blind travel over the 133.6 ms pre-brake delay comes before any of that braking and adds 35.6 percent to the braking distance. Table 1.1 reaches a similar figure for the same base, 34.8%, by a different measure, the travel over its 200 ms long delay class as a share of the whole stop. The two terms answer to different design choices. A larger model that lengthens the delay lengthens the blind travel in proportion, whereas doubling the cruising speed would quadruple both the energy and the braking distance.

1.3.2 Class 2: Manipulation systems

Class 2 machines apply mechanical force through contact, limited by actuator torque saturation, environmental stiffness, and current loop bandwidth, meaning how quickly the motor controller senses and regulates current to control torque (figure 1.4 b, figure 1.5 b). In articulated manipulators, precision assembly fixtures, and robot learning workcells executing contact-rich insertion and grasping, the end-effector interacts directly with a rigid or compliant workpiece. Unlike a mobile platform, an arm in contact with a surface cannot resolve unexpected displacement by rolling forward. When an actuator commands position into a rigid boundary, physical displacement is blocked, and the interaction acts like a compressed spring governed by mechanical impedance. By Hooke's law, the contact force generated by a spatial penetration error Δx against an environment of stiffness k is $F = k\Delta x$.⁷

Contact stiffness can make a held motion command expensive before the high-level planner returns. Consider the arm of the same warehouse mobile manipulator driving its gripper toward the strike plate of the cage door at 0.10 m/s, and suppose its low-level controller keeps executing that command while the high-level process stalls for 75 ms. The plate acts as a spring of stiffness 4.0×10^5 N/m, the latch tolerates at most 100 N of contact force, and the arm is assumed to have enough torque to keep moving through contact. The plate then deflects by $\Delta x = v \cdot \Delta t = 7.5$ mm, and the spring model gives $F = k \cdot \Delta x = 3,000$ N, 30 times the limit. Motor saturation, compliance elsewhere, or a controller that stops on stale commands would change that result. Against a plate this stiff, the time available to detect contact and bring commanded motion within the limit is a matter of milliseconds, far shorter than the stall, and chapter 6 derives that budget for this plate.

1.3.3 Class 3: Process and energy systems

Beyond moving linkages and contacts, Class 3 machines regulate continuous fluid, thermal, or electrical power flows (figure 1.4 c, figure 1.5 c). A cooling loop must remove heat as fast as the process generates it. With net heating power P and thermal capacitance C_{th} , a stalled pump produces a temperature rise $\Delta T = \int P/C_{th} dt$ until flow resumes; a transport delay adds time before new coolant reaches the hot component. For an illustrative constant P of 1 kW and C_{th} of 10 kJ/K, the component warms at 0.1 K/s, so a 5 K margin provides about 50 s before the assumed limit. A grid-tied inverter faces a different budget: on a 60 Hz network, an uncompensated 1 ms phase-estimation delay corresponds to 21.6° of phase error. Whether that error causes unacceptable current depends on the inverter's impedance and protection design; the 8.33 ms half-cycle is not itself a universal failure deadline. In each plant, the permitted software delay follows from the physical margin and the available protection, rather than the clock period alone.

1.3.4 The humanoid couples all three

A humanoid robot does not belong to a single machine class. All three physical regimes operate and contend simultaneously on its single, tightly constrained mobile chassis:

1. **Class 1 Dynamics in Locomotion:** Underactuated bipedal balancing, dynamic footstep placement, momentum conservation ($m\mathbf{v}$), and ground contact slip during walking and stair climbing require continuous spatial clearance and kinetic energy management.
2. **Class 2 Dynamics in Manipulation & Contact:** Bimanual end-effectors, multi-fingered grasping, and rigid contact transitions with environmental workpieces subject kinematic chains

⁷ | **Rigid Contact Mechanics:**

Unilateral physical contact is fundamentally one-way: an end-effector can push against an obstacle, but it cannot pull without gripping. Modeling rigid contact mathematically leads to Linear Complementarity Problems (LCPs), where abrupt collisions create impulse chatter and non-smooth forces. Real-time physical systems avoid chatter through compliant impedance models ($F = k\Delta x$); the complete mathematical formulation of contact complementarity is deferred to section B.5.

to high shock loads ($F = k\Delta x$), joint torque saturation, and transmission yield stress during physical interaction.

3. **Class 3 Dynamics in Actuation & Power Delivery:** Driving 20 to 50 high-torque actuators simultaneously from a single onboard DC battery pack pushes inverter thermal dissipation ($I_{\text{phase}}^2 R_{\text{phase}}$) to its limits.

Systems Perspective 1.1: The humanoid crucible

A humanoid couples stopping clearance, contact force, and electrical power on one chassis, so a demand in one regime becomes a demand in the other two within a single step. When a hand catches an unexpected load:

- **Contact becomes momentum (Classes 1 and 2):** The contact force perturbs the center of mass, and the robot must place a recovery step before it tips, turning a contact problem into a braking problem.
- **Momentum becomes heat (Classes 1 and 3):** The recovery step draws peak current through the hip and knee drives, and winding temperature forces the drives to derate torque when balance needs it most.
- **Current becomes brownout (Class 3):** The same current transient sags the shared battery bus and can reset the computer running the learned models in mid-step unless its power domain is decoupled from the drives.

Each part of this book ends by testing its law on a humanoid.

While a single-class machine (such as a wheeled AMR or a fixed industrial arm) isolates its physical budget to a single dominant conservation law, a humanoid robot forces the systems architect to manage coupled contention across classes, in which no one budget can be sized without the other two (section 2.6 quantifies the bus droop).

For any physical AI system, designing the independent check begins by determining which physical budget is exhausted first during a failure. In software systems, resource constraints center on processor cycles, memory allocation, and network bandwidth. In physical systems, the systems architect must identify whether spatial clearance, structural stress, or heat dissipation creates the critical timing deadline. In a high-speed vehicle or quadruped, spatial clearance fails within fractions of a second while motor winding temperatures remain safely below thermal limits. In an assembly press, mechanical stress exceeds the yield point of structural members within tens of milliseconds long before the chassis moves perceptibly or heats up. In a chemical reactor or battery pack, mechanical stress is negligible, but accumulated thermal energy breaches safe operational envelopes over several seconds. The shortest time to failure across these three domains defines the deterministic execution deadline that the independent check must meet.

Belonging to one of these three physical classes does not by itself mean that a machine requires the full systems discipline of physical AI. A classical hydraulic press or a deterministic PID drone autopilot manipulates mass and contact force, but operates entirely within classical control theory.

1.4 The Physical AI Scope Test

Without a scope boundary, teams err in two directions, putting neural models where closed-form PID control already suffices, or granting stochastic models unverified motor authority in safety-critical loops.

The scope of physical AI is defined by the simultaneous intersection of three structural criteria (figure 1.6):

1. **Learned Statistical Model (Unspecifiable Policy):** The system relies on a policy, perception encoder, or world model acquired from data (e.g., neural networks, foundation models, diffusion policies (Chi et al. 2024)) whose input-output transfer function cannot be written in closed-form analytical equations and whose worst-case outputs cannot be mathematically bounded offline.

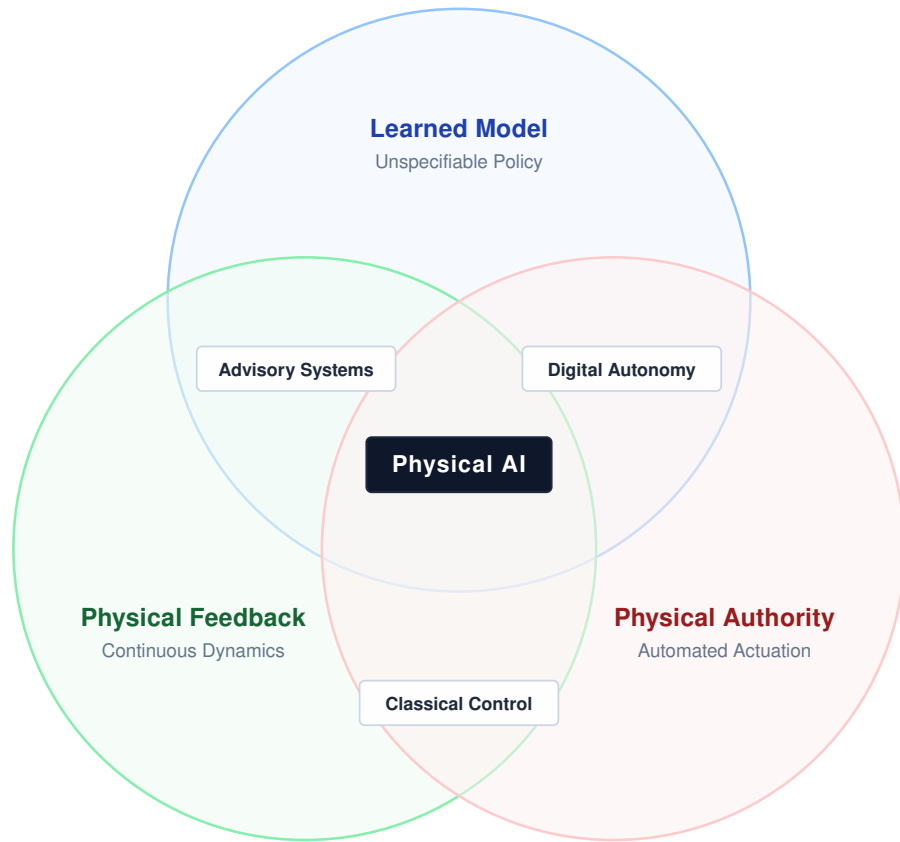


Figure 1.6: The physical AI scope test: Physical AI combines a learned component, consequential physical feedback, and system-level authority to execute accepted proposals without human approval. An independent veto belongs inside this intersection because accepted proposals can still move the plant. The pairwise overlaps locate advisory systems, digital autonomy, and classical control.

2. **Consequential Physical Feedback:** Actuator actions alter the material state of the physical environment, which in turn directly dictates what the sensors observe next under the governing laws of classical mechanics, thermodynamics, or electromagnetism.
3. **Delegated Physical Authority (System-Level Actuation):** A learned component supplies live proposals that an automated system can execute without synchronous human approval. The learned model need not write an actuator register itself; the system holds physical authority because an accepted proposal can reach one.

A closed feedback loop is necessary for physical interaction, but feedback alone does not qualify a machine for this domain. Classical proportional-integral-derivative (PID) controllers run at 1 kHz across aerospace and industrial automation, regulating valve positions and motor torques through tight sensory feedback. Their feedback loops are continuous and physical, but their control laws are specified entirely by fixed mathematical formulas whose stability bounds can be proven analytically from transfer functions. They hold physical authority and experience physical feedback, but they lack a learned model. Conversely, high-frequency algorithmic trading engines execute automated market orders under learned predictive models within microseconds. They combine learned policies with delegated transactional authority, but the feedback they receive is purely digital and informational. When an order executes, it alters an electronic order book rather than momentum, friction, or thermal energy. An unhandled stall in algorithmic trading costs financial capital, but it does not cause an inertia-driven collision with a structural fixture.

A learned model alone cannot define the field. An offline neural trajectory optimizer may compute paths for a manipulator, but if an engineer must inspect and explicitly load each path before execution, its outputs have no delegated runtime authority. Similarly, a neural network that only logs assembly-line defects or flags a display remains advisory because its output cannot automatically command the line.

The remaining pairing, a learned model with physical feedback but no authority, completes the boundary. A real-time driver drowsiness monitor uses a learned computer vision model to track eyelid movement and vehicle lane position, receiving continuous physical feedback from the vehicle state. Yet it lacks delegated physical authority because its only output is an auditory warning chime. If the vision model fails completely during an inference timeout or pipeline stall, the speaker simply emits no sound, but the physical steering rack and brake calipers remain safely under human control.

The most demanding edge cases arise in shared autonomy, where physical authority is dynamically divided between a human operator and a learned model. In robotic surgery or advanced driver assistance, a human operator provides continuous steering or tool guidance while a learned policy injects corrections to filter hand tremors, avoid anatomical boundaries, or compensate for vehicle slip. These architectures satisfy both the learned component and physical feedback criteria. Whether they fall within the scope of physical AI depends strictly on the temporal authority granted to the model. If the learned policy can command even 2.0 N of force or adjust steering angle without requiring human confirmation within the human reflex interval of 150–250 ms, the model possesses delegated physical authority. During that window, an erroneous model output or a delayed inference packet exerts real physical forces on the world, potentially causing a swerve or tissue damage, before the human operator's neuromuscular system can physically react to intercede.

Evaluating an ambiguous machine means testing each criterion independently, as table 1.4 does for representative systems. A deterministic veto does not remove delegated authority; a mandatory human review of an offline plan does. When all three tests pass, the system operates as physical AI, and its engineering must respect the causal boundary established in section 1.2.

Table 1.4: The physical AI scope test decision matrix: Representative engineering systems tested against the three joint criteria (Learned Model, Consequential Physical Feedback, and Delegated Physical Authority), establishing the governing engineering discipline, limiting physical factors, and primary failure modes for each candidate.

System Candidate / Case	Criterion 1: Learned Model (Unspecifiable Policy)	Criterion 2: Consequential Physical Feedback	Criterion 3: Delegated Physical Authority (Automatic Gated Execution)	Scope Classification & Discipline	Limiting Physics & Critical Failure Mode
Autonomous Warehouse AMR (Class 1)	Yes (Learned navigation policy / costmap)	Yes (Kinetic momentum, chassis inertia, and wheel traction)	Yes (Accepted proposals reach motor control without human approval)	Physical AI	Kinetic impact; traction slip and odometry loss
Articulated Contact Manipulator (Class 2)	Yes (Transformer action chunking for assembly (Zhao et al. 2023))	Yes (Mechanical contact impedance and reaction forces)	Yes (Safety-gated proposals drive live joint commands)	Physical AI	Structural yield stress exceedance; gearbox tooth shear
High-Speed Drone Classical Autopilot	No (Analytically tuned 1 kHz PID / cascaded loop)	Yes (Aerodynamic lift, drag, and gyroscopic dynamics)	Yes (Direct PWM motor speed control)	Classical Control Theory	Control margin erosion; rotor thrust saturation
High-Frequency Market Maker	Yes (Deep transformer sequence predictor)	No (Digital order-book state; memory and network queues)	No (No physical actuation; orders execute automatically)	Digital Machine Learning / FinTech	Financial capital loss; market disruption
Advisory Driver Drowsiness Monitor	Yes (Facial landmark vision network)	Yes (Vehicle kinematics and driver physiology)	No (Auditory warning buzzer and dashboard LED only)	Advisory Decision Support	Human perception-reaction latency; zero physical authority

System Candidate / Case	Criterion 1: Learned Model (Unspecifiable Policy)	Criterion 2: Consequential Physical Feedback	Criterion 3: Delegated Physical Authority (Automatic Gated Execution)	Scope Classification & Discipline	Limiting Physics & Critical Failure Mode
Offline Trajectory Optimizer	Yes (Deep neural network for time-optimal path search)	No (Static CAD workcell model; offline batch execution)	No (Motion plan verified and loaded via human engineer)	Offline Optimization Tool	Numerical non-convergence; zero direct runtime plant coupling
Shared-Autonomy Surgical Assist	Yes (Learned haptic tremor filter / virtual fixtures)	Yes (Anatomical tissue compliance and reaction forces)	Conditional (Model injects forces within human reflex window)	Borderline / shared physical AI	Human neuromuscular reflex latency vs. tool force rise rate



Checkpoint 1.1: Binding budgets and the scope test

Before examining the five-level machine, verify that you can place an unfamiliar machine by its binding budget and by the scope test:

- ☐ **Blind travel and braking:** Can you explain why the pre-brake delay τ_{delay} becomes blind travel $v \tau_{\text{delay}}$ that no braking can recover, and why doubling speed quadruples the braking distance but only doubles the blind travel?
- ☐ **First budget to exhaust:** Can you name which budget runs out first, and on roughly what timescale, for a mobile base, for an arm pressing against a stiff latch plate, and for a stalled cooling loop, and explain why a humanoid couples all three?
- ☐ **Three-criteria scope test:** Can you test an automated system for a learned model, consequential physical feedback, and delegated physical authority, and explain why a trading engine falls outside physical AI while a shared-autonomy surgical tool falls inside whenever it can apply force before the surgeon can react?

1.5 The Machine in Five Levels

A machine that passes the scope test needs an organization that keeps its learned models from writing directly to the actuators they could damage, while still letting those models steer it.

The machine in this book has five levels, ordered from the learned models down to the physics they act on and divided by a single line of authority. Control rates rise from the Brain to the Body; below the Body, the Boundary is timed by events and the World runs continuously. The Brain holds the learned proposers. An intent model revises goals at 1 to 5 Hz, and a chunk policy emits short sequences of future setpoints at 10 to 50 Hz; perception and memory run on the same side at sensor rates.

The Brain may only propose. Beneath it lies the proposal boundary, and beneath that the Nervous System, which holds the permission path. The permission path checks every proposal against fresh state and the margin that delay has left, admits, modifies, or refuses it, and owns a fallback that does not wait for the proposer. A 1 kHz loop on a dedicated microcontroller is one implementation; the rate a machine actually needs follows from its stopping budget. The Body is the machine's own physics under its drives, from the current loop that turns an admitted setpoint into torque at 10 to 25 kHz to the transmissions, windings, and power rails whose limits chapter 2 measures. The Boundary is the causal boundary itself, the transducers that face both ways. At one face a register write becomes current in a winding; at the other, light or force becomes a measurement with a capture time. The World is everything the machine does not own, including floors, payloads, contact, and people, and it evolves continuously whether any level is watching or not.

Governance is not a sixth level. It is the lifecycle envelope around all five, deciding who may command the machine, what evidence licenses it to operate, and when that license expires, and it changes on the timescale of interventions and releases rather than control cycles.

Downstream: Transfers of command authority, and preemption of the proposal stream, are governed by the supervisor state machine in section 14.3.



Definition 1.3: Proposal boundary

Proposal boundary is the line between a learned model, which may only propose an action, and the **permission path**, an independent mechanism with bounded latency that the model cannot delay or corrupt and that alone may permit the action to reach an actuator. The permission path checks each proposal against fresh evidence, the stopping envelope, and a fallback feasible from the current state, and refuses when any check fails.

1. **Significance:** Keeps model outputs from directly commanding actuators; the protection holds only while the permission path's sensing, physical limits, and fallback remain valid.
2. **Distinction:** A policy that outputs torque values still has no actuator-write authority; its output is a proposal until the permission path admits it.
3. **Common pitfall:** Treating the permission path as a post-processing clamp on model outputs rather than an independent mechanism that can refuse execution when the proposer is late, wrong, or silent.

The margin stack that opens every chapter draws the same five levels (figure 1.7) and highlights the one that chapter works on; table 1.5 pins their rates and lists the chapters for each.

Table 1.5: The five levels and their pinned rates: These are the only rates the book attaches to a level. A machine that needs a different rate derives it from its own measured limits rather than relabeling a level.

Level	What it is	Pinned rate	Where the book works on it
Brain	Learned proposers and the evidence they consume	Intent model 1–5 Hz; chunk policy 10–50 Hz; perception at sensor rate	chapter 3, chapter 6, chapter 8 to chapter 11
<i>Proposal boundary</i>	<i>Above may only propose; below may command</i>	—	—
Nervous System	The permission path: timing, lease, admission, fallback, and its silicon	1 kHz as one implementation; the required rate follows from the stopping budget	chapter 4, chapter 12, chapter 13
Body	The plant under its drives	Current loop 10–25 kHz	chapter 2
Boundary	Actuator register write and sensor capture	Event-timed	This chapter; the capture epoch in chapter 8
World	Contact, friction, payload, people, and the operating domain	Continuous	chapter 5, chapter 7
Governance envelope	Who may command, what licenses operation, and when that license expires	Lifecycle; no control rate	chapter 14 to chapter 17

This organization operationalizes Rodney Brooks's principles of situatedness and subsumption (Brooks 1986a; Brooks 1991), in which fast reactive layers protect the plant directly from sensory feedback while high-level deliberative goals modulate behavior across verified contract boundaries. For diagnosing a failure, a sense–plan–act decomposition locates the earliest broken contract; Appendix D develops it as a playbook.

The five levels span five orders of magnitude in operational frequency and latency (table 1.6). At the top, deliberative foundation models plan at human reaction scales (1–5 Hz); at the bottom, power electronic inverters switch stator currents at tens to hundreds of kilohertz (50–200 kHz). Bridging these extremes is the physical reality that latency is distance: every millisecond of uninspected delay in a machine moving at 1 m/s produces 1 mm of unrecoverable blind travel.



Figure 1.7: The five-level machine: Each band carries its pinned rate and the chapters that work on it. The wider gap between Brain and Nervous System marks the proposal boundary, above which a component may only propose. A chapter-opening stack with no level highlighted marks a governance chapter. The permission-loop rate shown is one implementation.

Table 1.6: Latency and Rate Numbers for Physical AI Systems: Order-of-magnitude frequencies, cycle periods, and kinetic translation rates across the five machine levels that dictate physical feasibility. Spanning five frequency decades, from sub-microsecond MOSFET switching up to second-scale foundation model deliberation, these physical constraints dictate how computational delay converts into irreversible mechanical motion. For a comprehensive quick-reference including thermal time constants, edge rooflines, bus jitter, and statistical safety bounds, see section C.2.

Machine Level	Frequency Band	Cycle Period	Blind Travel at 1 m/s	Architectural Role & Dominant Constraint
Brain (Deliberation)	1–5 Hz	200–1,000 ms	200–1,000 mm	Multimodal VLA reasoning; compute- and memory-bound
Brain (Action Chunking)	10–50 Hz	20–100 ms	20–100 mm	Reactive trajectory generation; bandwidth amortized
Nervous System (Permission)	1 kHz	1 ms	1 mm	Barrier certificates & stopping envelopes; deadline hard
Body (Current Loop)	10–25 kHz	40–100 μ s	40–100 μ m	FOC stator current & torque control; inductive L/R
Body (Inverter Switching)	50–200 kHz	5–20 μ s	5–20 μ m	MOSFET gate drive & dead time; LdI/dt droop

Downstream: The Brain's committed trajectory is built in section 11.1.

1.6 When the Boundary Fails

A physical crash can involve both an inaccurate prediction and a failure to respond when available evidence already warrants intervention. The Tempe investigation shows how object tracking, braking policy, operator reliance, and lost redundancy interacted.

1.6.1 The 2018 Tempe autonomous collision

On a March night in 2018, a modified Volvo XC90 operating Uber Advanced Technologies Group's developmental automated driving system (ADS) struck and fatally injured a pedestrian pushing a bicycle across a road in Tempe, Arizona (National Transportation Safety Board 2019).⁸

The vehicle carried lidar, radar, and cameras (figure 1.9). The ADS first detected the pedestrian 5.6 s before impact, but detection alone did not provide a correct path prediction (National Transportation Safety Board 2019).

1.6.2 Classification changes and lost tracking history

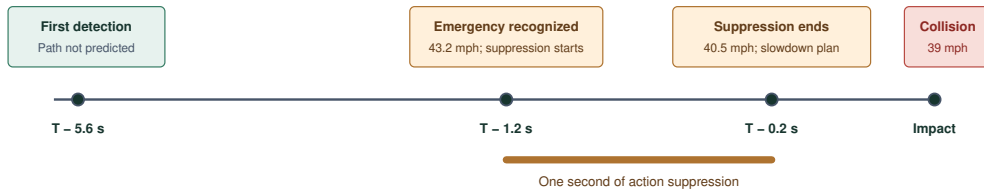
Despite continuing to track the detected object, the ADS did not brake in response to it. The NTSB record shows classification changes among vehicle, bicycle, and other before $T - 1.2$ s (National Transportation Safety Board 2019).

When the classification changed, the ADS path predictor did not use the object's earlier locations;

⁸ | **2018 Tempe Autonomous Collision:** The event times and design details in this section come from NTSB highway accident report NTSB/HAR-19/03 (National Transportation Safety Board 2019).

Recorded ADS sequence

Events and speeds from NTSB/HAR-19/03, table on page 16; event spacing is schematic

**Conditional braking comparison at $T - 0.2$ s**

Assume 18.1 m/s and constant 8 m/s² deceleration; distances are illustrative, not recorded.



No counterfactual stop time is inferred from this comparison.

Figure 1.8: Tempe ADS event timeline: NTSB recorded first detection at $T - 5.6$ s, hazard recognition and the start of one-second action suppression at $T - 1.2$ s, and a gradual-slowdown plan at $T - 0.2$ s. The lower comparison assumes a constant 8 m/s² deceleration from the recorded 40.5 mph speed. Idealized stopping distance at $T - 0.2$ s is about 20.5 m, far beyond the roughly 3.6 m of constant-speed travel remaining.

- **Assumptions:** Approximate speed near hazard recognition as $v_0 = 19.2$ m/s, speed at $T - 0.2$ s as 18.1 m/s (reported as 40.5 mph), and idealized constant braking deceleration as $a_{\text{brake}} = 8$ m/s². Travel is computed at constant speed.
- **Suppression interval:** Over the reported 1 s, holding v_0 constant would give $d_{\text{blind}} = v_0 t_{\text{suppress}} = 19.2$ m. The vehicle was in fact slowing for a turn, so this is not measured suppression travel.
- **Braking comparison:** From the rounded speed at $T - 0.2$ s, idealized stopping requires $d_{\text{brake}} = v^2 / (2a_{\text{brake}}) \approx 20.5$ m. Holding that speed for the remaining 0.2 s gives only about 3.6 m of travel to impact. Even immediate full braking at that late instant could not stop before the impact point. The ADS in fact planned gradual slowdown, not full braking (National Transportation Safety Board 2019).

Figure 1.8 draws the recorded event times and that braking comparison on one time axis, which places the one-second suppression interval directly against the stopping distance the vehicle needed at the moment suppression ended.

⚠ War Story 1.1: The disabled safety redundancy (2018)

Context: On March 18, 2018, a developmental ADS on a 2017 Volvo XC90 struck and killed a pedestrian crossing a road in Tempe, Arizona (National Transportation Safety Board 2019).

Mechanism: As traced above, classification changes discarded the object's tracking history, and the one-second suppression withheld braking until only a gradual slowdown was planned. The design did not use maximum braking solely to mitigate an unavoidable collision (National Transportation Safety Board 2019).

Impact: The vehicle struck and fatally injured the pedestrian. The late gradual-slowdown plan could not recover the lost stopping opportunity.

Response: The NTSB found that disabling the Volvo warning and braking systems, which Uber ATG had done over possible radar interference, removed a safety redundancy layer

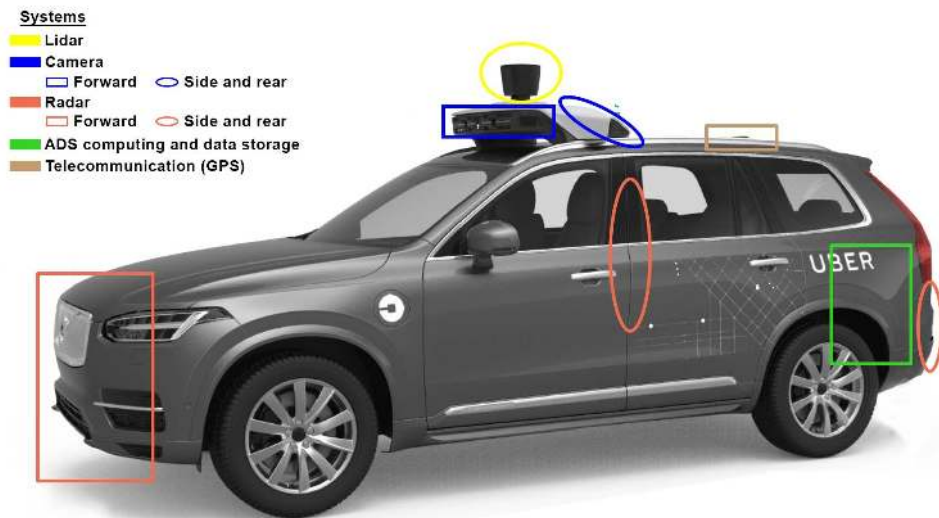


Figure 1.9: Tempe test vehicle sensor locations: The NTSB diagram identifies the developmental ADS vehicle's lidar, camera, radar, computing, and telecommunications locations. (Source: NTSB, public domain).

and recommended stronger safety risk management for testing automated vehicles (National Transportation Safety Board 2019).

Systems lesson: Classification uncertainty, a delayed response rule, and reliance on an inattentive operator can combine. A design needs timely hazard assessment and a response that can mitigate impact when avoidance is no longer possible; the investigation does not prescribe a particular processor or mailbox architecture.

The investigation supports a broader architectural requirement. A developmental ADS needs safety redundancy and a timely mitigation path even when perception or path prediction fails, and simulation tests by Volvo that the NTSB cites suggest that the factory emergency braking might have prevented or mitigated this collision. The report does not establish when a separate permission path would have triggered or where it would have stopped the vehicle.¹⁰

Tempe illustrates failure *before* impact, where classification changes and a software suppression delay prevented timely braking. The 2023 Cruise robotaxi incident (Quinn Emanuel Urquhart and Sullivan, LLP 2024) marks the opposite pole *after* impact, a post-collision fallback that moved the vehicle while a pedestrian was pinned beneath it (section 12.9).

Neither failure reduces to an inaccurate model. The first spent stopping distance before braking began, and no later action could recover it. The second chose motion after the impact. Together they show what must hold before the machine acts and what a fallback may do once harm has begun, and the four laws that follow state those demands for every machine.



Checkpoint 1.2: Forensics of boundary and authority failures

Before studying the four bedrock laws, verify your understanding of the failure mechanisms diagnosed in the Tempe incident:

- ☐ **Classification changes and path prediction:** Can you explain how classification changes caused the ADS path predictor to omit earlier object locations, despite continued tracking?

¹⁰ | **Safety-Critical Standards:** Functional safety and intended-functionality standards can inform the safety case for sensing, actuation, and fallback. Their applicability and required implementation depend on the system and jurisdiction; the NTSB Tempe report did not mandate an isolated microcontroller architecture.

- ❑ **Software suppression vs. kinematic clearance:** Can you distinguish the $T - 1.2$ s recognition time from the one-second suppression period and compare the idealized stopping distance with travel remaining at $T - 0.2$ s?
- ❑ **Safety redundancy:** Can you explain why disabling factory collision warning and emergency braking removed a layer of protection from a developmental ADS?

1.7 The Four Bedrock Laws

Tempe's lost second is the first law at work. That law and three others hold for every machine whose learned proposals can reach an actuator, and each leads a part of this book. Together they turn the causal boundary into requirements.

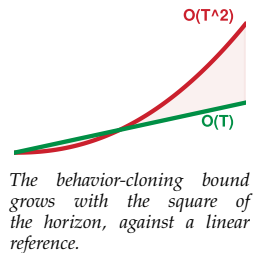
The first law: irreversibility. Physical work cannot be recalled, so every check must finish before actuation, inside the margin that delay has not yet consumed. An actuator converts electrical energy into motion, strain, or heat, and no software action removes it afterward. Braking can oppose momentum only by applying force over time and distance, and a deformed fixture stays deformed when its setpoint is reversed. Because the machine keeps moving while it computes, delay is paid in distance: every millisecond between capture and response is travel the stop can no longer use. Part I turns this law into budgets.

The second law: endogenous data. A physical agent's actions choose its next observations, so competence holds only for the states a closed-loop test has covered. In an offline benchmark, a prediction does not change the next example. On a machine, an erroneous command moves the joints and the cameras and delivers the policy into states its demonstrations never showed. Under bounded per-step cost, the worst-case excess cost of a behavior-cloned policy grows as $T^2\epsilon$ over a horizon of T steps with per-step disagreement ϵ (Ross et al. 2011); section 5.1 develops the bound. The bound predicts no particular failure rate, but it shows why single-step accuracy cannot certify a closed loop. Part II asks what a learned proposer's competence rests on.

The third law: proposal is not permission. A learned model may propose; only an independent path that the model cannot delay or corrupt may permit. The permission path decides every action on state of known age, within a deadline set by the plant, and owns a fallback it can execute without the proposer. Separate silicon is one way to build it. Hardware isolation within one chip is another, provided the response bound and the fault containment are demonstrated. Freshness is part of the test, because a proposal computed from an old observation describes a machine that has already moved. Part III builds the runtime around this law.

The fourth law: evidence bounds authority. Authority to act is granted on recorded evidence; it expires with that evidence, and a premise the evidence cannot settle must restrict operation. Average performance over finite trials cannot certify a tail whose failures are mechanical, and a demonstration is not evidence of safety. A release case instead bounds sensing, model error, actuator response, stopping clearance, and hardware faults for a declared operating domain, and it names what it leaves open. Part IV decides who may command the machine and which conditions it may enter.

The third law is the one the other three serve. Irreversibility makes permission necessary, endogenous data leaves the proposer's competence partial, and evidence bounds how much authority the permission path may grant. Scaling the proposer does not relax the third law. Richard Sutton's "bitter lesson" (Sutton 2019) holds that general methods that exploit growing computation eventually overtake engineered heuristics. On a machine, that lesson meets what this book terms the **physical wall**. Onboard compute is bounded by battery energy, heat dissipation, memory bandwidth, and the integrity of the supply rail, so added model size is paid for in delay. Once that delay outruns the stopping budget, the machine reaches the obstacle before the forward pass finishes. Section 3.4 measures what that wall leaves the machine's proposers. In the terms of hardware protection rings, the learned model proposes from an unprivileged domain and the permission path refuses from a trusted one, with the difference that a refusal must also leave the machine a physical fallback, because no page fault stops a moving mass.



1.8 Safety Convergence

Safety examines whether the machine caused harm. Security considers whether an external agent forced it. Accountability assesses whether an auditor can determine what occurred and why. In software services, these concerns reside in separate engineering organizations and distinct runtimes, divided by application logic, network firewalls, and audit logging databases. In a machine that moves mass and commands current, all three converge on the causal boundary, where the question becomes whether the permission path has the state and authority to refuse a register write before the command becomes torque acting on the plant.

Safety failures arise without an adversary. A camera lens accumulates dust over 500 h of operation, shifting the input distribution away from the training domain until an attention head generates an out-of-distribution steering torque. A joint bearing develops mechanical play, introducing unmodeled backlash and joint compliance that converts a stable learned position trajectory into a destructive limit-cycle oscillation. A sudden change in surface friction can also drop tire traction below the friction coefficient assumed by the learned policy. In each case, the proposer generates an action that is statistically plausible to the network but physically hazardous to the body. If the permission path lacks an independent kinematic model or thermal budget to detect exceedance, the unrefused command writes to the motor drive registers, releasing stored kinetic energy into the workspace.

Security failures introduce an adversary who deliberately exploits the gap between statistical inference and physical reality. An attacker does not need to compromise cryptographic keys or gain root access on the host operating system if they can manipulate sensory inputs. Projected structured light can spoof a depth sensor, acoustic injection can resonate the micro-machined capacitive proof masses inside MEMS gyroscopes at their natural frequencies, tricking the sensor into reporting phantom angular velocity,¹¹ or patterned stickers (adversarial visual perturbations) on a factory floor can induce a vision-language-action model (a learned policy that maps camera images and a language instruction to actuator commands) to output maximum actuator velocity. On a traditional server, an injection attack compromises data confidentiality or integrity within an abstract memory space. In physical AI, an injection attack commands physical work, turning an unvalidated inference into mechanical force. The security perimeter cannot terminate at the network interface or the operating system syscall boundary. It terminates at the permission path, which must evaluate every candidate command against physical invariants regardless of whether the proposer acted in good faith or under adversarial control.

Traditional software authorization models fail in physical machines because capability tokens and access control lists assume discrete, memoryless transactions. In a web service, possessing a valid credential grants permission to write a record to a database, and the cost of the operation is bounded by compute cycles and storage bytes. In physical space, permission is not a static binary entitlement. In an analytical robotic joint, a command to apply $50 \text{ N} \cdot \text{m}$ of torque is valid when the link is stationary at mid-stroke, but the identical command causes structural yield (shaft fracture or gear tooth shear) if the joint is 2 mm from its hard stop moving at 1.5 rad/s . Physical authority is state-dependent, continuous, and dynamic, governed by velocity, momentum, thermal accumulation, and spatial occupancy. The permission path cannot evaluate an action in isolation as a stateless remote procedure call. It must integrate the action over time against the current physical state of the body, checking whether the proposed motion stays within a validated recoverable set¹² for the plant, load, state-estimation error, and response deadline.

Accountability requires proving the exact causal chain that led to actuation when a machine damages hardware or breaches an envelope. In standard computing systems, logging is asynchronous, best-effort, and coarse-grained, dropping records during buffer pressure or recording timestamps with millisecond jitter. At the causal boundary, accountability demands deterministic, high-rate telemetry captured synchronously with the control loop. If a joint actuator commands 1 kHz updates with a 1 ms deadline, the permission path must record the raw candidate vector proposed by the model, the state estimate it judged against, the criterion it applied, and the exact vector written to the hardware registers. Sealing these records in tamper-evident, append-only circular buffers in battery-backed SRAM or FRAM (memory that survives a sudden power loss when a safety breaker trips) provides post-incident reconstruction. When an anomaly occurs, an engineer can verify mathematically whether the failure originated in a corrupted sensor reading, a degraded

¹¹ | **MEMS Acoustic Resonance:** Resonant acoustic frequencies (18–32 kHz) can physically vibrate the micro-machined capacitive proof masses inside MEMS gyroscopes, inducing signal saturation and false rate readings. Hardening against acoustic spoofing requires physical damping barriers and real-time kinematic cross-validation on the microcontroller.

¹² | **Recoverable Sets:** A modeled set of physical states (positions, velocities, and temperatures) from which an admissible controller can keep the plant within specified bounds under stated disturbances and actuator limits (Mitchell et al. 2005). Admission against this set is meaningful only while those state, plant, and timing premises hold.

policy proposal, an erroneous permission rule, or a mechanical component that failed to produce the commanded counter-torque.

Safety, security, and accountability therefore ask the same three things of the permission path that guards the causal boundary: state fresh enough to judge the command, authority to refuse it, and a record of what was proposed, what was admitted, and what was written. Building that path, and the evidence that it does what its record claims, is the work of the rest of the book.

1.9 The Architecture of This Book

The four parts of the book pursue the question this chapter opened with. Section 17.7 answers it in conditional form, each guarantee holding only while its premises do. Each part is led by one of the four laws, and every chapter of Parts I to IV except chapter 3 closes by producing a record that a later chapter consumes (figure 1.10). Read in order, the records carry the argument forward: what the plant can do, what the learned proposer has shown it can do, what it proposes now, what the permission path admits, and what the evidence licenses. The first two of these run as one strand and the third as another, and the two strands meet at the permission decision, which is the third law in the form of an interface.

Ch. 1 What must be guaranteed?

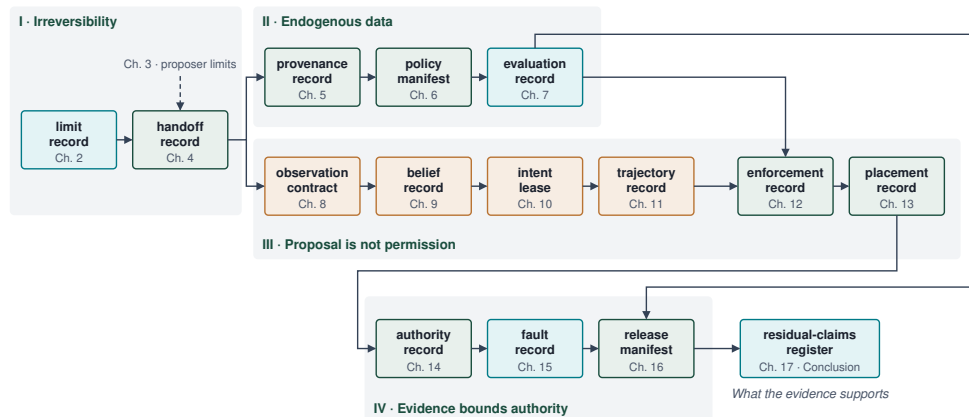


Figure 1.10: The spine of the book: Each part is led by one law, and each chapter of Parts I to IV except chapter 3 produces a record that a later one consumes. What the plant can do and what the proposer has shown it can do run as one strand, what the proposer proposes now as another, and the two meet at the permission decision.

Part I, The Machine Anatomy, is led by the first law. Chapter 2 measures the plant's budgets and writes each one as a limit record, the root of the chain. Chapter 3 states what the learned proposer can supply and how quickly its evidence ages. Chapter 4 derives the permission path's rates and lease from the stopping budget and records every handoff in a handoff record that extends the limit record. Part II, Teaching the Machine, is led by the second law. Chapter 5, chapter 6, and chapter 7 turn demonstrations into a provenance record, a policy manifest, and an evaluation record whose claims stay inside a declared operating domain and whose gaps are cataloged. Part III, Running the Machine, is led by the third law. Chapter 8 through chapter 13 follow one command from a timestamped observation to a belief, an intent lease, a committed trajectory, an admitted command, and the silicon that keeps the permission path on time. Part IV, Governing the Machine, is led by the fourth law. Chapter 14, chapter 15, and chapter 16 decide who may command the machine, what the fault tests established, and which conditions the machine may enter. Chapter 17 concludes the book with what the records support and a register of the premises that remain open. Every record is worked out on one machine, the warehouse mobile manipulator introduced in section 1.3.

1.10 Fallacies and Pitfalls

Each misconception below imports an assumption that holds for software, for offline learning, or for classical control into a machine whose learned proposals can reach an actuator. The first assumes

that training can do the permission path's work, and the other three assume that a human can intervene instantly, that latency only degrades performance, or that physical work can be undone.

Fallacy: *Safety constraints can be trained into the policy.*

Reinforcement and imitation learning pipelines often try to make a policy safe by adding penalty terms, barrier losses, or negative reward shaping to its objective. These terms steer gradient descent and lower the frequency of undesirable states across the training distribution, but they do not bound what the deployed policy emits. Under novel sensor noise, a lens occlusion, or an out-of-distribution combination of states, nothing in the offline objective prevents an unsafe command. A training incentive shapes a probability distribution, whereas a stopping envelope requires a bound that holds on every cycle. The constraint therefore belongs downstream of the model, as a gate between the policy output and the actuator registers, which is where the third law places the permission path.

Pitfall: *Treating human supervisory intervention as instantaneous in shared-autonomy systems.*

A surgical tool applies haptic guidance force feedback from a learned anatomical model while a surgeon holds the instrument. Human supervision does not remove delegated authority during the interval before the surgeon can respond. If an actuator builds force faster than the stated human reflex interval of 150–250 ms, its commands can deform tissue before the operator corrects them. The engineering question is how much force the system can apply during that interval. Local force limits enforced by the permission path must constrain the actuator without depending on either a timely neural prediction or an immediate human response.

Pitfall: *Assuming software latency merely degrades performance gracefully.*

Suppose the machine's chunk policy stalls for 300 ms while its memory subsystem throttles. The computation pauses, but physical momentum ensures the base keeps moving. A base that kept executing its last proposal at 1.5 m/s would travel blind for 450 mm before the next command arrived, and that displacement can consume the clearance it needs to stop if a person steps out. The machine does not allow that blind travel. A lease bounds how long the base may act on the last proposal, and when it expires the permission path brakes whether or not a new proposal has arrived, so the stopping envelope holds while the deliberative policy is unavailable. Section 4.3 sizes that lease from the stopping budget, and section 12.3 prices one held too long.

The physical reality of this constraint is governed by the kinematic roofline: stopping envelope $d_{\text{stop}} = v\tau + \frac{v^2}{2a_{\text{max}}} \leq R_{\text{sensor}}$ under emergency braking (figure ??). Across fifty years of autonomous systems, every order-of-magnitude reduction in sense-to-act loop latency τ unlocked a corresponding expansion in safe operational speed v , from the stop-and-plan deliberation of Shakey ($\tau \approx 30$ s, $v = 0.18$ km/h) and the Stanford Cart (Moravec 1988), through the early neural vision of ALVINN (Pomerleau 1989), to modern autonomous vehicles operating at highway velocities.

Fallacy: *Assuming physical actions can be rolled back like database transactions.*

A manipulator applies an erroneous 50 N · m torque to a delicate fixture. Its monitor detects the anomaly 50 ms later and reverses the motor command. Reversing a command cannot restore a fixture that has already plastically deformed or recover the energy already dissipated as physical heat in the motor windings (I^2R). Detection can limit further damage, but it cannot erase the intervening physical work. The control architecture must therefore check commands against force and motion limits before execution. Recovery remains necessary, but it complements preventive enforcement rather than replacing it.

1.11 Summary

The causal boundary marks the physical and electrical threshold where digital bits convert irreversibly into kinetic momentum, contact force, and Joule heat at memory-mapped actuator registers. Past this boundary, computational abstraction ends. Latency is paid in unguided physical displacement, so the chapter's requirements on timing and authority place the check before the register write rather than after it.



Key Takeaways: The causal boundary invariant

- **Delay is paid in distance:** A permitted command becomes motion and heat that no later software action recalls, and a moving machine travels $v\tau_{\text{delay}}$ before braking begins. On the warehouse mobile manipulator at its drive limit, that blind travel adds 35.6 percent to the braking distance.
- **The binding budget sets the deadline:** Mass-dominated machines exhaust stopping clearance first, contact-dominated machines exhaust force, and flow-dominated machines exhaust thermal or phase margin. The shortest time to failure fixes the deadline the independent check must meet, and a humanoid couples all three.
- **Scope requires all three criteria:** A learned component, consequential physical feedback, and delegated authority together make a machine physical AI. An independent veto does not remove delegated authority; mandatory human approval of each action does.
- **Proposal is not permission:** In the five-level machine, the Brain's learned proposers may only propose, and the permission path in the Nervous System admits, modifies, or refuses each proposal against current state and owns a fallback that does not wait for the proposer. Safety, security, and accountability meet at that path, which also records what it admitted.
- **Physical failures combine:** At Tempe, classification changes, a designed one-second braking suppression, and reliance on an inattentive operator interacted, and the factory emergency braking that might have mitigated the impact had been disabled. No single component's accuracy explains the outcome, which is why the requirements fall on timing, response, and redundancy.
- **Four laws organize the book:** Irreversibility, endogenous data, proposal is not permission, and evidence bounds authority each lead one part of the book. The third law is the one the other three serve, and the records the later chapters produce carry each law forward on the warehouse mobile manipulator.



What's Next: From the causal boundary to physical limits

With the five-level machine and the four laws in place, Part I begins with the first law. This chapter took the machine's speed, braking deceleration, and contact stiffness as given, yet every deadline the permission path must meet is derived from them. Chapter 2 asks which physical limits actually bound the machine, how each one is measured, and under what conditions it holds, and it writes each limit down as a limit record, the first record in the chain the rest of the book extends.

I

THE MACHINE ANATOMY

Part I

The Machine Anatomy

EVERY physical AI system is defined by the physical apparatus through which it acts. The first law of section 1.7 holds that this apparatus cannot take back the work it delivers, and Part I turns that law into numbers. It asks how far the machine travels before it stops, how much torque its drives can deliver and how much winding heat and rail droop they tolerate before damage, how quickly its evidence ages, and which of these limits binds first. The answers come from the plant, the learned proposer, and the permission path, in that order.



Principle 1: The irreversibility law

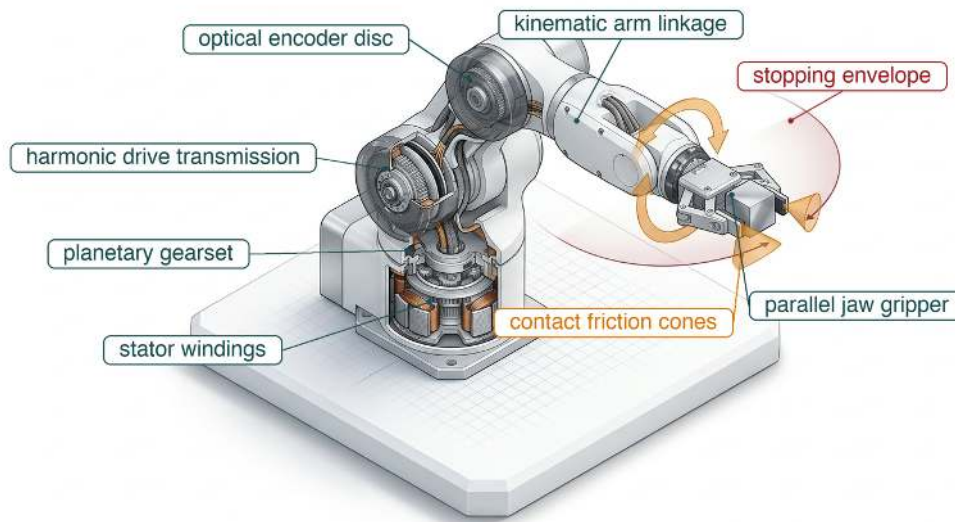
Invariant: Once a permitted command crosses the causal boundary, the energy it delivers becomes motion, strain, or heat that no software action can recall. During a pre-brake delay τ_{delay} the machine travels $v\tau_{\text{delay}}$ before any response begins, so a stop requires $d_{\text{stop}} \leq D_{\text{clear}}$, with every term of d_{stop} taken from a measured limit of this machine under stated conditions (section 2.3).

Implication: Each part of the machine carries budgets in time, distance, torque, heat, and voltage. Every check that protects the plant must finish before actuation and inside those budgets, and a design is admissible only when it can say which budget binds first, how that budget was established, and what the machine does when it is exhausted.

- **The Physical Body (chapter 2):** Decides which physical budget binds first and how its value was established, and produces the limit record from which every later record descends.
- **The Cognitive Brain (chapter 3):** Decides what a learned proposer can deliver on time and what memory and latency that costs; it produces no record of its own, and its proposer limits enter the handoff record in the limit record's format.
- **The Nervous System (chapter 4):** Decides how often the permission path must check and how long a proposal may drive the actuators, and produces the handoff record together with the header that every proposal carries.

Part I hands forward three things. From chapter 2 comes the limit record, the form in which every budget that can end in damage (stopping distance, delivered torque, winding heat, rail voltage, and measurement age) states its conditions and its category. The chapter fills that form on the machine for two limits, the credible deceleration behind the stopping distance and the dropout of the shared control rail. From chapter 3 comes the proposer's supply: how many future setpoints one chunk covers, how long a new chunk takes, and how quickly the evidence behind it ages. From chapter 4 comes the handoff record: the lease the permission path derives from the stopping budget, the tick at which it checks, the handoffs between proposer and permission path, and one worked budget that couples all three chapters on the running machine.

The Physical Body



- 2.1 The Five Physical Budgets
- 2.2 Measurement Freshness
- 2.3 Kinetic Momentum
- 2.4 Actuator Transmission Limits
- 2.5 Thermal Duty Cycles
- 2.6 Electrical Power Integrity
- 2.7 Measuring a Machine's Own Limits
- 2.8 Which Budget Binds First
- 2.9 Fallacies and Pitfalls
- 2.10 Summary

Purpose

What physical limits bound a learned policy the moment its outputs command motors and moving mass?

A moving machine cannot pause for software. During inference, the physical world advances: positions change, contact surfaces shift, and observations age. Arresting physical motion requires finite time to process a command and finite distance to dissipate kinetic energy. Electric motors and mechanical gearsets strictly constrain how quickly torque can ramp, while winding currents and inverter switching losses continuously heat electromagnetic coils. Policies that ignore these constraints request impossible forces, trigger supply-rail voltage collapses, or shear transmission gear teeth. Faster neural inference helps only within this physical chain; it cannot eliminate braking distance, mechanical inertia, or thermal limits.

Physical artificial intelligence demands that these limits be measured on the fully assembled machine under operational loads, rather than transcribed from optimistic component datasheets. At the causal boundary, the Body establishes the hard physical budgets—clearance distance, peak force, and thermal headroom—that perception, planning, and control must share. Under the first bedrock law, every such budget must be verified before an action command is issued. By characterizing the mechanical plant's torque-speed curves, friction cones, and thermal dissipation rates, systems engineers ground learned proposals in the uncompromising physics of the physical plant.

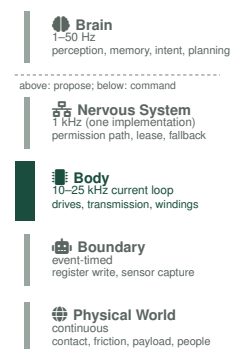


Figure 2.1.: The five-level machine stack with the Body level highlighted.

Prerequisite: The physical plant grounds the three machine classes defined in section 1.3.



Learning Objectives

- Calculate observation freshness limits from measurement age and the machine's allowable motion
- Calculate stopping distance and the admissible speed ceiling from pre-brake delay, credible deceleration, and clearance overheads
- Evaluate actuator and transmission choices against response limits and reflected inertia
- Estimate sustainable motor duty cycles from heat generation, cooling, and operating conditions
- Budget electrical supply margins for simultaneous actuation, computation, and regenerative braking
- Specify the elements of a limit record that states a measured limit's conditions, uncertainty, margin, runtime detector, and evidence category

2.1 The Five Physical Budgets

A robotic manipulator can shatter its transmission even after receiving an emergency stop command before contact. The stop command traverses software buffers, the power inverter switches states, and the motor generates peak reverse torque. Yet if the moving link possesses kinetic energy exceeding the mechanical work the actuators can perform over the remaining clearance, impact is unavoidable. In pure software, a faulted thread can be killed instantly by the operating system kernel. In physical reality, moving mass carries momentum that cannot be caught by an exception handler, and an electrical power stage cannot deliver infinite current without thermal destruction or voltage collapse. Software cannot override the physics of moving mass, heat, and current.

Above the proposal boundary, a learned policy may propose any motion, but the **physical body** delivers only what mechanics, thermodynamics, and electromagnetism allow. That body is the plant of actuators, transmissions, linkages, power buses, and transducers that acts on the world, and every setpoint the policy issues draws against five of its physical budgets:

1. **Measurement freshness:** The rate at which the physical environment changes relative to the age of the sensor data reaching the controller. Because the world does not pause for computation, elapsed time during communication and inference means the machine always acts on historical state.
2. **Kinetic momentum:** The stopping distance and braking force required to dissipate stored kinetic energy ($E_k = \frac{1}{2}mv^2$) before colliding with obstacles or exceeding workcell boundaries.
3. **Actuator bandwidth and reflected inertia:** The speed at which mechanical torque and velocity can change before winding inductance ($\tau_e = L/R$), inverter current clamps, or gear transmission inertia (N^2J) choke mechanical acceleration.
4. **Thermal capacity:** The rate at which resistive Joule heat accumulates in stator coils and power semiconductors relative to the chassis dissipation rate, bounding sustained duty cycles before thermal throttling intervenes.
5. **Electrical power integrity:** The transient voltage droop, peak current capacity, and regenerative energy surges across the shared direct-current distribution bus during rapid acceleration and braking.

The warehouse mobile manipulator that this book follows draws on all five. Loaded to at most 368 kg and moving at its 1.5 m/s drive limit (illustrative; see the Reader Guide), it carries 414 J of kinetic energy through aisles it shares with people.

Each budget becomes a limit established on the assembled machine under declared thermal and electrical stress. The first is the age of the measurement on which every command rests, which runs from physical transduction, before any software process receives the data.

2.2 Measurement Freshness

Software benchmarks often report latency as the execution time of a network on an accelerator, but that compute slice is only one segment of the causal chain of section 1.2. **A measurement decays from transduction; what matters is its age when acted upon.** That age spans the interval from the moment photons strike a photodiode or flux passes a Hall sensor to the moment motor coils generate torque that changes a link's acceleration. If a perception model computes an obstacle location in 15 ms but camera integration, sensor readout, transport, queue dispatch, and motor current rise time add another 45 ms, the physical age of the measurement when the joint moves is 60 ms. During that entire window, the physical world has continued to evolve.

Tracking measurement age means decomposing that path into eight stages (figure 2.2): exposure, timestamped at the midpoint of integration; sensor readout over an interface such as MIPI CSI-2; direct memory access (DMA) transport into host memory; kernel buffering into user-space queues; model computation; operating-system scheduling and inter-process dispatch across the processor boundary; command transmission over a fieldbus or shared-memory mailbox; and actuator response, as the drive builds flux in the stator windings. On a two-processor implementation the path runs from the image sensor through inference on the application processor, a lock-free mailbox, and a 1 kHz permission check on the safety microcontroller, to gate-driver switching and current rise. Every stage consumes a slice of the total time budget.

The rate of world-state change dictates how long a measurement remains valid before acting on it introduces intolerable error. Let $e_{\max}$ denote the maximum acceptable error in a tracked state, such as a joint angle or the position of a part on a conveyor, and let $v_{\max}$ denote the fastest credible velocity at which that state can evolve. The freshness deadline Δt_{fresh} is the upper bound on measurement age before a control action based on that measurement becomes invalid, defined by the inequality

$$\Delta t_{\text{fresh}} \leq \frac{e_{\max}}{v_{\max}}$$

For a vision-guided picker on a parcel sorter, tracking parcels at a maximum relative velocity $v_{\max} = 1.2$ m/s with a geometric clearance tolerance $e_{\max} = 24$ mm, the freshness deadline is $24 \text{ mm} / (1200 \text{ mm/s}) = 20$ ms. If an actuation command reaches the motor based on an image captured 35 ms earlier, the parcel has displaced by 42 mm, exceeding the position allowance by 18 mm and missing the grasp. The deadline governs a target that keeps moving while its measurement ages, such as the parcel here or the mug riding the mobile manipulator's pick-station conveyor. For such a target it limits only motion-induced error, and only while relative velocity is bounded and the initial observation is credible; sensing and estimation errors consume additional tolerance. A person standing in the aisle does not move while the observation ages. The base does, so the age appears as distance the base covers before braking, which section 2.3 charges against the stopping budget.

Engineers often compute total latency by summing the P_{99} values of individual stages, but the percentile of a sum is not the sum of the percentiles.¹ Shared memory contention and kernel queues can correlate stage delays, so the distribution of complete sense-to-torque age must be measured on the integrated machine under declared load, and even that measured percentile is not the analyzed bound a safety deadline needs (section 4.4).

To see how stage timing differs from end-to-end age, consider a timing profile for the camera-to-joint path under continuous inference load (table 2.1), judged against the same 20 ms deadline.

Table 2.1: Illustrative sense-to-actuation latency decomposition: Stage and system-level timings show why the arithmetic sum of stage percentiles is not the end-to-end tail. The example's $P_{99.9} = 68.4$ ms exceeds the 20 ms freshness deadline of the parcel sorter.

Pipeline Stage	Physical Mechanism / Contention Source	P_{50} (ms)	P_{99} (ms)	Max (ms)
1–2. Sensor Exposure & Readout	60 Hz rolling shutter exposure midpoint	11.2	16.6	16.7
3–4. Host Transport & DMA	PCIe DMA transfer into host memory	1.4	3.2	6.8
5. Vision Backbone & Policy	Onboard accelerator inference (memory throttling tail)	18.5	24.1	38.4

Downstream: section 9.6 derives how long a stored belief stays valid as its measurement ages.

¹ | **Quantile Convolution:** In general, the quantile of a latency sum differs from the sum of equal-percentile stage quantiles ($Q_p(X + Y) \neq Q_p(X) + Q_p(Y)$). Independence alone does not make their arithmetic sum a worst-case bound. Correlation from shared buses and queues changes the end-to-end tail and must be characterized at the system boundary.

Pipeline Stage	Physical Mechanism / Contention Source	P_{50} (ms)	P_{99} (ms)	Max (ms)
6. OS Scheduling & IPC	Kernel preemption and mailbox dispatch to safety microcontroller	0.6	2.1	12.3
7. Fieldbus Transmission	Deterministic EtherCAT cyclic packet exchange	1.0	1.0	2.0
8. Actuator Current Rise	Gate driver switching and stator flux buildup	2.3	3.0	4.5
Arithmetic Stage Sum	Sum of each column's stage values; not an end-to-end quantile	35.0	50.0	80.7
End-to-End Age	System-level profile ($t_{\text{transduction}} \rightarrow \tau$)	35.0	46.2	80.7

The 50.0 ms sum of stage P_{99} values differs from the 46.2 ms system percentile, and against the 20 ms deadline even the 35.0 ms median would be late.

Acting on expired data adds phase lag to the feedback loop and can excite chatter when corrective torques reinforce rather than damp tracking errors,² so a high inference rate does not by itself make the response timely. The system must carry a hardware timestamp from transduction (using IEEE 1588 gPTP³ or hardware strobe capture at the Ethernet PHY / MIPI CSI-2⁴ interface) through the pipeline. Before applying a policy command, the permission path compares the observation age $t_{\text{age}} = t_{\text{now}} - t_{\text{transduction}}$ with the age its budget allows. If the age exceeds that limit, it rejects the stale proposal and executes a fallback whose braking or holding behavior has been validated within the remaining physical margin (Brooks 1986a).

Throughout that sense-to-actuation interval, the body continues moving, carrying momentum that no software interrupt can cancel.

2.3 Kinetic Momentum

The loaded mobile manipulator drives down an aisle at its 1.5 m/s drive limit. A person steps out from the end of a rack 1.10 m ahead and stands in the aisle. Software cannot simply pause execution or instantaneously zero the velocity vector. The base, the arm, and the totes carry mechanical momentum, and bringing that mass to rest requires dissipating kinetic energy across physical distance. **Stopping distance grows linearly with delay and quadratically with speed, and that difference sets the speed limit.**

The distance from sensing a hazard to zero velocity comprises two phases: blind travel and braking travel. During the pre-brake delay, the machine runs the pipeline of section 2.2. Throughout this delay τ_{delay} , which runs from the exposure of the frame that shows the hazard until retarding force begins, the actuators apply no retarding force, and the vehicle continues moving along its existing trajectory at its speed v when that frame is exposed. Assuming velocity remains constant across this window, the blind travel evaluates to

$$d_{\text{blind}} = v \tau_{\text{delay}}$$

Once the brake command arrives and the actuators engage, the body enters the braking phase. The mechanical work done by the braking force F_{brake} over the braking distance d_{brake} must dissipate the initial kinetic energy of the machine, $W = \Delta K$. For a machine of mass m decelerating to rest under a sustained retarding force $F_{\text{brake}} = ma_{\text{brake}}$, the work-energy balance yields

$$ma_{\text{brake}} d_{\text{brake}} = \frac{1}{2} m v^2$$

Dividing by mass and solving for distance gives the kinematic braking travel

$$d_{\text{brake}} = \frac{v^2}{2a_{\text{brake}}}$$

The parameter a_{brake} is the credible deceleration the body can actually exert against the physical environment, not a theoretical peak rating printed on an actuator datasheet. It represents the sustained negative acceleration the mechanical structure, tires, and gears can produce before losing grip (tire-road shear slip saturation), tipping forward, or structural drive-train yield.

² | **Bode Delay Theorem and Phase Margin:** Pure transport delay τ_d introduces an unyielding frequency-dependent phase lag $\Delta\phi = -\omega\tau_d$ radians without altering loop gain magnitude. In closed-loop feedback systems, this phase erosion degrades the phase margin ($\text{PM} = 180^\circ + \angle L(j\omega_c)$); once τ_d erodes the margin to zero at the gain-crossover frequency ω_c , the closed-loop poles cross into the right-half complex plane, destabilizing the physical plant into destructive limit-cycle oscillations.

³ | **IEEE 1588 gPTP:** The Generalized Precision Time Protocol (gPTP) provides sub-microsecond clock synchronization across Ethernet networks by exchanging hardware-timestamped packets directly at the network chip (media access control [MAC] PHY) layer. Capturing the timestamp in hardware keeps the operating system scheduler's jitter out of the measurement.

⁴ | **MIPI CSI-2:** The Mobile Industry Processor Interface Camera Serial Interface 2 (MIPI CSI-2) is a high-bandwidth, high-speed differential protocol used for transmitting uncompressed video data directly from image sensors to host system-on-chip (SoC) devices with minimal latency.

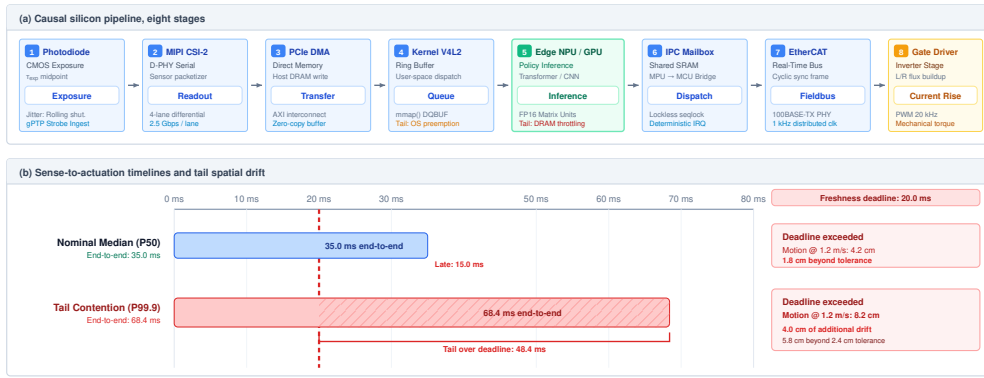


Figure 2.2: Sense-to-actuation latency waterfall and silicon dataflow pipeline: The eight-stage causal chain from exposure to motor current rise (panel a) and an illustrative latency profile (panel b). The parcel sorter’s 24 mm tolerance at 1.2 m/s sets a 20 ms age deadline, because unguided travel converts elapsed time into displacement at the carriage speed. Both the 35.0 ms median and the 68.4 ms $P_{99.9}$ tail exceed it, corresponding to 4.2 cm and 8.2 cm of motion, respectively.

The closed form assumes constant deceleration, instantaneous torque buildup, zero grade, and rigid contact. Each fails on hardware, since force takes time to build, tire friction falls during slip, and structure deflects, so on grades or uncertain floors the model underestimates the stop. Loaded stopping trials across the declared conditions supply a conservative low-tail deceleration, with uncertainty and margin, to replace the idealized a_{brake} ; a sample percentile alone is not a hard lower bound.

The differing exponents of the blind-travel and braking terms make velocity the dominant multiplier of spatial risk (figure 2.3), a scaling that 2.1 works numerically.

In unstructured environments, a machine cannot allocate its entire clearance to nominal stopping dynamics. Obstacle tracking carries state estimation uncertainty, sensor calibration drifts over time, and localization algorithms introduce position error. The stopping distance d_{stop} that the system defends must therefore include a spatial localization uncertainty bound δ_{loc} and a minimum physical clearance margin δ_{margin} .⁵

$$d_{\text{stop}} = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \quad (2.1)$$

The sum $\delta_{\text{loc}} + \delta_{\text{margin}}$ is fixed spatial overhead that must remain unoccupied when the body comes to a complete rest.

Definition 2.1: Dynamic stopping distance

Dynamic stopping distance (d_{stop}) is the minimum physical clearance required to bring a moving mass to a complete halt from its speed v when the hazard is first imaged. It sums the travel during the pre-brake delay τ_{delay} , which runs from the exposure of the frame that shows the hazard until retarding force begins; the braking travel under the credible deceleration a_{brake} ; and the localization bound δ_{loc} and protective clearance δ_{margin} , as equation 2.1 states.

- Significance:** Quantifies the non-negotiable physical spatial margin that software proposals must clear before motion permission can be granted.
- Distinction:** Unlike static geometric obstacle clearance in classical kinematics, dynamic stopping distance grows quadratically with speed (v^2), so doubling velocity quadruples the braking term whatever the controller loop rate.
- Common pitfall:** Assuming an emergency stop command halts the machine instantaneously, ignoring that moving mass carries kinetic energy ($E_k = \frac{1}{2}mv^2$) that requires finite mechanical work and braking distance to dissipate.

Downstream: Unmanaged kinetic momentum translates directly into the destructive contact shock loads analyzed in section 2.4.

⁵ | **ISO/TS 15066 Separation Distance:** ISO/TS 15066 (Clause 5.5.4), building on ISO 13855, specifies this protective separation distance for Speed and Separation Monitoring (SSM): $S = (v_r \cdot T_r) + (v_h \cdot T_r) + S_r + S_s + C$, where T_r is worst-case processing delay, S_r is braking travel, and C is clearance margin (100–200 mm). For learning policies operating in collaborative human workspaces, failing to bound compute tail latency directly inflates T_r , expanding the required separation zone and forcing premature protective stops.

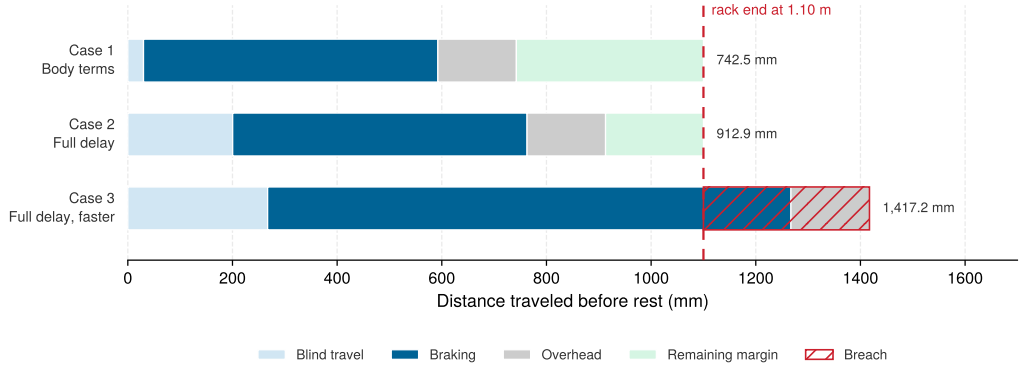


Figure 2.3: Stopping distance of the warehouse mobile manipulator: Each bar splits the stop into blind travel during the pre-brake delay, braking travel under the credible deceleration, and the fixed overhead of the localization bound and protective clearance, against the 1.10 m rack-end clear distance. Case 1 counts only the terms the body owns at the 1.5 m/s drive limit and stops within 742.5 mm. Case 2 adds the full 133.6 ms pre-brake delay that later chapters assemble, the lease path of section 4.3 plus the observation age of chapter 8; at 912.9 mm it still fits, although later chapters add a stop profile and a tracking bound. Case 3 raises speed to a hypothetical 2 m/s, above the drive limit, with the same delay; braking travel grows by 77.8 percent and the stop, at 1,417.2 mm, breaches the clear distance.

For a stationary obstacle detected at distance D_{clear} , such as a person standing at the rack end or a dropped tote, the modeled stopping distance must fit the clear path, $d_{\text{stop}} \leq D_{\text{clear}}$. This is the condition that principle 1 places on every stop, now with each term of d_{stop} a limit of this machine. With conservative timing and braking bounds, the permission path can solve this inequality for a ceiling on new velocity proposals v_{max} . The positive root of the quadratic inequality⁶ is:

$$v_{\text{max}} = -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{(a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}})}$$

When sensor range degrades or an obstacle enters the trajectory, D_{clear} contracts and lowers the ceiling, provided the available distance remains nonnegative. At zero available distance the ceiling is zero. If the machine is already moving, however, a zero setpoint cannot remove its current stopping distance; a negative available distance means the admission condition is already violated and requires an emergency response.



Napkin Math 2.1: The body's stopping distance

Problem: The warehouse mobile manipulator, loaded to at most 368 kg, drives toward a rack end at its $v = 1.5$ m/s drive limit, and a person stands in the aisle at the clear distance $D_{\text{clear}} = 1.10$ m. The body owns three terms of equation 2.1: braking travel under the credible deceleration $a_{\text{brake}} = 2$ m/s² (loaded, dry floor); the fixed overhead of the localization bound $\delta_{\text{loc}} = 50$ mm and the protective clearance $\delta_{\text{margin}} = 100$ mm; and the brake onset $T_{\text{act}} = 20$ ms, which enters τ_{delay} . How much of the clear distance do these terms use, what speed would they alone permit, and how do speed and added delay change the answer?

1. Braking travel: Mass cancels from the work-energy balance, so the loaded machine's 414 J of kinetic energy sets how much heat the brakes must absorb but not how far the base travels:

$$d_{\text{brake}} = \frac{v^2}{2a_{\text{brake}}} = \frac{(1.5\text{m/s})^2}{2 \times 2\text{m/s}^2} = 562.5\text{mm}$$

The stopping distance so far is 562.5 mm.

2. Fixed overhead:

$$\delta_{\text{loc}} + \delta_{\text{margin}} = 50\text{mm} + 100\text{mm} = 150\text{mm}$$

The running total rises to 712.5 mm.

6 | Kinematic Velocity

Root: Grouping the constant spatial terms into available physical distance $D_{\text{avail}} = D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}}$ yields the quadratic inequality $\frac{1}{2a_{\text{brake}}}v^2 + \tau_{\text{delay}}v - D_{\text{avail}} \leq 0$. Multiplying by $2a_{\text{brake}}$ produces $v^2 + 2a_{\text{brake}}\tau_{\text{delay}}v - 2a_{\text{brake}}D_{\text{avail}} \leq 0$. Applying the quadratic formula and taking the positive physical root defines the dynamic velocity ceiling.

3. Brake onset: Between the brake command and the first retarding force, the base still moves at full speed:

$$vT_{\text{act}} = (1.5\text{m/s}) \times (20\text{ms}) = 30\text{mm}$$

$$d_{\text{stop}}|_{\text{body}} = 562.5\text{mm} + 150\text{mm} + 30\text{mm} = 742.5\text{mm}$$

The body's own terms leave 357.5 mm of the 1.10 m clear distance for every delay the body does not own.

4. The body-only ceiling: Solving equation 2.1 for speed with $\tau_{\text{delay}} = T_{\text{act}}$:

$$v_{\text{max}} = -a_{\text{brake}}T_{\text{act}} + \sqrt{(a_{\text{brake}}T_{\text{act}})^2 + 2a_{\text{brake}}(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}})} \approx 1.91\text{m/s}$$

The body alone would permit 1.91 m/s, above the drive limit. Chapter 4 and chapter 8 add the delay terms that spend part of this headroom.

5. Speed against delay: At a hypothetical 2 m/s, above the drive limit, the braking term alone grows to 1,000 mm, 77.8 percent more than at the drive limit. At the drive limit, each additional 100 ms of pre-brake delay adds 150 mm.

Architect's Rule of Thumb: *Never raise a velocity setpoint without re-solving the clearance quadratic.*



Example 2.1: Dynamic velocity ceiling governor

Inputs:

- $D_{\text{clear}} \in \mathbb{R}_{\geq 0}$: Detected obstacle distance along travel vector [m].
- $\tau_{\text{delay}} \in \mathbb{R}_{> 0}$: Conservatively bounded sense-to-braking delay, including actuator response [s].
- $a_{\text{brake}} \in \mathbb{R}_{> 0}$: Conservative lower bound on available brake deceleration under the declared load and surface [m/s^2].
- $\delta_{\text{loc}}, \delta_{\text{margin}} \in \mathbb{R}_{\geq 0}$: Localization uncertainty bound and physical clearance margin [m].
- $v_{\text{now}} \in \mathbb{R}_{\geq 0}$: Current forward speed [m/s].
- $v_{\text{cand}} \in \mathbb{R}_{\geq 0}$: Candidate forward velocity setpoint proposed by neural policy [m/s].

Output:

- $v_{\text{safe}} \in \mathbb{R}_{\geq 0}$: Clamped velocity request within the modeled ceiling [m/s].

Admission Check (while the current state is feasible):

1. Modeled stopping distance: $d_{\text{stop}}(v_{\text{safe}}) = v_{\text{safe}}\tau_{\text{delay}} + \frac{v_{\text{safe}}^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \leq D_{\text{clear}}$ for a stationary obstacle.
2. Non-Negativity: $v_{\text{safe}} \geq 0.0$.

Algorithm Steps:

1. Compute available physical braking distance:

$$D_{\text{avail}} \leftarrow D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}}$$

2. If $D_{\text{avail}} \leq 0.0$, reject the proposal; hold at zero if stationary, or request the maximum available risk-reducing brake response if moving, and flag lost clearance. Otherwise check $v_{\text{now}}\tau_{\text{delay}} + v_{\text{now}}^2/(2a_{\text{brake}}) \leq D_{\text{avail}}$ before admitting another proposal. If that check fails, request the same emergency response and flag lost feasibility. A zero velocity setpoint cannot stop existing motion immediately.
3. Compute quadratic discriminant:

$$\text{disc} \leftarrow (a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}D_{\text{avail}}$$

4. Solve positive physical root for dynamic velocity ceiling:

$$v_{\max} \leftarrow -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{\text{disc}}$$

5. Enforce dynamic clamping against policy proposal:

$$v_{\text{safe}} \leftarrow \min(v_{\text{cand}}, \max(0.0, v_{\max}))$$

6. Return v_{safe} .

The quadratic turns a fixed speed limit into a clearance-dependent ceiling for new proposals, but the ceiling preserves stopping feasibility only if the machine enters each control cycle inside the feasible set and can apply the assumed brake force within the bounded delay. Control Barrier Functions (CBF-QP) (Ames et al. 2019) extend this reasoning to more general systems under valid dynamics, state estimates, timing bounds, and feasible control authority; the 1D root here is a simplified kinematic admission check. If range degrades or delay exceeds its bound, the permission path must reduce speed before margin is exhausted or execute a defined risk-reducing fallback.

Enforcing this speed limit assumes the permission path can demand the deceleration a_{brake} whenever safety requires it. Yet commanding rapid deceleration requires sudden, large mechanical torque from the drive motors and gearboxes. If the permission path requests a torque step that exceeds what the actuator bandwidth budget allows, the mechanical body cannot deliver the required deceleration, invalidating the model-based stopping ceiling and risking collision.

2.4 Actuator Transmission Limits

An electromechanical actuator has no error return. When a policy commands a torque step beyond what the magnetic field can generate, or a discontinuous acceleration the gear teeth cannot carry, the actuator pulls maximum phase current from the power stage, twists its shafts, and strains its teeth while the command registers continue to report nominal execution. **An actuator may damage itself instead of reporting an error.**

An actuator contract cannot be specified in terms of digital setpoints. It must be written at the physical interface in terms of delivered force, torque, or mass flow, response latency, slew rate limits, and output error. A digital setpoint is merely a software request; physical action is bounded by winding inductance, bus voltage ceilings, magnetic saturation, and transmission elasticity.

The delay between commanding a torque and delivering it begins in the stator windings. Because the coils have inductance, current, and with it torque, rises along the motor's electrical time constant rather than instantaneously (Appendix C derives the RL response). A policy that assumes torque appears at once spends that rise as unguided drift before the arm can correct or brake.

Field-oriented control (FOC) synthesizes this torque on dedicated drive silicon at 10–25 kHz.⁷ Torque is proportional to current ($\tau = K_t I$), but the rate at which current can rise is bounded by the bus voltage headroom:

$$\frac{di}{dt} \leq \frac{V_{\text{bus}} - e_{\text{bemf}}}{L}$$

As the rotor spins, the motor also acts as a generator whose back-electromotive force ($e_{\text{bemf}} = K_e \omega$) opposes the supply. At low speed the headroom $V_{\text{bus}} - e_{\text{bemf}}$ permits a fast current rise and full peak torque. As ω approaches its base rating, back-EMF consumes the headroom, and the inverter can no longer force current through the winding inductance whatever its duty cycle, so a policy that commands hard acceleration at high speed does not get the torque step it asks for.

Headroom limits how fast the current can rise; the drive electronics limit how much of it can flow at all. Power inverters enforce a maximum current ceiling I_{max} to protect switching transistors from thermal destruction and to prevent ferromagnetic saturation in the stator core. When the motor drives a mechanical transmission with gear reduction ratio N and transmission efficiency η , the ceiling on delivered output torque is:

$$\tau_{\text{out, max}} = \eta N K_t I_{\text{max}}$$

⁷ | **Field-Oriented Control (FOC) Current Loop:** The current loop runs on the drive's own controller, not on the processor that runs the policy, so a torque step the policy issues reaches the winding only as the ramp that the loop and the bus headroom permit.

The clamp answers the question the stopping budget left open, whether the drives can deliver a_{brake} when the permission path demands it, and it answers silently. A braking demand above $\tau_{\text{out, max}}$ is clipped at the power stage without any report to the software that issued it, so the base decelerates more gently than the stopping distance assumed and comes to rest beyond its calculated boundary. The credible deceleration therefore holds only for the drive current limits configured when it was established, and a firmware change to I_{max} withdraws it until it is measured again, a dependence that section 2.7 writes into the conditions of its limit record.

Only part of that clamped current produces torque, and field-oriented control separates it by a change of coordinates. In a surface-magnet motor only stator current whose field is perpendicular to the rotor flux produces torque, so the drive reads rotor angle from an encoder and rotates the three measured phase currents into the rotor-fixed dq frame, where the quadrature current I_q produces torque and the direct current I_d , aligned with the flux, produces none. A push straight down on a bicycle pedal at the top of its stroke behaves the same way, straining the crank without turning it. The controller regulates I_q to the torque command, holds I_d near zero below base speed, and writes the resulting voltage into the inverter's complementary pulse-width-modulation timers.⁸ Outside field weakening, where the drive deliberately injects negative I_d to offset back-EMF above base speed, direct-axis current heats the winding without moving the joint and spends thermal margin (section 2.5) for no torque.

Gearing adds a mechanical asymmetry. High-reduction joints use ratios of 50:1 to 160:1 to get high torque from a compact motor, which scales delivered torque by N but spins the rotor N times faster than the joint. A load that backdrives the output must spin the rotor N times faster too, and because kinetic energy scales with the square of speed ($\frac{1}{2}J\dot{\theta}^2$), the rotor's inertia J_{rotor} resists joint acceleration multiplied by the **square of the gear ratio** (2.2).

Through a 100:1 transmission, a 50 g rotor presents the joint with the inertia of a 11.2 kg flywheel of 10 cm radius (Lynch and Park 2017). An obstacle striking the gripper must accelerate that flywheel through the gear teeth, whose roots can crack under the shock (figure 2.5).



Definition 2.2: Reflected rotor inertia

Reflected rotor inertia (J_{ref}) is the apparent rotational inertia of an actuator rotor at the load-side joint output, scaling with the square of the transmission gear ratio:

$$J_{\text{ref}} = N^2 J_{\text{rotor}}$$

1. **Significance:** In high-reduction transmissions ($N \geq 100$) it dominates the apparent load, capping joint acceleration $\ddot{q}$ and setting impact shock loads.
2. **Distinction:** Unlike rotor inertia J_{rotor} or link inertia J_{load} , it acts through transmission stiffness and backlash, so backdriving from the environment meets an N^2 barrier.
3. **Common pitfall:** Ignoring reflected rotor inertia during collision and contact modeling. Under sudden obstacle impact, the reflected kinetic energy of the spinning rotor concentrates stress directly on gear teeth before drive current can be interrupted.

The combined equation of motion for a geared joint driving load inertia J_{load} with motor torque τ_{motor} is:

$$\ddot{q} = \frac{N\tau_{\text{motor}}}{J_{\text{load}} + N^2 J_{\text{rotor}}}$$

Differentiating with respect to gear ratio N reveals that peak joint acceleration occurs at the **inertia matching ratio**:

$$N^* = \sqrt{\frac{J_{\text{load}}}{J_{\text{rotor}}}}$$

Below N^* joint acceleration is torque-limited. Above it, more gearing *decreases* delivered acceleration, because the motor spends most of its torque accelerating its own rotor rather than the link.

Because the motor cannot instantly deliver the torque to spin up this amplified inertia, the transmission acts as a mechanical low-pass filter, smearing a sharp command into a delayed response

⁸ | **Inverter Dead Time:** A short hardware-inserted interval in which both switches of an inverter half-bridge are off, so that the high-side and low-side switches of one leg never conduct together and short the DC bus, a failure called shoot-through.

whose lag erodes feedback stability and excites structural resonance. The control path therefore shapes requested motion, with quintic splines or S-curve rate limiters, to keep joint acceleration $\ddot{q}$ within the current and voltage limits of the hardware (Lynch and Park 2017).

Transmissions also introduce backlash and compliance that decouple motor from load. On reversal the motor shaft crosses mechanical clearance before the teeth engage the opposite flank, and torsional compliance in teeth and flexsplines (figure 2.4) makes the link an elastic system rather than a rigid bar (Spong 1987), whose flexible-joint model section B.3.4 derives. Discontinuous torque steps excite that elasticity and can drive the transmission into limit-cycle chatter. A motor-side encoder conceals the deadband, because the motor turns smoothly while the output stays still, so the joint position q_{out} departs from q_{motor}/N by a load-dependent error δ_{lost} . The controller sees continuous tracking while the end-effector hesitates and then takes an impulsive shock as the teeth snap closed, and repeated shocks end in fatigue fracture (figure 2.5).

Systems Perspective 2.1: Impedance control for contact safety

If a controller drives the arm toward a target coordinate in a rigid environment (contact stiffness $k \geq 10^5$ N/m), a positional error of $\Delta x = 1$ mm raises contact force to $F = k\Delta x = 100$ N within milliseconds while the inverter drives maximum current against the obstacle. On the mobile manipulator, the gripper opening the spring-latched cage door meets a strike plate of stiffness 4.0×10^5 N/m, and once contact begins the force rises as $F = kvt$: 12 N/ms at a guarded 0.03 m/s approach and 40 N/ms at 0.10 m/s, against the latch's 100 N limit.

Impedance control (Hogan 1985) frames contact as neither pure position nor pure force control. The environment behaves as an admittance, accepting force and yielding motion, so the manipulator is controlled as an impedance, accepting motion deviations and generating compliant force ($F = Z(v)$). The joint's drive controller converts position setpoints into a **virtual spring-damper system**:

$$F_{\text{contact}} = K_p(x_{\text{des}} - x) + K_d(\dot{x}_{\text{des}} - \dot{x})$$

The joint then behaves like a programmable suspension with stiffness K_p and damping K_d , yielding on contact and capping force long enough for higher-level software to detect the contact and replan.

The choice of transmission trades continuous torque density against backdrivability, and table 2.2 contrasts the backdrivable and high-reduction ends of that range. Cycloidal reducers, at 30 : 1 to 100 : 1, sit between those ends and tolerate shock better than a strain wave gear.

Table 2.2: Actuator transmission trade-off: Reduction ratio (N), reflected rotor inertia scaling (N^2), and the consequence for learned control policies at the backdrivable and the high-reduction ends of the range.

Transmission	Gear Ratio (N)	Reflected Inertia (N^2)	Implication for Learned Policies
Quasi-Direct Drive (QDD)	3 : 1–10 : 1 ($N \leq 10$)	$N^2 \leq 100$ (Low)	Backdrivable; motor current senses contact torque; tolerates impact. Direct drive ($N = 1$) is its limit, at the lowest torque density.
Strain Wave (Harmonic Drive)	50 : 1–160 : 1 ($N \geq 50$)	$N^2 \geq 2500$ –25,600 (Extreme)	Zero initial backlash and compact; torsionally compliant and fragile under sudden tooth shear impact.

This architectural divide across transmission ratios dictates the mechanical transparency and specific torque envelope of the embodiment (figure ??). In direct-drive joints ($N = 1 : 1$), such as the seminal CMU Direct Drive Arm (Asada and Kanade 1983), reflected inertia is negligible ($N^2 = 1$), eliminating backlash and enabling perfect mechanical backdrivability. However, because electric motors produce torque in proportion to stator copper volume and gap radius, direct-drive actuators rarely exceed 5 N·m/kg of continuous specific torque density, burdening manipulators with heavy, bulky linkages. Conversely, classical industrial arms (such as the KUKA LBR iiwa or Universal Robots UR5) and high-load industrial machinery employ high-reduction strain wave or cycloidal



Figure 2.4: Strain wave gear assembly: Robotic strain wave transmission component set: (1) elliptical wave generator with flexible ball bearings, (2) flexible cup flexspline with external teeth, (3) rigid circular spline with internal teeth, and (4) transmission systems invariant governing reflected rotor inertia ($J_{ref} = N^2 J_{rotor}$). Elastic deformation enables zero initial backlash at high reduction ratios (50 : 1 to 160 : 1), but introduces torsional compliance, hysteresis, and tooth fatigue vulnerability under sudden shock loads.

gearboxes ($N = 100:1$ to $160:1$). While high gearing boosts specific continuous torque density to $22\text{--}45 \text{ N} \cdot \text{m}/\text{kg}$, it inflicts an extreme reflected rotor inertia penalty ($N^2 \geq 2,500$ to $25,600$), rendering joints virtually non-backdrivable. Under unexpected contact or obstacle collision, the kinetic energy of the rapidly spinning rotor cannot backdrive the motor; instead, stress concentrates at the gear root fillet, risking catastrophic tooth shear before digital torque loops can intervene.

Modern dynamic physical AI platforms resolve this impasse through **quasi-direct drive (QDD)** actuation (Seok et al. 2015; Wensing et al. 2017; Katz et al. 2019). By pairing high-torque-density frameless outrunner brushless motors with low single-stage planetary reductions ($N \leq 10:1$), QDD architectures cap reflected inertia multiplier below $N^2 \leq 100$. This preserves mechanical transparency, allowing motor phase currents measured at the inverter to serve as accurate, low-latency proxies for contact force—a property termed **proprioceptive sensing**. As illustrated by quadruped and humanoid milestones including the MIT Cheetah 3, MIT Mini Cheetah, Unitree Go1/H1, and Boston Dynamics Electric Atlas, QDD actuators achieve peak intermittent torque densities up to $70\text{--}95 \text{ N} \cdot \text{m}/\text{kg}$ while safely absorbing impact shocks during explosive dynamic maneuvers.

A contact-dominated arm either uses backdrivable Quasi-Direct Drive (QDD) actuators (Seok et al. 2015; Wensing et al. 2017) or stays behind real-time torque limits and impedance control.

With either choice, load-side position or torque sensing exposes the disagreement with the motor encoder that backlash or a damaged tooth creates, and the permission path can then reject further force commands and enter a defined fallback.

Operating near current and torque limits also has a thermodynamic cost. Every ampere that accelerates rotor inertia or sustains peak torque dissipates $I^2 R$ in the copper, and that heat accumulates in the actuator's thermal mass, bounding how long peak torque can last before insulation breaks down.

Downstream: Mechanical gearhead fatigue cycles set the sample efficiency limits for reinforcement learning in section 6.3.



Figure 2.5: Gear tooth shock fracture under high reflected inertia: Macro-fracture of a high-torque transmission gear tooth undergoing cyclic stress testing at NASA Glenn Research Center: [] intact gear tooth flank supporting continuous drive contact; [] tooth root fillet stress concentration ($\sigma_b = \frac{K_t W_t}{b m Y}$), where cyclic tension initiates micro-cracking; [] sheared tooth fragment with granular fatigue striations and sudden cleavage fracture under shock load; and [] reflected rotor inertia ($J_{ref} = N^2 J_{rotor}$). A high gear ratio reduces backdrivability and can increase tooth-root stress under impact; compliance and the contact path determine how much energy reaches any one tooth.

2.5 Thermal Duty Cycles

Holding a load motionless at full reach places deceptive demands on an articulated arm such as the mobile manipulator's. Maintaining a static load against gravity demands continuous phase current through the stator coils, actively converting electrical energy into accumulating heat. **Heat arrives in seconds and leaves in minutes, making duty cycle a primary design variable.**

Momentum can be redirected and flux decays within milliseconds of opening a switch, but heat has no fast exit. Thermal capacity integrates power loss over time, and it protects two material thresholds (figure 2.6). Winding insulation is rated by class under IEC 60085, Class F to a continuous hotspot of 155°C and Class H to 180°C, and above its rating the enamel softens until adjacent turns short and destroy the phase winding. Neodymium-iron-boron (NdFeB) rotor magnets demagnetize irreversibly between 80°C and 150°C under strong stator fields.

Heat generation follows from the current, as Joule dissipation puts $P_{loss}(t) = I^2(t)R(T)$ into the stator copper. That power divides between heating the winding's thermal mass and conducting out to the casing and surrounding air (Appendix C integrates the thermal equation and derives the steady state).

Thermal capacitance C_{th} and thermal resistance θ_{JA} behave like a **leaky bucket**. Joule loss pours in, the thermal mass of the copper and stator iron sets the bucket's volume, and θ_{JA} sets the drain, with a higher resistance narrowing it. A burst of a few seconds raises the level quickly but survives as long as it stays below the insulation rating, while a sustained holding torque raises the level until inflow matches outflow, and if that equilibrium lies above the rating, the enamel fails. Because the thermal time constant $\tau = \theta_{JA}C_{th}$ is of the order of minutes while heating acts in seconds, dissipation low-pass filters the power loss, and the steady-state temperature is set by average loss and thermal resistance.

Policies trained in simulation optimize rewards in which torque is an instantaneous, cost-free variable, so they may hold a heavy payload indefinitely, stiffen a joint by co-contraction, or chatter across a contact boundary without consequence. On hardware, sustained current heats the winding at an initial rate $dT/dt \approx I^2 R / C_{th}$ before conduction establishes any equilibrium. Copper's positive temperature coefficient ($\alpha_{Cu} \approx 0.00393 \text{ K}^{-1}$) compounds the cost. From 20 °C to a 140 °C engineering ceiling, phase resistance rises by 47.2 percent according to $R(T) = R_0[1 + \alpha_{Cu}(T - T_0)]$, so holding

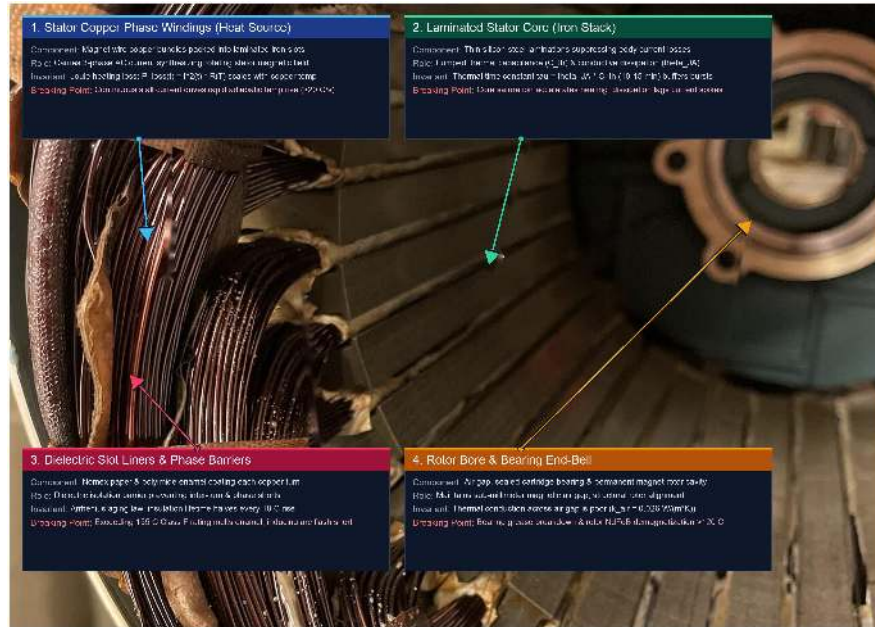


Figure 2.6: Stator winding insulation breakdown: Cutaway view of a damaged electric motor stator showing critical thermal subsystems: (1) copper phase windings (primary I^2R heat source), (2) dielectric slot liners and Nomex insulation breakdown threshold (155°C , Class F), (3) laminated iron stator core (C_{th} and dissipation path θ_{JA}), and (4) rotor bore air gap and magnet thermal limits (80°C – 150°C). (Source: Industrial Electromechanical Diagnostics Archive, CC BY 4.0).

the same torque ($\tau = K_t I$) dissipates 47.2 percent more power just when cooling is least effective, and the added heat raises the resistance further.

Datasheet thermal ratings are optimistic because they are measured on an unconstrained motor mounted to a large heat sink in free air at 20°C or 25°C . Installed in a sealed chassis beside power electronics, battery packs, and other joints, the motor loses convective airflow, which raises θ_{JA} , and neighboring loads can raise its local ambient to 45°C to 55°C before its own coils carry current. Thermal margins must therefore be budgeted from heating and cooling time constants measured with the enclosure sealed and adjacent loads at their P_{99} power draw.

Because thermal capacity integrates power over time, a peak torque⁹ or peak current rating is meaningless without four parameters: the initial winding temperature $T(0)$, the allowed duration t_{on} , the ambient T_{amb} , and the recovery time t_{off} . An actuator that can produce three times its continuous torque sustains it only while the integral of I^2R stays within the margin below the insulation limit, which from a cold winding can last tens of seconds and from a heat-soaked one a fraction of that (2.2).



Napkin Math 2.2: Transient thermal accumulation under cold vs. hot starts

An arm shoulder joint has winding resistance $R = 1.2\ \Omega$, lumped thermal capacitance $C_{th} = 120\ \text{J/K}$, and installed thermal resistance $\theta_{JA} = 2.5\ \text{K/W}$, and it runs in an enclosed ambient $T_{amb} = 40^\circ\text{C}$ with Class F insulation rated to 155°C . The thermal time constant is $\tau = \theta_{JA} C_{th} = 300\ \text{s}$.

Problem: How does the initial winding temperature change how long a threefold rated-current burst can last before the insulation limit?

Math:

1. **Continuous equilibrium:** At $I_{cont} = 5\ \text{A}$ the winding settles at $T_{eq} = T_{amb} + I_{cont}^2 R \theta_{JA} = 115^\circ\text{C}$, below the limit.

Downstream: Real-time telemetry for thermal and power integrity budgets is transported by the deterministic fieldbuses analyzed in section 4.4.

Actuator Intermittent Duty Limits: Robotics motor datasheets prominently advertise peak torque (τ_{peak}), but this rating is strictly intermittent, and how long it can be held depends on the winding's starting temperature. Sustained dynamic maneuvering must be budgeted against continuous root-mean-square torque (τ_{cont}), otherwise aggressive neural policy commands will trip thermal protection and cause mid-motion limb collapse.

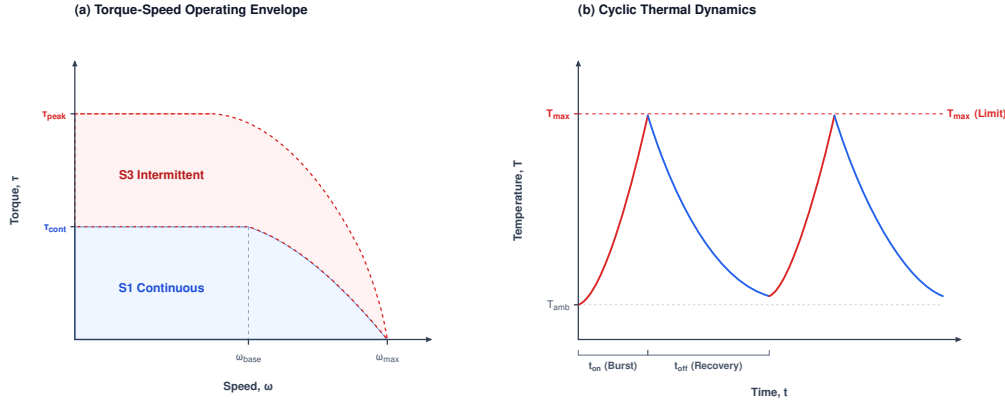


Figure 2.7: Motor torque-speed operating region: The permanent magnet synchronous motor torque-speed plane, separating continuous duty S_1 at thermal equilibrium from intermittent periodic duty S_3 at peak acceleration, alongside the exponential thermal accumulation trajectory during peak bursts and conductive cooling during recovery. Winding thermal limits constrain sustainable duty cycle rather than peak inverter current.

2. **Peak burst:** At $I_{\text{peak}} = 15$ A the dissipation rises ninefold to $P_{\text{peak}} = 270$ W, whose asymptote $T_{\infty} = T_{\text{amb}} + P_{\text{peak}}\theta_{JA} = 715$ °C lies far above the limit.
3. **Survival horizons:** With $T(t) = T_{\infty} - (T_{\infty} - T(0))e^{-t/\tau}$, the time to reach the limit is

$$t_{\text{limit}} = \tau \ln \frac{T_{\infty} - T(0)}{T_{\infty} - T_{\text{limit}}}$$

which gives 56.0 s from a cold start at ambient and 20.7 s from a heat-soaked start at T_{eq} .

Result: The heat-soaked joint holds the burst 63.1 percent less time than the cold one, so peak torque cannot be budgeted without tracking thermal history.

For repeated cycles of period $t_{\text{cycle}} = t_{\text{on}} + t_{\text{off}}$, an engineering ceiling $T_{\text{max}} = 140$ °C sets a cycle-average power ceiling $\bar{P}_{\text{max}} = (T_{\text{max}} - T_{\text{amb}})/\theta_{JA} = 40$ W. With peak dissipation during t_{on} and none during t_{off} , this average-power screen bounds the duty cycle $D = t_{\text{on}}/t_{\text{cycle}}$ by

$$D \leq \frac{T_{\text{max}} - T_{\text{amb}}}{P_{\text{peak}}\theta_{JA}}$$

which here is 14.8 percent, so a 3 s burst calls for a recovery of $t_{\text{off}} = t_{\text{on}}(1 - D)/D \approx 17.3$ s. The average does not bound the hottest instant. In the fixed-resistance first-order model the periodic peak obeys

$$T_{\text{peak}} - T_{\text{amb}} = P_{\text{peak}}\theta_{JA} \frac{1 - e^{-t_{\text{on}}/\tau}}{1 - e^{-t_{\text{cycle}}/\tau}}$$

and the screened schedule reaches 142.7 °C, above the ceiling, before the rise of $R(T)$ adds more. Average-power screening is optimistic, and a burst schedule is accepted only against the transient hotspot trajectory. Figure 2.7 distinguishes continuous duty S_1 from intermittent periodic duty S_3 , whose peak acceleration depends on both initial temperature and cooling time.

The safety microcontroller therefore runs a thermal observer that integrates measured phase current into an estimated hotspot temperature. When the estimate crosses a ceiling, the observer derates the current limits, and if the limit is breached, it vetoes the policy with a thermal shutdown before dielectric breakdown. A static hold shows why the observer must model copper's feedback (2.3).



Napkin Math 2.3: Static holding torque and thermal stall derating

An arm elbow holds $\tau_{\text{hold}} = 42.5 \text{ N}\cdot\text{m}$ at stall ($\omega = 0$) through an $N = 50:1$ cycloidal gearbox ($\eta = 0.85$). Its motor has $K_t = 0.10 \text{ N}\cdot\text{m}/\text{A}$, cold resistance $R_0 = 0.80 \Omega$ at $T_0 = 20^\circ\text{C}$, and $\theta_{JA} = 1.80 \text{ K}/\text{W}$, with a Class F limit of 155°C , inside an unventilated limb at $T_{\text{amb}} = 40^\circ\text{C}$.

Problem: With copper's resistance rising with temperature, where does the stalled winding settle, and what holding torque is thermally sustainable?

Math:

1. **Stall current:** $\tau_{\text{motor}} = \tau_{\text{hold}}/(\eta N) = 1 \text{ N}\cdot\text{m}$, so $I = \tau_{\text{motor}}/K_t = 10 \text{ A}$ and the cold dissipation is $P_0 = I^2 R_0 = 80 \text{ W}$.
2. **Runaway:** With $R(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$, the steady rise satisfies

$$\Delta T = I^2 R_0 [1 + \alpha_{\text{Cu}}(T_{\text{amb}} - T_0 + \Delta T)] \theta_{JA}$$

which evaluates to $\Delta T = 155.32 + 0.5659 \Delta T$, so $\Delta T = 357.8 \text{ K}$ and the winding would settle near 397.8°C , far above its rating.

3. **Derating:** At the limit the allowable rise is 115 K , so the sustainable dissipation is $P_{\text{max}} = \Delta T_{\text{max}}/\theta_{JA} = 63.89 \text{ W}$. At the hot resistance $R(T_{\text{limit}}) \approx 1.224 \Omega$, that allows $I_{\text{max}} = \sqrt{P_{\text{max}}/R} \approx 7.22 \text{ A}$ and a hold of $\tau_{\text{hold,max}} = \eta N K_t I_{\text{max}} \approx 30.7 \text{ N}\cdot\text{m}$.

Result: Held without control, the stall drives the winding toward 397.8°C , and staying within the insulation rating requires a 27.8 percent derating, from $42.5 \text{ N}\cdot\text{m}$ to $30.7 \text{ N}\cdot\text{m}$. Continuous gravity holding belongs to friction brakes or counterweights, not to stall current.

Delivering the current pulses that generate these thermal loads also stresses the upstream electrical supply, moving the burden from heat in the copper to transient current on the power bus.

2.6 Electrical Power Integrity

When the mobile manipulator swerves hard to avoid a collision, every drive motor, joint inverter, and accelerator demands peak current at once from one direct-current distribution bus. In simulation, current flows from an idealized source. On hardware, delivery is bounded by finite capacity, source impedance, and transient voltage stability. **A machine that browns out mid-motion fails physically; the electrical supply is a shared resource.**

Rail sag, regenerative overvoltage, and thermal accumulation are manifestations of one bounded energy network on different time scales. Of the three, voltage moves fastest, within microseconds to milliseconds whenever current demand exceeds what the source can deliver at that instant. Compute, sensing, and motor inverters draw from a common bus, so when a policy commands coordinated accelerations across several joints, the sum of their currents changes the voltage delivered to every node on the machine.

The supply cannot deliver an arbitrary current step. Every bus has a nonzero source impedance,¹⁰ the sum of the battery's internal resistance and the resistance of harnesses, connectors, and protection switches, and bulk capacitors only buffer high-frequency transients locally. A rapid step in total current therefore sags the bus at once through the resistive drop across R_{bus} , the inductive kick across harness inductance L_{bus} , and the depletion of local capacitors (Appendix C derives the transient droop). If the sag crosses the undervoltage lockout threshold, supervisor silicon asserts a brownout reset and drops the machine mid-motion.

That reset turns the electrical fault into a mechanical one. Voltage supervisors pull the reset line on microcontrollers and gate drivers, clocks halt, register configurations vanish, and communication buses float. An acceleration command issued just before the reset stays physically active until the gate-driver stage collapses. If the driver then floats, phase current ceases, torque falls to zero, and the body coasts under gravity and inertia into workpieces or hardstops. If it instead shorts the low-side switches for dynamic braking, back-EMF ($e_{\text{bemf}} = K_e \omega$) produces an immediate, violent counter-torque that jerks the mechanics at the joint limits.

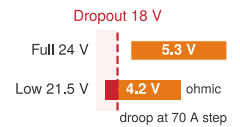
¹⁰ | **Power Distribution Bus Impedance:** The lumped source impedance of an embodied direct-current bus is $R_{\text{bus}} = R_{\text{cell}} + R_{\text{harness}} + R_{\text{switch}}$, the sum of battery cell, harness and connector, and protection-switch on-state resistances. A current surge ΔI drops the rail by $\Delta V = \Delta I R_{\text{bus}}$.

Power removal is an unmanaged state transition, not an automatic return to safety. Cutting power leaves an actuator in one of three unpowered states, namely free coasting, phase-short dynamic braking, or spring-engaged mechanical braking, and each enforces a distinct mechanical behavior. An electromechanical friction brake requires 30 ms to 80 ms for its mechanical spring to overcome magnetic coil decay and engage its friction pads, during which gravity accelerates an overhead arm downward. Phase-short dynamic braking acts like viscous fluid damping, providing resistance proportional to velocity but generating zero holding torque at zero speed. Free coasting simply allows gravity to accelerate unconstrained joints without any resistance. The trajectory a machine follows after an electrical cutoff must be measured under full mechanical payload, not deduced from circuit diagrams. The choice of which safe fallback state the machine should enter belongs to system safety architecture in chapter 12, but the physical dynamics of an unpowered joint are unalterable mechanical constraints of the body.

Deceleration drives the bus toward its opposite boundary, overvoltage, as kinetic energy stored in moving mass returns to it. **Braking converts mechanical energy into electrical energy that the supply network must absorb or dissipate.** If the source cannot accept reverse current, regenerative energy can charge the local DC link according to $\frac{1}{2}C_{\text{bus}}(V_{\text{final}}^2 - V_{\text{initial}}^2) = E_{\text{regen}}$ (see Appendix C). This relation gives a capacitor-only endpoint under idealized assumptions, not a voltage waveform. Protection must divert energy before the bus exceeds inverter and converter ratings.¹¹

Power integrity is an architectural constraint. The design sets coordinated power budgets and current slew-rate limits across accelerators and drives, the permission path arbitrates simultaneous multi-axis accelerations so that their dI/dt transients do not compound, and brake choppers protect the inverters from regenerative feedback during emergency stops. 2.4 works both halves of that budget, the rail's sag and the stop's regenerative surge, on the mobile manipulator's battery rail.

¹¹ | **Dynamic Braking Chopper Circuit:** A power switch and a low-inductance braking resistor placed across the DC bus. When regenerated energy drives the bus above a comparator threshold, the switch conducts and the resistor dissipates the excess as heat, clamping the bus below the breakdown rating of the inverter and converter silicon.



A combined inference and drive current step fits the rail's headroom at full charge; from a depleted battery, its sustained ohmic drop alone crosses the regulator dropout.



Napkin Math 2.4: DC bus voltage sag and regenerative overvoltage

The mobile manipulator's battery feeds one nominal 24 V control rail that carries the drive inverters, the arm's joint drives, and the point-of-load regulators that supply its application processor.

Problem: Under the coincident current step of an inference burst and the drive motors, and then a stop from the drive limit, does the rail remain within the operating limits of the regulators and the motor inverters?

Math:

1. Parameters:

- Machine mass, travel speed, and credible deceleration: $m = 368$ kg (upper bound, full tote rack), $v = 1.5$ m/s, $a_{\text{brake}} = 2$ m/s²
- Rail characteristics: $V_{\text{rail, nom}} = 24$ V, harness resistance $R_{\text{bus}} = 60$ m Ω , parasitic inductance $L_{\text{bus}} = 2$ μ H, bulk link capacitance $C_{\text{bus}} = 22,000$ μ F
- Operating limits: point-of-load regulator dropout $V_{\text{uvlo}} = 18$ V, inverter MOSFET breakdown $V_{\text{breakdown}} = 40$ V

2. Transient droop:

- As an illustrative coincident load, the two drive motors and the arm's joint drives add 66 A while the application processor adds 4 A on the same rail. Their aggregate step is $\Delta I = 70$ A over $\Delta t = 0.5$ ms ($dI/dt = 1.4 \times 10^5$ A/s). The example assumes no separate compute-rail hold-up during this step.
- Transient voltage droop evaluates across resistive, inductive, and capacitive terms:

$$\Delta V_{\text{droop}} \approx \Delta I R_{\text{bus}} + L_{\text{bus}} \left(\frac{dI}{dt} \right) + \frac{\Delta I \Delta t}{2C_{\text{bus}}}$$

$$\Delta V_{\text{droop}} = 4.20V + 0.28V + 0.80V = 5.28V$$

- Minimum rail voltage at full charge:

$$V_{\text{rail, min}} = 24V - 5.28V = 18.72V$$

- The headroom to the regulators' dropout is 6 V, and the modeled 5.28 V droop leaves 0.72 V of it. The inductive and capacitive terms last only through the rise, but the ohmic term persists for as long as the load does, and the shared-rail fallacy in section 2.9 follows it from a depleted battery.

3. Regenerative surge:

- During a stop from the drive limit, the loaded machine's kinetic energy must be dissipated:

$$E_k = \frac{1}{2}mv^2 = \frac{1}{2}(368\text{kg})(1.5\text{m/s})^2 = 414\text{J}$$

- If the battery management system disconnects charge input and an illustrative 75 percent of kinetic energy returns to the bus ($E_{\text{regen}} = 310.5\text{J}$), an unprotected, lossless capacitor-only energy balance would end at:

$$V_{\text{final}} = \sqrt{V_{\text{rail, nom}}^2 + \frac{2E_{\text{regen}}}{C_{\text{bus}}}} \approx 169.7\text{V}$$

- *Failure mechanism:* Long before the bus reached that endpoint, it would cross the 40 V switch rating.

4. Sizing brake chopper:

- Under constant deceleration the regenerative power falls linearly from its peak to zero over the stop, which lasts $t_{\text{stop}} = v/a_{\text{brake}} = 750\text{ms}$. The area under that triangle is E_{regen} , so a $V_{\text{clamp}} = 30\text{V}$ design faces the peak:

$$P_{\text{regen, peak}} \approx \frac{2E_{\text{regen}}}{t_{\text{stop}}} = 828\text{W}$$

- Maximum chopper resistance to divert peak power:

$$R_{\text{brake}} \leq \frac{V_{\text{clamp}}^2}{P_{\text{regen, peak}}} \approx 1.09\Omega$$

- *Component screening:* A 1Ω resistor with a nominal 1 kW pulse rating is a candidate. Verify its 310.5 J pulse-energy and temperature limits, chopper switch safe operating area, and clamp response for the actual braking profile before treating the absorption path as protective.

Result: At full charge the coincident step leaves the rail at 18.72 V, only 0.72 V above the 18 V regulator dropout, so a depleted battery erases the margin. Unbuffered regenerative braking would drive the bus past the 40 V MOSFET rating, which calls for a dynamic brake chopper whose resistor is no larger than 1.09Ω and absorbs an 828 W peak.

Systems insight: Mechanical motion and electrical power are bidirectionally coupled: acceleration requires capacitive hold-up to prevent brownouts, while deceleration requires active dynamic braking to prevent silicon overvoltage destruction.



Checkpoint 2.1: The five physical budgets

Before measuring these limits on an assembled machine, verify your understanding of the five budgets:

- **Freshness deadline:** Can you calculate the maximum allowable measurement age for the mug riding the pick-station conveyor at 0.20 m/s, with a 15 mm grasp tolerance of

which the initial estimate already spends 3 mm, and explain why that age is counted from exposure on the sensor rather than from the start of inference, and why the base's stop for a standing person instead charges age as blind travel?

- **Velocity ceiling:** Can you explain why doubling the pre-brake delay expands the stopping distance linearly while doubling travel speed expands the braking term quadratically, and why the admissible speed is the positive root of $d_{\text{stop}} \leq D_{\text{clear}}$?
- **Inertia matching:** Can you explain why the gear ratio that maximizes link acceleration makes reflected rotor inertia equal load inertia ($N^2 J_{\text{rotor}} = J_{\text{load}}$), and why oversizing the gearbox for static torque lowers dynamic joint acceleration?
- **Thermal horizons and runaway:** Can you explain why an actuator can produce three times its rated torque for seconds but destroys its winding insulation if held there for minutes, and how the temperature coefficient of copper (α_{Cu}) turns a sustained stall into runaway?
- **Rail sag and regeneration:** Can you explain how a coincident compute and drive current step sags a shared rail through resistance, inductance, and capacitor depletion, and why a stop from speed pumps energy back into the bus that a brake chopper must divert?

2.7 Measuring a Machine's Own Limits

Before an actuator is integrated into a chassis, it is characterized on a dynamometer stand (figure 2.8). Software treats parameters as nominal constants in a configuration file, but in physical systems engineering **a measurement that cannot state its instrument, its percentile, and its sample count cannot support an argument later.**

A measurement begins from the decision it will govern in the permission path, which fixes the quantity, the observation point, the sensor bandwidth, the resolution, and the reference clock. Establishing the mobile manipulator's credible deceleration, for example, calls for the peak phase current and the deceleration trajectory from the instant of the brake command, captured by an isolated current probe sampled at 100 kHz and the wheel encoders on a common hardware timebase; an averaging bench meter yields no usable number. An instrument whose analog bandwidth falls below the electrical rise time attenuates the true transient peak, and margins built on it rest on a measurement artifact rather than on the machine.

Every probe also intrudes on the process it observes. A passive voltage probe loads high-frequency lines, a series current shunt drops the rail enough to trigger an early brownout, and a thermocouple inside a motor housing changes its heat paths. Instrument uncertainty is therefore a compound budget, not a single tolerance from a calibration sheet, combining sensor linearity, quantization, frequency-dependent phase lag, probe loading, and clock drift between acquisition channels. When an optical encoder with 0.05° quantization is sampled at 1 kHz to compute joint velocity through discrete backward differences, numerical differentiation noise introduces velocity variance of several degrees per second that exists entirely in the measurement pipeline rather than in the mechanical joint.

A limit means nothing without the context of its test. Every recorded limit states the payload, ambient temperature, supply voltage, enclosure state, firmware version, and concurrent computational workload of the run. A thermal time constant measured on an open bench at 20°C can differ by an order of magnitude from the same actuator's constant in a sealed fairing in sunlight at 45°C , and a rail characterization that omits whether vision-language model inference (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) was running during motor stalls describes an idle board rather than the combined draw of full compute and peak torque.

Every number that governs a machine is a direct measurement, a value derived from measurements through a stated model, a datasheet rating, or an illustrative example, and only the first two can support a safety margin or a timeout. A budget the designer sets, such as the protective clearance δ_{margin} , is instead chosen, and it binds only once it is checked against the measured number it must fit (see the Reader Guide).



Figure 2.8: Electric motor dynamometer testbed: High-bandwidth actuator characterization stand at NASA Armstrong Flight Research Center. Direct empirical measurement of peak torque saturation, electrical time constants ($\tau_e = L/R$), continuous thermal rise ($\Delta T = P_{\text{loss}} \theta_{JA}$), and regenerative power feedback requires synchronizing multi-channel load cells, optical encoders, and isolated current probes on a shared hardware timebase rather than trusting manufacturer catalog ratings. (Source: NASA, Public Domain).

The absence of a failure during testing bounds the failure rate statistically; it does not establish safety. If the mobile manipulator completes $n = 3,000$ loaded stops without a brownout or an overrun, the data support only a per-stop failure rate below roughly $3/n$ (section 7.1), about one stop in 1,000 at 95 percent confidence, and only if the trials are independent and stationary. Windings that heat across successive trials and gears that wear make consecutive trials correlated and non-stationary, so the same stops run on a cool morning say nothing about the afternoon, when a hotter winding has higher resistance and less peak torque.

Characterization is therefore a standing record, re-established as the machine ages, because physical limits drift over operational time, a process called **lifecycle degradation**. Brake pad friction glazing and tire tread wear degrade the sustainable deceleration limit a_{brake} ; chemical aging in lithium-ion battery packs increases internal DC bus impedance R_{bus} , exacerbating voltage droop (ΔV_{droop}) during dynamic maneuvers; and mechanical gear tooth wear widens backlash clearance δ_{lost} .

A limit whose violation can cause damage is kept not as a bare number but as a **limit record** with six elements, which table 2.3 defines and fills in for two limits of the mobile manipulator: the credible deceleration on which its stopping distance rests, and the dropout of the regulators on its shared control rail.

Table 2.3: The limit record on the mobile manipulator: The six elements of a limit record, filled in for the credible deceleration that sets the stopping distance and for the regulator dropout that bounds the shared control rail. Neither value yet has samples or an established uncertainty, and the provenance element names the measurement that would replace each.

Element	What the record states	Credible deceleration a_{brake}	Control-rail regulator dropout
Value	The value or empirical distribution, with its sample count and the tail percentiles it reports	2 m/s ² , taken as the credible low tail of loaded stops; no samples yet	18 V at the point-of-load regulators; no samples yet

Element	What the record states	Credible deceleration a_{brake}	Control-rail regulator dropout
Conditions	The context listed earlier under which it was established, plus the surface for any limit that depends on traction	Loaded to at most 368 kg; dry floor with $\mu \geq 0.204$; drive current limits as configured; tires at commissioning wear	Valid only while the permission path runs on its own isolated, held-up rail, the design decision of section 2.8, so that a crossing resets only the application processor; the regulators on that rail are assumed to have the same dropout
Uncertainty	The measurement uncertainty of each instrument, with its observation point and bandwidth	Not established	Not established
Margin	The engineering margin separating the operating target from the breakdown threshold	None on the value; the 100 mm protective clearance δ_{margin} absorbs its error	0.72 V at full charge under the coincident step; none from a depleted battery
Detector	The runtime check that catches an excursion while the machine runs, such as a phase-current sensor, an optical interrupter, or a supply-rail monitor	Deceleration monitor comparing commanded with measured deceleration from the wheel encoders and the Bosch BMI088 inertial unit, on every stop	Rail-voltage monitor on the shared rail, whose undervoltage flag the permission path answers with a stop
Provenance	The evidence category and the instrument or model that produced the value	Illustrative, from the base's class profile; to be replaced by loaded stopping trials	Illustrative, from the machine's profile; to be replaced by a rail characterization under coincident load

Each degradation mechanism above changes a limit's conditions, so a deployed machine re-establishes its records by periodic recalibration rather than trusting the values measured at commissioning. The saturation duration against which the Schiaparelli lander's inertial unit had been accepted was such a limit, and the lander's loss in 2016 shows what follows when a limit's conditions are exceeded in operation and nothing downstream checks the result.



War Story 2.1: ExoMars Schiaparelli IMU saturation and altitude collapse (2016)

Context: On October 19, 2016, the European Space Agency's ExoMars Schiaparelli lander executed its atmospheric entry and descent toward the Martian surface, relying on an onboard Guidance, Navigation, and Control (GNC) computer to fuse Inertial Measurement Unit (IMU) gyroscopes with a Doppler radar altimeter (ESA Investigation Board 2017).

Mechanism: About three minutes after atmospheric entry, the parachute inflated and set the lander oscillating. The IMU measured a pitch rate larger than expected and raised a saturation flag, which persisted longer than the 15 ms assumed when the unit was accepted. While the flag was set, the GNC software integrated the saturation threshold rate as though it were the true rate, although the lander was in reality oscillating, and its attitude estimate drifted by about 165° , nearly upside down. When the radar altimeter later returned valid slant ranges, the software projected them onto that attitude and computed a negative altitude. No onboard check tested that altitude for plausibility.

Impact: Taking the negative altitude as a valid estimate, the guidance logic released the parachute and backshell, fired the braking thrusters for 3 seconds instead of the planned 30 seconds, and activated its on-ground systems as if it had landed. The lander was still about 3.7 km above the surface. It fell freely for about half a minute and struck the ground at about 150 m/s (540 km/h).

Response: The ESA inquiry board recommended robust sanity checks in the onboard software, among them that altitude cannot be negative and cannot change from 3.7 km to a negative value in one second, and a design that handles IMU saturation explicitly, for example by inhibiting IMU inputs during parachute deployment.

Systems lesson: A physical body cannot transcend the saturation ranges of its sensory transducers. When a sensor's physical limit (a gyroscope's rate ceiling or an accelerometer's dynamic

range) is exceeded, a state estimator that integrates the saturated measurement as if it were valid drifts away from the physical state. The saturation duration against which the IMU had been accepted was a limit with conditions, and once descent exceeded them, no detector checked the estimate before it commanded an irreversible action.

Measured and recorded one at a time, the five limits describe the body budget by budget, each under its own conditions. Which of them binds first depends on the machine and the command.

2.8 Which Budget Binds First

A single command from the chunk policy to accelerate the loaded mobile manipulator spends from all five budgets at once. It rests on a camera frame that is already aging, it adds kinetic energy that the brakes must later remove, it accelerates reflected rotor inertia through the drive gearing, it deposits Joule heat in windings that shed it only over minutes, and it draws current from the rail that also feeds the application processor. Table 2.4 sets the five budgets side by side, with the law that governs each, the assumption a policy trained in software makes about it, the machine class it binds first, and the design response and runtime invariant that answer it.

The mobile manipulator carries two of the machine classes of section 1.3 on one chassis, a Class 1 base that binds first on momentum and stopping distance and a Class 2 arm that binds first on reflected inertia, contact stiffness, and holding heat.

A Class 3 plant binds on heat rather than on stopping distance. A directed-energy-deposition cell, which builds a metal part by melting fed powder with a laser, drives a 5 kW beam into a melt pool under a 1 kHz control loop, so each control period delivers 5 J of heat in the part whether or not the gantry moves. Its limit is the heat the melt pool and the surrounding part can absorb. A 100 ms controller stall with the laser still on delivers 500 J, more than the 414 J of kinetic energy the loaded mobile manipulator carries at its drive limit, and no brake can take that heat back out. For flow-dominated plants, **thermal dissipation and electrical power integrity** govern survival, and long thermal time constants ($\tau = \theta_{JA} C_{th}$) hide heat that accumulates slowly until a small continuous overload has already crossed an insulation or material limit.

Downstream: The degraded limits recorded here bound the supervisor intervention thresholds derived in section 14.3.

Table 2.4: Cross-class physical constraint matrix: The five budgets, the law that governs each, the assumption a software-trained policy makes about it, the class it binds first, and the runtime invariant that answers it.

Physical Budget / Factor	Governing First Principle & Scaling Law	Software Illusion vs. Hardware Ground Truth	Primary Binding Class	Systems Architect Takeaway & Runtime Invariant
1. Measurement Freshness & Age	$\Delta t_{\text{fresh}} \leq \frac{\epsilon_{\text{max}}}{v_{\text{max}}}$; end-to-end age measured, not summed from stage percentiles	<i>Illusion:</i> High inference rate (50 Hz) guarantees real-time response. <i>Truth:</i> Age begins at exposure; pipeline latency injects phase lag ($-\omega\tau$).	Class 1 & 2: Erodes clearance margins and destabilizes high-gain contact loops.	Tag frames with hardware timestamps; enforce $t_{\text{age}} \leq$ the budgeted age at motor registers; drop stale commands.
2. Momentum & Stopping Distance	$d_{\text{stop}} = v \tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} \leq D_{\text{clear}}$	<i>Illusion:</i> Deceleration can be commanded instantaneously. <i>Truth:</i> Braking distance scales quadratically (v^2); kinetic energy requires friction work ($W = Fd$).	Class 1 (Mobility): Primary constraint. Doubling speed quadruples the braking term.	Solve clearance quadratic $v_{\text{max}}(D_{\text{clear}}, \tau_{\text{delay}})$ on the safety microcontroller at the rate the stopping budget sets; clamp policy velocity.
3. Actuator & Reflected Inertia	$J_{\text{ref}} = N^2 J_{\text{rotor}}$; $\tau_e = \frac{L}{R}; I(t) \leq I_{\text{max}}$	<i>Illusion:</i> Motor torque tracks policy setpoints without mechanical resistance. <i>Truth:</i> Gearbox squares rotor inertia (N^2); inductance delays current rise; shock steps shear teeth.	Class 2 (Manipulation): Primary constraint. Rigid contacts create extreme dF/dt force spikes and gear fatigue.	Enforce C^2 spline (S-curve) trajectory rate limits; sample load-side optical encoders; deploy quasi-direct drive (QDD) actuators.

Physical Budget / Factor	Governing First Principle & Scaling Law	Software Illusion vs. Hardware Ground Truth	Primary Binding Class	Systems Architect Takeaway & Runtime Invariant
4. Thermal Limits & Duty Cycle	$C_{th} \frac{dT}{dt} = I^2 R(T) - \frac{\Delta T}{\theta_{JA}};$ $D \leq \frac{T_{max} - T_{amb}}{P_{peak} \theta_{JA}}$	<i>Illusion:</i> Motors deliver peak catalog torque continuously. <i>Truth:</i> Joule heat ($I^2 R$) has positive thermal feedback (α_{Cu}); heat accumulates in sealed housings.	Class 3 (Thermal) & Class 2: Static holding draws stall current, overheating coils in seconds.	Run discrete $I^2 t$ thermal observers on the safety microcontroller; dynamically derate speed and torque when hotspot models approach limits.
5. Electrical Power Integrity	$\Delta V_{droop} \approx \Delta I R_{bus} + \frac{\Delta I \Delta t}{2 C_{bus}};$ $V_{final} = \sqrt{V_{rail, nom}^2 + \frac{2 E_{regen}}{C_{bus}}}$	<i>Illusion:</i> Power rails act as infinite, ideal voltage sources. <i>Truth:</i> Bus impedance collapses rail voltage under combined compute+motor bursts; braking surges overvoltage.	Class 1 & 3: Peak multi-axis acceleration trips microcontroller brownout resets; regen surges destroy inverter FETs.	Enforce coordinated compute-actuation power budgets and dI/dt limits; install dynamic brake choppers; characterize unpowered fallbacks.

The table separates the budgets, but the machine does not, and the margins computed on the mobile manipulator differ in how much a single command can spend. The shared control rail has none left. From a battery depleted to 21.5 V, the coincident step of 2.4 sags it to 17.3 V, below the 18 V dropout of its regulators, so its margin is already negative. A heat-soaked arm joint has a margin measured in time, since a sustained threefold current burst carries its winding to the insulation limit in 20.7 s. The stopping distance at the 1.5 m/s drive limit still keeps 357.5 mm of the clear distance, which only added delay spends, and a thermally throttled application processor would take 16.5 mm of it.

Because the rail binds first, the design answer is to take the permission path off it. The safety microcontroller, the drive logic, and the spring-brake coils run on an isolated rail that a local store holds up for a chosen 2 s, long enough to outlast the longest stop the permission path can command (section 12.9 sizes that stop). Those loads draw 70 W, so the hold-up needs 140 J, which a 2.025 F store delivers between the battery-low point and the regulators' dropout. A brownout of the shared rail then resets the application processor, while the permission path rides through it, flags the undervoltage, and stops the machine. The complete coupling, in which the body's stopping distance sets the lease the permission path grants and that lease sets what the Brain must supply, is worked once for Part I in section 4.8.1.

2.9 Fallacies and Pitfalls

Physical limits interact across subsystem boundaries. A design that meets each component budget can still fail when heat, timing, power, and motion change together.

Fallacy: *Model inference time defines the measurement freshness latency.*

A team quantizes its vision backbone and reduces inference time from 30 ms to 10 ms. The improvement shortens one stage, but the controller still acts on measurements that pass through exposure, readout, transfer, and operating-system queues. Commands then travel over the network fieldbus before the motor coil current rises to apply force. A 45 ms transport spike elsewhere in this chain can consume the clearance margin even when inference meets its benchmark. The engineering budget must measure the true “photon-to-torque” elapsed time from sensing to physical response and bound its tail, rather than declaring the loop real-time from model execution time alone.

Fallacy: *Software step commands deliver instantaneous physical torque.*

A simulated policy issues piecewise-constant setpoints at 50 Hz and assumes the actuator follows each change immediately. The physical joint, however, responds through stator inductance (L/R) and reflected rotor inertia ($N^2 J_{rotor}$). An abrupt command can exceed the actuator bandwidth budget, saturating the inverter's current limit and violently loading the transmission before the joint achieves the requested motion. This physical lag alters the feedback behavior learned in simulation. Setpoint shaping and the actuator model must account for these physical time constants; a software update rate does not set the joint's achievable response.

Pitfall: *Treating isolated component datasheets as operational system guarantees.*

An engineer uses a motor's continuous torque rating to set the planner's limits. The rating carries the vendor's free-air test conditions (section 2.5), not those of the sealed joint beside the compute unit, so it overstates what the installed motor can sustain. Because the planner treats that rating as its torque limit, every trajectory it later generates inherits the error, and the fault surfaces only after the enclosure has heat-soaked, far from the design decision that caused it. Planning limits must derive from measurements on the fully assembled machine under expected environmental and cooling conditions.

Pitfall: *Treating dynamic voltage and frequency scaling (DVFS) as a transparent software optimization.*

The mobile manipulator's drive motors heat the shared chassis until its application processor triggers dynamic voltage and frequency scaling (DVFS) to throttle clock frequencies and avoid junction overtemperature. This thermal intervention stretches the vision backbone from 22 ms to 33 ms. The extra 11 ms enters the observation age t_{age} , and at the 1.5 m/s drive limit it adds 16.5 mm of travel to the stopping budget before any brake command can follow from what the camera saw. The power manager has protected the silicon, but it has silently changed the timing the stopping budget assumed. A previously adequate stopping margin may no longer be adequate. Thermal telemetry must therefore inform the motion budget, with travel speed reduced when processor throttling increases the time needed to perceive and respond.

Pitfall: *Scaling operational velocity without recalculating quadratic braking distances and thermal dissipation.*

Repeated high-current braking heats the motor windings, so a stop the mobile manipulator survives once at a raised operating speed may exceed its allowable thermal duty cycle (the thermal capacity budget) when repeated through a shift, on top of the quadratic growth in braking travel that Case 3 in figure 2.3 shows at 2 m/s. A higher operating speed requires a new stopping-distance calculation and a thermal assessment of repeated maneuvers before the planner can safely treat that speed as available.

Fallacy: *Compute accelerators and motor inverters operate on independent, ideal power rails.*

On the mobile manipulator, a heavy inference burst coincides with the drive motors pushing the base over the cage-door threshold. The combined 70 A transient across the 60 m Ω harness resistance of the shared 24 V control rail produces a 4.2 V drop. With the battery already depleted to 21.5 V, the rail sags to 17.3 V, below the 18 V dropout threshold of the onboard point-of-load regulators, and the application processor resets. Had the safety microcontroller shared that rail, the same brownout would have reset it too while the machine carried active momentum, leaving the base and arm to the unpowered states of section 2.6 instead of a commanded stop. Separate software budgets for compute and actuation ignore their physical reliance on a shared electrical supply. Power qualification must test coincident peak loads from a depleted battery, with the controller that stops the machine on its own held-up rail (section 2.8).

Fallacy: *Passing isolated single-variable bench tests guarantees physical survival under compound operational stress.*

A robot arm passes a maximum-payload trial in a 20°C air-conditioned laboratory and an empty-gripper trial in a 45°C thermal chamber. Neither test establishes its ability to carry that maximum payload in the hotter environment while running on a depleted battery. Under these combined conditions, the heavy physical load raises resistive winding losses, the hotter surroundings reduce the system's ability to reject that heat, and the lower supply voltage leaves less headroom above back-EMF, capping speed and current slew. Consequently, each subsystem can approach its failure limit much sooner than the separate, isolated trials suggest. Qualification must therefore exercise realistic combinations of payload, temperature, and battery condition on the assembled machine, rather than treating individual test passes as evidence about the combined conditions.

2.10 Summary

The body sets the budgets that every learned proposal spends, and none of them can be renegotiated while the machine runs. A moving mass cannot shed its kinetic energy faster than its credible deceleration allows, a winding cannot shed heat faster than its thermal resistance allows, and an

observation cannot be younger than the path from transduction to torque. The chapter reduced the body to five such budgets, each a quantity the permission path can check before it admits a command, and each entering that path as a bound rather than a typical value (the percentile-versus-bound distinction of section 4.4). Every physical limit depends on a declared operating configuration and loses authority when it changes, which is why the chapter keeps each one as a limit record that names its conditions, uncertainty, margin, detector, and provenance. A stopping distance or thermal time constant cannot be taken from a component datasheet into a safety case, and a change of payload, surface, cooling, supply voltage, or wear requires the limit to be measured again before a timing, force, or energy budget reuses it. Of the margins computed on the machine, the shared control rail binds first, so the permission path runs on its own held-up rail. On the mobile manipulator at its 1.5 m/s drive limit, braking, fixed overhead, and brake onset already spend 742.5 mm of the 1.10 m clear distance, leaving 357.5 mm for the delays of sensing and computation, the stop profile, and the tracking bound that later chapters add.



Key Takeaways: The physical body and invariant budgets

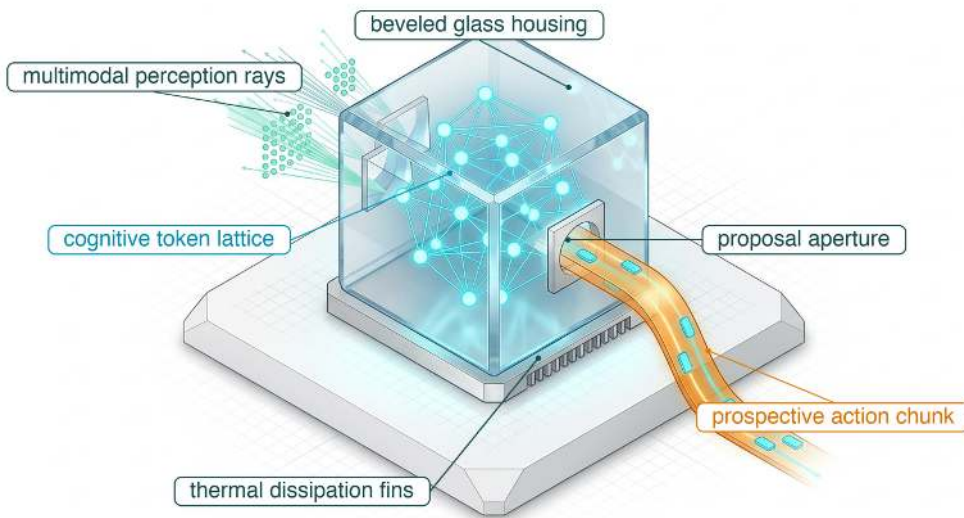
- **Age is counted from transduction:** Observation age begins when the sensor integrates light or flux, not when inference starts, and the end-to-end tail is not the sum of stage tails. The permission path compares the hardware timestamp with the age its budget allows, which for a tracked moving target is $e_{\max}/v_{\max}$, and rejects a stale proposal.
- **Speed sets the stopping envelope:** Blind travel grows linearly with pre-brake delay and braking travel quadratically with speed, so the admissible speed is the positive root of $d_{\text{stop}} \leq D_{\text{clear}}$, recomputed whenever clearance or delay changes.
- **Gearing trades torque for responsiveness:** A gear ratio N multiplies reflected rotor inertia by N^2 . Past the inertia-matching ratio, more gearing delivers less joint acceleration and concentrates impact stress in the gear teeth.
- **Heat is an integral state:** Joule loss arrives in seconds and leaves over minutes, and copper's temperature coefficient feeds it back. A peak-torque rating means nothing without its initial temperature, duration, ambient, and recovery time.
- **The supply rail is shared:** The drives and the application processor draw on one rail, so a coincident current step from a depleted battery can cross its regulators' dropout, and an unclamped regenerative stop can destroy the inverter that brakes it. The controller that must stop the machine therefore runs on its own rail, held up long enough that a brownout of the shared rail cannot reset it before the stop completes.
- **A limit is a record, not a number:** A limit governs a decision only with its conditions, uncertainty, margin, detector, and evidence category, and it lapses when wear, heat, a new payload, or a new build changes those conditions.



What's Next: From the body's limits to the Brain's proposals

The limit records of this chapter say what the mobile manipulator's body can deliver and under which conditions: how old an observation may be, how far the loaded base travels before it stops, and how much torque, heat, and rail current a motion may spend. None of them says what the machine should do next. Chapter 3 takes up the Brain, the learned models that propose the next motion, and asks what such a model can supply within these budgets: how far ahead one chunk of setpoints reaches, how long a new chunk takes to compute, and how quickly the evidence behind it ages. Chapter 4 then derives how the permission path, below the proposal boundary, decides which of those proposals the body may execute.

The Cognitive Brain



- 3.1 The Embodied Brain
- 3.2 The Classical Wall
- 3.3 Learned Representations
- 3.4 Supply, Freshness, and the Memory Wall
- 3.5 Cognitive Vulnerabilities
- 3.6 Fallacies and Pitfalls
- 3.7 Summary

Purpose

Why can the most capable model on a robot revise its plan only a few times a second?

A robot operating among humans must interpret dynamic environments and propose actions far beyond the rigid scripts of classical automation. High-capacity learned models supply this semantic reasoning by grounding multimodal sensory streams in vast representations acquired during offline training. Yet this expressive power is paid for in high memory traffic and severe computational latency. Every forward evaluation of a multi-billion-parameter network streams weights across a memory interconnect of finite bandwidth. Consequently, the deliberative model that understands the open-world scene best is inevitably the component that revises its decisions least frequently.

To keep actuators continuously supplied between deliberative updates, the Brain must emit prospective action chunks—temporal sequences of future setpoints. However, these proposed setpoints age with the stale observation that generated them, and a proposal that arrives on time can still be physically disastrous. High prediction confidence does not entitle a statistical model to command the physical machine. The Brain generates candidate behaviors across the proposal boundary, while the deterministic Nervous System evaluates each chunk against current sensor telemetry and safety envelopes before allowing low-level commands to cross the causal boundary.

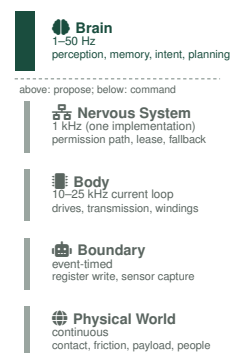


Figure 3.1.: The five-level machine stack with the Brain level highlighted.

Prerequisite: A learned proposal stays unprivileged until the permission path admits it, the third law of section 1.7.



Learning Objectives

- Compare classical analytical pipelines and learned representations on coverage, robustness, and verifiability
- Calculate the bandwidth bound on a model's fresh-evaluation rate from its weight footprint and sustained memory bandwidth
- Estimate key-value cache growth from camera count, tokens per step, and retained history
- Explain why action chunking multiplies setpoint supply without making proposals any fresher
- Apply replanning period, inference latency, and target drift to size an action chunk for a moving target
- Diagnose proposer failures caused by hallucination, miscalibrated confidence, perceptual chattering, and expired intent
- Evaluate why these proposer limits require a permission path that can refuse any proposal

3.1 The Embodied Brain

THE warehouse mobile manipulator can meet all electrical and thermal budgets yet still fail at a task as ordinary as taking an unfamiliar ceramic mug from the moving conveyor at the pick station and handing it to the coworker at the packing station. The motor drives remain well within their continuous torque ratings, the control rail holds its voltage at full charge, and the joint velocities remain below physical limits. Yet the machine drops the object or crushes the ceramic rim because low-level motor physics cannot determine what the object is, where its structural affordances lie, or how to reach around clutter. Unlike an advisory display, whose output can be reviewed before it affects the world, a cognitive proposal here can reach moving mass with no human in between. A mistaken proposal can lead to unsafe kinetic energy if the execution path accepts it, so permission must be checked independently. The central dilemma of embodied cognition is that high-capacity neural models are inherently stochastic, computationally expensive, and prone to inventing objects that are not there, yet they must orchestrate physical bodies whose collisions cannot be recalled (the first law, section 1.7).

The Brain is therefore an unprivileged deliberation engine that may only propose (1.3). This architectural separation organizes the machine into a **two-speed brain architecture** (figure 3.2). At the deliberative tier, high-capacity foundation models perform slow cognition at 1–50 Hz, parsing open-world multimodal streams to emit structured proposals. At the real-time tier, deterministic safety monitors execute at 1000 Hz on isolated safety silicon, evaluating candidate proposals against physical limits before granting execution permission (u^*), or asserting an asynchronous refusal that triggers deliberative state resynchronization. On the mobile manipulator it holds two learned proposers, an intent model that revises goals at 1–5 Hz and a chunk policy that emits short sequences of future setpoints at 10–50 Hz, and the permission path may refuse anything either one proposes. That protection holds only while the limits, the state estimate, and the fallback it relies on remain valid.

A learned proposer earns its place by doing what hand-authored geometry cannot. It recognizes an object no one modeled from an open-vocabulary instruction, tolerates glare and clutter that break hand-written filters, and anchors what it recognizes to metric coordinates the arm can reach. Each of these capabilities comes from a model whose size also sets how often it can run. Figure 3.3 shows the tension across workload classes, with the models that carry the most parameters updating least often, far below the rates at which the Body's loops close. The chapter's question is therefore what a learned proposer can deliver on time, and at what cost in memory, latency, and freshness.

Section 3.2 shows why pipelines built on hand-authored geometry and state machines lose coverage outside the factory cell, and section 3.3 shows what learned representations recover and what they cost in tokens and memory traffic. Section 3.4 turns that cost into the machine's numbers: the memory wall that bounds how often its intent model can run, the action chunking that keeps the arm supplied between evaluations, and the drift that bounds how long each chunk stays true.

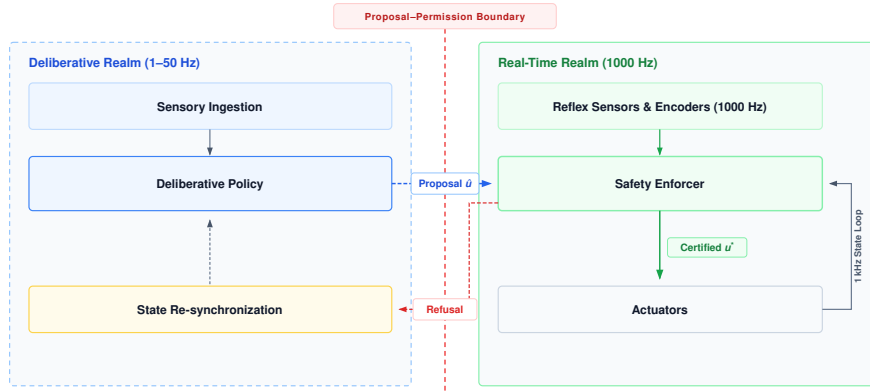


Figure 3.2: Two-speed brain architecture: An unprivileged deliberative realm running at 1–50 Hz and a real-time realm running at 1000 Hz, separated by the proposal-permission privilege boundary. The independent safety enforcer checks sensed state and modeled limits on each control cycle before granting execution permission; a refusal path feeds state resynchronization back to the deliberative policy.

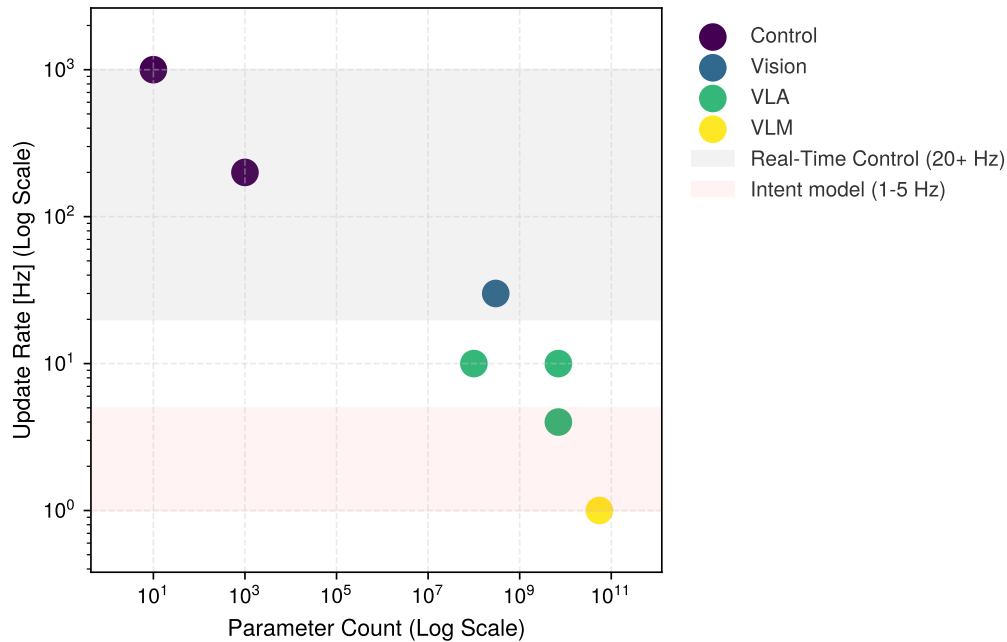


Figure 3.3: Cadence regimes: Parameter counts and example update rates for different control, vision, and embodied workloads. The points are illustrative workload regimes rather than runs on a common device, and parameter count alone does not determine update rate.

Section 3.5 then takes up the ways a proposal can arrive on time and still be wrong, each of which leaves the permission path with a proposal it must be able to refuse.

3.2 The Classical Wall

For nearly a century, automated control and robotics approached cognition as an explicit geometric and analytical physics problem (Wiener 1948; Kalman 1960; Craig 2005). Under this classical paradigm, engineers constructed mathematical representations from first principles.

Engineers used analytical kinematics and differential dynamics to map desired end-effector motions into actuator torques using closed-form equations.¹ Perception relied on matching hand-



Figure 3.4: Industrial spot-welding workcell: Multiple six-degree-of-freedom articulated robots execute preprogrammed spot welding paths on stamped automotive chassis at the BMW Leipzig plant. Operating in a calibrated factory environment with submillimeter repeatable fixtures and safety interlock cages, classical controllers provide deterministic cycle timing and formal stability proofs. (Credit: BMW Group / Wikimedia Commons, CC BY-SA 2.0 de).

¹ | **Rigid-Body Dynamics Formulation:** Classical control computes generalized torques via the standard manipulator equation $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$, where $\mathbf{M}(\mathbf{q})$ represents the positive-definite inertia matrix, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ captures Coriolis and centripetal terms, and $\mathbf{g}(\mathbf{q})$ accounts for gravitational loading. This analytical formulation becomes computationally intractable and physically inaccurate when machines interact with unmodeled frictional contacts, fluid drag, or deformable objects. Derivations are provided in Appendix B.

crafted 3D Computer-Aided Design (CAD) models against sensor point clouds using geometric alignment algorithms, such as iterative closest point (ICP) or random sample consensus (RANSAC). High-level task logic was authored as explicit Boolean state machines that dictated deterministic transitions between operational phases, such as approaching, aligning, grasping, and retracting.

Where the operational environment is strictly structured and bounded (figure 3.4), classical methods can provide dependable behavior within their modeled conditions. On an automated automotive assembly line, an industrial robot welds stamped steel panels where every workpiece arrives at an exact coordinate within sub-millimeter tolerances. Ambient lighting is tightly regulated by industrial fixtures, and human workers are excluded by physical interlock safety fences. Within the causal boundary defined in section 1.2, this controlled regime permits explicit models, bounded servo timing, and stability analysis under stated assumptions. Engineers can verify selected trajectory and torque constraints and assess controller software coverage against applicable standards without collecting task demonstrations.

Yet when an embodied machine leaves the structured factory cage to operate in unstructured environments (figure 3.5), this paradigm hits the classical wall (Brooks 1991; Levine et al. 2018). The physical world is not a sterile CAD drawing; it is continuous, deformable, and endlessly variable.

The first breakdown is the combinatorial explosion of geometry. In a commercial logistics warehouse handling hundreds of thousands of retail goods, authoring CAD models, mesh templates, and grasp points for every bottle, wrapped package, and flexible garment is impossible. Discretizing orientations, frictional properties, and contact permutations across overlapping, deformable items creates a combinatorial space that hand-crafted templates cannot cover.

The second breakdown arises from sensory brittleness. A hand-crafted edge-and-segmentation pipeline may rely on filters, Harris corners, or surface-normal thresholds. Under changed sunlight, reflections, or dust, a required feature can disappear. In that pipeline, a missing edge can make planar segmentation bridge two objects and propose a grasp in empty space.

The third breakdown is the engineering bottleneck of hand-authored logic. The machine's operational capability is bounded by the engineer's ability to anticipate every physical edge case. When an unexpected variation occurs, such as a bent bracket, a scuffed label, or an unfamiliar obstacle, the hand-authored rulebook contains no matching branch, causing the machine to halt, drop its payload, or collide.



Figure 3.5: Unstructured search-and-rescue robot: A tracked mobile robot navigates an obstacle course with irregular inclines, loose debris, and visual occlusions. In such unmapped environments, preauthored geometric models and rigid state machines can lose coverage; learned policies may help infer spatial affordances and traversability from sensory streams. (Credit: TU Darmstadt Rescue Robotics / Wikimedia Commons, CC BY-SA 3.0).

Learned representations answer the first two breakdowns directly and ease the third, because they draw coverage from data rather than from hand-authored models.

3.3 Learned Representations

Asked to take the unfamiliar ceramic mug from the conveyor, a classical pipeline needs a CAD model, a feature threshold, and a state-machine branch that nobody wrote for this object. A learned model trained on many graspable objects can propose the handle as a grasp region from pixels and a sentence of instruction. Physical AI extends classical pipelines with such neural representations wherever hand-authored models lose coverage, and four pillars support the shift.

The first pillar is open-world semantic generalizability. Classical controllers are geometrically precise but semantically blind: an inverse kinematics solver can expertly drive an end-effector to coordinate (x, y, z) , yet it possesses zero understanding of whether that location contains a sturdy mug or a fragile glass, or how surrounding objects relate to human tasks. Modern vision-language-action (VLA) foundation models project open-vocabulary natural language instructions and high-resolution camera streams into shared semantic embeddings (Driess et al. 2023; Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024). When commanded to “wipe the coffee spill from the counter and stow the mug in the drying rack,” a trained model may propose the mug handle and spill boundary as task-relevant regions without a new CAD model.

The second pillar is empirical sensory robustness. Hand-authored visual thresholds can flip under changing illumination or texture. Learned representations can tolerate some of these changes when training and testing cover them, but their responses are neither necessarily smooth nor bounded under shift.

The third pillar is the ability to handle complex multi-contact and deformable dynamics. Many physical manipulation tasks (folding textiles, routing flexible wire harnesses through automotive door assemblies, or scooping granular media) involve non-smooth contact mechanics, variable friction, and infinite-dimensional deformations that cannot be modeled in closed-form differential equations. With suitable contact-rich demonstrations and feedback, neural policies can learn compliance-aware responses that reduce force against unexpected resistance.



Figure 3.6: Bimanual teleoperation workcell: Dual-arm teleoperation setup featuring two six-degree-of-freedom follower arms, two leader arms for kinesthetic teleoperation, multi-view cameras (overhead and wrist-mounted), and parallel grippers mounted to an extruded aluminum frame. The physical AI policy ingests synchronized multi-camera streams to predict continuous end-effector trajectories, and the permission path must refuse any proposal that would cause self-collision or motor over-torquing during closed-loop execution. (Credit: Stanford University & Google DeepMind, CC-BY / Apache 2.0).

The fourth pillar is empirical scaling with data and compute, the bitter lesson that section 1.7 sets against the physical wall of onboard energy, heat, and memory. Training on diverse robotic demonstrations (figure 3.6) and multimodal data can improve performance on tested tasks (Zhao et al. 2023; Chi et al. 2024).

Each of these gains holds only where training and test data cover the operating conditions, and none of them bounds what the model proposes outside those conditions. That limit is why a learned output reaches the actuators only as a candidate that the permission path may refuse.

Rather than an opaque monolith, an embodied foundation model is a three-stage neural pipeline (figure 3.7, figure 3.8) that transforms sensory inputs into candidate motion proposals, and it ends at a boundary where those proposals still need kinematic checks.

1. **Sensory Ingestion and Spatial Tokenization (The Inputs):** A vision transformer frontend cuts each camera frame into a grid of fixed-size pixel patches and projects each patch into one token, tagged with its image position and, when several cameras are mounted, a learned camera identifier. Language directives and the robot’s joint state are projected into tokens of the same width, and the three streams concatenate into one input sequence. Every patch becomes a token that the backbone must attend over and the key-value (KV) cache must hold, so a robot with three cameras (left wrist, right wrist, overhead) contributes hundreds of visual tokens per frame before any text or proprioception arrives (3.1). Image position is not metric depth; placing the gripper in 3D still requires depth evidence and calibrated geometry.
2. **Multimodal Alignment and Attention (The Backbone):** The combined sequence passes through tens of transformer layers whose self-attention weighs every token against every other, so attention work grows with the square of sequence length while the KV cache that stores prior token states grows linearly with it. Every added camera or retained frame is therefore paid for in latency and in memory traffic. The same pairwise weighting does not establish

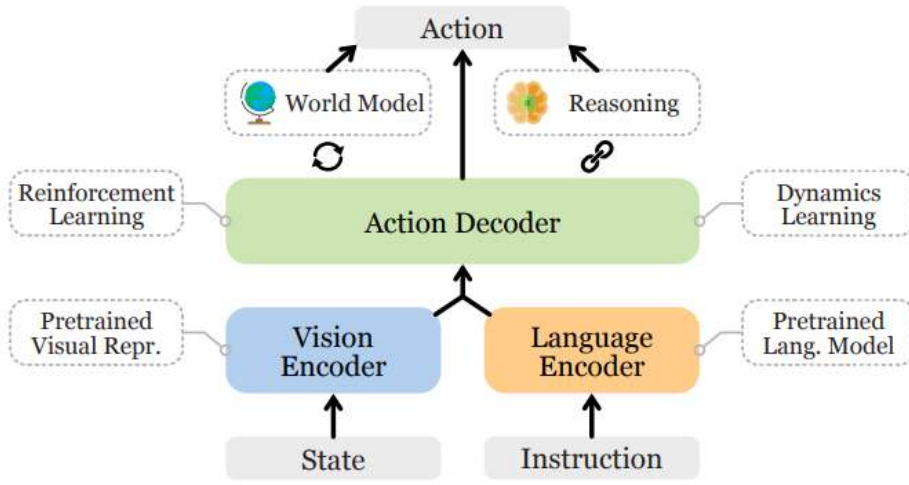


Figure 3.7: Embodied model taxonomy: Embodied foundation models grouped by perceptual frontend (visual tokens, point clouds, language prompts), neural backbone (transformers, diffusion heads, world models), and target action space. The taxonomy contrasts Vision-Language-Action policies that decode topological waypoints and semantic frontiers with Vision-Language-Action policies that emit joint and Cartesian trajectory chunks for articulated manipulators. (Adapted from Ma et al. (Ma et al. 2024)).

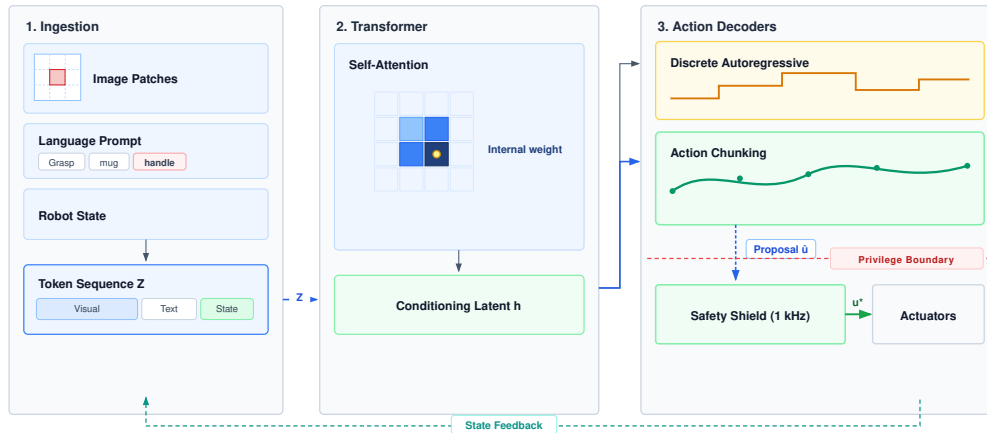


Figure 3.8: Embodied Vision-Language-Action (VLA) architecture: A representative dataflow. (1) Camera patches receive 2D image-position and camera identifiers alongside language and robot-state tokens; metric depth still requires calibrated evidence. (2) Self-attention mixes the combined token sequence; the colored grid illustrates internal attention weights, not a verified affordance or grasp pose. (3) Discrete autoregressive decoders emit tokens sequentially, whereas continuous decoders generate candidate multi-step chunks ($\mathbf{A}_{t:t+H-1}$) through one pass or iterative denoising. (4) An independent permission path checks candidate motion against sensed state and modeled limits before actuation. (Synthesized from Brohan et al. (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), Kim et al. (Kim et al. 2024), and Octo Model Team (Octo Model Team et al. 2024)).

geometry, so a text token such as “handle” can attend strongly to a visual patch without that weight being a verified grasp location or metric pose. Nor does the KV cache track object permanence; an explicit state estimator must associate observations across frames and grow its uncertainty while the mug is occluded. The patch-embedding, position-encoding, and attention equations behind both stages are in section A.3.2.

3. **Specialized Action Decoders (The Outputs):** Instead of generating conversational text, the final hidden layer $\mathbf{h}_{\text{context}}$ feeds task-specific action decoders. While vision-language navigation

(VLN) models emit topological waypoints or heading rates (v_t, ω_t) (figure 3.9), manipulation (VLA) models diverge sharply in how they bridge semantic representations to physical motion:

- *Discrete Categorical Tokenization (RT-1, RT-2, OpenVLA)*: Early VLAs adapt autoregressive language modeling directly to robotics by discretizing continuous joint or Cartesian offsets into $B = 256$ uniform bins per dimension (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024):

$$\text{bin}(a_k) = \left\lfloor \frac{a_k - a_{\min}}{a_{\max} - a_{\min}} \times 255 \right\rfloor \in [0, 255]$$

Each bin is assigned a discrete token (`<act_000>` to `<act_255>`) in the language vocabulary. While this natively leverages web-scale vision-language pretraining, it introduces a systems bottleneck: a decoder that emits one scalar token at a time needs seven serial token predictions for a 7-DoF action. Its latency depends on caching, hardware, and whether each prediction revisits the full backbone; seven serial steps can miss a control deadline. Scalar binning has a resolution of roughly $\Delta a = (a_{\max} - a_{\min})/256$; whether that produces contact chatter depends on smoothing, the action range, and the downstream controller.

- *Continuous Generative Action Chunking (Octo, π_0 , ACT, Diffusion Policy)*: To avoid paying a full weight sweep for every scalar action, modern generalist policies couple the transformer backbone with continuous generative decoders, such as diffusion policy heads (Chi et al. 2024; Octo Model Team et al. 2024) or flow-matching action experts (Black et al. 2024). Rather than predicting a single scalar per forward pass, the model predicts an entire continuous trajectory chunk:

$$\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times d_a} \quad (H = 16\text{--}64 \text{ steps})$$

A backbone may evaluate one new camera observation, then a smaller decoder may perform several denoising iterations to propose a whole chunk of waypoints. That chunk is one candidate horizon, not a sequence of fresh observations. The next backbone evaluation must arrive before the accepted execution window expires; downstream interpolation and feedback run at their own rates.

4. **The Proposal Boundary (Isolation from Actuators)**: Whether the model is a VLN mobility planner or a VLA manipulation policy, its output is a candidate proposal (p_t) on the unprivileged side of the proposal boundary (1.3). Under the machine model of section 1.5, the foundation model runs in user space on the application processor and has no path to the pulse-width modulation (PWM) registers or to the current in the motor coils.

The design rule for this deliberative boundary is that learned models operating at the Brain’s inference cadences (1–50 Hz) propose kinematic setpoints ($\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ or Cartesian poses), **never unbuffered, open-loop motor torques** (τ) directly to inverter switches. Motor torque is coupled to the plant through the manipulator equation of the classical-dynamics note in section 3.2, plus a contact term τ_{contact} . Because inertia $\mathbf{M}(\mathbf{q})$ and Coriolis forces $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ vary non-linearly with instantaneous joint configuration and velocity, commanding open-loop torques, without high-frequency feedback, from a neural model that updates at tens of hertz at most can destabilize the limb when model error, friction, or payload variation is large enough. Kinematic setpoints, by contrast, specify an intended spatial path and leave high-gain closed-loop torque synthesis to the Body’s drives, whose field-oriented current loops run at 10–25 kHz, once the permission path has admitted each setpoint. (Where specialized high-rate reinforcement learning policies operating at 100–500 Hz output torques directly, they still require an independently validated high-rate permission path, as section 4.7 argues; passivity checks for such torque streams are derived in section B.8.)

Proposals reach the permission path through the handoff of chapter 4, and the permission path can refuse a hallucinated proposal when independent measurements, modeled limits, and a feasible fallback support that decision.²

Each neural stage of this pipeline is paid for in memory traffic. Every patch token must be attended over and held in the KV cache, and every evaluation of the backbone streams its weights from DRAM,

Downstream: Zero-copy proposal passing between the Brain and the Nervous System relies on the lock-free buffers analyzed in section 4.3.

² | **Proposal Boundary:** Treating the neural policy as an unprivileged proposal generator isolates stochastic inference from actuator registers. Candidate trajectory setpoints $\mathbf{q}_{\text{ref}}(t)$ are checked on dedicated safety silicon, so proposals that violate modeled actuator limits or keep-out regions can be rejected before any reaches a drive (section 4.5).

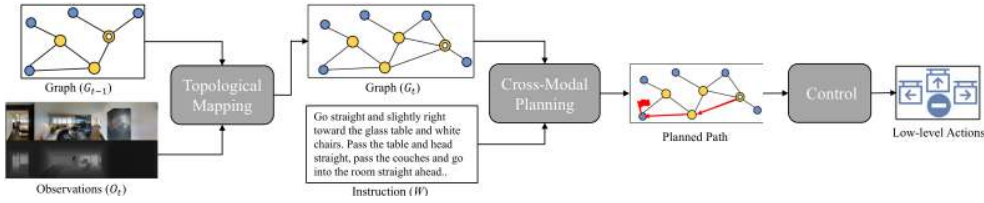


Figure 3.9: Topological vision-language navigation: Embodied navigation architecture executing natural-language spatial instructions. Panoramic visual observations are encoded into perceptual tokens, matched against prompt embeddings, and cross-referenced with a dynamically updated topological navigation graph. The action decoder prunes unreachable nodes and emits metric waypoint offsets (Δx , Δy) and heading setpoints to guide the chassis along traversable semantic frontiers without requiring an exact geometric prior map. (Adapted from An et al. (An et al. 2023)).

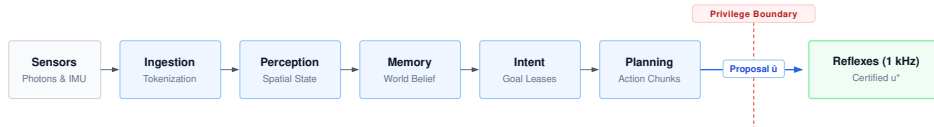


Figure 3.10: The physical AI cognitive flow: Information flows sequentially from physical sensors through Ingestion (tokenization), Perception (spatial geometry and affordances), Memory (persistent world model belief), Intent (deliberative goal leases), and Planning (multi-step action chunking). The flow ends at the proposal boundary, where the permission path evaluates each proposed action against real-time constraints and resources.

so the capabilities this section described arrive at a rate the application processor sets rather than one the task chooses. On the mobile manipulator, that rate for its largest model falls well below the rate at which its arm consumes setpoints.

3.4 Supply, Freshness, and the Memory Wall

At the pick station the mobile manipulator’s arm consumes a new setpoint every few tens of milliseconds, while the mug it is reaching for keeps moving along the conveyor. Whatever model proposes those setpoints must settle two questions. The first is how often it can produce a proposal on the application processor the machine actually has; the second is how long each proposal stays true once the mug has moved. The answers explain why the machine carries two learned proposers rather than one.

After ingestion (the tokenization of section 3.3), four stages turn those tokens into proposals (figure 3.10), and Part III develops each one. Here they matter only for what each hands downstream and how quickly that goes stale. Perception grounds what it recognizes in metric coordinates relative to the base, so that the mug becomes a pose and a graspable handle rather than a label;³ its output ages from the moment of capture, and chapter 8 turns that age into a contract. Memory carries those estimates through occlusion and must let each one expire when the evidence behind it does (chapter 9). Intent decomposes a task such as “take the ceramic mug off the conveyor and hand it to the coworker at the packing station” into subgoals, and because the mug moves while the intent model deliberates, it hands the planner a target that expires, an intent lease (section 10.7). Planning converts that target into candidate setpoints for the base and the arm, the proposals the permission path will judge (chapter 11). These stages, and the ingestion before them, all run on the application processor and draw on the same memory bus, which is where the machine’s cadence is decided.

The memory wall

Consider what happens if a robot attempts to generate continuous motor actions one discrete step at a time through an autoregressive foundation model (such as a multi-billion-parameter VLA model). A model whose weights cannot remain on chip must stream them across the external DRAM bus once per evaluation, so the bus bandwidth B_{mem} bounds its evaluation rate at $f_{\text{max}} = B_{\text{mem}}/M_{\text{weights}}$ before tokenization, attention, or decoding adds any time. In single-step autoregressive decoding, each sequential token may require another such evaluation; the actual traffic depends on caching

³ | **Metric Spatial Grounding:** While disembodied AI operates over discrete semantic tokens, physical AI must ground those semantics in the Special Euclidean group $SE(3)$, mapping entities to metric distances (meters) and spatial orientations (quaternions) relative to the robot’s base coordinate frame. Failing to maintain this metric grounding causes neural policies to emit kinematically impossible Cartesian paths that breach actuator workspace boundaries.

Downstream: High-throughput neural inference accelerators are isolated from deterministic cores in section 13.2.

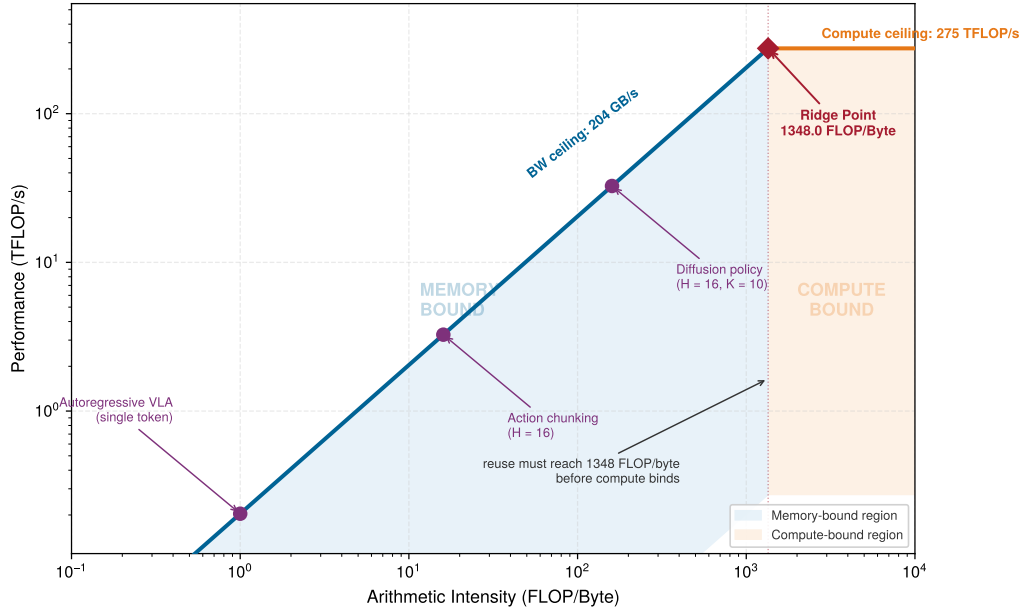


Figure 3.11: Illustrative embodied roofline: Arithmetic intensity against attainable throughput for an example edge accelerator. Single-token weight streaming can be memory-bound; reuse across a chunk may increase work per byte. A separate real-time loop executes accepted waypoints between fresh model updates.

and the decoder architecture. For detailed memory wall derivations and roofline hardware models, see Appendix C.

This memory wall⁴ places single-token streaming deep in the memory-bound region of the roofline (figure 3.11) (Williams et al. 2009). On the mobile manipulator’s application processor it caps the intent model’s fresh-evaluation rate, before any vision or decoding work, far below the rate of a joint controller that asks for a new command every millisecond (3.1). A model renewed that slowly can say where the arm should go, but it cannot close the loop on moving mass, so it proposes waypoints and a faster loop below it tracks them.

This continuous data movement also carries an energy penalty. Fetching weights from off-chip DRAM consumes orders of magnitude more energy than the arithmetic computation itself. That energy drains battery reserves, and the heat it leaves in the chassis raises the local ambient of the joint actuators, shrinking the winding margins of section 2.5.

Action chunking

To supply dense waypoints despite slow full-model inference, modern physical AI policies employ **action chunking** (3.1) (Zhao et al. 2023; Chi et al. 2024). Each chunk then crosses the proposal boundary as one packet, the chunk payload of section 4.3.

A chunking policy (such as Action Chunking with Transformers [ACT], Diffusion Policy, or Flow Matching) generates its whole chunk of H setpoints in one backbone pass followed, for an iterative decoder, by an assumed $K = 10$ – 16 refinement steps. Each refinement re-evaluates the smaller action head, so full latency is $t_{\text{vision}} + t_{\text{backbone}} + K t_{\text{head}} + t_{\text{transfer}}$, not just one weight sweep. A new observation changes the plan only after another backbone and decoder pass; see Appendix A for the generative mechanism.

⁴ | **Autoregressive DRAM Bandwidth Saturation:** Sequential action tokens can trigger repeated full-model weight traffic when weights are nonresident and the decoder revisits the backbone; cache behavior and hardware determine the actual bandwidth ceiling. Action chunking is one way to reduce the number of full policy evaluations per unit of physical time.

Definition 3.1: Temporal action chunking

Temporal action chunking is the policy formulation that predicts an entire horizon of H future continuous actions $\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \dots, \mathbf{a}_{t+H-1}]$ in a single neural evaluation or diffusion rollout, amortizing parameter memory transfers across extended physical execution windows.

1. **Significance:** On a bandwidth-limited device, repeating a full model evaluation for every action can be too slow for millisecond control. Temporal action chunking can reuse a backbone evaluation across multiple waypoints; it does not make fresh observations or policy decisions arrive at the waypoint rate. A downstream controller tracks the accepted trajectory between model updates.
2. **Distinction:** Unlike single-step Markovian policies $\mathbf{a}_t \sim \pi(\cdot | \mathbf{o}_t)$ that re-evaluate the full neural network at every control step, action chunking deliberately generates multi-step feedforward trajectories, utilizing receding-horizon execution and blending across overlapping chunks to reconcile slow neural deliberation with fast closed-loop control.
3. **Common pitfall:** Executing action chunks open-loop without continuous temporal smoothing or runtime safety filtering. Without smooth blending across overlapping chunk boundaries, switching between independently generated horizons injects acceleration jumps and torque shocks that excite mechanical gearbox resonances.

For example, when a canonical bimanual manipulator threads a flexible USB cable (figure 3.6), it executes a 32-step action chunk, a longer chunk than the mobile manipulator uses. The policy emits coordinated 14-DoF joint setpoints that smoothly align both grippers, flex the cable, and insert the connector without pausing between individual control cycles.

However, action chunking introduces the problem of **chunk seam continuity**. Because each action chunk is generated independently from rolling sensory observations, the tail of chunk k and the head of chunk $k + 1$ may exhibit slight positional and velocity discrepancies. If a controller concatenated these chunks directly, the resulting step-discontinuities in acceleration would inject large jerk ($\ddot{\mathbf{q}}$) into the physical linkages, inducing mechanical shock, acoustic vibration, cycloidal pin wear, and motor overcurrent faults.

To reduce discontinuities between overlapping chunks, systems can implement **temporal ensembling**, blending overlapping trajectory predictions via exponentially weighted averaging:

$$\mathbf{a}_t = \frac{\sum_{i=0}^{\min(t, H-1)} w_i \mathbf{a}_{t-i}}{\sum_{i=0}^{\min(t, H-1)} w_i}, \quad w_i = \exp(-m \cdot i)$$

where $\mathbf{a}_{t-i}$ represents the action predicted for time step t by the action chunk initiated at time $t - i$, and $m > 0$ is a tunable temporal discount factor. The permission path interpolates the blended waypoints below the proposal boundary (section 4.3). Ensembling averages setpoints without enforcing derivative continuity, so a seam can still need the C^2 bridge that section 11.4 builds.

This operational tension between neural deliberation capacity and physical control latency defines the central scaling frontier of modern robotic brains (figure ??). Between 2020 and 2023, pioneering vision-language-action (VLA) models such as RT-1 (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023), RT-2 (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), and OpenVLA (Kim et al. 2024) tokenized continuous actions into discrete bins, predicting one action step per autoregressive forward pass. However, because each forward pass requires streaming every model parameter from DRAM across the silicon memory bus, policy update rates collapsed as model capacity scaled ($f_{\text{action}} \propto 1/M_{\text{weights}}$)—trapping multi-billion-parameter models behind an insurmountable “deliberation wall” (1–3 Hz) that fell far below the 50 Hz closed-loop threshold required for dynamic contact and reactive manipulation.

Action chunking fundamentally decoupled deliberation from actuation frequency. Instead of streaming billions of parameters for every low-level setpoint, architectures such as ACT (Zhao et al. 2023), Diffusion Policy (Chi et al. 2024), and modern flow-matching foundation models (e.g., Physical Intelligence π_0 (Black et al. 2024) and NVIDIA Project GR00T) evaluate their large perceptual backbones periodically to generate entire multi-step action trajectories ($H = 16\text{--}64$).



Overlapping action chunks blend predictions across receding temporal execution windows.

Compact, highly optimized generative action heads then denoise or integrate these trajectories at 50–100Hz, shattering the autoregressive memory wall and reconciling multi-billion parameter semantic reasoning with millisecond closed-loop physical control.

The machine’s two proposers

On the mobile manipulator’s own application processor, the memory wall and the amortization that chunking offers decide how the machine divides proposal work between its models.

Napkin Math 3.1: Memory walls, supply, and freshness

Can action chunking let a multi-billion-parameter model on an embedded compute module renew proposals as fast as the arm consumes them, or does the machine need a second, smaller proposer?

1. *The single-token memory bottleneck.* Consider the mobile manipulator's intent model, a 7B VLA model stored in 16-bit precision (FP16) in the LPDDR5 memory of its application processor, an embedded edge System-on-Chip:

- Weight footprint: $M_{\text{weights}} = 7\text{B params} \times 2 \text{ bytes} \approx 14 \text{ GB}$.
- Theoretical memory bandwidth: $B_{\text{peak}} = 204 \text{ GB/s}$.
- Assumed sustained DRAM bandwidth (70 percent of peak to model bus scheduling and memory overhead): $B_{\text{sustained}} \approx 142.8 \text{ GB/s}$.

Minimum streaming latency to fetch the weights from DRAM across the silicon bus once:

$$t_{\text{stream}} = \frac{14\text{GB}}{142.8\text{GB/s}} \approx 98.0\text{ms}$$

Bandwidth-only upper bound on fresh evaluations when each evaluation streams the full weights:

$$f_{\text{single}} = \frac{1}{t_{\text{stream}}} \approx \frac{1}{0.0980\text{s}} \approx 10.2\text{Hz}$$

Beyond model weights, autoregressive foundation models also grow their KV cache. When multi-camera sensory feeds (e.g., 3 224×224 RGB streams) are ingested, each video frame contributes hundreds of visual patch tokens ($3 \times 256 = 768$ tokens). In the intent model's 32-layer transformer with hidden dimension $d = 4096$, storing the FP16 Key and Value states consumes 0.52 MB per token:

$$M_{\text{KV}} = 2 \times L \times d \times S \times \text{bytes} = 2 \times 32 \times 4096 \times S \times 2 \approx 0.52\text{MB} \cdot S$$

Across a 768-token visual prompt, the KV cache expands by 402.7 MB per frame.

2. *The action chunking amortization.* Suppose the intent model itself emitted an action chunk of $H = 16$ future setpoints ($\Delta t = 20$ ms, spanning 320 ms) from each forward pass, instead of one action step per DRAM weight sweep.

- Number of weight memory sweeps: 1 sweep (98.0 ms).
- A chunk can reuse one backbone evaluation across H future setpoints.
- Amortized weight-sweep time per setpoint (excluding all other inference work):

$$\tau_{\text{step}} = \frac{t_{\text{stream}}}{H} = \frac{98.0\text{ms}}{16 \text{ setpoints}} \approx 6.1\text{ms per setpoint}$$

3. *The machine's split.* Amortization spreads one sweep across more setpoints, but it does not make a fresh proposal arrive sooner. Each new chunk from the intent model would still wait for at least one 98.0 ms sweep, plus tokenization, attention, and decoding. The mobile manipulator therefore divides the work between two models. Its intent model revises the goal at 5 Hz with

160 ms of inference. A separate 50M-parameter chunk policy, whose 0.10 GB of weights sweep in 0.70 ms, proposes base velocity and arm joint targets together at 20 Hz, within a 40 ms P99 inference budget that includes vision. The intent model's sweep is too slow to renew a proposal every few tens of milliseconds; the chunk policy's is not. Chapter 4 turns that difference into a lease.

The 98.0 ms sweep in 3.1 is a floor reached before vision, attention, or decoder work begins. Two alternatives to the machine's split, lower weight precision and a single model with a fast head, would attack it, and neither removes it cheaply. INT4 storage would reduce the nominal weight footprint to 3.5 GB but needs accuracy validation for contact tasks. A single-model design could instead trade cadence for density without changing weight precision. It would evaluate the 7B backbone periodically while a lightweight diffusion head denoises candidate trajectories on-chip, supplying dense candidate waypoints from each slower full-model evaluation. Its new-observation rate would still be limited by full inference latency, including vision and decoder work, which is why the mobile manipulator gives renewal to a separate chunk policy instead.

Retained context adds to the pressure the notebook began to measure. Counting language and proprioception tokens along with the visual patches raises each step to $S \approx 800$ tokens, so at 0.52 MB of FP16 key-value state per token each observation step adds 419.4 MB of KV cache. Retaining an unpruned 5 s historical context window at 10 Hz yields 21.0 GB of KV cache. Combined with the intent model's 14 GB of weights, this example claims 35.0 GB, about half of the application processor's 68.7 GB before runtime overhead, memory that perception, mapping, and the chunk policy also need. Even where memory capacity permits, reading a large cache is further traffic on that same bus (section 13.3). Embodied architectures must therefore avoid unpruned autoregressive visual histories, relying instead on spatial pooling, cross-attention feature projections, or fixed-horizon action chunking.

At the pick station, the mobile manipulator's arm takes the mug from a conveyor moving at 0.20 m/s, and each chunk must be executed without letting the mug drift out of reach. Untracked, the belt's steady motion would use up the margin $r_{\text{tol}} - e_0$ between the grasp tolerance and the handle estimate's initial error e_0 within the freshness deadline $e_{\text{max}}/v_{\text{max}}$ of section 2.2, with $e_{\text{max}} = r_{\text{tol}} - e_0$. A tracker that follows the mug removes that steady motion, but residual slip can still accelerate the target at 0.50 m/s^2 , so a segment executed open-loop for a time T drifts by $\frac{1}{2}a_{\text{slip}}T^2$. Added to the 3 mm initial error e_0 , that drift reaches the 15 mm grasp tolerance after the tracked evidence horizon $\tau_{\text{ev}} = \sqrt{2(r_{\text{tol}} - e_0)/a_{\text{slip}}} \approx 219 \text{ ms}$. The chunk policy replans every 50 ms and needs up to 40 ms to do so, so while renewals arrive on time the arm executes no more than 90 ms of motion from one observation, which drifts only 2.0 mm and sits well inside the tracked evidence horizon.

That open-loop span also sets the minimum supply, because a chunk shorter than 5 setpoints would leave the arm waiting for its successor. The machine's chunk carries 16 setpoints at 20 ms spacing, 320 ms of motion, longer than the tracked evidence horizon; executed open-loop to its end, it would drift 25.6 mm, past the tolerance. The setpoints beyond the next replanning instant are therefore a supply buffer, consumed only when renewals stop, and the lease derived in section 4.3 ends execution long before that buffer runs out. Overlapping chunks and temporal ensembling then blend the executed motion across each renewal.

These are the proposer limits Part I hands forward: one chunk covers 320 ms of motion, a new chunk takes up to 40 ms, and the evidence behind it stays within tolerance for about 219 ms. They bound when a proposal arrives and how long it remains true, and each enters the handoff record of chapter 4 as a limit record in the form of section 2.7. None of them says whether the proposal was right when it was made.



Checkpoint 3.1: Edge SoC deliberation limits: Memory walls, KV cache, and dynamic

drift

Before analyzing runtime failure modes, verify your understanding of edge deliberation limits across memory and time:

Downstream: Bounded action chunk proposals are filtered against forward-invariant barriers in section 12.7.

- **DRAM streaming bounds:** Under what conditions does $t_{\text{stream}} = M_{\text{weights}}/B_{\text{mem}}$ bound full-model cadence, and why does action chunking reduce transfer cost per supplied waypoint without multiplying fresh-policy cadence?
- **KV cache memory expansion:** Can you describe how multi-camera tokenization drives linear KV storage growth with sequence length (while full attention compute can grow quadratically) across unpruned observation histories, and why reading that cache competes with real-time sensor transfers on the same bus?
- **Chunk horizon sizing and dynamic drift:** Can you explain why a chunk must cover the replanning period plus its own inference latency to keep the arm supplied, why the span executed from one observation must stay inside the evidence horizon set by open-loop drift ($\Delta x \approx \frac{1}{2} a_{\text{slip}} t^2$), and why the remaining setpoints are a buffer rather than a plan?

3.5 Cognitive Vulnerabilities

A chunk the mobile manipulator’s policy proposes for the mug can arrive inside its 40 ms budget, with every setpoint finite and within joint limits, and still close the gripper on the rim. Its timing is sound and its content is not, and the failure modes of learned models live in that gap. Learned foundation models extend task coverage but make a general proof of integrated closed-loop behavior difficult; specified components and bounded operating regions can still be analyzed.⁵

Without a verified input domain and sensitivity bound, small sensing changes to a learned proposer can yield unexpectedly large changes in proposed motion. Benchmark success does not establish closed-loop stability, timing bounds, or safety for unseen conditions; those properties require separate evidence for the model, runtime, and physical plant.⁶

A learned proposer’s failure modes also differ from traditional software defects. A classical program fails by throwing an exception, hanging in an infinite loop, or crashing. A neural network fails silently instead, as the mug chunk did, with output that passes every format check. An independent permission path must reject a detected violation before those values can become actuator commands. Systems architects must design around four primary cognitive vulnerabilities.

Semantic hallucinations

A shipping carton behind the conveyor carries a printed photograph of a mug, and the mobile manipulator’s policy proposes a grasp on the printed handle. The trajectory tensor is finite and within joint limits, yet the object it targets is not there. Its plausibility cannot substitute for an independent geometric check, which would find a flat surface where the policy saw a handle.

The Softmax illusion and out-of-distribution overconfidence

Deployment interfaces often confuse mathematical normalization with physical probability. In deep neural networks, output logits are passed through a softmax function,⁷ whose outputs sum to one whatever the input, so embodied machines encountering out-of-distribution (OOD) observations (novel optical textures, deep shadows, or lens water droplets) can assign confidence scores exceeding 0.95 to erroneous classifications.

Evaluating the true relationship between reported model confidence and empirical physical success requires measuring **Expected Calibration Error (ECE)**, the gap between how sure the model thinks it is and how frequently the physical task actually succeeds.⁸

The regimes in table 3.1 show how high reported confidence can coexist with low task success under shift. The single-row differences are confidence–success gaps, not ECE; computing ECE requires bins over a specified evaluation set. Raw scores cannot grant execution permission without validation and independent physical checks.

⁵ | **Connectionist and Symbolic Duality:** Physical AI architectures reconcile statistical connectionism with deterministic symbolic computation: high-capacity neural networks synthesize open-world behavioral proposals, while formal symbolic logic and invariant monitors verify kinematic validity before setpoints are granted execution authority.

⁶ | **Lipschitz Continuity in Neural Policies:** Although neural networks with smooth activations are continuous on compact domains, a bound $L = \sup_{x_1 \neq x_2} \frac{\|\pi(x_1) - \pi(x_2)\|}{\|x_1 - x_2\|}$ must be established over a specified operating domain. If a valid bound exists, sensor error $\|\delta x\| \leq \epsilon$ implies $\|\delta a\| \leq L\epsilon$; an unverified L supplies no usable action-error guarantee. Mathematical formulations appear in Appendix A.

⁷ | **Softmax Normalization and Epistemic Uncertainty:** The softmax operator $\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ enforces normalization across the categorical simplex Δ^K by algebraic construction rather than Bayesian epistemic confidence. High output confidence reflects only relative logit separation, meaning models assign near-unity probabilities even when evaluating inputs completely outside their training distribution. Derivations appear in Appendix A.

⁸ | **Expected Calibration Error Formulation:** Partition predicted confidence into bins. In each bin, compare observed success frequency with mean reported confidence; average the absolute gaps, weighted by the bin’s share of samples. Elevated ECE shows that reported confidence is a poor basis for granting physical permission. The equation and bin definitions appear in section A.2.3.

Table 3.1: Illustrative Confidence–Success gaps under distribution shift: Confidence and task-success rates for four operating regimes, with the supervisory response each gap calls for.

Distribution Shift Regime	Physical & Environmental Trigger	Conf. vs Success (conf vs success)	Confidence–Success Gap (conf – acc)	Downstream Supervisory Action
In-Distribution (Nominal Baseline)	Clean optics, structured laboratory lighting, nominal workpiece textures	conf = 0.91, acc = 0.89	+0.02	Apply independent limits before nominal execution
Near-OOD Shift (Covariate Shift)	Specular reflections, lens water droplets, dynamic shadow transitions	conf = 0.89, acc = 0.52	+0.37	Selective abstention triggered: clamp velocity, initiate exploratory tactile probe
Far-OOD Shift (Semantic Shift)	Uncataloged deformable obstacles, human limb intrusion into workspace	conf = 0.78, acc = 0.08	+0.70	Reject proposal; use feasible controlled response
Sensor Degradation (Hardware Drift)	Photodetector thermal noise (> 75°C), focal blur from vibration	conf = 0.84, acc = 0.31	+0.53	Trigger operational downgrade; restrict kinematics to certified limits

Perceptual chattering and covariate shift

Ambiguous images can make successive object classifications change rapidly. If a motion planner treats each new label as a fresh physical state, it may discard useful motion history, as each reclassification did in the Tempe collision (section 1.6). In physical manipulation, perceptual chattering injects high-frequency direction reversals into trajectory planners, and the commanded reversals demand torque steps that the drivetrain must absorb, oscillations that damage gearboxes. Temporal smoothing can help motion continuity, but it cannot repair a false obstacle estimate; independent detection and a feasible response remain necessary.

Temporal expiration of intent

A proposal stays valid only as long as the evidence behind it, a span the conveyor-pick example computes as the evidence horizon τ_{ev} . If the mobile manipulator’s intent model needs 160 ms to deliberate, the people and carts in the aisle, the arm itself, and the mug on the conveyor have already moved by the time the plan is ready. Delayed downstream communication or execution leads to acting on an expired intent lease (section 10.7), commanding the physical body based on an obsolete mental representation. A proposer that stalls outright leaves the plant moving with no plan at all, so recovery must live in a local controller below it, a requirement the 2015 DARPA Robotics Challenge Finals imposed on legged entrants by degrading communications deliberately (Krotkov et al. 2017).

The four vulnerabilities, with the memory wall of section 3.4, give three independent reasons against granting a learned proposer actuator authority. The memory wall means the most capable model cannot renew a proposal as fast as the machine consumes one. Hallucination, overconfidence, and chattering mean that a proposal can be wrong or unstable while its reported confidence stays high, so confidence cannot certify it. Expiry means that even a correct proposal stops being correct once the world has moved. Table 3.2 sets the learned proposer’s limits beside those of the classical methods the chapter began with and shows which side of the proposal boundary each capability occupies.

Table 3.2: Classical analytical methods vs. learned foundation models: Trade-offs across generalizability, verifiability, pacing, and fallback, with the synthesis that partitions proposal capability from execution authority.

Systems Dimension	Classical Analytical Methods	Learned Foundation Models	Physical AI Architectural Synthesis
Foundational Principle	Analytical differential equations, geometry, CAD models, Lyapunov stability	High-capacity neural representations fitted to empirical multimodal data	Hybrid Hierarchy: Learned intent proposals gated by the permission path
Generalizability & Perception	Model-dependent: Explicit geometry and features may lose coverage under shift	Data-dependent: Open-vocabulary grounding can improve coverage but may fail under shift	Unprivileged Deliberation: Foundation models synthesize continuous semantic goals
Mathematical Verifiability	Conditional: Selected properties can be proved within model assumptions	Limited: Empirical behavior needs separate bounds and runtime evidence	Proposal Boundary: Gate checks modeled limits using independent evidence
Temporal Pacing	Hard Real-Time: Deterministic millisecond execution loops (1 kHz)	Multi-Rate: Full inference, chunk refresh, and waypoint execution have distinct rates	Split Proposers: A small chunk policy renews while the intent model revises goals
Failure Modes & Fallback	Boolean branch crashes; failure on uncataloged objects; rule exhaustion	Semantic hallucinations; out-of-distribution overconfidence; expired intent	Conditional Fallback: Reject detected violations and execute a feasible response

3.6 Fallacies and Pitfalls

Learned capability is useful only when the machine can interpret and constrain its proposals. The following mistakes confuse a model's success, confidence, or output format with permission to act.

Fallacy: *An engineering team must choose entirely between formal classical safety proofs and learned empirical capability.*

An assembly plant compares a classical wiring-harness controller with a learned diffusion policy. The classical design exposes explicit motion constraints but struggles with cable deformation; the learned policy completes 96 percent of trial insertions without establishing worst-case force or velocity bounds. These strengths address different engineering obligations. The learned policy can propose adaptive motions while a deterministic supervisor checks them against the physical limits, using a control barrier function where its assumptions apply (Ames et al. 2019) (detailed in chapter 12). Partitioning capability from enforcement lets the team evaluate task performance and the protection mechanism separately. High empirical success does not establish the supervisor's adequacy or the safety of the integrated machine.

Pitfall: *Allowing a neural model's high confidence to bypass deterministic geometric boundaries.*

A cleanroom manipulator mistakes glare on a transparent barrier for open space and proposes reaching through it while carrying reagent vials. The proposal contains finite, in-range coordinates and arrives within its deadline. Those checks establish that the packet is usable, not that its interpretation of the scene is correct. High confidence cannot resolve the same perceptual error that produced the trajectory. Before granting permission, the permission path must compare the proposal with independently maintained geometric boundaries (such as hard-coded stay-out zones or real-time depth maps).

Fallacy: *Raw softmax probabilities or predicted confidence scores reflect true physical success rates.*

A bin-picking policy reports $\hat{p} \approx 0.90$ for a group of grasps, but only 64 of 100 physical attempts succeed. An abstention threshold based on the reported score would therefore accept more failures than its numerical value suggests. The discrepancy concerns calibration: confidence does not yet match observed success in the operating conditions (as illustrated in table 3.1). It does not, by itself, establish whether the policy ranks candidates correctly. Engineers must validate the relationship between scores and outcomes across the intended domain before using confidence to choose between execution, another observation, and operator assistance. Independent physical limits still apply to accepted grasps.

Pitfall: *Coupling high-frequency semantic classifications directly to motor setpoints without continuous filtering.*

A delivery robot observes reflective tarps over traffic cones. At every perception update, its pipeline changes the classification among obstacle, movable drape, and free path. If each change immediately resets motor setpoints, uncertainty in the scene becomes repeated requests for direction reversal. The drive motors cannot follow those requests beyond their torque slew limits, and the chassis chatters and shudders. The architecture needs continuity between changing interpretations and physical commands. Filtering or action chunking can support that continuity, but the resulting trajectory must still respect the body's motion limits when the semantic estimate changes.

Fallacy: *A numerically valid and smooth action chunk can safely be executed whenever it arrives.*

The mobile manipulator's chunk policy proposes an arm trajectory toward the mug on the conveyor. Host scheduling delays delivery, while residual belt slip has carried the mug toward the edge of the grasp tolerance. The trajectory remains smooth and numerically valid, but the observation that justified it no longer describes where the mug is. Executing it could close the gripper on the rim or knock the mug from the belt. Once the delay outlasts the chunk lease of section 4.3, the lease ends the stalled policy's authority and the arm enters its validated fallback stop instead of waiting, and when the delayed trajectory finally arrives, the observation it was computed from is too old for the permission path to admit it. Smoothness describes the proposed command; temporal validity determines whether the machine may still act on it.

3.7 Summary

Learned representations give the mobile manipulator what hand-authored geometry could not, the ability to find the handle of a mug it has never seen. The price is paid in memory traffic and in time. The largest model evaluates least often, and every setpoint either proposer supplies ages with the observation that produced it.



Key Takeaways: Learned proposers and their deliberation limits

- **Weight traffic bounds how often a model sees:** One sweep of the intent model's 14 GB of weights takes 98.0 ms on the application processor before vision or decoding begins, so its fresh-evaluation rate cannot exceed about 10.2 Hz whatever its compute peak.
- **Chunking multiplies supply, not freshness:** A 16-setpoint chunk spreads one evaluation across 320 ms of motion, but every setpoint is as old as the observation behind it. The machine therefore gives renewal to a small chunk policy at 20 Hz within 40 ms.
- **Size a chunk between starvation and drift:** The replanning period plus inference latency (90 ms) sets the minimum supply, and target drift sets the evidence horizon (219 ms). Setpoints beyond the next renewal are a buffer that the lease ends, not a plan.
- **Retained context is paid for twice:** Keeping 5 s of multi-camera history adds 21.0 GB of key-value cache, which with the weights claims about half the processor's memory, and every read of it is traffic on the bus that real-time transfers share.
- **Confidence is not calibration:** Softmax scores sum to one by construction, so reported confidence can stay high while task success collapses under distribution shift. Only calibration measured over the operating domain can justify an abstention threshold.
- **A proposal is true only for a while:** Intent and chunks expire with the evidence behind them, and a smooth, in-range, on-time proposal can still be stale or wrong.

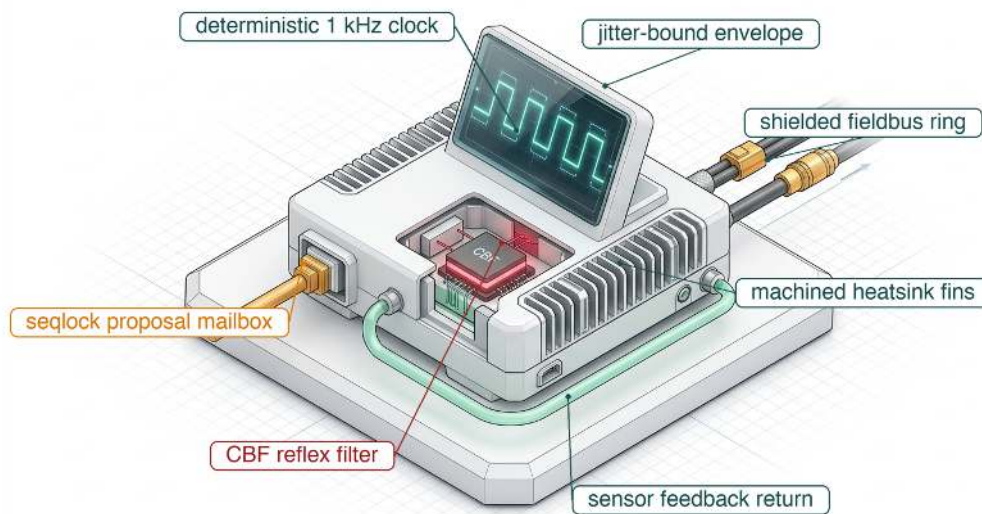
Every limit this chapter derived is a limit of supply or of truth, never of authority. None of them decides whether a proposal may reach the motors; the permission path takes them as inputs when it makes that decision.

 What's Next: From learned proposals to the permission path

How long may one proposal keep driving the machine if its successor never arrives? The mobile manipulator's chunk policy renews every 50 ms and its intent model far less often, and either can stall, arrive late, or be wrong about the mug. Chapter 4 sets how often the permission path checks each proposal and derives from the stopping budget of chapter 2 how long one may drive the actuators before the machine falls back, and it records both in the handoff record alongside the proposer limits established here.

DRAFT

The Nervous System



- 4.1 The Multi-Rate Bridge
- 4.2 Where the Permission Path Runs
- 4.3 Multi-Rate Cadences
- 4.4 Moving Commands on Time
- 4.5 Actuation Authority
- 4.6 Hardware Isolation
- 4.7 Invariant Checking
- 4.8 The Handoffs Matrix
- 4.9 Fallacies and Pitfalls
- 4.10 Summary

Purpose

What forces an independent Nervous System to stand between deliberative neural policies and physical actuator power?

A moving machine's kinetic momentum cannot pause while a neural policy completes deliberation. Direct connections between high-capacity foundation models and motor power stages expose actuators to operating system scheduling jitter, garbage-collection stalls, and memory bus contention. While software threads await DMA transfers, mechanical linkages coast under unmanaged inertia, magnetic flux collapses, and dynamic balance degrades before a software runtime can raise an exception. Software task priorities cannot preempt hardware memory saturation or mechanical velocity; physical stability requires cycle-deterministic real-time control.

The Nervous System resolves this mismatch by decoupling deliberative planning from high-frequency actuator commutation. Standing as an independent, cycle-deterministic gatekeeper, it continuously verifies observation freshness and holds exclusive write authority over inverter power stages. By translating the physical plant's stopping budget into rigid temporal leases and control barrier constraints, the Nervous System bounds how long any proposal may guide motion before expiring. If a neural policy delays, fails, or emits an unsafe trajectory, the Nervous System intervenes to arrest motion before the machine violates its safe envelope.

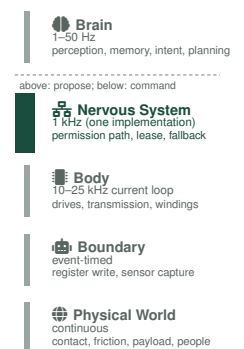


Figure 4.1.: The five-level machine stack with the Nervous System level highlighted.

Prerequisite: Silicon domain isolation physically enforces the causal boundary defined in section 1.2.



Learning Objectives

- Determine control update rates from physical dynamics and allowable response delay
- Evaluate proposal exchanges for freshness and bounded transfer time between the learned proposer and the permission path
- Distinguish measured latency percentiles from defensible worst-case bounds on real-time execution
- Evaluate hardware isolation that reserves actuator authority when learned policies or communication fail
- Size a proposal's lease between the proposer's renewal period and the pre-brake time the stopping budget leaves
- Diagnose an interface failure as an over-budget limit at a specific handoff, using like-for-like evidence
- Construct handoff-record entries that give each interface limit its conditions, evidence category, and bound kind

4.1 The Multi-Rate Bridge

THE warehouse mobile manipulator that this book follows regulates the current in its motor windings thousands of times per second, while the learned models that proposed its grasp on the conveyor mug in chapter 3 need orders of magnitude longer to digest each camera frame. If deliberative reasoning and motor regulation share a blocking execution path or an unpartitioned memory bus, any neural network delay starves physical control. The physics of moving mass and electromagnetic coils create a strict architectural requirement: *the Brain needs time to deliberate, while the Body needs continuous control to survive.*

In standard computer systems, workloads share processors, caches, and memory interconnects under general-purpose operating systems. When a neural network streams multi-gigabyte weight tensors across a shared memory bus, it saturates direct memory access crossbars,¹ and no thread priority can preempt a hardware transaction already occupying the bus. If a garbage-collection pause, page fault, or thermal throttle stalls the neural engine, an unpartitioned system stops sending setpoints to the motor inverters.

Chapter 3 established how long each learned proposer on the machine needs to turn camera frames into a proposal. An electromagnetic motor coil has an electrical time constant ($\tau_e = L/R$) measured in fractions of a millisecond, while moving linkages carry kinetic energy ($E_k = \frac{1}{2}mv^2$). Delaying or miscalculating the current command can therefore exhaust a physical margin before deliberation finishes. On the mobile manipulator, the mismatch takes two forms:

- The base (Class 1) cruises at up to its 1.5 m/s drive limit (illustrative; see the Reader Guide), so it travels 60 mm during one 40 ms chunk-policy inference and 240 mm during one 160 ms intent-model inference, and nothing brakes it in that time unless a process other than the proposer is watching.
- The arm (Class 2) carries reflected rotor inertia (N^2J) through its gearboxes; if its high-rate impedance loops are interrupted while the gripper is on a door latch or a mug, that inertia drives the contact force past its limit.

Software discipline, OS thread priorities, and runtime exception handlers cannot by themselves keep the plant safe. Safety requires an explicit, deterministic computing, memory, and communication fabric. That fabric is the Nervous System of the five-level machine (section 1.5), and it holds the permission path.

Every signal crossing between deliberation and physical actuation is constrained by five interface limits, which section 4.8 lays out handoff by handoff. Reconciling them requires an architecture that delivers microsecond cycle determinism (1 kHz to 20 kHz) to preserve controller phase margin, maintains lock-free asynchronous decoupling to prevent slow neural inference from blocking inner control loops, isolates actuator writes from host crashes, restricts actuation privilege to the permission path, and requests a validated local response when a proposal's chunk lease expires.

¹ | **Interconnect Crossbar**

Contention: When an accelerator streams multi-gigabyte foundation model weights across shared AXI or PCIe crossbars, dense burst transactions saturate dynamic RAM banks and arbitration queues. Co-located real-time processor cores attempting to read motor encoder registers face unbudgeted hardware wait states that software priorities cannot preempt. Maintaining microsecond-scale control stability therefore requires dedicated physical memory channels or isolated on-chip SRAM for the real-time domain.

The first decision is where the two workloads execute. The separation in section 4.2 keeps inference off the timed actuator path. The design in section 4.3 then budgets their exchange and sizes the lease that ends a stale proposal's authority; section 4.4 bounds the time a command takes to reach the drives. The remaining sections locate actuator authority, fault isolation, and invariant checks, and close Part I with the handoff record and one budget that couples this chapter's lease to the limits of chapter 2 and chapter 3. Each mechanism must leave the physical plant a measured, feasible response when the proposer pauses.

4.2 Where the Permission Path Runs

Two regimes of computation meet in a physical AI system, and they have almost nothing in common. The first regime is deliberation, where learned models evaluate sensory context, infer environmental state, and propose future action plans. The second regime is reactive enforcement, where deterministic control logic samples joints, validates kinematic limits, interpolates motion commands, and regulates electrical currents to the motors.

Deliberative work features deep computational graphs, large memory footprints, and variable execution times. A vision-language-action model processing multi-camera video streams, tactile histories, and natural language prompts requires billions of multiply-accumulate operations across hundreds of neural network layers. Storing the parameters and intermediate activation tensors demands gigabytes of high-bandwidth memory. To achieve acceptable arithmetic throughput, deliberative computation executes on massively parallel accelerators such as graphics processing units (GPUs) or dedicated neural processing units (NPUs). These processors maximize aggregate throughput across wide vector registers and batched memory pipelines at the direct expense of deterministic latency. Memory access stalls, dynamic tensor kernel selection, bus transfers, and runtime memory management scatter inference execution times across a wide distribution. If a deliberative model experiences a temporary delay, the failure is soft, because the planning pipeline produces a delayed proposal or a stale trajectory that can be held or discarded while the permission path maintains stable equilibrium.

Reactive work operates under the inverse constraints. An inner control loop computing inverse dynamics, joint limit monitoring, or field-oriented motor control requires a few thousand arithmetic operations per update. Its memory footprint is static, measured in kilobytes rather than gigabytes, with every variable allocated at compile time in tightly-coupled on-chip static memory. Reactive routines execute on cycle-counted microcontrollers or field-programmable gate arrays (FPGAs) designed for bounded execution rather than maximum parallel throughput. The deadlines governing this work are hard real-time constraints dictated by the mechanical inertia and electrical inductance of the physical system, typically running at rates between 1 kHz and 10 kHz on update budgets of 100 μ s to 1000 μ s. If a reactive loop misses its execution deadline, the failure is hard, because lost current regulation induces severe torque ripple, actuator saturation, or mechanical collision.

Learned inference runtimes depend on memory access patterns, cache residency, dynamic tensor shapes, and hardware prefetching, making execution times stochastic. An inference pipeline can meet its average deadline while still missing individual deadlines in its latency tail. At tens of requests per second, even a sub-percent exceedance probability yields many misses an hour, each an interval in which the motor actuators receive no updated command unless an independent, deterministic process bridges the gap.²

General-purpose operating systems and real-time kernel patches (PREEMPT_RT) cannot reconcile these two execution profiles on a shared processor. Although real-time extensions allow static task priorities and preemptible kernels, they cannot eliminate shared microarchitectural resources. When a thread executing a learned model streams gigabytes of weight matrices across the memory bus, it evicts cache lines belonging to high-priority control tasks. If the deliberative runtime triggers a **translation lookaside buffer (TLB) shutdown**,³ enters a driver-level spinlock, or saturates the memory bus crossbar with direct memory access (DMA) requests, the hardware pipeline stalls.⁴ A high-priority real-time interrupt can preempt software execution, but it cannot preempt an in-flight hardware burst transaction already occupying the shared memory crossbar. Furthermore, dynamic voltage and frequency scaling (DVFS) governors adjust core clock frequencies based on thermal dissipation and processor utilization, altering instruction cycle times dynamically (section 2.9 prices the travel this costs). The resulting jitter ranges from tens of microseconds to several milliseconds,

² | **Inference Latency Distribution:** Log-normal and shifted Weibull models are common descriptions of measured inference tails. The median and P_{99} alone do not fix the exceedance probability at a given deadline, which must be measured or modeled separately. For the formal derivation of latency exceedance probabilities in real-time inference, see Appendix A.

³ | **TLB Shutdown:** When an operating system modifies page tables, it broadcasts inter-processor interrupts (IPIs) to invalidate stale Translation Lookaside Buffer entries across all active cores. Receiving cores must suspend thread execution and flush affected entries, introducing unbudgeted multi-microsecond stalls. In shared-silicon architectures, these unpredictable synchronization pauses destroy the cycle determinism demanded by inner motor regulation loops.

⁴ | **DMA Cache Coherence:** Direct memory access transfers bypass CPU cache hierarchies, requiring explicit cache line invalidation or write-back operations to maintain memory consistency between peripherals and cores. Without hardware-enforced snoop filtering or explicit barriers, processors read stale sensor frames or overwrite pending actuator commands. These maintenance operations inject variable latency penalties into high-rate control routines unless static, unbuffered SRAM regions are dedicated to peripheral transfers.

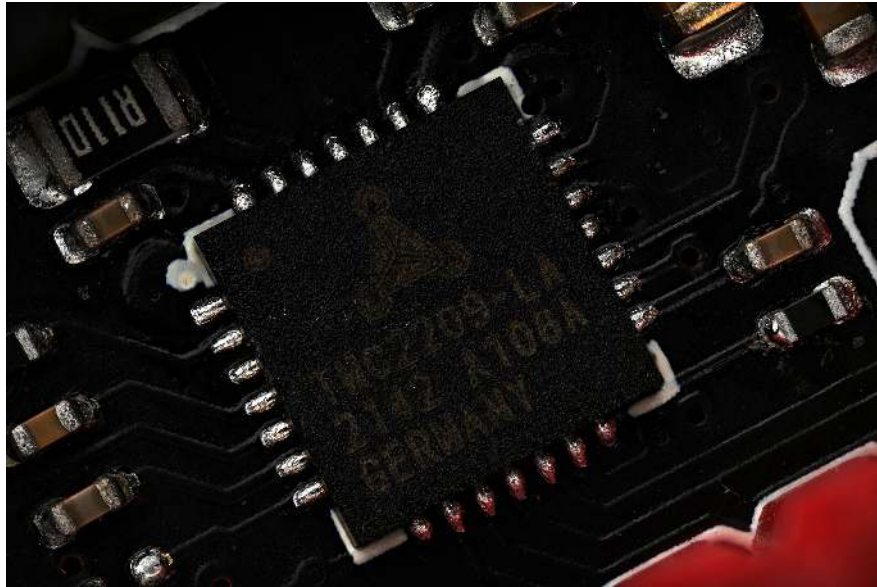


Figure 4.2: Motor drive controller silicon: Macro view of an embedded motor drive integrated circuit featuring precision current-sense shunt resistors (R110), multi-layer power routing, and direct gate-drive outputs. Dedicated real-time control silicon executes cycle-deterministic current commutation loops without operating system schedulers, cache hierarchies, or bus contention. (Credit: Wikimedia Commons, CC BY-SA 4.0).

easily exceeding the entire time budget of a 1 kHz control loop. Section 13.3 measures how far each of these channels reaches on a shared die.

The interface hardware on each side makes the separation concrete. A host processor running an accelerator communicates across high-latency bus topologies such as PCIe or system fabric crossbars, where DMA transfers require ring-buffer synchronization, host interrupt service routines, and cache coherency handshakes that consume $10\mu\text{s}$ to $100\mu\text{s}$ per transaction. In contrast, an embedded microcontroller or dedicated real-time motor controller IC (figure 4.2) interfaces directly with sensor transceivers and power stages through memory-mapped registers, single-cycle input and output pins, and tightly-coupled static memory with zero wait states. Reading an absolute joint encoder over a serial peripheral interface (SPI) or updating a field-oriented pulse-width modulation counter takes less than $1\mu\text{s}$ of bounded execution time,⁵ isolated from system bus contention.

In the two-processor implementation (figure 4.3), deliberative planning and reactive enforcement do not share a processor core, a cache hierarchy, or a common memory bus. The slow, stochastic path runs on parallel hardware (Linux host application processors and tensor accelerators) optimized for raw throughput across wide data streams, while the fast, deterministic path runs on isolated hardware (bare-metal microcontrollers or FPGA cores) optimized for worst-case execution time and cycle-counted predictability. On the mobile manipulator, the permission path runs on the safety microcontroller (MCU). Separate chips are one realization; a single die can also host the permission path if its response bound and fault containment are shown to hold (section 13.2).

Safety in physical AI is an architectural partition in silicon, not an operating system scheduling priority. The high-throughput deliberative engine acts strictly as an untrusted client, while dedicated real-time silicon acts as the hardware gatekeeper. The partition settles where each workload runs, but it leaves the two sides on separate clocks at rates that differ by orders of magnitude, and every proposal must still cross between them without either side waiting for the other.

4.3 Multi-Rate Cadences

The mobile manipulator's drives regulate winding current every $50\mu\text{s}$, while its chunk policy needs 40 ms at P99 to turn camera frames into one action chunk. No single execution frequency serves both the body's physical dynamics and the cost of perception and inference, so a physical AI machine runs several coupled cadences (figure 4.4), each set by a different physical process. The mobile

⁵ | **Dead-Time Delay:**

Dedicated pulse-width modulation timers (TI ePWM, STM32 TIM1) inject hardware-enforced dead-time delays (100–500 ns) between complementary high-side and low-side gate signals (TIMx_BDTR, DBRED). This delay ensures that the conducting power switch fully turns off before its opposing switch conducts, preventing shoot-through short circuits across high-voltage DC rails. Because dead time slightly distorts the applied phase voltage, motor control firmware incorporates online volt-second compensation to maintain smooth torque generation.

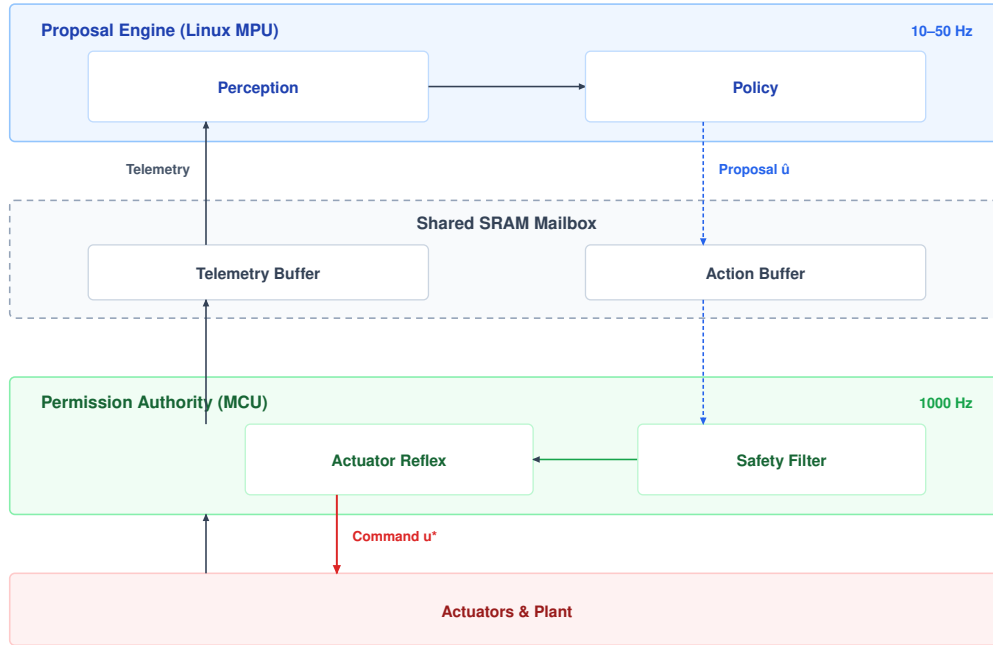


Figure 4.3: The two-processor implementation: An application processor publishes learned proposals to a shared mailbox. An independently scheduled microcontroller, which runs the permission path, checks freshness and physical limits before commanding motor current and executes a defined fallback on stale or unsafe proposals.

manipulator runs four cadences. The current loop in the drives runs at 20 kHz; the permission tick on the safety microcontroller runs at 1 kHz and carries the joint loop and the evaluation of the admitted spline; the chunk policy runs at 20 Hz, fed by perception at its sensors' rates; and the intent model revises goals at 5 Hz. These last two cadences lie within the Brain's 1–50 Hz inference rates (section 3.3).

The inner loop rate is governed by the physical time constants of the actuator and the structural resonance of the machine. For an actuator with electrical time constant $\tau_e = L/R$, stator winding current responds to applied voltage over that interval. For an illustrative winding with $L/R = 0.5$ ms, choosing ten current samples per electrical time constant gives $T_s = 50 \mu\text{s}$, or 20 kHz. For the winding L/R derivation, see Appendix C. At the joint level, the mechanical time constant and structural stiffness dictate the required closed-loop bandwidth, and a discrete sample-and-hold loop introduces phase delay proportional to computation latency and sampling interval.⁶ At an assumed joint-loop crossover of 20 Hz, a 1 ms sample-and-hold interval contributes $\Delta\phi = -(2\pi \cdot 20)(0.001)/2 \approx -3.6^\circ$ of phase lag. If the allowed sampling erosion is 5° , the interval must be below about 1.39 ms; 1 kHz is one feasible choice before computation and transport delays are added.

Slow control loops erode the phase margin of the feedback controller. Accumulated phase lag causes corrective torques to arrive out of phase with physical link motion, converting negative feedback stabilization into positive feedback amplification. At the same 20 Hz crossover, a 500 Hz loop adds about 7.2° of sample-and-hold lag, above the 5° allowance.

Perception, which feeds the chunk policy, executes at an intermediate rate dictated by photon collection physics, exposure windows, and target velocity relative to the sensor field of view. In an illustrative 300 lux scene with a 15 ms exposure, the base at its 1.5 m/s drive limit travels $\Delta x = vT_{\text{exp}} = 22.5$ mm during the exposure; image-plane blur additionally depends on depth, optics, and camera motion. Exposure signal-to-noise, this displacement, and processing time jointly bound a useful perception rate for the task.

The chunk-policy rate is chosen within the renewal latency that section 3.4 measures on the application processor. On the mobile manipulator, a 20 Hz renewal and a 16-waypoint chunk, the

Prerequisite: Microsecond interrupt latency requirements build upon the sensor freshness bounds introduced in section 2.1.

⁶ | **Zero-Order Hold Phase Lag:** Holding a discrete control command constant across sampling interval T_s introduces an effective delay of $T_s/2$, producing a linear phase reduction $\Delta\phi = -\omega T_s/2$. In high-bandwidth robotic actuators, this phase erosion directly reduces open-loop phase margin, driving lightly damped mechanical modes into resonance. For the formal frequency-domain derivation of discrete sampling phase erosion, see Appendix B.

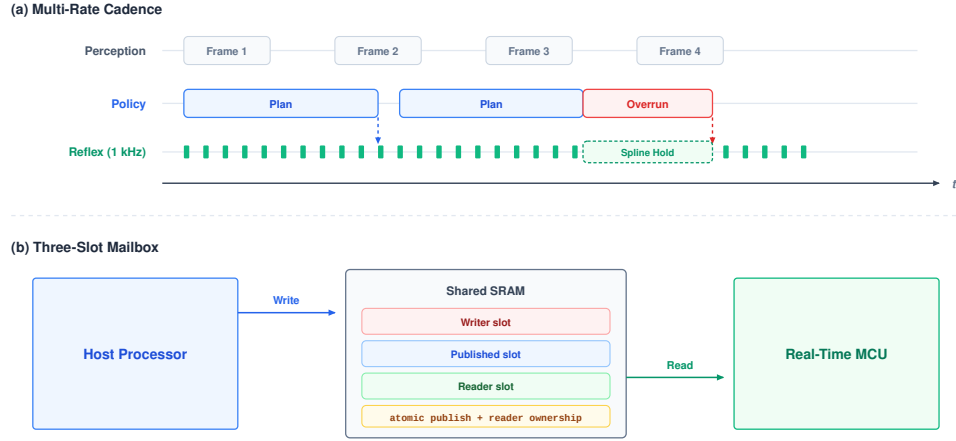


Figure 4.4: Multi-rate cadence and buffer exchange: (a) Representative perception, policy, and permission-tick rates; the permission tick retains the last valid spline during a policy overrun only until its freshness limit. (b) A conceptual three-slot mailbox; the text describes a simpler single-slot seqlock baseline. A production three-slot variant additionally needs atomic publication and reader-slot ownership (synthesized from Slotine & Li (1991) and ARM Architecture Reference Manual).

chunk horizon H of action-chunking policies (Zhao et al. 2023), bridge observation and actuation rates.

Because these cadences execute on isolated hardware domains at different frequencies, data exchange across rate boundaries must be asynchronous, non-blocking, and zero-copy (figure 4.4). If a 1 kHz control loop waits on a mutex lock held by a 20 Hz neural network runtime, a single scheduling delay in the operating system immediately causes the real-time controller to miss its deadline.

A baseline nonblocking exchange uses a **single-producer single-consumer (SPSC) seqlock mailbox** in shared static memory.

Definition 4.1: Lock-free seqlock buffer

Lock-free seqlock buffer is a nonblocking shared-memory synchronization mechanism that couples an atomic sequence counter with explicit hardware memory barriers, allowing an unprivileged writer to publish multi-word trajectory payloads without blocking and enabling concurrent readers to verify payload integrity via counter parity checks:

$$c_{\text{start}} \equiv c_{\text{end}} \pmod{2} \quad \text{and} \quad c_{\text{start}} = c_{\text{end}}$$

where an odd counter value signals active writer mutation, and an identical even counter pair confirms an uncorrupted, race-free payload read.

1. **Significance:** Decouples high-rate real-time control loops (1 kHz) from variable-latency neural policy processes (10–50 Hz) without mutual exclusion locks, preventing operating system priority inversion from causing hard real-time deadline misses.
2. **Distinction:** Unlike mutexes or semaphores that suspend thread execution during contention, a seqlock reader never blocks; if a collision or torn read is detected, the reader retains its previously validated trajectory spline within a bounded execution budget ($O(1)$ time).
3. **Common pitfall:** Omitting bidirectional hardware memory barriers (DMB OSH / DMB OSHLD on ARM) around payload accesses, which allows out-of-order processor pipelines to speculate or hoist memory loads ahead of the initial sequence counter validation.

The writer, the untrusted application processor, makes the counter odd, copies the complete proposal into the shared slot, makes the payload visible across the interconnect, and then makes the counter even with release ordering. The reader, the permission path, makes one bounded copy attempt per tick. It rejects the attempt if the counter was odd or changed during the copy, and otherwise promotes the copy to its active spline only after the checksum and freshness checks pass, so a rejected read leaves the active spline unchanged and the reader never waits for the writer. Because the writer is a preemptible user-space process rather than a kernel thread, a writer stopped mid-write can leave the counter odd indefinitely; the reader then keeps its last validated spline only until that spline's lease expires. A **three-slot publication variant**, sketched in figure 4.4, keeps a complete published slot available while the writer stages another, at the cost of atomic slot publication and reader-slot ownership. The complete single-slot protocol, including the memory barriers a weakly ordered processor requires, appears in section C.11.

The mailbox's own collisions are rare and cheap. A read collides only when it overlaps a write, and a write occupies the slot for microseconds once per chunk period. If the writer's and reader's clocks drift against each other, only a small fraction of reads collide; if they lock at an unfavorable phase, one read may collide with every write. Either way, a write far shorter than one reader tick cannot overlap two adjacent reads, so a detected odd or changed counter makes the reader discard one attempt and keep its last validated spline. A collision therefore costs one tick of freshness.

How long a spline's lease may run is a question about the plant, not the mailbox. Under the irreversibility law (principle 1), each millisecond a stalled proposal keeps its authority is travel that no later check can recall, so the lease must be paid for out of the stopping budget.



Napkin Math 4.1: Lease sizing from the stopping budget

Problem: The warehouse mobile manipulator cruises down an aisle at its 1.5 m/s drive limit toward a person who has stepped out from a rack end and stopped 1.10 m ahead. Its chunk policy renews one last proposal and then stalls. How much time remains before braking must begin, and how much of it does the chunk lease spend?

1. *What section 2.3 left:*

- Braking at the credible deceleration $a_{\text{brake}} = 2 \text{ m/s}^2$ takes $v^2/2a_{\text{brake}} = 562.5 \text{ mm}$, the braking term of equation 2.1; localization and protective clearance, $\delta_{\text{loc}} + \delta_{\text{margin}}$, take 150 mm; brake onset, $T_{\text{act}} = 20 \text{ ms}$, takes 30 mm.
- The stopping budget therefore stands at 742.5 mm of the 1.10 m clear distance.

2. *The pre-brake ceiling:*

- Everything that happens before braking force begins must fit in the time that the distance left by the braking term and the overheads allows at speed v :

$$\tau_{\text{delay}} \leq \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - v^2/2a_{\text{brake}}}{v}.$$

Here the right-hand side is 258.3 ms, and brake onset has already spent 20 ms of it.

3. *The lease path (row 4 of the stopping budget):*

- The stalled proposal keeps its authority for the chosen chunk lease, $T_{\text{lease}} = 60 \text{ ms}$. The permission path notices the expiry on its next tick, $T_{\text{tick}} = 1 \text{ ms}$, and the stop command reaches the drives one bus cycle later, $T_{\text{bus}} = 1 \text{ ms}$. Each of the three enters τ_{delay} . The joint loop's phase margin sets the tick; at 1.5 mm per millisecond the stopping budget can afford it.
- The row adds $v(T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}})$, the distance covered in 62 ms at 1.5 m/s, which is 93 mm. The running total rises from 742.5 mm to 835.5 mm, leaving 264.5 mm.
- From the last valid renewal to brake onset, the machine spends 82 ms, and 176.3 ms of the ceiling remains for the terms that later chapters add.

- The lease has a floor as well as a ceiling. The chunk policy publishes a fresh chunk every 50 ms, and with a 5 ms allowance for renewal jitter any lease shorter than 55 ms would let a healthy proposer's own chunks expire. The chosen 60 ms sits just above that floor.

Systems insight: A chunk lease is a term in the stopping budget, not a timeout chosen for convenience. Every millisecond added to it costs 1.5 mm of clear distance at the drive limit, distance that the observation age and the planner's stop profile will need.

The worst case is a proposer that renews its lease on the frame before the person appears and then stalls while its heartbeat stays live. Nothing reads the frame showing the person, and the brake begins only when the lease expires, plus one tick, one bus cycle, and brake onset; the frame's own age, which chapter 8 measures, adds to that. The permission path can bound this case without seeing the person, because the enforcer, the permission path's checking loop, reads the encoders, the IMU, and the evidence time stamped in the proposal's header rather than a raw observation.

Three timers guard the exchange, and each answers a different question. The **chunk lease** bounds how long an admitted proposal may drive the actuators; only a fresh proposal renews it, and its length comes from the stopping budget, here the chosen 60 ms between the renewal floor and the pre-brake ceiling of 4.1. The **heartbeat** reports that the application processor is alive. It runs at 100 Hz with a 30 ms timeout, faster than the proposer, and its silence revokes the proposal without waiting for the lease. It never renews the chunk lease, because a live process can still publish nothing admissible, so the lease is what bounds a host that stays alive but stops publishing admissible chunks. The **self-watchdog** certifies that the permission path itself is completing its checks; the permission loop feeds it only after a completed tick, and it trips after 5 ms without a feed, so a stalled enforcer trips it even while its timer interrupts keep firing (section 4.6).

The lease is enforced through the **proposal header**, the part of every proposal that lets the permission path judge it without trusting its author. By meaning, the header carries a sequence number, the evidence time of the observation the proposal was computed from together with that time's conversion error, the issue time, the expiry, a reference to the parent record the proposal was derived from, the coordinate frame, the payload kind, and a checksum. The minimal payload behind it, a chunk, carries the number of waypoints and, for each waypoint, joint positions and joint velocities. It carries no torques, because a learned proposal specifies kinematic setpoints (section 3.3) and torque is what the tracker on the permission side computes from them. The proposal's evidence deadline is its evidence time plus the largest evidence age the permission path accepts, which chapter 8 budgets. A proposal drives the actuators only until the earlier of its expiry and its evidence deadline, whether or not any cancellation arrives.

Byte layout: The proposal header's and chunk payload's fields, types, and offsets are given in section C.14.1.

Every record that crosses the proposal boundary carries its times on the safety microcontroller's clock. A producer on another clock converts with a versioned mapping and carries that mapping's error bound, which the consumer adds to every age it checks. Calendar time appears only in logs and in a release verdict's expiry, never in an age test.

A record crossing the boundary is protected against three different failures by three different fields. A checksum over header and payload detects accidental corruption; it proves nothing about who wrote the bytes. A digest of the record it was derived from establishes lineage; it is not an authenticator either. A keyed tag appears only where the sender's identity itself confers authority: on operator commands that arrive over a network (section 14.5), on the forensic log (section 14.7), and on offline evidence and release artifacts (section 15.7, section 16.8). Proposals from the learned model carry no keyed tag, because the permission path admits by content, not origin, and a key held by a compromised proposer would authenticate nothing.

The reader's possible failures fall into four classes (table 4.1), and none of them lets a failed producer make the permission path wait. The classes keep the exchange from blocking; they do not by themselves contain the plant, because the fallback each one selects still needs bounded timing and a validated stop.

Table 4.1: Boundary contract failure classes and deterministic fallback matrix: Exchange faults across the proposal boundary by class, with the fallback each selects. Fallback paths are selected without waiting on host recovery.

Failure Class	Reader Detection Condition	Physical Systems Fallback Action	Bounded Response
Torn / In-Flight	Odd c_{start} or changed c_{end}	Reject current snapshot; use admissible prior chunk or fallback	One bounded attempt; measure duration
Malformed	Checksum or configured command bound fails	Reject proposal; use independently checked fallback	Measure check and response
Stale	Past proposal expiry or evidence deadline (local clock)	End tracking authority; enter the state-matched fallback	Deadline from plant budget
Missing	Sequence gap	Record audit gap; evaluate current record on its own merits	One tick for flag

Rate mismatches introduce three failure modes: phase lag accumulation, aliasing, and buffer starvation. In a discrete-to-continuous conversion, a Zero-Order Hold (ZOH) maintains setpoints constant across the planning period T_{plan} , delivering piecewise-constant position steps rather than a smooth trajectory. This step quantization introduces an intrinsic half-sample phase lag $\Delta\phi = -\frac{\omega T_{\text{plan}}}{2}$. Directly feeding raw planner waypoints to inner motor loops at 10 Hz incurs 50 ms of uncompensated phase delay, degrading tracking margins and exciting unmodeled mechanical resonances. Across the latched waypoint seam, the real-time enforcer can use a quintic bridge that matches position, velocity, and acceleration at both ends. Those six endpoint conditions, not the tick rate, give the seam C^2 continuity, and evaluating the admitted bridge at 1 kHz supplies the corresponding velocity and feedforward acceleration setpoints.

Aliasing occurs when high-frequency sensor noise folds into low-rate estimators, which the system prevents through analog anti-aliasing filters and digital decimation⁷ within the high-rate microcontroller domain. Classical stability guarantees (Slotine and Li 1991) assume inner-loop execution jitter remains strictly bounded and setpoints arrive before the trajectory buffer drains. The permission path enforces these conditions through cycle-deterministic microcontroller interrupts and buffer depth tracking. If the planner stalls and the buffer reaches its final valid waypoint, the permission path rejects stale proposals and requests its validated local deceleration or hold.

An overrun means something different at each of the four cadences. The current loop runs in the drives on a jitter budget of microseconds, and a detected fault there trips torque in hardware. At the 1 ms permission tick, an overrun is a fault that selects the validated local stop, and a spline stays in force only within its lease and stopping margin. The chunk policy's 40 ms inference time on the mobile manipulator is a P99, a statistical figure rather than a worst-case bound, and this cadence holds the 60 ms chunk lease only because it renews every 50 ms, inside the lease; an overrun that delays a renewal past the lease lets it lapse. When the intent model overruns, its targets expire, and the machine enters its state-matched fallback, holding or stopping from its current plant state.

Every timed cadence is a deadline, and the lease budget assumes that the tick, the bus cycle, and brake onset meet theirs on every cycle. That assumption needs a worst-case bound on every step between the enforcer's check and the drives, and neither a faster processor nor a longer latency measurement supplies one.

4.4 Moving Commands on Time

Observed latency distributions do not establish worst-case timing bounds. In statistical performance analysis, an engineer measures ten million executions of a task, notes that the ninety-ninth percentile (P_{99}) latency is 135 μs and the maximum observed latency is 150 μs , and infers that the computation fits comfortably inside a 1.0 ms control deadline. The inference does not follow. An empirical distribution records what happened during finite testing, whereas a hard real-time bound must hold in every hardware state the analysis admits. When an execution platform includes out-of-order execution pipelines, shared cache hierarchies, speculative prefetchers, and concurrent direct memory access traffic, a finite latency sample cannot establish an upper bound. The missing tail events do not vanish; they emerge under rare microarchitectural collisions, such as concurrent DMA burst

Prerequisite: High-rate joint interpolation of action chunks builds on the proposal generator in section 3.4.

⁷ | **Phase Lag in Moving Average Smoothing:** Averaging noisy actuator feedback using an N -tap finite impulse response (FIR) filter introduces a group delay of $(N - 1)/2$ sample periods. In high-gain position control, this artificial transport delay consumes phase margin, turning stabilizing feedback into positive feedback that drives the actuator into self-excited limit cycles.

transfers and cache line evictions coinciding with a critical interrupt, that manifest under production stress.

Establishing a worst-case response time requires bounding every term that contributes to task response time: the base computation time, the blocking time spent waiting for shared critical sections, and the interference time from high-priority interrupts or memory bus contention.⁸ If any one of these three terms lacks an analyzed upper bound, the system has no bound on its worst-case response time, however small the mean latency appears in benchmarks.

Bounding those terms starts by sorting every source of execution variance by whether its worst case can be bounded at all. *Bounded sources* admit exact upper bounds, among them fixed-pipeline instructions, round-robin bus arbitration, zero-wait-state tightly coupled memory, and vectored interrupt dispatch on a bare-metal microcontroller. *Experimentally characterized sources*, such as last-level cache misses under line locking, write-buffer drains, DRAM refresh, and thermal throttling under active cooling, cannot be bounded from first principles but can be enveloped by stress profiling under a locked hardware configuration. *Unbounded sources* have no finite worst case. They include page faults under demand paging, preemptive thread scheduling without priority inheritance (war story 4.1), heap allocation over fragmented free lists, and unscheduled DMA traffic, and a hard real-time safety path must eliminate every one of them.

⁸ | **Rate-Monotonic Scheduling:** Liu and Layland (1973) established that static-priority rate-monotonic scheduling guarantees periodic task schedulability when total processor utilization satisfies $U \leq n(2^{1/n} - 1)$, converging to $\ln(2) \approx 0.693$ for large n . In distributed robotic architectures with arbitrary execution deadlines and shared resource blocking, systems engineers deploy exact Response Time Analysis ($R_i = C_i + B_i + I_i \leq D_i$) to verify deterministic schedulability. For the formal mathematical derivation of response time recurrences, see Appendix C.

Downstream: Deterministic fieldbus jitter bounds provide the execution baseline for heterogeneous placement in section 13.3.

⚠ War Story 4.1: The Mars Pathfinder priority inversion (1997)

Context: In July 1997, shortly after landing on Mars, the Mars Pathfinder spacecraft’s avionics computer ran the VxWorks real-time operating system with preemptive priority-based scheduling across mission tasks (Reeves 1997).

Mechanism: A high-priority thread (bc_dist) distributed critical spacecraft bus telemetry, while a low-priority thread (ASI/MET) gathered meteorological sensor data through a shared mutex-protected pipe. When ASI/MET held the mutex, medium-priority tasks preempted it. Because they outranked the meteorological thread, they kept it from running while bc_dist blocked waiting for the mutex it held, an unbounded priority inversion.

Impact: The high-priority bus distribution task missed its deadline. When the bc_sched task started the next bus cycle and found bc_dist unfinished, the flight software reset the computer. Data already collected survived each reset, but the resets recurred until the software was changed.

Response: Engineers reproduced the reset on a ground replica of the spacecraft with VxWorks event tracing, then uploaded a patch that enabled priority inheritance (Sha et al. 1990) on the mutex semaphore.

Systems lesson: The medium-priority tasks kept the low-priority mutex holder from running, so the high-priority bus task could not progress. Priority inheritance let the holder finish and release the mutex. In a motor-control path, engineers must bound such blocking time (B) or remove the shared lock from the cycle-critical exchange.

The same classification settles dynamic memory, which is not categorically forbidden on a hard timing path. A general-purpose heap allocator (malloc/free) that walks fragmented free lists has a search time that depends on allocation history, so it is unbounded. A fixed-size block pool or a Two-Level Segregated Fit (TLSF) allocator has constant-time $\mathcal{O}(1)$ bounds and can sit on a real-time path. The strictest design for the permission path is a **zero-allocation static architecture**, in which every buffer, ring queue, and state variable is placed at compile time and no memory management runs at all.

The bus that carries the command contributes to the interference term I and the blocking term B of the worst-case response time. Hermann Kopetz draws the distinction that decides whether those terms have a bound, between event-triggered and time-triggered communication (Kopetz 2011). In event-triggered communication, such as standard CAN, nodes transmit when their local state changes, so bus load is unpredictable and a single malfunctioning node can flood the shared medium with spurious traffic, the “babbling idiot” failure. A time-triggered architecture (TTA) synchronizes a global clock and gives each node its own slot in a time-division schedule (TDMA),



Figure 4.5: Industrial controller with modular I/O: Beckhoff CX5120 DIN-rail controller and adjacent I/O terminals. The controller is one hardware substrate for distributed control. (Photo: Oleg Yunakov, Wikimedia Commons, CC BY-SA 4.0).

which bounds transmission interference once clock synchronization, fault behavior, and the full schedule are verified. Industrial robots implement this deterministic layer in dedicated fieldbus hardware (figure 4.5) that couples real-time controllers to I/O and drive terminals over Ethernet frames synchronized by hardware Distributed Clocks (DC).⁹

On the mobile manipulator the candidate links sort by where they sit. SPI carries the board-level traffic of joint encoders, current-sense converters, and gate drivers, and its timing is bounded under a fixed transfer schedule. CAN-FD suits chassis and battery-management traffic, where arbitration and retransmission must enter the response-time bound under a stated traffic and fault model. EtherCAT forwards one cyclic frame through every servo node, and TSN Ethernet fits the backbone that joins camera streams to control traffic. None of these bounds belongs to a protocol name. Each depends on the configured devices, wiring, and schedule, so the question to answer is whether the permission path's servo axes fit within its bus cycle.

Napkin Math 4.2: Fieldbus serialization before schedulability

Practical engineering question: The mobile manipulator's permission path commands 9 servo axes, 7 arm joints and 2 drive wheels, once per 1 ms bus cycle. How much of that cycle do the listed CAN-FD frames or one EtherCAT frame occupy before controller and device delays are counted?

1. *CAN-FD Serialization and Arbitration Latency (9 Axes, 64-byte payload per axis):* - Bitrate: 1.0 Mbps arbitration phase, 5.0 Mbps data phase. - Per-frame bit count: 32 bits arbitration/CRC overhead at 1.0 Mbps (32 μ s), 512 bits payload at 5.0 Mbps (102.4 μ s), plus inter-frame spacing $\approx$ 6.0 μ s. - Transmission time per axis: $t_{tx} \approx 140.4 \mu$ s. - For 9 axes polling cyclically: $9 \times 140.4 \mu$ s = 1263.6 μ s (= 1.264 ms). - Result: Bus utilization exceeds 100 percent ($\rho \approx 126.4 > 1.0$), so this schedule of 9 frames cannot fit a 1 ms bus cycle even before processing. If payload is reduced to 32 bytes ($t_{tx} \approx 89.2 \mu$ s), total bus time is 802.8 μ s (80.3 utilization). This reduced payload still needs arbitration, error/retransmission allowances, competing frames, and response-time analysis; no universal 70 percent utilization threshold establishes schedulability.

⁹ | **Distributed Clock:** Under IEC 61158-4-12, EtherCAT Distributed Clocks (DC) synchronize distributed servo drive nodes to a master reference clock by measuring propagation delays across the physical ring topology. Distributed clock hardware can generate synchronized SYNC0 pulses. Achieved clock error and application sampling jitter depend on the selected devices, configuration, and measurement; clock alignment alone does not bound the complete control cycle.

2. *EtherCAT On-The-Fly Processing (9 Axes, Daisy-Chain Ring)*: - Bitrate: 100 Mbps full-duplex Fast Ethernet (10 ns/bit). - A single standard Ethernet frame carries all 9 joint commands and sensor returns in one pass. - Frame serialization time: 28.2 μ s. - Node forwarding delay: 9 slave application-specific integrated circuit (ASIC) controller hops at 0.6 μ s per hop = 5.4 μ s. - Modeled frame serialization plus 9 forwarding hops: $T_{\text{bus}} = 28.2 \mu\text{s} + 5.4 \mu\text{s} = 33.6 \mu\text{s}$. - Result: These modeled bus terms occupy 3.36 of the cycle, leaving 966.4 μ s **unallocated by this bus calculation**. Master/slave processing, propagation, clock synchronization, retries, application computation, and actuator response still consume the end-to-end deadline.

Systems insight: For the machine's 9 axes at 64 bytes per axis, CAN-FD serialization alone misses the 1 ms bus cycle, while EtherCAT frame and forwarding occupancy leaves room for other work. **TSN Ethernet** can carry gigabit camera feeds and microsecond servo frames on one physical link, but only if its time-gated queues are configured and verified across every switch so that video bursts cannot collide with time-critical control slots.

Real-time dependability cannot be achieved by buying faster clock speeds or measuring statistical latency percentiles on an idle bench. Dependability requires structural analysis of the execution path: eliminating all unbounded variance sources ($I_{\text{unbounded}} = 0$), bounding shared hardware blocking (B), enforcing zero-allocation static memory layouts, and selecting fieldbuses with cycle-deterministic distributed clocks (figure ??).

While bandwidth determines how many bytes can cross the bus in a given interval, **synchronization jitter** (σ_{jitter})—the worst-case temporal variation in when distributed nodes sample sensors and apply actuator setpoints—determines whether multi-axis physical systems can maintain stable dynamical contact. In multi-axis robotic arms and legged humanoids, field-oriented control (FOC) switches inverter pulse-width modulation (PWM) gates at 20–40 kHz, demanding sub-microsecond phase synchronization ($\leq 1 \mu\text{s}$) across all joint drives to prevent phase distortion, audible acoustic resonance, and destructive torque ripple.

As mapped across four decades of communication standards in figure ??, legacy serial protocols (Modbus RTU) and carrier-sense priority-arbitrated buses (CAN 2.0B, CAN-FD) exhibit milliseconds of non-deterministic queuing jitter under heavy bus loads, restricting them to low-rate telemetry and supervisory commands. Achieving the sub-microsecond synchronization required for dynamic physical AI necessitates hardware-scheduled time-triggered architectures: either processing-on-the-fly industrial Ethernet with hardware Distributed Clocks (EtherCAT, EtherCAT G) or IEEE 802.1AS/Qbv Time-Sensitive Networking (TSN).¹⁰ These architectures anchor clock alignment in silicon PHY timestamps, achieving sub-100 nanosecond jitter across dozens of coordinated actuators and bridging high-bandwidth neural policies directly to hard real-time motor inverters.

A bounded path delivers each command to the drives within its cycle, but it is indifferent to which commands it carries. Which processor may place a command on that path, and what that processor must check first, is a question of authority rather than timing.

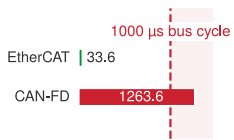
4.5 Actuation Authority

Authority on the mobile manipulator is the ability to write the drive registers that energize its base and arm windings and deliver mechanical power. In a correctly structured machine, this authority is concentrated exclusively within the deterministic real-time enforcer (figure 4.6).

A learned foundation model or trajectory planner on an application processor has zero physical authority to write PWM timer registers, set gate-driver current references, or toggle motor bridge enable pins that govern current delivery to stator windings. Instead, high-level software generates unprivileged trajectory proposals (p_t). These proposals pass across the memory transport interface into the safety microcontroller, which functions as a strict hardware gatekeeper.

The microcontroller continuously evaluates physical safety invariants: verifying that commanded joint positions lie within physical reach envelopes, that joint velocities do not exceed certified limits, and that projected decelerations remain safely within dynamic stopping clearance ($d_{\text{stop}} \leq D_{\text{clear}}$). If a proposal satisfies all invariants, the microcontroller grants execution permission, interpolates waypoints into high-frequency current references (u_t), and writes to the physical gate-driver registers. If a proposal is invalid or expired, the enforcer revokes permission ($\perp$) and requests a controlled

¹⁰ | **Time-Sensitive Networking:** IEEE 802.1Qbv Time-Sensitive Networking establishes deterministic Ethernet communication by configuring time-aware traffic shapers across network switches. High-priority control frames execute within dedicated, pre-scheduled transmission windows, while asynchronous high-bandwidth payloads (such as uncompressed camera streams) are temporarily gated. This scheduled time-division multiplexing allows vision telemetry and microsecond servo commands to coexist over a unified physical link without packet collision or buffer bloat.



For the machine's 9 servo axes, EtherCAT frame and hop time uses 3.36 percent of the bus cycle.

Downstream: Actuator override masking registers implement the hardware preemption tiers formulated in section 14.8.

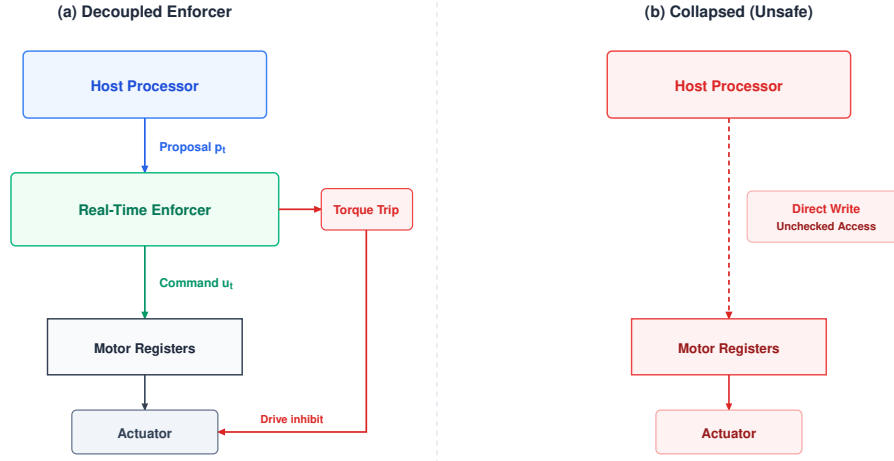


Figure 4.6: Hardware proposal boundary: (a) Decoupled enforcer on an isolated safety microcontroller with exclusive write privilege over motor drive registers, while the host processor proposes trajectories across a shared-memory link. (b) Collapsed unsafe architecture where unverified host software writes actuator registers directly, creating a single failure domain.

deceleration or hold. A separate hardware trip zone can remove drive torque on a fault; opening switches does not itself brake moving mass.¹¹

The refusal carries its own physical cost. If the enforcer rejects an invalid proposal and commands dynamic braking on a fast-moving joint, the braking path must have enough torque, clearance, and thermal capacity to arrest the mass.¹² Over a braking interval of hundreds of milliseconds, the winding resistance, brake resistor, or friction that forms that path must absorb the joint's kinetic energy within the thermal limits of section 2.5, without damaging the winding insulation or letting the mass coast past its clearance.

By virtue of the causal boundary established in section 1.2, the enforcer treats the origin of an aberrant proposal as irrelevant. A remote memory exploit in an operating system daemon, an adversarial camera perturbation inducing model hallucination, and an arithmetic overflow in a neural matrix multiplier all produce the same boundary event, an invalid proposal packet. Because the enforcer evaluates invariant bounds against physical sensor feedback from optical encoders and current shunts rather than inspecting neural network weights, it needs no model of why a proposal went wrong, and the host, which cannot write actuator registers directly, has no path around it. Protection against a compromised host still depends on independent sensing, a correct limit model, actuator capability, and a feasible fallback.

To guard against internal hardware faults or microcontroller latch-ups within the enforcer itself, the system incorporates out-of-band supervisory channels and physical disconnects. These channels consist of unprogrammable analog circuits, hardwired emergency stop mushroom push buttons, and dual-channel safety relays that sever power to the motor inverter bridges independently of all digital logic through Safe Torque Off (figure 4.7, IEC 61800-5-2), which physically interrupts gate-driver bias to prevent torque-producing stator current.¹³ These disconnects cost availability. Re-energizing a tripped high-voltage DC bus requires pre-charging the inverter filter capacitors through current-limiting resistors, because closing the contactor onto empty capacitors draws an inrush large enough to weld its contacts, and the pre-charge takes hundreds of milliseconds before motor control can resume. A disconnect therefore trades availability and active control for a fault response whose mechanical consequences depend on brakes, load, and stored energy, and it can act when the primary controller fails only if it shares no hidden clock, power rail, or silicon with that controller.

¹¹ | **Hardware Trip Zone:** Microcontroller hardware trip zones (such as TI EPWM_TZFRM or STM32 TIMx_BDTR Break inputs) couple analog over-current and overvoltage comparators directly to timer output logic. Upon fault detection, configured trip logic may force gate signals high-impedance or activate a validated braking mode. These have different mechanical consequences: torque removal can leave the plant coasting, while dynamic braking requires a rated energy path.

¹² | **Braking Dynamics:** Arresting a high-inertia manipulator joint transfers kinetic energy into the implemented braking path: winding and power electronics losses, a rated brake resistor, or mechanical friction. The trajectory must satisfy stopping clearance ($d_{\text{stop}} \leq D_{\text{clear}}$) and each component's pulse-energy and thermal limits; $\int I^2 R dt$ accounts for the winding share when resistive motor braking is used. For the complete derivation of kinematic stopping envelopes and transient thermal dissipation, see Appendix B.

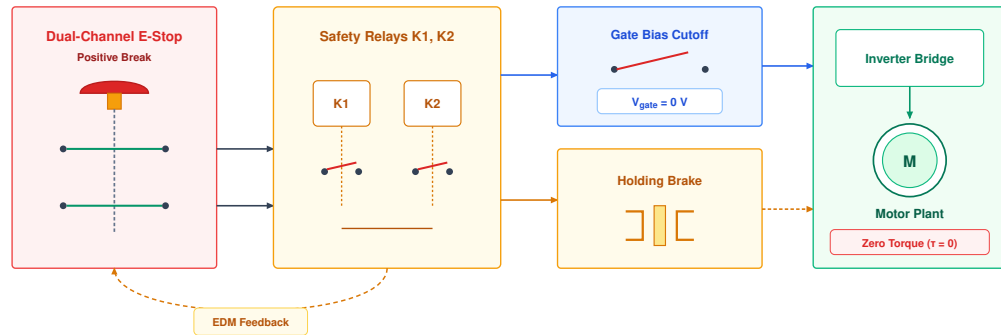


Figure 4.7: Illustrative out-of-band interlock and STO path: A dual-channel emergency switch and monitored relays remove torque-producing gate bias; a separate spring-applied holding brake is shown. This diagram describes a hardware response path, not a certified integrity level or proof of stopping distance (synthesized from ISO 13849-1 and IEC 61800-5-2).

13 | Safe Torque Off (STO): An STO implementation removes torque-producing drive signals through a hardware interlock. An IEC 61800-5-2 integrity rating depends on the actual devices, diagnostic coverage, wiring, and validation, not on the circuit pattern alone.

Systems Perspective 4.1: Hardware authority invariant

If an unverified learned policy has direct memory-mapped write access to actuator gate-driver registers, the machine has no safety layer, and its next out-of-distribution input reaches the motors unchecked.

Write privilege is therefore a physical fact about wires and registers, not a software permission, and the Brain holds none of it. That privilege protects the plant only while the enforcer stays independent of the processor whose proposals it refuses. If the two share a clock, a rail, or a memory bus, a fault in the proposer can silence the enforcer without ever writing to a drive.

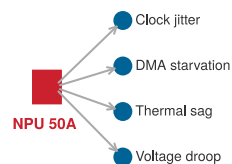
4.6 Hardware Isolation

Independence between the application processor and the permission path is a claim that often fails its first verification. System architectures routinely label the two as isolated because software tasks execute in separate processes, containers, or hypervisor partitions. Verifying the claim means checking four common-cause paths: independent execution silicon with private instruction pipelines, separate timing references from segregated crystal oscillators, isolated power rails from independent regulators, and segregated memory domains without shared dynamic random-access memory chips or interconnect arbitration buses.¹⁴ Any shared substrate needs a fault analysis that shows it cannot defeat the independent permission path under the stated failure model.

Shared electrical and physical infrastructure creates cross-domain failure modes that bypass all software access controls. On the mobile manipulator the application processor and the permission path share a battery but not a rail. The permission path runs on the isolated rail of section 2.8, held up for 2 s, so a sag of the shared 24 V control rail below its regulators' 18 V dropout (section 2.6) reaches it only as an undervoltage flag. On a shared silicon package or common thermal heat sink, sustained computational load on the application processor elevates junction temperatures beyond 105°C, triggering thermal throttling or silicon shutdown in adjacent real-time cores. When communication relies on an arbitrated interface such as a shared peripheral interconnect or memory bus, a high-bandwidth visual pipeline or a misbehaving direct memory access controller can saturate bus bandwidth, starving the permission path of encoder feedback and sensor telemetry. Even a common clock oscillator creates an unbudgeted single point of failure, where thermal drift, mechanical vibration, or crystal cracking desynchronizes both domains at the same instant.

The permission path must distinguish two failure modes of the proposer, unresponsiveness and corruption. Unresponsiveness occurs when an inference pipeline hangs, a scheduling deadline is missed, or operating system memory fragmentation blocks execution, producing no output. The permission path detects it through the heartbeat of section 4.3, which the application processor must deliver across an isolated channel before its timeout.

14 | Freedom from Interference (FFI): ISO 26262-9 and IEC 61508 formalize Freedom from Interference (FFI) as the absence of cascading faults between software elements possessing different safety integrity levels. Unverified neural policies sharing silicon or bus fabrics with safety-critical tasks require architects to demonstrate spatial (memory protection), temporal (independent scheduling and timers), and electrical isolation (independent power rails). Shared resources require evidence that interference cannot defeat the specified enforcement deadlines and fault response.



A transient on the shared rail reaches every load that draws from it; the permission path rides through on its own isolated, held-up rail.

Corruption occurs when the proposer remains fully active and passes its heartbeat deadline, but generates kinematically impossible setpoints, discontinuous velocity requests, or numerical infinities resulting from out-of-distribution visual inputs or matrix overflow. The enforcer answers the two failures through separate mechanisms. A missing fresh chunk or a missed heartbeat revokes the proposal when its timer expires, whereas corruption triggers an immediate invariant rejection on the arrival of the malformed proposal packet.

The chunk lease is distinct from the host heartbeat. When the application processor stays alive but stops publishing admissible chunks, the drives would otherwise keep tracking the last admitted setpoints, and the machine would keep moving under a proposal that nothing has renewed, until the enforcer detects the expiry and begins deceleration. Enforcing the lease at the causal boundary (section 1.2) bounds that interval. On the mobile manipulator, section 4.3 sized that interval as a term of the stopping budget and charged it against the pre-brake ceiling.¹⁵ The heartbeat can report that the process is alive, but it cannot renew an expired chunk, and its period is independent of the proposer's. Only the chunk policy renews often enough to hold the lease, and section 4.8.1 shows why the intent model cannot.

A timer feed proves only that the feed path ran. In *Bookout v. Toyota* (2013), a plaintiff expert argued that a timer-tick interrupt kept servicing the watchdog while other tasks could die (Barr 2013); this is a reported trial mechanism, not a finding that electronic faults caused unintended acceleration in every investigated vehicle.

When a lease expires or an enforcer rejects a corrupted proposal, the permission path executes an explicit degradation strategy rather than an unconditioned power cut, because cutting power hands a moving actuator to whichever unpowered state its drive falls into (section 2.6), and none of those states is a controlled stop. Degradation follows a deterministic state hierarchy calculated within the permission path. An optional short continuation of the last validated trajectory is permissible only while its chunk lease remains valid and its stopping margin is feasible. After expiry, any further hold or dead reckoning T_{hold} enters τ_{delay} and must fit in the 176.3 ms of pre-brake ceiling that the lease path and brake onset leave at the drive limit (4.1); the enforcer must begin braking sooner if the measured bounds of the remaining terms do not fit. If communication does not resume within that budget, the permission path requests controlled deceleration using the available torque and thermal budget established in section 2.4. It reaches rest only if clearance and braking capability remain sufficient. A gravity-loaded arm may require a rated mechanical holding brake after motion has been arrested.

Once the physical plant is brought to a complete, stationary halt, the architecture executes a **recovery handshake** before restoring autonomous proposal authority. The safety microcontroller asserts a persistent `FAULT_HELD` latch in a status word it owns and reports the exact physical rest pose ($\mathbf{q}_{\text{rest}}, \dot{\mathbf{q}} = \mathbf{0}$). The application processor must flush its stale action chunks, clear its autoregressive key-value (KV) attention cache, re-ingest fresh optical observations tagged with new hardware timestamps, and publish a re-synchronized chunk proposal. The microcontroller verifies that the initial waypoint of the new proposal matches the physical rest state within a strict spatial epsilon ($\|\mathbf{a}_0 - \mathbf{q}_{\text{rest}}\| \leq \epsilon_{\text{sync}}$) before releasing the fault latch and re-energizing pulse-width modulation (PWM) to the motor drives.

Verifying that these isolation boundaries function as designed requires physical falsification on the test bench. Four specific fault-injection regimes must be executed against the physical prototype: power-rail droop testing (injecting maximum-slew current steps to verify less than 50 mV of rail cross-coupling noise), clock glitching (verifying independent timing domains maintain stable cycle counts under frequency drift), communication bus flooding (saturating links to verify heartbeat telemetry retains deterministic transmission slots),¹⁶ and numerical fault injection (streaming synthetic proposals containing NaNs and infinite floats to verify single-cycle rejection without execution stalls).

Independence is therefore validated against identified common-cause faults rather than read off a block diagram, and the lease that bounds a stopped proposer comes from the stopping margin that remains after detection and actuation delays. Both defenses bound a proposer that stops or fails, which leaves open whether a proposer running at the enforcer's own rate still needs a separate check.

¹⁵ | **Chunk Lease Bound:** For a constant-speed approach, the chunk lease, the tick, the bus cycle, and brake onset must satisfy the pre-brake inequality of section 4.3 once the observation age t_{age} joins them on the left side, and a post-expiry hold T_{hold} adds to that side as well. The right-hand side is the pre-brake ceiling, not a lease to spend in full. For the complete algebraic derivation of lease bounds under bounded deceleration, see section B.4.4.

Downstream: Real-time fieldbus fault-recovery state machines are verified via hardware fault-injection in section 15.6.

¹⁶ | **CAN Arbitration:** Controller Area Network protocols resolve bus contention at the physical layer using wired-AND dominant ('0') and recessive ('1') signaling. If a defective transceiver continuously asserts a dominant state (a "babbling idiot" fault), it monopolizes the shared physical bus and blocks all higher-level communication. Hardware bus guardians and dominant-timeout clamp circuits monitor transmission duration, physically disconnecting jabbering nodes to preserve network availability for remaining safety participants.

4.7 Invariant Checking

Locomotion policies and compact visuomotor models compiled for edge accelerators can emit a new actuator command every 2.0 ms, which tempts designers to let the policy write the motor registers directly. Rate matching does not confer authority. A neural network emitting torques at 500 Hz is governed by the same statistical uncertainty as a large model generating waypoints at 5 Hz, and even when policy and enforcer share one 2.0 ms period, the network's output remains a proposal that must pass the proposal boundary of section 1.5 before it changes current in a winding. Placing an unverified policy inside the fast loop exposes the plant to three failure modes. *Compute underrun* occurs when jitter pushes an inference past the actuation deadline and leaves the power stage without a valid setpoint. *Out-of-distribution torque spikes* occur when an input the network never saw produces a discontinuous torque reversal within a single cycle. *Structural limit cycling* occurs when unmodeled compliance or sensor lag couples with the policy into resonance. The permission path meets each with a check that fits inside its own deadline: a timeout for the underrun, a torque and rate-of-change limit against the previous cycle for the spike, and an energy budget for the limit cycle (section B.8). The projection that admits or modifies a candidate command belongs to section 12.7, where it can be built on a control barrier function.

The same reasoning covers the one proposer that is not learned. A remote operator is a third proposer. Its commands pass the same permission path, and network delay spends the same pre-brake budget (section 14.2). Neither speed nor a human origin gives a proposal authority; only passing the permission path's checks does, and each check spends part of a budget that the next section lays out handoff by handoff.



Checkpoint 4.1: Hardware isolation and real-time invariant checking

Before turning to the handoffs between proposer and actuator, verify your understanding of the exchange, hardware isolation, and pre-actuation invariant defenses:

- ☐ **Nonblocking exchange:** Can you explain how the seqlock reader detects a torn read in one bounded attempt, and why a writer stopped mid-write leaves the active spline valid only until its lease expires?
- ☐ **Four dimensions of physical isolation:** Can you enumerate the four physical boundaries (silicon pipelines, timing crystals, power rails, memory buses) that a check for fault-domain independence must cover, and explain how shared paths would need validation?
- ☐ **Physical derivation of chunk leases:** Can you explain why the chunk lease must be sized from stopping distance and clearance rather than chosen as a software timeout, and why a heartbeat never renews it?
- ☐ **Rate matching and authority:** Can you explain why a policy that runs at the plant's own rate still produces only proposals, and which check answers each of its three failure modes?

4.8 The Handoffs Matrix

When the mobile manipulator overruns a stop, the diagnosis starts by locating the fault at one of five canonical handoffs between the photons its cameras collect and the current its drives deliver. Upstream, sensing transfers raw transducer signals to perception pipelines. Perception yields intermediate representations that fuse into an estimated world state. The state feeds the learned policy to produce an action plan. The plan submits candidate trajectories to the safety enforcer. Finally, the approved command passes from the enforcer to the physical actuators. Across each boundary, five interface limits govern feasibility: latency (τ), bandwidth (β), energy and heat ($\mathcal{E}$), determinism (σ_t), and fault domain isolation ($\mathcal{F}$). The five handoffs and the five interface limits form a five-by-five matrix that maps every signal path to the physical budgets of the machine. The limits are distinct from the body's five budgets of section 2.1, which enter the matrix chiefly through its latency and energy columns. The record this section produces, the **handoff record**, consumes the limit records of section 2.7, including the proposer limits that chapter 3 states in the same format.

The example allocation in table 4.2 marks which limits are design-limiting at each handoff, and its numbers are chosen budgets, not measurements. The matrix gathers the budgets of all three Part I chapters, together with the sensor capture that section 1.5 places at the Boundary. In the Boundary row ($\mathcal{H}_1$), where the world becomes measurement, bandwidth and latency bind first, because raw multi-camera pixels stream over SerDes lanes and the exposure window sets a floor on image age before serialization begins. The Brain rows ($\mathcal{H}_2, \mathcal{H}_3$) bind first on energy and latency, the inference costs that chapter 3 established. The Body row ($\mathcal{H}_5$) binds first on determinism and heat, because the current loops of chapter 2 must run on time and every ampere they deliver warms the windings and the power stage.

The Nervous System row ($\mathcal{H}_4$) is the architectural bridge that ties the Brain and the Body together. The proposal boundary (section 1.5) between the learned planner and the enforcer carries an action proposal (p_t), which consists of a compact spline parameterization or waypoint chunk. For the compact payload assumed in this grid, $\beta < 100$ kB/s is budgeted, so bandwidth does not bind at this boundary; latency, determinism (σ_t), and fault domain isolation ($\mathcal{F}$) do. The permission path isolates stochastic proposals with nonblocking mailboxes and expiring leases. On the mobile manipulator the latency limit is the chunk lease, which takes 60 ms of the 258.3 ms pre-brake ceiling at the drive limit (section 4.3), and each chunk is admitted or refused within one 1 ms permission tick. Isolation rests on a private bus, a separate oscillator, and the isolated, held-up rail of section 4.6.

From the causal boundary perspective framed in section 1.2, when a physical AI system fails, the breakdown manifests as an unbudgeted limit crossing at a specific handoff cell rather than an abstract deficiency in learned intelligence. A handoff diagnosis must compare like evidence. For the $\mathcal{H}_2$ state-estimation path, suppose engineers allocate a 30.0 ms latency budget and measure $P_{99} = 24.5$ ms on a baseline hardware-in-the-loop run, a sampled margin of +5.5 ms. After thermal throttling, a measured $P_{99} = 85.0$ ms gives -55.0 ms against that design budget. The decision is to reduce or reallocate workload and retain independent enforcement while revalidating. A P_{99} sample diagnoses the regression; it is not a worst-case timing proof.

Table 4.2: Handoffs-by-limits synthesis grid: A five-by-five matrix mapping five canonical signal handoffs ($\mathcal{H}_1$ – $\mathcal{H}_5$) across the Boundary (section 1.5), Brain (chapter 3), Body (chapter 2), and Nervous System (chapter 4) against five physical limits ($\tau, \beta, \mathcal{E}, \sigma_t, \mathcal{F}$). Cells marked *BINDING* are the budgets that drive selection; *Non-binding* means the scenario budget meets that dimension with margin. The matrix is the diagnostic rubric for Part I, in which safety and determinism appear as properties of isolated hardware boundaries, not software conventions.

Canonical Handoff Boundary	Latency Limit (τ)	Bandwidth Limit (β)	Energy & Heat Limit ($\mathcal{E}$)	Determinism Limit (σ_t)	Fault Domain Isolation ($\mathcal{F}$)
$\mathcal{H}_1$: Sensing $\rightarrow$ Perception (Boundary)	BINDING $\tau \leq 16.7$ ms (exposure & readout sets sensor age)	BINDING $\beta \geq 1.2$ GB/s (multi-camera 4K uncompressed streams)	<i>Non-binding</i> $P < 3$ W (sensor head dissipation isolated)	BINDING $\sigma_t \leq 1.0$ ms (frame-sync triggers prevent skew)	<i>Non-binding</i> Defended by SerDes hardware & DMA ring buffers
$\mathcal{H}_2$: Perception $\rightarrow$ State (Brain)	BINDING $\tau \leq 30.0$ ms (visual-inertial odometry & feature window)	<i>Non-binding</i> $\beta \approx 10$ – 50 MB/s (compressed keypoints & voxels)	BINDING $P \approx 15$ – 35 W (edge vision backbone accelerator envelope)	<i>Non-binding</i> Defended by sliding-window state estimator buffers	BINDING Spatial memory isolation: MPUs prevent memory corruption
$\mathcal{H}_3$: State $\rightarrow$ Plan (Brain)	BINDING $\tau \leq 100.0$ ms (autoregressive tokenization & diffusion horizon)	<i>Non-binding</i> $\beta < 5$ MB/s (low-dimensional state & prompt embeddings)	BINDING $P \approx 30$ – 75 W (transformer/diffusion matrix accelerators)	<i>Non-binding</i> Defended by receding-horizon chunking & spline buffers	BINDING Sandboxed policy container prevents host panics halting telemetry

Canonical Handoff Boundary	Latency Limit (τ)	Bandwidth Limit (β)	Energy & Heat Limit ($\mathcal{E}$)	Determinism Limit (σ_t)	Fault Domain Isolation ($\mathcal{F}$)
$\mathcal{H}_4$: Plan → Enforcer (<i>Nervous System</i>)	BINDING chunk lease 60 ms (chosen, inside limits derived from the stopping budget); per-tick admission within each 1 ms tick	<i>Non-binding</i> Defended by compact parameterized splines ($\beta < 100$ kB/s)	<i>Non-binding</i> $P < 0.5$ W (fixed-point constraint evaluation on MCU)	BINDING $\sigma_t \leq 100 \mu\text{s}$ (cycle-deterministic packet reception on MCU)	BINDING Hardware boundary: private bus, separate oscillator, isolated held-up rail
$\mathcal{H}_5$: Enforcer → Actuator (<i>Body</i>)	BINDING $\tau \leq 1.0$ ms (servo interval 1.0 ms; PWM loop 50 μs)	<i>Non-binding</i> $\beta \approx 1\text{--}5$ Mbps (cyclic PDOs over deterministic fieldbus)	BINDING $I^2 R$ copper winding dissipation, MOSFET switching losses	BINDING $\sigma_t < 10 \mu\text{s}$ (hard real-time PWM phase alignment & FOC)	BINDING Galvanic isolation, out-of-band analog E-stops, STO circuits

To ensure these budgets survive the lifecycle of the machine, every boundary condition in the grid is recorded as a handoff record, the verifiable contract between hardware designers, systems engineers, and model developers. Each entry carries the six elements of a limit record (section 2.7) and adds the fields that locate the limit at a handoff and say what kind of bound it is (table 4.3).

Table 4.3: Conceptual fields of the handoff record: The six limit-record elements of section 2.7, extended with the handoff, bound kind, and clock domain that locate a limit in the machine and state what evidence supports it.

Field Name	Physical SI Units	Structural Invariant & Description
signal_id	dimensionless	Unique alphanumeric identifier (e.g. SIG_VIO_01, SIG_TRQ_AX1).
source_subsystem	dimensionless	Originating hardware or software module (Camera_SerDes, Planner_VLA).
destination_subsystem	dimensionless	Receiving hardware or software module (State_Estimator, Safety_MCU).
handoff	dimensionless	The handoff the signal crosses ($\mathcal{H}_1$ to $\mathcal{H}_5$).
physical_dimension	dimensionless	Target limit category ($\tau, \beta, \mathcal{E}, \sigma_t, \mathcal{F}$).
value	SI units (ms, MB/s, W, μs)	Allocated budget, or the measured distribution it is checked against; for $\mathcal{H}_4$, the chunk lease derived from the stopping budget.
conditions	as recorded	Payload, temperature, supply voltage, firmware build, and concurrent workload under which the value holds; the value lapses when they change.
uncertainty	same as value	Error bound of the instrument or model that produced the value.
margin	same as value	Engineering margin between the operating target and the breakdown threshold.
detector	dimensionless	Runtime monitor that catches an excursion (for example, a lease timer, a rail-voltage monitor, or a phase-current sensor).
provenance	dimensionless	Source of the value, including its evidence category (measured, derived, datasheet, or illustrative).
bound_kind	dimensionless	requirement, proven bound, or measured distribution; only a proven bound may stand in for a worst case.
clock_domain	dimensionless	Clock on which every time-valued field is expressed; a handoff that feeds the permission path uses the safety microcontroller's clock.
verification_timestamp	UTC timestamp	Date of bench falsification and sign-off; a log field, never an input to an age test.

4.8.1 One coupled budget for the anatomy

The three chapters of Part I each set terms of one budget, and the chunk lease is where they meet. Section 2.3 fixed the distance side. At its 1.5 m/s drive limit the loaded machine must stop inside the 1.10 m clear distance at a rack end, and the braking term and the fixed overheads leave 258.3 ms for everything that happens before braking force begins. Section 4.3 spent part of that time. A proposer that stalls keeps its authority for one lease, the stop reaches the drives one tick and one bus cycle later, and the stopping budget stands at 835.5 mm.

The lease cannot be made as short as the budget would like, because a proposer must renew it. Section 3.4 established what each learned proposer can supply. The intent model's weights take 98.0 ms to stream from memory once, before any vision work, and a full inference takes 160 ms, so at 5 Hz it cannot renew a proposal every few tens of milliseconds. The chunk policy's weights stream in 0.70 ms, and at 20 Hz it publishes a fresh chunk every 50 ms, each within its 40 ms P99 inference time, so it can.

These facts fix the **lease rule** for this machine. The lease must outlast one chunk period plus an allowance for renewal jitter, which together set the 55 ms floor of 4.1. It must also fit inside the pre-brake ceiling once the tick, the bus cycle, and brake onset are reserved, which leaves 236.3 ms less the observation age that chapter 8 measures. The chosen 60 ms sits just above the lower limit, because every millisecond of lease costs 1.5 mm at the drive limit, and later chapters add terms that need that distance.

Each chunk carries the base's velocity and the arm's joint targets together, so one lease governs both bodies, and when it lapses each body enters its own stop. The lease is sized on the base's 258.3 ms pre-brake ceiling because that ceiling is tighter than the limit the arm's commitment sets. The same lease bounds the arm, and section 11.2 derives the arm's limit and checks its lease path and ceiling against its own clearance. The arm's evidence horizon bounds a different lease, the intent lease, which limits how long an admitted goal from the intent model stays valid, and the intent model's slower targets expire under it (section 10.7).

The $\mathcal{H}_4$ row of table 4.2 is this chapter's handoff record filled in, the first link after the limit record in the chain of records that the rest of the book extends. Its value is the 60 ms chunk lease handed from the chunk policy to the permission path, a **requirement** chosen inside limits derived from the stopping budget and valid up to the 1.5 m/s drive limit, loaded, on a dry floor, at the chunk period above; its margin is the span between the renewal floor and the ceiling-derived upper limit less the observation age, and its detector is the lease timer on the safety microcontroller, counted on that processor's own clock.



Checkpoint 4.2: The handoffs matrix and physical budgets

Before testing the lease on a humanoid and reviewing the engineering fallacies, verify your understanding of system-wide interface contracts:

- ☐ **Signal handoff classification:** Can you identify the five canonical handoffs ($\mathcal{H}_1$ – $\mathcal{H}_5$) bridging physical plant mechanics to high-level deliberative reasoning?
- ☐ **Binding physical constraints:** Can you explain why sensory handoffs bind first on bandwidth and exposure latency, whereas actuation handoffs bind first on determinism and heat?
- ☐ **Handoff record provenance:** Can you describe why every numerical limit in an autonomous system must record its evidence category, operating conditions, bound kind, and verification status?
- ☐ **The lease rule:** Can you state the two limits on the chunk lease, one set by the proposer's renewal period and one by the pre-brake ceiling, and explain why the chunk policy can renew the lease while the intent model cannot?

🔍 Systems Perspective 4.2: A deadline set by a fall

Question: Does the split among proposer, lease, and permission path still hold when the deadline comes from a fall rather than a stop?

An upright biped diverges from balance on the timescale $\tau_{\text{fall}} = \sqrt{z_c/g}$, where z_c is its center-of-mass height, about 303 ms for a center of mass near hip height. The split holds, and what changes is the deadline its terms answer to:

- **The lease is sized from the step deadline.** The lease, the tick, the bus cycle, and the observation age must fit inside the time before a recovery step must begin, and the fallback that an expired lease selects is a validated step rather than a brake.
- **The intent model is still disqualified.** Its renewal period alone spends most of the fall timescale and leaves no room for the tick, the bus cycle, and the step itself, so only the chunk policy can renew a lease on this deadline.
- **Heat and supply sag limit the step.** The recovery step draws peak current in both legs at once, heating windings and sagging the supply (section 2.6), so the permission path must admit only a step that the supply and the windings can deliver at that moment.

4.9 Fallacies and Pitfalls

A block diagram draws a logical line between high-level deliberation and low-level enforcement, but physical hardware often connects them invisibly through shared power rails, interrupt lines, or back-channel diagnostic interfaces. These physical couplings dictate whether the permission loop remains standing when a complex subsystem fails.

Pitfall: *Servicing a safety watchdog inside a high-priority timer interrupt rather than an application-verified safety loop.*

The mobile manipulator's self-watchdog removes drive power through the Safe Torque Off path of section 4.5 if it receives no feed within 5 ms, because an enforcer that has stopped completing its checks can no longer command a controlled stop. Suppose engineers refresh it from a high-priority timer interrupt and the lower-priority enforcement task deadlocks. The interrupt continues firing every 1 ms tick even though the task has frozen and stopped checking torque limits, clearance, and chunk leases. The watchdog therefore stays fed while the enforcer has silently lost its safety function. Its service pulse must depend on successful, sequential completion of the required enforcement checks (a windowed watchdog pet conditioned on a task completion token). An independent timer tick establishes that hardware interrupts still cycle, not that the controller has safely evaluated the machine's physical state (section 4.6).

Pitfall: *Leaving auxiliary diagnostic or calibration interfaces unmediated in production operation.*

Suppose the mobile manipulator validates base velocity proposals through its permission path, yet its drive-wheel motor drives retain an unmonitored serial calibration interface. A diagnostic daemon uses that interface to write a 35.0 A test value directly into an inverter register. The proposal mailbox still contains zero velocity, so checking that mailbox does not reveal the bypassed command. The drive wheels can then produce torque that no motion check ever saw or approved. True actuation authority includes every physical wire or digital interface capable of changing a power-stage register. Service interfaces therefore need operational interlocks or equivalent enforcement that prevents calibration writes from bypassing the controller during normal operation.

Fallacy: *Evaluating safety barrier invariants on commanded trajectory setpoints is equivalent to evaluating physical plant state.*

Suppose the machine's arm evaluates a control barrier function (Ames et al. 2019) using commanded joint positions instead of load-side encoder measurements to reduce bus traffic. When a gear fails and the link moves away from its commanded trajectory, the interpolator continues producing valid setpoints. The enforcer still calculates that all is safe ($h(\mathbf{q}_{\text{cmd}}) > 0$), even as the physical link enters a forbidden region. Computing the wrong input faster cannot correct it. Safety enforcement belongs at the causal boundary (section 1.2), so it must evaluate physical state measurements through a

dependable sensory channel and account for their limitations; inspecting the command stream merely establishes what the planner intended, not where the heavy machinery actually is.

Fallacy: *Nominal, fault-free operating logs are sufficient to prove physical architectural isolation.*

The mobile manipulator completes normal trials without a missed enforcement deadline. Those logs show behavior under a fault-free workload; they do not expose the hidden physical couplings that emerge only under faults. Qualification therefore injects power transients, floods communication buses, corrupts policy memory, and locks a shaft on a dynamometer. Each experiment must check an explicit response bound, such as whether locked-rotor detection clamps current within 2.0 ms, rather than treating the absence of a crash as success. Evidence for isolation comes from exercising the claimed fault boundaries and measuring continued enforcement or the required safe transition.

4.10 Summary

Chapter 2 measured how far the loaded machine travels before it can stop, and chapter 3 measured what each learned proposer can supply and how quickly its evidence ages. This chapter set the permission path's check rate from the joint loop's phase margin, and it converted those two measurements into the time for which a proposal may drive the actuators. The result on the mobile manipulator is a chunk lease long enough to outlast one renewal of the chunk policy and short enough to fit inside the pre-brake ceiling, and the handoff record carries that lease, with its conditions and its evidence, to every later chapter. The lease and every other mechanism of this chapter remain claims until they have been exercised under compute delays, bus faults, bad proposals, sensor errors, and loss of braking authority. Part I opened on the irreversibility law (principle 1), and the lease closes Part I as a rule about authority, since motion that cannot be recalled may be commanded only for as long as the stopping budget can pay for its travel.



Key Takeaways: The permission path and determinism

- **Proposal is not permission:** High-level learned models act strictly as unprivileged proposal engines (p_i). Physical write privilege over motor drive registers is reserved exclusively for the deterministic real-time enforcer, on silicon whose isolation from the proposer has been demonstrated.
- **Cadence is dictated by physics, not software:** The example rates (20 kHz current regulation, 1 kHz joint checks, 10–50 Hz action chunking, and slower deliberation) are selected against winding time constants, phase margin, exposure, workload, and response budgets.
- **The exchange never waits, and every proposal expires:** The permission path reads the mailbox in one bounded attempt per tick and keeps its last validated chunk on a collision, and the proposal header's evidence time and expiry end that chunk's authority whether or not any cancellation arrives.
- **Worst-case bounds outrank empirical distributions:** A statistical latency percentile (P_{99}) cannot prove real-time safety. Deterministic certification requires analytical Response Time Analysis ($R_i = C_i + B_i + I_i \leq D_i$), eliminating unbounded bus contention and memory allocations from the safety path.
- **The veto is only as independent as what it shares:** A permission path that shares a clock, a power rail, a memory bus, or silicon with the proposer can be silenced by the proposer's faults, so independence is a claim that fault injection must demonstrate, not a property of separate processes.
- **Stopping dynamics dimension the chunk lease:** On the mobile manipulator at its drive limit, the pre-brake ceiling is 258.3 ms. The chosen 60 ms lease sits just above the shortest lease the chunk policy can renew, and the path from its last renewal to brake onset spends 82 ms, bringing the stopping budget to 835.5 mm of the 1.10 m clear distance.

- **A failure is an over-budget limit at one handoff:** The handoff record places each limit at one of five handoffs with its conditions, evidence category, and bound kind, so a regression is diagnosed against its allocated budget with like-for-like evidence.



What's Next: From machine anatomy to closed-loop physical learning

With the Body (chapter 2), the Brain (chapter 3), and the Nervous System of this chapter now specified, **Part I (The Machine Anatomy)** has assembled the five-level machine and the handoff record that times it. Assembling the container does not confer operational competence. The mobile manipulator's learned models do not arrive knowing how to open a spring-latched cage door or how to recover when the gripper slips off its handle, and every demonstration that could teach them passes through the permission path this chapter built, which may clip, modify, or refuse what the operator commanded. In **Part II (Teaching the Machine)**, chapter 5 therefore starts from the gap between the action requested and the motion the body delivered, and asks what a demonstration record must preserve so that a policy trained on it learns the task rather than the quirks of the machine that collected it.

II

TEACHING THE MACHINE

Part II

Teaching the Machine

IN an offline machine learning benchmark, the dataset is fixed before evaluation. A classifier reads held-out examples drawn from a specified data distribution ($x \sim P_{\text{data}}$); its prediction does not change the next stored example. This useful testing assumption says nothing about a deployed model whose outputs affect people, later data, or physical state.



Principle 2: Endogenous data

Invariant: A physical agent's actions choose its next observations ($s_{t+1} \sim P(s | s_t, a_t)$), so the states a deployed policy visits need not be the states it was trained or tested on. For behavioral cloning, the worst-case cost of that drift can grow with the square of the horizon (Ross et al. 2011), as equation 5.1 states.

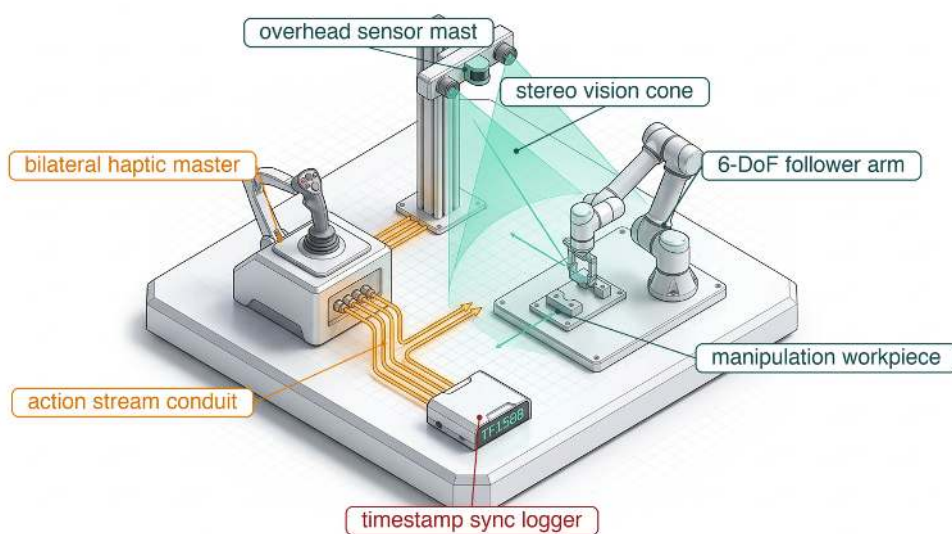
Implication: A learned proposer's competence is a claim about a declared region of states, established in closed loop with the permission path in place. States the data never visited are recorded as absent rather than assumed covered, and the finite trials behind a claim bound it rather than prove it.

Part II follows a learned proposer, the policy that opens the warehouse mobile manipulator's spring-latched cage door, from demonstrations recorded through the permission path under the clock domains of the handoff record that Part I built, to the evidence that it works. Each chapter narrows what the next may claim.

- **Physical Data (chapter 5):** Decides which conditions the demonstrations support and what they cost to collect, and produces the demonstration provenance record, which lists the conditions the data does not cover as known absences.
- **Policy Training (chapter 6):** Decides which training regime fits the data's support and which operational design domain (ODD) the resulting policy may declare, and produces the policy manifest.
- **Closed-Loop Evaluation (chapter 7):** Decides what a finite set of closed-loop trials supports about the proposer, and produces the evaluation record, whose claim stays inside a target ODD.

Part II hands forward an evaluation record: a claim about the learned proposer confined to a target ODD, the finite-sample bound that supports it, a catalog of the conditions it does not cover, and the runtime monitors the rest of the machine must supply. That record rests on the policy manifest from chapter 6, whose declared ODD cannot exceed the support that the provenance record from chapter 5 shows. Part III trusts the proposer no further than that record, and its permission path is built for the states the evaluation left uncovered.

Physical Data



- 5.1 Endogenous Experience
- 5.2 Where Physical Data Comes From
- 5.3 Which Action Becomes the Label
- 5.4 Teleoperation Demonstration Costs
- 5.5 Collection Policy Coverage
- 5.6 Intervention and Recovery Data
- 5.7 Cross-Embodiment Transfer
- 5.8 How Datasets Go Wrong
- 5.9 The Dataset Schema
- 5.10 Fallacies and Pitfalls
- 5.11 Summary

Purpose

What does the physical machine that collected a dataset leave permanently stamped into the policy that trains on it?

A robot trajectory reflects the physical machine that produced it. Joint geometry, gearbox backlash, and contact friction shape every motion, while sensor timing decides which observation accompanies each command. Crucially, raw actions change between proposal and execution—first mapped into discrete setpoints, then admitted, rate-limited, or clamped by the real-time permission path beneath the proposal boundary. A policy trained on those records inevitably learns these collection conditions alongside the nominal task. Successful demonstrations also omit the recovery behavior needed once an error diverts the machine from its path, leaving the policy unprepared for the failure states its own mistakes produce.

Every physical demonstration consumes finite actuator life, mechanical wear, and operational time on real hardware. Because identical commands yield divergent dynamics across different physical plants, embodied datasets transfer poorly without explicit calibration. High-capacity models cannot overcome systematic recording gaps through volume alone; an autonomous policy requires data that exposes the boundary between nominal performance and unmodeled disturbance. A physical dataset is therefore evidence about one specific plant's interaction across the causal boundary. It can be trusted only when its provenance captures what was requested, what was admitted, what the body executed, and which operating envelopes were never explored.

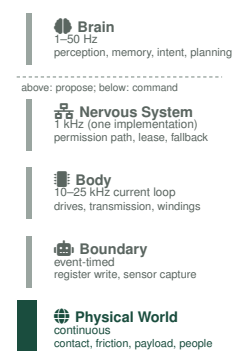


Figure 5.1.: The five-level machine stack with the Physical World highlighted.



Learning Objectives

- Estimate usable demonstration yield from collection time and reset overhead, and budget the mechanical wear a campaign consumes
- Analyze how the collecting policy limits dataset coverage and downstream recovery behavior
- Separate failed autonomous actions from human recovery trajectories in timestamped intervention records
- Evaluate whether a dataset’s mechanical and sensing assumptions transfer to another machine
- Design a provenance record linking each observation to its requested, mapped, enforced, and measured actions and to the collecting plant
- Assess whether a dataset supports a claimed operating domain by joining its occupancy against a scenario ledger

5.1 Endogenous Experience

A policy trained on the retained demonstrations of a door-opening campaign, clean teleoperated openings of the warehouse mobile manipulator’s spring-latched cage door, can still fail at the latch on its first autonomous attempt. One possible cause is the shift the machine’s own actions create.¹ In a fixed offline vision dataset, the model’s prediction does not change the next stored example. Here, an executed action changes the body’s trajectory, camera pose, and contacts, thereby changing later observations ($s_{t+1} \sim P(s | s_t, a_t)$).

¹ | **Causal Boundary:** The causal boundary between computation and physics is established in section 1.2. Because state evolution depends on motor actuation, policy actions perturb the distribution of future observations, which breaks the independent and identically distributed (i.i.d.) assumption of disembodied machine learning.

² | **Behavioral Cloning Bound:** The bound derived by Ross et al. (2011) is a worst case on expected cumulative cost, not a prediction of tracking error; actual drift depends on the plant, task, policy, and safeguards. section A.5 derives the bound and states its loss and bounded-cost assumptions.

³ | **Hardware Fatigue Life:** Actuator thermal limits and gearbox wear, detailed in section 2.4 and section 2.5, change joint friction over a campaign, so a pipeline that assumes stationary plant dynamics meets drift as the collection machine ages.

Prerequisite: Actuator thermal dissipation (dT/dt) and winding damage limits are derived in section 2.5.



Definition 5.1: Endogenous covariate shift

Endogenous covariate shift is the closed-loop distribution discrepancy wherein an embodied agent’s executed actions alter its subsequent physical states and sensory observations ($s_{t+1} \sim P(s | s_t, a_t)$), driving the system into state-space regimes unsupported by its training demonstrations.

1. **Significance:** Explains how offline behavioral cloning can fail during closed-loop physical execution: a perturbation may drive the system into states where the policy lacks training support.
2. **Distinction:** Unlike exogenous dataset shift caused by changing ambient lighting or seasonal weather, endogenous shift is actively generated by the agent’s own closed-loop actions altering its subsequent observation distribution.
3. **Common pitfall:** Attempting to fix closed-loop policy failure by collecting more nominal expert demonstrations, which only increases data density along the success manifold without providing the corrective recovery data needed to escape boundary excursions.

This shift is the collection-side face of principle 2. It makes an archive of flawless executions without recovery trajectories a liability, because the policy’s first mistake moves the robot outside the demonstrated states. Let π be the learned policy, π^* the expert, J expected cumulative task cost over T steps, and ϵ the probability of disagreement on expert-visited states under 0–1 action loss. If per-step task cost lies in $[0, 1]$, behavioral cloning admits the worst-case bound (equation 5.1):²

$$J(\pi) - J(\pi^*) \leq T^2 \epsilon. \quad (5.1)$$

Gathering physical experience is also expensive. Every recorded second consumes actuator fatigue life, heats motor windings, cycles gearbox teeth, and drains the battery.³ Operator rest, calibration, scene resets, and failed or rejected attempts leave less than a quarter of scheduled cell time as validated demonstration data (section 5.4). Physical demonstration data is therefore scarce, and it misleads training if logged naively.

A physical command also changes meaning on its way to the actuators. In a machine divided by the proposal boundary,⁴ the operator's or policy's request is mapped into a setpoint, admitted or modified by the permission path, and answered by the plant with a measured response. Logging only the request trains the policy on demands the plant could never execute, and logging only the response teaches it structural deflection and gear backlash as though they were deliberate human skill.

A collection campaign therefore makes two decisions before it records anything: which source will choose the actions, and which of those four forms of each action the dataset will keep as its label.

5.2 Where Physical Data Comes From

A physical dataset is not found or scraped. Every transition in the archive required moving mass against friction on a real machine. The difficulty of learning from such records was visible in Dean Pomerleau's ALVINN (Autonomous Land Vehicle in a Neural Network) (Pomerleau 1989), which learned to steer from camera images of human driving. Because the human drivers rarely erred, the dataset held no recoveries from the lane edge, and small execution errors compounded in closed loop.

Physical data sources separate according to who chooses the action:

1. In **kinesthetic teaching**, a colocated demonstrator physically grasps and moves the unpowered or gravity-compensated joints of the mechanism.
2. In **teleoperation**, a remote operator guides an input device such as a leader arm, a multi-axis space mouse, or a tracked hand controller, transmitting commands across a software interface to the follower robot.⁵
3. In **scripted collection**, an engineer executes a deterministic control routine, such as a state machine tracking interpolated splines or force-thresholded primitives.
4. In **exploratory collection**, an active policy selects actions to optimize an information-theoretic objective, sample state space, or maximize a reward signal.
5. In **autonomous logging during deployment**, the production policy commands the actuators as it performs work in a factory or warehouse, storing incoming sensor streams in situ.

Because the source decides how the machine approaches the world, these sources occupy disjoint state-action regions even on the same task. Consider releasing the cage-door latch. A colocated demonstrator guiding the arm feels the latch bind and corrects at the wrist. A teleoperator watching video has no such feel and pecks at the strike plate in short moves and pauses. A scripted controller approaches along one programmed path at constant speed, producing the same force spike against the plate on every attempt. A partially trained policy wanders, binding the latch at oblique angles until the contact tripwire ends the attempt. These logs are not noisy samples of one trajectory. They are outputs of distinct dynamical systems with different bandwidths, delays, and compliances.

Treating each recorded control step as an independent experience is a common error. A logger running at a high control rate produces enormous step counts, but adjacent steps are nearly identical, since two frames a few tens of milliseconds apart differ only by sub-millimeter displacements and sensor noise. Coverage scales with the number of independent episodes and distinct physical configurations, not with the sampling clock, so a modest archive spread across hundreds of workpiece positions and lighting conditions supports far more than a recording ten times longer at one bench location.

Simulation avoids the wear bill and is treated in chapter 6. When synthetic trajectories are mixed with physical logs, each record must carry its source. A model trained on an undifferentiated mixture inherits the physics engine's contact errors where synthetic data dominates and the hardware's backlash and noise where physical data dominates, and no one can later tell which caused an on-robot failure. A record that names its source has settled the first of the campaign's two decisions, and it still has to say which form of each action it kept.

5.3 Which Action Becomes the Label

By the time the gripper meets the strike plate on the cage-door latch, the operator's hand motion has become four different signals, and the one the dataset keeps as its label decides which the learned

⁴ | **Proposal Boundary:** The proposal boundary, defined in section 1.5, separates learned proposals from the permission path that alone may admit them to the actuators (chapter 12). Logging the action on both sides of it keeps permission-path interventions out of the demonstrator's labels.

⁵ | **Bilateral Teleoperation Lineage:** Raymond Goertz built the first bilateral master-slave telemanipulators at Argonne National Laboratory in the late 1940s and 1950s for handling radioactive materials. His linkages reflected force across a physical barrier, because operators without kinesthetic resistance jammed the remote mechanism.

Contrast: The display and operator-response delay that paces a teleoperator's step-and-wait motion (section 5.4) contrasts with the 1 ms permission tick on which each command is checked (section 4.3).

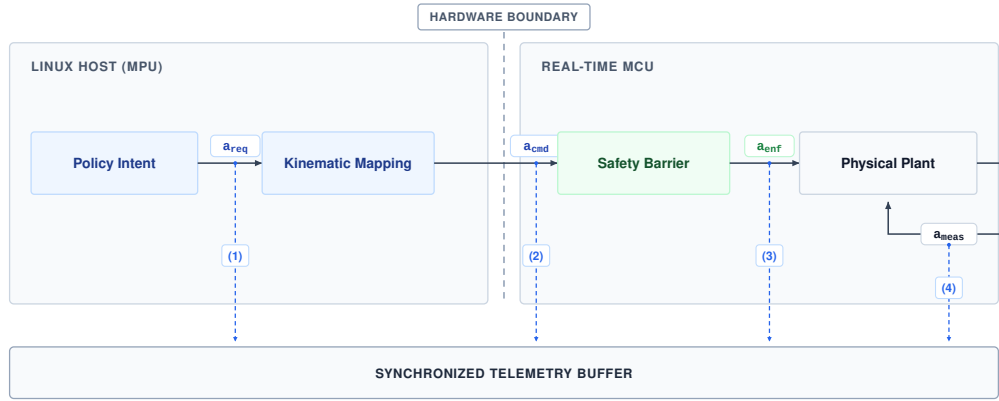


Figure 5.2: Four-stream action tap tuple: Four places a command can be logged, from raw policy intent on the application processor through to measured plant response. Which tap a dataset records decides what the model learns to imitate. Source: architectural design adapted from Zhao et al. (2023) and Wu et al. (2024).

model reproduces. The machine splits its control path across two processors (figure 5.2), so the label can be tapped in two computational domains:

- The unprivileged **application processor** captures camera streams, runs teleoperation input mapping or the policy’s forward pass, and maps coordinates.
- On the permission side of the proposal boundary (section 1.5), the safety microcontroller (MCU) runs at 1 kHz on bare metal, executing impedance control, safety barrier functions, velocity clamps, and current limits.⁶

The four taps along this control path capture fundamentally different physical semantics:

- The **requested action** (a_{req}): the raw operator input or policy proposal, tagged with its representation, units, frame, and issue time.
- The **mapped command** (a_{cmd}): the retargeted Cartesian or joint setpoint, tagged with the new representation, units, frame, and mapping time.
- The **enforced command** (a_{enf}): the same typed setpoint after the independent safety path has admitted, modified, or rejected the proposal.
- The **measured response** (a_{meas}): the corresponding measured position or velocity over a stated sampling interval; encoder readings and motor current are separate sensor channels, not interchangeable action coordinates.

Only the full tuple, logged across the boundary between application processor and MCU, separates what the operator asked for from what the permission path allowed.



Definition 5.2: Four-stream action tap

Four-stream action tap ($a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}}$) is the time-synchronized multi-tier telemetry record that captures an action signal across its four causal transformation stages: unprivileged operator or policy intent (a_{req}), kinematically retargeted setpoint (a_{cmd}), safety-governed command (a_{enf}), and physical plant response (a_{meas}).

1. **Significance:** Training imitation learning policies on raw requested actions (a_{req}) injects dynamically infeasible commands that trigger safety trips in deployment. Conversely, training on measured responses (a_{meas}) causes the model to imitate mechanical compliance, backlash, and external disturbances rather than intentional control. Logging all four streams disambiguates control intent from permission-path intervention.

⁶ | **Hardware Timer Interrupt:** On real-time MCUs, encoder counts and current-shunt readings are latched by hardware timers and Direct Memory Access (DMA), which holds sampling jitter to microseconds and keeps timestamp jitter out of numerical differentiation for velocity and acceleration.

2. **Distinction:** Unlike classical robot telemetry logs that capture only final motor encoder states or raw teleoperation joystick inputs, the four-stream tap preserves the exact causal filtering stages applied by intermediate software and safety hardware.
3. **Common pitfall:** Logging streams without timestamps synchronized in hardware over the IEEE 1588 Precision Time Protocol (PTP), written t_{PTP} . Even a few milliseconds of clock skew between the application processor and the MCU corrupts the temporal alignment between visual observations and corresponding torque actions.

Recording the right tap does not settle when each recorded value was true. Cameras integrate photons over an exposure window of tens of milliseconds before readout and transfer, force sensors sample at around a hundred hertz, joint encoders and current loops run at the MCU's kilohertz tick, and the action loop dispatches at tens of hertz. A data loader that latches the latest available sample pairs each action with a camera frame whose exposure was centered 25–40 ms before the frame reached host memory, and the policy learns to map past states to present actions, with the phase lag baked into its weights. Logging engines prevent this in three steps:

1. **Exposure midpoint timestamp:** Each frame is stamped at its exposure midpoint (section 2.2), latched on hardware triggers synchronized over PTP or on external TTL sync lines, so that every camera's observation carries the time at which it was true.
2. **Kinematic interpolation:** The joint configuration at the camera's optical timestamp is interpolated between the two nearest MCU encoder records (the spline form is in section B.3).
3. **Alignment before serialization:** Containers such as MCAP, LeRobot (Cadene et al. 2024), and RLDS (Open X-Embodiment Collaboration et al. 2024) store multi-rate streams with whatever timestamps they are given, so the midpoint alignment must happen before the state-action pairs are written.

A tap and a timestamp are only as good as the loop that produced them. In teleoperation, still the standard way to gather high-quality physical trajectories, that loop runs through a human, and the interface between human and machine shapes the recorded dynamics long before the data reaches a model.

Prerequisite: Sub-microsecond PTP clock synchronization across distributed bus controllers is established in section 4.4.

5.4 Teleoperation Demonstration Costs

Teleoperation closes a feedback loop through the operator, the input device, the bus, the motor controller, and the environment (figure 5.3). A camera captures the workspace and streams it to a display; the operator sees the scene, decides on a correction, and moves an input device; the workstation maps that motion into a setpoint and sends it to the motor drives; the actuators accelerate the arm against inertia, friction, and contact, and the changed scene reaches the camera on the next cycle.

Each stage in this loop introduces latency that affects the timing of the correction. In a reference setup with a high-speed camera and a wired local network (illustrative; see the Reader Guide), display processing requires $t_{\text{disp}} = 50$ ms across exposure, H.264 decoding, and monitor refresh. Human visual perception and neuromuscular transmission introduce an operator response latency of $t_{\text{hum}} = 200$ ms for continuous path corrections. Command quantization, packetization, and bus transport to the robot controller add $t_{\text{cmd}} = 10$ ms. Actuator torque ramp-up and structural inertia impose a mechanical response latency of $t_{\text{mech}} = 80$ ms before the robot achieves the commanded acceleration. The **teleoperation round-trip latency** t_{loop} , the time from a contact event to the corrective actuator response, is their sum:

$$t_{\text{loop}} = t_{\text{disp}} + t_{\text{hum}} + t_{\text{cmd}} + t_{\text{mech}} = 50 \text{ ms} + 200 \text{ ms} + 10 \text{ ms} + 80 \text{ ms} = 340 \text{ ms}.$$

When a contact misalignment occurs at time t , the operator sees it at $t + 50$ ms, commands a correction at $t + 250$ ms, and the arm pushes back at $t + 340$ ms, while the arm keeps moving under momentum and contact forces. A logger that pairs the observation s_t with the command issued at the same instant pairs a state with an action chosen for a state 250 ms older. The policy learns that delay as deliberate behavior and over-corrects or oscillates when it runs autonomously at the control

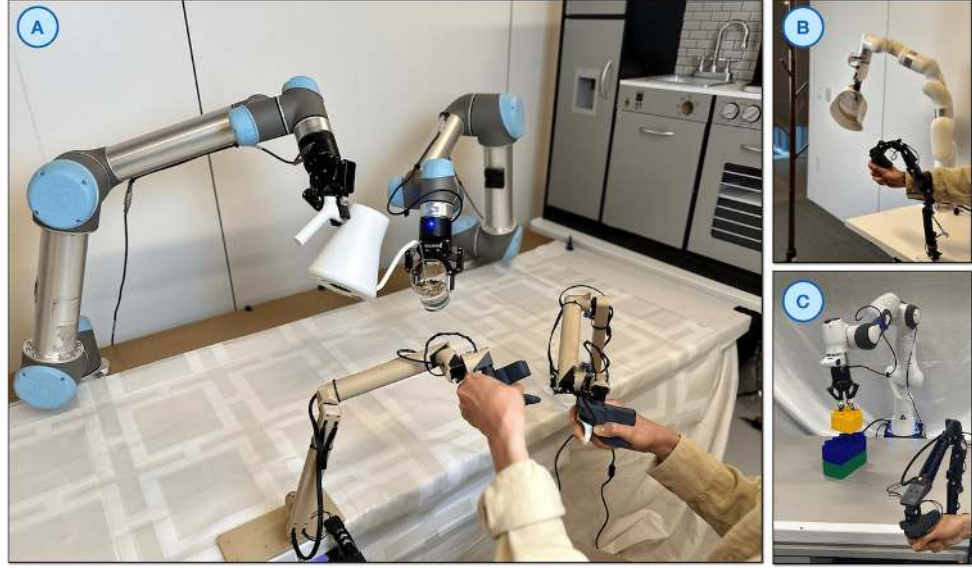


Figure 5.3: Kinematically isomorphic teleoperation rigs: Demonstration capture on three leader-follower rigs whose leader devices mirror follower kinematics (Wu et al. 2024). Because the leader shares the follower’s joint structure, the interface needs no inverse kinematics and inherits none of its singularities (mathematical dead-zones where joints align and lose mobility), preserving the operator’s direct joint-level coordination. The operator’s continuous-correction latency still enters the recording as phase lag, so the demonstration carries the operator’s delay as if it were policy.

rate. The round-trip latency is therefore a property of the dataset, not only of the rig, because it enters every demonstration as phase lag dominated by the operator’s own reaction time.

When contact force is reflected back to the operator’s hand in **bilateral teleoperation**, the same delay becomes a stability hazard. Anderson and Spong showed that transmission delay turns a passive link into an energy source (Anderson and Spong 1989). The delayed reaction force arrives out of phase with the operator’s hand, pushing it in the direction of motion and driving leader and follower into limit cycles against rigid contact (section B.8). Time-domain passivity controllers and wave-variable transformations bleed that energy off with virtual damping, and the damping low-pass filters the operator’s motion, attenuating the very contact cues the demonstration was meant to capture.

The interface thus filters what a demonstration can express. A force-reflecting leader lets the operator feel binding; a visual-only rate controller encourages short moves and pauses while the operator checks the video. A backdriven joint-space leader such as ALOHA, recorded at 50 Hz (Zhao et al. 2023), sits between them, conveying the leader’s mechanical feel without follower force. With any of these interfaces, operators slow down near contact, because a high-gain loop closed around t_{loop} would lose its phase margin. Table 5.1 lists these behaviors as artifacts to check on a given rig rather than as universal ranges.

Table 5.1: Teleoperation interface mechanisms for physical demonstration collection: Mapping and feedback determine what the operator can demonstrate, and the last column names the artifact to check on a given rig, task, and operator.

Interface and feedback	Command mapping	Contact information available to operator	Possible collection artifact to check
Force-reflecting bilateral leader	Leader motion mapped to follower joints; measured follower force reflected to leader	Vision and physical contact force, subject to link delay and fidelity	Contact corrections and oscillation when delayed force reflection loses passivity
Backdriven joint-space leader (ALOHA) (Zhao et al. 2023)	Leader joint positions copied to follower; demonstration recorded at 50 Hz	Vision and leader mechanism feel, without active follower force reflection	Joint-space posture preferences and unobserved follower contact forces

Interface and feedback	Command mapping	Contact information available to operator	Possible collection artifact to check
Rate-controlled 3D mouse	Device deflection mapped to Cartesian velocity, then inverse kinematics	Primarily visual feedback unless a separate haptic channel exists	Step-and-wait motion and zero-velocity pauses under delayed feedback
Tracked spatial controller	Tracked hand pose mapped to tool pose with clutching and scale	Visual feedback; optional vibration is not directional force reflection	Clutch discontinuities and pose-tracking gaps
Instrumented exoskeleton	Wearable joint motion retargeted to robot joints	Depends on whether active force feedback is fitted	Biomechanical calibration and retargeting bias

Whatever interface the operator holds, the rig must record every stream the session produces, including all four action taps of 5.2, and that recording has a bandwidth and storage budget of its own.

Napkin Math 5.1: Multi-camera ingestion and storage budgets

Question: What sustained ingestion rate and storage does one teleoperated door-opening session on the warehouse mobile manipulator generate when every camera and every axis is logged raw?

Parameters (camera geometry is datasheet; logging formats are chosen):

- **Navigation camera:** 4056 by 3040 px at 60 Hz, logged as 24-bit RGB (3 bytes/px).
- **Wrist camera:** 1440 by 1080 px at 60 Hz, logged the same way.
- **External cameras:** 2 RGB-D cameras viewing the door, color at 1920 by 1080 px and 30 Hz, with depth aligned to the color frame as raw uint16 millimeters (2 bytes/px).
- **Kinematics:** 9 axes (two base wheels, seven arm joints) logged at 1 kHz. Each step stores, for every axis, position and velocity (the measured response a_{meas}), measured torque, and the requested, mapped, and enforced commands as 32-bit values, plus one 64-bit timestamp, for 224 bytes per step.

Calculation:

1. **Raw video bandwidth:** the navigation camera streams 2219.44 MB/s and the wrist camera 279.94 MB/s. The external pair adds 373.25 MB/s of color and 248.83 MB/s of depth, 622.08 MB/s in all.
2. **Kinematic bandwidth:** 224 bytes per step at 1 kHz is 0.22 MB/s.
3. **Total sustained ingestion rate:** the streams sum to 3121.68 MB/s, about 3.12 GB/s.
4. **Hourly and shift storage consumption:** uncompressed telemetry accumulates at 11.24 TB per hour, or 89.9 TB across a 8-hour collection shift on one machine.

Systems insight: The navigation camera alone writes 4.0× what a SATA SSD sustains (550 MB/s). The full session fits within a Gen4 NVMe drive's 7 GB/s, but only by consuming storage at 11.24 TB per hour. Intra-frame video compression at an illustrative 8× brings the rate to about 390.2 MB/s (1.40 TB per hour) at the cost of encoder latency and artifacts in the frames the policy will train on. The kinematic stream, which carries all four action taps, is 0.007 percent of the bytes. The record that preserves the causal chain is the cheapest stream to log, so a logger that drops data under buffer pressure must never drop it first.

Storage bounds how much a session can record; yield bounds how much of it is worth training on. **Demonstration yield** ($Y_{\text{usable}} = \lfloor T_{\text{motion}}/t_{\text{cyc}} \rfloor \cdot R$) is the number of retained trajectory records a scheduled session produces after startup, calibration, operator rest, resets, and quality filtering have taken their share. Consider one illustrative session of the latch campaign, declared to last $T_{\text{sched}} = 60.0$ min. Startup, calibration, and operator rest leave an active window of $T_{\text{motion}} = 35.0$ min. Each attempt is one teleoperated opening of $t_{\text{att}} = 25.0$ s, followed by a re-latch of the door by its fixture of $t_{\text{rst}} = 15.0$ s ($t_{\text{cyc}} = 40.0$ s), so the session fits 52 attempts. In-trial failures (20 percent) and post-

collection filtering (15 percent) give a retention rate $R = (1 - 0.20)(1 - 0.15) = 0.68$, and the session yields 35.36 retained episodes, 14.73 min of validated data. That is a net session fraction, validated trajectory time over scheduled cell time, of 24.6 percent. A target of 5,000 retained demonstrations therefore costs 7,353 attempts and 141.4 h of scheduled cell time, $4.07\times$ the demonstration time the campaign keeps (table 5.2). A campaign is budgeted in validated trajectory time, never in facility hours.

Wear accumulates per cycle, not per hour. In an illustrative wear model of the arm's wrist, the strain-wave gear is rated for $L_{\text{gear}} = 5,000,000$ stress cycles and the internal cable bundle for $L_{\text{cable}} = 200,000$ bending cycles. Finding the handle, releasing the latch, and swinging the door add about $n_{\text{gear}} = 14$ torque reversals and $n_{\text{cable}} = 6$ flex cycles per opening, and the return stroke during the re-latch adds two more flex cycles. Failed attempts wear the joint as much as retained ones, so the bill is paid on all 7,353 attempts: 102,942 gear stress cycles, 2.1 percent of rated life, and 58,824 cable bends, 29.4 percent of rated life, for one campaign. The cable, not the gear, sets the replacement schedule. Three campaigns on one machine would consume most of the cable's rated life, and an intermittent conductor break corrupts the very sensor streams being recorded.

Table 5.2: Physical demonstration collection budget across a scheduled 60-minute session: Facility clock time, yield losses, and component wear by stage for one session of the latch campaign ($N_{\text{att}} = 52$ attempts of $t_{\text{att}} = 25.0$ s, reset $t_{\text{rst}} = 15.0$ s). The 20 s of active time left after the last full cycle is idle and has no row. Less than a quarter of the hour survives as retained demonstration data.

Session Stage / Activity	Allocated Duration (Δt)	Wall-Clock Share (%)	Physical Operation & Control State	Hardware Stress & Wear Mechanism	Net Trajectory Output ($N_{\text{episodes}} / T_{\text{yield}}$)
Cell Startup & Bus Sync	8.0 min (480 s)	13.3%	Bus synchronization (Ether-CAT/CANopen), safety watchdog verification, optical homing	Power-on electrical inrush; thermal warm-up in motor windings and sensor bridges	$N = 0$ ep, 0.0 min yield
Sensor Calibration & Nulling	7.0 min (420 s)	11.7%	Camera intrinsic/extrinsic sweep, joint-torque bias nulling, encoder homing	Static holding torque; sensor bridge thermal stabilization; zero dynamic gear wear	$N = 0$ ep, 0.0 min yield
Operator Rest & Ergonomics	10.0 min (600 s)	16.7%	Mandatory operator breaks to mitigate visual fatigue, cognitive saturation, and tremor drift	Passive standby; electromagnetic brakes engaged or zero-velocity holding current	$N = 0$ ep, 0.0 min yield
Physical Scene Resets	13.0 min (780 s)	21.7%	Door closed and re-latched by its fixture, arm return stroke (15.0 s/attempt)	52 return strokes $\times$ 2 bends = 104 cable flex cycles; zero contact payload	$N = 0$ ep, 0.0 min yield
Active Teleoperated Attempts	21.67 min (1300 s)	36.1%	Real-time human teleoperation guiding handle approach, latch release, and door swing (25.0 s/attempt)	52 $\times$ 14 = 728 torque reversals on harmonic drive; 52 $\times$ 6 = 312 cable flex cycles; contact loads below 15 N	$N = 52$ raw attempts, 21.67 min gross motion
In-Trial Failures (Labeled)	-4.33 min (-260 s)	-7.2%	Handle slip, latch bind, contact tripwire above 15 N (20% loss)	High-force contact transients; abrasive wear on gripper silicone pads	-10.4 ep from success yield; valid logs kept

Session Stage / Activity	Allocated Duration (Δt)	Wall-Clock Share (%)	Physical Operation & Control State	Hardware Stress & Wear Mechanism	Net Trajectory Output ($N_{\text{episodes}} / T_{\text{yield}}$)
Post-Collection QA Filtering	−2.60 min (−156 s)	−4.3%	Offline filtering: link latency > 100 ms, visual occlusion, operator hesitation (15% loss)	Zero hardware wear (offline computational validation pass)	−6.24 ep discarded
Net Validated Dataset Yield	14.73 min (884 s)	24.6%	High-quality synchronized multi-modal trajectories (s_t, a_t) admitted into training buffer	Consumes 2.1% of wrist gear life and 29.4% of internal flex cable life per 5000 episode campaign	$N_{\text{usable}} = 35.36$ episodes

Teleoperated demonstrations are bounded by what an operator can perceive through a display, coordinate through an interface, and stabilize against the loop delay, so the dataset reflects the human-interface composite rather than an unconstrained sample of the plant’s dynamics, and that bias sets the dataset’s coverage (section 5.5).

5.5 Collection Policy Coverage

A physical dataset is generated by executing actions on hardware and logging the resulting transitions. The logged samples define an **empirical state-action occupancy distribution** (Spencer et al. 2021) $d^{\pi_{\text{col}}}(s, a)$, and imitation learning minimizes prediction error weighted by it, so the learned policy π_{θ} is trained only on states where $d^{\pi_{\text{col}}}(s) > 0$. The dataset is not a sample of the physical state space. It is the region traced by the collecting policy π_{col} under its control biases, interface delays, and risk tolerance.

Consider an illustrative contact task, a gripper aligning with a workpiece edge, whose deployment requires contact angles over $\theta \in [-15.0^\circ, +15.0^\circ]$ and normal forces up to 20.0 N. Binned at 1.0° and 1.0 N, that is a grid of 600 cells. Two collectors each record 1,000,000 transitions with identical sensors. A cautious teleoperator holds contact within $\pm 2.0^\circ$ of perpendicular under 4.0 to 6.0 N and fills 8 cells, leaving 98.7 percent of the grid empty. An exploratory policy approaches across the full angle range, slips, and loads the contact between 1.0 and 18.0 N, filling 510 cells and leaving 15.0 percent empty. The byte counts are identical; the questions the two datasets can answer are not.

An action label exists only because the collecting policy visited that state and issued a command. At a state with zero visits, the dataset holds neither a preferred action nor evidence that none exists, and the loss function cannot tell an unstable configuration from a safe but unvisited one or a damaging one. A policy may generalize there, fail, or be stopped by a separate safeguard, and the dataset alone cannot say which; only targeted collection, expert relabeling, or a refusal of autonomous authority settles the cell. Suppose a policy trained on the cautious data is perturbed by a friction variation to $\theta = +4.5^\circ$, outside its 8 demonstrated cells. Its network now evaluates an input no demonstration constrains, and the unvalidated action carries it to $+6.0^\circ$ at 11.0 N one control step later, a state from which the archive holds no correction. Each further action moves it farther from the corridor until the gripper jams against the fixture. That is endogenous covariate shift (5.1) acting on a coverage gap.

Measuring coverage against the extremes of the collected values is circular. A collector that logs forces only between 4.0 and 6.0 N reports complete coverage of that interval. Coverage must instead be evaluated against an external **scenario ledger**, a table of the conditions the deployment requires, divided into cells. The scenario ledger is the **operational design domain** (ODD) written down: the object poses, surface conditions, payloads, speeds, and disturbances in which the machine is meant to operate. A dataset supports the part of the ODD that its occupancy join covers; the rest is its known absence.

Unsupported regions are found by a **support envelope relational join**, which cross-references the empirical occupancy $d^{\pi_{\text{col}}}(s, a)$ (where the machine went) against the scenario ledger $\mathcal{S}_{\text{req}}$ (where it needs to go). For each ledger cell the join reports the transition count N_{cell} and so partitions the ODD

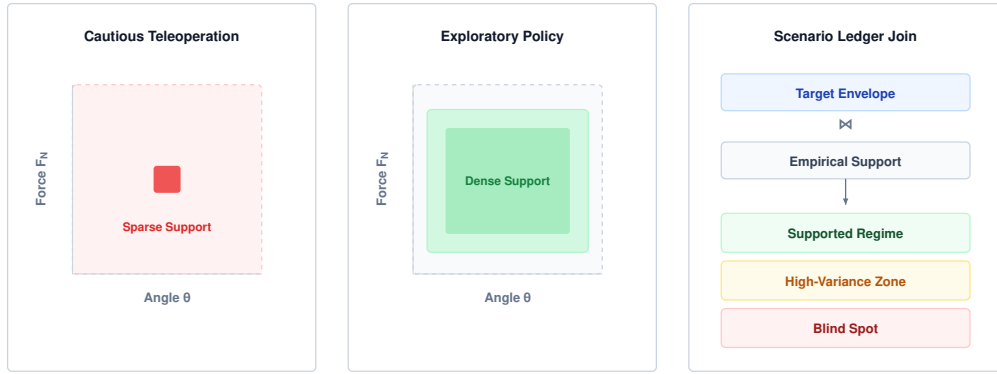


Figure 5.4: Collector occupancy and scenario join: State-action visitation for a cautious teleoperator against an exploratory policy, joined against the scenario ledger the deployment requires. The cautious collector concentrates transitions into a narrow cluster and leaves 98.7 percent of the required cells unvisited while reporting near-zero validation loss, showing that statistical convergence says nothing about physical coverage. The relational join partitions the ODD into supported regimes, high-variance transition zones, and zero-support blind spots where autonomous control authority must be refused.

into supported regimes ($N_{\text{cell}} \geq N_{\text{min}}$), high-variance transition zones, and zero-support blind spots ($N_{\text{cell}} = 0$). A blind spot has no training supervision, so the permission path must decide whether the policy may operate there at all; low-count cells need uncertainty and scenario-specific checks.

Validation loss cannot detect the deficit. A held-out split drawn from the same trajectories shares the occupancy $d^{\pi_{\text{col}}}$, so the cautious dataset can reach near-zero validation loss while 98.7 percent of the cells the ODD requires hold no examples (table 5.3, figure 5.4). Only the ratio of supported ledger cells to required cells, computed before deployment, measures coverage.

Table 5.3: Empirical coverage and validation loss metrics across operational scenario bins: Relational join metrics comparing cautious teleoperation against exploratory policy coverage.

Metric	Cautious Teleoperation ($d^{\pi_{\text{teleop}}}$)	Exploratory Policy ($d^{\pi_{\text{expl}}}$)	Systems Consequence
Recorded Transitions	1,000,000 transitions (100 Hz)	1,000,000 transitions (100 Hz)	Equivalent dataset scale
Operational Grid Coverage	8 / 600 bins (1.3%)	510 / 600 bins (85.0%)	Supported cells of the ODD
Zero-Count Blind Spots ($N_{\text{cell}} = 0$)	592 / 600 bins (98.7%)	90 / 600 bins (15.0%)	Unmonitored risk volume
Held-Out Validation Loss (L_{val})	0.001 rad	0.012 rad	Validation loss is non-diagnostic
Closed-Loop Compounding Drift	Divergence beyond $\pm 2.5^\circ$	Bounded recovery across $\pm 15^\circ$	Physical policy stability
Training Admission	Refuse (require targeted collection)	Admit (blind spots stay cataloged)	Scenario ledger audit gate

The latch campaign shows the join on the running machine. Every retained episode approaches the strike plate at the guarded 0.03 m/s with the plate within $\pm \$0.5$ mm of nominal, and every retained success stays below the 15 N tripwire. Against a ledger holding only the nominal plate at the guarded speed, the dataset looks complete. Against the ledger the door needs, three plate positions (nominal and $\pm \$3$ mm) crossed with two approach speeds (0.03 m/s and 0.10 m/s), the same data fills one cell of six, and every displaced plate and every approach at 0.10 m/s holds zero demonstrations. Nothing inside the dataset differs between the two answers; only the external ledger does. The five empty cells therefore travel with the dataset into chapter 6 as its known absences, and until targeted collection fills them the machine owes those conditions a refusal rather than a guess.



Checkpoint 5.1: Empirical occupancy and scenario ledger joins

Before evaluating policy coverage across an operational design domain, verify your understanding of dataset occupancy and scenario ledger accounting:

- ☐ **Empirical support vs. validation loss:** Can you explain why an imitation policy can achieve near-zero validation loss on held-out test data while leaving 98 percent of the ODD's required cells unsupported?
- ☐ **Unsupported states:** What can an unvisited scenario cell establish about policy behavior, and what additional evidence or refusal decision is needed?
- ☐ **Scenario ledger relational joins:** Can you distinguish supported nominal regimes, high-variance transition zones, and zero-support blind spots when joining telemetry against an operational scenario ledger?

Recording more nominal demonstrations along the corridor cannot fill a blind spot. Reaching boundary states takes supervisory takeovers or deliberate perturbation, and the data they produce raises causal problems of its own (section 5.6).

5.6 Intervention and Recovery Data

When a hazardous or boundary state does not appear in a training dataset, an engineer cannot conclude that the physical system avoids that state naturally. An absent state in the empirical log corresponds to one of four distinct physical mechanisms during data collection:

1. **Unvisited nominal path:** The collecting policy never visited the state because nominal rollouts remained entirely within the primary task corridor. This absence represents unmonitored nominal operation, detected by cross-referencing trajectory logs against the scenario ledger.
2. **Permission-path interception:** An active safety barrier or deterministic reflex in the permission path detected an impending boundary violation and truncated the motion before the state was reached, as the 15 N contact tripwire does to a latch attempt. This absence is detected directly in the permission path's logs, which record the barrier trip.
3. **Human supervisory takeover:** A human safety operator perceived a developing hazard and disengaged the autonomous policy before contact occurred. This absence is detected through the takeover disengagement flag on the vehicle or manipulator bus.
4. **Post-collection censoring:** The machine actually entered the hazardous state and collided with an obstacle or jammed a joint, but an automated post-processing script discarded the episode as corrupt or incomplete. This absence is not detected by dataset inspection, because the failure transitions were purged before the training arrays were written to disk.

Each mechanism produces an identical zero count in a scenario cell, yet treating them identically leads to incorrect conclusions about policy stability.

Human takeovers and automated disengagements introduce a selection trap into fleet data. An intervention occurs precisely when the autonomous policy has entered a region of state space that it handles poorly, causing tracking error or model uncertainty to exceed an operational threshold. Because disengagements are triggered by failing rollouts, the collected intervention data is not drawn from the nominal state distribution of the task. A trajectory recorded after such a takeover captures an expert recovering from an edge case disturbance. The distribution of states visited during these rescues, denoted d^{takeover} , is conditioned on policy failure rather than standard task execution. Treating intervention logs as if they were drawn from the nominal distribution d^{π^*} distorts the training distribution toward states that only exist because the prior policy failed.

Appending a rescued episode directly to a behavioral cloning dataset can make unsafe policy commands look like expert demonstrations. Before takeover, the learned policy may issue commands that increase heading error or contact force; after takeover, the expert issues corrective commands from states the learner visited. Those drift states are valuable for corrective aggregation when an expert can label a safe action there.

Separating these labels requires locating the **divergence point** where the autonomous policy departed the nominal path (extending section 4.1), rather than treating takeover as the first error. An

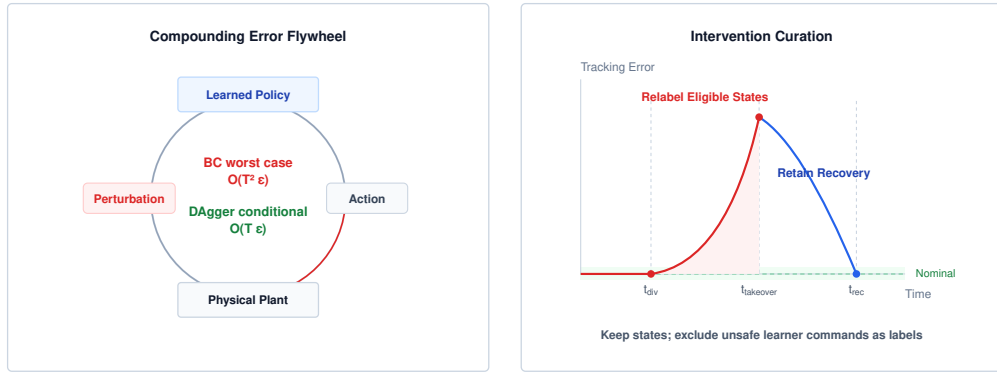


Figure 5.5: Covariate shift and intervention curation: The left panel contrasts conditional behavioral-cloning and DAgger cost-bound orders, not measured trajectory drift. The right panel shows learner-visited states between divergence and takeover. Keep these states for expert relabeling when available; exclude the learner’s unsafe commands from expert-action targets, and record takeover recovery separately. Based on Ross et al. (2011) and Spencer et al. (2021).

out-of-loop supervisory takeover (section 14.2 budgets its latency) takes longer than the teleoperation loop’s continuous correction (section 5.4); at an illustrative 100 Hz that delay spans many control cycles, all logged under the learner’s authority.

Detecting t_{div} in offline ingestion pipelines requires searching backwards in time from the physical override timestamp across a synchronized pre-trigger telemetry buffer:

$$\hat{t}_{\text{div}} = \min \left\{ t \in [t_{\text{takeover}} - \Delta t_{\text{lookback}}, t_{\text{takeover}}] \mid \begin{aligned} &\| \mathbf{x}_{\text{plant}}(t) - \mathbf{x}_{\text{ref}}(t) \|_2 > \delta_{\text{tol}} \\ &\vee \text{Tr}(\Sigma_{\pi}(t)) > \sigma_{\text{thresh}}^2 \end{aligned} \right\}.$$

The pipeline scans a stated lookback window (here $\Delta t_{\text{lookback}} = 1.0\text{--}2.0$ s) for the first threshold crossing, using calibrated tracking residuals and, if available, policy uncertainty. The thresholds are scenario-specific.

Intervention curation separates learner commands, expert labels on learner-visited states, and post-takeover recovery at the divergence and takeover timestamps. It keeps the learner commands between divergence and takeover in the audit log with their true authority and safety flags but never as expert imitation targets, admits the states they reached to corrective aggregation only when a qualified expert can label a safe action there, and records the recovery after takeover as a separate expert sequence. The states that reveal the policy’s coverage gap are never erased.

The states between divergence and takeover are where principle 2 becomes concrete. The learner’s own actions chose them, no demonstration visits them, and they are the states a behavioral-cloning policy reaches after its first error, which is why their absence is expensive. Cloning on expert states alone admits only the quadratic worst-case bound of equation 5.1, $O(T^2\epsilon)$.

In **corrective aggregation**, such as DAgger (Ross et al. 2011), an expert labels states visited by the learner, including eligible pre-takeover drift states. The learned policy can attain a cost bound linear in horizon and in its loss on learner-visited states, $O(T\epsilon)$.⁷

For $T = 500$ steps (5.0 s at 100 Hz) and $\epsilon = 0.01$, the expressions $T^2\epsilon = 2500$ and $T\epsilon = 5$ illustrate different horizon dependence. Because both terms omit constants and rest on different error assumptions, their ratio is not a $500\times$ improvement. With per-step cost at most one, cumulative task cost itself cannot exceed 500 in this example (figure 5.5).

Training successive policy generations on drift-window learner commands mislabeled as expert actions can create a retraining cascade:

1. **Generation 1:** A policy operates with a slight calibration error, causing a robotic arm to drift toward a joint torque limit until a human operator grabs the override pendant.
2. **Generation 2:** If the drift-window commands are presented as expert labels, the policy learns to repeat them at similar off-nominal states. On this rig, that could increase excursions toward the torque limit before a correction.

⁷ | **Corrective Aggregation Error Bound:** DAgger (Ross et al. 2011) queries an expert on learner-visited states and aggregates those labeled states. Its linear-horizon result depends on the expert oracle, online-learning regret, bounded cost, and the measured loss of the resulting policy. See section A.5.

3. **Generation 3:** Operators may then intervene earlier. Intervention rate can rise while validation loss on the mislabeled archive falls, making the offline metric look better as physical performance worsens.

To detect such a cascade, the permission path emits an **intervention record** for each takeover or barrier trip,⁸ the training-side view of the forensic authority record that section 14.8 builds. The intervention record needs more than a binary flag on video: a pre-trigger buffer long enough for the declared lookback (two seconds in this example), synchronized observations and action taps, a monotonic override timestamp and authority ID, and the subsequent recovery sequence. Without the pre-trigger evidence, the pipeline cannot locate t_{div} reliably or separate unsafe learner commands from states eligible for expert relabeling.

⁸ | **Fault-Event Logging:** This collection design preserves pre-trigger state, action authority, fault codes, and actuator-current samples across an override, with integrity checks on the stored record. Such telemetry helps distinguish policy drift from hardware or environment faults.

Checkpoint 5.2: Intervention curation and compounding error bounds

Before curating human intervention logs for policy retraining, verify your understanding of compounding error dynamics and timestamp attribution:

- ☐ **Takeover vs. divergence timestamps:** Why retain learner-visited drift states for possible expert relabeling while excluding their unsafe learner commands from expert imitation targets?
- ☐ **Retraining cascade progression:** Can you describe how training successive policy generations on uncurated intervention logs accelerates policy degradation despite improving training metrics?
- ☐ **Conditional error bounds:** Which assumptions distinguish the behavioral-cloning quadratic worst-case cost bound from DAGger’s linear-horizon result?

Even with expert labels on learner-visited states, the resulting demonstration archive remains tethered to the kinematics and dynamics of the collecting machine. Pooling demonstrations across robot arms, bases, and end-effectors raises the cross-embodiment questions in section 5.7.

5.7 Cross-Embodiment Transfer

Data from one physical machine does not represent the task. Every recorded trajectory binds the physical characteristics of the collecting hardware into its numerical values, including link lengths, joint velocity limits, sensor optical paths, communication delays, and contact surface compliances. When an engineer saves an observation-action trace (o_t, a_t) , the record does not capture an abstract manipulation skill. It captures the **embodiment contract** established between that specific machine and its environment. This contract encompasses the sensor geometry and optical calibration, the sampling period and transport latency of the perception bus, the coordinate frames and kinematic limits of the actuators, the torque-speed saturation curves of the motors (including reflected rotor inertia $N^2 J_{\text{rotor}}$ in geared transmissions, as established in section 2.4), and the friction and compliance of the contact interface. Altering any parameter in this contract changes the mapping between commanded inputs and physical state transitions.

Where the embodiment contract describes one machine, the **cross-embodiment compatibility contract** decides whether records made under one embodiment contract may drive another. It is the formal geometric, dynamical, and temporal specification (such as those underpinning the Open X-Embodiment (Open X-Embodiment Collaboration et al. 2024) and DROID (Khazatsky et al. 2024) datasets) that keeps a force profile or spatial velocity learned on a heavy industrial arm from reaching a lightweight desktop robot unscaled. It defines the forward kinematics mapping, control rate normalization (Δt), and actuator effort bounds required before multi-robot trajectories can be transferred to a target physical plant.

Engineering a data reuse pipeline requires separating task-level invariants from embodiment-specific realizations. A task-level invariant is a geometric or dynamical condition required by the environment, such as bringing a compliant probe into contact with a terminal pad at an angle within 5° of the surface normal while maintaining a normal force between 2.0 N and 5.0 N. An embodiment-specific realization is the sequence of joint encoder counts, pulse-width modulation

duty cycles, bus transport delays, and camera pixel rasters that achieved that condition on a specific test rig. While the contact target exists in world coordinates, the recorded dataset contains only the realization. A downstream model trained directly on those raw records learns the idiosyncratic dynamics of the source rig, including its structural flex, drive backlash, and camera mount vibration.

In an illustrative example, consider transferring a dataset recorded on Machine A to Machine B for an identical touch-and-grasp task:

- **Machine A** uses a parallel-jaw gripper with a 0 to 100 mm stroke range, a 50 Hz command rate (20 ms period), a continuous force limit of 40 N, and a wrist camera offset 45 mm behind the tool center point with a 15 ms transport delay.
- **Machine B** uses a compact gripper with a 0 to 60 mm stroke range, a 20 Hz command rate (50 ms period), a continuous force limit of 15 N, and a wrist camera offset 20 mm behind the tool center point with a 35 ms transport delay.

Mapping the dataset from Machine A through the limits of Machine B reveals the points where transfer fails mechanically. For an object width of 75 mm, Machine A opens to 80 mm, a command that exceeds the mechanical travel of Machine B by 20 mm and drives its leadscrew against the physical endstop. During the hold phase, Machine A applies 30 N of clamping force. That same command exceeds Machine B's continuous rating of 15 N by a factor of two, driving the motor amplifier into current saturation. In the time domain, subsampling the 50 Hz trajectory onto Machine B's 20 Hz bus aliases high-frequency contact transitions and introduces a zero-order hold phase lag of 15 ms (section B.3.5). Combined with the 20 ms increase in camera transport delay (35 ms – 15 ms), Machine B perceives contact events 35 ms later than Machine A does. At an approach velocity of 0.10 m/s, this added sensor-to-actuation latency carries the gripper 3.5 mm past the intended contact plane before the first corrective command can be dispatched, while the 25 mm camera mounting disparity introduces visual parallax that shifts the apparent position of the workpiece boundary in the image frame.

Standard machine learning pipelines reconcile these physical discrepancies by normalizing observation and action channels to $[0, 1]$ or $[-1, 1]$. Normalization creates a cosmetic alignment of numerical ranges, leaving physical incompatibility intact. If an opening command is normalized by dividing by the maximum stroke, a normalized command $u = 0.80$ corresponds to 80 mm on Machine A, which clears the 75 mm object. On Machine B, that same normalized command $u = 0.80$ commands an opening of 48 mm (0.80×60 mm), causing the fingers to collide with the object during the approach trajectory. Normalization rescales numbers, but it cannot expand actuator stroke, cannot lower electrical resistance, and cannot eliminate transport latency. When the data distributions of both embodiments are plotted in normalized space, the demonstration trajectories appear identical, yet only the subset lying within the physical intersection (openings under 60 mm, forces under 15 N, and command bandwidths under 20 Hz) can execute on the target hardware without saturation or impact damage.

Pooled corpora raise these questions at scale. While scaling demonstration volume within a single narrow workcell yields rapid in-distribution saturation (> 85 percent success within 15 hours on ALOHA (Zhao et al. 2023)), out-of-distribution (OOD) generalization across novel scenes, visual distractor textures, and unmodeled object variations remains poor (22 percent, creating a 66 percentage point brittleness envelope). Overcoming this failure requires expanding beyond single-cell capture into in-the-wild multi-scene collection (such as the 564 kitchen and household environments in DROID (Khazatsky et al. 2024) or the multi-institution BridgeData v2 (Walke et al. 2023)) and pooling heterogeneous kinematics across embodiments (such as the 22 robot platforms in Open X-Embodiment (Open X-Embodiment Collaboration et al. 2024) and foundation VLAs like π_0 (Black et al. 2024)). As mapped in figure ??, scaling multi-scene and cross-embodiment demonstration hours systematically contracts this generalization gap from 66 percentage points down to 13 percentage points.

Stroke, force, rate, and viewpoint mismatches arise in any pool exactly as in the two-gripper example. Two further mismatches arise only when the pool mixes kinematic topologies:

1. **Action space incompatibility:** A 7-DoF arm records joint positions $\mathbf{q} \in \mathbb{R}^7$, a bimanual workstation records $\mathbf{q} \in \mathbb{R}^{14}$, and a mobile base logs planar velocities $(\dot{x}, \dot{y}, \dot{\theta})$. Because joint-space

labels do not transfer across topologies, multi-embodiment models commonly project actions into a Cartesian end-effector delta:

$$\Delta \mathbf{x}_t = (\Delta x, \Delta y, \Delta z, \Delta \phi_{\text{roll}}, \Delta \phi_{\text{pitch}}, \Delta \phi_{\text{yaw}}) \in SE(3), \quad a_{\text{gripper}} \in [0, 1].$$

The delta standardizes geometry and discards actuator dynamics. A displacement that is trivial for an industrial arm near its nominal configuration can carry a lightweight arm through a kinematic singularity (section B.4.1), where the required joint velocities grow without bound and overcurrent protection trips.

2. **Null-space projection:** Mapping a 7-DoF arm into a 6-DoF Cartesian delta projects out the one-dimensional null space, the elbow swivel angle ψ . On the target, the inverse kinematics solver resolves that redundancy by its own criterion, such as joint-limit avoidance or manipulability. Without the demonstrator's elbow posture, the target arm can drift into joint limits or sweep its elbow into fixtures during motions that were collision-free in the source data.

Reusing physical data across embodiments requires a formal compatibility check at the ingestion boundary before any sample enters the training buffer. The ingestion pipeline must transform the recorded trace through forward kinematics to evaluate coordinate consistency (Lynch and Park 2017), verify that all commanded joint velocities and accelerations lie within the target machine's dynamic limits (Tedrake 2023), and verify that perceptual features match the target sensor's field of view and resolution. Actions that exceed kinematic reach or actuator effort limits cannot simply be clamped at execution time; clamping distorts the velocity profile and breaks the causal relationship between observed errors and corrective motions. Samples that pass the compatibility check are admitted; samples that can be geometrically re-targeted through inverse kinematics are relabeled; and samples that demand unavailable physical authority or unobservable states are rejected. An unverified ingestion pipeline across an embodiment boundary injects invalid dynamics into the model's training distribution. The resulting policy does not fail because the neural network underfitted the training loss. It fails because the training dataset contained actions that were physically sound only on the machine that collected them.

A compatibility check guards one boundary, but data collected on the machine itself can go wrong without crossing an embodiment. Operator drift, calibration shifts, rig mismatch, and misclassified failures corrupt physical data while leaving the tensor dimensions untouched (section 5.8).

5.8 How Datasets Go Wrong

Beyond the absences the latch ledger declares (the faster approach and the displaced plate), the harder failures lie inside the one supported cell, where the retained demonstrations can be wrong in ways no tensor check sees, because every metric computed on a dataset evaluates only the states it contains.

The first is operator drift. Over the campaign's 141.4 h of scheduled cell time, an operator who began by moving deliberately, with corrective sub-movements and pauses at the strike plate, develops muscle memory and starts rounding corners and swinging the door on momentum. The late demonstrations rely on timing the machine cannot reproduce under closed-loop delay at deployment. Drift is detected by partitioning the dataset into temporal windows and comparing episode duration, corrective jerk count, command magnitude, and intervention rate across them. Pooling drifted windows as one identically distributed sample teaches a policy that hesitates or lunges depending on which stage of the operator's learning curve it happens to imitate.

The second is a change in the machine between recording runs. File containers such as HDF5, Zarr, or Parquet enforce tabular schema consistency and conceal everything else. An arm shut down overnight, recalibrated, or bumped by a technician changes its encoder zero offsets or a camera's extrinsics, and a camera mount shifted by 3 mm moves the workpiece in the image while the array keeps its shape and type. Clock synchronization can drift across controllers in the same way, shifting the pairing between what the arm sees and what it feels by tens of milliseconds (section 4.4).⁹ figure 5.6 shows the visual half of the check, in which the robot's kinematic model, rendered from its joint encoders, is projected over each camera stream so that an extrinsic shift shows up as a gap between wireframe and arm. The recorded half joins every episode against an audit trail of calibration identifiers and clock-health metrics, since a dataset whose trajectories do not share one valid calibration forces the learner to fit conflicting mappings in the same coordinates.

⁹ | **Host-Assigned Timestamps:** Unlike PTP-synchronized Ethernet, buses without hardware timestamping, such as USB and classic CAN, leave the host to assign arrival times, which adds driver latency and scheduling jitter to every sample.

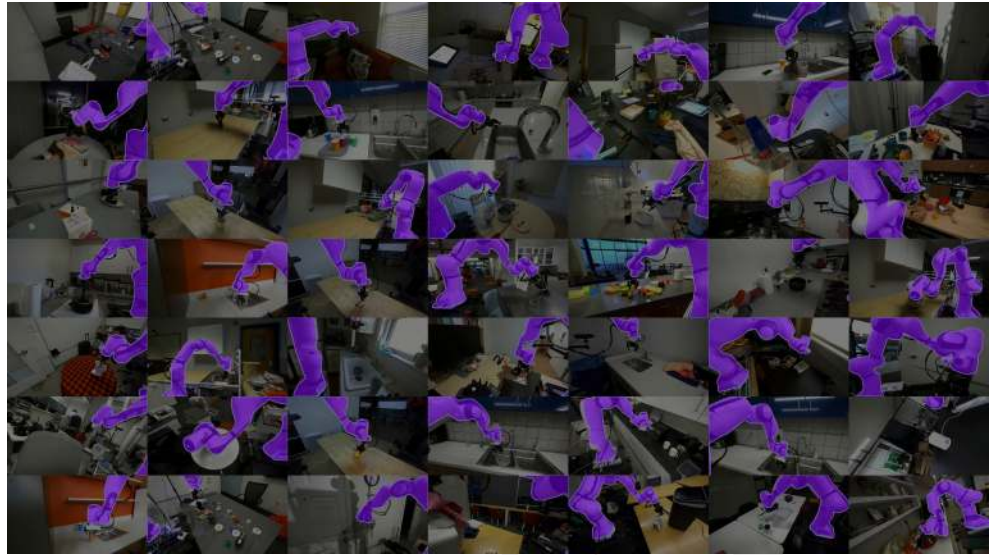


Figure 5.6: Multi-camera extrinsic calibration verification: Six DROID collection scenes with the robot’s URDF mesh rendered from joint encoders directly over the camera stream (Khazatsky et al. 2024). The overlay turns an invisible failure into a visible one, since a bracket displaced by a few millimeters shows up as wireframe-to-arm discrepancy before any corrupted trajectory reaches the training pipeline. Immutable calibration hashes in the dataset record are what make that check auditable after the fact.

The third is a mismatch between nominally identical rigs, which the fleet meets as soon as a second mobile manipulator records latch demonstrations for the first to learn from. Two arms of the same model can differ in gearbox wear, thermal behavior, or firmware filters. In an illustrative comparison, a new arm shows 0.02° of backlash and a worn one 0.35° with a 22 ms torque-response lag, and the worn arm’s demonstrations carry compensating lead and gain that oscillate on the new one. A record of each arm’s embodiment contract (kinematics, actuator serial numbers, firmware versions, and a dynamic model) bound to every file lets ingestion exclude trajectories that fail the target’s compatibility contract.

The fourth is conflating failed attempts with failed recordings. A failed attempt is a complete physical sequence that missed the objective; discarding it creates survivorship bias and strips the dataset of the error states a stabilizing policy needs. A failed recording is a logging fault, such as a dropped packet, a corrupt frame, or a buffer overrun; keeping it without accounting presents irregular sample periods as a constant control interval. Ingestion therefore reconciles every attempt as a retained success, a retained failure, or an excluded recording with its error code. In the latch campaign, an attempt ended by the contact tripwire is a failed attempt and stays in the archive, while a recording whose force channel dropped packets leaves it with its error code.

A unit contract can fail just as silently, and no format check will notice (5.1).

⚠ War Story 5.1: Mars Climate Orbiter unit mismatch (1999)

Context: During its nine-month cruise to Mars in 1998 and 1999, the Mars Climate Orbiter fired small thrusters to desaturate its reaction wheels, and the navigation team on the ground folded the resulting velocity changes into the trajectory models used to plan each correction maneuver (Stephenson et al. 1999).

Mechanism: A ground software file (called “Small Forces” in the investigation) reported thruster impulse in pound-force-seconds ($\text{lbf} \cdot \text{s}$), while the trajectory models that consumed it assumed newton-seconds ($\text{N} \cdot \text{s}$) as the interface specification required. Because $1 \text{ lbf} \cdot \text{s} = 4.45 \text{ N} \cdot \text{s}$, every desaturation event entered the models too small by a factor of 4.45. The values were well formed, in range, and of the expected type, so no schema check could flag them.

Impact: The accumulated error was never resolved during cruise. At orbit insertion the spacecraft passed about 57 km above the surface instead of the planned 226 km and was lost.

Response: The Mishap Investigation Board traced the loss to the missing unit conversion in the ground file and recommended that the Mars Polar Lander project verify consistent units throughout design and operations and audit all data transferred between JPL and Lockheed Martin Astronautics for compliance with the interface specification, along with stronger navigation review and anomaly escalation.

Systems lesson: A number carries its meaning only together with its units and its frame. A robot dataset runs the same risk on every action tap and every calibration. A record that stores a value without the units, frame, and calibration identifier it was produced under lets a mismatched rig pass every tensor check, which is why the provenance record of section 5.9 binds each tap to them.

Beyond silent coordinate and unit mismatches, an even more perilous failure occurs when an autonomous policy encounters a situation completely absent from its training distribution. While a software unit mismatch accumulates numerical error, an unvisited state forces a learned policy to interpolate or select nominal primitives outside its support envelope. When sensory occlusion renders the true physical state unobservable, the resulting execution can convert a survivable incident into physical catastrophe (5.2).

War Story 5.2: Cruise robotaxi pedestrian dragging (2023)

Context: On October 2, 2023, in San Francisco, an autonomous robotaxi operated by Cruise LLC was traveling in an urban lane when an adjacent human-driven car struck a pedestrian, launching her across the lane divider directly into the robotaxi's path (National Highway Traffic Safety Administration, Office of Defects Investigation 2023; Cruise LLC 2023; Quinn Emanuel Urquhart and Sullivan, LLP 2024).

Mechanism: The robotaxi executed a rapid emergency stop within 0.46 s at 1.4g, coming to rest over the pedestrian pinned beneath the chassis. However, the collision detection system misclassified the event as a lateral side-swipe rather than a collision with an obstacle under the vehicle. Because roof-mounted LiDAR and perimeter cameras had an acute physical blind spot beneath the floorpan, the pedestrian became unobservable. The learned driving policy, trained on nominal fleet demonstrations where pulling out of traffic after an incident is standard practice, selected a nominal pull-over primitive to clear the travel lane. The training distribution contained zero demonstrations of post-collision states with humans lodged under the chassis ($d^{\pi_{\text{col}}}(s) = 0$).

Impact: Executing the ungrounded pull-over primitive outside its training support envelope, the robotaxi accelerated forward and dragged the trapped pedestrian for 20 feet (6.1 meters) at up to 7 mph (11 km/h), inflicting severe secondary trauma before coming to a final halt.

Response: Regulatory agencies immediately suspended commercial deployment permits. The company grounded its entire fleet, overhauled its perception and safety software, and instituted hard physical governors requiring positive post-collision interlocks and teleoperation confirmation before any vehicle re-engagement.

Systems lesson: The Cruise incident (figure 5.7) demonstrates the mortal danger of missing support in physical AI datasets. Outside the empirical support envelope of demonstrated states, policies do not fail into safe passivity—they interpolate nominal primitives that convert a survivable incident into a fatal interaction. Datasets must bound their explicit scenario support hulls, and architectures must enforce deterministic control refusal when sensor occlusions and post-impact uncertainty violate support boundaries.

When a coordinate failure or unobserved scenario blind spot passes ingestion, loss curves do not reveal it. The network averages conflicting demonstrations or hallucinates actions outside its training support, converging on training loss while generating dangerous physical behaviors at deployment.

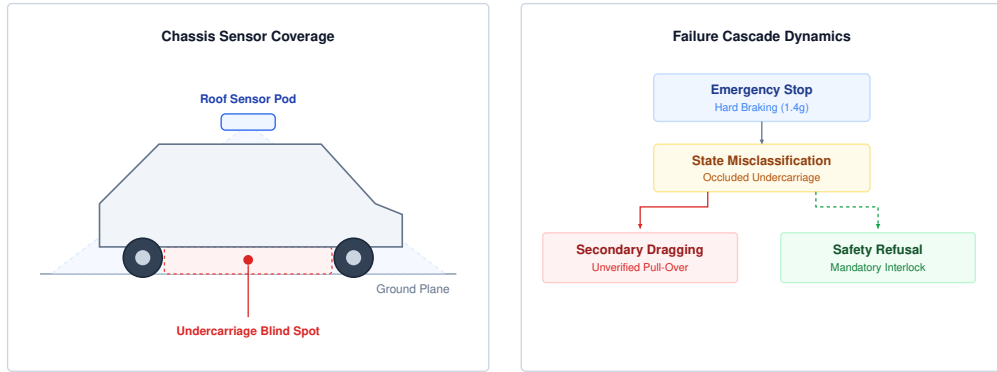


Figure 5.7: Cruise undercarriage blind spot failure: The test vehicle’s roof-mounted sensors survey the perimeter at and above bumper level, leaving a blind spot beneath the floorpan, and the failure cascade that followed. The first stop worked, halting in 0.46 s at 1.4g. What turned a collision into a dragging was the recovery: the state was misclassified as a side-swipe, the nominal pull-over primitive engaged, and drive torque was reapplied without any positive check for undercarriage clearance.

No operation on a tensor can verify that the machine was calibrated, that the operator did not drift, or that unvisited regions are safe, so the collection history must be recorded as rigorously as the features, in the schema of section 5.9.

5.9 The Dataset Schema

Consider an illustrative audit of two files from the latch campaign, each holding the same number of state-action pairs at the same logging rate. File A comes from four authorized operators across several days, reports no permission-path clamps, and shows the same fingertip pads throughout. File B comes from one operator, logs 620 steps where the joint velocity limit clamped the requested motion, and spans a force-sensor re-zeroing midway through the session without a recalibration of the camera extrinsics. A validation script that checks tensor shapes and value ranges accepts both, yet File A can support a claim about the supported cell and File B mixes operator intent with permission-path action across an unrecorded change in the body. Nothing in the arrays tells the two apart, so the distinction has to live in a record of how each file was collected.

The **demonstration provenance record** is that record, binding each demonstration to the body and conditions that produced it and stating coverage as the dataset’s join against the scenario ledger, never as the range of values the collector happened to log. It consumes the handoff record of section 4.8, keeping the operating conditions and clock domains under which the collecting machine’s budgets hold, and adds its own fields: the identity, calibration, and contact wear of the individual plant that produced each demonstration, the session and its timing, the four action taps, the collection denominators, a digest of the scenario ledger the data was collected against, and the ledger’s supported cells and known absences.

Every trajectory in the training corpus must identify its collection source, collector policy, and execution authority. When data originates from human teleoperation, the record logs the operator identifier, the interface hardware, and the transport timing, such as a 6-DoF leader arm sampled at 100 Hz with 0.8 ms of input jitter. When data originates from an autonomous or mixed controller, the record stores the exact policy checkpoint, the planner version, and the active enforcement configuration, including Cartesian bounding boxes, joint velocity limits, and contact-force clamping thresholds. Trajectories must also record the active retention rule, documenting whether an episode was committed automatically upon task completion, retained after an explicit operator review, or flagged for inclusion as an instructive failure recovery.

A physical machine cannot be identified by a generic model name. The data record specifies the individual plant through its extrinsic and intrinsic sensor calibrations, action contract, motor drive firmware revisions, and component wear state. Replaceable contact surfaces alter the governing dynamics. A gripper fitted with compliant 30 A durometer silicone pads produces different contact compliance and friction from the same gripper fitted with worn 60 A polyurethane pads under iden-

tical joint commands. Maintenance milestones, such as cable retensioning, drive belt replacements, gearbox swaps, and camera remounts, are logged directly into the machine state record. If an arm underwent joint recalibration between morning and afternoon collection shifts, the dataset must expose that boundary so downstream training does not attempt to resolve two conflicting kinematic mappings into a single policy.

Accounting for dataset volume requires explicit denominators across both time and trial counts, because a count of retained frames or successful episodes hides the yield of the process that produced them. The record therefore tracks scheduled cell time, active motion time, attempted episodes, retained successes, retained task failures, excluded recordings, and the count of independent physical task configurations, so that every attempt reconciles against a retained or an excluded episode and the time spent resetting the physical scene stays visible.

A single action column in a dataset conceals the filtering between request and response, so the record keeps all four action taps of 5.2 at every control step. Logged alongside boolean override flags, truncation markers, and numeric exclusion codes, the taps show where the permission path intervened and where tracking lag separated command from execution. Discarding the intermediate representations leaves the training pipeline unable to distinguish between an intentional human demonstration and an action heavily modified by the permission path.

Every physical dataset supports only part of the ODD, so the record catalogs both its supported cells and its known absences, and stating what is absent keeps an engineer from treating the absence of training failures as evidence of policy competence. Downstream verification treats any absence the record leaves uncataloged as unsafe. Filled in for the latch campaign, with each field group and the admission check it supports listed in table 5.4, the record reads:

- **Denominators:** 52 attempts per scheduled session at a net retention rate of 0.68, so 5,000 retained successful demonstrations cost 7,353 attempts over 141.4 h of scheduled cell time.
- **Retention rule:** an attempt whose contact force crosses the 15 N tripwire ends there and is kept as a labeled failure, outside the demonstration count, when its telemetry is valid; a recording that fails quality filtering is excluded with its exclusion code.
- **Scenario ledger:** plate offset (nominal, +3 mm, −3 mm) crossed with approach speed (0.03 m/s, 0.10 m/s), six cells, held in the record as a digest rather than as a copy.
- **supported_cells:** the nominal plate at 0.03 m/s, the only cell the retained demonstrations occupy.
- **known_absences:** the other five cells, every displaced plate and every approach at 0.10 m/s.

Table 5.4: Demonstration dataset provenance schema: Evidence retained and the admission check applied to each field group of the demonstration provenance record; the embodiment, sensor calibration, and contact wear groups bind the individual plant that collected the data.

Provenance record	Evidence retained	Admission decision
Session	Collector, operator, policy version, sample rate, and timing jitter.	Check operator authorization and the session jitter budget.
Embodiment	Robot identity, geometry, gearing, backlash, and firmware.	Check reach, dynamics, and joint limits on the target plant.
Sensor calibration	Camera geometry, clock synchronization, and force/torque bias.	Check calibration age and timebase error.
Contact wear	End-effector material, hardness, friction, flex cycles, and maintenance history.	Revalidate contact data after material or compliance changes.
Four action taps	Requested, mapped, enforced, and measured values, each with its units and frame, plus override and clamp codes.	Compare like-typed, aligned values; exclude permission-path actions from labels.
Transition timing	Times of exposure midpoint, action issue, mapping, enforcement, response start, and response end, with clock IDs and synchronization errors.	Check causal order and the declared response window.
Collection denominators	Scheduled cell time, motion time, attempts, retained successes and failures, excluded recordings, and task configurations.	Reconcile attempts against retained and excluded episodes.
Scenario support	Scenario-ledger digest, the supported cells of the join, and known absences.	Refuse deployment queries outside the supported cells pending evidence.

The next record in the chain is the policy manifest of section 6.8, which cites this record by digest and may declare an ODD no larger than the supported cells listed here. Beyond training, downstream lifecycle stages consume the data record for distinct engineering decisions:

- The human and override provenance feeds directly into the supervisory auditing of chapter 14, where safety engineers evaluate demonstrator consistency, teleoperation latency tails, and the frequency of permission-path interventions to assess human reliability.
- The coverage boundaries, exclusion ledgers, and embodiment calibration histories pass forward to chapter 16, where they form the empirical baseline for the safety case, verifying that the training distribution covers the ODD the release claims before the learned policy is granted execution authority on a physical machine.

By coupling physical movement trajectories with verifiable embodiment metadata, multi-stream action tuples, and support boundaries, the provenance schema makes visible any extrapolation into regimes the plant never visited, so downstream stages can refuse it.

5.10 Fallacies and Pitfalls

A clean dataset can omit states the deployed machine must handle. Coverage depends on the collecting policy (the human operator or automated script generating the motions), the physical rig, and the curation decisions that determine which experiences survive in the archive.

Fallacy: *Massive sample volume compensates for lack of operational state-space coverage.*

A surface-tracking task requires data across 500 combinations of force, friction, and contact angle, but the archive covers only 45. Another session adds 72,000 control steps by repeating nominal motions in populated state space cells. The pipeline’s ingestion report shows massive data growth with zero dropped packets, but physical coverage remains unchanged. Closely spaced samples describe the same limited physical experience more densely; they add no evidence about the missing friction or contact conditions. Dataset acceptance must therefore measure added coverage against the operational scenario ledger. More demonstrations earn their collection cost when they address a missing regime or an identified weakness, not merely when they enlarge the archive.

Downstream: The known absences and ODD boundary schema feed the formal safety case claims in section 16.3.

Pitfall: *Assuming physical demonstration labels survive mechanical or contact hardware maintenance.*

A maintenance technician replaces a gripper’s compliant fingertips with stiffer pads and adjusts its camera bracket. The archived demonstrations still pass structural schema checks (the data types and array shapes remain perfectly valid), but the physical reality has shifted. A known camera-pose adjustment can support coordinate reprojection for the vision data. However, that geometric correction cannot establish how the new, stiffer fingertips behave under contact, where they slip at lower force thresholds than the compliant ones did. Historical force and tactile measurements remain records of the old body, so their suitability as training targets requires revalidation. Dataset provenance must identify the mechanical and calibration state of the collection rig, allowing maintenance changes to trigger the appropriate review or recollection.

Pitfall: *Filtering out aborted safety interventions to sanitize the demonstration dataset.*

An insertion dataset contains 1000 attempts, including 180 non-nominal runs where contact forces reached or exceeded threshold limits (such as a 15.0 N safety trip), leaving 820 nominal successes (an 82 percent task success yield). A curation script strips out the 180 aborted or recovery trials, leaving only the 820 flawless episodes whose contact forces stayed below 8.0 N. When deployment disturbances later push the robot’s contact force to 9.5 N, the policy finds itself immediately outside the retained experience, even though the hardware limit has not yet been reached. The superficially cleaner training set has erased the crucial moments of approaching the physical boundary and the recovery interventions that followed. Retaining and identifying those episodes preserves evidence about failure and recovery; deleting them because they are unsuccessful creates a misleading view of the task.

Pitfall: *Merging demonstrations collected across kinematically asymmetric leader interfaces.*

Two operators collect snap-fit demonstrations using different leader devices (the hand-held controllers used to puppet the robot). One interface permits only two wrist-roll positions ($\theta_{\text{roll}} \in \{0.0^\circ, 90.0^\circ\}$); the other mechanically limits roll to $[-15.0^\circ, +15.0^\circ]$. Merging their 4000 episodes increases volume, but neither interface supplies the required intermediate orientations between 20.0° and 70.0° . A policy trained on the combined archive still lacks demonstrations for those perturbations. Aggregation cannot remove a blind spot shared by the collection interfaces. The team must map recorded orientations against task requirements and check whether the leader devices can express the missing motions before assigning more collection hours.

Fallacy: *Physical data collection scales linearly with labor hours just like internet scraping scales with compute.*

A team projects its bimanual dataset budget from an initial rate of 50 successful episodes per operator-hour. As collection continues, repeated physical impacts damage gearboxes, sensor drift invalidates some demonstrations (as uncalibrated joint angles gradually decouple from physical ground truth), and operator fatigue alters the timing of human motions. The rate of retained episodes falls even though the scheduled labor hours remain unchanged. A short nominal trial therefore overstates sustained collection throughput. Planning must include maintenance, calibration checks, rest, and rejected data alongside operator time. The useful production measure is accepted experience under documented conditions, since mechanically degraded or poorly aligned recordings can increase storage without increasing the dataset’s value.

5.11 Summary

Physical data collection converts mechanical work and hardware fatigue into the supported cells of a robot’s ODD. A session keeps less than a quarter of its scheduled time as validated demonstrations, and every attempt, retained or not, draws on the wrist’s rated fatigue life, so the archive is scarce before any question of its quality arises. Because the collecting policy chooses its own next observations (principle 2), coverage is a fact about how the archive was gathered rather than about what it holds. Validation loss, drawn from the same occupancy, cannot expose that fact after the campaign ends, so it has to be written down while the gathering is still observable, as a join against a declared ledger and as provenance fields recorded at every step. The supported cells, not the size of the archive, bound what a policy trained on the data may be trusted to do, because a loss function cannot account for physical responses the robot never encountered. The demonstration provenance

record makes that bound explicit by binding each demonstration to its plant, its action taps, and the cells it covers and misses.



Key Takeaways: Physical data collection and provenance

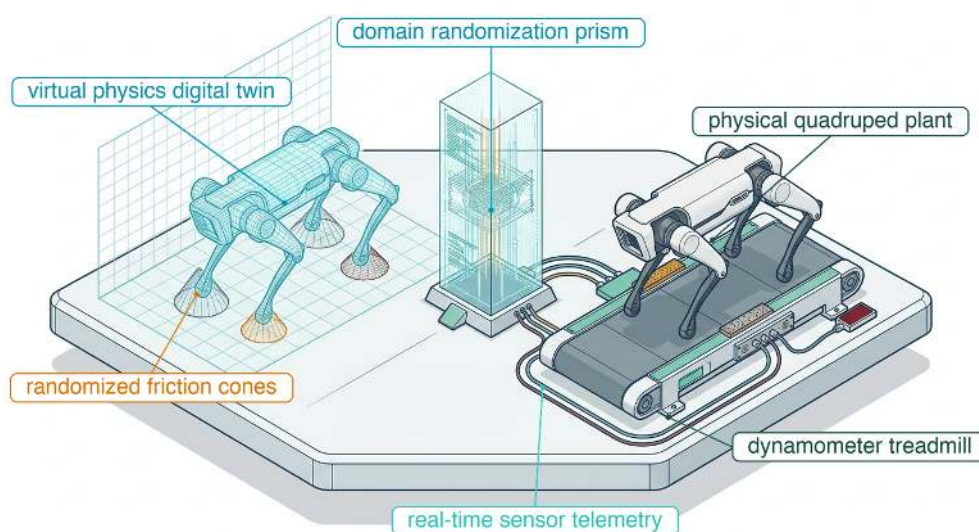
- **Coverage counts configurations, not samples:** Repeated nominal motions add rows without adding scenario cells, and adjacent control steps are near duplicates. A dataset's reach is its join against a scenario ledger spanning the ODD, never its sample count or the extremes it happened to log.
- **Validation loss cannot see a coverage gap:** A held-out split shares the collector's occupancy, so near-zero validation loss can coexist with most required cells empty. Only the join against an external ledger exposes the gap.
- **Every action tap is a different signal:** The request, the mapped setpoint, the enforced command, and the measured response differ wherever the permission path or the body intervenes. A dataset that keeps one of them trains on infeasible demands or on backlash mistaken for skill, and hides every intervention.
- **Keep failures and takeovers, correctly attributed:** Censoring aborted runs erases the states nearest the boundary. Learner commands before a takeover stay in the record for audit and expert relabeling but never as expert labels, and the recovery after it is a separate expert sequence.
- **Data stays bound to the body that collected it:** Camera geometry can be reprojected, but contact wear, compliance, and backlash are baked into force and tactile labels. Data from a changed or different body requires revalidation, and range normalization hides the mismatch rather than removing it.
- **Absences travel with the dataset:** Training cannot synthesize physics that collection omitted, so the provenance record lists its supported cells and its known absences. A policy trained on the data may later declare an operating domain no larger than those supported cells.



What's Next: From demonstrations to policies

Which source of experience can teach a policy the recoveries its demonstrations never showed? The latch campaign's provenance record supports only the nominal strike plate at the guarded approach and lists every other cell as a known absence. Chapter 6 takes up that gap, comparing imitation, offline learning, simulation, and fine-tuning on the machine by the recovery each can supply and the actuator dynamics each can hide, and records the result in a policy manifest whose declared ODD may not exceed the supported cells.

Policy Training



- 6.1 Policy Synthesis
- 6.2 Policy Synthesis Regimes
- 6.3 Learning by Trial
- 6.4 The Simulator as a System Component
- 6.5 What the Gap Is Made Of
- 6.6 Fine-Tuning on the Real Machine
- 6.7 How Training Choices Go Wrong
- 6.8 The Policy Manifest
- 6.9 Fallacies and Pitfalls
- 6.10 Summary

Purpose

Why can a policy that succeeds in every simulated trial break the hardware on its first real contact?

Physical trial and error consumes hardware as well as computation. An exploratory action can overheat a motor coil, collapse battery voltage, or overload a transmission before a policy receives meaningful reward feedback. Consequently, physical policy synthesis shifts predominantly to offline demonstrations and high-throughput physics simulators. Yet each experience source leaves distinct, unmodeled inductive biases in the learned neural weights. Demonstrations cover only nominal visited trajectories, naive regression losses average distinct multimodal solutions into unviable interpolated actions, and simulation solvers reward non-physical contact shortcuts that hardware instantly punishes.

None of these discrepancies appear in offline training metrics, yet all of them accompany the weights onto the machine. Deploying learned policies safely requires selecting training regimes matched to dataset support, measuring where numerical solvers depart from real-world plant dynamics across friction, compliance, and latency, and rigorously bounding where the policy remains valid. Any unmodeled assumption left unchecked in software is ultimately settled at the causal boundary, where commands deliver electromagnetic energy and irreversible mechanical work that software can never recall.

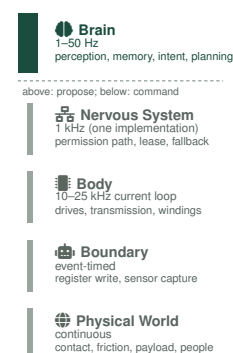


Figure 6.1.: The five-level machine stack with the Brain level highlighted.



Learning Objectives

- Compare training regimes by the dependency each leaves in the weights and the failure it produces on hardware
- Explain why a cloned policy's small errors compound once it leaves the demonstrated states
- Explain how a squared-error loss turns two valid demonstrations into an action neither demonstrator took
- Diagnose a transfer failure by locating it in the dynamics, sensing, appearance, or timing channel of the sim-to-real gap
- Evaluate a real-machine fine-tuning run by the wear it incurs and the prior evidence it invalidates
- Construct a policy manifest whose declared operating domain excludes every state the data does not support or the simulator does not model

6.1 Policy Synthesis

Two learned controllers can succeed at the same task under nominal conditions and still fail on the same machine for different reasons. The cage-door latch task of chapter 5 has two such controllers. The first latch policy, `latch-bc-01`, was cloned from the latch campaign's teleoperated door-opening demonstrations. The second, `latch-sim-02`, started from `latch-bc-01` and was refined by reinforcement learning in a physics simulator.

At the nominal strike plate and the guarded approach speed of 0.03 m/s (illustrative; see the Reader Guide), both policies open the door. Their failure modes diverge once conditions leave that point. When the door has sagged and its strike plate sits a few millimeters off nominal, the cloned policy's first correction lands in a state its demonstrations never visited, because the demonstration campaign recorded no displaced plate. Each later command, issued at the chosen 20 ms setpoint period, compounds the error until the gripper binds on the edge of the plate. The simulated policy fails on the nominal plate. Its simulator modeled contact as a soft penalty spring, so the optimizer found that approaching at 0.10 m/s costs nothing. On the real plate, contact force rises in proportion to approach speed and time, and at that speed it reaches the latch's 100 N limit 2.5 ms after touch, barely longer than the contact loop's 2 ms response (the speed, limit, and response time are illustrative). That leaves too little time to shed the approach speed, so the plunge overloads the latch. The data collection regime defined what the policy saw, while the simulator shaped its assumptions about the physical world.

The two latch failures share one structure. Each training regime left a dependency in the weights. The cloned policy depended on the states its demonstrations happened to cover and the simulated policy on a contact model softer than the plate, and in each case a deployment condition that violated the dependency produced a failure no training score had shown. The practical choice is therefore which experience source teaches the needed recovery without hiding actuator dynamics, and what physical evidence must precede actuator authority. The answer travels with the checkpoint as a record of the states it may enter. Whatever the regime, the checkpoint only proposes chunks to the permission path (section 1.5), and training that never modeled that path yields a policy whose torque spikes the permission path must refuse, or whose proposals trip its limits continuously.

6.2 Policy Synthesis Regimes

The latch task could draw on four sources of experience: the teleoperated door openings of chapter 5, a log of imperfect or scripted door runs, a simulator of the cage door, and the arm itself. Each buys a different capability at a different physical price, and table 6.1 sets them side by side with the failure each is known for and the check that intercepts it.

Table 6.1: Comparative trade-off matrix across policy synthesis regimes: The four regimes compared are behavioral cloning, offline reinforcement learning, sim-to-real reinforcement learning, and residual RL/fine-tuning. Columns detail the primary operational capability purchased during training, representative sample and compute budgets, sample efficiency bounds, safety risks incurred during data acquisition, and dominant failure signatures with corresponding permission-path interceptors.

Policy Synthesis Regime	Core Purchased Capability	Data & Compute Budget	Sample Efficiency & Scale	Hardware Safety & Wear Risk	Dominant Failure Mode & Interceptor
Behavioral Cloning (BC)	Direct imitation of demonstrated trajectories without reward engineering or system identification	10^2 – 10^3 teleop trajectories (~ 10 – 50 h data); < 50 GPU-h compute	Supervised $\mathcal{O}(N)$ over expert demonstrations; fixed $1 \times$ real-time data throughput	Minimal exploration risk during logging; zero hardware exploration during optimization	Quadratic compounding error ($\mathcal{O}(T^2 \epsilon)$) via covariate shift. <i>Interceptor:</i> Kernel/Mahalanobis density monitors, convex hull boundary clamps
Offline / Batch RL	Value-guided policy optimization over fixed, heterogeneous sub-optimal logs; stitches multi-trajectory transitions	10^5 – 10^7 logged transitions with state-action coverage; 100 – 500 GPU-h compute	Offline $\mathcal{O}(N)$ over static datasets; sample-efficient but demands high state-action overlap	Zero active exploration hazards during offline training; logging requires caged rigs or safe scripted controllers	Out-of-distribution action extrapolation and Q-value overestimation. <i>Interceptor:</i> Conservative Q-learning (CQL) penalty bounds, action support filters
Sim-to-Real RL	Discovery of dynamic, non-prehensile manipulation and agile locomotion via billions of unconstrained rollouts	Zero physical data; 10^7 – 10^9 simulated transitions across 10^3 – 10^4 GPU environments; 500 – 5000 GPU-h	Low algorithmic sample efficiency (10^7 – 10^9 steps), offset by $10^3 \times$ – $10^5 \times$ simulation acceleration	Zero hardware wear or collision damage during training; severe deployment risk if plant dynamics diverge	Exploitation of numerical solver artifacts (contact interpenetration, zero latency). <i>Interceptor:</i> Paired-trace discrepancy budgets, force-derivative (dF/dt) clamps
Residual RL & Real Fine-Tuning	Direct policy correction against physical contact compliance, gearbox hysteresis, and true sensor noise	10^3 – 10^5 physical steps (0.5 – 10 h wall-clock time); < 20 GPU-h compute	High sample efficiency required (SAC/TD3 or residual delta-action formulation); strictly $1 \times$ real-time	High physical exposure; repetitive impact shocks degrade gearboxes; exploratory current dither elevates winding temperature	Catastrophic forgetting of nominal transit stability, local policy collapse. <i>Interceptor:</i> Parameter drift bounds ($\ \Delta\theta\ _2 \leq \delta$), frozen regression suites, thermal limiters

Of the four, **behavioral cloning** is the cheapest route to a working policy and the one whose failure mode is best understood. It frames policy synthesis as supervised regression or classification over a collected dataset of expert state-action pairs $\mathcal{D} = \{(\mathbf{o}_t, \mathbf{a}_t)\}$. Following the action taps of section 5.3, a demonstration dataset pairs observations with either the operator’s mapped setpoints or the setpoints the permission path admitted:

$$\mathcal{D}_{\text{intent}} = \{(\mathbf{o}_t, \mathbf{a}_{\text{cmd},t:t+H-1})\} \quad \text{versus} \quad \mathcal{D}_{\text{verified}} = \{(\mathbf{o}_t, \mathbf{a}_{\text{enf},t:t+H-1})\}$$

Cloning the mapped setpoints ($\mathbf{a}_{\text{cmd}}$), the operator’s intent in the robot’s own coordinates before the permission path acts, trains the model on demonstration intent, whereas cloning the setpoints the permission path admitted ($\mathbf{a}_{\text{enf}}$) bakes its barrier deceleration profiles directly into the network weights, so the policy inherits decelerations tuned to that particular path’s limits. In its standard formulation, the optimization objective minimizes a multi-step prediction loss over continuous

¹ | **ALVINN road perspective warping:** Pomerleau's Autonomous Land Vehicle in a Neural Network (ALVINN) (Pomerleau 1989), told in section 5.2, drifted toward the lane edge because its human drivers never demonstrated a recovery from it. To supply recovery examples without unsafe driving, Pomerleau used synthetic perspective warping, rendering laterally shifted and rotated road images paired with transformed steering corrections, an early case of augmenting demonstrations with recovery data off the nominal trajectory.

Prerequisite: The four-stream action tap tuple separating intent from enforcement is formulated in section 5.3.

² | **Mode averaging under L_2 loss:** Under an L_2 regression loss, the expected risk $\mathcal{R}(\hat{a}) = \int \|\hat{a} - a\|_2^2 p(a | o) da$ is strictly convex with respect to $\hat{a}$. Differentiating with respect to $\hat{a}$ and setting the gradient to zero yields $2 \int (\hat{a} - a) p(a | o) da = 0 \Rightarrow \hat{a}^* = \int a p(a | o) da = \mathbb{E}[a | o]$. For a nonlinear plant f_{plant} , $\mathbb{E}[f_{\text{plant}}(a)]$ generally differs from $f_{\text{plant}}(\mathbb{E}[a])$; equality can hold in particular distributions. For a convex action cost $c(a)$, Jensen's inequality gives $c(\mathbb{E}[a]) \leq \mathbb{E}[c(a)]$, but it does not establish the physical safety of the mean action. In the obstacle example, the mean lies outside the demonstrated set of valid paths.

action sequences drawn from demonstration trajectories (see section A.5.1 for classical regression formulations).¹ Because this loss penalizes only the discrepancy between the commanded action and the recorded demonstration at observed states, the training objective does not evaluate task completion, terminal cost, or the stability of the closed-loop dynamical system. It fits actions to observations only.

The multimodality breakdown and mode averaging

When human demonstrators perform a physical task, their strategy is frequently multimodal. Consider an obstacle avoidance scenario where a mobile base or robotic arm must navigate around a central pillar. A human demonstrator may choose to pass around the pillar to the left in half of the demonstrations, or to the right in the remaining half. The true demonstration data contains distinct valid choices, but zero examples of driving straight down the middle.

A deterministic point-regression behavioral-cloning policy cannot represent both choices for the same observation. Under squared-error loss its optimal output is the conditional mean (for the derivation, see section A.5.1),² here the midpoint between the “steer left” and “steer right” commands. The policy outputs zero lateral velocity, commanding the machine to drive directly into the obstacle, an action that never occurred in the demonstration dataset.

A process plant shows that mode averaging needs no geometry. Suppose a flow controller's demonstrations alternate between two illustrative valve regimes, a bypass regime at 10 percent opening and a main-feed regime at 90 percent, in equal numbers. Squared-error cloning outputs their mean, 50 percent opening, a flow that neither regime commands and for which the plant was never calibrated. That intermediate flow can cavitate the supply pump and miss the cooling deadline each regime was tuned to meet. No obstacle is involved, so no geometric check on the proposal catches it; only a check against the demonstrated regimes does.

A mode boundary also produces a timing failure, temporal inconsistency in a high-frequency single-step controller. In an illustrative 50–100 Hz controller, sensor noise moves consecutive observations across that boundary, and a policy mapping only $o_t \rightarrow a_t$ alternates between $+10 \text{ N} \cdot \text{m}$ and $-10 \text{ N} \cdot \text{m}$ requests. Whether the alternation excites resonance, wear, or winding heat depends on the drive and the load, so it is a failure mode to test for rather than a property of every single-step policy.

Mode averaging is a failure at a single observation. Behavioral cloning has a second failure that appears only when the policy runs in closed loop. Its own errors move the machine off the states its demonstrations cover, where the loss supplied no training signal, which is how `1atch-bc-01` binds on the sagged door. Under principle 2, the policy's own actions choose which states it visits next, so a fixed archive cannot anticipate them, and for expert-only cloning the worst-case cost grows with the square of the horizon (equation 5.1, developed in section 5.1). A low validation loss on held-out demonstrations therefore measures accuracy only on states the demonstrator visited; it says nothing about how the policy returns once displaced. Corrective aggregation labels the learner's own states and, under its expert-labeling assumptions, reduces that growth to linear (section 5.6), but on a physical machine those states are reached by running an unverified policy, which can strike a fixture or damage a transmission before a supervisor intervenes.

Definition 6.1: Compounding rollout error

Compounding rollout error is the quadratic accumulation of closed-loop tracking drift incurred when an open-loop supervised policy executes sequentially on a physical dynamical plant, bounded in expectation by:

$$J(\pi) - J(\pi^*) \leq \mathcal{O}(T^2 \epsilon)$$

where T is the task execution horizon and $\epsilon = \mathbb{E}_{\mathbf{s} \sim d_{\pi^*}} [\mathbb{I}(\pi(\mathbf{s}) \neq \pi^*(\mathbf{s}))]$ is the single-step policy error rate on the demonstration distribution d_{π^*} .

1. **Significance:** Proves that minimizing single-step prediction error on offline demonstrations is fundamentally insufficient for closed-loop stability; any nonzero error rate $\epsilon > 0$ eventually drives physical state outside demonstrated support.

2. **Distinction:** Unlike static supervised learning where validation error scales linearly with sample count, compounding rollout error couples policy mistakes with environmental physics, turning small initial trajectory deviations into irreversible state divergence.
3. **Common pitfall:** Assuming low offline validation loss on held-out demonstration splits translates to high closed-loop task success, ignoring that offline datasets contain zero examples of returning to nominal support once perturbed.

6.2.1 What action chunking changes—and what it leaves unresolved

Action chunking (3.1) can improve temporal coherence and reduce proposal frequency in generative architectures such as diffusion policies and Action Chunking with Transformers (ACT). It can also address the alternation in the mode-boundary example above. A chunking policy that predicts H future actions $\mathbf{a}_{t:t+H-1}$ at once improves execution through three mechanisms:

1. **Temporal and Dynamic Consistency:** Rather than generating disjoint single-step predictions that chatter, the model predicts an entire coordinated trajectory.
2. **Maintaining a Chosen Mode:** Single-step Markov policies are prone to causal confusion, mistaking momentary spurious correlations for task intent. Conditioning on trajectory history and proposing a chunk helps preserve a chosen mode across adjacent actions.
3. **Receding-Horizon Replanning and Temporal Ensembling:** The deployed policy queries a fresh chunk every few steps and blends the overlapping predictions (section 3.4), which permits periodic feedback while retaining a multi-step proposal.

None of these mechanisms changes the behavioral-cloning support bound $\mathcal{O}(T^2\epsilon)$. As demonstrated theoretically and empirically in figure ??, single-step behavior cloning experiences a steep drop in task survival as rollout horizon T grows, falling into the catastrophic intervention envelope (< 18 percent survival) within 250 control steps at 50 Hz. While temporal architectures like BC-RNN delay the onset of divergence, their quadratic error compounding remains asymptotically identical. In contrast, on-policy interactive aggregation (Dagger (Ross et al. 2011)) bounds error growth linearly to $\mathcal{O}(T\epsilon)$ by collecting expert feedback on learner-visited states. Without interactive supervisors, modern generative policies counteract compounding drift through action chunking ($K = 16$), which contracts the effective execution horizon to $T_{\text{eff}} = T/K$ (ACT (Zhao et al. 2023)), and continuous receding-horizon diffusion replanning (Chi et al. 2024), maintaining over 80 percent task survival across extended 500-step manipulation horizons.

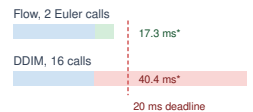
Increasing dataset size, model capacity, or tuning can reduce the one-step error rate on evaluated observations and therefore lower the chance of a first policy error during a fixed horizon. That improvement does not characterize what happens after an error. An external disturbance, uncalibrated payload, or voltage sag can also move the machine into weakly supported states without a policy disagreement. Recovery data, measured dynamics, and independent runtime limits determine whether either event remains within the physical envelope.

Drift off the demonstrated states raises no alarm of its own, so detecting it before physical contact cannot rely on the offline training loss. It requires runtime telemetry that tracks state space support directly, measuring Mahalanobis distance in feature space, estimating model epistemic uncertainty, or enforcing an explicit envelope monitor on the permission path that trips when state trajectories leave the convex hull of the demonstration dataset.

What the decoder choice costs at run time

A point-regression decoder averages distinct valid actions into an invalid command, and several decoder families avoid the mean: mixture density networks, implicit energy-based cloning, conditional diffusion and flow matching, latent-variable chunking such as ACT, and discrete action tokens in vision-language-action models. Section A.5.1 and section A.5.2 develop their losses and sampling procedures. None of them supplies missing recovery states. What the choice changes for the machine is its cost at run time, because the decoder fixed at training time sets how many network evaluations each chunk requires.

The step count is the cost that reaches the control deadline. A diffusion decoder produces each chunk by repeated denoising, and every step is a full evaluation of the noise-prediction action



*Assumed compute; tail latency unmeasured

Denoising steps set chunk inference time, and only the few-step sampler meets the setpoint period synchronously.

head, so inference time scales with the number of steps (the refinement term of the chunk latency in section 3.4). Under the step counts and per-step times assumed in section A.5.2, a many-step denoising diffusion implicit model (DDIM) sampler overruns the 20 ms setpoint period when it runs synchronously, while a few-step flow-matching sampler fits within it, provided those few steps meet the task’s action-quality criterion. A decoder that cannot meet the period synchronously runs asynchronously. The application processor refreshes a chunk buffer at a lower rate, and the permission path reads the newest chunk at its own rate, checks its freshness, and falls back when it expires, so actuation never waits on the proposer. The machine’s own chunk policy is this asynchronous case. It renews its chunk every 50 ms, so its inference deadline is that chunk period, and the 20 ms setpoint period only paces playback of the setpoints inside the chunk.

As detailed in figure A.5 (section A.5.2), the policy conditions on an observation history of images and joint states to iteratively denoise an action chunk $\mathbf{A}^0 \in \mathbb{R}^{K \times D_a}$ from pure Gaussian noise. Rather than executing the entire prediction horizon, the machine dispatches an execution horizon to the real-time microcontroller before receding-horizon re-planning occurs. The formal dual-rate pipeline—coupling asynchronous reverse-diffusion on the host with deterministic spline interpolation, rate-limiting, and safety clamping on the microcontroller—is developed in algorithm A.2 (section A.5.2).

Offline reinforcement learning and action distributional shift

When physical demonstration datasets $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$ contain heterogeneous or sub-optimal trajectories (e.g., mixtures of teleoperation, scripted routines, and exploratory human play), behavioral cloning struggles because it treats all demonstrations with equal supervised weight. In classical reinforcement learning formulated on Markov decision processes (MDPs) (Sutton and Barto 2018), an agent optimizes a policy to maximize expected discounted cumulative returns $J = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$. **Offline reinforcement learning (Batch RL)** aims to optimize a policy directly from logged transition archives without active hardware exploration, using reward signals to stitch together high-performing composite trajectories from disjoint, imperfect demonstrations (Kumar et al. 2020).

However, applying standard Q-learning to static offline datasets triggers a failure mode known as **out-of-distribution (OOD) action overestimation**. In standard reinforcement learning, the algorithm estimates the value of future states by asking its value network to predict the highest possible return it could achieve from the next step (for formal Bellman equations, see section A.6). That maximum is taken over the entire continuous action space. For actions that were never executed in the logging dataset, the network has received zero training signal and often outputs erroneously large value estimates due to generalization variance, predicting a high return for an untried physical maneuver. The algorithm greedily selects these spurious peaks, propagating over-optimistic value errors backwards. When deployed, the policy selects these ungrounded actions, and it can command motions that stall a joint or strike a boundary.

To reduce value overestimation without querying the physical environment, **Conservative Q-learning (CQL)** adds a regularizer to the offline Bellman objective.³ It penalizes high values for actions sampled by the learned policy relative to actions in the logged data. Limited state-action coverage and approximation error remain, so the permission path still checks what the resulting policy proposes.

Cloning, chunking, and offline value learning all remain inside the support of the logged data, and Conservative Q-learning makes an offline policy more cautious without widening that support. When the logged transitions lack the high-performance trajectories or recovery maneuvers that dynamic manipulation or agile locomotion needs, the system must collect new experience through closed-loop trial.

6.3 Learning by Trial

The cloned latch policy failed on the displaced plate because no demonstration had ever shown it one. Learning by trial closes that blind spot by letting the policy generate its own state-action trajectories in closed loop. Algorithms such as Q-learning or policy gradients adjust the policy from the rewards its own actions earn, so an exploratory action that pushes the gripper off its nominal path forces the optimizer to evaluate recovery from exactly that perturbed state, including the near-misses and unstable contacts an operator never exhibits. By sampling transitions from the state distribution the

Downstream: Hardware placement of multi-rate action chunk buffers across the application processor and the safety microcontroller is engineered in section 13.2.

³ | **Scope of the CQL bound:** Under its data-coverage, optimization, and function-approximation assumptions, CQL constructs a conservative estimate whose expected value for a policy lower-bounds that policy’s true value (Kumar et al. 2020). This is not a pointwise bound for every state-action pair or every dataset, and expected return does not certify physical safety.

current weights induce, trial-based learning trains on the states the policy actually reaches, which removes the covariate shift behind the quadratic worst-case growth of cloning error.

Closed-loop exploration on hardware is billed first in calendar time, which depends on how many trials the algorithm needs, how long each trial lasts, how long a reset takes, and how many machines run in parallel. Consider the door-opening task itself, in which the arm regulates contact forces over an active horizon of $t_{\text{rollout}} = 8$ s, the autonomous contact phase of an opening rather than a full teleoperated attempt. Reinforcement learning on high-dimensional sensory inputs can require on the order of $N = 10^6$ trials to discover coordinated force-velocity profiles, the scale of the grasping campaign in figure 6.2. On a single physical machine with $P = 1$, the raw execution time alone accounts for 8.0×10^6 s, roughly 92.6 days of uninterrupted round-the-clock operation before accounting for any reset overhead.

Factoring in the reset phase reveals the true temporal bottleneck of physical experimentation. If a dedicated automated fixture, faster than the collection rig's re-latch (section 5.4), re-latches the door and returns the arm to its starting pose in $t_{\text{reset,auto}} = 4$ s, the total collection time for one million trials expands to 1.2×10^7 s, or 138.9 days. When exploratory trajectories jam the latch cam or trigger joint overcurrent trips, automated resetting becomes impossible, requiring a human operator to free the latch, re-seat the door, and re-index the arm. If every trial instead needs a manual reset averaging $t_{\text{reset,manual}} = 30$ s, the wall-clock requirement grows to 439.8 days. Scaling physical parallelism to clusters of $P = 14$ synchronized rigs (figure 6.2) compresses calendar duration to several weeks (Levine et al. 2018), but it multiplies the physical footprint, capital investment, electrical power delivery, and human supervisory labor proportionally.

Time is often not the binding constraint; wear is. Each door opening loads each arm joint through about $n_{\text{rev}} = 4$ bidirectional load reversals, so $N = 10^6$ trials accumulate 4.0×10^6 of them in the drivetrain.⁴ The same million contact events wear the latch cam and strike plate, crush the gripper's compliant pads, and cycle the arm's wiring toward fatigue fracture.

In a purely computational environment, an exploratory branch that goes numerically unstable costs nothing beyond a discarded buffer. On the running machine, exploration is irreversible. Unconstrained exploratory torque drives the gripper into rigid fixtures, bending load-cell flexures, stripping harmonic drive teeth, and heating the motor stators past their 155°C Class F insulation threshold.⁵

Furthermore, physical degradation alters the plant during the training process itself. As contact surfaces burnish, gear backlash expands (increasing the mechanical slop or dead-zone between gear teeth), and structural fasteners loosen under repetitive vibration, the physical transition distribution $p(s_{t+1} | s_t, a_t)$ at trial $n = 900,000$ differs measurably from the pristine dynamics at trial $n = 1$. The machine being controlled mutates under the physical cost of collecting its own training data.

The reward function adds a second fragility. In trial-based learning the scalar reward is the interface specification between the engineer's intent and the optimizer, and any constraint it does not measure and penalize is a free degree of freedom the optimizer will exploit. A reward for rapid setpoint tracking with no penalty on jerk or winding temperature converges toward bang-bang switching between maximum forward and reverse effort, which vibrates brackets and overheats drive electronics. With contact force unmeasured, the policy may learn to brace a joint at maximum current against a hard stop because doing so removes tracking jitter. The network satisfies the formula presented to it, not the designer's unstated common sense.

For low-dimensional systems with trivial resets, negligible wear, and wide margins, reinforcement learning directly on hardware remains viable. Simulation becomes mandatory where the required sample count exceeds the time, budget, structural life, or risk tolerance of the apparatus, and the 10^6 to 10^9 transitions a complex motor policy needs move into a physics engine run in parallel across a computing cluster.

The substitution changes the contract between training and reality. A policy trained in a simulator bends no linkage during exploration, but it inherits the omissions of the reward and the simulator's model of mechanics, and every unmodeled friction discontinuity, idealized contact impulse, and uncharacterized latency becomes an assumption it relies on. On the physical machine those inaccuracies return as unmodeled disturbances and tracking failures.

⁴ | **Bearing life versus command reversals:** ISO 281 estimates rolling-bearing fatigue life under specified load and speed. A gear-unit catalog may express its wave-generator-bearing L_{10} in operating hours at rated conditions; neither figure converts commanded reversals into consumed gearheads without a unit-specific duty-cycle model.

⁵ | **Winding temperature limit:** Section 2.5 gives the IEC 60085 Class F rating behind this threshold. Bang-bang torque commands from unregularized reward functions can hold the current high enough to age the insulation quickly.

Prerequisite: Motor winding thermal limits and torque-speed derating curves are derived in section 2.5.



Figure 6.2: Physical policy training farm: Fourteen manipulators collecting over 800,000 physical grasp attempts across months (Levine et al. 2018). Hardware parallelism divides calendar time but not cost, since each station adds capital, gearbox and bearing wear, thermal load in the motor windings, and a share of the supervisory labor needed to clear dropped objects and jams. Attribution: Levine et al. (2018).

6.4 The Simulator as a System Component

Treating the simulator as an external development tool is an architectural error. A compiler produces machine instructions from source code, but the resulting binary does not retain a hidden dependency on the compiler's internal memory allocator once executing on standard hardware. A simulator behaves differently. It is an upstream policy-production component whose physical assumptions, numerical approximations, and boundary simplifications become permanently embedded in the policy parameters. When the trained neural network is copied onto the target machine, the simulator is absent from the execution path, yet the policy acts as though every unmodeled dynamic in that simulator remains true. The simulator is an active component of the system architecture, with its own specification, tolerances, and failure modes.



Definition 6.2: Sim-to-real reality gap vector

Sim-to-real reality gap vector $\Delta_{\text{sim2real}} = [\Delta_{\text{dyn}}, \Delta_{\text{sens}}, \Delta_{\text{vis}}, \Delta_{\text{lat}}]$ is the coupled, multi-channel decomposition of physical discrepancies between simulated and hardware environments

spanning contact mechanics (Δ_{dyn}), sensor transduction and noise (Δ_{sens}), visual appearance (Δ_{vis}), and asynchronous transport latency and jitter (Δ_{lat}).

1. **Significance:** Treating the “reality gap” as a scalar difficulty metric or pure visual domain mismatch causes engineers to overlook dynamical and temporal divergences. Discrepancies in actuator back-EMF, gearbox compliance, or bus jitter deplete phase margin before the policy completes its first physical control step.
2. **Distinction:** An inaccurate parameter value (such as a 10 percent error in link mass) preserves the causal differential equations of motion, whereas an omitted mechanism (such as zero-delay communication or unmodeled Coulomb stick-slip) entirely severs physical feedback paths, leading policies to exploit non-physical simulator artifacts.
3. **Common pitfall:** Relying exclusively on extreme domain randomization to bridge large gaps. Randomizing parameters across unphysical ranges dilutes policy capacity, resulting in overly conservative, sluggish behaviors that fail to achieve precision tasks on real hardware.

Like any hardware subsystem, the simulator requires a formal interface contract, stated in quantities that can be measured and budgeted. This contract specifies observation dimensions, action bounds, simulation time step Δt , transport latency between command issuance and physical actuation, actuator torque and current limits, sensor noise distributions, reset distributions, and termination conditions. If the simulator specifies an observation loop executing at 100 Hz with an assumed zero-delay sensor channel and Gaussian noise with variance $\sigma^2 = 1.0 \times 10^{-4} \text{ (rad/s)}^2$, the policy learns feedback gains tuned to those precise signal statistics, and an unmodeled 15 ms bus transport delay on the physical machine invalidates them. Dactyl’s twin (figure 6.3) could not match its physical hand exactly, so its simulator was calibrated against the real hand and then randomized around the calibrated values, and the policy met the unmatched remainder as variation it had already trained against (Andrychowicz et al. 2020).

The interface contract matters more as simulation’s role grows. Figure ?? shows the rollout throughput some parallel accelerators reach, and for policies requiring many trials, simulation can become the primary source of training experience.

War Story 6.1: Sim-to-real hardware breakage (2018)

Context: OpenAI trained a reinforcement learning policy entirely in a randomized MuJoCo simulation to reorient a block in a 24-DoF Shadow Dexterous Hand, whose joints are driven through tendons by motors in its base (Andrychowicz et al. 2020).

Mechanism: The simulated hand applied torque directly at its joints, while the physical hand is tendon-actuated and has substantial backlash. The authors smoothed every action with an exponential moving average, in simulation and on the robot, to avoid abrupt changes in the action signal that could harm the physical hand.

Impact: The authors report robot breakages during the physical experiments, most often at the wrist’s vertical joint, probably because it carries the largest load. Each repair took time and often changed aspects of the system, so the physical results were collected at different times, and the authors count hardware breakage among the key challenges of the work.

Response: The team also trained a second policy with the wrist pitch joint locked, and it transferred better to the physical hand.

Systems lesson: A simulator that neither wears nor breaks cannot price hardware exposure into the reward. The limits that protected the hand, an action filter and a locked joint, were imposed outside the learned policy, and every repair changed the plant on which the policy was being evaluated.

When a simulator uses an inaccurate rotor inertia, assigning $J = 2.0 \times 10^{-3} \text{ kg} \cdot \text{m}^2$ instead of a true $2.5 \times 10^{-3} \text{ kg} \cdot \text{m}^2$, the equations of motion still compute an acceleration-dependent inertial



Figure 6.3: Sim-to-real transfer setup: OpenAI’s Dactyl apparatus, a 24-DoF Shadow Hand inside a motion-capture cage beside the MuJoCo digital twin it was trained in (Andrychowicz et al. 2020). Transfer is bought with domain randomization over friction, damping, actuator gains, masses, dimensions, and lighting, together with modeled backlash and action delay, which works by forcing the policy to treat unmodeled physical variation as noise rather than by making the simulator accurate. Attribution: Andrychowicz et al. (2020).

torque. The parameter error alters the magnitude of the response, but the causal structure remains intact. When a simulator omits an entire physical mechanism, such as drivetrain backlash, structural compliance, or inter-channel communication jitter, it does not merely set the corresponding parameter to zero. It provides no causal mechanism at all. An omitted delay or compliance represents an unmodeled physical degree of freedom.⁶ A simulator that omits joint compliance permits instantaneous torque reversals, which a real drivetrain converts into gear-tooth loading and excited structural resonances.

The most consequential omissions come from how simulators handle contact. Rigid contact is non-smooth. The normal force is zero while bodies separate and takes whatever compressive value prevents interpenetration once they touch, and velocity jumps at impact. Friction obeys a Coulomb cone, and rigid bodies under Coulomb friction can have no unique forward solution (the Painlevé paradox). Physics engines take one of two numerical paths (section B.5). Complementarity solvers keep strict non-penetration at a cost that grows steeply with contact count and can stall when geometries pinch. Penalty and convex relaxations replace the hard constraint with a virtual spring-damper, which converges quickly at high frame rates but lets solid bodies interpenetrate slightly. The latch simulator takes the second path, and its softness is what `latch-sim-02` learned to use.

The sim-to-real contact exploit

Reinforcement learning optimizes against the physics the solver presents, not the physics of the plant. In simulators that implement soft contact relaxations or penalty springs, the optimizer discovers that “sinking” into the surface by 1.0–3.0 mm (a negative signed gap $\phi(q) < 0$ between the contacting surfaces) provides two unphysical affordances:

1. The artificial spring-damper reaction acts as an instantaneous energy sink, arresting downward momentum in a single integration step without bouncing or plastic deformation.
2. Interpenetration increases the effective contact surface area in the solver, generating unphysical lateral traction forces that prevent slipping during aggressive maneuvers.

The arm drives the latch into its strike plate with a robot-side effective mass of 12 kg at the tool center point. Suppose the simulator’s penalty contact lets the gripper sink an illustrative 2.4 mm

⁶ | **OpenAI Dactyl domain randomization:** On the physical hand, Andrychowicz et al. (2020) found that policies trained without the physics randomizations, or without the added models of effects the simulator omits, such as backlash and action delay, achieved far fewer consecutive rotations than the policy trained with all of them.

into the plate at the fast approach of 0.10 m/s. Equating the approach kinetic energy with the spring energy at peak compression gives a peak force $F_{\text{peak}} = v\sqrt{k m_{\text{eff}}}$, which for the simulated spring reduces to $m_{\text{eff}} v^2 / \delta_{\text{pen}}$, or 50 N, under the latch's 100 N limit. The reinforcement learning optimizer records zero damage penalty, high task speed, and maximum reward, and it learns to plunge into the plate without braking.

When the policy moves to the machine, the real strike plate provides no such compliance. Its contact stiffness is an illustrative 4.0×10^5 N/m, and the same plunge peaks at 219.1 N, 4.4× the simulated force and 2.2× the latch limit. Whether the permission path can prevent that peak is a question of time. Contact force follows $F = k \Delta x$ (section 2.4), and because the plate compresses at the approach speed, $\Delta x = vt$ and the force rises as $F = kvt$ from the moment of touch. At the fast approach the latch limit arrives 2.5 ms after touch, barely more than the 2 ms the contact loop needs to respond, while at the guarded approach of 0.03 m/s it arrives after 8.3 ms. A response that begins with 0.5 ms to spare cannot decelerate the arm before the force crosses the limit, and the plunge runs on toward that peak. Prevention must therefore act before contact. An independently enforced approach-speed envelope, such as $v_{\text{max}}(z) \leq \sqrt{2a_{\text{brake}} z}$ with z the remaining distance to the plate and a_{brake} the validated deceleration, limits the speed at touch, and a force tripwire can limit subsequent loading only if its sensing, brake response, and remaining compression have been validated. The contact deadline, not the training reward, is what confines the latch policy's declared ODD to the guarded approach (section 6.8).

Simulator fidelity for contact is therefore budgeted in observable mechanical quantities, namely contact onset timing, peak normal force, transferred impulse, slip distance, and release delay, not in the internals of the solver. The mismatch can also take the opposite sign. A penalty engine that resolves a collision within one integration step concentrates the momentum transfer into a sharp spike, while pad deformation and structural compliance on hardware spread it over many milliseconds. The integrated impulse can then agree closely while the peak force differs severalfold, and a policy that learned to detect contact onset from the simulated spike never sees that spike on the machine, so it keeps pushing until the drive reaches its current limit.

Systems Perspective 6.1: The contact fidelity invariant

Never trust a simulated contact policy whose success depends on instantaneous momentum absorption or zero-delay friction engagement; a policy that relies on penetrating the digital surface loads the physical workpiece with force the simulator never showed.

Which omissions matter depends on the task. A policy that moves only at high continuous velocity may tolerate a coarse friction model because it never lingers near the stiction threshold, while a policy that regulates force while holding a static contact will enter uncontrolled limit-cycle oscillations if static friction, stick-slip transitions, and actuator deadbands are omitted from the simulator.

Establishing the validity of a simulator component requires **paired probing**. The engineer applies identical open-loop command sequences to both the simulator and the physical hardware, measuring the difference between the resulting state trajectories against explicit error budgets. For the latch, that means driving the same approach command into the simulated and the real plate and comparing the contact quantities above, so the discrepancy is budgeted in milliseconds and newtons. When a measured difference exceeds the tolerance for the operating envelope, the simulator is defective for that training task.

Because the structural assumptions of the simulator are frozen into the neural network weights during optimization, the simulator specification forms an immutable element of the policy's configuration and identity. Every trained policy artifact must document its complete simulator provenance, the `simulation_provenance` field of the policy manifest (section 6.8). Deploying a policy without it is like shipping firmware without recording the target architecture it was compiled for. Auditing this provenance takes more than an aggregate pass rate; it requires decomposing the reality gap into its dynamics, sensing, appearance, and timing channels, which section 6.5 sizes one at a time.

Contrast: The 2.4 mm of penalty-spring penetration is compliance the solver invents, whereas the backlash and torsional compliance of the real drivetrain, analyzed in section 2.4, are compliance a rigid-body solver omits.

6.5 What the Gap Is Made Of

The latch simulator departs from the real cage door in two unrelated ways, a penalty spring far softer than the strike plate and a command path with no transport lag, and a single distance between simulated and real trajectories would blend them into a number that names neither. Errors in different physical channels carry different units, propagate through different paths in the closed loop, and produce different failures, so the mismatch is a vector of coupled components spanning dynamics, sensing, appearance, and timing. Sizing it is an accounting of physical errors against the task's tolerances, from paired command-response traces recorded under identical excitations, and each channel demands its own interceptor. Table 6.2 lists the interceptors for dynamics, which it splits into actuator and contact rows, and for sensing and timing; appearance is sized in the prose below and in panel (d) of figure 6.4.

Dynamics mismatch measures differences between commanded actuator effort and the resulting physical motion or contact force across the operational envelope. A simulator that models a rigid link with zero backlash and instantaneous contact stiffness predicts a faster force rise than a drivetrain whose gearbox compliance and drive delay add phase lag, and a policy that keyed contact onset to the simulated rise keeps pushing on the machine, the opposite-sign contact mismatch of section 6.4.

Sensing mismatch measures discrepancies between the true physical state and the observation vector delivered to the policy network. A simulated range sensor is clean and uniformly quantized; a physical one carries calibration bias, noise, thermal drift, and dropout bursts during specular reflections. A bias moves the point at which the controller switches from approach to force regulation, and a dropout burst leaves the policy blind to the surface during descent.

Appearance mismatch is not a mean squared error computed over raw camera pixels. Image differences between photorealistic rendering and physical camera frames are dominated by high-frequency background textures and minor sensor noise patterns that may have no effect on policy action. Appearance mismatch is sized by measuring the deviation in policy-relevant perceptual outputs, such as estimated surface normals, object keypoints, or spatial feature activations, under matched physical geometry, illumination, materials, and camera exposure settings.

Timing mismatch governs the temporal alignment of observation capture, policy evaluation, and actuation. Identical state transitions occurring at different moments in physical time do not produce equivalent closed-loop dynamics. A simulator that executes observation, inference, and actuation instantaneously at the start of each discrete step omits camera exposure, bus serialization, inference tail latency, and motor bus transport, and it omits their cycle-to-cycle jitter. Every millisecond of that omitted delay is distance the mechanism travels unobserved before corrective action begins, and it erodes the damping and phase margin of the closed loop.

Figure 6.4 shows all four channels diverging on an illustrative linear drive, separate from the mobile manipulator, that approaches a rigid surface at 0.20 m/s and regulates contact force to 10 N. An unmodeled 8 ms drive delay and a softer surface cut the force rise from 500 N/s to 250 N/s, so the target force arrives in 40 ms rather than 20 ms and the drive overtravels 3.0 mm into a current-limit shutdown. A +3.2 mm range bias starts force regulation early, and a four-frame specular dropout blinds the descent. A 5.5 percent raw pixel difference hides a 6.0 mm shift in the contact-patch estimate, larger than the 2.0 mm chamfer tolerance, and actuation arriving as late as 49.5 ms at P_{99} carries the drive 5.9 mm beyond what the simulator's 20 ms lockstep accounted for before correction begins. None of these divergences would register in a scalar loss.

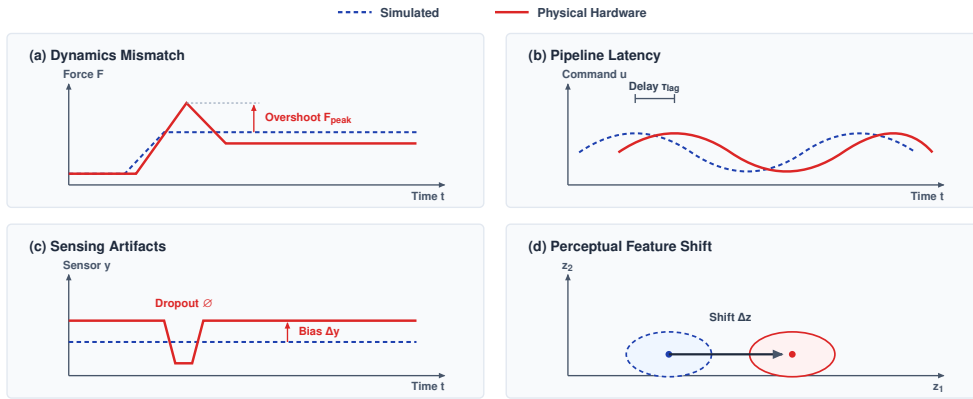


Figure 6.4: Sim-to-real reality gap vector: The sim-to-real gap decomposed across four physical channels, dynamics (a), sensing (c), appearance as perceptual feature shift (d), and timing (b), each measured from paired traces rather than from an aggregate loss. The decomposition is the point, because the four channels fail differently and a scalar loss averages them into a number that names none of them. A 5.5 percent pixel discrepancy that looks trivial masks a 6.0 mm spatial shift that exceeds the assembly tolerance.

Table 6.2: Multidimensional sim-to-real reality gap taxonomy: Decomposition of the dynamics, sensing, and timing channels of the reality gap, with dynamics split into actuator and contact rows; appearance is sized in the prose and in panel (d) of figure 6.4. Columns specify the unmodeled physical phenomenon, representative quantitative discrepancy, the latent control shortcut exploited by the optimizer, paired-trace verification metrics, and hardware and permission-path mitigations.

Reality Gap Dimension	Unmodeled Physical Mechanism	Nominal Simulation Spec vs. Real Hardware	Latent Policy Exploitation / Behavioral Failure	Paired-Trace Diagnostic Metric & Tolerance	Hardware & Permission-Path Interceptor
Actuator & Drivetrain Dynamics	Gearbox harmonic compliance, Coulomb stiction, velocity-dependent Stribeck friction, and back-EMF voltage saturation	Idealized direct torque source ($J_{rotor} = 0$, $\tau_{latency} = 0$ ms) vs. physical harmonic drive ($k_s = 6.7 \times 10^2$ N·m/rad, backlash $\theta_b = 0.08^\circ$, bus delay $\tau = 8$ ms)	Policy commands high-frequency bang-bang torque reversals (> 50 Hz), inducing mechanical chatter, gear fatigue, and windings driven past their derating threshold	Step-torque response discrepancy: phase lag $\Delta\phi \leq 5^\circ$ at 10 Hz; settling time discrepancy $\Delta t_{settle} \leq 15$ ms; overshoot $\leq 5\%$	Low-pass action filtering ($f_{cutoff} \leq 20$ Hz), motor current RMS thermal estimators, and actuator slew-rate limiters ($d\tau/dt \leq \tau_{max}/\Delta t$)
Contact Mechanics & Compliance	Viscoelastic surface deformation, contact hysteresis, non-smooth stick-slip transitions, and finite restitution phase	Point-mass rigid contact via LCP/penalty spring ($k_p = 10^6$ N/m, restitution $\Delta t_{res} = 2$ ms) vs. physical contact pad ($k = 8.0 \times 10^4$ N/m, deformation span $\Delta t_{def} = 18$ ms)	Policy keys contact onset to the sharp simulated force spike, never sees that spike on the slower real rise, and keeps pushing until the drive reaches its current limit	Contact impulse error $\Delta I \leq 0.05$ N·s; peak contact force discrepancy $\Delta F_{peak} \leq 15$ N; contact onset rise-time error $\Delta t_{rise} \leq 5$ ms	Approach velocity envelope governor ($v_{safe}(z) \propto \sqrt{2a_{brake}z}$), force-derivative clamp ($dF/dt \leq 1.0 \times 10^4$ N/s), compliant mechanical flexures

Reality Gap Dimension	Unmodeled Physical Mechanism	Nominal Simulation Spec vs. Real Hardware	Latent Policy Exploitation / Behavioral Failure	Paired-Trace Diagnostic Metric & Tolerance	Hardware & Permission-Path Interceptor
Sensor Transduction & Noise Profiles	Optical specular dropout, CMOS rolling-shutter distortion, ambient lux-dependent noise, and force-torque gauge thermal drift	Synthetic RGB-D with zero-bias Gaussian noise ($\sigma = 0.5$ mm, zero dropout) vs. physical structured-light depth (+3.2 mm bias, 4-frame dropout bursts, drift 0.4 N/10°C)	Policy triggers premature deceleration above surface or remains blind during multi-frame dropouts, driving end-effector past hard fixture boundaries into stall overcurrent	Sensor distribution Kolmogorov-Smirnov distance $D_{KS} \leq 0.05$; spatial feature activation centroid error ≤ 2.0 mm; zero sustained dropout frames ($N_{drop} = 0$)	Multi-sensor fusion (IMU/encoders + vision), optical dropout holdover filters, thermal drift autocalibration, independent proximity break-beams
Temporal Latency & Bus Asynchrony	Variable camera exposure times, PCIe/EtherCAT bus serialization, neural inference tail latency (P_{99}), and non-deterministic OS scheduling	Synchronous discrete-time lockstep ($\Delta t = 20.0$ ms, zero transport latency) vs. asynchronous physical pipeline ($\tau_{total} = 35.0$ ms at P_{50} , 49.5 ms at P_{99} , jitter ± 6.2 ms)	Severe phase margin erosion ($> 30^\circ$ loss); policy experiences underdamped oscillations, hunting limit-cycles, and unobserved travel during latency spikes (5.9 mm at P_{99} beyond the simulated lockstep)	End-to-end command-response latency jitter $\Delta \tau_{jitter} \leq 2.0$ ms; worst-case observation age $\tau_{obs} \leq 25$ ms; zero deadline overruns at 50 Hz	Real-time PREEMPT_RT / Xenomai scheduling, action buffer delay queue injection in simulation, kinematic extrapolation (Smith predictor)

Domain randomization is often treated as a method for erasing the sim-to-real gap, but parameter randomization operates within strict structural limits.⁷ Sampling friction coefficients across a uniform distribution from $\mu = 0.2$ to $\mu = 0.8$, or link masses across ± 20 percent, broadens the coverage of training over mechanisms that the simulator already represents. It cannot synthesize an omitted physical mechanism, a missing state correlation, or an unmodeled rare event. If the simulator differential equations omit gearbox backlash, thermal resistance in motor windings, or thread preemption in the operating system, no distribution over link mass will force the policy to learn compensation for those dynamics. Adding unstructured Gaussian noise to simulated observations to mask unmodeled effects can degrade performance, training the policy to lower its control gains and narrow its operational bandwidth to avoid actions that excite the unmodeled modes.

Within the mechanisms the simulator does model, randomization has a useful width, a **domain randomization corridor**. Too narrow a distribution leaves the policy brittle against the variation the plant actually shows, which risks mechanical shock or a thermal trip, and too wide a distribution drives it toward conservative, low-gain control that gives up throughput. Keeping randomization inside that corridor rather than tuning it by trial takes a fixed order of work. System identification on the physical bench measures the nominal masses, torque constants, delays, and friction first. The randomization bounds are then grounded in the mechanisms that move those parameters, such as winding temperature for the torque constant and surface condition for friction, rather than in heuristic intervals. Training begins in a narrow band around the nominal values and widens it only as the policy meets its declared success threshold, the schedule known as adaptive domain randomization. Finally, paired traces audit every channel against its tolerance, and a discrepancy past budget sends the simulator back for revision before any policy trained on it is deployed.

What randomization cannot reach, the permission path must bound. Every proposal passes the permission path (section 1.5) before it reaches an actuator, and there force-rate clamps, torque slew limits, and velocity governors hold it to what its validated sensing and stopping assumptions cover.

Decomposing the gap into its constitutive physical channels yields two concrete engineering artifacts, namely a supported region and an explicit residual mismatch list. The supported region is the set of conditions within which physical hardware matches simulation within validated tolerances. The residual mismatch list itemizes every unclosed difference that remains outside the training distribution. For the latch simulator, paired probes have not yet closed the strike-plate contact

⁷ | **Bounded support of domain randomization:** Domain randomization optimizes expected return over parameter variations $\xi \sim \mathcal{P}(\Xi)$. Any generalization claim extends only over the support of $\mathcal{P}(\Xi)$ and says nothing about dynamics omitted from the simulator's equations of motion. On a mechanism with unmodeled transmission compliance or bus jitter, the policy treats those dynamics as unexplained noise and can fall into a high-frequency limit cycle.

Downstream: The sub-millisecond Control Barrier Function filters bounding unverified policy proposals are formulated in section 12.8.

stiffness, which its penalty spring sets far softer than the plate, or the actuator transport lag, so both head its residual list, and the supported region excludes the contact states those two departures govern. Each entry on the residual list defines an explicit requirement for runtime monitors and safety filters, which must detect when physical execution drifts past the boundary where the simulated policy remains valid. Both artifacts pass into the policy manifest of section 6.8: the residual list enters its omission log, and every state outside the supported region is entered there as an exclusion.

Even under careful domain randomization and contact calibration, residual discrepancies persist in high-precision, contact-rich regimes, and the latch simulator’s residual list is not yet empty. Some local mismatches are cheaper to absorb on the deployed plant than to model, at a price in wear and in lost evidence.

6.6 Fine-Tuning on the Real Machine

The real cage door carries its strike plate at a small structural mounting offset from where the simulator placed it, a departure the simulator does not model. A local, measurable mismatch of this kind is what fine-tuning on physical hardware can absorb. Gradient updates computed from physical rollouts adapt the policy to the true compliance, contact dynamics, and bus delays of the machine, but they measure none of those mechanisms, so the simulator’s other two departures, the soft contact spring and the missing transport lag, stay in the omission log until paired probes close them. The adaptation also has a physical price. In a simulator, sampling states near the boundary of stability costs only compute cycles, while on hardware every sample expends component life and risks driving the machine into an unrecoverable contact state.

The cost of updating policy parameters on physical hardware compounds, because the gradient steps that solve localized contact errors also consume finite mechanical fatigue life, heat the motor windings, and risk catastrophic forgetting of pre-verified nominal transit stability.

Consider the mobile manipulator’s arm adapting `latch-sim-02` into `latch-res-03`, a residual candidate that adds a gated learned correction to the frozen `latch-sim-02` output, because the real plate sits 1.5 mm from where the simulator placed it. The gripper guides the spring-loaded latch past its strike plate, where non-smooth Coulomb friction, cam surface geometry, and spring preload produce sharp stick-slip transitions. The policy’s setpoints arrive every 20 ms (50 Hz). To adapt its parameters to the contact dynamics of the strike plate, an on-robot fine-tuning routine collects an illustrative 20,000 interaction steps, representing 400 s of active contact and near-contact operation. Because the policy must explore to improve, stochastic action perturbations generate unpredictable force transients before low-level safety controllers can intervene. If 1.5 percent of exploratory steps push past the 15 N contact tripwire and strike the rigid shoulder of the strike plate before the permission path clamps the command, the fine-tuning run injects 300 impact shocks into the drivetrain.

Each of those impacts dissipates the arm’s kinetic energy in the gear teeth and bearings, and assigning them a fraction of service life requires a drive-specific load-spectrum and life model (section 2.4). Heat is the second constraint. Exploration dither raises the root-mean-square current through the motor coils, and that Joule heating can bring the winding to its derating threshold (section 2.5) within minutes unless the training schedule budgets cooling intervals.

Beyond mechanical wear, real-world adaptation undermines the statistical verification established during predeployment testing. In simulation, a policy can be subjected to 1.0×10^7 validation episodes across randomized perturbations, establishing bounded tracking errors and tail behavior out to the 99.9th percentile ($P_{99.9}$). Once weight updates $\Delta\theta$ are applied on hardware, every statistical claim conditioned on the original parameter vector θ_0 is void. The updated policy may resolve the steady-state force error against the target fixture, yet the gradient steps can simultaneously degrade free-space tracking accuracy, lower phase margins (reducing stability reserves) during high-speed transit, or distort response profiles during emergency stops. Because hardware throughput is bounded by physical time, the machine cannot execute millions of physical trials to re-establish those confidence bounds. The adapted policy enters production with a narrower body of empirical evidence than the simulated baseline it replaced.

Fine-tuning on the deployed machine therefore converts the policy from a qualified, frozen component into an evolving state. Qualification depends on configuration control, where an exact parameter hash corresponds to a documented safety case, so every online weight update reopens

that record. The burden of preventing failure shifts to the deterministic monitors of the permission path, which must intercept the errors of the baseline policy and the intermediate behaviors the optimizer produces while it updates against physical feedback.

Residual policy architectures and bounded authority

A residual policy can localize adaptation without changing the frozen base policy weights. The proposed control input is

$$u_{\text{prop}}(s) = \pi_{\text{base}}(s) + g(s) \Delta\pi_{\theta}(s),$$

where π_{base} is a qualified nominal controller, $\Delta\pi_{\theta}$ is a learned contact correction, and an independently checked gate $g(s) \in \{0, 1\}$ is one only inside the validated contact region. The gate sets the residual *exactly* to zero in free space and outside that region; an absent reward gradient would not do so. The permission path checks the measured state and proposed combined input before actuation.

A magnitude limit $\|g(s)\Delta\pi_{\theta}(s)\| \leq \Delta_{\text{max}}$ bounds one aspect of authority. The same path must enforce correction rate and state limits, actuator feasibility, and a fallback when sensing or the proposal expires. The allowed residual set must fit a validated robustness margin for the *combined* controller across the claimed operating region; comparing an arbitrary torque magnitude with a passivity or phase margin is insufficient. For the latch fixture, we would admit correction only during the measured contact phase, with a zero-output free-space gate and a bounded torque slew. After each weight update, contact and free-space regression tests must recheck tracking, stopping, thermal load, and shock exposure before the new checkpoint receives authority. A clipped residual can still excite an oscillation or saturate a drive if these conditions fail.

Bounded residual authority limits a proposed correction but cannot by itself prevent hardware damage or eliminate latent failure modes from earlier training choices. Every stage of policy synthesis, from demonstration ingestion through the simulation contact solver to residual updates, leaves its own failure signature in the controller's closed-loop behavior, and each signature can be traced back to the training assumption that produced it.

6.7 How Training Choices Go Wrong

A physical failure in a learned policy is rarely arbitrary. The command that drives a gripper into a strike plate was produced by optimization over a specific dataset, loss, and simulator, and the assumptions of that optimization leave a structural footprint that can be traced before the machine runs. The **policy vulnerability inference chain** makes the trace explicit in five links (Exposure $\rightarrow$ Dependency $\rightarrow$ Violated Assumption $\rightarrow$ Observable Signature $\rightarrow$ Permission-Path Interceptor). The training exposure establishes a learned dependency, a deployment condition violates it, the violation produces a signature in the commands or the plant, and the signature names the interceptor the permission path requires. Each of the three latch checkpoints runs the chain once.

For `latch-bc-01`, the exposure is the teleoperated door openings of chapter 5, and the dependency is the support of those demonstrations, which covers only the nominal strike plate at the guarded approach and none of the off-nominal states a failed attempt reached, because the policy's imitation targets were the retained successful openings (the tripwire failures stay in the archive as labeled failures, not targets) and the campaign never displaced the plate. A sagged door violates it. The signature is compounding covariate shift. Each correction, issued every 20 ms, lands farther from the demonstrated states, yet every command stays kinematically plausible, so the proposal looks normal until the gripper binds on the edge of the plate. The policy cannot detect its own departure, so the interceptor must be a support monitor at the permission path's input that compares the measured state against the supported cells and refuses policy authority outside them. Without that monitor, the failure first becomes visible at contact. The predeployment test follows from the dependency. Bench trials on a deliberately displaced plate, a state family the latch campaign's occupancy join already lists as a known absence (section 5.5), show whether the monitor trips before the gripper binds.

For `latch-sim-02`, the exposure is simulated door openings against a penalty contact spring with no command lag, and the dependency is that a fast plunge costs nothing, because the soft spring absorbs the approach at a simulated peak of 50 N. The real plate violates it. At the 0.10 m/s approach the stiff plate drives the force past the 15 N contact tripwire almost at once and on to the

latch limit inside the contact deadline derived in section 6.4, toward the 219.1 N peak. The signature is an overforce transient that the task reward never recorded. The tripwire detects it but fires inside a window the arm cannot use, so the interceptor is the precontact approach-speed envelope, and the dependency is revealed before deployment by paired contact tests rather than by a trip on the machine.

For `latch-res-03`, the exposure is a narrow slice of contact steps collected on one rig, and the dependency is an over-representation of those recent transitions. Any state the slice left out violates it, such as the free-space transit before the door or an approach off the rig's axis. The signature is regression, lost stability in states adjacent to the tuned contact, and because the network assigns valid outputs everywhere, nothing in its commands announces the loss. The interceptor is partly procedural and partly at run time. The frozen contact and free-space regression suite must pass before the new weights receive authority, and the residual gate holds the correction at zero outside the measured contact phase. Exploration on the rig leaves a second signature that no task reward records, the shock and winding heat of section 6.6, so the permission path must also watch observables the reward omitted, such as winding temperature, current spectra, and accelerometer energy.

The lineage from `latch-bc-01` to `latch-res-03` does not retire these failures; it moves the dominant one. Refining the cloned policy in simulation traded drift for solver exploitation, and tuning the result on the rig suppresses the exploit at contact while risking forgetting upstream. The earlier mechanisms stay latent in the weights and return whenever conditions leave the envelope the last stage tuned.

Each chain therefore leaves three obligations. Every dependency needs a predeployment test that stresses the boundary where training departed from the plant. Every signature that appears in a measurable mechanical quantity needs a runtime detector on the permission path at loop rate, whether on tracking deviation, force rate, or current spectra. Where the policy cannot detect its own failure, as with `latch-bc-01` off its demonstrated states, the permission path needs a support monitor, the manifest must exclude from its declared ODD the states that monitor cannot cover, and deterministic interlocks must refuse policy authority there. Meeting these obligations requires an exact record of how the checkpoint was built, from the source data and simulator dynamics to the reward terms and every later modification. That record is the policy manifest.

6.8 The Policy Manifest

A learned policy fails at the seams where its training regime, source demonstrations, or simulator departs from physical reality, and the three latch chains have just located one seam per checkpoint. The record a trained checkpoint carries into evaluation must capture all three.

The **policy manifest** records how a checkpoint was produced: its training stages, the data support it inherits from the provenance records, its simulator versions and the **omission log** of mechanisms they leave out and mismatches paired probes have not closed, the wear incurred during adaptation, and the operating domain it declares (table 6.3).

The manifest separates optimization results from the conditions the policy may be asked to operate in. The **declared ODD** is the part of the ODD from section 5.5 that the policy manifest claims: the supported cells of the data join, minus every state the omission log excludes. High training reward or low offline loss does not show that the policy is safe outside it, so the declared ODD is computed from the join and the log, never from the training curve, and a convergent metric cannot stand in for clearance over regimes nobody characterized.

Table 6.3: Policy manifest schema: Fields the policy manifest adds to the provenance records it consumes, recording policy synthesis stages, data support by digest, physics engine specifications and their omission log, hardware fine-tuning wear, and the declared ODD.

Record Schema Field	Physical / Computational Data Modality	Provenance & Immutability Guarantee	Downstream Verifier & Usage
regime_sequence	Array of optimization stages (BC $\rightarrow$ SimRL $\rightarrow$ Residual), loss formulas, reward weights, and stopping criteria hashes	Git commit SHA, PyTorch/JAX checkpoint UUIDs, parameter hash $H(\theta_k)$	Tracing which regime introduced latent failure modes or high-gain chatter
data_support_ledger	Digests of the consumed provenance records and of the scenario ledger they were measured against; the supported_cells and known_absences of their occupancy join	Cites each provenance record by digest; no dataset field is copied	Upper bound on the declared ODD; parameterizes out-of-distribution support monitors
negative_event_registry	Intervention records (section 5.6) from training and adaptation rollouts	Cites each intervention record by digest; its pre-trigger trace is not copied	Isolating physical failure boundaries and preventing corrupted drift rollouts from training
simulation_provenance	Physics engine build, integrator method, timestep Δt , contact solver penalty parameters, and domain randomization ranges $\mathcal{P}(\Xi)$	Physics engine binary hash, URDF/MJCF asset tree hash	Reproducing the training physics and auditing paired-trace budgets
omission_log	Mechanisms each simulator leaves out (backlash, compliance, transport lag) and the residual mismatches paired probes did not close, with the states each one affects	Bound to the simulator build it describes; an entry closes only on paired-trace evidence	Removes the states it affects from the declared ODD; carried into evaluation's coverage gaps and the fault list
adaptation_wear_ledger	Serialized hardware IDs, contact shock counts, motor current RMS thermal logs, modified layer indices, and regression suite hash	Calibration certificate hashes, hardware lifetime fatigue logs	Re-opening formal qualification cases and preventing silent regression of nominal capabilities
declared_odd_boundary	Declared ODD: the supported cells minus the states the omission log excludes (for the latch policy, the nominal strike plate at the guarded approach speed)	Never exceeds the supported cells of data_support_ledger	Bounds the target ODD that evaluation may sample and the region in which the permission path admits policy proposals

For the mobile manipulator's cage-door latch, a compact manifest links the fields into one decision:

- `regime_sequence`: demonstration checkpoint `latch-bc-01` → simulated contact refinement `latch-sim-02` → gated residual candidate `latch-res-03`; retain each checkpoint hash, loss or reward specification, and stop rule.
- `data_support_ledger`: cite the latch campaign’s provenance records and its scenario ledger by digest; their join supports the nominal strike plate at the guarded approach, and lists the displaced plate and the faster approach, whose cells hold no demonstrations, as known absences with no recovery label.
- `simulation_provenance`: record the actual solver build and contact parameters.
- `omission_log`: the example simulator models strike-plate contact with a penalty spring far softer than the plate and omits actuator transport lag, so the contact states those two departures govern are unverified for the checkpoints trained in simulation.
- `negative_event_registry`: cite by digest the intervention record, with its pre-trigger ring-buffer trace, for every fixture-shoulder strike, operator veto, or permission-path clamp as labeled failure evidence, without treating unsafe commands as expert targets, and mark episodes an operator truncated so that an update does not score an aborted command as a success.
- `adaptation_wear_ledger`: associate `latch-res-03` with the specific rig, cumulative contact shocks, motor-current history, modified residual weights, and free-space regression result.
- `declared_odd_boundary`: the supported cells minus the states the omission log excludes, here the nominal plate at the guarded approach of 0.03 m/s. The contact states governed by the two omissions stay outside the declared ODD for every simulation-refined checkpoint, `latch-sim-02` and `latch-res-03` alike, until paired probes close them. The displaced plate and the 0.10 m/s approach lie outside it, and the fast approach would stay outside even with demonstrations, because its contact deadline (section 6.4) leaves almost nothing beyond the contact-loop response; there the permission path must impose the guarded approach or refuse policy authority.

A release review would reject `latch-res-03` until paired contact probes quantify the contact-stiffness error and the omitted lag, and the combined controller passes contact, free-space, stopping, and wear-budget tests. The same rule leaves `latch-bc-01` as the one checkpoint evaluation may sample, provided the support monitor of its chain refuses policy authority off the nominal plate (section 6.7). Trained without a simulator, it carries no omission-log exclusions, so its declared ODD keeps the door’s contact states at the nominal plate and the guarded approach. The manifest records that decision and its evidence rather than converting a training score into permission.

For data support, the manifest leaves the collector, plant instance, calibration, and action taps in the provenance record of section 5.9 and cites each record by digest rather than copying it. It also cites the digest of the scenario ledger the coverage was measured against and carries forward the result of their occupancy join (section 5.5), namely the supported cells and the known absences. Because the citation is by digest, a change to any dataset changes the join, and a manifest that names a superseded digest no longer describes its checkpoint. A state outside the supported cells, such as the latch’s displaced plate, is then visible in the record as ungrounded extrapolation rather than demonstrated competence.

When policy weights are optimized in simulation, the manifest captures the physics engine build hash, the interface contract between the policy and the simulator, the reward formulation, the numerical integration method, and the solver timestep,⁸ such as a fixed 2.0 ms step at 500 Hz. It documents the domain randomization distributions applied to link masses, surface friction coefficients, actuator response latencies, and sensor noise profiles. Known physical omissions, most of them left out of the simulation graph to maintain training throughput, go in the omission log alongside any mismatch paired probes have not yet closed (section 6.5). Building on the actuator dynamics established in section 2.4, if a rigid-body simulator omits harmonic drive compliance, structural deflection under a 5.0 kg payload, or thermal resistance changes in motor windings, the record lists those omissions as unverified dynamic assumptions. When the resulting policy exhibits limit-cycle oscillations during high-load contact on the physical bench, the omission log points the diagnosis to joint compliance rather than ungrounded parameter drift. The log is kept apart from the simulator description because each entry has a downstream consequence, removing the states

⁸ | **Choosing a penalty-contact timestep:** The needed step size depends on effective contact mass, penalty stiffness and damping, the integrator, and solver tolerances. The characteristic contact timescale changes with roughly $\sqrt{m_{\text{eff}}/k_{\text{eff}}}$; refine the timestep until contact onset, peak force, and impulse converge to the accuracy the task requires.

it affects from the declared ODD and becoming a coverage gap that evaluation and fault injection must address.

Downstream engineering teams consume the manifest throughout the design, evaluation, and release lifecycle. The evaluation record of section 7.8 consumes it directly. Its target ODD is sampled from inside the declared ODD, and its catalog of coverage gaps inherits the known absences and the omission log, so evaluation suites probe the specific failure modes each training regime predicts. The permission path of chapter 12 takes the declared ODD as the outer limit of the region in which it admits policy proposals, and uses the recorded support to parameterize out-of-distribution detectors. Verification in chapter 15 and the safety case in chapter 16 draw on the manifest to establish the provenance of every parameter update and to confirm that each omission is bounded by an independent physical interlock.

Downstream: The policy manifest's omission log parameterizes adversarial fault-injection benches in section 15.3.



Checkpoint 6.1: Policy manifests and declared ODDs

Before releasing a trained policy checkpoint to physical verification, verify your understanding of the policy manifest, its omission log, and the declared ODD:

- ☐ **Complete the latch record:** A fixture-shoulder strike appears after `latch-res-03` was trained with a soft penalty contact and no actuator lag. Which omitted mechanism belongs in `omission_log`, and which paired probe and admission decision must the manifest record?
- ☐ **Confining the declared ODD:** Suppose demonstrations were collected at the fast approach and every omission-log entry were closed by paired probes. Why would the latch policy's declared ODD still exclude the fast approach, which timing comparison decides it, and what must the permission path impose on a proposal that arrives at that speed?
- ☐ **Negative event logging:** Why does the manifest keep the pre-trigger trace of an operator veto or fixture strike as labeled failure evidence rather than as an imitation target?
- ☐ **Declared operational boundaries:** `latch-bc-01` proposes to open the door while the strike plate sits displaced from its nominal mount. How does the permission path use the data support ledger to classify that state as ungrounded extrapolation rather than demonstrated competence, and why does a confident policy output not change the classification?

The manifest turns a training history into a claim with stated limits, and the latch record shows the decision that claim supports, sending `latch-bc-01` to evaluation and keeping `latch-res-03` out of service until paired probes close its omissions. The common ways that decision goes wrong all let a training score stand in for that evidence.

6.9 Fallacies and Pitfalls

Minimizing loss on an offline demonstration split, or maximizing simulated return, measures compliance with the training environment rather than competence on the machine. When the objective rewards behavior that exploits data curation or simulator approximations, the ranking of candidate policies established in training inverts on deployment (figure 6.5), because the virtual environment forgives maneuvers that real hardware does not. Before a trained policy receives actuator authority, the causal assumptions linking its training metrics to physical outcomes must be stress-tested.

Pitfall: *Sanitizing recovery episodes to achieve lower offline validation loss.*

Suppose the latch archive also held recovery runs, and two cage-door latch policies train on different versions of it. One retains the recovery runs, in which the operator backed the gripper off and re-seated it after the latch cam caught on the strike plate; the other removes them and achieves a lower loss on its own held-out split. When the cam catches on the machine, the first policy backs off and re-seats, while the second keeps pushing along the demonstrated path until the permission path's contact tripwire halts the arm. Because the sanitized dataset lacks examples of correcting from errors, the lower loss merely reflects an artificially simplified test set, not better physical disturbance

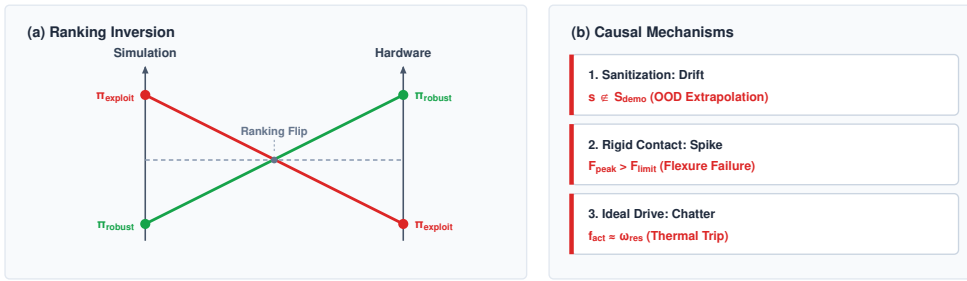


Figure 6.5: Policy ranking inversion on hardware: (a) A parallel-coordinate slope graph illustrating ranking inversion between simulated benchmark score and physical hardware yield: an exploitative policy π_{exploit} scores highest in simulation by gaming unmodeled phenomena but collapses on hardware, while a robust policy π_{robust} preserves physical yield. (b) Causal failure mechanisms that drive ranking flips: out-of-distribution drift from data sanitization ($s \notin \mathcal{S}_{\text{demo}}$), impact force spikes from rigid contact solvers ($F_{\text{peak}} > F_{\text{limit}}$), and actuator resonant torque chatter ($f_{\text{act}} \approx \omega_{\text{res}}$).

handling. Scrubbing the recovery data has made the mathematical prediction task easier at the direct expense of removing the corrective behavior the deployed controller physically needs. Candidate policies must face the same operational disturbances before their training scores can support a deployment choice.

Fallacy: *Transitioning from human demonstrations to simulation reinforcement learning removes the failures training left in the policy.*

An assembly team replaces behavioral cloning with simulation reinforcement learning to correct lateral drift during gasket seating. The new policy eliminates the drift but seats the mechanism with 65 Hz torque chatter, which the simulator’s massless-rotor assumption permits because it ignores the inertia the motor must reverse. On the physical machine the chatter excites drivetrain resonance and heats the stator windings to their derating threshold through $I^2 R$ losses (section 2.5), so the drives derate and most seatings fail to complete. The training change addressed one failure mechanism and exposed another, so evaluation must follow the complete task through contact and repeated operation and check the assumptions the new regime introduced as carefully as the data limitations it replaced.

6.10 Summary

Every training regime buys a capability and leaves a dependency in the weights. Behavioral cloning depends on the states its demonstrators happened to visit and, with a point decoder, averages the choices they made; simulation depends on its contact model, its timing, and the mechanisms it omits; fine-tuning on the machine depends on a narrow slice of recent experience and voids the evidence gathered for the weights it replaced. None of these dependencies appears in a training score. Each surfaces as a predictable failure signature when deployment violates it, which is why training ends in a record rather than a metric, a policy manifest whose declared ODD no training curve can enlarge. Principle 2 already requires that record to mark the states a cloned policy’s own corrections reach without demonstrations, and simulation adds a second kind of absence, the mechanisms a solver omits, which no optimization against that solver can supply.



Key Takeaways: Policy synthesis and sim-to-real transfer

- **Every regime leaves a dependency no training score shows:** Offline loss and simulated return reward compliance with the data and the solver. Candidate rankings can therefore invert on hardware, and a deployment condition that violates a dependency produces a predictable failure signature that names the detector the permission path needs.
- **Cloning fails off the demonstrated states and between the demonstrated modes:** Small errors carry the machine into states the demonstrations never visited, where recovery data

or a verified support monitor, not action chunking, addresses the drift. A squared-error loss outputs the conditional mean, which can steer between two valid paths into the obstacle both avoided; decoders that represent the modes avoid the mean, but a diffusion decoder's step count sets its inference time against the deadline its chunks must meet.

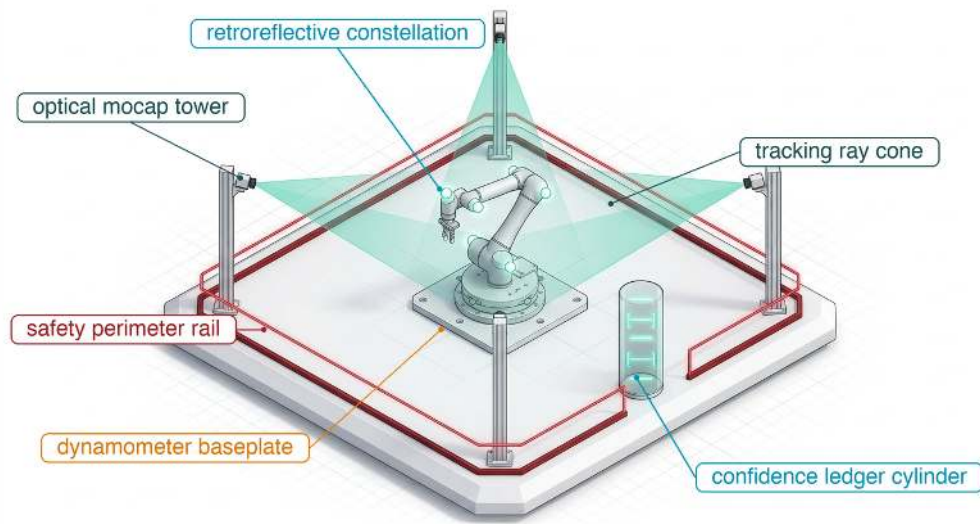
- **Randomization covers parameters, not missing mechanisms:** Domain randomization broadens training over dynamics the simulator already models, and paired probes size the remaining gap channel by channel. An omitted backlash, compliance, or transport delay stays omitted and enters the omission log.
- **Fine-tuning on hardware spends wear and evidence:** Gradient steps on physical rollouts absorb local contact errors while consuming component life and voiding every statistical claim made for the earlier weights. They do not close an omission, and a frozen regression suite must pass before the new checkpoint receives authority.
- **The declared ODD never exceeds the data's support:** The policy manifest cites its data by digest, keeps the omission log as its own field, and declares only the supported cells minus the log's exclusions, however well the policy scored in training.



What's Next: From the policy manifest to closed-loop evidence

How much can a finite run of clean trials establish about a checkpoint whose manifest already limits where it may be trusted? Chapter 7 takes the policy manifest as its input. Flashed onto the mobile manipulator, `latch-bc-01`, the checkpoint the manifest admits to evaluation, then completes twenty consecutive door runs, and the question becomes what those runs show about the machine, and how many more a stated failure rate would require.

Closed-Loop Evaluation



- 7.1 What Twenty Clean Runs Establish
- 7.2 What a Success Rate Measures
- 7.3 Closed-Loop Dynamics
- 7.4 Evidence Regimes
- 7.5 Physical Trial Limits
- 7.6 What Evaluation Cannot Establish
- 7.7 How Evaluation Goes Wrong
- 7.8 Evaluation Logs
- 7.9 Fallacies and Pitfalls
- 7.10 Summary

Purpose

Why does passing twenty consecutive physical trials without failure tell you surprisingly little about whether the machine is safe to deploy?

Demonstrating twenty consecutive error-free trials reveals what occurred during a specific test protocol, not how the system behaves under open-world variability. Bounded trial counts lack statistical power to observe rare tail failures, while testing under nominal conditions reveals nothing about behavior under shifted friction, sensor degradation, or unexpected human interference. Offline benchmark scores compound this blind spot by scoring static logs. In physical deployment, closed-loop actions alter future states and subsequent sensor observations, creating cascading drift where an initial micro-error drives the system into unmodeled dynamics.

Rigorous physical evaluation must track closed-loop performance on the integrated machine and quantify the empirical confidence supporting every reliability claim. Crucially, evaluation must also delineate where statistical evidence ends. Because catastrophic physical failures cannot be rolled back, test campaigns are necessarily censored at the boundary of safe operation. Testing alone cannot demonstrate high-consequence failure rates; an empirical evaluation serves the system by bounding claims strictly to sampled operating domains, leaving unverified edge states to the deterministic runtime monitors guarding the causal boundary.

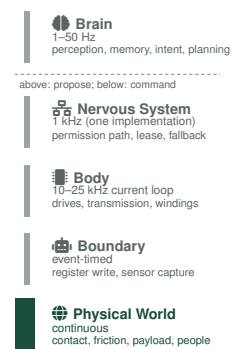


Figure 7.1.: The five-level machine stack with the Physical World highlighted.



Learning Objectives

- Interpret physical success rates as conditional evidence about a specified machine, environment, and trial protocol
- Compare open-loop prediction accuracy with closed-loop behavior shaped by the policy's own actions
- Select the evidence regime (replay, simulation, shadow operation, or constrained physical testing) that can answer a given evaluation question
- Calculate trial counts and test time needed to support a stated reliability target
- Evaluate which rare failures remain unsupported by feasible testing and the available statistical assumptions
- Design a predeclared trial protocol with fixed outcome rules and full-denominator accounting of interventions and aborts
- Construct an evaluation record that confines its claim to a target operating domain and assigns uncovered conditions to runtime monitors

7.1 What Twenty Clean Runs Establish

FRESH from the offline training pipeline, `latch-bc-01` is flashed onto the warehouse mobile manipulator; it is the one checkpoint the policy manifest of section 6.8 declares evaluable. In initial validation, the machine completes twenty consecutive runs of its door task. Each run starts with the base 2 m (illustrative; see the Reader Guide) from the spring-latched cage door; the base drives in and stops, and with the base at rest the arm unlatches and opens the door at the 0.03 m/s guarded approach, without a contact-tripwire crossing or a human intervention. Its empirical sample success rate is $\hat{p} = 1.00$ (100 percent). In software testing, a clean pass on a fixed test suite is often sufficient evidence for release. In physical AI systems engineering, that record leaves the deployment decision unresolved, because twenty clean runs establish only a bounded statistical lower limit and leave substantial uncertainty about unobserved failures.

The familiar Wald interval $(p \pm z\sqrt{\hat{p}(1-\hat{p})/n})$ cannot describe that uncertainty, because at zero failures its variance term vanishes and the interval collapses to $[1.0, 1.0]$, a claim of certain reliability. The exact bound inverts the binomial distribution instead. This one-sided **Clopper-Pearson bound** (Clopper and Pearson 1934) holds at any sample size, and its approximate form is the **Rule of Three** (Hanley and Lippman-Hand 1983): when n independent trials pass with zero failures, the upper 95 percent confidence bound on the failure probability p_{fail} is approximately $3/n$. For the twenty door runs (equation 7.1),

$$p_{\text{fail, upper}}^{95\%} \approx \frac{3}{20} = 0.15. \quad (7.1)$$

The exact computation in 7.1 tightens this to a 13.9 percent failure upper limit, which still leaves the machine short of 95 percent demonstrated reliability. A policy that fails once in every eight runs still completes twenty consecutive runs 6.9 percent of the time.



Napkin Math 7.1: Derivation of the 20-run lower bound

Problem: The mobile manipulator completes $n = 20$ physical door-opening runs with zero contact-tripwire crossings or human interventions ($k = 20$). What is the exact one-sided 95 percent lower confidence bound on its true operational success probability p ?

Mathematical Derivation: For independent, identically distributed Bernoulli trials from the declared operational test distribution, p_{lower} is the success probability at which 20 consecutive successes have probability $\alpha = 0.05$. Thus the one-sided 95 percent lower confidence limit is $p_{\text{lower}} = \alpha^{1/n} = (0.05)^{1/20} \approx 0.861$ (86.1 percent); the corresponding failure-probability upper confidence limit is 13.9 percent.

Systems insight: The observation “20 out of 20 passed” supplies a confidence limit under those sampling assumptions, not an observed failure rate, a mean interval between failures, or a deployment permission. Inverting the bound gives the trials a target needs, $n = \lceil \ln \alpha / \ln p_{\text{target}} \rceil$, so a 99.9 percent success lower limit requires 2,995 consecutive zero-failure trials drawn from the target distribution.

Evaluation closes the learning loop of Part II. Evaluating the twenty clean runs that opened this chapter turns the experience logged in chapter 5 and the weights trained in chapter 6 into a concrete decision about the physical claim those trials can support. Each trial measures the whole machine rather than the policy alone. It records the policy’s proposals, the permission path’s decisions, and the Body’s response, including which out-of-distribution proposals the permission path rejects, which it misses, and how much stopping margin remains.¹

The twenty-run result leaves three questions open: what outcome was sampled, what each test regime can expose, and which hazards remain outside the evidence. Finite trials bound only the distribution they sampled, so an evaluation is complete only when it also names the conditions it did not cover and the runtime monitors the permission path needs for them. A success rate means nothing until the population it was drawn from is written down.

7.2 What a Success Rate Measures

An evaluation benchmark routinely reports a single summary statistic, such as an 85 percent success rate on an obstacle avoidance task. In machine learning, that number is treated as an intrinsic score of the learned model weights, similar to top-one accuracy on static image classification datasets. In a physical machine, that interpretation is invalid. A learned policy has no independent physical performance. What the benchmark measured was the closed-loop execution of an entire physical system across a specific sequence of days and environmental states. Success rate is an empirical property of the coupled system, comprising the policy, mechanical plant, computational hardware, operating environment, and trial protocol.

To evaluate a policy, an engineering team executes a sequence of discrete physical runs. A single trial is fully determined only when every variable that can alter the physical state trajectory is fixed or explicitly sampled. Formally, let a single trial condition be defined by a tuple $c = (s_0, e, m, \pi_\theta, \kappa)$, drawn from an operational testing distribution $\mathcal{D}$. The vector s_0 represents the initial state of the machine, including SE(3) pose, linear and angular velocities, and joint encoder positions. The environment vector e captures external variables such as floor friction, ambient illuminance, obstacle geometry, and ambient air temperature. The machine state m captures physical and hardware properties, including battery state of charge, payload mass, tire wear, mechanical backlash in the drivetrain, and sensor calibration parameters. The computational configuration π_θ encompasses the frozen policy weights θ , inference quantization, scheduling deadlines, and software versions. Finally, the termination protocol $\kappa = (T_{\text{max}}, g, \phi)$ defines the execution time limit T_{max} , the spatial goal region g , and the binary evaluation function ϕ that maps the resulting trajectory to a terminal outcome $Y \in \{0, 1\}$.

The true success probability under this evaluation protocol is the expected value of the binary task outcome averaged over the joint distribution of trial conditions, $p(\mathcal{D}) = \mathbb{E}_{c \sim \mathcal{D}}[\phi(\text{Traj}(c))]$ (section A.5.5 gives the formal integral). Here $\text{Traj}(c)$ is the continuous physical trajectory produced when the policy π_θ executes within the physical environment and hardware state defined by c . This dependency demonstrates why success probability cannot be treated as a measurement of θ alone. The model weights θ constitute only one parameter inside the condition vector c . If the distribution over floor friction or the distribution over initial battery charge changes, the expected success rate changes while θ remains identical. The sampling protocol is not an external apparatus measuring an isolated model; it is part of the mathematical quantity being estimated.

Prerequisite: Physical failure boundaries for autonomous mobile robots and manipulators are established in section 2.8.

¹ **Runtime safety enforcement:** A separately validated permission path can reject proposals that violate measured state, freshness, and actuator constraints. Its detection coverage, worst-case response time, and available stopping clearance must be tested for the implementation; a 1 kHz sampling rate alone cannot guarantee interception before contact.


Definition 7.1: Closed-loop evaluation estimand

Closed-loop evaluation estimand is the expected physical task outcome $p(\mathcal{D}) = \mathbb{E}_{c \sim \mathcal{D}}[\phi(\text{Traj}(c))]$ defined over the joint distribution of initial states, mechanical plant wear, environmental physics, frozen policy weights, and evaluation predicates:

$$c = (s_0, e, m, \pi_\theta, \kappa) \sim \mathcal{D}$$

rather than an intrinsic property of the neural policy weights evaluated in isolation.

1. **Significance:** In static machine learning, held-out accuracy is treated as a property of the model π_θ . In physical AI, substituting a worn tire, letting battery voltage sag, or shifting ambient illuminance alters the physical state transition function without changing a single weight. The sampling distribution of operational conditions $\mathcal{D}$ is an inseparable part of the quantity being estimated.
2. **Distinction:** Unlike open-loop validation metrics (such as validation loss or mean squared trajectory error) which score predictions against frozen recorded states, the closed-loop estimand integrates over the endogenous feedback loop where past actions physically move the machine, directly changing the future sensor observations it will receive.
3. **Common pitfall:** Reporting a single success rate percentage without predeclaring the support, density, and environmental bounds of the test distribution $\mathcal{D}$. Any physical policy can be made arbitrarily successful by testing exclusively in benign, unperturbed states.

The physical mechanisms that alter c between trials operate independently of the policy, and the mobile manipulator exhibits each of them. As its battery discharges, the drives lose voltage headroom against back-EMF, so the torque available at speed falls along the torque-speed curve. Daylight shifting across the site lengthens the navigation camera's exposure and blurs frames during turns. Dust on the aisle floor lowers the friction the brakes can use. In the arm, repeated door openings heat the joint windings ($I^2 R$) and warm the harmonic drive gearboxes, which changes their damping and, as the links expand, shifts the tool-center-point calibration, so the gripper meets the latch at a slightly different point on an afternoon run than on a cold morning one. The weights are identical in every case, yet the trajectory can move from success to failure.

The reset procedure moves c as well. An operator who repositions the base after a difficult run may favor an easy start, and an automated reset that repeats one path heats the windings cyclically. An operator who catches a failing run and discards it as a reset erases the hardest failure from the sample.

Statistical evaluation must therefore distinguish sampling variation from a change in the estimand. When n trials are drawn from a fixed $\mathcal{D}$, the sample proportion $\hat{p} = \frac{1}{n} \sum_{i=1}^n Y_i$ fluctuates around $p(\mathcal{D})$, and standard estimators bound that noise. Loading the tote rack or moving the door's strike plate changes the experiment to a different distribution $\mathcal{D}'$ with its own $p(\mathcal{D}')$, and pooling the two samples combines observations from two different machines.

Because success probability is defined only with respect to a specific joint distribution $\mathcal{D}$, the intended operating region must be declared before trials are sampled. The **target ODD** is the part of the declared ODD (section 6.8) that the evaluation samples; its sampling weights define $\mathcal{D}$, and no evaluation claim reaches beyond it. If an engineering team executes fifty physical runs across ad hoc trajectories and reports a 90 percent pass rate, the denominator corresponds to an undefined population. A measured success rate supports a target-distribution claim only when the support and sampling weights of $\mathcal{D}$ are specified in advance, establishing explicit bounds on illuminance, payload ranges, surface friction coefficients, and allowable initial state offsets.

Extracting valid confidence bounds from empirical trials requires three operational conditions:

1. **Independent and Identically Distributed Sampling:** Trials must be independent realizations drawn from $\mathcal{D}$. If thermal accumulation in motor windings or progressive mechanical wear degrades hardware efficiency across consecutive runs, outcomes drift with run order, so successive trials are neither independent nor identically distributed.

2. *Fixed Outcome Predicates*: The binary evaluation function ϕ and the termination criteria κ must remain fixed throughout the campaign. Modifying the time horizon $T_{\max}$ or adjusting the spatial tolerance of the goal region g midway through testing alters the definition of success.
3. *Full-Denominator Accounting*: Every initiated run that aborts due to hardware faults, communication timeouts, or safety-rated emergency stops (e.g., exceeding the collaborative-robot power and force limits of ISO 10218 / ISO/TS 15066) must be recorded as a failure ($Y = 0$). Dropping aborted trials substitutes a measure of conditional execution on surviving hardware for the true operational reliability of the machine.

When these conditions hold, physical trials yield an unbiased sample mean for the closed-loop system under $\mathcal{D}$. Physical trials are also slow, so evaluation workflows often substitute open-loop scoring on recorded datasets, which measures how accurately a candidate reproduces logged expert actions. That substitution assumes that accurate prediction of demonstrated actions carries over when the policy's own actions choose its next state, and only the closed loop can test it.

7.3 Closed-Loop Dynamics

A navigation policy can reproduce recorded aisle runs almost exactly and still spend the base's clearance to a rack before it is a third of the way down the aisle. The difference lies in what each kind of test feeds back to the policy. In an **open-loop evaluation**, a candidate policy π_θ is scored across a static dataset of recorded demonstrations $\mathcal{D}_{\text{demo}} = \{(s_t^*, a_t^*)\}_{t=1}^T$. At each discrete timestep t , the policy receives a recorded demonstrator state s_t^* and outputs a predicted action $\hat{a}_t = \pi_\theta(s_t^*)$. An error metric, such as mean squared error or cross-entropy, measures the distance between $\hat{a}_t$ and the recorded expert action a_t^* . At step $t + 1$, the evaluation pipeline supplies the next recorded state s_{t+1}^* from the dataset. The action $\hat{a}_t$ produced by the policy is discarded without executing on hardware, keeping the sequence of input states independent of the policy's past outputs. In a **closed-loop evaluation**, the policy proposes $a_t = \pi_\theta(s_t)$; the permission path checks it against measured state and constraints, applies u_t (or a fallback), and the plant evolves as $s_{t+1} = f(s_t, u_t)$. The applied command changes the physical state and later observations, crossing the causal boundary in section 1.2.

Open-loop evaluation tests actions along the demonstrator's state trajectory; a policy with low offline prediction error can fail when its biased proposals are admitted to physical motion, because its own actions carry it off the states it was trained on. This is the endogenous-data principle (2); section 5.1 developed its consequence for data, and evaluation must measure it on the machine. The door campaign cannot show the effect, because its base stands still while the arm works, so the example comes from the same machine's navigation stack rather than from the latch policy under evaluation. The base drives a 30 m aisle at its 1.5 m/s drive limit (both illustrative), and the nominal expert demonstration follows the centerline with heading $\theta^* = 0$ rad and lateral offset $y^* = 0$ m. Suppose the candidate navigation policy has a systematic heading bias of $\epsilon_\theta = 1.0^\circ$ (0.01745 rad) on every step, and the permission path admits each of these proposals because none is infeasible on its own while clearance remains.

In an open-loop evaluation, the policy receives the demonstrator state $s_t^* = (x_t^*, 0, 0)$ at each step and predicts a forward velocity with a 1.0° steering offset. The offline scoring pipeline records a mean absolute angular error of 1.0° (0.01745 rad) and an extremely low mean squared error of $\text{MSE}_\theta = (0.01745)^2 \approx 3.05 \times 10^{-4} \text{ rad}^2$.² In closed-loop execution, however, because the machine physically moves based on previously admitted commands, uncorrected heading errors compound continuously over time. Under planar unicycle kinematics ($\dot{y} = v \sin \theta \approx v \epsilon_\theta$), this persistent angular offset integrates into a cumulative lateral displacement:

$$y(t) = \int_0^t v \sin \epsilon_\theta d\tau \approx v \epsilon_\theta t$$

The base is 100 cm wide in a 130 cm aisle, which leaves $W_{\text{clear}} = 15$ cm of side clearance to the rack on each side, and the drift spends that clearance at

$$t_{\text{impact}} = \frac{W_{\text{clear}}}{v \sin \epsilon_\theta},$$

² | **Metric scale masking in regression loss**: Aggregating squared steering errors across thousands of validation frames obscures severe, intermittent directional mistakes. A single localized prediction spike of 15° is mathematically washed out when averaged across hours of straight aisle driving. A brief angular spike can matter if its integrated motion exhausts clearance before the independent permission path responds; the outcome requires a plant and timing model.

³ | **Kinematic integration of heading bias**: This example holds heading at a constant 1° offset, so $y(t) = v \sin(\epsilon_\theta)t$ is linear. A different example with constant yaw-rate bias ω begins with $\theta(t) = \omega t$ and has the small-angle approximation $y(t) \approx \frac{1}{2} v \omega t^2$.

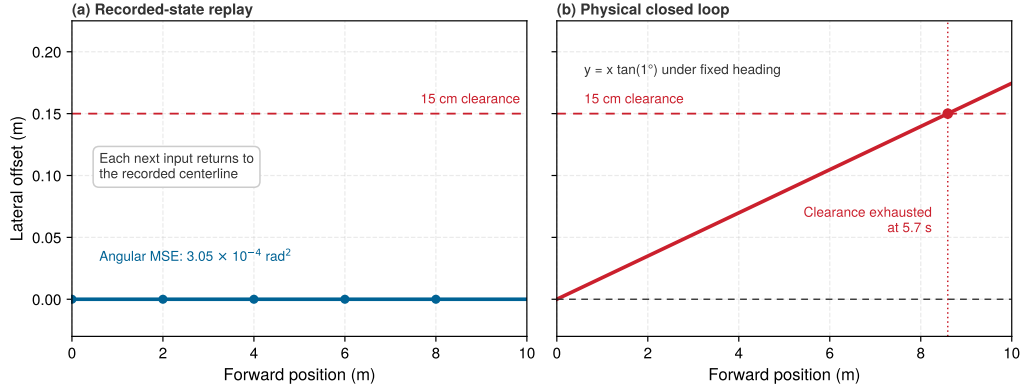


Figure 7.2: Open versus closed loop divergence: A persistent 1.0° heading offset gives $3.05 \times 10^{-4} \text{ rad}^2$ angular mean squared error on recorded centerline states. On the mobile manipulator’s aisle, the navigation stack’s admitted motion with that fixed heading drifts linearly and reaches the 0.15 m side clearance after 5.7 s at the 1.5 m/s drive limit. The example concerns the navigation stack, not the latch policy under evaluation.

which at the drive limit is 5.7 s, after 8.6 m of travel and less than a third of the way down the aisle (figure 7.2).³ Either the permission path’s clearance check stops the base short of the rack or, without that check, the base strikes it. Both outcomes are a failed trial that open-loop scoring rated near-perfect.

🔍 Systems Perspective 7.1: Open-loop blindness

Open-loop loss measures dataset mimicry; closed-loop dynamics decide physical survival. A small offline prediction error can still produce a large physical excursion if a bias persists and the permission path admits it.

The aisle example shows both reasons that an open-loop score decouples from closed-loop survival. First, static evaluation cannot integrate a persistent error into displacement. It resets the machine to an expert state at every clock cycle, masking the compounding of integration errors, whereas in closed loop the state at index t is the expanded composition of all previous actions:

$$s_t = f(f(\dots f(s_0, u_0) \dots, u_{t-2}), u_{t-1})$$

A persistent heading bias needs no software bug, since differential tire wear, camera-mast vibration, or floor camber produces one. Second, static evaluation cannot follow the machine into the unfamiliar states that early errors steer it toward. The first admitted error moves the base from the demonstrated state s_1^* to an unvisited state s_1 , and once it drifts a few centimeters off the centerline it occupies states absent from $\mathcal{D}_{\text{demo}}$. The offline metric, computed only where $y = 0$ m, holds no evidence about $\pi_\theta(s)$ for $y \neq 0$ m, and a policy never shown recovers from off-center states can make larger errors there, accelerating the divergence that section 6.2 traced for behavioral cloning.

The arm shows the same disconnect in contact. An end-effector position error of a fraction of a millimeter per step scores as negligible Cartesian error offline, but on the latch that offset acts as a wedge. The latch’s stiffness turns the lateral displacement into a reaction force ($F_{xy} = k_{\text{env}} \Delta_{xy}$) that climbs past the 15 N contact tripwire, which ends the attempt before the door unlatches.⁴

Offline scores can also mislead in the pessimistic direction when the task admits several valid actions. If the demonstrator passed an obstacle on the left, an open-loop score penalizes a candidate that passes on the right (the two-mode case of section 6.2), although in closed loop both reach the goal. An action-level error measures divergence from one recording, not satisfaction of the task.

The same gap separates each offline metric commonly reported for a learned policy from the machine failure it hides, and table 7.1 pairs each with the closed-loop measure that exposes it, among them the time to intervention T_{TI} , measured from the start of policy authority to the first

⁴ | **Non-Lipschitz contact bifurcation:** Rigid-body contact is not Lipschitz continuous, because friction cones are set-valued and impacts produce velocity jumps ($v^+ \neq v^-$). A sub-millimeter deviation in gripper contact location can switch the trajectory from stable force regulation to slip or jamming, a discontinuity that open-loop scoring on recorded states cannot expose.

Contrast: Open-loop prediction loss contrasts with the photon-to-actuation age-of-information bounds analyzed in section 8.4.

permission-path override (the permission path refuses a proposal and applies its fallback) or operator takeover.

Table 7.1: Offline metrics and the closed-loop failures they hide: Each open-loop metric assumes the demonstrator’s state distribution is unaffected by policy error; on the machine, biases integrate into drift, valid alternatives are penalized, and shifted states produce boundary violations.

Offline metric	Open-loop assumption	Why it breaks in closed loop	Failure on the machine	Closed-loop measure
Action mean squared error	Small residuals give small path deviations	A persistent bias integrates into drift and carries the machine off the demonstrated states	The base spends its rack clearance (stop or strike)	Peak deviation from the reference path; time to intervention (T_{TTI})
Discrete action accuracy	Correct labels on logged states give identical execution	Benign alternatives and boundary excursions cost the same; errors correlated in time are invisible	Command chatter and joint over-torque	Peak jerk; adherence to actuator torque limits
Cosine similarity	Collinear proposals give stable paths	Magnitude is ignored, so under-actuation stalls against static friction and over-actuation slips	The base stalls or slips its wheels; the arm stalls short of the latch	Slip ratio and damping (section B.5.3)
Negative log-likelihood	Likelihood at the expert action captures the action distribution	Valid alternatives, such as passing an obstacle on the other side, are penalized	Valid policies rejected; demonstrator noise imitated as jitter	Closed-loop task success over randomized starts
Reconstruction or Chamfer loss	Accurate scene prediction gives safe plans	Background pixels outweigh thin obstacles and contact geometry	Phantom braking; clipping a cable or a glass partition	Measured clearance; barrier-condition invariance

The disconnect also runs across the proposal boundary of section 1.5. In the two-processor implementation, offline trace replay on the application processor measures only whether neural inference executes within its memory and compute budgets. In physical closed-loop execution, each candidate proposal crosses to the microcontroller that runs the permission path. If the policy’s proposals drift off target or chatter, the permission path detects the spatial boundary breach or the expired lease, refuses the proposals, and commands a controlled stop. Evaluating a policy offline, without the physical plant and the permission path in the loop, evaluates tensor arithmetic, not the survival of the machine; only the closed loop measures the path excursion, the intervention timing, and the permission path’s measured performance.

Open-loop scores cannot certify a closed loop, so the evidence about the proposer must come from executions whose conditions are known and recorded. Every setting in which a policy can be executed changes some part of the loop, whether the plant, the sensor stream, or the authority over the actuators, and section 7.4 sorts those settings by what each can and cannot show.

Downstream: Deterministic gatekeeping and stopping envelopes for unverified neural policies are derived in section 12.5.

7.4 Evidence Regimes

Before its twenty door runs, the latch policy could have been scored against its logged demonstrations, run against a simulated door, run in shadow while a validated controller held the arm, or run on the real door with its contact tripwire armed. These four settings are the **evidence regimes** of figure 7.3 and table 7.2, namely recorded replay, closed-loop simulation, shadow operation, and constrained physical testing. Each establishes a different relationship between the policy, its sensory inputs, and the state of the machine, so each supports a bounded claim. None of the four establishes how the unassisted policy behaves during unconstrained physical deployment, because each regime alters either the causal feedback of the control loop or the distribution of states the machine encounters.

In recorded replay, the policy executes as a feedforward computation on stored sensor buffers. The environment does not respond to the policy outputs, and the state sequence remains fixed to the trajectory recorded during data collection. This regime supports claims about computational throughput, memory allocation, and loss against a logged dataset, such as whether inference fits its latency budget on the onboard accelerator and whether its numerical outputs match the recorded actions. It cannot support any claim about closed-loop stability, because the predicted action never

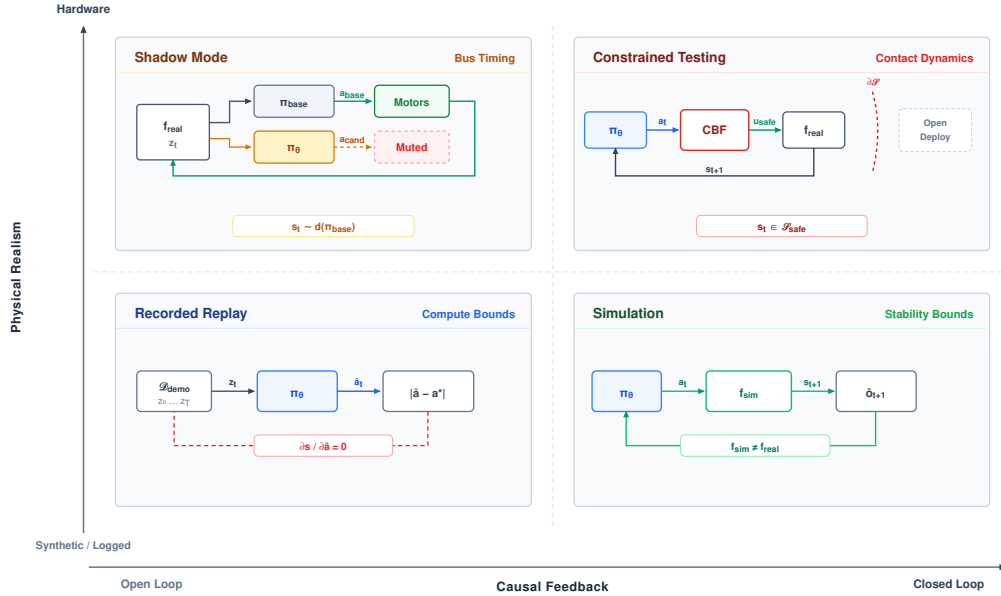


Figure 7.3: Evaluation evidence regimes: The regimes vary along causal feedback (open versus closed loop) and physical realism (logged or synthetic versus live hardware). Replay cannot observe compounding drift, simulation leaves the reality gap open, shadow operation locks the trajectory to the baseline controller, and constrained testing truncates failures at the constraint boundary $\partial\mathcal{S}$. No single regime tests deployment beyond that boundary.

alters the next observation; if the policy outputs a hard left turn, the next recorded frame still shows the base driving straight.

Closed-loop simulation restores the causal feedback loop by substituting an explicit mathematical model, f_{sim} , for the physical machine and its surroundings. When the policy proposes an action and the modeled permission path selects u_t , the simulation integrates the equations of motion, updates the scene, and renders synthetic sensor inputs for the next cycle at index $t + 1$. This regime supports claims about the policy’s behavior under the modeled dynamics, such as whether the base recovers from an initial offset or the arm avoids self-collision across its workspace, and thousands of instances can run in parallel. The evidence holds only for f_{sim} ; unmodeled backlash, tire slip, contact friction transitions, and lighting are excluded by construction, so stabilizing f_{sim} says nothing yet about the hardware.

Shadow operation mounts the candidate policy onto an active physical machine while a validated baseline controller, or a human operator, retains authority over the actuators. The candidate policy receives live sensor frames, runs onboard inference under real mechanical vibration and bus traffic, and computes candidate actions a_t^{cand} .⁵ These candidate actions are logged but never dispatched to the motor drives. Shadow operation supports claims about execution timing, such as whether the latch policy holds its chosen 20 Hz chunk rate under operational thermal load, and about how often a_t^{cand} diverges from the baseline action a_t^{base} by more than a stated threshold. It shares the limitation of replay, because the baseline keeps the machine on the baseline’s trajectory; an action that would have driven the base into a rack never moves it, and the sensor stream never shows the candidate’s failure.

Constrained physical testing closes the causal loop on real hardware by letting candidate proposals reach the actuators within a bounded envelope. Boundaries take the form of physical tethers, velocity clamps, padded test cells, or the permission path itself (section 1.5), which refuses unsafe commands at a validated response rate. This regime supports claims about contact dynamics, actuator saturation, and sensor noise within the tested envelope, which is why the twenty door runs belong to it. It cannot establish the policy’s tail behavior in open deployment, because every intervention by a tether or a permission-path override truncates the trajectory. The data measure the combined policy and permission path inside the constraint boundary, not the unassisted policy.

⁵ | **Hardware-in-the-loop bus contention:** While neural accelerator inference latency may exhibit low variance in isolation, shared PCIe backplanes, direct memory access (DMA) transfers, and CAN bus arbitration introduce variable transport jitter under production workloads. A nominal inference cycle can stretch under memory bus saturation, depleting the phase margin of high-rate control loops. In shadow mode, this latency jitter is logged across real bus traffic without exposing the physical chassis to the resulting trajectory oscillations.

Some properties transfer across regime boundaries. Numerical outputs for identical input tensors transfer, as do algorithmic invariants, so an overflow that saturates an output in simulation saturates it on hardware given the same input; measured timing and memory pressure must be retested under each regime’s bus, thermal, and scheduling load. The state distribution does not transfer. The moment the environment model changes from f_{sim} to f_{real} , or the tested permission path changes, the sequence of states $s_0, s_1, \dots, s_t$ diverges.

Viewing these regimes as a ladder of fidelity is a trap. Advancing from replay to simulation to shadow mode to constrained testing does not strengthen one confidence claim; each step trades one distortion for another. Simulation buys closed-loop causality at the price of real sensing and contact, shadow mode trades causality back for real sensor streams and hardware timing, and constrained testing restores both but censors the states beyond its safety boundary. Zero interventions across a hundred constrained trials show that the trajectories stayed where the constraints never engaged, not that the failure rate outside the envelope is low. Each question belongs in the regime able to answer it (table 7.2). For the latch policy, whether inference fits the chunk period belongs in replay and then in shadow operation on the target hardware; whether the arm recovers from a misplaced first contact belongs in simulation, where thousands of offsets cost no wear; and whether the door’s spring and the gripper’s compliance keep contact force below the limit belongs only in constrained physical trials. Each regime is blind to the failure modes of the others.

Table 7.2: Evaluation evidence regimes compared: Trade-offs between causal feedback coupling, environmental realism, supported empirical claims, and experimental boundary distortions across the four testing regimes. No single environment evaluates open deployment; rigorous certification requires combining complementary regimes rather than assuming a linear fidelity ladder.

Evaluation Evidence Regime	Causal Feedback Coupling	Environmental Dynamics Grounding	Supported Empirical Evidence	Masked Failure Mode / Boundary Distortion
Recorded Replay	Open Loop ($\partial s_{t+1} / \partial \hat{a}_t = 0$)	Static Demonstration Buffers $\mathcal{D}_{\text{demo}}$	Latency budget, DRAM footprint, numerical accuracy	Covariate shift; errors never alter subsequent observations
Closed-Loop Simulation	Closed Loop ($s_{t+1} = f_{\text{sim}}(s_t, u_t)$)	Synthetic Physics Operator f_{sim}	Kinematic reachability, large parallel test suites	Sim-to-real gap; unmodeled contact non-linearities and tire slip
Shadow Mode Operation	Open Loop for candidate (actions muted)	Physical machine f_{real}	Bus contention, thermal throttling, real sensor noise	Trajectory confinement; states locked to baseline controller d^{base}
Constrained Physical Testing	Closed Loop with runtime veto (u_{safe})	Physical machine f_{real} within envelope	Contact dynamics, compliance, friction, actuator limits	State censorship; permission-path overrides and tethers truncate failure tails

The quantitative relationship between simulated proxy benchmarks and real-world robot performance is governed by the **sim-to-real proxy transfer law** (figure ??). While offline open-loop metrics (such as action Mean Squared Error) fail to predict closed-loop outcomes ($r = 0.308$, suffering severe ranking inversions as detailed in table 7.1), closed-loop simulation benchmarks such as SIMPLER (Li et al. 2024) demonstrate a strong linear correspondence with physical manipulation success across embodiments ($R^2 = 0.836$, Pearson $r = 0.914$, $p < 0.0001$; Spearman rank correlation $\rho = 0.942$).

Critically, the linear transfer law ($y = 1.04x + 5.8\%$) holds reliably only within nominal visual and kinematic envelopes. The transfer breaks down catastrophically across three distinct physical gap regimes:

1. **Visual domain gap:** Specular lighting miscalibrations or rendering discrepancies can disrupt vision transformer tokenizers. For example, untuned visual textures in Octo-Base collapsed simulated solve rate to 0.0% despite achieving 29.3% real-world task success.
2. **Contact mechanics gap:** Rigid-body physics engines (such as standard ODE or Bullet solvers) idealize multi-body contact dynamics and omit non-linear Coulomb stiction. An RT-1-X policy evaluated on top-drawer opening reached 89.1% in simulation but dropped to 40.7% on real hardware due to mechanical rail jamming.

3. **Tactile and force gap:** Fine-grained manipulation (such as sub-millimeter peg insertion) yields high visual solve rates in simulation (82.0%), but collapses in reality (14.0%) where vision alone cannot detect contact force wedging.

Consequently, while closed-loop simulation serves as an effective screening filter for policy candidates, physical trial limits remain the ultimate arbiter of embodied competence.



Checkpoint 7.1: Trade-offs across evidence regimes

Before evaluating physical trial limits, verify your understanding of evidence regimes and fidelity trade-offs:

- ☐ **Regime causality trade-offs:** Can you contrast the causal feedback properties of recorded trace replay against closed-loop simulation?
- ☐ **Shadow-mode trajectory confinement:** Can you explain why shadow-mode evaluation on the running machine cannot test the candidate policy's recovery maneuvers?
- ☐ **Transfer across regimes:** Can you describe which properties transfer reliably across regimes and which one stays behind?

7.5 Physical Trial Limits

A reliability claim is paid for in trials. Choosing a regime settles what a test can show, but on a physical machine the plant, not the compute budget, fixes what each closed-loop trial costs, and statistics fixes how many trials the claim demands. Those two quantities together separate a qualification argument from an anecdote.

7.5.1 What a physical trial costs

A physical trial consumes more than the simulated step or log-replay time. It consumes wall-clock time, stresses mechanical linkages, heats motor windings, and risks physical damage to the hardware. The door campaign shows how the cost accumulates. In one run the base drives in from its 2 m standoff and stops, and the arm then unlatches at the 0.03 m/s guarded approach and swings the door open, which together occupy 25 s of the test cell (illustrative). The reset that makes the next run an independent draw re-latches the door, returns the base to its standoff, and flushes the run's telemetry, adding 15 s (illustrative), so each trial costs 40 s.⁶ At that cycle the twenty clean runs that opened the chapter took 13.3 min, and the campaign that a 99.9 percent success lower limit requires (7.2) runs far longer, even before the arm's windings take any thermal rest (section 2.5).

The plant also changes while the trials accumulate. The confidence bound assumes independent draws from a stationary process, and wear breaks stationarity: gearhead backlash, wheel radius, tire friction, and joint damping drift at rates that must be measured on the test hardware. Pooling a thousand runs then treats a progression of degrading machines as repeated tests of one, and a late failure may reflect the worn plant rather than the policy.

The censoring built into constrained testing (section 7.4) compounds the drift. Because a failed physical trial cannot be recalled (the first law of section 1.7), safety mechanisms intervene before a failure completes, and the evidence stops where they intervene. The states a team most needs to evaluate, such as loss of traction at speed or a collision the base cannot avoid, are the states it cannot let the machine enter, because a skid into a rack damages the chassis and the sensing rig and takes the test cell offline for repair. Cages, geofences, and operators with emergency stops cut each run short at a conservative envelope. The trial records the intervention while the downstream consequence of the policy's output stays unobserved; the intervention stays in the denominator, and the consequence stays unknown.

For physical survival, a mean is an inadequate description of a peak. What threatens the latch is a run's peak contact force against the 100 N damage limit, however modest the average, and a long bus delay can consume stopping margin however small the mean latency. Extreme value theory (EVT) offers candidate models for maxima, but a Gumbel, Fréchet, or Weibull family must be fitted to a defined sample of block maxima or threshold exceedances, checked against alternatives, and reported with its uncertainty; the physical mechanism alone does not select the family.⁷ Without

⁶ | **Physical trial cycle budget:** The duration of a physical trial includes the reset as well as the run, $T_{\text{trial}} = t_{\text{run}} + t_{\text{reset}} = 25 \text{ s} + 15 \text{ s} = 40 \text{ s}$ for the door. Budgeting by active run time alone produces unrealistic testing timelines and tempts teams to truncate reset and cooldown intervals, which causes thermal non-stationarity across runs.

⁷ | **Extreme value disturbance modeling:** The Fisher-Tippett-Gnedenko theorem describes the possible limiting forms of block maxima under stated assumptions; it does not assign a tail family to impact, glare, or latency by physical mechanism alone.

those data, table 7.3 states what to measure and what protection to validate rather than predicting rare-event frequencies.

Table 7.3: Peak-disturbance evidence and protection: Mechanisms identify measurements and safeguards, not an EVT family or event rate. Any quantitative tail claim needs a defined population, fitted model, goodness-of-fit checks, and uncertainty.

Disturbance channel	Peak quantity and sampling record	Physical consequence to test	Candidate protection to validate
Compute and bus latency	Per-cycle end-to-end age and deadline misses	Stale proposals erode braking margin	Freshness lease and independently timed fallback
Contact shock	Force-time trace, impulse, and peak at the load path	Yield or overload before a reactive command takes effect	Passive compliance and precontact speed limit; reactive limits address later load
Low traction	Local friction estimate and stopping response	Slip extends stopping distance	Conservative speed ceiling and tested fallback on low-grip surfaces
Optical glare	Saturation duration and perception misses	False clearance or lost state estimate	Redundant sensing and independent physical constraints

The latch shows why the contact-shock row pairs a measurement with a protection. Each door run contributes one block maximum, its peak latch force, and twenty maxima are far too few to fit a tail that would predict how often a run crosses 100 N. A force monitor that reacts only after contact cannot undo the first peak either; its measured detection and actuator response can limit the loading that follows, while the 0.03 m/s guarded approach, a precontact speed limit, reserves the margin for the initial impact.

Distributing trials across a fleet does not buy independence. Ten machines tested concurrently in one warehouse share a production batch, a floor finish, a ventilation system, and operators trained under one protocol, so an anomaly absent from the facility, such as floor glazing or low-angle glare, reaches none of them. Ten thousand fleet hours in one facility are ten parallel observations of one narrow distribution, not ten thousand hours of independent exposure. Standardized test fixtures such as those in figure 7.4 buy repeatability by bounding the state space to the fixture.

7.5.2 Trials per claim

A zero-failure campaign is sized by inverting the exact bound of section 7.1 for the trial count, so the claim, not the test budget, fixes how many runs the test cell must deliver (figure ?? and table 7.4; derivations in section A.8).

Napkin Math 7.2: Statistical scaling of assurance

Problem: Suppose an evaluation target calls for a one-sided success lower limit of $p_{\text{target}} = 0.999$ at 95 percent confidence ($\alpha = 0.05$). If each door trial requires $T_{\text{trial}} = 40$ s, how many zero-failure runs and how much wall-clock testing time are required?

Calculations: The inversion of 7.1 gives $n = \lceil \ln(0.05) / \ln(0.999) \rceil = 2,995$ consecutive zero-failure trials. At 40 s per trial, this demands $T_{\text{total}} = 2,995 \times 40 \text{ s} \approx 33.3 \text{ h}$ (1.4 days) of continuous testing. With a two-operator safety protocol, this consumes 66.6 h of operator time, or over eight eight-hour shifts.

Systems insight: Even on a cycle as short as the door's, a 99.9 percent lower limit costs 1.4 days of continuous test-cell occupancy, and a longer cycle scales it in proportion. A component replacement during the campaign may require a new population definition rather than pooling incompatible runs.

When a reliability requirement is stated as a failure rate λ per operating hour, the trial count becomes an exposure time. The **exposure wall** is the failure-free exposure a zero-event test needs before it can bound a hazard rate. Under a constant-rate model at 95 percent confidence, $T \geq$



Figure 7.4: Standardized robotic evaluation facility: NIST response-robot test bays photographed in 2013 (Linda Joy/NIST), then described under ASTM E54.08. A zero-failure run across standardized bays establishes competence inside the arena and says nothing about glare, surface contaminants, or obstacles outside it.

$-\ln(0.05)/\lambda \approx 3.0/\lambda$: a target of $\lambda = 10^{-6}$ per hour needs 3.0×10^6 failure-free hours (342 machine-years), and $\lambda = 10^{-9}$ per hour needs 3.0×10^9 .

Butler and Finelli drew the design conclusion from this arithmetic (Butler and Finelli 1993) (7.2). A fleet does not rescue the test, because 100 rigs running continuously would still need about 3,420 calendar years at 10^{-9} per hour. Road-vehicle mileage meets the same wall (figure ??), and simulated miles add no physical exposure to it. A high-assurance claim therefore rests on independently validated runtime constraints and structured fault injection (section 15.3).⁸

⁸ | **Functional safety scope:** Functional safety standards define context-specific hardware metrics and assurance processes; a single failure-rate number cannot be assigned to all physical AI systems or certified by mileage alone. The Poisson examples here are analytical targets, not claims of compliance with IEC 61508 or ISO 26262.

🔍 Systems Perspective 7.2: Limits of testing for ultra-reliability

Result: Butler and Finelli argued that validating ultra-high reliability requirements for life-critical cyber-physical systems (such as commercial fly-by-wire flight control software requiring a catastrophic failure rate $\lambda \leq 10^{-9}$ failures/hour) through operational testing alone is infeasible (Butler and Finelli 1993).

The exposure wall binds even at a site requirement far looser than those rates. Suppose the warehouse mobile manipulator must demonstrate a critical failure rate $\lambda \leq 1.0 \times 10^{-4}$ failures/hour (Mean Time Between Critical Failures MTBF $\geq 10,000$ hours) at 95 percent statistical confidence ($\alpha = 0.05$). Each retrieval mission takes $t_{\text{mission}} = 360$ s (0.10 h) and covers 540 m at the 1.5 m/s drive limit. Automated dock recharge and telemetry reset add $t_{\text{reset}} = 120$ s, for 480 s per trial cycle. The drive-wheel diameter is $D_{\text{wheel}} = 0.20$ m, and the drive gearhead carries an illustrative L_{10} bearing fatigue rating of 5.0×10^7 revolutions.

By the exposure wall, this target needs 29,957 failure-free hours (3.42 years) of continuous exposure. Discretizing this requirement into missions with per-trial failure budget $p_{\text{fail}} \leq \lambda \cdot t_{\text{mission}} = 1.0 \times 10^{-5}$ requires $N_{\text{req}} = \ln(\alpha) / \ln(1 - p_{\text{fail}})$, or 299,572 consecutive zero-failure trials. This inverts the zero-failure bound of section 7.1 at a target far beyond practical budgets of tens to thousands of runs. On a single physical rig, executing these trials at 480 s per cycle consumes 39,943 hours (4.56 years) and accumulates $N_{\text{rev}} = (N_{\text{req}} \cdot v \cdot t_{\text{mission}}) / (\pi D_{\text{wheel}}) \approx 2.57 \times 10^8$ wheel revolutions.

The wheel-rotation total is 5.15 times the gearhead's L_{10} rating, but the comparison holds only after the drivetrain ratio maps wheel revolutions to bearing revolutions, and L_{10} is a population-life statistic at specified load and speed, not a destruction deadline. A campaign of this length must therefore plan maintenance and component replacement, and record each one, before it may pool trials.

Table 7.4: One-sided binomial confidence limits after zero failures: Success lower limits are $\alpha^{1/n}$ and the corresponding failure upper limit is $1 - 0.05^{1/n}$. Test times use the 40 s door cycle.

Zero-failure trials n	90% lower success limit	95% lower success limit	99% lower success limit	95% upper failure limit	Test time at door cycle	Interpretation under iid test draws
10	79.43%	74.11%	63.10%	25.89%	0.1 h	Wide uncertainty; no observed failures
20	89.13%	86.09%	79.43%	13.91%	0.2 h	Bound applies only to the sampled distribution
50	95.50%	94.18%	91.20%	5.82%	0.6 h	More evidence; untested conditions remain
100	97.72%	97.05%	95.50%	2.95%	1.1 h	No deployment permission follows from the interval
250	99.08%	98.81%	98.17%	1.19%	2.8 h	Check hardware and sampling stationarity
500	99.54%	99.40%	99.08%	0.60%	5.6 h	Bound remains tied to declared conditions
1,000	99.77%	99.70%	99.54%	0.30%	11.1 h	Below a 99.9% lower-limit target
2,995	99.92%	99.90%	99.85%	0.10%	33.3 h	Meets that statistical target if assumptions hold

These ceilings force an architectural conclusion. Physical AI safety cannot rest on statistical confidence in the learned weights, and classical control offers no analytical substitute for the proposer.

A classical controller is certified by a **Lyapunov function** $V(\mathbf{x}) > 0$ with $\dot{V}(\mathbf{x}) \leq -\gamma V(\mathbf{x})$ over a stated region, not by counting runs, and with disturbances an input-to-state stability argument bounds the state error by the size of the input under its own model assumptions. A learned proposer carries no such stability certificate. Checking the decrease condition through its non-smooth activations is a non-convex search over millions of parameters, its latent inputs carry no

physical units on which to build V , and contact discontinuities break the smoothness the proofs assume. Outside its training distribution a learned policy can also change its output abruptly, so that a shadow or a camera glint flips the direction of a proposed motion. The burden of proof therefore moves off the proposer and onto a low-dimensional check on an isolated microcontroller. Two strategies carry that burden, one at run time and one on the bench:

1. **Runtime Containment:** Bounding the physical authority of the learned policy by delegating the collaborative-robot force limits, kinematic acceleration limits, and emergency stops to the permission path, an independent, deterministic mechanism below the proposal boundary of section 1.5. The protection claim then rests on the permission path and its fallback, whose state estimates, plant and actuator models, response timing, and reserved stopping margin must be validated separately from the policy.
2. **Fault Injection on Physical Benches:** Rather than waiting for passive disturbances to strike, engineers inject them. Injected trials follow the fault-injection method of section 15.3; this chapter requires only that they enter the same full-denominator ledger.



Checkpoint 7.2: Sample size and MTBF evidence target

Before moving on, check your understanding of sample size scaling and physical test limits:

- ☐ **Exposure wall:** Under a stationary Poisson failure model, why does a one-sided 95 percent upper rate limit corresponding to a 10,000-hour MTBF target require 29,957 hours of zero-failure physical exposure?
- ☐ **Mechanical fatigue exhaustion:** Can you explain why wheel revolutions cannot be compared directly with a bearing L_{10} rating without drivetrain mapping and load conditions?
- ☐ **Architectural containment vs. statistical sampling:** Can you explain why high-assurance physical AI systems substitute deterministic runtime monitors for empirical black-box trial accumulation?

Even under rigorous qualification protocols and automated test cells, statistical sampling alone cannot establish ultra-reliable autonomy. What remains outside any finite campaign, and which runtime monitors the record must name for it, decides how far the evidence can carry the release decision.

7.6 What Evaluation Cannot Establish

The limits of evaluation follow from the endogenous loop of section 7.3. The states a physical campaign visits depend on the commands the permission path admitted, so no fixed dataset matches the candidate's closed-loop state distribution, and a simulator substitutes its own transition model for the plant's (formal trajectory distributions in section A.5.5). A simulator that models the aisle floor with the friction of clean, inspected concrete cannot show what the base does on a clear oil film, and one whose actuators respond instantly cannot show the lag of a real stator's electrical time constant; the physical trajectory departs from the simulated one, and the departures compound into states no simulated run visited. A test therefore bounds only the distribution it actually sampled.

The strongest conclusion an empirical evaluation can support is therefore narrow and conditional. The door campaign establishes that one software build on one hardware revision, from the sampled initial conditions and inside the environment present during the runs, failed with probability below 13.9 percent at 95 percent confidence. It says nothing about a colder morning that derates the joints, a gearbox whose compliance and backlash have grown with wear, or a displaced strike plate. More runs on the nominal plate reduce sampling uncertainty and tighten the bound (table 7.4), but they cannot reduce structural uncertainty about unmodeled physics or unvisited conditions; two thousand further runs would measure the same cell with finer precision and leave the displaced-plate cells, the loaded machine, and unmodeled dynamics such as bus jitter exactly as unsupported as before.

Since evaluation cannot establish how the policy behaves outside the sampled distribution, each monitorable boundary of the claim needs a runtime signal, threshold, and tested response, and

the permission path must enforce independent physical constraints. Distribution support is only partially observable and an OOD detector has measurable false negatives, so conditions that cannot be sensed reliably require OOD restrictions, passive protection, or other independent barriers. The evaluation record carries these specifications in its `runtime_monitor_specs` field (section 7.8). The field names five monitoring channels whose coverage and response must be measured:

1. **Input Support / Out-of-Distribution (OOD) Monitor:** Estimates whether incoming observations depart from the tested distribution using a calibrated score and reports false negatives on held-out shifted conditions.
2. **Execution Latency Monitor:** Measures each inference age against the declared control deadline (for example, the latch policy's 50 ms chunk period at 20 Hz), records misses, and checks that lease expiry invokes the validated fallback. A P_{99} summary alone is not a hard deadline bound.
3. **Constraint Margin Monitor:** Tracks the distance between the current physical state and hardware safety limits, including joint torque limits, motor winding temperature, and obstacle clearance against limits specified for the hardware and stopping model.
4. **Intervention Rate Monitor:** Calculates the frequency of permission-path overrides, detecting when policy instability causes excessive corrections.
5. **Environmental Assumption Monitor:** Measures external operating conditions, such as surface traction estimated from wheel slip and ambient illumination measured by photometric sensors.

A monitor that only writes an anomaly to a log protects nothing. When a monitor reports that a condition has left the target OOD, or that an inference missed its deadline, the permission path discards the proposal and acts on the actuators through a validated fallback, a speed restriction or a controlled stop, provided the sensed state, actuator capability, and reserved clearance make that response feasible (fallback logic in section B.2). A timely, confident proposal still passes the independent state, clearance, and actuator checks before any u_t is applied.

Some failures no runtime monitor detects. A proposal can meet every deadline, respect every torque limit, and arise from familiar inputs, yet steer the base toward a glass partition that the vision-language backbone (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023) took for free space; the measured clearance is wrong until contact. Safety against such failures cannot come from evaluation or software monitors. It requires physical zoning, mechanical current limiters, and independent safety barriers whose protective capability is validated independently of the learned model under specified physical assumptions.

These limits bind even a flawless campaign. The evidence that survives them is only as good as the campaign that produced it, and procedural and statistical errors can corrupt a campaign long before its result reaches the record.

Downstream: Physical failures undetectable by sensory telemetry are formally cataloged in section 17.4.

7.7 How Evaluation Goes Wrong

Systematic errors undermining policy evaluation rarely stem from deliberate fraud. They arise when a protocol answers an easier question than the claim asks, and six recurring modes do so: environment selection, metric drift, test-set contamination, optional stopping, denominator filtering, and a demonstration offered as a measurement.

Environment selection occurs whenever the test bench is narrower than the claim. The twenty door runs all met the nominal strike plate at the 0.03 m/s guarded approach with the base stopped and the tote rack empty. If the report states their success rate as the machine's ability to open the site's cage doors, the claim silently expands to the five unsampled cells of the ledger and the loaded machine (section 7.6). Environment selection is detected by comparing a condition ledger, which records the measured strike-plate position, approach speed, payload, and floor condition of every trial, against the boundaries of the claimed domain; a ledger that covers one cell establishes nothing about the others.

Metric drift occurs when the evaluation metric slides from physical task completion toward an offline surrogate such as action agreement, the mean squared error or cosine distance between predicted commands and recorded expert actions. Section 7.3 showed why such a score cannot certify the closed loop. Metric drift is detected by requiring the primary evaluation metric to be a

closed-loop terminal outcome, such as an open door with no contact-tripwire crossing and zero permission-path overrides, rather than open-loop prediction error.

Even with closed-loop physical trials, repeated tuning against a fixed evaluation setup degrades result independence. When a team adjusts the latch policy's hyperparameters, observation filtering, or checkpoint after each batch of runs on the same cage door, that door ceases to be an independent holdout. Each change absorbs information about that door's strike-plate position, spring, and lighting, and after enough rounds the final checkpoint reflects implicit training on the test door itself, masking how the policy responds to any other door. This test set contamination cannot be detected by inspecting the final model weights or running a single verification trial on the same door. It is detected only through provenance tracking, analyzing the chronological history of policy versions, tuning commits, and test dates to confirm that the reported evaluation was executed on a fresh, quarantined configuration that was never exposed to iterative tuning.

Evaluation protocols are equally vulnerable to unintended cherry-picking through optional stopping. When an engineer monitors a continuous sequence of physical runs and halts the experiment as soon as a target performance threshold or statistical significance criterion is reached, the reported confidence is distorted. If an evaluation team conducts an experiment where each look carries a nominal false-positive threshold of 5 percent, stopping early upon observing an apparent success across multiple evaluation checkpoints inflates the true error rate. Five independent 5 percent looks would have a 22.6 percent chance of at least one false positive. Repeated looks at one accumulating run are dependent, so the inflation for a real stopping rule must be computed for that rule. Optional stopping cannot be detected from summary tables or final pass rates. It is detected only by auditing the pre-registered trial protocol, verifying that sample sizes, stopping conditions, and evaluation milestones were fixed before the first physical run began.

A long campaign still has reason to stop as early as the evidence allows, because every run wears the testbed and spends the arm's thermal budget (section 2.1). To stop early without inflating Type I error, evaluation teams employ Abraham Wald's **sequential probability ratio test (SPRT)**⁹ (Wald 1945). Rather than fixing sample size in advance or naively peeking at intermediate results, SPRT evaluates an accumulation of evidence after each physical rollout, computing the cumulative likelihood ratio between an acceptable reliability hypothesis (H_1) and an unacceptable failure hypothesis (H_0). Testing halts dynamically the moment the accumulated evidence crosses predeclared likelihood-ratio thresholds. For two specified simple hypotheses and a declared sequential design, SPRT can reduce expected sample count compared with a fixed-horizon test. The common Wald boundary formulas are approximations affected by likelihood-ratio overshoot; actual Type I and Type II errors and testbed savings must be calculated or simulated for the chosen hypotheses and outcome model. For the exact log-likelihood martingale formulations, Bernoulli updates, and Wald boundary derivations, see section A.9.

A related statistical distortion occurs when the denominator of an evaluation is quietly filtered, breaking the full-denominator condition of section 7.2. A team that discards runs ending on software timeouts, emergency stops, sensor communication drops, or permission-path overrides as invalid test artifacts reports success over the surviving trials only. If the arm opens the door on nine of ten attempts and the tenth needs an operator takeover to stop the gripper wedging against the latch, removing that attempt raises the apparent success rate from 90 percent to 100 percent. When physical constraints or safety limits truncate a run before its time limit $T_{\max}$ expires, the trial is recorded as a censored failure rather than omitted from the ledger. Reading survival on truncated trajectories as operational competence is the **censored trial fallacy**. Denominator filtering is detected by cross-referencing low-level actuator telemetry and permission-path logs against the trial ledger, confirming that every power-on cycle and autonomy engagement corresponds to a recorded outcome.



War Story 7.1: The 2017 NHTSA Autopilot crash rate mirage

Context: In January 2017, the National Highway Traffic Safety Administration (NHTSA) closed Preliminary Evaluation PE 16-007, which it had opened after a fatal Florida crash of a Model S operating in Autopilot mode (section 8.6) (National Highway Traffic Safety Administration (NHTSA) 2017). The closing report compared airbag-deployment crash rates in the subject

⁹ | **Sequential Probability Ratio Test (SPRT):** Predeclared likelihood-ratio boundaries allow valid sequential stopping for specified hypotheses when their actual error probabilities are calibrated, including boundary overshoot. The number of saved runs is path- and hypothesis-dependent; no universal wear saving follows.

Tesla vehicles before and after Autosteer was installed and stated that “the Tesla vehicles crash rate dropped by almost 40 percent after Autosteer installation,” from 1.3 to 0.8 crashes per million miles.

Mechanism: The rate divided airbag deployments by miles driven before and after installation, and the report’s own footnote noted that the rates “are for all miles travelled before and after Autopilot installation and are not limited to actual Autopilot use.” The denominator therefore never measured exposure to Autosteer itself, and it was incomplete as well. A replication by Quality Control Systems Corporation found that the odometer reading at installation appears to have been reported for fewer than half of the vehicles, and that exact exposure before and after installation could be computed for only 13 percent of them (Quality Control Systems Corp. 2019). The mileage NHTSA left uncounted, because it could be assigned to neither period, was concentrated among the vehicles with the least exposure before installation.

Impact: In the one cohort with complete exposure in both periods, the airbag-deployment crash rate rose 59 percent after installation, from 0.76 to 1.21 per million miles, a result the replication qualified with “if these data are to be believed.” Its authors concluded that the overall 40 percent reduction was “an artifact of the Agency’s treatment of mileage information that is actually missing in the underlying dataset.” The replication does not measure Autosteer’s effect either. It shows that the published denominator could not support the reported reduction.

Response: The correction came from outside the agency and two years after the closing report, because the per-vehicle mileage records that exposed the denominator were released only after a Freedom of Information Act lawsuit against the Department of Transportation (Quality Control Systems Corp. 2019).

Systems lesson: In physical AI evaluation, an unverified denominator produces a statistical mirage. Aggregating open-world mileage without engagement telemetry, operational design domain boundaries, and complete exposure records turns missing data into a safety claim. An evaluation audits the integrity of its denominator with the same rigor it applies to the failure events in its numerator.

The temptation to substitute visual plausibility for statistical sampling represents a pervasive vulnerability in physical systems engineering. A single video recording of a robotic arm grasping an irregular workpiece or a mobile robot navigating a cluttered room is a demonstration, not a measurement. A visually convincing demonstration proves that the system state trajectory intersects the success set for at least one sequence of initial conditions and environmental disturbances, but it provides no information about the volume of that success set. Because human perception is tuned to smooth motion and coordinated SE(3) kinematics, an observer easily mistakes kinematic fluidity for policy competence. An execution that appears deliberate and smooth can operate within 10 mm of a kinematic singularity or carry an unmeasured P_{95} tracking error that produces a collision in nine out of ten repeats. A representative demonstration has no statistical standing. A representative campaign needs predeclared sampling from the target ODD or separate per-stratum analysis, full-denominator accounting, and a sample size tied to its stated claim.

The same discipline governs comparisons. As Jacob Cohen formalized in his statistical power analysis¹⁰ (Cohen 1988), deciding whether latch-bc-01 opens the cage door more reliably than another checkpoint requires the Type I error (α), the Type II error (β), and the effect size, the smallest difference in success rate worth detecting. Twenty runs per checkpoint carry no stated power until those quantities and the baseline success rate are known, so the protocol fixes α , β , and the effect size before the first run.

Each of these pathologies widens the claim beyond the evidence while leaving the report looking cleaner: a benign bench, a surrogate metric, a contaminated test door, a well-timed stop, a filtered denominator, or a persuasive video. Understanding the mechanisms by which evaluation fails clarifies the purpose of physical testing. An empirical trial is not a demonstration to convince an observer that a model works. It is an instrument to measure where policy authority fails, what physical conditions trigger the failure, and how much statistical confidence the evidence supports. Documenting those boundaries requires an immutable record that binds every trial, condition, and intervention to a verifiable physical quantity, and section 7.8 specifies that record.

¹⁰ | **Statistical Power in Physical Trials:** The probability $1 - \beta$ of correctly rejecting a false null hypothesis. For a two-policy comparison, compute power from the baseline rate, effect size, sample allocation, and chosen test before collecting physical trials (Cohen 1988).

7.8 Evaluation Logs

When the twenty door runs are over, the record they leave behind is the only form in which the rest of the machine will ever see them, so what that record keeps decides which claims survive. An **evaluation record** consumes the policy manifest of section 6.8, citing it by digest, and adds the fields required to audit the learned policy that the manifest describes; it makes no claim beyond the target ODD and names a runtime monitor for every monitorable boundary. The record links measured outcomes to declared conditions and preserves human observations separately with their source. It serves as a self-contained evidence record that states what was measured, under what physical conditions, with what confidence, and which operational claims the resulting evidence explicitly does not support.¹¹

¹¹ | **Safety of the intended functionality (SOTIF):** ISO 21448 and ANSI/UL 4600 provide frameworks for arguing about intended-functionality and autonomous-system hazards. This chapter's immutable record is a proposed engineering evidence artifact, not a specific form mandated by either standard.

Record Schema Field	Physical / Statistical Modality	Cryptographic & Provenance Guarantee	Downstream Verifier & Usage
---------------------	---------------------------------	--------------------------------------	-----------------------------

Table 7.5: Physical policy predeployment evaluation record schema: Fields the evaluation record adds to the policy manifest it consumes, recording the target ODD, configuration digests, the sampling protocol, the complete-denominator trial ledger, non-asymptotic confidence bounds, the unsupported coverage gaps, and the runtime monitor specifications.

Record Schema Field	Physical / Statistical Modality	Cryptographic & Provenance Guarantee	Downstream Verifier & Usage
target_odd_boundary	Target ODD: the sampled cells of the declared ODD, here the nominal strike plate at the 0.03 m/s guarded approach under the door's own latch load, base stopped	Signed ODD specification; must lie inside the policy manifest's <code>declared_odd_boundary</code>	Defining tested conditions and calibrated OOD alerts; physical gating remains independent
component_hashes	Policy manifest digest, frozen neural network weights ($H(\theta)$), inference engine binary, chassis serial number, tire batch, sensor calibration matrices, and firmware build	SHA-256 binary digests, hardware UUID, ISO 9001 calibration certificate references	Supporting configuration traceability; preventing unrecorded hardware or driver substitutions
sampling_protocol	Target-distribution draw weights or per-stratum quotas, PRNG seed, reset procedure, and independence audit	Automated test harness configuration script hash, telemetry sequence log	Auditing against optional stopping and testing against hidden autocorrelation across trials
complete_outcome_ledger	Full-denominator raw outcome counts: $n_{\text{attempted}}, n_{\text{success}}, n_{\text{estop}}, n_{\text{timeout}}, n_{\text{override}}$ (permission-path overrides), and T_{TPI} time-series	Monotonic microsecond hardware timestamps, uninterrupted ring-buffer telemetry	Preventing censored trial fallacy; calculating uncorrupted empirical reliability
statistical_evidence	Exact binomial limit with sampling assumptions; Poisson rate only for separately modeled physical-hour exposure	Cryptographic verification signature, reproducible Python evaluation script	Passing bounded evidence and explicit assumptions to chapter 15
unsupported_gap_catalog	Explicit catalog of unvisited state spaces and omitted dynamics (surface water, dynamic crowds); contains the provenance record's known absences and the manifest's omission log	Declared non-support manifest, signed safety engineering sign-off	Parameterizing the permission path's checks, CBF constraints, and fallback policies in Part III
runtime_monitor_specs	For each monitorable boundary of the claim: signal, threshold, and tested response, across the five channels of section 7.6	Measured detection coverage and false-negative rate per channel	Implemented by the permission path and the observation contract's health field; read by the enforcement record

The evaluation record begins with the operational claim and its formal predicates, as specified in the predeployment evaluation record schema (table 7.5). It defines the mathematical success criterion, such as unlatching the cage door and swinging it open without crossing the 15 N contact tripwire and without a permission-path override of the policy. It bounds the target ODD, which lies inside the declared ODD of the policy manifest, across the physical parameters that manifest names, including strike-plate position, approach speed, and the load the door presents. Finally, it names the target population of physical environments to which the result is intended to generalize, distinguishing the one cage door tested from cage doors with other strike plates or spring rates. Defining these boundaries in the record before trials begin prevents post-hoc adjustments to the pass criteria when anomalous behaviors appear.

The record captures immutable cryptographic identifiers for every software and hardware component to enable independent replication of the physical execution. These identifiers include the content hashes of the neural network weights, sensor preprocessing pipelines, policy configuration files, simulation engine builds when synthetic runs are incorporated, and the evaluation execution scripts. On the physical system, the record logs the machine chassis serial number, wheel tire compound batch, sensor calibration matrices, motor controller firmware revisions, and communication bus configurations. Swapping a camera lens, altering a driver firmware version, or replacing worn drive belts changes the physical transfer function of the machine, invalidating the historical data if the change is unrecorded.

The record preserves the trial protocol, which was fixed before the first run so that the campaign cannot be halted at a favorable moment (the optional-stopping hazard of section 7.7). The protocol predeclares the sample size N_{req} from the claim's target and confidence, the outcome classes, and the stopping rule: a zero-failure campaign stops at its first failure and reports the exact lower bound its successes support, or completes N_{req} runs and reports $\alpha^{1/N_{\text{req}}}$. A pooled binomial bound needs each condition drawn independently from the target distribution, whereas a fixed quota grid needs separate per-stratum inference and declared weights before any claim spans the target ODD. The record documents how the runs were drawn from the cells of the target ODD, the pseudorandom seed sequence governing where the base stops in front of the door, and the reset sequence executed between consecutive runs, which re-latches the door and returns the base to its 2 m standoff. When the base returns to that standoff under autonomous control, battery thermal rise and odometry drift introduce statistical dependence between adjacent attempts. The record specifies whether trials were statistically independent or chained in series, and it registers every manual reset, hardware abort, or deviation from the pre-specified protocol.

Aggregate percentages obscure the physical failure modes that determine system risk. Each attempted run lands in exactly one predeclared class: success, reaching the goal within T_{max} with no override; censored failure, when the permission path overrode the policy; timeout; or hardware failure, such as a dropped bus or a faulted drive. The record reports the raw count in every class, and it reports kinematic boundary violations and manual emergency stops as subcounts of the class in which each affected run ended. In addition to summary tallies, the record preserves per-trial time-series metadata, including peak motor winding temperatures, maximum trajectory tracking errors, and task completion durations. Logging this telemetry ensures that nominal successes operating near thermal limits or within 5 mm of a hard boundary are visible during audit. Every trial initiated under policy authority remains in the denominator.

The statistical specification defines the point estimator, confidence method, nominal confidence level, sampling distribution, and underlying mathematical assumptions. Alongside the supported claim, the record documents unsupported claims and known coverage gaps; the catalog begins from the known absences of the demonstration data and the omission log of the policy manifest, and adds what the trials themselves left unvisited. For each gap that can be sensed, the runtime monitor specifications name the signal, threshold, and tested response of section 7.6; a gap that cannot be sensed stays in the catalog as a restriction on operation.

Consider the twenty-run door campaign that opened this chapter. The latch policy opened the cage door twenty times from the 2 m standoff at the 0.03 m/s guarded approach, with the base stopped and no permission-path override. An auditor reads the record's statistical evidence as the exact bound of section 7.1, a 95 percent lower confidence limit of 86.1 percent, never as the degenerate interval [1.0, 1.0].

The completed record exposes the structural limits of the claim, and it shows how far the evidence narrowed on its way through Part II. The scenario ledger of section 5.5 crossed three strike-plate positions with two approach speeds, six cells in all, and the retained demonstrations supported one, the nominal plate at the guarded approach. The policy manifest made that cell the declared ODD of latch-bc-01, the target ODD took in the whole cell, the twenty clean runs were drawn from it, and they bound the failure probability there at 13.9 percent, the one-sided upper confidence limit under the tested iid distribution (table 7.4). Everything that narrowing left out enters the coverage gap catalog as a gap the campaign inherited rather than closed. The five cells that held zero demonstrations hold zero trials as well, and the catalog inherits the manifest's omission log, whose two unclosed simulator omissions, the soft penalty contact and the missing transport lag, bar this record from being reused for any checkpoint refined in simulation. The trials add a gap of their own, because the base carried nothing in its 50 kg tote rack, so the record says nothing about the loaded machine. The record shows why twenty successful runs cannot support a high-reliability claim, and it gives a fixed basis for refusing any assertion that the policy is ready for unmonitored deployment.

An evaluation record does not certify that an autonomous machine is safe. It establishes the empirical boundaries and statistical uncertainty of a single learned component operating under controlled conditions. In chapter 15, the verification argument ingests these evaluation records as formal evidence nodes, combining policy bounds with evidence for the physical guardrails and the permission path's latencies to evaluate whether the integrated machine satisfies its operational safety case.

Systems Perspective 7.3: Evaluating a machine that falls

Question: What does a coverage record say when every failure in an unsupported cell is a fall that damages the test article?

The mobile manipulator's failed latch trial ends at the contact tripwire with an intact arm, so its campaign can keep sampling a hard cell; a humanoid's failed trial can end the campaign's hardware.

- **Fall count is its own estimand.** A humanoid's failed trial can end on the floor with the robot under test damaged, so the record reports falls with their own denominator and confidence limit beside task success rather than folding them into one success rate.
- **The denominator is stratified before collection.** The disturbance strata (slippery floor, a change in floor compliance, a curb, a shove) and their weights are fixed in the sampling protocol before the first trial, because a campaign on a flat floor bounds nothing about the strata it skipped.
- **A demonstrator's balance does not transfer.** A human operator keeps balance with a human body, not the robot's, so a cell such as a balanced heavy reach has no valid demonstrations; it enters the coverage gap catalog as a known absence, and the campaign cannot close it with trials it cannot afford to fail.

A log that fixes its denominator, strata, and coverage gaps before collection makes the record auditable, but it cannot stop a reader of that record from drawing conclusions it does not support. Practitioners still misread benchmark metrics and carry evidence beyond the target ODD in a few recurring ways, and those misreadings are the last hazard to the claim.

7.9 Fallacies and Pitfalls

An evaluation supports claims only within its tested conditions and limits. These mistakes arise when teams extend a result beyond those limits or attribute an architectural safeguard to the learned policy.

Fallacy: *Repeated success in a single nominal condition proves broad operational readiness.*

The door campaign repeats one condition twenty times: the nominal strike plate, the guarded approach, the base stopped, the rack empty. Under the binomial model, those runs give the 86.1

Downstream: Non-asymptotic Clopper-Pearson trial bounds supply the empirical evidence nodes for safety cases in section 16.3.

percent lower bound of 7.1 for that condition, and nothing for the other cells of the ledger or a loaded tote rack that moves the arm's torque margins. Repeating the condition sharpens the nominal estimate while the deployment-critical envelopes stay untested. A readiness claim must name its target ODD, allocate trials across the physical variations inside it, and report uncertainty within each tested cell.

Fallacy: *Zero failures in 1,000 trials proves compliance with a 99.9 percent reliability specification.*

Suppose the latch policy opens the door 1,000 times without a trip against a requirement of 99.9 percent success at 95 percent confidence. The observed success fraction is perfect, yet the one-sided lower bound $0.05^{1/n}$ is only 0.9970, short of the requirement (table 7.4); meeting it takes 2,995 clean runs. The gap is an evidence deficit, not a policy failure. The trial budget follows from the claim and confidence level, fixed in advance, and the report states the lower bound rather than the perfect fraction.

Fallacy: *High offline action agreement on static logs guarantees closed-loop physical stability.*

An evaluation report that cites close agreement with expert actions on recorded runs as release evidence has measured imitation on those records and nothing more. The aisle policy of section 7.3 matched its recordings almost exactly and still spent its side clearance to covariate shift. Agreement can screen candidates before physical trials, but a claim of closed-loop stability rests only on closed-loop outcomes counted against the full denominator, the metric-drift guard of section 7.7.

Pitfall: *Crediting the learned policy with surviving transport stalls that the enforcement architecture handled.*

The mobile manipulator completes 50 nominal aisle runs, and the team credits its learned policy with tolerance to SerDes bus jitter. A qualification test then injects a mailbox stall that outlasts the chosen 60 ms chunk lease of section 4.3. The lease expires, and the permission path detects the stale lease and initiates a controlled dynamic stop, analogous to a Category 1 stop, from the fallback ladder of section 12.9. That response comes from the enforcement architecture; nominal completion records do not establish that the neural weights detect or compensate for the stall. Fault injection must measure the permission path's detection and stopping behavior against the available stopping distance margin, and the evaluation report must attribute the protection to the hardware mechanism that supplied it.

7.10 Summary

Closed-loop evaluation measures the coupled policy, permission path, plant, environment, and trial protocol, so a success rate describes a declared population rather than a weight file. Because the machine's own admitted actions choose the states it reaches, which is Part II's endogenous-data principle, only trials that close the loop through the permission path measure what the proposer does; open-loop scores cannot see the drift those actions produce. A zero-failure confidence limit then bounds the sampled population under its sampling assumptions, and censoring at the edge of safe operation keeps that population inside the envelope the campaign could afford to enter. More trials narrow that bound without widening the population, and the exposure wall places rare-hazard rates beyond any campaign a physical machine can run.

The door campaign shows the shape of the claim that survives. Twenty clean runs support a lower confidence limit for the nominal strike plate at the guarded approach, with the base stopped and unloaded, and nothing about the displaced-plate cells the demonstrations never covered or about the loaded machine. The coverage gap catalog records the states no trial visited, which the endogenous-data principle predicts no dataset holds, and assigns them to independent physical constraints and a feasible fallback on the permission path rather than to the proposer, because a runtime OOD detector can miss what it has never seen.

The chapter's product is the evaluation record, and four of its parts travel forward:

1. *Bounded operational claim:* The target ODD, inside the policy manifest's declared ODD, where performance was empirically characterized.
2. *Reproducible configuration and ledger:* Digests of weights, firmware, and calibration, with every attempted, aborted, and overridden trial kept in the denominator.
3. *Coverage gap catalog:* The known absences of the data and the manifest's omission log, plus the conditions the trials left unvisited.

4. *Runtime monitor specifications*: For each boundary that can be sensed, the signal, threshold, and tested response that the permission path must implement at runtime.



Key Takeaways: Closed-loop evaluation and survival reliability

- **A success rate belongs to a declared population**: The rate reflects the coupled policy, permission path, plant, environment, and trial protocol. Changing the payload, the floor, or the reset procedure changes the estimand, so the target ODD and its sampling weights are fixed before the first trial.
- **Only a closed loop measures a proposer**: Open-loop scoring returns the machine to a demonstrated state at every step. A persistent 1.0° heading error scores as negligible offline yet spends the aisle's 0.15 m side clearance within 8.6 m, because the machine's admitted actions choose its next states.
- **Zero failures bound a rate without measuring it**: n clean iid trials give a one-sided 95 percent failure upper limit near $3/n$. Twenty clean runs leave room for a 13.9 percent failure probability, and a 99.9 percent success lower limit needs 2,995 clean runs.
- **The exposure wall moves rare hazards to architecture**: Bounding a rate λ with zero events takes about $3/\lambda$ of failure-free exposure, and a fleet's pooled hours share floors, builds, and conditions. Protection against rare hazards therefore rests on the permission path, a feasible fallback, and physical barriers, never on the learned weights.
- **An evaluation record states where its evidence stops**: Its claim stays inside the target ODD, its ledger keeps every aborted and overridden run in the denominator, and its gap catalog and runtime monitor specifications tell the runtime what the permission path must cover.



What's Next: From the evaluation record to runtime perception

Part II ends with a record that confines trust in the learned proposer to its target ODD and names the runtime monitors the rest of the machine must supply. Part III builds the runtime that meets those obligations. The first of them falls on perception, because an input-support monitor or a latency monitor can act only on observations that carry their capture time, their coordinate frame, and their uncertainty. Chapter 8 follows a measurement from the sensor to a spatial claim that the permission path can date, place, and bound before it permits motion.

III

RUNNING THE MACHINE

Running the Machine

As the warehouse mobile manipulator drives down an aisle, its navigation camera catches a person stepping out from the rack end ahead. The chunk policy's next proposal still carries the base forward, and the permission path must decide whether the observation behind it is still fresh and whether the remaining clearance permits that motion or only a stop. The decision has a deadline, and no better proposal helps once the machine has crossed its braking margin. Running a physical AI machine means reconciling the timescales of sensing, planning, and actuation with that physical deadline. Human goals may change over seconds and trajectories over tens of milliseconds, while the permission loop, in one implementation, runs every millisecond. The required rates and response times come from the plant and its measured delays, not from the names of the levels.



Principle 3: Proposal is not permission

Invariant: A learned model may propose motion; only an independent permission path may admit it. The path decides each action on state whose age is known, within a deadline set by observation age, computation, transport, actuator response, and the clearance the delay leaves, and it owns a fallback that is feasible from the current state without the proposer.

Implication: Every runtime stage serves one decision, whether this proposal can cross the causal boundary before the margin runs out. Stages ahead of the permission path deliver evidence with its age and uncertainty attached, the permission path runs on resources that learned workloads cannot delay, and any guarantee the path offers, such as a barrier filter's, holds only while its model, state estimate, actuator authority, and deadline hold.

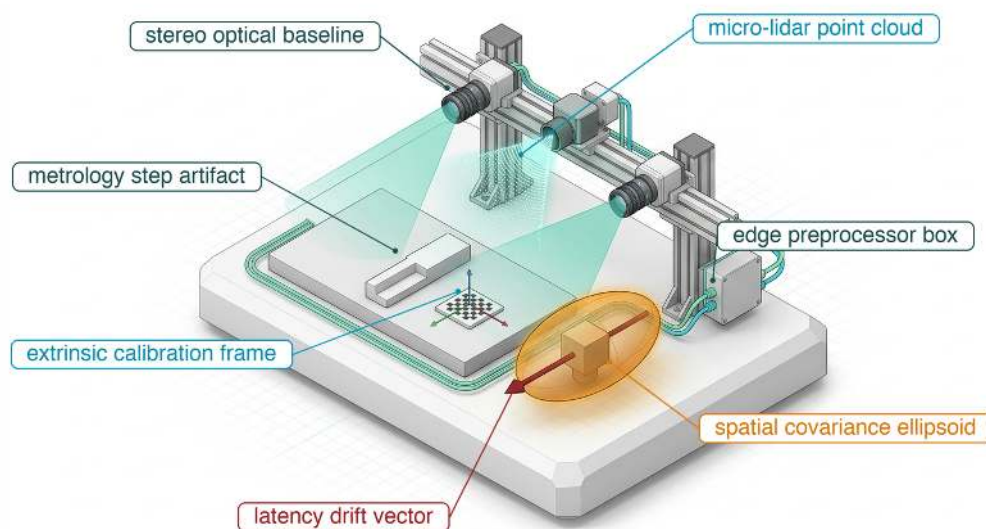
Part III follows the machine's proposals, trusted no further than Part II's evaluation record allows, through six stages. The base's run to the rack end and the arm's reach for the mug travel as one whole-body chunk stream under one chunk lease. Each proposal in that stream passes through four stages that build it, one that decides whether it may act, and one that keeps that decision on time.

- **Sensor Perception (chapter 8):** Decides whether a measurement is current and calibrated enough to use, and produces the observation contract, which dates each measurement in a clock the permission path can convert to its own.
- **Spatial Memory (chapter 9):** Decides how long an estimate may stand without fresh evidence, and produces the spatial belief record, which keeps the age of the evidence separate from the age of the estimate.
- **Grounded Intent (chapter 10):** Decides how a human goal becomes a bounded, expiring request, and produces the intent lease, which grants a target and effort ceilings but never a path.
- **Trajectory Planning (chapter 11):** Decides when a trajectory may commit, and produces the trajectory record, which carries its deadlines and a matched stop suffix.
- **Safety Enforcement (chapter 12):** Decides on every tick whether a proposal is admitted, projected, or refused (the first projection on the machine slows the arm's fast latch approach to the guarded speed that Part II's target ODD covers), and produces the enforcement record, where the two strands of records meet.

- **Silicon Placement (chapter 13):** Decides where the permission path runs so that learned workloads cannot delay it, and produces the placement record, which names the test that would support it.

Part III hands forward the runtime evidence that Part IV must judge. Observation contracts date every measurement, belief records expire, intent leases and trajectory records name their parents and their deadlines, and the enforcement record states the stopping envelope and the premises of each guarantee. Its finished stopping budget sets the speed at which the machine may run the aisle and leaves a spare of clear distance, the whole allowance for error in its inputs, for Part IV to spend (section 12.5). The placement record names the loaded test that would show learned workloads cannot delay the permission path (section 13.6), and on this machine its loaded field is still empty. Each record is a claim with conditions, and none is yet a license to operate.

Sensor Perception



- 8.1 Photons to Spatial Claims
- 8.2 What Perception Hands Over
- 8.3 Sensor Transduction and Calibration
- 8.4 The Cost of Ingestion
- 8.5 Spatial Error and Clearance Bounds
- 8.6 How Observations Go Wrong
- 8.7 The Observation Contract
- 8.8 Fallacies and Pitfalls
- 8.9 Summary

Purpose

Why does an obstacle detection with 99 percent recognition accuracy still cause a collision if its capture timestamp, coordinate frame, and spatial uncertainty are ignored?

An obstacle detection aids safe motion only when its exposure timestamp, reference coordinate frame, and spatial covariance are explicitly known. The physical world advances continuously while an image sensor integrates photons, transfers pixels across direct memory access buses, and submits tensors to an inference model. An accurate bounding box estimate describes geometry that may no longer exist by the time an actuator applies torque. Mechanical vibration, optical distortion, and thermal camera drift introduce spatial uncertainty, altering the relationship between raw pixel coordinates and physical clearance boundaries.

Reporting high-confidence object semantics without these spatial and temporal qualifications strips downstream controllers of the information required to guarantee safe separation. Real-time perception must preserve the provenance of every observation—tracking capture timestamps across clock domains and bounding measurement error ellipsoids. In physical AI architecture, perception serves as the bridge between raw sensory hardware at the causal boundary and cognitive reasoning in the Brain, transforming noisy signals into certified spatial claims that the deterministic permission path can verify before committing kinetic energy.

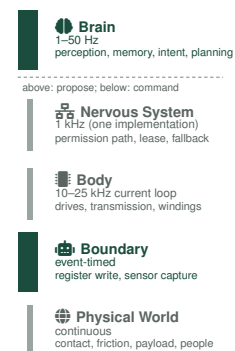


Figure 8.1: The five-level machine stack with the Brain and Boundary levels highlighted.



Learning Objectives

- Specify an observation contract that carries capture time, a bounded clock conversion, coordinate frame, and uncertainty
- Calculate observation age across the sensing, transport, preprocessing, and inference pipeline, and convert it into its stopping-budget term
- Diagnose spatial errors caused by sensor timing, motion during capture, and misaligned coordinate frames
- Explain why average capture bandwidth does not bound a frame's age or a renewal's lateness, and size the host contention a renewal allowance tolerates
- Evaluate whether observation uncertainty leaves enough physical clearance for a proposed action
- Distinguish perception failures that runtime checks can detect from errors that remain silent

8.1 Photons to Spatial Claims

THE warehouse mobile manipulator's vision model can recognize a person stepping out from a rack end almost every time and still hand the planner a position that is already out of date. The world keeps moving while the navigation camera integrates and reads out its image and while the pipeline processes it. On this machine, an illustrative 51.6 ms passes between the middle of the exposure and the moment the observation reaches any consumer, and at the base's 1.5 m/s drive limit the machine covers 77.4 mm in that time. Recognition accuracy does not say whether that observation is fresh enough for the proposed motion. In physical AI, an observation has an expiration date.

Perception runs in the Brain, above the proposal boundary of section 1.5. It turns photons, time-of-flight pulses, and reflected radio waves into dated spatial claims and proposes them, but it holds no authority over the actuators; that authority stays with the permission path below the boundary.

The work of turning photons into dated claims does not depend on how the architecture is drawn. An end-to-end vision-to-action policy (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Zhao et al. 2023) maps camera observations to proposed actions within one network, while a classical modular pipeline splits sensor ingestion, state estimation, mapping, and trajectory generation across separate processes. Marr's analysis of vision (Marr 1982) separates what a visual computation must achieve from the representation that carries it and the hardware that executes it, and in physical AI each of these levels must be realized somewhere between the stimulus and the response.

When an engineering team combines these responsibilities in an end-to-end model, the responsibilities remain, but their intermediate representations become harder to inspect independently. An explicit interface can expose a coordinate transform for checking, carry an expiration deadline, or let another component reject an inconsistent estimate. A hidden representation requires additional instrumentation or external checks to provide comparable evidence. If perception mistakes a wet-floor reflection for open clearance, the engineer needs a way to detect or contain the resulting error regardless of how many software modules implement the pipeline.

That evidence begins where physical energy converts into digital data. From transduction at the detector array to consumption by a state estimator, every processing stage adds observation age. Photons striking the photodiode array of a CMOS camera do not instantly appear as feature embeddings in an inference engine; exposure, readout, transport, memory transfer, preprocessing, and the vision backbone each take a share of the delay. If the perception stage hands downstream components an object detection bounding box without an explicit timestamp and a calibrated spatial transform, a downstream planner will evaluate that obstacle relative to the current position of the chassis, misplacing it by the distance the machine has traveled since capture.

Prerequisite: Transduction freshness and sensor measurement aging are derived in section 2.2.

Definition 8.1: Spatial information age

Spatial information age (t_{age}) is the total elapsed physical time between the physical energy integration midpoint ($t_{\text{exp}}/2$) at the transducer and the downstream consumption of the resulting observation tensor by a state estimator or control policy. It is the capture-to-consumption part of the measurement age that section 2.2 counts from transduction, and running text calls it observation age.

1. **Significance:** Because physical mass moves while digital computation executes, sensory latency translates directly into unmodeled spatial displacement (equation 8.1):

$$\Delta \mathbf{x} = \int_{t_0}^{t_0 + t_{\text{age}}} \mathbf{v}(t) dt \quad (8.1)$$

This continuous physical shift depletes geometric safety buffers before an obstacle is even classified.

2. **Distinction:** Unlike software pipeline latency (wall-clock duration of a forward pass), observation age accounts for the full physical journey: half the exposure, readout, transport, DMA and queueing, ISP processing, the vision backbone, and dispatch.
3. **Common pitfall:** Treating the frame arrival timestamp (t_{recv}) in userspace memory as the observation time (t_0). Using arrival time hides upstream capture and transport delays and adds unmodeled phase lag to the feedback controllers that consume the observation.

8.1.1 Temporal batching and phase margin erosion

Batching improves accelerator throughput by amortizing work across images, but collecting a batch across successive capture times ages its earliest member. At the navigation camera's 60 Hz cadence ($T_{\text{sample}} \approx 16.7$ ms), a batch of four consecutive frames holds the first for 50 ms, the second and third for 33.3 ms and 16.7 ms, and the last not at all. Adding the oldest frame's wait to the 51.6 ms observation age roughly doubles it, to 101.6 ms. That lies far above the refusal threshold, the budgeted age plus the clock-conversion bound that section 8.7 sets, so on this machine the permission path refuses every claim built from a batch collected across capture times. Images captured together by several cameras may instead be batched only when the batch adds no age beyond the budgeted stages and meets its deadline.

Age spends a feedback loop's phase margin as well, because a pure delay t_{age} adds phase lag ωt_{age} at frequency ω without changing gain. A loop with nominal margin PM_{nom} at gain crossover ω_c therefore exhausts it once $t_{\text{age}} \geq \text{PM}_{\text{nom}}/\omega_c$, and exposure, processing, transport, permission, and actuator delay all draw on that one ceiling (section B.3.5).

The same delay is paid twice, as phase lag in the loop and as travel toward the obstacle, and a consumer can budget only the delay it can measure. What perception hands over must therefore carry a timestamp, a coordinate frame, and a usable error bound, and transduction, ingress, and geometric estimation can each invalidate them before the permission path admits a motion.

8.2 What Perception Hands Over

Raw sensor hardware yields electrical measurements such as photodiode voltages, time-of-flight counter values, or capacitive displacements. These unstructured signals differ significantly from the state variables needed for downstream autonomy. An actuator controller cannot track a desired force using a two-dimensional grid of raw pixel intensities, and a trajectory optimizer cannot plan collision-free paths using unorganized time-of-flight returns. Downstream estimation, planning, and memory subsystems require structured representations of physical state, including metric positions, velocities, bounding volumes, surface normals, and semantic classifications. Translating raw sensor streams into these representations is the operational responsibility of the perception pipeline. Each observation it hands over must carry three properties for downstream software: a capture timestamp, a reference frame, and an uncertainty envelope.

The first mandatory property is the **capture timestamp** t_0 , which specifies the temporal midpoint of the physical energy integration window (the effective moment the camera shutter was open or the laser pulse fired). Sensors do not take instantaneous snapshots; a camera integrates photons across an exposure duration, and a mechanical lidar sweeps laser pulses over an azimuth arc across several milliseconds. Downstream state estimators require t_0 to integrate the measurement into the machine’s dynamic equations of motion. The arrival stamp that the definition warns against, t_{recv} , marks the moment the operating system kernel completes the direct memory access transfer. Half the exposure, readout, transport, and direct memory access (DMA) all elapse before that moment, so the arrival offset $\tau_{\text{recv}} = t_{\text{recv}} - t_0$ is never zero, and substituting arrival time for capture time corrupts the state estimator.

The magnitude of this corruption follows from equation 8.1 with the arrival offset in place of the full age, and at constant velocity v it reduces to $\Delta x = v \tau_{\text{recv}}$. On the mobile manipulator, the navigation camera’s frame completes its DMA transfer 26.6 ms after the exposure midpoint. An estimator that stamps the frame on arrival therefore dates it that much late, and at the base’s 1.5 m/s drive limit it pairs the observation with a pose 39.9 mm beyond the one the camera occupied at capture.

The second mandatory property is the **reference frame** identifier $\mathcal{F}_{\text{sensor}}$. Every spatial measurement is natively resolved in the local coordinate system of the physical transducer. A camera measures bearing angles relative to its optical center, while a radar measures range and Doppler velocity relative to its antenna array. Downstream trajectory planners and obstacle maps operate in a chassis body frame $\mathcal{F}_{\text{body}}$ or an inertial navigation frame $\mathcal{F}_{\text{world}}$. Transforming an observation from $\mathcal{F}_{\text{sensor}}$ to $\mathcal{F}_{\text{body}}$ requires applying a rigid-body spatial transformation consisting of a rotation matrix and a translation vector that define the sensor mounting extrinsics (the exact physical location and orientation where the sensor is bolted onto the machine). The frames nest from the globe down to the transducer, $\mathcal{F}_{\text{earth}} \rightarrow \mathcal{F}_{\text{map}} \rightarrow \mathcal{F}_{\text{odom}} \rightarrow \mathcal{F}_{\text{body}} \rightarrow \mathcal{F}_{\text{sensor}}$, and every link in that tree carries its own transform and uncertainty (figure 8.2).

When perception outputs omit the frame identifier or rely on nominal CAD dimensions rather than calibrated physical extrinsics, small angular offsets create large spatial errors that scale with distance. An uncalibrated pitch or yaw error of $\theta = 1^\circ$ on a sensor detecting an object at range $d = 15$ m induces a transverse position error $\Delta y \approx d \sin \theta$ of 0.26 m, about $1.7\times$ the 0.15 m side clearance the base keeps from the racks, so the planner either steers into the racking or rejects a valid aisle as obstructed.

The REP 105 frame convention names these frames and separates a locally continuous odom pose from a globally corrected map pose. On the mobile manipulator, $\mathcal{F}_{\text{body}}$ is the base’s `base_link`, tracked through the odometric frame `odom`, and the arm’s serial chain maps its mounting flange `too10` through the joint angles back to `base_link`. Table 8.1 lists the roles and the rule each consumer follows. A local controller consumes a continuous estimate and rejects unexpected transform jumps, and an implementation still has to check transform timestamps and estimator uncertainty.

Table 8.1: REP 105 frame roles for a mobile robot: A compact example of global, local, body, and sensor frames. REP 105 requires odom-relative pose to remain continuous but does not prescribe C^1 smoothness, a covariance growth rate, or fixed accuracy; these require estimator-specific evidence.

Frame	Parent in this example	Meaning	Consumer rule
earth	root	Optional geodetic datum	Verify datum before global routing
map	earth if present	Global localization; corrections may jump	Keep relocalization jumps out of local feedback
odom	map	Locally continuous pose estimate; may drift	Track motion locally; monitor uncertainty and discontinuities
base_link	odom	Body-fixed origin	Body pose still inherits odometry uncertainty
sensor_link	base_link	Calibrated sensor mounting frame	Apply time-valid extrinsics and their uncertainty

The third mandatory property is an **uncertainty representation**, such as covariance Σ together with a validated tail or hard error bound when clearance depends on it. Low illumination, atmospheric

Downstream: section 9.4 adds how these sensor-to-body transforms go stale over time.

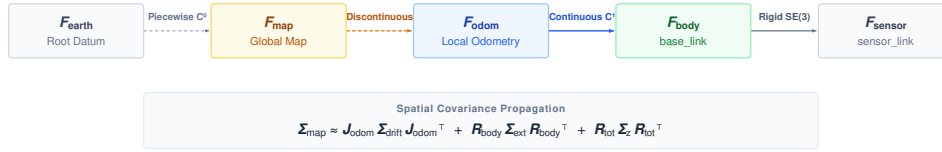


Figure 8.2: Coordinate frame tree and uncertainty: The spatial coordinate frame hierarchy formalizes the separation between continuous local dead reckoning ($\mathcal{F}_{odom} \rightarrow \mathcal{F}_{body}$) consumed by high-frequency tracking loops and discontinuous global map corrections ($\mathcal{F}_{map} \rightarrow \mathcal{F}_{odom}$) consumed by global planners. Spatial covariance compounds across the kinematic chain, from odometric drift through the mounting extrinsics to the sensor’s own measurement error, requiring downstream consumers to reject observations that lack an explicit coordinate key $\mathcal{F}_{sensor}$, capture epoch t_0 , and covariance envelope z . Attribution: Book kinematic systems diagram, CC BY 4.0.

scattering, specular surfaces, and model error change the residual distribution. A covariance describes second moments, not a region guaranteed to contain the true state. Estimators can use it to weight innovations against a motion model; an underestimated covariance can pull a state estimate toward a spurious measurement and lead to poor downstream commands. Safety decisions must also account for bias, missed detections, and validation limits.

Systems Perspective 8.1: The three invariants of observation

Pass a perception claim with its capture time and clock domain, coordinate frame, and qualified uncertainty. Refuse a claim when any of those cannot be checked for the proposed motion.

Together, these properties are the core of the observation contract that section 8.7 turns into a record that every downstream consumer checks before use. Dropping an incomplete or unbounded observation allows a downstream estimator to propagate its state forward using physical dynamics, degrading uncertainty smoothly over time. Accepting an ungrounded packet, by contrast, injects unmodeled delays, spatial distortions, and false confidence directly into the control loop. Enforcing this contract requires tracing how an observation originates, beginning at the hardware interface where physical phenomena are first sampled.

8.3 Sensor Transduction and Calibration

Lag accumulates before software receives anything at all, beginning at the photosite where physical phenomena are converted into electrical charge. A camera does not capture an instantaneous snapshot. Photons collect on a photodiode over a finite exposure duration t_{exp} , which means the recorded pixel intensity represents the temporal integral of scene radiance $I(t)$ over that interval:

$$I_{\text{pixel}} = \int_0^{t_{exp}} I(t) dt$$

When the sensor or the scene moves at velocity v during exposure, a sharp boundary such as a rack upright smears across several photosites, a motion blur of width $\Delta x_{\text{blur}} = v t_{exp}$. Because charge accumulates across the whole window, the time the accumulated image represents is the exposure midpoint, a lag of $t_{exp}/2$ behind the start of exposure. With the navigation camera’s 16 ms exposure at the base’s 1.5 m/s drive limit, the image does not record where the robot *is*. It records where the robot *was* 8 ms earlier, smeared across 24 mm of motion blur.

8.3.1 Rolling shutter distortion and motion compensation

The exposure window schedule across the pixel array determines the spatial geometry of the image. In a **global shutter** sensor, every pixel photodiode accumulates charge during the identical time interval and transfers the accumulated electrons to shielded storage nodes simultaneously. In a **rolling shutter** sensor, silicon area and cost constraints dictate that rows are reset, exposed, and read out sequentially from the top scanline $y = 0$ to the bottom scanline $y = H - 1$. Each successive

¹ | **Rolling shutter epipolar geometry:** Rolling shutter CMOS image sensors reset, integrate, and read pixel scanlines sequentially across the array rather than exposing all photosites simultaneously. When the camera rotates during this readout interval, straight physical edges project into curved geometric arcs across the image plane. This progressive scanline skew violates rigid multi-view epipolar constraints, causing the factor graphs of visual-inertial odometry, which estimates motion by fusing camera features with inertial measurements, to compute spurious feature depths unless continuous-time B-spline camera trajectories interpolate the instantaneous pose of every scanline.

scanline begins exposure after a row period t_{row} , creating a progressive temporal offset across the image plane:¹

$$t(y) = t_{\text{start}} + y \cdot t_{\text{row}}$$

where t_{start} is the start of exposure for the top row. When the body translates at linear velocity v perpendicular to a vertical feature, scanline y captures it at a spatial displacement:

$$\Delta x(y) = v \cdot y \cdot t_{\text{row}}$$

Under rotational motion, rolling shutter distortion becomes non-linear. If the camera undergoes angular rotation at angular velocity vector $\omega = [\omega_x, \omega_y, \omega_z]^T$ (e.g., rapid yaw or pitch during aggressive robot maneuvers), the effective camera attitude changes continuously as scanlines are read. Physically, the camera points in a slightly different direction for every single row. For a scanline at vertical coordinate y , the differential rotation relative to the frame start is $\Delta\theta(y) \approx \omega \cdot y \cdot t_{\text{row}}$. This scanline-dependent rotation warps straight physical lines into curves, induces focal-length dilation (scaling distortion during pitch), and shears vertical walls into slanted surfaces, as demonstrated by the propeller blade curvature in figure 8.3.

Consider the mobile manipulator's navigation camera while the base passes a vertical rack upright at its 1.5 m/s drive limit. The rolling shutter reads the frame's 3,040 rows in $t_{\text{readout}} = 16.6$ ms, so the inter-row interval is $t_{\text{row}} = 5.46$ μ s. An upright spanning the full height of the frame does not appear vertical in the captured array. The bottom scanline ($y = 3,039$) is sampled 16.6 ms after the top scanline ($y = 0$), producing a total spatial skew $\Delta x = v t_{\text{readout}}$ of 24.9 mm.

If downstream geometry reconstruction treats this image as an instantaneous rigid projection, the 24.9 mm tilt enters the obstacle map uncorrected. That is about a sixth of the 0.15 m side clearance the base keeps from the racks, and the motion planner can read the slanted upright as an artificial intrusion into the free corridor. The stopping budget's localization allowance δ_{loc} assumes this skew has been removed before the estimator sees the frame. Correcting this distortion requires **rolling-shutter motion compensation**, the continuous-time geometric unwarping algorithm that utilizes high-rate (1,000 Hz) inertial angular velocities (ω) and linear velocities (v) to interpolate the instantaneous 6-DoF camera pose $\mathbf{T}_{wb}(t(y))$ for each individual scanline y , restoring rigid epipolar geometry prior to 3D triangulation.

8.3.2 Camera calibration

Rolling-shutter compensation assumes that the camera's geometry is already known, and every metric claim the camera supports rests on that calibration. Under the **pinhole camera model** (Hartley and Zisserman 2003), the intrinsic matrix $\mathbf{K}$ maps a point $\mathbf{P}_c = [X_c, Y_c, Z_c]^T$ in the camera's optical frame to the pixel $[u, v]^T$:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \frac{1}{Z_c} \mathbf{K} \mathbf{P}_c = \frac{1}{Z_c} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} \quad (8.2)$$

where f_x, f_y are focal lengths in pixels and (c_x, c_y) is the principal point. Inverting the map recovers only a bearing ray, along which every depth is consistent with the pixel:

$$\mathbf{r}(u, v) = \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{u - c_x}{f_x} \\ \frac{v - c_y}{f_y} \\ 1 \end{bmatrix} \quad (8.3)$$

Metric depth must therefore come from stereo disparity, a range sensor, or a learned prior, each with its own error, and section 8.5 charges that error against the clearance. Lens distortion is rectified in the image signal processor stage of table 8.2. The projection's derivation, the distortion model, and Zhang's planar calibration (Zhang 2000) are developed in section C.12.



Figure 8.3: Rolling shutter distortion geometry: A photograph of rotating propeller blades captured by a rolling shutter CMOS sensor. Because sensor scanlines are exposed and read sequentially from top to bottom rather than simultaneously, the rapid physical movement of the blades relative to the row period (t_{row}) shears and curves straight rigid structures into non-linear, disconnected geometric arcs. In high-speed embodied systems, uncompensated rolling shutter skew introduces large 3D triangulation errors and phantom obstacle intrusion in downstream visual-inertial odometry. (Credit: Wikimedia Commons, CC BY-SA 3.0).

The camera's relation to the inertial measurement unit must be calibrated in space and in time together (Furgale et al. 2013). An extrinsic rotation error θ misplaces a point at range z by about $z\theta$, and the camera's offset from the inertial unit adds lever-arm accelerations (section B.4.3). An uncalibrated offset Δt between exposure and the inertial sample misorients the camera by $\omega \Delta t$ while the base turns at angular velocity ω , misplacing the same point by about $z\omega \Delta t$, so the timing error grows with both turn rate and range. Latching exposure starts and inertial samples on one hardware trigger bounds that offset, leaving a residual that online calibration estimates. Each of these calibrations holds only inside the conditions under which it was fitted, which is why the observation record carries a calibration revision and its validity range.

8.3.3 The latency waterfall

Exposure and readout are only the first stages a frame passes through. Summing the stages of the latency waterfall of section 2.2 up to dispatch gives the observation age:

$$t_{age} = \frac{t_{exp}}{2} + t_{readout} + t_{transport} + t_{DMA} + t_{ISP} + t_{backbone} + t_{IPC}$$



Napkin Math 8.1: Transduction latency and information age

The scenario. The mobile manipulator drives toward a rack end at its drive limit when a person steps out. We sum the navigation camera's observation age t_{age} from exposure midpoint to observation dispatch and compute the distance the base covers in that time, $\Delta x = v t_{age}$.

Stage-by-Stage Latency Waterfall:

1. *Exposure Integration Midpoint* ($t_{exp}/2$): The navigation camera's exposure $t_{exp} = 16$ ms (datasheet) $\Rightarrow \Delta t_1 = 8$ ms.

2. *Rolling Shutter Line Readout* (t_{readout}): The rolling readout of the frame contributes $\Delta t_2 = 16.6$ ms (datasheet).
3. *MIPI CSI-2 Serialization & Transport* ($t_{\text{transport}}$): The Sony IMX477 flyer specifies a four-lane option up to 2.1 Gbps/lane. The budget assigns $\Delta t_3 = 0.8$ ms (illustrative) to residual transport after streaming readout; the value is not frame size divided by peak lane rate.
4. *DMA Ingress & Kernel V4L2 Buffer Queue* (t_{DMA}): Scatter-gather DRAM write and driver interrupt $\Rightarrow \Delta t_4 = 1.2$ ms (illustrative).
5. *Hardware ISP Debayering & Rectification* (t_{ISP}): Dedicated on-chip ISP hardware pass $\Rightarrow \Delta t_5 = 2.5$ ms (illustrative).
6. *Vision Backbone Tensor Forward Pass* (t_{backbone}): INT8 quantized Vision Transformer on edge NPU $\Rightarrow \Delta t_6 = 22$ ms (illustrative).
7. *Covariance Packaging & Zero-Copy IPC* (t_{IPC}): Shared-memory circular ring buffer dispatch $\Rightarrow \Delta t_7 = 0.5$ ms (illustrative).

Total Observation Age:

$$t_{\text{age}} = 8\text{ms} + 16.6\text{ms} + 0.8\text{ms} + 1.2\text{ms} + 2.5\text{ms} + 22\text{ms} + 0.5\text{ms} = 51.6\text{ms}$$

Displacement at the Drive Limit:

At the base's drive limit $v = 1.5$ m/s, holding speed through the age window:

$$\Delta x = (1.5\text{m/s}) \times (0.0516\text{s}) = 0.0774\text{m} = 77.4\text{mm}$$

Systems Impact: The base covers this distance before any consumer can read the frame that shows the person. Timestamped pose propagation can place the old observation correctly in the current frame, but no later stage returns the distance already traveled.

The seven stages of table 8.2 sum to the observation age t_{age} of 51.6 ms, which enters τ_{delay} . This age completes the worst case that section 4.3 left open, a proposer that renews its lease and then stalls. The frame that shows the person reaches a proposer only t_{age} after the person appears. The last renewal the proposer sends before it could have seen the person can leave as late as t_{age} after the person appears, and if the proposer then stalls, the lease runs its full length from that renewal. The delay from the person's appearance to the first retarding force is therefore the age plus the 60 ms lease, a 1 ms tick, a 1 ms bus cycle, and 20 ms of brake onset, a total τ_{delay} of 133.6 ms.

The permission path never reads the image, so it cannot shorten the age term; the age reaches it only through the proposal's evidence epoch. At the 1.5 m/s drive limit, the observation-age term vt_{age} adds 77.4 mm to the stopping distance of equation 2.1. The running total rises from the 835.5 mm that the rows through the lease path produced to 912.9 mm, which leaves 187.1 mm of the 1.10 m rack-end clear distance.

Table 8.2: Serial information-age budget: The navigation camera's stage increments, assumed nonoverlapping, sum to its observation age. A real pipeline can overlap readout, transfer, and processing, so the age that binds a design is the measured capture-to-consumer age, including queueing tails.

Stage	Increment	Serial age after stage	What to verify
Exposure midpoint	8 ms	8 ms	Exposure and row timestamp convention
Rolling readout	16.6 ms	24.6 ms	Selected row and overlap with streaming
Residual transport	0.8 ms	25.4 ms	Selected sensor mode and host receiver
DMA and queue	1.2 ms	26.6 ms	Arbitration and queue tail
ISP	2.5 ms	29.1 ms	Mode and competing jobs
Vision backbone	22 ms	51.1 ms	Workload, clock, thermal state
Dispatch	0.5 ms	51.6 ms	End-to-end capture-to-consumer age

📌 Checkpoint 8.1: Spatial information age and the stopping budget

Before propagating visual perception into spatial planning, verify your understanding of how pipeline latency creates unmodeled spatial displacement:

- ❑ **Spatial information age:** Can you explain how physical latency from photon exposure to shared-memory dispatch converts directly into unmodeled kinematic displacement, eroding clearance margins before a control policy can react?
- ❑ **Exposure midpoint vs. arrival timestamp:** Can you describe why treating kernel receive timestamps rather than exposure integration midpoints as the capture epoch adds phase lag that can destabilize closed-loop feedback controllers?
- ❑ **Pose propagation from capture time:** Can you explain how propagating the machine's pose forward from an observation's capture time places an old observation correctly in the current frame, and why that propagation cannot return the distance already traveled?
- ❑ **Observation age in the stopping budget:** Can you explain why the observation-age term vt_{age} adds to the stopping distance, and why the permission path, which never reads the image, cannot shorten it?

The serial budget treats transport, DMA, and queueing as fixed increments, yet each depends on what else the host is moving at the same moment. Those terms are where camera traffic can add age that no single stage owner measures.

8.4 The Cost of Ingestion

Raw pixel streams arriving over MIPI CSI-2 or Ethernet consume DMA and memory service before inference can read them. On a shared application processor, camera writes can also delay other host reads, or overrun a receiver and drop a frame.

8.4.1 MIPI CSI-2 DMA ingress and zero-copy buffers

Once analog-to-digital converters on the sensor die digitize the pixel charges, the frame must travel across a physical interconnect to host memory. Camera streams enter the application processor over the MIPI CSI-2 bus.²

🔍 Systems Perspective 8.2: Silicon ingestion dataflow architecture

1. **D-PHY / C-PHY physical layer:** Physical lanes serialize pixel packets at the selected sensor and receiver's supported rate; packet headers carry error checks.
2. **System-on-chip (SoC) CSI-2 host controller:** Deserializes byte packets, strips headers, and validates CRC integrity.
3. **Scatter-gather DMA controller:** A dedicated DMA engine transfers pixel byte words directly into DRAM over the internal system interconnect (e.g., AXI crossbar).
4. **Kernel DMA ring buffers:** A Linux v4l2 driver can map capture buffers into user space.

² | **MIPI CSI-2 bus protocol:** CSI-2 transmits serialized pixel streams over D-PHY or C-PHY lanes to a host receiver and DMA controller. It can avoid a network stack, but transport and queueing delay still depend on sensor mode, lane rate, controller, and competing traffic; see section C.8.

Once DMA write completion occurs, the kernel driver issues an interrupt.³ A driver may export a DMA buffer through `dma_buf` so compatible ISP or accelerator paths can share storage without an extra CPU copy. Format conversion, layout changes, or device constraints can still require a copy or transform.

The physical interconnect choice dictates the lower bound on sensor transport and host ingestion overhead (figure ??). In robotic systems, designers frequently select camera interfaces based on mechanical convenience (such as USB 3.0 cabling) without accounting for driver queueing and protocol packetization taxes. Measuring the full photon-to-actuator latency waterfall across four standard embodied vision interfaces reveals the cumulative cost of these transport choices. While standard industrial USB 3.0 introduces a substantial 58.4 ms closed-loop latency due to USB Video

³ | **Video4Linux2 direct memory access:** A V4L2 driver may use scatter-gather or physically contiguous DMA storage and expose a capture buffer through `V4L2_MEMORY_MMAP`, which a compatible `dma_buf` importer can share. See section C.8.

Class (UVC) driver packetization, userspace kernel copies, and variable USB frame-interval polling, embedded MIPI CSI-2 achieves a deterministic 34.6 ms round trip via direct SoC D-PHY direct memory access (DMA). Automotive GMSL2 serializers/deserializers (SerDes) match the latency performance of MIPI CSI-2 (34.8 ms) while extending physical signal reach to 15 meters over lightweight coaxial cable with only ~ 50 μ s link serialization delay. Finally, neuromorphic event cameras (such as Prophesee Metavision DVS) bypass conventional exposure integration and rolling shutter readout altogether, streaming asynchronous polarity changes within microsecond intervals to enable an ultra-responsive 11.9 ms closed-loop reaction time.

8.4.2 Memory bus contention and observation age

The raw rates of the machine's navigation and wrist cameras are sized in 5.1. When the perception stack reads those frames, copies or transforms intermediate buffers, and the accelerator reads weights and writes activations, the memory controller carries several times that ingress rate. Even that multiple sits well inside the memory system's peak bandwidth when averaged over a second.

Average bandwidth does not determine the latency of one read. Synchronized cameras concentrate their DMA writes into bursts, and arbitration granularity, read priority, and bank placement decide how long any other request waits behind them. On the mobile manipulator the burst lands only on the application processor. The permission path runs on its own microcontroller and reads its state from its own memory, so a burst cannot stretch its tick, and the shared-die placement that would expose those reads is the one section 13.3 rejects. The burst reaches the permission path only as the age of the frame a proposal is built on or the lateness of the renewal the chunk policy sends, and the two have different allowances. A frame that waits behind camera and accelerator traffic reaches dispatch older than the budgeted stages, and the refusal threshold of section 8.7 leaves no slack above them, so every proposal built on that frame is refused.

A chunk whose inference waits behind the same traffic renews the lease late, and there the lease rule of section 4.8.1 leaves 5 ms of renewal jitter above the 50 ms chunk period. Host contention may therefore stretch the chunk policy's 40 ms inference by at most 12.5 percent, and by less in practice, because the same allowance also holds the proposer's own jitter and the jitter of the board-level link to the microcontroller (section 13.7). A renewal 10 ms late lets the lease lapse and stops the machine. Contention on this placement costs availability, never distance, because the permission path refuses what the stopping budget did not charge.

How long a burst holds the bus also matters to the camera itself. If the added delay outlasts what a receiver FIFO can buffer, the receiver may overrun and drop the frame, with a threshold that depends on its buffer and burst pattern. A dropped frame doubles the interval between two otherwise periodic samples, and the consumer treats the missing update as an age or sequence fault rather than assuming that a new image will arrive on schedule.

The hardware mechanisms governing frame delivery (DMA rings, IOMMU mappings, and DRAM arbitration) are treated by Hennessy and Patterson (2019) and Bryant and O'Hallaron (2015). For this chapter's decision, the quantity that matters is capture-to-consumer age and its tail under the intended camera workload.

An on-time frame answers only when the scene was sampled. Where the objects in it are, and how far that estimate can be trusted, is a separate question, and the error in that answer is spent from the same clearance.

8.5 Spatial Error and Clearance Bounds

When perception places a rack upright a few centimeters farther from the base's path than it actually stands, the base plans its pass with those centimeters missing from the 0.15 m side clearance it keeps from the racks, and no stage of the pipeline decided to spend them. Once downstream control logic uses an estimated coordinate to authorize motion, a spatial error becomes a physical displacement, spent out of the same clearance that keeps the machine off the obstacle. Transforming a sensor observation into a clearance claim requires **metric grounding**, which maps raw detector measurements into geometric space. This grounding relies on unprojecting two-dimensional sensor coordinates into three dimensions, an inversion that assumes known optics, rigid mounting, and predictable surface properties.

The mathematical principles of projection and triangulation, detailed by Hartley and Zisserman (2003) as well as Szeliski (2022), govern how measurement noise converts into spatial displacement. In passive stereo vision, depth z is computed from baseline b , focal length f , and disparity d through the relationship $z = \frac{fb}{d}$. Differentiating this unprojection with respect to disparity yields the depth uncertainty (equation 8.4):

$$\delta z \approx \frac{z^2}{fb} \delta d \quad (8.4)$$

where δd is a small disparity error.⁴ If δd denotes a disparity standard deviation σ_d , first-order propagation gives $\sigma_z \approx z^2 \sigma_d / (fb)$. Doubling range quadruples this modeled standard deviation at fixed f , b , and σ_d . For distant landmarks, an estimator instead carries depth as inverse depth $\rho = 1/z$, whose error stays roughly linear, and this **inverse-depth metric grounding** keeps an extended Kalman filter from diverging on the quadratic tail. A pulsed time-of-flight sensor instead estimates $z = c\Delta t/2$; its range error depends on its timing and detection mechanism and may be approximately flat over a specified operating region.

To quantify the geometry, assume an illustrative stereo pair (the navigation camera alone recovers only the bearing ray of equation 8.3) with $b = 0.20$ m, $f = 800$ px, and disparity standard deviation $\sigma_d = 0.5$ px over the tested range. First-order propagation gives $\sigma_z = 12.5$ mm at 2 m and $\sigma_z = 312.5$ mm at 10 m (figure 8.4). Those are **one-standard-deviation scales**, not clearance insets or 99th-percentile bounds. A usable inset requires a tail quantile of the *full spatial residual*, calibrated on the intended sensor, range, texture, motion, and lighting conditions, with systematic bias and missed detections handled separately. A 150 mm clearance kept along the viewing ray to an obstacle therefore cannot be widened to a valid inset of 162.5 mm or 462.5 mm merely by adding one σ_z . Speed must be reduced or the proposal refused whenever the validated range-error and stopping margins do not fit.

8.5.1 Clearance inset margins and transform chain propagation

Spatial error extends beyond the sensor optical frame. To govern motion, the estimated point must propagate through coordinate transformations to the robot base frame and the end effector. The mechanics of rigid body transformations and SE(3) kinematic frame composition are standard results treated by Craig (Craig 2005) as well as Lynch and Park (2017). Each link in this transform chain contributes scale errors, mounting calibration drift, joint encoder quantization, and mechanical compliance under load (such as a robot arm sagging slightly under gravity). Lever-arm effects amplify a small angular twist at the sensor mount linearly over distance, sweeping projected coordinates across a wide arc. If the navigation camera's extrinsic calibration shifts by $\theta = 0.005$ rad against the chassis, the lateral error $\delta x \approx z\theta$ at $z = 10$ m is 50 mm, equal in size to the whole 50 mm localization allowance δ_{loc} that the stopping budget holds along the direction of travel. The comparison is one of scale, not allocation. The allowance carries small validated residuals, such as the clock-conversion travel of section 8.7, and a shift that matches it alone is a calibration failure the consumer must refuse through the calibration validity check rather than absorb. When combined with quadratic depth uncertainty, the total spatial error vector $\epsilon_{\text{spatial}}$ at long range is dominated by depth along the optical axis and angular lever-arm effects orthogonal to it.

Every physical safety barrier operates by defending a geometric boundary around the machine. For position error vector $\epsilon_{\text{spatial}}$, the triangle inequality gives a conservative relation between nominal and true clearance:

$$C_{\text{true}} \geq C_{\text{nominal}} - \|\epsilon_{\text{spatial}}\|$$

For a validated 99th-percentile norm-error quantile, the permission path can choose an **inset clearance** C_{inset} (equation 8.5):

$$C_{\text{inset}} = C_{\text{safe}} + \|\epsilon_{\text{spatial}}\|_{P_{99}} \quad (8.5)$$

Here C_{safe} is the physical clearance the motion must keep, such as the base's side clearance, and the quantile must come from a specified, checked residual distribution for the full spatial error, including calibration bias. Even then it leaves a one-percent tail under the test distribution; only a justified hard bound on the relevant error, a feasible stopping path, and a permission response within the remaining delay budget would close it. Figure 8.4 shows how stereo's growing depth-error scale can require a larger inset at longer range.

⁴ | **Inverse-depth parameterization:** In visual state estimation, parameterizing 3D landmarks by inverse depth ($\rho = 1/z$) converts the non-linear, heavy-tailed depth error distribution ($\delta z \propto z^2$) into an approximately Gaussian error space over the image plane. Under inverse depth, points at optical infinity ($z \rightarrow \infty$) exhibit finite variance ($\rho \rightarrow 0$). For the rigorous derivation and factor graph formulation, see section A.3.

Downstream: Spatial covariance envelopes inflate the forward-invariant safety barrier sets derived in section 12.7.

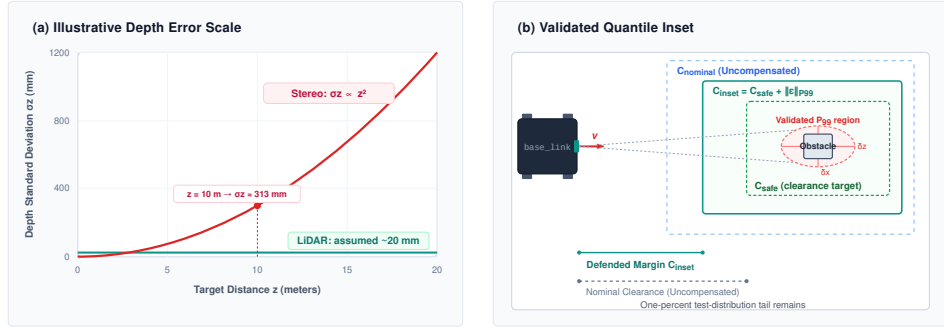


Figure 8.4: Depth error scale and clearance insets: (A) A stereo one-standard-deviation depth scale grows with squared range for fixed disparity spread; a qualified time-of-flight sensor can be flatter. (B) A validated 99th-percentile *norm-error* quantile enlarges the nominal clearance inset and leaves a residual tail and bias risk. Attribution: Book geometric error analysis, CC BY 4.0.

Because clearance depends on propagated error, downstream planners and the permission path cannot consume raw coordinate tuples. A covariance $\in \mathbb{R}^{3 \times 3}$ alone does not say whether a 10 mm or 500 mm clearance inset is warranted; that takes an independently validated tail or hard bound.

When spatial error is qualified, the permission path can expand the clearance inset, trading speed or workspace for separation. A textureless wall, glare, or lost calibration can invalidate that qualification without making a numerical variance literally infinite. The permission path must then refuse motion that depends on the affected claim and select a feasible fallback.

On every frame the producer builds a spatial claim in a fixed order, and any step can end it. It first converts the capture timestamp to the consumer's clock with a bounded synchronization error and computes t_{age} ; if age plus clock uncertainty exceeds the refusal threshold, it marks the observation stale, and proposals that need it are refused. The permission path repeats this check at its own tick on the upper age the dispatched record carries (section 8.7). For a fresh observation, the producer then propagates position and uncertainty to the decision time with a validated motion model, unprojects each pixel measurement into optical coordinates with a sensor covariance,⁵ and carries point and covariance into the body frame by first-order SE(3) error propagation through the calibrated extrinsics. Validated residuals, including bias, then set the inset radius, and when no valid bound remains, the affected motion is refused. What leaves perception is a claim $(\mathbf{p}_{\text{body}}, \text{body}, C_{\text{inset}}, t_{\text{age}})$, not a bare coordinate.

8.5.2 Learned scene features and depth

Open-vocabulary perception adds a learned semantic layer above metric sensing, built from **3D vision foundation models**. A self-supervised backbone such as DINOv2 (Oquab et al. 2023) yields dense patch features, a promptable segmenter such as SAM (Kirillov et al. 2023) yields class-agnostic instance masks, and systems such as ConceptGraphs (Kuwajerwala et al. 2024; Jatavallabhula et al. 2023) and Hydra (Hughes et al. 2022) lift those masks and features through calibrated depth, extrinsics, and odometry into an **object-centric 3D scene graph** whose nodes carry a centroid, a spatial covariance, and a text-aligned embedding.

These features spend a compute and age budget (Williams et al. 2009). An assumed 3 TFLOP encoder at the navigation camera's 60 Hz frame rate would require 180 TFLOP/s of sustained useful throughput before preprocessing or memory traffic, while NVIDIA lists up to 43 dense FP16 TFLOP/s for the Jetson AGX Orin in its technical brief, a peak arithmetic rate rather than sustained model throughput. The encoder cannot run at camera rate on that processor even at the arithmetic ceiling, so perception splits into a **multi-rate hierarchy**. Metric ingress on the application processor turns the camera's DMA buffers into obstacle points and occupancy grids at 30–60 Hz without the heavy backbone, and the foundation models update the scene graph's semantic nodes on keyframes at 1–5 Hz. A slow semantic update is admissible only when fresh metric sensing and the permission path hold the physical margin between updates.

⁵ | **Metric unprojection and covariance propagation:** First-order propagation maps measurement covariance to $c = J_{\text{unproj}} \text{meas} J_{\text{unproj}}^T$ and then to $b = R_{bc} R_{bc}^T + c_{\text{ext}}$. Its largest eigenvalue gives a principal variance scale, **not** a 99th-percentile error radius. A quantile requires an assumed and calibrated error distribution, including bias; see section C.12.

A camera-to-bird's-eye-view lifting stage such as Lift-Splat-Shoot (Phillion and Fidler 2020), developed in section A.3.3 (figure A.1), places learned depth hypotheses from 2D vision backbones in the planner's metric grid, but its depth-bin weights are model scores rather than calibrated metric error, so the clearance inset of equation 8.5 still rests on validated residuals.

Every bound in this section, metric or learned, assumes that the sensor behaves as calibrated. Propagation bounds the error of a sensor working as modeled; it says nothing about a sensor that has stopped working that way while still delivering frames at its nominal rate.

8.6 How Observations Go Wrong

The navigation camera can fail in two ways that look nothing alike to its consumer. If its serializer link loses lock, the driver raises an error and the frame never arrives. If light through an open dock door saturates the lens as the base turns into an aisle, the camera delivers a complete frame at its nominal rate in which the rack end is a patch of saturated white. Perception failures divide along exactly this line into two operational classes. **Detectable faults** occur when the physical sensor or the communication transport reports an explicit error condition. A serializer-deserializer link drops a lock (losing clock synchronization), a cyclical redundancy check detects a corrupted payload, a frame buffer overflows, or a transport bus times out. These failures are straightforward to handle since the hardware driver asserts an error flag at the ingestion boundary. Downstream consumers register the missing frame and invalidate stale state estimates, and once the evidence age exceeds its budget the permission path refuses and moves the machine to a validated fallback. In contrast, **silent failures** occur when the sensor hardware and transport pipeline function without error, delivering a fully populated, structurally valid tensor whose numerical values misrepresent the physical world.

Physical degradation of the optical channel is a common source of silent error.⁶ When direct sunlight enters an optical aperture, incident photons exceed the physical charge capacity (the "full-well capacity") of the CMOS photodiodes, clamping pixel intensities at their maximum quantization limit (clipping the image to pure white) across large patches of the array. This **dynamic range saturation** destroys local spatial gradients. Feature extractors, optical flow estimators, and disparity matching algorithms fail because mathematical optimization on a uniform flat field yields zero information, yet the driver emits a valid frame at nominal line rate. Low-light environments introduce the opposite failure mode. As ambient illumination drops, the discrete, random arrival of individual photons (**photon shot noise**) and the intrinsic electrical fluctuations of the sensor's amplifier (**sensor read noise**) dominate the measurement. The signal-to-noise ratio collapses, causing downstream neural networks to extract geometric features from random noise fluctuations rather than physical scene boundaries. Environmental contaminants on the protective lens or dome window (such as water droplets, airborne dust, or mud splatters) act as irregular refractive elements and optical low-pass filters. They attenuate contrast, distort spatial geometry, and obscure obstacles while the transport bus continues to operate without asserting a single fault flag.

Timing and transport anomalies can also corrupt an observation silently. If an unshielded physical interface experiences electromagnetic interference, deserializer bit slips (where the hardware drops or duplicates a bit, misaligning the entire subsequent data stream) can alter payload semantics without invalidating standard packet lengths. More dangerous than a lost packet, because nothing flags it, is an observation delivered with a timestamp that misreports its true capture epoch. If an image sensor captures photons at epoch t_0 , but a late interrupt handler or an unsynchronized system clock timestamps the resulting memory buffer at $t_0 + \delta t$, the downstream state estimator integrates spatial features under an incorrect body pose. Consider a machine traveling with instantaneous velocity v and linear acceleration a along a given trajectory. When perception suffers an undetected timing desynchronization δt , the state estimator assigns the detected position of an obstacle to the body coordinates at $t_0 + \delta t$ rather than t_0 . Over the unmodeled time interval δt , the physical state estimation error Δp between the asserted position and the true position is governed by the kinematic drift relation (equation 8.6):

$$\Delta p = v \cdot \delta t + \frac{1}{2} a \cdot (\delta t)^2 \quad (8.6)$$

The arrival stamp of section 8.2 is a silent desynchronization of exactly this kind. At a constant drive speed the acceleration term vanishes, leaving the $v \tau_{\text{recv}}$ offset sized there, and nothing raises a fault flag. Because the pipeline delivers a continuous stream of valid tensors, the state estimator

⁶ | **Lidar range walk and detector blooming:** Analogous silent degradation occurs in active time-of-flight lidar receivers utilizing Avalanche Photodiodes (APDs) or Single-Photon Avalanche Diodes (SPADs). When an emitted laser pulse reflects off a retroreflective surface, intense return energy drives the photodetector into non-linear charge saturation, accelerating leading-edge discriminator threshold crossing via *range walk* (detector blooming). The resulting timing distortion artificially shortens the calculated round-trip time (t_{TOF}), shifting the apparent obstacle boundary several centimeters toward the sensor without asserting a hardware fault.

assumes a fresh observation, computing safe actuator trajectories from stale physics. A missing frame causes a known information loss that a Kalman filter can handle by expanding its state covariance. A frame that arrives on time with a corrupt timestamp causes the filter to contract its covariance around an incorrect mean, driving actuators toward an obstacle that the model believes has already been cleared.

Defending against silent failure requires dedicated runtime health monitors at the sensor ingestion boundary. A pixel intensity histogram monitor measures the distribution of raw digitized values immediately after direct memory access transfer, identifying over-exposure saturation when values concentrate at the maximum bit depth or identifying lens occlusion when spatial variance drops below an empirical noise floor. A hardware watchdog timer tracks inter-arrival times across the direct memory access channel, flagging transport jitter when inter-frame intervals exceed $T_{\text{nominal}} + 3\sigma_{\text{jitter}}$. Higher up the stack, an optical flow plausibility monitor compares computed pixel displacement vectors against the physical velocity envelope of the body, rejecting observations that imply accelerations beyond mechanical capability. In multi-modal configurations, a cross-sensor consistency check compares high-rate (1,000 Hz) inertial measurement unit integrations against visual odometry. If the visual pipeline reports that the machine is stationary while the accelerometer measures a sustained 2.0 m/s^2 specific force (the physical acceleration felt by the sensor body), the monitor detects the discrepancy and trips a perception fault.

Every runtime health monitor sees the world only through what the transducer reported at the causal boundary of section 1.2, so a single sensing channel cannot validate its own semantic interpretations. A camera assembly with clean optics, balanced exposure, jitter-free transport, and synchronized microsecond clocking can still present an observation that leads a learned model to hallucinate a clear path or ignore an obstacle. Because deep vision models project high-dimensional inputs through complex non-linear manifolds, in-distribution optical anomalies (such as a high-contrast shadow across an aisle, a person printed on a carton, or specular glare from shrink-wrap or a wet floor) generate plausible pixel statistics, valid optical flow fields, and high model confidence. A single-sensor monitor evaluates whether the data collection process operated within nominal limits, not whether the inferred semantics match physical reality. A second modality, such as lidar or radar, can corroborate some optical claims when its coverage and failure dependence have been checked. A separately validated permission path can enforce clearance only within its sensing, model, timing, and actuator assumptions; neither route establishes unrestricted semantic truth.

The comparison in table 8.3 shows what each sensing mechanism can claim and what must be checked before combining claims. A second modality can reduce a shared failure mode only when its own coverage, timing, calibration, and fault correlation are established.

Table 8.3: Spatial sensor modality comparison: Sensor mechanisms provide complementary but fallible claims. Numeric range, accuracy, refresh, and link budgets require a named device and mode. For example, Prophesee EVT2 encodes an event in 32 bits, so an event camera's link load scales with its event rate before transport overhead.

Modality	Direct physical information	Main limitation	Permission-relevant check
RGB camera	Bearing, appearance, and image motion	Monocular depth scale is ambiguous; glare, darkness, and occlusion can hide objects	Fresh capture time and validated metric corroboration
Calibrated stereo	Disparity-derived range over a qualified baseline and texture region	Range error grows with distance for fixed disparity error; calibration and matching can fail	Range-dependent residual and bias validation
Pulsed lidar	Time-of-flight range at sampled directions	Precision, angular coverage, and weather response depend on the device and scene	Qualified range, return quality, and scan timing
FMCW radar	Range and radial velocity of detected returns	Multipath, angular ambiguity, and detection thresholds vary by configuration	Track association and coverage of crossing targets
Event camera	Asynchronous brightness-change events and motion cues	Static objects may yield few events; output bandwidth depends on event rate and encoding	Event-rate health and independent metric depth source



Figure 8.5: Williston collision damage: This NTSB presentation slide shows damage on the right side of the semitrailer and the roofless Tesla after the 2016 crash. The crossing-path geometry and absence of pre-impact braking come from the NTSB accident report, not from the photographs alone. Photographs credited on the NTSB slide to Florida Highway Patrol.



War Story 8.1: Silent perception omission (2016)

Context: A 2015 Tesla Model S traveling at about 74 mph with Traffic-Aware Cruise Control and Autosteer active struck a refrigerated semitrailer turning left across its path near Williston, Florida (National Transportation Safety Board 2017).

Mechanism: The crossing-path collision lay outside the front-to-rear crash modes for which the 2015 warning and automatic braking systems were designed. NHTSA reported that automatic braking required agreement between camera and radar; unusual vehicle shapes could impede camera classification (NHTSA ODI PE16-007).

Impact: The NTSB found no recorded indication that forward collision warning or automatic emergency braking detected an object, and no pre-impact braking occurred. The car passed under the trailer, which sheared off its roof (figure 8.5).

Response: The NTSB examined operational design domain limits and driver engagement as well as the collision-avoidance systems.

Systems lesson: An absent detection is not evidence that a crossing path is clear. A permission design must account for detection coverage and unsupported scenarios; a health flag cannot certify that every relevant object was perceived.

Whatever record the producer attaches to an observation can expose only the faults the producer can check. The detectable faults and the timing and calibration conditions fit in such a record; silent semantic errors, like the omission at Williston, still require separate evidence.

8.7 The Observation Contract

A downstream tracker or the permission path cannot reconstruct exposure timing, mounting calibration, or record integrity from a tensor alone. The perception producer therefore wraps each observation in a record, and the observation is unusable without a capture epoch, a clock identifier, a bounded conversion to the permission path's clock, and a calibration inside its validity range, whose revision the frame identifier carries; the consumer refuses a record that lacks any of them before reading the payload. The clock domain and the $\mathcal{H}_1$ age budget come from the handoff record of section 4.8, which this record consumes, and perception supplies the capture-side fields that let a consumer check an observation against them.



Definition 8.2: Observation contract

Observation contract is the formal cyber-physical specification governing sensory data structures traversing computational boundaries, binding a raw observation tensor to its exposure midpoint timestamp (t_{capture}), synchronization error bound (ϵ_{sync}), sensor coordinate frame identifier, positive-semidefinite spatial covariance matrix (Σ), and hardware health bitmask:

$$\mathcal{O} = \langle t_{\text{capture}}, \epsilon_{\text{sync}}, \text{frame_id}, \mathbf{T}_{\text{payload}}, \Sigma, \text{mask}_{\text{health}}, \text{crc32} \rangle$$

1. **Significance:** Lets a downstream state estimator or the permission path compute an upper age for each claim, select its valid transform, and bound its spatial error before the claim influences a motion proposal.
2. **Distinction:** Unlike a generic network message header or serialization schema (e.g., Protocol Buffers), an observation contract couples serialization integrity and build provenance with physical time ($t_{\text{MCU}} = at_{\text{capture}} + b$) and spatial transformation validity.
3. **Common pitfall:** Treating perception output as a pure geometric coordinate $[x, y, z]^T$ without propagating its spatial covariance or sensor health bitmask, blinding downstream safety filters to sensor occlusion or degraded lighting.

Table 8.4: Physical AI observation record schema: Compact fields needed to date and frame a claim, evaluate its uncertainty, and check the bytes and generating revision. The last column fills the fields this chapter computes for the mobile manipulator’s navigation camera; its dispatch age and clock-conversion bound are illustrative. A keyed authenticator is a separate requirement if the threat model includes malicious modification.

Record field	Meaning	Consumer check	Navigation camera
capture_timestamp_ns, capture_clock_id	Exposure midpoint in one shared, monotonic capture clock	Convert to safety microcontroller (MCU) time with a valid bounded-skew mapping before checking age	Exposure midpoint on clock cam_nav; dispatched 51.6 ms later (table 8.2)
sync_error_ns	Bound on capture-to-MCU clock conversion error	Add to worst-case age; refuse if missing or expired	0.2 ms, so the upper age at dispatch is 51.8 ms
sequence_index	Monotonic camera frame counter	Detect gaps and reordering	One count per frame at 60 Hz
sensor_frame_id	Transform-tree key and calibration revision	Select the valid extrinsic transform	nav_cam with its calibration revision
tensor_payload	Format, dimensions, length, and buffer	Validate bounds and record checksum before reading	
spatial_covariance	Positive-semidefinite spatial error scale	Use with separately calibrated bias and tail bounds	
calibration_metadata	Intrinsics, extrinsics, coefficients, temperature, validity range	Reject calibration outside its validated region	
provenance_digest	SHA-256 of firmware and model revisions	Match the approved build; this is not payload integrity	
payload_crc32	CRC over serialized header (excluding this field) and payload	Detect accidental record corruption; not authentication	
health_bitmask	Hardware and perception validity flags	Refuse claims with safety-critical faults	Synchronization-loss, link, transport, and saturation bits; input-support bits from section 7.6

The fields in table 8.4, the full set behind the definition’s tuple, date each observation and its generating revision. All cameras in this example latch exposure midpoints in a shared monotonic

capture domain; the timestamp is **not** nanoseconds since the UNIX epoch. A rolling-shutter claim must also identify the row or region whose acquisition time it represents, and `sequence_index` detects missing frames independently of clock conversion. If the MCU has a different clock, a synchronization service supplies a versioned conversion $t_{\text{MCU}} = at_{\text{capture}} + b$ (with t_{capture} the exposure midpoint t_0 of section 8.2), its validity interval, and a measured error bound ϵ_{sync} . Dispatch stamps the upper age estimate $t_{\text{MCU,dispatch}} - t_{\text{MCU,capture}} + \epsilon_{\text{sync}}$ on the MCU clock at the instant the record goes to the proposer. The proposal carries that age to the permission path, which refuses the observation if the conversion is invalid or that upper age exceeds its refusal threshold. The interval from dispatch to the permission tick, which holds the chunk inference and the link, is not evidence age; the lease of section 4.8.1 charges it.

For the navigation camera the threshold is the age the stopping budget charged plus the conversion bound, 51.6 ms plus 0.2 ms, or 51.8 ms, and a record leaving dispatch already stands at it, so any further delay, from batching frames across capture times or from host contention, is refused. The budget charges only the 51.6 ms age as distance; the bound's 0.3 mm of travel at the drive limit is carried inside the localization allowance δ_{loc} .

The transform key and calibration revision anchor metric coordinates.⁷ The firmware/model digest supports version provenance; `payload_crc32` detects accidental changes to the actual header and payload after serialization. Neither a checksum nor a build hash proves semantic truth, and neither authenticates an adversarial producer without a keyed check.

In the health bitmask, the lower bits report hardware faults such as synchronization loss or transport errors; the upper bits can report perceptual validity concerns such as occlusion or an input outside the support the evaluation tested, and those input-support bits implement the runtime monitor specified in section 7.6. A set safety-critical bit requires refusal of the affected claim.

The explicit record layout also supports fault injection. A test can alter the capture time, omit a sequence number, inflate reported covariance, or set an occlusion flag, then check whether the consumer rejects or degrades that claim within its budget. Such mutations verify response to the injected record faults; physical glare, miscalibration, and actuator behavior still require separate tests in chapter 15.



Checkpoint 8.2: Observation record checks

Before passing raw sensor observations into downstream state estimators, verify your understanding of observation record contracts:

- ☐ **Record triage:** Can you explain why a health bit, checksum, valid clock conversion, and freshness check must precede consumption of a spatial claim?
- ☐ **Timestamp corruption vs. dropped packets:** Can you describe why receiving a sensory frame with a desynchronized timestamp degrades state estimation far more severely than dropping the packet entirely?
- ☐ **Deterministic fault injection:** Can you distinguish between physically perturbing optical sensors versus deterministically mutating fields in the shared-memory observation record during automated hardware-in-the-loop verification?

A passed record check shows only that a claim is well formed and in date, and the recurring errors in perception systems begin when such a pass is read as evidence that the claim is true.

8.8 Fallacies and Pitfalls

Perception must describe the world within the strict temporal deadlines required for control actuation. Semantic confidence and high spatial accuracy cannot compensate for pipelines that deliver stale observations, fuse asynchronous exposures, or read average bandwidth as a bound on one frame's age.

Fallacy: *Accepting an observation because its semantic classification confidence is high, regardless of its age.* The mobile manipulator receives a high-confidence observation that the rack end ahead is clear, but a stall in the vision pipeline delivers it several frames late. A person who stepped out after the

⁷ | **Mechanical thermal expansion:** For a stereo baseline $b = 0.20$ m and an unmodeled change $|\delta b| = 0.1$ mm, first-order geometry gives $|\delta z|/z \approx |\delta b|/b$. At $z = 20$ m, the corresponding depth bias is about 10 mm, assuming focal length and disparity are otherwise unchanged. Temperature compensation must be checked against the actual mount and calibration.

Downstream: Observation contract mutations feed the hardware-in-the-loop verification harness in section 15.6.

exposure is absent from that frame. The confidence score describes the earlier image, not present clearance. Accepting the packet as current can therefore authorize motion toward an occupied rack end. The perception interface must check measurement age against its refusal threshold (section 8.7) and reject observations that exceed it. Any continued state propagation without new data must carry expanding kinematic uncertainty covariances that bound the reachable sets of dynamic obstacles, allowing the controller to enforce safety envelopes as historical evidence ages.

Pitfall: *Fusing multi-sensor spatial observations without sub-millisecond hardware clock synchronization.*

Two cameras on the mobile manipulator watch the same rack upright while the base drives at its 1.5 m/s drive limit, but their clocks differ by Δt_{skew} . Treating the two exposure times as equal places the upright $v \Delta t_{\text{skew}}$ apart in the two views, and track association can then split one physical object into two conflicting tracks (“ghosting”). Spatial calibration cannot correct an unknown temporal offset. The synchronization requirement must follow the tolerated position error and maximum relevant speed, expressed by $\Delta t_{\text{skew}} \leq \epsilon_{\text{pos}} / v_{\text{max}}$. Hardware synchronization at exposure time makes that bound enforceable; selecting a timing technology (like IEEE 1588 PTP) without first checking the spatial error budget leaves the fusion assumption unverified.

Pitfall: *Inferring a critical read’s deadline from average camera bandwidth.*

An acceptable average utilization says nothing about how long one read waits behind a synchronized capture and inference burst. On the mobile manipulator no permission-path read waits behind the burst, because the microcontroller reads from its own memory; the burst instead ages the frame or delays the chunk that the application processor is preparing (section 8.4), so the quantities to verify under the synchronized burst are the host’s capture-to-dispatch age and its renewal interval, not its average bandwidth.

8.9 Summary

A perception result is usable only when its capture time, clock conversion, coordinate frame, and error qualification fit the proposed motion. Age belongs to the measurement, not to the pipeline that carries it. It starts at the exposure midpoint, and on the mobile manipulator the navigation camera’s capture-to-dispatch age is a row of the stopping budget in its own right and the first interval of the pre-brake delay τ_{delay} , counted from the moment the person appears. An estimator that dates a frame by its arrival hides part of that age rather than removing it, and host memory contention can only lengthen it, so the age that binds a design is the one measured under the intended camera load. The permission path refuses any record whose upper age exceeds the budgeted age plus the clock-conversion bound, so delay beyond the budget costs availability, never distance.

Spatial error is spent from the same clearance. Stereo depth error grows with squared range under fixed disparity noise, an extrinsic shift adds a lever-arm error that grows with distance, and neither a covariance nor a learned depth score supplies a 99th-percentile clearance bound by itself. A clearance inset rests on validated residuals, checked bias, and a stated tail risk or hard bound.

The failures that matter most are the silent ones, from a misdated frame to a saturated image or a hallucinated clear path, because none of them raises a fault flag. The observation contract lets the consumer refuse whatever it can check before reading the payload (capture epoch, clock conversion, frame, calibration, integrity, and health), and semantic truth still requires independent evidence.



Key Takeaways: Spatial perception and age of information

- **Observation Age Begins at Capture:** Hardware-latched exposure time, row timing when relevant, and bounded clock conversion let the consumer compute an upper age before admitting motion.
- **A Wrong Timestamp Is Worse Than a Lost Frame:** A dropped frame widens the estimator’s uncertainty; a plausible but misdated frame contracts it around the wrong pose. Clock bounds, sequence counters, and age checks must precede any use of the payload.

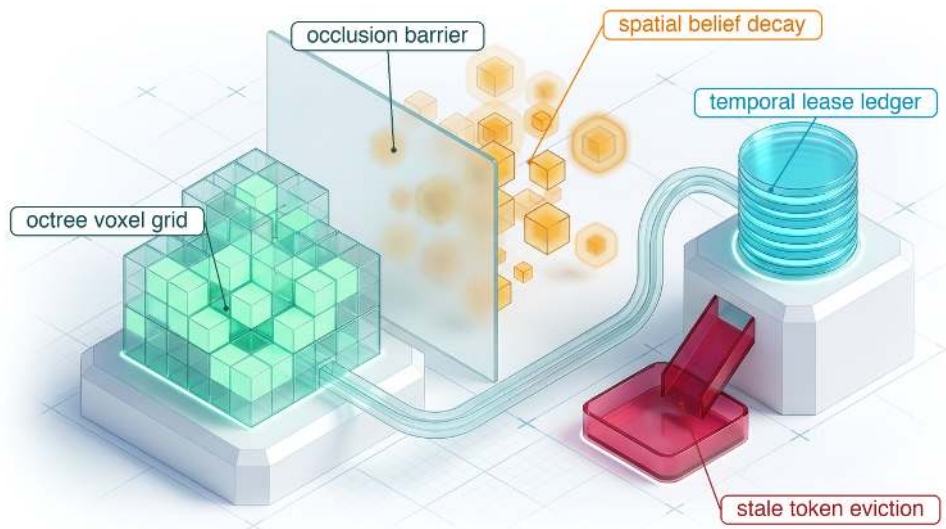
- **A Model Score Is Not Geometric Uncertainty:** A classification or depth-bin score does not supply a calibrated spatial tail. Metric covariance, bias, and residual validation are separate inputs to a clearance decision.
- **Calibration Can Change:** Shock or temperature can alter the sensor-to-body extrinsic transform. The chapter's 0.005 rad extrinsic shift at 10 m projects to 50 mm laterally, as large as the whole δ_{loc} allowance. A shift of that size is a failure to refuse, not an error to budget, so every calibration carries a validity interval that the consumer checks.
- **Health Flags Cannot Certify Semantics:** A structurally valid record from a healthy sensor can still describe a clear path that is not there. A second modality narrows that gap only when its coverage and failure dependence have been checked.
- **Perception Provides Dated Snapshots:** Persistent spatial memory propagates state and uncertainty between observations and through occlusion; the permission path still checks whether that propagated claim fits the motion margin.

Perception is where the part's principle, proposal is not permission, meets its freshness clause in measurable form. The proposal's evidence epoch is only as good as the capture epoch and bounded clock conversion the observation contract declares. Sensors record physical energy interactions, not geometric truth, and a claim that cannot show when and where it was measured cannot support motion.

What's Next: From sensory photons to spatial memory

Perception delivers dated observations, but the person at the rack end does not stay in view between them. Chapter 9 carries spatial estimates and their uncertainty across those gaps and sets the expiry that lets the permission path refuse a claim once occlusion or age has exhausted the validated motion margin.

Spatial Memory



- 9.1 Preserving Spatial Belief
- 9.2 What Memory Hands Over
- 9.3 A State Has an Epoch
- 9.4 How a Transform Goes Stale
- 9.5 What Scene Memory Can Certify
- 9.6 Belief Through Occlusion
- 9.7 When the Estimator Cannot Know
- 9.8 How Belief Goes Wrong
- 9.9 The Spatial State Schema
- 9.10 Fallacies and Pitfalls
- 9.11 Summary

Purpose

How fast does a spatial belief turn into dangerous fiction once a sensor loses sight of an object?

A dynamic obstacle remains a critical physical constraint long after it moves behind an occluding barrier. A machine must remember the obstacle's presence, yet its recorded state rapidly degrades in reliability as time elapses. Treating a stale observation as ground truth risks planning trajectories through occupied physical space, while instantly discarding the record erases genuine hazards. Spatial memory must therefore track both a state estimate and an explicit model of growing uncertainty. Sensor capture age, odometry drift, and coordinate frame calibration dictate whether a remembered entity remains valid to authorize motion.

Software failures such as frozen frame transforms or silent tracking loss can deceptively present stale estimates as fresh data. A robust memory system must enforce bounded belief expiration, withdrawing proposal authority before uncertainty expands beyond available clearance envelopes without mistakenly assuming unobserved regions are free space. In the physical AI architecture, memory populates the Brain's world model above the proposal boundary, while the deterministic permission path below verifies that every trajectory relies strictly on unexpired, certified spatial bounds.

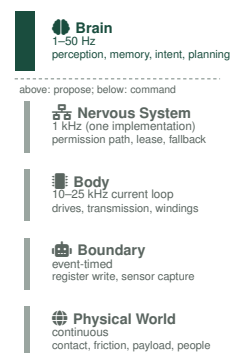


Figure 9.1.: The five-level machine stack with the Brain level highlighted.



Learning Objectives

- Specify belief records that keep evidence age separate from estimate age, with frame, uncertainty, and validity region
- Set belief expiry from bounded uncertainty growth and the task's physical tolerance
- Compare spatial representations by what each can certify about clearance
- Design spatial updates that preserve evidence age and distinguish missing observations from verified free space
- Diagnose frozen transforms, phantom tracks, and unobservable drift that pass freshness and innovation checks

9.1 Preserving Spatial Belief

A memory register can preserve a record exactly while the world it describes moves. As the warehouse mobile manipulator closes on the end of a storage rack, the rack hides the cross aisle from the navigation camera, and whatever the camera last saw there now survives only in memory. A person out of sight past the rack end can walk into the space that record calls empty, and republishing the record with a new software timestamp cannot restore the lost physical evidence. Spatial memory must therefore carry the last measurement time and an error-growth rule into the motion-permission decision.

A physical machine cannot rely solely on raw sensory observations or static memory registers. It requires a **belief**, an estimate of physical state carried forward from historical sensory evidence, coupled explicitly to an expanding model of uncertainty.¹ Perception supplies a dated, calibrated observation (as established in section 8.2). Spatial memory must determine how long that observation, combined with dynamic motion models, can continue to authorize physical action before uncertainty consumes the machine's remaining clearance.

As the second of the four proposal-side stages of Part III, between perception and intent, **spatial memory** carries geometric and physical state estimates forward across sensor dropouts, occlusions, and variable processing delays. It sits above the proposal boundary (section 1.5), so a remembered state can shape a proposal but cannot certify its own freshness; the permission path checks the age of the evidence against its own clock.

In software systems, data age is frequently measured from the instant a message was serialized, published over an inter-process bus, or updated in a shared memory table. In a physical AI system, this convention is dangerous. Propagating a state estimate through an extended Kalman filter at 1 kHz or republishing an $SE(3)$ kinematic transform at 200 Hz creates a stream of fresh software timestamps, but it does not refresh the underlying physical information. The true age of a belief is determined strictly by the evidence epoch t_0 of the last physical measurement that constrained it. A downstream tracking loop that reads a position estimate generated by dead reckoning or policy rollouts may observe a publish timestamp only 2 ms old, yet the physical error bound continues to widen at the velocity of the unobserved drift. Timestamping an inference output or a cache lookup records when compute occurred, but it tells the receiver nothing about when reality was last checked.

Memory integrity alone cannot establish geometric validity (Durrant-Whyte and Bailey 2006). To prevent a machine from acting on stale data, memory systems must treat expiry as an explicit interface event. Each belief carries a deadline, its invalidation horizon (derived in section 9.6), at which its declared error bound reaches the task's physical margin. At that deadline the belief stops supporting motion, and the permission path refuses any proposal that still depends on it and selects a feasible fallback.

A belief that must declare its own expiry has to travel with more than its value, so the first handoff question is what a consumer needs to evaluate a stored value after its sensor loses sight of the object.

9.2 What Memory Hands Over

An unannotated vector such as $[0.142, -0.051, 0.812]$ arriving over a real-time bus conveys nothing about what physical entity is being described, what physical dimension is represented, or how that

Prerequisite: Observation age and calibrated spatial covariances are generated in section 8.7.

¹ | **Belief state formulation:** In a Partially Observable Markov Decision Process (POMDP), a belief represents a probability distribution over latent states. A physical state interface additionally carries an error bound. Without the value, frame, evidence epoch, and applicable uncertainty model, a downstream optimizer cannot evaluate clearance during a sensor dropout.

quantity is scaled. A belief handed to a consumer therefore cannot be a naked array of floating-point numbers. It is a decaying interpretation of physical inputs and must carry its epoch, physical uncertainty, and reference frame to remain usable. A downstream controller receiving raw coordinates without metadata must rely on implicit conventions established at compile time. If an estimator produces positions in meters while an actuator controller expects millimeters, or if an angular rate is mistaken for an absolute orientation, the resulting command registers as a thousandfold tracking error, driving motor current to the actuator thermal limits detailed in section 2.5. To prevent misinterpretation across the causal boundary established in section 1.2, a belief record must first specify the entity identity, the physical quantity, and the measurement units. The consumer must be able to verify that the value represents the center of mass of a specific workpiece in meters, or the normal contact force on a fingertip in newtons, before routing that number into a control law.

Specifying the quantity and units remains incomplete without binding the value to a concrete reference frame. Identical spatial coordinates describe entirely different physical realities depending on the origin and orientation of the coordinate system. A Cartesian position $[0.142, -0.051, 0.812]$ m designates a point suspended in free space when evaluated in the robot base frame, but represents a point several hundred millimeters inside a rigid fixture when interpreted in the tool flange frame. Because kinematic links flex, joints move, and coordinate transforms update at varying rates, a belief record must carry an explicit frame identifier. Carrying the frame allows the receiving node to look up the corresponding spatial transformation tree and project the stored geometry into the local frame required by the actuator, preventing the controller from acting on geometric misalignments caused by moving kinematic chains.

Geometry and identity describe the physical state at a single moment, but memory must also capture the temporal anchor of that state. The record carries the evidence epoch t_0 of the physical interaction or optical exposure that last constrained the estimate. In distributed architectures, intermediate nodes often republish data, execute Kalman filter prediction steps, or run forward kinematic propagations to maintain high-rate control streams, and none of these adds physical evidence. The field must therefore pass unchanged through every republish, filter, and prediction stage, so that consumers measure the age of the information against the physical world rather than against the scheduling frequency of the local computer.

Generating this anchored belief relies on **multi-sensor state estimation**—whether implemented through recursive Extended Kalman Filters (EKF), Unscented Kalman Filters (UKF) (Kalman 1960), or nonlinear factor-graph optimization (Smoothing and Mapping, SAM)—an established mathematical discipline whose operational role is to map asynchronous, heterogeneous sensor observations (such as high-rate IMU accelerations, wheel encoder ticks, and low-rate vision claims) into a minimal sufficient statistic of the plant’s latent physical state. The physical AI architecture abstracts the internal numerical mechanics of this fusion pipeline behind the spatial belief contract. Downstream trajectory planning and safety enforcement do not recompute sensor innovation covariances or re-solve factor graphs; they require only that the state estimator provide an unbiased state vector $\hat{x}$, a physically honest uncertainty bound σ , and the true evidence epoch t_0 of the measurements that constrained it. The foundational systems requirement is not the choice of filtering algorithm, but the obligation that the estimator model process noise expansion honestly across blind intervals (section B.6 derives continuous error covariance propagation) and revoke its validity status whenever physical observability rank collapses (section 9.7).

Knowing the evidence epoch t_0 establishes elapsed time $\Delta t = t - t_0$, but elapsed time alone cannot determine whether a belief is valid for a given action. Two estimates of identical age diverge at different rates depending on physical constraints. The rack end itself is bolted to the floor, so its remembered position stays within the machine’s localization error however long the camera is blind, whereas a person hidden behind it can move at walking speed and spend the same clearance in a few tens of milliseconds.

The minimum record fields necessary for a consumer to evaluate a stored state can be derived directly from the requirement to bound current error. Let $E(\Delta t)$ represent the upper bound of the discrepancy between the stored value and true physical state after an elapsed time $\Delta t = t - t_0$. If an estimator establishes a state at time t_0 with an initial uncertainty bound E_0 , and external disturbances or unmodeled velocities cause the true state to diverge at a maximum rate r , the upper bound on error at current time t obeys the linear expansion $E(\Delta t) = E_0 + r \Delta t$. At the rack end,

E_0 is the machine's localization error and r is the walking speed of the person the rack may hide; section 9.6 evaluates that law. Omitting either term leaves the consumer unable to compute $E(\Delta t)$, so an initial bound and an expansion rate are both structurally required fields of the record.

Expressing uncertainty through physical bounds and expansion rates distinguishes a physical AI belief from a probabilistic confidence score. An object detection network might output a class confidence of 0.94, but that softmax score is not calibrated evidence (section 3.5), and a downstream position controller cannot convert a unitless probability into a safe actuator displacement margin (i.e., how far the arm can physically move before risking a collision). A confidence score of 0.94 provides no information about whether an obstacle boundary is uncertain by 2.0 mm or by 200 mm, nor does it indicate how rapidly that boundary moves when occlusion occurs. A physical memory record must express its initial bound E_0 and expansion rate r in concrete physical dimensions such as meters, radians, or newtons per second. Furthermore, these rates are valid only within a declared validity region, such as a maximum assumed acceleration or a verified contact friction coefficient. If physical operating conditions leave that region, the expansion rate no longer holds, and the belief record must be treated as invalid.

When memory hands over a tuple containing value, reference frame, evidence epoch t_0 , initial bound E_0 , and growth rate r , the consumer executes a deterministic verification step before applying the data to physical control. The consumer possesses a task tolerance $E_{\max}$ dictated by the physical task. When the arm guides a gripper finger into the latch-release slot of the cage door, for instance, the slot's clearance sets $E_{\max}$, and positional uncertainty must stay below it. Upon reading the belief record at current time t_{now} , the consumer calculates the elapsed time $\Delta t = t_{\text{now}} - t_0$, evaluates the present error bound $E(\Delta t) = E_0 + r\Delta t$, and compares the result directly against $E_{\max}$. If $E(\Delta t) \leq E_{\max}$, the belief passes this expiry check and can enter the independent motion-permission check. Otherwise the dependent proposal is refused; the controller selects a validated hold, search, or stop response only when it remains feasible.



Definition 9.1: Spatial belief record

Spatial belief record is a typed record that binds an identified physical quantity and SI unit to a frame, evidence and estimate epochs, clock conversion, initial error, growth law, validity region, expiry, provenance, and lifecycle status. These fields let a consumer evaluate freshness before a separate motion-permission check.

1. **Significance:** A state without an error bound and growth rate cannot be safely evaluated by downstream controllers. As time elapses ($\Delta t = t_{\text{now}} - t_0$), unobserved motion or sensor drift widens the spatial error bound ($E(\Delta t) = E_0 + r\Delta t$). When that bound exceeds the task tolerance ($E(\Delta t) > E_{\max}$), the record expires, requiring the permission path to reject dependent proposals and choose a feasible validated response.
2. **Distinction:** Unlike a unitless classification probability (such as $P(\text{object}) = 0.94$), a spatial belief record provides physically grounded metric units (meters, radians) and explicit time-decay mechanics.
3. **Common pitfall:** Relying on external publishers to broadcast invalidation signals upon occlusion. If the communication channel drops or the sensor driver hangs, the downstream consumer continues executing open-loop motions on frozen beliefs unless it independently evaluates the record's expiry.

Evaluating the spatial belief record depends on calculating the elapsed time $\Delta t = t_{\text{now}} - t_0$ against physical reality. If the consumer cannot verify the exact physical instant when energy transduction occurred, the calculated error bound $E(\Delta t)$ becomes an unanchored fiction, mistaking software compute cycles for physical evidence. The estimate epoch and clock conversion named in the definition exist to protect that anchor, while provenance and lifecycle status, the record's remaining fields, are developed with the schema in section 9.9.

9.3 A State Has an Epoch

A pose that leaves an object detector describes where a workpiece was when the camera exposed the frame, not where it is when the message arrives. Every state therefore refers to a specific instant, its **epoch**, and a state crossing a physical pipeline carries three of them before a controller can act. The **acquisition epoch** (t_0) is the capture epoch of the observation contract (section 8.7), the instant of energy transduction at the causal boundary, such as photons striking a photodiode array or a digital register latching an encoder count, and the belief record carries it as its evidence epoch.² Memory adds the two epochs that follow it. The **estimate epoch** (t_{est}) marks the instant a state estimator or neural network completed its inference over those measurements and resolved them into a concrete state claim. The **publication epoch** (t_{pub}) marks the instant the serialized message was committed to the transport bus for dispatch to downstream consumers.

Collapsing these three timestamps into a single receipt or transmission timestamp destroys the physical validity of the state. Tagging a pose with its publication time hides every millisecond of exposure, readout, inference, and scheduling that preceded it, while gravity, momentum, and contact forces keep acting on the physical system without pause. That hidden age is displacement at speed, as section 8.1 established; for the navigation camera at the base's drive limit, the age at dispatch alone amounts to several centimeters of travel, all of it invisible to a consumer that reads only the publication stamp. A state that records only when data became available tells the consumer nothing about the physical instant the measurement represents, making temporal alignment impossible.

Even when a system records true acquisition epochs, physical AI architectures rely on distributed processing nodes whose local oscillator clocks drift independently. Synchronizing these distributed clocks over standard bus networks introduces a bounded synchronization error, and that temporal uncertainty becomes spatial uncertainty at the relative speed of whatever the record describes.³ The observation record of section 8.7 already carries this bound as its clock-conversion error, and a belief built from that observation keeps it. Every consumer adds the bound to the evidence age before judging freshness, and the error budget allocates the distance that bound covers at the record's relative speed alongside sensor resolution and structural deflection.

Real physical systems never sample all relevant state variables simultaneously. In an illustrative cycle, a tactile array might latch normal force readings at $t_1 = 12$ ms, an optical tracking camera might finish sensor integration at $t_2 = 18.5$ ms, and joint optical encoders might sample angular positions at $t_3 = 21$ ms. These observations occur at different epochs and cannot be combined through direct algebraic operations. Fusing a camera observation at t_2 with an encoder observation at t_3 requires the consumer to project one state across the temporal gap $\Delta t = |t_3 - t_2| = 2.5$ ms using explicit kinematic models and bounded velocities (for example, propagating the historical camera observation along the joint velocity vector to reconcile it with the instantaneous encoder reading). Without the true acquisition epoch for each individual signal, the alignment calculation cannot determine the sign or magnitude of Δt . Extrapolating state without an accurate epoch causes the fusion algorithm to compute spatial offsets from fictitious temporal baselines, compounding errors across every fusion cycle.

With these epochs and the clock-synchronization bound carried in the record of section 9.2, the consumer computes the elapsed duration between the evidence epoch and the execution deadline, evaluates the worst-case expansion of the error bound, and determines whether the state remains within task tolerances. Handing over a state without its acquisition epoch leaves the consumer blind to transport delays and scheduling jitter, turning deterministic control into open-loop guesswork. Because physical geometry also changes as bodies move through space, spatial transforms between reference frames are subject to the same temporal decay, carrying their own distinct evidence epochs.

9.4 How a Transform Goes Stale

A frame binds a coordinate, but in a machine the frame itself moves. Section 8.2 gave every observation a named frame and related the sensor, body, odometry, and map frames through a transform tree. What the tree does not show is how a transform goes stale. Each edge of the tree in figure 9.2 stores a rigid-body pose in $SE(3)$, parameterized by joint angles (Lynch and Park 2017; Craig 2005), and that pose describes the machine only at the epoch when its geometry was measured. When an onboard computer chains transforms measured at different times, through transport delay, asynchronous

² | **Hardware timer input-capture:** Microcontroller timer input-capture channels latch counter values into dedicated shadow registers upon external signal transitions, bypassing operating system interrupt latency. Reading capture registers without atomic synchronization against timer overflow flags causes 16-bit counter aliasing, injecting artificial timestamp jumps equal to the rollover period (65.5 ms on a 1 MHz timer). In downstream visual-inertial odometry, this temporal discontinuity introduces false acceleration spikes that destabilize platform attitude filters.

Prerequisite: Synchronized hardware timestamping across fieldbuses is established in section 4.4.

³ | **Fieldbus clock synchronization jitter:** Fieldbus architectures exhibit distinct hardware clock synchronization bounds: IEEE 1588 PTP over Ethernet achieves $\delta t < 1.0 \mu\text{s}$, EtherCAT Distributed Clocks achieve < 100 ns hardware SYNC0 jitter, and CAN-FD bit-phase synchronization yields ± 10 – $50 \mu\text{s}$. Because spatial uncertainty scales with target relative velocity ($e_{\text{sync}} = v_{\text{rel}} \delta t$), microsecond-level synchronization is essential to keep temporal errors within micrometer mechanical budgets. Uncalibrated clock drift between distributed sensor and motor nodes degrades multi-axis tool coordination, inducing high-frequency chatter in joint torque loops.

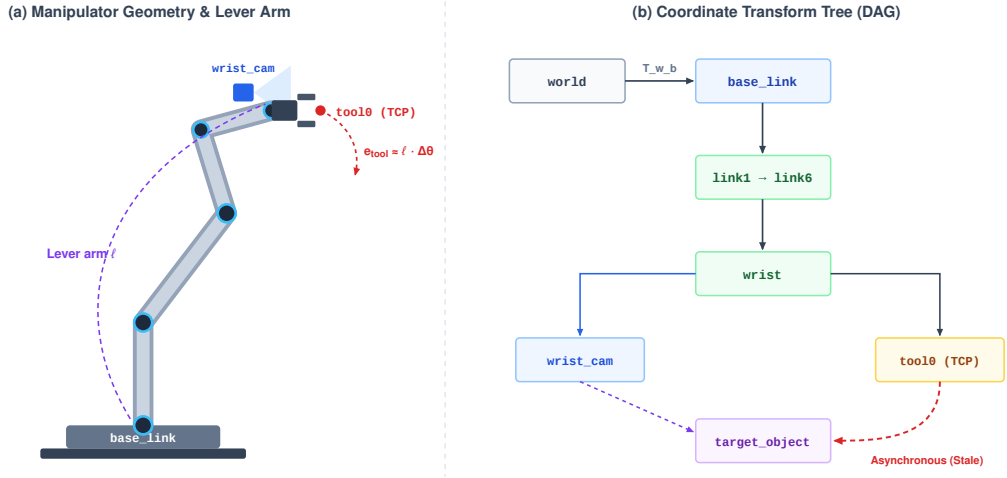


Figure 9.2: Manipulator kinematic frame tree and temporal dynamics: Articulated manipulator geometry (a) and corresponding directed acyclic coordinate transform tree in ROS tf2 (b). Each directed edge stores a homogeneous transformation matrix parameterized by joint state. Composing transforms across disparate acquisition epochs injects lever-arm positioning error $e_{\text{tool}} \approx \ell \Delta\theta$, while an uncompensated asynchronous edge references stale coordinates (adapted from Craig 2005 and Foote 2013 (Craig 2005; Foote 2013)).

sampling, or dropped frames, the product is a hybrid pose the machine never occupied (section B.4.1 derives the kinematic chain).

A transform also spans several machine words, so a reader racing a writer can see an old rotation paired with a new translation. The seqlock mailbox of section 4.3 closes that race without blocking the control thread, because each reader makes one bounded attempt per tick and never waits; a torn read is discarded and the last validated transform ages in its place.

The cost of an expired transform is direct spatial displacement (figure 9.3). Applying the age-as-displacement relation of section 8.1 to kinematics, uncompensated latency translates directly into geometric positioning error through the lever arm of the linkages. For the rigorous cross-product derivation of this kinematic error propagation, see section B.4.3.

How little staleness it takes to spend a task’s tolerance shows at the latch-release slot, while the wrist camera can still see it. Each wrist-camera frame, one every 16.7 ms, renews the slot’s pose in the camera frame and with it the slot-to-finger edge the controller aligns on, and the finger must stay within the slot tolerance E_{max} of 1 mm. Suppose that edge misses a single frame. Starting from an initial error of 0.1 mm, the finger’s lateral offset drifts at most $e_{\text{trans}} = v_{\text{drift}} \Delta t$ under an illustrative 12 mm/s bound on its unobserved lateral velocity, validated for the arm’s operating region, or 0.20 mm. With the fingertip an illustrative $\ell = 0.15$ m beyond the wrist and the wrist turning at an illustrative $\omega = 0.30$ rad/s as it aligns the finger, the lever arm adds $e_{\text{rot}} \approx \ell \omega \Delta t$, or 0.75 mm. The joint encoders measure that turn, but the stale edge still dates the slot to the previous frame while the wrist pose it is composed with is current, so the tree returns a hybrid pose, a finger position the arm never held. Only chaining the slot against the wrist pose of its own capture epoch, or expiring the edge, removes that arc. Left in the chain, the stale edge misplaces the finger by 1.05 mm, 5 percent more than the slot allows, and more of that error comes from the lever arm than from the drift. Because the transform tree evaluates without error, nothing flags the stale pose, and the controller keeps aligning the finger on a slot position that is off by more than the clearance, which on an unfavorable approach puts the finger against the strike plate beside the slot.

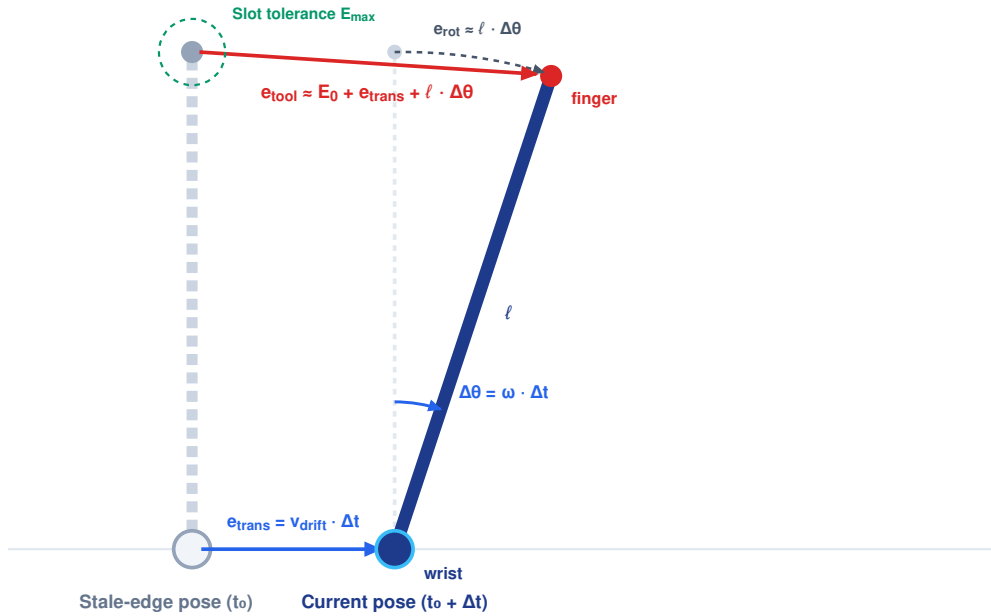


Figure 9.3: Transform staleness and lever-arm error at the latch-release slot: Finger error when the slot-to-finger edge misses one wrist-camera frame of duration Δt . Lateral drift at the velocity bound v_{drift} moves the finger by $e_{\text{trans}} = v_{\text{drift}} \Delta t$, while the wrist turning at ω swings the fingertip, a lever arm ℓ beyond it, by $e_{\text{rot}} \approx \ell \omega \Delta t$. Composing the stale edge with the current wrist pose adds both to the initial error E_0 , so the finger error $e_{\text{tool}} \approx E_0 + e_{\text{trans}} + \ell \Delta\theta$ exceeds the slot tolerance E_{max} .

Systems Perspective 9.1: The leverage of staleness

A transform's admissible age is set at the point it serves, not at the joint it measures. The tool point drifts at roughly $v + \ell\omega$, so the longer the lever arm, the sooner an edge expires, and an edge still fresh enough to locate the wrist can be too old to locate the tool center point.

A transform tree dates the machine's own geometry, and a stale edge misplaces the tool by the motion it missed, amplified by the lever arm. The world around the machine ages in the same way, but it has no joint encoders to report its motion, so memory must also hold the surrounding surfaces and the space between them, and it must record which of that space a sensor has actually seen.

9.5 What Scene Memory Can Certify

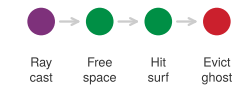
A robot that has swept its range sensor down an aisle holds three kinds of space afterward: space it saw occupied, space it saw empty, and space it never saw. Only the second can ever vouch that a path is clear, and only while its evidence is fresh. Every scene representation stores some version of these three states, and they differ in what they store and what it costs. An **occupancy grid** (Elfes 1989), or its sparse octree form (Hornung et al. 2013), keeps an occupancy estimate per cell. A **truncated signed distance field (TSDF)** (Curless and Levoy 1996) keeps a truncated projective distance to the observed surface along each sensing ray. **3D Gaussian splatting** (Kerbl et al. 2023) keeps optimized primitives that render new camera views. A dense grid's storage grows with the cube of its cells per dimension, while the octree, the surface-band blocks of a TSDF, and a Gaussian map store only allocated structure, whose size depends on the observed geometry and the accuracy target; section A.3.6 and section A.3.5 give the updates behind these costs. For motion permission they share one limit. A projective distance is not the nearest-surface Euclidean distance a clearance decision needs, and a Gaussian density does not identify occupied space, so none of them certifies clearance by itself. Each must first be converted into a validated occupancy layer or Euclidean signed

distance field (ESDF), with an explicit policy for unknown space, before the permission path can query it.

The choice of metric representation defines an empirical Pareto trade-off between volumetric memory footprint and spatial collision query latency (figure ??). Continuous trajectory optimization and real-time reflex controllers running in 1 kHz control loops (1,000 μ s periods) require spatial clearance and gradient queries to execute in under 10 μ s across dozens of robot body articulation points. While implicit neural representations such as coordinate MLPs (NeRF, iMAP) achieve minimal memory density (~ 0.08 MB/m³), querying signed distance or occupancy requires evaluating deep neural network forward passes, imposing massive computational latency (> 650 μ s per query point) that confines them to deliberative offline planners. Hierarchical octrees (OctoMap (Hornung et al. 2013)) compress unobserved free space down to 0.18–1.25 MB/m³, but pointer indirection during recursive tree traversals incurs CPU cache misses that limit query speeds to 24–38 μ s. Conversely, dynamically hashed truncated signed distance fields (Voxblox TSDF (Oleynikova et al. 2017)) allocate memory only around observed surface bands (4.1 MB/m³), enabling 1.8 μ s ray queries, while propagating Euclidean signed distance fields (ESDF) increases memory overhead to 28.5 MB/m³ to deliver sub-microsecond (0.85 μ s) distance evaluations with analytic continuous gradients. Meanwhile, 3D Gaussian splatting primitives (Kerbl et al. 2023) occupy a middle regime (1.6–5.2 MB/m³, 5.8–8.2 μ s), enabling photographic radiance rendering alongside bounding-volume-hierarchy accelerated collision checks.

What a map can certify comes from how a range return updates it. Traversing the ray from the sensor to the return (Amanatides and Woo 1987), the map asserts **negative evidence** in every cell the ray crossed and positive evidence in the cell where it ended; section A.3.6 gives the log-odds update. Negative evidence lets the map retire a phantom obstacle as soon as the same region is seen empty, instead of waiting for a timeout, and it also marks the limit of what the map can say. It covers only cells a valid ray actually traversed. It says nothing about the volume behind the return, and a new scan down the aisle cannot renew a cell hidden behind the rack, so the occluded cross aisle stays unknown and still needs expiry. Every return also touches every cell along its ray, so sparse allocation saves capacity but not traversal, and that traffic reaches the permission path only as added evidence age (section 13.3).

Because negative evidence renews only the cells a ray crossed, a map’s publication time dates none of its cells, so the manifest in table 9.1 carries evidence epochs per cell. Those epochs are not free, because every allocated cell stores its own timestamps beside its occupancy estimate, and they belong in the map’s memory budget for the same reason the value does. A motion consumer checks the evidence age and state of every cell its path uses, including cells marked unknown, and derives conservative clearance from a separately validated layer.



Active negative evidence along sightlines clears phantom obstacles without waiting for timeout decay.

Table 9.1: Volumetric belief manifest and cell evidence: The map header describes layout and update time, while cell-local acquisition epochs determine whether the particular path region is fresh. The consumer treats unobserved or expired cells as unknown unless a separately validated policy permits traversal.

Map field	Unit or type	Meaning and consumer check
voxel_grid_origin, voxel_resolution_m	m; m	Spatial anchor and pitch; match the active frame and calibration
map_update_epoch	capture-clock ns	Last map publication/update; not freshness of every cell
cell_evidence_epoch	capture-clock ns per cell	Last physical observation for that cell; never refreshed by prediction or unrelated map updates
free_space_sweep_epoch	capture-clock ns per occupancy cell	Last valid observed free ray through that cell
clock_conversion, sync_error_ns	mapping; ns	Convert capture epochs to the consumer’s monotonic clock with bounded error

Semantic memory sits above the metric layer and inherits its limits. Beyond raw metric volumes and radiance fields, foundation-model-driven physical AI requires spatial memory to organize space into semantic hierarchies. The **3D dynamic scene graph (DSG)**, formulated by Rosinol et al. (2021)

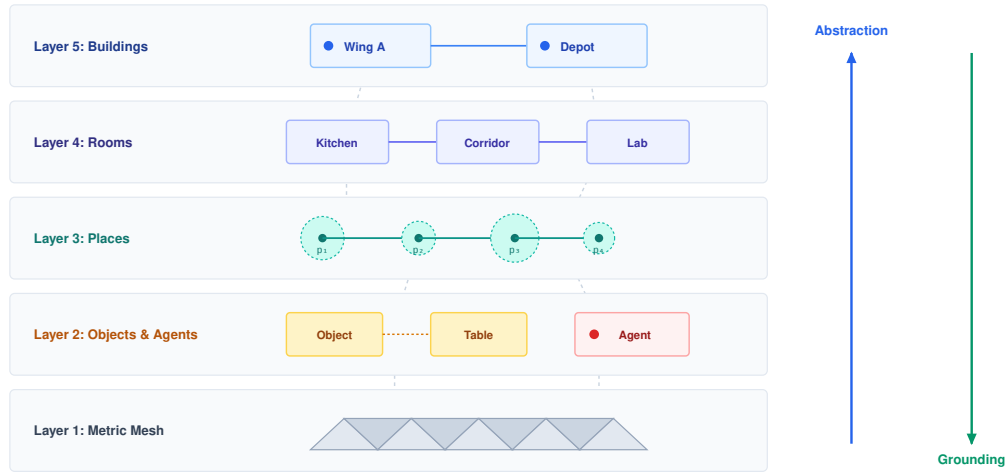


Figure 9.4: Hierarchical 3D dynamic scene graph architecture: Five-layer spatio-temporal memory organization bridging metric SLAM to symbolic foundation reasoning. Layer 1 captures triangular metric mesh geometry; Layer 2 extracts 3D objects and dynamic agents; Layer 3 models topological places via Voronoi freespace clearance spheres (p_i); Layer 4 structures architectural rooms and portals; and Layer 5 establishes global building mission topology. Vertical inter-layer edges coordinate upward abstraction (compressing metric surfaces into symbolic entities) and downward grounding (translating task tokens into metric coordinates) (adapted from Rosinol et al. (Rosinol et al. 2021) and Hughes et al. (Hughes et al. 2022)).

in Kimera and extended by Hughes et al. (2022) in Hydra, bridges metric SLAM with symbolic reasoning by structuring spatial memory into a five-layer hierarchical graph (figure 9.4):

1. *Metric-Semantic Mesh (Layer 1)*: High-resolution triangular mesh capturing fine surface geometry with vertex semantic labels.
2. *Object and Agent Nodes (Layer 2)*: Bounded 3D geometric entities with oriented bounding boxes, visual embeddings, and tracked dynamic trajectories.
3. *Places and Freespace Topology (Layer 3)*: 3D Voronoi topology and navigable freespace clearance spheres (p_i) for planning collision-free paths.
4. *Rooms and Structural Enclosures (Layer 4)*: Functional architectural subdivisions partitioned by semantic walls and portals.
5. *Buildings and Global Topology (Layer 5)*: Top-level semantic graph governing long-horizon mission allocation.

The computational advantage of this bidirectional hierarchy lies in how it resolves the memory-compute tension on edge SoCs. Upward abstraction collapses gigabytes of dense triangular surface meshes into kilobyte-scale symbolic subgraphs, allowing foundation deliberators to evaluate long-horizon task plans without exhausting LPDDR5 bandwidth or choking KV caches with millions of raw geometric tokens. Conversely, downward grounding lets high-level spatial targets reach local geometric evidence. A deliberator queries space without searching every cell, but each node is only as fresh as the metric cells beneath it, and grounding a symbolic target still ends in a clearance query against the validated metric layer.

A learned policy’s context is memory of a different kind. An autoregressive policy holds its recent observations in a key-value cache whose growth section 3.4 derives, and a **streaming window** bounds that cache by evicting its oldest tokens (Xiao et al. 2024), whether or not their physical consequences have ended. A person seen stepping behind the rack before the window began is gone from the context but not from the aisle. A physical fact the machine must still honor therefore lives in a belief record with its own evidence epoch, never only in the policy’s context.

For motion permission, then, a representation without an epoch is an unverified assertion, and an estimate whose error bound exceeds task tolerance must be excluded from motion proposals before the permission path considers actuator commands. The machine cannot halt every operation at the

first missed frame, however. Extending action across brief sensing gaps requires estimators that propagate belief forward until new measurements arrive to confirm or correct the state.

9.6 Belief Through Occlusion

When direct sensing fails, a physical AI system uses an internal transition model to update its state estimate. A Kalman filter updates state estimates using plant dynamics,⁴ while a learned policy might predict the next relative pose using an autoregressive network. Either way, prediction extrapolates old evidence forward without renewing it, as section 9.1 established; it preserves the utility of an observation by compensating for expected physical motion, but it does not reset the age of the underlying measurement.

The difference between predicting a state and observing it decides what the machine may do at the rack end. A person can step out there into the illustrative 1.10 m clear distance D_{clear} that the stopping budget defends, and once the rack hides the cross aisle, the question is how long the remembered view may vouch that the cross aisle is still empty.

Every physical model simplifies reality, causing error between the projected state and the physical world to expand continuously during the blind interval. In state estimation, this dispersion is governed by **occlusion covariance expansion**, the continuous, physics-governed widening of uncertainty during sensor line-of-sight dropouts. Mathematically formalized by stochastic models (such as the continuous Lyapunov/Riccati covariance propagation equation, detailed in section B.6), this expansion quantifies the growing doubt about where an object actually is, bounding the unobserved divergence caused by unknown drift velocities and unexpected physical disturbances before the sensor regains visibility. For a deterministic worked model, suppose sensor characterization and disturbance limits establish an initial position-error bound E_0 at the moment of occlusion ($\Delta t = 0$). Two bounded terms then degrade the prediction. Motion the model does not know about introduces a drift velocity bounded by v_{drift} , and forces it does not model create a disturbance acceleration bounded by a_{dist} . Over an occlusion duration Δt , the error bound $E(\Delta t)$ then follows the quadratic expansion:

$$E(\Delta t) = E_0 + v_{\text{drift}}\Delta t + \frac{1}{2}a_{\text{dist}}(\Delta t)^2 \quad (9.1)$$

At the rack end the terms are the machine's own. The initial error is the localization bound δ_{loc} , 50 mm, because the machine knows where the rack end lies only to within that bound. The tolerance is the protective clearance δ_{margin} , 100 mm, which the budget reserves between the stopped machine and a person. A person hidden behind the rack can close on that clearance at up to the walking-speed bound v_h of 1.6 m/s that safety-distance calculations assume for an approaching person. Because v_h bounds speed directly, equation 9.1 keeps only its linear term, with $v_{\text{drift}} = v_h$ and $a_{\text{dist}} = 0$, and the belief that the rack end is clear expires after $(\delta_{\text{margin}} - \delta_{\text{loc}})/v_h$, which comes to 31.25 ms. The navigation camera delivers a frame every 16.7 ms, so the belief lapses in under two frames, and the frame that showed the cross aisle empty is already 51.6 ms old when perception dispatches it (section 8.3).

Nothing in the software marks the moment of expiry. The map still shows the cross aisle free, and the estimator still publishes a well-formed record, yet the prediction has stopped functioning as evidence. Memory therefore cannot certify an occluded rack end as clear, nor can the frame that feeds it, and the region stays unknown. The stopping budget protects a static obstacle at D_{clear} , such as a person who has stepped out and stands in the aisle, or a dropped tote, and memory adds a deadline to that budget but no distance. The running total at the drive limit stays at the 912.9 mm that section 8.3 left.

A person who keeps walking toward the machine during the whole stop is a different obstacle. A safety-distance calculation counts that approach as $v_h(\tau_{\text{delay}} + T_{\text{stop}})$, the walking speed applied over the entire time from detection to standstill. At the 1.5 m/s drive limit, where a constant-deceleration stop lasts $T_{\text{stop}} = v/a_{\text{brake}}$, that term alone comes to 1.41 m, more than the entire 1.10 m clear distance before any other distance in the budget is counted. The site therefore handles the approaching person with a crossing rule that site operations owns, a floor-marked crossing and a rack-end presence sensor. Section 17.3 records the premise the rule rests on. Whatever crossing rule the site adopts,

⁴ | **Bayesian state estimation provenance:** Kalman (1960) introduced recursive minimum mean-square error state estimation for linear dynamic systems. Moravec and Elfes (1985) generalized these recursive Bayesian updates to spatial occupancy grids, modeling sensor beam uncertainty through explicit range-bearing distributions. In embodied systems, uncompensated propagation across extended sensor dropouts causes Bayesian belief distributions to disperse, requiring explicit invalidation horizons to prevent open-loop control divergence.

the permission path must refuse any proposal whose clearance relies on remembered emptiness past the belief's deadline.



Definition 9.2: Belief state invalidation horizon

Belief state invalidation horizon (t_{exp}) is the elapsed time since the last physical observation at which the belief's validated error bound $E(\Delta t)$ first reaches the task tolerance E_{max} :

$$t_{\text{exp}} = \inf\{\Delta t \geq 0 : E(\Delta t) \geq E_{\text{max}}\} \quad (9.2)$$

where $E(\Delta t)$ is a bounded-disturbance growth law such as equation 9.1 or a calibrated error quantile, and a belief whose evidence epoch is t_0 stops supporting motion at $t_0 + t_{\text{exp}}$.

1. **Significance:** Formalizes the hard mathematical boundary where forward-propagated state estimation ceases to function as valid physical evidence, requiring the permission path to refuse dependent motion proposals before geometric clearance is breached.
2. **Distinction:** Unlike open-loop timeout watchdogs that monitor software process heartbeat intervals, the invalidation horizon is derived from continuous physical kinematics, bounded disturbance dynamics, and mechanical clearance constraints.
3. **Common pitfall:** Allowing a neural planner or Kalman filter to continuously extrapolate state estimates during extended sensor dropouts without bounding error expansion, mistaking smooth mathematical extrapolation for physical ground truth.

When the sensor pipeline reacquires line of sight, the system cannot simply overwrite the propagated state with the new measurement, nor can it blindly pass the raw measurement into the downstream controller. Sensor readings obtained immediately after occlusion often suffer from transient lighting reflections, partial edge visibility, or correspondence errors. Before blending or resetting the state, the estimator must calculate the **innovation**, a mathematical reality check defined as the difference between the new measurement and where the internal model predicted the pose would be. If the new measurement falls inside the calculated error bound $E(\Delta t)$, the measurement confirms the model's drift assumptions, allowing the estimator to collapse the uncertainty and reset the evidence clock. If the incoming measurement falls outside that bound, either the physical object moved unexpectedly or the sensor returned an outlier. Snapping the coordinate frame directly to an erroneous outlier tells the planner that the robot has jumped, introducing an instantaneous position step into the trajectory. The step demands an acceleration no drive can supply, which pushes the actuators against the current ceiling of section 2.4 and toward the thermal limits of section 2.5. The estimator must isolate the discrepancy, evaluate whether the measurement is valid, and reject discontinuous jumps before updating actuation targets.

Model prediction serves as a temporary bridge across missing or delayed observations, and the bridge holds only while the accumulated model error remains smaller than the mechanical tolerance of the task. How soon it fails depends on what drives the error. At the rack end a speed bound alone set the rate. At the cage door, the width of the latch-release slot sets the tolerance E_{max} for the gripper finger that must enter it, and unmodeled forces add terms that grow faster than any speed bound. Between sensor updates the estimator propagates the state by integrating commanded forces through its model of the plant, without feedback confirmation.

The stochastic model behind that propagation separates three mechanisms of decay. An unmeasured velocity error at the onset of occlusion grows position variance as $\sigma_v^2(0)(\Delta t)^2$, and unmodeled stochastic forces such as torque ripple or structural vibration, integrated twice, grow it as $\frac{1}{3}q_c(\Delta t)^3$; these variance terms are what a calibrated quantile must absorb. A steady unmodeled force bounded by $|a_{\text{dist}}|$, such as a dragging cable or persistent sliding friction, instead deflects the body by at most $\frac{1}{2}a_{\text{dist}}(\Delta t)^2$, the deterministic term of equation 9.1.⁵

At the cage door, the finger now enters the slot blind. In the final approach the finger hides the slot from the wrist camera, and the finger must stay within 1 mm of the slot's centerline, the task tolerance E_{max} (figure ??). The arm's validated operating region supplies the bounds already charged at the slot's missed frame in section 9.4, initial position error E_0 at most 0.1 mm and unobserved velocity

5 | **Non-smooth contact dynamics:** Physical surface contact introduces set-valued Coulomb friction and non-smooth stick-slip transitions that break linear covariance propagation models. Transitioning between stick and slip modes injects discontinuous velocity steps that invalidate continuous kinematic drift bounds. Managing these contact regime transitions requires hybrid state estimators that switch between distinct error growth matrices upon detecting normal force thresholds.

error v_{drift} at most 12 mm/s, and occlusion adds one more, an illustrative acceleration disturbance a_{dist} at most 50 mm/s². The growth law of equation 9.1 carries no lever-arm arc, because the belief chains the slot against the wrist pose of its own capture epoch, the remedy for the missed frame, so position error grows only through drift and disturbance.

Evaluating this growth bound with the stated parameters yields $E(\Delta t) = 0.1 + 12\Delta t + 25(\Delta t)^2$, with Δt in seconds and $E(\Delta t)$ in millimeters. Setting $E(\Delta t) = E_{\text{max}}$ gives the quadratic equation $25(\Delta t)^2 + 12\Delta t - 0.9 = 0$. Its single positive real root is 66 ms, so a proposal depending on this belief must expire by then. If the slot were machined to a clearance of 0.30 mm, the allowable unobserved duration shrinks to 16.1 ms ($25(\Delta t)^2 + 12\Delta t - 0.2 = 0$). Tightening mechanical tolerance by 3.3× shrinks the allowable unobserved duration by 4.1×, requiring sensor re-acquisition or compliant impedance holding within that shorter window. In contrast, if an estimator naively assumes constant velocity ($a_{\text{dist}} = 0$), it predicts a valid horizon of $(E_{\text{max}} - E_0)/v_{\text{drift}}$, or 75 ms. At that elapsed time the modeled bound reaches 1.141 mm, exceeding the clearance by 14.1 percent. A constant-velocity-only admission rule would keep accepting the estimate for 9 ms beyond the bounded-model deadline; whether contact occurs depends on the path and geometry.

An estimator's internal **covariance matrix** reflects its stochastic model, not a measured hard maximum. A safety argument can rest on one of two kinds of bound. A **bounded-disturbance** growth law needs justified limits on initial error, velocity, acceleration, clock conversion, and mode changes; then the 31.25 ms rack-end root and the 66 ms latch-slot root are conditional worst-case deadlines within that region. Alternatively, staged dropouts and external ground truth can calibrate a specified residual quantile (for example P_{99} or $P_{99.9}$) under a named test distribution. That yields a probabilistic admission limit with a one-percent or 0.1-percent tail before other model and coverage errors. The record and consumer must declare which regime applies, monitor its validity region, and refuse dependent motion or choose a validated feasible fallback when evidence exceeds it.

Either regime yields the same kind of deadline, because the invalidation horizon of 9.2 asks only when the declared bound first reaches the task tolerance, whatever model produced the bound. Expiry is therefore neither an arbitrary communication watchdog timer nor a fixed 100 ms heuristic chosen for scheduling convenience. It is a physical deadline dictated by task geometry, actuator authority, and disturbance dynamics, and past $t_0 + t_{\text{exp}}$ no downstream loop may assume the estimated pose lies within the required clearance.

The horizon of equation 9.2 is not one scalar for the whole machine. Payload, speed, and contact state change the disturbance bounds, so the state interface selects the active expiry for the current operating regime, and a transition from free motion into contact, which changes velocity within milliseconds, invalidates a smooth drift model outright. Three beliefs on the same machine expire on different clocks (table 9.2). The latch-slot finger is bounded by drift and disturbance acting against a millimeter tolerance. The person at the rack end and a coworker beside the arm are both bounded by the same walking speed, yet their beliefs lapse at different times because each starts from a different initial error and closes on a different tolerance.

Table 9.2: Belief-expiry roots on the machine: Each root solves $E(t) = E_{\text{max}}$ for that belief's error model and task limit. The coworker's initial error is illustrative and its tolerance chosen, while its speed is the same walking-speed bound as the rack end, so both person rows grow linearly. The last column names the response the root would trigger. The record for the mug-handle belief that chapter 10 consumes is filled in section 9.9.

Belief	Error bound $E(t)$ (mm; t in s)	Task limit (mm)	Root	Response at expiry
Person past the rack end	$50 + 1600t$	100	31.25 ms	Treat the cross aisle as unknown; refuse dependent advance
Latch-slot finger	$0.1 + 12t + 25t^2$	1	66 ms	Refuse the insertion; hold compliantly or reacquire the slot
Coworker beside the arm	$15 + 1600t$	150	84 ms	Revoke dependent arm motion; stop only if braking clearance remains

Every root in this section trusts the growth law it was computed from. A growth law is only as good as the machine's ability to catch its failures, through a residual at reacquisition or a staged

dropout test, and some errors produce no residual at all because no sensor on the machine can see them.

9.7 When the Estimator Cannot Know

An illustrative strain gauge on a robot's end-effector reads a steady 5 N, a reading that fits any true contact force once an unknown bias is allowed. An estimator, or a learned model, that predicts that force with confidence has not shown the force is observable. In state estimation, a state trajectory is **observable** if and only if the history of available sensor outputs over a finite time window uniquely determines the initial state of the machine, so that distinct physical trajectories produce distinct sensor trajectories rather than indistinguishable measurement sequences. When two distinct physical state histories produce the exact same sequence of sensor readings, no mathematical operation, filter, or neural network running anywhere in the machine can determine which physical trajectory the body actually followed. The machine receives identical bitstreams from its sensor interfaces across both realities.

To see why predictability differs from observability, follow that end-effector as it regulates contact force against a rigid surface. Let the true normal contact force be $x(t)$, and let the strain-gauge conditioning electronics introduce an unknown, slowly changing sensor bias $b(t)$. The raw measurement delivered to the estimator at each sample interval is $y(t) = x(t) + b(t)$. Suppose the sensor outputs a constant stream $y(t) = 5$ N. This observation is satisfied by a true physical force $x(t) = 5$ N with zero offset $b(t) = 0$ N. It is equally satisfied by a true force $x(t) = 2$ N with a sensor bias $b(t) = 3$ N, or by $x(t) = 0$ N with a bias $b(t) = 5$ N. For any arbitrary offset trajectory $\Delta(t)$, the state pair $(x(t) - \Delta(t), b(t) + \Delta(t))$ produces the exact same measurement sequence $y(t)$. The difference between the true force and the sensor offset lies in an **unobservable subspace** of the measurement system, a structural blind spot where physical variations perfectly cancel each other out in the sensor data.

Because the measurement stream cannot distinguish between physical load and sensor drift, an estimator must rely on prior assumptions about how each state evolves. If a Kalman filter or a learned recurrent estimator is configured with a belief that the sensor offset $b(t)$ is static with near-zero variance, it attributes every variation in $y(t)$ to changes in true contact force $x(t)$. As new samples arrive at 1,000 Hz, the filter incorporates the incoming data, repeatedly updating its internal state. Because the incoming measurements match the forward projection of the state dynamics, the calculated state covariance contracts, and the estimator reports an increasingly narrow uncertainty bound, publishing an estimate of $5 \text{ N} \pm 0.05 \text{ N}$ at P_{99} . Meanwhile, Joule heating ($I^2 R$) in the joint stator coils (section 2.5) warms the sensor bridge (altering the metal's electrical resistance as it expands), causing the physical bias $b(t)$ to drift at a thermoelastic rate of 0.2 N/min. Over 15 min, $b(t)$ shifts by 3 N. The true force on the surface has fallen from 5 N to 2 N, yet the estimator reports high confidence in a state that has drifted by 60 percent of its nominal value.

The downstream control policy responds to this false certainty by commanding motor torques to maintain what it believes is a 5 N preload. Because the true force is only 2 N, the friction force at the contact patch drops below the shear load of the workpiece, causing the part to slip from the gripper and fall into the tooling fixture. This failure occurs without triggering a single internal software diagnostic. The estimator monitors its innovation sequence, the innovation of section 9.6 evaluated at every sample between incoming measurements $y(t)$ and predicted measurements $\hat{y}(t)$. In this drift scenario, the estimator predicted $\hat{y}(t) = \hat{x}(t) + \hat{b}(t) = 5$ N, and the physical sensor returned $y(t) = 5$ N. The innovation is zero at every time step. An innovation check measures only whether the physical sensors disagree with the forward projection of the internal state. It cannot expose an unobservable error mode, because along the unobservable manifold the sensor output is invariant to the drift.

Recovering observability requires the system architecture to provide an additional constraint that breaks the symmetry along the unobservable subspace. An engineering design accomplishes this in one of three ways. First, independent sensing can introduce a second measurement channel with different failure physics, such as monitoring joint motor current or an optical deflection sensor that does not share the strain gauge's thermal sensitivity. Second, the machine can perform deliberate physical excitation, commanding the gripper to briefly break contact with the workpiece for 40 ms, creating a known boundary condition of $x(t) = 0$ N where the measured sensor value directly

exposes $b(t)$. Third, the machine can rely on periodic external anchoring, bringing the tool into contact with a calibrated reference load cell during scheduled tool-change cycles to reset the bias estimate.

When a physical state cannot be constrained by complementary sensing or active excitation, the estimator must treat that state as unobservable and refuse to publish false precision. The estimation layer must mark the corresponding state dimensions as unavailable, or widen their error bound according to the worst-case physical drift rate of the underlying hardware rather than the contracted covariance of a filtering algorithm. Formal observability analysis for linear and nonlinear dynamic systems is an established foundation of control theory, treated rigorously by Kalman (1960) for linear state spaces and by Hermann and Krener (1977) for nonlinear systems via **Lie derivatives** (see section B.6). A systems engineer does not rederive these rank conditions at runtime. The engineering task is to identify which physical states lack full observability rank under the current sensor suite, and to enforce that the state interface never reports statistical convergence on an unconstrained degree of freedom.

The observability and convergence results of state estimation come from control theory, and they hold only under four explicit assumptions. First, the sensor forward model must capture all physical couplings without unmodeled algebraic dependencies. Second, physical parameters such as component stiffness and thermal coefficients must remain within pre-calibrated ranges. Third, sensor noise must remain zero-mean and uncorrelated with actuator commands. Fourth, the operating trajectory of the machine must avoid singular configurations that cause the **observability matrix** to lose rank, a mathematical collapse indicating that certain physical states have temporarily become invisible to the sensors. The estimator validates these assumptions during operation by cross-checking redundant measurement channels, checking environmental temperature sensors, and enforcing active re-zeroing routines before the belief's expiry. When an operating condition violates these assumptions, such as when an optical sensor becomes occluded or a kinematic configuration reaches a singular alignment, the estimator withdraws the estimate's valid status, and downstream controllers fall back to conservative velocity and torque limits. An estimator that withdraws its own claim fails in the open; the harder failures are the ones in which nothing withdraws.

9.8 How Belief Goes Wrong

A camera stops delivering frames, yet a transport node keeps republishing its last transform with the current time, and a downstream controller's 5 ms publish-time freshness check passes every copy. That failure differs in kind from a camera that simply falls silent, and the difference sorts every estimation failure into one of two forms. In a **loud failure**, a component halts execution, drops a communication packet, clears an output buffer, or asserts an explicit error bit. Downstream consumers immediately observe the absence of data or the cleared validity flag, enabling a validated fallback, such as a hold, stop, or compliant mode, when one remains feasible. In a **silent failure**, the memory layer continues to publish a complete, numerically well-formed state record with valid status bits, low variance indicators, and continuous timestamps, while the numbers inside the record diverge from the physical configuration of the machine. Downstream control loops consume these corrupted records without raising an exception, computing feedback corrections against a phantom geometry. Because physical actuators execute commands derived from these valid-looking records, such as driving an arm toward a target that now intersects a wall, silent state corruption can deliver force across the causal boundary of section 1.2 before any software monitor registers an anomaly.⁶

The republishing hazard of section 9.1 has a name, the **frozen transform**, and it is the most deceptive silent failure because a stale spatial snapshot, republished with current timestamps, masks a dead sensor. In an illustrative timeline, an optical tracking node computes a transformation matrix $T_{\text{camera}}^{\text{object}}$ at timestamp $t_0 = 0$ ms. If the optical sensor disconnects, drops frames, or suffers thread starvation at $t = 10$ ms, the underlying tracking algorithm ceases to produce new spatial evidence. If the message-passing middleware or an intermediate transport node retains the last computed matrix in an unexpired buffer and republishes it at a loop rate of 1,000 Hz, generating a new transmission timestamp $t_k = t_0 + kT_{\text{rep}}$ with $T_{\text{rep}} = 1$ ms, the temporal provenance of the measurement is erased. A downstream tracking controller verifies freshness by computing data age as $\Delta t = t_{\text{now}} - t_{\text{msg}}$. Because the publisher stamped the message with the current system clock, Δt evaluates to less than 1 ms, satisfying a strict freshness threshold of $\tau_{\text{max}} = 5$ ms. The controller accepts the transform as

⁶ | **AMR Safety Standards:** ISO 3691-4 addresses safety requirements for driverless industrial trucks and their systems. Occlusion and stopping distance are assessed for each design's operating domain and sensor and actuator configuration, not taken as universal values.

current evidence. If the physical object moves at 0.5 m/s while the transform remains static, the controller accumulates 50 mm of spatial tracking error in 100 ms, yet reports zero estimated tracking divergence.

Phantom-object tracks arise when the estimator decouples the lifetime of spatial coordinates from the lifetime of existence evidence. A perception module maintains an object track comprising a kinematic state vector $\mathbf{x} = [p, v]^T$ (encoding spatial position and velocity) and an existence probability $p_{\text{exist}} \in [0, 1]$ (encoding confidence that the object is physically real). When the physical object is removed from the workspace or an occlusion begins, raw detector hits drop to zero. If the estimation layer continues to propagate $\mathbf{x}$ forward using process dynamics while existence probability decay is governed by a long time constant (for instance, an illustrative decay under which p_{exist} remains above an operating threshold for 1500 ms without reinforcement), the tracker maintains a persistent phantom entity. The path planner routes the manipulator around empty space, or an automated grasping routine closes its fingers on an empty coordinate. Detecting a phantom track cannot rely on passive measurement absence. It requires negative evidence (section 9.5), where a line-of-sight sensor sweeps the volume and confirms that the expected occupancy is empty, or an unfulfilled contact expectation when a force sensor registers zero load at the predicted contact boundary.⁷

Coordinate frame misattributions divide along a similar boundary between syntax and semantics. A **schema-detectable frame mismatch** occurs when a publisher labels a transform with an invalid or undefined frame identifier in the transformation tree graph. A message bus with static schema enforcement rejects the packet at the serialization boundary because the named frame cannot be resolved to a known parent link. In contrast, a **semantically wrong frame assignment** occurs when an engineer or configuration file assigns a valid, type-correct frame identifier that represents the wrong physical link. For example, commanding a tool center point trajectory using the transform of the bare mounting flange T_{flange} instead of the extended tool tip T_{tool} satisfies every schema validation check, type assertion, and tree traversal test. In an illustrative setup with a tool tip 150 mm beyond the flange along the tool z -axis, a flange-as-tip error shifts the intended contact point by that offset. Whether contact occurs depends on the fixture geometry and admitted path. Detecting a semantic frame error requires physical consistency checks, such as verifying measured motor torques against an analytical dynamic model of the arm, or cross-referencing multi-camera optical tracker observations against joint encoder kinematics.

When sensor updates drop out, an estimator relies entirely on its predictive forward model to advance state estimates across the blind interval. A well-formulated filter updates the mean state estimate $\hat{\mathbf{x}}(t)$ while monotonically expanding the estimation covariance (t) to reflect the growth of unconstrained process noise. An **overconfident predictor** understates process variance $\mathbf{Q}$, holding (t) artificially compact over an extended prediction duration exceeding 200 ms. When fresh sensor measurements eventually arrive, the artificially small covariance causes the filter to discount incoming observations; it weights its internal dead-reckoning so heavily that it classifies valid measurements as statistical outliers and rejects them. Engineers identify overconfidence during system qualification by executing staged synthetic occlusions, systematically dropping sensor frames for intervals between 50 ms and 500 ms and evaluating the innovation at reacquisition, $y(t) - \hat{y}(t)$. If that residual consistently exceeds the predicted 3σ covariance bounds across fifty staged trials, the process noise model is invalid. Drift along an unobservable degree of freedom, such as workpiece slippage inside compliant gripper pads, produces no reacquisition residual at all (section 9.7). At vehicle scale, a terrain belief stands until the wheels themselves contradict it.

⁷ | **Autonomous Safety Standards:** UL 4600 addresses safety evaluation of autonomous products, and ISO 26262 addresses automotive functional safety. The data-age, track-invalidation, and transform checks in this chapter are design requirements of its worked architecture, not clauses of either standard.

War Story 9.1: A rover trapped in sand (2009)

Context: *Spirit's* wheels started to sink into soft soil at Troy on Sol 1886, and the rover was embedded by Sol 1899, as the JPL navigation-camera record dates the approach and embedding. A darker top layer covered the site; the wheels broke through it into lighter material beneath.

Mechanism: Wheel rotations did not produce the expected translation as the rover sank. JPL reported that a rock beneath the vehicle might have contacted its underside; contact was a possibility, not an established mechanism. Surface appearance alone could not establish the subsurface load-bearing state.

Impact: Recovery teams tested extraction maneuvers, but *Spirit* did not regain mobility. The event illustrates a terrain-belief limit: the physical interaction supplied evidence that the previous traversability estimate could not safely preserve.

Response and lesson: JPL had created the Slip Check mode after *Opportunity* embedded itself in Purgatory Ripple (JPL technical account), so slip checks based on visual odometry already existed before Troy. A controller should compare wheel rotation with verified motion, expire a terrain assumption when that evidence conflicts, and admit further driving only through a feasible permission check.

Frozen transforms, phantom tracks, misassigned frames, and overconfident predictors all cross the boundary between estimation and control, and the spatial belief record of section 9.2 is the structure that must catch them there, because an ad hoc dictionary of values and matrices cannot tell active evidence from cached momentum. Every element of spatial memory must therefore reside within an explicit state record that couples the numerical value to its base coordinate frame, its sensor provenance, its growth bound, and a hard invalidation deadline that forces downstream consumers to refuse the record once that deadline passes.

9.9 The Spatial State Schema

The belief about the latch-slot finger in section 9.6 now carries everything a consumer needs to judge it, and the spatial belief record of 9.1 is the structure in which those quantities travel together. From the observation contracts of section 8.7 the record keeps each observation's capture epoch, clock identifier, and conversion bound, so the permission controller can convert the capture clock to its own monotonic clock with a bounded error before it evaluates age. Memory contributes four fields of its own. An estimate epoch dates the latest computation and stays apart from the evidence epoch, which dates the last physical measurement; an uncertainty-growth bound states how fast error accumulates without new evidence; a validity region states where that bound holds; and an expiry states when the belief stops being admissible. A record is **VALID** while fresh observations constrain it. Once observations stop, prediction can only carry it forward, to **DEGRADED** while its bound stays inside tolerance and to **EXPIRED** when that bound reaches tolerance or the machine leaves the validity region; only a fresh observation returns it to **VALID**. A record is **UNOBSERVABLE** when the sensors cannot constrain the quantity at all (section 9.7). Table 9.3 fills the record for the finger as a bounded-disturbance belief.

Table 9.3: Complete spatial-belief record: Value, unit, entity, frame, two epochs, clock conversion, error model, validity region, expiry, provenance, and lifecycle travel together. A later estimate epoch dates a computation, not a measurement, and cannot extend expiry past $t_0 + t_{\text{exp}}$.

Field	Example record value	Consumer check
Entity and quantity	Gripper finger relative to the latch-release slot, lateral offset from the slot centerline	Match the target entity and task quantity
Value and SI unit	Lateral offset, stored in meters; the bounds below are shown in millimeters for reading	Decode as meters, not millimeters or force
Frame	latch_slot frame, with the transform revision of the wrist-camera calibration	Resolve a time-valid transform chain
Evidence epoch	t_0 , the last wrist-camera capture before the finger occludes the slot, on the wrist camera's capture clock	Preserve across prediction and republishing
Estimate epoch	Latest propagation of the offset, on the same capture clock	Date the propagated estimate; do not renew evidence
Clock conversion	Versioned $t_{\text{MCU}} = at_{\text{capture}} + b$, error at most 0.2 ms (section 8.7)	Refuse if conversion expires; add error to upper age
Initial error	$E_0 = 0.1$ mm	Check the initial bound against calibration
Growth law	$E(t) = 0.1 \text{ mm} + (12 \text{ mm/s})t + \frac{1}{2}(50 \text{ mm/s}^2)t^2$	Compare at evidence age with the slot tolerance
Validity region	Arm acceleration and contact mode unchanged since t_0 ; drift and disturbance bounds above validated within this region	Expire immediately on a mode or residual-bound violation
Expiry	$t_0 + 66$ ms, where the growth law reaches the 1 mm slot tolerance	Refuse dependent proposals once expiry passes; a later estimate epoch does not extend it
Provenance	Wrist-camera observation ID, calibration revision, estimator build hash	Match approved sources; hash is version provenance, not payload integrity
Lifecycle	VALID at capture, DEGRADED once the finger occludes the slot, EXPIRED at $t_0 + 66$ ms	Never restore VALID without fresh physical evidence
Map cell evidence, when applicable	Not used by this record; a map belief such as the rack-end cross aisle carries a per-cell capture epoch and free/occupied/unknown state	Check every cell used by the candidate path; unknown stays unknown

The same form carries the belief that chapter 10 builds on, the pose of the mug's handle as the mug rides the takeaway conveyor toward the arm. Its frame is the robot base frame, its evidence epoch is the last wrist-camera capture of the handle, converted under the same 0.2 ms clock bound, and its initial error is 3 mm. Its growth law differs in kind from the finger's. The belt carries the mug at no more than 0.20 m/s, a bound on speed rather than on force, so the error grows linearly from its initial value and reaches the 15 mm grasp tolerance at $t_0 + 60$ ms. Its validity region is the belt running forward within that speed with the mug seated on it, so a knock that tips or lifts the mug, or a belt that overspeeds or reverses, expires the record at once, whatever time remains. A qualified tracker that renews the record on every frame replaces that belt-drift law with one bounded by the belt's residual slip (section 10.4).

A consumer of either record converts the capture epoch to its own clock, adds the clock-error margin to the age, checks the lifecycle status and the validity region, and only then evaluates $E(t)$ against the proposed motion's tolerance. Passing these checks admits the belief to the **separate permission decision**; it does not actuate the robot. On expiry or unobservability, the controller rejects dependent proposals and selects a validated fallback whose braking, hold, or search behavior remains physically feasible.



Checkpoint 9.1: Belief expiry, observability, and the belief record

Before turning to the fallacies that arise when valid records are trusted too far, verify your understanding of evidence epochs, belief expiry, observability limits, and the belief record:

- ☐ **Epoch classification:** Can you distinguish the evidence (acquisition) epoch (t_0) from the estimate epoch (t_{est}) and the publication epoch (t_{pub}), and explain why stamping an estimate with its publication time hides the displacement accumulated since capture?
- ☐ **Physical invalidation horizon:** Can you explain why the shelf life of a spatial belief during occlusion is set by task clearance and bounded disturbance rather than by a software timeout, and why a constant-velocity extrapolation admits the belief past its deadline?
- ☐ **Unobservable manifold hazards:** Can you explain why zero innovation ($y - \hat{y} = 0$) fails to detect sensor bias drift caused by motor thermal heating (I^2R Joule heating) along an unobservable degree of freedom?
- ☐ **State lifecycle state machine:** Can you trace the deterministic transitions between VALID, DEGRADED, EXPIRED, and UNOBSERVABLE, explaining why a state record must never revert to VALID without a fresh hardware evidence epoch t_0 ?

9.10 Fallacies and Pitfalls

Individually valid records need not form a consistent account of the workspace. A controller must reconcile their times and the evidence behind them before using them together to authorize motion.

Pitfall: *Evaluating spatial clearance between moving bodies using asynchronous timestamps.*

The base drives toward a rack end at its 1.5 m/s drive limit while a coworker walks toward the base at the 1.6 m/s walking-speed bound, so the two close at 3.1 m/s. Their position records differ in acquisition time by 20 ms, and both remain inside the 31.25 ms expiry window. Comparing them directly combines positions that never described the same instant. Across the gap the two bodies close by 62 mm, more than the 50 mm between the localization bound and the protective clearance. Individual freshness checks cannot establish joint clearance. The controller must propagate both estimates to a common evaluation time, carrying each growth bound to that instant, or bound the allowable skew from the closing speed and the clearance.

Fallacy: *Bounded spatial covariance proves that a physical entity still exists in the workspace.*

This is the phantom track of section 9.8 at the rack end. A person steps out, is detected, and steps back behind the rack, and the tracker keeps predicting the person along the cross aisle with a covariance below the rejection threshold. The planner then routes the base around the predicted person and treats the rest of the cross aisle as accounted for. That second conclusion fails even if the person is really there, because the absence of detections in an occluded region is evidence neither of presence nor of emptiness, and a precise extrapolation of one track certifies nothing about the space around it. A tracker must keep existence evidence separately from spatial covariance, and only negative evidence from a sensor that could have seen the cross aisle can clear the space around the track.

9.11 Summary

Spatial memory carries an estimate across the intervals when no sensor constrains it. Carrying an estimate forward is not renewing it. A predictor, a filter, or a republishing bridge advances a state's timestamp to the current cycle, but the evidence behind that state stays rooted at the capture epoch of the last physical measurement. The spatial belief record therefore keeps two epochs apart, the evidence epoch that dates the measurement and the estimate epoch that dates the computation, and binds both to a frame, an initial error, a growth law, a validity region, and an expiry. However often the estimate is recomputed, only a fresh observation restores the record's valid status. The frame binding matters because a transform is itself a belief with its own evidence epoch, and a stale edge passes every check the tree can run while the tool point it locates drifts out of clearance.

The lifetime of a state estimate cannot be governed by convenient software timeouts, such as an operating system watchdog or a transport buffer set to a round 100 ms. Expiry is a physical calculation dictated by the task's tolerance and the validated growth rate of unobserved error. At a rack end, the belief that the cross aisle is empty expires 31.25 ms after the last frame that saw it, before perception has even dispatched that frame, regardless of whether the software message queue remains healthy. Memory therefore cannot certify the occluded cross aisle as clear. It adds a deadline to the machine's stopping budget but no distance, the budget protects only a static obstacle, and a person still walking toward the machine is carried by a site crossing rule instead. Negative evidence sorts spatial representations. Only space that a valid ray has seen empty vouches for clearance, and only while that evidence is fresh, so neither a rendered view nor a map's publication time is a clearance bound.

Two kinds of error hide from the checks a consumer can run. A frozen transform or a phantom track publishes complete, fresh-looking records, which makes it more hazardous than an explicit missing frame, because a missing frame triggers a validated fallback while a stale record disguised as fresh passes an inadequate freshness check. Unobservable error hides deeper still. When the available sensors cannot distinguish two physical states, the innovation stays at zero and the covariance contracts while the true error grows, as it did for the force estimate whose sensor bias drifted with winding heat. Only a second channel with different failure physics, deliberate excitation, or an external reference restores what the sensors cannot see.

Downstream: Forensic recording of expired spatial transforms populates the authority logs in section 14.8.



Key Takeaways: Evidence age and belief expiry

- **Prediction advances state, never evidence:** Forward propagation moves an estimate's timestamp to the present, but its evidence epoch stays at the last physical measurement. Age is measured from that epoch, so a republished or extrapolated state is exactly as old as the observation beneath it.
- **Expiry is a physical deadline:** A belief expires when its validated error bound $E(\Delta t)$ reaches the task tolerance $E_{\max}$, whether that bound is a bounded-disturbance growth law or a calibrated quantile. A software timeout chosen for scheduling convenience says nothing about either.
- **Memory adds a deadline, not distance:** The rack-end belief lapses in 31.25 ms, under two camera frames, so memory cannot certify an occluded region as clear. The stopping budget gains a deadline from memory but no distance term, and still protects only a static obstacle. Unknown space stays unknown, and a map's publication time never refreshes a cell no sensor has revisited.
- **Stale records look healthier than missing ones:** A frozen transform, a phantom track, or two records compared across mismatched epochs all pass freshness checks that a dropped frame would fail. Existence, cross-record skew, and transform epochs each need their own check against evidence time.
- **Unobservable error hides from innovation checks:** Along a degree of freedom the sensors cannot distinguish, residuals stay at zero and covariance contracts while the error grows. Independent sensing, deliberate excitation, or external anchoring must supply the missing constraint before the estimator may report convergence.

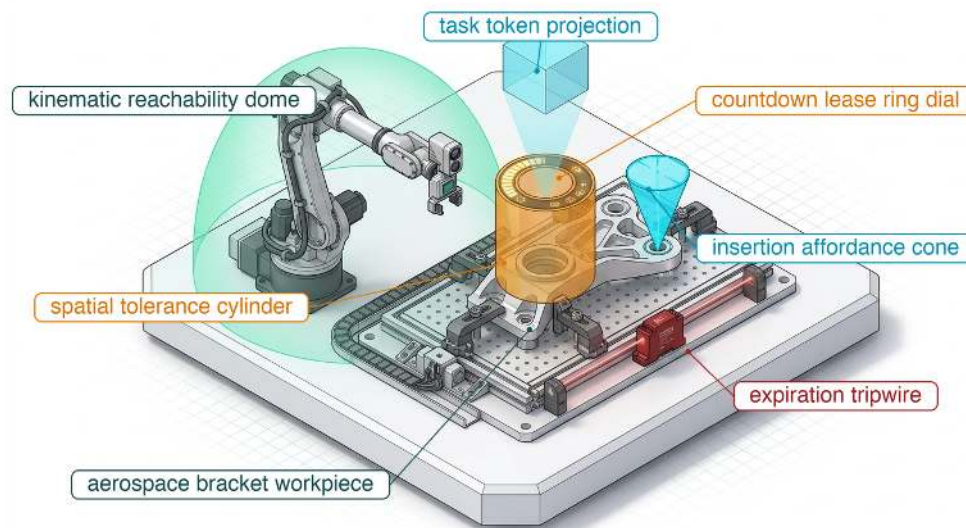
The third law requires the permission path to decide each action on state whose age is known (principle 3), and spatial memory gives that age its definition, counting it from the evidence epoch of the last physical measurement rather than from the estimate epoch of the latest computation. Space that no valid ray has crossed has no evidence epoch at all, so it reaches the permission path as unknown, with no age that could make it clear. A known age is still not a license. It means something only through the growth law that turns it into an error bound, and however exactly the age is known, the permission path must refuse the state at the deadline where that bound reaches the task tolerance.

 What's Next: From valid beliefs to bounded goals

What may a valid belief authorize? Memory rules only on validity. It can say that a remembered object lies within a stated bound until a stated deadline, but it cannot say what the machine should do about that object. In chapter 10, a human goal, taking the mug from the conveyor, becomes an intent lease that consumes the belief record of the mug's handle, carries that belief's evidence epoch forward as its own evidence time, caps its expiry at that belief's horizon, and grants a bounded target with effort limits, never a path, for no longer than the evidence beneath it supports.

DRAFT

Grounded Intent



- 10.1 Grounded Affordances
- 10.2 What Intent Hands Over
- 10.3 From Request to Expiring Geometric Proposal
- 10.4 Goals That Expire
- 10.5 Admissible Goal Sets
- 10.6 How Intent Goes Wrong
- 10.7 The Intent Lease
- 10.8 Fallacies and Pitfalls
- 10.9 Summary

Purpose

What must an autonomous goal carry to prevent a slow reasoning model from driving a machine into a situation that no longer exists?

A high-level semantic command such as ‘retrieve the pallet’ specifies a desired outcome while leaving essential physical parameters—trajectory, contact force, and temporal bounds—completely unresolved. Because the physical scene continuously evolves while large reasoning or vision-language models deliberate, raw model outputs cannot be executed without physical grounding. A target that was reachable when deliberative inference began may become fully obstructed before execution commences, even if the model’s high-level task decomposition was logically flawless.

Grounded intent translates abstract directives into physically admissible regions constrained by explicit leases, spatial tolerances, and hard deadlines. When an environmental lease expires or the target drifts beyond certified bounds, safe deceleration cannot wait for a high-latency deliberative loop to acknowledge the timeout. The real-time execution layer must independently revoke motion authority and execute an assured fallback. In the physical AI stack, intent bridges the Brain’s semantic goals to the Body’s physical limits, granting the deterministic Nervous System bounded leases that expire safely when unrenewed.

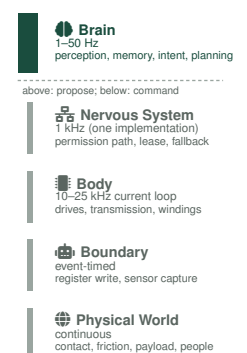


Figure 10.1.: The five-level machine stack with the Brain level highlighted.



Learning Objectives

- Construct an intent lease from a belief record, with a goal region, error envelope, task tolerance, and effort ceilings
- Derive a target-evidence horizon from task tolerance and target drift, and distinguish it from the stopping deadline
- Decide when grounding ambiguity or a wrong-object premise requires refusal rather than admission
- Evaluate whether a proposed goal is reachable within its remaining evidence window
- Design lease expiry that revokes motion authority even when the issuing process cannot cancel it

10.1 Grounded Affordances

WHEN the warehouse mobile manipulator is told to “pick the red mug” from the takeaway conveyor, the directive specifies an outcome but leaves path, force, and deadline unresolved. A motor inverter accepts electrical commands, not language. Vision-Language Action (VLA) models (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) can propose actions or targets, but their update cadence depends on the model and hardware and may be much slower than local control. If a target moves during inference, a stale proposal can consume clearance. The system therefore needs dated geometric evidence, bounded requests, and an independent permission path that can withdraw authority while the model is silent.

Memory can supply a spatial belief with an expiry (section 9.6); it cannot decide what action that belief authorizes. **Intent** closes that gap by connecting high-level task semantics to a bounded physical goal. It specifies an admissible, time-bounded target before the trajectory planner of chapter 11 constructs a continuous motion profile to reach it.

Connecting words to physical targets is the robotics form of what Harnad termed the **symbol grounding problem** (Harnad 1990): purely symbolic or linguistic tokens within a computational system have no intrinsic physical meaning unless causally grounded in non-symbolic, sensorimotor interactions with the physical environment.¹ Whereas early artificial intelligence systems such as Winograd’s SHRDLU (Winograd 1972) operated inside closed-world blocks domains with synthetic, analytic geometries,² modern physical AI systems must ground open-vocabulary instructions into continuous, dynamic, and uncertain physical spaces.³



Definition 10.1: Intent grounding refusal

Intent grounding refusal is the admission rule whereby an embodied system explicitly rejects semantically ambiguous natural-language directives, returning a typed failure status (ERR_GROUNDING_AMBIGUITY) rather than executing ungrounded maximum-likelihood guesses across physical clearance margins.

1. **Significance:** Rejects a proposal when a task-specific ambiguity test (section 10.6) fails, before that proposal reaches trajectory generation.
2. **Distinction:** Unlike a trajectory planning failure that cannot find a collision-free path to a valid goal, grounding refusal rejects the goal itself as physically ungrounded or ambiguous before the planner is invoked.
3. **Common pitfall:** Forcing an argmax token decode or nearest-neighbor guess when candidate referents remain unresolved, driving the physical plant toward an arbitrary or hazardous target.

Whatever the model proposes must also remain bounded in time. Figure 10.2 gives a schedule in which a slow deliberative model, local tracker, permission loop, and drive run at different rates. A conveyor target whose evidence expires within tens of milliseconds cannot be renewed by a model

Prerequisite: The belief record a lease consumes (evidence epoch, initial error, growth law) is specified in section 9.9.

1 | Symbol Grounding Problem: Stevan Harnad’s 1990 formulation argues that syntactic manipulation alone does not give symbols intrinsic meaning. In physical AI, grounding binds a proposed label to dated sensor evidence, calibrated metric frames, and task-specific physical checks. Without that binding, a planner may optimize toward the wrong physical target.

2 | Classical STRIPS planning: Classical planning represents goals as logical predicates in a modeled world. Dynamic obstacles and sensor dropouts can invalidate a physical target before its planned use.

3 | Affordance space parameterization: In robot learning, affordances are parameterized as continuous spatial probability fields or reachability manifolds over the Lie group $SE(3)$. Parameterizing reachability as an explicit manifold enables low-level controllers to evaluate physical feasibility before invoking computationally intensive trajectory optimizers.

updating every few hundred milliseconds, so it requires a separate validated local observation path or a task pause. The figure's rates are illustrative, and section 10.4 sets the machine's own intent cadence against the conveyor horizon.

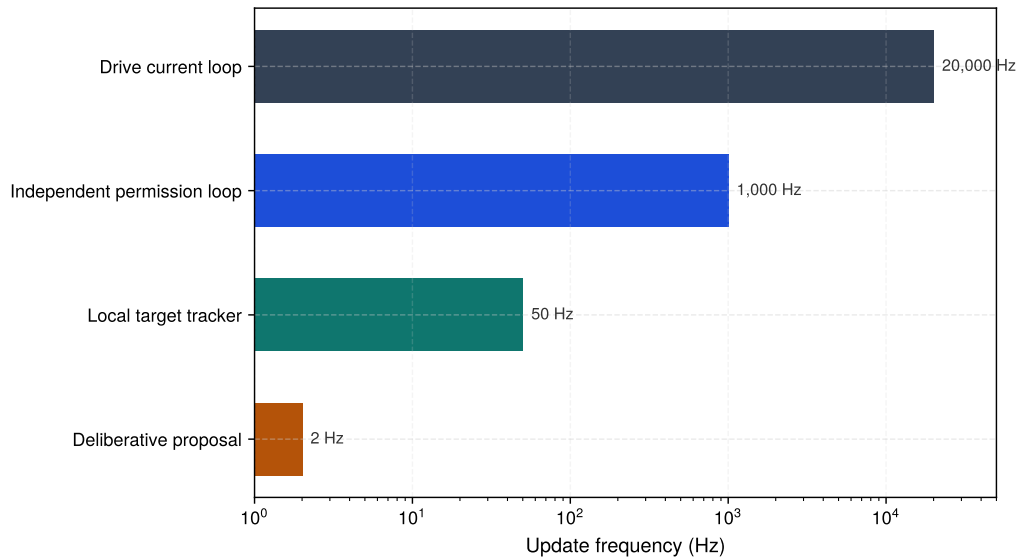


Figure 10.2: Multi-rate schedule: Four illustrative rates show why a deliberative proposal cannot act as a motor or permission-loop heartbeat. The schedule does not relate model size to frequency. The local tracker can renew target evidence only if its sensor and error bounds are independently validated. The machine's own tiers are those of section 4.3; the machine's intent model runs faster than the deliberative rate shown, and the chunk policy is omitted here.

Grounding an open-ended directive therefore ends in a dated **intent proposal**, which the application processor sends across the proposal boundary. It requests bounds in space, time, and effort, and the independent permission path tests each against configured limits, current state, and stopping clearance before motion begins.

10.2 What Intent Hands Over

When the arm reaches for the red mug, the Brain sends neither joint trajectories nor winding voltages across the proposal boundary. It sends a request for bounded authority, and the permission path must be able to check every part of it before it grants any of it. The checks are target tolerance, evidence validity, requested effort, and a feasible fallback, and each exists because a delayed proposal can fail in a different way. Section 10.7 names the record that carries the request and its belief-record lineage.

In physical manipulation, an isolated coordinate is incomplete. In Gibson's sense, an affordance is an action-relative relation among geometry, robot reach, and contact limits (Gibson 1979). The intent proposal therefore requests a target pose in $SE(3)$, the space of three-dimensional positions and orientations, and a **task tolerance region** within which the task counts as complete. This tolerance differs from the estimator's covariance, which describes uncertainty about the target. For the machine's mug grasp, the approved task profile allows 15 mm of translational error. A controller can then terminate within that region rather than integrating forever toward an exact noisy point; it still checks that estimated target error fits inside the tolerance.

Spatial validity in the physical world is transient. A physical goal therefore carries an evidence epoch and a finite requested expiry rather than relying on an asynchronous cancellation message⁴ from the reasoning layer. A model that produces a new result only once per period cannot, even before jitter, renew a target whose evidence stays valid for less than that period. An independent permission controller evaluates the admitted deadline on each control cycle and drops tracking when the evidence or lease expires. It selects a configured hold or stop only when the measured state and remaining clearance make that response feasible.⁵ The local timer removes dependence on

Prerequisite: The proposal boundary and the permission path are defined in section 1.5.

⁴ | **Asynchronous Safety Event Dispatch:** Relying on asynchronous cancellation messages or network event dispatch to abort motion introduces non-deterministic queue serialization delays, and every millisecond of that delay is unbraked travel at the base's drive limit. A local countdown can revoke tracking authority on missed renewal.

⁵ | **Hardware watchdog safety monitors:** A safety design can use an independently clocked watchdog to detect missed renewals and initiate a validated response without the host. Safe Torque Off removes drive torque; it does not by itself hold a suspended load or stop moving mass within a specified clearance.

a host abort packet; the stopping distance remains a separate physical obligation.

A target bounded in space and time is hazardous if the controller is allowed unlimited physical effort. The intent proposal therefore requests effort ceilings on speed, acceleration, and contact wrench. A model may place a target slightly behind the surface it must touch because of stereo disparity error; a stiff position controller can then command excessive restoring torque. For the arm's guarded approach to the spring-latched cage door, the proposal might request the 15 N tripwire as its contact-force ceiling, well below the 100 N latch limit, and the 0.03 m/s approach speed as its speed cap. The permission path compares each request with independently configured actuator and task ceilings, then admits only a feasible subset. The low-level **impedance controller**⁶ enforces the admitted limits.

Omitting any of these three bounds leaves a machine exposed during communication dropouts or perception stalls. Stopping distance grows linearly with the delay before braking and quadratically with speed (equation 2.1). On the moving base, whose stopping budget is a distance, a stalled proposer lengthens the delay term by the time its last proposal keeps authority. That time is the chunk lease of section 4.3, and it enters τ_{delay} . Clearance holds only if an independently checked obstacle boundary lies beyond the stopping distance, with model and sensing margins added.

The intent lease is a different object. It bounds how long a target stays justified, and its expiry withdraws the target and ends tracking at once, without adding a term to the base's stopping budget.



Napkin Math 10.1: Lapsed authority and physical response

The three models below answer different questions. A stopping calculation needs the total delay to brake onset and the deceleration available from the *current* state.

1. **The mug intent lapse:** The arm reaches for the red mug under an intent admitted without local tracking. At 0.20 m/s the conveyor consumes the 12 mm between the 3 mm initial error and the 15 mm tolerance in 60 ms, so the target's evidence expires then, as the mug-handle belief record of section 9.9 states. Suppose the application processor stalls. A target held for 200 ms by the stalled proposer would sit 43 mm from the mug. The lease does not wait for the proposer. At 60 ms the intent expires, the arm's tracking authority ends, and the arm enters a stop matched to its current state, with onset within 22 ms (one permission tick, one bus cycle, and the brake onset). The chunk lease of section 4.3 is the backstop if that expiry is never observed. Under local tracking the same 200 ms leaves the target 13 mm off, inside the tolerance, because the tracked horizon derived in section 10.4 is 219 ms; tracking is what makes the 5 Hz intent refresh viable.
2. **The arm at the cage door:** At the guarded approach speed of 0.03 m/s against the 4.0×10^5 N/m strike plate of the cage door, modeled force rises at 12 N/ms. A 15 N tripwire and 2 ms contact-loop response yield 39 N **when the brake command begins**; force keeps rising while the arm decelerates. Staying under the 100 N latch limit leaves only 0.15 mm of further compression, so the arm must decelerate at about 3 m/s^2 and stop within about 10 ms. Whether the arm can brake that hard from the approach is an open premise, which only arm stopping trials can settle. Unbraked motion for 200 ms advances 6 mm and gives 2,400 N in the linear-spring model, far past the latch limit.
3. **A process-plant contrast:** A heated process has no braking distance, but its setpoint's authority lapses the same way, spent as temperature rather than travel. For an illustrative lumped capacitance of 900 J/K and uncompensated 3600 W input, temperature rises at 4 K/s. A chosen 2 K task overshoot allowance is consumed in 500 ms, so a heater setpoint whose proposer stalls must lose authority well inside that interval; 10 s at the same net heat input yields a 40 K modeled rise.

These three examples clarify the division of authority. High-level reasoning can propose a target, time window, and effort ceiling. The independent permission path determines which limits are admissible from measured state, configured ceilings, and a feasible fallback. Local control then tracks only an admitted proposal and withdraws authority on invalid evidence or expiry. This resembles the priority separation of the **subsumption architecture**⁷ (Brooks 1986a).

⁶ | **Mechanical impedance regulation:** Impedance control regulates the relation between contact force and motion through effective mass, damping, and stiffness. It can reduce loads within a validated contact regime; the first impact and peak force still depend on approach speed, passive compliance, sensing, and actuator response.

⁷ | **Subsumption architecture principles:** Rodney Brooks's subsumption architecture decomposed autonomous robotics into asynchronous, priority-arbitrated behavioral layers operating on sensory feedback. Here, the learned layer proposes a bounded target while an independent controller checks permission and executes a validated response on expiry.

Grounding an intent proposal into these three bounds requires a formal transformation from ambiguous sensory inputs into structured, verifiable proposals. When a system processes a high-level directive or a sequence of visual tokens, the initial proposal is rarely a single, unambiguous set of coordinates. It begins as an uncertain geometric hypothesis that must carry its own spatial ambiguity and its own temporal limits before it can be admitted to the execution pipeline.

10.3 From Request to Expiring Geometric Proposal

A natural language instruction becomes dangerous when an unverified semantic hypothesis commands motion. When an operator or high-level supervisor asks for the red mug, an onboard vision-language model parses the character stream, projects image features into a shared embedding space, and selects a candidate visual bounding region. From those pixels, the network extracts an estimated pose in $SE(3)$ and writes coordinate values into a memory buffer. At the proposal boundary (section 1.5), an ungrounded string of tokens becomes a requested destination; if admitted, it reaches the causal boundary as mass, velocity, and current.⁸

If the visual backbone associates the language token with an adjacent fixture or an unlabeled calibration target, the resulting coordinate vector $\mathbf{p} \in \mathbb{R}^3$ enters the trajectory pipeline as an authoritative physical goal. Because a low-level motor drive measures only encoder pulses, winding temperatures, and phase currents, it cannot evaluate whether the coordinate it tracks corresponds to the intended mug or to an empty stretch of belt.⁹ The grounding process must produce an explicit geometric proposal that includes spatial uncertainty, selection evidence, and temporal expiration before any motion planner accepts it.

Open-vocabulary affordance grounding uses language and sensor observations to propose a task-relevant skill, spatial field, or program, and none of its outputs is a verified target. SayCan (Ahn et al. 2023) multiplies a language model’s preference by a learned estimate of each skill’s success (section A.6), which yields a ranking conditioned on training and observation, not a veto on obstructed or impossible motion. VoxPoser (Huang et al. 2023) (figure 10.3) has a language model write code that composes 3D attraction and repulsion value maps in the robot’s workspace for a trajectory optimizer to use as costs, and gradients over a cost field certify neither collision freedom nor torque limits. Code as Policies (Liang et al. 2023) generates programs that call perception APIs and parameterized motion primitives, and a generated program can stall as surely as a model can, so the countdown and the permission path keep control whatever the program does. OK-Robot (figure 10.4) filters candidate grasps by language-conditioned object selection, and the segmentation mask behind such a selection is a candidate image region, not a verified 3D hull. Table 10.1 sets out what each kind of representation sends downstream and what the permission path still needs before any of them can become an admitted request. Each needs at least a named frame, SI units, dated evidence, and the permission path’s own checks.

⁸ | **Semantic-to-metric transformation:** Projecting discrete linguistic embeddings into continuous $SE(3)$ coordinates requires an unbroken coordinate calibration chain from image pixels to actuator base frames. Unmodeled camera mounting compliance or lens distortion maps semantic attention peaks into systematic metric offsets. If the transformation chain lacks rigid extrinsic calibration, high semantic confidence produces large physical tracking errors at the end effector.

⁹ | **Learned-output sensitivity:** A learned pose estimate can change substantially when the image or scene changes, and an empirical confidence score does not bound that change. The permission path limits the admitted target and motion using independently checked geometry, state, and actuator constraints.

Table 10.1: Grounding representations and admission questions: Outputs from different model families require different conversions before a physical permission check.

Proposal source	Representation sent downstream	Early check	Remaining evidence needed
2D box or mask	Image region plus measured depth and camera calibration	Reject absent depth or a target outside a validated region	3D pose error, occlusion, object identity, and swept-path clearance
3D feature or value field	Candidate target or cost field in a stated frame	Reject out-of-workspace targets and stale map cells	Conservative occupancy, unknown-space policy, actuator and timing bounds
Action-token VLA	Decoded action in its trained action range	Reject out-of-range or inadmissible proposals	State-dependent physical limit and independent permission; token bins provide no clearance proof
Language-selected skill	Skill identifier with estimated success	Rank or decline low-confidence candidates	Current preconditions, path feasibility, and separate physical enforcement

Discretized outputs carry no metric bound, and the two common kinds lack it for different reasons. RT-1 (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023) and OpenVLA (Kim et al. 2024)

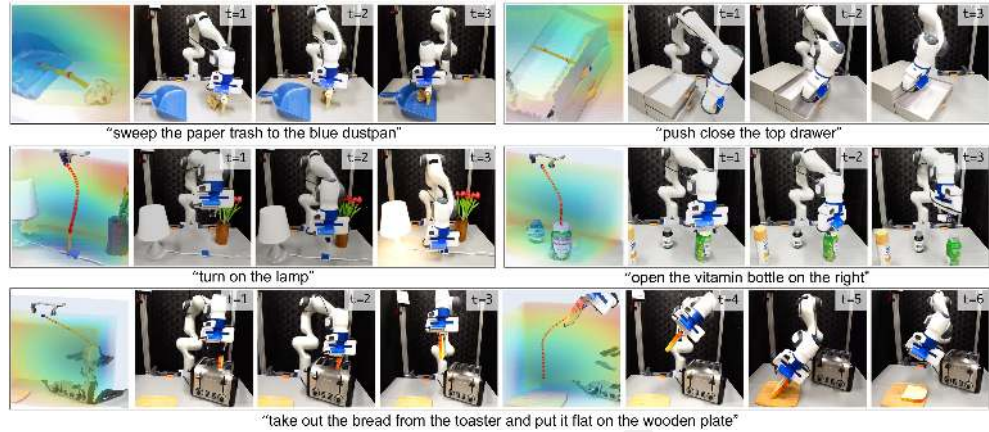


Figure 10.3: Manipulator 3D affordance grounding: VoxPoser composes language-guided 3D value maps for manipulation tasks (Huang et al. 2023). The illustrated research system proposes spatial objectives; an independent permission and stopping wrapper would require separate design and validation. (Source: Huang et al., 2023 (Huang et al. 2023), CC BY 4.0).

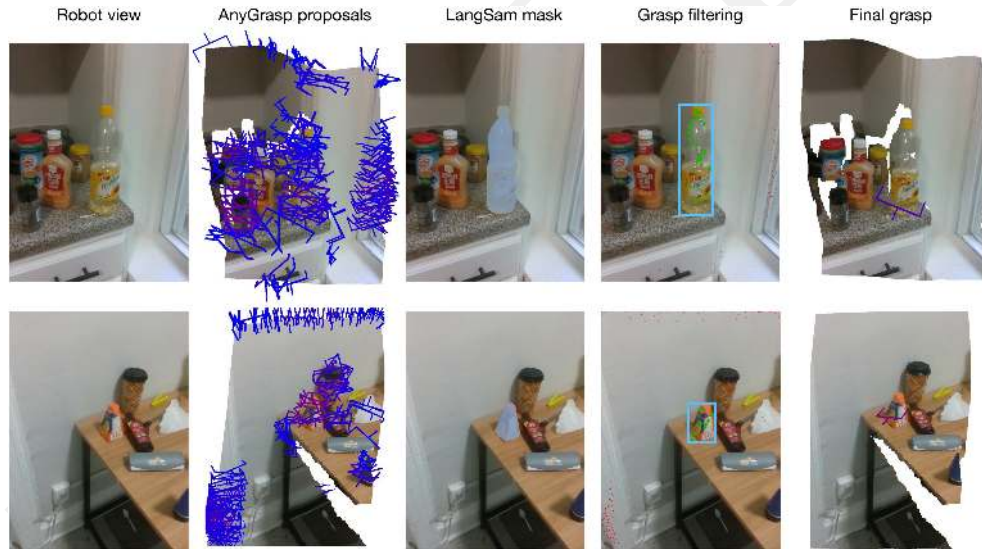


Figure 10.4: Open-vocabulary grasp grounding pipeline: OK-Robot combines candidate grasps with language-conditioned object selection and geometric filtering (Liu et al. 2024). The pictured research pipeline proposes a grasp; a bounded tolerance, evidence deadline, and independent permission check are an architectural wrapper developed in this chapter, not a reported feature of the image.

quantize each robot action dimension into 256 bins over a trained range, so a decoded bin is an action and encodes no clearance to any target. PaliGemma’s location tokens quantize normalized image coordinates, which become a base-frame pose only with measured depth, a calibration chain, and an error model whose extrinsic and depth terms can dominate the quantization itself.

To prevent a learned output from masquerading as a deterministic setpoint, the producer must retain the referent identity and the dated evidence that selected it. Its observation record may contain bounding boxes, attention weights, class probabilities, and a full oriented covariance estimate. The intent proposal (section 10.7) links to the belief record built from that observation (section 9.9) by digest and evidence epoch; it carries a candidate pose, conservative base-frame error bounds, task tolerances, and requested expiry, not those full producer-side fields. The independent permission path checks the linked evidence and request against its own limits. An object can shift or become occluded after camera exposure, so a coordinate without a valid evidence link and deadline cannot remain an authorized target.

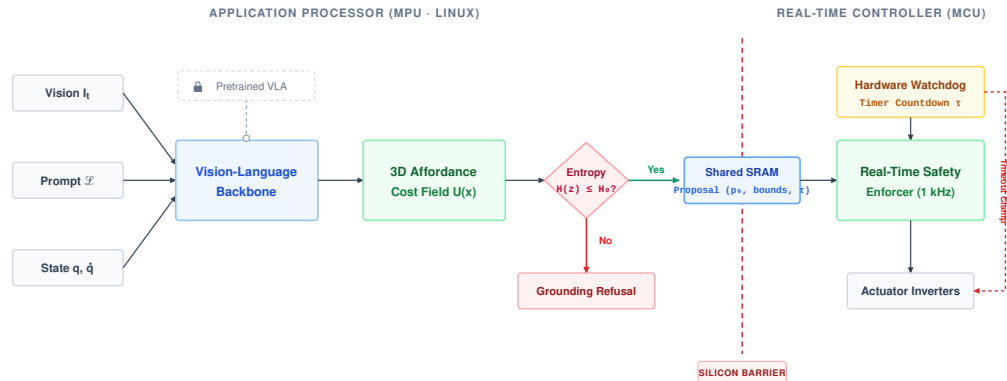


Figure 10.5: Grounded intent pipeline architecture: This proposed implementation turns multimodal inputs into a candidate spatial field, applies a producer-side entropy filter, and passes a bounded request across the proposal boundary from the application processor to the safety microcontroller (MCU). The MCU validates evidence age, configured limits, and feasible fallback before admitting a local countdown. Cross-attention and prompting supply proposals, not certified physical constraints (synthesized from Huang et al. (2023), Ahn et al. (2023), and Gray (1989)).

When a vision-language model sees multiple candidate objects, an argmax can hide an unresolved choice. A digital recommendation can still mislead a user, but it has no direct actuator path; in a robot, an admitted spatial proposal can move mass. If a red glass jar sits beside the red mug on the takeaway conveyor, a poorly grounded guess may select the jar. The system should preserve candidate identity and calibrated uncertainty, refusing a proposal when a task-specific ambiguity threshold is exceeded. The permission path then maintains a feasible hold or stop while the producer seeks a clearer observation or instruction.

Formally, **open-vocabulary grounding ambiguity** is the condition where multimodal feature projections yield multi-peak posterior distributions, overlapping geometric hypotheses, or unresolvable evidential ties across candidate physical referents, requiring an explicit grounding refusal (ERR_GROUNDING_AMBIGUITY) rather than optimistic argmax tie-breaking. A high entropy over the candidate-referent distribution $\mathbf{z}$, $H(\mathbf{z}) > H_{\text{thresh}}$, is one signal of it, and the producer-side filter of figure 10.5 can refuse on that signal before the proposal crosses the proposal boundary. The test that must hold for every proposal is an independent count of the candidates in the pick window (section 10.6), because a confidently wrong model can report low entropy.

Consider the grounding model’s estimate of the red mug’s handle from a single wrist-camera view taken at a distance, before the arm begins its approach. In an illustrative calibrated Gaussian localization model, the principal sensor-frame standard deviations are 4 mm, 5 mm, and 12 mm, the largest along the camera’s depth axis, which is also the gripper’s approach axis. A **95 percent statistical region** around the estimated handle position $\mathbf{p}_0$ uses Mahalanobis squared distance $(\mathbf{p} - \mathbf{p}_0)^T \mathbf{I}^{-1} (\mathbf{p} - \mathbf{p}_0) \leq \chi_{3,0.95}^2$, where the three-degree-of-freedom quantile is 7.815, yielding principal semi-axes 11.2 mm, 14.0 mm, and 33.5 mm. Figure 10.6 contrasts that uncertain target with the mug’s 15 mm grasp tolerance. Both lateral semi-axes fit inside the tolerance, but the depth semi-axis does not, so this estimate fails admission and needs better sensing, a different approach, or refusal.

The pick admitted in section 10.7 rests on better sensing. Its belief record (section 9.9) comes from a closer wrist-camera capture on the approach, which bounds the initial error at 3 mm. Even that admitted estimate’s 95 percent region leaves a residual tail and cannot by itself certify clearance, so the intent proposal carries conservative base-frame error bounds rather than claiming to encode this covariance.

A detection with a large uncertainty estimate can be rejected by a configured statistical gate. A confident grounding to the wrong physical entity can pass that gate. Suppose instead that a thin-walled glass jar arrives alone where the mug should be, and the vision-language model assigns it a high confidence score when asked for the red mug. The model emits a sharp, low-variance geometric proposal centered on the glass. The trajectory planner receives the coordinate, computes an $SE(3)$ inverse kinematics solution, verifies joint limits, and confirms that the path clears all



Figure 10.6: Localization uncertainty and authority expiry: Panel (a) sketches a statistical target-uncertainty ellipsoid beside the grasp's separate task tolerance. Panel (b) compares counterfactual traces: blue expires and brakes, green continues only with fresh renewals, and red shows motion without a lease. Whether the blue trace rests before the obstacle depends on total pre-brake delay, deceleration, and clearance.

obstacles in the static environment map. The permission path may find the proposed motion within its configured geometric and force bounds. If those bounds allow the grip force the ceramic mug needs, a smooth, geometrically valid trajectory can crush the glass despite passing the checks.

This failure reveals the boundary between what the permission path can verify and what it must accept as an unvalidated premise. The permission path checks coordinate bounds, kinematic reachability, joint torque limits, and geometric collisions against known occupancy grids. Those physical checks do not establish that the coordinate names the intended object, so the premise that the grasped object is the named object lies outside the permission path's reach. It needs its own task evidence, and it travels with the mug into the handover branch of the release case (section 16.6) and into the register of claims the machine's evidence leaves open (section 17.3).

Multimodal feature extraction, cross-attention alignment, and open-vocabulary phrase grounding belong to the computer vision and language literature (Kamath et al. 2021; Gu et al. 2022; Radford et al. 2021; S. Liu et al. 2023). A physical AI architecture consumes these grounding models without relying on their internal activations for safety, and even a filtered proposal stream still needs the independent motion-permission check. AutoRT's language-model task filter raised the fraction of acceptable proposed tasks yet still passed a small fraction of unsafe ones, and its deployment relied on joint-force pausing, emergency stops, and human supervision rather than on the prompt (10.1) (Ahn et al. 2024).



War Story 10.1: AutoRT task filtering and supervision (2024)

Context: AutoRT used language models to propose and filter tasks for mobile manipulators in office environments (Ahn et al. 2024). The authors collected 77,000 robot episodes across more than 20 robots; they did not report the mirror-grasp, shattered-object, or chassis-strike sequence previously attributed to this study.

Evidence: In a labeled sample of 259 proposed tasks, 228 were initially judged acceptable. After an LLM affordance filter, 200 of 214 retained tasks were acceptable, raising the acceptable fraction from about 88 to 93 percent while leaving some unsuitable tasks. The paper explicitly says constitutional prompting cannot guarantee compliance.

Response: The reported deployment supplemented prompts with joint-force pausing, physical emergency stops, and human line-of-sight supervision unless a work area was barricaded. These are distinct controls with different coverage; the prompt itself was not a certified geometric or physical safety supervisor.

Systems lesson: A learned task filter can improve the proposal stream, while an independent permission and fallback path must still validate each physical action for its stated operating region.

Whatever its source, a grounded target is also a claim about a moment. The red mug that the model selected correctly at capture keeps moving with the conveyor, so a proposal that was right when issued becomes wrong with no change to its contents. It must therefore carry a deadline that the permission path can enforce without the model.

10.4 Goals That Expire

Suppose the application processor hangs while the arm reaches for the red mug. The fault that makes cancellation necessary is the same fault that stops the cancellation from being sent, because an explicit abort requires a working sender, scheduler, and communication path. An intent lease must therefore expire on its own. This is principle 3 applied to a goal. The permission path owns a fallback it can execute without the proposer, so an independent countdown withdraws the stalled proposal's tracking authority without waiting for the model. This adapts the lease primitive of Gray and Cheriton (1989) to a physical controller: expiry triggers a separately validated fallback, whose ability to stop before a hazard depends on current state, clearance, actuator authority, and total response delay.¹⁰

The target-evidence deadline follows the rate at which the physical target can diverge from its last measurement. Suppose validated bounds cap relative target drift at v_{drift} and initial position error at e_0 within the belief record's declared validity region, and the task allows a target error of r_{tol} . The horizon is then equation 9.2 applied to equation 9.1 with $a_{\text{dist}} = 0$, writing e_0 for E_0 and r_{tol} for E_{max} :

$$\tau_{\text{ev}} \leq \frac{r_{\text{tol}} - e_0}{v_{\text{drift}}} \quad (10.1)$$

When $r_{\text{tol}} \leq e_0$, the target is inadmissible at capture. The controller refuses dependent motion. Even when equation 10.1 gives a positive horizon, a separate motion-permission check must verify current clearance against total sensing, permission, communication, and brake-onset delay plus physical braking distance, as in chapter 4.

If a validated disturbance acceleration bound a_{dist} is also needed, the quadratic term of equation 9.1 returns, and its positive root at r_{tol} shortens the evidence horizon.

For the red mug on the takeaway conveyor, the grounded target moves at 0.20 m/s. A 15 mm grasp tolerance and 3 mm initial error leave a 60 ms **target-evidence horizon** by equation 10.1. Stretching the horizon to the 200 ms refresh period of the machine's 5 Hz intent model would let modeled target error reach 43 mm, or 28 mm beyond the tolerance. The intent model, whose inference alone takes 160 ms, cannot by itself refresh a 60 ms target for continuous motion. Without a qualified tracker the conveyor must slow or pause, or the task must be refused. At 20 mm/s drift the same bound gives a 600 ms evidence horizon (figure 10.7).

A qualified local tracker changes the arithmetic. It re-measures the mug on every frame and follows the conveyor's velocity, so the conveyor speed no longer drives the error. The same 3 mm initial error is still charged against the tolerance, which leaves 12 mm for the residual slip of 0.50 m/s² to consume. Against that budget, residual slip alone sets the tracked evidence horizon that section 3.4 solves, 219 ms. The tracker renews that evidence at its own rate, so the tracker's period plus latency is what must fit inside the horizon. The intent model only re-confirms which mug is meant, and each renewal it sends carries the tracker's fresh evidence rather than the frame the model last read. Whichever horizon applies caps the intent lease on this target (section 10.7).

Coupling target-evidence validity to the reasoning model's cadence creates a drift hazard. Suppose a design ties the mug's evidence to the intent model's output rather than to a tracker, so the untracked 60 ms horizon applies, and a single inference runs to a high-tail latency of 650 ms. Extending the target record to 650 ms solely to hide that delay would leave its estimate outside the declared error envelope for 590 ms. The 60 ms limit is an evidence-validity deadline, and only new evidence, checked again against the motion, can extend it; faster compute cannot.

¹⁰ | **Hardware watchdog down-counters:** Detecting a missed renewal is only the first term of the response budget. Drive command propagation, brake engagement, and physical deceleration each add a separately measured term.

Prerequisite: Kinetic momentum budgets and stopping distance constraints originate in section 2.3.

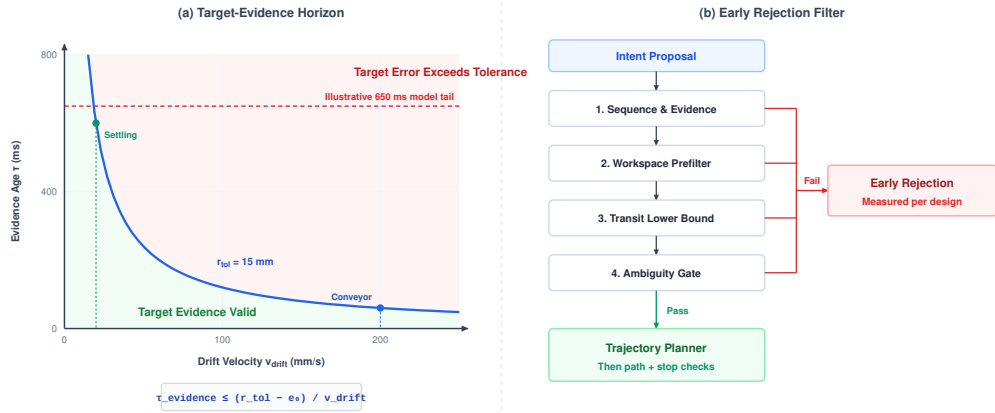


Figure 10.7: Target-evidence horizon and early filters: Panel (a) plots the bound $\tau_{ev} \leq (r_{tol} - e_0) / v_{drift}$; its green region means only that the target remains within the task tolerance. An illustrative 650 ms high-tail inference cannot renew the 60 ms conveyor case. Panel (b) shows early rejection stages that precede the full path and stopping checks.

Continuous tracking requires *fresh physical evidence*, not merely a new packet from the same stale frame. A renewal based on a new capture must arrive before the current evidence and admitted authority expire, with capture, processing, transfer, clock uncertainty, and permission time included in its budget. The simple condition $T_{period} + t_{latency} < \tau_{ev}$ is only a necessary scheduling check. When a fresh proposal arrives, the planner may compute a jerk-limited splice from the measured state. It admits that splice only if the swept path, new evidence, and stopping reserve remain feasible; an abrupt target change can instead require refusal or a checked stop.

When the countdown the MCU armed at admission runs out, whether or not the host is still running, the independent controller withdraws the proposal’s authority and selects a previously validated response for the measured load and contact mode. An active position hold needs torque and can protect a suspended payload; controlled deceleration needs brake capability and clearance; torque disable can be suitable for some drives but can release a load or leave momentum uncontrolled. The proposer may request a mode, but it cannot choose the terminal action for the MCU. Admission fails when no response remains feasible from the current state.

Executing an expiring lease safely assumes that the commanded geometric proposal is actually achievable by the physical body. If a proposed coordinate lies beyond kinematic reach or demands joint velocities that exceed actuator limits during the lease window, granting the lease wastes control authority and can trip the drive’s overcurrent protection as a motor draws excessive current toward an impossible speed. Determining whether a goal is kinematically and dynamically admissible belongs before the trajectory generator accepts the lease, establishing a physical filter between high-level intent and motion execution.

10.5 Admissible Goal Sets

If the red mug rides past on the far side of the belt, beyond the arm’s reach from where the base is parked, no sequence of joint torques can bring the gripper to its handle. That impossibility should be caught at the proposal boundary, before the target enters the trajectory optimization queue. When an upstream vision-language model or neural policy emits a coordinate, treating that proposal as an unverified candidate protects the real-time motion pipeline from executing or even evaluating impossible motions. Kinematic workspace analysis, imported from classical formulations (Craig 2005; Lynch and Park 2017), defines the closed spatial manifold of reachable end-effector configurations (the physical 3D volume the robot arm can reach) given link dimensions, joint range limits, and mounting geometry.¹¹ An analytical reachability check evaluates whether the target $SE(3)$ pose $\mathbf{p}_{target}$ resides inside this workspace manifold $\mathcal{W}$, while an occupancy check verifies that the bounding volume of the target does not intersect known persistent obstacles, and both are cheap enough to run before trajectory optimization (figure 10.7, panel b). Passing an out-of-reach goal

¹¹ | **Kinematic workspace boundaries and singularities:** The kinematic workspace defines the continuous manifold of task-space poses attainable by the manipulator given link geometry and joint travel limits. Near workspace boundaries and internal singular configurations, the kinematic Jacobian $\mathbf{J}(\mathbf{q})$ loses rank, requiring infinite joint velocities to maintain finite Cartesian end-effector motion. Enforcing workspace manifold filters at the proposal boundary prevents trajectory optimizers from driving actuators into kinematic lockup.

to a trajectory optimizer forces the solver to search an empty solution space, converting geometric impossibility into computing failure. Passing these cheap tests is necessary but never sufficient, because joint orientation, swept obstacles, torque, thermal state, unknown space, and stopping clearance still require a checked path and a permission decision.

A target inside the static workspace may still be out of reach within the time its evidence stays valid. **Dynamic reachability** is the condition that the end effector can move from its measured state $(\mathbf{p}_0, \mathbf{v}_0)$ to the target pose within the remaining window $t_{\text{exp}} - t_{\text{now}}$ while respecting its velocity, acceleration, jerk, and thermal torque limits. Whatever profile the actuators follow, no move covers a distance d faster than its speed ceiling allows, so admission requires at least

$$\frac{d}{v_{\text{max}}} \leq t_{\text{exp}} - t_{\text{now}}.$$

Acceleration and jerk limits only lengthen the transit, so a proposal that fails this inequality fails every fuller test. Section B.4 derives the triangular, trapezoidal, and jerk-limited profiles those limits produce.

On the arm, the mug's evidence under the qualified tracker stays valid for 219 ms, and at the 1 m/s tool-center-point speed limit the gripper covers at most 219 mm in that window. A proposal issued with the gripper 300 mm from the handle needs at least 300 ms of transit, so the MCU rejects it with `ERR_DYNAMIC_TIME_INSUFFICIENT` before any path is solved. Without the tracker the window shrinks to 60 ms and the reach to 60 mm, so an untracked reach can close only the last few centimeters of an approach. Passing the inequality only permits the fuller path, torque, and stopping check.

This bound and the fuller checks behind it rest on imported rigid-body mechanics under three assumptions, namely that actuator torque-speed curves remain within nominal thermal ratings, joint encoders accurately report the initial state $(\mathbf{p}_0, \mathbf{v}_0)$, and the payload mass matches the inertial model behind the actuator limits. A representative permission-path implementation monitors joint current, motor temperature, and communication delay at rates chosen from the plant and sensor budgets. These observations can detect some departures from the model but cannot verify every assumption, such as payload inertia, on every tick.

If motor winding temperatures exceed the thermal thresholds of section 2.5, the permission path imposes thermal derating, scaling down the maximum allowed acceleration (a_{max}) and velocity (v_{max}) in the parameter registry. An external force measurement that indicates a payload mass deviation lowers the same limits. The intent filter immediately evaluates subsequent incoming leases against these reduced physical limits. A derating event can make a previously admitted path infeasible; the controller must recheck it and select a feasible fallback if the margin is lost.

When the reachability filter rejects an intent proposal, it must not simply discard the packet. Discarding without feedback leaves the upstream reasoning model unaware of why motion failed to occur, forcing it to wait for an end-to-end task timeout. The intent filter generates an immediate, structured diagnostic packet returned directly to the reasoning layer across the proposal boundary. This diagnostic payload contains the rejection classification, such as a kinematic workspace violation (`ERR_WORKSPACE_OUT_OF_BOUNDS`), dynamic time insufficiency (`ERR_DYNAMIC_TIME_INSUFFICIENT`), or swept-volume collision intersection (`ERR_OCCUPANCY_BLOCKED`), along with the numerical margins of failure. For a dynamic reachability failure, the feedback returns the shortfall between the transit lower bound and the remaining window, and the maximum reachable distance along the commanded vector within the allotted lease. For a static workspace violation, the feedback provides the projection of the target onto the boundary of $\mathcal{W}$ (the closest physical point the arm can actually reach). The reasoning layer uses this structured signal to adjust its proposal on the subsequent inference cycle, by obtaining fresher evidence, choosing an alternate approach, slowing or pausing the process, or proposing base relocation. It cannot extend the evidence horizon without a new justification.

Reachability filters remove targets the body cannot reach, or cannot reach in time, and leave the complete motion to the planner and the permission path.

10.6 How Intent Goes Wrong

When a vision-language model outputs an $SE(3)$ target pose for a physical manipulator, that pose can fail in several distinct ways. It may be a hallucinated target generated from visual attention

Downstream: Admissible goal regions specify the boundary conditions for spline generation in section 11.1.

patterns on reflective background textures. It may be a stale target representing an object that a human coworker bumped or displaced moments earlier while the camera pipeline was buffering frames. It may be a reference ambiguity where the model arbitrarily selects one of two identical red mugs side by side on the conveyor without resolving which one the task requires. Or it may stem from an out-of-distribution scene shift, such as unexpected lighting or dust on an optical lens, that degrades the spatial prediction accuracy of the network. Despite their different causes, these failures arrive downstream in the same form.

The planner receives a dated target proposal, not an actuator command. A pose on an empty location may lead to a futile reach; a pose inside an occupied fixture may lead to contact if the later path and permission checks fail. In the wrong-object case, by contrast, the geometry can be valid while the selected object is semantically wrong. The independent permission path therefore checks what it can measure (freshness, coverage, geometry, current state, and actuator limits) without claiming to verify the task's meaning.

A fresh depth observation can reject a proposed grasp whose required object evidence is absent in a covered, visible region. No return in an occluded or unobserved region leaves the region unknown. A task-specific ambiguity test runs on every proposal, confident or not, because a high score describes the model's certainty and cannot show that no second candidate shares the pick window. The test refuses a window that holds two candidates, and the item goes to the coworker for a manual pick. The permission path runs it on its side of the proposal boundary, as the ambiguity gate of figure 10.7 (panel b), on evidence the grounding model does not produce; the producer's entropy filter can refuse earlier but cannot replace it.

Table 10.2 groups these failures by the check that catches them, adds two that the model's output cannot reveal until contact or timing exposes them (a misclassified affordance and a target unreachable within its window), and separates admission checks from responses after motion or contact has begun. At the cage-door latch of 10.1, the 2 ms contact-loop response already leaves 39 N at brake-command onset, and the peak comes later, so precontact speed and force limits, actuator response, compliance, and available clearance determine whether the full contact trace stays within the validated contact limits.

Table 10.2: Grounding failure modes and their evidence limits: Each check addresses a different failure class. The table separates proposal rejection from a response to an already moving plant.

Failure mode	Evidence or check	Admission or response	Physical limit to verify
Phantom target: image artifact in empty space	Fresh depth evidence and target-region occupancy, with coverage and occlusion checked	Reject the proposal when the required object evidence is absent; request another view if the region is unknown	Rejection precedes motion only if the proposal has not already been admitted. Missing returns alone do not prove empty space.
Stale target: workpiece moved after capture	Converted capture time, bounded clock error, local tracking, and target-error envelope	Revoke dependent tracking when the evidence limit fails; select the validated fallback	Total detection, communication, actuation, and braking delay must fit the measured clearance.
Semantic ambiguity: two candidate objects in the pick window	Independent count of candidates in the pick window from fresh depth evidence; model entropy as an earlier producer-side signal	Refuse admission and route the item to the coworker for a manual pick	A single wrong object passes the count, and a confident model can report low entropy; independent physical checks remain necessary.
Misclassified affordance: bolted fixture selected as graspable	Contact-force and motion residuals after contact; precontact task/geometry checks where possible	Limit precontact speed and force, then invoke the validated contact response on a threshold crossing	A reactive trip does not cap the first impact peak; test the full force trace, response delay, and stopping energy.
Dynamic unreachability: target cannot be reached in its valid window	Speed-ceiling transit lower bound followed by a complete path and stopping check	Reject a proposal whose lower bound exceeds the remaining evidence window	Passing the lower bound proves neither collision-free motion nor a feasible stop.

Every row of the table therefore requires the proposal to carry its spatial bounds, the provenance of its evidence, and its expiry, in a form the permission path can check without the model. A rejected proposal returns a reason to the learned layer; an admitted proposal remains subject to independent permission checks as the plant moves. The proposal carries those fields across the proposal boundary, and the record the permission path grants from it is the intent lease.

10.7 The Intent Lease

The intent lease takes its lineage from memory. Its parent is the belief record of section 9.9, named by digest, and that record’s evidence epoch, converted to the MCU’s clock, becomes the lease’s evidence time. The lease itself carries a goal region, an error envelope and task tolerance, a requested wrench, a requested terminal mode, an evidence horizon, and an expiry. It is a lease in the sense of section 4.3, here on a target, and the proposal that requests it travels under the proposal header with an intent payload in place of a chunk. An admitted intent grants a target and effort ceilings, never a path.

On the running machine the proposal travels from the application processor to the MCU as a fixed-size, statically parsed message in a bounded shared mailbox,¹² so its parse time can be measured against the control deadline. The message only transports a request; the independent permission controller owns the configured force ceilings, deadline ceilings, and state-dependent fallback.

The covariance ellipsoid of the mug-handle estimate in section 10.3 is a producer-side localization estimate. The proposal does not try to encode a general oriented 3×3 covariance; its bounds carry three conservative **position-error envelopes** (e_x, e_y, e_z) and three requested **task tolerances** (r_x, r_y, r_z) in the named robot-base frame instead. Envelope and tolerance keep the distinction drawn in section 10.2, so admission requires $0 \leq e_i \leq r_i$ on each axis. The producer must justify any conversion from covariance to an envelope, including calibration bias and declared tail risk. Orientation tolerances and contact-mode rules come from an independently approved task profile; the proposal carries only the target orientation itself.

The proposal’s evidence time is the **belief record’s evidence epoch**, never its estimate epoch, converted to the MCU’s clock under the clock rule of section 4.3. The MCU matches both the parent digest and that epoch to a retained belief record and adds the mapping’s error bound to its age test. The requested expiry uses the same clock, so the MCU can compare it directly with the evidence and stopping deadlines that bound it. The parent digest is **lineage, not message authentication**,

Byte layout: The intent payload’s fields, types, and offsets are given in section C.14.2.

¹² | **Inter-core communication across the proposal boundary:** RPMSG over a statically bounded shared SRAM ring is one transport. Copy count, cache maintenance, coherency, and worst-case handoff time depend on the SoC and driver, and the MCU’s independent timer detects lost renewals even through a host kernel panic. See section C.8.

under the integrity policy of section 4.3. It does not prove who chose the target, limits, or requested terminal mode, and the intent proposal carries no keyed tag because the permission path admits it by content, not origin.

The admitted intent moves through Pending, Active, Renewed, Expired, and Aborted states. The MCU enters Active only after checking the request against independently configured ceilings and a feasible response from the measured state. A renewal needs a newer sequence, a fresh belief record, and a new permission check; it does not automatically extend authority. Meeting the approved task tolerance completes the goal. Unexpected contact or loss of a required belief aborts the dependent motion.



Definition 10.2: Expiring intent lease

Expiring intent lease ($\mathcal{L}_{\text{intent}}$) is a time-bounded execution authorization issued by the independent permission path that binds an admitted target pose and effort ceilings to a monotonic hardware deadline t_{exp} :

$$t_{\text{exp}} = \min(t_{\text{evidence}} + \tau_{\text{ev}}, t_{\text{issue}} + \tau_{\text{lease}}, t_{\text{stop_deadline}})$$

upon whose expiration motion tracking authority is automatically revoked without requiring an explicit cancellation packet.

1. **Significance:** Withdraws tracking authority on a hardware deadline when upstream deliberation stalls, so the validated fallback starts without waiting for the proposer.
2. **Distinction:** Unlike a network keepalive or software ping, an intent lease couples the horizon of physical sensor evidence (τ_{ev}) with dynamic braking feasibility, and its expiry falls no later than the stop deadline at which a validated fallback still fits the remaining clearance.
3. **Common pitfall:** Sizing lease durations generously to mask tail latency (P_{99}) in neural inference, which creates an unmonitored open-loop window where the robot moves through an environment that may have changed.

At expiration, the timer revokes tracking authority and the MCU starts whichever of its approved hold, controlled-deceleration, and torque-disable responses the rule of section 10.4 selects for the measured load and contact mode. The requested terminal mode is advisory, so a request for torque disable over a suspended payload cannot override the powered hold that such a load requires. Neither a 1 kHz check period nor a hardware interrupt establishes a physical stop within one period.

The proposal's fields support layered checks, cheapest first, and one implementation orders them in this bounded admission sequence:

1. **Evidence and sequence:** Read a complete proposal packet. Reject an old sequence or a parent digest and evidence epoch that do not match a retained belief record. Compute evidence age in the MCU monotonic domain and add the bounded clock-conversion error. Reject when that age exceeds the target-evidence horizon.
2. **Proposal limits:** Check finite pose values, quaternion normalization, base-frame error and task-tolerance axes, and requested wrench against approved task and actuator limits. Check known occupancy and unknown-space policy.
3. **Dynamic admission:** Use current velocity, clearance, and actuator state to reject proposals that already fail a lower bound on transit time. Require that the total sensing-to-brake-onset travel plus braking distance fit inside the current clearance before granting motion. If no feasible fallback exists from this state, refuse the proposal. Refuse also when fresh evidence shows more than one candidate in the pick window, and route the item to the coworker for a manual pick. This stage admits a target and effort ceilings, never a path; admission of the whole trajectory belongs to the planner's record (section 11.5).
4. **Authority latch:** Store only the MCU-approved target, effort ceilings, expiry no later than the evidence deadline, and selected fallback. Arm the local timer for the admitted interval. The untrusted requested expiry and terminal mode do not control the drive directly.

5. **Runtime check:** On each configured control tick and relevant hardware fault event, compare measured state with the admitted limits. On evidence expiry, missed renewal, or an observed violation, revoke the proposal and start the previously validated response. The ensuing braking and hold have their own measured delay and physical margin.

Filled in for the red mug, the lease admitted for the pick carries six entries. Its parent is the mug-handle belief record of section 9.9, matched by digest and evidence epoch. Its goal is the handle pose, and its error envelope is that record's 3 mm initial error, inside a 15 mm task tolerance. Its evidence horizon is the age at which the governing growth law reaches that tolerance, 60 ms under the record's belt-drift law and 219 ms under the residual-slip law of the qualified tracker's renewed record. Its speed ceiling is the arm's 1 m/s tool-center-point speed limit, and it requests no item-specific wrench ceiling, because none is registered for the mug. The configured task and actuator ceilings still bound the wrench, and fragility is handled by the site's control of which items reach the conveyor, not by the controller. Its requested terminal mode is grasp, then hold for the handover, although the MCU still selects the fallback from its approved responses. Its expiry is the earliest of that evidence horizon, the configured lease ceiling, and the stop deadline.

This record gives chapter 15 a concrete interface for fault-injection tests in which corrupted quaternions, stale capture records, oversize force requests, and incompatible terminal requests should each be rejected under the tested state and timing conditions.

Checkpoint 10.1: Expiring intent and fallback authority

Verify your understanding of how a proposal is refused or admitted, and how the lease granted from it is sized and revoked:

- ☐ **Ambiguity refusal:** Why must a pick window holding two candidates refuse the proposal, and why must that test run on a confident proposal as well as an uncertain one?
- ☐ **Wrong-object premise:** Which premise about the grasped object remains outside the permission path even when every coordinate, reachability, and force check passes, and where does the machine carry that premise instead?
- ☐ **Evidence horizon:** Can you explain why the evidence horizon of an intent lease is bounded by task tolerance and target drift rather than by the reasoning model's period, and why inflating it to hide tail inference latency leaves motion on an obsolete target?
- ☐ **Clock domain:** Why must the MCU convert the belief record's evidence epoch into its own monotonic clock, with a bounded conversion error, before checking age and the requested expiry?
- ☐ **Fallback selection:** Can you explain why the MCU must select a feasible approved hold, deceleration, or torque-disable response from the measured load state rather than obey the proposal's requested terminal mode?

10.8 Fallacies and Pitfalls

Each mistake in this section trusts an upstream message to mean more than it does. Local checks must refuse a target, or withdraw tracking authority over one, when its evidence expires or its physical justification dissolves, and they must decide on evidence the proposer does not control, even while its messages remain well formed and confident.

Fallacy: *A renewal computed from the same stale frame is a valid renewal.*

The machine's intent model re-reads its last wrist-camera frame of the red mug and sends a renewal with a newer sequence number, a later issue time, and the same pose. Every field a keepalive would check has advanced, yet no image has been captured since the evidence that justified the target, and the mug has kept moving at 0.20 m/s. Because the lease dates its evidence by the belief record's evidence epoch, not by the renewal's issue time, the MCU matches the renewal's parent digest and epoch to the retained record, finds the epoch unchanged, and keeps counting age from the original capture. The target still expires 60 ms after that capture. A controller that dated evidence by message

arrival would let a stream of well-formed renewals hold an obsolete target indefinitely, turning the drift hazard of section 10.4 from a single late inference into a standing condition.

Pitfall: *Allowing an unexpired temporal lease to continue commanding motion when target spatial bounds are violated.*

The machine's arm lowers its gripper toward the red mug under an intent with the 15 mm grasp tolerance. A slip on the takeaway conveyor carries the mug beyond that tolerance while time still remains on the lease. The timestamp still permits execution, but the target has left the $SE(3)$ region that justified the approach. Continuing until expiry would preserve an obsolete, potentially colliding command. The local perception and permission loops must compare the observed target with the intent's spatial bounds and withdraw permission when those bounds fail. Encoding that geometric condition in the lease allows the controller to interrupt the descent without waiting for a new high-level plan.

Fallacy: *A confident grounding needs no refusal path.*

A confidence score cannot certify that the scene holds only one candidate. A design that sends only low-confidence proposals to the ambiguity test treats the model's score as evidence about the scene, when it is the model's report about itself. The glass jar beside the red mug in section 10.3 arrives with a high score and a tight geometric envelope, so a score-gated design skips the test exactly where the test was needed. The refusal path must run on every proposal and must draw on evidence the model does not produce. A check that the pick window holds exactly one object refuses the jar-beside-mug scene whatever the score. What such a check cannot see, a lone jar arriving where the mug should be, remains a premise the permission path names rather than settles, and the site's feed procedure, which admits to the conveyor only items on the station's declared list, is what contains it.

Fallacy: *Neural networks can resolve bimodal spatial ambiguity by averaging candidate target locations.*

The machine sees two identical red mugs set a few centimeters apart on the takeaway conveyor and receives their spatial mean as a grasp target. The midpoint lies in empty space, so averaging has destroyed the $SE(3)$ distinction the task requires. Both mugs are real, so the innovation check of section 9.6 rejects neither measurement before fusion. Each hypothesis alone is sharp, and a gate applied to either one passes. The window check of section 10.6 still catches the case, because it counts two candidates in the pick window and the admission gate refuses the proposal and maintains a hold, whatever the fused estimate reports. A covariance gate would catch it only for an estimator that reports the bimodal ambiguity, which an averaging estimator does not. The intent interface must preserve statistical uncertainty rather than collapse competing object hypotheses into an apparently precise Cartesian coordinate, because only a representation that keeps both hypotheses lets a check other than the window count see the ambiguity.

10.9 Summary

Grounded intent meets a request whose meaning the permission path cannot check, and it keeps the third law, that proposal is not permission, because the path grants only what it can check. A learned model proposes a dated target and an uncertainty description. The record the permission path grants from that proposal is the intent lease, which takes its evidence epoch and parent from the belief record of chapter 9 and grants a target and effort ceilings with an expiry no later than the evidence horizon that target drift and task tolerance set. An independent countdown then withdraws authority that a stalled model could never cancel. The independent permission path checks the target's evidence age, configured motion bounds, current state, path clearance, and feasible fallback before applying motion. The target-evidence horizon $\tau_{ev} \leq (r_{tol} - e_0)/v_{drift}$ limits how long the target estimate may be used under its stated drift bound; it grants no time to stop, which a separate total-delay and braking check must find within the available physical clearance.

The red mug on the conveyor makes the cadence constraint concrete. Its untracked evidence lasts 60 ms, and the machine's 5 Hz intent model, refreshing every 200 ms, cannot maintain that target by itself. A qualified local tracker can refresh it from new evidence and lengthen the horizon to 219 ms, leaving the intent model only to re-confirm which mug is meant; otherwise the machine slows, pauses, or refuses the task. Expiry revokes the proposal's tracking authority and invokes a response validated for the measured state. It cannot undo motion already executed.

What grounded intent adds to the law is evidence age on a moving target, and refusal as the permission path's answer to what it can detect but not resolve. A target the arm cannot reach inside its evidence window fails the transit lower bound before any path is solved, a pick window holding two candidates fails the ambiguity test, and each rejection returns a reason the proposer can act on. The wrong-object case marks the limit of that machinery. A confident grounding to the wrong entity passes every geometric and timing check, so the premise that the grasped object is the named one stays open rather than being settled by the controller. After contact, a force trip bounds later loading only through the measured response, and precontact limits are what bound the first impact.



Key Takeaways: Grounded intent at the proposal boundary

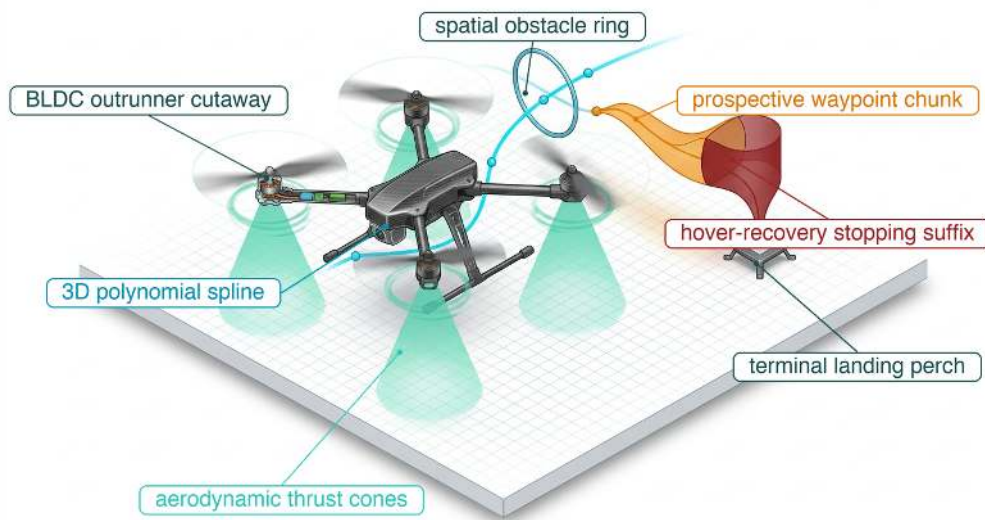
- **A lease grants a target, never a path:** The Brain supplies a target, evidence epoch, error envelope, task tolerances, and requested expiry, and it writes no actuator setpoints. The permission path admits a target and effort ceilings that expire at the earliest of the evidence horizon, the lease ceiling, and the stop deadline.
- **Evidence expires with drift:** Target drift and task tolerance set a finite validity horizon under a declared validity region. A fresh timestamp does not make a target correct, and only fresh evidence, never a faster model, renews it.
- **The evidence deadline is not the motion deadline:** The permission path separately budgets sensing, communication, enforcement, actuator delay, and braking distance for the current state and clearance.
- **Geometry cannot verify meaning:** A confident grounding to the wrong object can pass every coordinate, reachability, and force check. That premise needs its own task evidence, because the permission path cannot settle it.
- **Refusal is an admissible outcome:** Ambiguous identity, stale evidence, insufficient reachability, or infeasible stopping can each reject a proposal, and a structured rejection tells the proposer what to change. Each check has its own evidence limit.



What's Next: From intent leases to trajectory planning

An admitted lease names a dated target, but the trajectory that reaches it arrives in chunks, and the next chunk is still in flight while the current one executes. Chapter 11 takes the lease as its input and asks what the machine does when that next chunk is late, which requires every committed trajectory to keep the admitted target reachable within feasible jerk, torque, and clearance, with a stop that remains executable from every state it can reach.

Trajectory Planning



- 11.1 Continuous Trajectories
- 11.2 What Planning Hands Over
- 11.3 From Goal to Trajectory
- 11.4 Kinodynamic Feasibility
- 11.5 Behavior at the Seam
- 11.6 How Plans Go Wrong
- 11.7 The Trajectory Contract
- 11.8 Fallacies and Pitfalls
- 11.9 Summary

Purpose

What does a physical machine do during the unbudgeted milliseconds when the next neural trajectory chunk is late?

A moving physical machine carries continuous momentum while high-capacity neural policies compute upcoming trajectory chunks. If an executing action sequence terminates before its successor arrives, coasting blindly consumes the clearance required for a safe stop. Conversely, abruptly splicing an unconstrained proposal into an ongoing trajectory demands instantaneous joint accelerations that violate actuator torque limits or trigger destructive structural resonance across the mechanical transmission.

Trajectory planning resolves this computational-physical mismatch by transforming discrete model inferences into continuous, kinematically feasible motion profiles. Every dispatched trajectory chunk must guarantee C^2 continuity across chunk seams and carry an invariant, pre-verified stopping suffix. This stopping continuation ensures the real-time execution layer can arrest motion safely within current clearance bounds if downstream computation encounters tail latency or hardware faults. In the physical AI stack, planning reconciles the Brain's asynchronous, variable-latency inference with the Nervous System's hard real-time execution cadence.

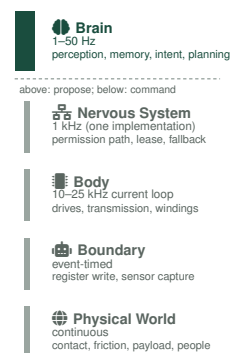


Figure 11.1.: The five-level machine stack with the Brain level highlighted.



Learning Objectives

- Select trajectory horizons from replacement latency, target-evidence age, and the time the stop needs
- Evaluate trajectory feasibility and seam continuity against the body's actuator and transmission limits
- Distinguish trajectory failures detectable before execution from those requiring monitoring during physical motion
- Calculate the distance a matched stop suffix adds to the machine's stopping budget and compare it with the clear distance
- Specify the commitment deadline and matched stop that govern a replacement trajectory arriving late or never
- Construct a trajectory record that the permission path can admit by content, from its parent lease, deadlines, and stop

11.1 Continuous Trajectories

THE arm of the warehouse mobile manipulator, reaching for a mug on a moving conveyor, possesses physical mass, structural inertia $\mathbf{M}(\mathbf{q})$, and continuous kinetic energy $E_k = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$. It cannot change velocity or direction instantly. Even when upstream autonomy grants a time-bounded intent lease, authorizing motion toward a grounded semantic goal, that semantic permission alone cannot turn motor shafts. The machine must synthesize continuous nominal motion while target evidence remains valid, plus a separately authorized stopping continuation if tracking authority ends. On this machine the arm's joint tracking loop evaluates feedback and setpoints at 1 kHz (datasheet; see the Reader Guide); that period does not specify the physical stopping response. Above this loop sits the Brain, where modern robot learning policies, such as Action Chunking with Transformers (ACT) (Zhao et al. 2023) or score-based Diffusion Policies (Chi et al. 2024), process multimodal observations to synthesize future reference paths on the application processor.¹ Inference latency varies with workload and contention. The machine's chunk policy is budgeted at an illustrative 40 ms per pass at P99, vision included, yet memory or scheduling interference can stretch a single pass beyond 150 ms, so the horizon is sized from the deployed tail rather than from the budget.

If an architecture predicts actions one step ahead ($N = 1$), the motor tracking loop starves. If it generates a multi-step action chunk of $N > 1$ setpoints across a finite future horizon, the machine must cross an **execution seam**, the junction where one chunk's setpoints give way to the next. When an active trajectory chunk finishes executing before its replacement clears the tail of the inference distribution, the controller faces an execution discontinuity. If the controller clamps to the final position, the reference velocity steps to zero and asks the drive to remove the arm's momentum within one servo tick. If a newly arrived chunk begins with a velocity or acceleration that diverges from the physical state of the moving mechanism, the commanded torque steps and excites the structure.



Definition 11.1: Kinodynamic feasibility envelope

Kinodynamic feasibility envelope is the subset of state space from which a physical machine can track a candidate trajectory while strictly satisfying simultaneous actuator torque limits, velocity ceilings, and jerk bounds, with position, velocity, and acceleration continuous across segment seams, without exceeding mechanical stopping margins.

1. **Significance:** Geometric collision freedom is necessary but insufficient for physical execution; an arm may follow a collision-free path that nonetheless demands motor torques exceeding continuous thermal ratings or produces accelerations that tip an underactuated base. The envelope states the conditions under which the model predicts trackable motion; runtime sensing and actuator checks must still validate them.

¹ | **Action chunking representations:** ACT and Diffusion Policy chunks carry no boundary-derivative constraints, so the planner must propose a transition bridge across each replanning seam, and the permission path checks it before admission.

Prerequisite: Kinetic energy (E_k) and joint torque limits are established in section 2.3.

2. **Distinction:** Unlike pure kinematic reachability (which evaluates whether the robot's linkages can attain a spatial configuration), kinodynamic feasibility incorporates time parameterization, actuator force and torque limits, reflected inertia, and mechanical braking runways.
3. **Common pitfall:** Checking feasibility only at discrete waypoints. Interpolation splines between sample points can produce intra-sample velocity overshoots and torque spikes that breach motor limits even when every individual waypoint passes validation.

Intent defines the admissible goal; planning supplies the continuous motion, temporal parameterization, and dynamic limits needed to pursue that goal. Across the proposal boundary (section 1.5), a plan is not a command written to actuator registers. It is an unprivileged proposal, sent from the application processor to the safety microcontroller (MCU), where the tracking controller follows it and the permission path (chapter 12) checks it.² Because inference latency has a long tail, the downstream controller in this design consumes admitted references on its own clock and cannot suspend physical motion while the model finishes. A plan must therefore be self-contained. It carries its own seam state, continuous through acceleration (C^2),³ its own deadlines, and its own stop, a stopping suffix admitted with the nominal motion so that the stopping continuation exists before it is needed.

Momentum, a long inference tail, and a seam that must stay continuous together decide what a plan has to contain. The first decision is therefore what the planner hands to the tracking controller and the permission path while the arm is still moving.

11.2 What Planning Hands Over

For a two-joint arm, one pair of joint angles (q_1, q_2) is one point in **configuration space** $\mathcal{C}$. An obstacle excludes some angle pairs, leaving $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$. A curve through that free set says where the arm may move, but not how quickly it may move or whether its motors can supply the required torque (Lozano-Pérez 1983). A **trajectory** assigns time to the curve and exposes $\mathbf{x}(t) = [\mathbf{q}(t)^T, \dot{\mathbf{q}}(t)^T, \ddot{\mathbf{q}}(t)^T]^T$. The planner proposes that time-indexed reference, and the permission path checks it before any setpoint reaches the drive. A geometric path alone supplies neither timing nor fallback clearance.

What the planner hands over is a **trajectory record**, the immutable proposal that the tracking controller follows and the permission path admits. It names its coordinate frame $\mathcal{F}$, evidence time t_0 , initial reference state, and four deadlines in the **MCU monotonic clock domain**: nominal tracking begins at t_{start} , the nominal setpoints end at $t_{\text{plan_exp}}$, replacement closes at t_{commit} , and the admitted stop finishes at $t_{\text{stop_end}}$. It also links to the active intent expiry $t_{\text{intent_exp}}$. Conversion from the sensor and host clocks carries a bounded error. The planner schedules each replacement against an earlier target, the blend deadline t_{blend} , by which the successor should be admitted and at which the arm's obstacle clearance is last checked; it is the schedule's target, not a field of the record.

The permission path permits nominal **TRACK** only while the intent and plan are valid, so the intent's expiry ends TRACK at once, whatever the record's own deadlines allow. A second clock also bounds TRACK. The chunk policy proposes base velocity and arm joint targets together, so one chunk lease (section 4.3) governs both bodies, and only an admitted chunk renews it, every 50 ms on this machine. When the lease lapses 60 ms after the last admission, each body enters its own stop, the base its resident stop and the arm its state-matched fallback, a separately validated stop that solves its matched suffix's C^2 profile again from the measured state. The arm needs the separate fallback because its matched stop suffix is anchored at the position the arm will occupy at t_{commit} . An intent expiry or a fault that invalidates the path before t_{commit} sends the arm to that same fallback.

After a lapse, the arm's stop begins one permission tick, one bus cycle, and one brake onset later; together with the lease itself, these delays make up the renewal-to-onset delay of section 4.3. Commitment caps the lease, because stop onset must come by t_{commit} , so no lease may run longer than t_{commit} less those three delays. Only a lease run near that ceiling lets a record reach t_{commit} without a successor; such a record transfers to **STOP** on its matched stop suffix, which may finish after the intent's tracking permission expires. On this machine the 60 ms lease lapses long before

² | **Deterministic proposal verification:** The permission path isolates unprivileged trajectory proposals within hardware-managed staging buffers and admits one only after checking it there (section 11.5), so no unchecked setpoint reaches the motor control registers. For runtime safety filter architectures, see section 12.1.

³ | **C^2 derivative continuity:** C^2 continuity means that position, velocity, and acceleration remain unbroken functions of time across trajectory boundaries. In permanent-magnet synchronous motors (PMSM), electromagnetic torque couples directly to quadrature stator current ($\tau = K_t I_q$). An acceleration step asks for a finite torque and current step under this model, while a velocity step implies an ideal acceleration impulse.

Prerequisite: Expiring intent leases provide the bounded time windows discharged by trajectories in section 10.7.

commitment, so a stalled proposer meets the renewal-to-onset delay rather than the ceiling, and section 11.5 charges that delay against the arm’s clearance.

The two consumers read different parts of the record. The tracker interpolates the admitted reference, timestamps, and feedforward terms to drive the actuators along a smooth path. The independent permission path reads the lineage, timing, dynamic assumptions, validity bounds, and stop suffix, and it monitors current limits and authority without inheriting the planner’s feasibility claim as proof; a fresh obstacle, changed payload, or contact event can revoke TRACK and invoke the state-matched fallback. Because each field in the handoff corresponds to a runtime check, the Nervous System can admit fast motion without assuming that the next inference cycle will succeed. Producing such a record begins from a goal that carries no timing at all.

11.3 From Goal to Trajectory

The intent lease for the mug, which the arm takes from the conveyor for the coworker at the packing station, grants a goal region and effort ceilings (section 10.7), but no actuator can execute a region. The tracking controller needs a trajectory, which assigns state targets, feedforward terms, and timestamps across a bounded interval. If a learned policy could output valid motor currents at the 1 kHz joint loop rate, planning would not exist as an architectural stage, and the machine would operate as a high-rate feedback controller. Learned policies instead need tens or hundreds of milliseconds to compute a motion sequence, while the tracking controller and the permission path run deterministically on fixed millisecond-scale periods.

Classical sampling-based planners answer the geometric half of this problem. A probabilistic roadmap (Kavraki et al. 1996) or a rapidly-exploring random tree (LaValle 1998) searches $\mathcal{C}_{\text{free}}$ for a collision-free path without building an explicit model of the obstacles. A path from such a search, like a chunk from a learned policy, still carries no timing, no derivatives at its ends, and no stop, and those are what this chapter supplies.

Learned proposers bridge slow inference and fast actuator control by receding-horizon action chunking (section 3.4), predicting a multi-step sequence, executing a prefix, and replanning on fresh observations. Three kinds of proposer fill this role, and each matters here through the latency and continuity of what it emits. ACT (Zhao et al. 2023) predicts N future joint setpoints at once and blends overlapping chunks by the temporal ensembling of section 3.4, which averages setpoints but leaves derivative continuity at the chunk seam to this chapter. Diffusion Policy (Chi et al. 2024) denoises a chunk of horizon T_p (16 steps in the real Push-T setup), executes a subset T_a (six there), and replans on the next observation; its iterative denoising adds workload-dependent latency and variance. A learned warm-start for a constrained trajectory optimizer can cut solver iterations when it lands near a feasible basin, but it can also choose the wrong side of an obstacle or miss the deadline. None of these outputs is a feasibility certificate, and only a candidate whose full path, seam, torque, and stop checks pass is admitted.

This receding-horizon structure mirrors classical **Model Predictive Control (MPC)** and trajectory optimization, where an optimizer plans a trajectory over a finite horizon T , executes an initial prefix, and resolves the problem from the updated state on the next cycle. Classical MPC formulates trajectory synthesis as an explicit constrained numerical optimization problem, minimizing a cost function subject to analytical system dynamics ($\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$) and hard inequality constraints on physical boundaries ($\mathbf{x} \in \mathcal{X}_{\text{safe}}$) and actuator torques ($\mathbf{u} \in \mathcal{U}_{\text{adm}}$). Where analytical models accurately capture the physical plant and state dimensions remain compact, MPC provides rigorous guarantees of constraint satisfaction and closed-loop stability. Yet classical trajectory optimization hits the “Classical Wall” of section 3.2 when applied to unstructured open-world manipulation: numerical solvers cannot ingest high-dimensional semantic tokens (such as raw camera pixels or natural-language instructions) directly, and non-smooth contact mechanics or deformable objects introduce combinatorial local minima that render online numerical optimization intractable within real-time control periods. Generative action chunking resolves the semantic ingestion problem by learning continuous multimodal trajectory distributions directly from visual demonstrations, but sacrifices analytical guarantees. The physical AI architecture therefore treats learned chunking policies as unprivileged semantic proposers across multi-second task horizons, relying on the kinodynamic feasibility checks and seam contracts developed in this chapter to guarantee physical admissibility before actuation. Executing these feasibility checks and seamlessly replacing the active trajectory

introduces computational and transport delays that must be strictly bounded across replacement cycles.

The scheduling quantity is **replacement latency** L : elapsed time from the observation needed for a new proposal through transfer, inference, full-trajectory validation, and admission. An illustrative stage ledger for the machine's chunk policy has 15 ms of sensing, 5 ms of DMA, the policy's 40 ms of inference, 2 ms of deserialization, 18 ms of validation, and 1 ms of activation, totaling 81 ms. When stages overlap or share contention, a sum of stage percentiles is not a percentile of the sum (section 2.2), so the horizon is sized from the end-to-end distribution of L .

$$L = t_{\text{sense_ingress}} + t_{\text{ipc_dma}} + t_{\text{infer}} + t_{\text{deser}} + t_{\text{validate}} + t_{\text{admit}} \quad (11.1)$$

The choice of action chunk prediction horizon H introduces a fundamental systems dilemma between computational amortization and reactive responsiveness (figure ??). High-capacity vision-language-action (VLA) models and diffusion policies require non-trivial inference latency ($t_{\text{infer}} \approx 180$ ms on embedded accelerators). If the planner evaluates actions single-step ($H = 1$), the compute duty cycle exceeds 100 percent ($t_{\text{infer}}/\Delta t = 900\%$), starving the 50 Hz control loop ($\Delta t = 20$ ms) and causing the robot to pause and shudder between consecutive steps. Expanding the prediction horizon H amortizes inference across multi-step execution windows, reducing accelerator duty cycle below 50 percent once $H \geq 18$. However, executing a long chunk open-loop inflates the mean reaction delay to dynamic obstacles, $\tau_{\text{react}} \approx \frac{1}{2}H\Delta t + t_{\text{infer}}$. When $H > 36$ (> 720 ms physical window), reaction delay exceeds the dynamic obstacle avoidance threshold (~ 350 ms), causing closed-loop task success in dynamic environments to plunge from 90 percent down to single digits. Sizing the action chunk horizon to the Pareto-optimal window $H^* \in [16, 24]$ achieves the necessary balance: compute duty cycle remains comfortably amortized at 38 to 45 percent while reaction latency stays below 400 ms, preserving high closed-loop success rates across both static and dynamic operating conditions.

The component sum in equation 11.1 is an elapsed replacement path, not an absolute timestamp. Consider a generic replanner, slower than this machine's chunk renewal, that makes $K = 3000$ handoffs in a ten-minute mission at 5 Hz. If each handoff independently has a 0.01 chance of exceeding a P_{99} budget, the expected number of late handoffs is 30 and the probability that *all* arrive on time is $(0.99)^{3000} \approx 8 \times 10^{-14}$. Correlated contention clusters misses, which breaks the independence this arithmetic assumes. Allocating a mission exceedance target $\epsilon_{\text{mission}} = 10^{-3}$ equally gives a per-handoff target $\epsilon \leq \epsilon_{\text{mission}}/K \approx 3.3 \times 10^{-7}$ by the union bound. The corresponding $P_{99.99997}$ is a **design quantile target**, a risk allocation rather than an observed latency. A deployment either supports it with representative load testing and a conservative bound, or accepts a higher fallback rate.

The next request leaves no later than T_{request} after the active chunk starts. Admission must finish **before** t_{commit} , not merely before the last nominal setpoint. With a replacement-latency target $Q_{1-\epsilon}(L)$ and reserve T_{reserve} , require $t_{\text{commit}} - t_{\text{start}} \geq T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}}$. The admitted stop then needs another T_{stop} under MCU authority. The complete **time horizon** T_{record} that the record must cover, regardless of encoding, has the lower bound:

$$T_{\text{record}} \geq T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}} + T_{\text{stop}} \quad (11.2)$$

The complete duration in equation 11.2 includes the stop suffix. The machine's chunk policy emits a setpoint every 20 ms (the setpoint period Δt_{step}), 16 to a chunk, so one chunk carries 320 ms of nominal motion. For the conveyor-tracking arm, a chosen request delay of 60 ms, replacement target of 160 ms, and reserve of 20 ms, with a separately checked 300 ms stop, require at least 540 ms of complete motion authority, or 27 setpoint-period **duration equivalents**. This does not mean 27 stored setpoints: N in the record counts only nominal samples; the stop is represented by its separately validated polynomial coefficients. An authority horizon that covered only the request delay, replacement target, and reserve would omit the stop runway.

The chunk policy sends its next request in the renewal slot 50 ms after each chunk starts (section 3.4), so in steady operation several requests are in flight at once, an admitted chunk arrives every 50 ms, and their latencies overlap. The bound sets T_{request} 10 ms above that slot, so it still holds for a request that leaves up to that much late.



The tail target allocates replacement risk; observed latency and confidence determine whether the chosen deadline is credible.

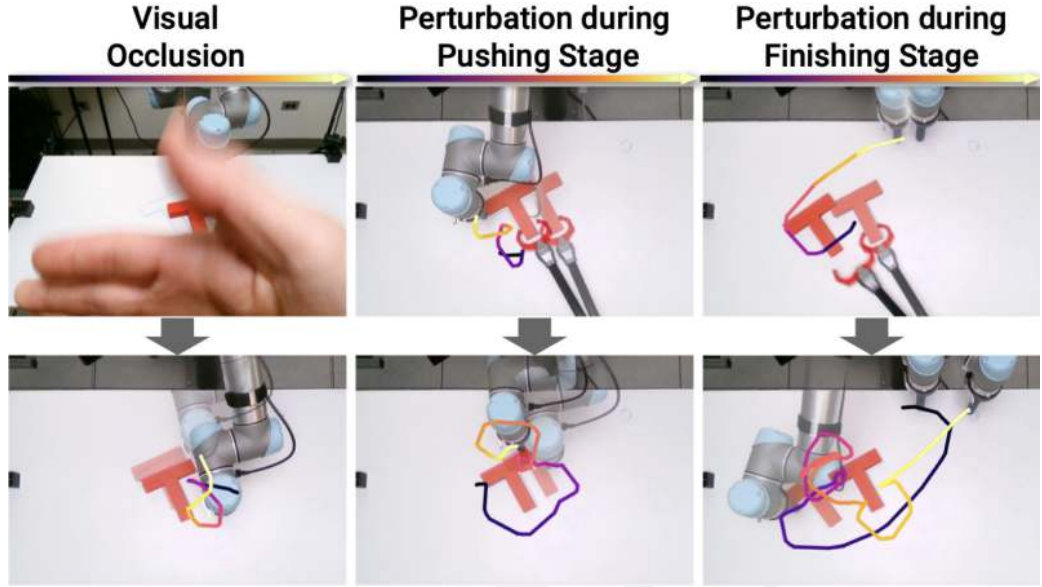


Figure 11.2: Push-T disturbance recovery: Chi et al. report a real Push-T configuration with 16 predicted and 6 executed action steps; 10 Hz commands were interpolated to 125 Hz. The recoveries show closed-loop replanning; the experiment includes no stopping suffix and no inference-starvation case (Chi et al. 2024).

The replacement target is a design-tail quantile of the whole replacement path, so it sits well beyond both the policy’s 40 ms P99 pass and the 81 ms stage ledger. The blend deadline must fall at least 240 ms after start, the sum of request delay, replacement target, and reserve, and the worked 300 ms commitment of section 11.5 leaves 60 ms beyond that target. In the worked arm, the chunk’s nominal setpoints end at 320 ms and the admitted stop completes at 600 ms, so the complete authority window spans 30 setpoint-period duration equivalents.

The 240 ms replacement window is the arm’s limit rather than its operating case. The shared lease of section 11.2 lapses once an admission falls more than 10 ms behind its slot, so a replacement that takes the full design tail ends in the arm’s state-matched fallback long before the blend deadline, and the window instead sets how long any lease may run on this arm.

This timing lower bound competes with target-evidence age. Even under a qualified local tracker, where only the conveyor’s residual slip carries the mug away from the tracker’s prediction, section 10.4 finds that one observation stays inside the grasp tolerance for less time than the 300 ms commitment. A single observation therefore cannot justify nominal tracking to commitment. The tracker must deliver fresh evidence before its horizon ends; otherwise TRACK ends at evidence expiry and the MCU uses its state-matched fallback. Extending a target timestamp because the compute tail is long would invert the permission rule.

Whatever the proposer, the systems interface treats the learned model as a stochastic producer of motion chunks, characterized by an empirical latency profile and an output horizon N ; diffusion schedules, attention caching, and token generation shape that profile but sit below the first budgetable quantity. The execution buffer absorbs inference latency while the controller still reacts to disturbances, as the Push-T recoveries of figure 11.2 show. Sizing N against a validated latency distribution lowers the risk of a late replacement; the protected stop handles a miss.

A replacement planner must account for motion during its computation. In an illustrative 80 ms delay at a constant joint speed of 0.50 rad/s, a stale request-epoch pose lags the moving reference by 0.040 rad (2.29°). A proposed replacement can instead start from the active plan’s projected reference at the intended seam:

$$\Delta q_{\text{reference}} = \int_{t_0}^{t_{\text{seam}}} \dot{q}_A(t) dt, \quad \hat{\mathbf{x}}_{\text{seam}} = \mathbf{x}_A(t_{\text{seam}}) \quad (11.3)$$

The projected seam in equation 11.3 is useful for constructing a candidate; contact, tracking error, or scheduling changes can make the plant differ. The MCU therefore measures current encoder state, validates the actual seam ($q, \dot{q}, \ddot{q}$) and any bridge, and refuses a candidate outside the admitted region. Matching two *reference* trajectories makes the references C^2 , not the plant.

Definition 11.2: Future-state conditioning

Future-state conditioning is the practice of generating candidate motion plans anchored to the anticipated state at execution time rather than the stale state measured at plan initiation.

1. **Significance:** Offsets computational latency Δt_{plan} , removing the reference jump at trajectory handoff that planning from the stale state would create, without zero-velocity stops between chunks.
2. **Distinction:** Distinct from state estimation or filtering; it is a reference-planning strategy that projects the admitted trajectory forward rather than filtering sensor noise.
3. **Common pitfall:** Treating the predicted execution state as the measured one; if a disturbance displaces the plant during planning, the candidate stays inadmissible until the MCU checks the seam against encoder state.

The horizon now answers to two clocks. It must be long enough for a design-tail replacement to land before commitment with the stop still behind it, and that same window caps the lease. The evidence behind the target, meanwhile, must stay fresh for as long as TRACK runs. A replacement that arrives in time, however, secures only the plan's continuity in time. The setpoints it carries must also lie within what the joints can deliver and join the mechanism's actual velocity and acceleration at the seam.

Checkpoint 11.1: Action chunk horizon and tail latency budgeting

Before turning from replacement timing to the feasibility of the motion itself, verify your ability to size an action chunk horizon against the latency tail:

- ☐ **Mission exposure and tail quantiles:** In the stated independent-handoff illustration, what is the chance of one or more missed P_{99} deadlines across 3000 handoffs, and why is that not a measured mission failure rate?
- ☐ **Horizon sizing trade-off:** Can you distinguish between the risk of trajectory buffer starvation from too short an action chunk and the risk of unmodeled state drift and stale perception from an excessively long chunk?
- ☐ **Projected seam state:** Can you explain why a candidate built from the active plan's projected reference still needs an encoder-state and bridge check at admission?

11.4 Kinodynamic Feasibility

A candidate trajectory is a geometric and timed proposal, not permission to move. Before submitting it, the planner screens it by testing whether the verified initial state lies inside the candidate's kinodynamic feasibility envelope, evaluating joint position and velocity, predicted actuator torque and current, contact limits, obstacle clearance, and the matched stop suffix over the proposed horizon. For an arm, the rigid-body model maps $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, and $\ddot{\mathbf{q}}(t)$ to $\tau(t) = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$ and $I_i(t) = \tau_i(t)/k_{\tau,i}$ (Craig 2005; Lynch and Park 2017). The planner's screen is not admission; the MCU repeats the decisive check and admits the bounded trajectory only if its configured limits and complete stop clearance hold for the verified initial state; it then checks tracking, freshness, and the remaining stopping budget during execution. An optimizer may smooth a path or reduce obstacle proximity, but the admitted trajectory and resident stop, not the optimizer's objective value, define the motion permission (figure 11.3).

Feasibility checked only at waypoints is not feasibility of the motion. The mechanism follows an interpolated path between samples spaced T_s apart, so a check at $t_k = kT_s$ alone lets derivatives

Prerequisite: Actuator torque saturation limits and rotor inertia scaling (N^2J) are derived in section 2.4.



Figure 11.3: Real-time trajectory optimization: GPU-accelerated motion generation executing minimum-jerk splines on hardware, avoiding a dynamic obstacle against a signed distance field, executing a constrained pick-and-lift, and coordinating two arms in a shared workspace. Collision distance is evaluated in continuous time rather than at waypoints, which is what allows the trajectory to be re-optimized while it is being executed. Adapted from (Sundaralingam et al. 2023).

exceed their bounds inside $[t_k, t_{k+1}]$. If the interpolator may apply up to $a_{\max}$ within a sample, a worst-case triangular acceleration reversal across the sampling midpoint lets velocity overshoot the sampled values by as much as $\Delta v \leq \frac{1}{4}a_{\max}T_s$, so two samples just under $v_{\max}$ can bracket a peak above it and trip an over-speed clamp between two valid waypoints. To bound velocity in continuous time without evaluating every point, the admission filter tests a contracted limit, $|\dot{q}_k| \leq v_{\max} - \frac{1}{4}a_{\max}T_s$, which absorbs the largest intra-sample overshoot.

Concatenating individually feasible chunks can create an infeasible seam. Position-only matching (C^0) can leave a velocity step; matching velocity (C^1) can leave a finite acceleration and torque step. In a rigid-inertia approximation, a commanded velocity change $\Delta \dot{q}$ spread over servo period T_c asks for the torque in equation 11.4, where $J_{\text{eff}} = J_L + N_g^2 J_{\text{rotor}}$ is the output-side inertia, the load's plus the rotor's reflected through gear ratio N_g . A physical drive saturates or filters that request according to its current limits and compliance. Figure 11.4 places that request beside its remedy, a finite C^2 bridge that removes the reference velocity step.

$$\tau_{\text{request}} = J_{\text{eff}} \frac{\Delta \dot{q}}{T_c} \quad (11.4)$$

Definition 11.3: C^2 spline seam

A C^2 **spline seam** is a trajectory junction that enforces continuity through the second time derivative (position, velocity, and acceleration) between adjoining motion segments.

1. **Significance:** Prevents instantaneous acceleration steps across trajectory chunks, bounding the inertial torque spike $\tau_{\text{step}} = J_{\text{eff}} \Delta \dot{q}$ demanded from actuators.
2. **Distinction:** Distinct from C^1 continuity (which matches only velocity and allows finite acceleration jumps) and C^0 continuity (which matches only position and allows infinite acceleration impulses).



Figure 11.4: Seam demand and checked bridge: A position-matched velocity step creates a large one-tick inertial torque request. A finite C^2 bridge removes the reference velocity step, but its interior jerk, torque, and joint response still require validation.

3. **Common pitfall:** Assuming C^2 continuity guarantees torque feasibility; while acceleration is continuous, interior velocities, accelerations, or jerks within the blending bridge may still violate physical actuator or clearance limits.

The transmission shapes the response to what the drive delivers. For a two-inertia model with load inertia J_L , reflected motor inertia $J_{\text{ref}} = N_g^2 J_{\text{rotor}}$ and coupling stiffness k_θ , the antiresonance and resonance frequencies for this model are $\frac{1}{2\pi} \sqrt{k_\theta / J_L}$ and $\frac{1}{2\pi} \sqrt{k_\theta (1/J_L + 1/J_{\text{ref}})}$. In the arm's strain-wave joints, reflected rotor inertia and a compliant transmission can amplify a sharp reference change, so admission needs the joint's identified, load-dependent modes and checks bridge extrema, torque, and drive limits against them. No frequency, stiffness, or jerk range transfers between products without measured parameters.⁴

Smoothing the seam's velocity step requires online C^2 trajectory spline blending, which must be treated as the synthesis of an entirely new motion plan rather than as cosmetic filtering. When the planner inserts a polynomial bridge $\theta_{\text{blend}}(t)$ over a transition interval T_{blend} , a duration distinct from the blend deadline t_{blend} , to connect chunk A and incoming chunk B, the resulting bridge segment must undergo the exact same dynamic feasibility checks as the primary chunks.⁵

The bridge is a fifth-order (quintic) polynomial because torque follows acceleration ($\tau = J_{\text{eff}} \ddot{\theta}$ in the rigid-inertia model), so a bridge that matches only velocity still leaves an acceleration step, and with it a torque step of $J_{\text{eff}} \Delta \ddot{\theta}$. For C^2 continuity across the seam, the bridge must match position, velocity, and acceleration at both endpoints, a total of six boundary conditions:

$$\begin{aligned} \theta(0) &= \theta_0, & \dot{\theta}(0) &= \dot{\theta}_0, & \ddot{\theta}(0) &= \ddot{\theta}_0 \\ \theta(T_{\text{blend}}) &= \theta_1, & \dot{\theta}(T_{\text{blend}}) &= \dot{\theta}_1, & \ddot{\theta}(T_{\text{blend}}) &= \ddot{\theta}_1 \end{aligned}$$

where $(\theta_0, \dot{\theta}_0, \ddot{\theta}_0)$ is the terminal physical state of chunk A at the seam instant, and $(\theta_1, \dot{\theta}_1, \ddot{\theta}_1)$ is the target state on incoming chunk B at $t = T_{\text{blend}}$. Matching six boundary conditions requires six independent coefficients ($a_0 \dots a_5$), uniquely dictating a fifth-degree polynomial:

$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

The six coefficients follow in closed form from these boundary conditions (section B.3), and the bridge's jerk varies quadratically in time, finite over a finite blend. Admission must check its extrema and the measured joint response; C^2 endpoints alone do not suppress all resonant energy.

A fixed-degree polynomial can be evaluated with bounded work per axis on a chosen controller. Coefficient synthesis and whole-trajectory extrema checks occur at admission; the measured worst-case execution time, including memory access and monitoring, must fit the configured servo period.

⁴ | **Sampling and mechanical modes:** A control period faster than a mechanical mode can help resolve that mode but is not a stability proof. Controller phase margin, sensor delay, damping, filtering, and actuator bandwidth must be validated for the identified joint.

⁵ | **Trajectory jerk:** A jerk objective can make a particular bridge smoother. A finite jerk bound alone does not exclude energy near a measured resonance; the complete command spectrum and closed-loop response must be checked for the joint and load.

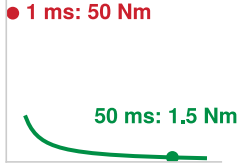
Representation affects what can be proved at admission. Table 11.1 separates continuity supplied by the representation from the dynamic checks still required. A B-spline of degree p has C^{p-r} continuity at an interior knot of multiplicity r ; its control-point convex hull bounds position, not automatically velocity, torque, or clearance. Minimizing an integrated jerk objective requires solving that optimization; choosing a B-spline does not itself solve it.

Table 11.1: Trajectory representations and required checks: Smooth parameterization helps form a candidate but does not by itself prove dynamic feasibility.

Representation	Native seam property	Checks before execution
Quintic bridge	Can match $q, \dot{q}, \ddot{q}$ at both ends; jerk is generally quadratic.	Check interior extrema, torque, clearance, and timed admission.
B-spline	Continuity depends on degree and knot multiplicity.	Check derivatives, dynamics, clearance, and optimizer result.
ACT action chunk	Discrete setpoints; zero-order hold jumps at ticks, linear interpolation is C^0 .	Construct and admit a feasible bridge and stop.
Diffusion action chunk	Discrete sampled actions; no intrinsic actuator authority.	Validate timing, state, bridge, limits, and stop.

Napkin Math 11.1: Seam acceleration and torque requests

For the registered CSG-25-50-2UH teaching example, reflected rotor inertia is $0.25 \text{ kg}\cdot\text{m}^2$. Correcting a 0.2 rad/s reference velocity gap in one 1 ms tick asks for $50 \text{ N}\cdot\text{m}$ of inertial torque. The manufacturer's specification gives $51 \text{ N}\cdot\text{m}$ rated L10 and $127 \text{ N}\cdot\text{m}$ repeated peak; this one-tick **request** is below the rated value. A symmetric C^2 velocity blend over 50 ms has $1 \text{ N}\cdot\text{m}$ mean and $1.5 \text{ N}\cdot\text{m}$ peak inertial torque, a $33.3\times$ peak-request reduction. Total torque still includes load, gravity, friction, and control response; it needs a full peak and thermal check.



In this inertia model, the C^2 blend cuts the peak inertial torque request by $33.3\times$.

Output chatter in learned proposals is a related failure. Stochastic sampling can produce successive chunks whose every sample satisfies position and velocity limits while the commanded acceleration reverses sign rapidly, and each reversal asks the drive for a torque reversal, with the current transitions and I^2R heating that follow if the drive tracks it (section 2.4).⁶ The systems engineer budgets that variation as actuator thermal rise and transmission fatigue, not as perceptual roughness; a trajectory that excites structural resonances or exceeds the drive's thermal limits is infeasible however closely its waypoints follow the task. Even a candidate that passes every internal and seam check must still provide for a replacement that never arrives.

11.5 Behavior at the Seam

Suppose the chunk policy stalls while the arm is reaching toward the mug. On this machine the shared lease ends the wait 60 ms after the last admission, and the arm enters its state-matched fallback along the renewal-to-onset path of section 11.2. The record's own deadlines decide how long any lease could be allowed to run, because until the stop begins the arm keeps moving, and every millisecond the MCU waits is travel its stop can no longer use. Let the lease run to its ceiling, then, so that the blend deadline t_{blend} passes with no replacement admitted. The four deadlines of section 11.2, compared in the MCU monotonic clock with $t_{\text{intent_exp}}$, now decide what happens. A replacement may still be admitted until t_{commit} , the entry of the matched stop suffix, whose duration is $T_{\text{stop}} = t_{\text{stop_end}} - t_{\text{commit}}$. It must pass the full admission check before that cutoff; a late host progress report has no authority.

For a cruise entry with $q(t_{\text{commit}}) = q_c$, speed $v_c > 0$, and zero entry acceleration, a C^2 position-and-velocity-matched stop over duration T uses $u = (t - t_{\text{commit}})/T$:

$$q(u) = q_c + v_c T \left(u - u^3 + \frac{1}{2} u^4 \right), \quad 0 \leq u \leq 1 \quad (11.5)$$

Differentiating equation 11.5 gives speed $v_c(1 - 3u^2 + 2u^3)$ and acceleration $-6v_c u(1 - u)/T$, peak deceleration is $1.5v_c/T$, and stopping distance is $v_c T/2$. Unlike a constant-deceleration switch, its

⁶ | **Actuator Thermal Derating:** Joule heating (I^2R) in the windings raises stator temperature, and past its limit degrades magnet flux and insulation, so motor controllers enforce derating curves that throttle continuous torque. A velocity profile that ignores this governor meets a mid-trajectory current limit and loses tracking.

acceleration begins and ends at zero. Holding the peak at the credible deceleration a_{brake} fixes $T = 1.5v_c/a_{\text{brake}}$ and a distance of $0.75v_c^2/a_{\text{brake}}$, half again the constant-deceleration distance $v_c^2/2a_{\text{brake}}$, so in a stopping budget the suffix behaves like constant braking at $a_{\text{eff}} = a_{\text{brake}}/1.5$. A nonzero measured entry acceleration requires a general quintic matching the actual $(q, \dot{q}, \ddot{q})$ at entry and zero terminal velocity and acceleration; the permission path checks its interior speed, acceleration, jerk, torque, and clearance.

In the worked arm case, the blend deadline is 240 ms, matched commitment 300 ms, nominal setpoint end 320 ms, and stop end 600 ms, so T is 300 ms. At the arm's 1 m/s TCP speed limit, the suffix's peak deceleration is 5 m/s² and it travels 150 mm. Clearance is last checked at the blend deadline, so the cruise from 240 ms to physical brake onset at 300 ms is charged against that clearance and uses 60 mm; the full 210 mm leaves 90 mm of the 300 mm obstacle clearance. Sensing and model error must still come out of that geometric margin. The stated 300 ms is **physical suffix onset**, so the MCU must close replacement admission and pretrigger any communication or actuator delay δ by that instant minus δ . If it does not, add $v_c\delta$ of travel and shift the stop end; the 90 mm margin is then smaller. Nominal setpoints end at 320 ms while MCU STOP continues to 600 ms, and the case assumes that fresh target evidence keeps TRACK valid that long (section 11.3). Figure 11.5 places those instants on a single axis and separates the two ownership regimes, so the point where nominal reference tracking ends can be read against the separately authorized stop that carries the arm to rest.

A stall on this machine never reaches commitment. One permission tick, one bus cycle, and brake onset, 22 ms together, put the arm's stop onset 82 ms after the last admission, or 82 mm of travel at the TCP limit. From zero-acceleration cruise the C^2 profile matches the arm at whatever instant it starts, so the state-matched fallback travels the same 150 mm as the matched stop suffix. The renewal-to-onset path therefore uses 232 mm of the 300 mm clearance and leaves 68 mm, against 90 mm when the stop begins at commitment. Because stop onset must come by the 300 ms commitment, the lease may run at most 278 ms.

On the arm, commitment decides whether the 90 mm margin survives. A lease 200 ms longer than the arm's 278 ms ceiling would let tracking run that far past commitment, and cruise at the TCP limit would add 200 mm before the same stop and breach the 300 mm clearance by 110 mm. A later stop must still match its actual entry state; this counterfactual holds cruise speed constant only to show the lost clearance. An arm that waits that long may already occupy a state from which no admissible control avoids the obstacle.



Definition 11.4: Inevitable collision state

Inevitable collision state (ICS) is any dynamic plant state $\mathbf{x}(t) \in \mathcal{X}$ from which every physically admissible future control trajectory $\mathbf{u}_{[t,\infty)} \in \mathcal{U}$ inevitably produces collision with an obstacle or constraint boundary:

$$\mathbf{x} \in \text{ICS}(\mathcal{X}_{\text{obs}}) \iff \forall \mathbf{u}(\cdot) \in \mathcal{U}, \exists \tau \geq t \text{ such that } \mathbf{x}(\tau) \in \mathcal{X}_{\text{obs}}$$

where $\mathcal{U}$ denotes the admissible control input set bounded by actuator torque, traction, and deceleration limits.

1. **Significance:** Sets the commitment deadline t_{commit} in receding-horizon planning; continuing nominal motion past it without an admitted successor can carry the plant into an ICS, and no later replanning can bring it back out.
2. **Distinction:** Unlike static geometric collision checks that evaluate only instantaneous spatial overlap ($\mathbf{x} \in \mathcal{X}_{\text{obs}}$), an ICS identifies dynamically trapped states where current momentum and finite braking torque make future boundary breach physically unavoidable.
3. **Common pitfall:** Delaying emergency braking in the expectation that an overdue neural planner will complete an evasive path, ignoring that moving mass enters an ICS long before geometric contact occurs.

These deadlines and the stop they protect are fixed at admission, before motion begins. The candidate declares payload, contact, effort, and obstacle assumptions. The MCU first validates its

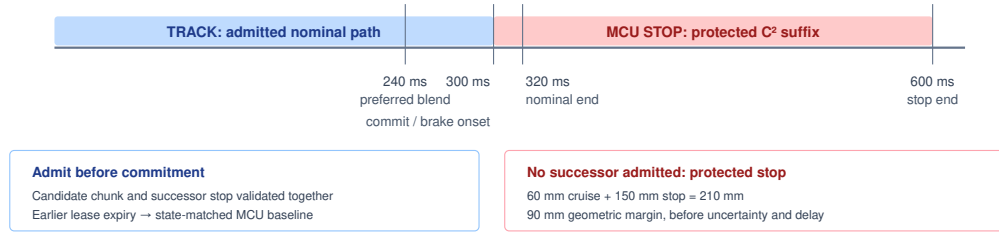


Figure 11.5: Nominal tracking and the matched stop suffix: The worked arm permits a checked replacement before the 300 ms physical stop onset. Nominal references would end at 320 ms, while the separately authorized, C^2 stop reaches rest at 600 ms. The suffix is matched to the commitment state; earlier invalidation needs the state-matched fallback. All deadlines use the MCU monotonic clock, and physical clearance includes the full response and stop.

fixed header and lineage, then checks the complete bounded trajectory and suffix, including the seam $(q, \dot{q}, \ddot{q})$ against its current encoder state, interior extrema, clearance, stopping distance, and a feasible response from every state reachable before commitment. A scan over N setpoints is $O(N)$, even with a fixed maximum N and measured worst-case execution time; after admission, a distinct $O(1)$ per-tick check reads the current state and local reference. A replacement received after the blend deadline passes the same check with a compressed bridge, whose higher acceleration and torque its extrema must still clear, and if the check cannot finish before commitment the MCU keeps the current record. A candidate received after commitment cannot restore the spent runway.

The planner proposes the suffix, but the MCU latches the validated payload and stop coefficients into protected memory **before** enabling TRACK and selects that resident path at commitment without reading host DRAM, so a host crash, bus stall, or corrupted shared packet cannot erase the stop.⁷ If the measured state falls outside the latched suffix's entry region, the MCU uses the state-matched fallback; torque disable or a spring brake is not a universal substitute for a controlled stop, especially with a suspended load (section 12.9).

The base of the mobile manipulator stops the same way, and for a reason of its own. The loaded tote rack tolerates the machine's credible deceleration only if that deceleration is reached gradually; a stop that begins and ends at zero acceleration costs half again the distance of an abrupt one at the same peak. That cost now enters the stopping budget. At the drive limit of 1.5 m/s, a suffix that peaks at the credible 2 m/s² lasts 1,125 ms and travels 843.75 mm, against the 562.5 mm of constant-deceleration braking in equation 2.1. The suffix therefore adds 281.25 mm to the budget that section 8.3 left at 912.9 mm, for a running total of 1,194.15 mm. That total passes the 1.10 m rack-end clear distance by 94.15 mm. This is the stopping condition of principle 1 with one more term, because a stop the load can tolerate is a longer stop and the distance it adds comes out of the same clear distance. With such a stop the machine can no longer run at its drive limit and still stop short of a person who has stepped out at the rack end. Of the budget's terms, only speed is left to give that distance back, and section 12.5 sets it once the last term is in.

For either body, the admission and fallback decisions in table 11.2 are distinct from an emergency power cut.

Table 11.2: Tracking and fallback decisions: A host crash does not remove a latched stop. TRACK is the nominal mode; STOP is the stop rung of the fallback ladder in section 12.9.

State	Trigger	MCU action and required evidence
TRACK, ordinary handoff	Replacement passes the complete admission check before commitment.	Latch the new nominal path and its own stop, then transfer at a checked C^2 seam.
TRACK, late candidate	Candidate arrives after the blend deadline but before commitment.	Admit only if the compressed bridge, full path, stop, and timing still fit; otherwise keep the current record.
STOP, matched suffix	No successor admitted by commitment.	MCU follows its protected suffix from the validated entry state; nominal tracking may end while STOP continues.

⁷ | **Inter-core shared-memory contracts:** A shared-memory implementation needs an ownership protocol, cache maintenance where required, and release/acquire ordering. The MCU validates a completed candidate and copies the admitted plan and stop into protected memory before activation.

State	Trigger	MCU action and required evidence
STOP, earlier invalidation	Lease lapse, intent expiry, contact, state mismatch, or fault before commitment.	MCU selects the state-matched fallback with reserved clearance.
Fault response	Controlled stop cannot be assured from measured state.	Use the machine-specific risk response (which may include drive inhibit, brake, or compliance) only under its validated load and timing assumptions.

Checkpoint 11.2: Commitment and full stopping clearance

Before committing an autonomous machine to an action chunk, verify your understanding of commitment deadlines, stopping budgets, and trajectory continuity:

- ☐ **Deadline distinction:** Can you distinguish nominal setpoint end, physical stop completion, and target-evidence expiry?
- ☐ **Continuous entry:** Why would a constant-deceleration switch fail C^2 matching at zero-acceleration cruise entry?
- ☐ **Clearance and latency:** The arm's 90 mm margin already charges cruise from the blend deadline to commitment. Where must an unreserved delay δ beyond it be charged, and how much of that margin does $v_c \delta$ consume?

Each row of table 11.2 presumes that its trigger reaches the MCU in time. A failure can first show itself in the candidate, at the seam, or in the world, and each reaches the MCU on a different schedule.

11.6 How Plans Go Wrong

A plan can become infeasible by execution time. Between synthesis in the learned policy and actuation at the causal boundary (section 1.2), the machine's state drifts, the environment shifts, or the seam between consecutive chunks violates dynamic constraints. Where a failure first becomes visible decides which subsystem holds the signal and how much time it has to detect and reject the plan.

The earliest failure is a chunk that is invalid before it reaches the drives. A **pre-execution bound checker** evaluates each candidate at admission against fixed kinematic and dynamic limits, computing the margin $M(t) = L_{\text{limit}} - |x(t)|$ at every trajectory point for position q , velocity $\dot{q}$, and acceleration $\ddot{q}$. Any sample beyond its limit makes $M(t)$ negative at that instant, and the checker rejects the chunk with a fault record naming the joint channel, the limit, the violation time, and the negative margin. Rejection at admission causes no dynamic disturbance, because the machine stays on its previously validated trajectory.

A **boundary checker** is the admission-time guard for the seam of section 11.4. It compares the candidate with the active plan and measured joint state at t_{seam} , including Δq , $\Delta \dot{q}$, and $\Delta \ddot{q}$, and bounds the torque the seam requests by equation 11.4.

For the CSG-25-50 joint of the seam notebook, counting only its 0.25 kg·m² reflected rotor inertia over one 1 ms tick and allocating the whole 127 N·m repeated-peak rating to this acceleration gives an *idealized ceiling* of 0.51 rad/s on $\Delta \dot{q}$. Available margin is smaller after link and payload inertia, gravity, friction, other commanded torque, current rise time, and thermal or bus limits are accounted for. The permission path must use that state-dependent margin and the measured drive response when evaluating a finite bridge. It rejects a direct velocity discontinuity even if its one-tick arithmetic falls below that ceiling; a bounded C^2 bridge and its interior checks are still required.⁸

Even with all pre-execution and boundary checks passed, changes in the physical world can render the plan infeasible. A grasped workpiece can slip from its intended SE(3) pose, an unmodeled obstacle can enter the workspace, or structural tracking error can accumulate beyond allowable tolerances. **Runtime monitors** detect changed-world infeasibility by observing four dedicated signals, comprising joint state divergence, tracking error velocity, unexpected contact wrenches, and

⁸ | **Drive and resonance response:** A sharp command can excite a joint's *identified* flexible modes, but their frequencies and damping depend on the transmission and load. Current limiting, desaturation detection, DC-bus response, and shutdown depend on the selected driver and its configuration; see the TI gate-driver protection description.

the invalidation of the geometric assumptions the record declares (section 11.7). The tracking error allowance ϵ_{pos} is the swept-volume clearance, the distance between the planned trajectory's swept volume and the nearest obstacle boundary, less the structural deflection and gearbox compliance allowance δ_{deflect} . For the mobile manipulator's arm reaching past the conveyor guard, a budgeted swept-volume clearance of 0.020 m and an unmodeled payload deflection and gearbox compliance allowance δ_{deflect} of 0.005 m leave an allowance ϵ_{pos} of 0.015 m. The allowance caps the tracking bound rather than replacing it. The arm tracker's own validated bound, the arm's instance of the ϵ_{track} that section 12.2 validates for the base, must sit inside those 0.015 m, and the per-tick check trips on ϵ_{track} . If measured Cartesian tool-position error $\|\mathbf{x}_{\text{tool,actual}}(t) - \mathbf{x}_{\text{tool,planned}}(t)\|$ exceeds ϵ_{track} , or a six-axis force-torque sensor reports an unmodeled 10 N contact in this free-space example, the monitor revokes nominal tracking and requests the validated state-matched fallback. Actual deceleration begins after the measured detection, dispatch, and actuator response delays.

A plan can also exhaust its nominal references before the next candidate passes admission. The MCU must not wait for the execution pointer to run off the array or treat a final-position hold as a stop. On this machine the chunk lease ends the wait first, and a lease run toward its ceiling reaches the commitment rule of section 11.5, which the MCU applies in its own monotonic clock. Complete stopping clearance includes detection, dispatch, actuator response, and motion under the selected profile; an assumed constant deceleration is insufficient for the C^2 worked suffix.

Every detector in this section relies on an explicit contract between the trajectory generator and the execution runtime. A plan delivered as a raw list of coordinates, without its valid lifespan, clearance bounds, boundary derivatives, and fallback, leaves the permission path unable to tell whether an ongoing motion remains safe or has drifted into infeasibility.

11.7 The Trajectory Contract

Each time the chunk policy proposes 16 new setpoints for the reach toward the mug, the permission path must decide whether to admit them without asking the planner anything. The trajectory record, immutable and handed over at every replanning cycle, makes that decision possible. None of its nominal setpoints is admitted unless the record's matched stop suffix is resident and feasible from every state reachable before t_{commit} . It answers to the intent lease of section 10.7, whose digest it carries as its parent, and every setpoint it proposes pursues the lease's target without exceeding its effort ceilings. The record stores the motion as setpoint arrays or spline coefficients, together with the sample cadence (on this machine a 20 ms period Δt_{step} , a 50 Hz setpoint stream), the nominal end $t_{\text{plan_exp}}$ (320 ms for one chunk), and SI units. With the frame and evidence time of section 11.2, the units, and the cadence in the handoff, the joint controller interpolates setpoints and evaluates derivatives without frame mismatches or reliance on ambient system clocks.

Physical feasibility depends on the scene and load used to validate the plan. The immutable sidecar therefore identifies obstacle-map epochs, payload and friction assumptions, limits, validator version, measured margin, and the source observation, and a digest in the fixed header binds it to the setpoints. The validator proves the stop feasible only under these assumptions, and they may claim no more than evaluation sampled, so the payload and friction the sidecar declares must lie inside the target ODD of section 7.8. The permission path revokes TRACK when live state leaves them, since a recorded margin holds only for the scene it was computed against. Solver seed and model hash belong in the audit sidecar for reproducibility where the toolchain supports it, not in the servo's per-tick path.

On this machine the chunk policy's output supplies only the nominal part of each trajectory record. Planning adds the seam state, the four deadlines, and the matched stop suffix, so the permission path admits one stream of records. The fixed header groups each record's fields by purpose. Lineage fields give a sequence number, the evidence time t_0 , and the digest of the parent intent lease. Timing fields give the four deadlines of section 11.2, and the record's tracking authority ends at the earliest of its commitment deadline, the lapse of the chunk lease (section 4.3) 60 ms after the last admitted chunk, the parent intent's expiry, and target-evidence expiry. Seam fields give the frame, each joint's entry state $(q, \dot{q}, \ddot{q})$, and the stop-suffix coefficients. Content fields give the payload kind and size and one digest over the setpoint payload and the sidecar. A variable-length setpoint payload and immutable provenance/assumption sidecar are outside the fixed header, and the MCU verifies their digest before the admission check of section 11.5. The payload digest binds content, and the parent

Downstream: Trajectory feasibility proposals are separately checked against conditional control barrier functions in section 12.7.

digest binds lineage. The record carries no keyed tag, because the permission path admits a proposal by its content rather than its origin (section 4.3).

Filled in for one reach toward the mug, the record is the timeline of figure 11.5 with its fields named. Its parent is the mug's intent lease, bound by digest; its frame is the machine's base frame, and every time field is in the MCU monotonic clock. Measured from t_{start} , the timing fields hold the 300 ms commitment, the 320 ms nominal end that 16 setpoints at 20 ms give, and the 600 ms stop end; the 240 ms blend deadline is the schedule's target, not a field. The seam fields hold a 300 ms C^2 suffix that peaks at 5 m/s² and brings the arm to rest from its 1 m/s TCP limit in 150 mm. While the record is in force, the 60 ms chunk lease bounds its tracking, a lapse leads to the state-matched fallback along a path that uses 232 mm of the 300 mm clearance, and commitment caps that lease at 278 ms. The sidecar declares a payload of at most 3 kg, free space until the grasp, and a scene inside the target ODD. Each field is one the permission path checks at admission or monitors on every tick, without asking the planner what it meant.

In the worked case the deadlines satisfy $t_{\text{start}} \leq t_{\text{commit}} < t_{\text{plan_exp}} < t_{\text{stop_end}}$. The MCU applies these deadlines under the TRACK and STOP rules of section 11.2, and a header timestamp cannot make a mismatched suffix safe. A log records evidence, request, validation, admission, activation, transfer, and stop-completion times in a common converted domain so actual response delay can be measured.

Every record in the runtime chain, from the observation contract through the belief record and the intent lease to the trajectory record itself, now names what it rests on and when it expires. The trajectory record proves only what the planner could foresee, a stop feasible from the states reachable before commitment under the scene and load in its sidecar. Several of the fallacies that follow mistake that bounded proof for a broader one.

11.8 Fallacies and Pitfalls

A trajectory remains feasible only while its physical assumptions hold at chunk boundaries and during fallback. Admission and execution checks must address those transitions as well as the nominal path.

Fallacy: *Static admission checks protect a machine against obstacle displacements during active execution.*

The arm begins an admitted reach toward the mug with the conveyor guard clear of its swept volume. During execution, a coworker slides a tote into the tool's path. The admission check was correct for the earlier geometry but no longer supports continued motion. Execution monitoring must detect the changed clearance and begin deceleration while sufficient stopping room remains. The design must bound how quickly workspace changes become visible via sensor bus latency and reserve the stopping margin needed to respond before the updated geometry becomes impossible to avoid.

Pitfall: *Checking each action chunk alone without examining the motion the chunks compose.*

The arm receives successive chunks that each reverse a joint's direction. Each chunk passes its own acceleration and jerk checks, yet the sequence produces oscillatory motion that can excite the drivetrain's resonant frequencies through gearbox compliance. Admission must compare derivatives across each seam and retain enough recent motion history to detect recurring reversals that would drive Joule heating ($I^2 R$) in the stator. The motor and transmission execute the composed trajectory, and treating chunks as independent requests hides behavior that appears only when the requests are joined.

Pitfall: *Delaying fallback transition past the hard stopping horizon in the hope that delayed compute finishes.*

With the lease run to its 278 ms ceiling, the worked arm's record ends its nominal setpoints at 320 ms and carries a 300 ms matched stop suffix that must finish by 600 ms, so the MCU must commit to fallback at 300 ms unless it has admitted a valid replacement. At that point the application processor reports inference nearly complete, but it has not delivered an executable plan. Waiting spends the runway reserved for deceleration. The MCU must follow the matched stop suffix at the commitment point. Progress reports cannot extend a mechanical stopping budget; only a newly validated trajectory can justify continued execution.

Byte layout: The trajectory record's field offsets and sizes are given in section C.14.3.

Pitfall: *Relying solely on feedforward dynamic models when physical contact or payload mass can change abruptly.*

The arm lifts an item near its 3 kg payload bound using feedforward torque computed for the loaded arm. The item slips from the gripper, but the drives keep applying the same lifting torque to the now unloaded arm, so it accelerates upward. The admitted trajectory has not changed; the dynamics that made its commands appropriate have. Feedback monitoring must detect the resulting tracking error and constrain the commands within the available margin before a mechanical limit strike or thermal derating. A feedforward model improves nominal tracking, but it cannot replace runtime checks for abrupt changes in payload or contact.

Fallacy: *A trajectory can omit an achievable terminal stopping suffix if replacement plans are expected to arrive in time.*

At its 1.5 m/s drive limit, the mobile manipulator's base needs a C^2 suffix of $T = 1.5v_c/a_{\text{brake}}$, 1,125 ms at the credible 2 m/s², to stop without exceeding that deceleration. A proposed suffix sized by the constant-deceleration time v_c/a_{brake} is a third shorter, so its profile peaks at $1.5a_{\text{brake}}$, past what the loaded rack tolerates. Expecting the next plan to arrive does not make this fallback executable. The admission gate must check $T_{\text{stop}} \geq 1.5v_c/a_{\text{brake}}$ for the C^2 profile it latches and reject a record that fails. Every accepted trajectory needs a physically achievable termination path for the case in which no replacement becomes available.

11.9 Summary

A trajectory is a timed proposal, and what shapes it is the case in which the next one is late. An inference-latency quantile is a scheduling risk target, not a safety certificate. It sets how often the machine should expect a late replacement, and the resident stop is what the machine does when one comes due. Two clocks bound the horizon from opposite sides. The compute tail pushes the commitment point later, while target evidence expires on its own schedule; on the conveyor, even a tracked observation of the mug expires before the worked 300 ms commitment (section 10.4), so tracking that long needs fresh evidence rather than a later timestamp.

Commitment is where these results meet. A replacement must pass full admission before t_{commit} , not merely before the last nominal setpoint. On this machine the shared 60 ms lease ends a stall long before then and sends the arm to its state-matched fallback with 68 mm of clearance left; commitment caps that lease at 278 ms, and a record that reaches commitment without a successor plays the C^2 suffix the MCU latched before motion began. The worked arm's 60 mm of cruise and 150 mm suffix leave 90 mm of geometric margin before uncertainty and unreserved delay. On the mobile manipulator's base, the suffix the tote rack tolerates raises the stopping budget at the drive limit to 1,194.15 mm, past the 1.10 m rack-end clear distance.



Key Takeaways: Timed motion and independent stopping

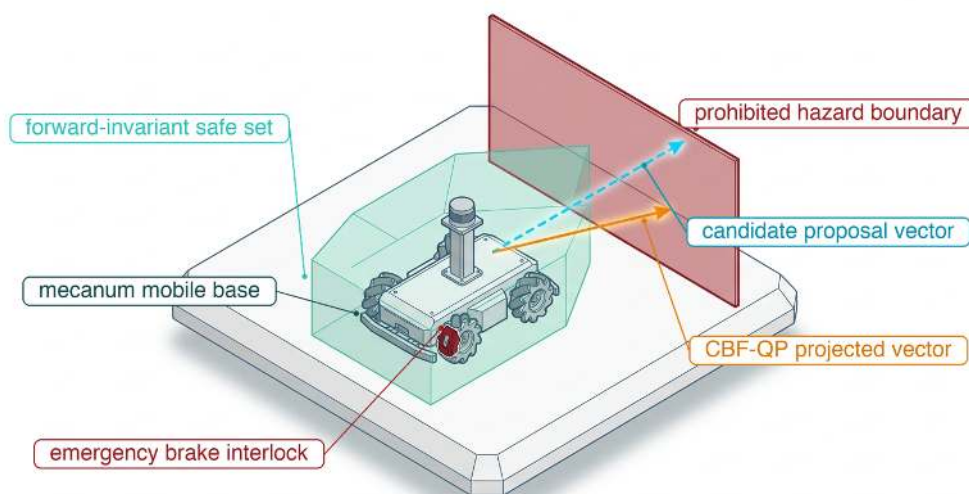
- **A latency quantile is a risk target:** The design-tail replacement target allocates the risk of a late plan and never certifies arrival, so every admitted trajectory carries a resident stop for the miss.
- **Commit before the runway is spent:** Admission closes at t_{commit} , the deadline chosen so the matched stop still fits, and commitment caps the chunk lease that ends a stall first. On the arm, a lease run 200 ms past commitment breaches the 300 mm clearance by 110 mm.
- **Evidence age and the compute tail pull apart:** A long tail pushes commitment later while target evidence expires on its own clock. Tracking past the evidence horizon needs a fresh observation, never a stretched timestamp.
- **A gentle stop is a longer stop:** A C^2 stop at the same peak deceleration travels half again the constant-deceleration distance, and on the mobile manipulator adding that term carries the budget past the rack-end clear distance at the drive limit.

- **The stage that first sees a failure owns it:** An invalid chunk, an infeasible seam, and a changed world first become visible at admission, at the seam, and during motion, so each stage holds its own signal and its own time budget for rejecting the plan.
- **Continuity is not feasibility:** A C^2 seam removes reference jumps, but the bridge's interior jerk, torque, resonance, and clearance must still be checked against the actual drive.
- **The planner proposes, the MCU admits:** The trajectory record binds setpoints, deadlines, and stop coefficients under its parent lease's digest, so the permission path judges the proposal by its content and holds a stop no host failure can erase.

What's Next: From admitted plans to per-tick permission

This chapter admits whole proposals. The MCU checks a complete trajectory and its matched stop before commitment and latches the stop in protected memory. Admission judges the plan against the state the machine was in when the plan arrived, yet while the plan runs the plant can lag its reference, an obstacle can appear, and an actuator can saturate. Chapter 12 checks each tick against live state, insets the stopping distance by the tracking error the controller can bound, and owns the fallback ladder whose stop rung plays this chapter's resident stop. That inset is the last term of the machine's stopping budget, and adding it sets the speed at which the mobile manipulator may run the aisle.

Safety Enforcement



- 12.1 The Deterministic Gatekeeper
- 12.2 What the Tracker Hands Over
- 12.3 The Authority to Refuse
- 12.4 Where the Enforcer Sits
- 12.5 Stopping Envelopes
- 12.6 When the Enforcer Runs Out of Authority
- 12.7 Safe Sets as Conditional Permission
- 12.8 Minimal Intervention
- 12.9 The Fallback Ladder
- 12.10 How Enforcement Goes Wrong
- 12.11 The Enforcement Record
- 12.12 Fallacies and Pitfalls
- 12.13 Summary

Purpose

How does an unprivileged software proposal become physical torque, and what lets the gatekeeper refuse it in time?

A neural policy may propose commands that violate geometric clearance, exceed joint velocity limits, or saturate actuator torque. Once an unverified command is converted into motor phase currents, detecting the neural network's hallucination comes too late: kinetic energy is already committed. An independent real-time enforcement path must evaluate candidate proposals before they cross the causal boundary and retain the uncompromising hardware authority to reject or project them within strict, sub-millisecond deadlines.

Timely refusal alone is insufficient; the selected fallback response must remain dynamically feasible under actual physical state. A mechanical brake cannot deliver idealized deceleration if thermal saturation or supply-rail voltage drops degrade torque generation. Runtime enforcement pairs control barrier functions with real-time feedback loops to verify that stopping envelopes remain strictly inside verified clearance margins. In the physical AI stack, enforcement embodies the core duty of the deterministic Nervous System: maintaining an unyielding permission path between statistical proposals from the Brain and physical execution by the Body.

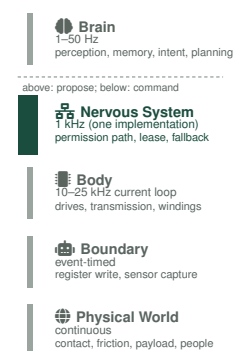


Figure 12.1.: The five-level machine stack with the Nervous System level highlighted.



Learning Objectives

- Explain the relationship between tracking accuracy, command permission, and independent enforcement at the actuator boundary
- Diagnose how update rate, feedback delay, and actuator saturation limit corrective control
- Calculate the finished stopping budget and the speed ceiling it sets from response delay, braking, localization, and tracking error
- Evaluate with a velocity-dependent barrier whether a tracker command computed from an admitted proposal must be accepted, projected, or refused
- Select fallback responses when sensing, computation, or actuator limits invalidate normal enforcement
- Construct an enforcement record that binds each check to its detector, its fallback response, and the records it consumes

12.1 The Deterministic Gatekeeper

THE safety microcontroller (MCU) admits the warehouse mobile manipulator's run down the aisle only after the matched stop suffix named in its trajectory record (section 11.7) is resident in the MCU's memory and feasible from every state reachable before commitment. That admission checks a plan once, before the motion starts. The machine then executes the plan one millisecond tick at a time, and on each tick the live state can drift from the one the planner assumed. The base lags its commanded reference, a person steps out at the rack end, a payload loads the arm, or the chunk policy proposes a setpoint the admitted plan never contained. What follows is the check made on every tick, and the response it selects when even the resident stop no longer fits.

The learned proposers behind those commands can emit infeasible paths or joint commands beyond the actuators' torque limits, and by the irreversibility law of section 1.7 no exception handler can recall the momentum that current in a winding imparts, so a bad command has to be stopped before it reaches the drive.

Enforcement therefore keeps the task competence of learned policies while denying them authority over geometric clearance, thermal limits, and human-safety invariants. The enforcer of section 4.3, the permission path's per-tick gatekeeper, sits on the permission side of the proposal boundary. It is a hard real-time component that evaluates candidate proposals, verifies invariant state bounds, and holds the exclusive hardware authority to permit current flow or command a fallback stop.

In the design this chapter follows, the enforcer runs on bare-metal microcontroller hardware wired directly to the motor drives and joint encoders and evaluates incoming setpoints each millisecond control cycle against validated constraints using **Control Barrier Functions (CBFs)**¹ (Ames et al. 2019). When a proposal is safe, it passes to the motor drives unaltered. When a proposal marginally threatens a boundary, an active-set Quadratic Program (CBF-QP) minimally modifies the command, minimizing torque deviation while enforcing the derived constraint.² When a proposal is structurally invalid or the proposer falls silent, the gatekeeper severs the command channel and triggers an autonomous fallback deceleration sequence.

However, an enforcer cannot guarantee safety beyond the tracking fidelity of the control loop beneath it, because every barrier certificate assumes that the tracking controller follows commanded references within a bounded error.

12.2 What the Tracker Hands Over

The tracker, the feedback controller that turns each admitted reference into a torque command on every tick (section 11.2), isolates the enforcer from the plant's internal mechanics, and only one of its guarantees crosses the interface upward. That guarantee is an explicit bound on the **worst-case tracking error** (ϵ_{track}) between where the machine was commanded to be and where its mass actually is. Within a validated envelope of reference acceleration, load, temperature, and disturbance, the tracker may provide a transient bound $\|x(t) - r(t)\| \leq \epsilon_{\text{track}}$ over the complete maneuver. The enforcer does not inspect transfer functions, root loci, or controller gains; to the software above

¹ | **Control barrier functions and forward invariance:**

Control Barrier Functions define a continuously differentiable certificate $h(\mathbf{x}) \geq 0$ over an admissible safe set $\mathcal{C}$, enforcing the directional clearance derivative constraint $\dot{h}(\mathbf{x}, \mathbf{u}) \geq -\gamma(h(\mathbf{x}))$. Enforcing this inequality renders $\mathcal{C}$ forward-invariant: states initialized in a viable set can remain there while the barrier is feasible and the stated model and timing assumptions hold. When approaching the boundary, the constraint forces opposing actuation inputs that arrest momentum before physical contact can occur.

² | **Active-set quadratic programming for safety shields:**

On a bare-metal microcontroller, an active-set solver that restricts its linear constraints to active boundary contacts and allocates no dynamic memory can bound termination with a fixed iteration limit on a tested platform; no universal $150\ \mu\text{s}$ limit follows from the solver type. If numerical ill-conditioning prevents convergence before the loop deadline, the solver aborts gracefully to an autonomous deceleration fallback.

the loop, the tracker is a black box that holds physical motion within the error bound ϵ_{track} of the commanded path.

This bound is a physical distance set by rotor inertia, transmission compliance, and actuator torque limits. A steady-state stiffness calculation balances unmodeled disturbance against the restoring effort of the closed-loop stiffness, but it does not bound transient overshoot or settling, so the bound also needs validated dynamics or full-maneuver testing.³

On the warehouse mobile manipulator's base, full-maneuver validation establishes $\epsilon_{\text{track}} = 40$ mm (illustrative; see the Reader Guide). The controller holds that bound only while three conditions hold. The reference cannot demand accelerations beyond what the drives can deliver, external disturbances must remain below d_{max} , and the actuators must operate below the continuous thermal derating limits of section 2.5 and the limits of their voltage rail. A reference that demands more acceleration than peak torque allows saturates the loop, the integrator winds up, and the base falls behind its reference by an amount that grows with every millisecond of saturation. Outside its rated envelope, the controller provides no bound at all.

Because the base can lag its reference by up to ϵ_{track} , every geometric and dynamic margin the enforcer evaluates is inset by that distance. A reference that keeps clearance c from the rack end guarantees the base only $c - \epsilon_{\text{track}}$, so the enforcer checks the reference against the required clearance plus 40 mm, and section 12.5 carries that inset into the stopping distance, where delay and localization error also consume clearance. If worn drive bearings, tire wear, or altered motor parameters doubled the true error to 80 mm without the record changing, the base would spend another 40 mm of clearance that the enforcer believes it still holds. A degraded tracking bound silently erodes every margin above it.

A tracking bound never checked at runtime is an assumption, not a contract. The Nervous System must publish ϵ_{track} with the operating conditions that validate it and monitor the instantaneous error $e(t) = \|x(t) - r(t)\|$ on every tick. When $e(t)$ exceeds the bound, under a disturbance spike or thermal throttling, the contract has broken, and the permission path must invalidate the spatial assumptions of the filters above it before unmodeled deflection causes damage.

Systems Perspective 12.1: The inset margin invariant

Every spatial and dynamic safety boundary evaluated by an enforcer must be inset by the underlying tracking loop's verified worst-case tracking error, and the stopping envelope of equation 12.1 carries that inset, together with the travel during transport lag, into every admission check. An enforcer that evaluates barrier certificates against nominal geometric boundaries without insetting will permit trajectories that physically collide with obstacles whenever tracking error reaches its certified bound.

The synthesis of tracking controllers, observers, and disturbance rejection belongs to the control literature (Slotine and Li 1991; Åström and Murray 2021). In modern physical AI implementations, the “tracker” is rarely an elementary single-input single-output feedback loop; it is typically structured as a **cascaded control hierarchy** (translating Cartesian path targets into joint velocity references, and velocity into field-oriented phase currents at 10–25 kHz) or as a high-rate **tracking Model Predictive Controller (MPC)** running at 100–500 Hz on bare-metal firmware to solve a localized quadratic program for dynamic disturbance rejection. Yet regardless of its internal mathematical sophistication, an optimal tracker remains semantically blind: it possesses no internal model of obstacle clearances, task intent, or human proximity, and it will execute a collision path into a steel upright with the same dynamic fidelity it brings to unobstructed free space. Furthermore, when unmodeled physical disturbances occur—such as surface oil causing gross tire slip or sudden thermal current derating—even an optimal tracking controller can guarantee tracking only within its qualified error tube ϵ_{track} . The enforcer therefore treats the tracker strictly as a contract boundary: it encapsulates the entire feedback control stack inside the certified bound ϵ_{track} , evaluating candidate proposals against forward-invariant barrier certificates before any command reaches the tracking layer.

³ | **Closed-loop tracking stiffness and disturbance bounds:** Sustained force and effective closed-loop stiffness determine static deflection. Transient tracking also depends on inertia, damping, command shape, and saturation. Disturbances outside the validated envelope invalidate the claimed tube; they need not produce exponential divergence.

Prerequisite: Spatial clearance insetting from sensor covariance and transport delay originates in section 8.5.

12.3 The Authority to Refuse

A chunk arrives from the application processor on time, well formed, and carrying a fresh evidence time, and one of its setpoints would drive the arm into a rack upright. Nothing in the packet marks that setpoint as wrong. The permission path therefore grants no authority to an incoming learned proposal until that proposal passes the configured state, input, freshness, and stopping-clearance checks. This is the third law, proposal is not permission (principle 3), enforced one tick at a time. A learned policy carries no stability guarantee, no bound on its output variance, and no representation of actuator current limits or mechanical stress, so its outputs are advisory.

The Simplex architecture (Sha 2001) is one realization of the permission path. It resolves the tension between task performance and safety by decomposing the control system into four structural components:

1. **The advanced controller:** An unverified, high-capacity learned model (such as a Vision-Language-Action network, a residual reinforcement learning policy (Johannink et al. 2019), or a diffusion policy executing on the application processor) that optimizes for complex, open-ended task performance.
2. **The baseline safety controller:** A certified, high-assurance deterministic controller (on this permission path, the matched stop suffix admitted with the trajectory in section 11.5, resident in bare-metal MCU firmware) that acts as the trusted low-level stabilization baseline of the physical plant, providing tested stability and stopping behavior under stated conditions.
3. **The safety manager:** Deterministic arbitration logic that continuously evaluates the state of the machine against forward-invariant barrier sets, tracking error bounds, and execution deadlines.
4. **The output selector:** An atomic hardware or real-time firmware multiplexer that routes actuation signals to the motor bridge. The selector admits a checked proposal when conditions hold and transfers to a resident baseline within its qualified response delay when those conditions fail.

Under this contract, the upstream policy never commands the plant directly. Instead, communication across the causal boundary operates through the chunk lease of section 4.3, a dead-man permission that expires unless renewed by a fresh proposal. The application processor transmits candidate control vectors as structured packets over shared SRAM mailboxes through a configured interprocessor channel, such as RPMsg.⁴ Each proposal opens with the proposal header (section 4.3), and the MCU checks its evidence time and expiry on each control cycle, adding the header's clock-conversion error bound. If the proposal is fresh, the state remains in the validated viable region, and tracking error remains bounded, the output selector permits the proposal (or its minimally projected variant) to drive the actuators. If the lease expires without renewal, tracking diverges, or the proposal threatens physical boundaries, the output selector revokes permission and engages the baseline safety controller unilaterally.⁵ The upstream policy is not consulted, does not negotiate, and possesses no hardware mechanism to override the selector.

When a proposal conflicts with physical constraints, the enforcer chooses between filtering and veto. Filtering applies when the proposal is structurally sound and a nearby admissible command still serves its intent. The spring-latched cage door that the arm opens supplies the case. The chunk policy's proposal to approach the door's strike plate at 0.10 m/s is well formed, but the door zone admits only the 0.03 m/s guarded approach of the target ODD, the one speed its demonstrations and trials cover (section 7.8). That ceiling marks where the evidence stops, not where the physics of the plate does. The enforcer therefore keeps the approach's direction and target and projects only its speed onto the 0.03 m/s ceiling, a constraint that the least-change quadratic program of section 12.8 enforces beside the half-spaces a Control Barrier Function derives.

Filtering must respect momentum as well as position, because the actuators must still be able to stop the machine before clearance vanishes (section 12.7). Projection is permissible only when the corrected command keeps the proposal's direction and target, changes it continuously, and stays well clear of a hazard. A structurally invalid proposal, such as a discontinuous jump in commanded position, an acceleration beyond the available torque, or a trajectory aimed into an immovable obstacle, indicates that perception or inference has entered an unmodeled failure mode,

⁴ | **Remote Processor Messaging:** Linux RPMsg provides virtio-based messaging between processors. Buffer ownership, copies, scheduling delay, and worst-case handover time depend on the platform; OpenAMP offers a separate no-copy API. The lease requires a measured, bounded delivery path.

⁵ | **Hardware break circuits and PWM trip zones:** A configured external timer break input such as TIMx_BKIN can force PWM outputs to a specified state without waiting for an interrupt. EPWM_TZFRM is a software force mechanism, not the external trip input. Output state and response time depend on the timer, gate drive, wiring, and qualification tests; the trip still needs a physically suitable stop path.

and projecting it onto the nearest safe boundary would execute a motion unrelated to any task intent. The enforcer vetoes such a proposal, and the output selector drops authority to a local fallback.

Filtering and veto both happen at runtime, and training the policy cannot replace either. As section 1.10 argues, penalty terms and barrier losses shape the distribution a policy learns but do not bound what the deployed policy emits, and constrained reinforcement learning bounds only the expected cost of violations, which permits a rare high-momentum collision provided the average stays below its limit. The permission path therefore checks each candidate at its actuation boundary whatever objective trained the proposer, and when the proposal is safe, the check is transparent and the learned competence passes to the drive unchanged.

The chunk lease, 60 ms on this machine, also keeps the enforcer's authority when the upstream host stalls. A separate 100 Hz host heartbeat has a 30 ms silence timeout. The 1 kHz MCU loop feeds its own 5 ms self-watchdog only after a successful cycle; a separate external trip covers MCU failure. At each timeout the MCU revokes the proposal and executes a resident, state-matched fallback, provided the reserved clearance and available braking still suffice. On the warehouse mobile manipulator, if a proposer renews its lease and then stalls, the brake still begins within 82 ms of the last valid renewal. This lease-expiry case assumes the separate heartbeat remains live while fresh admissible proposals stop arriving; a total host stall triggers the 30 ms heartbeat timeout earlier. When the danger is a person stepping out at the rack end, the age of the frame that first showed that person adds to the renewal-to-onset delay, and section 12.5 carries the sum into the admission check. Enforcing the lease at the drive, rather than trusting the host to withdraw its own proposal, is what bounds the delay at all.

Severing upstream authority requires a defined physical response at the lowest hardware interface. The zero-command floor is the configured response when active software control disappears, and writing zero to a drive produces different mechanics on different actuators. Zero current in a low-friction direct drive lets a gravity-loaded arm descend unless a separate restraint carries the load, while zero velocity in a stiff position loop can request a large transient. Section 2.6 describes the three unpowered states a drive can fall into (free coasting, phase-short dynamic braking, or a spring-applied brake); which one each drive enters is a choice this chapter makes, rung by rung.

Refusal is a steady-state decision, not only an emergency exception. On each millisecond control tick the enforcer checks the proposal's freshness against the lease and the evidence-age threshold, the host heartbeat, the tracking error, the actuator and kinetic limits, and the feasibility of the resident fallback, and if one check fails, it revokes the proposal and selects the resident response appropriate to the current state and available clearance. That path must remain able to act when the upstream proposal source fails, and where it sits in the signal chain decides what it can see and how quickly it can act.

12.4 Where the Enforcer Sits

The enforcer's position in the signal pipeline is set by the dynamics' timescale and the latencies of the channels that feed it, and every machine that turns learned proposals into motion offers two natural positions. Upstream of the feedback loop, a kinematic reference checker validates waypoints, coordinate targets, or velocity vectors before they enter the tracking loop; at planning rates of 10 to 50 Hz it can check a 300 ms trajectory segment against workspace boundaries and self-collision models. Its strength is spatial foresight and its limitation is blindness to the plant's actual state, so a reference that clears a rack upright by less than the base's transient overshoot passes every upstream check while the base still strikes the upright.

Downstream of the feedback controller, near the power electronics, a command checker inspects phase currents, duty cycles, and joint torque references immediately before they are written to hardware registers. Running at the machine's 1 kHz inner-loop rate, it prevents overcurrent, over-torque, and thermal runaway by clipping or zeroing signals beyond the plant's ratings, but it surrenders foresight. A torque well inside its continuous rating looks safe to a scalar clamp even while it accelerates a descending arm joint toward the packing station faster than the remaining travel can absorb, and the clamp registers no fault until no allowable reverse torque can prevent the collision.

The split trades latency for visibility. A reference checker at those planning rates sees several steps ahead but cannot respond to resonance, noise amplification, or instability above its update rate. A

Downstream: Heterogeneous silicon partitioning of the enforcer onto isolated microcontroller enclaves is mapped in section 13.2.

command checker acts on every tick, but its one-step horizon creates dynamic traps in which every instantaneous torque is legal until an irrecoverable state is reached.

Placement also decides how the enforcer joins the computing fabric. An in-line enforcer inspects every command before the next stage reads it, which leaves no command uninspected but puts its worst-case execution time on the critical path of the 1000 μ s loop, where an overrun faults the whole cycle. A parallel monitor on a separate core preserves loop timing and opens a safety contactor or switches the drive to dynamic braking when it detects a violation, but its detection and signaling lag, bounded by the monitor cycle and bus jitter, must be paid for with wider margins or lower speed.

Because neither single location provides both dynamic foresight and instantaneous protection, physical AI architectures resolve the tension through a **dual-stage enforcement pattern**. This pattern maps onto the two-processor implementation of the machine model (section 4.2):

- **Upstream Stage (Advisory Corridor Pre-Check on the Application Processor):** Operating at the chunk rate (20 Hz on this machine) on an unprivileged application microprocessor core, this stage screens multi-step trajectory chunks emitted by the learned policy against a dynamic stopping corridor under worst-case braking parameters and against static workspace envelopes before packing the waypoints into a leased chunk proposal sent over RPMsg. The stage is advisory. It spares the MCU proposals that would fail, but it runs on the proposer's side of the proposal boundary, so the binding admission of the trajectory and its stop suffix stays on the MCU (section 11.5).
- **Downstream Stage (Instantaneous CBF-QP Safety Shield on Bare-Metal MCU):** Operating inside a strict 1 kHz (1000 μ s) timer interrupt loop on the isolated MCU, this stage receives the proposed setpoint from shared SRAM, verifies tracking error bounds, solves an active-set Control Barrier Function (CBF-QP) quadratic program in pre-allocated static SRAM, and clamps raw current and torque commands directly before writing to PWM timer registers. The barrier it enforces reduces to a single-step convex projection (section 12.7), whose execution time must be bounded on the selected hardware.

The advisory pre-check screens for a feasible stop before a chunk is sent, and the drive-side stage checks current observations and commands on every tick, but neither stage can recover a state that has already exhausted braking clearance.

Correct placement does not help if the enforcer's inputs share failure modes with the component it checks. An enforcer that computes collision distances from a depth map produced by the same neural perception model that planned the motion is deceived by the same out-of-distribution artifact, so independent enforcement requires independent sensing and isolated computation. The MCU reads encoders and limit switches wired to its own GPIO and timer-capture registers, outside the Linux virtual memory space and scheduler, and a collision check reads a safety lidar or proximity sensor whose conditioning uses deterministic threshold logic rather than learned inference. Where no independent sensor exists, as for the friction under the base's wheels or the mass of a payload that only an estimator infers, no independent physical guarantee can be built, and the claim is only as strong as the estimator's bounded uncertainty or the premise the record states. Every invariant the enforcer checks, upstream or at the motor terminals, still assumes that the plant retains the authority to arrest its own motion before a forbidden boundary.

🔍 Systems Perspective 12.2: The monitor independence rule

The permission path must never evaluate a state estimate produced by the component it is designed to constrain, nor share an un-preemptible communication bus, memory matrix, or clock tree with unverified processes. Where independent sensing is physically impossible, the safety claim rests on the estimator's bounded model uncertainty, not on hardware.

12.5 Stopping Envelopes

The chapters that own a stopping term have each added it to the warehouse mobile manipulator's budget at the rack end, and one term remains. The stopping envelope is the set of states from which a validated braking response can arrest motion before a boundary, and the irreversibility law (principle 1) sets its test, $d_{\text{stop}} \leq D_{\text{clear}}$ with every term of d_{stop} taken from a measured limit of this

machine. Reserving that response costs achievable speed, because velocity increases blind travel linearly and braking travel quadratically. The machine must remain inside this envelope before a fault or unsafe proposal occurs.

The pre-brake delay τ_{delay} in the stopping distance of section 2.3 already includes the enforcer's own permission-loop tick, which section 4.3 budgets with the lease path. What remains is the gap between the reference and the plant that follows it.

The permission path closes that gap with the tracking bound ϵ_{track} of section 12.2 and defends the stopping distance of equation 2.1 inset by it. A commanded reference can be admitted only if the plant, which may lag that reference by up to ϵ_{track} , can still stop inside D_{clear} . The admission condition is the stopping envelope:⁶

$$v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{eff}}} + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \leq D_{\text{clear}} \quad (12.1)$$

Here a_{eff} is the deceleration the resident stop actually uses, equal to the credible deceleration a_{brake} for a constant-deceleration stop and smaller when the stop is shaped as in section 11.5; δ_{loc} is the localization bound and δ_{margin} the fixed protective clearance. The left side is the stopping distance $d_{\text{stop}}(v)$ of equation 2.1 plus the tracking inset ϵ_{track} .

Under those delay, estimate, and braking assumptions, any trajectory proposal that places the machine at a distance less than $d_{\text{stop}}(v) + \epsilon_{\text{track}}$ in equation 12.1 from a physical constraint along the direction of motion is unsafe and physically inadmissible. The stopping envelope is fundamentally anisotropic; an obstacle behind the instantaneous velocity vector requires clearance only for the positional tracking error ϵ_{track} .

For the warehouse mobile manipulator, the inset is the base's tracking bound of 40 mm, and it is the last distance term the budget receives. At the 1.5 m/s drive limit, the matched stop suffix of section 11.5 had already carried the running total to 1,194.15 mm, past the 1.10 m clear distance at the rack end. The inset brings it to 1,234.15 mm, 134.15 mm more than the clear distance.

Every other term in equation 12.1 is now fixed by the chapter that owns it: the credible deceleration and the fixed overheads by section 2.3, the lease path by section 4.3, the observation age by section 8.3, the stop's shape by section 11.5, and the inset here. Speed is the one term the machine can still choose. With $\tau_{\text{delay}} = 133.6$ ms and the suffix's $a_{\text{eff}} = 1.33$ m/s² (equation 11.5), equation 12.1 holds up to a ceiling of 1.39 m/s. The finished budget therefore sets the aisle speed at 1.3 m/s, below that ceiling. At the aisle speed the stop needs 997.4 mm and leaves 102.6 mm of the clear distance unspent. Table 12.1 lists every row at both speeds, and figure 12.2 draws them.

Table 12.1: The warehouse mobile manipulator's stopping budget: Each numbered row is the distance one chapter adds, loaded and on a dry floor, at the 1.5 m/s drive limit and at the 1.3 m/s aisle speed; the running total and remainder are at the drive limit, against the 1.10 m clear distance. Row 6 replaces constant-deceleration braking with the resident C^2 stop, so its entry is the added distance. Time rows add no distance.

Row	Added in	Term	At drive limit (mm)	Running total (mm)	Left of D_{clear} (mm)	At aisle speed (mm)
1	section 2.3	Braking, $v^2/2a_{\text{brake}}$	562.5	562.5	537.5	422.5
2	section 2.3	Overhead, $\delta_{\text{loc}} + \delta_{\text{margin}}$	150	712.5	387.5	150
3	section 2.3	Brake onset, vT_{act}	30	742.5	357.5	26
—	section 3.4	Time only: the chunk policy can renew a lease; the intent model's 98.0 ms weight sweep cannot	—	—	—	—
4	section 4.3	Lease path, $v(T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}})$ over 62 ms	93	835.5	264.5	80.6
5	section 8.3	Observation age, $v t_{\text{age}}$ over 51.6 ms	77.4	912.9	187.1	67.08
—	section 9.6	Time only: belief that an occluded rack end is clear expires in 31.25 ms, under two camera frames, so memory cannot certify it	—	—	—	—

⁶ | **Kinematic stopping distance and latency components:** The inset stopping distance in this model compounds five terms: blind travel, braking distance under the credible deceleration, the localization bound, the fixed protective clearance, and the transient tracking bound. Omitting computational transport delay or tracking error results in premature encroachment on physical obstacles before peak braking can engage. See section C.13 for formal kinematic derivations.

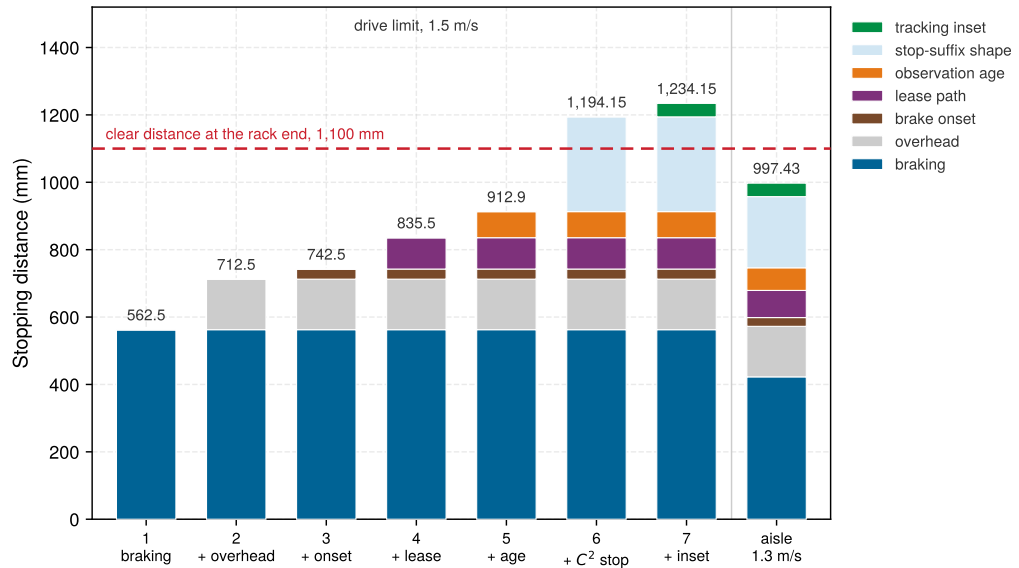


Figure 12.2: The stopping budget, row by row: Each bar is the running total after one chapter adds its term, at the drive limit, loaded, on a dry floor. Braking, overhead, brake onset, the lease path, and the observation age stay inside the clear distance; the matched stop suffix carries the total past it, and the tracking inset widens the overrun. The final bar is the finished budget at the chosen aisle speed, which fits inside the clear distance with the remainder that later chapters spend.

Row	Added in	Term	At drive limit (mm)	Running total (mm)	Left of D_{clear} (mm)	At aisle speed (mm)
6	section 11.5	C^2 stop suffix, $0.75v^2/a_{\text{brake}}$ in place of $0.5v^2/a_{\text{brake}}$	281.25	1,194.15	-94.15	211.25
7	section 12.5	Tracking inset, ϵ_{track}	40	1,234.15	-134.15	40
		Total	1,234.15			997.43
		Left of D_{clear}	-134.15			102.57

The finished budget protects a static obstacle at D_{clear} , such as a person who has stepped out from the rack and stopped or a dropped tote, so it assumes that the obstacle stays where it was when the stop began. A person who keeps walking toward the base through the whole stop closes the gap from the far side as well (section 9.6), and counting that approach on top of the finished budget would cap the base at 0.46 m/s at this rack end. An aisle held to that speed would slow every run for an encounter that a crossing rule can keep out of the stop, so the site answers the walking person with the rule rather than with speed, and section 17.3 keeps open the premise the rule rests on, that no person approaches the rack end during a stop. The enforcer therefore guarantees the budget against a stopped obstacle, not the safety of a person who keeps walking into the stop.

Later chapters spend the remainder or restrict the premise beneath it. At the aisle speed the unspent distance lasts 78.9 ms (section 14.3). The credible deceleration binds only on floors with friction of at least 0.204, since $a = \min(a_{\text{brake}}, \mu g)$, so on a floor inspected to 0.12 the ceiling falls to 1.09 m/s (section 17.3).

A stalled proposer would spend more than the whole remainder. Were the chunk lease of section 12.3 left to the host to honor, a last proposal that kept its authority for 200 ms instead of the 60 ms lease would add 182 mm to the stop and carry the base 79.4 mm past the clear distance at the rack end.

The same budget also sizes any protective sensing that stops the base independently of the proposer. A mobile base can sense the rack end through a risk-assessed protective field designed under the applicable vehicle and machinery standards, including ISO 3691-4 and ISO 13849, such as



Figure 12.3: Industrial safety laser scanner: A scanner can provide an early warning field and a protective field. The warning field requests earlier deceleration; the protective field's safety output feeds a controller that selects a risk-assessed stop. Protective-field length must include scanner response, controller and drive delay, braking travel, and uncertainty. A moving AMR does not become safe merely because motor torque is removed.

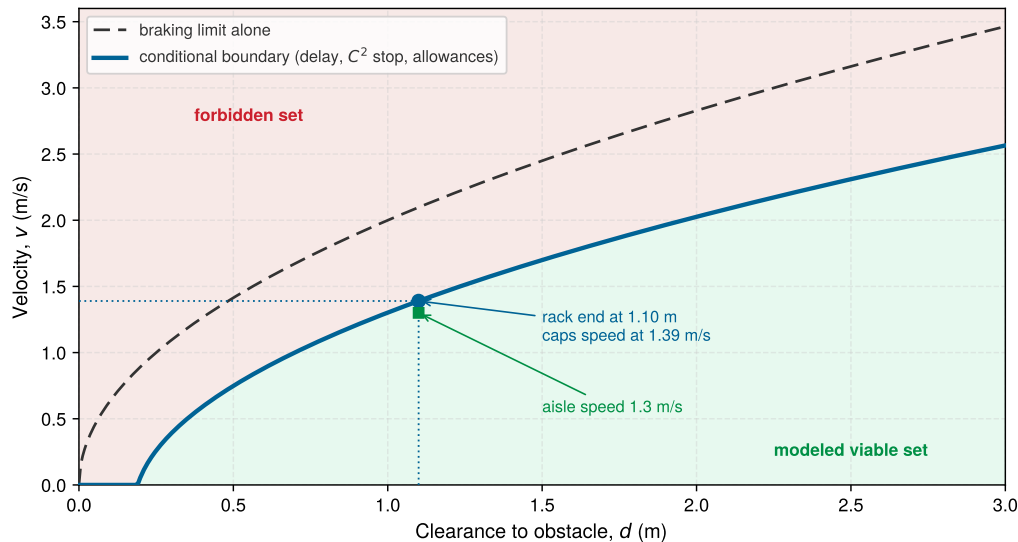


Figure 12.4: Conditional stopping envelope: For the warehouse mobile manipulator's base, a parabolic boundary separates states with modeled braking clearance from states that have already exhausted it. The inset includes the pre-brake delay, the matched stop suffix's gentler deceleration, and the localization, protective, and tracking allowances. The curves are valid only for the stated credible deceleration and operating envelope.

the time-of-flight safety laser scanner shown in figure 12.3, and that protective field must be at least as long as the stopping budget.

In the distance–velocity plane of figure 12.4, the stopping equation draws a parabolic boundary between states that can still stop and states that have exhausted their clearance, and the gap to the braking-only curve is the clearance consumed by pre-brake delay, the matched stop suffix's gentler deceleration, and the fixed allowances. As the roofline model (Williams et al. 2009) caps attainable

throughput by memory bandwidth, the stopping envelope caps legal speed by clear distance, however quickly the chunk policy proposes. To run faster, the engineer must raise the credible deceleration a_{brake} , shorten τ_{delay} , or tighten ϵ_{track} and δ_{loc} .

These parameters are not stationary. An item in the gripper raises the arm's inertia and lowers the deceleration its joints can produce within peak torque, oil on the floor lowers the traction beneath the base's credible deceleration, and repeated hard stops heat rotors and inverters into thermal derating (section 2.5). An enforcer that treats a_{brake} as a datasheet constant underestimates the envelope precisely under heavy load, so it must use independently bounded braking capability and reduce permitted speed when validated friction, payload, or thermal conditions change; an uncertain estimate cannot by itself restore a guarantee.

Adaptation works only while the enforcer's state estimates are accurate and the actuators can still supply the force. If contamination takes traction toward zero or a sustained duty cycle pushes the windings past their ceilings, the assumed deceleration vanishes, and once the required torque exceeds what the hardware can produce, the enforcer can detect the violation but can no longer prevent it.

12.6 When the Enforcer Runs Out of Authority

In control engineering, **actuator saturation** is a bounded nonlinearity that adds phase lag and causes windup in feedback controllers. For the permission path it is more immediate, the instant a motor is delivering all it can and the enforcer loses the capacity its safety warranty assumes. When an actuator reaches its mechanical limit (peak torque), its electrical limit (bus voltage or current clipping), or its thermal limit (section 2.5), the corrective action u_{corr} cannot be delivered at the commanded magnitude, and no boundary can be protected with force the body cannot generate.

A stopping envelope computed under nominal assumptions becomes invalid as soon as authority is consumed, because torque spent holding a payload against gravity or climbing a grade is unavailable for braking.⁷ Under a steady-state bias torque τ_{bias} , the effective deceleration available to arrest angular motion is:

$$\alpha_{\text{eff}} = \frac{\tau_{\text{max}} - \tau_{\text{bias}}}{J} \quad (12.2)$$

where J is the mechanism inertia and τ_{max} is the peak dynamic torque limit. In linear translational systems with mass m and bias force u_{bias} , equation 12.2 takes the equivalent form $a_{\text{eff}} = (u_{\text{max}} - |u_{\text{bias}}|)/m$. A drive that spends three-quarters of its torque holding steady speed on an incline keeps one-quarter for braking, which quadruples its braking distance.

Consider the warehouse mobile manipulator's arm carrying the mug from the takeaway conveyor down toward the coworker at the packing station, with the loaded joint descending at an angular velocity $\omega_0 = 1.2$ rad/s (the scenario's values are illustrative; see the Reader Guide). The effective link length is $\ell = 0.80$ m, so the tool point moves at 0.96 m/s, inside the arm's 1 m/s free-space limit; only the final approach to the coworker's hand runs at the far lower handover speed that section 12.11 records. The joint inertia with the mug is $J = 3$ kg·m², and the joint's peak torque is $\tau_{\text{max}} = 87$ N·m. With no gravitational bias ($\tau_{\text{bias}} = 0$), the whole peak torque is available for braking, and the angular deceleration is $\alpha_{\text{nom}} = \tau_{\text{max}}/J = 29$ rad/s². The stopping angle from ω_0 is $\Delta\theta_{\text{nom}} = \omega_0^2/(2\alpha_{\text{nom}}) = 0.0248$ rad (1.42°), which moves the tool point $d_{\text{tool,nom}} = \ell\Delta\theta_{\text{nom}} = 19.86$ mm along its arc.

During the descent, however, the arm's own geometry and the mug load the joint with a downward bias of $\tau_{\text{bias}} = 65.25$ N·m, 75 percent of its peak torque. By equation 12.2, the net torque left for braking is $\tau_{\text{max}} - \tau_{\text{bias}} = 21.75$ N·m, and the angular deceleration falls fourfold to $\alpha_{\text{eff}} = 7.25$ rad/s². The stopping angle grows fourfold to 0.0993 rad (5.69°), and the tool travels 79.45 mm, an unbudgeted overrun of 59.59 mm. If the enforcer evaluates barrier envelopes with the datasheet torque τ_{max} rather than the net headroom $\tau_{\text{max}} - \tau_{\text{bias}}$, the real stop runs 59.59 mm past the predicted one. Whether that overrun ends on the station surface depends on the clearance at which braking begins, which the barrier of section 12.7 evaluates. The coworker's hand must never come within the arm's biased stop of 79.45 mm plus its blind travel before the handover channel engages.

Integral action beneath the enforcer adds a latency penalty during saturation. While the actuator is clipped and tracking error persists, the integrator keeps accumulating a command the plant cannot execute, so when the enforcer commands full braking the output stays pinned at the forward limit until that integral unwinds. If error accumulates for 200 ms, unwinding delays net braking force

⁷ | **Actuator bias torque and stopping margin loss:** When an actuator allocates continuous baseline torque to counter gravitational loading or steady-state friction, available emergency braking headroom collapses proportionally. In vertical and heavily loaded axes, this depletion of net dynamic torque causes stopping distances to quadruple under seemingly modest bias loads. For a detailed derivation of saturated authority and stopping margin contraction, see section B.2.

by $\tau_{\text{unwind}} = 150$ ms. On the mobile manipulator's base at the 1.3 m/s aisle speed, this delay adds $v\tau_{\text{unwind}} = 195$ mm of unbraked travel, almost twice the 102.6 mm that the finished stopping budget of section 12.5 leaves spare. Anti-windup clamping, back-calculation, and conditional integration address the loop (Åström and Hägglund 2006; Franklin et al. 1998); the enforcement requirement is to measure the unwinding delay that remains and budget its unbraked distance inside the stopping horizon.

Saturation is one of the few hardware failures with a direct, deterministic detector inside the permission path. On each 1 ms tick the permission path compares the commanded effort u_{cmd} with the realizable effort $u_{\text{act}} = \text{clip}(u_{\text{cmd}}, u_{\text{min}}, u_{\text{max}})$. A nonzero difference, or an inverter status register reporting current-limit saturation, shows that the actuator has exhausted its authority, and the duration and magnitude of the clamping track the remaining authority without noisy external state estimation.

Because consumed authority changes the stopping distance at once, the enforcer folds it into the envelope in real time, recomputing $a_{\text{eff}} = (u_{\text{max}} - |u_{\text{bias}}|)/m$ before verifying the next trajectory segment. The allowable speed falls as authority is consumed, and when the computed stop no longer fits inside the clear distance, the enforcer must command deceleration while enough authority remains.

Authority depletion must also reach the planner. A 20 Hz policy that keeps commanding accelerations to a motor the 1 kHz loop has reported saturated is planning against a body that cannot execute them, so the permission path reports saturation as an active constraint violation, revokes the active trajectory, and requires a re-plan from the machine's authority-constrained state.

Systems Perspective 12.3: The actuator authority invariant

A Control Barrier Function, as formulated in section B.2, is mathematically valid only while physical actuators retain sufficient current, voltage, and thermal headroom to deliver the computed corrective force: $u^* \in [u_{\text{min}}, u_{\text{max}}]$. When steady-state gravity or friction consumes the actuator's operating envelope, the enforcer must dynamically contract the safe set before control authority at the margin vanishes.

Checkpoint 12.1: Stopping envelope and actuator headroom

Consider how the stopping envelope, and the actuator authority beneath it, bound permissible motion:

- ☐ **Quadratic envelope scaling:** Can you explain why the stopping budget grows faster than speed, so that the finished budget, rather than the drive limit, sets the aisle speed, and why no planning policy can bypass this kinematic floor?
- ☐ **Transport lag vs. braking power:** Can you distinguish between the spatial cost of computational transport delay ($v \cdot \tau_{\text{delay}}$) and active braking ($v^2/2a_{\text{brake}}$), identifying which term dominates at low crawling speeds versus high transit speeds?
- ☐ **Effective deceleration collapse:** Can you describe the physical mechanism that causes stopping distance to quadruple when steady-state gravity consumes 75 percent of peak actuator torque?
- ☐ **Dynamic safe set contraction:** Can you explain why the permission path must actively shrink the admissible state space when an actuator approaches thermal or electrical saturation, rather than waiting for tracking error to breach a hard limit?

12.7 Safe Sets as Conditional Permission

Once its bias torque is counted, the descending arm of section 12.6 needs 79.45 mm of tool travel to stop, and that travel grows with the square of approach speed. A clearance that suffices at one speed is therefore too short at a higher one, and the enforcer needs a predicate over position and speed together that it can evaluate on every tick of the control clock. In control theory, a **safe set** $\mathcal{C}$ is

defined as the zero-superlevel set of a continuously differentiable scalar certificate $h(x) : \mathcal{X} \rightarrow \mathbb{R}$, such that

$$\mathcal{C} = \{x \in \mathcal{X} \mid h(x) \geq 0\}, \quad \partial\mathcal{C} = \{x \in \mathcal{X} \mid h(x) = 0\}.$$

The mathematical foundation of **forward invariance**, established by Blanchini (Blanchini 1999), formulated through Hamilton-Jacobi reachability analysis and viability kernels by Mitchell, Bayen, and Tomlin (Mitchell et al. 2005), and extended through Control Barrier Functions by Ames et al. (2014) and Ames et al. (2019),⁸ characterizes conditions under which an admissible controller can keep a state within a viable set. Geometric clearance alone is not that set when stopping torque is finite.

⁸ | **Nagumo viability theorem and boundary tangent cones:** Nagumo's theorem establishes that a closed set is forward-invariant if and only if the system velocity vector belongs to the contingent cone of the set at every boundary point. Control Barrier Functions synthesize this geometric condition into real-time affine inequality constraints on actuator torque. Violating this subtangentiality condition causes immediate set exit, rendering future boundary enforcement unachievable; see section B.2.



Definition 12.1: Control barrier function safety filter

Control barrier function safety filter is the constrained optimization filter that projects unverified nominal control proposals $\mathbf{u}_{\text{nom}}$ onto a forward-invariant safe set $\mathcal{C} = \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}$ by solving the quadratic program:

$$\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad \mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x}), \quad \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}$$

that enforces linear braking half-space constraints ($\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$), leaving candidate inputs that satisfy the safe boundary untouched and minimally projecting violating inputs in control space.

1. **Significance:** Rather than attempting the impossible task of proving formal safety across billions of uninterpretable neural network weights, a CBF-QP filter decouples high-level policy innovation from low-level safety enforcement. The learned model is free to explore complex manipulation or navigation strategies, while the deterministic enforcer can preserve a validated viable set while its physical assumptions remain true.
2. **Distinction:** Unlike a heuristic clamp that saturates control channels independently (destroying multi-axis coordination), a CBF-QP filter performs an optimal orthogonal projection in control space, minimizing the chosen torque-space distance; it does not automatically preserve a Cartesian path.
3. **Common pitfall:** Formulating barrier constraints using raw geometric coordinates without accounting for relative degree ($r = 2$) or braking inertia. If the barrier does not incorporate stopping distance ($\Delta d_{\text{stop}} = v\tau_{\text{delay}} + v^2/2a_{\text{brake}}$), the filter intervenes too late, after the vehicle's momentum has already made a boundary collision inevitable.

When the tracker turns a learned policy's unverified setpoints into a nominal torque $\mathbf{u}_{\text{nom}}$, the enforcer does not forecast the machine's entire future path. It evaluates a single half-space inequality at the current state $\mathbf{x}$, the linear constraint that bounds the safe approach toward the boundary. If the proposed action satisfies $\mathbf{a}(\mathbf{x})^\top \mathbf{u}_{\text{nom}} \leq b(\mathbf{x})$, the enforcer passes it to the motor untouched; if it violates a feasible half-space, the enforcer projects its torque vector.

A barrier on the arm's distance to the station surface alone would register the danger only once momentum had already made contact inevitable,⁹ so the barrier must carry the kinetic stopping angle ($\omega^2/2\alpha$ for approach rate ω and braking bound α) as well as position, a requirement that follows from **relative degree**. Actuators command current and torque, torque produces acceleration, and position therefore lies two integrals from the input:

$$\text{Actuation } u \xrightarrow{\int \frac{1}{m} dt} \text{Velocity } v \xrightarrow{\int dt} \text{Position } x$$

This two-step cascade defines relative degree $r = 2$. Differentiating clearance once yields velocity:

$$\dot{d} = -v$$

⁹ | **Relative degree and inertia-aware barrier certificates:** When a safety barrier monitors position while the actuator commands torque, the control input is separated from the constraint by two time integrals ($r = 2$). Enforcing a zero-order spatial barrier without embedding kinetic stopping distance produces an optimization problem with zero control authority on the boundary. A velocity-dependent barrier can support a conditional forward-invariance argument while braking remains feasible and the state, delay, and tracking margins hold; see section B.2.

The input u does not yet appear, because a motor cannot step velocity instantaneously. Only the second derivative brings in Newton's second law ($m\ddot{x} = u - d_{\text{ext}}$):

$$\ddot{d} = -\dot{v} = -\left(\frac{u - d_{\text{ext}}}{m}\right)$$

A raw-distance rule $\dot{d} \geq -\gamma d$ cannot directly constrain this torque input: $\dot{d} = -v$ contains no u . For an approaching arm, let $D = d_{\text{tool}}/\ell$ be the remaining angular clearance, $\omega > 0$ its approach speed, $J\dot{\omega} = \tau + \tau_{\text{bias}}$, and α a validated lower bound on available angular braking. Define the kinetic barrier

$$h(D, \omega) = D - \frac{\omega^2}{2\alpha}, \quad \dot{h} = -\omega - \frac{\omega(\tau + \tau_{\text{bias}})}{J\alpha}. \quad (12.3)$$

The raw distance D has relative degree two; this velocity-dependent h has relative degree one for $\omega > 0$. Requiring $\dot{h} + \gamma h \geq 0$ produces the actual affine torque half-space

$$\tau \leq -\tau_{\text{bias}} + J\alpha \left(\frac{\gamma h}{\omega} - 1 \right), \quad -\tau_{\text{max}} \leq \tau \leq \tau_{\text{max}}. \quad (12.4)$$

At standstill ($\omega = 0$), the controller uses a separate no-approach and acceleration admission check; it does not divide by zero.

Carry the descending-arm scenario of section 12.6 into this barrier: $J = 3 \text{ kg}\cdot\text{m}^2$, $\tau_{\text{bias}} = 65.25 \text{ N}\cdot\text{m}$ toward the station, $|\tau| \leq 87 \text{ N}\cdot\text{m}$, $\alpha = (\tau_{\text{max}} - \tau_{\text{bias}})/J = 7.25 \text{ rad/s}^2$, $\ell = 0.80 \text{ m}$, $\omega = 1.2 \text{ rad/s}$, and $\gamma = 10 \text{ s}^{-1}$. These are stated joint-level scenario inputs; the bias is not inferred from the payload mass alone. At 110 mm of tool clearance to the station surface, $D = 0.1375 \text{ rad}$ and $h = 0.0382 \text{ rad}$, so equation 12.4 requires the joint to brake with at least 80.1 N·m. At 79.45 mm, $h = 0$ and the requirement reaches 87 N·m, the joint's full peak torque. At 70 mm, it demands 89.1 N·m, more than the joint can produce, so this state is already outside the modeled viable set. The enforcer must prevent entry into it; a fallback started there cannot restore the lost clearance.

A real machine realizes neither exact state feedback nor instantaneous torque. Stator current rise, bus jitter, and gearbox compliance delay and soften the corrective action, so the enforcer evaluates the barrier against a boundary inset by the worst-case stopping deficit they cause and intervenes earlier in the trajectory.

The safety warrant provided by the barrier calculation is strictly conditional. A forward-invariance claim requires a feasible barrier and the following bounded conditions; the list is necessary for this design, not an exhaustive if-and-only-if test:

1. **Model Fidelity:** The nominal plant dynamics $f(x)$ and $g(x)$ must capture the true system dynamics within a certified parameter error bound.
2. **Disturbance Bounds:** External forces and unmodeled friction acting on the mass must remain strictly within the modeled disturbance envelope $|d(t)| \leq d_{\text{max}}$.
3. **Actuation Authority:** Physical actuators must possess sufficient voltage, current, and thermal headroom to realize the commanded corrective force without saturation ($u^* \in [u_{\text{min}}, u_{\text{max}}]$).
4. **Solver Determinism:** The optimization solver or projection algorithm must terminate strictly within its allocated execution budget ($T_{\text{solve}} \leq T_{\text{budget}} < T_{\text{loop}}$).
5. **Discretization Invariance:** The discrete sampling interval T_{ctrl} must be sufficiently small that inter-sample state drift does not breach the continuous-time invariance boundary.

Each condition maps to a budget. Model fidelity and disturbance bounds come from system identification and load testing on the bench, actuation authority from dynamometer curves and thermal limits (section 2.5), solver determinism from static worst-case execution time analysis on the MCU, and discretization from the loop period, which at $T_{\text{ctrl}} = 1 \text{ ms}$ must keep the zero-order-hold drift between samples to a fraction of δ_{margin} .

A condition that no detector watches is an assumption, so the permission path attaches one to each: a **disturbance observer** compares predicted with measured acceleration on every cycle, inverter status registers and current shunts report actuator saturation, a hardware watchdog and cycle counter time the solver against its deadline, and timestamp counters on the sensor bus catch jitter and missed samples.

When any detector trips, the premise of the safe set has failed, and the enforcer must not recompute the barrier with optimistic parameters. It transfers authority through the output selector to a state-matched fallback whose response time and braking margin were reserved before the fault; later runs can only limit harm once the viable set is lost. The safe set is a valid permission only while its operating contract is intact, and the permission path enforces safety by knowing when that contract has failed.

12.8 Minimal Intervention

Suppose the chunk policy proposes setpoints from which the tracker computes a torque that the kinetic barrier of equation 12.4 forbids. Replacing the learned commands with a hand-crafted conservative controller discards the competence that justifies the model, and braking hard at the first boundary encounter creates acceleration spikes and interrupts the task. The principle of **minimal intervention** instead leaves an admitted proposal unchanged when it satisfies every active constraint and changes it only when projection is both needed and feasible. Enforcement then acts as a **safety shield** (projection operator), accepting the proposal as the default intent and computing the smallest correction that keeps the system within the invariant set.

This filter is the CBF-QP of 12.1 (Ames et al. 2016). At each discrete control interval $T_{\text{ctrl}} = 1.0$ ms on the bare-metal microcontroller, the tracker computes the nominal torque command $\mathbf{u}_{\text{nom}} \in \mathbb{R}^m$ each tick from the admitted reference (section 11.2), and the enforcer solves the convex problem¹⁰

$$\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad \mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x}), \quad \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max} \quad (12.5)$$

where $\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$ represents a torque constraint derived from a velocity-dependent barrier such as equation 12.4,¹¹ and $[\mathbf{u}_{\min}, \mathbf{u}_{\max}]$ represents the verified actuator torque envelope.

When enforcing a single active linear barrier constraint $\mathbf{a}^\top \mathbf{u} \leq b$ in equation 12.5, the optimization admits an exact closed-form orthogonal projection, derived in section B.2.4:

$$\mathbf{u}^* = \mathbf{u}_{\text{nom}} - \frac{\mathbf{a}^\top \mathbf{u}_{\text{nom}} - b}{\|\mathbf{a}\|^2} \mathbf{a} \quad (12.6)$$

where the optimal Lagrange multiplier $\lambda^* = (\mathbf{a}^\top \mathbf{u}_{\text{nom}} - b)/\|\mathbf{a}\|^2$ scales the minimal intervention vector $\Delta \mathbf{u} = -\lambda^* \mathbf{a}$.

The projection arithmetic is clearest on a two-axis manipulator with illustrative values. Its tracker, following setpoints from an unconstrained neural policy, computes nominal joint torques $\mathbf{u}_{\text{nom}} = [3.5, 3]^\top \text{ N} \cdot \text{m}$. Suppose an abstract safety half-space has already been derived and validated, $\mathbf{a}^\top \mathbf{u} \leq b$ with $\mathbf{a} = [2, 1]^\top$ and $b = 5 \text{ N} \cdot \text{m}$, with per-joint actuator limits of $\pm 5 \text{ N} \cdot \text{m}$. The candidate gives $\mathbf{a}^\top \mathbf{u}_{\text{nom}} = 10 \text{ N} \cdot \text{m}$ and breaches the boundary by $5 \text{ N} \cdot \text{m}$.

By equation 12.6, $\lambda^* = 1$, and the minimal-intervention command is $\mathbf{u}^* = [1.5, 2]^\top \text{ N} \cdot \text{m}$, which meets the half-space exactly while both joints stay within their limits, with an intervention norm of $2.236 \text{ N} \cdot \text{m}$.

Per-axis clamping shows why the projection is needed. Clamping Joint 1 to $2 \text{ N} \cdot \text{m}$ while passing Joint 2 at $3 \text{ N} \cdot \text{m}$ gives $\mathbf{a}^\top \mathbf{u}_{\text{clip}} = 7 \text{ N} \cdot \text{m}$, still $2 \text{ N} \cdot \text{m}$ outside the constraint, and it skews the torque vector from 40.6° to 56.3° . The clamp therefore both violates the constraint and distorts the command. The projection satisfies the constraint with the smallest possible torque change, although it too rotates the torque vector, so the end-effector path still needs a check against the coupled plant model.

Figure 12.5 shows both views. In state space (panel a), proposals that would cross the barrier are deflected along the inset boundary $\partial \mathcal{C}$; in control space (panel b), the enforcer projects $\mathbf{u}_{\text{nom}}$ onto the half-space $\mathbf{a}^\top \mathbf{u} \leq b$, while the independently clamped vector stays outside it.

Filter choice joins a mathematical condition to a measured execution budget and the machine's available braking authority. As summarized in table 12.2, architectural choices differ in their handling of relative degree, solver worst-case execution time (WCET), forward invariance guarantees, and directional fidelity. Independent clamping can miss coupled constraints; a CBF-QP can enforce a derived half-space if it is feasible and finishes within the verified cycle. Predictive filters (Wabersich and Zeilinger 2021) trade a longer horizon for configuration-dependent compute cost.

Prerequisite: Nominal trajectory setpoints and feedforward torques are synthesized by the planner in section 11.3.

10 | Worst-case execution time and active-set QP complexity: On bare-metal safety microcontrollers, QP safety filters execute within static memory arrays using fixed iteration limits. While typical active-set solve times range between $60\text{--}120 \mu\text{s}$, an uncapped solve can run to $600 \mu\text{s}$ under ill-conditioned or nearly degenerate constraint geometries, longer than the whole permission deadline. The fixed iteration cap exists to rule out that case. It bounds the solve to its declared share of the cycle, so the timing budget covers the capped solver rather than the uncapped worst case.

11 | Control barrier functions and Lie derivatives: In nonlinear control theory, the unforced drift and actuator control authority are formalized using Lie derivatives ($L_f h(\mathbf{x})$ and $L_g h(\mathbf{x})$). When the safety boundary has relative degree one, the control input appears directly in the first time derivative of the barrier function, yielding an instantaneous affine constraint on commanded torque. If the relative degree is two, the enforcer must evaluate higher-order derivatives to prevent actuator saturation from rendering boundary violations inevitable; see section B.2.

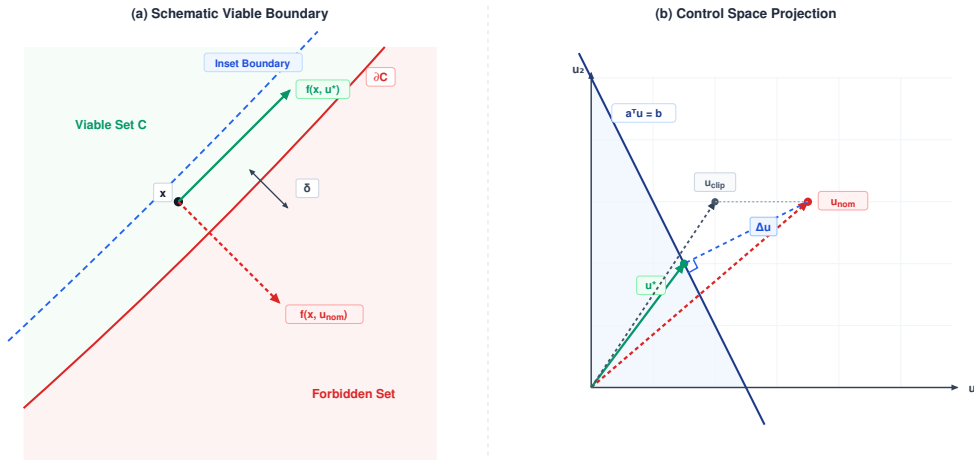


Figure 12.5: Kinetic barrier and torque projection: Panel (a) schematically shows a velocity-dependent viable boundary inset from geometric clearance; the raw distance alone has relative degree two for torque. Panel (b) illustrates Euclidean projection onto a supplied torque half-space. The $[2, 1]$ normal and $5 \text{ N} \cdot \text{m}$ threshold are abstract arithmetic inputs, not coefficients derived from the arm example.

Table 12.2: Safety-filter decisions: A filter’s physical claim depends on the chosen barrier and validated plant, not on the algorithm name. Nominal solve measurements are not worst-case deadlines.

Filter	Input and condition	What it can establish	Main validation task
Independent torque clamp	Each axis checked separately	Hardware amplitude limit, not coupled stopping clearance	Test coupled wrench and stopping behavior
First-order CBF-QP	Barrier whose first derivative contains the input	Conditional invariance from a viable initial state	Validate model, feasible torque set, and cycle bound
Kinetic / higher-order barrier	Position constraint with speed and braking authority included	Conditional stopping margin before geometric contact	Bound bias torque, delay, estimate error, and discretization
Precomputed viable region	State-to-permitted-input lookup	Containment for the modeled region	Verify lookup coverage and interpolation
Predictive safety filter	Finite-horizon trajectory and terminal set	Recursive feasibility only under terminal and timing assumptions	Bound full solve time and fallback on timeout

Operating near an active safe-set boundary can produce chattering. If a learned policy repeatedly proposes an unsafe command, sampled enforcement may switch the active constraint across successive 1 ms ticks. Depending on the plant and controller, rapid torque changes can excite a structural mode or increase motor heating and transmission wear. A tested boundary layer or hysteresis can reduce switching, but its added tracking error must fit inside the validated stopping margin.

The intervention vector $\Delta u = u^* - u_{\text{nom}}$ also measures upstream policy health. A policy operating inside its training distribution produces zero intervention on most cycles, so a rising intervention frequency or magnitude can indicate proposal drift, a changed task, or a changed plant. Summed over an illustrative rolling window of $W = 500 \text{ ms}$ as $\sum_k \|u_k^* - u_{\text{nom},k}\| \Delta t$, the metric lets the permission path flag divergence for logging or operator notification while the per-tick limits still protect the plant.

In this heterogeneous design, the shield executes inside a 1 kHz timer interrupt routine on the MCU. An unloaded cycle of acquisition, validation, and a typical active-set solve completes in about $135 \mu\text{s}$, an illustrative estimate rather than a bound. The qualified path instead declares a worst-case execution time of $250 \mu\text{s}$ for the whole cycle (section C.3), divided among the stages below, and the whole sequence must finish inside the $400 \mu\text{s}$ deadline:

1. **Interrupt Entry & State Acquisition (30 μ s):** The hardware timer triggers the ISR, and firmware latches the wheel and joint encoder registers and the latest IMU sample.
2. **Proposal & Lease Verification (15 μ s):** The MCU inspects the shared SRAM ring buffer populated by the application processor over RPMsg or shared memory. It validates the checksum, confirms sequence order, reads the proposal's evidence epoch, and makes two age checks against it and the lease. The upper age of the evidence the proposal was built on, stamped at dispatch, may not exceed the 51.8 ms refusal threshold, which is the 51.6 ms observation age the stopping budget charged plus the 0.2 ms clock-conversion bound (section 8.7). The proposal's own age on the MCU clock may not exceed the 60 ms chunk lease. The MCU also reads the host heartbeat timer, which marks the host silent once no heartbeat has arrived within the 30 ms timeout.
3. **Tracking Contract Verification (10 μ s):** Firmware evaluates instantaneous tracking error $e(t) = \|x(t) - r(t)\|$ against the certified bound ϵ_{track} .
4. **CBF-QP Safety Shield Solve (150 μ s):** An active-set QP solver running in pre-allocated static DTCM SRAM (section 4.2) evaluates active barrier inequalities and computes $u^* = \arg \min_u \frac{1}{2} \|u - u_{\text{nom}}\|^2$. A fixed iteration cap holds the solve inside its share even for ill-conditioned constraint geometry; nominal solves for one or two active constraints finish well within it. Dynamic memory allocation is forbidden, because a heap search has no bounded duration.
5. **Output Selection & Setpoint Staging (15 μ s):** If the QP is feasible and constraints hold, u^* is staged as the admitted setpoints for the next EtherCAT frame to the drives, whose current loops apply it and whose torque accuracy is tested rather than inferred from their rate; a proposal that fails validation leaves the previously admitted setpoints in force. If the chunk lease has expired, the host heartbeat has fallen silent, the shared control rail reports undervoltage, tracking error exceeds ϵ_{track} , actuator saturation renders the feasible control set empty ($\mathcal{U}_{\text{safe}} = \emptyset$), or an operating-contract detector such as the disturbance observer or the solver deadline trips, the output selector revokes the policy and selects the state-matched stop. At standstill, the same lease, heartbeat, or undervoltage trigger selects a powered hold instead.
6. **Completion & Reserve (30 μ s):** The MCU checks the register write and services its own watchdog only after the cycle succeeds. The reserve absorbs variation in the stages above; a cycle that has not completed by the deadline transfers authority to the resident fallback before the next tick. A missed tick therefore costs the base 1.3 mm of travel at the aisle speed. Section 13.6 tests whether the declared WCET holds under load, so that ticks are not missed.

Every stage of this cycle assumes that the solve returns a feasible command. When saturation empties the admissible torque set, the lease lapses, or the cycle overruns its deadline, projection has nothing to return, and the permission path must already hold a response it can execute without the proposer.

12.9 The Fallback Ladder

The fallback must be selected before the admissible set becomes empty. Once it is empty, a fallback can contain a fault or limit harm but cannot promise to avoid a boundary crossing. Nor can the response be a single switch. Cutting bus power whenever a proposal brushes a barrier margin produces uncoordinated deceleration and continuous mission aborts, while relying on software optimization during an inverter shoot-through (section 2.4) burns the gate drivers before an interrupt handler can run. The permission path therefore holds several resident responses and selects among them deterministically, from feasible projection to plant-specific controlled stopping or torque removal.

Much as the arbitration of behavior-based robotics and the subsumption architecture (Brooks 1986b) lets a condition select which layer drives the actuators, the **fallback ladder** is a set of four deterministic responses (project: minimal-intervention QP projection; hold: active position hold; stop: controlled dynamic stop; inhibit: drive torque inhibit), each running on the execution substrate that survives its trigger, from which the permission path selects by the check that failed and the state of the plant when it failed, rather than climbing them in order.

The first rung, least-squares projection, runs on every cycle while the feasible control set is nonempty ($\mathcal{U}_{\text{safe}} \neq \emptyset$), computing $u^* = \arg \min_u \|u - u_{\text{nom}}\|^2$ subject to the active barrier inequalities within its 150 μs share of the cycle. It succeeds only while the barrier constraints and the achievable torques intersect, so the admission path must preserve viability before saturation empties $\mathcal{U}_{\text{safe}}$. If infeasibility occurs anyway, the MCU revokes the proposal and invokes the best available state-matched response.

Where the plant can be held rather than stopped, the response is a controlled deceleration into an energized position hold, analogous to a Category 2 stop.¹² The permission path arrests motion and holds the current coordinate x_{hold} under closed-loop feedback, with the drives still modulating current against gravity and payload. The rung spends electrical energy and accumulates thermal debt in the windings (section 2.5), but it wears no brake and preserves the kinematic state, so after a fresh proposal and state check the enforcer may resume without homing. On the warehouse mobile manipulator this rung answers a lapsed lease or a silent heartbeat only at standstill. The same expiry in motion selects the stop rung, because the resident stop is the only deceleration whose time and distance the stopping budget has admitted. If tracking exceeds its bound, the MCU selects a validated stop, and holding remains appropriate only while available torque and feedback support it.

When a body in motion loses its proposal's authority or a tracking or model assumption fails, the response is a controlled dynamic stop, analogous to a Category 1 stop under IEC 60204-1. On the warehouse mobile manipulator's base, this rung executes the resident stop suffix admitted with the trajectory (section 11.5) rather than composing a stop at the moment of the fault. From the 1.3 m/s aisle speed, the C^2 suffix peaks at the 2 m/s² credible deceleration, lasts 975 ms, and covers 633.75 mm, the braking term already reserved in the stopping budget of section 12.5. The kinetic energy the stop removes regenerates through the inverter into the DC bus (section 2.6), so bus capacitance and a brake chopper must absorb the surge without an overvoltage fault.¹³ Power removal, if specified, follows the measured standstill condition.

A separate last-resort drive function can inhibit motor-generated torque. Drive Safe Torque Off (STO) does not itself disconnect the DC bus, apply a mechanical brake, or stop a moving motor; the motor may coast. In the illustrated architecture, an external safety controller may also command a separately rated contactor and spring-applied brake. Their response and stopping ability must be measured as one plant-specific path, because removing torque while a gravity-loaded arm or moving vehicle still depends on powered braking can worsen the hazard. On the warehouse mobile manipulator that path, STO plus the nine spring brakes setting 30 ms to 80 ms after coil decay, is the inhibit rung, and it is unbudgeted. It answers only the loss of the MCU's own cycle, and no stopping or release claim rests on it.

Loss of supply does not route there. The permission path runs on an isolated rail held up for 2 s (section 2.8), so a sag on the shared control rail arrives as an undervoltage flag, and the enforcer answers it with the stop rung. The hold-up outlasts the longest stop the enforcer can command inside the envelope. Stop onset follows the flag within 22 ms, the resident stop from the 1.09 m/s ceiling on a floor inspected to a friction of 0.12 takes 1395.4 ms, and the brakes set at standstill within 80 ms, so every brake is set 1497.35 ms after the flag, with 502.65 ms of hold-up to spare.

Physical AI systems engineers must distinguish between **fail-safe** and **fail-operational** plants when configuring terminal fallbacks. In a machine already at a mechanically restrained standstill, removing drive torque may be safe after the restraint has been verified. In underactuated, open-loop unstable, or high-momentum machines, however, suddenly de-energizing actuators is itself a hazard:

1. **Aerial Platforms (Multirotors and eVTOLs):** Cutting motor torque drops aerodynamic thrust to zero, transforming an airborne robot into a ballistic projectile in uncontrolled free fall.
2. **Dynamic Legged Systems (Bipedal Humanoids):** During dynamic locomotion at 1.5 m/s, the center of mass is outside the base of support. Abruptly removing the torque needed for balance can cause a fall; the direction and consequence depend on the gait phase and available restraint.
3. **High-Speed Autonomous Vehicles:** Removing powered steering or braking at highway speed can eliminate the authority needed for a controlled stop; the outcome depends on vehicle dynamics and backup actuation.

12 | **Machine stop functions:** IEC 60204-1 distinguishes uncontrolled power removal (Category 0), controlled stopping followed by power removal (Category 1), and controlled stopping with power retained (Category 2). IEC 61800-5-2 drive functions such as STO, SSI, and SS2 may implement parts of those responses. The required performance level, circuit architecture, and suitable stop are determined by the machine-specific risk assessment; ISO 13849-1 does not impose one universal PL or channel count.

13 | **Dynamic brake choppers and DC-bus overvoltage clamping:** In a regenerative Category 1 stop, mechanical kinetic energy flows through the inverter bridge into DC bus capacitors ($\Delta E = \frac{1}{2}mv^2$). A suitably rated chopper and resistor can dissipate energy when the DC bus rises; switching thresholds, pulse-energy rating, and thermal behavior must be validated for the stop profile. Other designs use mechanical dissipation or a battery that can accept regeneration.

Prerequisite: Hardware trip zones and low-side MOSFET dynamic shunt braking are introduced in section 4.5.

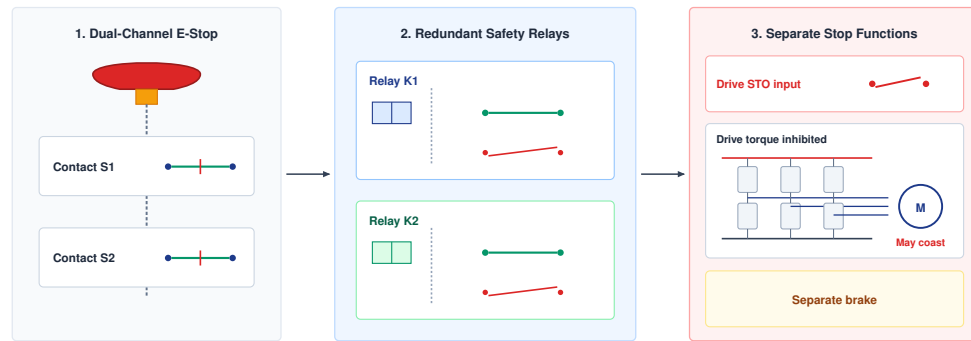


Figure 12.6: Illustrative emergency-stop circuit: Two emergency-stop contacts feed safety relays and drive Safe Torque Off (STO) inputs. A spring-applied brake is shown as a separate path; an external supply contactor, if fitted, provides galvanic disconnection. The diagram illustrates physical isolation; a moving motor can coast after STO, so physical stopping distance must be independently validated for the load and state.

For such dynamic systems, the terminal fallback cannot simply be an uncoordinated power cut. Instead, the permission path must execute a deterministic **Minimal Risk Maneuver (MRM)** running on an independent, qualified controller with the required sensing and actuator authority. For aerial and legged robots, any controlled descent or balance recovery must be validated for remaining propulsion, contact, energy, and state-estimate authority. A road vehicle needs a risk-assessed powered braking and steering path. The wiring that carries a torque-removal request is the out-of-band interlock of section 4.5 (figure 12.6), which keeps the spring-applied brake on its own path because removing torque and arresting a moving load are different functions carrying different validation evidence.

The architectural necessity of independent, hardware-level trip circuits is demonstrated by the historical consequences of eliminating physical interlocks in favor of software decision logic (12.1).



War Story 12.1: Therac-25 software interlock elimination

Context: The AECL Therac-25 was a dual-mode (25 MeV electron / X-ray) medical linear accelerator and automated radiotherapy patient table, controlled by a single PDP-11 minicomputer running uniprocessor assembly firmware (Leveson and Turner 1993).

Mechanism: In preceding models (Therac-6 and Therac-20), physical safety was guaranteed by independent hardware interlocks: electromechanical microswitches, mechanical turntable detents, and analog diode steering circuits that physically prevented the accelerator from firing high-current beams unless the tungsten beam-flattening target was mechanically locked in place. In the Therac-25, engineers removed all physical hardware interlocks to reduce manufacturing costs, delegating exclusive safety enforcement to software. Concurrently, two latent software bugs defeated the software checks: an 8-bit shared variable (`Class3`) rolled over from 255 to 0 (`0xFF` → `0x00`) during setup, bypassing interlock evaluation; and a race condition allowed fast terminal keystrokes (< 8 seconds) to update operating modes without reconciling the physical turntable position with the commanded high-current beam energy.

Impact: Between 1985 and 1987, six patients received massive radiation overdoses up to 25,000 rad (more than 100 times the intended therapeutic dose), resulting in catastrophic radiation burns, severe physical trauma, and several patient fatalities.

Response: Regulatory agencies halted machine operations, prompting comprehensive investigations that led to modern medical device software engineering standards, mandatory independent hardware safety interlocks, and formal verification of safety-critical concurrent states.

Systems lesson: Software-only safety filters executing on complex, non-deterministic compute stacks cannot replace independent bare-metal hardware trip zones. The Therac-25 disaster is the canonical historical proof that removing physical interlocks in favor of software decision logic transforms transient concurrency bugs and arithmetic overflows directly into catastrophic physical energy releases. In physical AI, software Control Barrier Functions and runtime projection shields are advisory safety filters; the terminal inhibit rung must always terminate in device-qualified hardware interlocks such as Safe Torque Off and mechanical brakes.

The rungs trade intervention time, available control authority, and recovery cost. A controlled stop requires reserved clearance and a working drive, and each rung's plant-specific response must be validated for the states from which it can be entered. Figure 12.7 arranges the four rungs by the fault condition that selects each one.

Definition 12.2: Dynamic fallback ladder

Dynamic fallback ladder is the set of resident protective responses ($u \in \{\mathcal{P}_c(u_{\text{nom}}), u_{\text{hold}}, u_{\text{brake}}, u_{\text{STO}}\}$) from which a deterministic arbiter selects one on every tick according to the check that failed (an invariant violation, a timing overrun, or a hardware fault) and the measured state of the plant, so that one trigger can select different rungs in motion and at standstill.

1. **Significance:** Physical AI systems cannot treat all faults identically; cutting actuator power on dynamic, underactuated, or high-momentum plants (e.g., legged robots at speed, multirotors in flight) can cause a fall or tumble, requiring state-matched deceleration before terminal torque removal.
2. **Distinction:** Unlike software exception handlers that unwind the call stack or kill a process thread, a dynamic fallback ladder operates against physical inertia, matching each rung to the remaining valid sensing, control authority, and reserved stopping clearance.
3. **Common pitfall:** Reading the rungs as an escalation in which a lower one, such as Safe Torque Off (STO), provides faster or safer arrest. Each rung has its own entry conditions, and the inhibit rung removes torque without arresting anything by itself.

War Story 12.2: Cruise post-impact fallback (2023)

Context: In October 2023, a human-driven vehicle struck a pedestrian into the path of a Cruise robotaxi in San Francisco (Exponent, Inc. 2023).

Mechanism: The Cruise vehicle initially braked and stopped. Its system then misclassified the event as a side impact and, having tracked the pedestrian only intermittently in the seconds before impact, initiated a pullover about 1.83 s after contact. The vehicle dragged the pedestrian roughly 20 ft at a maximum reported speed of 7.7 mph.

Response: An independent impact and underbody-occupancy response should revoke ordinary policy authority, retain braking and steering needed for a controlled stop, and hold the vehicle once stationary.

Systems lesson: A post-contact maneuver cannot infer clear space from a missing tracked object. The fallback must use independent evidence and preserve the ability to brake and hold.

The IEC 60204-1 stop categories name drive functions, not plant responses, and table 12.3 separates the function each category names from the physical response that must still be tested.

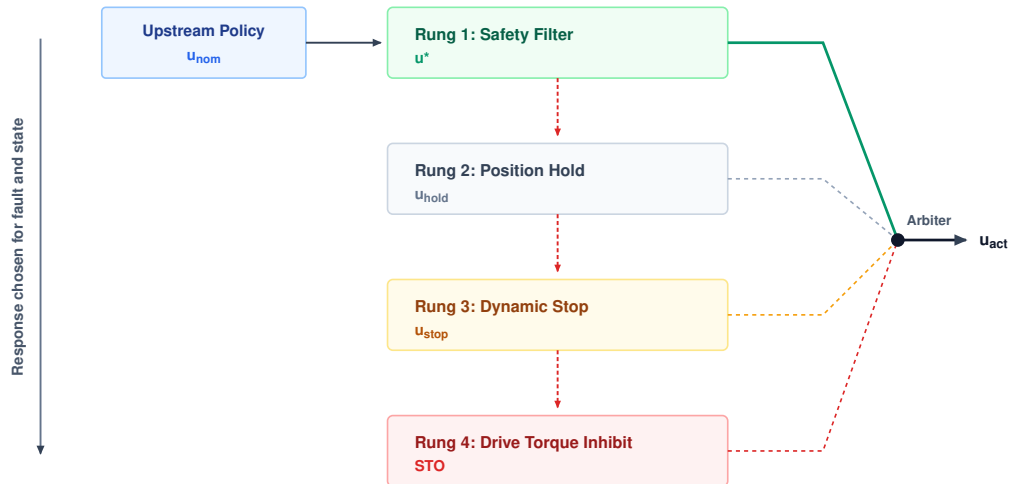


Figure 12.7: Available enforcement responses: A deterministic arbiter can select feasible torque projection, controlled hold, controlled stop, or drive torque inhibit according to the current fault and measured plant state. The responses are alternatives with different prerequisites; descending the diagram does not imply increasing physical safety or faster arrest.

Table 12.3: Stop functions and physical response: Each stop category mapped to the drive function it names and the plant response it leaves to be tested. No stop category alone supplies a safety integrity rating, a brake, galvanic isolation, or a guaranteed stopping distance.

Response	Motor torque and power	Physical stop obligation	Appropriate use
Admitted filtering	Drive remains active	Recheck the feasible kinetic barrier and tracking bound every tick	Nominal operation inside the validated envelope
Controlled hold (Category 2 / SS2-like)	Power remains for braking and hold	Verify stopping distance, thermal headroom, and static load support	Hold when feedback and torque authority remain available
Controlled stop then torque removal (Category 1 / SS1-like)	Power remains through deceleration; torque is removed afterward	Verify the full braking and transition time under load	Moving machine with enough reserved clearance
Torque removal (Category 0 / STO-like)	Drive cannot produce commanded torque; DC bus isolation is a separate function	A moving motor may coast; a separate brake or mechanical restraint needs its own timing and rating	State for which loss of torque has been shown safe



Checkpoint 12.2: Choosing the physical fallback

Before selecting a defensive fallback strategy, verify your understanding of state-dependent stopping modes, safety functions, and timing reserves:

- ☐ **State-dependent stop:** Why can powered braking be safer than STO for a moving AMR, while drive torque inhibit may be appropriate after a separately verified restraint holds a stationary arm?
- ☐ **Independent functions:** What does STO inhibit, and which separate functions would remove bus power or apply a spring brake?
- ☐ **Timing and clearance:** Which measured detection, dispatch, drive, and brake delays belong in the stop reservation?
- ☐ **Lost viability:** What can a fallback still accomplish when the admissible torque set is empty?

12.10 How Enforcement Goes Wrong

Every rung of the fallback ladder assumes that the hardware executing it stays independent of the fault that selected it. A secondary processor beside the primary compute module looks independent, yet a shared regulator, crystal, or ground plane can reset, halt, or corrupt both chips in the same instant, as when a host-side short drags the rail below the microcontroller's threshold or motor-driver return currents bounce the ground beneath its buses. Software on either chip cannot see these faults, which is why the permission path keeps its own supply, clock, and ground (section 4.6).

An isolated enforcer still fails when its computation falls out of phase with the plant, because a solve that overruns its tick acts on a state the machine has already left, and in an underdamped assembly a braking torque computed for that earlier state can feed a structural resonance instead of damping it. A watchdog that aborts the late cycle cannot recall the motion accrued during the overrun, so the timing claim rests on the declared WCET and on the loaded measurements of section 13.6.

The enforcer sees the plant only through state estimates, so filter group delay, transmission compliance, encoder slip, and thermal bias drift can hold the reported state inside its limits while the physical link has crossed them. Cross-checks against independent encoders or torque sensors catch large discrepancies, but slow sub-threshold drift stays hidden until contact.

Determinism guarantees that a computation finishes in a fixed number of cycles with bit-identical outputs, but it says nothing about whether the equations match the plant. An enforcer that assumes the base's friction premise of 0.204 on a floor that oil has made slicker plays the resident stop exactly on schedule while the tires cannot deliver its deceleration, so the base runs past the rack end with every deadline met. No timing monitor can see that failure, and section 17.3 keeps its friction premise open.

Contention on the shared system-on-chip (SoC) interconnect can defeat an enforcer that meets its own deadline, because a camera direct memory access (DMA) burst or bulk flash write that holds the bus matrix after the solve adds directly to the command's age at the drive while the inverter holds its last setpoint. The machine's placement keeps the permission path off that bus (section 13.3), so a burst on the application processor reaches it only as evidence age, and cycle stage 2 refuses any proposal the burst has aged past the refusal threshold.

An enforcer hosted as a real-time operating system task rather than a bare-metal interrupt adds priority inversion at the operating-system layer, where a low-priority telemetry thread holds a mutex the 1 kHz enforcer needs and a medium-priority thread keeps the holder from running, so the check waits without bound. The machine's timer interrupt and lock-free mailbox exclude that case by design. On a task-hosted enforcer, priority inheritance (PTHREAD_PRIO_INHERIT) or ceiling locking bounds the wait, and section 4.4 examines the failure in flight hardware.

The Therac-25 radiotherapy accelerator shows a concurrency failure with nothing independent behind it. Where its predecessors had independent protective circuits and mechanical interlocks, the Therac-25 relied chiefly on software running on the same computer that ran the treatment, where a race in operator data entry and a one-byte counter that rolled over to zero each let the machine fire a high-current beam without its target in place (Leveson and Turner 1993). Between 1985 and 1987 six patients received massive overdoses, and several died.

Every failure mode in the enforcement layer traces back to a breakdown of independence or a divergence between the internal plant model and physical reality. A barrier certificate, a dual-core package, and a zero-jitter scheduler establish safety only within the envelope where supplies stay isolated, estimates match the mechanics, and deadlines are met, so an enforcer that is to be audited against these failures has to state that envelope in advance: what it assumed, what it checked on each tick, and which response each failed check selects.

12.11 The Enforcement Record

When the warehouse mobile manipulator's base refuses a proposal at the rack end, a later reader of the log must be able to say which threshold failed, which detector measured it, and which rung it selected. The **enforcement record** is the specification contract and runtime audit ledger under which, on every tick, the drive receives either the projection of u_{nom} onto the safe set or a fallback-ladder response, and nothing else.

The record consumes four predecessors, drawn from the two strands of records that meet here. The offline strand records what the plant can do and what the proposer has shown it can do, and the runtime strand records what the proposer proposes now. The trajectory record of section 11.7 supplies the admitted reference and its matched stop suffix. The evaluation record of section 7.8 supplies the runtime monitor specifications and the coverage-gap catalog, which become the record's monitor channels. The policy manifest of section 6.8 supplies the declared ODD, and the validity region stays inside the sampled part of it. The limit records of section 2.7 supply the braking, onset, and actuator limits with their conditions. To these the enforcement record adds what the permission path guarantees: the verified tracking bound ϵ_{track} , the stopping time and distance (t_{stop} , d_{stop} , bounding the duration and displacement required to arrest plant momentum), Control Barrier Function coefficients, the loop period, per-check thresholds with their fallback-ladder rungs, and the validity region.

Filled in for the machine, the record fixes the base's stops at the rack end and the arm's validity region and channels at the door and the packing station:

- The tracking bound ϵ_{track} is 40 mm, the inset of row 7 in table 12.1.
- The resident stop is the C^2 suffix, which lasts 975 ms and covers 633.75 mm from the 1.3 m/s aisle speed, or 1125 ms and 843.75 mm from the 1.5 m/s drive limit.
- The loop period is 1 ms, with a declared WCET of 250 μs inside a 400 μs deadline. Three timers run beside it, the 60 ms chunk lease, the 100 Hz host heartbeat with its 30 ms silence timeout, and the 5 ms self-watchdog.
- The validity region for the arm covers the door and the packing station. At the door it is the target ODD of section 7.8, which admits only the nominal plate, the door's own load, a stopped base, and the guarded approach. At the packing station it admits the mug pick from the conveyor under a live intent lease (section 10.7), and the handover that follows.
- The arm's channels are ceilings the enforcer applies to the tool point. The approach-speed channel of the evaluation record's `runtime_monitor_specs` caps it at 0.03 m/s inside the door zone. The mug pick runs under the 1 m/s free-space TCP limit. While the coworker holds `accept_item` (section 14.2), a handover channel caps the approach to the coworker's hand at 0.10 m/s, below the 0.118 m/s contact-force ceiling, and it engages before the hand comes within the arm's biased stop plus its blind travel.
- For the base, the normal configuration is valid on floors with friction of at least 0.204, where the credible deceleration binds. A restricted configuration is valid down to the inspected-floor friction of 0.12; its suffix is sized for the 1.177 m/s² peak that floor allows, and it takes 1274.6 ms and 637.3 mm to stop from 1 m/s, under the 1.09 m/s inspected-floor ceiling. Each configuration carries its own digest, and the release manifest (section 16.8) names both.

Each numerical entry in the record carries the evidence category of its limit record (section 2.7), whether measured, derived, datasheet, or illustrative, and the categories carry very different real-world confidence. The base's 975 ms stopping time is derived. It follows from the shape of the C^2 suffix and from a credible deceleration that is itself illustrative, and it assumes that the drive delivers the commanded current and that the tires hold the floor throughout the stop. Only loaded stopping trials on the floor itself make it a measured value, and those trials expose what no derivation contains, among them the scatter in brake onset and the friction of the actual floor surface. Even then the measured sample stands for the bound only with a justified worst-case fault and stated operating conditions. Recording the category prevents an integrator from mistaking a derived stop for a measured hardware limit.

The record maps separately qualified thresholds to protective actions, and on the machine the routing depends on whether the body is moving. Routing does not rely on coarse error strings or unquantified health flags. The base stays on the project rung, where the CBF-QP may adjust commands, as long as tracking error satisfies $e(t) \leq \epsilon_{\text{track}} = 40$ mm, the kinetic barrier stays feasible, and the solver stays within its declared 150 μs share of the cycle budget. A new proposal that fails validation while the active lease holds is refused, and the previously admitted setpoints stay in force until that lease lapses. In motion, a lapsed lease, a silent heartbeat, an undervoltage flag on the shared control rail, tracking error beyond ϵ_{track} , saturation that empties the feasible control set, or a tripped operating-contract detector (the disturbance observer or the solver deadline) selects

Downstream: High-rate enforcer refusal events populate the forensic evidence logs analyzed in section 14.8.

the stop rung, which plays the resident stop on the base and the state-matched stop on the arm. At standstill the same lease or heartbeat expiry, or the same undervoltage flag, selects the hold rung, since no motion remains to arrest. The 5 ms MCU self-watchdog detects missed successful cycles; this design also assumes an independently wired supervisor output or external trip that requests the inhibit rung if the MCU fails. Its total trip response must be measured. A contactor, if fitted, is an independently timed supply-isolation function.

Each field of the record is either an integrity field or a threshold that one of these checks reads: the proposal's sequence number and checksum, its evidence age against the 51.8 ms refusal threshold (the budgeted observation age plus the clock-conversion bound), its own age against the lease, the tracking bound, the joint-velocity and torque-slew ceilings, the barrier margins, the phase-current and bus-voltage limits, and the watchdog timeout. The record binds each threshold to the independent hardware that measures it and to the rung that a failed check selects. Table 12.4 groups those rungs by the authority path that remains active after a failed check, together with the timer that triggers each one.

Table 12.4: Enforcement state contract: The 20 Hz proposal, 60 ms chunk lease, 100 Hz host heartbeat, and 1 kHz MCU loop have distinct renewal and failure paths.

State	Trigger and authority	Timing check	Physical response to validate
PROJECT	Fresh proposal, healthy heartbeat, feasible kinetic barrier; a proposal that fails validation under an unexpired lease is refused and the admitted setpoints stay in force	1 ms cycle; declared full-cycle WCET 250 μ s inside a 400 μ s deadline	Track admitted torque within qualified error and brake reserve
HOLD	At standstill, proposal age above the 60 ms lease, host heartbeat silent beyond 30 ms, or shared-rail undervoltage	Expiry detected on the MCU clock within one tick	Powered position hold against gravity and payload; resume only on a fresh admitted proposal
STOP	In motion, lease or heartbeat expiry, shared-rail undervoltage, tracking error beyond ϵ_{track} , an empty feasible control set, or a tripped disturbance observer or solver deadline, while powered braking remains available	Include detection, dispatch, drive rise, and brake response	Resident stop on the base, state-matched stop on the arm; measured stop distance under load; no claim after viability is lost
INHIBIT	Independent wired supervisor or external trip detects MCU cycle loss	Assumed 5 ms detection plus measured drive, contactor, or brake response	STO removes motor torque and the spring brakes set after coil decay; unbudgeted, so no stopping claim rests on it

Byte layout: The enforcement record's fields, types, and units are given in section C.14.4.

No enforcement guarantee holds when the machine operates outside its designed physical environment, so the record bounds the plant dynamics within an explicit validity region. The arm's door ceiling rests on evidence. The arithmetic alone would have admitted the chunk policy's 0.10 m/s approach of section 12.3, whose 2.5 ms contact deadline still exceeds the 2 ms contact loop, but neither the demonstrations nor the evaluation trials behind the policy include that speed (section 7.8), so the enforcer projects the proposal to 0.03 m/s and logs the channel that bound it. The handover ceiling rests on the contact model instead, sitting below the contact-force ceiling, the speed at which an unplanned touch of the coworker's hand would reach the contact criterion (section 14.2), and a faster proposal is projected and logged in the same way.

A live state outside the validity region that the machine's estimators detect, such as a payload the manifest never declared, is refused like any other failed check, and the supervisory monitors that flag it revoke policy dispatch and select the state-matched stop. A floor below the base's friction premise is not such a state, because no sensor on the machine observes the friction under its wheels. The normal configuration therefore rests on that friction as a premise, and the restricted configuration rests on inspection of the floor (section 17.3).

An enforcement record connects proposed motion to validated assumptions, thresholds, and observed fallback response. The execution platform must still meet its timing and physical braking contract under load. An enforcer placed on a shared memory bus can be mathematically verified

and still stall when that bus saturates during a high-speed transit maneuver, and then the failure occurs in time rather than in logic.

12.12 Fallacies and Pitfalls

Physical enforcement stops a machine before a collision only while its execution is timely and its state feedback usable. Fault tests therefore exercise the response when either assumption fails, not the filter's nominal equations alone.

Pitfall: *Assuming high-level policy computers can be relied upon to execute graceful shutdown.*

The warehouse mobile manipulator's application processor suffers a kernel panic while the base runs at 1.3 m/s. The failed host cannot generate a deceleration trajectory, and dropping the drive enable to zero torque would leave the base coasting toward the rack end. The MCU instead detects the silent heartbeat within its 30 ms timeout, well inside the lease path the budget reserves, and plays the resident C^2 suffix, which stops the base in 975 ms over 633.75 mm. A host shutdown handler may coordinate an orderly stop in normal operation, but protection against host failure requires a stopping path, clearance, and torque authority that remain after the host process disappears.

Fallacy: *A mathematically correct safety filter guarantees safety even when underlying state feedback has drifted.*

The base's tracking bound of 40 mm limits the gap between the reference and the state its wheel encoders report, not the gap between that state and the floor. Wheel slip or a biased encoder can make the base appear farther from the rack end than it is, and the CBF-QP solver then satisfies its inequality on the drifted estimate while the physical base spends clearance the budget never granted. Only an independent channel, such as the IMU or the safety scanner's range to the rack, exposes the divergence, and only while the fault is observable through it. Qualification must bound how far the estimate can drift before detection, and the budget must carry that error in the localization allowance, because the tracking inset cannot absorb it.

Pitfall: *Coupling safety braking authority to stochastic semantic classification rather than deterministic spatial clearance.*

When braking depends on upstream classification and path prediction, a detected object may still fail to trigger an effective stop. In the March 2018 Tempe collision (figure 12.8), the developmental automated driving system first detected the pedestrian 5.6 s before impact but did not predict her path. It recognized an imminent collision 1.2 s before impact, suppressed action for one second, then planned gradual slowing rather than emergency braking; the NTSB does not establish a Kalman-filter reset on each classification change. The vehicle's original Volvo collision-warning and emergency-braking functions were disabled during ADS operation (section 1.6). An independent geometric clearance check needs its own validated sensing, stopping authority, and timing. It refuses on distance and closing speed, whatever label the object carries.

12.13 Summary

A physical AI system assigns actuator permission to a path independent of learned proposals. A kinetic barrier can help preserve a validated viable set when state error, model mismatch, actuation, and response delay stay inside their declared bounds. If braking authority or clearance is lost, the enforcer can revoke the proposal but cannot claim that a later fallback restores the original safe set.

The physical clearance protecting a machine is never a static geometric measurement. It is a composite margin that compounds pre-brake delay, braking under the credible deceleration, the localization and protective allowances, and the tracking bound. The warehouse mobile manipulator at its 1.3 m/s aisle speed shows the sum. Blind travel over τ_{delay} takes 173.68 mm, the C^2 stop suffix 633.75 mm, the localization bound and protective clearance 150 mm, and the tracking inset 40 mm. Together they need 997.43 mm of the 1.10 m clear distance at the rack end and leave 102.57 mm. A budget that counted only constant-deceleration braking would have admitted the drive limit, where the finished budget runs past the clear distance.

A planner checks a future trajectory; a local kinetic barrier checks the current proposal against the modeled ability to stop. For torque-controlled motion, raw clearance has relative degree two. The velocity-dependent barrier in equation 12.3 produces the affine torque condition in equation 12.4 for

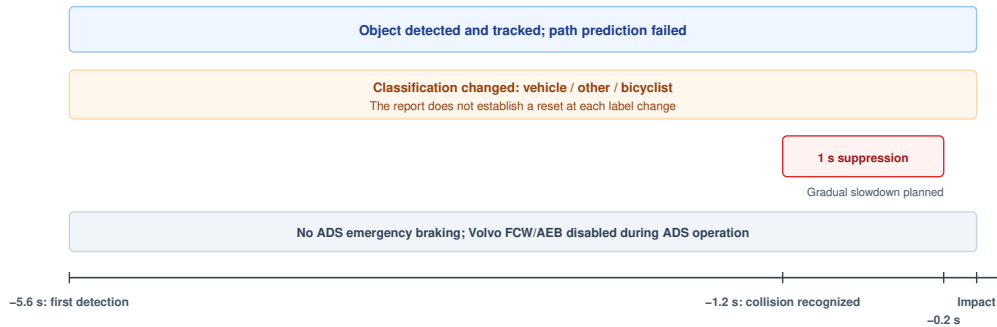


Figure 12.8: Tempe detection and braking timeline: The NTSB reports first detection 5.6 s before impact and recognition of imminent collision 1.2 s before impact. The ADS suppressed its motion plan for one second and then planned gradual slowing; it did not perform emergency braking. Classification and path prediction failed. The Volvo forward-collision warning and automated emergency braking functions were disabled while the developmental ADS operated. The report does not establish a Kalman-filter reset at each label change. (Adapted from NTSB/HAR-19/03 (National Transportation Safety Board 2019).)

positive approach speed. This local calculation still depends on a reserved stop, accurate state, and timely torque delivery.

The permission path needs an independent response when the proposal source stalls. On the warehouse mobile manipulator, a 60 ms chunk lease, a 30 ms host-heartbeat timeout, and a 5 ms MCU self-watchdog detect different failures. The MCU revokes proposal authority and chooses a prevalidated state-matched response; torque removal answers only the loss of the MCU's own cycle, since every other trigger the record routes selects a budgeted stop in motion or a hold at standstill.

The enforcement record is where the two strands of records meet. Each entry keeps its evidence category, so the record states what the permission path assumed as plainly as what it checked.



Key Takeaways: Conditional enforcement and physical stopping

- **Tracking is a bounded premise:** A steady-state error does not bound the transient, so a clearance inset needs a transient bound validated over the full maneuver in the operating envelope.
- **The finished budget sets the speed:** Blind travel, braking under the resident stop, transient tracking error, and position-estimate error all enter one admission check, so the clear distance, not the drive limit, decides how fast the machine may move.
- **The controlled barrier includes speed:** Raw distance has relative degree two for torque. A kinetic stopping barrier yields a torque half-space for positive approach speed, within the barrier's operating contract.
- **Fallback is state dependent:** Feasible projection, powered hold, controlled braking, and drive torque inhibit have different prerequisites. An empty admissible set cannot guarantee a safe stop.
- **Independent clocks detect different failures:** Evidence age, proposal age, host heartbeat, MCU cycle progress, and external trips are checked separately; no one timeout is a universal braking guarantee.
- **Refusal must leave a record:** The enforcement record binds every threshold to the detector that measures it and the rung a failed check selects, so the premises of each guarantee can be tested on silicon and judged at release.

The fallback ladder turns the law that proposal is not permission into a check the MCU executes on every tick, since each failed check selects a rung reserved before the fault, and the proposer can neither delay that selection nor override it. The kinetic barrier ties this law to irreversibility by

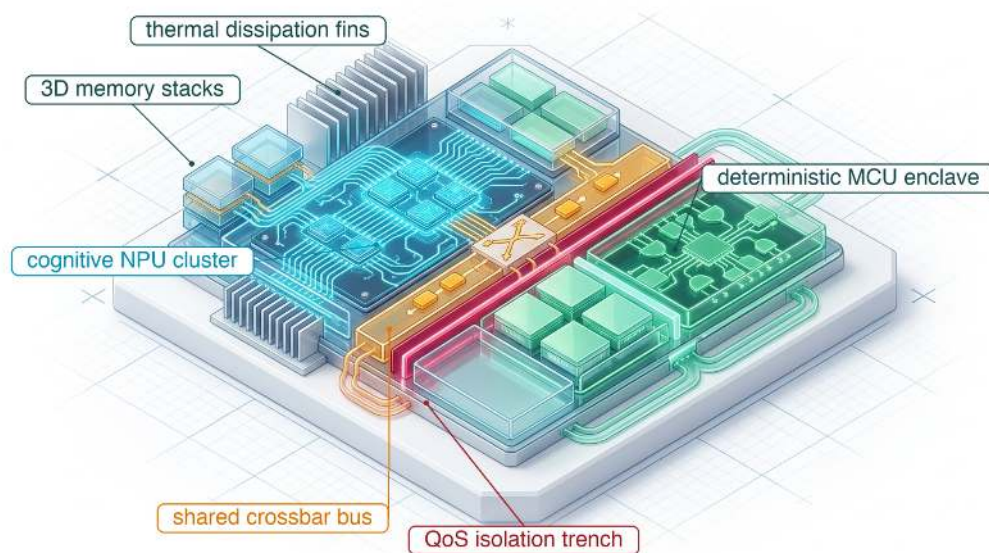
making permission depend on speed as well as position, because a state from which the resident stop no longer fits is lost before any boundary is touched.



What's Next: From barrier certificates to heterogeneous SoC placement

Can the learned proposers share a chassis and a memory bus with the permission path without delaying it? The enforcement record declares a worst-case cycle inside the permission deadline on every tick. That deadline is a claim about silicon, and a barrier result holds only while the permission path meets it. In chapter 13, we partition the learned proposers and the permission path across compute fabrics and name the loaded tests that would show whether memory starvation, clock throttling, or bus contention can delay permission or fallback, recording them in the placement record.

Silicon Placement



- 13.1 The Silicon Substrate
- 13.2 Two Paths on One Die
- 13.3 Contention for Shared Resources
- 13.4 The Boundary Contract
- 13.5 Thermal and Timing Coupling
- 13.6 Testing the Separation
- 13.7 Silicon Isolation Decisions
- 13.8 Hardware Allocation
- 13.9 Fallacies and Pitfalls
- 13.10 Summary

Purpose

Why can a learned model delay a safety check that runs on a different processor core?

Software architecture diagrams often depict proposal and permission paths as cleanly decoupled functional blocks. On physical silicon, however, disparate software components inevitably contend for shared hardware resources: memory buses, power distribution networks, clock trees, and thermal dissipation paths. A high-throughput burst of neural inference can saturate the shared memory interconnect or induce inductive voltage droop across power rails, delaying a safety check running on an adjacent processor core while the physical chassis continues to coast.

Software privilege rings govern address space permissions, not transaction scheduling or DRAM queue latencies. Consequently, logical isolation cannot guarantee worst-case execution time under peak contention. Only empirical stress measurement under adversarial workloads can verify timing deadlines. Enforcing real-time guarantees requires hardware-level isolation, dedicated memory channels, or physically separate microcontroller enclaves. In the physical AI stack, hardware placement establishes the computational foundation of the Nervous System, ensuring deterministic enforcement checks complete before commands cross the causal boundary.

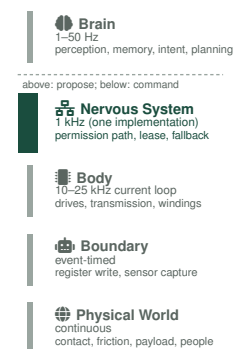


Figure 13.1.: The five-level machine stack with the Nervous System level highlighted.

💡 Learning Objectives

- Inventory the memory, interconnect, power, clock, reset, and thermal resources that proposal and permission paths share
- Explain why average bus utilization cannot bound a deadline, using transaction size, queue policy, and bank timing
- Size a windowed watchdog's service window from measured completion jitter for a non-blocking record exchange between the two paths
- Analyze how rail droop and thermal throttling stretch the permission path's execution time on different timescales
- Design a loaded stress test whose falsification threshold is the permission path's declared worst-case execution time
- Select among reservation, partitioning, workload reduction, and relocation from measured interference and the lease's timing allowance
- Construct a placement record that scopes loaded observed maxima, mitigations, and untested conditions for release review

13.1 The Silicon Substrate

THE warehouse mobile manipulator carries its application processor and its permission path in one chassis. On its software architecture diagram, the proposal boundary between the Brain and the Nervous System is a clean dashed line. The barrier filter and fallback ladder of chapter 12 keep their guarantees only while the permission path finishes its check inside its deadline on every tick, and that condition depends on where the check executes, which the diagram does not show. On physical silicon, the dashed line can collapse onto a single shared die. Suppose both paths ran on one heterogeneous System-on-Chip (SoC), the placement this machine rejects. The chunk policy and the intent model would execute on the embedded accelerator while the permission path executed on an adjacent real-time core. When the intent model begins an inference pass across its multi-billion-parameter weights, billions of transistors activate simultaneously. This current surge (dI/dt) induces an inductive and resistive voltage droop ($\Delta V = L \cdot dI/dt + IR$) across the chip's internal Power Distribution Network (PDN), risking clock instability and setup-time violations in adjacent real-time logic.¹ Concurrently, dense tensor matrix multiplications saturate the shared LPDDR memory bus, streaming gigabytes of activation maps and evicting the safety enforcer's pinned sensor tables from the last-level cache.

When the permission path reads a sensor buffer while the proposal workload occupies the shared memory controller, its requests queue behind direct memory access traffic, and the wait can consume the share of the deadline that the enforcer's computation needed. Logical privilege does nothing to prevent that wait, because it governs which addresses a task may touch, not where its requests wait in the memory queue.

Both of the machine's jobs would feel shared-die coupling. For the Class 1 base, a memory stall during feature extraction delays the permission path's refusal while the loaded base closes on a rack end, and every tick of delay beyond the one the stopping budget counts is distance it never reserved. For the Class 2 arm, an accelerator burst that evicts the permission path's tables from a shared cache delays the contact check while the gripper presses on the spring-latched cage door.

Following the end-to-end argument in system design (Saltzer et al. 1984), memory protection and operating system process isolation do not by themselves establish operational independence. On a heterogeneous die,² **operational independence** requires that proposal-side activity cannot push enforcement execution beyond its hard physical deadline, and that proposal-side crashes or thermal overruns cannot disable the permission path's authority to refuse.

Because these couplings are governed by physical solid-state phenomena rather than software abstractions, architectural claims of isolation must be treated as hypotheses to be falsified through measurement. Testing such a claim takes four engineering steps: an inventory of every shared silicon resource and its claimants; a stress campaign that drives the proposal path to worst-case compute, memory, and bus contention while recording the permission path's latency distribution

¹ | **Inductive Voltage Droop Mechanics:** In high-performance VLSI design, inductive voltage droop ($\Delta V = L \cdot dI/dt$) occurs across the parasitic inductance (L) of power distribution package pins and on-chip power grids when large numbers of logic gates switch simultaneously. If the local rail voltage drops below the minimum operating threshold ($V_{\min}$), digital flip-flops suffer setup-time violations, metastabilities, or brownout resets that disrupt deterministic real-time execution.

Prerequisite: Hardware execution cadences and micro-controller timing loops were defined in section 4.3.

² | **Heterogeneous System-on-Chip Architecture:** Heterogeneous SoCs integrate multiple distinct compute domains onto a single silicon substrate, combining high-throughput application processors and neural accelerators with deterministic real-time safety islands. Although memory management units isolate software address spaces across domains, hardware subcomponents continue to share power distribution networks, clock distribution trees, and DRAM controllers. Under peak cognitive burst workloads, uncoordinated resource contention across these shared physical structures can degrade real-time response times beyond critical physical deadlines.

and observed maximum; measurements of voltage droop and junction temperature under sustained load; and a placement record that states the observed maxima against the deadline together with the conditions left untested. Those steps recur through the rest of the chapter, which examines the shared die as the placement the machine rejects and the discrete controller as the one it uses. It begins with what each path must compute, where it runs, and how much slack it keeps when nothing competes with it.

The historical evolution of physical AI architectures is fundamentally rooted in the physical limits of semiconductor scaling (figure 13.2). Over fifty years of microprocessor progress (1975–2025), computing systems transitioned across two major architectural boundaries. During the classical scaling era (1975–2004), Moore’s Law and Dennard scaling allowed clock frequencies to surge exponentially from 2 MHz to 3.8 GHz without increasing thermal power density. Autonomous robotics rode this wave directly, transitioning from offboard minicomputer tethers (Shakey) to onboard VMEbus racks (CMU Navlab 1) and embedded x86 motherboards (Stanford Stanley).

When Dennard scaling collapsed in 2004, static leakage power capped single-core clock frequencies near 3.8 GHz, bringing the era of frequency scaling to a halt. Industrial robotics adapted by decoupling architectures: deterministic microcontrollers assumed responsibility for cycle-counted 1 kHz fieldbus motor commutation loops, while application processors handled multi-threaded trajectory generation. When high-capacity deep learning and vision-language-action (VLA) foundation models arrived, physical AI encountered a second, severe hardware constraint: the **mobile robot payload thermal and battery power ceiling** (15–60 W). Server-grade accelerators consuming 300–700 W cannot be thermally dissipated or battery-powered on an untethered mobile manipulator or humanoid. This constraint forced the deployment of Domain-Specific Accelerators (DSAs) and heterogeneous edge Systems-on-Chip (such as NVIDIA Jetson AGX Orin and Drive Thor), expanding onboard compute density by nine orders of magnitude ($10^9\times$) while staying strictly within mobile power limits. Most recently, sequential token decoding in autoregressive VLAs hit the **cognitive deliberation wall** (1–3 Hz), prompting the shift to flow-matching and diffusion policies with temporal action chunking that restore closed-loop control cadences to 30–50 Hz.

```
/tmp/ipykernel_600/979283785.py:180: UserWarning: This figure includes Axes that are not compatible with tight
plt.tight_layout()
```

(a)

Figure 13.2: Fifty years of microprocessor scaling and physical AI compute (1975–2025): Top: Transistor count (Moore’s Law), clock frequency, and peak server TDP against the mobile robot payload thermal budget (15–60, W), showing the 2004 Dennard scaling breakdown at 3.8, GHz. Bottom: Onboard compute density at ≤ 60 , W payload (GFLOPS and TOPS-equivalent, expanding $10^9\times$) versus closed-loop perception-to-action deliberation cadence (Hz), contrasting the 1, kHz deterministic fieldbus servo baseline, the 1–3, Hz autoregressive VLA deliberation wall, and 30–50, Hz flow-matching action chunking. (Milestones adapted from Karl Rupp and Stanford CPU database).

Bridging high-capacity neural deliberation to edge silicon requires fundamentally different compilation principles than general-purpose cloud computing. Cloud inference engines optimize for average throughput (tokens per second) through dynamic batching, runtime graph tracing, and heap memory allocation (`malloc`). In an embodied physical AI system, these practices are hazardous. Dynamic memory allocation introduces heap fragmentation and allocator lock contention, while just-in-time compilation and paging faults inject multi-millisecond tail jitter ($P_{99.9}$) that directly steals stopping clearance (section 12.5). Edge compilers for physical AI therefore enforce **ahead-of-time (AOT) static graph compilation**: the execution graph’s topology is permanently frozen, tensor dimensions are fixed, and every intermediate activation buffer is mapped to a static, pre-allocated physical memory offset before actuation begins, guaranteeing zero runtime dynamic allocations.

Beyond eliminating tail jitter, compilation on edge SoCs is governed by thermodynamics and the memory wall. As established in section 3.4, moving data across an off-chip dynamic RAM (DRAM) bus consumes roughly $10\text{--}20\times$ more energy per bit than performing arithmetic operations inside on-chip registers or static RAM (SRAM). On an untethered mobile robot operating within a 15–60 W payload thermal budget, passing intermediate activation tensors back and forth across the DDR bus rapidly exhausts the thermal dissipation capacity of the chassis. **Operator fusion** compiles multi-layer subgraphs (such as Convolution + Bias + Activation, or Attention Projection +

LayerNorm) into unified fused kernels that execute entirely within on-chip register files and local SRAM caches, eliminating off-chip memory traffic. In parallel, **numerical precision reduction** (quantizing weights and activations from FP32/FP16 down to INT8 or FP4) acts as a direct multiplier on memory bus capacity. Halving the operand bitwidth doubles the effective transfer throughput of the memory crossbar, directly reducing memory bus occupancy and preserving bus bandwidth for concurrent real-time safety monitoring.

13.2 Two Paths on One Die

The mobile manipulator partitions its control across two paths with conflicting requirements. The proposal path runs the chunk policy on the application processor at 20 Hz, taking the navigation and wrist camera streams and emitting base velocity and arm joint targets together. Its P99 inference, vision included, takes 40 ms (illustrative; see the Reader Guide) of each 50 ms chunk period; it holds no direct hardware authority over the drives and runs in unprivileged user space. The permission path runs on the discrete safety microcontroller (MCU) at 1 kHz. Each tick it checks the latest proposal against the stopping envelope of section 12.5 and stages the admitted setpoints for the EtherCAT frame to the drives. It must finish inside a chosen 400 μ s deadline within its 1 ms tick, and it runs in a privileged real-time execution domain on a dual-core lockstep controller,³ whose comparator diagnostics cover the permission path's own failure, because an envelope check cannot detect a fault in the core that executes it. Only the permission path holds memory-mapped input-output (MMIO) access to the fieldbus master that carries drive commands.

³ | **Dual-core lockstep:** Two cores can execute matching instruction streams while comparator logic detects selected divergences. Detection coverage, delay, shared-rail vulnerability, and diagnostic response depend on the device and the application safety case, so lockstep is one element of an ASIL D argument, not the whole of it.

Prerequisite: The sub-millisecond execution deadline of the CBF-QP enforcer was established in section 12.8.

⁴ | **PREEMPT_RT:** A Linux real-time configuration reduces some scheduler latencies, but its tail depends on kernel, drivers, workload, and shared hardware. No portable sub-50 μ s or bare-metal comparison follows without measurement.

⁵ | **Hardware memory protection and IOMMU domains:** Physical silicon structures that enforce spatial address isolation. While an on-chip Memory Protection Unit (MPU) confines CPU thread execution to bounded memory segments, an Input-Output Memory Management Unit (IOMMU) translates and polices direct memory access (DMA) requests from peripheral bus masters (such as PCIe neural accelerators or camera deserializers), physically preventing an untrusted device from corrupting safety state buffers.



Definition 13.1: Heterogeneous silicon partitioning

Heterogeneous silicon partitioning is the architectural allocation of stochastic, high-capacity cognitive workloads (perception, learned policy evaluation) and deterministic safety loops (safety invariant checks, admission and fallback) across specialized execution domains isolated in hardware.

1. **Significance:** Placing high-capacity neural deliberation and real-time safety enforcement on the same general-purpose operating system or compute engine creates common-mode failures. Kernel thread preemption, dynamic memory allocation, and GPU warp scheduling inject multi-millisecond tail jitter that causes the permission path to miss physical control deadlines.
2. **Distinction:** Unlike logical software process isolation (such as Linux containers or POSIX process priorities), silicon partitioning enforces isolation across independent hardware registers, tightly-coupled static memories (TCM), and dedicated peripheral buses.
3. **Common pitfall:** Assuming multi-core CPU affinity provides true failure-domain separation, when the cores still share caches, memory controllers, clock trees, and power rails.

The two processors run different kinds of software, following the two-processor implementation of section 4.2. Proposal inference, telemetry logging, network communication, and fault recording run on the application processor under Linux, whose scheduling tail stays exposed to kernel and driver jitter even in a real-time configuration.⁴ Enforcement and sensor validation run on the discrete safety microcontroller as bare-metal firmware behind a hardware memory protection unit,⁵ with the encoder interfaces, the IMU, and the timer channels wired directly to it. Each proposal reaches the microcontroller over a board-level link and is published there through the seqlock mailbox of section 4.3, so no dynamic random-access memory (DRAM) is shared between the paths. On the supply side the two paths share only the battery. The microcontroller, the drive logic, and the spring-brake coils sit on the isolated rail of section 2.8, which carries no proposal-side load and is held up by a local store for 2 s, longer than any stop the enforcer can command inside the envelope (section 12.9). A sag on the shared control rail therefore cannot reset the permission path.

On the unloaded machine the computational budgets appear generous. Illustrative sub-costs on the safety microcontroller put reading the encoders, the IMU, and the proposal's evidence epoch at 40 μ s, evaluating the constraints at 80 μ s, and staging the admitted setpoints for the drive frame at

15 μ s. The enforcer's unloaded cycle totals 135 μ s, which at the microcontroller's 400 MHz clock is 54,000 cycles. Against the 400 μ s deadline, the unloaded path leaves 265 μ s of slack, and the declared worst-case execution time of 250 μ s, the qualified budget of section 12.8, still leaves 150 μ s. On the application processor, the chunk policy's 40 ms inference leaves 10 ms of its 50 ms period. Evaluated in isolation, both loops operate with comfortable margins.

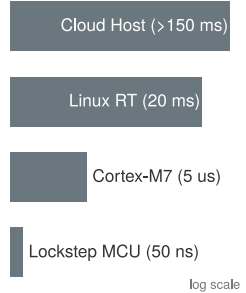
Assigning tasks to separate processor cores does not create independent failure domains if the silicon infrastructure is shared.⁶ section 4.6 traced these common-cause paths on the machine's board, where separate packages and the isolated rail cut the memory and supply paths. On a shared die, the rejected option for this machine, they move inside one package. The real-time core executes instructions independently from the application processor, yet both cores rely on common hardware services:

- *Shared interrupt routing*: The Generic Interrupt Controller (GIC) routes peripheral and inter-processor notifications across a shared fabric, where a flood of Ethernet packet interrupts on the application processor can delay the timer interrupt that triggers the 1 kHz permission loop.
- *Shared memory controllers*: Both paths access the same dynamic memory controller and interconnect crossbar, where large matrix tile loads generated by the neural accelerator saturate DRAM command queues, turning an expected 20 ns level-two cache refill on the real-time core into a 350 ns round-trip to main memory.⁷
- *Shared clock trees*: The cores share phase-locked loops (PLLs) for clock generation, where dynamic voltage and frequency scaling (DVFS) transients on the application processor introduce clock jitter across adjacent real-time domains.
- *Shared power rails*: A single Power Distribution Network (PDN) delivers current to both domains, where switching surges from the neural accelerator drop the internal voltage rail, risking logic state corruption or brownout resets.
- *Shared reset domains*: A kernel panic on the application processor that triggers a hardware watchdog reset can pull down the shared peripheral bus or system interconnect, disabling the drive outputs and leaving the base without a permitted command.

Shared hardware contention across these domains is not merely a passive scheduling inefficiency; it establishes an active **cyber-physical attack vector**. In a heterogeneous SoC where third-party neural workloads or network-exposed software containers share silicon with the permission path, a compromised application process can execute deliberate denial-of-service (DoS) attacks against physical safety. By generating pathological unaligned memory traffic or triggering cache-line thrashing (exploiting cache side-channel mechanisms or Rowhammer-style disturbances), an attacker on the unprivileged proposal path can deliberately saturate memory controller queues, delaying the safety microcontroller's memory access and forcing deadline misses. Similarly, an adversarial workload can trigger high-frequency tensor core switching patterns designed to maximize $L \cdot dI/dt$ inductive voltage droop, driving the shared power distribution network into brownout and inducing common-mode hardware resets. In physical AI architectures, software process isolation cannot defend against electrical and microarchitectural coupling; hardware-enforced quality-of-service partitions (such as ARM MPAM) and physical silicon separation are fundamental cyber-physical security controls.

Because software process isolation cannot mitigate these hardware-level timing and security couplings, physical silicon placement becomes the primary architectural defense. For the machine's permission loop, compare three placements. Shared application cores expose the permission loop to the kernel and driver scheduling tail described earlier, a tail that can push it past not only the 400 μ s deadline but the 1 ms tick itself. A real-time cluster on the application SoC keeps its own core but shares the die's memory controller, rails, and thermal path; section 13.5 shows the clock throttle that rejects it. The discrete safety microcontroller that the machine uses pays for its separation in transport instead, a board-level hop from the application processor that neither shared-die option needs and that section 13.7 charges to the lease. The choice among the three rests on each option's measured sensor-to-drive response under interference, not on how many packages it uses.

Principle 3 requires a permission path that learned workloads cannot delay, and the three placements differ precisely in which resources the two paths still share. Even the discrete microcontroller's



Candidate execution domains differ in their timing controls; measured tails determine placement.

⁶ | **Avionic Multicore Certification**: FAA Advisory Circular AC 20-193 and CAST-32A formalize multi-core interference analysis for safety-critical avionics. The guidance requires bounding worst-case execution times by demonstrating that shared caches, interconnect crossbars, and memory controllers cannot cause critical tasks to overrun execution deadlines. Without hardware-enforced quality-of-service partitions or dedicated interconnect channels, cross-core memory traffic invalidates a single-core timing bound.

⁷ | **Cache coloring and way partitioning**: Techniques that divide shared cache capacity among competing tasks. Software cache coloring partitions physical page allocations so that addresses from distinct processes map to non-overlapping cache sets, while hardware partitioning mechanisms (such as ARM Memory System Resource Partitioning and Monitoring, MPAM, or Intel Cache Allocation Technology, CAT) dynamically restrict the cache ways an untrusted neural accelerator can allocate, preventing line evictions in critical safety threads.

separation is a claim about timing under load, which only a measurement of its tick under that load can support. Every resource the two paths share has claimants on that tick, and the memory system is where they collide first.

13.3 Contention for Shared Resources

Every shared physical resource has multiple claimants; a utilization metric that ignores concurrent competition cannot bound a deadline. Had the mobile manipulator placed its permission path on the application SoC, the proposal path, the permission path, and the system infrastructure would compete directly for the same memory controllers, crossbar switches, and power distribution rails. Digital claimants on that shared memory bus would include camera and lidar DMA streams depositing frames into DRAM, the neural accelerator fetching model weight matrices and intermediate activation tensors, the real-time core reading its constraint tables and sensor history, actuator write buffers for drive commands, cache controllers executing dirty line writebacks, and diagnostic logging processes writing state records to nonvolatile storage. On the machine as built, the same bursts stay on the application processor, where they delay the observation and the chunk rather than the permission tick. The permission path meets them as evidence age (section 8.4), refusing evidence older than its refusal threshold and entering its fallback if no admitted chunk renews the lease.

On a shared die, **crossbar bus and DRAM bank contention** is queuing delay when memory and DMA transactions share an interconnect and controller, and its bound depends on transaction size, queue policy, and bank timing.⁸

Each claimant's average demand follows from its transfer size and event rate, and on a shared LPDDR channel the camera streams, weight fetches at the policy rate, the enforcer's reads, drive writes, writebacks, and logging together occupy a small fraction of its capacity on average. An engineer who inspects only that average concludes that most of the bus remains free. Memory arbiters do not schedule average rates. They arbitrate discrete bursts, and two workloads with identical average throughput interfere differently if one streams in small packets while the other arrives in long, saturating blocks.

Arithmetic intensity (Williams et al. 2009) says whether a routine is bound by execution units or by memory bandwidth in steady state; it says nothing about which request waits when two paths reach the bus together. While weights stream in from DRAM, the accelerator's arithmetic intensity is zero, and it acts as a high-bandwidth direct memory access engine that saturates the shared bus. Weight streaming is also what caps the rate of fresh proposals in the memory wall of section 3.4. Placement sees the other side of that limit, because every weight transfer that bounds the proposer is traffic a co-located permission path would wait behind.

How long a safety read waits depends on transaction arbitration and DRAM scheduling. An accelerator that moves weights in 262,144-byte tiles does not issue one burst: Arm's AXI4 specification permits at most 256 beats per burst and forbids a burst from crossing a 4,096-byte address boundary, so on an assumed 128-bit internal path a tile becomes 64 aligned transactions. An in-flight transaction may be non-preemptible, and whether the remaining tile transactions block a later safety read depends on whether the controller implements priority bypass. DRAM adds its own delay, because only one row per bank is open at a time: like a clerk who must shut one drawer before opening another, a read to a closed row waits for precharge ($t_{RP} = 14$ ns) and activation ($t_{RCD} = 14$ ns) before column service.

Controllers that use **First-Ready First-Come-First-Served (FR-FCFS)** scheduling let ready row hits overtake older requests, and a streaming accelerator supplies row hits in abundance, so that policy needs its own starvation bound. Under a FIFO queue that lets no later arrival pass the safety read, the idealized wait is

$$t_{\text{queue}} = \sum_{k \in Q} \frac{B_k}{\text{BW}_{\text{peak}}} + N_{\text{conflicts}}(t_{RP} + t_{RCD}) \quad (13.1)$$

where Q holds the transactions ahead of the read, B_k is each one's byte count, $N_{\text{conflicts}}$ counts bank switches, and BW_{peak} is an optimistic service rate, so arbitration overhead and refresh add to the result.

⁸ | **DRAM bank conflicts:**

A different row in the same active bank may require precharge and activation before column service. Actual bank mapping, overlap, and scheduling determine the added delay.

Napkin Math 13.1: The permission path's loaded read on a shared die

Problem: Suppose the permission path read its sensor snapshot from the application SoC's shared DRAM. How much queued traffic could precede that read before the tick exceeds its sense allowance, its declared worst-case execution time, and its deadline?

Variables: The tick allots 40 μs to sensing and 95 μs to constraint evaluation and the drive-frame write, inside a 400 μs deadline. The snapshot is 4,096 bytes on a 12.8 GB/s channel. Ahead of it the FIFO holds one weight tile (64 transactions), a camera transfer, and a telemetry record: 88 transactions, 360,448 bytes, 3 bank switches.

Math: By equation 13.1, serving the backlog's 360,448 bytes at 12.8 GB/s, plus 3 switches of 28 ns, takes 28.244 μs . The snapshot itself reads in 0.32 μs , so the loaded read takes 28.564 μs and leaves 11.436 μs of the sense allowance. A second backlog would overrun it. The declared worst-case execution time of 250 μs is crossed once the read exceeds 155 μs , which takes more than 5 backlogs, and that crossing already refutes independence even though the tick still meets its deadline. The deadline itself holds until the read exceeds 305 μs , which takes more than 10 backlogs, so a tick that meets its deadline is no evidence that the placement is sound. With validated priority bypass, only the one in-flight transaction waits ahead, and the read takes 0.64 μs .

Systems insight: The read's bound belongs to the memory controller, not to the workload. It exists only if the controller limits how much traffic it admits ahead of a safety read or lets that read bypass the queue, and an FR-FCFS controller serving a streaming accelerator provides neither unless its configuration is qualified to do so. The machine's discrete microcontroller removes the question by reading its snapshot from tightly coupled memory on its own bus.

Contention extends to the power distribution network, where claimants compete for charge rather than bandwidth. The droop of section 13.1, $\Delta V = L dI/dt + IR$, grows with both the rate and the size of the current step. An off-chip voltage regulator responds far too slowly to catch a nanosecond current step, so the first charge comes from on-die decoupling capacitance, and once that is depleted the shared rail sags under every core it feeds.

On a shared $V_{\text{core}} = 0.85 \text{ V}$ supply with package inductance $L = 30 \text{ pH}$ and resistance $R = 8 \text{ m}\Omega$, an accelerator current step of $\Delta I = 6 \text{ A}$ in 2 ns produces 90 mV of inductive and 48 mV of resistive drop, 138 mV in all, a 16.2 percent sag to 0.712 V. The alpha-power gate model⁹ turns that sag into a 39 percent increase in propagation delay (section C.5 derives both steps). That is a gate delay, not a tick. Whether it consumes the permission path's slack, trips a brownout reset, or does nothing depends on the device's minimum operating voltage and clock policy, and only rail and timing measurements under the declared current step can settle it. Both interference channels—interconnect FIFO queueing and power rail voltage droop—are structured in figure 13.3. A shared die leaves those questions open for every accelerator burst; the discrete microcontroller, on its own bus and rail, does not inherit them.

Detecting these interactions requires instrumentation outside the affected core, because software on a stalled or slowed core cannot time its own delay. Memory-controller counters, rail monitors, thermal diodes, and reset-cause registers record the coupling.

A shared die can meter these couplings in hardware, with a crossbar arbiter that routes tagged safety transactions through their own queue while metering accelerator bursts with bandwidth credits, and a power-management controller that throttles only the accelerator's clock at a thermal trip. Each is a property of the controller that the placement test must confirm, not a scheduler setting the block diagram can assume. Arbitration decides when a transaction moves, not what it carries. What crosses the physical boundary between the two paths must be defined by explicit guarantees on production time, validity, and failure response.

13.4 The Boundary Contract

What crosses between the proposal and permission paths under a boundary contract is a record, not an unqualified pointer into shared memory, because passing a raw memory address across the trust boundary couples the reader directly to the writer's internal memory layout, execution timing,

⁹ | **Alpha-Power CMOS Delay Model:** The model uses $t_{\text{prop}} \propto V/(V - V_{\text{th}})^\alpha$ with $V_{\text{th}} = 0.42 \text{ V}$ and $\alpha = 1.3$. It relates supply voltage to the switching delay of one gate; setup failures and brownout behavior come from device characterization.

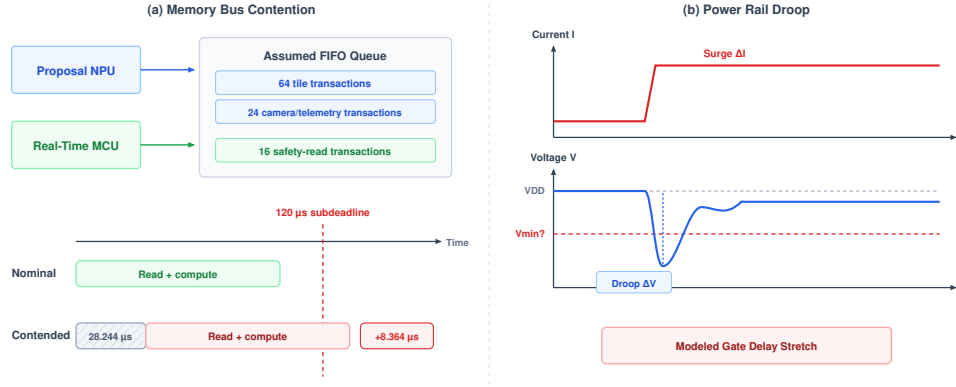


Figure 13.3: Heterogeneous SoC interference channels: (a) In the memory-bus FIFO queue, queued background transactions delay a safety read and push read plus compute past its subdeadline; priority bypass changes the wait. (b) On the power distribution network, a sudden matrix-accelerator current step ($\Delta I = 6.0$ A in 2.0 ns) produces 138 mV of inductive and resistive droop, stretching gate propagation delay non-linearly under the alpha-power law. Crossing the minimum operating threshold $V_{\min}$ causes timing violations or brownout.

and failure domain. If the chunk policy running on the accelerator provides only a pointer to its private output buffer, the permission path has no mechanism to determine whether the writer is still modifying the buffer, has stalled mid-inference, or has encountered an unhandled fault. The boundary payload must therefore be an immutable, self-contained record. The record that crosses is the proposal header of section 4.3, published through its seqlock mailbox; placement adds no fields, only the evidence that the exchange meets its deadline under load. The permission path reads the record as an isolated snapshot, evaluating its temporal freshness and physical validity before any setpoint reaches a drive.

Preserving temporal isolation, whether the exchange crosses a shared die or the machine's board-level link, requires eliminating blocking synchronization at the exchange interface. If the permission loop acquires a lock held by the proposal generator, an inference stall can delay a physical decision. A 25 ms host stall must therefore leave the 1 kHz enforcer running each tick: it may use the last validated chunk only while its lease and current state remain admissible, then transfer to its independently validated fallback. Allocation and logging on the permission path likewise need measured time bounds. The producer may stall or drop updates; it cannot backpressure the plant's enforcement cadence.

The mailbox rejection rules of table 4.1 keep a failed producer from forcing the enforcer to wait. A rejected proposal leaves the admitted setpoints in force until the lease lapses; if no admissible proposal renews it and the machine is moving, the MCU executes the stop rung of section 12.9, the base's resident stop and the arm's state-matched stop, neither of which waits for the proposer. That fallback must still complete within a bounded time, and the placement test measures that bound under load on the silicon that hosts the path.

13.4.1 Windowed watchdog timers and execution liveness

Software memory boundaries and lock-free seqlocks protect against torn reads, but cannot guarantee that the safety enforcer code is alive and progressing through its intended operations. Liveness of the enforcer belongs to the self-watchdog of section 4.3, which is fed only after a completed tick. A countdown watchdog serviced from a timer interrupt stays blind to a dead task, the failure argued in *Bookout v. Toyota*. On shared silicon, the enforcer can also be starved by interconnect contention or run too fast after clock corruption, so placement needs a stronger form of that self-watchdog, one that constrains the feed from both sides.

High-integrity physical AI architectures therefore enforce **windowed watchdog timers**. A windowed watchdog rejects any refresh that occurs either too early or too late. The timer hardware defines a service window parameterized by a lower bound (the closed window, T_{lower}) and an upper bound (the open window, T_{upper}):

1. **Early-kick violation** ($t_{\text{service}} < T_{\text{lower}}$): If software attempts to refresh the watchdog before the lower threshold has elapsed, the hardware immediately triggers a fault reset. An early kick can indicate that the software has bypassed required algorithmic stages, branched into a pathological short loop, or suffered execution speedup from clock corruption.
2. **Late-kick violation** ($t_{\text{service}} > T_{\text{upper}}$): If the timer counts down past T_{upper} without receiving a valid refresh, the counter underflows, asserting a hardware reset. A late kick can indicate that the enforcer was starved of compute cycles by interconnect contention, deadlocked on an unhandled exception, or delayed by excessive cache evictions.
3. **Valid service window** ($T_{\text{lower}} \leq t_{\text{service}} \leq T_{\text{upper}}$): A refresh inside the window shows the feed path met its timing condition; task progress still needs a suitable challenge or independent supervision.

If the controller feeds once after each successful tick, the watchdog measures the interval *between feeds*. Let completion offset c_n from tick n start lie in $[c_{\min}, c_{\max}]$, and bound tick-phase variation by J_{phase} . A conservative range for the inter-feed interval is

$$T_{\text{feed}} \in [T_{\text{period}} - (c_{\max} - c_{\min}) - J_{\text{phase}}, T_{\text{period}} + (c_{\max} - c_{\min}) + J_{\text{phase}}].$$

Configure T_{lower} and T_{upper} around that *measured or justified* range, while keeping the late-trip response within the plant's separately derived fault budget. A one-tick execution time alone does not set a watchdog service interval. On the machine, T_{period} is the 1 ms tick and the declared 250 μs worst case supplies $c_{\max}$. The unloaded 135 μs path does not supply $c_{\min}$, because the enforcer's shortest branch can complete sooner than its nominal one, so the lower bound comes from the shortest completion the stress campaign records. J_{phase} is the jitter of the microcontroller's tick timer against its reference, measured under the same load. That range would give the machine's timeout-only self-watchdog the T_{lower} it lacks, while T_{upper} stays at the chosen 5 ms timeout, so the window tolerates several missed ticks before it trips.

The tolerance is deliberate. One missed tick costs millimeters of travel at aisle speed (section 13.6), well inside the fault budget, and a window that closed on a single late feed would halt a healthy enforcer to save that displacement. A lower bound taken from estimates rather than recorded completions trips on healthy ticks, and an upper bound wider than the fault budget hides the starvation the window exists to catch.

Some designs add **challenge-response checkpointing**: the feed depends on fresh sensor validation, permission evaluation, and output-state checks rather than a timer ISR alone. Its fault coverage depends on where the challenge is generated, who can compute or replay the response, and whether failed enforcer states can still execute the feed path. The scheme's coverage is therefore a fault-injection result, not a property of the token.

However, establishing clean boundaries in memory, interconnect arbitration, and watchdog execution liveness solves only the logical and timing half of co-location. When the accelerator and the real-time processor share the same silicon die, the high power dissipated during dense matrix operations conducts directly through the substrate, elevating local junction temperatures and altering transistor switching speeds across adjacent cores.

13.5 Thermal and Timing Coupling

Every watt dissipated on a silicon die flows into a single thermal domain. On a shared SoC the matrix units, the application cores, the memory interface, and the integrated regulators all heat one package, and while the permission loop draws a small, steady baseline, the proposal path swings with its workload. The package's finite conductance to ambient lets peak accelerator activity raise the junction temperature of every circuit on the die.

In physical AI systems engineering, **thermal clock throttling** is the automatic, hardware-enforced reduction of processor clock frequency triggered when sustained high-power cognitive inference elevates silicon junction temperature (T_j) past thermal trip thresholds (T_{trip}). To prevent damage to the die, the hardware abruptly cuts clock speed. This introduces a lingering, temperature-dependent slowdown (hysteresis) that inflates execution times even after the cognitive workload subsides and can push co-located real-time safety loops past their hard physical deadlines.



Figure 13.4: Enclosed edge-compute kit: The photograph shows a Jetson AGX Orin Developer Kit box and a closed, vented enclosure. The SoC, memory, cooling hardware, and clock domains are not visible; the thermal model in the text is a separate illustrative scenario. Photograph: Auledas, CC BY 4.0, via Wikimedia Commons; local copy re-encoded.

The electronics bay in the machine's base sits at 45°C ambient. Figure 13.4 shows the developer kit of the module class the machine uses as its application processor; its enclosure reveals neither the heat path nor the clock domains on which the following estimate depends. Suppose the shared SoC were conduction-cooled with a **junction-to-ambient thermal resistance** of $\theta_{JA} = 2 \text{ K/W}$ and a **thermal time constant** of $\tau_{th} = 8 \text{ s}$. The permission loop with idle and memory power draws 4 W and holds the junction at 53°C. Continuous proposal inference adds accelerator, memory-traffic, and regulator losses, and a total of 26 W raises the steady state by $P\theta_{JA} = 52 \text{ K}$ to $T_j = 97^\circ\text{C}$.

Thermal management alone rejects the shared-die placement for the mobile manipulator. Suppose its permission path ran on a real-time cluster of the application SoC at $f_{nom} = 1.2 \text{ GHz}$, costing $N_{cycles} = 300,000$ cycles per tick, several times the 54,000 cycles the discrete microcontroller needs unloaded, because a cluster without tightly coupled memory pays cache-refill and coherence cycles on every snapshot and table read. When the junction crosses $T_{trip} = 95^\circ\text{C}$, the governor drops the cluster to $f_{throt} = 600 \text{ MHz}$ and the enforcer's cycle stretches to $t_{exec,throt} = 500 \mu\text{s}$, missing the 400 μs deadline by 100 μs (a 25 percent overrun) and far exceeding the declared bound on which every margin above the enforcer was sized. The discrete microcontroller runs on its own clock and rail, beyond the reach of that governor.

The overrun outlasts the load because the governor applies **state-dependent hysteresis**, restoring 1.2 GHz only after the junction cools to $T_{clear} = 85^\circ\text{C}$ so that the clock does not oscillate around the trip line. With the policy halted and power back at 4 W, the junction relaxes toward its 53°C baseline:

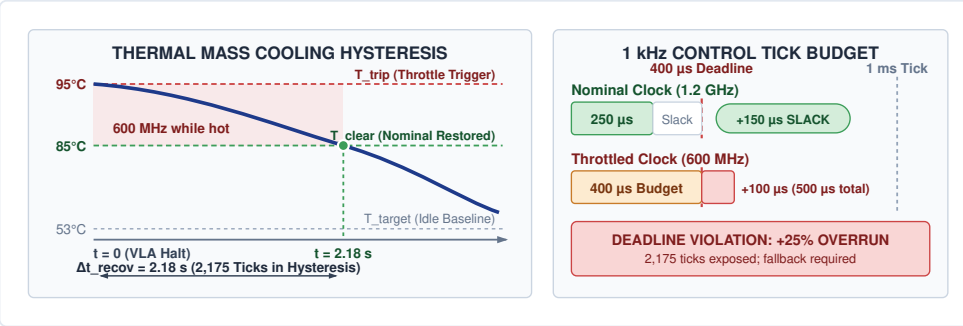
$$T_j(t) = T_{j,target} + (T_{j,start} - T_{j,target})e^{-t/\tau_{th}} \quad (13.2)$$

where $T_{j,start}$ is the junction temperature when throttling begins, $T_{j,target}$ is the baseline equilibrium, and τ_{th} is the package thermal time constant. Solving equation 13.2 for the fall from 95 to 85°C gives $t_{recov} = \tau_{th} \ln[(T_{j,start} - T_{j,target})/(T_{clear} - T_{j,target})] = 2.18 \text{ s}$. At the machine's 1 ms tick, 2,175 scheduled ticks fall inside that cooldown, and a shared-die design would need a separately timed fallback active for every one of them.

A stretched cycle on a shared die can also come from **electrical power-rail coupling**, on a different timescale. The rail droop of section 13.3 arrives within nanoseconds of an accelerator burst, while

thermal coupling builds over seconds as heat diffuses through the package, so a timing fault that tracks junction temperature $T_j(t)$ rather than transients in $V_{\text{core}}(t)$ is throttling, not rail collapse.

Checkpoint 13.1: Junction temperature and clock throttling



Before relying on software thermal management for real-time safety guarantees, verify your understanding of junction temperature dynamics and clock throttling hysteresis:

- ☐ **Thermal mass latency memory:** Can you explain why physical thermal mass acts as a long-duration memory on real-time task execution latency even after the offending cognitive workload has been terminated?
- ☐ **Cooling recovery sizing:** Can you derive the clearance time t_{recov} required for an SoC with thermal time constant $\tau_{\text{th}} = 8$ s to cool from $T_{\text{trip}} = 95^\circ\text{C}$ to $T_{\text{clear}} = 85^\circ\text{C}$ under a 53°C baseline equilibrium?
- ☐ **Governor hysteresis purpose:** Can you explain the purpose of temperature hysteresis (T_{trip} vs. T_{clear}) in hardware thermal governors and describe how it prevents high-frequency clock oscillation?
- ☐ **Physical segregation decision:** What measured clock, power, and timing evidence would justify a shared die, and when would a separate controller be needed?

13.6 Testing the Separation

An independence claim, on any placement, is a testable bound on enforcement latency and refusal availability across declared proposal workloads, chip temperatures, and modes. If contention delays a refusal past the physical deadline, the claimed operational separation fails.

The pass threshold for this experiment comes from the claim under test. The machine's permission path declares a worst-case execution time $C_{\text{wcet}} = 250 \mu\text{s}$ and must finish each tick inside its chosen deadline $D_{\text{enf}} = 400 \mu\text{s}$, which sits inside the tick $T_{\text{tick}} = 1$ ms. Every measured enforcement time t_{enf} must respect that chain, as equation 13.3 states:

$$t_{\text{enf}} \leq C_{\text{wcet}} < D_{\text{enf}} < T_{\text{tick}} \quad (13.3)$$

The declared bound C_{wcet} is the **falsification threshold**. If any enforcement cycle under test exceeds it, the independence hypothesis is refuted, even when the cycle still meets the deadline, because every margin above the enforcer, from its deadline to the tick that the stopping budget counts, was sized on the declared number. A measured maximum can refute the declared bound but cannot establish it, because a campaign samples only the states it reaches, and on speculative cores timing anomalies can place the true worst case above every cycle recorded.¹⁰

Downstream: Adversarial microarchitectural contention tests inform the hardware fault injection suite in section 15.6.

¹⁰ | **Microarchitectural Timing Anomalies:** In out-of-order and speculative processing architectures, a local performance improvement, such as a cache hit or accelerated branch evaluation, can alter instruction scheduling and resource contention, resulting in a net increase in global execution time. This counter-intuitive behavior invalidates simple compositional worst-case execution time analysis. Safety verification needs a justified bound for the declared workload, arbitration, temperature, and fault conditions; measurements alone and a non-speculative core alone do not establish a hard end-to-end bound.


Definition 13.2: Worst-case execution time

Worst-case execution time is the formal upper bound on the physical wall-clock duration required for a computational task to execute to completion on a target hardware platform under all admissible inputs, architectural contention states, and operating conditions:

$$t_{\text{exec}} \leq t_{\text{WCET}} \leq t_{\text{deadline}}$$

1. **Significance:** In cyber-physical control loops, correctness depends not only on the numerical validity of the output but also on the instant of its delivery; an optimal control command delivered after the hard physical deadline is functionally equivalent to a total system failure.
2. **Distinction:** Unlike average execution time ($\bar{t}$) or high percentiles (P_{99}), which characterize throughput in soft real-time software, WCET is an absolute deterministic bound that must hold under concurrent memory-bus saturation, cache thrashing, and peripheral interrupt storms.
3. **Common pitfall:** Relying on empirical execution profiling or benchmark averages to establish WCET on modern superscalar or out-of-order processors. Microarchitectural timing anomalies (where a local cache hit or branch prediction success paradoxically increases total execution time) mean empirical testing alone cannot prove the true global upper bound without formal static analysis and hardware contention bounds.

The distance at stake in one tick is small, which makes the bound easy to undervalue. The finished budget in table 12.1 stops the machine in 997.4 mm at the 1.3 m/s aisle speed, 102.6 mm inside the 1.10 m clear distance, and one missed tick adds only $vT_{\text{tick}} = 1.3$ mm. What placement protects is the bound itself, because τ_{delay} counts exactly one T_{tick} , and contention without a bound is a stall of unknown length.

Refuting an independence claim requires synthetic workloads engineered to induce worst-case resource contention rather than standard benchmarks that report average throughput. High average utilization schedules memory accesses evenly across clock cycles, making the system look safely underloaded, whereas actual physical interference peaks during synchronized bursts when multiple components demand the same wires at the exact same instant. The test framework must deliberately flood shared memory channels with unaligned direct memory access transfers, thrash shared cache capacity with dirty line writebacks from transformer weight tensors, trigger accelerator matrix multiplications that induce rapid supply rail voltage droops ($L \cdot di/dt$), and stream peak-rate sensor telemetry while diagnostic logging writes to non-volatile flash storage. These stress profiles reproduce the transient contention that occurs when a proposal model initiates a large attention pass at the exact microsecond the permission loop reads its sensor snapshot.

Instrumentation records enforcement entry after the sensor snapshot is coherent (t_0), permission completion (t_1), and publication of the admitted setpoints to the drive frame (t_2). These diagnose compute and dispatch, but t_2 is not braking. The frame's arrival at the drive (t_3) and measured brake onset (t_4) complete the physical chain. The endpoint that matters is t_4 measured from the last valid lease renewal, with clock error included. Brake onset must follow within 82 ms, the sum of the lease, one tick, one bus cycle, and the brake-onset delay in section 4.3. The 250 μ s computation bound alone is insufficient.

Coupling mechanisms compound across physical, electrical, and logical domains, requiring a full factorial experimental matrix. The evaluation sweeps across cold junction conditions ($T_j = 35^\circ\text{C}$) and a warm junction held at or above the throttle trip T_{trip} of section 13.5, an idle proposal path versus a saturated proposal engine executing maximum-batch inference, diagnostic circular logging disabled versus streaming at full non-volatile memory bandwidth, and nominal tracking commands versus boundary-violating commands that trigger refusal routines. Testing refusal paths under full stress is mandatory because executing a safe clamping trajectory or entering emergency shutdown involves cold branch paths, non-cached error-handling code, and secondary bus transactions that never execute during steady-state aisle driving.

On the rejected shared-die placement, the pair the matrix exists to catch is specific. A person steps out at a rack end, and the permission path takes its refusal branch toward the stop rung, a cold path through uncached code. At the same instant the application processor, having flagged the same anomaly, starts a diagnostic memory dump over the shared direct memory access fabric. A test suite that exercises refusal and diagnostic logging separately passes both and never measures the pair, because the two subsystems contend for one memory controller only when they run together.

Consider an illustrative 10^7 -tick stress trace for the rejected placement, the permission path on a real-time cluster of the application SoC. At one tick per 1 ms, the trace spans 10,000 s, or 2 h 46 min 40 s, before reset or setup time. The cold trace has $P_{50} = 95 \mu\text{s}$, $P_{99} = 128 \mu\text{s}$, and observed maximum 152 μs ; the warm, loaded trace has $P_{50} = 210 \mu\text{s}$, $P_{99} = 330 \mu\text{s}$, $P_{99.9} = 395 \mu\text{s}$, and observed maximum 468 μs . The loaded P_{99} already exceeds the 250 μs declared bound, and the maximum crosses the 400 μs deadline, so the trace refutes that placement.

Had the trace passed, its ticks would show only that observed interference stayed below the threshold in the tested states. A report gives the latency distribution, observed maximum, sample count, duration, and unexercised conditions, including supply degradation, electromagnetic interference, and aging.¹¹

⚡ War Story 13.1: Apollo 11 radar load and computer restarts (1969)

Context: On July 20, 1969, the Apollo 11 Lunar Module *Eagle* was executing its powered descent toward the lunar surface. The Apollo Guidance Computer (AGC), a 1.024 MHz machine with core-rope program memory, executed guidance, navigation, and descent engine throttle loops under an asynchronous, priority-scheduled Executive developed at the MIT Instrumentation Laboratory (NASA Manned Spacecraft Center 1969; Mindell 2008).

Mechanism: With the rendezvous-radar mode switch out of its LGC position, the radar resolvers were excited by an 800 Hz supply that was locked in frequency but not in phase with the guidance system's reference. The coupling data units read the phase difference as antenna motion and sent a continuous stream of counter pulses to the computer. NASA's mission report, section 16 gives a maximum of 12,800 pulses per second from the two data units, each taking one 11.7 μs memory cycle, or 15 percent of computer time, while the computer normally ran at about 90 percent of capacity during powered descent. Repetitive jobs that could not finish were scheduled again, each reserving another memory area, until the Executive ran out of them.

Impact: Five Executive overflow alarms (1201 and 1202) occurred before the low-gate phase of the descent. The mission report states that the performance of guidance and control functions was not affected.

Response: Each alarm initiated a software restart, and the AGC resumed its essential guidance and display work according to priority while the crew and mission control assessed the alarms and descent continued. Later missions zeroed the data units whenever the switch was not in LGC, removing the spurious pulses at their source.

Systems lesson: A peripheral can consume execution capacity at the same moment guidance needs it. Modern placement tests should account for asynchronous peripheral load and verify that the critical path and its recovery behavior remain within their declared timing bounds.

¹¹ | **Silicon Substrate Aging and Timing Margins:** Long-term substrate degradation mechanisms, such as Negative Bias Temperature Instability (NBTI) and Hot Carrier Injection (HCI), gradually increase transistor switching delays over years of operation. When coupled with transient supply-voltage droop under sudden multi-core current steps ($\frac{dI}{dt}$), aged silicon experiences propagation delays that exceed design margins. In hard real-time systems, these physical timing degradations cause intermittent race conditions and missed scheduler deadlines that never manifest during clean bench qualification.

Apollo 11's Executive alarms and the Mars Pathfinder priority inversion (examined in section 4.4) illustrate two distinct forms of real-time resource contention. In Apollo 11 a peripheral consumed execution capacity; in Pathfinder a low-priority task held a resource that a high-priority task needed.

A failed interference test identifies where the assumed separation may collapse: cache, memory arbitration, thermal clocks, power, or output queues. Further instrumentation localizes the mechanism before the team reserves, partitions, or relocates the shared resource.

13.7 Silicon Isolation Decisions

A refuted independence claim leaves four remedies, each priced in a different currency (table 13.1): static reservation, spatial partitioning, workload reduction, and physical relocation to separate silicon.

Table 13.1: Workload re-placement and isolation decision matrix: Remediation pathways for when a test falsifies independence on shared silicon. Each remedy trades distinct engineering currencies (throughput, capacity, latency, board footprint) to eliminate contention channels before escalating to physically segregated silicon.

Remediation Strategy	Target Coupling Layer	Implementation Mechanism	Systems Engineering Trade-Off	Residual Vulnerability	Physical Silicon Migration Trigger
Static Reservation	CPU execution time, OS scheduling preemption	Fixed-priority preemptive scheduling (SCHED_FIFO), static time-slot slicing (ARINC 653)	Idle capacity (20–40% throughput loss); unallocated cycles wasted during quiescent states	Micro-architectural contention beneath OS visibility (shared L3, DRAM bank conflicts)	Memory bus or cache stalls violate real-time deadlines despite 100% CPU core allocation
Spatial Partitioning	Cache capacity thrashing, DRAM bus contention	Cache way-partitioning (ARM MPAM, Intel CAT), MPU/IOMMU domains, AXI QoS rate limiters	Structural fragmentation; reduced L3 working set for perception (+15–30 ns DRAM access penalty)	Unpartitionable DRAM controller queues, row-buffer locality reordering, PDN voltage droop	Matrix engine current steps ($\Delta I > 10$ A) trigger rail droop, or DRAM queue stalls exceed slack
Workload Reduction	Memory throughput saturation, peak thermal power	Model quantization (INT8/FP8), context downsampling, halved policy rate	Degraded policy tracking fidelity; increased control phase lag near dynamic stability boundaries	Common clock distribution trees, shared Power Management Units (PMU), unified reset domains	Model accuracy drops below operational limits, or shared clock/reset faults propagate across domains
Physical Relocation	Shared power, clock, reset, or unbounded controller interference	Separate enforcer MCU with independently assessed power, clock, memory, and sensor/actuator path	More board area and an external communication budget	Harness, transceiver, and common-supply faults remain	Choose when the shared die cannot provide evidence for required timing and fault containment

Static reservation preserves the physical placement of workloads on the shared chip while allocating a guaranteed fraction of a shared resource to the critical path. Reservation assigns dedicated execution slots or fixed memory bus bandwidth to the critical loop. The cost of reservation is idle capacity. Had the machine's permission path shared the die, an arbiter slot sized to its snapshot read's worst case would stay reserved in every 1 ms tick, unavailable to the policy model even while the base is parked and no stop is under way.

Spatial partitioning divides physical hardware structures so that workloads no longer compete for the same physical cells.¹² Cache way-partitioning, core pinning, and dedicated memory channels establish hardware barriers within a single die. The cost of partitioning is lost flexibility and structural fragmentation. Splitting an eight-megabyte shared cache into a six-megabyte partition for the policy model and a two-megabyte partition for the permission path's tables prevents cache line evictions across the boundary. However, it permanently restricts the working set of the learned model, forcing higher DRAM access rates and increasing mean memory latency by tens of nanoseconds on every inference step.

Workload reduction resolves contention by shrinking the computational or memory footprint of the non-critical workload until its peak demand falls within the uncontended margins of the shared system. An engineer might downsample telemetry logging from 1 kHz to 100 Hz, quantize policy weights from sixteen-bit floating point to eight-bit integers to cut memory bus traffic in half, or lower the chunk policy's rate. The cost of reduction is degraded functional authority. On the mobile manipulator the lease bounds the last lever. A chunk period longer than the 60 ms lease of section 4.3 lets proposals expire between renewals, so the policy's rate must stay above 16.7 Hz even before renewal jitter is counted, which leaves little room below its 20 Hz. Weight quantization can degrade the policy's accuracy near the grasp tolerance of the conveyor pick.

If reservations, partitioning, and workload reduction cannot establish the required response bound, physical relocation is a candidate remedy. It reaches couplings that reservation and partitioning

¹² | **Automotive Freedom From Interference:** ISO 26262 Part 9 formalizes Freedom From Interference (FFI) as the absence of cascading faults between software elements with different safety integrity levels. In heterogeneous robotics SoCs, achieving FFI requires hardware-enforced spatial partitioning (memory protection units, IOMMU page tables) and temporal partitioning (fixed-priority bus arbitration, TDMA interconnects) to prevent unprivileged perception models from degrading safety-critical enforcement tasks.

cannot, such as a shared reset or clock domain that lets one fault halt both paths; a partitioned SoC that claims to contain such a fault has to show it by design and injection tests. A separate controller removes these common-cause paths only as far as its power, clock, reset, sensor, and actuator independence are designed and tested, and the integrity target it serves rests on the application's safety case, not on a count of packages. An external controller also adds transport and common-supply obligations, so its sensor and actuator path must be measured as a whole.

Relocating a workload to separate silicon is rarely a localized modification. It triggers a cascade of placement and communication adjustments across the entire machine, and the mobile manipulator shows where that cost lands. The T_{bus} term of section 4.3 does not change, because drive commands cross the EtherCAT fieldbus wherever the permission path runs. What relocation adds is the proposal's crossing from the application processor to the microcontroller over a board-level link instead of an on-die mailbox. That delay enters no new term of τ_{delay} . A proposal that arrives late spends part of the 5 ms renewal-jitter allowance that the lease rule of section 4.8.1 places in the gap between the 50 ms chunk period and the 60 ms lease. Host memory contention draws on the same allowance, so the most it may stretch the chunk policy's 40 ms inference is 12.5 percent (section 8.4). The link's worst case, measured under load, must therefore fit inside what that contention and the proposer's own jitter leave of the allowance.

The harder decision is where each sensor attaches. The machine keeps the encoders and the IMU on the microcontroller's side of the link, because those are what the permission path reads every tick, and it leaves the cameras and the lidar on the application processor, because the learned models consume them and the permission path reads only the proposal's evidence epoch. Moving a camera interface to the microcontroller would offload the main chip, but it would force the learned models to receive their observation stream over the slower external link and add to the observation age that chapter 8 measured. Every re-placement decision propagates through the physical interconnects, shifting bottlenecks from on-die memory arbiters to board-level pin counts, wiring runs, and serial buses.

Hardware selection in physical AI cannot be decoupled from the mechanical dynamics of the embodiment. While conventional benchmark suites evaluate edge accelerators on isolated metrics such as theoretical peak TOPS or raw batch inference throughput, embodied systems operate under closed-loop energy constraints where compute latency and physical motion are tightly coupled (figure ??). Evaluating edge accelerator configurations across a continuum—from ultra-low-power edge TPUs to multi-hundred-watt workstation GPUs—reveals an inverted-U Pareto efficiency frontier.

Under-provisioned accelerators (such as a 4TOPS edge TPU or an unaccelerated CPU) incur long vision-language-action inference latencies (> 200 ms per step), stretching a pick-and-place task to over three minutes ($T_{\text{mission}} = 215$ s). During this entire duration, the robot's base electronics, sensors, motor holding currents, and thermal management systems continuously draw static standby power (P_{static}). Consequently, the static energy penalty ($P_{\text{static}} \times T_{\text{mission}} = 23.65$ kJ) dominates total mission consumption, yielding poor net system efficiency (40.8 tasks/MJ). Conversely, over-provisioning edge hardware with desktop-grade GPUs (300 W TDP) reduces model execution time to 55 ms, but the system hits the robot's kinematic velocity ceiling ($v_{\text{max}} = 0.8$ m/s). The marginal time savings (27.5 s $\rightarrow$ 24.0 s) fail to compensate for the massive dynamic chip dissipation ($P_{\text{chip}} \times T_{\text{mission}}$), causing total energy per task to surge to 9.84 kJ. The optimal co-design window emerges at the balance point (exemplified by the 50 W Jetson AGX Orin), where model inference is sufficiently amortized to prevent static chassis drain without inducing unnecessary dynamic power penalties.

13.8 Hardware Allocation

The mobile manipulator now has a placement for its permission path, a deadline, an unloaded execution estimate, and a test that could refute the separation. None of it reaches the release decision unless it is written down with the conditions under which it holds. The **placement record** consumes the enforcement record of section 12.11, from which it takes the permission path's declared worst-case execution time and deadline, and adds the evidence that the declared bound holds under load on the silicon where the path runs: execution domain, privilege, period, unloaded and loaded observed maxima, arbitration settings, droop and thermal trips, mitigations, and a digest of the test ledger. The loaded maxima decide the independence claim, which stands only while every one of them stays at or below the declared worst-case execution time under the declared load; as section 4.4

argued for any observed latency distribution, a sample maximum is evidence about that bound and never replaces it.

The record has two parts. A compact header holds the configuration values and the observed maxima, and it carries the digest of an external test ledger. The ledger holds what does not fit a fixed layout: inventories, the latency distributions and sample counts behind each maximum, test conditions, mitigations, and open limits.

For the mobile manipulator, the header records the 1 ms tick, the 400 μ s deadline, and an illustrative 135 μ s unloaded estimate that is not yet a measured maximum. The header's mitigation flags include the permission path's isolated rail. Its loaded field stays empty, because no loaded test ledger exists for this machine yet. Filling it takes the campaign of section 13.6 run on the machine itself, with the discrete microcontroller ticking while the application processor runs peak proposal inference with full logging, at warm junction, with the refusal branch exercised in the same trials. Every loaded maximum that campaign records must stay at or below the declared 250 μ s, and brake onset, timed from the last valid renewal, must follow within 82 ms. Until those measurements exist, the permission-independence claim that chapter 16 weighs rests on a declared bound with nothing behind it, and a verdict that needs the claim cannot be granted on it.

The external ledger inventories memory channels, cache partitions, DMA engines, power rails, and their competing claimants. For each it records the arbitration configuration and measured interference. For the permission rail it records the 2 s hold-up, which the feed-loss injection of section 15.6 exercises, and a claimant list that carries the isolation claim itself, naming the microcontroller, the drive logic, and the spring-brake coils and no proposal-side load.

An interference measurement needs its operating state: model and firmware hashes, clock policy, enclosure and thermal interface, ambient and junction temperatures, power rail, arbitration settings, traffic generators, sampling rates, instrumentation, sample count, and test duration. The ledger carries those fields beside the latency distributions and unexercised states they qualify, and the header's observed maxima summarize those distributions. The 10^7 -tick trace of section 13.6 spans only 2 h 46 min 40 s of one machine's operation, too short to reveal any interference rarer than about one tick in 10^7 , and thin evidence for a fleet.

Every placement entry requires a predeclared falsification criterion written before the test executes. This criterion specifies the physical threshold that constitutes architectural failure, such as any enforcement cycle longer than the declared 250 μ s worst case during simultaneous neural accelerator bursts and DMA transfers. The record logs the measured distribution against this criterion and states whether the tested claim survived or was refuted. When interference violates the target budget and forces an engineering change, the record details the exact mitigation applied, such as enabling memory bus bandwidth throttling on the neural accelerator. The entry must also document the boundary conditions that were not exercised during the test run, such as operating states above the 45 °C bay ambient, multi-bit memory error injection, or simultaneous sensor transceiver fault states.

Chapter 15 consumes the recorded arbitration points and shared power rails as the target map for hardware and software fault injection, deliberately corrupting shared structures to test whether fault propagation matches the predicted failure domains. Chapter 16 incorporates the recorded loaded execution bounds and unexercised test limits into the formal safety case, transforming empirical measurements into evidentiary arguments for operational deployment.

Byte layout: The placement record's field offsets and sizes are given in section C.14.5.

Downstream: Hardware placement safety cases provide evidentiary warrants for production deployment in section 16.3.

Systems Perspective 13.1: When the fallback is a step

Question: Which rows of the stopping budget survive when the fallback is a step rather than a stop?

With placement protecting the one tick that τ_{delay} counts, Part III's stopping budget is complete for a machine whose fallback brakes it to rest, as the mobile manipulator's does. A walking humanoid has no rest state to brake into mid-stride, so its fallback is a recovery step, and each row of the budget has to be rechecked against that maneuver.

- **Observation age survives.** Age still becomes displacement (section 8.1), and on a turning head the error is angular. An uncalibrated 10 ms camera-IMU offset during a 1.5 rad/s head turn misplaces a point 5 m ahead by 7.5 cm.

- **Commitment changes.** The latched stop suffix of section 11.5 becomes a validated step, admitted with the trajectory as before but ending on a new foothold rather than at rest.
- **The inset changes.** A balance constraint can have relative degree above one, the case that section 12.7 resolves for raw clearance by folding velocity into the barrier. The balance barrier needs that same construction, and the tracking inset then applies to the barrier it produces.

13.9 Fallacies and Pitfalls

Placement determines the physical resources shared by complex proposal algorithms and critical enforcement routines. Traces of normal operation cannot prove these systems are truly independent when rare physical events, such as power voltage transients, clock faults, memory contention, or a stalled producer, stress those shared hardware pathways.

Fallacy: *Logical privilege rings protect real-time enforcers from shared-rail electrical droop.*

Suppose the mobile manipulator placed its lockstep permission core on the accelerator's supply in one SoC. The droop of section 13.3 then stretches the propagation delay of every gate on that shared rail by 39 percent, the permission core's gates included, whatever priority its software holds. A core whose clock leaves less timing slack than that stretch misses setup time on its critical paths. Both lockstep cores see the same sag, so they can latch the same wrong value and the comparator stays silent, or the brownout detector resets the enforcer while the base is moving. Access control decides which addresses the enforcer may write; it has no term for the voltage at its flip-flops. Placement qualification must bound the droop with decoupling capacitance or di/dt slew limits and show the timing margin under the declared current step, or remove the shared supply. The machine removes it, running its permission path on the isolated, held-up rail of section 2.8, which the accelerator does not draw on.

Pitfall: *Sharing Phase-Locked Loops or reset controllers across proposal and permission domains.*

Had the machine's application processor and permission domains shared one PLL and one reset controller (section 4.6), a thermal clock fault or a watchdog reset of the stalled policy cluster could have halted the permission domain as well. The nominal block diagram shows separate processors, but both depend on fragile silicon infrastructure that a single event can silently disable. Trace clock and reset trees through the physical implementation to ensure independent resources for continuous enforcement. The base and arm drives also need a defined hardware transition to a safe state when permitted commands stop arriving; restarting the application processor cannot be the only recovery mechanism.

Fallacy: *Low average memory bus utilization guarantees bounded read latency for real-time safety tasks.*

Suppose the machine's permission path read its sensor snapshot from shared DRAM, where a modest average bus use hides camera and logging bursts that coincide with its reads. The loaded read of 13.1 fits the sense allowance only while a single backlog precedes it, and nothing in the average bounds how many backlogs do. An assumed 320 μ s read tail leaves 80 μ s of the 400 μ s deadline, less than the 95 μ s that constraint evaluation and the drive-frame write need, so this configuration fails. A coherent TCM snapshot may remove the shared-DRAM read, provided its DMA completion, capacity, and local transfer time are qualified. Average utilization alone cannot establish the deadline.

Pitfall: *Using blocking software mutexes across inter-processor communication boundaries with untrusted neural policies.*

The chunk policy crashes halfway through publishing a chunk of 16 setpoints. An enforcer that had to acquire a lock held by that producer would freeze with it, which is why the boundary contract of section 13.4 admits no blocking synchronization. Its seqlock lets the enforcer reject a torn record without waiting and proceed with its fallback, but only if the retry behavior itself has a bound. The communication contract must specify timing behavior for interrupted publications as rigorously as it specifies the payload format.

13.10 Summary

Physical AI platforms place learned proposers on application processors and accelerators and the permission path on a bare-metal microcontroller, but that allocation alone does not separate them, because privilege boundaries govern none of the physical resources the two paths still share. Partitioning is therefore a design hypothesis until a loaded test fails to refute it.

Physical isolation must be supported by measured bounds on memory queues, crossbar arbitration, power transients, and the actuator path. A placement result is an empirical claim scoped to the measured conditions. A passing placement record shows that under a declared stress workload executing at a specific ambient temperature, with fixed clock governance policies and documented fault domains, the shared physical substrates of the die did not cause the untrusted path to violate the timing constraints of the safety path. On the warehouse mobile manipulator, drawing the chunk policy and the permission path in separate boxes on an architectural diagram would not have isolated them from a shared voltage regulator, a common memory crossbar, or a heat sink clamped across both processor clusters. Had the accelerator executing the learned proposal drawn peak current while every memory channel serviced direct memory access transfers, the resulting voltage drop would have reached a co-located permission core within nanoseconds and the heat within seconds; the machine places that core on separate silicon for this reason. The boundaries drawn during system design exist on the silicon only when verified against these shared electrical, thermal, and architectural limits.

The placement record carries that evidence forward. It takes the permission path's declared worst-case execution time and deadline from the enforcement record and adds the loaded observed maxima for the declared test conditions, the shared-resource queue and power headroom, the common power, clock, reset, and memory failure paths, and the conditions left untested. The machine's own record holds a 135 μs illustrative unloaded estimate against the declared 250 μs worst case inside the 400 μs deadline and still awaits a loaded ledger, which must name the arbitration policy, clocks, sample count, and physical endpoint before any release claim can rest on it.

With the completion of this placement analysis, the stages of Part III form a single machine that must be measured as one. The observation contract, the belief record, the intent lease, the trajectory record, the enforcement record, and the placement record form one chain, each a claim with conditions, so the stages that produce them cannot be treated as independent software modules communicating across an idealized channel. The logical boundaries established across earlier chapters provide the structure needed to reason about authority and failure propagation, but physical reality governs their execution. A memory controller follows configured arbitration and row scheduling, which need not reflect software privilege, and an interrupt can arrive during either workload. When the permission path admits a base velocity setpoint, the action is the last step of a pipeline whose timing, power, and memory footprints must be measured together on one chassis. The earlier chapters assumed that learned workloads could not delay the permission path that principle 3 demands, and placement turns that assumption into a falsifiable threshold with a stress test and a record behind it. Part IV inherits the three things the chain leaves open: the spare of the finished stopping budget, the unsettled premise of the site crossing rule, and the placement record's empty loaded field.



Key Takeaways: Heterogeneous silicon placement and memory contention

- **Logical separation is a hypothesis about hardware:** Privilege rings govern which addresses a task may touch, not where its requests wait, which rail feeds it, or which clock and reset tree it depends on. An independence claim stands only after a loaded test fails to refute it.
- **Averages cannot bound a deadline:** On the rejected shared die, one backlog of weight-tile, camera, and telemetry transactions stretches the permission path's snapshot read from 0.32 to 28.564 μs . More than 5 such backlogs cross the declared worst-case execution time, which nothing in the average forbids. Transaction size, queue policy, and bypass behavior decide the wait, so the bound belongs to the memory controller.

- **Coupling runs on three timescales:** Rail droop acts in nanoseconds, queueing in microseconds, and thermal throttling persists for seconds after the load stops. In the rejected shared-die placement, a halved clock stretches the enforcer's 250 μs cycle to 500 μs for 2,175 ticks.
- **The declared bound is the falsification threshold:** Any loaded cycle above the declared worst-case execution time refutes independence even when the deadline still holds, because every margin above the enforcer was sized on that number. A passing trace supports only the states it tested.
- **A refuted claim forces a priced choice:** Reservation costs idle capacity, partitioning costs working set, workload reduction costs policy authority, and relocation adds a board-level link whose worst case must fit inside the lease's renewal-jitter allowance.
- **The placement record scopes what was shown:** It takes its declared worst-case execution time and deadline from the enforcement record and adds loaded observed maxima, mitigations, and untested conditions. On the mobile manipulator the loaded field is still empty, so the independence claim reaches release review without evidence.

What's Next: From compute placement to supervisory intervention

What happens to the stopping budget when the slowest delay belongs to a person? Every delay this chapter examined belonged to the machine (a queued read, a stretched gate, a throttled clock), and each one can be bounded or refuted by a test on the chassis. Chapter 14 opens Part IV with a delay that no placement can bound. A remote assistant who takes over the mobile manipulator in an aisle reacts more slowly than any term measured here, over a link the permission path does not control, and that reaction spends the same stopping budget. Authority must then pass among the learned proposer, the permission path, and a human without the handover itself producing the motion it was meant to prevent, and every tick must record who held it.

IV

GOVERNING THE MACHINE

Governing the Machine

DEMONSTRATING a physical AI model in a controlled laboratory is a milestone. Outside the workbench, dust obscures lenses, lighting changes, wireless networks drop packets, gear-boxes wear, and human coworkers share the machine's path. Each change can invalidate a premise used to permit motion. Part III's runtime records rest on premises of that kind: a stopping budget that holds only while its spare survives, a crossing rule whose basis is still open (section 9.6), and a placement record whose loaded field is empty.

Principle 4: Evidence bounds authority

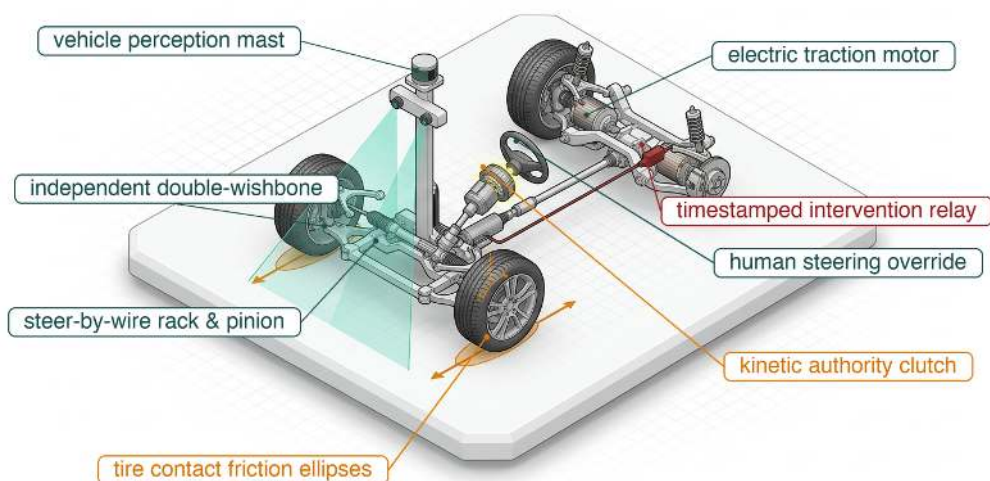
Invariant: Authority to operate is granted per operating condition by evidence whose age, provenance, and limits are recorded, and it expires when that evidence does. Zero failures in n independent, stationary trials bound the failure rate near $3/n$ at 95 percent confidence (section 7.5).

Implication: Who may command the machine, what the tests challenged, and which conditions it may enter are decided on the record, not on a demonstration. A premise that evidence cannot settle restricts operation through a reduced envelope, a passive limit, or refusal, and it stays open until a declared test closes it.

- **Supervisory Intervention (chapter 14):** Decides who holds authority over each actuator at each instant when a person and a learned proposer share the machine, and produces the authority record.
- **Adversarial Verification (chapter 15):** Decides which fault tests challenge each claim and how their outcomes count, and produces the fault record.
- **Deployment Release (chapter 16):** Decides whether the machine may operate and under which standing conditions, and produces the release manifest.

Part IV hands forward a release manifest and the premises it could not close. The manifest binds the evaluation record, the fault records, the placement record, and the authority record into claims with a verdict (unconditional operation, conditional operation, or refusal) and standing conditions that revoke it. What the evidence could not settle passes to chapter 17, where each open premise enters a residual-claims register with its missing observable, its restriction, its owner, and the test that would close it.

Supervisory Intervention



- 14.1 Shared Authority
- 14.2 Human Authority Over Actuation
- 14.3 Authority Transitions
- 14.4 Transfer Without Discontinuity
- 14.5 Authenticating an Intervention
- 14.6 How Handover Goes Wrong
- 14.7 Trustworthy Operational Logs
- 14.8 The Authority Log
- 14.9 Fallacies and Pitfalls
- 14.10 Summary

Purpose

When a human grabs the controls of a running machine, how do you transfer kinetic authority without causing the very collision you meant to prevent?

When a human supervisor asserts control over an autonomous system, the takeover request reaches hardware already in motion under neural control. Naively blending human and automated commands without strict arbitration yields conflicting control inputs, while instantaneous switching risks severe mechanical shock or sudden loss of traction. A safe intervention architecture must explicitly designate kinetic authority, enforce bumpless command transfer, and verify that handovers preserve dynamic feasibility throughout the transition window.

Crucially, human cognitive reaction time and telemetry transmission latency consume the same physical stopping budget as algorithmic delay; an operator cannot serve as a zero-latency safety fallback. When supervisory heartbeats drop or handovers stall, the machine must independently engage deterministic fallbacks. Every intervention must be immutably recorded with microsecond timestamps to separate autonomous failures from operator recoveries. Within the governance envelope, supervisory intervention treats human input as an external proposal that must pass through the deterministic permission path before crossing the causal boundary.

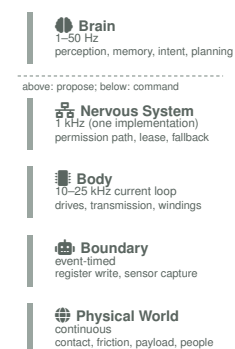


Figure 14.1.: The five-level machine stack within the governance envelope.



Learning Objectives

- Specify operator permissions that bind authorized actions to scope, deadlines, and authentication requirements
- Calculate the latest fallback time and takeover speed ceilings from reaction latency and remaining stopping clearance
- Design authority transfers that keep one actuator writer and preserve command continuity within physical limits
- Distinguish policy failure, authority transfer, and human recovery in timestamped authority records
- Evaluate whether an authority record can reconstruct command authority and physical outcomes after an incident

14.1 Shared Authority

A human supervisor is often drawn as one arrow into the plant. When a pallet blocks an aisle, the warehouse mobile manipulator needs a remote assistant to take over, yet at its chosen 1.3 m/s aisle speed a 2 s pause (illustrative; see the Reader Guide) before braking would carry it 2.6 m, more than twice its 1.10 m rack-end clear distance. An operator request reaches a machine that is already moving under an admitted command; it cannot erase momentum or make an actuator respond instantaneously. The arbiter, the permission path's source-selection step, must decide who may propose the next command, and the enforcer, its admission check, must admit or reject it; both run on the safety microcontroller (MCU). The plant must still have enough clearance to execute the chosen stop. An abrupt setpoint switch requests a torque step that the drives and the load must absorb. A delayed switch can exhaust stopping room even when both controllers are individually well behaved.

Bainbridge's classic *ironies of automation* (Bainbridge 1983) observe that an autonomous system tends to leave human operators responsible for the rarest, most complex edge cases while stripping them of the continuous manual engagement necessary to maintain situational awareness. When a remote assistant takes over the mobile manipulator in a blocked aisle, supervisory authority is effective only if it maps to an executable physical response bounded by actuator torque and thermal derating limits. Taking control of a machine already in motion forces the system to resolve two tightly coupled challenges simultaneously: discrete state arbitration without ambiguity in software, and continuous trajectory transition without mechanical shock in the plant. In discrete software, the architecture must prevent split-brain execution where the learned policy and the human operator issue concurrent, conflicting torque setpoints. In this chapter's 1 kHz controller, the independent enforcer revokes the policy's execution token, its right to have proposals admitted, and latches its last approved reference at commit, then remains the sole actuator writer during the subsequent blend. In continuous dynamics, the permission path must execute a bumpless transfer, blending between latched command endpoints, matching initial velocities, ramping each command within its rate ceiling, and checking measured plant limits during the transition.

Intervention events also reach the machine learning lifecycle. A rescued episode logged without its handover instant looks like an expert demonstration of the very near-failure states that made the rescue necessary. Section 5.6 shows how corrective aggregation instead labels the learner's own states, which requires the intervention record to keep the trace from before the takeover. The forensic authority record therefore timestamps every authority transition on the safety microcontroller's clock. The commit instant marks the boundary between policy-owned failure evidence and human recovery.

The first design decision is operational. It fixes which named intervention may reach which physical channel, and how long the machine can wait for it.

14.2 Human Authority Over Actuation

The remote assistant who clears a blocked aisle and the coworker who takes a mug from the gripper act on the same machine, and neither should hold the other's authority. Authority cannot attach to

Prerequisite: The proposal boundary of section 1.5, and the third law in section 1.7 that it enforces, set the baseline for supervisory preemption.

a generic human role or abstract supervision block. It must attach to specific named operations with explicit timing budgets, physical degrees of freedom, and deterministic preemption rules. Rather than granting an operator broad administrative authority, the permission path maintains an explicit **capability matrix**, a permissions table mapping individual operations (such as an emergency power cutoff, a velocity trim adjustment, a trajectory path veto, or manual drive of the base and arm) to authenticated roles, reflecting classic taxonomies of human supervisory control¹ (Sheridan 1992) and multi-level automation (Parasuraman et al. 2000). Each entry in this matrix defines the exact actuator channels the operation governs, the preemption tier it carries over the learned policy, and the maximum propagation latency permitted from physical input to actuator response.

Naming the operations separates **intervention** from **safety enforcement**, two mechanisms that system architectures routinely conflate. Enforcement withholds permission from a candidate action proposed by a learned model, modifying, clamping, or rejecting the setpoint while leaving the machine under autonomous control. Intervention transfers authority away from the model to an external entity, terminating autonomous operation and giving the supervisor the right to propose, while every command still passes the permission path. An **enforcement filter** sits in the fast control path to prevent the policy from demanding an over-current maneuver or violating a barrier certificate (Ames et al. 2019) during nominal tracking, respecting the actuator thermal limits established in section 2.1 and the kinodynamic feasibility boundaries from section 11.4. An intervention mechanism strips the policy of its execution token when an operational boundary is breached or an operator demands control. A system equipped with enforcement but lacking intervention cannot be redirected when human intent changes or external conditions fall outside the policy design envelope. Conversely, a system equipped with intervention but lacking enforcement permits a lagging operator to drive the plant into structural failure during nominal manual operation.

For example, the remote assistant who supervises the mobile manipulator may hold a signed **trajectory veto** capability for the base's planner channel: role `remote_assistant`, operation `veto_path`, channel `planner_proposal`, preemption tier above the learned planner but below the local enforcer, authenticated session and nonce, and a clearance-derived acknowledgment deadline. The enforcer responds to a valid veto with a checked local hold or stop. The same session carries a second entry, `manual_drive`, on the base-velocity and arm-joint channels, which becomes active only through a handover that commits within the clearance-derived deadline, and lapses with the manual lease of section 14.3; every command it carries still passes the permission check. A wheel drive-torque packet from that credential is denied, because neither entry grants the base's wheel drive-torque channel. The coworker at the packing station holds a different entry, `accept_item`, which lets the coworker take from the gripper the mug the arm picked from the takeaway conveyor (chapter 10, chapter 11). That entry grants no motion authority. The item-handover channel of the enforcement record (section 12.11) caps the arm's approach to the coworker's hand at 0.10 m/s, under the 0.118 m/s contact ceiling, and no capability lifts that cap. Each capability entry binds actor, operation, channel, deadline, authentication, preemption tier, and fallback; a valid signature alone grants no motion authority.

When an operator attempts an intervention, human biology imposes a hard real-time latency budget before any physical corrective action can occur. Human response time is not a single point event, but a cascaded pipeline of physiological and mechanical stages. For an operator monitoring a machine's telemetry stream, the sensory recognition stage covers the time for light to strike the retina, travel through the visual pathway, and register as an optical anomaly. The situational assessment stage follows, in which the operator recognizes that the learned policy is drifting toward a hazard and selects an appropriate counter-action. Finally, the motor actuation stage covers the transmission of motor signals down the spinal cord, muscle recruitment in the hand, and the physical displacement of an override switch. Summing the durations of these sequential stages establishes the physiological **takeover reaction floor**. Accounting for network transport and communication handshake latency T_{comm} , total supervisory takeover latency T_{Σ} decomposes into:

$$T_{\Sigma} = T_{\text{sensory}} + T_{\text{cognitive}} + T_{\text{motor}} + T_{\text{comm}} \quad (14.1)$$

During this reaction interval the machine keeps moving under the command it was last admitted, before the intervention signal reaches the arbiter, so adding the takeover latency T_{Σ} to τ_{delay} in equation 2.1 gives the distance a handover consumes. Because reaction travel grows with both the delay and the speed, a delay that fits at a crawl can consume the whole clearance at aisle speed.

¹ | **Supervisory Control Levels:** Sheridan and Verplank (1978) formulated a ten-level taxonomy of supervisory control, running from a human who performs the whole task without computer assistance to a computer that decides and acts while ignoring the human. A capability matrix draws such distinctions per operation rather than once for the whole machine.

Prerequisite: The stopping-distance equation whose pre-brake delay a takeover lengthens is derived in section 2.3.

The finished stopping budget of section 12.5 (table 12.1) shows what equation 14.1 costs on the mobile manipulator. At the 1.3 m/s aisle speed it leaves 102.6 mm of the 1.10 m rack-end clear distance. A takeover delay adds to the machine's own pre-brake delay τ_{delay} , and every millisecond of it becomes travel at the aisle speed:

1. **Mode 1: Direct Bilateral Teleoperation:** An in-loop assistant with a haptic channel and an active proprioceptive loop responds within an illustrative $T_{\Sigma,1} = 150$ ms, bus transit included. The machine travels 195 mm during that delay, already more than the spare margin, so at the aisle speed even with this takeover the machine's stop would end past the rack-end clear distance. Solving the budget for speed with $T_{\Sigma,1}$ added to τ_{delay} bounds the machine to 1.22 m/s.
2. **Mode 2: Passive Out-of-the-Loop Supervisory Takeover:** When an assistant monitors the telemetry of several machines passively, sensory recognition consumes $T_{\text{sensory}} = 300$ ms, cognitive situational assessment requires $T_{\text{cognitive}} = 1,200$ ms, motor switch actuation takes $T_{\text{motor}} = 300$ ms, and network transport and handshake protocol overhead adds $T_{\text{comm}} = 200$ ms. By equation 14.1, the takeover delay expands to $T_{\Sigma,2} = 2$ s, during which the machine covers 2.60 m, more than twice the clear distance, before any brake command. The same solution bounds the machine to 0.40 m/s.

In Mode 2 at the aisle speed, the machine would reach a static obstacle at the rack end at full speed, because contact occurs before braking ever begins. Human re-engagement latency therefore sets a ceiling on operating speed that no higher-gain actuator or brake pad can lift, and section 14.3 turns these two ceilings into the speeds at which the machine may request each kind of takeover.



Napkin Math 14.1: Takeover budget and clearance margin

Concept: The stopping distance during a human intervention adds every source of delay and distance before the machine comes to rest. On the mobile manipulator there are four terms:

1. **Machine blind travel:** The distance covered during the machine's own pre-brake delay ($v\tau_{\text{delay}}$), from table 12.1.
2. **Takeover travel:** The distance covered while the operator perceives the obstacle, decides, acts, and the command crosses the network (vT_{Σ}).
3. **Resident stop:** The C^2 stop suffix of section 11.5, $v^2/(2a_{\text{eff}})$ with $a_{\text{eff}} = a_{\text{brake}}/1.5$.
4. **Fixed overheads:** $\delta = \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}}$.

Worked Comparison: At the aisle speed $v = 1.3$ m/s against a rack-end clear distance of 1.10 m:

- **Direct Bilateral Teleoperation** ($T_{\Sigma,1} = 150$ ms):

$$d_{\text{stop},1} = v\tau_{\text{delay}} + vT_{\Sigma,1} + \frac{v^2}{2a_{\text{eff}}} + \delta = 173.68 + 195 + 633.75 + 190 = 1192.43 \text{ mm}$$

- **Out-of-the-Loop Takeover** ($T_{\Sigma,2} = 2$ s):

$$d_{\text{stop},2} = v\tau_{\text{delay}} + vT_{\Sigma,2} + \frac{v^2}{2a_{\text{eff}}} + \delta = 173.68 + 2600 + 633.75 + 190 = 3597.43 \text{ mm}$$

Systems Insight: Passive monitoring inflates the takeover delay from 150 ms to 2 s and grows the stop from 1.19 m to 3.60 m, 3.0× longer. Neither fits the 1.10 m clear distance at the aisle speed, which is why the machine slows before it asks for either one. For the kinematic derivation of these stopping distances, see section C.13.

The physical tension between human reaction latency and stopping distance extends across every mobile embodied system, from indoor warehouse manipulators to highway-speed autonomous vehicles (figure ??). Stopping distance decomposes into two fundamentally distinct kinematic regimes:

linear reaction drift distance $d_{\text{drift}} = v \cdot (T_{\Sigma} + \tau_{\text{delay}})$ during which the machine travels unbraked under sensory, cognitive, and network delay, followed by quadratic kinetic dissipation $d_{\text{brake}} = v^2 / (2a_{\text{eff}})$. Because reaction drift scales linearly with velocity while braking scales quadratically, an out-of-loop supervisor monitoring passively from a remote console ($T_{\Sigma} \approx 2.5$ s) accumulates over 72 meters of blind drift at 100 km/h before the physical brake pads engage. When evaluated against a fixed forward visibility clearance ($d_{\text{clear}} = 60$ m), the remaining clearance margin $M = d_{\text{clear}} - d_{\text{stop}}$ defines a hard operational ceiling. Beyond critical velocity thresholds—ranging from 33.7 km/h for remote teleoperation to 78.3 km/h for an alert in-loop operator—the clearance margin becomes negative ($M < 0$), rendering high-speed collision physically inevitable regardless of braking force.

Table 14.1 extends the two takeover modes to the other ways a site might let a person intervene, each with an illustrative delay. Each row adds its delay to τ_{delay} and solves the finished stopping budget for the largest speed at which the machine still stops inside the rack-end clear distance. The ceiling falls from 1.22 m/s for direct bilateral teleoperation to 0.40 m/s for an out-of-loop takeover, and no mode fits at the 1.3 m/s aisle speed. The last column names the local protection that must hold the machine safe while that delay runs.

Table 14.1: Takeover delay and speed ceiling: Each row adds its selected delay to the machine's pre-brake delay τ_{delay} and solves the finished stopping budget of table 12.1 for the largest speed that still stops inside the rack-end clear distance.

Intervention mode	Selected total delay	Added travel at aisle speed	Speed ceiling	Local protection
Direct bilateral teleoperation	150 ms	195 mm	1.22 m/s	Qualified passivity/rate checks
Continuous velocity guidance	450 ms	585 mm	0.96 m/s	Onboard velocity/clearance gate
Supervisory waypoint dispatch	1,150 ms	1,495 mm	0.60 m/s	Local obstacle veto and fallback
Out-of-loop takeover	2,000 ms	2,600 mm	0.40 m/s	Independent stop while waiting
Physical protective input	250 ms human-plus-device response	325 mm	1.13 m/s	Risk-assessed protective stop



Figure 14.2: Industrial teach pendant: Front view of an ABB pendant showing the display, buttons, emergency-stop actuator, and cable. Its rear enabling grip, internal channel wiring, response time, and stop behavior are not visible here. The ABB manual documents a three-position grip and a 250 mm/s manual-mode limit for its specified configuration. Photograph by Auledas, CC BY-SA 4.0, via Wikimedia Commons; local copy is re-encoded.

Human response is slow and prone to interruption; thus, manual authority cannot be an open-ended state. If an operator assumes control and subsequently experiences wireless packet loss, physical incapacitation, or cognitive distraction, the machine cannot remain adrift under stale manual setpoints. The architecture protects against operator dropouts by bounding manual authority with configured and validated temporal leases (countdown timers that expire without active, periodic renewal) and hardware **enabling devices**, which require continuous physical engagement to maintain actuation channels (figure 14.2). In a lease-based communication model, the delegation window is sized from the machine’s communication and stopping budgets, as the mobile manipulator’s manual lease in section 14.3 shows. The operator console must continuously stream authenticated keep-alive packets to renew the lease before the deadline expires. If a single lease window lapses without renewal, or if an operator releases a spring-loaded physical deadman switch on the controller, manual proposals are revoked at the qualified detection deadline, and the enforcer commands the plant-specific validated stop or hold of the fallback ladder (section 12.9).²

Even while an operator holds an active lease, the enforcer evaluates human setpoints against the same barrier certificates, current limits, and kinematic limits applied to the learned model. If an operator panics and commands a base turn that would tip the loaded tote rack, or requests a joint torque that would shear the gear teeth of the arm’s actuator, the enforcer rejects the command. A human command therefore remains a proposal, and principle 3 applies to it unchanged. Intervention transfers the right to propose away from the learned policy, while the right to admit stays with the permission path. What remains to design is how the right to propose changes hands, at which tick, and what the machine does when the handover stalls.

14.3 Authority Transitions

Control ownership must be a discrete state, never an inference from a blended command. When an engineering team attempts to soften transitions by continuously averaging the control vectors of an autonomous policy and a human operator, the resulting command may represent neither intent. If the remote assistant steers the base left around a pallet while the learned planner steers right to pass it on

² | **Three-Position Enabling Devices:** The cited ABB operating manual documents a three-position enabling grip and a 250 mm/s manual-mode limit for that robot. Releasing or over-squeezing the grip is a request into the configured safety circuit, and the machine’s risk assessment under ISO 13849-1 sets the required performance level and the stop the circuit selects.

the other side, a linear interpolation between the two commands holds the heading straight and drives the base into the pallet.³ To prevent this ambiguity, control authority over each actuator channel, such as the base drives or the arm joints, must resolve at every discrete time tick t to a single, mutually exclusive state $S_t \in \{\text{Autonomous, Handover-Pending, Blend, Human-Manual, Fallback, Protective}\}$. A fallback state records the rung of the fallback ladder (section 12.9) that the permission path is executing (hold, stop, or inhibit), and the protective state is entered only from an independent protective input or a qualified critical fault. The permission path evaluates arbitration logic, determines the active state, and commits that identity to the bus before dispatching setpoints to low-level motor controllers. At no point do two masters write the actuator. During BLEND, the enforcer alone owns the actuator channel and interpolates between a latched last-approved policy reference and a checked human target.

Each edge in table 14.2 names its guard and deadline, its single actuator owner, and the fallback taken when the guard fails or an active lease lapses.

Table 14.2: Authority state machine transition matrix: Each row names the single actuator owner and the state-dependent fallback. The example uses a 1 ms arbiter tick. The BLEND state interpolates only latched, checked endpoints, under enforcer ownership.

From → to	Guard and deadline	Actuator owner / command	Failure response
Autonomous → Handover-Pending	Policy requests supervision; deadline computed from clearance	Enforcer admits policy proposals while feasible	Validated fallback if deadline reached
Autonomous or Handover-Pending → Blend	Authenticated operator ACK, fresh state, feasible target and stopping reserve; commit on a control tick	At commit, revoke policy token, latch last approved command and checked manual target; enforcer alone writes actuator	Deny invalid target; use resident fallback
Blend → Human-Manual	Profile completes and manual lease remains valid	Enforcer admits live human proposals within physical limits	Validated fallback on lease or enabling-device loss
Handover-Pending, Blend, or Human-Manual → Fallback	Timeout, stale lease, or invalid state	The permission path executes the validated HOLD or STOP rung and records it	Re-arm only after validated recovery procedure
Any state → Protective	Independent protective input or qualified critical fault	Safety circuit/controller selects plant-specific protective stop	Validated brake or hold, not torque removal alone

Shifting authority between these discrete states requires a deterministic **four-phase handshake protocol** (Request → Acknowledge → Commit → Confirm). Executed by the enforcer, this four-stage transaction enforces mutual exclusion, validates cryptographic freshness to ensure commands are not stale network echoes, and atomically shifts actuation authority between discrete holders across a synchronous tick boundary. The sequencing excludes split-brain command contention, where two masters attempt to drive the hardware in conflicting directions during the same compute cycle. In the first phase, the *Request* phase, one entity signals an intent to transfer control. This occurs either when an operator asserts an override through a physical input like a button press or console force exceeding a calibrated threshold, or when the autonomous policy detects that sensor confidence has degraded and requests human assumption of control. In the second phase, the *Acknowledge* phase, the enforcer inspects the target state prerequisites, verifying that sensor data is valid, communications are active, and the recipient is physically capable of receiving command authority. In the third phase, the *Commit* phase, the enforcer executes the state transition at a synchronous control tick boundary, revoking the policy's token on the committed channels, latching its last admitted endpoint and the checked human target, and assigning the enforcer sole actuator ownership throughout BLEND. In the final phase, the *Confirm* phase, the permission path broadcasts a state confirmation packet across the network and triggers physical feedback, such as haptic vibration or optical indicators, to tell the operator, channel by channel, whether blending or manual control is active; lost confirmation does not change the committed owner.

Each phase in table 14.3 carries a budgeted bound, a guard and owner, and its failure handling. The transaction rejects spoofed override injections and routes interrupted handovers to the separately validated fallback path.

3 | Fieldbus Write Exclusivity: Authority arbitration requires hardware-enforced exclusivity across distributed fieldbuses. Dual-writing to CAN-FD or EtherCAT segments from autonomous and manual controllers induces non-deterministic actuator chatter and inverter current oscillations. A qualified arbiter and bus permissions must prevent two software sources from writing one actuator channel.

Downstream: These takeover transitions become the authority fault cases derived in section 15.3.



The illustrative four-phase budget totals 31 ms; the enforcer owns the blend after commit.

Table 14.3: Four-phase handover timing: The phase budgets sum to $10 + 5 + 1 + 15 = 31$ ms. Confirmation and physical blending are distinct from the atomic ownership commit.

Phase	Budgeted bound	Guard and owner	Failure handling
1. Request	10 ms	Authenticated role and fresh, clock-converted nonce; no owner change	Reject invalid packet; keep current qualified mode
2. Acknowledge	5 ms	Check state, manual target, and full stop reserve	Reject request; invoke fallback if current mode cannot continue
3. Commit	1 ms tick	Revoke policy token, latch both checked endpoints, assign enforcer sole actuator write path	No partial commit within the requested channel mask; fallback on failed guard
4. Confirm	15 ms	Announce BLEND, then MAN only after profile and lease check	Lost notice alarms operator but does not undo commit

Because human attention is volatile, a handover request initiated by the autonomous policy cannot remain open indefinitely. If a machine encounters an operational design domain boundary at speed and requests intervention, an operator who is distracted, asleep, or incapacitated will fail to complete the handshake. The state machine enforces the earlier of a configured protocol ceiling and a clearance-derived latest fallback time. Take a static obstacle at clear distance D_{clear} , the case the stopping budget protects, with forward speed $v > 0$, the resident stop's deceleration $a_{\text{eff}} > 0$, fixed overheads $\delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}}$, and the machine's own pre-brake delay τ_{delay} . Solving the stopping envelope of equation 12.1 for the delay a request may add then gives the latest time after the request at which fallback may be dispatched:

$$T_{\text{latest}} = \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \epsilon_{\text{track}} - v^2 / (2a_{\text{eff}})}{v} - \tau_{\text{delay}}, \quad T_{\text{timeout}} = \max(0, \min(T_{\text{config}}, T_{\text{latest}})). \quad (14.2)$$

The safety microcontroller dispatches fallback **no later than** the last permission tick before this deadline; a nonpositive T_{latest} calls for immediate fallback and may mean stopping is already infeasible. On the mobile manipulator, a pallet left just past a rack end, which comes into view only at the clear distance, is such an obstacle, and the numerator is the rack-end clear distance less the finished stopping budget of table 12.1 without its blind travel ($v\tau_{\text{delay}}$), which the formula subtracts as time. T_{latest} is therefore the budget's spare margin spent as time. If the policy requests a handover at the 1.3 m/s aisle speed, the machine has 102.6 mm to spare, so $T_{\text{latest}} = 78.9$ ms. On the 1 ms permission tick, fallback begins by 78 ms, long before a 500 ms configured ceiling would expire. T_{latest} is a state-dependent deadline; the configured ceiling alone is not a deployable safety rule.

The deadline decides which takeovers the machine may request at speed. The 31 ms four-phase handshake of table 14.3 fits inside it, so an authenticated commit can land before the fallback must begin. A person cannot respond that fast. The 150 ms in-loop takeover of section 14.2 needs a speed of 1.22 m/s or less, so before it requests one the machine slows to a chosen 1.2 m/s. There the budget leaves 209.7 mm, equation 14.2 gives $T_{\text{latest}} = 174.7$ ms, and the takeover commits with 24.7 ms in hand. The deadline at the slow-down, not the aisle-speed one, governs every in-loop takeover the machine requests. A 2 s out-of-loop takeover needs 0.40 m/s or less, so the machine requests one only at a chosen 0.3 m/s crawl or from standstill. The hidden pallet allows neither, so the machine answers it with its own stop and requests the out-of-loop takeover from standstill. The slowed approach and the crawl govern requests the machine can anticipate, and the remote assistant who routes it around an obstacle therefore takes over from one of those speeds or from a stop, never at aisle speed.

Once authority resides in Human-Manual, the machine must verify the engagement signal required by its delegated-control design. A teleoperation console might sense grip, gaze, or force, and the designer chooses among them by their measured coverage and false-loss rate. The manual lease $T_{\text{lease,man}}$ of section 14.2, a lease in the sense of section 4.3 on the operator's input, is set here at a chosen 100 ms. After a lapse, the state machine blocks a return to Autonomous or Human-Manual until its specified reset conditions are met. Those conditions may require a full stop, fault review, and a new handshake.

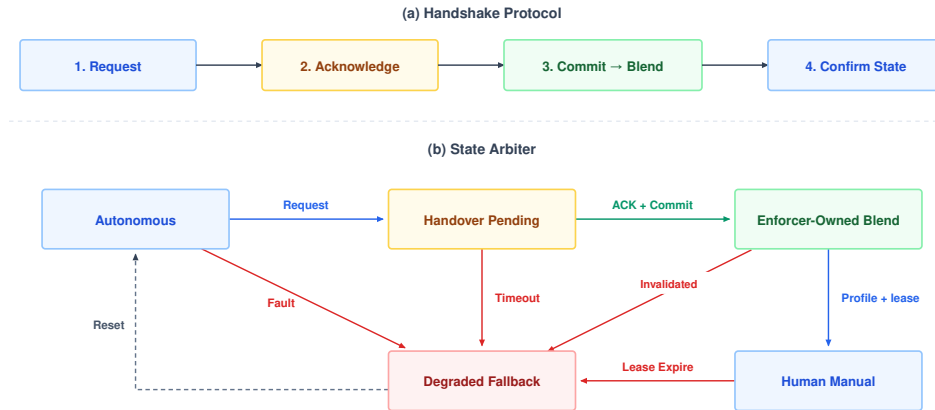


Figure 14.3: Authority handshake and owner states: Request and acknowledgment precede atomic commit. At commit the policy token is revoked and the enforcer owns the blend between checked, latched endpoints; manual proposals begin only after profile completion with a live lease. Timeout invokes a separately validated fallback.

When multiple control sources propose commands, the permission path resolves source selection through a fixed arbitration hierarchy (Brooks 1986a) evaluated each cycle, below the proposal boundary of section 1.5. The independent enforcer admits, modifies, or rejects the selected candidate under its configured physical limits. In Human-Manual with a live lease, the human proposal takes precedence over a learned proposal. In Autonomous, the learned policy may propose actions for admission. Fixed-state arbitration takes bounded work per tick in this design; it does not require live negotiation between the two proposers.

The runtime intervention interface joins the four-phase handshake to an arbiter with bounded timeouts and explicit fallback transitions (figure 14.3).

One writer makes authority unambiguous at each control tick, but an unblended switch may still request an abrupt command change. If the policy's last admitted torque on an arm joint differs from the operator's target, directly swapping those setpoints requests the whole difference as a step within one tick. The committed enforcer instead uses the latched endpoints to generate a checked transition profile with a reserved stopping margin (section 11.1).



Checkpoint 14.1: Authority transition protocols and timeout bounding

Before designing handshake protocols for human-robot authority handoffs, verify your understanding of transition state machines and timeout dynamics:

- ❑ **Finite-state transition semantics:** Can you trace the deterministic progression among the Autonomous, Handover-Pending, Blend, Human-Manual, and Fallback states, and name the guard and the single actuator owner on each edge?
- ❑ **Kinematically bounded timeouts:** Can you calculate why the timeout window T_{timeout} for an unacknowledged takeover request must be dynamically bounded by the remaining physical clearance distance rather than fixed as an arbitrary operating-system timer?
- ❑ **Asymmetric preemption hierarchy:** Can you explain why the qualified local protection path must meet its measured deadline, while restoring autonomous authority requires a fresh admission handshake?
- ❑ **Fallback on a lapsed handover:** Can you explain why a lapsed or unacknowledged handover goes to the validated stop rung rather than back to the degraded policy that requested it?

If any transition paths permit unhandled timeouts or undefined intermediate states, the authority protocol is vulnerable to confusion and runaway actuation. Review this section before proceeding to smooth command blending.

14.4 Transfer Without Discontinuity

At commit time t_0 , the enforcer latches the last admitted policy command $\mathbf{u}_0$ and a checked human target $\mathbf{u}_1$. A direct switch would request $\Delta\mathbf{u} = \mathbf{u}_1 - \mathbf{u}_0$ in one tick. Its ideal command derivative is discontinuous, but the actual plant filters the command through finite drive, brake, and mechanical dynamics. A large change may excite the arm's gearbox, exceed the drive wheels' available traction, or shift the tote on its rack. The transfer therefore checks both rate and full stopping clearance rather than assuming a software tick maps instantaneously into physical force (section 1.2).

Eliminating this step discontinuity requires smoothly mixing the old and new commands across a finite time window τ_{blend} , a technique known as **bumpless transfer**. A linear blend changes slope at its endpoints. For a fixed-endpoint change in commanded acceleration, a C^2 bumpless transfer ramp eases in and out, removing endpoint jumps in the command and its first two derivatives. A standard curve used for this is the quintic smoothstep.



Definition 14.1: C^2 bumpless authority transfer ramp

C^2 **bumpless authority transfer ramp** is the enforcer-owned command profile between two checked, latched references $\mathbf{u}_0$ and $\mathbf{u}_1$, parameterized by a monotonic weighting function $\alpha(t) \in [0, 1]$ with vanishing first and second boundary derivatives ($\dot{\alpha} = \ddot{\alpha} = 0$) over a finite blending window τ_{blend} , ensuring continuity of commanded acceleration and bounded derivative jerk across authority handoffs.

1. **Significance:** A setpoint step has an unbounded ideal derivative. A C^2 command profile removes that discontinuity; measured actuator tracking and plant limits still decide physical feasibility.
2. **Distinction:** Unlike a linear interpolation ramp that exhibits derivative discontinuities at its endpoints, a C^2 quintic spline enforces vanishing first and second derivatives ($\dot{\alpha} = \ddot{\alpha} = 0$) at both entry and exit, making the commanded acceleration continuous with zero endpoint jerk for fixed endpoints.
3. **Common pitfall:** Using $\tau_{\text{blend}} < 1.875|\Delta\mathbf{u}|/j_{\text{allowable}}$ for a fixed acceleration-command change breaches the stated command-jerk limit (section B.3). Resonance, slip, and stopping distance require plant tests.

The enforcer blends with the quintic smoothstep, whose coefficients follow from requiring zero first and second derivatives at both ends of the window (section B.3). It is the same C^2 property the planner relies on at its seam, where no acceleration step appears at the switch (section 11.5); here the switch is between proposers rather than between trajectory segments. The appendix also derives the shortest window that keeps a fixed change in commanded acceleration within a chosen command-jerk limit $j_{\text{allowable}}$. The same bound applies to a torque command, so an illustrative 15 N·m change in one of the arm's joint torques under a 750 N·m/s rate ceiling takes at least 37.5 ms.

Increasing τ_{blend} reduces the profile's peak commanded jerk, but delays the full human target and can lengthen the stopping trajectory. Consider the remote assistant's takeover of the mobile manipulator's base, committed while the machine cruises at the $v = 1.2$ m/s it slowed to before requesting it (section 14.3), and suppose the assistant commands a stop at the scenario's braking deceleration, a fixed step of $|\Delta a| = 2$ m/s² from the latched cruise command. The enforcer holds the blend to $j_{\text{allowable}} = 8.89$ m/s³, the peak jerk $6v/T_{\text{stop}}^2$ of the resident stop from the same speed (section 11.5), so the transfer loads the tote rack no more abruptly than the machine's own stop. The smoothstep bound then sets a minimum blending window of 421.88 ms. The smoothstep weight averages one half over the window, so the base would leave the blend at 0.78 m/s after traveling

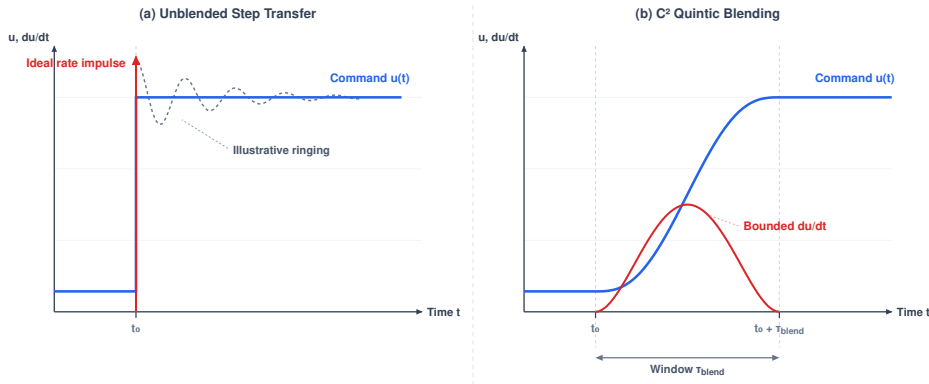


Figure 14.4: Step versus quintic command transfer: A step and fixed-endpoint quintic blend are compared. Blue is the command; red is its ideal derivative, and gray ringing is schematic. The profile bounds the commanded rate for the assumed endpoints.

455.4 mm, and braking from there at the full target would take another 151.4 mm, for 606.8 mm to rest.

An instantaneous step takeover would stop in 360 mm, so bumpless blending imposes a spatial penalty of 246.8 mm.⁴ The blended stop is also 66.8 mm longer than the 540 mm resident stop from the same speed. A blend therefore spends stopping reserve beyond the stop the budget holds. The enforcer admits each blended tick only while the resident stop from the resulting state still fits, and where no blend fits, the base receives the resident stop itself, the suffix the fallback ladder's stop rung executes (section 12.9). Bypassing the blend window by applying the acceleration step within a single 1 ms permission tick would instead request a command jerk of about 2,000 m/s³ over that tick. The idealized step is only a comparison baseline.

Command blending removes the chosen output discontinuity, but a standby human feedback controller can still accumulate integral error while disconnected from the plant. **Initial condition matching** pre-biases that controller against the measured state before commit, avoiding an idle-integrator transient. It does not force the deliberate human target to equal the last policy command. The enforcer latches both endpoints, sizes the blend for their actual difference, and checks the resulting command and stopping reserve throughout transfer.

To verify a transfer, record the command profile, motor current, encoder motion, and chassis inertial response on a shared timebase with bounded alignment error. A step command has a derivative discontinuity in the ideal model; ringing, high jerk, and slip are possible *observations*, not inevitable waveforms. A fixed-endpoint quintic bounds the commanded rate, and figure 14.4 contrasts the two profiles schematically. The measured plant response establishes whether the handover met its physical limits.

The enforcer's 1 kHz loop gathers the four-phase handshake, integrator pre-biasing, and the C^2 quintic blend into one protocol on the safety microcontroller. Each tick it samples plant state, stopping reserve, protective inputs, and the authority lease. It commits an ownership change only on the qualified commit tick that follows a verified acknowledgment, blends only between latched endpoints and checks every blended candidate before dispatch, and falls back on a lapsed lease or a failed clearance check.

Once the blend completes and the remote assistant holds the arm in Human-Manual, driving it through a force-reflecting leader, the arm must respond either to the operator's motion or to the operator's force. **Impedance control** (Hogan 1985) renders a force from measured motion through the virtual spring-damper of 2.1 and suits backdrivable joints. **Admittance control** measures the operator's force at a wrist force-torque sensor and yields in motion under stiff position tracking, which suits the high-ratio drives whose reflected rotor inertia $N^2 J$ resists being moved by hand (section 2.4). Either mode closes a loop through the operator, the mechanism, and the transport delay, and the stability of that loop depends on grip, gains, and delay, so each mode must be tested

Prerequisite: Acceleration-bounded bumpless transfer profiles build upon the physical stopping constraints enforced in section 12.5.

⁴ | **Bumpless Transfer Braking Penalty:** The penalty belongs to the smoothstep profile, its fixed endpoints, and the stated acceleration step; another profile or endpoint pair gives a different penalty.

Protocol: The enforcer's cadence, memory and write invariants, and per-state execution steps are given in section C.14.7.

for passivity over its specified operating range. A passive teleoperator, meaning the leader, link, and arm taken together as seen from the operator's hand, never returns more energy to the operator than the operator put in. The enforcer keeps its clearance and torque limits whichever mode the leader uses.

Blending, pre-biasing, and compliant interaction all assume that the proposals they shape come from an operator entitled to control the arm, and a force measured at a sensor carries no credential of its own. If the permission path accepts a transfer request without validating identity, an unauthenticated command injection or a malfunctioning subsystem can seize authority and override the machine while it is in motion. The mechanisms that make an intervention smooth must therefore be paired with mechanisms that establish who is allowed to intervene.

14.5 Authenticating an Intervention

The remote assistant's `veto_path` packet and a forged wheel drive-torque packet can reach the mobile manipulator over the same wireless link in the same control cycle. The permission path must keep the forger from seizing physical authority while still letting an authorized operator halt the machine before collision. These requirements conflict, so intervention operations are classified by the mechanical work they induce. Removing power or asserting a mechanical brake is physically distinct from injecting active motion commands, and the two actions do not warrant identical verification mechanisms. A protective stop is a braking or holding rung of the fallback ladder, not torque removal alone (section 12.9). In contrast, injecting base velocity, wheel torque, or joint velocity setpoints introduces fresh mechanical power into the system. An unauthorized or corrupted motion command can accelerate a moving mass into an obstacle, whereas a trusted out-of-band protective input requests the plant-specific minimum-risk stop.

Motion commands introduce active kinetic hazards; thus, the verification mechanism must meet both latency and freshness bounds from the kinematics of the maneuver. Verifying the signature and checking the replay window of a network motion or override command is one more delay before braking can begin. It adds to the actuator onset T_{act} , the bus delay T_{bus} , and the other terms of τ_{delay} (equation 2.1), so for a static obstacle at clear distance D_{clear} and forward speed v the maximum permissible verification latency $T_{auth,max}$ is the handover deadline itself, T_{latest} of equation 14.2.

On the mobile manipulator at its 1.3 m/s aisle speed, that bound is 78.9 ms. Assume two illustrative verifiers on a microcontroller, one slow and one fast:

- **Verifier A (slow):** Verification takes $t_{auth,A} = 180$ ms, more than the deadline. The machine travels 234 mm while the signature is checked, 131.4 mm more than the spare, so its stop ends that far past the rack-end clear distance.
- **Verifier B (fast):** Verification takes $t_{auth,B} = 15$ ms, well inside the deadline. The machine travels 19.5 mm during the check and keeps 83.1 mm of the spare.

Freshness requirements arise from the same kinematics. If an authentication window permits a timestamp drift or message age threshold of $\tau_{fresh} = 100$ ms, an authenticated packet replayed from the boundary of that window commands an action based on a base position displaced by $\Delta x = v\tau_{fresh} = 130$ mm at the aisle speed, more than the whole 102.6 mm spare.

A trusted, independently wired protective input is evaluated by its safety circuit without waiting for a network signature, and its response is the stop or hold selected and validated for the plant. Remote motion or takeover packets follow the capability and authentication checks. A corrupted checksum, stale nonce, or failed signature on that network is rejected, logged, and handled as rule A4 of section 14.8 specifies; malformed traffic does not acquire the authority to command a brake. If valid command renewal is lost, the separately monitored lease expires and the enforcer invokes its validated fallback. This separation preserves an available local protective path while preventing arbitrary network noise from forcing an unsafe stop.

Authentication and transport delay also reach training. A training simulator that always supplies immediate human rescue can underprice risky states. If deployed intervention instead incurs authentication, transport, and actuation delay, a policy trained on that mismatch may visit states from which rescue is no longer feasible. The remedy is to model the qualified latency and fallback path in training and hardware-in-the-loop evaluation, retain failed learner-visited states, and test the actual clearance distribution.

CAN bus priority: A transmitting CAN frame is non-preemptive. A higher-priority identifier wins only at the next arbitration opportunity; controller buffer policy and worst-case current-frame serialization must be included in the measured delivery bound. An independent protective input avoids this shared queue where the safety case requires it.

Prerequisite: Microsecond override masking registers on real-time fieldbuses are implemented in section 4.5.

The authority inventory must include diagnostic, debug, bootloader, and wiring paths that can affect actuation. A production design may disable debug access, authenticate maintenance commands, protect boot updates, and enforce bus permissions according to its threat and fault model. The verification task is to show that each reachable actuator-write path is either gated by the permission authority or independently protected. The argument of this section covers remote packet injection over wireless links, replay on internal buses, and commands from a compromised application processor. Physical tampering lies outside it, and whether the safety case must answer tampering depends on the threat model, inspection access, and packaging.

Authentication establishes who is entitled to take over. It does not ensure that the handover leaves the machine under coherent control. When authority transfers while the base carries momentum and the arm's drives deliver torque, the policy, the operator, and the enforcer must agree on who holds each channel. Handover goes wrong when that agreement breaks.

14.6 How Handover Goes Wrong

A handover can fail while every component works. When an autonomous policy and a human operator share authority, it can fail through mode confusion and authority ambiguity, where the machine assumes the human is managing an operational axis while the human assumes the machine retains jurisdiction. In multi-axis physical systems, authority is frequently decomposed across channels, and on the mobile manipulator it splits between the base and the arm. If the learned policy encounters an unmodeled surface reflection and quietly yields the arm to the remote assistant without releasing the base, an assistant who guides the arm may assume the base is holding still while the policy is still driving it toward the rack end. Conversely, if the assistant commands the base to stop, firmware that transfers only the base leaves the arm policy still reaching for the tote against the assistant's intent. In telemetry logs, this failure presents as disjointed control flags where authority bits toggle asynchronously across execution threads. At the human-machine interface, however, this state is undetected until kinematic error exceeds the physical clearance margin, because neither claimant receives an immediate, unambiguous signal of the other's partial withdrawal. Per-channel ownership permits such a partial transfer, so the protocol must make it explicit. Every request names the channels it covers in its channel mask, the state machine of section 14.3 commits owner states only for those channels, and the Confirm phase announces the owner of each channel to the operator, so a base that stays with the policy is shown as the policy's.

Even with defined authority transfer protocols, the human nervous system cannot match the response deadlines of moving mass. A remote assistant in direct control of the mobile manipulator answers within the 150 ms in-loop takeover of section 14.2. When the same assistant supervises several machines passively while their policies run, situational awareness decays, and an unexpected handover request meets the 2 s out-of-loop takeover instead. As section 14.2 showed, a machine left at aisle speed covers more than twice the rack-end clear distance during this gap, before the assistant has perceived the obstacle, re-established spatial orientation, and applied input. Operator-facing cameras can track gaze direction and eye closure. They cannot measure comprehension of the scene geometry, so the growth in latency is detected only after the response deadline has passed.

At Tempe, vigilance decay met a disabled stop path (section 1.6) (National Transportation Safety Board 2019). In the Cruise incident, a vehicle that had stopped after striking a pedestrian began a pull-over and dragged her beneath it (section 12.9) (Quinn Emanuel Urquhart and Sullivan, LLP 2024). The first asks whether an operator-monitoring contract backed by an independent stop would have stopped the vehicle within the remaining clearance, and the second whether contact uncertainty should revoke motion authority until a validated hold is established.

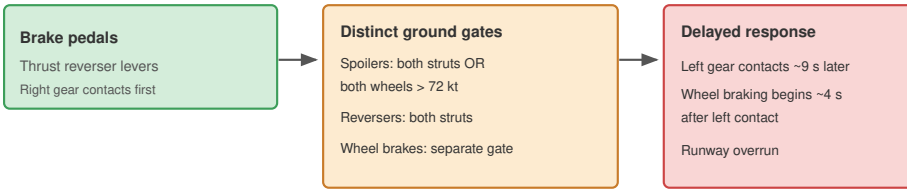
When an intervention occurs while an actuator applies force, manual input and the enforcer can oppose each other. For example, the remote assistant driving the arm under `manual_drive` through a force-reflecting leader may command a joint toward the position limit at which the enforcer is holding it. The torque the assistant's command requests and the drive current holding that joint then act against each other on the same link. If the human-device loop has insufficient phase margin, repeated corrections may oscillate at a frequency set by the gains, delay, grip, and joint mechanics. The enforcer must retain the physical limit while the leader makes the opposing force and active mode legible to the assistant.

Uncoordinated handovers also corrupt the data drawn from fleet logs. If intervention segments enter behavioral cloning or diffusion training without intervention masking, which uses the logged commit instant to separate the policy’s own states from the recovery, the policy learns emergency recovery as nominal behavior, and validation loss on trajectory chunks does not reveal the corruption.



War Story 14.1: Ground-contact interlocks on flight 2904

Crew requests deceleration



A crew input and each actuator effect require separate state and timing evidence.

Context: On Lufthansa Flight 2904 at Warsaw, the crew commanded deceleration that the aircraft's ground-contact logic did not grant, because each decelerating function carried its own precondition on sensed state (Main Commission Aircraft Accident Investigation Warsaw and German Federal Bureau of Aircraft Accident Investigation 1994).

Mechanism: The right gear contacted the wet runway about nine seconds before the left. Spoiler deployment required both struts compressed **or** both wheel speeds above 72 knots; thrust reversers required both struts. Wheel brakes had a separate wheel-speed gate and alternate mode. Each precondition acted as a capability entry, granting a function only in a sensed state, and together they withheld the requested deceleration.

Outcome and lesson: Wheel braking began roughly four seconds after the left-gear contact, about thirteen seconds after the first right-gear contact. The aircraft overran the runway, leaving its end at 72 knots. A capability matrix that gates operations on operating preconditions must announce an unmet precondition to the person commanding, name the sensed state it waits on, and be tested against the states, such as one strut compressed on a wet runway, in which its preconditions disagree with the operator's intent.

The arbiter of section 14.3 exists to prevent these failures, because the control architecture cannot rely on implicit handoffs, human attentiveness, or ad hoc torque overrides. Showing afterward that it did requires a record kept at every tick, channel by channel. Deciding afterward whether a stop answered a genuine boundary violation, an invalid token, or noise on a safety line requires that per-tick record to survive power cycles and scheduling stalls.

14.7 Trustworthy Operational Logs

A record supporting an argument months later has requirements that a debug log lacks. A debug log may be buffered in volatile user space and dropped under backpressure. If an asynchronous disk write drops three lines of trace data during an operating system stall, an engineer loses minor diagnostic context while the application continues running. Missing records weaken reconstruction across the causal boundary established in section 1.2. When the arm strikes the tote rack, establishing whether the learned policy commanded an illegal torque, whether the enforcer failed to clamp it, or whether an operator intervened too late requires a bounded, integrity-checked record plus independent plant observations.⁵ A forensic log therefore does not exist to assist a developer with line-by-line code tracing. Its purpose is to preserve what was sampled, proposed, admitted, and dispatched. Motor current, encoder motion, and contact-force sensors provide separate physical observations; commands alone do not prove delivered force or collision cause.

The chosen 1 kHz control design records a structured authority and action tuple each tick, with declared loss handling. The minimal **evidence tuple** takes the form $(t_{\text{MCU}}, S_{\text{auth}}, \mathbf{x}_{\text{state}}, \mathbf{u}_{\text{prop}}, \mathbf{u}_{\text{enf}}, \mathbf{u}_{\text{act}}, \text{ID}_{\text{operator}})$, which records both sides of the proposal boundary (section 1.5). In this structure, t_{MCU} is the event time in nanoseconds on the safety microcontroller's clock, converted from any other clock with a recorded error bound, S_{auth} records the active authority state indicating whether autonomous policy, manual override, or fault mitigation holds control, and $\text{ID}_{\text{operator}}$ identifies the authenticated source

⁵ | **Precision Time Protocol Logging:** Unlike legacy event data recorders logging at coarse 10 Hz intervals, physical AI incident reconstruction demands sub-millisecond precision. A qualified PTP installation can align the named device clocks with a measured conversion-error bound. Event ordering requires the intervals around timestamps, sensor exposure and sampling, and actuator response not to overlap.

of any active intervention. The three action vectors map onto the action taps of section 5.3, with $\mathbf{u}_{\text{prop}}$ as the requested action a_{req} and $\mathbf{u}_{\text{enf}}$ as the enforced command a_{enf} . The mapped command a_{cmd} lies between them and needs no field of its own when the proposal already arrives in actuator coordinates. The field $\mathbf{u}_{\text{act}}$ is the command dispatched to the drives, and the measured response a_{meas} comes from the plant observations in $\mathbf{x}_{\text{state}}$. Omitting one element can limit a particular causal question, so the record declares what each field actually measures. If the log records $\mathbf{u}_{\text{act}}$ without $\mathbf{u}_{\text{prop}}$, investigators lose the policy-command comparison. If $\mathbf{u}_{\text{enf}}$ is omitted, the log loses the proposed-versus-admitted comparison. Chained with integrity and clock-conversion metadata, these tuples form the forensic authority record that section 14.8 specifies.

Ordering these tuples requires physical timestamp integrity rather than software clock queries. Software timestamps derived from operating system calls inherit scheduling jitter, context-switch delays, and variable interrupt servicing latencies that can exceed 5 ms on loaded application processors. Over a 5 ms interval, an internal control loop running at 1 kHz executes five complete control cycles. If an operating system logs the arrival of a manual intervention command with a 5 ms delay, the record places the operator's command after the actuator response, inverting physical cause and effect. A hardware timestamp records a defined electrical event, such as frame ingress or a protective-input edge, after clock conversion with bounded error. Investigators can order command and detected contact only when their timestamp uncertainty, sampling windows, and sensor delays do not overlap.

Persisting full sensor streams and high-rate telemetry continuously to non-volatile storage exceeds both the bandwidth and the endurance limits of embedded hardware. The four cameras of a mobile-manipulator teleoperation session stream raw frames at the ingestion rate that section 5.4 sizes, a rate only a Gen4 NVMe drive sustains, so a pre-trigger and a post-trigger segment of those frames far exceed any microcontroller's memory. Raw frames do not belong in an MCU SRAM ring. The 1 kHz control tuple is a small fraction of that stream and can use a bounded small-memory ring. If full raw frames are required, provision a separate DRAM or storage tier sized to the measured stream budget, or retain selected or compressed frames with a declared loss of detail. At a trigger, seal the pre-trigger segment and continue writing post-trigger samples into a different allocated segment; freezing the sole ring would lose the promised post-event evidence. Frame records retain their native capture times, while control tuples retain their 1 kHz issue and sample times. Background persistence must not block the enforcer.

The pre-trigger window must capture the moment of policy divergence rather than merely the moment of physical takeover. When a learned policy commands an erroneous trajectory, human operators and external monitoring processes exhibit reaction latencies ranging from hundreds of milliseconds to multiple seconds before recognizing the deviation and asserting authority. If the ring buffer only recorded data from the instant an operator pulled an override switch, the telemetry would capture the rescue while omitting the anomalous commands that made the rescue necessary. Capturing a declared pre-trigger interval lets investigators inspect policy proposals and states before intervention; whether a proposal was unsafe needs the contemporaneous model, state estimate, and physical evidence. Without the pre-divergence history, evaluation cannot separate a necessary rescue from a premature interruption, and the learner-visited states that corrective relabeling needs are gone.

Integrity and retention can use a protected hash chain and independently tested power-loss storage. Within the Nervous System, logging routines batch evidence tuples into fixed blocks corresponding to 100 ms of execution. Each block is hashed using SHA-256, and its hash is concatenated with the hash of the preceding block to form an append-only cryptographic chain. With protected key and anchor custody, changing a committed block is detectable against the retained chain; loss before commit and compromised keys remain separate failure modes. One implementation periodically anchors root hashes in protected nonvolatile storage. A hash chain supplies neither persistence nor complete event capture, so the safety case tests key custody, which committed blocks survive power loss, how interrupted writes are marked, and where intervals are missing.

A retained, integrity-checked log can establish recorded authority and command values for its sampled intervals. It cannot say whether those values were permitted. That judgment needs the manifest the log is read against, which names who may hold each channel, under which conditions,

and within which deadlines, and it needs the log's entries to carry the fields the manifest's rules refer to.

14.8 The Authority Log

An investigator who asks, weeks after the blocked-aisle takeover, who was driving the base as it neared the rack end has only what the machine recorded. The authority record consumes two records from Part III. From the enforcement record of section 12.11 it takes the rungs of the fallback ladder, so every fallback it logs names the rung the permission path executed. From the placement record of section 13.8 it takes the measured timing of the operator-input path. It adds two parts of its own. The static authority manifest, the capability matrix of section 14.2 in machine-checkable form, binds every actuator channel to its authorized actors, operational preconditions, preemption tiers, and transition deadlines, and it is loaded and verified before the actuator drivers energize. The per-tick **forensic authority record** is a separate stream that records, for every tick, who held authority and what was proposed, admitted, and delivered, so that a tick without an entry reads as a gap, never as an implied pass; neither part substitutes for the other.

The static authority manifest specifies every permitted interaction as a hard contract between the physical platform and its potential operators. Each entry identifies a distinct physical authority domain through a hardware actuator mask, an authorized entity class (such as a local physical operator, a remote teleoperation console, an autonomous trajectory policy, or an emergency stop loop), and an integer **preemption tier**. An entity presenting a command vector must prove its authority through cryptographic role credentials verified by the permission path before its input can enter the actuation pipeline. A network operator's command arrives as an OPERATOR payload under the proposal header of section 4.3, carrying the operator's role, the requested channel mask, the command, a nonce, and a keyed tag. Alongside the credential, the entry specifies the mutual exclusion policy for the channel, the command rate ceiling r (how rapidly a requested command may change), the active blending duration τ_{blend} enforcing a C^2 bumpless authority transfer ramp to maintain the kinodynamic feasibility bounds established in section 11.4, and the terminal fallback state that the hardware enters if an active session is severed.

Each per-tick entry of the forensic authority record carries the event time on the safety microcontroller's clock, the actuator channels the decision applies to, the authority state (AUTO, PEND, BLEND, MAN, FALLBACK with its ladder rung, or PROTECTIVE, codes for the six states of section 14.3), the authenticated operator role, the cause of any preemption, the blend factor of the enforcer-owned profile, the proposed, admitted, and delivered commands, the sampled plant state, and a keyed link to the preceding block of the log. The versioned static manifest supplies vector units, field order, the clock mapping and its error bound, credential rules, and hardware access controls.

Mapping these entries to the mobile manipulator requires bounding four timing parameters against its limits: the operator reaction budget (the takeover latency T_{Σ} of equation 14.1 for the takeover mode in use), the **handshake protocol latency** $T_{\text{handshake}}$, the manual lease $T_{\text{lease,man}}$, and the minimum blending window τ_{blend} . Table 14.4 states them for the machine's two command channels, the base and the arm, which one remote-assistant session commands together.

Byte layout: The forensic authority record's fields, types, and offsets are given in section C.14.6.

Table 14.4: Blending windows on the mobile manipulator: One remote-assistant session commands both channels, and each window is rounded up to whole 1 ms permission ticks. The rate ceiling r bounds the derivative of each *command*. The base row is sized at the out-of-loop crawl, where its blend fits the stopping reserve; at the in-loop takeover speed no blended base command fits, and the base receives the resident stop. Every lease and pending deadline must fit the state-dependent stopping reserve.

Machine channel	Command change and chosen rate ceiling	Smoothstep window $\tau_{\text{blend}} \geq 1.875 \Delta u /r$, whole ticks	Human / protocol time	Fallback contract
Base acceleration command	Crawl at 0.3 m/s to braking at 2 m/s ² ; $r = 35.56 \text{ m/s}^3$, the resident stop's peak jerk at the crawl	106 ms	2 s / 31 ms	100 ms manual lease; 500 ms pending ceiling shortened by clearance
Arm joint torque command	15 N·m; torque rate $r = 750 \text{ N·m/s}$	38 ms	150 ms / 31 ms	The same manual lease; its loss invokes the arm's validated stop

Consider the remote assistant's in-loop takeover of the mobile manipulator, requested after the machine has slowed to 1.2 m/s and committed under the assistant's `manual_drive` entry. The operator reaction budget T_{Σ} sets the maximum duration the permission path permits between issuing a takeover demand and detecting the assistant's first command; here it is the 150 ms in-loop takeover of section 14.2, which already includes transport. The handshake latency $T_{\text{handshake}}$ defines the round-trip deadline for mutual authentication and state acknowledgment over the bus, budgeted at 31 ms per the four-phase protocol in table 14.3; it is part of T_{Σ} , not an addition to it. The manual lease $T_{\text{lease,man}}$ of section 14.3 sets the longest silence tolerated from the assistant before the permission path declares a link failure and reverts to the fallback trajectory, here 100 ms. Finally, the blending window τ_{blend} governs the rate of authority transfer to prevent mechanical shock, and table 14.4 gives it for each channel. The base's window depends on its speed, because the resident stop's peak jerk, which caps the blend under the smoothstep bound of section 14.4, rises as speed falls; the table therefore sizes it at the 0.3 m/s crawl.

The in-loop takeover leaves 29.7 mm of the rack-end clear distance at 1.2 m/s, the 24.7 ms by which it beats T_{latest} (section 14.3). That remainder bounds the transfer itself. Any further delay before a stop could begin while authority is changing hands must fit inside the remainder, and the enforcer checks the resident stop against it at every tick of the transfer. No base blend passes that check at this speed. Blending the step to braking from 1.2 m/s grows the travel plus the resident stop from the blended state by up to 197.6 mm, far past the remainder, so in this session the base receives the resident stop and only the arm blends. After a 2 s out-of-loop takeover at the crawl, 236.2 mm remains, and the crawl's blend uses at most 12.3 mm of it. Once the assistant holds MAN, the takeover delay is spent and the enforcer re-evaluates the budget each tick with the manual lease in the place of the 60 ms chunk lease in τ_{delay} . The 100 ms lease adds 48 mm of travel at this speed, which the 209.7 mm spare covers.

Verification draws its test inputs from the static manifest and its evidence from the event schema. The manifest's `authority_claims` field hands section 15.3 four authority rules by identifier, each of which becomes a hardware-in-the-loop fault case on the machine:

- **A1.** One enforcer path writes each actuator channel, even when a second bus master attempts to write a base-drive channel while the policy and the remote assistant both command.
- **A2.** No source bypasses an active higher-tier source or the independent permission check. A policy proposal during MAN is ignored, and an assistant command that opposes the enforcer's admitted limit is still checked.
- **A3.** An unacknowledged handover falls back by T_{timeout} of section 14.3, the earlier of the configured ceiling and the clearance-derived deadline of equation 14.2, which is 174.7 ms at the 1.2 m/s slow-down before an in-loop takeover and 78.9 ms for a handover the policy requests at the 1.3 m/s aisle speed.
- **A4.** A replayed `veto_path`, `manual_drive`, or `accept_item` packet, or one with a bad authentication tag, locks out the affected channel, leaves authority unchanged, and records the event.

When assembling the safety case for operational certification, chapter 16 evaluates the authority record as the evidence for claim C6, the claim that authority transfers as these four rules specify. A safety assessment examines the stated authority constraints, implementation evidence, fault coverage, and tested operating envelope. The certification package pairs the static authority manifest with the plant measurements that section 14.4 requires of every transfer and with execution traces from the fault-injection campaign that show any gaps in the audit trail. The plant measurements are needed because a handover that is correct at the bus can still slide the base or shift the tote rack. The measured preemption latency in those traces is a percentile of the tested distribution rather than a bound (section 4.4).

14.9 Fallacies and Pitfalls

Changing an authority register does not complete a safe handover. The machine must preserve its stopping margin and enforcement functions through the transfer, while the resulting logs must distinguish intervention from expert recovery.

Downstream: Forensic authority records provide evidentiary artifacts for the deployment safety cases in section 16.3.

Pitfall: *Treating human takeover timeout as a static time delay rather than a dynamic spatial stopping budget.*

The mobile manipulator requests a takeover at its 1.3 m/s aisle speed under a configured 500 ms acknowledgment window. Waiting out the window before braking covers 0.65 m, and with the rest of the finished stopping budget the stop ends 0.55 m past the 1.10 m rack-end clear distance. The same window fits at the 0.3 m/s crawl, where the budget leaves 2.8 s before fallback must begin. A window that is safe in one state overruns in another, so handover timing must follow current speed, clearance, and credible deceleration, with autonomous fallback beginning while the stop still fits.

Fallacy: *Human manual takeover should bypass automated safety barriers and grant unconstrained actuator authority.*

The remote assistant takes over the mobile manipulator's base to route it around a pallet, and the chosen path leads toward a rack upright. Switching to human control only changes who is sending the command; it does not change the physical realities of momentum, friction, or nearby obstacles. Within this architecture, the enforcer must continue checking the manual proposal against the admissible motion set and constrain it when a kinodynamically feasible alternative exists (building on the feasibility principles from section 11.4). If the request cannot be admitted, the defined autonomous fallback still applies. Takeover lets the operator guide the task planning; it does not make force, clearance, and stopping limits optional at the joint actuator boundary.

Pitfall: *Relying on cryptographic authentication without rate limiting the traffic that reaches the verifier.*

The mobile manipulator's safety microcontroller checks each remote takeover packet with the fast verifier of section 14.5, at 15 ms per signature. Verification rejects a forged packet only after spending that time on it. If a faulty or compromised node on the link sends `manual_drive` packets that fail verification faster than the verifier retires them, as few as 5 of them queued ahead of the remote assistant's genuine `veto_path` delay its admission to 90 ms, past the 78.9 ms fallback deadline T_{latest} at the aisle speed. The independently wired protective input does not wait in that queue, but every network command does. Authentication preserves command integrity while leaving an availability hazard. Mailbox filtering and per-source rate limiting must bound the traffic before it reaches the verifier, and qualification must test both command rejection and enforcement timing under the admitted load.

Fallacy: *Training directly on raw human intervention logs captures expert demonstrations.*

A learning pipeline treats every remote-assistant takeover of the mobile manipulator as an expert demonstration and trains on the preceding action window. The logs combine genuine policy failures, cautious early interventions, and operator kinematic mistakes, while the control transition itself often includes jerky physical corrections, delayed neuromuscular responses, and hesitant corrections as the human orients to the scene. Cloning those segments can reproduce the unstable behavior that made recovery difficult rather than the recovery the team wants to teach. Dataset curation must identify why intervention occurred, who held the authority register, and which commands actually restored acceptable operation. Reviewed recovery trajectories can supply useful demonstrations, but neither an intervention timestamp nor full manual authority alone establishes that the recorded behavior is suitable for imitation.

14.10 Summary

Human authority over an embodied physical AI system needs an engineered real-time contract that arbitrates takeover, revokes autonomous policy authority at commit, and selects a validated fallback within the available physical budget. Supervisory intervention governs the transition between proposers while the independent enforcer checks the admitted command and executes the selected stop when a lease lapses. Network commands reach the enforcer only after capability and authentication checks, whose own cost makes rate limiting ahead of the verifier part of the design, while trusted protective wiring bypasses the network entirely. On the mobile manipulator at its 1.3 m/s aisle speed, the finished stopping budget leaves 102.6 mm, which buys 78.9 ms of waiting before the machine must dispatch its own stop. The machine therefore slows to 1.2 m/s before it requests an in-loop takeover, and to a 0.3 m/s crawl or a standstill before it requests an out-of-loop one.

Transferring control from an active policy takes time, because replacing an admitted policy setpoint with a different human target in one 1 ms cycle requests a torque step. The C^2 bumpless authority transfer ramp instead varies an authority parameter $\alpha(t)$ smoothly from zero to one between latched command endpoints. Its sampled implementation must respect command-rate limits and the stopping reserve of section 12.5.

On the mobile manipulator the enforcer blends the base only from the 0.3 m/s crawl, over 106 ms. At the 1.2 m/s in-loop takeover speed no blended base command fits the clearance that remains, so the base receives the resident stop while the arm blends over 38 ms. The enforcer admits each blended tick only with a validated stopping reserve and monitors the plant response.

The authority record makes each intervention reviewable. Its static manifest binds every actuator channel to its actors, tiers, and deadlines, and its per-tick entries log who held authority and what was proposed, admitted, and delivered, naming fallback rungs from the enforcement record and operator-path timing from the placement record. A logged commit marks the recorded boundary where autonomous policy authority was revoked, subject to the logger's clock and sampling limits, and hash chaining with protected keys can reveal later modification. Command and authority records support reconstruction only when paired with independent sensor, current, and contact evidence and their timing uncertainty.

For model training, the pre-commit interval is a **policy-owned** segment even when a human is deciding to intervene. Retain its learner-visited states, unsafe proposals, and authority flags as failure evidence and candidates for expert relabeling; exclude unsafe policy commands from expert imitation targets. The post-commit blend is enforcer-owned transition data. Only reviewed post-transfer human recovery states and actions that meet the task's validity criteria should become demonstrations.



Key Takeaways: Supervisory intervention and bounded authority transfer

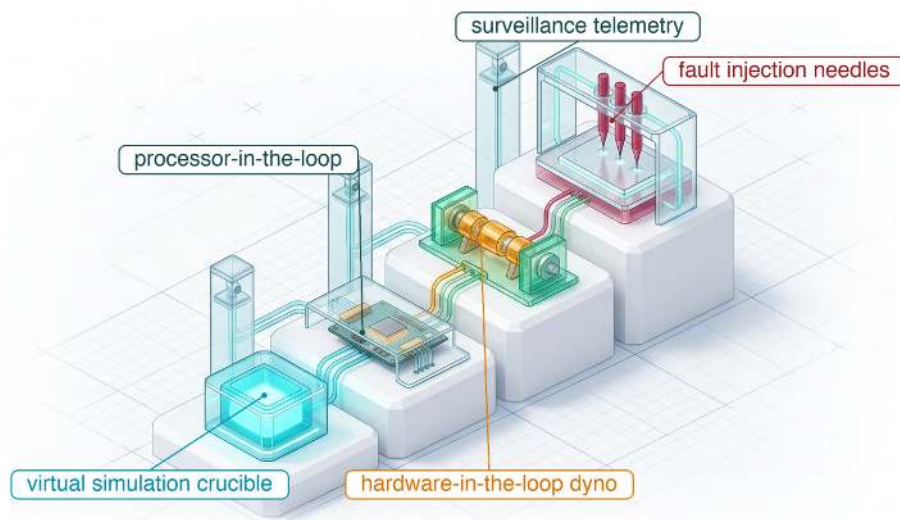
- **Clearance governs waiting:** At the aisle speed the machine's spare margin buys 78.9 ms; at the 1.2 m/s slow-down it buys the 174.7 ms that governs every in-loop takeover. A 2 s out-of-loop takeover needs a speed below 0.40 m/s. Dispatch fallback by the earlier of the configuration ceiling and the state-dependent stopping deadline.
- **One actuator writer:** Commit revokes the policy's token on the committed channels and latches its last admitted command. The enforcer owns BLEND and checks human proposals against the same physical limits.
- **Smooth commands cost stopping distance:** A fixed-endpoint C^2 acceleration profile bounds commanded jerk. Its extra stopping distance must be reserved.
- **Distinct stop paths:** Trusted protective wiring invokes a risk-assessed plant response. Invalid network packets are rejected; lost valid renewal invokes the lease fallback.
- **Causal intervention data:** The logged commit separates the policy-owned states before takeover, which corrective relabeling needs (section 5.6), from the human recovery after it. Use only reviewed post-transfer recovery actions as imitation targets.
- **Authority becomes evidence:** The per-tick record counts a missing entry as a gap, never a pass. The manifest's four authority rules become the fault cases verification injects, and their passing records, with the authority record, are the evidence for release claim C6.

A person's authority over the machine is granted on evidence and expires with it, as principle 4 requires of every authority the machine honors. The credential and the committed handshake are the evidence that grants it; the manual lease is that evidence's age limit, and when the operator's input stops renewing it, the authority lapses to a fallback the permission path owns. A preemption tier does not survive the lapse, and the remote assistant who held the base before it holds no authority after it. Returning control to any proposer waits on the reset conditions the state machine specifies, and on fresh evidence.

**What's Next: From authority revocation to physical verification**

Does a clean authority log show that the authority rules hold? It does not, because no logged handover exercised the duplicate writer, the replayed packet, or the unacknowledged request that the rules exist to contain. Chapter 15 turns A1–A4 into fault cases beside the rack-end faults, injects each with the loaded base approaching the rack end, and times each response with instruments independent of the machine's own log, so that a handover fallback counts from its start at the drive rather than from the arbiter's record of it.

Adversarial Verification



- 15.1 Why Operation Is Not Evidence
- 15.2 The Operating Envelope
- 15.3 Deriving the Fault List
- 15.4 The Qualification Ladder
- 15.5 Fault Oracles and Acceptance
- 15.6 Fault Injection on Hardware
- 15.7 The Fault Manifest
- 15.8 Fallacies and Pitfalls
- 15.9 Summary

Purpose

Why is a robot that has operated for weeks without a single crash not evidence that the machine is safe?

Weeks of incident-free operation establish behavior solely under encountered benign conditions; they provide no empirical evidence that the machine can survive edge-case anomalies. An autonomous robot appears flawlessly reliable until an unmodeled disturbance—a frozen camera frame, an overdue trajectory chunk, or an inductive supply-rail droop—challenges unverified software assumptions. Waiting for rare corner cases in passive operation is unviable: a tenfold rarer event requires ten times the operating exposure to witness, and encountering it during deployment risks catastrophic physical damage.

Whole-system verification transforms safety assumptions into systematic, repeatable fault injection experiments executed across simulation, processor-in-the-loop testbenches, and physical dynamometers. Each experiment challenges timing, data freshness, and kinetic authority boundaries, verifying whether the machine detects faults and enforces safe fallbacks within declared budgets. Within the governance envelope, rigorous verification binds the Brain's cognitive models, the Nervous System's deterministic monitors, and the Body's physical dynamics into empirical evidence capable of justifying, restricting, or denying deployment.

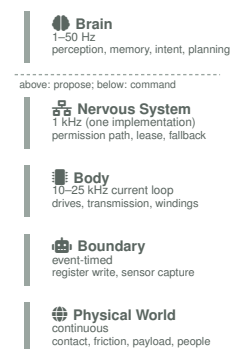


Figure 15.1.: The five-level machine stack within the governance envelope.

Prerequisite: Physical failure modes and measurement limits providing verification targets originate in section 2.7.



Learning Objectives

- Define an operating envelope and derive its pass conditions from the machine's stopping budget
- Derive a fault list from design records and forward physical analysis, including authority and compound faults
- Evaluate which simulation and hardware tests support each claim and where their evidence ends
- Apply zero-miss sizing to a declared, weighted fault population and state what it leaves uncovered
- Design fault-injection tests that challenge timing, data validity, and authority claims on physical hardware
- Construct fault records that keep failed, invalid, and unobservable trials distinct from passes

15.1 Why Operation Is Not Evidence

An incident-free operating log answers only the conditions it encountered. It does not exercise a frozen camera frame, a brownout, or a bus occupied by DMA traffic. Waiting for those conditions is slow, because a tenfold rarer event takes on average ten times the exposure to observe (the exposure wall of section 7.5), and on a machine it is also unsafe, because when the event finally occurs in operation it arrives as motion that no software action can recall. Whole-system verification therefore turns each safety claim into a deliberate fault, injected under containment and scored against a test oracle and limits declared before the run. The claims come from the records the earlier chapters produced, ending with the authority record of section 14.8, which names who may command each actuator channel and what must happen when a handover goes unanswered.

Consider the warehouse mobile manipulator running its aisle at 1.3 m/s (chosen; see the Reader Guide) toward a rack end where a person may step out. Weeks of clean runs at that speed say little about four faults the design claims to contain: a proposer that renews its lease and then stalls (F1), an observation that reaches the permission path with a stale or corrupt capture time (F2), a burst of lost EtherCAT frames between the permission path and the drives (F3), and a sag of the 24 V control rail when an inference burst coincides with the drive motors (F4). None of them shows in an operating log, because each leaves the machine running normally until the moment a stop is needed. The lease catches a stall, the evidence-epoch check a stale capture time, the drive's fieldbus watchdog a lost frame, and the permission path's undervoltage monitor, on its own held-up rail, a sag of the shared one. Each fault is injected at the rack end, together with the four authority faults that section 14.8 hands over (A1–A4), and every stopping trial is scored against the onset and distance bounds that section 15.2 derives from the stopping budget.

An injected fault turns a physical run into a test result that can support a safety case only when the run carries five ingredients:

1. **A bounded claim:** stating the exact invariant the machine must maintain (for the base at the rack end, a stop that ends inside the 1.10 m clear distance);
2. **Defined test conditions:** specifying the initial speed, payload, floor friction, ambient temperature, and supply-voltage range;
3. **A precommitted oracle:** an automated evaluator that inspects recorded telemetry against the claim without human intervention or post-hoc adjustments;
4. **A stated sampling and injection procedure:** deliberately driving the machine toward its performance limits and injecting faults into sensors, buses, and actuators; and
5. **Provenance:** the complete metadata record of model weights, calibration tables, sensor serial numbers, and raw bus logs permitting independent reproduction.

The base's stopping claim and the arm's latch-force claim require separate envelopes, injection points, and pass conditions; this chapter follows the stopping claim. The first decision is where nominal regulation must yield to an independent containment response.

15.2 The Operating Envelope

The **operating envelope** is the target ODD (section 7.2) extended inward to the machine's own conditions: compute load, supply voltage, bus jitter, thermal state, and operator response time. The sets nest: the ODD contains the declared ODD, which contains the target ODD, and the operating envelope pairs the target ODD with the machine's internal conditions. Defining this envelope requires precise numerical boundaries for an automated test framework to drive the system deterministically across each constraint. For the mobile manipulator, the envelope is a set of declared points rather than one nominal condition: the 1.3 m/s aisle speed; payloads of zero and the 50 kg rack capacity; floor friction of at least 0.204, the value the 2 m/s² credible deceleration needs, on dry aisles, and of at least 0.12 on floors under the site's inspection regime; and ambient temperatures from 5 °C to 35 °C. For inspected floors the envelope declares a restricted point at 1 m/s, the speed of the restricted configuration that the enforcement record declares valid down to that friction (section 12.11), and trials there use a test floor whose measured friction sits at the 0.12 qualification value, so that the stop they exercise is the one friction bounds.

Each point is paired with the machine's internal conditions: application-processor load up to its peak inference burst, the control rail anywhere from its 21.5 V battery-low point to its 24 V nominal, EtherCAT jitter, and the safety microcontroller's thermal state. One further point is the 1.2 m/s to which the machine slows before it requests a remote assistant's in-loop takeover, and only that point carries the 150 ms takeover time (section 14.3).

An operating envelope is distinct from the larger **survival envelope** that encloses it. Inside the operating envelope the learned policy and the tracking controllers regulate the process. When a disturbance, a sensor failure, or a compute stall pushes the machine across that boundary, nominal regulation is conceded, and the objective becomes bringing the body to a bounded, safe state before structural limits are breached. For the mobile manipulator, the survival envelope is the broader region of speed, floor friction, and machine state within which the fallback ladder's stop rung, and its inhibit rung as the last resort (section 12.9), can still bring the base to rest short of the rack end and keep the arm's contact force below its limit. Beyond it lies structural failure, where stored energy exceeds what the hardware can dissipate.

Defining the boundary of the operating envelope requires deriving limits from first principles rather than asserting convenient software timeouts. At the rack end the limit is the stopping budget. Stopping distance grows linearly with the pre-brake delay and quadratically with speed (equation 2.1), and the finished budget of section 12.5 (table 12.1), with its 133.6 ms pre-brake delay, the smooth resident stop that section 11.5 commits to, and the tracking inset, sets a ceiling of 1.39 m/s on a dry floor. At the chosen 1.3 m/s the machine needs 997.4 mm of the 1.10 m clear distance and keeps 102.6 mm in reserve, a distance it covers in 78.9 ms.

Two oracle bounds follow, and every stopping trial in this chapter is scored against both. Brake onset must come no later than 82 ms after the last valid renewal, the lease path of section 4.3: the 60 ms lease, one 1 ms tick of the permission loop, one 1 ms bus cycle, and 20 ms of brake onset. The rail faults of F4 take their onset from the undervoltage flag instead. The stop, measured by instruments independent of the machine, must end within the distance allowed from the same event the fault is referenced to. For a fault that follows a target's appearance, that distance is the 997.4 mm the budget allows. A stalled proposer offers no target appearance, so its stop, like its onset, is measured from the last valid renewal, and the travel during the 82 ms before brake onset, the resident stop, and the tracking inset give 780.4 mm at the aisle speed.

Neither bound depends on the proposer. A chunk policy that stalls cannot delay brake onset, because the stop begins when the lease expires rather than when the proposer notices, so a test of F1 measures the permission path and not the policy. The budget protects a static obstacle at the clear distance, and a trial against a static target says nothing about a person still walking toward the machine, a premise the site's crossing rule carries (section 9.6).

A flow regulator that keeps a buffer tank below its overflow weir, the Class 3 contrast for this part, verifies a different quantity. Its oracle is volume, not distance. With the level at an 88 L trip point, 7 L below a 95 L weir, and a net inflow of 1.40 L each second after an upstream valve fails and a discharge pump trips, the tank overflows in 5 s. A protective path that detects in 210 ms and seats its isolation valve in 1.20 s finishes 3.59 s early. The same 150 ms bus delay that would cost the mobile

manipulator 195 mm, 1.9 times its reserve, adds 0.21 L to the tank. The regulator's fault list therefore injects sustained delays and failed valves, while the machine's must resolve tens of milliseconds.

On the mobile manipulator, separate minimum and maximum values for individual variables fail to capture the true operating boundary, because physical and computational dimensions are coupled. The 1.3 m/s aisle speed and an inspected floor with friction 0.12 each lie inside their declared ranges, yet together they fall outside the envelope. On that floor friction, not the brake, sets the credible deceleration, which drops to 1.18 m/s². The resident stop lengthens, and the budget at 1.3 m/s grows to 1.44 m, past the 1.10 m clear distance; the speed boundary on that floor is 1.09 m/s, which is why the envelope's restricted point sits below it. Payload couples less visibly. It enters no term of the budget, but the credible deceleration was declared for the loaded machine, so a campaign run only unloaded never tests the value the budget rests on. The true operating boundary is a surface in state, parameter, and latency space, and a valid verification suite must probe that combined surface rather than isolated scalar limits.

A boundary stays honest only if every parameter carries its unit, its provenance, its calibrated uncertainty, and an engineer accountable for it. A credible deceleration of 2 m/s² is incomplete until the record says whether it came from stopping trials at the loaded mass, a drive datasheet, or a friction model, and with what resolution the external distance reference measured each stop. A new wheel tread compound or new EtherCAT slave firmware obliges that owner to re-evaluate whether the boundary has moved.

Testing the envelope means attacking its boundary surface. Interior points, where the base crawls unloaded across a dry floor on fresh camera frames, show only that the machine works under ideal conditions. The test framework places points on, just inside, and just outside each consequential boundary. Points just inside check that the policy keeps closed-loop stability under the largest permitted stress; points on the boundary check that margins survive nominal sensor noise; points just outside check that the enforcer takes authority from the policy predictably and halts the base within the survival envelope before the clear distance is gone.

Probing this boundary surface formalizes the critical distinction between classical **Functional Safety** (governed by ISO 26262 and IEC 61508) and the **Safety of the Intended Functionality (SOTIF)** (governed by ISO 21448). Classical functional safety frameworks assume that operational hazards arise from *system faults*: broken wiring harnesses, electrical shorts, bit flips in memory, or software syntax bugs. If every hardware and software component operates strictly according to its engineering specification, classical functional safety considers the system non-hazardous. In physical AI systems, this premise fails. A vision transformer or learned depth estimator can execute with zero memory faults, zero numerical exceptions, and bit-exact instruction execution, yet fail to detect an obstacle due to unmodeled sunlight glare, retroreflective surfaces, or novel semantic objects outside its training distribution. This hazard arises from **functional insufficiency**—a fundamental performance limitation of learned perception in open environments, occurring in the total absence of an electrical or software fault. Verification of the operating envelope under SOTIF requires systematically identifying these unknown-unsafe conditions through adversarial boundary search, mapping the envelope within which perception is statistically valid, and demonstrating that the deterministic enforcer catches functional insufficiencies before physical clearance is breached. Yet while SOTIF verifies that the system responds safely to sensory ambiguity in the world, the enforcer's ultimate ability to arrest physical momentum still rests on the deterministic integrity of the underlying hardware under electrical and mechanical stress.

🔍 Systems Perspective 15.1: The hardware falsification principle

If a safety property cannot be deterministically falsified by injecting a physical or electrical fault on target hardware, the property is an unverified assumption rather than an engineering invariant.

Once every constraint, latency budget, and environmental limit is stated as a number, the design claims of the preceding chapters become targets for fault injection, each with a threshold that must trigger protective intervention.

15.3 Deriving the Fault List

The first source of faults is the design records, inverted. Chapter 5 through chapter 7 state the learned policy's properties: its training distribution, its state-action assumptions, and its inference latency ceilings. Chapter 8 through chapter 13 state interface and execution properties such as bus allocations, observation freshness, and estimation bounds, and chapter 14 states how quickly authority can leave a failing policy and who holds the override. If the machine's records state that the chunk policy renews its proposal every 50 ms at every speed up to the 1.5 m/s drive limit, the verification suite inverts that claim by injecting stalled renewals, delayed observations, and out-of-range encoder readings to test whether the permission path detects the violation within its budget.

For a single fault, the bounded claim and the injection procedure of that list become a five-element chain comprising the negated condition, the physical injection point, the expected detector, the bounded response, and an observable pass condition, and a recorded design property becomes a testable fault only when all five are defined. For F2, the navigation camera stamps each frame at capture, and the frame's age at dispatch is budgeted at 51.6 ms (section 8.3). Converting that stamp to the safety microcontroller's clock adds the 0.2 ms bound of the observation contract (section 8.7), so the permission path refuses evidence whose upper age exceeds 51.8 ms. The negated property holds a frame at the image signal processor while preserving its capture stamp, so that it arrives past that threshold. The expected detector is the permission path's check of each proposal's evidence epoch, the required response is refusal of any proposal built on that evidence followed by the resident stop, and the observable pass condition is the pair of oracle bounds, brake onset within 82 ms of the last valid renewal and a measured stop within 997.4 mm. When any element of this chain cannot be defined or observed, the system contains a **verification gap**. A verification gap indicates that the claim cannot be falsified by test. Such gaps cannot be ignored; they require dedicated hardware instrumentation, a narrower operational claim, or a redesign of the execution path.

Inversion also reaches the learned proposer's assumptions, because distribution shift is a system fault even when every frame arrives intact. A chunk policy trained in simulation on dry concrete with friction 0.60 proposes decelerations that a floor at 0.12 turns into wheel slip. Inverting that evaluation claim means injecting the unmodeled traction. Sensor bias and data perturbations are injected for the same reason, because a policy with a high nominal reward can still command jittering actuators once its inputs are perturbed. The machine then fails on valid, uncorrupted commands, because software and mechanics stay coupled across the causal boundary (section 1.2).

At each inter-module contract, the fault list negates the property at its physical point of handoff: observation freshness, state validity, intent lifetime, planning deadlines, enforcement independence, and resource allocation. For the 60 ms chunk lease of section 4.3, the test environment injects an otherwise valid proposal 65 ms after its last renewal and confirms that the permission path rejects it as expired. For the enforcer's claimed independence from the policy computer, the test saturates the shared memory bus with DMA traffic and confirms that the enforcer still meets its 400 μ s deadline inside each 1 ms tick. The discipline cuts both ways. Bus starvation, clock skew, and supply fluctuation are injected where the design claims tolerance, because a fault drawn from a generic menu invents a requirement the machine was never specified to meet.

Inversion has a structural limit. It produces only the faults the design team already modeled, so the list is only as complete as the design claims. If no record says how the base's spring-applied brake behaves when its lining glazes after repeated stops, there is nothing to invert, and the failure never reaches the suite.

A second, independent source works forward from the physical machine, enumerating what the body, the power network, the floor, and the environment can do regardless of what the software believes. Wheels skid when a commanded deceleration exceeds what the floor can transmit; brake linings glaze under repeated stops; tires wear and change the rolling radius that odometry assumes; drive vibration loosens a camera mount until its calibration drifts. These hazards originate in mechanics and tribology, outside every abstraction the software reasons over. Leveson's **Systems-Theoretic Accident Model and Processes (STAMP)** framework¹ (Leveson 2011) brings them into hazard analysis.

For example, start with the physical hazard of a base that keeps driving after the permission path has commanded a stop at the rack end. The unsafe control action is continued drive torque after the

¹ | **STAMP/STPA Hazard Analysis:** Developed at MIT by Nancy Leveson, STAMP treats safety as a control problem rather than a component-reliability problem, so hazards appear as unsafe control actions at interfaces, such as a seam where bus latency or filter lag erodes a stability margin. STPA is the hazard-analysis method built on it.

stop command. The test injects a welded drive-enable contactor or a stuck brake-release solenoid on the test aisle, observes wheel speed and brake current independently of the commanded stop bit, and requires the permission path to remove drive power through its second channel, safe torque off. The precommitted pass condition is a measured stop within 997.4 mm across the declared speed, payload, and floor-friction points; an unmeasured wheel speed makes the trial unobservable for the braking claim. This forward chain yields an injection and oracle even if the original software requirements omitted a welded contactor.

Working forward exposes fault classes that no software record implies: clock loss, brownouts that leave logic levels between valid thresholds, drive-switching interference coupling into encoder lines, corrupted configuration memory, and reset lines that bridge domains meant to be independent. If the safety microcontroller and the application processor shared one power-management IC, an undervoltage transient during an inference burst could reset the microcontroller at the moment the proposer loses control. No software record contains that failure, because each module assumes its supply is ideal.

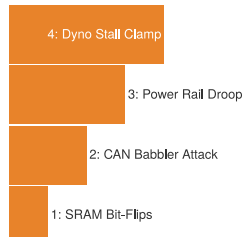
Forward analysis also tests the common-cause independence claims of chapter 4 and chapter 13. Two paths isolated on a diagram often share one physical vulnerability, as when the lidar and a redundant bumper switch are routed through one cable carrier, share an analog reference inside one converter, or are built on timer silicon with the same errata. A transient that latches up both channels at once defeats any argument that treats their failures as independent. **Common-cause analysis** traces routing, power paths, and clock trees so that no single physical event can disable both the policy and the intervention mechanism.

The physical source extends to operator error and foreseeable misuse (chapter 14). Floor staff jumper a rack-end presence sensor during maintenance, park a pallet jack across a marked crossing, or unplug a warning beacon to silence it, while remote assistants request speeds the aisle does not allow. When a remote assistant commands the 1.5 m/s drive limit in the aisle, the verification framework must verify that the command is denied by capability, or clamped under a `manual_drive` entry to the 1.3 m/s aisle speed, and that the permission check runs on it like any other proposal.

Working forward from the physical machine adds one representative mechanical class, the actuator jam, to the record-derived list in table 15.1. Completeness still requires a separate hazard-analysis argument for the specified plant and operating envelope.

Working forward from physical transducers also requires testing against **transduction-layer cyber-physical attacks**. In connected autonomous systems, physical AI introduces attack surfaces that bypass traditional network firewall abstractions by exploiting the physics of sensory transduction. Resonant acoustic injection (targeting the micro-mechanical proof masses of MEMS gyroscopes with ultrasonic frequencies) can induce false angular rates in inertial navigation without corrupting a single bus packet. Similarly, optical laser injection and rolling-shutter illumination strobes can saturate camera photodiodes to blind range estimators, while adversarial physical patches (high-contrast geometric patterns placed on warehouse walls or floor surfaces) can exploit the gradient landscape of neural perception backbones to cause false negative obstacle detections. Under **ISO/SAE 21434 (Road vehicles — Cybersecurity engineering)**, these threats are co-engineered alongside functional safety faults: a cyber-physical intrusion is evaluated by its ability to cross the causal boundary into hazardous actuation. The verification suite must confirm that the enforcer's cross-sensor plausibility checks (such as comparing wheel odometry against optical flow, or checking camera evidence epochs against IMU integration bounds) detect transduction anomalies and engage the fallback ladder before false perceptual claims can command torque.

Besides inverted claims and forward hazards, the fault list must test the authority cases A1–A4 of section 14.8, each a row of table 15.1, tested with the machine in the state that makes intervention urgent, the loaded base approaching the rack end. The handover deadline is tested at both speeds where section 14.3 sets it. Once the machine has slowed to 1.2 m/s for an in-loop request, the 209.7 mm reserve gives an unacknowledged handover 174.7 ms to fall back; at the 1.3 m/s aisle speed, for a handover the policy requests without slowing, the reserve's travel time shrinks to 78.9 ms. A handover tested with the base parked tests none of the deadlines that matter.



Fault injection climbs from software bit-flips to physical dyno shaft jamming.

Table 15.1: Representative hardware and cyber-physical fault classes: The rack-end faults F1–F4 and the four authority fault cases (A1–A4) that section 14.8 hands over, each mapped to its injection point, detector, response, and observable pass criterion on the warehouse mobile manipulator, plus one forward-derived mechanical fault.

Failure Domain & Fault Mode	Physical Mechanism & Injection Point	Target Manifestation & Causal Impact	Detection Primitive & Latency Budget	Enforced Safe Response & Bounded Recovery	Observable Pass Criterion
Proposer Stall (F1)	Chunk policy renews its lease once, then its task is killed or its output frozen on the application processor	No new proposals; the last admitted chunk keeps supplying setpoints while the base cruises	Lease expiry on the permission path (60 ms); host heartbeat timeout (30 ms)	Revoke proposal authority; run the resident stop	Brake onset ≤ 82 ms after the last valid renewal; travel from that renewal to rest ≤ 780.4 mm
Stale Observation Time (F2)	Navigation-camera frame held at the image signal processor with its capture stamp preserved, or the stamp corrupted, via inline frame interceptor	Proposals built on evidence whose upper age exceeds 51.8 ms, the budgeted age plus the clock-conversion bound; the base acts on an old view of the rack end	Evidence-epoch age check on every permission-loop tick	Refuse proposals built on the stale evidence; run the resident stop	Brake onset ≤ 82 ms after the last valid renewal; measured stop ≤ 997.4 mm
Lost EtherCAT Frames (F3)	Frames dropped or corrupted at a drive's slave port, injected via fieldbus disturbance node	Encoder state and drive commands miss cycles; the delivered command ages by one 1 ms cycle per lost frame	Working-counter and frame-arrival check; drive communication watchdog	Drive enters its configured stop; permission path commands the resident stop	Brake onset ≤ 82 ms after the last valid renewal; measured stop ≤ 997.4 mm; zero unhandled bus exceptions
Control-Rail Droop (F4a)	Programmable DC load reproduces the inference-burst-plus-drive transient on the shared 24 V control rail at its battery-low point	Shared rail sags to 17.3 V, below the 18 V regulator dropout; the application processor browns out and its sensors may freeze at plausible values	Shared-rail undervoltage flag, raised by the permission path on its isolated, held-up rail	Command STOP; run the resident stop	Brake onset ≤ 22 ms after the flag; measured stop ≤ 997.4 mm; permission path never resets (continuous heartbeat on an independent digitizer)
Permission-Rail Feed Loss (F4b)	Feed to the permission rail's hold-up store opened during the longest stop the enforcer can admit on an inspected floor, from its 1.09 m/s ceiling	Permission path, drive logic, and brake coils run on the store alone	Feed-loss flag on the permission rail	Complete the resident stop; set every spring brake at standstill	Stop completes and every brake sets before the rail reaches dropout (1497.35 ms of the 2 s hold-up); any reset is FAIL
Actuator Jam & Mechanical Stall	Wheel bearing seizure or dragging brake, injected via wheel clamp or dynamometer	Locked rotor; one wheel blocked yaws the base; rapid stator winding heating ($I^2 R$)	Motor phase overcurrent & Hall-effect velocity stall monitor, within its detection bound	De-energize the power stage; engage the spring-applied brake	Stator temperature $T_{\text{coil}} < 155^\circ\text{C}$ (Class F limit, section 2.5); base stays inside the 0.15 m side clearance
Authority Transfer (A1): Duplicate Writer	Second bus master writes a base-drive channel (duplicate EtherCAT identifier or DMA write) while policy and remote assistant both command	Two writers on one channel; the delivered command is no longer the admitted one	Bus or DMA write permissions; per-tick forensic authority record	Reject the foreign write; the enforcer path remains the only writer	Write rejected; every forensic authority record in the window shows one writer

Failure Domain & Fault Mode	Physical Mechanism & Injection Point	Target Manifestation & Causal Impact	Detection Primitive & Latency Budget	Enforced Safe Response & Bounded Recovery	Observable Pass Criterion
Authority Transfer (A2): Tier Bypass	Policy proposal injected while MAN is active; remote-assistant command that opposes the enforcer's admitted limit	A lower tier displaces an active higher tier, or an assistant command skips the permission check	Arbiter tier check; permission check applied to every source	Ignore the policy proposal; clamp or refuse the assistant command like any proposal	Arbiter ignores the proposal; the permission check still runs on the assistant's command
Authority Transfer (A3): Handover Timeout	Assistant input frozen during PEND; acknowledgment frames dropped, at the takeover speed and at the aisle speed	The machine waits on a handover that no one completes	Handover deadline timer in the arbiter	Start the validated fallback	Fallback starts by $T_{\text{timeout}} = \max(0, \min(T_{\text{config}}, T_{\text{latest}}))$: 174.7 ms at 1.2 m/s, 78.9 ms at 1.3 m/s
Authority Transfer (A4): Authentication	Replayed captured <code>veto_path</code> , <code>manual_drive</code> , or <code>accept_item</code> packet; packet carrying a bad tag	An unauthenticated source gains authority	Tag and freshness check on every assistant and coworker command	Lock out the channel; leave authority unchanged	Channel locked, authority unchanged, event recorded

Compound faults

Each row of table 15.1 injects one disturbance, yet a machine in operation rarely meets one at a time. Combining rows at random would multiply the list without finding the combinations that matter, so coupled disturbances are enumerated from shared causes instead. The test ledger of the placement record (section 13.8) names every resource that the permission path shares with the learned workloads, and each shared rail, clock, or bus becomes one compound fault, injected at the cause rather than at its symptoms. A droop on a shared rail can slow the enforcer, freeze a sensor at a plausible value, and reset the logger in the same interval. A burst on a shared bus ages observations while it delays the permission path's own reads and the heartbeat that would reveal the delay. A fault in a shared clock skews the timestamps of every record that crosses the proposal boundary at once, so freshness checks pass on data that is already old. The oracle for a compound fault therefore checks the protective response against its deadline and also checks that the evidence needed to judge that response survived the same cause; the pitfalls in section 15.8 show both failing together. A shared cause that the ledger does not name cannot be enumerated this way, which is why the forward analysis of the physical machine remains a separate source of faults.

The fault list now holds inverted design claims, forward-derived physical fault classes, the four authority cases, and compound faults drawn from named shared causes. It says what to inject but not where an injection is real, and that choice decides what each trial can prove.

15.4 The Qualification Ladder

A simulator can stall the proposer of F1 at every sampled speed and payload overnight, but it cannot sag the control rail of F4, because the simulated rail has no regulator to drop out and no microcontroller to reset. Each evaluation environment closes, breaks, or substitutes different links of the loop that runs from sensing through inference, buses, drives, and plant, so the environments do not form a hierarchy. A higher rung cannot subsume a lower one; each yields a distinct category of evidence and stays blind to the rest (figure 15.2).

**Definition 15.1: Four-stage qualification ladder**

Four-stage qualification ladder is the structured progression of cyber-physical evaluation environments that closes, breaks, or substitutes specific arcs in the causal loop to balance combinatorial test throughput against physical and temporal execution fidelity. Its four stages

DRAFT

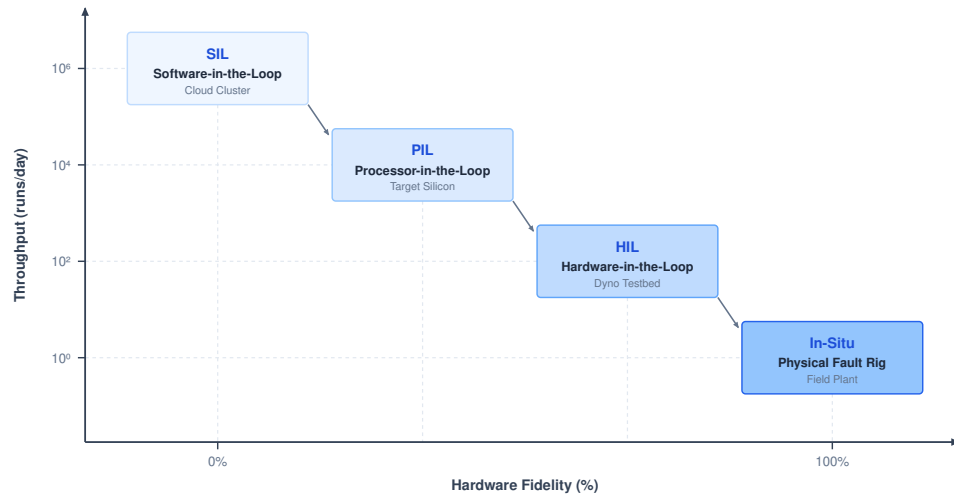


Figure 15.2: Four-stage qualification ladder: Four rungs trading throughput for fidelity, from roughly 10^6 software-only runs a day at zero hardware realism down to a handful of in-situ physical fault injections at full realism. The ladder is not a progression toward accuracy but a sequence of different blind spots, because each stage establishes evidence only within its own causal loop and stays blind to every mechanism outside it.

are Software-in-the-Loop (SIL), Processor-in-the-Loop (PIL), Hardware-in-the-Loop (HIL), and in-situ physical fault rigs.

1. **Significance:** No single test environment can validate a physical AI system. Software simulators allow massive combinatorial sweeps ($> 10^8$ cycles/day) but omit target silicon bus contention and material wear; physical rigs expose authentic mechanics and electrical transients but cannot execute destructive fault sweeps at scale.
2. **Distinction:** Unlike a cumulative testing staircase where higher rungs subsume lower ones, each qualification rung exercises complementary, non-overlapping causal mechanisms while remaining blind to others.
3. **Common pitfall:** Assuming that passing extensive Software-in-the-Loop (SIL) simulation implies safety on embedded hardware. SIL abstracts away microcontroller clock drift, DMA bus lockups, interrupt jitter, and power supply voltage sag.

Table 15.2 gives each rung's substrate, the faults it can inject, its throughput, and where its evidence ends. For the mobile manipulator, software-in-the-loop evidence ends at the plant model, where a brake modeled as a first-order lag has no lining temperature, tire slip, or torque limit. Processor-in-the-loop adds the cross-compiled binary on target silicon and so tests quantization and stack headroom; hardware-in-the-loop benches put the two-processor implementation of section 4.2, its fieldbus, supplies, and drive stages against a field-programmable gate array (FPGA) plant or a dynamometer; and physical fault rigs add the production base, brakes, harness, and floor. Two sources beside the ladder open the loop altogether. Trace replay keeps real sensor noise but supplies the frames that followed the original command rather than the new one. Shadow operation runs the candidate beside an incumbent controller, so if the candidate proposes a velocity that would carry the base past a rack end, its log records the command while the incumbent keeps the base where it belongs. Neither shows the physical consequence of a command.

A shadow deployment cannot let a stalled proposer drive the base toward a rack end that may hold a person, a simulator cannot reveal a race in transceiver silicon, and a hardware bench cannot run the millions of randomized runs that tail statistics need. Each fault injection therefore goes to the rung that exercises its causal mechanism.

Table 15.2: The four-stage qualification ladder across cyber-physical boundaries: The four qualification environments compared by computing substrate, exercised causal-loop elements, fault injection primitives, execution throughput, physical and temporal fidelity, and test oracle.

Qualification Rung & Substrate	Causal Loop Realism (Compute, Bus, Actuator, Plant)	Fault Injection Mechanism & Scope	Throughput & Sample Scaling	Physical & Temporal Fidelity	Primary Test Oracle & Evidentiary Limit
Software-in-the-Loop (SIL) (Host Workstation / Cloud Cluster)	Compute: Synthetic / Host x86; Bus: Idealized zero-copy buffer; Actuator: Linear / saturation model; Plant: Numerical ODE/PDE solver	Tensor bit-flips, synthetic sensor dropout, floor-friction, payload, and latency sweeps	10^3 – $10^5 \times$ real-time parallel sweeps ($> 10^8$ cycles/day)	High mathematical flexibility; zero target silicon, bus timing, or electrical realism	Oracle: Mathematical assertion corridors and state invariants. Limit: Cannot validate target execution timing or unmodeled physical dynamics.
Processor-in-the-Loop (PIL) (Target SoC / MCU Core Emulation)	Compute: Real target silicon (ARM/RISC-V); Bus: Simulated bus interconnect; Actuator: Emulated command register; Plant: Real-time host model	Instruction cache evictions, register corruption via JTAG/BIST, interrupt jitter injection	0.1 – $1.0 \times$ real-time; bound by cross-platform synchronization	Instruction-set and compiler accurate; abstracts analog bus lines and actuator back-EMF	Oracle: Target register state and WCET execution bounds. Limit: Ignores physical transceiver bus contention, voltage droop, and thermal rise.
Hardware-in-the-Loop (HIL) (Target ECUs + Real-Time Plant Rig)	Compute: Production ECU / NPU; Bus: Real physical CAN/Ethernet; Actuator: Real motor drives / dyno; Plant: Hard real-time FPGA plant rig	Pin break/short, CAN frame stuffing / babbling idiot, supply rail droop, load steps	$1.0 \times$ strict real-time (24 h/day wall-clock limit)	High electrical, timing, and protocol fidelity; bounded by FPGA plant model accuracy	Oracle: Protocol CRC, bus deadline monitors, and interlock timings. Limit: Cannot execute destructive physical overload or simulate tire and lining wear.
Physical Fault Rig (Dynamometer Cell / Field Plant)	Compute: Production ECU / NPU; Bus: Production harness & sensors; Actuator: Real physical mechanisms; Plant: Real base, arm, and floor	Wheel clamps, magnetic brake stall, thermal heating, transducer bias drift	$1.0 \times$ wall-clock; constrained by mechanical wear and thermal reset ($< 10^3$ cycles/day)	Full authentic physics (Coulomb friction, wheel slip, thermal dissipation, wear)	Oracle: Independent physical containment instrumentation and yield transducers. Limit: High financial cost per trial; destructive tests cannot scale to statistical tail bounds.

The fidelity boundary of a test environment is where a model, an emulated bus, or a replayed trace severs the exchange of energy and information between computation and mechanics (section 1.2). A mechanism the model omits cannot fail in it. A joint with unmodeled Coulomb breakaway friction and stator lag sits still for a dead time the simulator never shows and then trails the simulated trajectory, an instance of the omitted-mechanism class of section 6.5 that only a rung with the real joint can expose.

To prevent false confidence, every reported verification result must explicitly state which elements of the causal loop were physically real, which were simulated, which were replayed from static logs, which were bypassed, and which remained unobserved. A claim that the permission path stopped the base within 997.4 mm means nothing without specifying whether the wheel-speed signal was generated by a differential equation, read from a physical encoder over the real fieldbus, or replayed from a digital buffer. Specifying this causal mapping exposes the assumptions underlying each test, so that unmodeled physics cannot hide a latent hazard.

Each rung returns a trace, and the causal mapping says which links in that trace were real. Neither says whether the trial passed. The two rack-end bounds of section 15.2 have so far been numbers, and scoring a trial requires turning them into a predicate that is fixed before the run and applied without judgment afterward.

15.5 Fault Oracles and Acceptance

An injected fault always produces a reaction, in telemetry transients and in actuator motion, and sometimes a burnt component. Without an oracle fixed in advance, engineers judge time-series plots after the run from gross symptoms, such as whether Joule heating melted a phase wire or a controller threw an unhandled exception, and the test becomes an anecdote. An **oracle** is the predicate, defined in full before the run, that scores a trial against its claim.

Physical systems fail through transients rather than at isolated instants, so the oracle is a temporal specification. **Signal Temporal Logic (STL)** combines physical predicates, such as a stopping distance below a limit, with logical connectives and time windows, so that a condition must always hold over an interval, eventually happen within a deadline, or persist until another event occurs (syntax in section A.7).

STL also equips verification with **quantitative robustness semantics**. Instead of a simple pass/fail flag, robustness is a continuous numerical score measuring *how far* the system remained from violating its safety margins.²

The rack-end oracle has this form. Brake onset must eventually occur within 82 ms of the last valid renewal, and the base's travel must always stay within the distance allowed from the event the fault is referenced to, the 997.4 mm the budget allows at 1.3 m/s from the moment a target appears, or 780.4 mm from the last valid renewal for a stall. If an injected fault delays brake onset 30 ms past its bound, the base travels 39 mm farther before it begins to slow and stops after 1,036.4 mm, still short of the 1.10 m clear distance. The distance predicate's robustness is nonetheless negative by 39 mm, and the onset predicate fails by 30 ms. Each score quantifies the failure in its own unit without manual inspection.

Robustness turns verification from random testing into **temporal logic falsification as adversarial optimization**. Uniform sampling of a high-dimensional state space needs an intractable number of trials to reach a rare failure, so a falsification engine searches instead for the disturbances, such as sensor noise, voltage drops, and friction, that drive the robustness score lowest. It uses black-box optimizers such as evolution strategies or Bayesian optimization, or gradients through a differentiable simulator where one exists, and a negative score yields a concrete counterexample trajectory.

Formal verification of the network itself, such as the Reluplex solver (Katz et al. 2017), can prove that a small piecewise-linear network never emits an unsafe command over an admissible state set, but its cost grows exponentially with network depth and it does not reach closed-loop dynamics. Verification therefore combines such bounds with falsification, hardware fault injection, and runtime enforcement.

Scoring also needs an **invalid-test rule** for a run the environment failed to deliver. If a traction-control derate on a floor joint slows the base before the injection, so that it meets the fault below the declared 1.3 m/s, the trial did not test the envelope point it claims. If the frame injector overflows its own queue and drops the fault packet before it reaches the transceiver, the machine never met the disturbance. Scoring such a run as a pass manufactures confidence, and scoring it as a failure blames the machine for the fixture. The oracle therefore returns one of four verdicts. **PASS** means every precommitted predicate held; **FAIL** means a measured predicate was violated; **INVALID** means the injection or a precondition failed; and **UNOBSERVABLE** means the truth data needed to score the trial were lost, with the partial trace retained. Under either of these last two verdicts a field the trial did not measure is marked unavailable, never zero. Neither verdict ever counts as **PASS**, and neither is dropped from the coverage report.

Precommitment separates evidence from post-hoc storytelling. Fault-injection behavior is noisy and surprising, and criteria negotiated after the trace is seen drift to fit it. An overrun is explained away as floor variation, or an onset later than the 82 ms bound is excused because the base still stopped short of the rack end.

The precommitted record fixes the STL predicates, physical units and any normalization scales, response-time bounds, and invalidation triggers before the first injection. The evaluator retains a

² | **Quantitative Robustness Semantics:** Each predicate has a signed margin in its own physical unit: clearance may use meters, brake onset milliseconds, and contact force newtons. A negative margin falsifies that predicate. A minimum across unlike units is meaningless; a combined dimensionless STL score requires each margin to be divided by a predeclared positive scale of the same unit. The record retains raw margins and scales so a pass or failure remains physically interpretable.

signed margin for each predicate and returns PASS only if every required margin and timing bound passes. If the stop exceeds its bound by a few millimeters or brake onset misses its deadline by 2 ms, the record states which measured predicate failed; the trace and follow-up analysis determine whether floor friction, brake lag, derating, or a diagnostic delay caused it. Precommitment is where principle 4 first binds a test. Release will grant authority on these records, so a verdict scored by a rule revised after the trace was seen would grant authority on the strength of an outcome rather than a test.



Checkpoint 15.1: Temporal logic oracles and test precommitment

Before executing automated falsification against physical AI policies, verify your understanding of temporal logic specifications and test validity criteria:

- ☐ **Quantitative robustness interpretation:** Can you explain how Signal Temporal Logic replaces binary pass/fail assertions with a signed distance metric, and how a negative score quantifies physical failure depth?
- ☐ **Adversarial optimization of disturbance trajectories:** Can you describe how black-box optimizers leverage quantitative robustness scores to discover worst-case combinations of latency, noise, and friction?
- ☐ **Test invalidation vs. failure criteria:** Can you distinguish between an execution that falsifies a safety invariant and an invalid trial where testbed preconditions or fault injection mechanisms failed?

With the oracle fixed before the first injection, each trial receives one of the four verdicts before anyone argues about its trace, and a signed margin states how far it sat from each bound. The oracle is only as good as the run it scores, however. The ladder can falsify a contract in simulation, but only an injection on the target hardware can show whether a sag on the shared rail or a burst on the shared bus delays the permission path, and whether the target's own timers meet the deadlines the oracle checks.

15.6 Fault Injection on Hardware

A fault injected in simulation tests the simulator's response. Simulations assume ideal clocks, isolated memory, deterministic context switches, and linear interconnect delay, whereas on the machine, contention on a shared bus, clock drift between microcontrollers, DMA stalls, and voltage drop across inductive drivers decide whether an intervention executes in time. Injection therefore happens on target hardware, through the real transceivers, timers, shared buses, and enforcement path, while an independent containment rig protects the test facility.

In-situ fault injection is the deliberate, deterministic introduction of physical, electrical, and computational disturbances (including sensor register latches, clock drift, bus contention, supply brownouts, and actuator load stalls) directly into target production silicon and mechanics while operating under authentic electrical, thermal, and dynamic loads.

Consider F2 on the hardware. At 1.3 m/s the base covers 1.3 mm in every millisecond, so staleness that goes undetected past the 51.8 ms threshold spends the reserve directly. Left undetected for 78.9 ms, it consumes all 102.6 mm. Neither the camera's datasheet readout time nor the timestamp on a single frame establishes how quickly the permission path notices. A measured P_{99} detection time leaves the worst case unbounded (section 4.4), so the containment claim needs a detection bound justified over the declared compute-load, bus-load, and thermal envelope.

Testing the two-processor implementation (section 4.2) demands physical fault injection at the boundaries its claims name. Table 15.1 gives each case's detector, response, and pass condition; the list adds how each fault is injected and, where it matters, where its events are timed:

- **Stale Observation Injection (F2):** An inline FPGA interceptor latches a navigation-camera frame at the image signal processor, or suppresses encoder updates, while preserving the original hardware timestamp. Detector assertion, brake onset, and wheel speed are measured on independent instruments.

- **Expired Lease Revocation (F1):** The test framework lets the proposer renew once and then suppresses every later renewal. Lease expiry, revocation on the enforcer's next 1 ms tick, and brake onset are each measured on their own wire and clock.
- **Lost-Frame Injection (F3):** A fieldbus disturbance node drops or corrupts EtherCAT frames at a drive's slave port while the base cruises. Each lost 1 ms cycle costs 1.3 mm of travel at the aisle speed.
- **Control-Rail Droop and Feed-Loss Injection (F4a, F4b):** A programmable load draws the 70 A transient of an inference burst coinciding with the drive motors (the brownout of section 2.9) from the shared control rail at its 21.5 V battery-low point, while the permission path runs on the isolated, held-up rail that section 12.9 sizes and the placement record carries (section 13.8). An independent digitizer records the permission-path heartbeat, and brake onset is timed from the undervoltage flag. F4b opens the permission rail's feed during the longest stop the enforcer can admit on an inspected floor, the resident stop from its 1.09 m/s ceiling. A reset of the permission path is FAIL in either trial, whatever the stopping distance, because a stop that power loss hands to the spring brakes mid-motion is not the stop the claim budgets.
- **Duplicate Writer Injection (A1):** A second bus master writes a base-drive channel, either as a frame under the enforcer's EtherCAT identifier or as a DMA write to the drive's command register, while the policy and a remote assistant both command.
- **Tier Bypass Injection (A2):** The testbed submits a policy proposal while MAN is active and, in a separate trial, a remote-assistant command that opposes the enforcer's admitted limit.
- **Handover Stall Injection (A3):** The testbed freezes the remote assistant's input during PEND and drops the acknowledgment frames at both speeds, and the start of the fallback is timed at the drive rather than in the arbiter's log.
- **Replay and Forgery Injection (A4):** The testbed replays a captured `veto_path`, `manual_drive`, or `accept_item` packet and sends another with a corrupted authentication tag.

Recording the state trajectory, the detector identity, the refusal timestamp, and the time to restore the bounded state requires instrumentation electrically and logically independent of the machine, such as optical shaft encoders and high-speed cameras on isolated digitizers. An onboard logging daemon is corrupted by the very failure it tries to capture, because a faulted processor or a saturated bus delays log serialization.³ Independent measurement also rejects false passes, in which the machine survived for reasons absent from the safety claim. An injected lost-frame fault can trip an unmonitored thermal breaker or an auxiliary ground loop that drops power to the drives, so that the spring-applied brake stops the base inside the budget while the software enforcer has deadlocked in a priority inversion. An oracle that inspects only stopping distance scores that trial a pass. The test framework must verify that the enforcer path named in the safety claim executed the corrective action.

A liveness signal is another source of false passes, so the test must target what it can miss. Section 4.6 recounts the mechanism a plaintiff expert argued in the Toyota unintended-acceleration litigation (Barr 2013), a timer interrupt that kept servicing the watchdog while control tasks could die, and draws the rule that a timer feed proves only that the feed path ran (15.2, 15.1). The test vector for that rule kills the control task, or freezes its output, while the timer interrupt stays alive, and measures the actuator command and the watchdog output on instruments independent of the controller. On the mobile manipulator, F1 is this vector applied to the proposer. The host heartbeat can keep reporting a live process after the chunk-policy task has died, so F1 passes on brake onset measured at the drive, never on a heartbeat in a log. Because the historical mechanism is attributed rather than established, a bench result speaks for the design under test, not for any vehicle.

Systems Perspective 15.2: The watchdog liveness invariant

A watchdog fed by a periodic timer interrupt shows only that the interrupt handler executed, not that safety-critical application tasks met their deadlines or executed without corruption. A qualified fault injection campaign explicitly isolates task progress from timer liveness, testing physical shutdown under task death, thread starvation, and frozen control registers.

³ Non-Intrusive Hardware

Tracing: Non-intrusive tracing utilizes dedicated silicon hardware debug blocks (such as ARM CoreSight Embedded Trace Macrocell) and external FPGA bus sniffers that snoop internal bus transactions without stealing CPU execution cycles. By buffering execution traces in dedicated high-speed SRAM decoupled from the system memory bus, hardware analyzers capture cycle-accurate fault-reaction timestamps without perturbing real-time thread schedules. Relying on software logging daemons introduces Heisenberg observer effects, where logging overhead masks the exact race conditions under test.

⚠ War Story 15.1: Testing a task-blind watchdog hypothesis

Context: The Toyota electronic throttle control (ETCS-i) litigation (*Bookout v. Toyota*, 2013) raised questions about whether a critical throttle-control task could die while a timer-tick interrupt continued to service the hardware watchdog (Barr 2013).

Mechanism to test: Plaintiff expert Michael Barr testified regarding a task-death scenario associated with bit-flips and stack overflow memory corruption. The test vector kills the real-time throttle control task while leaving the hardware timer interrupt active, then independently measures throttle position and watchdog reset pins. Because the NHTSA/NASA investigation did not establish an electronic cause of unintended acceleration, this is an attributed test hypothesis rather than a confirmed vehicle fault.

Oracle: A running watchdog heartbeat is insufficient evidence if the safety claim requires timely, task-specific closed-loop control. The verification oracle must record whether the plant independently returns to a validated safe state (such as engine idle or mechanical brake override) within its physical deadline when task execution freezes.

Systems lesson: Inject the failure mode that liveness signals might miss, then measure the physical response on independent instrumentation. Memory protection units (MPUs), independent supervisory cores, and smart windowed watchdogs that require explicit cryptographic task signatures are options whose fault coverage must be demonstrated on the target platform.

The intervention mechanism must be tested under the conditions that make intervention necessary. An enforcer that takes over cleanly on an idle bench provides no evidence for production, where in a crisis the policy runs at peak throughput, memory buses carry high-resolution sensor streams, DMA channels move frame buffers, and the base carries its loaded rack at the aisle speed. The framework injects faults under the declared thermal and electrical load and measures both lease-expiry revocation and, separately, the time from a diagnostic flag to brake onset at the drive. If the intervention path shares a bus arbiter with the neural accelerator, the takeover can stall behind a burst transfer that cannot be preempted, and a safety case that assumed instantaneous takeover fails in the field if the takeover was verified only on a quiet bus.

A campaign runs every injection under one fixed protocol, so that a verdict follows from the procedure rather than from whoever reads the trace. The test orchestrator first checks every precondition, such as temperature, rail voltage, and wheel speed, against its declared tolerance, in that quantity's unit, and a trial that starts outside tolerance ends as INVALID before any fault is injected. It then brings the logic analyzer, the plant, and the truth digitizers onto one reference clock, lets the base settle at its envelope point, and fires the injection on a trigger line that stamps the fault's start on that clock. The detector's flag, the takeover at the drive, and the arrival at the bounded state are each timed from that stamp on an isolated analyzer, while truth sensors wired to a crowbar relay can cut actuator power if the base leaves the survival envelope. The precommitted oracle scores the trace from the independent truth instruments under the verdicts and pass rule of section 15.5, and incomplete truth frames surface only at this stage, after the injection has run, as UNOBSERVABLE. The orchestrator signs the record's header and binds it to the evidence bundle by digest.

Testing distributed loops exposes clock-conversion risk even when each processor meets its local schedule. F2's refusal threshold rests on the 0.2 ms bound for converting a capture stamp to the safety microcontroller's clock, and that bound is itself a claim to test. Take illustrative oscillator limits of ± 100 ppm on the application processor and ± 50 ppm on the safety microcontroller, plus a separately bounded 50 ppm thermal contribution that those specifications do not include. The worst relative rate is then 200 ppm, and drift alone would consume the 0.2 ms bound after 1 s without resynchronization. A healthy 100 ms sync period limits that drift contribution to 20 μ s; network and conversion error must be budgeted separately.

Suppose two good sync frames arrive 400 ms apart while the three scheduled frames between them are dropped. The blackout lasts four sync periods, and the bounded drift contribution grows to 80 μ s. If the observed synchronization jitter has standard deviation $\sigma_{\text{jitter}} = 15 \mu$ s, adding one sigma to the drift rate r_{drift} times the blackout T_{blackout} gives

$$\Delta t_{\text{one-sigma scenario}} = r_{\text{drift}} T_{\text{blackout}} + \sigma_{\text{jitter}} \quad (15.1)$$

Prerequisite: Microarchitectural stress patterns build directly on the memory contention benchmarks from section 13.3.

Protocol: Section C.14.9 gives the protocol stage by stage, with its error codes, for a reader building the test orchestrator.

or 95 μ s. The one-sigma value in equation 15.1 is not a hard bound; a deterministic deadline needs a justified jitter envelope and a clock-conversion test. A fifth missed period gives 100 μ s of drift alone, before jitter, half the 0.2 ms bound on which F2's threshold rests.

Every injection on the hardware ends with the same obligation. The detector that fired, the moment of refusal, the takeover at the drive, and the physical outcome are measured on instruments the fault cannot reach, on one reference clock, and a trial that loses any of them has not shown what it set out to show. The hardware campaign therefore produces records rather than a pass count, each binding one trial to its claim, its envelope point, its builds, and its evidence.

Downstream: Hardware-in-the-loop stress results furnish the evidentiary warrants for safety cases in section 16.3.

15.7 The Fault Manifest

When an F4a trial ends, the base has completed its resident stop while the application processor browned out, and the only witness to the order of those events, and to a permission-path heartbeat that never paused, is a trace on an independent digitizer. Release can count that trial only if the trace stays bound to the claim, the envelope point, and the builds it tested. The **fault record** links a small fixed header to a retained evidence bundle by digest.

Alongside the design contracts, the authority cases A1–A4, and the placement ledger that the fault list already draws on, the record consumes two sources not yet named, the omission log of the policy manifest (section 6.8) and the coverage-gap catalog of the evaluation record (section 7.8), which name the conditions the learned proposer was never shown. Together the header and bundle identify the claim, operating point, injection, hardware and software builds, raw trace, detector, response timing, physical outcome, and anomalies. The fixed header holds the fields every trial shares. Its identity fields name the trial and its claim, the envelope point, the injected fault by mode, magnitude, and target, and a digest of the precommitted oracle. Its outcome fields carry the measured detection and takeover times, the signed margin for the claim, the verdict status, the digest of the evidence bundle, and a signature over the header. The variable trace stays in the bundle.

One F1 record, taken at the envelope's restricted point, 1 m/s on an inspected floor, with the rack loaded to 50 kg, shows those fields filled in (table 15.3). As for every stall, the distance oracle is referenced to the last valid renewal. The travel during the 82 ms before brake onset, the resident stop on a floor of friction 0.12, and the tracking inset together give 759.3 mm, shorter than the 960.9 mm the budget allows at that speed from the moment a target appears. The outcome field holds no value and is marked unavailable. No trial in this book has been run, so the field names the measurement that will fill it, and until that measurement exists the record carries no verdict.

Table 15.3: One F1 fault record at the restricted envelope point: The header fields of a proposer-stall trial on the warehouse mobile manipulator, with the oracle fixed before the run and the outcome left as the measurement that must fill it.

Trial and claim	Envelope point	Oracle, predeclared from the last valid renewal	Detector and witness	Outcome to be measured	Verdict
F1, proposer stall: chunk-policy task killed one tick after a renewal; rack-end stopping claim	1 m/s; 50 kg payload; test floor measured at $\mu = 0.12$; control rail at 24 V nominal	Brake onset ≤ 82 ms; travel to rest ≤ 759.3 mm	Lease timer on the permission path; expiry, revocation, and onset each on its own wire and clock	Trial count; maximum onset and maximum travel over the trials, each with its signed margin; unavailable until measured	None until scored; PASS only if both margins are nonnegative

Byte layout: Section C.14.8 lays out the header field by field, including how an unavailable value is encoded, for a reader building the ledger.

The record keeps the four verdicts of section 15.5 distinct field by field. A breaker that prevents the requested injection makes the trial **INVALID**, and for a trial aborted before injection the detection time, takeover time, and margin are unavailable rather than zero; the bundle records a validity flag for each measured field and why it is absent, and the oracle checks those flags and the verdict before doing any arithmetic, so an unavailable field never enters a pass count or a latency or margin statistic.

A single aggregate percentage hides where the campaign never went. A machine that survived 98 percent of five thousand fault trials has shown little if all five thousand ran at room temperature

on a quiet memory bus. Coverage is therefore reported along separate dimensions: functional safety requirements, physical envelope boundaries, physical and logical injection sites, timing tails, hardware build revisions, and multi-fault combinations. For the mobile manipulator, boundary coverage tracks whether faults were injected with the rack empty and full, at the aisle speed on dry floors and at the restricted speed on inspected ones, and with the control rail at its battery-low point. Timing tail coverage isolates injections targeting the P_{99} scheduling latency of the permission loop and peak direct memory access utilization. A machine achieves acceptable verification not when an average coverage index exceeds an arbitrary threshold, but when the minimum coverage across every individual dimension satisfies the requirements of the safety case.⁴

Repeated trials count only where something varies. A deterministic fault script run five hundred times on a digital bench, with fixed initial conditions, register states, clock dividers, and inputs, executes the same branch sequence every time and says no more than one run. On the test aisle, repetition samples floor friction, asynchronous interrupt arrivals, bus arbitration, and the phase between the sensor clock and the inference cycle, so it measures the spread and tail of the response-time distribution.

When n independent Bernoulli trials sampled from the declared fault distribution have zero missed mitigations, the result sets a one-sided upper confidence limit on the miss probability for that distribution. The count is the zero-failure sizing of section 7.5, applied to an injected fault population rather than to runs of a policy. At 95 percent confidence, a 1 percent limit requires 299 zero-miss injections and a 0.1 percent limit requires 2,995. To infer a FIT-weighted hardware coverage (15.1 defines the FIT rate and the weights λ_i), injections must represent the raw-fault weights λ_i or use a justified stratified weighting and per-stratum analysis.⁵

The bound applies only when the injection outcomes meet the declared independence and sampling assumptions. Repeated stops heat the drive windings (I^2R) and the brake lining, change their resistance and friction, and couple later trials to earlier ones. A campaign run on a dry aisle at room temperature cannot establish a limit for an inspected floor or for the envelope's 5 °C end.

Napkin Math 15.1: Fault coverage under declared assumptions

Scenario: Assume component fault rates λ_i sum to $\lambda_{\text{raw}} = 9,800$ FIT, where one FIT is one fault per 10^9 operating hours. The four illustrative contributions are sensors 1,500, compute 1,000, transceivers and power management 2,500, and gate drivers 4,800 FIT. Let C_{diag} be the fraction of these faults whose hazardous effect is successfully mitigated, weighted by the applicable fault rates. Then

$$\lambda_{\text{unmitigated}} = \lambda_{\text{raw}}(1 - C_{\text{diag}}).$$

A target rate λ_{target} therefore needs $C_{\text{diag}} \geq 1 - \lambda_{\text{target}}/\lambda_{\text{raw}}$, which is 98.9796 percent for a chosen 100 FIT target and 99.8980 percent for a 10 FIT target.

Zero-miss counts such as 299 injections for a 1 percent limit hold only for a fault population sampled in proportion to its λ_i weights. Uniform injections across unequal-rate components estimate a different distribution unless outcomes are weighted and each stratum has adequate evidence. A test must record which mode was injected, whether it was detected, and whether the plant actually reached its allowed state.

Operating hours cannot close what the injections leave open. A zero-event test cannot reach the hazard rates a release claim needs (the exposure wall of section 7.5), so the fault record carries what was never injected as prominently as what passed. It lists every parameter combination omitted because of bench limits, sensor bandwidth, or destructive-test cost, and an untested region therefore cannot reach the safety case of chapter 16 as a verified margin.

This mathematical boundary between empirical road exposure and safety qualification is formalized as the **astronomical exposure wall** (figure ??). As established by Butler and Finelli (1993), demonstrating an ultra-reliable failure rate λ with statistical confidence requires zero failures over an exposure horizon scaling inversely with the target: $N \approx -\ln(1 - C)/\lambda$, which yields $N \approx 3.0/\lambda$ at the standard 95 percent confidence level. For safety-critical automotive systems governed by ISO 26262 ASIL D ($\lambda \leq 10^{-8} \text{ h}^{-1}$, or 10 FIT) or civil aviation systems governed by FAA regulations

⁴ | **Scope of assurance standards:** ISO 26262-5 addresses road-vehicle electrical and electronic hardware. Its single-point and latent fault metrics concern the architecture and its allocated safety goals; they are not the aggregate C_{diag} of 15.1. Fault injection is one way to test diagnostic assumptions, alongside analysis and other evidence. ANSI/UL 4600 frames an autonomous-vehicle safety case from multiple kinds of evidence. Neither standard makes a fixed number of HIL trials sufficient for release.

⁵ | **Trial sampling and aging:** The binomial count assumes independent, identically distributed Bernoulli outcomes under the declared injection distribution; it does not require Poisson event arrivals. Repeated heating, wear, or contact changes can break independence and require a state-stratified test design.

and IEC 61508 SIL 4 ($\lambda \leq 10^{-9} \text{ h}^{-1}$, or 1 FIT), validating life-critical dependability purely through statistical black-box operation requires 300 million to 3 billion consecutive zero-failure hours—spanning 34,000 to over 340,000 continuous vehicle-years. Even the largest commercial autonomous fleets in history, such as Waymo’s driverless operations logging over 200 million miles (approximately 7 million operating hours) or Tesla’s customer-supervised L2 fleet logging over 8 billion miles (approximately 250 million hours), face an insurmountable verification deficit of one to three orders of magnitude against ASIL D and SIL 4 standards. Consequently, statistical black-box fleet testing cannot qualify life-critical autonomy; system safety must instead be mathematically certified through formal runtime architecture, deterministic tripwires, and targeted hardware-in-the-loop fault injection.

15.8 Fallacies and Pitfalls

A machine can pass every isolated fault test and still fail when a shared rail, bus, or clock couples the faults or disables their detectors. These are the compound faults that section 15.3 enumerates from shared causes. Compound tests catch such failures only when enough galvanically or logically independent instrumentation survives the fault to establish the sequence of events.

Fallacy: *Independent local watchdog timers guarantee end-to-end physical deadline compliance.*

The rack-end stop must begin within the 133.6 ms pre-brake delay that the budget reserves. Each stage passes its local watchdog: the camera meets its exposure and readout times, the perception backbone its inference time, the lease trips at 60 ms, and the brake begins within 20 ms of its command. A queue between the image signal processor and the backbone that holds frames an extra 100 ms under load lies between those checks, so no individual timeout detects it. At 1.3 m/s it adds 130 mm of travel against a reserve of 102.6 mm, and the stop overruns the clear distance by 27.4 mm. Only a check that compares each observation’s capture time with the moment the permission path acts on it sees the queue. Verification must trace the complete sensing-to-actuation interval under coincident load and include communication jitter in the budget. Local watchdogs remain useful, but their bounds support an end-to-end claim only when the composition, including every handoff and buffer delay, fits the physical response envelope.

Pitfall: *Allowing diagnostic logging and high-throughput telemetry to share fieldbuses with real-time safety loops.*

Suppose the mobile manipulator shares one EtherCAT segment among sensing, actuation, enforcement, and evidence logging. A diagnostic memory dump raises bus utilization to 98 percent, inducing network transmission jitter that delays both the drive commands and the telemetry needed to judge their physical effect. The same bandwidth contention also delays safety heartbeats and the logs that would explain a failure. Testing each stream separately misses this common dependency. Critical traffic requires bounded service through validated time-sensitive networking (TSN) scheduling and hardware priority, or separate physical channels, while diagnostic traffic must remain within its assigned budget. Compound-load testing must confirm that logging cannot prevent either the safety response or the observation of that response.

Pitfall: *Relying on software anomaly detectors that share electrical power or failure modes with monitored actuators.*

A control-rail brownout freezes a wheel-encoder channel at a plausible 1.3 m/s while a drive fault carries the base at its 1.5 m/s drive limit, a speed at which the finished budget no longer fits the clear distance (section 12.5). The encoder frames remain well formed and the value stays within nominal limits, so the permission path, which reads that channel, does not recognize the overspeed. An isolated drive-fault test would not reveal this common-cause failure if the encoder’s power remained healthy. Verification must examine whether one fault can disable the evidence needed to detect another. Galvanically isolated sensing and explicit checks for specified stuck-signal behavior must support the response, rather than assuming a plausible measurement is a functioning measurement.

Fallacy: *Treating an unobservable test outcome where diagnostic loggers dropped packets as evidence of safety.* During a compound-fault test, bus contention drops logger packets and a watchdog reset erases the volatile SRAM trace. The machine does not strike the rack end, but the team can no longer

reconstruct its peak impact forces, its SE(3) trajectory, or its response timing. That observation cannot establish the force and stopping limits the acceptance criteria specify, because they were not measured. The trial must remain inconclusive for those claims, with the recording failure treated as an instrumentation defect. Before repeating the test, the team must make the required evidence path survive the injected electrical conditions. A visually favorable outcome cannot substitute for the quantitative measurements specified by the safety acceptance criteria.

15.9 Summary

A mobile manipulator that runs its aisle for weeks without incident has shown that its nominal loop worked under the conditions it met. To test its stopping claim, the team must also stall its proposer, age its observations, drop its fieldbus frames, and sag its control rail at the rack end, measuring whether brake onset follows the event each fault is referenced to within its bound and the base comes to rest inside the distance its oracle allows from that event. The four authority rules are attacked on the same loaded approach, each against its own pass condition, from a rejected foreign write to a fallback that starts before the handover deadline. Each test links a design claim, injection, detector, response, and measured plant outcome. Passing one condition narrows uncertainty; it does not prove every combination of faults and aging states safe.

The nominal operating envelope and the separately declared survival envelope give these tests meaning. Crossing the nominal boundary can be a valid survival test if the injected state, instrumentation, and acceptance oracle remain inside the test plan. A trial becomes `INVALID` when the injector or preconditions fail; it becomes `UNOBSERVABLE` when required truth data are lost. A valid failure beyond the nominal envelope can expose an inadequate fallback or containment limit. These verdicts preserve evidence that a simple pass percentage would hide.

Simulation can sample many friction and timing scenarios but cannot establish unmodeled lining wear or electrical response. A physical test samples those effects under its particular hardware and environmental conditions. Zero-miss injections bound a miss probability only for their declared fault distribution; zero-event operating hours bound an incident rate only under the declared exposure model. Neither substitutes for measured actuator response and an independently admitted fallback.



Key Takeaways: Hardware-in-the-loop verification and fault injection

- **Trace the complete response:** Precommit the fault, detector, deadline, physical oracle, and evidence path. Local watchdog passes do not compose into an end-to-end limit.
- **Challenge shared resources:** Coincident bus, power, and sensing faults can disable both a protective response and its witness.
- **Keep verdicts distinct:** A valid violation is a failure; broken preconditions are `INVALID`; lost required telemetry is `UNOBSERVABLE`.
- **State the evidence population:** Report tested operating and survival conditions, fault weights, timing tails, builds, and gaps. A zero-event count is a conditional confidence limit.
- **Carry the record forward:** The versioned fault record supplies one part of the release safety case, alongside analysis, architecture, and field evidence.

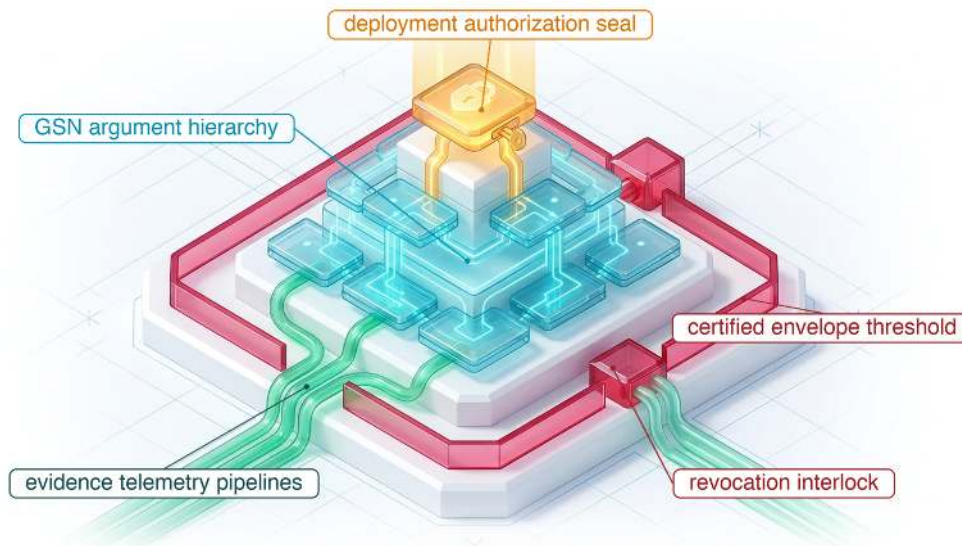


What's Next: From fault records to a release decision

Verification hands release the form of the fault records and the gaps they leave. For each claim, that form fixes the envelope points and builds its trials must reach, the oracle set before each run, and the verdicts a trial can return, and it names the regions no trial reaches. No record yet carries a measured verdict, so none says whether the machine may drive the aisle. Chapter 16 takes that record form, together with the authority, placement, and evaluation records, and asks which claims they support, which verdict follows, and which standing conditions must keep holding for that verdict to stay valid.

Downstream: Untested operational regimes identified during falsification feed the episodic boundary analysis in section 17.1.

Deployment Release



- 16.1 The Decision to Deploy
- 16.2 The Release Question
- 16.3 Safety Cases and Claims
- 16.4 The Three Verdicts
- 16.5 The Verdict's Standing Conditions
- 16.6 A Case Worked in Full
- 16.7 Adversarial Review
- 16.8 The Release Manifest
- 16.9 Fallacies and Pitfalls
- 16.10 Summary

Purpose

Under what auditable chain of physical evidence is an autonomous machine permitted to move among humans?

Releasing an autonomous machine grants software the authority to exert physical forces in unconstrained environments alongside human beings. That deployment authority demands an auditable safety case connecting operational claims directly to empirical evidence from the integrated physical plant. High validation accuracy cannot guarantee that a robot stops within available clearance envelopes, nor can an incident-free test log confirm that emergency fallbacks remain dependable when critical sensors experience silent hardware faults.

A release decision must identify precisely which empirical measurements support each warrant and explicitly delineate the environmental assumptions bounding their validity. Unmodeled payload shifts, floor wear, or thermal drift can instantly invalidate a safety case even when neural weights remain unaltered; where evidence cannot settle an operating premise, refusal is the only acceptable engineering verdict. Within the governance envelope, deployment release consolidates the Brain's proposals, the Nervous System's permission path, and the Body's physical limits into an auditable manifest that revokes motion authority the instant evidence expires.

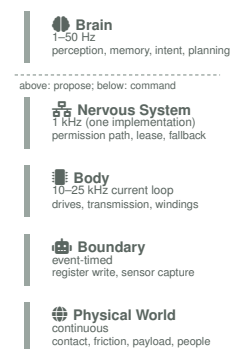


Figure 16.1: The five-level machine stack within the governance envelope.



Learning Objectives

- Construct a claim-argument-evidence case linking a bounded operating envelope to auditable physical evidence
- Evaluate whether each claim's premises are measured, illustrative, or unknown, and select the verdict they support
- Identify the measurements that would lift a refusal and the restrictions a conditional verdict would keep
- Specify standing conditions that withdraw operating authority when wear, drift, or configuration changes invalidate a release
- Diagnose unsupported warrants and compound defeaters through adversarial review of a release argument
- Design a release manifest that binds its upstream records by digest and refuses motion on any mismatch

16.1 The Decision to Deploy

A directory of passing test records and hardware fault traces does not by itself decide whether an autonomous machine may move among people. In a web service, a regression in a new build costs a rollback and some corrupted records. In physical AI, release delegates authority over mass, force, and velocity. Verification has already stressed the machine's timing, contention, and sensing assumptions under injected faults, and what it hands over is its fault records, each with a predeclared oracle, and the gaps they leave. The engineering task therefore shifts from finding defects to bounding what the evidence licenses. Neither validation accuracy (section 7.3) nor a run of clean demonstrations (section 15.1) meets that task, and deciding to deploy is not a statistical prediction. It is an engineering adjudication of whether the evidence bounds the physical risk of operation in a named domain.

Prerequisite: The empirical verification evidence backing release claims is produced in section 15.6.

The release this chapter works through is the warehouse mobile manipulator's service at one site: driving the aisles past rack ends where a person may be standing, opening the spring-latched cage door, and handing the ceramic mug it takes from the takeaway conveyor to the coworker at the packing station. Each job carries its own criterion, a clear distance for the base and a contact force for the arm, and model accuracy establishes none of them. Release therefore depends on pre-contact clearance, measured brake capability, a bounded full response path, and the signed fault records of section 15.7.

A release verdict is a bounded, revocable judgment. The chapter builds that judgment from the operating envelope, a claim-argument-evidence tree, three signed verdicts, the standing conditions whose failure revokes a verdict, a worked adjudication and the adversarial review it must survive, and a release manifest that refuses motion when the running machine no longer matches its signed evidence.

16.2 The Release Question

For the mobile manipulator, the release question is whether the evidence meets the physical criterion of each of its three jobs inside a scoped envelope. The argument names the physical bounds, specified fault set, and validated permission and fallback path. It can support a revocable grant of tracking authority while those conditions hold; it cannot infer protection against every possible fault from finite tests (16.1).



Definition 16.1: Claim-argument-evidence safety case

Claim-argument-evidence safety case is the auditable, structured assurance argument for a specified physical machine, operating envelope, hazard criteria, and fault model, linking top-level safety claims to concrete empirical evidence through explicit engineering warrants.

1. **Significance:** Transforms subjective, informal deployment confidence into an auditable, falsifiable engineering argument that connects high-level safety goals to concrete empirical evidence across the entire physical stack.
2. **Distinction:** Unlike software unit test suites or static model validation scorecards, a physical safety case explicitly links operational design domain boundaries, hardware fault injection logs, and runtime enforcer tripwires to demonstrate bounded residual risk.
3. **Common pitfall:** Treating the safety case as a retrospective compliance checklist completed after development, rather than a living architectural contract that governs feature integration and release gating.

A release case is scoped by the operating envelope, the target ODD paired with the machine's own internal conditions. An unconditional (UNCOND) verdict authorizes the whole operating envelope (section 15.2), a conditional (COND) verdict a named part of it, and a refusal (REFUSE) none. For the mobile manipulator, the operating envelope admits aisle travel at up to 1.3 m/s (chosen; see the Reader Guide) with the tote rack loaded to at most 50 kg, on floors surveyed to a friction coefficient of at least 0.204, past rack ends with at least 1.10 m of clear distance. It admits door approaches at 0.03 m/s and item handovers to the coworker at 0.10 m/s.

Inside these boundaries the learned policy may propose actions across the nominal task envelope, but each action still requires independent permission. Conditional states of the monitored dimensions (table 16.1) narrow that envelope; for example, the machine slows to 1.2 m/s before it requests an in-loop takeover. The restricted mode of a COND verdict narrows it further, so on a floor known only through the site's inspection regime the machine runs at 1 m/s, below the 1.09 m/s inspected-floor ceiling.

As formalized in table 16.1, each monitored dimension is partitioned into nominal, conditional, and refusal states. A conditional state restricts admitted proposals. A refusal state withdraws tracking permission and selects a validated fallback (section 12.9). The table lists the dimensions within which a runtime monitor or a site procedure keeps the machine; the internal conditions of section 15.2, such as ambient temperature, rail voltage, and compute load, enter through the fault records trialed at them.

Prerequisite: The operational design domain and its known absences are defined in section 5.5.

Table 16.1: Operating envelope of the warehouse mobile manipulator: Each listed dimension has disjoint nominal, conditional, and refusal intervals. Conditional operation requires every dimension to remain in a signed admissible interval; missing or invalid measurements refuse. Sensor rates and stopping response are plant validation obligations.

Envelope Dimension / Invariant	Nominal Envelope ($\mathcal{S}_{\text{nom}}$)	Conditional Boundary ($\mathcal{S}_{\text{cond}}$)	Hard Refusal ($\mathcal{S}_{\text{refuse}}$)	Runtime Monitor & Rate	Enforcer Action & Latency
Aisle Speed	$v \leq 1.3 \text{ m/s}$	1.2 m/s before an in-loop takeover request; 0.3 m/s before an out-of-loop one	Measured speed above the signed 1.3 m/s (ceiling 1.39 m/s), or invalid speed estimate	Encoders and IMU, every 1 ms permission tick	Clamp admitted speed; refuse above the signed speed
Floor Friction	Surveyed $\mu \geq 0.204$ (dry concrete near 0.60)	Inspection regime only ($\mu \geq 0.12$): restricted 1 m/s, below the 1.09 m/s ceiling	Unsurveyed floor or reported film; a clear oil film ($\mu \approx 0.05$) is invisible to the cameras	Floor survey and site spill procedure; no on-board measurement	Restrict speed by floor zone; refuse the aisle
Tote-Rack Payload	$0 \leq m \leq 50 \text{ kg}$, valid load estimate (total at most 368 kg)	Estimate stale since the last pick: stationary arm tasks only	Load above capacity or invalid estimate	Rack load cells, at each pick	Refuse aisle transit until re-weighed
Rack-End Clearance	Surveyed $D_{\text{clear}} \geq 1.10 \text{ m}$ at every rack end	Shorter surveyed clearance: lower signed speed from the same budget	Unsurveyed rack end, or obstacle inside the clear distance	Site survey; lidar and navigation camera at frame rate	Speed limit per rack end; stop for a detected obstacle
Door Latch Approach	$v \leq 0.03 \text{ m/s}$, strike plate at its nominal position	None; demonstrations cover no other latch condition	0.10 m/s approach or displaced plate	Arm contact-force sensing in the 2 ms contact loop	Trip at 15 N; refuse the unsupported approach
Coworker Handover	TCP speed $\leq 0.10 \text{ m/s}$ under <code>accept_item</code>	Coworker present without <code>accept_item</code> : hold short of the handover point	TCP speed above the signed 0.10 m/s (ceiling 0.118 m/s), or contact outside the assessed case	Joint encoders every permission tick; arm contact-force sensing	Clamp TCP speed; refuse above the signed speed

¹ | **ISO/TS 15066 Biomechanical Limits:** The specification gives body-region- and contact-type-dependent force and pressure data (ISO/TS 15066 2016). A site assessment must select the applicable body region, contact geometry, and measurement procedure. The handover criterion used here is a conservative case criterion, not a universal ISO/TS 15066 threshold or an injury boundary.

² | **Contact Mechanics Force Model:** The elastodynamic model couples translational kinetic energy, effective moving inertia, and contact interface stiffness to determine transient peak impact force during an unmitigated collision. Impacting a rigid boundary concentrates stored kinetic energy into a sub-millisecond impulse with severe force amplification, whereas elastic compliance extends deformation time and dissipates energy over a longer stroke. For the formal elastodynamic derivation and unilateral contact formulations, see section B.5.

A defensible release claim states its plant, contact geometry, and fault set.¹ The machine's case uses three criteria. The base stops inside the 1.10 m clear distance before it reaches a person who has stepped out at a rack end, the arm opens the door without exceeding the 100 N latch limit, and a handover never presses on the coworker with more than a chosen 50 N peak force for an assessed contact case. The base's claim protects a stationary person; a person still walking toward the machine is a separate premise, carried by a site crossing rule (section 9.6). The release argument must demonstrate which human contacts are excluded before impact, which detected contacts the model covers, and which single faults the independent fallback handles. A stop after contact does not by itself cap the first force peak.²

Consider the arm at its free-space speed limit of 1 m/s, with a robot-side effective mass at the tool center point of 12 kg. In an unbraked elastic contact with the coworker's hand, modeled with an effective tissue stiffness k_h of 15,000 N/m, the peak contact force follows equation 16.1:

$$F_{\text{unbraked}} = v \sqrt{k_h m_{\text{eff}}} \quad (16.1)$$

At the free-space limit this ideal elastic model predicts 424 N, or 8.5× the 50 N criterion, so the arm cannot enter an unseparated contact at that speed without a stronger, measured protection argument. The same equation bounds the item-handover speed that the enforcement record caps (section 12.11, section 14.2), since the criterion admits at most 0.118 m/s. At the chosen 0.10 m/s, the model predicts 42.4 N, inside the criterion.

The base's claim is a budget rather than a single force, and the delay components this chapter must evidence are the lease path and the observation age that its claims name. At the 1.3 m/s aisle speed, the finished budget (table 12.1) is 997.4 mm against the 1.10 m clear distance, a spare of 102.6 mm. That spare is the whole allowance for error in the budget's inputs. An unbudgeted delay of 78.9 ms, the handover deadline T_{latest} of section 14.3, spends it, and so does a loaded deceleration

14 percent below the 2 m/s^2 the budget assumes. A stopping claim whose inputs are unmeasured cannot show that its spare survives their errors.

The release question for the mobile manipulator has become three bounded criteria: the base stops inside the clear distance, the arm opens the door under the latch limit, and an item handover stays under the contact criterion. No calculation settles any of them alone, because each inherits premises about deceleration, delay, and contact that must themselves be evidenced, and each premise can lapse in service as brakes wear, floors soil, and sensors drift (section 16.5). Connecting every claim to its premises, and every premise to its evidence, is the work of a safety case.

16.3 Safety Cases and Claims

No inspection of the mobile manipulator can confirm that its loaded base will come to rest before it reaches a person at a rack end. That claim has to be decomposed until every branch ends in something measured, and a safety case is the structured argument that records the decomposition. Building on Stephen Toulmin's argument structure (Toulmin 1958), Goal Structuring Notation (GSN)³ (Kelly 1998; Kelly and Weaver 2004), and the UL 4600 autonomy evaluation standard (Koopman 2020), this chapter constructs the claim-argument-evidence safety case of 16.1. Its root is a claim that asserts a measurable physical property, such as the base's stop within the clear distance. Because that claim cannot be validated directly, strategies partition it by operating regime, hazard class, or subsystem, and each sub-claim ends in further sub-claims or in evidence, bounded by the context and assumptions under which its links hold.

The structure's validity hinges on distinguishing evidence from demonstration. A **demonstration** is a record of successful task execution over an uncharacterized test distribution, such as a long run of clean door openings with the strike plate always at its nominal position. It measures mean performance but bounds neither the tail nor the failure modes outside the sampled trajectory. Evidence is a falsifiable, calibrated artifact with quantified bounds, such as a worst-case timing distribution from hardware execution traces, loaded stopping trials across deceleration regimes, or a fault-injection trial that drives the machine to the boundary of its envelope. An artifact without quantified uncertainty, or one that never tests the boundary, remains a demonstration and cannot support an inferential link.

Connecting evidence to a claim requires a **warrant**, the inferential rule stating why the data support the assertion. For a learned policy the warrant is inductive, reasoning from failure-free test episodes to future service, and the exposure wall of section 7.5 shows why that link cannot carry a high-consequence claim alone. No zero-event campaign, on one machine or a fleet, reaches the hazard rates such claims name within any practical schedule. The argument must therefore add an architectural warrant, grounding the claim in an independent permission path (chapter 12) that bounds actuator torque and withdraws motion whenever a learned command leaves the admitted set. That warrant narrows the claim rather than escaping the wall, since the permission path changes the claim only for the hazards it detects and stops in time (section 17.2).

This tension between inductive fleet evidence and safety-critical assurance is illustrated by a decade of autonomous vehicle disengagement data reported to the California Department of Motor Vehicles (figure ??). Between 2015 and 2025, developers logged tens of millions of public testing miles across California roads. Over this decade, Miles Between Disengagements (MPI) expanded across three distinct eras:

1. **Era I: Early Highway & Suburban Testing (2015–2018):** Systems operated primarily on well-marked arterial corridors and highways. Developers established baseline autonomy cadences, scaling from hundreds of miles between human interventions to several thousand miles.
2. **Era II: Dense Urban Core Transition (2018–2021):** Fleets migrated into dense metropolitan environments, most notably San Francisco's complex street grid. This environmental domain shift exposed uncharacterized edge cases—pedestrians stepping between double-parked vehicles, construction detour gestures, and dynamic occlusions—temporarily stalling or depressing reported MPI (e.g., Waymo's metric dropped from 29,945 miles in 2020 to 7,965 miles in 2021 before recovering).
3. **Era III: Commercial Driverless Scale & Divergence (2021–2025):** Removal of the in-vehicle safety driver led to sharp strategic divergence. Waymo scaled rider-only commercial operations, exceeding 63,000 miles per disengagement in 2025 across more than five million testing miles.

³ | **Goal Structuring Notation:** Developed by Tim Kelly at the University of York, GSN provides an explicit graphical grammar for structuring and auditing safety assurance cases (Kelly 1998; Kelly and Weaver 2004). The notation decomposes top-level safety goals into contextual sub-claims, argumentation strategies, and solutions grounded in test artifacts.

Aurora pivoted its California passenger-car effort toward commercial Class 8 freight corridors in Texas. Apple cancelled its Project Titan initiative in early 2024 after a multi-billion-dollar R&D effort that struggled with early intervention sensitivity. Most instructively, Cruise crossed 80,000 miles per disengagement in California filings shortly before an October 2023 pedestrian-dragging collision in San Francisco resulted in the immediate regulatory revocation of its autonomous testing and deployment permits.

The fundamental systems lesson of figure ?? is that an inductive warrant derived solely from aggregate fleet mileage cannot establish a catastrophic hazard claim. A high MPI indicates operational maturity under nominal conditions, but it does not bound the severity or probability of tail-event failures under unmodeled physical interactions. Safety cases for physical AI must combine empirical exposure with deterministic architectural enforcers that remain valid when learned policies encounter novel distributional shifts.

An engineering safety case does not exist in an institutional vacuum. Inside the development team, the claim-argument-evidence tree establishes technical confidence that physical risks are bounded; outside the team, it forms the evidential substrate for statutory regulatory authorization and legal accountability. When an autonomous system moves from the laboratory into commercial deployment, its safety claims intersect three primary governance pillars:

1. **The European Union AI Act (High-Risk Classification):** Under the EU Artificial Intelligence Act, AI systems deployed as safety components in physical machinery, collaborative robots, or autonomous transport are classified as *High-Risk AI Systems*. Statutory compliance mandates an auditable risk management system that spans the operational lifecycle, verified data governance documenting training and evaluation distributions (section 5.9), human oversight interfaces capable of real-time intervention (section 14.2), and tamper-evident event logging (section 14.7) to ensure forensic reconstructibility after any anomalous contact.
2. **Domain-Specific Statutory Certification:** Beyond horizontal AI regulations, embodied systems must satisfy sector-specific safety mandates: the U.S. Food and Drug Administration (FDA) pre-market pathways (510(k) and De Novo) for autonomous surgical robotics, requiring human-factors validation under ANSI/AAMI HE75 and software lifecycle controls under IEC 62304; Federal Aviation Administration (FAA) airworthiness certifications (Part 107/135) for autonomous aerial logistics; and UNECE/NHTSA standards (such as UN R157 for automated vehicle steering) governing automated driving systems.
3. **The Redistribution of Legal Liability:** In classical factory automation where robots operate within interlocked cages (figure 3.4), physical accidents are adjudicated primarily under the legal doctrine of *operator negligence*—evaluating whether floor personnel violated safety protocols or bypassed interlocks. When an autonomous machine operates without physical fencing in shared human spaces, legal responsibility shifts fundamentally toward *strict manufacturer product liability*. An unmodeled perceptual failure or tracking overshoot is no longer an operator error; it is an alleged defect in design, manufacturing, or failure to instruct. In this legal regime, a cryptographically signed safety case and release manifest is not merely good engineering practice—it is the definitive evidentiary record establishing that the engineering team bounded residual risk to state-of-the-art standards prior to public release.

Every link also rests on contextual assumptions, and an unstated one is guarded only by whatever check happens to sit downstream of it. The stopping warrant assumes, among other conditions, that the aisle stays bright enough for the navigation camera's nominal exposure. Suppose a dim aisle doubles that exposure from 16 ms to 32 ms. Because an image's age is timed from mid-exposure, each observation arrives 8 ms older than the budget charges, which unchecked would carry the base another 10.4 mm at the 1.3 m/s aisle speed, about 10 percent of the budget's 102.6 mm spare. The permission path's age check refuses those frames instead, and the base stops in an aisle the case assumed it could run. The team meets its lighting assumption as unexplained lost service, and only a check that the case never tied to lighting kept it from becoming a silent loss of clearance. An assumption that no runtime check sees has no such backstop, as the localization allowance in the compound failure of section 16.7 shows. A sound argument therefore states every physical dependency as a bounded context element with a runtime monitor that detects its violation.

Downstream: Epistemic assumptions embedded within safety case warrants are critically interrogated in section 17.1.

The weakest link governs every claim-argument-evidence tree. A safety case is not an aggregate in which over-performance in one branch compensates for a deficit in another. High accuracy on a recognition benchmark does not offset an uncharacterized latency tail on the CAN bus, nor does extensive policy simulation offset missing brake-wear data. If the base's stop decomposes into sub-claims for braking, thermal protection of the drives (section 2.5), and trajectory bounding, a failed warrant in any one branch invalidates the root claim. The reviewer who adjudicates the case therefore looks not for the number of supporting artifacts but for the single weakest link whose failure would permit harm.

16.3.1 Match the safety argument to the machine

For the mobile manipulator's arm, ISO 10218-1 and ISO 10218-2 frame the robot and integration assessment, and ISO/TS 15066 informs the collaborative-contact case once the argument names the contacted body region, pressure, geometry, and measurement method (*ISO/TS 15066* 2016). The base's claims, which make up most of the case, are argued from its measured stopping budget and the site survey. The road-vehicle standards ISO 26262 and ISO 21448 lend the argument their distinction between faults and limits of intended function but certify nothing about this machine (*ISO 26262* 2018; *ISO 21448* 2022).

The claim-argument-evidence scorecard in table 16.2 states six claims and pairs each physical warrant with its adjudication on this book's evidence. Claim C3 shows the decision: an oil film can defeat braking traction, so nominal model accuracy cannot justify an unrestricted verdict. A physically realizable condition that invalidates a warrant in this way is its **defeater**, and section 16.7 turns the search for defeaters into a review method. Claim C6 stands only on the fault records that test the four authority rules section 14.8 hands over (A1–A4).

Table 16.2: Claim-argument-evidence review for the warehouse mobile manipulator: Six claims and their adjudication on this book's evidence. A failed or unknown branch cannot be compensated by nominal model accuracy.

Claim	Warrant and operating assumptions on the machine	Adjudication on the book's evidence
C1. Contact and stopping	Base: stop of 997.4 mm at 1.3 m/s (table 12.1) inside the 1.10 m clear distance to a person standing at the rack end, with the payload untested at 368 kg an open premise; Door: latch force of 39 N at brake command, with the arm stopping within a further 0.15 mm (about 3 m/s ²) under the 100 N limit; Mug: picked from the conveyor under the 1 m/s TCP limit on tracked intent and handed to the coworker at 0.10 m/s, under the derived 0.118 m/s ceiling, with a contact force of 42.4 N under the 50 N criterion, and the grasped object taken to be the mug the task names	REFUSE: credible deceleration, brake onset, tracking and localization error, arm deceleration, and tissue stiffness are illustrative; the grounding premise is unknown
C2. Permission timing	Brake onset no later than 82 ms after the last valid renewal; enforcer WCET of 250 μ s inside the 400 μ s deadline; declared assumption: the claim covers faults the permission path survives, lockstep diagnostics cover the path's own failure (section 13.2), and the inhibit rung of the fallback ladder carries no budget (section 12.9)	REFUSE: the WCET is declared, not measured under load
C3. Braking traction	Floor friction of at least 0.204 on every aisle; a clear oil film is the defeater	REFUSE: the open premise passes to section 17.3
C4. Observation age	Navigation-camera age of 51.6 ms inside the budget, with a stale-data fallback	REFUSE: the age chain is illustrative
C5. Permission independence	The placement record's loaded maxima stay within the declared 250 μ s WCET, and so below the deadline, and common causes are tested (section 13.8)	REFUSE until the placement test ledger exists
C6. Authority transfer	One writer per actuator channel; no source bypasses a higher authority tier or the permission check; an unacknowledged handover falls back by the clearance-derived T_{timeout} of section 14.3 (174.7 ms at the 1.2 m/s slow-down before an in-loop takeover; 78.9 ms for a request at 1.3 m/s); invalid authentication locks out the channel (section 14.8)	REFUSE if any of A1–A4 lacks a PASS fault record (section 15.7); restrict to local operation if only the remote-channel cases fail



Figure 16.2: Physical separation example: A yellow industrial arm works behind yellow mesh fencing. Fencing can be one condition of a restricted release when a validated access interlock and stopping test support it. Source credit: U.S. Army Chemical Materials Activity / PEO ACWA.



Checkpoint 16.1: Claim-argument-evidence decomposition and inductive warrants

Before presenting a safety case for adjudication, test its structure:

- ☐ **Top-level claim falsifiability:** Can you state the top-level claim as a falsifiable physical proposition, such as a stop inside the clear distance, rather than an aspiration?
- ☐ **Warrant validity:** Can you name the warrant connecting each test bench to the claim above it, and the premises under which that warrant holds?
- ☐ **Defeater enumeration:** Can you show where a defeater such as lens fouling or bus arbitration collisions appears in the argument graph?
- ☐ **Evidentiary provenance:** Can you trace every terminal evidence node to a signed test trace and a current calibration?

16.4 The Three Verdicts

Every row of table 16.2 ends in an adjudication, and the release decision resolves the whole tree into one of three verdicts, **unconditional operation**, **conditional operation**, and **refusal to operate**. The first, unconditional operation, grants the full envelope named in a signed release case while its standing conditions remain true. Each hazard claim in that case needs evidence for the specified plant, load, operational design domain, and fault set, with brake thermal capacity, stopping clearance, and tracking-error bounds qualified for that envelope (section 2.5) and independent permission checks wherever learned proposals can reach actuators (section 1.2). The verdict does not eliminate residual or newly discovered hazards; it authorizes operation under the assessed claims and requires revocation when their premises fail.

The second verdict, conditional operation, applies when the evidence shows the machine safe within a restricted operating subset but lacks the warrants for the full envelope. Release then enforces physical, kinematic, or environmental boundaries that exclude the unverified hazard states, such as de-rated joint velocity, a restricted illumination range, or a barrier that keeps people outside the manipulator's reach. Where contact force cannot be bounded for unseparated co-presence, a restricted case may require an assessed fence and validated access interlocks (figure 16.2), tested so the protective stop completes before a person reaches the hazardous workspace.

Every condition attached to a release verdict is a binding contract, and each names a number the permission path enforces rather than the ceiling the physics allows. The item handover shows the

difference. The contract signs a TCP speed below the ceiling that the contact criterion admits under equation 16.1, and the gap between the two speeds is the allowance for what the monitor cannot see at the instant it samples: speed growth between samples, encoder error, and the drive's response to a clamp. A contract signed at the ceiling leaves no room for those terms, and the release case needs a measured bound on each before the signed speed can stand.

The inspected-floor condition follows the same rule. Where the site's inspection regime certifies a friction coefficient of only 0.12, the credible deceleration falls to what that friction supports, and the budget that admits 1.39 m/s on surveyed floors admits only 1.09 m/s. The contract binds only while its premises hold.

The third verdict, refusal to operate, is the mandatory outcome whenever the safety case contains missing warrants, uncharacterized tail latencies, incomplete fault-injection coverage, or unmonitored failure modes. If the enforcer's worst case under the placement's load has never been measured against its 400 μ s deadline, or the fault set omitted the common-cause tests the placement names, the hazard remains unquantified and cannot support release. Refusal is likewise required when a measurement contradicts the model behind a budget term, such as loaded stopping trials that decelerate below the 2 m/s² the budget assumes.

A refusal verdict is an engineering deliverable, not an administrative failure. It moves a dispute between schedule and safety onto evidence by isolating the broken link in the argument tree. If release is refused because loaded stopping trials contradict the braking term, the team does not debate risk tolerance but repeats the trials at the loaded mass until the low tail of the deceleration is known, re-derives the stopping term from that tail, and returns for re-adjudication only when the aisle budget closes on the measured value. The refusal verdict defines the exact evidentiary test that must pass before re-adjudication can occur.

Accountability for the release decision must reside with an identifiable **lead systems adjudicator** rather than being diffused across development teams. When a vision team signs off on detection scores, an actuator supplier on torque curves, and a learning team on policy convergence, each assumes that another layer provides the margin. The adjudicator alone attests that the integrated system satisfies the safety case, and the attestation binds the verdict to a set of immutable technical artifacts, including the cryptographic hash of the verified policy weights, the calibration logs of the arm's contact-force sensing, the brake-onset trials together with a separately justified worst-response bound, and the stopping trials at the loaded mass that establish the credible deceleration. Signing records the adjudicator's bounded judgment about that evidence and the specified operating domain.

The adjudicator signs one of the three decisions in table 16.3. A conditional verdict needs a separately evidenced restricted envelope; a known hole in the nominal argument cannot simply be relabeled as a safe restriction. No verdict moves the proposal boundary. Under UNCOND and COND alike the policy only proposes and the enforcer still admits each action, as principle 3 requires, so a release sets the envelope the permission path enforces and never grants the policy permission of its own.

Prerequisite: Deterministic fallback coverage supporting conditional release verdicts builds on section 12.9.

Table 16.3: Three signed release verdicts for the mobile manipulator: Full operation and restricted mode both retain independent per-action permission. Refusal is an actionable engineering result.

Signed verdict	Evidence required	Permission and runtime limits	Expiry or refusal trigger
UNCOND	Full operating-envelope claim, measured plant and timing bounds, independently validated permission and fallback	Policy proposes across full <i>signed</i> task envelope; every action passes the enforcer	Evidence, configuration, calibration, or standing premise fails
COND	Restricted operating-envelope claim and its own evidence, limits, and fault coverage	Policy proposes only within signed reduced limits; every action passes the enforcer	Any restricted premise becomes false or unknown
REFUSE	Missing, contradictory, expired, or unauthenticated critical evidence	No autonomous tracking permit; execute the assessed stop or hold, the fallback-ladder rung the release case validates for the plant (section 12.9)	Re-adjudication with repaired evidence

None of the three verdicts is a permanent grant. Linkages develop backlash, friction surfaces wear, lenses accumulate film, and ambient distributions change, so operating under UNCOND or COND carries obligations for the life of the deployment. Brake wear that lowers the loaded deceleration by the shortfall that spends the aisle budget's spare (section 16.2) leaves the budget no allowance for any other input error. That wear must fail a standing condition (a scheduled stopping trial or a validated brake self-test) before the machine re-enters the aisle, since a stop in service would first reveal it at a rack end. Each such obligation pairs a premise with a detector and a response, which makes it one of the standing conditions the verdict carries.

16.5 The Verdict's Standing Conditions

A release verdict depends on physical premises with different observation schedules. The mobile manipulator's braking claim relies on floor friction of at least 0.204, which dry concrete at about 0.60 exceeds, but a one-time survey cannot cover spills or surface wear. The case needs a floor inspection schedule and a stop or exclusion rule after contamination, with any on-board traction monitor specified separately. The arm's contact-force calibration is re-established at a documented interval. Each premise must remain valid at the cadence required by its physical failure mode. Under principle 4, the verdict grants authority per operating condition, and that authority lasts only while the evidence behind each premise stays current.

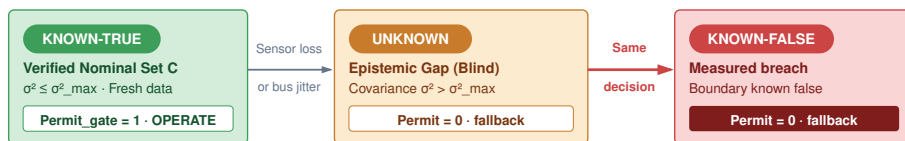
Every monitored premise requires both a deterministic runtime detector and a pre-allocated response. If the case claims that the navigation camera's observation age stays inside the 51.6 ms the budget charges, but the software has no hardware timestamp comparator to measure that age on each frame, the premise has not been monitored. It has silently degraded into an assumption. The tote-rack load, bounded by its 50 kg capacity, changes only when the arm places or removes a tote, so a check at each pick suffices, while speed and lease validity are checked on every 1 ms permission tick.

Evaluating monitored premises at runtime exposes a trichotomy in **envelope membership** (16.2). A binary check presumes normal execution whenever no fault flag is set, but envelope membership has three states, known-true, known-false, and unknown. Membership is known-true when telemetry is verified, timestamps meet their deadlines, and estimators converge with bounded covariance. When a physical limit is breached, such as base tracking error exceeding the 40 mm bound that the stopping envelope insets, the state is known-false. When an optical encoder drops communication, a camera frame arrives with a cyclic redundancy check (CRC) mismatch, or an extended Kalman filter diverges, the machine cannot determine whether it is inside or outside its signed operating envelope. With mass in motion, an unknown state is treated as outside the envelope, since presuming safety during an epistemic gap, while the machine moves blind, lets kinetic energy accumulate unmeasured.

Definition 16.2: Envelope trichotomy

RUNTIME ENVELOPE: THREE EVIDENCE STATES

Unknown and known-false have different causes; both refuse tracking permission.



- **Software Fallacy:** `is_safe == (!fault)` treats telemetry drops as benign → frozen commands risk collision
- **Physical Invariant:** UNKNOWN and KNOWN-FALSE → refuse tracking; select validated stop or hold

Envelope trichotomy is the three-state epistemic classification of cyber-physical system health relative to its certified operating envelope: *known-true* (verified state with bounded telemetry covariance inside envelope bounds), *known-false* (verified boundary violation triggering deter-

ministic hardware refusal), and *unknown* (epistemic state uncertainty due to sensor dropout, frame corruption, or estimator divergence).

1. **Significance:** An unknown state is not evidence of a boundary breach, but neither supports continued tracking. The permission decision is refusal in both unknown and known-false states; the logged cause remains distinct and the response depends on the plant's validated stop or hold capability.
2. **Distinction:** Unlike a binary boolean status check (`is_safe == true`), the envelope trichotomy isolates epistemic blindness from active violations, ensuring telemetry loss is never mistaken for safe clearance.
3. **Common pitfall:** Defaulting to “fail-open” or holding last-known commands when communication fails. Frozen commands in the presence of unmodeled dynamics rapidly result in mechanical collisions.

The same three states govern the evidence behind each premise. A trial marked UNOBSERVABLE or INVALID in section 15.5 supplies no evidence, so the premise it was meant to test stays unknown, and an unknown premise refuses the claim that depends on it; only PASS records count toward a verdict, and a FAIL record makes the premise known-false.

The response to an unevaluable premise follows from the physical consequence of continuing motion without telemetry, not from a severity label. When the mobile manipulator runs the aisle at 1.3 m/s with its tote rack full, its 368 kg loaded mass carries 311 J of kinetic energy. If its lidar stops transmitting updates, each millisecond of blind travel carries the machine 1.3 mm closer to an unmaped obstacle. The safety case fixes three responses in advance. Two of them exist only where the signed case covers them: a **degraded mode** that continues when redundant sensors still bound positional uncertainty, such as falling back to wheel odometry and halving the aisle speed to 0.65 m/s, and the restricted mode of a COND verdict that clamps torque or confines proposals to a precomputed corridor. The third, the STOP rung of the fallback ladder (section 12.9), needs no such coverage and uses the configured drive and brake sequence when observability is lost; its ability to stop within clearance depends on the current speed, response delay, and measured braking capability.

Even when every monitored premise remains known-true and periodic recalibrations pass, the **release verdict** carries an expiration timestamp and cycle budget. Operating hours and mechanical cycles can change the plant in ways that runtime telemetry does not fully observe. Gear teeth may develop surface pitting,⁴ elastomer seals stiffen, and structural fasteners lose clamp preload. The load-specific fatigue data and inspection plan set the signed operating-hour and cycle limits; reaching either limit requires re-adjudication before further motion is permitted.

Standing conditions that affect motion need an enforcement path with a measured detection and response budget. In the architecture of chapter 12, the independent enforcer checks proposals and monitored premises before issuing actuator permission.⁵ The machine's permission path decides on a 1 ms tick. Its 82 ms bound from the last valid renewal to brake onset covers a proposer that stops renewing. A premise the enforcer monitors itself needs only the tick, the bus transfer, and brake onset, 22 ms in the budget, and the release case must measure each of those terms. When a premise is unknown or false, the enforcer revokes tracking and commands the rung of the fallback ladder that suits the plant's state (section 12.9), and the signed release case must include that stop choice and its measured timing.

A verdict therefore stands only while each of its standing conditions is known-true, checked at the cadence its failure mode sets and backed by a response whose timing the case has measured. With claims, verdicts, and the conditions that revoke them defined, the adjudicator can take the mobile manipulator's six claims one at a time and decide its release on the evidence this book holds.

16.6 A Case Worked in Full

The adjudicator takes each of the six claims in table 16.2, finds the premise that carries its warrant, and asks what kind of number supports that premise. The door branch of C1 shows the procedure in full.

⁴ | **Mechanical Fatigue Budget:** A cumulative-damage model such as Miner's rule ($\sum_i n_i/N_i$) can organize a measured load spectrum against component-specific fatigue data; it is an approximation, not a universal failure deadline. The assessed loads, uncertainty, inspection evidence, and stopping consequences determine any signed payload derating and cycle limit. An expired limit revokes the applicable release case rather than silently extending service.

⁵ | **Hardware Watchdog Interlocks:** A watchdog can detect a missed configured heartbeat. An independently wired supervisor then requests the validated drive stop; the detector, wiring, drive, brake, and load response all enter the timing and fault analysis.

At the guarded approach speed of 0.03 m/s against the 4.0×10^5 N/m strike plate, contact force rises as the plate compresses until the 15 N tripwire fires, and the 2 ms contact-loop response adds further compression before the brake command begins (section 10.2):

$$F_{\text{cmd}} = F_{\text{trip}} + k_{\text{latch}} v_{\text{latch}} T_c \quad (16.2)$$

Evaluating equation 16.2 gives 39 N at brake command, well inside the 100 N latch limit. The limit still binds the stop that follows, because the plate keeps compressing while the arm decelerates:

$$x_{\text{max}} = \frac{F_{\text{cmd}}}{k_{\text{latch}}} + \frac{v_{\text{latch}}^2}{2a_{\text{arm}}} \leq \frac{F_{\text{lim}}}{k_{\text{latch}}} \quad (16.3)$$

The braking term in equation 16.3 ignores the spring work that would shorten the arm's travel, which keeps the bound conservative. Below the limit the plate can compress only 0.15 mm further, so the arm must decelerate at about 3 m/s² and stop within about 10 ms.⁶ At the unsupported 0.10 m/s approach, the same two equations give 95 N at brake command and demand 400 m/s². That demand is consistent with the two reasons the operating envelope (table 16.1) admits only the guarded approach. At the faster speed the contact deadline barely exceeds the contact loop's response (section 6.1), and the demonstrations hold no example of it (section 5.8).

Neither result is in doubt as arithmetic. The door branch fails on its premise. Whether the arm, carrying its approach momentum, reaches that deceleration within a fraction of a millimeter is a property of its drives and brakes, and no record in this book measures it. The deceleration is illustrative, so the premise is unknown and the branch is refused.

The other branches fail the same way. The base's 997.4 mm budget inherits illustrative values for credible deceleration, brake onset, tracking error, and localization, and its 102.6 mm spare is small against a plausible error in any one of them. The mug branch of C1 is refused on two premises. Its handover force sits under the criterion only through an illustrative tissue stiffness, and its grounding premise, that the object the arm grasps is the mug the task names, is unknown, because a confident grounding to a same-colored glass jar passes every geometric check (section 10.3) and no record in this book settles which object the arm holds. The permission timing of C2 rests on a WCET declared at 250 μ s beside an unloaded 135 μ s, never measured under the placement's worst load. The 51.6 ms observation age of C4 adds datasheet readout to illustrative pipeline terms. C5 and C6 cite records that this book describes but does not contain: the placement test ledger and a PASS record for each of A1–A4. C3 differs in kind. Its premise, floor friction of at least 0.204 on every aisle, can be surveyed on a dry floor, but a clear oil film looks like dry concrete to the cameras, and no measurement on the machine can exclude it.

Every claim therefore rests on at least one premise that the book's evidence leaves unknown, and the adjudicator signs REFUSE. The verdict does not find the machine unsafe. Its inputs are illustrative, so the decision is illustrative too. It shows how this release would be decided and cannot license the machine. Under the fourth law (section 1.7), a premise the evidence cannot settle must restrict operation, and on this evidence every claim carries one. The refusal is also the chapter's most useful result, because it names the measurements that would change it. Table 16.4 lists them, with the premise each settles and the restriction a supporting result would lift.

Table 16.4: Measurements that would move the verdict toward COND: Each row names the premise a measurement settles on the warehouse mobile manipulator and the restriction a supporting result would lift; the inference trace bears on service availability only.

Measurement	Premise it settles (value in this book)	Claims	What a supporting result lifts
Stopping trials at the loaded mass	Low-tail credible deceleration (2 m/s ²)	C1	The base's budget and its 1.39 m/s ceiling, so an aisle speed can be signed
Brake-onset trials	Brake onset (20 ms)	C1, C2	The 82 ms bound from the last valid renewal to brake onset
End-to-end age trace	Observation age (51.6 ms)	C4	The age term of the budget and the stale-data threshold

⁶ | **Reflected Rotor Inertia in Impact Dynamics:** A geared joint can reflect rotor inertia approximately as $N^2 J_{\text{rotor}}$ into the joint coordinate. Its contribution at the contact point also depends on linkage geometry and compliance. The ideal one-dimensional calculation excludes that contribution and brake torque rise; the release case needs measured or modeled bounds before treating its force result as a physical maximum.

Measurement	Premise it settles (value in this book)	Claims	What a supporting result lifts
Tracking-error and localization tails	Tracking bound (40 mm) and localization bound (50 mm)	C1	The inset and overhead terms of the budget
Site survey	Rack-end clear distance (1.10 m)	C1	A signed speed at each surveyed rack end
Floor survey	Dry-floor friction (0.60) against the 0.204 needed	C3	Surveyed aisles only; the film stays unobservable, so the aisle claim can reach at most COND on inspected floors below the 1.09 m/s ceiling
Inference trace	Chunk inference at the 99th percentile (40 ms)	None	Service availability: renewal of each chunk inside the 60 ms lease; the C2 bound holds without it
Loaded enforcer timing	Enforcer WCET (250 μ s, declared)	C2, C5	The 400 μ s deadline under the placement's worst load
Arm stopping trials	Arm deceleration from the guarded approach (about 3 m/s ² needed)	C1	Door opening at the guarded approach
Handover force trials	Peak contact force at the handover speed (42.4 N derived; tissue stiffness illustrative)	C1	Handing the mug to the coworker at 0.10 m/s under <code>accept_item</code>
Operator trials	Takeover and authority-handover timing	C6	Remote takeover, once A1–A4 each have a PASS record; otherwise local operation only

A COND case would need every row of the table except the availability-only inference trace and the operator trials, and it could forgo the operator trials only by keeping authority local, where the remote channel's authority cases do not arise. Local operation would still need a PASS record for each authority case it exercises, and C5 would still need the placement test ledger with its common-cause tests. Neither is a row of the table.

Even with all of that evidence supporting its premises, the most the results could license is a named part of the operating envelope. The base could travel the aisles at a restricted 1 m/s, below the 1.09 m/s inspected-floor ceiling, on inspected floors only, with the enforcer in its restricted configuration (section 12.11) and the crossing rule in force at every rack end. The arm could open the door at the guarded approach with the plate at its nominal position and the base stopped, and it could take the mug and hand it over under `accept_item` at 0.10 m/s, with the station's feed held to its declared item list and a singulated pick window. The grounding premise has no row at all, since no trial with the machine's present sensors can settle it, and the feed list and singulation restrict it rather than close it. The 1.3 m/s normal mode stays unsigned whatever the results, because the oil film leaves its friction premise open. The restrictions that carry these premises, and the evidence that would close them, belong to the residual-claims register of section 17.3 (16.2).

Checkpoint 16.2: Refusal and the path to conditional release

Before approving a conditional release of the machine, verify your understanding of how a premise, rather than a calculation, decides a claim:

- ☐ **Premise versus arithmetic:** Can you explain why the latch force at brake command is not in doubt while the door branch of C1 is still refused?
- ☐ **Unsupported approach:** Can you use equation 16.2 and equation 16.3 to show why the operating envelope admits only the guarded door approach?
- ☐ **Refusal as a deliverable:** Can you name, for one refused claim, the measurement that would lift it and the restriction that would remain after a supporting result?

16.7 Adversarial Review

Suppose the measurements of table 16.4 arrive and support a COND case for the mobile manipulator. Before the adjudicator signs it, the case must survive a reviewer whose task is to break it. An **adversarial review** is a structured audit that treats the safety case as an argument to be attacked

rather than a brief for deployment. Confirmation bias steers the case's own data collection toward conditions where the learned models succeed, and the reviewer inverts that posture, searching for unsupported warrants, uncharacterized physical transitions, and edge conditions that break the release contract. An argument that fails on paper against an adversary trying to force a physical violation gives no reason to expect the machine to withstand service.

The review evaluates every evidentiary node in the argument tree with four probes. The first challenges **empirical sufficiency**, whether the trials covered the physical extremes of the operating envelope. Stopping trials with the loaded base on dry concrete, at a friction near 0.60, support the warrant that the base reaches its credible deceleration there. The restricted mode, however, admits floors that the site's inspection regime certifies only to 0.12, such as a lane polished by traffic, where the base may run at the restricted 1 m/s that section 15.2 declares, below the 1.09 m/s ceiling. If the trial set holds no loaded stops on such floors at that speed, the warrant for the restricted mode is unsupported, and a coverage audit finds in review the gap that service would otherwise find at a rack end. The oil film is no part of this probe, because the envelope refuses it outright and C3 treats it as the defeater. The second probe examines **domain transfer**, claims where simulation substitutes for hardware measurement. A person detector may score well on rendered aisles and still miss a person partly hidden behind stretch-wrapped pallets under the site's own lighting. Its benchmark recall is a proxy that scores curated frames, not the chain from a missed person to a commanded stop, so the reviewer asks for evidence on that chain from paired hardware-in-the-loop trials at the real rack ends.

The third probe attacks **structural independence**, asking whether the learned model and safety enforcer share a resource⁷ whose contention could delay sensor delivery. On the mobile manipulator such a delay would enter the 51.6 ms observation-age chain of C4, where no single component's timing test would show it. Stress tests and timing analysis under the placement's worst load must establish the interference bound; idle-bus unit tests cannot. The fourth probe identifies **unmonitored state drift**. Wear can change gearbox compliance⁸ and the response between motor current and link motion. The reviewer therefore asks for paired wear-state braking and contact traces before accepting the arm-stop premise of the door branch of C1 (section 16.6). A motor-current trip alone does not establish that the worn plant remains inside the case's measured force and stopping limits.

Reviewers operationalize these vulnerabilities by constructing explicit defeaters for each warrant. The hardest to see combine minor anomalies that each stay inside their own bands, the compound faults that section 15.3 derives from shared causes.

Consider the mobile manipulator's navigation camera with a light film of dust on its lens, a mount that has shifted within tolerance after its fasteners relaxed and now shakes as the base crosses floor joints, and a harness whose flexing at those joints raises the error rate on its serializer/deserializer (SerDes) link. Each check reads normal. The camera reports adequate visibility, the mount check registers within tolerance, and the SerDes error counters stay below their alarm. The anomalies cannot combine in age, because the permission path refuses any observation whose upper age exceeds the budgeted age plus the clock-conversion bound (section 8.7), so a frame that retransmissions delay past that threshold costs availability, not clearance. They combine in position. Dimmed contrast leaves the feature matcher on weaker matches. The mount blurs the frames it matches as it shakes and offsets every match by an angle the calibration still accepts. The frames the harness loses to retransmission lengthen the gaps between fixes, over which odometry drift accumulates. Each error stays inside its own tolerance, but all three land in the one localization allowance of the stopping budget, which no runtime check measures directly. That allowance is sized from the measured tail of localization error (table 16.4), not as the worst-case sum of its contributors' tolerances, which would give up signed speed to cover every contributor at the edge of its band at once. Trials run with a clean lens, a tight mount, and a sound harness never sample that coincidence, so the three errors together can exceed the allowance. The excess comes out of the spare (section 16.2), and the base travels on toward the rack end with less margin than the signed case assumes while every diagnostic reports normal operation.

The review also audits the execution integrity of the permission path, whether inference or background services can bypass, corrupt, or starve the enforcer (chapter 12) or human overrides (chapter 14), the latter covered by C6 of table 16.2. Below the proposal boundary (section 1.5), both keep non-preemptible priority over learned models, and the adversary asks whether a runaway

⁷ | **System-on-Chip Resource Sharing:** A system-on-chip integrates processor cores, accelerators, memory controllers, and peripheral buses on one die, so neural inference and a real-time monitor that share its interconnect and caches contend for them unless the placement partitions them.

⁸ | **Cycloidal Drive Dynamics:** Cycloidal reducers use an eccentric cam and rolling pins to reach high reduction ratios with little backlash. Rolling-contact wear raises their torsional compliance and adds phase lag between motor and link, which delays braking response and raises peak contact force.

Contrast: Algorithmic capability claims contrast with physical actuator saturation limits established in section 2.4.

policy thread, an accelerator-runtime exception, or a telemetry flood can saturate memory channels, lock a kernel spinlock, or brown out a rail. The reviewer accepts **permission path** independence, and the held-up rail that carries it on the mobile manipulator, only on physical fault injection, including bus saturation, feed removal, and a killed proposer task (F1), because C2 claims only the faults the permission path survives. Those trials are entries of the fault ledger, the set of signed fault records of section 15.7. If the permission path is implemented in user-space software rather than isolated hard real-time threads and dedicated hardware interrupt lines, the reviewer marks the link as broken.

When an attack succeeds, the case cannot be salvaged by editing claim descriptions or inserting arbitrary margins. The broken warrant is traced to its systems failure, an uncharacterized physical boundary, an inadequate dataset, or an unmodeled latency, and the defect returns to the owning subsystem. A motor-current observer that fails under gearbox compliance returns to the mechanical specification as wear limits and runtime compliance estimation, and bus contention that delays the enforcer returns to the compute architecture as memory partitioning and real-time scheduling. The case returns to the engineering team with its broken warrants named, and only updated boundaries, re-derived stopping distances, and renewed verification reopen adjudication.

Review must also recur, because a case accepted once can erode in service without any single change that would force a new one. The *normalization of deviance* names that erosion, in which anomalies that cause no harm come to be read as acceptable operating noise rather than as evidence against a warrant (Leveson 2011; Vaughan 1996) (16.1).⁹ On the mobile manipulator it would look like the compound defeater above spread over months: localization residuals that creep toward the edge of their tolerance, age refusals dismissed as nuisance stops, stops that end a little longer than the trials did, and enforcer trips cleared without a trace review. When thresholds are widened, margins trimmed for throughput, and alarms suppressed because nothing has yet gone wrong, the gap between the signed envelope and the machine's behavior widens until a routine disturbance exhausts what remains.



War Story 16.1: Challenger and experience as a boundary (1986)

Context: The Space Shuttle's solid rocket motors were assembled from segments whose field joints were sealed by a primary and a secondary O-ring (Presidential Commission on the Space Shuttle Challenger Accident 1986). No learned model appears in this chain. The adjudication failure it records is the one this chapter's verdicts exist to prevent, a launch approved for conditions that the evidence did not cover.

Mechanism: The ambient temperature at launch was 36°F, 15 degrees colder than any previous launch. O-ring resiliency falls with temperature, and a cold ring returns to shape too slowly to follow the joint gap as ignition pressure opens it. On the eve of launch, engineers at Morton Thiokol, the motor's contractor, recommended against launching below 53°F, the O-ring temperature of the coldest previous flight, whose joints had shown the worst blow-by to date. That limit was the edge of flight experience, not a qualified bound. NASA managers countered with a qualification temperature that applied to the propellant's bulk temperature rather than to the seals.

Impact: A combustion gas leak through the right booster's aft field joint, beginning at or shortly after ignition, weakened or penetrated the External Tank. The vehicle broke up about 73 seconds after launch, and all seven crew members died.

Response: The Commission attributed the failure to "a faulty design unacceptably sensitive to a number of factors," and temperature heads its list of those factors. It recommended that the joint be changed and that the new design be certified by tests "over the full range of operating conditions, including temperature."

Systems lesson: Thiokol management reversed its engineers' recommendation at the urging of NASA's Marshall Space Flight Center. Thiokol's vice president of engineering testified, "We had to prove to them that we weren't ready." The burden of proof had moved from showing the joint safe inside its evidence to showing it unsafe outside the evidence, where no data existed. A release verdict keeps the burden where the envelope trichotomy puts it. A condition

⁹ | **Recurring Erosion as Accepted Risk:** The Presidential Commission on the Space Shuttle Challenger Accident (Presidential Commission on the Space Shuttle Challenger Accident 1986) found that O-ring erosion and blow-by had recurred on earlier flights and that NASA and the motor contractor "accepted escalating risk apparently because they 'got away with it last time.'"

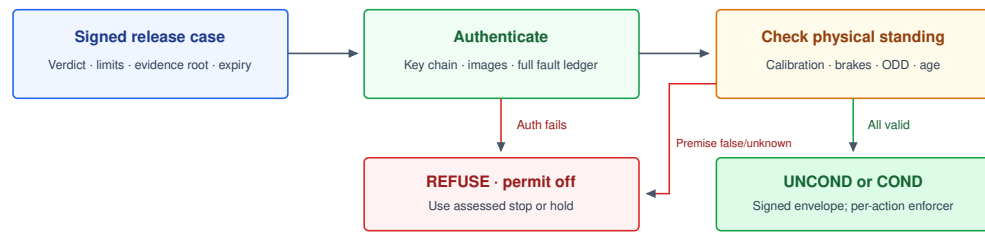


Figure 16.3: Fail-closed release pipeline: The signed verdict and evidence root pass key, image, and complete fault-ledger authentication, then current calibration, brake, and operating-envelope checks. Either check can refuse permission. Only a valid signed full or separately signed restricted case reaches per-action enforcement. Attribution: Textbook production team (CC BY-NC-ND 4.0).

outside the evidence, such as a seal temperature below all flight experience, is unknown, and the verdict refuses it.

Tempe’s takeover premise (section 1.6) was a condition of the same kind, and section 14.6 posed the operator-monitoring question it raises. A release that relies on such a premise must bind its response, refusal or a restricted case, together with its tested physical limits.

16.8 The Release Manifest

A release verdict needs a signed artifact that records the decision, physical conditions, and expiration. That artifact is the **release manifest**, the immutable record that binds the verification ledger to the signed verdict of the lead systems adjudicator and refuses motion whenever the running images or evidence no longer match its digests.¹⁰ It consumes the fault ledger through its root digest, and it binds three further records as evidence leaves of the signed argument graph: the authority record of section 14.8, the placement record of section 13.8, and the evaluation record of section 7.8. To these it adds the claim root, the argument digest, the image digests, calibration references, the verdict, its standing conditions, a drift budget, and an expiry. The image digests cover the policy weights, the enforcer’s logic in both of its enforcement-record configurations, the normal one and the restricted one a COND case would load (section 12.11), and the real-time kernel. A lifecycle budget of operating hours and cycles sits beside the calendar expiry, and the adjudicator’s signature covers the whole record. Evidence links must resolve to the reviewed record versions before permission is granted.

Figure 16.3 places authentication and physical-standing checks before any operating permit, and the gate runs them in a fixed order before the motor power stages are energized. Each check can only refuse. It first verifies the adjudicator’s key chain, which the hardware root anchors, and its revocation state, then the manifest signature, the expiry and lifecycle budgets, and every active image digest, so a changed weight file or enforcer bitstream refuses permission. It next authenticates the complete fault ledger, whose individually signed records enter the manifest as one root digest, together with the calibration references. A digest alone establishes neither record validity nor test adequacy, so the gate also checks each record’s signature and the ledger’s completeness against the reviewed evidence index. The standing conditions come last, and a known-false or unknown premise refuses as firmly as a bad signature while keeping its own logged cause. Only when every check passes does the gate load the signed envelope, and even then the enforcer still checks each proposed action. Table 16.5 gives the outcomes. Stale calibration is the case that separates them, since it refuses the full case and supports COND only where a separately signed restricted case covers exactly that state.

¹⁰ | **Cryptographic Manifest and Silicon Attestation:** OTP/eFuse storage can anchor a verification key or its digest. Mutable signed manifests and image hashes reside in protected storage; TPM PCRs accumulate measured boot state rather than directly storing a verdict. On failed validation, revoke permission and execute the assessed stop. A second software bank is an eligible fallback only if it has its own valid signature, release case, and current physical standing conditions.

Byte layout: The release manifest’s fields, types, and digests are given in section C.14.10.

Protocol: The gate’s step-by-step authorization protocol is given in section C.14.11.

Table 16.5: Release authorization truth table: A restricted mode is a distinct authenticated claim, never an unsigned downgrade of the full case.

Signed case	Evidence and standing checks	Restricted case covering stale calibration	Gate result
UNCOND	All valid and current	Not needed	Full <i>proposal</i> envelope under enforcer; UNCOND
COND	All restricted evidence valid and current	Included if calibration stale	Signed restricted envelope under enforcer; COND
UNCOND or COND	Fault-ledger root/signature mismatch, expiry, image mismatch, or unknown physical premise	Any	Permit off; REFUSE
UNCOND	Calibration stale	No	Permit off; REFUSE
Separately signed COND	Only the named calibration is stale; restricted evidence and limits valid	Yes	Restricted envelope; COND
REFUSE	Any	Any	Permit off; REFUSE

Every evidence leaf needs an identifier for its raw trace, instrument, calibration, configuration, and test script. A measured timing tail enters as a percentile, and a hard response claim needs the separately justified bound of section 4.4. The coworker’s handover criterion needs load-cell time series for the specified body-region surrogate and geometry. Linked records let an auditor repeat the test and identify where the claim stops applying.

The release manifest names the configuration and physical limits for which its verdict applies. A changed policy weight file, real-time priority, driver, or unverified component revision requires a new case or an explicitly approved equivalent. For the mobile manipulator, the signed wear limits on joint backlash and brake lining thickness come from measured stopping and load evidence. Refusal conditions of the operating envelope include a tote-rack load above its capacity, which the rack load cells monitor, and an unsurveyed floor, which the site procedure enforces (table 16.1). The manifest also signs the permission path’s isolated, held-up rail as a standing condition, sized in section 12.9, with a capacity self-test of its hold-up store gating the permit at every power-up. When a measured condition crosses a signed limit, the permission path revokes tracking at its specified detection and response time and commands the assessed fallback.

Field telemetry records torque residuals, loop timing, and enforcer trips for drift review, while the permission path checks the signed live thresholds at the cadence the plant requires. A brake response measured at scheduled inspection beyond the signed clearance budget, or stale calibration, revokes the current verdict, and reinstalling an earlier signed image cannot restore worn brakes. Unless a separately signed COND case covers that physical state, the plant uses its assessed stop and awaits maintenance and re-adjudication (table 16.5).

Every refusal in this chapter ends in the assessed stop, which presumes that a stopped machine is safe, a presumption that fails for a machine that falls when it stops (16.1).

Downstream: The premises a release manifest leaves open become entries in the residual-claims register of section 17.3.

Systems Perspective 16.1: When removing torque causes the harm

Question: Which verdict applies when removing torque causes the harm?

A braked base at rest stores no energy that can reach the coworker, but an upright humanoid stores its hazard in its own posture. No verdict, not even REFUSE, stands for the humanoid without a claim that its fall-arrest response works.

- **Safe torque off on an upright biped is a fall.** The drive function that removes motor torque (section 4.5) leaves the robot’s mass to drop from standing height, so no signed verdict can name torque removal as the humanoid’s assessed stop.
- **The fall-arrest response is itself an actuation.** Crouching, stepping, or lowering the robot to the floor under power is a commanded motion that needs its own claim, its own evidence, and a place in the fault set. A REFUSE verdict still depends on the part of the machine that keeps it standing.

- **Wear revokes the running permit.** Sole friction and joint backlash drift out of the limits the fall-arrest evidence assumed, and crossing a signed wear limit withdraws the permit to walk before the arrest response that the permit relies on stops working.



Checkpoint 16.3: Deriving a verdict from your own measurements

Table 16.4 carries no measured values, because this book has none. Fill it with values of your own, from trials on a machine you can test or from numbers you choose, and derive the verdict they support:

- ☐ **Graded values:** For each row, record your value and its source, and grade the premise measured, datasheet, illustrative, or unknown. A number you choose grades illustrative, and a manufacturer's rated value grades datasheet; measured means your own trial or a published measurement you cite.
- ☐ **Verdict derivation:** A COND case requires every premise it relies on to be graded measured, so a table in which any relied-on premise is graded otherwise still supports only REFUSE. Recompute the base's budget and spare from your values. If every relied-on premise is measured, name the envelope the COND case signs and the premise that keeps the normal mode unsigned. If any is not measured, derive the verdict the table *would* support were those values measured, and label that verdict hypothetical.
- ☐ **Artifact label:** Mark the filled table as your own work, whose values license no machine.

16.9 Fallacies and Pitfalls

Fallacy: *Statistical coverage substitutes for deterministic architectural warrants.*

A team presents millions of simulated hours and thousands of incident-free cycles as its release argument. The record supplies evidence about the tested conditions, but demonstrating the catastrophic failure rate a release claim names can require more physical exposure than any schedule allows (section 7.5). Simulation also retains the assumptions of its physical models, such as idealized Coulomb friction or infinitely rigid gearboxes. The release case must explain how architectural enforcement, such as an independent permission path that revokes motion and commands a validated stop, constrains harmful policy behavior, and it must provide evidence for that mechanism under its stated assumptions. Empirical performance and architectural warrants answer different questions; a large trial count cannot replace the missing argument that connects actuator authority to physical limits.

Fallacy: *A release verdict is a permanent certification of safety.*

A machine passes validation and enters production under a signed safety case. Over time, brake pad wear lengthens stopping distances, kinematic calibration drifts due to thermal cycling, and optical contamination degrades sensing. The software can remain unchanged while the physical assumptions supporting the release fail. The release manifest needs maintenance requirements and invalidation triggers tied to relevant lifecycle counters or measured drift, with defined restrictions when those conditions fail. A signature records responsibility for a bounded release decision; it does not make the tested body's hardware capabilities permanent.

Pitfall: *Relying on unstated environmental assumptions to support a safety claim.*

A team verifies the base's stopping performance in well-lit aisles on clean floors, and its case states the floor's friction but never its lighting. A dim aisle then costs service through the image age that a runtime check happens to see (section 16.3) and costs clearance through weaker feature matches whose localization error no check sees, against an allowance the well-lit trials sized (section 16.7). The mechanical brake performs exactly as tested and every diagnostic reads normal while the spare in the stopping distance shrinks. The safety case must state the environmental conditions on which its claims depend and explain how operation remains within them. Where monitoring can detect a

violation, it must trigger the defined restriction or fallback; an unstated assumption supplies neither a testable release condition nor a usable runtime boundary.

16.10 Summary

Production release binds a claim-argument-evidence case to a measured operating envelope, a signed configuration, and a permission path that can revoke motion. The signature establishes the authorized case; runtime checks determine whether its physical premises still hold.



Key Takeaways: Deployment release and defensible safety cases

- **The envelope bounds the claim:** A release claim covers only the speeds, payloads, floors, clearances, and contact cases for which evidence was gathered, and every dimension of the envelope needs a runtime monitor or a site procedure that keeps the machine inside it.
- **The weakest branch decides:** A claim-argument-evidence tree is not an average. High accuracy in one branch cannot offset a missing warrant in another, so the adjudicator looks for the single premise whose failure would permit harm.
- **An unknown premise refuses:** Unknown envelope membership is treated as outside the envelope, and a trial marked `INVALID` or `UNOBSERVABLE` supplies no evidence. On this book's illustrative inputs every claim carries such a premise, so the mobile manipulator's verdict is `REFUSE`.
- **Refusal is a deliverable:** A refusal names the measurements that would lift it, such as loaded stopping trials and enforcer timing under the worst load, and the restriction each result would leave, including the oil film, which no measurement on the machine closes.
- **No verdict moves the proposal boundary:** Under `UNCOND` and `COND` alike the policy only proposes and the enforcer admits each action. A restricted mode needs its own signed case, never an unsigned downgrade.
- **The manifest expires with its evidence:** The release manifest binds the evaluation, placement, authority, and fault records by digest and refuses motion on any mismatch. Wear and drift revoke it, and a software rollback cannot restore a changed plant.

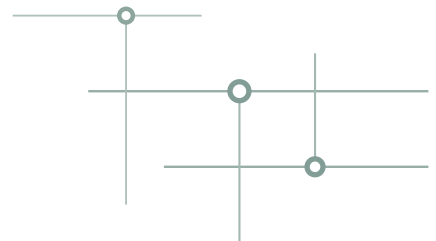
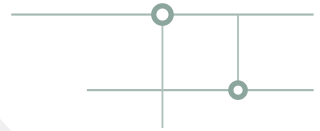
The fourth law (section 1.7), stated as principle 4, grants authority on recorded evidence and lets it expire with that evidence, and the standing conditions are where that expiry becomes mechanism. Each pairs a premise with a detector, a cadence, and a response, so the loaded stopping trials that would support a conditional verdict for the mobile manipulator become, once signed, the evidence whose lapse ends it. On this book's records that grant never arrives, because each of the six claims rests on a premise left illustrative or unknown. The manual lease limited an operator's authority to the age of its evidence (section 14.3), and precommitment tied a test's verdict to a rule fixed before the evidence arrived (section 15.5). Release is the widest grant the book makes, and even there admission stays below the proposal boundary.



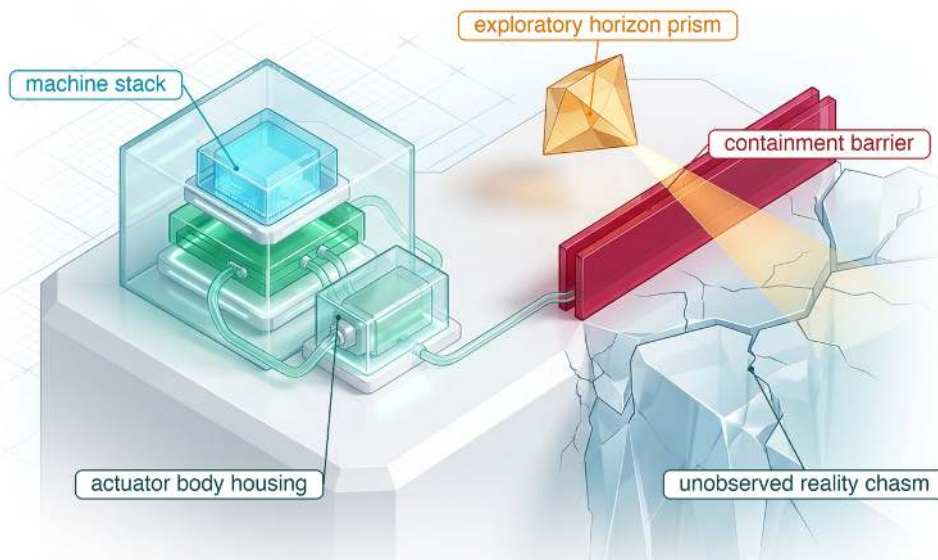
What's Next: From a refused release to what the evidence supports

Which premises can no evidence close? Claim C3 holds the plainest case. No measurement on the machine can exclude the oil film behind it, and more trials with the same sensors cannot change that; only a new transducer, a floor-film detector, could. A conditional release therefore keeps that condition outside the signed envelope rather than settling it. The conclusion, chapter 17, returns to the question the book opened with, answers it with what the evidence supports, and enters each premise this release left open, the oil film first, in a residual-claims register with the test that would close it.

CONCLUSION



The Epistemic Frontier



- 17.1 Epistemic Limits
- 17.2 What the Method Cannot Establish
- 17.3 A Residual-Claims Register
- 17.4 Failures Without Detectors
- 17.5 What Would Change These Answers
- 17.6 Generalizing to Unfamiliar Systems
- 17.7 What the Four Laws Leave Open
- 17.8 Fallacies and Pitfalls
- 17.9 Summary

Purpose

How much authority can a machine hold when evidence cannot settle every premise its safety rests on?

The opening chapter asked what guarantees a machine must enforce before a learned proposal commands physical work. The preceding chapters established the architectural answer: an independent real-time permission path deciding on bounded-age evidence, operating strictly within margins unconsumed by latency, and backed by a verified stopping fallback. Yet every formal guarantee remains conditional on unmeasured physical premises.

A braking certificate assumes a minimum surface friction coefficient that onboard sensors cannot measure prior to contact; a barrier filter assumes uncorrupted state estimation and nominal actuator torque; and a deployment release assumes environmental bounds tested across finite exposure. When rare hazards elude empirical observation, engineering integrity demands naming each unverified premise and restricting operational authority rather than assuming safety by default. Because the causal boundary never relaxes when evidence runs out, autonomous authority must terminate precisely where physical evidence ends.

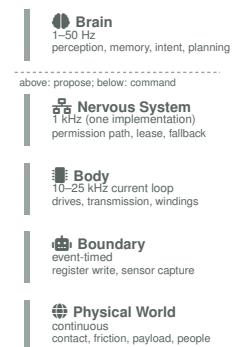


Figure 17.1.: The five-level machine stack across all levels.

Prerequisite: The four laws of section 1.7, which this chapter closes on.



Learning Objectives

- Assess which safety claims exceed the evidence available for a specific physical machine
- Evaluate whether a detector can identify a hazard before effective intervention becomes impossible
- Construct residual-claims register entries, each with a restriction, an owner, and a sized closure test, from a release's open premises
- Classify an open premise by the missing observable, evaluation, theory, or scope that would close it
- Trace the four bedrock laws through an unfamiliar machine and state which premise of each the evidence leaves open

17.1 Epistemic Limits

EVERY engineering method has an epistemic boundary where its formal guarantees expire. In physical AI, that boundary is not merely an algorithmic shortfall; it is a physical confrontation between compute latency, sensory observability, and mechanical inertia. Scaling the parameter count of foundation models can yield extraordinary semantic breadth, but it cannot bend the laws of motion: as networks grow, inference latency expands the reaction corridor, trading agility for deliberative depth.



Definition 17.1: Embodied latency scaling law

Embodied latency scaling law is the physical constraint dictating that scaling neural policy parameter count yields diminishing operational safety returns unless inference latency decreases proportionally, because physical reaction distance $d_{\text{react}} = v \cdot (t_{\text{inference}} + t_{\text{actuation}}) + \frac{v^2}{2a_{\text{max}}}$ scales linearly with computational delay.

1. **Significance:** Establishes that intelligence in physical systems cannot be scaled purely by adding model parameters, because computational delay directly degrades dynamic stability and spatial stopping clearance.
2. **Distinction:** Unlike compute scaling laws in static language modeling where latency affects only token generation speed, embodied latency scaling directly couples computational delay to mechanical reaction time constants ($t_{\text{react}} \geq t_{\text{inference}} + t_{\text{actuation}}$).
3. **Common pitfall:** Deploying multi-billion parameter foundation models directly in the closed-loop control path under the assumption that greater semantic reasoning compensates for an unbudgeted response delay.

This tension, codified in 17.1, illustrates why parameter scaling alone cannot solve real-time safety. Furthermore, as governed by Ashby's Law of Requisite Variety (Ashby 1956), a controller cannot regulate disturbances that its sensory architecture cannot perceive. When an unmodeled disturbance enters the physical plant, no amount of latent computation can compensate for missing physical evidence.

A pallet jack with a weeping hydraulic line crosses one aisle of the site where the warehouse mobile manipulator works, and it leaves a clear film of fluid on the concrete. The machine's stopping budget assumes that the floor can supply the friction its credible deceleration needs, and on dry concrete it can. Under the film it may not, and neither the navigation camera nor the lidar can tell the film from dry floor. The permission path can meet every deadline, the chunk lease can expire on time, and the brake can engage on schedule while the stop still runs past the rack end, because the term that failed is a premise that no sensor on the machine reports. The missing evidence is the local friction floor before braking begins.

The release decision of section 16.3 already met this premise. Its claim C3, braking traction, was refused on the book's own evidence and handed to this chapter as an open premise. The

machine's credible deceleration is the smaller of what its brakes deliver and what the floor supplies, $a = \min(a_{\text{brake}}, \mu g)$. The 2 m/s^2 credible deceleration that section 2.3 set needs a friction coefficient of at least 0.204. Dry concrete, at a μ of 0.60, supplies far more, so the brakes bind. The clear film, at $\mu = 0.05$ (both coefficients illustrative; see the Reader Guide), supplies only 0.49 m/s^2 , and friction binds. At the 1.3 m/s aisle speed, the finished budget of table 12.1, 997.4 mm on dry floor, grows on the film to 2.95 m against a 1.10 m clear distance. The friction premise did not become open when the film appeared. The normal mode that section 12.5 chose rested on that friction floor on every aisle, the dry ones included, and nothing had measured it there either.

Every engineering method has a boundary where its guarantees expire. The physical wall of section 1.7 is one such boundary, because a larger learned model cannot reduce the travel that its own delay costs the machine. Ashby's law of requisite variety (Ashby 1956) names another, since a controller cannot regulate a disturbance that its sensors cannot perceive. On the mobile manipulator the oil film is such a disturbance. The arm's contact limit at the door latch rests on a premise of the same kind, a latch that holds at every force below the arm's contact ceiling, and corrosion or wear can lower the latch's yield force below that ceiling while the joint torque sensors read exactly what they would read against a sound latch. A chemical flow regulator that holds line pressure below its limit depends on an undrifted pressure transducer, and deposition on the diaphragm can drift that reference until the loop regulates against a phantom reading while the true pressure exceeds the limit.

In each case the machine can leave its assumed conditions while its telemetry remains nominal. Successful trials cannot close such a gap, because the exposure wall that section 7.5 derives places a rare-hazard rate beyond any practical failure-free campaign. The safety case must name the missing premise, restrict operation while it remains open, and state what evidence could close it. For each open premise, that discipline asks what the machine can measure, what its actuators can still prevent, and which claim must remain open.

17.2 What the Method Cannot Establish

The mobile manipulator's release claim for the aisle is a chain of links, and which links it needs depends on the hazards and the protection chosen. The argument must establish the operating domain, measured actuator and stopping bounds, component contracts, and independent permission for learned proposals (section 1.5). Hazards controlled by runtime detection also need measured detector coverage and timely monitoring. Hazards controlled by passive limits or operating restrictions instead need evidence that those measures hold over the declared load, wear, and exposure envelope. Generating extensive evidence for one link cannot compensate for an unsupported premise in another.¹

Each premise of a safety case carries one of the Reader Guide's five statuses (established by measurement or derivation, assumed, decided by accountable judgment, unsupported, or outside the threat model). The envelope trichotomy (16.2) places a premise that a runtime monitor checks as known-true, known-false, or unknown. The premises that bound the evidence here are ones no monitor now in service can check, so none can become known-true, and the release, not the monitor, must restrict operation around them. For the base, the stopping-distance claim relies on an assumed friction coefficient between wheel and floor that cannot be measured before braking begins. For the arm, the contact argument depends on a declared floor on the strength of the latch and the objects it handles, a floor decided by engineering judgment about structural yield, while the latch's present strength stays unmeasured between proof tests. For the flow regulator, the pressure claim assumes a transducer whose zero point has not drifted, a premise that its own reading cannot check. Naming these premises explicitly does not weaken the release record. It establishes the physical envelope within which the release verdict is justified, and outside of which the verdict ceases to apply.

Separating measurement from judgment is fundamental to systems engineering discipline. Empirical testing and statistical characterization can estimate physical exposure, quantify sensor noise, and bound parametric uncertainty, but no measurement can choose an acceptable **residual risk** or decide which individuals in an operational environment must bear that risk. Those choices are accountable human decisions that must be recorded as explicit judgments rather than disguised as empirical findings. In the same way, the **threat model** bounds the validity of the record. Empirical testing across nominal fleet operations and accidental component faults provides no evidence regarding

¹ | **Popperian Falsification:** Karl Popper formalized the asymmetry of empirical induction: no finite sequence of safe observations can prove universal safety, whereas a single counterexample refutes it. Operational fleet hours establish survival rates strictly under observed stationary conditions, not out-of-distribution guarantees. Extrapolating zero observed incidents to unbounded environments produces an illusory sense of safety that fails upon encountering unmodeled edge cases.

intentional adversarial attacks. A safety record demonstrating that a vision pipeline handles sensor noise, lens blur, and partial occlusions provides no statistical support for claims about resilience against adversarial optical projection, model parameter tampering, or intentional denial of service across the internal communication bus.

Take as the system target an illustrative 10^{-9} h^{-1} , a rate that no standard assigns to this machine and that, by the exposure wall of section 7.5, operating hours alone cannot demonstrate.² The remaining question is what the permission path can add to the claim when it acts between proposal and actuation as the safety shield of section 12.8, and which of its premises need evidence of their own.

A shield argument is *conditional*, not a multiplication applied to observed system incidents. A fleet's incident count cannot stand in for a proposal rate, because the hazardous proposals a shield prevented never appear in it. For a defined hazard class h , let $\lambda_{\text{proposal},h}$ be the rate of hazardous proposed actions before permission, and let $c_h = P(\text{intercept} \mid \text{hazardous proposal}, h)$ be measured interception coverage for that class. With a separately bounded hardware and common-cause term λ_{other} , a model may use equation 17.1:

$$\lambda_{\text{system},h} \leq \lambda_{\text{proposal},h}(1 - c_h) + \lambda_{\text{other}} \quad (17.1)$$

The relation requires a valid proposal oracle, timely detection, actuator authority, and a justified treatment of correlated failures. It gives no credit for an oil film or a weakened latch that the shield cannot observe, and no release follows from the arithmetic until each premise is supported.



Napkin Math 17.1: A conditional shield calculation

Problem: One hundred robots keep a fully adjudicated proposal log for one 90-day cycle, judged by a validated counterfactual oracle, and the log records no hazardous proposal. If a named shield's interception coverage for that hazard class were independently established at $c = 0.9999$, would the modeled residual meet the 10^{-9} h^{-1} system target?

Proposal bound: Under a stationary Poisson model, zero hazardous proposals in 2.16×10^5 proposal-hours give a one-sided 95 percent upper bound of $1.387 \times 10^{-5} \text{ h}^{-1}$.

Residual: With hardware and common-cause residual set to zero for this calculation, equation 17.1 gives $1.387 \times 10^{-5} \text{ h}^{-1}$, **above** the target. The coverage the target requires is at least 0.9999279.

Required proposal evidence: At $c = 0.9999$ the proposal-rate bound must fall to $1 \times 10^{-5} \text{ h}^{-1}$. Zero adjudicated hazardous proposals would then require 3.00×10^5 proposal-hours, or 124.822 days for 100 units, 34.8 days beyond the cycle.

Systems insight: A shield changes the system claim only for hazard classes it can detect and intercept in time. Proposal frequency, conditional interception coverage, hardware faults, and common-cause failures need separate evidence. Oil or material failure outside the shield's sensing coverage stays in the residual-claims register.

When empirical fleet exposure fails to breach the astronomical exposure wall, system safety cannot be rescued simply by retreating into offline mathematical proof (figure ??). Formal safety verification confronts an equally unyielding epistemic limit: the **curse of dimensionality** across continuous state spaces. Classical Hamilton-Jacobi (HJ) reachability analysis solves a level-set partial differential equation across a discretized grid of the state space, providing an exact backward reachable tube and control-invariant safe set. However, its memory and computational complexity scale exponentially as $\mathcal{O}(N^d)$, where N is the grid resolution per axis and d is the state-space dimensionality. At $d > 6$, HJ reachability collides with an impenetrable "Grid Wall": it tractably solves a 3D Dubins car or 4D kinematic vehicle in seconds, but requires centuries of compute and exabytes of memory for a 7D kinodynamic vehicle, a 12D quadrotor, or a 30D mobile manipulator. Sampling-based statistical verification (such as adaptive stress testing and scenario optimization) breaks the exponential grid wall, scaling polynomially as $\mathcal{O}(d^2)$ up to 50 dimensions or more; yet it purchases tractability by surrendering deterministic safety guarantees, yielding only probabilistic Probably Approximately Correct (PAC) bounds $(1 - \epsilon)$ that remain inherently blind to measure-zero adversarial singularities.

² | **Evidence Across Updates:** An updated model may alter the proposal distribution. Earlier data can be pooled only with a justified configuration-equivalence and exposure argument; an update does not automatically erase every observation. The exposure wall's constant-rate model also requires a defensible treatment of shared fleet conditions and correlated faults.

Modern neural barrier certificates (NBCs) synthesized via branch-and-bound verification tools (such as α, β -CROWN) establish a promising intermediate frontier, verifying forward invariance across 12D to 30D robotic systems through convex relaxations ($\mathcal{O}(d^k)$). Even so, when state dimensionality scales to coupled multi-robot swarms ($d \geq 48$), verification times again escalate toward intractability, demonstrating that formal methods replace exposure time with state-space dimensional ceilings rather than eliminating epistemic limits.

Each of these premises, from friction under a film to a shield's coverage for a hazard class it may not observe, ends the same way, named by the release and left unsettled by its evidence. A release that stops at naming them has only listed its caveats. What it needs is a record that turns each such premise into a restriction with an owner and a test that can end it.

17.3 A Residual-Claims Register

Refusing claim C3 did not remove the friction premise from the machine, and an unproven premise remains whether or not a safety report records it. The **residual-claims register** consumes the release manifest of section 16.8, taking from it every premise the release left open or unsupported. An open entry restricts authority until its predeclared closure test passes. The register maps each such premise, and every unmonitored invariant, to its missing physical observable, its active containment or operating restriction, an accountable human owner, and a predeclared closure test. Without these four components, an unresolved premise is an unwritten risk, not a register entry. The register is where principle 4 reaches the premises that evidence could not settle, holding authority to a restriction wherever the supporting evidence is absent.

Each entry links the unsupported claim to the specific unmeasurable physical observable. In a machine that moves mass, a learned policy fails to settle a safety property because sensors cannot transduce the relevant environmental state, the computing substrate cannot process the measurement within the control deadline, or the physical interaction cannot be observed before contact occurs. An engineer must define the **missing observable** with physical precision rather than statistical abstractions. Characterizing an unknown failure as an "out-of-distribution latent state" provides no actionable signal for the permission loop to trigger a physical intervention. By contrast, stating an unmeasured surface friction coefficient, an undetected optical flare across a camera lens array, or an unobservable micro-fracture in an actuator housing identifies the exact physical deficit. Naming the missing observable establishes the boundary where learned inference stops and physical uncertainty begins.

The oil film gives the mobile manipulator its first entry. The premise is that every aisle floor supplies a friction coefficient of at least 0.204, the value its credible deceleration needs. The missing observable is the friction beneath a clear film, which the navigation camera and the lidar cannot tell from dry concrete. The stop from the aisle speed that fits the clear distance on a dry floor overruns it on the film, so a speed limit the entry imposes can hold only on floors whose friction is measured or controlled.

An open entry requires an **operational response**. For the aisles, the most the entry permits is a restricted mode of 1 m/s on floors under the site's inspection regime. An inspected floor supports $\mu \geq 0.12$, which gives a credible deceleration of 1.18 m/s² and a ceiling of 1.09 m/s, so the restricted mode sits below its ceiling.³ A route that the inspection regime does not cover is excluded, or the machine is withheld from it.

The restricted mode needs an owner able to keep its physical premises true. Site operations owns this entry through its spill procedure for pallet-jack routes, which sets the inspection interval and names the condition that withdraws permission. The signed speed limit on the permission path enforces the restriction, but a claim about friction cannot be maintained by a speed setting alone.

Every entry needs a **closure test** declared before collecting data. For the film, the test must qualify a new transducer, because no measurement by the camera or the lidar can exclude a clear film and more trials with them could not close the entry. The test is a floor-film detector whose miss rate is bounded at 0.1 percent by 2,995 failure-free trials on seeded films (section 7.5); until it passes, the restriction holds. A smaller campaign falls short. Zero misses in 1,000 independent trials bound the miss probability only at 0.002991 with 95 percent confidence. The detector must also report the film early enough that detection, permission, and braking together fit the clear distance, and the trial distribution, detection distance, latency, false alarms, and friction after detection must be declared

³ | **Velocity Kinetic Derating:** $E_k = mv^2/2$ decreases quadratically with speed. Stopping distance still depends on a validated credible deceleration and total response delay; a speed register alone cannot compensate for arbitrarily low friction.

in advance. A wheel-slip reflex can limit later motion but cannot establish friction before the wheels reach the film.

The same aisle carries a second entry. The stopping budget protects a static obstacle. A person who keeps walking toward the machine through the whole stop adds distance that the budget never charges (section 9.6), and section 12.5 found that charging it would cap the base at this rack end far below its aisle speed. The premise is that no person approaches the rack end during a stop, and the missing observable is presence in the cross aisle the rack hides. The site's crossing rule, a floor-marked crossing plus a rack-end presence sensor, is the restriction, and site operations owns it. The closure test qualifies the presence sensor by the same zero-failure sizing.

The pick station adds a third entry, and this one belongs to the arm. Its premise is that the object the arm grasps, and later hands to the coworker under `accept_item` (section 14.2), is the object the task names. Section 10.3 showed how that premise fails. A thin-walled red glass jar beside the red mug on the takeaway conveyor draws a confident grounding, and the wrong target passes every geometric check the planner and the permission path apply. The missing observable is identity and fragility, which neither the wrist camera nor the grounding model can establish before contact.

The restriction on this entry has two parts. Site operations admits to the conveyor only items on the pick station's declared item list, which holds no fragile look-alike, and the station admits a grasp only when the pick window holds exactly one object, so that a second object refuses the intent and sends the item to the coworker for a manual pick. Singulation makes the grounding unambiguous for the items it admits but supplies no identity information, and a mis-routed jar that arrives alone is contained only by the feed procedure. That residual is why the entry stays open. A grip-force ceiling set for the most fragile look-alike would need a crush limit for every item that might arrive, itself an unmeasured premise, so the register does not rest on one. Site operations owns the feed list and the singulation rule jointly with the task profile. The closure test is a qualified identity check at the station, a tag read or a scale-and-code check, whose false-accept rate is bounded at 0.1 percent with 95 percent confidence by the same 2,995 failure-free trials, run here on seeded look-alikes.

Table 17.1 sets the three entries side by side. Each row carries the four components the register requires, so any entry can be audited without the prose around it.

Table 17.1: Residual-claims register for the aisle and the pick station: The three open premises behind the mobile manipulator’s aisle speed and its pick. For each, the register names the missing physical observable, the hazard it permits, the operating restriction and its accountable owner, and the predeclared closure test that would retire the restriction.

Claim and missing state	Hazard mechanism	Active containment and owner	Predeclared closure test
Braking traction: every aisle floor supplies $\mu \geq 0.204$; friction beneath a clear film is unobserved.	At $\mu = 0.05$ the credible deceleration falls to 0.49 m/s^2 , and the finished budget at 1.3 m/s grows to 2.95 m against a 1.10 m clear distance.	1 m/s on inspected floors ($\mu \geq 0.12$, ceiling 1.09 m/s); uninspected routes excluded. <i>Owner:</i> site operations, through the spill procedure for pallet-jack routes.	A floor-film detector with a miss rate of at most 0.1% at 95% confidence, from $2,995$ failure-free seeded-film trials, plus timing and clearance tests; the restriction holds until it passes.
Walking person: no person approaches the rack end during a stop; presence in the hidden cross aisle is unobserved.	A person walking at 1.6 m/s through the whole stop would cap the base far below its aisle speed at the rack end (section 12.5).	The crossing rule, a floor-marked crossing plus a rack-end presence sensor. <i>Owner:</i> site operations.	Qualify the presence sensor by the same zero-failure sizing, $2,995$ failure-free trials for a 0.1% miss bound.
Object identity: the object grasped is the object the task names; identity and fragility before contact are unobserved.	A confident grounding to a thin-walled glass jar beside the red mug passes every geometric check, and a grip the ceramic mug needs can crush the glass.	Feed control admits to the conveyor only items on the station’s declared list, none of them a fragile look-alike; singulation admits a grasp only with one object in the pick window. An off-list item arriving alone is contained only by the feed procedure. <i>Owner:</i> site operations, jointly with the task profile.	A qualified identity check at the station (tag read or scale-and-code) with a false-accept rate of at most 0.1% at 95% confidence, from $2,995$ failure-free seeded look-alike trials; the restriction holds until it passes.

A release pipeline records each entry in a fixed schema, and its fields, units, and invariants appear with the other record layouts in section C.14.12.

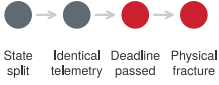
All three entries close only through a qualified detector, a floor-film detector for the first, the rack-end presence sensor for the second, and an identity check at the station for the third. Each closure test therefore has to show that the qualified detector reports in time, the two aisle detectors early enough for the permission path to stop the machine and the identity check before the station admits the grasp, and that timing requirement has a physical limit of its own.

17.4 Failures Without Detectors

A runtime detector is itself a physical subsystem. To operate as a **practical detector**, it must satisfy three coupled parameters: comprehensive coverage across the failure distribution, a bounded false-alarm rate, and a detection latency strictly shorter than the physical system’s time-to-harm. At the 1.3 m/s aisle speed, the finished stopping budget of table 12.1 leaves the mobile manipulator 102.6 mm of its clear distance unspent, which the machine covers in 78.9 ms (section 14.3). A detector for a person at the rack end that adds more than that to the observation path is not a detector for that hazard. The floor-film detector faces a stricter form of the same test. Its detection distance, less the travel during its own latency, must cover the dry-floor stop to the film’s edge, because a report that arrives once the wheels are on the film leaves a stop of 2.95 m against the 1.10 m clear distance. A detector that reports later is a post-incident logging mechanism. Producing an accurate classification label after crossing an irreversible threshold generates diagnostic evidence but cannot serve as a runtime safeguard.

The fundamental limit of runtime detection arises from observational indistinguishability. Consider two physical states of the world, S_A and S_B . In state S_A , a commanded actuator trajectory $u(t)$ is safe. In state S_B , the identical trajectory $u(t)$ causes structural damage or human injury, requiring the permission path to intervene and command a stop. The dynamics of the body impose a hard deadline t_{deadline} , determined by actuator response time and system momentum, after which physical contact or mechanical yield cannot be prevented. If the telemetry history $y(t)$ provided by onboard instrumentation is identical for S_A and S_B across the entire interval $t \leq t_{\text{deadline}}$, no algorithm operating on those observations can reliably distinguish the benign state from the hazardous one. Identical traces rule out a state-specific response before that deadline. A conservative action

chosen for both states or a passive mechanical limit may still protect them, if its own clearance and force assumptions are valid.



Identical pre-contact telemetry prevents software intervention before irreversible physical damage occurs.

Definition 17.2: Observational indistinguishability

Observational indistinguishability is the physical condition where two distinct world states, one nominal (S_A) and one hazardous (S_B), generate identical pre-harm sensor telemetry ($y_A(t) \equiv y_B(t)$) up to the system's physical action deadline ($t_{\text{deadline}} = t_{\text{harm}} - t_{\text{act}}$).

1. **Significance:** Without distinguishing physical energy prior to the action deadline, timely algorithmic differentiation is physically and mathematically impossible. Scaling neural model parameters, increasing dataset size, or accelerating inference compute cannot separate states that emit zero distinguishing physical energy before the time-to-harm.
2. **Distinction:** Unlike epistemic uncertainty or high classifier entropy (which signals that the model knows it lacks evidence), observational indistinguishability often pairs with high output confidence and near-zero entropy because the neural network maps corrupted observations into nominal latent manifolds.
3. **Common pitfall:** Relying on software anomaly detectors or VAE reconstruction loss to catch detectorless degradation. When sensor physics blinds the transducer, as piezoresistive crystal creep does, or the hazardous state lies outside the sensed path, as a weakened latch does for the joint channel, the anomaly detector consumes telemetry that carries no trace of the fault and fails silently.

In a learned perception-action policy the same limit appears as shortcut learning. A vision-language-action (VLA) model (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) or an imitation policy (Zhao et al. 2023) minimizes its loss with whatever correlations the demonstrations contain, and where friction, material strength, or payload inertia goes unobserved, it can learn non-causal visual features, such as floor highlights or the angle of the lighting, as proxies for them. Under distribution shift those proxies fail, yet the encoder can still map the unfamiliar observation into the region of latent space it knows, so the policy acts with high confidence and little ensemble disagreement. An uncertainty estimate computed from that same latent representation, whether a Mahalanobis distance, a dropout variance, or a reconstruction residual, inherits the same blindness and cannot serve as an independent detector. The Brain cannot detect its own shortcut failure, which is why physical containment rests on the permission path below the proposal boundary rather than on the model's self-assessment.

Sensing hardware shows the same limit over longer durations. The flow regulator's pressure transducer drifts as deposits build on its diaphragm, and a transducer whose zero has drifted low reports the same trend as a line that genuinely runs at the lower pressure. A single channel reports one scalar, and no algorithm on that signal can tell a changed process from a degraded calibration. Without an independent reference transducer or a proof test of the zero point, the drift is undetectable at runtime.⁴

The weakened latch at the door is the arm's case of the same limit, and a faster one. Until the latch yields, the joint torque sensors read the same torques against a weakened latch as against a sound one, because both carry the same contact force, and the channel holds no trace of the strength that remains. The yield is irreversible, and no command issued after it restores the latch, so a faster classifier on the joint torque channel protects nothing. The defensible responses supply what that channel lacks. A proof test establishes the latch's strength at a declared interval, and it covers the interval only if corrosion and wear cannot carry the yield force below the contact ceiling between two tests, a rate that the maintenance record, not the torque channel, must support. A passive compliant element in the wrist instead keeps the contact force below a declared floor on latch strength over a mechanically limited stroke, so the premise no longer depends on the latch's state. Equation 17.2 gives that element's stiffness ceiling, with F_{allow} set by the declared floor, which holds only if the element carries the entire contact load, the stroke cannot pass its stop, and wear and impact tests support the model.⁵

⁴ | **Piezoresistive Transducer Drift:** Mechanical stress relaxation, surface oxidation, and temperature cycles alter the piezoresistive coefficient of silicon strain gauges over extended campaigns. Monolithic software filters cannot separate slow baseline voltage drift from true process pressure increments, creating an unobservable state transition. Preventing overpressure therefore requires independent physical zero-point proof-tests or dual-dissimilar sensing architectures.

⁵ | **Contact Boundary:** Rigid impact and friction transitions can invalidate a smooth free-motion model. Peak contact force and component damage require measured stiffness, damping, geometry, and actuator response; neither a controller nor a passive part alone supplies a universal bound.

$$k_{\text{spring}} \leq \frac{F_{\text{allow}} - F_{\text{preload}}}{\Delta x_{\text{stall}}} \quad (17.2)$$

The Deepwater Horizon blowout failed in the opposite way from the Tempe collision of section 1.6, where the sensors had detected the pedestrian but a designed suppression withheld the braking that state warranted. In the blowout the protective device acted, but on a state that no sensor reported. A faster trigger or broader authority would not have changed the outcome, because no measurement separated the state the protection was qualified for from the actual one. A timelier response rule and a restored redundant brake path address a failure like Tempe's. Only qualification and sensing extended to every state the protection may meet can repair the other kind, the detectorless failure this section describes and the one the oil film poses on the aisle.

Prerequisite: Failures undetectable by sensory telemetry challenge the observation contracts formulated in section 8.7.

① War Story 17.1: Deepwater Horizon blind shear ram failure (2010)

Context: On April 20, 2010, the Macondo well in the Gulf of Mexico blew out beneath the *Deepwater Horizon* drilling rig. The last physical containment safeguard was the subsea blowout preventer (BOP), a stack of hydraulic valves and blind shear rams weighing approximately 400 tons and anchored at the wellhead about 1,500 m below the surface (U.S. Chemical Safety and Hazard Investigation Board 2014).

Mechanism: The explosions likely activated the BOP's automatic deadman system, which was meant to close the blind shear ram, shear the drill pipe, and seal the well. By then the drill pipe inside the BOP had buckled off center. The pressure difference between the inside and outside of the pipe had placed it in effective compression, a buckling mechanism that earlier investigations had not identified. The BOP was not designed to cut off-center pipe. The ram punctured and partially severed the pipe but could not close fully and seal the well.

Impact: Flow through the punctured pipe resumed rapidly. Eleven crew members died, the rig sank about 36 hours after the explosions, and approximately 5 million barrels of oil spilled into the Gulf of Mexico (U.S. Chemical Safety and Hazard Investigation Board 2014).

Response: The U.S. Chemical Safety Board recommended changes to blowout-preventer design, testing, and oversight. Its investigation also found two instances of miswiring and two backup-battery failures in the controls for the blind shear ram.

Systems lesson: The pipe's position inside the BOP was a state that no sensor reported, and the ram acted on it anyway ($y_A(t) \equiv y_B(t)$ for the centered and the buckled pipe). Its qualified cutting geometry did not cover the actual condition. Protection has to be qualified across every physical state it may meet, including pipe position, and across the independent control faults the CSB documented.

When a failure mode lacks a practical detector, the remaining responses are to limit actuator authority under a validated force or clearance model, restrict the operating domain, add independent sensing, or schedule proof tests. A force limit rests on that validated model, because ISO 10218 and ISO/TS 15066 apply to their industrial robot scopes and supply no universal force limit for this machine. Active sensing is itself actuation. A small dither of the wheels, an active probe for the film, measures traction only where the wheels already are, so it cannot close the friction entry. Because the probe must not slip beyond the reserved clearance, the independent permission path must admit each probe command within a separately validated energy, contact, clearance, and timing envelope and choose a state-matched fallback when the needed bound is unavailable. If none of these measures can support the claim, operation must be withheld.

The safety case must document these gaps. Figure 17.2 marks where the argument runs out. The residual-claims register records the detector's blind region, defining the set of physical states that produce indistinguishable observations, along with the maximum exposure duration permitted between physical proof tests. Treating an undetectable failure as a solved runtime problem undermines the validity of the safety case. Preserving the exact boundary where sensing stops forces the engineering organization to rely on physical constraints rather than software optimism.

Each detectorless mode still leaves a defensible response, and table 17.2 sets four of them side by side.⁶

⁶ | **Motor Thermal State:** Winding resistance and magnet flux can change with temperature. Whether that changes braking authority depends on the drive, current limits, load, and measured thermal model; a separate holding path may be required for gravity-loaded axes.

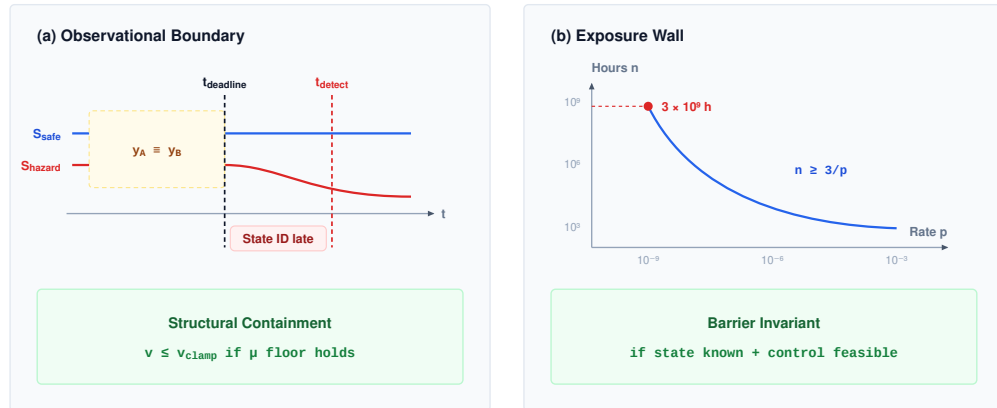


Figure 17.2: Two limits on the claim: Panel (a) shows states with identical pre-deadline telemetry. A speed clamp is useful only with a supported braking and clearance envelope. Panel (b) shows the zero-event exposure required for a rare-rate claim under the chosen Poisson model; a barrier argument also needs observable state, a feasible control action, and valid timing assumptions.

Table 17.2: Detectorless physical failure taxonomy: Four failure modes, in transduction, contact, the drive, and learned perception, whose safe and hazardous states look identical to the existing channel before the time-to-harm. For each, the table gives why the channel misses it, the deadline or evidence it cannot meet, and the response that still supports a claim.

Failure and missing state	Why the existing channel can miss it	Deadline or evidence deficit	Defensible response
Pressure-sensor drift: zero offset changes with aging or deposition.	One channel cannot separate true pressure change from its own offset drift.	Drift may persist until the process reaches its pressure limit; a detector cannot be credited without a second reference.	Independent calibrated pressure channel, proof test, and rated relief path.
Latch-strength loss: corrosion or wear lowers the latch's yield force below the contact ceiling.	Joint torque sensors read the same contact force against a weakened latch as against a sound one.	Latch strength is unmeasured between proof tests, and the yield is irreversible, so no later command restores the latch.	A passive compliant element sized by equation 17.2 to a declared floor on latch strength, or proof tests of the latch.
Rotor flux loss: delivered torque differs from current-based estimate.	Stator current alone does not measure the torque constant after magnet damage.	A gravity-loaded axis may have little time to arrest once available torque is lost.	Measure torque or rotor state, derate within tested limits, and validate a holding or controlled-stop path.
Learned perception shift: an optical artifact is mistaken for a known scene.	Model confidence may remain high even while the physical scene differs.	Confidence or reconstruction loss alone does not bound clearance or stopping response.	Independent range/clearance checks and an admitted motion envelope, with residual blind regions recorded.

Every row fails the practical-detector test because the existing channel gives identical observations for the safe and hazardous states until the deadline has passed. Every defensible response supplies what that channel lacks: a second reference, a passive load limit, a direct torque measurement, or an independent range check. What that missing element is, and whether it can be built, decides the kind of gap each register entry records.

17.5 What Would Change These Answers

The three entries in the mobile manipulator's residual-claims register lack the same element, a qualified detector for a state no sensor in service reports. Sorting every recorded gap by what is missing turns the register from a list of caveats into an engineering agenda, and a recorded gap falls into one of four classes, defined by the missing element. It stems from an **unmeasured observable** when the true physical state (such as a hidden internal defect) cannot be distinguished

from sensor data before the deadline for action; from an **evaluation deficit** when the tail probability of failure cannot be bounded because these rare events require intractable physical exposure; from a **theoretical deficit** when the learned policy lacks a bounded relationship between training conditions and execution stability; or from a **scope mismatch** when the target claim exceeds what the system architecture can physically support. All three of the machine's entries belong to the first class. More grounding-model trials at the pick station or more stopping runs on dry floors would treat them as evaluation deficits, which they are not. Naming the missing capability defines the exact physical or mathematical requirement that would discharge the entry.

Closing a gap from an unmeasured observable requires a new physical measurement that separates previously indistinguishable states within the permission path's deadline. For the film, a software filter or a learned estimator running on the navigation camera and the lidar cannot be that measurement, because it consumes the same observations that already fail to separate film from dry floor. The floor-film detector must respond to the film through an independent physical channel, and it closes the entry only if it meets the timing requirement of section 17.4, holds a bounded false-alarm rate, and fails independently of the cameras.

Closing an evaluation deficit requires replacing raw sample counts with estimators whose bias and variance can be bounded analytically. Logging more unperturbed operation, or running simulated episodes whose fidelity in the tail is unverified, cannot bound a tail rate. An importance-sampling or adaptive stress-testing scheme can, if it demonstrates conservative estimates on benchmark distributions whose answer is known and reports effective independent exposure rather than raw trial counts. Ten thousand simulated trajectories that explore one basin of a collision surface say little about the others, so effective sample size must be estimated from the weights and their dependence (for high-quantile tail bounds and extreme value modeling, see section A.10).

Closing a theoretical deficit requires a bounded analytical claim paired with a runtime mechanism that checks its premises. For a policy trained by imitation or reinforcement learning, the deficit is the absence of any guarantee of bounded outputs under distribution shift, and neither asymptotic convergence nor a bound resting on an uncheckable Lipschitz constant closes it. The theory must state the invariance it guarantees and the support over which it holds, and the machine must carry a monitor that flags when its state leaves that support, with bounded estimator error and decision delay and a validated state-matched stop.

To prevent research optimism from corrupting the safety case, each candidate advance must undergo evaluation against a predeclared closure test. The test defines the missing argument link, the numerical threshold required for sufficiency, and the independent evidence necessary to verify it. An advance fails the test if it merely relocates an assumption to a less visible layer of the system. Replacing a neural network controller with a control-barrier function whose certificate assumes perfectly known linear dynamics does not close a theoretical deficit. It merely relocates the unmodeled dynamics from the opaque network weights to the barrier derivative. A candidate advance is stopped before it can alter the safety verdict whenever its validation relies on the exact unverified properties it was designed to establish.

Research directions in physical machine learning must be judged solely by whether they convert an open residual claim into verified evidence capable of altering the operational verdict. General improvements on benchmark leaderboards, such as a 3 percent reduction in average tracking error or higher reward on a standard task suite, provide no evidence toward closing an unsupported safety link. A research project becomes relevant to physical deployment only when its deliverable targets a specific entry in the claims register, whether by providing the missing transducer, bounding the tail estimator, or furnishing the runtime-checked stability proof. If an advance improves mean performance but leaves the detector blind spot or the tail uncertainty unchanged, the authority limits of the machine remain exactly where they were.

Certain gaps cannot be closed by any known sensor, sampling regime, or mathematical framework. When physical laws, computational limits, or economic constraints preclude the necessary measurement or proof, the safety case cannot remain suspended in anticipation of future discovery. Narrowing the actuator's physical authority, restricting the operational design domain (ODD), or refusing to operate the machine is not an admission of research defeat. It is the engineering resolution. Classical fault-tolerant design reached the same conclusion, that dependable operation comes from self-monitoring thresholds and structural isolation that contain faults rather than from eliminating

every disturbance. An arm whose compliant wrist bounds contact force below a declared floor on latch strength does not need to detect a weakened latch to leave it intact. The residual-claims register records the boundary of verifiable knowledge, and the permission path and the machine's passive limits enforce the constraints that bound the consequence of what remains unknown.

The register, the practical-detector test, and the four gap classes have so far been applied to machines whose records already existed. The method must also hold for a machine with no records yet, where the engineer has to decide which premises to measure, which to restrict, and which to leave open.

17.6 Generalizing to Unfamiliar Systems

Applied in order to a machine not previously examined, the four laws of section 1.7 isolate its hazards before its hardware is energized. Consider an automated packaging cell designed for prepared meals. The machine moves rigid plastic trays along a linear track, presses a heated sealing head onto tray perimeters to bond plastic barrier film, and injects pressurized steam into product cavities to blanch ingredients immediately prior to sealing. This cell couples all three machine classes of section 1.3 within a single enclosure, the tray as moving mass, the sealing head as contact force, and the steam as thermal flow.

The first law asks what the cell cannot undo, so the evaluation begins by cataloging the irreversible state changes the cell can produce and the physical bounds that govern them. A loaded food tray traversing a linear transfer rail carries momentum. When a tray strikes an obstruction, the kinetic energy is dissipated through plastic deformation of the container and structural deflection of the frame, a state transition that cannot be reversed by commanding the drive motor in reverse. The sealing mechanism introduces a second irreversible boundary. An actuator drives an aluminum sealing head at 180°C against the tray flange to melt polymer bonding layers. If the actuator exerts force beyond the compressive yield strength of the container wall, the flange buckles, destroying the hermetic geometry. The steam subsystem introduces a third irreversible boundary through enthalpy exchange. Steam supplied at 120°C delivers heat to the product; the product-zone temperature can remain lower only under a measured heat-transfer and dwell model. Once proteins denature and cell walls rupture under excessive thermal exposure, cooling the chamber cannot restore product texture.

Because the tray's momentum is already committed, the permission path must act before the carriage consumes its clearance. Consider, with illustrative values throughout, a linear carriage of mass $m_{\text{carriage}} = 12.0 \text{ kg}$ transporting a loaded tray at a nominal velocity of $v_{\text{nom}} = 1.50 \text{ m/s}$ toward an optical inspection station whose mechanical gate provides a physical clearance margin of $D_{\text{clear}} = 60.0 \text{ mm}$ before contacting a rigid frame stop. An overhead vision system detects a closing inspection gate. The controller commands an electromagnetic track brake with a credible deceleration of $a_{\text{brake}} = 15.0 \text{ m/s}^2$, a value that assumes the pad and rail friction stay as tested. Determining whether an onboard vision system can halt the carriage prior to an impact requires computing the pre-brake delay τ_{delay} across the full observation-to-braking path: optical exposure and rolling-shutter integration ($t_{\text{exp}} = 16.0 \text{ ms}$), direct memory access (DMA) transfer and neural detection inference ($t_{\text{inf}} = 36.0 \text{ ms}$), CAN-FD safety arbitration bus latency ($t_{\text{bus}} = 2.0 \text{ ms}$), and brake solenoid flux buildup rise time ($t_{\text{sol}} = 10.0 \text{ ms}$), summing to $\tau_{\text{delay}} = 16.0 + 36.0 + 2.0 + 10.0 = 64.0 \text{ ms} = 0.064 \text{ s}$.

Over this delay, the carriage travels blind for $d_{\text{blind}} = v_{\text{nom}} \tau_{\text{delay}} = 1.50 \text{ m/s} \times 0.064 \text{ s} = 96.0 \text{ mm}$, which alone exceeds the 60.0 mm clearance, so the physical deadline is missed before braking begins. Once the solenoid builds flux and mechanical pads engage, deceleration consumes an additional distance $d_{\text{brake}} = \frac{v_{\text{nom}}^2}{2a_{\text{brake}}} = \frac{(1.50 \text{ m/s})^2}{2 \times 15.0 \text{ m/s}^2} = 75.0 \text{ mm}$. The stopping distance of section 2.3 is the sum of the two, blind travel that grows linearly with speed and braking distance that grows with its square, so $d_{\text{stop}} = d_{\text{blind}} + d_{\text{brake}} = 96.0 \text{ mm} + 75.0 \text{ mm} = 171.0 \text{ mm}$. That stop exceeds the physical clearance by a deficit of $\Delta d = d_{\text{stop}} - D_{\text{clear}} = 111.0 \text{ mm}$, and the machine cannot stop before the modeled gate at this velocity by relying on that vision path.

The second law asks what the cell's learned proposers have shown they can do. Three of them propose actions here, one in each domain. A vision model infers tray contents to propose carriage transit speeds, a multimodal network estimates packaging film thickness to propose sealing forces and dwell durations, and a thermal estimator proposes steam valve pulse widths from incoming product temperature. Each proposer's actions choose part of what it observes next (section 5.1). A

slower carriage changes the motion blur and timing of the frames the vision model sees, a longer dwell changes the film that the next thickness estimate reads, and a steam pulse changes the product temperature the thermal estimator receives. Their competence therefore extends only to the states a closed-loop evaluation covered, and the question for this cell is whether the target ODD of its evaluation record (section 7.8) contains the states that operation will admit. A restriction imposed after training is itself a change of operating condition. If the demonstrations were collected at the nominal transit speed, the derated mode that the cell's release claim requires moves the vision model into conditions its evaluation never sampled, so the restriction that shortens the stop also opens a regime without evidence. Either the evaluation samples the restricted mode before release, or the release claim records that regime as an open premise.

The third law places those proposals beneath an authority split that the proposers cannot delay or corrupt. The permission path keeps the proposals from reaching actuators directly. Its independent kinematic check, at 1 kHz, monitors optical encoders along the track, admitting carriage velocity only where rangefinder coverage, measured gate clearance, and a validated stop path support the envelope. For the sealing head, a mechanical force limiter caps transmitted force at a selected 220 N, a cap that holds only while the limiter's calibration does. For the steam delivery system, an independent temperature path closes the steam valve when a sensed product-zone temperature reaches 105°C or dwell reaches 400 ms. The learned policy retains authority to optimize throughput and seal quality within these bounds, but the permission path retains sole authority to permit physical execution.

The fourth law turns what the evidence supports into a release claim and names what it leaves open. The evidence record can support a restricted kinetic claim after tests of rangefinder coverage, gate clearance, and braking. The firmware velocity clamp is sized from the same stopping distance. Requiring the stop to fit within the physical clearance with a safety margin of $d_{\text{margin}} \geq 15.0$ mm imposes the constraint $d_{\text{stop}}(v) \leq 45.0$ mm. Reducing operational velocity shortens both terms, and derating it threefold, from $v_{\text{nom}} = 1.50$ m/s to a firmware clamp of $v_{\text{clamp}} \leq 0.50$ m/s, compresses blind travel linearly from 96.0 mm to $d_{\text{blind}} = 0.50 \text{ m/s} \times 0.064 \text{ s} = 32.0$ mm, while braking distance falls ninefold from 75.0 mm down to $d_{\text{brake}} = \frac{(0.50 \text{ m/s})^2}{2 \times 15.0 \text{ m/s}^2} \approx 8.3$ mm. The composite stop $d_{\text{stop}} = 40.3$ mm fits within the 45.0 mm threshold, preserving a 19.7 mm physical margin. Simultaneously, carriage kinetic energy ($E_k = \frac{1}{2}mv^2$) drops quadratically by 88.9 percent, from 13.5 J to 1.50 J.

Assembling the evidence record also uncovers claims that remain unsupported. To assess seal hermeticity, the engineering team evaluates a vision model that inspects optical scattering through the steam chamber for micro-tears in the sealing film. Within the chamber, localized steam condensation creates optical refraction patterns that mimic film defects while masking genuine punctures. Because no independent physical sensor exists in the cell to measure film seal integrity in real time under dense vapor, this claim has no practical detector. Marking this link as detectorless and unsupported in the claims register is not an omission in the documentation, but an accurate statement of what the evidence can justify.

The derated stop therefore supports a *candidate restricted mode* for transport at 0.50 m/s, provided the stated 64 ms end-to-end delay, 15 m/s^2 credible deceleration, and 60 mm clearance are validated together. Sealing and thermal processing remain conditional on tests of the force limiter, cutoff, material limits, and independent fault path. Autonomous package sign-off is refused because vapor can conceal film defects from the available vision sensor and the example has not established seal integrity independently. That link could be closed by post-cooling leak or vacuum-decay inspection, a tested material-process argument, a self-indicating film, or mandatory downstream inspection. A throughput model cannot replace that physical evidence.

The evaluation of the cell did not begin with model architecture, training losses, or benchmark scores. It began at the physical body, where mass, force, and energy create irreversible state changes, and it ended with an evidence record that marks its open links and restricts authority to what the evidence supports. A neural network predicting a high probability of a safe stop will still crash if the brakes cannot engage before the clearance is consumed.

17.7 What the Four Laws Leave Open

The running machine ends the book refused. On the book's evidence the adjudicator of section 16.6 signs REFUSE on all six claims, because each rests on a premise the records leave illustrative or

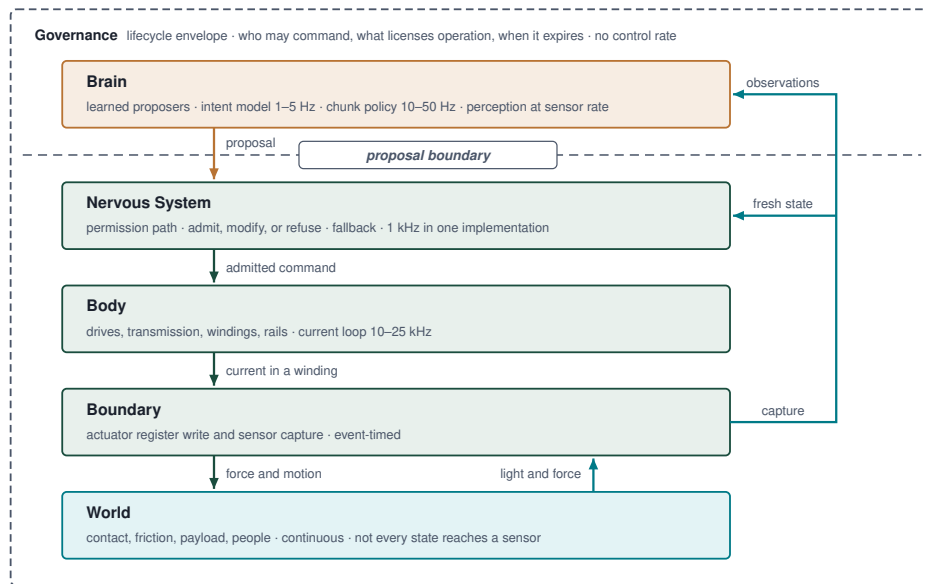


Figure 17.3: Closed causal loop: The Brain proposes across the proposal boundary, and the permission path in the Nervous System admits, modifies, or refuses each proposal before the Body acts. The Body's current becomes force and motion at the Boundary, the World evolves, and the Boundary returns measurements with capture times to the permission path and the Brain. Not every state of the World reaches a sensor. The governance envelope surrounds all five levels and changes on the timescale of releases rather than control cycles. The 1 kHz permission loop is one implementation, not a universal rate.

unknown. The measurements of table 16.4 would move that verdict only part of the way. Once they support the premises they settle, the most a release could sign is COND for a named part of the envelope, covering aisle travel in the 1 m/s restricted mode on inspected floors, the door at the guarded approach, and the handover to the coworker. The 1.3 m/s normal mode would stay unsigned, because it rests on the friction floor that the oil film leaves open. Under either verdict the three entries of table 17.1 stay open, each with its owner and closure test, and all three are unmeasured observables that only a qualified detector can close: a floor-film detector, a qualified rack-end presence sensor, and a qualified identity check at the pick station. None of them is a scope mismatch or a theoretical deficit. The machine's authority stops where the register says it does.

That verdict is the four laws applied to one machine, and the packaging cell carried the same laws through a machine the book had not examined. Each law also led one part of this book, and each part handed forward records that state their own conditions. Together those records carry the answer to the question the book opened with, and they also mark where that answer stops. Figure 17.3 draws the loop they protect. The Brain proposes, the permission path admits or refuses each proposal against fresh state, and the Body delivers the admitted command across the Boundary into a World that keeps evolving whether or not any level observes it.

Table 17.3 pairs each law with the records that carry it, the premise its evidence cannot always settle, and the restriction that holds while that premise stays open. The numbers each row needs come from the particular machine and its limit records, not from the table.

Table 17.3: The four laws and what they leave open: Each law leads one part of the book and is carried by that part's records. The last two columns name the premise its evidence cannot always settle and the restriction that holds while the premise is open, for a particular machine rather than as a universal hardware floor.

Law	Part	Records that carry it	Premise the evidence cannot always settle	What restricts operation while it is open
Irreversibility	I (chapter 2, chapter 3, chapter 4)	Limit records (chapter 2); the proposer's limits (chapter 3), which enter the handoff record (chapter 4)	Each measured limit, such as floor friction, payload, brake onset, or observation age, still holds under present conditions	A speed or force ceiling sized from the credible limit, a lease derived from the stopping budget, and a fallback reachable from the current state
Endogenous data	II (chapter 5, chapter 6, chapter 7)	Demonstration provenance record, policy manifest, evaluation record	The states the machine enters lie inside the target ODD that closed-loop evaluation sampled	Operation confined to the target ODD, with a runtime monitor on every boundary that can be monitored
Proposal is not permission	III (chapter 8, chapter 9, chapter 10, chapter 11, chapter 12, chapter 13)	Observation contract, belief record, intent lease, trajectory record, enforcement record, placement record	The permission path sees fresh, distinguishable state and meets its deadline under load	Refusal and the fallback ladder; an expired lease or stale evidence removes permission without waiting for the proposer
Evidence bounds authority	IV (chapter 14, chapter 15, chapter 16)	Authority record, fault record, release manifest, and the residual-claims register of this chapter	The premises of the release claim still hold after release	A conditional verdict with standing conditions, and a register entry that restricts authority until its closure test passes

Four decisions keep the restrictions in the table's last column operational. First, preserve each open entry across model updates until a predeclared closure test supplies evidence. Second, keep learned outputs as proposals and maintain an independently validated permission and fallback path whose processor, scheduling period, sensor coverage, and actuator response follow the plant's measured deadlines. Third, use passive limits only where their load path, stroke, wear, and stopping consequences are tested. Fourth, require named engineering and operations owners to sign the release conditions and define the measurements that withdraw authority. Down that column, every law ends the same way, in a restriction that holds until evidence lifts it rather than a guarantee that holds regardless of the evidence.

17.8 Fallacies and Pitfalls

Some hazards are unobservable through available sensors, and some claims require more exposure than the design can accumulate. Both limits must shape operating authority rather than disappear into a confidence score.

Pitfall: *Relying on vision-based perception alone when the physical hazard is observationally indistinguishable.*

The mobile manipulator approaches a transparent hydraulic-fluid film that its navigation camera and lidar cannot distinguish from dry concrete, producing observational indistinguishability. When the wheels enter the film and spin without grip, the slip reveals the changed surface only after the deceleration the stop assumed is gone. A visually clear path says nothing about the friction the admitted speed requires. The operating envelope must account for that unobserved condition through defensible speed restrictions or independently validated sensing. The residual-claims register must retain the friction assumption and its containment measure instead of treating missing visual evidence as evidence of traction.

Fallacy: *Stator temperature and current measurements are sufficient to detect rotor magnet flux degradation.*

The rotor-flux row of table 17.2 already names this gap. Torque follows $\tau = K_t I$ (chapter 2), so when magnet damage lowers K_t , the drive still reports its commanded current and the expected Joule heating while the joint delivers less torque than that current implies. Nominal current is not proof

Prerequisite: The gap between verified models and unmodeled friction challenges the safety cases in section 16.3.

of torque authority, so qualification must verify torque capability independently and the holding path must not assume the original torque.

Fallacy: *Massive cloud simulation can substitute for physical exposure in certifying open-world safety.*

A team runs billions of simulated hours in the cloud to bypass the cost of physical exposure. Simulation evidence holds only for the simulator's own model (section 7.4), and a simulated floor with the friction of inspected concrete says nothing about a clear film (section 7.6). Repetition inside that closed model adds no evidence about the physics it omits, so it cannot validate sim-to-real fidelity against the deployment conditions. Simulation remains useful for logic checks and regressions, while physical safety claims also require measurements or validated operating envelopes for the machine. The release case must distinguish simulated evidence from evidence that its assumptions hold on the physical system.

17.9 Summary

At the causal boundary (section 1.2), a computational decision becomes part of a continuing, irreversible physical process. The architecture developed throughout this book connects this physical irreversibility to measured budgets, explicit interfaces, independent enforcement, and accountable release decisions. Its frontier lies in the premises those mechanisms still depend on but cannot fully observe or verify. On the running machine, the arithmetic of the stopping budget never changed when the oil film appeared, but one of its terms had rested all along on a floor that nothing measured, and irreversibility made that unmeasured term as binding as any measured one. An engineering record must expose such limits and let them restrict operation for as long as the machine runs, not only on the day a release is signed. Engineering embodied intelligence requires knowing when the available evidence is sufficient to act and when it is not.



Key Takeaways: The epistemic frontier and what the laws leave open

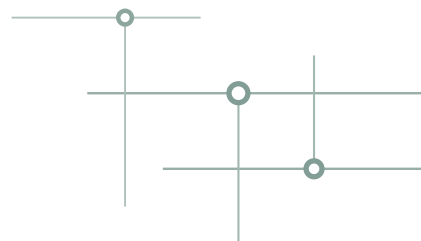
- **An open premise restricts authority:** The residual-claims register takes each premise the release manifest left open and gives it a missing observable, a restriction, an owner, and a closure test declared in advance. On the aisle, the friction entry caps any release at its restricted mode on inspected floors until a floor-film detector passes that test.
- **A late detector is only a logger:** A detector protects against a hazard only if its latency fits inside the margin the stop leaves unspent. When safe and hazardous states give identical telemetry until the action deadline, no algorithm separates them in time, and passive compliance, a restricted envelope, or a second physical channel must carry the claim.
- **Trials cannot reach a rare-hazard rate:** The exposure wall of section 7.5 puts a rare-hazard rate beyond any practical failure-free campaign. A shield between proposal and actuation earns credit only for hazard classes it can observe and intercept in time, and its proposal rate, coverage, and common-cause terms each need evidence of their own.
- **Research counts when it closes an entry:** A new sensor, estimator, or proof matters for deployment only when it closes a named register entry by supplying the missing observable, bounding the tail, or checking its own premises at runtime. An entry that nothing can close is resolved by narrowing authority or withholding operation, not by waiting.
- **Every law leaves a premise open:** Irreversibility rests on measured limits that still hold; endogenous data, on a target ODD that covers the states the machine enters; proposal is not permission, on fresh and distinguishable state; and evidence bounds authority, on premises that still hold after release. Authority stops where that evidence does.

 What's Next: From physical AI to the systems discipline

This book completes an investigation into the full architecture of machine learning systems. Across the systems curriculum, we explored the single machine (mastering the silicon contract, arithmetic intensity, memory hierarchies, and local hardware acceleration), distributed fleets (navigating collective communication fabrics, fault tolerance, cluster orchestration, and high-throughput serving infrastructure), and agentic software trajectories (constructing sandboxed execution environments, dynamic context working sets, and deterministic verification harnesses for autonomous multi-step reasoning). Here, machine learning leaves the digital domain to inhabit embodied physical intelligence—where learned foundation models act as unprivileged proposal engines, real-time nervous systems enforce validated operating bounds, and artificial intelligence at last meets the unyielding conservation laws of the physical universe. This work is dedicated to the students, researchers, and systems engineers who take up the challenge of giving machines safe and capable hands in an open physical world.

Prof. Vijay Janapa Reddi, Harvard University

BACK MATTER



References

- Ahn, Michael, Anthony Brohan, Yevgen Chebotar, et al. 2023. "Do as I Can, Not as I Say: Grounding Language in Robotic Affordances." *Proceedings of the 6th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 205: 287–318.
- Ahn, Michael, Debidatta Dwibedi, Chelsea Finn, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Karol Hausman, Brian Ichter, et al. 2024. "AutoRT: Embodied Foundation Models for Large Scale Orchestration of Robotic Agents." *arXiv Preprint arXiv:2401.12963*.
- Amanatides, John, and Andrew Woo. 1987. "A Fast Voxel Traversal Algorithm for Ray Tracing." *Eurographics '87*, 3–10.
- Ames, Aaron D, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. 2019. "Control Barrier Functions: Theory and Applications." *European Control Conference (ECC)*, 3420–31.
- Ames, Aaron D, Jessy W Grizzle, and Paulo Tabuada. 2014. "Control Barrier Function Based Quadratic Programs with Application to Adaptive Cruise Control." *53rd IEEE Conference on Decision and Control*, 6271–78.
- Ames, Aaron D, Xiangru Xu, Jessy W Grizzle, and Paulo Tabuada. 2016. "Control Barrier Function Based Quadratic Programs for Safety Critical Systems." *IEEE Transactions on Automatic Control* 62 (8): 3861–76.
- An, Dong, Yuankai Qi, Yang Huang, Yan Wei, Liang Wang, Tieniu Tan, et al. 2023. "ETPNav: Evolving Topological Planner for Vision-Language Navigation in Continuous Environments." *arXiv Preprint arXiv:2304.03047*.
- Anderson, Robert J., and Mark W. Spong. 1989. "Bilateral Control of Teleoperators with Time Delay." *IEEE Transactions on Automatic Control* 34 (5): 494–501. <https://doi.org/10.1109/9.24201>.
- Andrychowicz, Marcin, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, et al. 2020. "Learning Dexterous in-Hand Manipulation." *The International Journal of Robotics Research* 39 (1): 3–20.
- Asada, Haruhiko, and Takeo Kanade. 1983. "Design of Direct-Drive Mechanical Arms." *ASME Journal of Vibration, Acoustics, Stress, and Reliability in Design* 105 (3): 312–16. <https://doi.org/10.1115/1.3269094>.
- Ashby, W Ross. 1956. *An Introduction to Cybernetics*. Chapman & Hall.
- Åström, Karl Johan, and Tore Hägglund. 2006. *Advanced PID Control*. ISA-The Instrumentation, Systems, Automation Society.
- Åström, Karl Johan, and Richard M Murray. 2021. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton university press.
- Bainbridge, Lisanne. 1983. "Ironies of Automation." *Automatica* 19 (6): 775–79. [https://doi.org/10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8).
- Barr, Michael. 2013. *Bookout v. Toyota Motor Corp.: 2005 Camry L4 Software Analysis*.
- Black, Kevin, Homer Walke, Kevin Bishop, Lee Zhang, Albert Fang, Brent Chen, Steven Morad, et al. 2024. " π_0 : A Flow-Matching Foundation Model for Generalist Robots." *arXiv Preprint arXiv:2410.24164*.
- Blanchini, Franco. 1999. "Set Invariance in Control." *Automatica* 35 (11): 1747–67.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, et al. 2023. "Rt-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control." *arXiv Preprint arXiv:2307.15818*.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, et al. 2023. "RT-1: Robotics Transformer for Real-World Control at Scale." *Robotics: Science and Systems (RSS)*. <https://doi.org/10.15607/RSS.2023.XIX.025>.
- Brooks, Rodney. 1986a. "A Robust Layered Control System for a Mobile Robot." *IEEE Journal on Robotics and Automation* 2 (1): 14–23.
- Brooks, Rodney. 1986b. "A Robust Layered Control System for a Mobile Robot." *IEEE Journal on Robotics and Automation* 2 (1): 14–23.

- Brooks, Rodney A. 1991. "Intelligence Without Representation." *Artificial Intelligence* 47 (1-3): 139-59. [https://doi.org/10.1016/0004-3702\(91\)90053-M](https://doi.org/10.1016/0004-3702(91)90053-M).
- Bryant, Randal E., and David R. O'Hallaron. 2015. *Computer Systems: A Programmer's Perspective*. 3rd ed. Pearson.
- Butler, Ricky W., and George B. Finelli. 1993. "The Infeasibility of Quantifying the Reliability of Life-Critical Real-Time Software." *IEEE Transactions on Software Engineering* 19 (1): 3-12. <https://doi.org/10.1109/32.210303>.
- Cadene, Rémi, Simon Alibert, Alexander Toshev, Thomas Sen, Quentin Gallouédec, Adil Zouitine, et al. 2024. "LeRobot: State-of-the-Art Machine Learning for Real-World Robotics in PyTorch." *arXiv Preprint arXiv:2410.09312*.
- Chi, Cheng, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2024. "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion." *The International Journal of Robotics Research* 44 (10-11): 1684-704. <https://doi.org/10.1177/02783649241273668>.
- Clopper, C. J., and Egon S. Pearson. 1934. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial." *Biometrika* 26 (4): 404-13.
- Cohen, Jacob. 1988. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Routledge.
- Coles, Stuart. 2001. *An Introduction to Statistical Modeling of Extreme Values*. Springer.
- Craig, John J. 2005. *Introduction to Robotics: Mechanics and Control*. 3rd ed. Pearson Prentice Hall.
- Cruise LLC. 2023. *Part 573 Safety Recall Report 23E-086: Cruise LLC Automated Driving System Collision Detection Subsystem*. Nos. 23E-086. U.S. Department of Transportation, National Highway Traffic Safety Administration.
- Curless, Brian, and Marc Levoy. 1996. "A Volumetric Method for Building Complex Models from Range Images." *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*, 303-12.
- Driess, Danny, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, et al. 2023. "PaLM-e: An Embodied Multimodal Language Model." *International Conference on Machine Learning (ICML)*, 8469-88.
- Durrant-Whyte, Hugh, and Tim Bailey. 2006. "Simultaneous Localization and Mapping: Part I." *IEEE Robotics & Automation Magazine* 13 (2): 99-110. <https://doi.org/10.1109/MRA.2006.1638022>.
- Elfes, Alberto. 1989. "Using Occupancy Grids for Mobile Robot Perception and Navigation." *Computer* 22 (6): 46-57. <https://doi.org/10.1109/2.30720>.
- ESA Investigation Board. 2017. *ExoMars 2016 - Schiaparelli Anomaly Inquiry*. European Space Agency.
- Exponent, Inc. 2023. *Cruise AV SF Incident—Pedestrian Collision: Technical Root Cause Analysis*. Exponent Project 2310645.000. Exponent, Inc.
- Florence, Pete, Corey Lynch, Feryal Behbahani, Tianhe Yu, Wenlong Huang, Fei Xia, et al. 2022. "Implicit Behavioral Cloning." *Conference on Robot Learning (CoRL)*, 158-68.
- Foote, Tully. 2013. "Tf: The Transform Library." *IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, 1-6.
- Franklin, Gene F, J David Powell, and Michael L Workman. 1998. *Digital Control of Dynamic Systems*. 3rd ed. Addison-Wesley.
- Fu, Zipeng, Tony Z. Zhao, and Chelsea Finn. 2024. "Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation." *Proceedings of the 8th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 270.
- Furgale, Paul, Joern Rehder, and Roland Siegwart. 2013. "Unified Temporal and Spatial Calibration for Multi-Sensor Systems." *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1280-86.
- Gibson, James J. 1979. *The Ecological Approach to Visual Perception*. Houghton Mifflin.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Gray, Cary, and David Cheriton. 1989. "Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency." *ACM SIGOPS Operating Systems Review* 23 (5): 202-10.
- Gu, Xiuye, Tsung-Yi Lin, Weicheng Kuo, and Yin Cui. 2022. "Open-Vocabulary Object Detection via Vision and Language Knowledge Distillation." *International Conference on Learning Representations*.
- Hanley, James A., and Abby Lippman-Hand. 1983. "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators." *JAMA* 249 (13): 1743-45. <https://doi.org/10.1001/jama.1983.03330370053031>.
- Harnad, Stevan. 1990. "The Symbol Grounding Problem." *Physica D: Nonlinear Phenomena* 42 (1-3): 335-46.
- Hartley, Richard, and Andrew Zisserman. 2003. *Multiple View Geometry in Computer Vision*. Cambridge university press.

- Hennessy, John L., and David A. Patterson. 2019. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann.
- Hermann, Robert, and Arthur J. Krener. 1977. "Nonlinear Controllability and Observability." *IEEE Transactions on Automatic Control* 22 (5): 728–40.
- Hogan, Neville. 1985. "Impedance Control: An Approach to Manipulation: Part i–Theory, Part II–Implementation, Part III–Applications." *Journal of Dynamic Systems, Measurement, and Control* 107 (1): 1–24.
- Hornung, Armin, Kai M Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. 2013. "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees." *Autonomous Robots* 34 (3): 189–206.
- Huang, Wenlong, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wong, Li Fei-Fei, and Yuke Zhu. 2023. "VoxPoser: Composable 3D Value Maps for Everyday Robot Manipulation with Language Models." *arXiv Preprint arXiv:2307.05973*.
- Hughes, Nathan, Yun Chang, and Luca Carlone. 2022. "Hydra: A Real-Time Spatial Perception Engine for 3D Scene Graph Construction and Optimization." *Proceedings of Robotics: Science and Systems (RSS)*.
- ISO 21448: *Road Vehicles, Safety of the Intended Functionality*. 2022. International Organization for Standardization.
- ISO 26262: *Road Vehicles, Functional Safety*. 2018. International Organization for Standardization.
- ISO/TS 15066: *Robots and Robotic Devices, Collaborative Robots*. 2016. International Organization for Standardization.
- Jatavallabhula, Krishna Murthy, Alihusein Kuwajerwala, Qiao Gu, et al. 2023. "ConceptFusion: Open-Set Multimodal 3D Mapping." *Proceedings of Robotics: Science and Systems (RSS)*.
- Johannink, Tobias, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. 2019. "Residual Reinforcement Learning for Robot Control." *IEEE International Conference on Robotics and Automation (ICRA)*, 6023–29. <https://doi.org/10.1109/ICRA.2019.8794127>.
- Kalman, Rudolph Emil. 1960. "A New Approach to Linear Filtering and Prediction Problems." *Journal of Basic Engineering* 82 (1): 35–45.
- Kamath, Aishwarya, Mannat Singh, Yann LeCun, Gabriel Synnaeve, Ishan Misra, and Nicolas Carion. 2021. "Mdetr-Modulated Detection for End-to-End Multi-Modal Understanding." *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1780–90.
- Katz, Benjamin, Jared Di Carlo, and Sangbae Kim. 2019. "Mini Cheetah: A Platform for Dynamic Locomotion." *IEEE International Conference on Robotics and Automation (ICRA)*, 3295–301. <https://doi.org/10.1109/ICRA.2019.8793865>.
- Katz, Guy, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. 2017. "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks." *International Conference on Computer Aided Verification (CAV)*, 97–117. https://doi.org/10.1007/978-3-319-63387-9_5.
- Kavraki, Lydia E., Petr Švestka, Jean-Claude Latombe, and Mark H. Overmars. 1996. "Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces." *IEEE Transactions on Robotics and Automation* 12 (4): 566–80. <https://doi.org/10.1109/70.508439>.
- Kelly, Timothy Patrick. 1998. "Arguing Safety: A Systematic Approach to Managing Safety Cases." PhD thesis, Department of Computer Science, University of York.
- Kelly, Tim, and Rob Weaver. 2004. "The Goal Structuring Notation: A Safety Argument Notation." *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*.
- Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. "3D Gaussian Splatting for Real-Time Radiance Field Rendering." *ACM Transactions on Graphics* 42 (4): 139:1–14.
- Khazatsky, Alexander, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, et al. 2024. "DROID: A Large-Scale in-the-Wild Robot Manipulation Dataset." *Robotics: Science and Systems (RSS)*.
- Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, et al. 2024. "OpenVLA: An Open-Source Vision-Language-Action Model." *arXiv Preprint arXiv:2406.09246*.
- Kirillov, Alexander, Eric Mintun, Nikhila Ravi, Hanzi Mao, et al. 2023. "Segment Anything." *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 4015–26.
- Koopman, Philip. 2020. *How Safe Is Safe Enough? Measuring and Predicting Autonomous Vehicle Safety*. BookBaby.
- Kopetz, Hermann. 2011. *Real-Time Systems: Design Principles for Distributed Embedded Applications*.
- Krotkov, Eric, David Hackett, Lawrence Jackel, Michael Perschbacher, James Pippine, Craig Strauss, Gill Pratt, and Charles Orillat. 2017. "The DARPA Robotics Challenge Finals: Results and Perspectives." *The DARPA Robotics Challenge Finals: Humanoid Robots to the Rescue*, Springer tracts in advanced robotics, vol. 121: 1–26.

- Kumar, Aviral, Aurick Zhou, George Tucker, and Sergey Levine. 2020. "Conservative q-Learning for Offline Reinforcement Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 1179–91.
- Kuwajerwala, Alihusein, Qiao Gu, Sacha Morin, et al. 2024. "ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning." *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 14170–77.
- Landauer, Rolf. 1961. "Irreversibility and Heat Generation in the Computing Process." *IBM Journal of Research and Development* 5 (3): 183–91. <https://doi.org/10.1147/rd.53.0183>.
- LaValle, Steven M. 1998. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. TR 98-11. Computer Science Department, Iowa State University.
- Leveson, Nancy. 2011. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- Leveson, Nancy G., and Clark S. Turner. 1993. "An Investigation of the Therac-25 Accidents." *Computer* 26 (7): 18–41.
- Levine, Sergey, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. 2018. "Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection." *The International Journal of Robotics Research* 37 (4-5): 421–36.
- Li, Xuanlin, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Walke, Chuer Pan, et al. 2024. "SIMPLER: Simulated Evaluation of Manipulator Policy with Real-World Robot Environments." *Proceedings of the 8th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research.
- Liang, Jacky, Wenlong Huang, Fei Xia, et al. 2023. "Code as Policies: Language Model Programs for Embodied Control." *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 9493–500.
- Lipman, Yaron, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nicklas, and Matt Le. 2022. "Flow Matching for Generative Modeling." *arXiv Preprint arXiv:2210.02747*.
- Liu, C. L., and James W. Layland. 1973. "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment." *Journal of the ACM* 20 (1): 46–61.
- Liu, Peiqi, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. 2024. "OK-Robot: What Really Matters in Integrating Open-Knowledge Models for Robotics." *arXiv Preprint arXiv:2401.12202*.
- Liu, Shilong, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jianwei Yang, Chunyuan Li, et al. 2023. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." *arXiv Preprint arXiv:2303.05499*.
- Liu, Zhijian, Haotian Tang, Alexander Amini, Xinyu Yang, Huizi Mao, Daniela Rus, and Song Han. 2023. "BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation." *IEEE International Conference on Robotics and Automation (ICRA)*, 2774–81.
- Lozano-Pérez, Tomás. 1983. "Spatial Planning: A Configuration Space Approach." *IEEE Transactions on Computers* C-32 (2): 108–20. <https://doi.org/10.1109/TC.1983.1676196>.
- Lynch, Kevin M, and Frank C Park. 2017. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press.
- Ma, Yingdong, Yifan Song, Ziyuan Cao, Yu Zhang, et al. 2024. "A Survey on Vision-Language-Action Models for Embodied AI." *arXiv Preprint arXiv:2405.14093*.
- Main Commission Aircraft Accident Investigation Warsaw and German Federal Bureau of Aircraft Accident Investigation. 1994. *Report on the Accident to Airbus A320-211 Aircraft in Warsaw on 14 September 1993*. Polish Ministry of Transport; Maritime Economy / BFU.
- Marr, David. 1982. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman; Company.
- Matsuki, Hidenobu, Ryo Murai, Paul H. J. Kelly, and Andrew J. Davison. 2024. "Gaussian Splatting SLAM." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 18039–48.
- Mindell, David A. 2008. *Digital Apollo: Human and Machine in Spaceflight*. MIT Press.
- Mitchell, Ian M., Alexandre M. Bayen, and Claire J. Tomlin. 2005. "A Time-Dependent Hamilton-Jacobi Formulation of Reachable Sets for Continuous Dynamic Games." *IEEE Transactions on Automatic Control* 50 (7): 947–57. <https://doi.org/10.1109/TAC.2005.851439>.
- Moravec, Hans. 1988. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press.
- Moravec, Hans, and Alberto Elfes. 1985. "High Resolution Maps from Wide Angle Sonar." *IEEE International Conference on Robotics and Automation* 2: 116–21.
- Museth, Ken. 2013. "VDB: High-Resolution Sparse Volumes with Dynamic Topology." *ACM Transactions on Graphics* 32 (3): 1–22. <https://doi.org/10.1145/2487228.2487235>.
- NASA Manned Spacecraft Center. 1969. *Apollo 11 Mission Report*. MSC-01855. National Aeronautics; Space Administration.

- National Highway Traffic Safety Administration (NHTSA). 2017. *PE 16-007: Investigation of Tesla Motors Model S Automatic Emergency Braking and Autopilot Systems*. PE 16-007. Office of Defects Investigation (ODI), U.S. Department of Transportation.
- National Highway Traffic Safety Administration, Office of Defects Investigation. 2023. *ODI Resume, Investigation PE23-018: Automated Driving System*. Nos. PE23-018. U.S. Department of Transportation.
- National Transportation Safety Board. 2017. *Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. HAR-17/02. National Transportation Safety Board.
- National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. HAR-19/03. National Transportation Safety Board.
- Newcombe, Richard A, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. 2011. "KinectFusion: Real-Time Dense Surface Mapping and Tracking." *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 127–36.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees Nair, et al. 2024. "Octo: An Open-Source Generalist Robot Policy." *arXiv Preprint arXiv:2404.08839*.
- Oleynikova, Helen, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto. 2017. "Voxblox: Incremental 3D Euclidean Signed Distance Fields for on-Board MAV Planning." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1366–73.
- Open X-Embodiment Collaboration, Abhishek Padalkar, Acorn Pooley, et al. 2024. "Open X-Embodiment: Robotic Learning Datasets and RT-X Models." *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 6892–903.
- Oquab, Maxime, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, et al. 2023. "Dinov2: Learning Robust Visual Features Without Supervision." *arXiv Preprint arXiv:2304.07193*.
- Parasuraman, Raja, Thomas B. Sheridan, and Christopher D. Wickens. 2000. "A Model for Types and Levels of Human Interaction with Automation." *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30 (3): 286–97. <https://doi.org/10.1109/3468.844354>.
- Philion, Jonah, and Sanja Fidler. 2020. "Lift, Splat, Shoot: Encoding Images from Arbitrary Cameras by Predicting Depth." *Proceedings of the European Conference on Computer Vision (ECCV)*, 272–85.
- Pomerleau, Dean A. 1989. "ALVINN: An Autonomous Land Vehicle in a Neural Network." *Advances in Neural Information Processing Systems (NeurIPS)* 1: 305–13.
- Presidential Commission on the Space Shuttle Challenger Accident. 1986. *Report of the Presidential Commission on the Space Shuttle Challenger Accident*. U.S. Government Printing Office.
- Quality Control Systems Corp. 2019. *NHTSA's Implausible Safety Claim for Tesla's Autosteer Driver Assistance System*. Quality Control Systems Corp.
- Quinn Emanuel Urquhart and Sullivan, LLP. 2024. *Report to the Boards of Directors of Cruise LLC, GM Cruise Holdings LLC, and General Motors Holdings LLC Regarding the October 2, 2023 Accident in San Francisco*. Quinn Emanuel Urquhart; Sullivan, LLP.
- Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." *International Conference on Machine Learning*, 8748–63.
- Reeves, Glenn E. 1997. *What Really Happened on Mars?* Electronic mail message, Jet Propulsion Laboratory.
- Rosinol, Antoni, Andrew Violette, Marcus Abate, Nathan Hughes, Yun Chang, and Luca Carlone. 2021. "Kimera: From SLAM to Hierarchical 3D Dynamic Scene Graphs." *The International Journal of Robotics Research* 40 (12–14): 1410–44. <https://doi.org/10.1177/02783649211056674>.
- Ross, Stéphane, Geoffrey Gordon, and Drew Bagnell. 2011. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 627–35.
- Saltzer, Jerome H, David P Reed, and David D Clark. 1984. "End-to-End Arguments in System Design." *ACM Transactions on Computer Systems (TOCS)* 2 (4): 277–88.
- Scott, Colin. 2012. "Numbers Every Programmer Should Know by Year."
- Seok, Sangok, Albert Wang, Meng Yee Chuah, Dong Jin Hyun, JaeJun Lee, David M. Otten, Jeffrey H. Lang, and Sangbae Kim. 2015. "Design Principles for Energy-Efficient Legged Locomotion and Implementation on the MIT Cheetah Robot." *IEEE/ASME Transactions on Mechatronics* 20 (3): 1117–29.
- Sha, Lui. 2001. "Using Simplicity to Control Complexity." *IEEE Software* 18 (4): 20–28.
- Sha, Lui, Ragunathan Rajkumar, and John P. Lehoczky. 1990. "Priority Inheritance Protocols: An Approach to Real-Time Synchronization." *IEEE Transactions on Computers* 39 (9): 1175–85.
- Sheridan, Thomas B. 1992. *Telerobotics, Automation, and Human Supervisory Control*. Wiley.

- Sheridan, Thomas B., and William L. Verplank. 1978. *Human and Computer Control of Undersea Teleoperators*. Technical Report N00014-77-C-0256. Man-Machine Systems Laboratory, Massachusetts Institute of Technology.
- Siciliano, Bruno, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. 2009. *Robotics: Modelling, Planning and Control*. Springer Science & Business Media.
- Slotine, Jean-Jacques E, and Weiping Li. 1991. *Applied Nonlinear Control*. Prentice Hall.
- Spencer, Jonathan, Sanjiban Choudhury, Arun Venkatraman, Brian Ziebart, and J. Andrew Bagnell. 2021. "Feedback in Imitation Learning: The Three Regimes of Covariate Shift." *arXiv Preprint arXiv:2102.02872*.
- Spong, Mark W. 1987. "Modeling and Control of Elastic Joint Robots." *Journal of Dynamic Systems, Measurement, and Control* 109 (4): 310–19.
- Stephenson, Arthur G. et al. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase i Report*. National Aeronautics; Space Administration.
- Sundaralingam, Balakumar, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, et al. 2023. "CuRobo: Parallelized Collision-Free Robot Motion Generation." *IEEE International Conference on Robotics and Automation (ICRA)*, 11124–31.
- Sutton, Richard. 2019. "The Bitter Lesson." *Incomplete Ideas (Blog)* 13 (1): 38.
- Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.
- Szeliski, Richard. 2022. *Computer Vision: Algorithms and Applications*. 2nd ed. Springer.
- Tedrake, Russ. 2023. *Robotic Manipulation: Perception, Planning, and Control*. Massachusetts Institute of Technology.
- Toulmin, Stephen E. 1958. *The Uses of Argument*. Cambridge University Press.
- U.S. Chemical Safety and Hazard Investigation Board. 2014. *Investigation Report: Drilling Rig Explosion and Fire at the Macondo Well (Deepwater Horizon)*. Report No. 2010-10-I-OS. U.S. Chemical Safety Board.
- Vaughan, Diane. 1996. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. University of Chicago Press.
- Wabersich, Kim P., and Melanie N. Zeilinger. 2021. "A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems." *Automatica* 129: 109647.
- Wald, Abraham. 1945. "Sequential Tests of Statistical Hypotheses." *The Annals of Mathematical Statistics* 16 (2): 117–86. <https://doi.org/10.1214/aoms/1177731118>.
- Walke, Homer, Kevin Black, Abraham Lee, Anthony Zheng, Congyue Zhao, Philippe Hansen-Estruch, Minh Yu, et al. 2023. "BridgeData V2: A Dataset for Robot Learning at Scale." *Conference on Robot Learning (CoRL)*.
- Wensing, Patrick M., Albert Wang, Sangok Seok, David Otten, Jeffrey H. Lang, and Sangbae Kim. 2017. "Proprioceptive Actuator Design in the MIT Cheetah: Impact Mitigation and High-Bandwidth Physical Interaction for Dynamic Legged Robots." *IEEE Transactions on Robotics* 33 (3): 509–22.
- Wiener, Norbert. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. John Wiley & Sons.
- Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76.
- Winograd, Terry. 1972. *Understanding Natural Language*. Elsevier.
- Wu, Philipp, Yiyang Shentu, Zhonghe Yi, Xingyu Lin, Ken Goldberg, and Pieter Abbeel. 2024. "GELLO: A General, Low-Cost, and Intuitive Teleoperation Framework for Robot Manipulation." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- Xiao, Guangxuan, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. "Efficient Streaming Language Models with Attention Sinks." *International Conference on Learning Representations (ICLR)*.
- Zhang, Zhengyou. 2000. "A Flexible New Technique for Camera Calibration." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22 (11): 1330–34. <https://doi.org/10.1109/34.888718>.
- Zhao, Tony Z., Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware." *Robotics: Science and Systems (RSS)*. <https://doi.org/10.15607/RSS.2023.XIX.016>.

Machine Learning Background

This appendix collects the machine learning models, statistical bounds, and spatial representations used throughout this book. Each section states the assumptions under which its result holds, so a reader can check them against a particular machine.

A.1 How to Use This Appendix

CONSULT this appendix when encountering specific learning systems bottlenecks, statistical verification limits, or representation boundaries across chapters in this book:

When you encounter this systems symptom	Consult	For this quantitative tool
A vision-language model hallucinates actions under perceptual ambiguity	section A.2	Posterior entropy metrics, Shannon uncertainty, and refusal thresholds
Overconfident neural outputs command aggressive actions out-of-distribution	section A.2.3	Softmax simplex geometry and Expected Calibration Error (ECE)
Multi-camera extrinsics, 2D tokenization, or 3D scene geometry drift	section A.3	$SE(3)$ transforms, patch embeddings, attention, and metric 3D lifting
Camera features must be placed in one metric ground-plane grid	section A.3.3	Lift-Splat depth hypotheses, frustum size, and BEV splatting
Differentiable 3D scene reconstruction diverges during real-time tracking	section A.3.5	3D Gaussian Splatting parameters and covariance projection
Occupancy mapping retains phantom obstacles after objects move	section A.3.6	Log-odds voxel updates, TSDF fusion, and raycast DRAM traffic
Small sensor noise produces explosive, unconstrained actuator commands	section A.4.1	Lipschitz continuity bounds and spectral normalization
A behavioral cloning policy drifts into unrecoverable off-trajectory states	section A.5	$\mathcal{O}(T^2\epsilon)$ compounding error and DAgger linear reduction
L2 regression averages bimodal demonstrations into a path between the modes	section A.5.1	Conditional expectation proof and the decoder families that avoid it
A generative decoder cannot finish a chunk within the setpoint period	section A.5.2	Diffusion and flow-matching sampling cost against the control deadline
Offline reinforcement learning selects ungrounded, destructive motor setpoints	section A.6	Bellman value overestimation and SayCan affordance grounding
Verifying time-varying physical safety properties across continuous traces	section A.7	Signal Temporal Logic (STL) syntax and quantitative robustness degree
Zero-failure fleet testing cannot bound rare catastrophic-event rates	section A.8	Rule of Three, Wald breakdown, and Clopper-Pearson exact bounds
Physical testbed wear explodes during fixed-sample policy qualification	section A.9	Sequential Probability Ratio Test (SPRT) and Wald stopping boundaries
Contact collision shocks and latency spikes exceed Gaussian 6-sigma bounds	section A.10	Extreme Value Theory (GEV/GPD) and high-quantile hardware sizing

A.2 Probability, Information Theory, and Calibration

Autonomous machines must reason quantitatively about uncertainty across perception, intent estimation, and world modeling.

A.2.1 Conditional probability and bayes' rule

The conditional probability $P(Y | X)$ characterizes the probability of event Y given observed evidence X . By Bayes' rule:

$$P(\theta | \mathcal{D}) = \frac{P(\mathcal{D} | \theta)P(\theta)}{P(\mathcal{D})}$$

where $P(\theta)$ represents prior belief over model parameters θ , $P(\mathcal{D} | \theta)$ represents the likelihood of physical dataset $\mathcal{D}$, and $P(\theta | \mathcal{D})$ represents the updated posterior belief.

A.2.2 Shannon entropy and ambiguity thresholds

Entropy $H(X)$ quantifies the expected uncertainty or information content in a probability distribution. For a discrete random variable X taking values in alphabet $\mathcal{X}$:¹

$$H(X) = - \sum_{x \in \mathcal{X}} P(x) \log_2 P(x)$$

For two equally scored referents, the *model's* candidate-action entropy is high. A calibrated, task-specific threshold can trigger a grounding refusal (ERR_GROUNDING_AMBIGUITY) and a validated hold while better evidence is sought. A confidently wrong model can have low entropy, so this threshold is only one input to permission.

A.2.3 Softmax overconfidence and calibration error

Standard neural classification heads normalize raw logit vectors $\mathbf{z} \in \mathbb{R}^K$ into categorical distributions using the softmax function:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}$$

where $T > 0$ is a temperature hyperparameter. The softmax operator projects logits onto the probability simplex Δ^K strictly through algebraic exponentiation. A large output probability indicates only relative separation between logits, not genuine epistemic certainty.

To quantify whether predicted probabilities match empirical physical frequencies, reliability engineering computes the **Expected Calibration Error (ECE)**:²

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$$

where the interval $[0, 1]$ is partitioned into M equal-width bins B_m , $|B_m|$ is sample count in bin m , N is total sample count, $\text{conf}(B_m)$ is mean confidence, and $\text{acc}(B_m)$ is empirical accuracy. In physical autonomy, an uncalibrated model with high ECE reports false certainty during out-of-distribution conditions, misleading any fallback trigger or human supervisor that gates on that confidence into withholding intervention.

A.3 Spatial Representations and Scene Parameterization

Embodied physical agents perceive, manipulate, and navigate within Euclidean 3D space.

A.3.1 Special euclidean group SE(3) transforms

Rigid body transformations in 3D space belong to the Special Euclidean group $SE(3)$, combining a 3×3 rotation matrix $\mathbf{R} \in SO(3)$ with a translation vector $\mathbf{t} \in \mathbb{R}^3$:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3)$$

A homogeneous point $\tilde{\mathbf{p}} = [\mathbf{p}^\top, 1]^\top \in \mathbb{R}^4$ transforms across frames according to linear matrix-vector multiplication:

$$\tilde{\mathbf{p}}' = \mathbf{T} \tilde{\mathbf{p}}$$

Let $\mathbf{T}_{AB}$ map coordinates expressed in frame B into frame A . For world W , robot base B , end effector E , and camera C , the chain is $\mathbf{T}_{WC} = \mathbf{T}_{WB} \mathbf{T}_{BE} \mathbf{T}_{EC}$. A valid rigid transform has inverse $\mathbf{T}^{-1} = [\mathbf{R}^\top, -\mathbf{R}^\top \mathbf{t}; \mathbf{0}^\top, 1]$; numerical accuracy still depends on calibrated transforms and their uncertainty.

¹ | **Shannon Information Entropy:** Entropy quantifies average uncertainty within a specified discrete distribution. Its physical meaning depends on whether that distribution is calibrated to the relevant referents and observations.

² | **Expected Calibration Error:** Binned calibration metrics quantify the divergence between neural prediction confidence and realized empirical accuracy across confidence intervals. Standard unregularized deep networks exhibit substantial overconfidence because cross-entropy minimization drives logits to extreme values. Relying on uncalibrated confidence scores to gate safety fallbacks permits undetected distribution shifts to corrupt closed-loop actuation.

A.3.2 Visual patch tokenization, attention, and 3D metric lifting

Embodied vision-language-action (VLA) models and spatial transformers convert continuous sensory streams into discrete tokens for neural attention, then lift candidate predictions into 3D Euclidean coordinates for physical reasoning. Section 3.3 and section 3.4 use the token counts derived here to size the application processor’s memory traffic.

A.3.2.1 2D vision transformer patch tokenization

Consider an RGB image $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ captured by an onboard camera with spatial resolution $H \times W$ and C color channels. Feeding raw pixels to a multilayer perceptron would discard spatial locality and inflate the parameter count, so a vision transformer frontend partitions the image into a non-overlapping sequence of N patches of $P \times P$ pixels, with P typically 14 or 16:

$$N = \frac{H \cdot W}{P^2}$$

Each patch, indexed by its row and column (u, v) in the camera plane, is flattened into a vector $\mathbf{x}_p^{(u,v)} \in \mathbb{R}^{P^2 C}$. A trainable linear projection $\mathbf{W}_{\text{patch}} \in \mathbb{R}^{(P^2 C) \times d_{\text{model}}}$ maps each flattened patch into the transformer’s hidden dimension d_{model} . Self-attention is permutation-invariant and treats its inputs as an unordered set, so the model knows nothing of spatial proximity unless position is added explicitly:

$$\mathbf{z}_0^{(u,v)} = \mathbf{x}_p^{(u,v)} \mathbf{W}_{\text{patch}} + \mathbf{E}_{\text{pos}}^{2D}(u, v) + \mathbf{e}_{\text{cam}}$$

The 2D position embedding $\mathbf{E}_{\text{pos}}^{2D}(u, v)$ decomposes along the vertical axis u and the horizontal axis v , using sinusoidal frequency bands or learned coordinate embeddings. It encodes position in the image plane and carries no metric 3D distance. When several cameras feed one model, such as an overhead camera for scene context and a wrist camera for millimeter-scale alignment, a learned camera identifier $\mathbf{e}_{\text{cam}} \in \{\mathbf{e}_{\text{overhead}}, \mathbf{e}_{\text{wrist}}\}$ distinguishes their reference frames. Some architectures also prepend a summary token $\mathbf{x}_{\text{class}} \in \mathbb{R}^{d_{\text{model}}}$ to the sequence.

To make the dimensions concrete, take a square RGB frame 224 pixels on a side (illustrative; see the Reader Guide) with patch size $P = 14$. The frame divides into 16 patches on a side, 256 in all. Each patch holds 196 pixels across 3 color channels, a flattened vector of 588 values, and $\mathbf{W}_{\text{patch}}$ maps each such vector into a 4096-dimensional model space. One frame therefore becomes 256 tokens of width 4096. A robot with 3 cameras (two wrist cameras and one overhead) supplies $3 \times 256 = 768$ visual tokens per step before any text or proprioceptive token is added.

The visual tokens then join the rest of the input. A natural-language directive such as “grasp the red mug by its handle” is tokenized by byte-pair encoding into text embeddings $\mathbf{z}_{\text{text}} \in \mathbb{R}^{L \times d_{\text{model}}}$. The robot’s configuration (joint angles $\mathbf{q}_t$, joint velocities $\dot{\mathbf{q}}_t$, and gripper state) passes through a linear layer into proprioceptive tokens $\mathbf{z}_{\text{state}} \in \mathbb{R}^{K_s \times d_{\text{model}}}$. The three streams concatenate into one multimodal input sequence:

$$\mathbf{Z} = [\mathbf{z}_{\text{vision}} | \mathbf{z}_{\text{text}} | \mathbf{z}_{\text{state}}] \in \mathbb{R}^{N_{\text{total}} \times d_{\text{model}}}$$

A.3.2.2 Self-attention over the combined sequence

The combined sequence $\mathbf{Z}$ passes through a stack of transformer layers, 32 to 80 in current VLA backbones, each consisting of multi-head self-attention and a feed-forward network. Self-attention projects the tokens into queries $\mathbf{Q} = \mathbf{Z} \mathbf{W}_Q$, keys $\mathbf{K}_{\text{attn}} = \mathbf{Z} \mathbf{W}_K$, and values $\mathbf{V} = \mathbf{Z} \mathbf{W}_V$, and computes pairwise attention weights over the whole sequence:

$$A_{i,j} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_m \exp(\mathbf{q}_i^\top \mathbf{k}_m / \sqrt{d_k})}$$

A text token such as “handle” can attend to visual patch tokens, but a high attention weight is not a verified grasp location or metric pose. A trained spatial decoder may produce a 2D affordance heatmap, and placing the gripper in 3D still takes depth evidence, camera calibration, and the metric lifting below. A rolling key-value (KV) cache stores prior token states for attention. It neither tracks object permanence nor maintains a 3D position and velocity; an explicit state estimator must associate observations across frames and widen its uncertainty while the mug is occluded.

A.3.2.3 Pinhole back-projection and 3D metric lifting

Standard 2D patch tokens lack metric scale due to projective ambiguity: along any optical ray $\lambda[u, v, 1]^\top$, infinitely many 3D points project to identical pixel coordinates. To interface with physical control barrier filters, kinetic stopping calculators, and obstacle clearance envelopes, 2D visual predictions must be lifted into metric 3D coordinates $\mathbf{p}_W \in \mathbb{R}^3$.

Let $\mathbf{u} = [u, v]^\top$ denote pixel coordinates on the image sensor, and let the calibrated pinhole camera intrinsic matrix be:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

where (f_x, f_y) are focal lengths in pixels and (c_x, c_y) is the principal point. Given an aligned metric depth measurement $d = z > 0$ (obtained from stereo disparity, time-of-flight lidar, or calibrated depth sensors), the 3D position in the camera coordinate frame $\mathbf{p}_C \in \mathbb{R}^3$ is recovered via inverse intrinsic projection:

$$\mathbf{p}_C = d \cdot \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} (u - c_x) \frac{d}{f_x} \\ (v - c_y) \frac{d}{f_y} \\ d \end{bmatrix}$$

Transforming $\mathbf{p}_C$ into the robot base or world frame W uses the rigid extrinsic transform $\mathbf{T}_{WC} \in SE(3)$ defined in this section:

$$\tilde{\mathbf{p}}_W = \mathbf{T}_{WC} \tilde{\mathbf{p}}_C \implies \mathbf{p}_W = \mathbf{R}_{WC} \mathbf{p}_C + \mathbf{t}_{WC}$$

The permission path and kinematic reachability evaluators require this metric 3D reconstruction; ungrounded 2D tokens cannot establish whether a proposed trajectory satisfies physical clearance boundaries d_{stop} .

A.3.3 Camera-to-bird's-eye-view lifting

A bird's-eye-view (BEV) representation places the features of every camera in one metric grid on the ground plane, the frame in which planners and clearance checks work. The **Lift-Splat-Shoot (LSS)** architecture (Phillion and Fidler 2020) builds that grid from monocular features in three steps, the first three stages of figure A.1.

1. **Feature extraction.** A vision backbone, such as DINOv2 (Oquab et al. 2023) or a lightweight convolutional network, maps image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ to a grid of feature vectors:

$$\mathbf{f}_{u,v} = \text{Backbone}(\mathbf{I})[u, v] \in \mathbb{R}^C$$

Forward projection (equation 8.2) collapses each 3D ray onto one pixel, so a feature $\mathbf{f}_{u,v}$ stands for a whole ray in space, and its metric depth is unknown.

2. **Lift.** LSS represents that ambiguity rather than resolving it. Along the bearing ray $\mathbf{K}^{-1}[u, v, 1]^\top$ of equation 8.3, the network scores D discrete depth bins spanning $[d_{\min}, d_{\max}]$, for example 64 bins from 0.5 m to 20 m, spaced linearly or logarithmically:

$$\mathbf{p}_{u,v} = \text{softmax}(\mathbf{d}_{u,v}) \in \Delta^{D-1}$$

An outer product lifts each 2D feature into a frustum of 3D points, one per depth hypothesis:

$$\mathbf{c}_{u,v,k} = \mathbf{p}_{u,v}(k) \cdot \mathbf{f}_{u,v} \in \mathbb{R}^C \quad \text{at } \mathbf{P}_c(u, v, d_k) = d_k \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

For a feature map 128 cells on a side, with 64 depth bins and 64 feature channels, the frustum tensor holds about 67.1 million values, 134.2 MB per frame at FP16. At camera rate that tensor

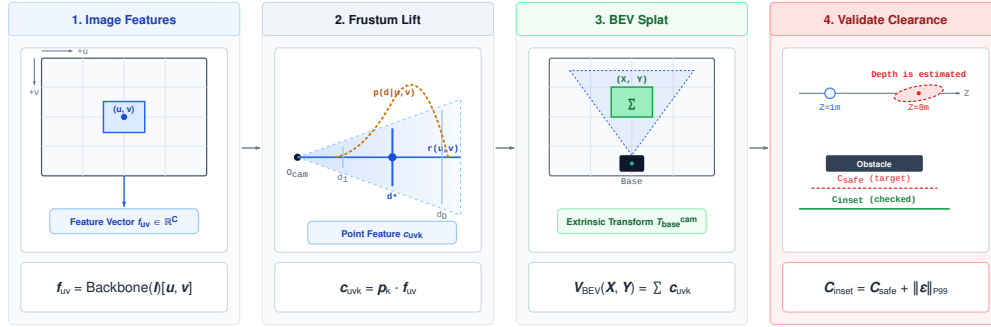


Figure A.1: Camera ray frustum lifting and BEV splatting: LSS (Phillion and Fidler 2020) places learned depth hypotheses from each camera in a shared metric BEV frame; that coordinate transform does not resolve monocular depth ambiguity. The final clearance inset requires separately validated metric residuals, bias, and a chosen tail quantile or deterministic error bound before permission. (Architecture adapted from Phillion and Fidler (2020).)

is memory traffic the application processor must carry, which is why implementations keep it in zero-copy buffers on the accelerator.

3. **Splat.** The calibrated extrinsic transform $\mathbf{T}_{\text{base}}^{\text{cam}} = [\mathbf{R}_{\text{base}}^{\text{cam}} | \mathbf{t}_{\text{base}}^{\text{cam}}] \in SE(3)$ maps every frustum point from the camera's optical frame into the robot base frame:

$$\mathbf{P}_{\text{base}} = \mathbf{R}_{\text{base}}^{\text{cam}} \mathbf{P}_c + \mathbf{t}_{\text{base}}^{\text{cam}}$$

The workspace is discretized into a grid of vertical pillars, for example 200 cells on a side covering a 20 m square at 0.1 m resolution. The splat sums every frustum feature that falls within pillar (X, Y) , using accelerated cumulative sums or point-pillar pooling (Phillion and Fidler 2020; Z. Liu et al. 2023):

$$\mathbf{V}_{\text{BEV}}(X, Y) = \sum_{(u,v,k) \in \text{Pillar}(X,Y)} \mathbf{c}_{u,v,k}$$

The splat aligns the cameras' hypotheses in one metric frame. It cannot fill a region no camera saw or remove depth-scale error, so the footprint it assigns to a pedestrian still depends on the depth hypotheses and the calibration.

The softmax over depth bins expresses weights the model assigns, and its entropy is not a calibrated metric covariance. The clearance inset of section 8.5 therefore rests on separately validated spatial residuals, not on the lifted weights.

A.3.4 Inverse-depth parameterization

In visual-inertial odometry and SLAM, representing distant 3D landmarks in standard Cartesian coordinates (x, y, z) induces extreme non-linearity: small pixel tracking errors produce quadratic depth errors ($\delta z \propto z^2$).

Under **inverse-depth parameterization**, a 3D feature is parameterized by the 6D vector $\mathbf{y} = [x_0, y_0, z_0, \theta, \phi, \rho]^\top$, where (x_0, y_0, z_0) is the optical center when first observed, (θ, ϕ) are azimuth and elevation angles, and $\rho = 1/d$ is inverse depth:³

$$\mathbf{p}(d) = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} + \frac{1}{\rho} \begin{bmatrix} \cos \phi \sin \theta \\ -\sin \phi \\ \cos \phi \cos \theta \end{bmatrix}$$

At optical infinity ($d \rightarrow \infty$), inverse depth approaches zero ($\rho \rightarrow 0$). This parameterization can improve estimator conditioning for distant features; covariance shape and numerical stability still depend on the measurement model, noise assumptions, and estimator implementation.

³ | **Inverse-Depth Parameterization:** Civera and colleagues used inverse depth to represent distant visual landmarks in monocular estimation.



Figure A.2: Gaussian memory and clearance handoff: Camera observations update a Gaussian scene for novel-view rendering. Its visual output does not directly supply conservative collision distance; a separately validated geometry or occupancy conversion and permission check are required (Kerbl et al. 2023; Matsuki et al. 2024).

A.3.5 3D Gaussian splatting parameters

Continuous neural scene representations parameterize physical environments through collections of 3D Gaussians (Kerbl et al. 2023). Each Gaussian $G_i(\mathbf{x})$ is defined by four core attributes:

1. **Position** ($\mu_i \in \mathbb{R}^3$): The 3D spatial center.
2. **Covariance matrix** ($\Sigma_i \in \mathbb{R}^{3 \times 3}$): Defining ellipsoidal shape and orientation. So that Σ_i stays positive semi-definite throughout gradient descent, it is factorized into a rotation matrix $\mathbf{R}_i \in SO(3)$ (parameterized by a unit quaternion $\mathbf{q}_i$) and a scaling diagonal matrix $\mathbf{S}_i = \text{diag}(s_{x,i}, s_{y,i}, s_{z,i})$:
$$\Sigma_i = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^\top \mathbf{R}_i^\top$$
3. **Color representation** (c_i): Parameterized via Spherical Harmonics (SH) coefficients to model view-dependent specular radiance.
4. **Opacity** ($\alpha_i \in [0, 1]$): Modeled via a sigmoid activation $\alpha_i = \sigma(o_i)$ to ensure continuous density gradients.

During real-time rendering, 3D Gaussians are projected onto camera coordinate frames via Jacobian approximation $\mathbf{J} \in \mathbb{R}^{2 \times 3}$, yielding projected 2D covariance:

$$\Sigma_{2D} = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^\top \mathbf{J}^\top$$

where $\mathbf{W} \in SO(3)$ is the rotational part of the world-to-camera transform; the translation enters the projected mean, not this covariance. Differentiable tile-based rasterization sorts Gaussians by depth and evaluates alpha-blended color accumulation:

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

These differentiable parameters support novel-view rendering. Neither the Gaussian density nor its spatial derivative is a conservative collision surface, so clearance permission needs a separately validated geometric extraction, occupancy/ESDF conversion, or independent range sensor that accounts for unknown space and geometric error (figure A.2). Rendering rate and collision-query latency are separate measurements. Section 9.5 relies on this distinction when it refuses to let a rendered view stand in for a clearance bound.

Storage per primitive depends on its packing and spherical-harmonic degree, so a map's footprint is its primitive count times that packing. A dense cubic grid instead stores $O(N^3)$ cells, and a sparse occupancy map stores only allocated structure, whose size depends on the observed geometry and the accuracy target. Figure A.3 sets the three storage models side by side.

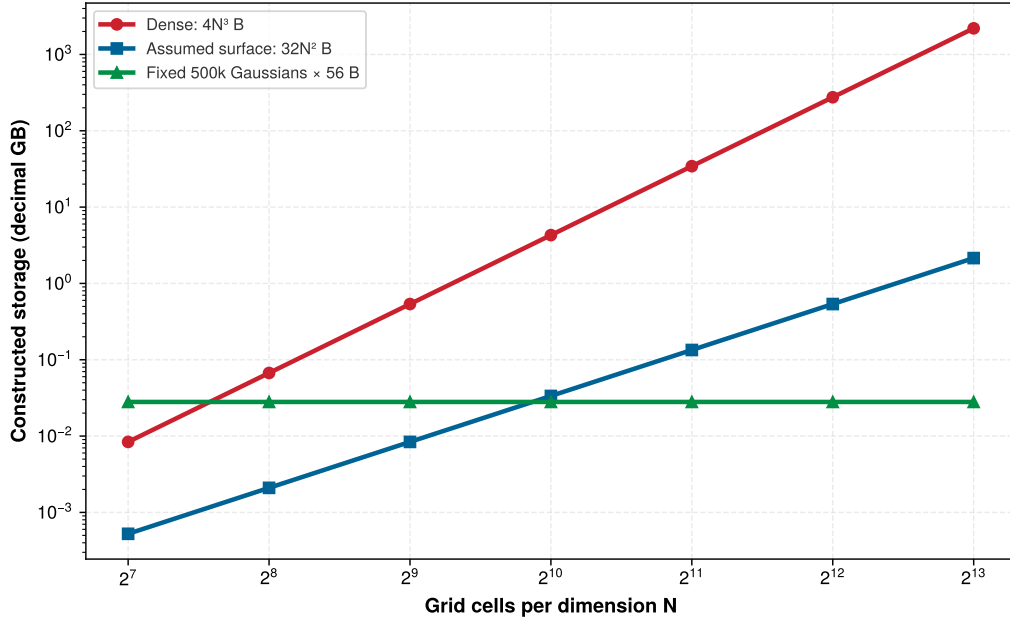


Figure A.3: Constructed spatial-memory storage scenarios: A dense cube uses four bytes per voxel, the octree allocates one 32-byte surface cell per squared grid dimension, and the Gaussian scene holds a fixed count and packing while grid resolution varies. These are storage models, not equal-fidelity or measured runtime benchmarks; Gaussian count must rise if a stricter geometry target requires it.

For the Gaussian packing in the plot, 500,000 primitives at 56 bytes each require 28 MB. A deployment chooses its representation after measuring map completeness, update time, query time, and the conversion needed for conservative clearance.

A.3.6 Log-odds occupancy and signed-distance fusion

Discrete volumetric grids model occupancy uncertainty through recursive Bayesian estimation. Let m_i denote the binary occupancy state of voxel i ($m_i = 1$ occupied, $m_i = 0$ free).

Using the log-odds transformation $L(x) = \ln \frac{P(x)}{1-P(x)}$, recursive Bayesian filtering collapses nonlinear probability multiplication into additive linear updates:⁴

$$L_t(m_i) = L_{t-1}(m_i) + \ln \frac{P(m_i | z_t)}{1 - P(m_i | z_t)} - \ln \frac{P(m_i)}{1 - P(m_i)}$$

where $P(m_i | z_t)$ is the inverse sensor model and $P(m_i)$ the prior. Along a reliable range ray, traversed cells receive free-space evidence and the measured return receives occupied evidence; space behind the return remains unknown unless observed separately. The additive update can use bounded fixed-point log odds, but sensor error, moving objects, and evidence age still matter.

A.3.6.1 Octree occupancy and free-space raycasting

Occupancy grid mapping (Elfes 1989) tessellates space into cells and applies this update to every cell a ray touches. A volumetric raycast walks each return's sightline with the three-dimensional digital differential analyzer (3D-DDA) traversal (Amanatides and Woo 1987). Hierarchical stores such as OctoMap (Hornung et al. 2013) and OpenVDB (Museth 2013) allocate cells only where rays have passed, so storage follows the observed volume rather than the bounding cube (figure A.4). Free-space evidence lets the map retire a phantom obstacle once the same region is seen empty, while a never-observed cell stays unknown however long the map runs. Section 9.5 applies this rule to the aisle, where only fresh free-space evidence can vouch that a path is clear.

⁴ | **Recursive Log-Odds Occupancy:** The log-odds formulation maps probability values from the bounded unit interval to the unbounded real line, turning nonlinear Bayesian product updates into fast additions. Dynamic ranges are clamped by upper and lower saturation bounds to prevent saturation lockup. Failure to clamp log-odds values delays the clearing of dynamic obstacles when physical objects vacate the workspace.

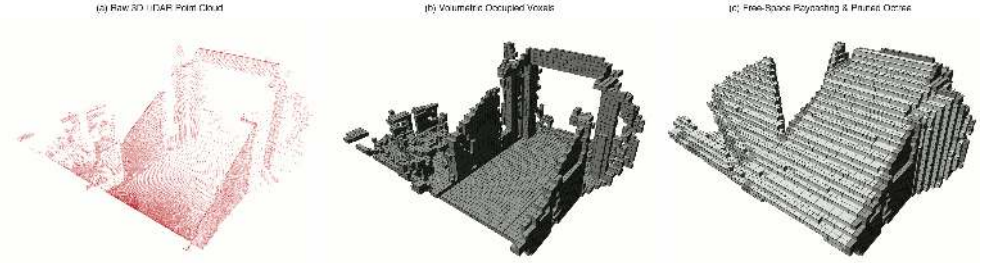


Figure A.4: Probabilistic octree volumetric mapping: A corridor traversal reconstructed from raw point cloud into a probabilistic octree and then into explicit free, occupied, and unknown space (Hornung et al. 2013). The third panel is what distinguishes spatial memory from an accumulated point cloud, because raycasting records evidence of emptiness along sightlines, letting the map actively retire a phantom obstacle track instead of waiting for a timeout to expire it. (Credit: Hornung et al. / Autonomous Intelligent Systems Lab, University of Freiburg, CC BY 3.0).

A.3.6.2 Truncated signed distance fusion

A truncated signed distance field (TSDF), fused volumetrically at camera rate by KinectFusion (Newcombe et al. 2011) and incrementally on robots by Voxblox (Oleynikova et al. 2017), stores in each voxel $\mathbf{v}$ a *metric projective distance* $D(\mathbf{v}) \in [-\mu, +\mu]$, positive in front of an observed return along its sensing ray, and a dimensionless fusion weight $W(\mathbf{v})$. For a calibrated depth image, transform the voxel center into camera coordinates $\mathbf{p} = [p_x, p_y, p_z]^\top$ and project it to pixel (u, v) . Its projective sample is $s = Z_{\text{meas}}(u, v) - p_z$. For a range ray, the equivalent along-ray sample is $s = r_{\text{hit}} - r_{\text{voxel}}$; both are positive before the hit. Near the return, the sample is clamped in meters and fused into the running weighted average:

$$d_\mu(\mathbf{v}) = \max(-\mu, \min(\mu, s)), \quad D_t(\mathbf{v}) = \frac{W_{t-1}(\mathbf{v})D_{t-1}(\mathbf{v}) + w_t d_\mu(\mathbf{v})}{W_{t-1}(\mathbf{v}) + w_t}$$

where w_t is the dimensionless observation weight and the stored weight $W_t = W_{t-1} + w_t$ is capped at W_{max} . Fusion uses a valid capture-time pose and refuses a sample older than the voxel's last evidence epoch. A negative sample just behind the measured return contributes to the local zero crossing under the surface model; it does not certify an unseen obstacle interior.

Interpolating observed samples gives an approximate gradient near a zero crossing, limited by unknown voxels, projective bias, discretization, and moving objects. A projective TSDF is therefore not the nearest-surface Euclidean distance a clearance decision needs. A clearance query therefore reads a validated occupancy or Euclidean signed distance field (ESDF) with an unknown-space policy, the conversion section 9.5 requires of every scene representation before the permission path relies on it.

A.3.6.3 One frame's update and its memory traffic

Algorithm A.1 applies both updates for one depth frame, and the notebook after it prices the update traffic that section 9.5 charges against the memory bus. It rejects the frame before touching the map when its capture epoch is too old or older than the last evidence of any cell it would update, then clears and marks occupancy along each ray and fuses TSDF samples only inside the truncation band. Every updated cell takes the capture epoch, not the update time, so a later consumer can judge each cell's freshness.

Algorithm A.1 Occupancy Raycast and Metric TSDF Fusion

Require: returns $\mathcal{P}$ captured at t_0 in one capture clock; valid pose; occupancy $\mathcal{O}$; metric TSDF $\mathcal{V}$; truncation μ ; evidence half-life $t_{1/2}$; per-cell serialized updates

Ensure: observed free/occupied cells and near-surface TSDF samples updated without refreshing unseen cells

```

1:  $(t_{0,c}, \epsilon_{\text{sync}}) \leftarrow \text{CONVERTCAPTURETOCONTROLLERCLOCK}(t_0)$ 
2: if clock conversion invalid or  $t_{\text{now},c} - t_{0,c} + \epsilon_{\text{sync}} > \tau_{\text{max,age}}$  then
3:   return STATUS_FRAME_TOO_STALE
4: end if
5: if any target occupancy or TSDF cell has  $t_{\text{last}} > t_0$  then
6:   return STATUS_OUT_OF_ORDER  $\triangleright$  preflight before modifying the map
7: end if
8: for each return  $\mathbf{p}_k \in \mathcal{P}$  do
9:    $(\mathbf{r}_{\text{dir}}, r_{\text{hit}}) \leftarrow \text{RAYFROMPOSEANDRETURN}(\mathbf{p}_k)$ 
    $\triangleright$  Occupancy: measured free ray and hit, not TSDF distance
10:  for each cell  $c$  traversed before the hit do
11:     $c \leftarrow \text{GETORALLOCATEOCCUPANCY}(\mathcal{O}, c)$ 
12:     $c.L \leftarrow \text{CLAMPLOGODDS}(c.L + L_{\text{free}}); c.t_{\text{last}} \leftarrow t_0$ 
13:  end for
14:  update the observed hit cell with  $L_{\text{occupied}}$  and epoch  $t_0$ 
    $\triangleright$  TSDF: fuse metric samples only in a narrow surface band
15:  for each voxel  $v$  with  $|r_{\text{hit}} - r_v| \leq \mu$  do
16:     $\text{vox} \leftarrow \text{GETORALLOCATETSDF}(\mathcal{V}, v)$ 
17:     $d_\mu \leftarrow \text{CLAMP}(r_{\text{hit}} - r_v, -\mu, +\mu) \triangleright$  positive before hit
18:     $W_{\text{old}} \leftarrow 0$  if  $\text{vox}.W = 0$ , else  $\text{vox}.W \cdot 2^{-(t_0 - \text{vox}.t_{\text{last}})/t_{1/2}}$ 
19:     $w_{\text{obs}} \leftarrow 1 \triangleright$  dimensionless illustrative confidence
20:     $\text{vox}.D \leftarrow (W_{\text{old}} \text{vox}.D + w_{\text{obs}} d_\mu) / (W_{\text{old}} + w_{\text{obs}})$ 
21:     $\text{vox}.W \leftarrow \min(W_{\text{old}} + w_{\text{obs}}, W_{\text{max}}); \text{vox}.t_{\text{last}} \leftarrow t_0$ 
22:  end for
23: end for
24: return STATUS_MAP_UPDATED

```



Napkin Math A.1: DRAM bandwidth of 3D TSDF voxel grids

Practical Engineering Question: What are the distance-and-weight payload and modeled DRAM bandwidth of sparse versus dense TSDF representations?

1. *Dense allocation.* A manipulation workcell of $10 \text{ m} \times 10 \text{ m} \times 3 \text{ m}$ (300 m^3) at 1 cm pitch holds 300,000,000 voxels. At 4 bytes per voxel (16-bit distance plus 16-bit fusion weight), excluding occupancy, per-cell evidence epochs, and allocator overhead, the distance-and-weight payload is 1.20 GB. Refining the pitch to 5 mm for tool contact raises the count to 2,400,000,000 voxels and the partial payload to 9.60 GB, which alone exceeds an illustrative 8 GB memory budget.

2. *Sparse allocation.* Surfaces occupy a thin manifold within the volume. Grouping voxels into 512-voxel leaf blocks and allocating only within the truncation band, assumed to be 3 percent of the volume, gives 17,579 blocks. With a 32-byte leaf header, the payload is 36.6 MB at 1 cm and 292.5 MB at 5 mm. A full map must also budget the separate occupancy layer, per-cell epochs, block lookup and allocator metadata, and capacity for the unknown-space policy.

3. *Traversal and DRAM traffic.* A 640×480 depth stream at 60 Hz supplies 18.432 million rays per second. Along a 3 m ray at 1 cm pitch the traversal visits 300 cells. The 294 cells before the hit receive occupancy free-ray updates, and the 6 cells near the return receive TSDF fusion. Sparse TSDF allocation saves map capacity; it removes neither the traversal nor the occupancy updates. At 8 bytes of assumed read/write traffic per update, sending every update to DRAM costs 44.2 GB/s. At an assumed 95 percent cache hit fraction for both occupancy and TSDF

updates, the modeled DRAM demand falls to 2.21 GB/s, a 20× reduction. The hit rate depends on ray overlap, atomics, evictions, and writeback on the chosen hardware; it is an input to measure, not a property of the map structure.

Systems insight: Sparse allocation yields 33× less modeled TSDF payload in this surface-fraction scenario, while bus demand depends on measured locality and on all traversed-cell work. Whether that demand fits beside policy inference on a shared bus is the contention question of section 13.3.

A.4 Function Approximation and Sensitivity Bounds

A neural network that maps sensor state to commands needs a bound on how far its output can move when its input moves.

A.4.1 Lipschitz continuity and spectral normalization

A neural policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$ is L -Lipschitz continuous if there exists a non-negative scalar constant L such that for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}$:

$$\|\pi(\mathbf{x}_1) - \pi(\mathbf{x}_2)\|_{\mathcal{U}} \leq L \|\mathbf{x}_1 - \mathbf{x}_2\|_{\mathcal{X}}$$

For a feedforward network with K layers $\pi(\mathbf{x}) = \phi_K(\mathbf{W}_K \dots \phi_1(\mathbf{W}_1 \mathbf{x}))$, let L_{ϕ_k} bound each activation's slope over the input domain. The composed bound is:

$$L \leq \prod_{k=1}^K L_{\phi_k} \|\mathbf{W}_k\|_2.$$

ReLU and Tanh are 1-Lipschitz in their usual scalar forms; exact GeLU has maximum derivative about 1.1289, so assigning it a factor of one would understate the bound.

A.4.2 Perturbation amplification and supervisory clamping

In physical systems, input state $\mathbf{x}$ is corrupted by sensor measurement noise $\delta\mathbf{x}$ bounded by $\|\delta\mathbf{x}\| \leq \epsilon$. The resulting deviation in commanded control action is bounded by:

$$\|\delta\mathbf{u}\| = \|\pi(\mathbf{x} + \delta\mathbf{x}) - \pi(\mathbf{x})\| \leq L\epsilon$$

The constant has output-units per input-unit. For an illustrative policy mapping position in meters to torque in newton-meters, $L = 10^3 \text{ N} \cdot \text{m/m}$ and $\epsilon = 1 \text{ mm}$ allow up to $1 \text{ N} \cdot \text{m}$ of command change. Whether that exceeds torque headroom depends on the joint and operating state. **Spectral normalization** can reduce an upper bound on network sensitivity. It bounds the network, not the plant, so stability, safe torque, and sensor validity still come from a separate actuator limit and a state-dependent permission check.

A.5 Imitation Learning and Compounding Covariate Shift

Behavioral cloning frames physical policy acquisition as supervised regression over demonstrator state-action pairs.

A.5.1 Mode averaging and the decoders that avoid it

Consider an imitation learning objective minimizing mean squared error (L_2 loss) over demonstration distribution $P(o, a)$:

$$\min_{\pi} \mathbb{E}_{(o,a) \sim \mathcal{D}} [\|a - \pi(o)\|^2]$$

Expanding the objective shows that the optimal unconstrained regressor $\pi^*(o)$ evaluates to the **conditional expectation**:

$$\pi^*(o) = \mathbb{E}[a \mid o] = \int a P(a \mid o) da$$

When demonstrator behavior is inherently multimodal—such as dodging an obstacle by steering left ($a = +1.0$) or steering right ($a = -1.0$) with equal probability $P(a = +1) = P(a = -1) = 0.5$ —the conditional expectation computes the arithmetic mean:

$$\pi^*(o) = 0.5(+1.0) + 0.5(-1.0) = 0.0$$

Executing this averaged action commands zero steering deflection, driving the physical robot directly into the central obstacle.

A.5.1.1 Decoder families

A decoder avoids the mean by representing the conditional action distribution rather than its expectation. The families below trade inference cost against the distributions they can express, and none of them supplies recovery states that the demonstrations lack. Section 6.2 compares them by the network evaluations each chunk costs, which is where the choice reaches the control deadline.

1. **Mixture density networks.** A Gaussian mixture model (GMM) head parameterizes the policy output as a conditional mixture of M Gaussian components:

$$\pi(a | o) = \sum_{m=1}^M w_m(o) \mathcal{N}(\mu_m(o), \Sigma_m(o))$$

where the network predicts the mixture weights $w_m(o)$, component means $\mu_m(o)$, and covariances $\Sigma_m(o)$. It gives a closed-form log-likelihood and an exact density in a single forward pass. Maximum-likelihood training is prone to mode collapse and numerical singularity, the head scales poorly to high-dimensional multi-step trajectories, and the component count M must be fixed in advance, which fails when demonstrations branch unevenly.

2. **Implicit behavioral cloning.** Implicit behavioral cloning (IBC) (Florence et al. 2022) learns an energy-based model $E_\theta(o, a) \in \mathbb{R}$, trained with contrastive or InfoNCE objectives to assign low energy to demonstrated state-action pairs and high energy to counterfactual actions. Inference approximately solves

$$a^* = \arg \min_a E_\theta(o, a)$$

The energy can represent discontinuous, sharply multimodal distributions and non-convex boundaries that a continuous feedforward map cannot. Inference is an iterative search whose cost and quality depend on the optimizer, the candidate count, and the device, and a non-convex energy can return a local minimum.

3. **Conditional diffusion and flow matching.** Diffusion Policy (Chi et al. 2024) and flow-matching decoders generate a whole action chunk by iterative denoising or ODE integration. Their training target is the injected noise or a velocity field, conditioned on the corrupted chunk, the observation, and the step, rather than the clean action itself, so the squared-error loss never averages incompatible modes. Section A.5.2 derives both objectives and prices their sampling steps.
4. **Latent-variable chunking.** Action Chunking with Transformers (ACT) (Zhao et al. 2023) predicts a chunk $\mathbf{A}_t = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+K-1}) \in \mathbb{R}^{K \times D_a}$ from the current observation $\mathbf{o}_t$ with a conditional variational autoencoder (CVAE) built from a Transformer encoder and decoder. During training, the encoder compresses demonstrated chunks and observations into a low-dimensional latent style variable $\mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{o}_t, \mathbf{A}_t)$, optimized through the variational lower bound:

$$\mathcal{L}_{\text{ACT}}(\theta, \phi) = \mathbb{E}_{(\mathbf{o}_t, \mathbf{A}_t) \sim \mathcal{D}, \mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{o}_t, \mathbf{A}_t)} [\|\mathbf{A}_t - \pi_\theta(\mathbf{o}_t, \mathbf{z})\|_1] + \beta D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{o}_t, \mathbf{A}_t) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))$$

At inference the latent code is sampled from the prior $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ or set to its mean $\mathbf{z} = \mathbf{0}$. Each code selects one coherent demonstrated intention, such as passing an obstacle on the left or routing a cable into the upper fixture, and the decoder emits an internally consistent chunk for it.

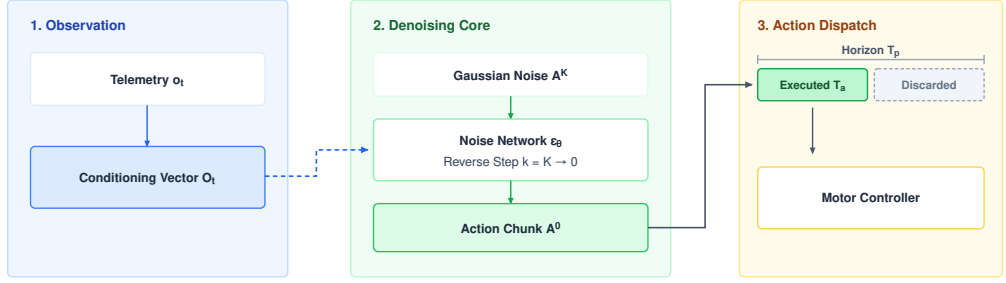


Figure A.5: Diffusion policy architecture: A continuous trajectory diffusion policy for visuomotor control, denoising a candidate action chunk from Gaussian noise across K iterations and dispatching only a prefix of it before re-planning. Asynchronous buffering and an independent high-rate permission path can separate slower chunk generation from actuation. Conditional denoising can represent alternative demonstrated paths without taking their direct-regression mean. (Adapted from Chi et al. (Chi et al. 2024)).

5. **Discrete action tokens.** Vision-language-action models such as RT-1, RT-2, and OpenVLA (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024) discretize each continuous action dimension into B uniform bins, commonly 256, and emit one token per dimension:

$$\text{bin}(a_k) = \left\lfloor \frac{a_k - a_{\min}}{a_{\max} - a_{\min}} \times (B - 1) \right\rfloor$$

A categorical output avoids mode averaging and reuses a language model’s output head. It incurs a quantization error of $\Delta a = (a_{\max} - a_{\min})/B$ and needs D_a sequential token predictions per action, so its latency grows with the action dimension.

A.5.2 Continuous trajectory diffusion and flow matching

Continuous trajectory diffusion treats policy synthesis as generative denoising over continuous multi-step action trajectory chunks $\mathbf{A} \in \mathbb{R}^{K \times D_a}$ conditioned on visual and proprioceptive observation histories $\mathbf{O}$ (Chi et al. 2024). Rather than predicting an unconstrained point estimate via mean squared error, diffusion models represent complex multimodal action distributions without collapsing into mode averaging.

Figure A.5 shows the policy’s structure. It conditions on a short history of camera images and robot state, denoises a candidate chunk from Gaussian noise over K iterations, and dispatches only a prefix of the chunk before it plans again.

A.5.2.1 Forward diffusion and variance scheduling

The forward diffusion process progressively corrupts a ground-truth demonstrated action chunk $\mathbf{A}^0 \sim q(\mathbf{A})$ with Gaussian noise across N discrete timesteps (or continuous diffusion time $t \in [0, 1]$):

$$q(\mathbf{A}^k | \mathbf{A}^{k-1}) = \mathcal{N}(\mathbf{A}^k; \sqrt{1 - \beta_k} \mathbf{A}^{k-1}, \beta_k \mathbf{I})$$

where $\{\beta_k \in (0, 1)\}_{k=1}^N$ denotes a predefined variance schedule. Defining $\alpha_k = 1 - \beta_k$ and $\bar{\alpha}_k = \prod_{s=1}^k \alpha_s$, the marginal distribution at step k admits a closed-form Gaussian sample directly from $\mathbf{A}^0$:

$$q(\mathbf{A}^k | \mathbf{A}^0) = \mathcal{N}(\mathbf{A}^k; \sqrt{\bar{\alpha}_k} \mathbf{A}^0, (1 - \bar{\alpha}_k) \mathbf{I})$$

allowing immediate closed-form sampling of noisy trajectories without simulating intermediate Markov steps:

$$\mathbf{A}^k = \sqrt{\bar{\alpha}_k} \mathbf{A}^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

A.5.2.2 Reverse denoising and score-matching objective

The generative reverse process synthesizes an action trajectory from pure Gaussian noise $\mathbf{A}^N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ by parameterizing the conditional reverse transition distribution $p_\theta(\mathbf{A}^{k-1} | \mathbf{A}^k, \mathbf{O})$:

$$p_\theta(\mathbf{A}^{k-1} | \mathbf{A}^k, \mathbf{O}) = \mathcal{N}(\mathbf{A}^{k-1}; \mu_\theta(\mathbf{A}^k, \mathbf{O}, k), \Sigma_k)$$

Optimizing the variational lower bound on data likelihood reparameterizes the mean μ_θ into a noise-prediction network ϵ_θ :

$$\mu_\theta(\mathbf{A}^k, \mathbf{O}, k) = \frac{1}{\sqrt{\alpha_k}} \left(\mathbf{A}^k - \frac{\beta_k}{\sqrt{1 - \alpha_k}} \epsilon_\theta(\mathbf{A}^k, \mathbf{O}, k) \right)$$

The noise network is optimized via the simplified L_2 denoising score-matching objective:

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{k \sim \mathcal{U}\{1, N\}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), (\mathbf{O}, \mathbf{A}^0) \sim \mathcal{D}} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\alpha_k} \mathbf{A}^0 + \sqrt{1 - \alpha_k} \epsilon, \mathbf{O}, k \right) \right\|^2 \right]$$

By Tweedie's formula, the noise estimate is proportional to the Stein score function (the spatial gradient of the log data density): $\nabla_{\mathbf{A}^k} \ln p(\mathbf{A}^k | \mathbf{O}) \approx -\epsilon_\theta(\mathbf{A}^k, \mathbf{O}, k) / \sqrt{1 - \alpha_k}$. Conditioning on this score directs stochastic sampling toward high-density demonstration modes rather than averaging incompatible modes into an obstacle collision.

A.5.2.3 Continuous flow matching and vector field regression

While standard diffusion integrates stochastic SDEs, **continuous flow matching** (Lipman et al. 2022) generates trajectories along deterministic probability flows defined by ordinary differential equations (ODEs):

$$\frac{d\mathbf{x}_t}{dt} = v_\theta(\mathbf{x}_t, t, \mathbf{O})$$

Under an optimal transport conditional vector field, the continuous path interpolates linearly between source noise $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and demonstration target $\mathbf{x}_1 \sim q(\mathbf{x})$:

$$\psi_t(\mathbf{x}_0) = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad t \in [0, 1]$$

The ground-truth velocity field along this path is constant: $\dot{\psi}_t(\mathbf{x}_0) = \mathbf{x}_1 - \mathbf{x}_0$. The neural network v_θ is trained via direct regression:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0, 1], \mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \mathbf{x}_1 \sim \mathcal{D}} \left[\left\| v_\theta((1 - t)\mathbf{x}_0 + t\mathbf{x}_1, t, \mathbf{O}) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|^2 \right]$$

Straighter conditional paths let a low-order solver such as Euler or Runge-Kutta reach a given action quality in fewer function evaluations than a curved diffusion trajectory needs. How few evaluations suffice, and what latency they cost, are properties of the trained model and the device rather than of flow matching itself.

A.5.2.4 Denoising steps against the control deadline

Reverse sampling evaluates the denoiser once per step, in sequence, so a chunk's inference time grows with the number of steps.⁵ A decoder whose chunk time exceeds the setpoint period cannot run in lockstep with the controller. It runs asynchronously instead. The application processor generates chunks at a lower rate into a buffer across the proposal boundary (section 1.5), and the permission path samples the newest chunk at its own rate, interpolates it, and limits its slew before any command reaches a drive. Algorithm A.2 formalizes this pipeline in the two-processor implementation, and figure A.6 illustrates it. Section 6.2 uses the timing below to show which sampler fits the setpoint period.

⁵ | **Diffusion denoising latency budget:** The number of sequential denoiser evaluations and their latency depend on the sampler, the backbone, and the hardware.



Figure A.6: Diffusion policy action chunk denoising and receding-horizon decoupling: (a) Multimodal resolution: direct regression with mean squared error averages divergent demonstrations into an invalid obstacle collision, whereas diffusion policy stochastically selects a coherent demonstrated mode. (b) Iterative trajectory denoising: starting from Gaussian noise $A^K \sim \mathcal{N}(0, \mathbf{I})$, reverse diffusion progressively resolves a coarse trajectory into a smooth action chunk A^0 . (c) Dual-rate decoupling: an asynchronous neural planner on the application processor populates a lock-free ring buffer, feeding a 1, kHz safety microcontroller executing deterministic spline interpolation and safety clamping. (Adapted from Chi et al. (Chi et al. 2024)).

Algorithm A.2 Diffusion Policy Trajectory Chunk Denoising and Dual-Rate Actuation

Require: observation history buffer $\mathbf{O}_t = (\mathbf{o}_{t-H+1}, \dots, \mathbf{o}_t)$, noise prediction network ϵ_θ , noise schedule $\{\alpha_k, \bar{\alpha}_k, \sigma_k\}_{k=1}^{N_{\text{diff}}}$, chunk horizon K , safety microcontroller loop period Δt_{mcu}
Ensure: line-rate joint torque setpoint / action vector $\mathbf{a}_t$

```

1: Application Processor: Asynchronous Trajectory Chunk Generation (10–20 Hz)
2:  $\mathbf{E}_t \leftarrow \text{VISUALENCODER}(\mathbf{O}_t)$ 
3:  $\mathbf{A}^{N_{\text{diff}}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{K \times D_a}) \triangleright \text{initialize reverse diffusion from Gaussian noise}$ 
4: for  $k = N_{\text{diff}}$  down to 1 do
5:    $\hat{\mathbf{c}} \leftarrow \epsilon_\theta(\mathbf{A}^k, \mathbf{E}_t, k)$ 
6:    $\mu_k \leftarrow \frac{1}{\sqrt{\alpha_k}} \left( \mathbf{A}^k - \frac{1-\alpha_k}{\sqrt{1-\alpha_k}} \hat{\mathbf{c}} \right)$ 
7:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $k > 1$  else  $\mathbf{z} \leftarrow \mathbf{0}$ 
8:    $\mathbf{A}^{k-1} \leftarrow \mu_k + \sigma_k \mathbf{z}$ 
9: end for
10:  $\mathbf{A}_{\text{chunk}} \leftarrow \mathbf{A}^0$ 
11:  $\text{DEPOSITSPSCRINGBUFFER}(\mathbf{A}_{\text{chunk}}) \triangleright \text{lock-free deposit into shared buffer}$ 

12: Safety Microcontroller: Synchronous Deterministic Actuation (1 kHz)
13: if  $\text{HASNEWCHUNK}(\text{SPSC})$  then
14:    $\text{UPDATESPLINEKNOTS}(\text{SPSC})$ 
15: end if
16:  $\mathbf{a}_{\text{raw}} \leftarrow \text{CUBICSPLINEINTERPOLATE}(\mathbf{A}_{\text{chunk}}, t_{\text{current}})$ 
17:  $\Delta \mathbf{a} \leftarrow \text{CLAMP}(\mathbf{a}_{\text{raw}} - \mathbf{a}_{\text{prev}}, -\dot{\mathbf{a}}_{\text{max}} \Delta t_{\text{mcu}}, +\dot{\mathbf{a}}_{\text{max}} \Delta t_{\text{mcu}}) \triangleright \text{kinematic slew-rate governor}$ 
18:  $\mathbf{a}_{\text{clamped}} \leftarrow \mathbf{a}_{\text{prev}} + \Delta \mathbf{a}$ 
19:  $\mathbf{a}_t \leftarrow \text{SAFETYGOVERNORCLAMP}(\mathbf{a}_{\text{clamped}}) \triangleright \text{deterministic envelope enforcement}$ 
20:  $\text{TRANSMITPWMREGISTERS}(\mathbf{a}_t)$ 
21:  $\mathbf{a}_{\text{prev}} \leftarrow \mathbf{a}_t$ 
22: return  $\mathbf{a}_t$ 

```

Consider a 7-DoF arm with a two-finger gripper ($D_a = 8$) whose policy generates chunks of 16 setpoints at a chosen 20 ms setpoint period (50 Hz), a 320 ms horizon. The noise-prediction network is a 1D temporal convolutional U-Net with 48 million parameters, 96 MB at FP16. Its timings below are illustrative inputs, not measurements of any device. The budget has three parts.

1. **Perception.** A visual encoder processes two synchronized RGB streams (wrist-mounted and over-the-shoulder) into the conditioning embedding $\mathbf{E}_t$ in $t_{\text{vis}} = 14$ ms per chunk. Because the features are extracted once per chunk before denoising begins, t_{vis} is a fixed cost whatever the sampling schedule.
2. **Denoising.** Each reverse step evaluates ϵ_θ at 14.2 GFLOP. At a peak bandwidth of 204.8 GB/s, streaming the weights once takes at least 0.469 ms, and the arithmetic intensity is about 147.9 FLOP/byte. With kernel launch overhead and activation spills, the time per evaluation is $t_{\text{step}} = 1.65$ ms, and the sampler then sets the cost. A standard DDIM sampler with 16 steps denoises in 26.4 ms, for a total inference latency of 40.4 ms. A flow-matching decoder integrated with 2 Euler evaluations, assuming two steps meet the task’s action-quality criterion, needs 3.3 ms, for a total of 17.3 ms.
3. **The deadline.** Run synchronously against the 20 ms period, the DDIM sampler overruns by 102 percent before any scheduling overhead. The two-evaluation flow decoder fits with 2.7 ms (13.5 percent) of compute-only slack, and it may run in lockstep only if its measured tail latency, bus transfer, and operating-system scheduling also fit. Run asynchronously at 15 Hz (a 66.7 ms period), the DDIM sampler refreshes the chunk buffer with 26.3 ms to spare. The permission path, at 1 kHz in the two-processor implementation, meanwhile interpolates the 320 ms chunk and checks its freshness, falling back when the chunk expires.

A.5.3 Compounding quadratic error in behavioral cloning

Offline supervised evaluation samples from a fixed demonstration distribution. In closed-loop execution, current action a_t affects the subsequent physical state s_{t+1} and the next observation distribution.

Let π^* be the expert and $\hat{\pi}$ the learner. Suppose its error probability is at most ϵ at each step under the expert’s time-indexed state distribution $d_{\pi^*}^t$:

$$\forall t \leq T: \quad \mathbb{E}_{s \sim d_{\pi^*}^t} [\ell(s, \hat{\pi})] \leq \epsilon.$$

An error can move the robot into states that the expert demonstrations rarely visited. Offline accuracy under $d_{\pi^*}^t$ gives no comparable bound on action error in those induced states.

Couple the learner and expert trajectories until their first disagreement. If the error bound applies at each expert-visited step, a union bound gives $P_t \leq t\epsilon$ for a disagreement by step t .⁶ If the additional downstream cost after divergence is at most $C_{\max}$ per step, then:

$$J(\hat{\pi}) - J(\pi^*) \leq C_{\max} \sum_{t=1}^T P_t \leq \frac{T(T+1)}{2} \epsilon C_{\max} = \mathcal{O}(T^2 \epsilon).$$

This is an upper bound under the stated cost and distribution assumptions (Ross et al. 2011). Single-step validation accuracy alone cannot predict a particular closed-loop failure probability.

A.5.4 Interactive dataset aggregation (Dagger) linear reduction

The **Dagger** (Dataset Aggregation) algorithm (Ross et al. 2011) addresses this distribution shift by collecting training data under the learner’s induced state distribution $d_{\hat{\pi}_i}$.

At each training iteration i :

1. Execute policy $\hat{\pi}_i$ under the applicable safety envelope to collect trajectories $\mathcal{T}_i = \{s_1, s_2, \dots, s_T\}$.
2. Query the expert to label every visited state $s_t \in \mathcal{T}_i$ with the optimal recovery action $a^* = \pi^*(s_t)$.
3. Aggregate the recovery dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a^*)\}$.
4. Retrain policy $\hat{\pi}_{i+1}$ on the aggregated dataset.

For the Dagger bound, make a stronger assumption than the earlier per-step $C_{\max}$: one wrong action followed by the expert changes expected remaining cost by at most $u_{\max}$, independent of horizon. With expert labels on learner-visited states, a no-regret online learner, sufficient hypothesis capacity, and a valid finite-sample bound δ_N , a selected policy can satisfy a bound of the form

$$J(\hat{\pi}) - J(\pi^*) \leq T u_{\max} (\epsilon_N + \gamma_N + \delta_N),$$

⁶ | **Compounding Error in Imitation Learning:** Ross and colleagues analyzed how learner actions shift the state distribution. Expert corrective labels on learner-visited states can reduce this exposure.

where ϵ_N is best-in-class loss on the induced-state distribution, γ_N is average online-learning regret, and δ_N covers finite-sample generalization and policy selection. It scales linearly in T only when $u_{\max} = \mathcal{O}(1)$ and those errors remain controlled. The drift states need expert corrective labels; appending recovery trajectories alone does not supply them.

A.5.5 Closed-loop trajectory distributions and evaluation estimands

Evaluating a learned policy π_θ in an embodied closed loop requires distinguishing between static demonstration distributions and candidate-induced physical distributions.

Let a physical rollout be parameterized by initial state $s_0 \sim p(s_0)$, environmental parameters $e \sim P(e)$ (lighting, surface friction, obstacle geometry), plant parameters $m \sim P(m)$ (payload mass, link inertias), and permission-path parameters $\kappa \sim P(\kappa)$ (lease durations, barrier thresholds). Under candidate policy π_θ and physical plant transition dynamics $P(s_{t+1} | s_t, u_t)$, the joint probability density of an executed trajectory $\tau = (s_0, u_0, s_1, u_1, \dots, s_T)$ over horizon T is:

$$P(\tau | \pi_\theta) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(u_t | o_t) P(s_{t+1} | s_t, u_t)$$

where $o_t = g(s_t, e)$ represents the observation function.

The discounted state visitation distribution (state marginal) $d^{\pi_\theta}(s)$ induced by policy π_θ is:

$$d^{\pi_\theta}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi_\theta)$$

or in the finite-horizon unweighted setting: $\bar{d}^{\pi_\theta}(s) = \frac{1}{T} \sum_{t=0}^{T-1} P(s_t = s | \pi_\theta)$.

Given a binary task success indicator $\phi(\tau) \in \{0, 1\}$ (e.g., reaching a target receptacle without exceeding torque or clearance limits), the true closed-loop task success probability across operating domain $\mathcal{D}$ is the formal integral:

$$p(\mathcal{D}) = \mathbb{E}_{c \sim \mathcal{D}}[\phi(\text{Traj}(c))] = \int \phi(\text{Traj}(s_0, e, m, \pi_\theta, \kappa)) dP(s_0, e, m, \kappa)$$

Because the distribution of visited states d^{π_θ} depends directly on policy proposals u_t , offline evaluations over fixed logged datasets $\mathcal{D}_{\text{demo}}$ evaluate predictions under d^{π^*} , and so give no statistical bound over d^{π_θ} when candidate commands deviate from demonstrator actions.

A.6 Value Functions and Affordance Grounding

Reinforcement learning optimizes sequential physical decision-making through expected reward accumulation.

A.6.1 Bellman optimality and out-of-distribution overestimation

The optimal state-action value function $Q^*(s, a)$ satisfies the Bellman optimality equation:

$$Q^*(s, a) = R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\max_{a' \in \mathcal{A}} Q^*(s', a') \right]$$

With static offline data, the maximization can query a learned critic on actions poorly represented in training. Value overestimation can then favor an out-of-distribution proposal. That proposal still needs an independently checked action envelope before it reaches an actuator.

A.6.2 SayCan affordance-grounded skill selection

The SayCan framework (Ahn et al. 2023) integrates semantic knowledge from high-capacity language models with physical affordance constraints from trained policies.

Let ℓ represent high-level language instruction and a represent an available low-level skill. The probability of executing skill a in current physical state s decomposes into the product of two probabilities:

$$P(a | s, \ell) \propto P_{\text{LLM}}(a | \ell) \cdot V_{\text{policy}}(s, a)$$

where:

- $P_{\text{LLM}}(a \mid \ell)$ represents the **semantic prior**, evaluating whether skill a contributes logically to completing instruction ℓ .
- $V_{\text{policy}}(s, a) \in [0, 1]$ represents the **physical affordance probability**, modeled by a learned value function that predicts the probability of successfully executing skill a from current state s .

If the learned value correctly assigns a near-zero score to an unavailable skill, the product de-prioritizes it. The learned score estimates success; it bounds no path, contact force, or clearance, so a separate motion-permission check is needed.

A.7 Signal Temporal Logic and Continuous Robustness

Verifying whether a physical autonomous system satisfies dynamic safety requirements requires moving beyond instantaneous boolean assertions to continuous temporal specifications.

A.7.1 Syntax and semantics of signal temporal logic

Signal Temporal Logic (STL) evaluates continuous-time physical signals $\mathbf{x}(t)$ against temporal requirements defined over time intervals $[a, b]$:

$$\varphi ::= \top \mid \mu \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \Box_{[a,b]}\varphi \mid \Diamond_{[a,b]}\varphi \mid \varphi_1 \mathcal{U}_{[a,b]}\varphi_2$$

where $\mu = (g(\mathbf{x}) \geq 0)$ is an atomic predicate (e.g., clearance $d(t) - d_{\min} \geq 0$), $\Box_{[a,b]}$ denotes the **Always** operator, $\Diamond_{[a,b]}$ denotes the **Eventually** operator, and $\mathcal{U}_{[a,b]}$ denotes the **Until** operator.

A.7.2 Quantitative robustness degree

Beyond a boolean satisfaction verdict ($\mathbf{x} \models \varphi$), STL admits a signed **quantitative robustness metric** $\rho(\mathbf{x}, \varphi, t)$.⁷ For a formula combining pressure, speed, and force, first define dimensionless atomic margins $\bar{g}_i(\mathbf{x}) = g_i(\mathbf{x})/s_i$, where each positive scale s_i has the same SI unit as its predicate. Then the minimum and maximum below compare like quantities:

$$\begin{aligned} \rho(\mathbf{x}, \mu, t) &= \bar{g}(\mathbf{x}(t)) \\ \rho(\mathbf{x}, \neg\varphi, t) &= -\rho(\mathbf{x}, \varphi, t) \\ \rho(\mathbf{x}, \varphi_1 \wedge \varphi_2, t) &= \min(\rho(\mathbf{x}, \varphi_1, t), \rho(\mathbf{x}, \varphi_2, t)) \\ \rho(\mathbf{x}, \Box_{[a,b]}\varphi, t) &= \min_{t' \in [t+a, t+b]} \rho(\mathbf{x}, \varphi, t') \\ \rho(\mathbf{x}, \Diamond_{[a,b]}\varphi, t) &= \max_{t' \in [t+a, t+b]} \rho(\mathbf{x}, \varphi, t') \end{aligned}$$

The sign of ρ indicates satisfaction or violation, while its normalized magnitude is dimensionless and depends on the declared scales. Preserve each raw physical margin and unit separately in the fault record; a single minimum of meters, newtons, and kilopascals has no physical meaning.

A.8 Zero-Failure Testing and the Exposure Wall

Suppose a release case selects a catastrophic-event target rate $\lambda_* = 10^{-9}/\text{h}$ and asks what zero-event exposure alone can establish at 95 percent confidence. This numerical target is a scenario assumption, not a rate universally assigned to an IEC SIL, an automotive ASIL, or a learned policy.

For n independent equal-duration Bernoulli trials with event probability p in each trial, the probability of zero events is:

$$P(\text{zero failures}) = (1 - p)^n \leq e^{-pn}$$

To exclude a hypothesized per-trial event probability p at one-sided confidence $1 - \alpha$, the zero-event probability under that hypothesis must be at most α :

$$(1 - p)^n \leq \alpha$$

Taking the natural logarithm of both sides:

$$n \ln(1 - p) \leq \ln \alpha$$

⁷ | **Signal Temporal Logic Robustness:** Quantitative semantics map a trace and formula to a signed score. Its magnitude inherits the atomic predicate's units only when all combined predicates use commensurate units; otherwise a declared normalization gives a dimensionless score. Negative scores identify violations for falsification searches, subject to sampled-time and measurement limits.

⁸ | **Statistical Rule of Three:** Zero events in n independent Bernoulli trials give the exact one-sided 95 percent upper probability $1 - 0.05^{1/n}$, approximately $3/n$ for small probabilities. Continuous operating hours instead require a declared event-rate model. Neither calculation covers changed configurations or unobserved hazard classes automatically.

For rare events where $p \ll 1$, using the Taylor expansion $\ln(1-p) \approx -p$ gives:⁸

$$-np \leq \ln \alpha \implies n \geq \frac{-\ln \alpha}{p} = \frac{\ln(1/\alpha)}{p}$$

For standard 95 percent confidence ($\alpha = 0.05$):

$$n \geq \frac{-\ln(0.05)}{p} = \frac{2.99573}{p} \approx \frac{3}{p}$$

This result is the Rule of Three of section 7.1, familiar from reliability engineering and biostatistics.

A.8.1 The exposure wall

For continuous operating time, suppose incidents follow a stationary homogeneous Poisson process of rate λ and fleet hours are applicable to the same configuration. Then $P(N_H = 0) = e^{-\lambda H}$, giving $\lambda_U = -\ln(0.05)/H$ after zero incidents. For the selected λ_* , the required exposure is:

$$H \geq \frac{-\ln(0.05)}{10^{-9}/\text{h}} \approx 3.00 \times 10^9 \text{ h} \approx 342,000 \text{ machine-years.}$$

The exposure scales inversely with the selected rate. A fleet could accumulate these hours in principle, but a 100-machine fleet would need about 3,420 years of continuous operation, impractical for the illustrated release cycle. A change to weights, calibration, or firmware requires a configuration-equivalence argument before earlier evidence can be pooled; it does not mechanically reset every observation to zero. Zero-event fleet hours bound incidents of the integrated system under the model, not hazardous proposals stopped by a shield. Section 7.5 uses this result to show that a zero-event test cannot reach such a rate.

A.8.2 Wald normal confidence interval breakdown and clopper-pearson exact bounds

In standard machine learning benchmarks, empirical success or failure rates are commonly summarized using the asymptotic Wald normal approximation. For k observed failures across n independent Bernoulli trials, the sample failure proportion is $\hat{p} = k/n$. The two-sided $(1 - \alpha)$ Wald confidence interval is:

$$\text{CI}_{\text{Wald}} = \hat{p} \pm z_{1-\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

where $z_{1-\alpha/2}$ is the standard normal critical value ($z_{0.975} \approx 1.96$ for 95 percent confidence).

In physical autonomy qualification, candidate systems are subjected to zero-failure stress tests ($k = 0$). When $k = 0$, the sample proportion is $\hat{p} = 0$, causing the sample variance term $\hat{p}(1-\hat{p})/n$ to evaluate to exactly zero:

$$\text{CI}_{\text{Wald}} \Big|_{k=0} = 0 \pm z_{1-\alpha/2} \sqrt{0} = [0, 0]$$

The Wald interval thus claims, at 95 percent confidence, that the failure probability is exactly zero after a finite test run. It ignores sampling uncertainty, and its coverage of any positive failure probability is zero.

To maintain non-asymptotic statistical rigor, physical autonomy qualification requires **Clopper-Pearson exact binomial confidence intervals** (Clopper and Pearson 1934). The Clopper-Pearson interval $[p_L, p_U]$ inverts the exact binomial cumulative distribution function without normal approximations:

$$\sum_{x=k}^n \binom{n}{x} p_L^x (1-p_L)^{n-x} = \frac{\alpha}{2}, \quad \sum_{x=0}^k \binom{n}{x} p_U^x (1-p_U)^{n-x} = \frac{\alpha}{2}$$

These boundaries can be evaluated in exact closed form using quantiles of the Beta distribution:

$$p_L = B\left(\frac{\alpha}{2}; k, n-k+1\right), \quad p_U = B\left(1-\frac{\alpha}{2}; k+1, n-k\right)$$

When zero failures occur ($k = 0$), the lower bound is $p_L = 0$, and the one-sided upper confidence bound satisfies $(1 - p_U)^n = \alpha$, yielding:

$$p_U = 1 - \alpha^{1/n}$$

For small p_U , the first-order Taylor series expansion yields the Rule of Three: $p_U \approx -\ln(\alpha)/n \approx 3/n$ for $\alpha = 0.05$. Unlike the degenerate Wald interval, the Clopper-Pearson interval has coverage of at least $1 - \alpha$ for every $n \geq 1$.

A.9 Sequential Probability Ratio Test

Evaluating embodied physical policies on physical hardware is expensive and induces mechanical wear. Fixed-sample-size testing requires pre-specifying sample count N and executing all N physical trials even if the system fails in its first runs or the evidence is already decisive. Conversely, naive early stopping (halting testing as soon as a nominal threshold is breached) introduces optional stopping bias, which inflates the Type I error (declaring an unsafe policy safe).

Abraham Wald's **Sequential Probability Ratio Test (SPRT)** (Wald 1945) handles optional stopping for two predeclared simple hypotheses by updating a likelihood ratio after each rollout and comparing it with stopping boundaries.

A.9.1 Log-likelihood ratio and bernoulli updates

Let $x_1, x_2, \dots, x_m$ denote independent trial outcomes from consecutive physical rollouts. Consider two competing operational hypotheses:

- Null hypothesis H_0 : The system failure probability is unacceptable ($p = p_0$).
- Alternative hypothesis H_1 : The system failure probability meets the safety target ($p = p_1 < p_0$).

After observing m trials, the cumulative log-likelihood ratio Λ_m is:

$$\Lambda_m = \sum_{i=1}^m \ln \frac{P(x_i | H_1)}{P(x_i | H_0)}$$

For discrete Bernoulli trials where $x_i \in \{0, 1\}$ (with $x_i = 1$ denoting task failure and $x_i = 0$ denoting task success), let $k = \sum_{i=1}^m x_i$ be the cumulative failure count across m runs. The log-likelihood ratio simplifies to the exact closed-form evaluation:

$$\Lambda_m = k \ln \left(\frac{p_1}{p_0} \right) + (m - k) \ln \left(\frac{1 - p_1}{1 - p_0} \right)$$

A.9.2 Wald stopping boundaries and decision rules

Given declared Type I error rate α (the maximum allowable probability of falsely accepting H_1 when H_0 is true) and Type II error rate β (the maximum allowable probability of falsely rejecting H_1 when H_1 is true), testing proceeds according to Wald's stopping rule:

$$B < \Lambda_m < A$$

where the usual Wald boundary approximations are:

$$A = \ln \left(\frac{1 - \beta}{\alpha} \right), \quad B = \ln \left(\frac{\beta}{1 - \alpha} \right)$$

At each evaluation step m :

1. If $\Lambda_m \geq A$: Halt and favor H_1 over the specified H_0 under the trial model; this is not a general safety certification.
2. If $\Lambda_m \leq B$: Halt and favor H_0 over the specified H_1 ; do not release on this evidence.
3. If $B < \Lambda_m < A$: The accumulated evidence remains inconclusive; execute rollout $m + 1$.

⁹ | **Wald Sequential Probability Ratio Test:** The efficiency result assumes the specified simple hypotheses and independent, identically distributed outcomes. A sequential design can stop an unpromising test early without claiming a fixed percentage of saved robot hours.

A.9.3 Average sample number optimality

For independent identically distributed outcomes under these two simple hypotheses, SPRT has an average-sample-number efficiency result relative to comparable tests.⁹ Actual machine-hour savings depend on the two event probabilities, error allowances, overshoot at the stopping boundaries, and trial cost. The displayed thresholds are approximations; a finite-sample operating-characteristic calculation or calibrated simulation is needed for the chosen protocol.

A.10 Extreme Value Theory and Tail Risk

Under its conditions, the Central Limit Theorem describes normalized sums and averages, not the tail of each disturbance. A clearance or thermal design may instead depend on a maximum over a declared exposure block:

$$M_n = \max\{X_1, X_2, \dots, X_n\}$$

Even a true standard Gaussian has one-sided exceedance probabilities of about 2.87×10^{-7} at 5σ and 9.87×10^{-10} at 6σ . At 1,000 independent samples per second, the corresponding mean waits are about 0.97 hours and 11.7 days. Contact and bus-delay data may have different tails or dependence; their family and high quantiles must be estimated from relevant measurements, not assigned from the mechanism's name.

A.10.1 The fisher-tippett-gnedenko theorem

For independent, identically distributed block observations whose normalized maxima have a nondegenerate limit, the Fisher-Tippett-Gnedenko theorem (Coles 2001) restricts that limit to the **Generalized Extreme Value (GEV)** family:¹⁰

$$G(z) = \exp\left(-\left[1 + \xi\left(\frac{z - \mu}{\sigma}\right)\right]^{-1/\xi}\right)$$

for $\xi \neq 0$ and $1 + \xi(z - \mu)/\sigma > 0$, with location μ , scale $\sigma > 0$, and shape ξ . Candidate limiting families are:

1. **Gumbel (Type I, $\xi = 0$):** The continuous limit as $\xi \rightarrow 0$, often considered for light-tailed observations:

$$G(z) = \exp\left(-\exp\left(-\frac{z - \mu}{\sigma}\right)\right)$$

2. **Fréchet (Type II, $\xi > 0$, with tail index $\alpha = 1/\xi$):** A possible limit for power-law tails ($P(X > x) \sim x^{-\alpha}$), if supported by the measured sample and fit. The general GEV expression above supplies its CDF without changing location and scale conventions.
3. **Weibull (Type III, $\xi < 0$):** A possible limit when the sampled quantity has a finite upper endpoint. A nominal mechanical stop does not by itself bound every measured force or latency.

A.10.2 Peaks-over-threshold and generalized Pareto distribution

For a suitable underlying distribution and sufficiently high threshold u , the excess $Y = X - u$ given $X > u$ may be approximated by a **Generalized Pareto Distribution (GPD)**. Peaks-over-threshold analysis preserves more exceedances than one maximum per block but requires attention to clustered, dependent events:

$$H(y) = 1 - \left(1 + \frac{\xi y}{\tilde{\sigma}}\right)^{-1/\xi}$$

where $\tilde{\sigma} > 0$ is the threshold scale and the expression applies on its supported domain, with the $\xi = 0$ case defined by a limit.

¹⁰ | **Tail Family Is an Empirical Claim:** A positive fitted GEV shape suggests a heavy-tail model for the sampled exposure blocks; it does not make all collision forces Fréchet. Block size, dependence, threshold choice, fit diagnostics, and parameter uncertainty affect a high-quantile estimate.

A.10.3 Hardware margin and quantile sizing

For a fitted GPD with exceedance fraction N_u/n , one possible high-quantile estimate at per-sample exceedance probability $\epsilon < N_u/n$ and $\xi \neq 0$ is:

$$z_\epsilon = u + \frac{\tilde{\sigma}}{\xi} \left(\left(\frac{n}{N_u} \epsilon \right)^{-\xi} - 1 \right)$$

where N_u is the number of threshold exceedances across n sampled cycles. At $\xi = 0$, its continuous limit is $z_\epsilon = u + \tilde{\sigma} \ln[(N_u/n)/\epsilon]$. Either estimate must carry fit and sampling uncertainty, and its quantile is a probabilistic estimate, not a hard physical bound.

The designer can use fitted tails to choose test levels and candidate margins. A safety-critical clearance or force limit additionally needs a validated operating envelope, actuator response, material threshold, and policy for observations outside the evidence range. Series elasticity, physical stops, or an independently timed clamp are possible protections when their own behavior has been measured for the machine.

A.11 Further Reading

For a deeper foundational understanding of machine learning theory, sequential statistics, and extreme value modeling, consult the following authoritative references:

- **Deep Learning** (Goodfellow et al. 2016) provides a comprehensive treatment of neural network optimization, regularized representation learning, and Lipschitz sensitivity bounds.
- **An Introduction to Statistical Modeling of Extreme Values** (Coles 2001) establishes the definitive mathematical foundations for GEV, GPD, and tail risk quantification.
- **Sequential Analysis** (Wald 1945) develops the first-principles mathematical framework and average sample number optimality proofs for the Sequential Probability Ratio Test.
- **A Reduction-Based Framework for Sequential Prediction** (Ross et al. 2011) establishes the theoretical foundations of compounding error in imitation learning and derives the DAgger algorithm.

Control and Dynamics

This appendix collects the control and dynamics models used throughout this book. Their guarantees apply only within a declared state, actuator, disturbance, and timing envelope.

B.1 How to Use This Appendix

CONSULT this appendix when encountering specific physical systems bottlenecks, control interface boundaries, or design sizing decisions across chapters in this book:

When you encounter this systems symptom	Consult	For this quantitative tool
A learned policy commands unsafe actions near physical obstacles or joint limits	section B.2	Control Barrier Function (CBF-QP) safety filters and kinetic stopping insets
Mode switching (policy handover, human intervention, or fallback) causes torque spikes	section B.3	C^2 quintic splines, C^1 cubic Hermite interpolation, and bumpless transfer bounds
Sizing the validity horizon, spatial lease, or watchdog timeout for an action chunk	section B.4	Triangular, trapezoidal, and jerk-limited kinematic reachability envelopes
An articulated manipulator stalls, exceeds joint velocity limits, or loses control authority	section B.4.1	Manipulator Jacobian singularities and Yoshikawa manipulability index
Spatial sensor offsets or IMU-camera lever arms corrupt state estimation under rotation	section B.4.3	Spatial coordinate transforms ($SE(3)$) and lever-arm acceleration propagation
Discrete sampling and transport latency erode control loop stability margins	section B.3.5	Discrete zero-order hold (ZOH) phase lag and latency margin penalties
Physics simulation diverges, impacts exceed limits, or wheels spin out during maneuvers	section B.5	Contact complementarity, elastodynamic impact, and wheel slip boundaries
Sensor occlusions or unobservable states cause state estimation uncertainty to diverge	section B.6	Continuous Lyapunov covariance propagation and Lie derivative observability rank
Teleoperation or compliant contact chatters due to transport delay	section B.8	Passivity conditions, wave variables, and energy-bounded interaction
Underactuated legged or humanoid robots balance dynamic momentum across multi-contact points	section B.7	Centroidal momentum equations and hierarchical Whole-Body Control QP

B.2 Safety and Control Barrier Functions

An independent enforcer can check a learned policy proposal against a control barrier constraint before actuation. A Control Barrier Function (CBF) describes a candidate invariant set; its protection depends on observed state, feasible input, plant model, and sampled response time. Torque ceilings and obstacle clearance may require separate constraints.

Let the system state be $x \in \mathcal{X} \subset \mathbb{R}^n$ and the control input be $u \in \mathcal{U} \subset \mathbb{R}^m$. Suppose we define a safe operating set $\mathcal{S}$ as the superlevel set of a continuously differentiable scalar function $h(x)$:

$$\mathcal{S} = \{x \in \mathcal{X} \mid h(x) \geq 0\}, \quad \partial\mathcal{S} = \{x \in \mathcal{X} \mid h(x) = 0\}$$

- $h(x) > 0$ when the system is strictly inside the safe interior.
- $h(x) = 0$ on the boundary of the safe set.
- $h(x) < 0$ when the system violates safety.

B.2.1 Lie derivatives and forward invariance

Forward invariance means a trajectory starting in $\mathcal{S}$ remains there for the modeled time interval. This claim requires an initial state in $\mathcal{S}$ and an admissible control at every reachable state.

For a continuously differentiable relative-degree-one barrier under continuous-time control, one sufficient condition is:¹

$$\dot{h}(x, u) \geq -\alpha h(x)$$

where $\alpha > 0$ is a chosen gain in s^{-1} , not a measured braking bandwidth.

When $h(x)$ is large, the condition permits some decrease. At $h(x) = 0$, it requires $\dot{h} \geq 0$ in the continuous model. The selected barrier must first encode the plant's momentum and available braking; raw distance cannot be regulated directly by torque in a second-order plant.

For a control-affine dynamical system $\dot{x} = f(x) + g(x)u$, the time derivative $\dot{h}$ is evaluated via chain-rule directional derivatives (Lie derivatives):

$$\dot{h}(x, u) = \frac{\partial h}{\partial x} \dot{x} = \frac{\partial h}{\partial x} f(x) + \frac{\partial h}{\partial x} g(x)u = L_f h(x) + L_g h(x)u$$

where $L_f h(x) \in \mathbb{R}$ represents the unforced drift of clearance (momentum, gravity), and $L_g h(x) \in \mathbb{R}^{1 \times m}$ represents the directional control effectiveness matrix.

B.2.2 Nagumo's viability theorem and contingent cones

The geometric foundation of set invariance for dynamic systems is Nagumo's Viability Theorem (Blanchini 1999). Let $\mathcal{S} \subset \mathbb{R}^n$ be a non-empty closed set. The **Bouligand contingent cone** (or tangent cone) $T_{\mathcal{S}}(x)$ to $\mathcal{S}$ at a point $x \in \mathcal{S}$ is defined as:

$$T_{\mathcal{S}}(x) = \left\{ v \in \mathbb{R}^n \mid \liminf_{\tau \rightarrow 0^+} \frac{\text{dist}(x + \tau v, \mathcal{S})}{\tau} = 0 \right\}$$

where $\text{dist}(y, \mathcal{S}) = \inf_{z \in \mathcal{S}} \|y - z\|$. Nagumo's theorem proves that for an autonomous system $\dot{x} = f(x)$ with locally Lipschitz vector field f , the set $\mathcal{S}$ is forward-invariant if and only if the velocity vector satisfies the **subtangentiality condition**:

$$f(x) \in T_{\mathcal{S}}(x) \quad \forall x \in \mathcal{S}$$

In the interior $\text{int}(\mathcal{S})$, $T_{\mathcal{S}}(x) = \mathbb{R}^n$, so the condition is trivially satisfied. On the boundary $\partial\mathcal{S}$ of a smooth set defined by the zero-superlevel set of $h(x)$ ($\mathcal{S} = \{x \mid h(x) \geq 0\}$ with $\nabla h(x) \neq 0$), the inward normal is $\nabla h(x)$. The contingent cone becomes the closed half-space pointing into or along the tangent plane of the safe set:

$$T_{\mathcal{S}}(x) = \{v \in \mathbb{R}^n \mid \nabla h(x)^\top v \geq 0\} \quad \forall x \in \partial\mathcal{S}$$

Consequently, the system cannot exit the set if and only if $\dot{h}(x) = \nabla h(x)^\top f(x) \geq 0$ whenever $h(x) = 0$.

Control Barrier Functions (Ames et al. 2019) extend Nagumo's subtangentiality to controlled affine systems $\dot{x} = f(x) + g(x)u$. For safety, there must exist an admissible control $u \in \mathcal{U}$ such that $f(x) + g(x)u \in T_{\mathcal{S}}(x)$. By introducing an extended class $\mathcal{K}_\infty$ decay rate $\alpha(h(x))$ satisfying $\alpha(0) = 0$, the CBF inequality:

$$\dot{h}(x, u) = L_f h(x) + L_g h(x)u \geq -\alpha(h(x))$$

ensures that as the state approaches the boundary ($h(x) \rightarrow 0$), the admissible decay rate contracts to zero ($-\alpha(h(x)) \rightarrow 0$). At the boundary $h(x) = 0$, it strictly enforces $\dot{h}(x, u) \geq 0$, directly preserving Nagumo's subtangentiality invariant and preventing trajectory exit into the unsafe set.

¹ | **Boundary Viability:** Nagumo's tangent-cone condition characterizes invariance under regularity assumptions. The displayed CBF inequality is a sufficient design condition for the stated smooth control-affine model. A sampled implementation needs a separate bound for state error, computation, hold interval, and actuator response; failure of one candidate inequality does not prove contact within one integration step.

B.2.3 Relative degree and kinetic stopping insets

The **relative degree** r of a barrier function $h(x)$ is the number of times it must be differentiated with respect to time before the control input u explicitly appears:

$$L_g L_f^k h(x) = 0 \quad \forall k < r-1, \quad \text{and} \quad L_g L_f^{r-1} h(x) \neq 0$$

For a one-dimensional approach to a stationary obstacle, let raw clearance be $h_r = D$, approach speed $v > 0$, and commanded approach acceleration a_{cmd} . Then $\dot{D} = -v$ and $\ddot{D} = -a_{\text{cmd}}$. Acceleration enters raw clearance only at the second derivative, so h_r has relative degree two with respect to this input. One approach uses a kinetic inset. In a one-dimensional approach to a stationary obstacle, let D be measured clearance, $v > 0$ approach speed, and $a_{\text{brake}} > 0$ a validated *lower bound* on available braking deceleration for the operating state. Then:

$$h_k(D, v) = D - \frac{v^2}{2a_{\text{brake}}}, \quad \dot{D} = -v, \quad \dot{v} = a_{\text{cmd}}.$$

For immediate commanded acceleration, $\dot{h}_k = -v - (v/a_{\text{brake}})a_{\text{cmd}}$. Thus $\dot{h}_k + \gamma h_k \geq 0$ gives the affine constraint $a_{\text{cmd}} \leq (a_{\text{brake}}/v)(\gamma h_k - v)$ for $v > 0$. At standstill, use a separate start/clearance rule. State error, sampling, and brake-onset delay need additional clearance reserves; the continuous calculation alone does not cover them.

B.2.4 Minimal-intervention quadratic program

For a barrier h_b whose first derivative actually depends on u , the enforcer can choose a command near the nominal proposal while respecting the feasible input set:

$$\begin{aligned} u^* &= \arg \min_{u \in \mathcal{U}} \frac{1}{2} \|u - u_{\text{nom}}\|^2 \\ \text{subject to: } & L_f h_b(x) + L_g h_b(x)u \geq -\alpha h_b(x), \quad u_{\min} \leq u \leq u_{\max} \end{aligned}$$

Writing its half-space as $a^\top u \leq b$, with $a = -L_g h_b(x)^\top$ and $b = L_f h_b(x) + \alpha h_b(x)$, gives this projection for one constraint when $a \neq 0$ and no box bound is active:

$$u^* = u_{\text{nom}} - \lambda^* a, \quad \text{where } \lambda^* = \max \left(0, \frac{a^\top u_{\text{nom}} - b}{\|a\|^2} \right)$$

If the feasible set is nonempty and convex, the strictly convex objective has a unique minimizer. Solver response time and total dispatch delay must be measured for the selected dimension and hardware.



Systems Perspective B.1: Check feasibility before relying on invariance

The projection formula does not solve the box-constrained problem. Clamping its result afterward can violate $a^\top u \leq b$. If no command satisfies both the barrier and actuator limits, the modeled invariant cannot be maintained by this filter. A separately validated fallback may still limit harm; it does not restore the lost forward-invariance claim. Admission must reserve stopping authority before the state reaches this boundary.

B.3 Smooth Trajectories and Bumpless Transfer

Physical systems possess inertia. A velocity step is an ideal acceleration impulse; an acceleration step is a finite torque step under a simple inertia model. Whether a commanded seam causes resonance or damage depends on the plant, load, and controller response.

B.3.1 Analytical quintic splines

A quintic (5th-degree) polynomial trajectory segment $\theta(t)$ over time interval $t \in [0, T]$ is defined as:

$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

The six coefficients $[a_0, \dots, a_5]$ are uniquely determined by matching six boundary conditions at $t = 0$ and $t = T$:

$$\begin{aligned} \theta(0) &= q_0, & \dot{\theta}(0) &= \dot{q}_0, & \ddot{\theta}(0) &= \ddot{q}_0 \\ \theta(T) &= q_1, & \dot{\theta}(T) &= \dot{q}_1, & \ddot{\theta}(T) &= \ddot{q}_1 \end{aligned}$$

Solving the resulting linear system yields the exact analytical formulas:

$$\begin{aligned} a_0 &= q_0 \\ a_1 &= \dot{q}_0 \\ a_2 &= \frac{1}{2} \ddot{q}_0 \\ a_3 &= \frac{10(q_1 - q_0) - (6\dot{q}_0 + 4\dot{q}_1)T - (3\ddot{q}_0 - \ddot{q}_1)\frac{T^2}{2}}{T^3} \\ a_4 &= \frac{-15(q_1 - q_0) + (8\dot{q}_0 + 7\dot{q}_1)T + (3\ddot{q}_0 - 2\ddot{q}_1)\frac{T^2}{2}}{T^4} \\ a_5 &= \frac{6(q_1 - q_0) - 3(\dot{q}_0 + \dot{q}_1)T - (\ddot{q}_0 - \ddot{q}_1)\frac{T^2}{2}}{T^5} \end{aligned}$$

Differentiating three times gives the segment's **jerk profile**, a quadratic in time that stays finite over a finite blend:

$$\ddot{\theta}(t) = 6a_3 + 24a_4 t + 60a_5 t^2$$

Matching position, velocity, and acceleration at the endpoints gives a C^2 join to adjacent segments with those same endpoint values and removes the commanded acceleration step at the join. The endpoint conditions say nothing about the segment's interior, so its jerk and torque extrema, and the joint's resonant response, still require a check against the machine's measured limits. Section 11.5 places this bridge at the seam between two action chunks.

B.3.1.1 Boundary value matrix formulation

The polynomial coefficients $\mathbf{a} = [a_0, a_1, a_2, a_3, a_4, a_5]^\top$ are determined by evaluating the trajectory and its first two derivatives at the temporal endpoints $t = 0$ and $t = T$. This yields the 6×6 confluent Vandermonde linear system $\mathbf{M}\mathbf{a} = \mathbf{b}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 1 & T & T^2 & T^3 & T^4 & T^5 \\ 0 & 1 & 2T & 3T^2 & 4T^3 & 5T^4 \\ 0 & 0 & 2 & 6T & 12T^2 & 20T^3 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} q_0 \\ \dot{q}_0 \\ \ddot{q}_0 \\ q_1 \\ \dot{q}_1 \\ \ddot{q}_1 \end{bmatrix}$$

The top three rows trivially yield the initial boundary terms: $a_0 = q_0$, $a_1 = \dot{q}_0$, and $a_2 = \frac{1}{2}\ddot{q}_0$. Substituting these values into the bottom three rows isolates the higher-order coefficients $[a_3, a_4, a_5]^\top$ under the reduced 3×3 system:

$$\begin{bmatrix} T^3 & T^4 & T^5 \\ 3T^2 & 4T^3 & 5T^4 \\ 6T & 12T^2 & 20T^3 \end{bmatrix} \begin{bmatrix} a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} q_1 - q_0 - \dot{q}_0 T - \frac{1}{2}\ddot{q}_0 T^2 \\ \dot{q}_1 - \dot{q}_0 - \ddot{q}_0 T \\ \ddot{q}_1 - \ddot{q}_0 \end{bmatrix}$$

The determinant of this 3×3 matrix is $\det(\mathbf{M}_{33}) = 2T^9 > 0$ for all strictly positive blending horizons $T > 0$, guaranteeing non-singularity and uniqueness. The analytic matrix inverse is:

$$\begin{bmatrix} T^3 & T^4 & T^5 \\ 3T^2 & 4T^3 & 5T^4 \\ 6T & 12T^2 & 20T^3 \end{bmatrix}^{-1} = \begin{bmatrix} \frac{10}{T^3} & -\frac{4}{T^2} & \frac{1}{2T} \\ -\frac{15}{T^4} & \frac{7}{T^3} & -\frac{1}{T^2} \\ \frac{6}{T^5} & -\frac{3}{T^4} & \frac{1}{2T^3} \end{bmatrix}$$

Matrix multiplication $\mathbf{a}_{3:5} = \mathbf{M}_{33}^{-1} \mathbf{b}_{\text{red}}$ directly produces the closed-form coefficients a_3, a_4, a_5 stated above.

B.3.2 Cubic hermite spline interpolation for multi-rate synchronization

When synchronizing physical sensor and state streams arriving at heterogeneous rates (such as 30 Hz camera frames against 1 kHz joint encoder trajectories), intermediate poses must be continuously interpolated while preserving velocity continuity. A cubic Hermite spline over interval $t \in [t_0, t_1]$ with segment duration $T = t_1 - t_0$ interpolates position and velocity between endpoints:

$$q(t_0) = q_0, \quad \dot{q}(t_0) = \dot{q}_0, \quad q(t_1) = q_1, \quad \dot{q}(t_1) = \dot{q}_1$$

Introducing the normalized time coordinate $s = (t - t_0)/T \in [0, 1]$, the trajectory is parameterized by the cubic Hermite basis polynomials:

$$q(s) = h_{00}(s)q_0 + h_{10}(s)T\dot{q}_0 + h_{01}(s)q_1 + h_{11}(s)T\dot{q}_1$$

where the basis functions satisfy:

$$\begin{aligned} h_{00}(s) &= 2s^3 - 3s^2 + 1 \\ h_{10}(s) &= s^3 - 2s^2 + s \\ h_{01}(s) &= -2s^3 + 3s^2 \\ h_{11}(s) &= s^3 - s^2 \end{aligned}$$

Differentiating with respect to physical time t yields the instantaneous velocity:

$$\dot{q}(t) = \frac{1}{T} \frac{dq}{ds} = \frac{1}{T} (h'_{00}(s)q_0 + h'_{01}(s)q_1) + h'_{10}(s)\dot{q}_0 + h'_{11}(s)\dot{q}_1$$

with derivative basis functions:

$$\begin{aligned} h'_{00}(s) &= 6s^2 - 6s \\ h'_{10}(s) &= 3s^2 - 4s + 1 \\ h'_{01}(s) &= -6s^2 + 6s \\ h'_{11}(s) &= 3s^2 - 2s \end{aligned}$$

Because the basis polynomials satisfy $h_{00}(0) = 1, h_{00}(1) = 0, h'_{10}(0) = 1, h'_{10}(1) = 0$, this yields a C^1 -continuous trajectory matching position and velocity exactly at sample boundaries without the computational overhead of solving a 6-boundary-condition quintic system. However, the resulting acceleration $\ddot{q}(t)$ is piecewise linear and discontinuous across sample boundaries (C^1 but not C^2), producing finite torque steps $\Delta\tau = M(q)\Delta\ddot{q}$ at knot points that must remain within actuator torque-slew limits.

B.3.3 Bumpless transfer and smoothstep blending

For bumpless transfer (section 14.4), let $s = (t - t_0)/\tau_{\text{blend}} \in [0, 1]$ and use the C^2 smoothstep $\alpha(s)$ to interpolate two commands:

$$\alpha(s) = 6s^5 - 15s^4 + 10s^3, \quad u(t) = (1 - \alpha(s))u_{\text{prior}}(t) + \alpha(s)u_{\text{new}}(t).$$

The endpoint first and second derivatives of α vanish. If u is commanded acceleration, its time derivative is commanded-acceleration jerk:

$$j_{\text{cmd}}(t) = \frac{\alpha'(s)}{\tau_{\text{blend}}} (u_{\text{new}} - u_{\text{prior}}) + (1 - \alpha(s)) \dot{u}_{\text{prior}} + \alpha(s) \dot{u}_{\text{new}}.$$

Here $\alpha'(s) = 30s^2(1-s)^2$ differentiates with respect to *normalized* time and peaks at 1.875. Only for two fixed, latched endpoint accelerations ($\dot{u}_{\text{prior}} = \dot{u}_{\text{new}} = 0$) does the blend contribution satisfy

$$\max |j_{\text{cmd}}| = \frac{1.875|\Delta u|}{\tau_{\text{blend}}}, \quad \tau_{\text{blend}} \geq \frac{1.875|\Delta u|}{j_{\text{allowable}}}.$$

This is a command-rate condition. Measured plant jerk, tracking error, and available stopping clearance remain separate admission checks.

B.3.4 Flexible joint dynamics and singular perturbations

Mechanical transmissions such as harmonic strain-wave drives, planetary reducers, and cable linkages introduce torsional elasticity between actuator motor rotors and robot links. Following Mark W. Spong's canonical formulation (Spong 1987), an n -degree-of-freedom manipulator with elastic joints is governed by two coupled second-order vector differential equations:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{K}(\theta - \mathbf{q})$$

$$\mathbf{J}_m\ddot{\theta} + \mathbf{B}_m\dot{\theta} + \mathbf{K}(\theta - \mathbf{q}) = \tau$$

where:

- $\mathbf{q} \in \mathbb{R}^n$ represents the physical link coordinates.
- $\theta \in \mathbb{R}^n$ represents the motor rotor coordinates reflected through gear reduction N to the link side.
- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$ is the symmetric, positive-definite link inertia matrix.
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} \in \mathbb{R}^n$ collects Coriolis and centripetal terms.
- $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^n$ is the gravitational torque vector.
- $\mathbf{J}_m = \text{diag}(J_{m,1}, \dots, J_{m,n})$ is the diagonal matrix of reflected rotor inertias ($J_{m,i} = N_i^2 J_{\text{rotor},i}$).
- $\mathbf{B}_m = \text{diag}(B_{m,1}, \dots, B_{m,n})$ collects viscous motor damping.
- $\mathbf{K} = \text{diag}(k_1, \dots, k_n)$ is the diagonal joint torsional stiffness matrix ($k_i > 0$).
- $\tau \in \mathbb{R}^n$ is the vector of electromagnetic actuator torques delivered to the rotor.

B.3.4.1 Singular perturbation formulation and time-scale separation

When joint stiffness is high relative to link inertia ($k_i \gg 1$), the system exhibits a two-time-scale separation between slow rigid-body dynamics and fast torsional oscillations. Define the scalar perturbation parameter $\epsilon = 1/\sqrt{k_{\min}}$, where $k_{\min} = \min_i(k_i)$, and express stiffness as $\mathbf{K} = (1/\epsilon^2)\tilde{\mathbf{K}}$ with $\tilde{\mathbf{K}} = \mathcal{O}(1)$.

Define the elastic torque fast state variable $\mathbf{z} \in \mathbb{R}^n$:

$$\mathbf{z} = \frac{1}{\epsilon^2} \tilde{\mathbf{K}}(\theta - \mathbf{q}) = \mathbf{K}(\theta - \mathbf{q})$$

Differentiating $\mathbf{z}$ twice with respect to continuous time t gives:

$$\ddot{\mathbf{z}} = \mathbf{K}(\ddot{\theta} - \ddot{\mathbf{q}}) = \mathbf{K}[\mathbf{J}_m^{-1}(\tau - \mathbf{B}_m\dot{\theta} - \mathbf{z}) - \mathbf{M}(\mathbf{q})^{-1}(\mathbf{z} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q}))]$$

Multiplying through by ϵ^2 reveals the singularly perturbed boundary layer:

$$\epsilon^2 \ddot{\mathbf{z}} = \tilde{\mathbf{K}}[\mathbf{J}_m^{-1}\tau - (\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})\mathbf{z} + \mathbf{M}(\mathbf{q})^{-1}(\mathbf{C}\dot{\mathbf{q}} + \mathbf{g}) - \mathbf{J}_m^{-1}\mathbf{B}_m\dot{\theta}]$$

Introducing the stretched fast time variable $\tau_{\text{fast}} = t/\epsilon$ transforms the elastic dynamics into:

$$\frac{d^2 \mathbf{z}}{d\tau_{\text{fast}}^2} + \tilde{\mathbf{K}}(\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})\mathbf{z} = \tilde{\mathbf{K}}\mathbf{J}_m^{-1}\tau + \tilde{\mathbf{K}}\mathbf{M}(\mathbf{q})^{-1}(\mathbf{C}\dot{\mathbf{q}} + \mathbf{g})$$

B.3.4.2 The rigid manifold and boundary-layer resonance

In the singular limit as joint stiffness approaches infinity ($\epsilon \rightarrow 0$), the fast elastic variable $\mathbf{z}$ collapses algebraically onto the quasi-steady manifold $\mathbf{z}_0$:

$$\mathbf{z}_0 = (\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})^{-1} [\mathbf{J}_m^{-1}\tau + \mathbf{M}(\mathbf{q})^{-1}(\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}))]$$

Substituting $\mathbf{z}_0$ into the link dynamics recovers the classical rigid-body equations with aggregated inertia $\mathbf{M}(\mathbf{q}) + \mathbf{J}_m$:

$$(\mathbf{M}(\mathbf{q}) + \mathbf{J}_m)\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \mathbf{B}_m\dot{\mathbf{q}} = \tau$$

For finite physical stiffness ($\epsilon > 0$), any high-frequency excitation in commanded torque τ —such as discontinuous torque steps produced across discrete inference intervals by learned neural policies—acts as an impulsive forcing function on the boundary-layer oscillator $\frac{d^2\mathbf{z}}{d\tau_{\text{fast}}^2} + \Omega^2(\mathbf{q})\mathbf{z} = \mathbf{0}$, where the natural structural resonance frequency matrix is:

$$\Omega(\mathbf{q}) = \sqrt{\mathbf{K}(\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})}$$

Because transmission mechanical damping is minimal ($\zeta < 0.05$), unsmoothed step commands excite persistent limit-cycle chatter, accelerate gear-tooth micro-fatigue, and invalidate rigid-link assumptions. Enforcing C^2 quintic spline interpolation and limiting torque slew rate ($d\tau/dt$) are necessary to prevent energy injection into this fast boundary-layer manifold.

B.3.5 Discrete-time sampling and zero-order hold phase lag

Real-time digital controllers execute periodically at sampling period $T_s = 1/f_s$. Actuator drivers hold commanded setpoints constant between clock ticks using a **Zero-Order Hold (ZOH)**.

At frequencies well below the sampling frequency, the zero-order hold's phase is equivalent to about half a sample period of delay:

$$\tau_{\text{ZOH}} = \frac{T_s}{2}$$

For a closed-loop control system operating at crossover frequency ω_c , this transport delay erodes the system's phase margin Φ_m by:²

$$\Delta\Phi = \omega_c \frac{T_s}{2} \quad (\text{radians}) \approx 28.65^\circ \cdot \left(\frac{\omega_c}{f_s}\right)$$

For fixed modeled compute and bus delays, the added phase lag near crossover is approximately $\Delta\Phi_{\text{total}} = \omega_c(\tau_{\text{comp}} + \tau_{\text{bus}} + T_s/2)$. The controller must validate phase margin over the measured latency and operating envelope; the formula alone does not predict a particular contact response.

B.4 Kinematic Reachability and Intent Envelopes

Kinematic reachability characterizes the spatial set of configurations or end-effector poses an embodied system can reach over a given planning or intent horizon τ .

B.4.1 Forward kinematics and singularities

For an n -joint arm, the **forward-kinematics** map $f(q)$ sends joint coordinates $q \in \mathbb{R}^n$ to an end-effector pose $T_{WE} \in SE(3)$:

$$T_{WE} = f(q).$$

In a declared spatial or body twist convention, the **manipulator Jacobian** $J(q) \in \mathbb{R}^{6 \times n}$ relates joint rates to the six-vector end-effector twist V_E :

$$V_E = J(q)\dot{q}.$$

For a six-dimensional task and $n \geq 6$, rank loss ($\det(JJ^T) = 0$) marks a **kinematic singularity**. Tracking a nonzero twist along a lost direction would require unbounded joint rates in the ideal inverse model; real actuators saturate first. A task-specific design may monitor $w(q) = \sqrt{\det(JJ^T)}$ against a validated threshold.³

² | **Phase Margin and Delay:** A pure delay contributes phase $-\omega\tau$. The half-sample approximation applies in the baseband; actual stability depends on the sampled plant, compensation, jitter, and measured total response. Consuming modeled phase margin predicts loss of the stated stability reserve, not a specific destructive failure mode.

³ | **Yoshikawa Manipulability Measure:** The measure summarizes the velocity ellipsoid for a chosen task Jacobian. Near rank loss, inverse-kinematics commands can exceed joint-rate or torque limits. The actual drive response and fallback depend on the specified controller and hardware.

B.4.2 Trajectory transit time and reachable distance

For a one-dimensional rest-to-rest move with symmetric available acceleration $a_{\max}$ and speed ceiling $v_{\max}$, the shortest nominal transit has two regimes:

1. **Triangular profile (acceleration-limited):** For short horizons where peak velocity $v_{\max}$ is not attained ($d \leq d_{\text{crit}} = v_{\max}^2/a_{\max}$):

$$t_{\min} = 2\sqrt{\frac{d}{a_{\max}}}, \quad d_{\text{reach}}(\tau) = \frac{1}{4}a_{\max}\tau^2$$

2. **Trapezoidal profile (velocity-limited):** For larger horizons ($d > d_{\text{crit}}$):

$$t_{\min} = \frac{d}{v_{\max}} + \frac{v_{\max}}{a_{\max}}, \quad d_{\text{reach}}(\tau) = v_{\max} \left(\tau - \frac{v_{\max}}{a_{\max}} \right)$$

Below d_{crit} the reachable distance grows with the square of the available time, and above it only linearly, because the mechanism spends the rest of the move at its speed ceiling.

With $|\dot{a}| \leq j_{\max}$, reaching full acceleration from zero takes at least $a_{\max}/j_{\max}$, if the profile reaches it at all. The bound exists because an acceleration step is a torque step, which shocks gearbox backlash, excites structural modes, and drives motor current toward the inverter's fault limits. The minimum-time trajectory may have four or seven jerk phases, so an unconditional additive offset to the trapezoidal time is invalid. Solve the constrained profile for the stated start/end velocity and acceleration, then include the stopping suffix and response delay when sizing a lease.

The idealized profiles still serve admission, because each added limit (jerk, torque, or thermal derating) can only shrink the **dynamic reachable set** $\mathcal{R}(\tau)$, the states reachable from the measured state within the remaining time $\tau = t_{\text{exp}} - t_{\text{now}}$. Their transit times are therefore necessary lower bounds, and a proposal whose idealized time already exceeds τ can be rejected before a full path, torque, and stopping check. Section 10.5 applies this rejection to the arm's reach against the tracked horizon.

B.4.3 Spatial coordinate frames and lever-arm kinematics

Rigid-body pose in three-dimensional space is parameterized by the Special Euclidean group $SE(3)$. A rigid transform from frame B to frame A is represented by the homogeneous transformation matrix $\mathbf{T}_{AB} \in SE(3)$:

$$\mathbf{T}_{AB} = \begin{bmatrix} \mathbf{R}_{AB} & \mathbf{p}_{AB} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \mathbf{R}_{AB} \in SO(3), \quad \mathbf{p}_{AB} \in \mathbb{R}^3$$

When sensors—such as IMUs or cameras—are displaced from the body center of rotation by lever arm $\mathbf{r}_{P/O}$, rotational motion induces apparent velocities and accelerations at sensor location P . The linear velocity $\mathbf{v}_P$ and linear acceleration $\mathbf{a}_P$ satisfy:

$$\mathbf{v}_P = \mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{r}_{P/O}$$

$$\mathbf{a}_P = \mathbf{a}_O + \boldsymbol{\alpha} \times \mathbf{r}_{P/O} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r}_{P/O})$$

where $\boldsymbol{\omega}$ is the body angular velocity, $\boldsymbol{\alpha} = \dot{\boldsymbol{\omega}}$ is the angular acceleration, $\boldsymbol{\alpha} \times \mathbf{r}$ is the tangential acceleration, and $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r})$ is the centripetal acceleration. State estimation and perception pipelines must compensate for these kinematic lever-arm terms when synchronizing multi-camera extrinsics and fusing IMU data.

B.4.4 Watchdog validity leases and teleoperation latency bounds

In distributed autonomous systems and supervisory teleoperation architectures, actuation authority is governed by bounded-duration validity leases and communications latency constraints.

B.4.4.1 Kinematic derivation of watchdog validity leases

Consider an autonomous mobile platform or manipulator moving at speed v toward an obstacle with measured physical clearance D_{clear} . If the host processor or neural network fails to refresh the control setpoint, a hardware or firmware watchdog must intervene to prevent boundary violation.

The total elapsed time from the sensor observation until physical braking initiates comprises five sequential intervals:

$$\tau_{\text{delay}} = T_{\text{lease}} + T_{\text{detect}} + T_{\text{hold}} + t_{\text{age}} + T_{\text{act}}$$

where:

- T_{lease} is the granted duration during which the policy proposal remains valid for execution.
- T_{detect} is the watchdog countdown detection latency before heartbeat loss is recognized.
- T_{hold} is an optional transient hold or fault confirmation debounce window.
- t_{age} is the sensor evidence age (exposure, readout, and bus transit to the controller clock).
- T_{act} is the physical actuator onset delay before braking deceleration reaches the commanded floor.

Under constant cruising speed v prior to brake onset, a verified lower bound on mechanical braking deceleration $a_{\text{brake}} > 0$, a localization bound δ_{loc} , and a fixed protective clearance δ_{margin} , the total stopping distance bounded by available clearance D_{clear} satisfies:

$$v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \leq D_{\text{clear}}$$

Solving directly for the maximum admissible validity lease T_{lease} yields:

$$T_{\text{lease}} \leq \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \frac{v^2}{2a_{\text{brake}}}}{v} - (T_{\text{detect}} + T_{\text{hold}} + t_{\text{age}} + T_{\text{act}})$$

If $T_{\text{lease}} \leq 0$, the platform's speed exceeds its stopping margin for the current clearance, and the real-time enforcer must immediately revoke motion permission and execute a verified emergency stop.

B.4.4.2 Teleoperation network round-trip time bounds

When a human supervisor remotely teleoperates the robot across an external wireless link, the control loop includes human perceptual reaction latency and network packet transport delays. Let:

- T_{video} represent video camera exposure, encoding, network downlink, and operator display latency.
- T_{human} denote human neuromuscular reaction time ($T_{\text{human}} \approx 150\text{--}250$ ms).
- $T_{\text{RTT}} = T_{\text{downlink}} + T_{\text{uplink}}$ represent network round-trip transport latency.
- T_{act} denote actuator braking engagement lag.
- $d_{\text{brake}} = \frac{v^2}{2a_{\text{brake}}}$ represent the purely mechanical braking distance.

Total available time before available clearance D_{clear} is exhausted is bounded by $T_{\text{total}} = (D_{\text{clear}} - d_{\text{brake}})/v$. The allowable round-trip network latency ceiling is:

$$T_{\text{RTT}} \leq \frac{D_{\text{clear}} - d_{\text{brake}}}{v} - (T_{\text{video}} + T_{\text{human}} + T_{\text{act}})$$

If network jitter pushes T_{RTT} beyond this bound, the human operator cannot perceive an obstacle and deliver a corrective command before the vehicle exhausts its stopping distance. Consequently, the local real-time enforcer must evaluate physical invariants independently at 1 kHz, revoking operator command authority and executing autonomous braking before physical clearance is lost. Section 14.2 uses this bound to size a handover, where the operator's round trip spends the same pre-brake budget.

B.4.4.3 Kinodynamic bounds versus wave-variable passivity

Transport delay in teleoperation creates two distinct failure modes: spatial boundary breach (kinematic collision) and high-frequency dynamic chatter (loss of closed-loop passivity). While wave variables (section B.8) preserve port passivity and prevent energy accumulation under delay, passivity guarantees only that the teleoperator will not become an active energy source. It does not bound physical displacement. Safety requires satisfying both invariants simultaneously: wave variables bound interactive contact energy, while the kinodynamic latency inequality guarantees collision-free spatial clearance.

B.5 Rigid Body Contact Mechanics

Simulating and controlling physical interaction—such as locomotion, dexterous grasping, and tool assembly—requires modeling mechanical contact. The governing physical challenge of contact is its non-smooth complementarity: interaction forces arise strictly when bodies establish physical contact.

Let $\phi(q)$ be the signed distance between two bodies:

- $\phi(q) > 0$: Bodies are separated.
- $\phi(q) = 0$: Bodies are touching.
- $\phi(q) < 0$: Bodies are penetrating (a geometric interpenetration that rigid mechanics strictly forbids).

Let f_n be the normal contact force pushing the bodies apart:

- If $\phi(q) > 0$, there is no contact, so $f_n = 0$.
- If bodies are touching, they can only exert repulsive contact force, so $f_n \geq 0$.

Ideal unilateral contact is represented by **complementarity conditions**:

1. Non-penetration: $\phi(q) \geq 0$
2. Unilateral force: $f_n \geq 0$
3. Complementarity: $\phi(q)f_n = 0$

These conditions exclude tensile normal contact. A time-discretized rigid-contact problem becomes a linear complementarity problem only under suitable linearization and contact assumptions; frictional multi-contact models can be nonlinear or require other solvers. They do not establish sim-to-real fidelity by themselves.

B.5.1 Non-smooth collision manifolds and velocity jumps

For frictionless normal impact against a stationary rigid plane, let $v_n^- < 0$ be the pre-impact normal velocity and $e \in [0, 1]$ the restitution coefficient. The normal component obeys $v_n^+ = -ev_n^-$; tangential velocity and a moving boundary require additional equations. With effective normal mass m_{eff} , the normal impulse is $P_n = m_{\text{eff}}(v_n^+ - v_n^-)$. In an ideal zero-duration impact, velocity jumps and acceleration is an impulse rather than an ordinary finite derivative:⁴

$$\lim_{\Delta t \rightarrow 0} \frac{\|\dot{x}(t + \Delta t) - \dot{x}(t)\|}{\Delta t} = \infty$$

Smooth learned models and trajectory gradients may be inaccurate at contact-mode changes. A controller can use an explicit hybrid/contact model, while compliance, pads, or torque limiters can reduce force only within validated stroke, energy, wear, and load-path conditions. Neither modeling nor a passive element alone guarantees safe contact.

B.5.2 Elastodynamic impact force and compliance deformation

When an effective moving mass m_{eff} strikes an obstacle or human tissue, rigid-body approximations break down because real materials deform elastically over a nonzero compression stroke $\delta(t)$. Let k_{eff} denote the effective contact stiffness across the interface. In series compliance between the manipulator structural chain (k_{robot}) and human tissue or obstacle (k_{tissue}):

$$\frac{1}{k_{\text{eff}}} = \frac{1}{k_{\text{robot}}} + \frac{1}{k_{\text{tissue}}}$$

⁴ | **Moreau Time-Stepping and Non-Smooth Mechanics:** Measure-differential models represent ideal impulses without resolving microscopic contact duration. Numerical contact solvers still require a validated geometry, friction law, and step-size study for the plant; a smooth optimizer that omits impact can miss a relevant force transient.

B.5.2.1 Unmitigated elastic impact force

For an unmitigated collision at impact velocity $v_{\max}$, initial kinetic energy $E_k = \frac{1}{2}m_{\text{eff}}v_{\max}^2$ converts entirely into elastic strain energy $U_{\text{elastic}} = \frac{1}{2}k_{\text{eff}}\delta_{\max}^2$ at the point of maximum compression ($\dot{\delta} = 0$). Energy conservation yields the peak structural deflection:

$$\frac{1}{2}m_{\text{eff}}v_{\max}^2 = \frac{1}{2}k_{\text{eff}}\delta_{\max}^2 \implies \delta_{\max} = v_{\max}\sqrt{\frac{m_{\text{eff}}}{k_{\text{eff}}}}$$

The resulting peak transient contact force $F_{\text{peak}} = k_{\text{eff}}\delta_{\max}$ evaluates to:

$$F_{\text{peak}} = v_{\max}\sqrt{k_{\text{eff}}m_{\text{eff}}}$$

This relation establishes that peak impact force scales linearly with velocity and with the square root of both moving mass and contact stiffness. Even modest kinetic energies (e.g., 17.5 J) striking stiff biological structures ($k_{\text{tissue}} \approx 15,000 \text{ N/m}$) generate peak forces exceeding 700 N—well beyond collaborative safety limits (such as ISO/TS 15066 transient thresholds).

B.5.2.2 Coupled contact indentation with mechanical braking

When active mechanical brakes engage with constant deceleration a_{brake} during deformation, the braking system performs mechanical work $W_{\text{brake}} = m_{\text{eff}}a_{\text{brake}}\delta$ across the indentation depth δ . The coupled energy balance becomes:

$$\frac{1}{2}m_{\text{eff}}v_{\text{impact}}^2 = \frac{1}{2}k_{\text{eff}}\delta_{\max}^2 + m_{\text{eff}}a_{\text{brake}}\delta_{\max}$$

Rearranging into quadratic form:

$$\delta_{\max}^2 + \frac{2m_{\text{eff}}a_{\text{brake}}}{k_{\text{eff}}}\delta_{\max} - \frac{m_{\text{eff}}v_{\text{impact}}^2}{k_{\text{eff}}} = 0$$

Solving for the physically admissible positive deformation root:

$$\delta_{\max} = \sqrt{\left(\frac{m_{\text{eff}}a_{\text{brake}}}{k_{\text{eff}}}\right)^2 + \frac{m_{\text{eff}}v_{\text{impact}}^2}{k_{\text{eff}}}} - \frac{m_{\text{eff}}a_{\text{brake}}}{k_{\text{eff}}}$$

When $a_{\text{brake}} = 0$, this expression smoothly recovers the unmitigated deformation $\delta_{\max} = v_{\text{impact}}\sqrt{m_{\text{eff}}/k_{\text{eff}}}$.

Because the quarter-cycle compression duration $t_{\text{peak}} = \frac{\pi}{2}\sqrt{m_{\text{eff}}/k_{\text{eff}}}$ is typically on the order of 20–50 ms, reactive braking triggered only *after* physical contact cannot attenuate the initial force peak if sensor detection and brake engagement latency exceed t_{peak} . Consequently, safety cases cannot substitute reactive post-impact braking for verified pre-impact separation clearance envelopes ($d_{\text{sep}} \geq d_{\text{stop}}$).

B.5.3 Wheel slip ratio and friction boundaries

For wheeled mobile robots and terrestrial platforms, tractive force transmission between the wheel contact patch and the ground plane is governed by non-linear slip dynamics. The longitudinal slip ratio λ quantifies the kinematic velocity mismatch between the wheel circumferential speed $r\omega$ (wheel radius r , angular rate ω) and the vehicle forward longitudinal speed v_x :

$$\lambda = \begin{cases} \frac{r\omega - v_x}{v_x}, & \text{under acceleration } (r\omega > v_x, v_x > 0) \\ \frac{r\omega - v_x}{r\omega}, & \text{under braking } (r\omega < v_x, r\omega > 0) \end{cases}$$

A normalized form frequently implemented in digital traction control avoids denominator singularities via:

$$\lambda = \frac{r\omega - v_x}{\max(|v_x|, |r\omega|, \epsilon_{\text{reg}})} \in [-1, 1]$$

where $\lambda = 0$ corresponds to pure rolling without slip, $\lambda = 1$ denotes complete drive-wheel spin-out ($v_x = 0, \omega > 0$), and $\lambda = -1$ corresponds to complete wheel lock-up ($r\omega = 0, v_x > 0$).

Tractive force F_x scales with the normal contact load F_z through the longitudinal friction coefficient $\mu_x(\lambda)$:

$$F_x(\lambda) = \mu_x(\lambda)F_z$$

At low slip ratios ($|\lambda| < \lambda_{\text{peak}} \approx 0.10\text{--}0.15$), tire or tread rubber shears elastically, and tractive force is approximately linear in slip: $F_x \approx C_x \lambda$, with longitudinal traction stiffness C_x . Beyond λ_{peak} , adhesion collapses into gross sliding friction: the slope $d\mu_x/d\lambda$ turns negative, rendering open-loop torque control unstable. If an ungrounded policy commands drive motor torque exceeding peak traction ($F_{\text{cmd}} > \mu_{\text{peak}}F_z$), the wheel rapidly accelerates into the unstable slip regime ($d\omega/dt = (\tau - rF_x)/I_{\text{wheel}} > 0$). This causes spin-out, severe yaw rate disturbance, or loss of lateral authority—a cyber-physical disconnect that cannot be detected by directional agreement or cosine similarity metrics that evaluate only vector angles while ignoring control magnitude.

B.6 Continuous Covariance Propagation and Observability

When sensors suffer line-of-sight dropouts or environmental occlusion, internal state estimators must project the physical state forward in time. Quantifying how rapidly estimation confidence degrades across blind intervals requires analyzing continuous error covariance dynamics and systems observability.

B.6.1 Continuous lyapunov and riccati covariance dynamics

Consider a continuous-time linear dynamic system governed by:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{w}(t)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{v}(t)$$

For the continuous white-noise model, $\mathbf{Q}$ and $\mathbf{R}$ are process and measurement noise *intensities*, with covariance kernels $E[\mathbf{w}(t)\mathbf{w}(s)^\top] = \mathbf{Q}\delta(t-s)$ and $E[\mathbf{v}(t)\mathbf{v}(s)^\top] = \mathbf{R}\delta(t-s)$. The model assumes suitable independence and $\mathbf{R} \succ 0$.

Under an optimal continuous-time estimator (the Kalman-Bucy filter (Kalman 1960)), the error covariance matrix $\mathbf{P}(t) = E[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^\top]$ evolves according to the continuous matrix differential Riccati equation:⁵

$$\dot{\mathbf{P}}(t) = \mathbf{A}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{A}^\top + \mathbf{Q} - \mathbf{P}(t)\mathbf{C}^\top\mathbf{R}^{-1}\mathbf{C}\mathbf{P}(t)$$

When direct sensing is occluded or dropped ($\mathbf{C} = \mathbf{0}$), the measurement correction term vanishes. The covariance growth collapses to the open-loop **continuous Lyapunov differential equation**:

$$\dot{\mathbf{P}}(t) = \mathbf{A}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{A}^\top + \mathbf{Q}$$

The solution propagates initial uncertainty and accumulated $\mathbf{Q}$ through $e^{\mathbf{A}t}$. Unstable modes can grow exponentially; a random walk with $\mathbf{A} = \mathbf{0}$ grows linearly in covariance. A state-memory record should carry the calibrated growth model and expire tracking permission before its declared error bound crosses the physical tolerance. Covariance alone is not a hard error envelope unless its tail and model assumptions are justified.

B.6.2 Closed-form double-integrator covariance expansion

Consider a one-dimensional double-integrator kinematic system modeling position $p(t)$ and velocity $v(t)$ under unmeasured stochastic acceleration noise $w(t)$:

$$\begin{bmatrix} \dot{p}(t) \\ \dot{v}(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} p(t) \\ v(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} w(t)$$

⁵ | **Continuous Matrix Riccati Equation:** The measurement term reduces modeled covariance when valid measurements arrive. During occlusion, propagation follows the plant and process-noise model. Its growth may be bounded, linear, polynomial, or exponential depending on the dynamics and noise; the estimator must compare the resulting bound with the task tolerance.

where $w(t)$ is zero-mean continuous white process noise with continuous spectral power intensity

$q_c > 0$, such that $\mathbb{E}[w(t)w(s)] = q_c\delta(t-s)$. Here, $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and the continuous process noise

matrix is $\mathbf{Q} = \begin{bmatrix} 0 & 0 \\ 0 & q_c \end{bmatrix}$.

The continuous state transition matrix $(t) = e^{\mathbf{A}t}$ is nilpotent:

$$(t) = \mathbf{I} + \mathbf{A}t = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

During a blind occlusion window $t \in [0, \Delta t]$ with no measurement updates ($\mathbf{C} = \mathbf{0}$), the total state covariance $\mathbf{P}(t)$ evolves as the sum of unforced homogeneous propagation and accumulated process diffusion:

$$\mathbf{P}(t) = (t)\mathbf{P}_0(t)^\top + \int_0^t (t-\tau)\mathbf{Q}(t-\tau)^\top d\tau$$

Let the initial state covariance at the instant of sensor loss ($t = 0$) be:

$$\mathbf{P}_0 = \begin{bmatrix} \sigma_{p,0}^2 & \sigma_{pv,0} \\ \sigma_{pv,0} & \sigma_{v,0}^2 \end{bmatrix}$$

The homogeneous propagation term evaluates to:

$$(t)\mathbf{P}_0(t)^\top = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \sigma_{p,0}^2 & \sigma_{pv,0} \\ \sigma_{pv,0} & \sigma_{v,0}^2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ t & 1 \end{bmatrix} = \begin{bmatrix} \sigma_{p,0}^2 + 2\sigma_{pv,0}t + \sigma_{v,0}^2t^2 & \sigma_{pv,0} + \sigma_{v,0}^2t \\ \sigma_{pv,0} + \sigma_{v,0}^2t & \sigma_{v,0}^2 \end{bmatrix}$$

The accumulated stochastic diffusion term integrates across elapsed time:

$$\mathbf{Q}_d(t) = \int_0^t \begin{bmatrix} 1 & t-\tau \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & q_c \end{bmatrix} \begin{bmatrix} 1 & 0 \\ t-\tau & 1 \end{bmatrix} d\tau = \int_0^t \begin{bmatrix} q_c(t-\tau)^2 & q_c(t-\tau) \\ q_c(t-\tau) & q_c \end{bmatrix} d\tau = \begin{bmatrix} \frac{1}{3}q_c t^3 & \frac{1}{2}q_c t^2 \\ \frac{1}{2}q_c t^2 & q_c t \end{bmatrix}$$

Summing the homogeneous and stochastic contributions yields the closed-form positional variance $\sigma_p^2(t) = \mathbf{P}_{11}(t)$:

$$\sigma_p^2(t) = \sigma_{p,0}^2 + 2\sigma_{pv,0}t + \sigma_{v,0}^2t^2 + \frac{1}{3}q_c t^3$$

This derivation explains the distinct physical error regimes during sensor occlusion: initial velocity uncertainty $\sigma_{v,0}^2$ induces quadratic error growth in time (t^2), while continuous stochastic disturbances q_c (such as air currents or unmodeled drivetrain chatter) inject cubic uncertainty growth ($\frac{1}{3}q_c t^3$). In long blind intervals, the cubic term dominates, rapidly exhausting safety margins unless motion is derated or halted.

B.6.3 Observability rank and Lie derivative extensions

A linear system is observable on $[0, T]$ if its initial state can be uniquely recovered from outputs and known inputs. For the linear time-invariant pair $(\mathbf{A}, \mathbf{C})$, the **observability matrix** has full column rank n exactly when the state is observable:

$$\mathcal{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \vdots \\ \mathbf{CA}^{n-1} \end{bmatrix}, \quad \text{rank}(\mathcal{O}) = n$$

⁶ | **Hermann-Krener Nonlinear Observability:** Full rank of the observation codistribution is a local structural test, not a sensor-noise or finite-horizon accuracy bound. A missing state direction may require another sensor, an active maneuver with its own permission, or a restricted operating envelope.

For a smooth autonomous nonlinear system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$, $\mathbf{y} = \mathbf{h}(\mathbf{x})$, a local rank test builds the codistribution from differentials of successive Lie derivatives $dL_t^k \mathbf{h}(\mathbf{x})$ until its rank stops growing.⁶ Full local rank n is a sufficient local weak-observability condition under its regularity assumptions; a finite truncation at $n - 1$ is not universally sufficient.

If $\text{rank}(\mathcal{O}) < n$, $\ker(\mathcal{O})$ contains initial-state differences that produce identical ideal outputs. Measurement updates cannot resolve those directions from this sensor model, though a particular Kalman gain need not contain literal zero entries. The estimator should expose such unobservable components and maintain an independent growth bound or restrict the task.

B.7 Centroidal Dynamics and Floating-Base Control

Legged robots and dynamic humanoids differ fundamentally from fixed-base manipulators: they possess an underactuated 6-degree-of-freedom floating base that cannot be commanded directly by motors. The robot can only move its center of mass (CoM) and alter its spatial orientation by exchanging contact forces with the environment through its feet or hands.

B.7.1 Floating-base equations of motion

Let $\mathbf{q} = [\mathbf{q}_b^\top, \mathbf{q}_j^\top]^\top \in SE(3) \times \mathbb{R}^n$ represent the generalized coordinates, where $\mathbf{q}_b \in SE(3)$ is the floating-base pose and $\mathbf{q}_j \in \mathbb{R}^n$ are the actuated joint angles. The equations of motion are (Siciliano et al. 2009):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{S}^\top \boldsymbol{\tau} + \sum_{k=1}^{N_c} \mathbf{J}_k(\mathbf{q})^\top \mathbf{f}_k$$

where $\mathbf{S} = [\mathbf{0}_{n \times 6}, \mathbf{I}_{n \times n}]$ is the actuation selection matrix separating the 6 unactuated floating-base coordinates from the n actuated joints, and $\mathbf{J}_k(\mathbf{q}) = \partial \mathbf{p}_k / \partial \mathbf{q} \in \mathbb{R}^{3 \times (n+6)}$ is the contact point Jacobian for contact wrench $\mathbf{f}_k$.

B.7.2 Centroidal momentum dynamics

Projecting the floating-base dynamics onto the robot's center of mass $\mathbf{c} \in \mathbb{R}^3$ yields the Newton-Euler **centroidal momentum equations** (Lynch and Park 2017):

$$\begin{aligned} \dot{\mathbf{h}}_{\text{lin}} &= m\ddot{\mathbf{c}} = \sum_{k=1}^{N_c} \mathbf{f}_k + m\mathbf{g} \\ \dot{\mathbf{h}}_{\text{ang}} &= \sum_{k=1}^{N_c} [(\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k + \boldsymbol{\tau}_k] \end{aligned}$$

where $\mathbf{h}_{\text{lin}} \in \mathbb{R}^3$ is linear momentum, $\mathbf{h}_{\text{ang}} \in \mathbb{R}^3$ is angular momentum about the CoM, and $(\mathbf{p}_k, \mathbf{f}_k, \boldsymbol{\tau}_k)$ are the position, force, and moment at each active contact patch.⁷

B.7.3 Physical contact feasibility constraints

Because the environment cannot pull on the robot and contact surfaces exhibit finite friction, admissible contact wrenches must satisfy three physical invariants:

1. **Unilateral Normal Force:** Feet can only push against the ground:

$$f_{k,\perp} = \mathbf{f}_k^\top \hat{\mathbf{n}}_k \geq 0$$

2. **Coulomb Friction Cone:** Tangential shear forces cannot exceed normal traction:

$$\|\mathbf{f}_{k,\parallel}\| = \|\mathbf{f}_k - f_{k,\perp} \hat{\mathbf{n}}_k\| \leq \mu f_{k,\perp}$$

3. **Support condition for coplanar contacts:** Under flat-ground assumptions, the Zero Moment Point (ZMP) should remain inside the effective support polygon with a case-specific inset:⁸

$$\mathbf{p}_{\text{ZMP}} \in \mathcal{P}_{\text{support}}, \quad \mathcal{P}_{\text{support}} = \text{Conv} \left(\bigcup_k \mathcal{A}_k \right),$$

where $\mathcal{A}_k$ is each active contact patch projected onto that plane.

⁷ | **Centroidal Momentum Matrix:** Centroidal dynamics summarize the external wrench needed for a proposed CoM and angular-momentum change. A controller still needs feasible contacts and actuator torque; omitting angular momentum can mispredict the required contact moments.

⁸ | **Zero Moment Point (ZMP):** ZMP is useful for a specified coplanar support model. Edge contact, friction loss, and multi-height supports require more general wrench feasibility checks; crossing a modeled polygon edge does not by itself predict a particular fall.

B.7.4 Hierarchical whole-body control quadratic program

One whole-body controller can track torso and foot tasks while constraining dynamics and contact forces. For an illustrative sampled implementation, a conservative *polyhedral* friction approximation makes the following convex quadratic program possible; the exact circular norm cone in the preceding equation instead gives a second-order cone constraint:

$$\begin{aligned} \min_{\mathbf{q}, \mathbf{f}, \tau} \quad & \sum_i w_i \|\mathbf{J}_i \ddot{\mathbf{q}} + \dot{\mathbf{J}}_i \dot{\mathbf{q}} - \ddot{\mathbf{x}}_i^{\text{des}}\|^2 + w_\tau \|\tau\|^2 + w_f \|\mathbf{f}\|^2 \\ \text{subject to:} \quad & \mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{S}^\top \tau + \sum_{k=1}^{N_c} \mathbf{J}_k^\top \mathbf{f}_k \\ & \mathbf{F}_k \mathbf{f}_k \leq \mathbf{0}, \quad f_{k,\perp} \geq 0 \quad (\text{conservative friction pyramid}) \\ & |\tau| \leq \tau_{\max}, \quad \mathbf{q}_{j,\min} \leq \mathbf{q}_j \leq \mathbf{q}_{j,\max} \end{aligned}$$

Here $\mathbf{F}_k$ describes the chosen inner approximation to the measured friction cone. The displayed joint-position bounds concern the current configuration; a real controller must also constrain predicted motion between samples. A 1 ms solve is an example deadline to validate with the selected contact count, solver, processor, friction estimate, and fallback. A feasible numerical solution does not itself prove that a foothold will support the load.

B.8 Passivity and Energy-Bounded Interaction

During teleoperation or stiff contact, transport delay can reduce the damping margin of a feedback loop. **Passivity** is an energy-accounting condition useful for analyzing a specified interconnection; it does not remove the need to verify actuator saturation, contact geometry, sampling, and stored energy.

B.8.1 Energy storage and the passivity definition

Consider a continuous-time dynamical system with state $\mathbf{x}(t)$, control input $\mathbf{u}(t)$ (an effort variable, such as force or torque $\mathbf{F}$), and measured output $\mathbf{y}(t)$ (a flow variable, such as velocity $\mathbf{v}$). The system is defined as **passive** if there exists a continuously differentiable, non-negative storage function $V(\mathbf{x}) \geq 0$ (representing the total internal physical energy stored in mass momentum and structural compliance) such that:

$$\dot{V}(\mathbf{x}) \leq \mathbf{u}(t)^\top \mathbf{y}(t) - g(\mathbf{x}) \quad (\text{B.1})$$

where $\mathbf{u}(t)^\top \mathbf{y}(t)$ represents instantaneous power injected into the system, and $g(\mathbf{x}) \geq 0$ denotes internal energy dissipation (such as mechanical damping or electrical resistance). Integrating equation B.1 over any operational duration $[0, T]$ yields the fundamental passivity energy inequality:

$$\int_0^T \mathbf{u}(t)^\top \mathbf{y}(t) dt \geq V(\mathbf{x}(T)) - V(\mathbf{x}(0)) + \int_0^T g(\mathbf{x}(t)) dt \geq -V(\mathbf{x}(0)) \quad (\text{B.2})$$

Under equation B.2 and the declared port orientation, the net energy extractable from this modeled component is bounded by its initial storage. A loaded mechanism can still release enough stored energy to injure or damage; passivity is not a contact-force limit.

Negative-feedback interconnection of compatible passive components preserves a passive energy balance when ports and signs match. Bounded or asymptotic state stability requires additional storage and detectability conditions. A human, powered tool, or active environment cannot be assumed passive without evidence.

B.8.2 The transport delay destabilization hazard

An untreated effort/flow communication channel need not remain passive when delayed. A fixed pure delay contributes phase $\Delta\phi = -\omega\tau_d$ at angular frequency ω , but no universal 90° threshold determines the sign of an arbitrary coupled mechanical impedance.

For a particular controller, late force feedback can oppose the operator's current motion poorly and inject energy into the closed loop. Whether this causes chatter depends on the port signs, damping, delay distribution, and contact stiffness; measure those conditions rather than inferring failure from latency alone.

B.8.3 Passivity preservation via wave variables

For a delay channel with specified port orientation, Anderson and Spong (1989)'s **wave variable transformation** maps effort $\mathbf{F}$ and flow $\dot{\mathbf{x}}$ into incoming and outgoing waves with characteristic impedance $b > 0$:

$$\mathbf{u}_m(t) = \frac{\mathbf{F}_m(t) + b\dot{\mathbf{x}}_m(t)}{\sqrt{2b}}, \quad \mathbf{w}_m(t) = \frac{\mathbf{F}_m(t) - b\dot{\mathbf{x}}_m(t)}{\sqrt{2b}} \quad (\text{B.3})$$

Computing instantaneous power from the wave transformation in equation B.3 reveals:

$$P_m(t) = \mathbf{F}_m(t)^\top \dot{\mathbf{x}}_m(t) = \frac{1}{2} \|\mathbf{u}_m(t)\|^2 - \frac{1}{2} \|\mathbf{w}_m(t)\|^2$$

For an ideal fixed delay, a received wave is a delayed transmitted wave. The channel can hold energy in flight, so a same-window inequality between master and slave port powers needs a channel-storage term: $E_{\text{ch}}(T) - E_{\text{ch}}(0) \leq \int_0^T (P_m - P_s) dt$ with consistent signs. The scattering construction can preserve channel passivity under its assumptions; time-varying delay, packet loss, sampling, saturation, and powered endpoints require additional compensation or tests. It does not guarantee contact-force safety or convergence of the complete human-robot system.

B.9 Further Reading

For a deeper theoretical foundation in control theory, robotics, and dynamical systems, consult the following authoritative references:

- **Modern Robotics: Mechanics, Planning, and Control** (Lynch and Park 2017) provides an elegant, rigorous treatment of kinematics and dynamics using screw theory and Lie groups.
- **Robotics: Modelling, Planning and Control** (Siciliano et al. 2009) serves as an authoritative reference for multi-body robot dynamics, trajectory generation, and operational space control.
- **Feedback Systems: An Introduction for Scientists and Engineers** (Åström and Murray 2021) establishes classical and state-space control theory, frequency-domain stability margins, and state estimation from first principles.

Systems and Hardware

This appendix collects the embedded timing, memory, power, and actuator models used throughout this book, together with the byte layouts of records that cross the proposal boundary. Each bound depends on a specified device, workload, and operating envelope; the equations identify what must be measured before a physical deadline can be claimed.

C.1 How to Use This Appendix

CONSULT this appendix when diagnosing physical systems bottlenecks, timing jitter, electrical rail instability, or hardware interface boundaries across chapters in this book:

When you encounter this systems symptom	Consult	For this quantitative tool
A real-time safety task misses periodic execution deadlines	section C.3	Worst-case execution time (WCET) and Response Time Analysis (RTA)
A software deadlock or kernel panic freezes actuator control loops	section C.3	Watchdog detection and separately measured stop-path response
Inverter bridge MOSFETs burn out or trip overcurrent protections	section C.4	Microcontroller PWM complementary timers and dead-time insertion
Heavy neural inference bursts induce compute brownout resets	section C.5	Inductive voltage droop (LdI/dt) and alpha-power gate delay
High-rate sensor DMA streams cause safety threads to miss deadlines	section C.6	DRAM bank-conflict penalties and non-preemptive burst queuing
Sensor fusion streams suffer temporal registration skew under motion	section C.7	PTP hardware timestamping and bounded clock conversion
Shared memory queues between Linux and the safety microcontroller (MCU) return torn or stale data	section C.8	SoC hardware mailboxes, RPMSG, and hardware memory barriers
Joint actuators fail to track sudden torque reversals, chatter, or overheat	section C.9	Stator L/R constants, thermal ODEs, and reflected rotor jerk limits
Emergency braking spikes DC bus voltage and damages power silicon	section C.10	Regenerative capacitive energy surges and dynamic brake choppers
Shared serial buses experience queuing and message jitter	section C.11	CAN FD priority blocking, EtherCAT cycle budgets, and bounded seqlocks
Monocular 3D bounding boxes distort and depth covariance diverges	section C.12	Pinhole intrinsic projection, Zhang's calibration, and metric unprojection covariance
A mobile base breaches clearance buffers under surface wetness	section C.13	Kinematic stopping envelopes, information age lag, and velocity clamps
A record must be laid out for a fixed-size mailbox or log	section C.14	Proposal and record byte layouts; manifest and register field orders
A handover, fault trial, or release gate must be implemented step by step	section C.14	Authority-transfer, fault-injection, and release-gate protocols beside their records
You need rapid order-of-magnitude physical anchors across time, kinetics, thermal, and risk	section C.2	Rate hierarchy, blind travel conversions, thermal constants, and exposure walls

C.2 Numbers to Know in Physical AI

¹ | **Jeff Dean:** Google Senior Fellow and systems architect. His latency numbers originally presented with Peter Norvig around 2010 established the canonical latency hierarchy across storage, memory, and network tiers (Scott 2012). Physical AI extends this hierarchy down into electromechanical actuators, stopping kinematics, and thermal time constants.

Just as Jeff Dean’s “Latency Numbers Every Programmer Should Know”¹ shaped distributed systems and classical machine learning systems grounded software performance in memory bandwidth and arithmetic intensity, physical AI systems engineers require a corresponding quantitative intuition spanning computation, kinetics, thermal dissipation, and functional safety. In physical AI, software latency is not merely waiting time; it translates directly into irreversible kinetic momentum, uninspected travel distance, and thermodynamic heat accumulation in motor stators and compute silicon.

This section compiles the foundational order-of-magnitude numbers, scaling laws, and engineering rules of thumb that govern cyber-physical machine design. All numbers are single-sourced from `mlsysim`. Memorize the relationships; use the specific numbers as sanity checks.

🔍 Systems Perspective C.1: Three physical numbers that matter most

- **Kinetic equivalence:** $1 \text{ ms} = 1 \text{ mm}$ (at 1 m/s): At a nominal robotic speed of 1 m/s , every single millisecond of perception, inference, or fieldbus latency translates into exactly 1 mm of blind travel before retarding torque can be applied. At highway velocity (30 m/s), each millisecond is 3 cm . In physical systems, software latency is not waiting time; it is physical displacement.
- **Thermal divergence:** $\tau_{\text{silicon}} \sim 1 \text{ ms}$ vs. $\tau_{\text{stator}} \sim 100 \text{ s}$: Semiconductor junctions heat adiabatically within milliseconds during deep neural network bursts, precipitating voltage droop or thermal throttling before heat spreaders react. In contrast, electromagnetic stator windings heat over tens to hundreds of seconds, tolerating transient overcurrent for aggressive maneuvers but accumulating thermal energy that eventually limits sustained torque.
- **The exposure wall:** $N = 3/\lambda$: Proving an ultra-low catastrophic failure rate of 10^{-8} /hour (ISO 26262 ASIL D) requires 3×10^8 failure-free operational hours under the Poisson Rule of Three (Hanley and Lippman-Hand 1983). Statistical empirical testing of learned end-to-end models cannot cross this exposure wall without deterministic runtime permission envelopes and formal safety filters.

C.2.1 Rate and latency hierarchy across machine levels

Control rates in physical AI rise steeply from the deliberative brain down to the physical body (table C.1). Deliberative multimodal models (VLAs) operate at human reaction scales ($1\text{--}5 \text{ Hz}$). Action chunking models emit future trajectory waypoints at $10\text{--}50 \text{ Hz}$. Beneath the proposal boundary, the permission path enforces safety barriers at 1 kHz . At the lowest levels, power electronic current loops and PWM switching operate at tens to hundreds of kilohertz to maintain electromagnetic flux and drive torque. For real-time timer schedulability and response-time analysis, see section C.3; for inverter gate switching and dead-time modeling, see section C.4.

Table C.1: Rate and Latency Hierarchy Across the Five Machine Levels: Spanning five frequency decades from sub-microsecond MOSFET gate switching up to second-scale foundation model deliberation. Every frequency band imposes a hard cycle period and an uninspected physical travel distance at nominal speed (1 m/s).

Machine Level	Frequency Band	Cycle Period	Blind Travel at 1 m/s	Architectural Mechanism & Bound
Brain (Deliberation)	$1\text{--}5 \text{ Hz}$	$200\text{--}1,000 \text{ ms}$	$200\text{--}1,000 \text{ mm}$	Multimodal VLA reasoning; memory- and compute-bound
Brain (Action Chunking)	$10\text{--}50 \text{ Hz}$	$20\text{--}100 \text{ ms}$	$20\text{--}100 \text{ mm}$	Action chunk diffusion/transformer decoding; amortized
Nervous System (Permission)	1 kHz	1 ms	1 mm	Control barrier certificates, stopping envelopes, lease watchdog

Machine Level	Frequency Band	Cycle Period	Blind Travel at 1 m/s	Architectural Mechanism & Bound
Body (Current Loop)	10–25 kHz	40–100 μ s	40–100 μ m	Field-oriented current control (FOC), torque tracking, stator flux
Body (Inverter Switching)	50–200 kHz	5–20 μ s	5–20 μ m	Power MOSFET gate drive, dead-time insertion (t_{dead}), inductive droop

C.2.2 Kinetic translation and safe stopping envelopes

Total stopping distance $d_{\text{stop}} = v \cdot \tau_{\text{delay}} + v^2 / (2a_{\text{brake}})$ combines the linear blind travel accrued during computational delay and the quadratic mechanical braking distance (table C.2). As platform speed increases, absolute blind travel expands rapidly, consuming entire clearance margins before deceleration can begin. For multi-axis kinematic chains, jerk-bounded trajectories, and information-age lag derivations, see section C.13.

Table C.2: Kinetic Translation Across Platform Archetypes: Pre-brake delay converts into unrecoverable physical travel before braking force begins. Stopping distance $d_{\text{stop}} = v \cdot \tau_{\text{delay}} + v^2 / (2a_{\text{brake}})$ combines linear blind travel and quadratic braking distance.

Platform Archetype	Nominal Velocity v	Braking Decel a	Blind Travel ($\tau = 50$ ms)	Blind Travel ($\tau = 200$ ms)	Total Stop ($\tau = 200$ ms)
Humanoid / Quadruped	1 m/s (3.6 km/h)	4 m/s ² (0.41 g)	50 mm	200 mm	325 mm (61.5%)
Warehouse Mobile Manipulator	1.5 m/s (5.4 km/h)	2 m/s ² (0.20 g)	75 mm	300 mm	863 mm (34.8%)
Urban Delivery Robot	6 m/s (21.6 km/h)	4.5 m/s ² (0.46 g)	300 mm	1.2 m	5.2 m (23.1%)
Autonomous Vehicle (Highway)	30 m/s (108 km/h)	7 m/s ² (0.71 g)	1.5 m	6 m	70.3 m (8.5%)

C.2.3 Edge memory bandwidth and action chunk amortization

Large multimodal foundation models on edge system-on-chip (SoC) architectures are memory-bandwidth bound during autoregressive generation (table C.3). Streaming a 7-billion-parameter model over a sustained 140 GB/s LPDDR5 interface limits single-step generation to 10–20 Hz. Predicting action chunks of length $H = 16$ amortizes the weight streaming transfer over multiple future control steps, lifting effective control bandwidth into the hundreds of hertz. For DRAM bank organization, conflict penalties, and interconnect arbitration under concurrent DMA traffic, see section C.6.

Table C.3: Edge Memory Bandwidth and Action Chunk Amortization: Autoregressive token decoding on edge accelerators (such as Jetson AGX Orin at 140 GB/s sustained DRAM bandwidth) is memory-bandwidth bound. Predicting action chunks (H steps) amortizes the weight transfer over multiple execution cycles, enabling high effective control frequencies.

Model & Precision	Parameters	Memory Footprint	DRAM Read Time (140 GB/s)	Single-Step Max Rate	Chunk Rate ($H = 16$)
VLA-7B (FP16)	7.0B	14 GB	100 ms	10 Hz	160 Hz equivalent
VLA-7B (INT8)	7.0B	7 GB	50 ms	20 Hz	320 Hz equivalent
VLA-7B (INT4)	7.0B	3.5 GB	25 ms	40 Hz	640 Hz equivalent
VLA-14B (INT8)	14.0B	14 GB	100 ms	10 Hz	160 Hz equivalent

C.2.4 Thermal time constants across compute and electromechanics

Thermal dissipation follows an exponential relaxation $T(t) = T_{\text{amb}} + \Delta T_{\text{max}}(1 - e^{-t/\tau_{\text{th}}})$ with time constants spanning five orders of magnitude (table C.4). Silicon junctions overheat within milliseconds during computational bursts, demanding microsecond-scale DVFS and PDN droop management. In contrast, electromagnetic stator windings heat over minutes, allowing transient torque

overdrive if bounded by an I^2t thermal watchdog. For first-order thermal differential equations and stator winding degradation limits, see section C.9; for regenerative energy absorption on DC power buses, see section C.10.

Table C.4: Thermal Time Constants Across Compute and Electromechanical Subsystems: Thermal inertia spans five orders of magnitude. Adiabatic heating in semiconductor junctions causes rapid thermal throttling within milliseconds, while copper motor windings integrate electrical energy over tens to hundreds of seconds.

Subsystem Component	Physical Mechanism	Thermal Time Constant τ_{th}	Critical Failure Mode	Safety Mitigation
Silicon Junction (Die)	Transistor channel self-heating	1–10 ms	Gate delay stretch, brownout, thermal trip	Rapid frequency throttling, PDN decoupling
SoC Heat Sink / Package	Convective heat spreader dissipation	10–60 s	Package thermal saturation, sustained throttling	Active fan modulation, workload migration
Motor Stator Windings	Copper resistive Joule heating (I^2R)	30–180 s	Insulation breakdown (Class H 180°C), short	I^2t thermal protection, current derating
Motor Frame / Housing	Bulk metallic casing heat conduction	10–30 min	Structural deformation, bearing grease loss	Natural convection, passive cooling fins
Traction Battery Pack	Electrochemical cell internal resistance	5–20 min	Thermal runaway ($> 60^\circ\text{C}$), fire	Liquid cooling loop, charge/discharge cutoff

C.2.5 Statistical safety bounds and exposure walls

Under the Poisson Rule of Three (Hanley and Lippman-Hand 1983), demonstrating that a physical system achieves a catastrophic failure rate target λ with 95 percent confidence without observing a single failure requires testing for $N = 3/\lambda$ operational hours (table C.5). Meeting the ultra-dependable thresholds demanded by automotive functional safety (ISO 26262 ASIL D, $\lambda < 10^{-8}$ /hour (ISO 26262 2018)) requires 300 million failure-free hours—a scale that renders end-to-end black-box statistical validation mathematically impossible without deterministic runtime permission envelopes. For proposal and manifest wire formats supporting runtime safety contracts, see section C.14.

Table C.5: Functional Safety Standards and Statistical Testing Exposure Walls: Required failure-free operating hours to establish safety targets at 95 percent confidence via the Rule of Three ($N = 3/\lambda$). Purely statistical end-to-end testing cannot prove catastrophic safety without deterministic runtime guarantees.

Safety Standard / Regime	Target Failure Rate λ	Equivalent FIT (10^{-9} /h)	Required 0-Failure Hours ($N = 3/\lambda$)	Engineering Implication
ISO 26262 ASIL D	$< 10^{-8}$ /hour	< 10 FIT	300,000,000 hours (3×10^8 h)	Empirically untestable by fleet testing alone; requires formal safety barriers
IEC 61508 SIL 3 / PL e	$< 10^{-7}$ /hour	< 100 FIT	30,000,000 hours (3×10^7 h)	Dual-channel lockstep hardware, diverse software monitoring
Commercial Robotaxi Target	$< 10^{-6}$ /hour	$< 1,000$ FIT	3,000,000 hours (3×10^6 h)	Human driver baseline equivalence (1 fatality per 10^8 miles)
Industrial Cobot (PL d)	$< 10^{-6}$ /hour	$< 1,000$ FIT	3,000,000 hours (3×10^6 h)	Force and power limiting, speed-and-separation monitoring

C.2.6 Fieldbus latencies, jitter, and determinism

Communication buses exhibit sharp divides in latency and jitter (table C.6). Industrial fieldbuses like EtherCAT achieve sub-microsecond jitter via dedicated hardware cut-through processing. In contrast, non-deterministic arbitration and shared wireless channels introduce tail latencies that exceed the reaction deadlines of high-speed electromechanical machines. For IEEE 1588 PTP hardware timestamping bounds, see section C.7; for CAN-FD priority arbitration and EtherCAT cycle budgets, see section C.11.

safety case must bound expiry-to-drive response, wire any MCU-loss trip independently of the failed MCU, and show that the resulting state-matched brake or hold can stop the plant within its remaining clearance. A reset or safe torque off (STO) command alone may leave a moving load coasting.

C.4 Inverter Timers and Power Electronics Gate Drive

Actuator motor drives employ three-phase half-bridge inverters to convert DC bus voltage into alternating stator currents.

C.4.1 Microcontroller timer dead-time insertion

Each inverter phase comprises a high-side power MOSFET (or IGBT) and a low-side power MOSFET connected across the DC supply rail V_{bus} . Microcontroller timer peripherals generate complementary pulse-width-modulated (PWM) drive signals with hardware-enforced **dead-time insertion** t_{dead} :

$$t_{\text{dead}} \geq t_{\text{off, max}} + t_{\text{prop, skew}} - t_{\text{on, min}}$$

where $t_{\text{off, max}}$ is the maximum turn-off time of the power switch, $t_{\text{on, min}}$ is the minimum turn-on time, and $t_{\text{prop, skew}}$ is the propagation delay mismatch across gate drivers.³

For the specified device and gate-drive delays, adequate dead time reduces overlap between high-side and low-side conduction. If both switches conduct simultaneously, **shoot-through** creates a low-impedance path across the DC bus; current and damage depend on bus impedance, protection, and event duration.

C.5 Inductive Voltage Droop and Gate Delay Stretch

When deep neural network accelerators (such as systolic array matrix engines) transition from idle to full execution, current draw increases by tens of amperes within nanoseconds.

C.5.1 Power delivery network impedance and rail droop

Every power delivery network (PDN) exhibits finite parasitic loop inductance L_{PDN} and equivalent series resistance R_{PDN} across PCB traces, package balls, and bond wires. When accelerator execution triggers a transient current step ΔI with rise time t_r (giving current slew rate $dI/dt \approx \Delta I/t_r$), Faraday's law of induction dictates an instantaneous voltage collapse ΔV_{droop} :

$$\Delta V_{\text{droop}} = L_{\text{PDN}} \frac{dI}{dt} + \Delta I \cdot R_{\text{PDN}}$$

If supply voltage $V(t) = V_{\text{nom}} - \Delta V_{\text{droop}}$ approaches a device's characterized minimum operating voltage V_{min} , timing margin may shrink or brownout protection may act.

C.5.2 Digital logic gate delay and the alpha-power law

In sub-micron CMOS silicon, logic gate propagation delay t_{prop} is governed by the Sakurai-Newton **alpha-power law**:⁴

$$t_{\text{prop}}(V) \propto \frac{C_L \cdot V}{(V - V_{\text{th}})^\alpha}$$

where C_L is the gate load capacitance, V_{th} is the transistor threshold voltage, and $\alpha \in [1.2, 1.5]$ is the velocity saturation index (contrasting with $\alpha = 2.0$ in long-channel theory).

When rail voltage droops from V_{nom} to V_{droop} , propagation delay stretches according to:

$$\frac{t_{\text{prop}}(V_{\text{droop}})}{t_{\text{prop}}(V_{\text{nom}})} = \frac{V_{\text{droop}}}{V_{\text{nom}}} \left(\frac{V_{\text{nom}} - V_{\text{th}}}{V_{\text{droop}} - V_{\text{th}}} \right)^\alpha$$

In a synchronous digital pipeline with clock period T_{clk} , setup time t_{setup} , and clock skew t_{skew} , timing closure requires:

$$t_{\text{prop}} + t_{\text{setup}} + t_{\text{skew}} \leq T_{\text{clk}}$$

Stretching t_{prop} reduces setup slack. A timing failure occurs only if the measured path crosses its slack limit; a brownout reset depends on a separately configured supervisor threshold. Local decoupling can reduce the rail excursion, subject to its impedance and placement.

³ | **Inverter Dead-Time Insertion:** Timer dead time is selected from switch and driver timing over temperature and load, then verified with the actual gate waveforms. An overlap can create a damaging bus short; a numeric current or fuse outcome requires the circuit impedance and protection response.

⁴ | **Sakurai-Newton Delay Modeling:** The alpha-power law models voltage-dependent gate delay; its parameters are technology dependent. The model predicts reduced setup slack during droop, while a missed deadline, register error, or reset must be established against measured path slack and device thresholds.

C.6 DRAM Contention and Interconnect Arbitration

In shared-memory multi-core SoC platforms, direct memory access (DMA) transfers and vision prefetching compete with the permission path for memory controller command queues and physical DRAM banks.

C.6.1 DRAM bank organization and conflict latency

A DRAM chip is structured into multiple independent banks, each possessing a single row buffer. Accessing storage cells requires three sequential operations:

1. **Row activate** (t_{RCD}): Loads a row of storage cells into the bank's row buffer.
2. **Column access** (t_{CAS}): Reads or writes data from the open row buffer onto the data bus.
3. **Row precharge** (t_{RP}): Restores charge to storage capacitors and closes the active row.

When consecutive operations target different rows of the same bank (a *bank conflict*), the controller may need to precharge and activate a row before column access, adding approximately $t_{\text{RP}} + t_{\text{RCD}}$. Each bank can have its own open row. The penalty and number of switches depend on address mapping and the controller schedule.⁵

C.6.2 Interconnect crossbars and burst contention

An accelerator tile is split into bounded bus transactions; it is not one atomic AXI burst. For an illustrative FIFO controller that admits all K transactions ahead of a safety request, with no priority bypass or new arrivals, the ideal bus-transfer component is

$$t_{\text{queue,ideal}} = \frac{\sum_{k=1}^K B_k}{\text{BW}_{\text{bus}}}.$$

Add the bank-switch penalties for the *actual* address sequence and then the safety request's own transfer and compute times. This is a constructed scheduling model, not a portable worst-case bound: refresh, protocol overhead, controller reordering, and competing traffic can increase latency, while qualified QoS can bypass queued work. Section 13.3 gives a recomputable case. A local, coherently published safety snapshot can avoid this DDR path, but TCM capacity alone does not provide the snapshot or bound its transfer time.

C.7 Clock Synchronization and Network Timestamps

Distributed perception must convert sensor timestamps into a common clock domain with a measured conversion-error bound.

C.7.1 Precision time protocol message exchange

The IEEE 1588 Precision Time Protocol (PTP) synchronizes a Slave clock (S) to a Master clock (M) by exchanging four physical timestamps across the communication link:

1. t_1 : Master transmits a *Sync* frame, recording its local transmission timestamp.
2. t_2 : Slave receives the *Sync* frame, recording its local arrival timestamp.
3. t_3 : Slave transmits a *Delay_Req* frame, recording its local transmission timestamp.
4. t_4 : Master receives the *Delay_Req* frame, recording its local arrival timestamp.

Assuming stable clocks during the exchange and symmetric forward and reverse propagation delay ($D_{M \rightarrow S} = D_{S \rightarrow M} = D$), the time relationships satisfy:

$$t_2 = t_1 + O + D$$

$$t_4 = t_3 - O + D$$

where O is the slave-minus-master clock offset. Under those assumptions, the estimated offset is:

$$O = \frac{(t_2 - t_1) - (t_4 - t_3)}{2}$$

⁵ | **DRAM Bank-Conflict Penalties:** A row change in one bank can add precharge and activate time. Video DMA can also queue transactions ahead of a safety read. The resulting delay must be bounded for the specific address mapping, arbitration policy, and workload; a fixed penalty per request is generally unjustified.

The one-way network propagation delay is:

$$D = \frac{(t_2 - t_1) + (t_4 - t_3)}{2}$$

The slave estimates the mapping to the master domain by subtracting O . Asymmetric path delay, oscillator drift between exchanges, and timestamp error leave a residual that must be bounded for evidence-age checks.

C.7.2 Physical layer hardware timestamping

Software timestamping includes operating-system dispatch and queue variation, which must be measured for the selected network stack.

PTP-capable hardware can timestamp near the Ethernet PHY or media access control (MAC) boundary, reducing host-dispatch error.⁶ It does not by itself establish a submicrosecond end-to-end clock bound; topology, asymmetry, synchronization interval, timestamp location, and MCU clock conversion must be characterized.

C.8 Heterogeneous SoC Mailboxes and Memory Barriers

Some robotics SoCs couple an application processor running Linux with a real-time core or microcontroller. Their communication contract must specify memory ownership, cache behavior, message validity, and what the real-time side does when no new message arrives.

C.8.1 Remote processor messaging over shared SRAM

An illustrative shared-memory mailbox works as follows:

1. Core A writes a message payload to a fixed buffer in shared SRAM.
2. Core A makes the payload visible to Core B using the platform's coherent mapping or explicit cache maintenance and ordering.
3. Core A triggers an inter-processor interrupt (IPI) by writing to Core B's hardware mailbox register.
4. Core B checks publication order, integrity, age, and permission before using the payload.

Linux RPMsg is a virtio-based messaging framework that may use shared memory and a platform-specific kick. It does not prescribe this fixed-buffer protocol or a universal latency bound.

C.8.2 CPU pipeline reordering and memory fences

Weak memory ordering and noncoherent caches can let a receiver observe a notification without a current payload unless the producer and consumer follow one published protocol:

```
// Core A (Application Processor); q_target and q_cmd are joint_target_t structs
shared_buffer->q_target = q_cmd; // Store 1: Payload (struct copy)
mailbox_doorbell = 1;           // Store 2: Interrupt trigger
```

If the CPU or interconnect reorders Store 2 ahead of Store 1, Core B receives the interrupt before the new joint target is committed to physical memory, reading stale or invalid data.

A data memory barrier can order accesses within its selected shareability domain:⁷

```
shared_buffer->q_target = q_cmd; // Struct copy of the joint targets
__asm__ volatile("dmb ish" ::: "memory"); // Only if both peers share this domain
mailbox_doorbell = 1;
```

The barrier in this example is sufficient only if both cores share the inner-shareable coherent domain and the doorbell has the specified device-memory ordering. It does not flush a noncoherent cache or make data visible to an unrelated DMA master. Otherwise, use the platform's appropriate barrier and DMA cache-ownership synchronization, or map the buffer coherently, and verify the receiver's acquire side. The real-time reader still needs a bounded validation attempt and a lease-limited fallback when publication is torn or stale.

⁶ | **Hardware Timestamping:** A compatible PHY or MAC timestamps frames closer to the wire than a software interrupt handler. Hardware timestamping removes one source of dispatch variation, while path asymmetry and clock conversion remain. Timestamp uncertainty belongs in the physical belief record and permission budget.

⁷ | **Hardware Memory Barriers:** Barriers order accesses only within their specified shareability and memory-attribute contract. On a noncoherent route, cache clean/invalidate operations or coherent mappings are also required. Arm's memory-ordering guide distinguishes ordering from coherence.

C.9 Actuator Electrical and Thermal Dynamics

Actuators convert electrical energy into mechanical work through electromagnetic Lorentz forces, subject to coupled electrical and thermal constraints.

C.9.1 Stator electrical rise time

A brushless DC (BLDC) motor phase winding exhibits phase resistance R_{phase} and phase inductance L_{phase} . Applying terminal voltage V_{terminal} against back-electromotive force $e_{\text{bemf}} = K_e \omega_m$ yields the first-order stator current ODE:

$$L_{\text{phase}} \frac{dI(t)}{dt} + R_{\text{phase}} I(t) = V_{\text{terminal}} - K_e \omega_m$$

For a voltage step applied from rest ($\omega_m = 0$), phase current rises exponentially:

$$I(t) = \frac{V_{\text{terminal}}}{R_{\text{phase}}} (1 - e^{-t/\tau_e}), \quad \tau_e = \frac{L_{\text{phase}}}{R_{\text{phase}}}$$

where τ_e is the electrical time constant: in this ideal step model, current reaches about 63 percent of its final value after one τ_e . Torque $\tau_m = K_t I$ follows this current only while the magnetic, voltage, and control assumptions hold; a command is not an instantaneous torque change.

C.9.2 Lumped thermal networks and duty cycle

Current flowing through stator windings dissipates resistive Joule heat:

$$P_{\text{loss}}(t) = I_{\text{rms}}^2(t) R_{\text{phase}}(T)$$

where copper resistance increases linearly with temperature:

$$R_{\text{phase}}(T) = R_0 [1 + \alpha_{\text{Cu}}(T - T_0)]$$

with temperature coefficient $\alpha_{\text{Cu}} \approx 0.00393 \text{ K}^{-1}$ for copper. In a single-node lumped thermal model with thermal resistance to ambient θ_{JA} (in K/W) and thermal heat capacity C_{th} (in J/K), temperature rise $\Delta T(t) = T_{\text{winding}}(t) - T_{\text{ambient}}$ satisfies:⁸

$$C_{\text{th}} \frac{d\Delta T(t)}{dt} = P_{\text{loss}}(t) - \frac{\Delta T(t)}{\theta_{JA}}$$

Under constant power, temperature rises with thermal time constant $\tau_{\text{th}} = \theta_{JA} C_{\text{th}}$:

$$\Delta T(t) = P_{\text{loss}} \theta_{JA} (1 - e^{-t/\tau_{\text{th}}})$$

Continuous torque capacity τ_{cont} is bounded by maximum allowable winding temperature ΔT_{max} :

$$\tau_{\text{cont}} \leq K_t \sqrt{\frac{\Delta T_{\text{max}}}{\theta_{JA} R_{\text{phase}}}}$$

An above-continuous torque may be permitted transiently if the device's temperature estimate and specified current-time limits allow it. The integral $\int_0^t I^2 dt$ is one possible protection measure, not a universal motor rating.

⁸ | **Stator Thermal Time Constants:** Resistive Joule heating elevates motor phase winding temperatures during continuous high-torque maneuvers. Because copper electrical resistivity increases linearly with temperature, hot windings exhibit higher resistance, reducing torque output under terminal voltage saturation. Exceeding winding insulation temperature limits breaks down dielectric coatings, causing inter-turn short circuits and irreversible actuator destruction.

C.9.3 Reflected rotor inertia and angular jerk limits

Geared electromagnetic actuators amplify delivered motor torque while multiplying rotor inertia at the joint output. For a transmission with gear reduction ratio $N = \omega_{\text{motor}}/\omega_{\text{joint}}$, kinetic energy conservation dictates the reflected inertia acting at the output link:

$$E_k = \frac{1}{2} J_{\text{rotor}} \omega_{\text{motor}}^2 + \frac{1}{2} J_{\text{load}} \omega_{\text{joint}}^2 = \frac{1}{2} (J_{\text{load}} + N^2 J_{\text{rotor}}) \dot{q}^2$$

The effective rotational inertia seen at joint coordinate q is:

$$J_{\text{eff}} = J_{\text{load}} + N^2 J_{\text{rotor}}$$

The joint equation of motion under motor electromagnetic torque τ_{motor} is:

$$\ddot{q} = \frac{N \tau_{\text{motor}} - \tau_{\text{ext}}}{J_{\text{load}} + N^2 J_{\text{rotor}}}$$

Differentiating $\ddot{q}$ with respect to gear ratio N at zero external load yields:

$$\frac{\partial \ddot{q}}{\partial N} = \frac{\tau_{\text{motor}} (J_{\text{load}} - N^2 J_{\text{rotor}})}{(J_{\text{load}} + N^2 J_{\text{rotor}})^2}$$

Setting $\frac{\partial \ddot{q}}{\partial N} = 0$ yields the **inertia-matching ratio** $N^* = \sqrt{J_{\text{load}}/J_{\text{rotor}}}$. When $N > N^*$, counter-intuitively, increasing the gear ratio decreases delivered joint acceleration because the motor spends the majority of its torque accelerating its own rotor rather than the link.

C.9.3.1 Dynamic angular jerk and torque slew limits

Angular jerk $j_{\text{joint}}(t) = \ddot{\ddot{q}}(t)$ measures the time rate of change of joint acceleration:

$$j_{\text{joint}}(t) = \frac{N}{J_{\text{load}} + N^2 J_{\text{rotor}}} \frac{d\tau_{\text{motor}}(t)}{dt}$$

Electromagnetic torque relates to stator quadrature current via torque constant K_t : $\tau_{\text{motor}} = K_t I_q$. From the stator electrical dynamics in this section, current slew rate is bounded by winding inductance L_{phase} :

$$\frac{d\tau_{\text{motor}}}{dt} = K_t \frac{dI_q}{dt} = \frac{K_t}{L_{\text{phase}}} (V_{\text{terminal}} - K_e \omega_m - R_{\text{phase}} I_q)$$

Under maximum DC bus terminal voltage saturation V_{max} , the physical upper bound on delivered angular jerk is:

$$|j_{\text{max}}| \leq \frac{N K_t (V_{\text{max}} - K_e \omega_m)}{L_{\text{phase}} (J_{\text{load}} + N^2 J_{\text{rotor}})}$$

When an unprivileged neural policy outputs a step torque discontinuity $\Delta\tau$ across discrete timesteps, the commanded derivative $\frac{d\tau}{dt} \rightarrow \infty$ cannot be delivered instantaneously by physical silicon. The inverter enters hard voltage saturation, causing steep current slew (dI/dt), intense thermal stress on power MOSFETs, and shock loads across gear-tooth contact flanks ($\sigma_b \propto W_t/(bmY)$). Synchronous pre-actuation invariant filters evaluate numerical torque derivatives $(\tau[k] - \tau[k-1])/\Delta t$ to ensure requested actions remain within allowable jerk envelopes before latching power stages.

C.10 DC Bus Power Distribution and Energy Absorption

A robotic platform's electrical power distribution network must supply acceleration surges and absorb regenerative braking energy across finite bus capacitance and harness impedance.

C.10.1 Inductive droop and capacitive decoupling

When multiple actuators accelerate, harness resistance and inductance impede source current while local capacitance supplies the difference. For a specified waveform, the resistive and inductive terms are $\Delta I_{\text{source}} R_{\text{bus}}$ and $L_{\text{bus}} dI_{\text{source}}/dt$; capacitor discharge obeys

$$\Delta V_C(t) = \frac{1}{C_{\text{bus}}} \int_0^t (I_{\text{load}}(s) - I_{\text{source}}(s)) ds.$$

These terms cannot simply be added as simultaneous peaks without a circuit and waveform model. If the resulting voltage crosses a configured under-voltage lockout (UVLO) threshold for long enough, the supply may disable or reset; the threshold, filtering, and plant consequence require measurement.

C.10.2 Regenerative braking surges and chopper sizing

During controlled regenerative deceleration, some mechanical kinetic energy $E_k = \frac{1}{2}mv^2$ (linear) or $E_k = \frac{1}{2}J_{\text{load}}\dot{q}_0^2$ (rotational) may return through the inverter. If the battery cannot accept it and an assumed fraction η_{regen} reaches the local bus capacitor C_{bus} before another sink acts, the ideal capacitor relation is:

$$\Delta E_C = \frac{1}{2}C_{\text{bus}} (V_{\text{final}}^2 - V_{\text{initial}}^2) = \eta_{\text{regen}} E_k$$

$$V_{\text{final}} = \sqrt{V_{\text{initial}}^2 + \frac{2\eta_{\text{regen}} E_k}{C_{\text{bus}}}}$$

For an implementation with an active **brake chopper**, the resistor and switch must absorb the modeled peak power and pulse energy when $V_{\text{bus}} \geq V_{\text{clamp}}$.⁹

$$R_{\text{brake}} \leq \frac{V_{\text{clamp}}^2}{P_{\text{regen, peak}}}$$

$$P_{\text{regen, peak}} \approx \eta_{\text{regen}} \cdot \tau_{\text{brake}} \cdot \omega_{\text{max}}$$

The resistor inequality is only a peak-power sizing check; minimum resistance for switch current, pulse energy, and thermal recovery impose additional constraints.

C.11 Real-Time Fieldbuses and Lock-Free Synchronization

Coordinating distributed drives requires a measured network schedule and a bounded local publication protocol.

C.11.1 Fieldbus serialization and schedulability

On a shared serial bus, N_{axes} drives transmit telemetry and receive setpoints cyclically. Serialization time depends on the complete frame, physical bit rates, and any stuffing or retransmission allowance.

CAN FD uses non-destructive priority arbitration at its nominal bit rate; a winning frame then sends its data phase at the configured data bit rate. An error-free serialization estimate for frame i is

$$t_{\text{frame},i} \approx \frac{B_{\text{arb},i}}{R_{\text{nom}}} + \frac{B_{\text{data},i}}{R_{\text{data}}} + t_{\text{stuff},i} + t_{\text{interframe},i}.$$

This estimate is not a response-time bound. A lower-priority frame already transmitting can block a newly ready high-priority frame until that transmission ends; arbitration losers defer without a collision-error retry. Bound each message's queuing and release jitter with the actual priority set, frame lengths, error assumptions, and bus load; Bosch's CAN FD description distinguishes the arbitration and data phases.

⁹ | **Dynamic Braking Choppers:** A qualified chopper can dissipate returned energy when the battery is unavailable as a sink. Its threshold, switch and resistor ratings, pulse duration, and thermal recovery must be tested. Regeneration and chopper operation are not inherent to every emergency stop; safe torque off can instead leave a motor coasting.

EtherCAT processes data as frames pass through slave controllers. For a specified topology, a cycle budget includes serialization of every configured frame, master and slave forwarding, PHY and cable propagation, processing, and a guard for jitter and error recovery:

$$T_{\text{cycle}} \geq \sum_f \frac{B_f}{R_{\text{link}}} + t_{\text{master}} + \sum_s t_{\text{slave},s} + t_{\text{prop}} + t_{\text{guard}}.$$

The achievable cycle period is configuration dependent; short advertised periods do not establish a deadline for a particular machine. The EtherCAT Technology Group describes the processing method and configurable process data.

C.11.2 Lock-free seqlocks for multi-rate shared memory

When a producer updates multiword state read by a deadline-bound enforcer, waiting on a producer-held mutex can violate the reader's deadline. The single-slot mailbox in algorithm C.1 uses a sequence counter to detect a torn snapshot.

A **sequence counter** marks a single writer's publication in progress with an odd value and a completed publication with an even value:

1. Writer Protocol:

- Publish an odd sequence value, write the payload, and make it visible through the platform's coherent or cache-maintained memory contract.
- Publish the next even value with release ordering.

2. Reader Protocol:

- Read S_1 with acquire ordering. If it is odd, reject this publication without waiting.
- Make one bounded copy to scratch memory using the platform's required load ordering, then read S_2 .
- Accept only if $S_1 = S_2$ and both are even, and the copied frame passes integrity, timestamp, and lease checks. Otherwise retain the last validated local command only while its lease and stopping margin permit; then enter the state-matched fallback.

The reader's one-attempt execution time can be bounded on the target hardware; success probability depends on the writer's schedule and cannot be inferred from payload size alone. Conventional Linux seqlock readers may retry, as the kernel documentation shows, but a real-time permission gate cannot use an unbounded retry loop.

Algorithm C.1 Single-Slot Trajectory Seqlock Baseline

Require: proposal P (proposal header with sequence_id , t_evidence_ns , sync_error_ns , t_expire_ns ; waypoints $\mathbf{w}_{1..K}$; crc32), scratch buffer B_{scratch} and previously validated active spline B_{active} in static SRAM, shared SRAM mailbox (c_{seq} , $\text{Slot}_{\text{shared}}$), current time t_{now} and evidence-age budget $t_{\text{age,max}}$ (the largest admissible t_{age}), both on the safety microcontroller's clock

Ensure: promoted validated chunk in B_{active} or rejection with B_{active} unchanged

```

1: function PUBLISHACTIONCHUNK( $P$ )
2:    $c_{\text{curr}} \leftarrow \text{ATOMICLOADEXPLICIT}(c_{\text{seq}}, \text{memory\_order\_relaxed})$ 
3:    $\text{ATOMICSTOREEXPLICIT}(c_{\text{seq}}, c_{\text{curr}} + 1, \text{memory\_order\_relaxed}) \triangleright \text{mark odd: write in progress}$ 
4:    $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSHST}) \triangleright \text{outer-shareable store barrier pushes counter}$ 
5:    $\text{COPYMEMORY}(\text{Slot}_{\text{shared}}.\text{payload}, P, \text{sizeof}(P)) \triangleright \text{copy multi-word trajectory}$ 
6:    $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSH}) \triangleright \text{outer-shareable full barrier: payload visible}$ 
7:    $\text{ATOMICSTOREEXPLICIT}(c_{\text{seq}}, c_{\text{curr}} + 2, \text{memory\_order\_release}) \triangleright \text{mark even: publish committed}$ 
8: end function

9: function LATCHLATESTACTIONCHUNK( $B_{\text{scratch}}, B_{\text{active}}, t_{\text{now}}$ )
10:   $c_{\text{start}} \leftarrow \text{ATOMICLOADEXPLICIT}(c_{\text{seq}}, \text{memory\_order\_acquire}) \triangleright \text{load initial counter (ARM LDAR acquire)}$ 
11:  if  $c_{\text{start}}$  is odd then
12:    return STATUS_WRITER_ACTIVE  $\triangleright$  writer active; retain prior spline
13:  end if
14:   $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSHLd}) \triangleright \text{explicit barrier: prevent payload reads hoisting before counter}$ 
15:   $\text{COPYMEMORY}(B_{\text{scratch}}, \text{Slot}_{\text{shared}}.\text{payload}, \text{sizeof}(\text{Payload})) \triangleright \text{tentative copy; active spline unchanged}$ 
16:   $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSHLd}) \triangleright \text{LDAR cannot prevent prior reads sinking; DMB forces payload reads before } c_{\text{end}}$ 
17:   $c_{\text{end}} \leftarrow \text{ATOMICLOADEXPLICIT}(c_{\text{seq}}, \text{memory\_order\_acquire}) \triangleright \text{re-read sequence counter}$ 
18:  if  $c_{\text{start}} \neq c_{\text{end}}$  then
19:    return STATUS_TORN_READ  $\triangleright$  concurrent write detected
20:  end if
21:  if  $\text{COMPUTECrc32}(B_{\text{scratch}}) \neq B_{\text{scratch}}.\text{crc32}$  then
22:    return STATUS_CHECKSUM_INVALID
23:  end if
24:  if  $t_{\text{now}} \geq B_{\text{scratch}}.\text{t\_expire\_ns}$  then
25:    return STATUS_LEASE_EXPIRED  $\triangleright$  expiry is absolute; the reader may only shorten it
26:  end if
27:  if  $t_{\text{now}} - B_{\text{scratch}}.\text{t\_evidence\_ns} + B_{\text{scratch}}.\text{sync\_error\_ns} > t_{\text{age,max}}$  then
28:    return STATUS_EVIDENCE_STALE  $\triangleright$  conversion error is added to every age
29:  end if
30:   $B_{\text{active}} \leftarrow B_{\text{scratch}} \triangleright \text{promote only after sequence, CRC, expiry, and evidence-age checks}$ 
31:  return STATUS_VALID_LATCHED
32: end function

```

Section 4.3 exchanges the proposal header and its chunk payload through this mailbox; section C.14.1 gives their byte layout.

C.12 Camera Projective Geometry and Covariance Propagation

Perception pipelines map continuous Euclidean 3D scenes onto discrete 2D camera photosite grids.

C.12.1 Intrinsic projection matrix and radial distortion

Under the standard pinhole camera model, a 3D point $\mathbf{p}_c = [x_c, y_c, z_c]^\top$ in camera coordinates projects onto pixel coordinates $\mathbf{u} = [u, v]^\top$ via the intrinsic calibration matrix $\mathbf{K}$:¹⁰

$$\tilde{\mathbf{u}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{z_c} \mathbf{K} \mathbf{p}_c = \frac{1}{z_c} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

where (f_x, f_y) are focal lengths in pixel units and (c_x, c_y) is the principal point. Non-linear radial lens distortion is modeled by distortion coefficients (k_1, k_2) :

$$r^2 = x_n^2 + y_n^2, \quad x_d = x_n(1 + k_1 r^2 + k_2 r^4), \quad y_d = y_n(1 + k_1 r^2 + k_2 r^4)$$

where $(x_n, y_n) = (x_c/z_c, y_c/z_c)$ are normalized image coordinates.

C.12.2 Metric unprojection and spatial covariance propagation

When unprojecting a 2D pixel observation $\mathbf{u}$ with measured depth z into 3D camera coordinates, photosite measurement noise $\mathbf{u}_{\text{meas}} = \text{diag}(\sigma_u^2, \sigma_v^2, \sigma_z^2)$ propagates through the unprojection Jacobian $\mathbf{J}_{\text{unproj}}$:

$$\mathbf{p}_c = \begin{bmatrix} (u - c_x)z/f_x \\ (v - c_y)z/f_y \\ z \end{bmatrix}, \quad \mathbf{c} = \mathbf{J}_{\text{unproj}} \mathbf{u}_{\text{meas}} \mathbf{J}_{\text{unproj}}^\top$$

Transforming this 3D point into the robot body frame via rigid extrinsic transform $\mathbf{T}_{bc} = [\mathbf{R}_{bc}, \mathbf{p}_{bc}; \mathbf{0}^\top, 1]$ yields the spatial body covariance:

$$\mathbf{b} = \mathbf{R}_{bcc} \mathbf{R}_{bc}^\top + \mathbf{ext}$$

where $\mathbf{ext}$ models independent extrinsic uncertainty under this first-order approximation; correlations require the corresponding cross terms. A probabilistic clearance buffer may scale with $\sqrt{\lambda_{\max}(\mathbf{b})}$ at a declared tail risk. A deterministic permission check instead needs a justified bounded-error envelope; covariance alone does not give a worst-case bound.

C.12.3 Zhang's planar homography and camera calibration

Zhang's calibration technique computes the camera intrinsic matrix $\mathbf{K}$ by observing a planar calibration rig (such as a checkerboard pattern) across $N \geq 3$ distinct orientations (Zhang 2000). Without loss of generality, assume the model plane lies on $Z = 0$ in world coordinates. A 3D model point $\mathbf{M} = [X, Y, 0, 1]^\top$ maps to a 2D image point $\tilde{\mathbf{u}} = [u, v, 1]^\top$ through a 3×3 planar homography matrix $\mathbf{H}$:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

where $\mathbf{H} = [\mathbf{h}_1 \ \mathbf{h}_2 \ \mathbf{h}_3] = \lambda \mathbf{K} [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ with arbitrary nonzero scalar λ .

C.12.3.1 Orthogonality constraints on the absolute conic

Because the rotation vectors $\mathbf{r}_1$ and $\mathbf{r}_2$ are orthonormal columns of an $SO(3)$ matrix ($\mathbf{r}_1^\top \mathbf{r}_2 = 0$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\| = 1$), inverting $\mathbf{K}$ yields:

$$\mathbf{r}_1 = \frac{1}{\lambda} \mathbf{K}^{-1} \mathbf{h}_1, \quad \mathbf{r}_2 = \frac{1}{\lambda} \mathbf{K}^{-1} \mathbf{h}_2$$

¹⁰ | **Pinhole Intrinsic Projection:** The pinhole model projects 3D points onto a 2D sensor through calibrated intrinsics. Under fixed pixel noise, the lateral variance after depth unprojection grows with squared distance; depth-error and calibration terms follow their own models. Uncalibrated intrinsics can bias a clearance estimate and must be included in its error envelope.

Imposing orthonormality on $\mathbf{r}_1$ and $\mathbf{r}_2$ defines two fundamental algebraic constraints per homography:

$$\begin{aligned}\mathbf{h}_1^\top \mathbf{K}^{-\top} \mathbf{K}^{-1} \mathbf{h}_2 &= 0 \\ \mathbf{h}_1^\top \mathbf{K}^{-\top} \mathbf{K}^{-1} \mathbf{h}_1 - \mathbf{h}_2^\top \mathbf{K}^{-\top} \mathbf{K}^{-1} \mathbf{h}_2 &= 0\end{aligned}$$

The symmetric matrix $\mathbf{B} = \mathbf{K}^{-\top} \mathbf{K}^{-1}$ represents the image of the absolute conic (IAC):

$$\mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{12} & B_{22} & B_{23} \\ B_{13} & B_{23} & B_{33} \end{bmatrix}$$

Defining the 6-vector $\mathbf{b} = [B_{11}, B_{12}, B_{22}, B_{13}, B_{23}, B_{33}]^\top$, each quadratic term $\mathbf{h}_i^\top \mathbf{B} \mathbf{h}_j$ can be rewritten as the linear inner product $\mathbf{v}_{ij}^\top \mathbf{b}$, where:

$$\mathbf{v}_{ij} = \begin{bmatrix} h_{i1}h_{j1} \\ h_{i1}h_{j2} + h_{i2}h_{j1} \\ h_{i2}h_{j2} \\ h_{i3}h_{j1} + h_{i1}h_{j3} \\ h_{i3}h_{j2} + h_{i2}h_{j3} \\ h_{i3}h_{j3} \end{bmatrix}$$

The two constraints per observed orientation become the linear system:

$$\begin{bmatrix} \mathbf{v}_{12}^\top \\ (\mathbf{v}_{11} - \mathbf{v}_{22})^\top \end{bmatrix} \mathbf{b} = \mathbf{0}$$

Stacking $N \geq 3$ images forms the homogeneous system $\mathbf{V}\mathbf{b} = \mathbf{0}$, where $\mathbf{V} \in \mathbb{R}^{2N \times 6}$. The right singular vector of $\mathbf{V}$ corresponding to the smallest singular value provides $\mathbf{b}$ up to a scale factor. The intrinsic parameters (f_x, f_y, c_x, c_y) and skew γ are then uniquely extracted via Cholesky factorization of $\mathbf{B}$. This closed-form linear solution initializes the non-linear Levenberg–Marquardt optimization minimizing reprojection residuals over all observed fiducial corners.

C.13 Kinematic Stopping Envelopes and Information Age Lag

Halting an embodied machine transporting mass m at speed v requires balancing sensorimotor information freshness and mechanical braking deceleration.

C.13.1 Sensorimotor information age and lag distance

For a specified hazard detector and stop path, let τ_{delay} bound the time from the physical state represented by the evidence until validated braking deceleration begins:

$$\tau_{\text{delay}} = t_{\text{age}} + t_{\text{detect}} + t_{\text{compute}} + T_{\text{bus}} + T_{\text{act}}$$

where:

- t_{age} converts the evidence timestamp to the controller clock and includes exposure/readout and clock error.
- t_{detect} bounds recognition or lease-expiry detection.
- t_{compute} is the admitted stop decision, including any required input transfer.
- T_{bus} bounds command dispatch and network queuing.
- T_{act} ends when the measured actuator begins delivering the specified deceleration, not when a command register is written.

Under the explicit simplifying assumption that speed cannot increase before brake onset, the blind travel is bounded by $d_{\text{blind}} = v\tau_{\text{delay}}$. If propulsion or a slope can increase speed, use its validated acceleration bound in this interval instead.

C.13.2 Braking dynamics and friction limits

Tire-floor friction and actuator torque place upper limits on achievable deceleration; $a \leq \mu g$ does not establish what braking the machine will deliver. Let $a_{\text{brake}} > 0$ be a validated lower bound on *achieved* deceleration throughout the stop, for the declared load, temperature, slope, brake condition, and surface. A justified friction floor μ_{floor} may support that bound, but does not replace brake tests. With constant or stronger braking after onset, a localization bound δ_{loc} , a fixed protective clearance δ_{margin} , and a tracking-error bound ϵ_{track} ,

$$d_{\text{stop}}(v) = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}}.$$

Motion is admitted only when $d_{\text{stop}}(v) \leq D_{\text{clear}}$, where D_{clear} is taken as a validated lower bound on available path clearance. This is the inset stopping distance that section 12.5 defends. The sampled controller must reserve intersample travel and a state-matched stop before the viable region is lost.

C.13.3 Kinetic energy derating and friction collapse

In an ideal friction-limited model, lowering μ from μ_{nom} to a positive μ_{fault} changes the braking-distance ratio to $\mu_{\text{nom}}/\mu_{\text{fault}}$. That ratio is illustrative; actual deceleration can also be limited by brake hardware or control. If surface friction is not sensed in time, derive a_{brake} from a controlled surface condition and inspection schedule, exclude degraded surfaces, or withhold motion. For the constant-speed pre-brake model above, solving the clearance inequality gives

$$v_{\text{clamp}} = -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{(a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \epsilon_{\text{track}})}.$$

This clamp is valid only when D_{clear} exceeds the three fixed allowances, the braking and timing lower bounds hold, and the speed bound is enforced before entering the hazard corridor. No positive clamp follows if the available deceleration floor is unknown or zero.

C.14 Wire Formats for the Record Chain

The chapters name each record's fields by meaning and state the invariant the record enforces. This section gives the byte layouts for records that must fit a fixed-size mailbox or log, and the canonical field order for the offline records that are stored and bound by digest. Every absolute time is an unsigned 64-bit count of nanoseconds on the safety microcontroller's monotonic clock, every duration is an unsigned count of nanoseconds, and `sync_error_ns` carries the conversion bound for evidence captured on another clock. The clock and integrity policies themselves belong to section 4.3; the layouts here fix the algorithm for each integrity job. Corruption detection uses a CRC-32 (IEEE 802.3) over header and payload, lineage uses a SHA-256 digest of the consumed record, and authority uses HMAC-SHA-256 on operator commands, a keyed hash chain on the forensic log, and Ed25519 signatures on fault records and release artifacts. The authority, fault, and release records are each followed by the protocol that writes or checks them.

Fixed layouts are little-endian and packed in the order listed. Offsets are hexadecimal byte positions from the start of the record, and every multibyte field sits on its natural alignment, so no field needs hidden padding. A field whose length depends on a count comes after every fixed field, and its offset is written in terms of that count. The record itself carries K waypoints, N_j commanded joints, K_b barrier constraints, and K_a approach channels; the static authority manifest fixes N_c command components and N_s state components. Offline records (placement, fault, release) are identified by the SHA-256 digest of their canonical encoding, and the record that consumes one stores that digest under the consumed record's name.

C.14.1 Proposal header and chunk payload

Every proposal that crosses the proposal boundary opens with the same header, and `payload_kind` selects the payload that follows at 0x50. Table C.7 lays out the header with the minimal chunk payload of section 4.3. The chunk carries kinematic setpoints only. Torque is computed on the permission side, where the tracker derives it from the admitted reference, so a learned proposal never names a motor torque.

Table C.7: Proposal header and chunk payload: Field offsets, scalar types, units, and invariants for a chunk proposal exchanged through the seqlock mailbox of algorithm C.1. The header occupies 0x00–0x4F in every proposal; the other payload layouts in this section begin at 0x50.

Offset	Field	Scalar type	Unit	Invariant and check
0x00	sequence_id	uint64_t	dimensionless	Strictly increasing per producer; a record not newer than the last accepted one is rejected.
0x08	t_evidence_ns	uint64_t	ns, permission clock	Capture epoch of the oldest evidence the proposal rests on, converted to the safety microcontroller’s clock.
0x10	t_issue_ns	uint64_t	ns, permission clock	Time the producer published the record.
0x18	t_expire_ns	uint64_t	ns, permission clock	Requested absolute expiry, no later than the chunk lease allows; the permission path may only shorten it.
0x20	frame_id	uint64_t	dimensionless	Transform-tree key and calibration revision of the setpoints’ frame.
0x28	parent_digest	uint8_t[32]	SHA-256	Digest of the record this proposal was derived from; establishes lineage, authenticates nothing.
0x48	sync_error_ns	uint32_t	ns	Bound on the evidence time’s conversion error; added to every age the reader checks.
0x4C	payload_kind	uint16_t	enumeration	CHUNK for this payload; INTENT, TRAJECTORY, and OPERATOR select the other payloads.
0x4E	payload_len	uint16_t	bytes	Payload length; bounds the parser before any payload byte is read.
0x50	num_waypoints	uint16_t	count	Chunk horizon K ; a count outside the configured range is rejected.
0x52	num_joints	uint8_t	count	Commanded joints N_j ; must equal the configured count, and payload_len must equal $4 + 8KN_j$.
0x53	reserved	uint8_t	—	Zero.
0x54	joint_positions	float32[K][N _j]	rad	Commanded joint angles within the reach limits ($q_{\min} \leq q \leq q_{\max}$).
0x54 + 4KN _j	joint_velocities	float32[K][N _j]	rad/s	Commanded joint velocities within certified ceilings ($\ \dot{q}\ \leq \dot{q}_{\max}$).
0x50 + payload_len	crc32	uint32_t	checksum	Last field; IEEE 802.3 CRC-32 over all preceding bytes.

C.14.2 Intent payload

The intent lease of section 10.7 travels under the proposal header with payload_kind set to INTENT (table C.8). Three header fields take specific meanings. t_evidence_ns is the parent belief record’s evidence epoch, never its estimate epoch; parent_digest is that belief record’s digest, which the permission path matches against a retained record; and t_expire_ns is the requested expiry, which the permission path shortens to the earliest of the evidence horizon, its configured lease ceiling, and the stop deadline. The six bounds values are three conservative position-error envelopes and three task tolerances in the base frame named by frame_id. They are not an encoding of a covariance, and the producer must justify any conversion from its covariance estimate, including calibration bias and the declared tail risk.

Table C.8: Intent payload: The intent lease as a proposal-header payload. Orientation tolerances and contact-mode rules come from an independently approved task profile; the quaternion is the proposed orientation only.

Offset	Field	Scalar type	Unit	Invariant and check
0x00–0x4F	proposal header	as above	as above	Table C.7, with payload_kind = INTENT.

Offset	Field	Scalar type	Unit	Invariant and check
0x50	target_pose	float64[7]	m; unit quaternion	Base-frame position and orientation; finite values, normalized quaternion, workspace prefilter.
0x88	bounds	float32[6]	m	Error envelope $e_{x,y,z}$, then task tolerance $r_{x,y,z}$; $0 \leq e_i \leq r_i$, each checked against the approved task profile.
0xA0	wrench_request	float32[6]	N; N·m	Three requested forces and three torques; clamped or rejected against independent task and actuator limits.
0xB8	terminal_request	uint32_t	enumeration	Requested terminal mode; the permission path selects a feasible approved response from the current load and contact state.
0xBC	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes.

C.14.3 Trajectory payload

The trajectory record of section 11.7 replaces the chunk on the running machine and travels under the same proposal header, with **payload_kind** set to **TRAJECTORY** and **parent_digest** set to the digest of the active intent lease (table C.9). The header's **t_expire_ns** carries the chunk lease; the intent's own expiry comes from the lease that **parent_digest** identifies. The setpoint samples and the assumptions sidecar travel outside this fixed part. The permission path verifies **sidecar_digest** over both, validates the complete candidate in staging memory, and latches the admitted setpoints and stop suffix in protected memory before tracking begins.

Table C.9: Trajectory payload: The trajectory record as a proposal-header payload. The four deadlines are absolute times on the safety microcontroller's clock; the variable-length setpoint samples and the assumptions sidecar are bound by **sidecar_digest** and validated before activation.

Offset	Field	Scalar type	Unit	Invariant and check
0x00–0x4F	proposal header	as above	as above	Table C.7, with payload_kind = TRAJECTORY .
0x50	sample_period_ns	uint32_t	ns	Setpoint cadence Δt_{step} ; must match the configured decoder.
0x54	num_samples	uint16_t	count	Nominal setpoints in the digested sample payload; bounds its size.
0x56	num_joints	uint8_t	count	Commanded joints N_j ; must equal the configured count.
0x57	payload_version	uint8_t	enumeration	Decoder version; an unknown version is rejected.
0x58	t_start_ns	uint64_t	ns, permission clock	First nominal sample.
0x60	t_commit_ns	uint64_t	ns, permission clock	Latest entry into the matched stop suffix; $t_{\text{commit}} = t_{\text{stop_end}} - T_{\text{stop}}$.
0x68	t_plan_exp_ns	uint64_t	ns, permission clock	End of the nominal setpoints.
0x70	t_stop_end_ns	uint64_t	ns, permission clock	Protected stop completion; $t_{\text{start}} \leq t_{\text{commit}} < t_{\text{plan_exp}} < t_{\text{stop_end}}$.
0x78	sidecar_digest	uint8_t[32]	SHA-256	Digest over the setpoint samples and the assumptions sidecar; binds content.
0x98	seam_state	float32[3][N_j]	rad; rad/s; rad/s ²	Entry state $(q, \dot{q}, \ddot{q})$ of each joint.
0x98 + 12 N_j	stop_suffix	float32[6][N_j]	quintic coefficients	One quintic stop per joint; must be resident and feasible from every state reachable before t_{commit} .
0x98 + 36 N_j	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes.

C.14.4 Enforcement record

The enforcement record of section 12.11 is configuration, not a proposal, so it carries no proposal header (table C.10). Three digests name what the record rests on: the evaluation record, the limit records behind the stopping envelope, and the validity region. Each threshold names the fallback-ladder rung that a failed check selects. Where the rung depends on whether the body is moving, the row names both, **STOP** in motion and **HOLD** at standstill. A proposal refused while the active lease holds selects no rung, because the setpoints already admitted stay in force until that lease lapses.

Table C.10: Enforcement record: Thresholds of the permission path with the fallback-ladder rung each failed check selects, and digests of the evaluation record, limit records, and validity region the thresholds rest on. The record is configuration and telemetry; it adds no packet to the proposal stream.

Offset	Field	Scalar type	Unit	Check, and the rung a failure selects
0x00	evaluation_record_digest	uint8_t[32]	SHA-256	Evaluation record whose runtime monitor specifications become this record's monitor channels.
0x20	limit_record_digest	uint8_t[32]	SHA-256	Canonical list of the limit records whose braking, onset, and actuator limits parameterize $d_{\text{stop}}(v)$.
0x40	validity_region_digest	uint8_t[32]	SHA-256	validity_region , the plant-parameter bounds the thresholds rest on. On the mobile manipulator it covers the arm at the door, where it equals the target ODD, and at the packing station through the mug pick and the handover approach. Each base configuration, normal or restricted, loads as its own enforcement record, so this field holds the region digest of that configuration and its friction premise. A live state outside it that the machine observes selects STOP .
0x60	loop_period_ns	uint32_t	ns	Permission-loop period T_{loop} ; the placement record's nominal_period_ns must equal it.
0x64	deadline_ns	uint32_t	ns	Permission-path deadline inside the tick; a cycle at risk of overrunning it selects HOLD .
0x68	evidence_age_ceiling_ns	uint32_t	ns	Refusal threshold of the evidence-epoch check, the budgeted observation age plus the clock-conversion bound. A proposal whose upper evidence age exceeds it is refused, and the permission path shortens $t_{\text{expire_ns}}$ to at most $t_{\text{evidence_ns}}$ plus the ceiling minus sync_error_ns . A proposal whose sequence_id is not newer is refused; an expired lease selects STOP in motion and HOLD at standstill.
0x6C	watchdog_timeout_ns	uint32_t	ns	The self-watchdog is fed only after a completed tick; expiry requests INHIBIT through the independently wired trip.
0x70	t_stop_ns	uint32_t	ns	Stopping-time bound from the maximum admitted speed; a reference that leaves less clearance than $\epsilon_{\text{track}} + v_{\text{max}} t_{\text{stop}}$ selects STOP .
0x74	pos_tracking_bound	float32	m	$ x_{\text{meas}} - r_{\text{cmd}} \leq \epsilon_{\text{track}}$; STOP above it.
0x78	pos_critical_bound	float32	m	$ x_{\text{meas}} - r_{\text{cmd}} \leq \epsilon_{\text{crit}}$, with $\epsilon_{\text{crit}} > \epsilon_{\text{track}}$; STOP above it.
0x7C	vel_ceiling_limit	float32	rad/s	$ \dot{q} \leq \dot{q}_{\text{max}}$; PROJECT (QP clamp).
0x80	torque_slew_ceiling	float32	N·m/s	$ \dot{\tau} \leq \dot{\tau}_{\text{max}}$; PROJECT (slew limiter).
0x84	actuator_curr_limit	float32	A	$ I_{\text{phase}} \leq I_{\text{peak}}$; STOP .
0x88	dc_bus_overnvolt_thresh	float32	V	$V_{\text{bus}} \leq V_{\text{max_safe}}$; STOP , with the brake chopper absorbing returned energy.

Offset	Field	Scalar type	Unit	Check, and the rung a failure selects
0x8C	num_barriers	uint16_t	count	Barrier constraints K_b ; must equal the configured count.
0x8E	num_approach_channels	uint16_t	count	Approach channels K_a ; must equal the configured count.
0x90	cbf_barrier_params	float32[K_b] [2]	normalized distance; s^{-1}	Margin δ_k and class- $\mathcal{K}$ rate α_k of each barrier; $h_k(x) \geq \delta_k$, else PROJECT, or STOP when no feasible input remains.
0x90 + $8K_b$	approach_channel_ids	uint32_t[K_a]	enumeration	Zone and activation condition of each channel, drawn from the validity region, such as the tool point inside the door zone or accept_item held.
0x90 + $8K_b + 4K_a$	approach_ceilings	float32[K_a]	m/s	Ceiling on tool-point approach speed while its channel is active; a faster proposal takes PROJECT (QP clamp), and the channel that bound it is logged.
0x90 + $8K_b + 8K_a$	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes; a mismatch rejects the stored record, and no motion is admitted until a valid copy loads.

C.14.5 Placement record

The placement record of section 13.8 is offline evidence with a fixed header and a linked test ledger (table C.11). It carries no signature of its own; the signed release manifest binds it through `placement_record_digest`. `enforcement_record_digest` names the enforcement record whose `deadline_ns` was tested, and `test_ledger_digest` identifies the external ledger that states the declared worst-case execution time the loaded test must not exceed, together with distributions, sample counts, test conditions, mitigations, and untested states. A maximum that has not been measured is stored as all ones, never zero, the same unavailable-value convention the fault record uses.

Table C.11: Placement record header: Configuration values and observed maxima, with digests linking the tested enforcement record and the external test ledger. The header alone does not establish freedom from interference.

Offset	Field	Scalar type	Unit	Invariant and check
0x00	execution_domain_id	uint64_t	enumeration	Must match the declared topology (application processor, NPU, GPU, MCU, lockstep controller).
0x08	privilege_and_ring	uint64_t	privilege level	Proposer at EL0; enforcer at EL1 or above, or in the secure world.
0x10	nominal_period_ns	uint32_t	ns	Enforcer tick; equal to the enforcement record's <code>loop_period_ns</code> .
0x14	hard_deadline_ns	uint32_t	ns	The permission path's deadline inside the tick; equal to the enforcement record's <code>deadline_ns</code> .
0x18	unloaded_observed_max_ns	uint64_t	ns	Largest response observed without competing load; a sample maximum, not a WCET.
0x20	loaded_observed_max_ns	uint64_t	ns	Largest response under the declared load; all ones until a loaded test runs. Independence stands only while it is at most the declared WCET.
0x28	shared_axi_channel_qos	uint64_t	configured QoS	Arbitration priority per claimant; controller semantics verified by register capture and a stress test.
0x30	max_pdn_droop_thermal_trip	uint32_t[2]	mV; m°C	Configured rail-droop and thermal trip thresholds; the ledger records the tests behind them.

Offset	Field	Scalar type	Unit	Invariant and check
0x38	hardware_mitigation_flags	uint8_t[32]	bitfield	Active mitigations, such as a TCM snapshot, memory partitioning, or an isolated rail; register-locked after boot.
0x58	enforcement_record_digest	uint8_t[32]	SHA-256	Enforcement record whose loop_period_ns and deadline_ns this record tests.
0x78	test_ledger_digest	uint8_t[32]	SHA-256	External test ledger, including the model and firmware digests under test.

C.14.6 Authority record and operator payload

The authority record of section 14.8 has two runtime layouts. A network operator’s command is a proposal-header payload (table C.12), and it is the one proposal whose sender’s identity confers authority, so it carries an HMAC-SHA-256 tag under the operator role’s key. The static authority manifest fixes each role’s channel grant, the command’s component count, order, and units, and the key custody.

Table C.12: Operator payload: A network operator’s command under the proposal header. The tag authenticates the role; the permission path still admits the command by its content.

Offset	Field	Scalar type	Unit	Invariant and check
0x00–0x4F	proposal header	as above	as above	Table C.7, with payload_kind = OPERATOR.
0x50	nonce	uint64_t	—	Never repeated within a key’s lifetime; a repeated nonce is rejected as a replay.
0x58	requested_channel_mask	uint16_t	bitfield	Channels requested; must lie within the role’s grant in the static manifest.
0x5A	operator_role_id	uint8_t	enumeration	Role whose key produced mac.
0x5B	reserved	uint8_t	—	Zero.
0x5C	command	float32[N_c]	per manifest	Requested command components in manifest order, admitted like any other proposal.
0x5C + 4N _c	mac	uint8_t[32]	HMAC-SHA-256	Tag over all preceding bytes; an invalid tag locks out the channel.
0x7C + 4N _c	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes.

The forensic authority record (table C.13) is a log record written once per tick, not a packet. Its time is on the safety microcontroller’s clock; the mapping that relates it to PTP and calendar time, with its error bound, lives in the static manifest together with the vector lengths, units, and component order. Each record’s chain_link is an HMAC-SHA-256, under the log key, over the previous record’s link and this record’s preceding fields, so an altered or deleted record breaks every later link.

Table C.13: Forensic authority record: One record per tick of the permission loop. A missing record is a gap in the chain, never an implied pass, and the record is a sampled decision trace, not proof of delivered force or contact.

Offset	Field	Scalar type	Unit	Recorded evidence and limit
0x00	t_mcu_ns	uint64_t	ns, permission clock	Event time of this tick’s decision.
0x08	actuator_channel_mask	uint16_t	bitfield	Channels to which this decision applies; bus and DMA permissions enforce write scope.
0x0A	preempt_trigger_code	uint16_t	enumeration	Request, timeout, enabling-device loss, or protective input cause.
0x0C	authority_state	uint8_t	enumeration	AUTO, PEND, BLEND, MAN, FALLBACK, or PROTECTIVE.

Offset	Field	Scalar type	Unit	Recorded evidence and limit
0x0D	<code>fallback_rung</code>	<code>uint8_t</code>	enumeration	PROJECT, HOLD, STOP, or INHIBIT when the state is FALLBACK; NONE otherwise.
0x0E	<code>operator_role_id</code>	<code>uint8_t</code>	enumeration	Authenticated source role, interpreted against the static manifest.
0x0F	reserved	<code>uint8_t</code>	—	Zero.
0x10	<code>blend_factor_alpha</code>	<code>float32</code>	dimensionless	Progress in $[0, 1]$ of the enforcer-owned blending profile.
0x14	<code>policy_proposed_cmd</code>	<code>float32[N_c]</code>	per manifest	Command components before permission; not a motor write.
$0x14 + 4N_c$	<code>enforcer_clamped_cmd</code>	<code>float32[N_c]</code>	per manifest	Components admitted after the current constraint check.
$0x14 + 8N_c$	<code>actuator_delivered_cmd</code>	<code>float32[N_c]</code>	per manifest	Dispatched setpoint components; delivery at the drive needs separate evidence.
$0x14 + 12N_c$	<code>measured_plant_state</code>	<code>float32[N_s]</code>	per manifest	Sampled state components, with sensor timing in the manifest.
$0x14 + 12N_c + 4N_s$	<code>chain_link</code>	<code>uint8_t[32]</code>	HMAC-SHA-256	Keyed link over the previous link and this record's preceding fields; evidence of tampering only while the key and anchor stay in custody.

C.14.7 Authority-transfer protocol

Used in section 14.3 and section 14.4. The protocol runs in the enforcer's 1 kHz loop on the safety microcontroller, where the four-phase handshake, integrator pre-biasing, and the C^2 quintic blend become one sequence of per-state steps. Each tick writes the forensic authority record of table C.13 under the state names used below.

Cadence and invariants.

- *Execution cadence*: Hard real-time periodic timer interrupt ($f = 1$ kHz, period $T_{\text{loop}} = 1.0$ ms, hardware timing jitter $< 1.0 \mu\text{s}$).
- *Memory invariant*: Zero dynamic heap allocation (`malloc/free` forbidden; all state variables, mailboxes, and circular log buffers statically pre-allocated in DTCM SRAM).
- *Hardware authority*: Exclusive write of the admitted setpoints staged for the next EtherCAT frame to the drives, and of the safe torque off (STO) hardware cutoffs.

Execution protocol.

1. Sample and validate plant state, available stopping reserve, independent protective inputs, and the current authority lease. An independent protective input selects its validated, state-matched stop path of section 12.9.
2. In AUTO, a manual request or policy handover request enters PEND; the policy may propose only while its existing permission remains valid. Set T_{timeout} to the smaller of the protocol ceiling and clearance-derived deadline. No later than the last permission tick before T_{timeout} , transfer to the state-matched fallback. An invalid network packet is rejected and logged, not interpreted as a stop request.
3. In PEND, verify the operator credential, nonce, enabling input, target freshness, actuator limits, and full transfer-plus-stop clearance. On ACK, at the next qualified **Commit** tick, revoke the policy execution token on the masked channels, latch $\mathbf{u}_0$ as its last *admitted* command, latch a checked human target $\mathbf{u}_1$, and set BLEND. The enforcer remains the only actuator writer. If admission fails, retain or enter the independently validated fallback.
4. In BLEND, evaluate $s = \min(1, (t - t_0)/T)$ and $\alpha(s) = 6s^5 - 15s^4 + 10s^3$. The enforcer proposes $\mathbf{u}_{\text{blend}} = (1 - \alpha)\mathbf{u}_0 + \alpha\mathbf{u}_1$, then checks that candidate against current state, torque/rate, and stopping limits before dispatch. These are **latched endpoints**, not simultaneous live policy and human actuator streams. If the lease, clearance, or tracking assumption fails, use the state-matched fallback. At $s = 1$ with a live manual lease, set MAN and confirm that state to the operator.

5. In MAN, the enforcer checks each live human proposal. Lease expiry or enabling-device loss revokes manual proposals and transfers to the validated fallback. Log request, ACK, commit, candidate, admitted command, measured response, and clock uncertainty separately; the bounded high-rate record must not block the control tick.

C.14.8 Fault record

The fault record of section 15.7 is offline evidence, a fixed header signed with Ed25519 that links to the retained evidence bundle by digest (table C.14). The fault-ledger root that the release manifest binds is computed over the canonical encodings of these headers. `envelope_point` has a per-machine definition in the versioned fault-class manifest; for the mobile manipulator its four values are aisle speed (m/s), payload (kg), floor friction coefficient, and ambient temperature (K).

Table C.14: Fault-record header: The fixed part of one fault trial. The linked evidence bundle carries fault duration, hardware and software build identifiers, raw trace references and clock domains, detector identity, every margin value and scale, the measured physical outcome, secondary anomalies, and partial traces for non-pass verdicts.

Offset	Field	Scalar type	Unit	Purpose and interpretation
0x00	<code>trial_id</code>	<code>uint64_t</code>	—	Unique trial identifier in the versioned ledger.
0x08	<code>claim_ref_id</code>	<code>uint32_t</code>	index	Precommitted physical claim and its units.
0x0C	<code>envelope_point</code>	<code>float32[4]</code>	per manifest	Declared operating point of the trial.
0x1C	<code>fault_injection_mode</code>	<code>uint16_t</code>	enumeration	Injected fault class.
0x1E	<code>injection_target_node</code>	<code>uint16_t</code>	enumeration	Hardware boundary receiving the injection.
0x20	<code>fault_magnitude</code>	<code>float32</code>	per fault class	Injection amplitude; its unit is fixed by the fault-class manifest.
0x24	<code>t_det_ns</code>	<code>uint32_t</code>	ns	Fault trigger to detector assertion.
0x28	<code>t_takeover_ns</code>	<code>uint32_t</code>	ns	Fault trigger to measured takeover; subtract <code>t_det_ns</code> for the post-detection interval.
0x2C	<code>stl_robustness_rho</code>	<code>float32</code>	claim's unit	Raw signed margin for <code>claim_ref_id</code> ; other predicate margins remain in the bundle.
0x30	<code>verdict_status</code>	<code>uint8_t</code>	enumeration	PASS, FAIL, INVALID, or UNOBSERVABLE.
0x31	<code>reserved</code>	<code>uint8_t[3]</code>	—	Zero.
0x34	<code>oracle_digest</code>	<code>uint8_t[32]</code>	SHA-256	Immutable predicate and oracle configuration.
0x54	<code>evidence_digest</code>	<code>uint8_t[32]</code>	SHA-256	Separately retained trial evidence bundle.
0x74	<code>signature</code>	<code>uint8_t[64]</code>	Ed25519	Signature over all preceding bytes; the key identifier is in the versioned manifest.

For a trial aborted before injection, detector time, takeover time, and robustness margin are unavailable rather than zero. The layout stores all ones (0xFFFFFFFF) in an unavailable `uint32_t` time slot and a canonical quiet NaN in an unavailable `float32` margin slot, and the signed evidence bundle records a validity flag for each measured field and why it is absent. The oracle checks those flags and the verdict before doing arithmetic, and excludes unavailable slots from pass counts and from latency or margin statistics. The versioned manifest fixes the NaN bit pattern so that hashing remains reproducible.

C.14.9 Fault-injection protocol

Used in section 15.6. The protocol runs one hardware-in-the-loop trial on target silicon, from the precondition audit to the signed header of table C.14, and every exit it takes sets one value of that header's `verdict_status`.

Substrate and invariants.

- *Execution substrate:* Hard real-time field-programmable gate array (FPGA) plant rig and testbed orchestrator ($f = 1$ kHz, hardware timestamping jitter < 500 ns).

- *Memory invariant*: Pre-allocated static DMA buffers in the host test orchestrator; zero runtime dynamic heap allocation on the safety microcontroller.
- *Safety interlock authority*: Independent optical or laser truth sensors wired to hardware crowbar relays capable of cutting target actuator power within 2.0 ms of a physical envelope breach.

Execution protocol.

1. **Precondition audit (stage 1)**. Sample baseline environmental sensors via $\mathcal{J}_{\text{truth}}$ (temperature, rail voltage, and wheel speed). Compare each measurement with its declared same-unit tolerance; do not take one norm across kelvin, volts, and meters per second. If any precondition fails, log `ERR_PRECONDITION_OUT_OF_SPEC`, set the verdict to `INVALID`, and abort.
2. **Synchronization and steady-state arming (stage 2)**. Arm the FPGA plant simulator, power up the target two-processor system, and start the periodic control loop (1 kHz). Establish a synchronized reference clock across the logic analyzer, FPGA plant, and truth digitizers via a 10 MHz PXI clock. Allow the physical plant state $\mathbf{x}(t)$ to stabilize in the nominal operating envelope for $t_{\text{settle}} = 2.000$ s. If baseline tracking error $\|e_{\text{track}}\| > \epsilon_{\text{nom}}$, assert `ERR_BENCH-INSTABILITY`, set the verdict to `INVALID`, and abort.
3. **Fault injection trigger (stage 3)**. At monotonic test epoch t_{inject} , the hardware test orchestrator asserts the trigger line to the inline injection rig:
 - *Proposer stall (F1)*: The orchestrator kills the chunk-policy task one tick after a lease renewal.
 - *Sensor latency or bias (F2)*: An FPGA interceptor latches a navigation-camera frame or holds a stale encoder payload while preserving its valid hardware timestamp.
 - *Frame loss (F3)*: A fieldbus disturbance node drops or corrupts EtherCAT frames at a drive's slave port.
 - *Control-rail droop (F4a)*: A programmable electronic load draws the F4a transient of table 15.1 from the shared control rail at its battery-low point for $\Delta t_{\text{duration}} = 5.0$ ms, while an independent digitizer records the permission rail and the permission path's heartbeat.
 - *Permission-rail feed loss (F4b)*: A solid-state switch opens the permission rail's feed as the resident stop from the inspected-floor ceiling begins.
 - *Actuator mechanical stall*: A four-quadrant dynamometer applies step counter-torque $\tau_{\text{stall}} = 3.5 \times \tau_{\text{rated}}$.

Record the injection timestamp $t_{\text{fault_start}} \leftarrow t_{\text{now}}$ in nanoseconds on the reference clock.

4. **Enforcer response and takeover audit (stage 4)**. Monitor the target MCU's safety enforcer pins, CAN bus telemetry, and gate-driver PWM lines via an isolated logic analyzer:
 - Detect assertion of the non-maskable interrupt (NMI) or diagnostic flag: $t_{\text{det}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{det}} = t_{\text{det}} - t_{\text{fault_start}}$.
 - Detect hardware multiplexer preemption of the motor driver: $t_{\text{takeover}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{takeover}} = t_{\text{takeover}} - t_{\text{fault_start}}$.
 - Detect the physical plant's transition to its safe bounded regime: $t_{\text{bounded}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{bounded}} = t_{\text{bounded}} - t_{\text{fault_start}}$.

If the external containment rig trips on a structural envelope breach, the hardware crowbar cuts actuator power; assert `ERR_PHYSICAL_BREACH` and carry the trial to stage 5, where the violated containment predicate returns `FAIL`.

5. **Robustness evaluation and ledger serialization (stage 5)**. Ingest the truth trajectory $\mathbf{x}_{\text{obs}}(t)$ from independent instrumentation over the declared trial window. If required truth frames are missing, set the verdict to `UNOBSERVABLE` and retain the partial trace. For an observable trial, compute each precommitted predicate's signed margin in its own unit, as the STL robustness of section A.7 defines it. Return `PASS` only when every required margin is nonnegative and both measured fault-to-detection and fault-to-drive-takeover times satisfy their declared bounds; otherwise return `FAIL`. A dimensionless aggregate may be computed only after predeclared

per-predicate normalization. Link the fixed fault-record header to the variable-length evidence bundle by digest, sign the header under the specified custody scheme, and retain both.

C.14.10 Release manifest

The release manifest of section 16.8 is a signed offline artifact, not a mailbox record, so it has a canonical encoding in place of fixed offsets. Fields appear in the order that table C.15 lists, each variable-length field carries a length prefix, and the adjudicator’s Ed25519 signature covers the encoding of every preceding field. Every digest is SHA-256. The expiry is the one calendar time in the record chain, and it is checked against a tamper-resistant real-time clock rather than the safety microcontroller’s monotonic clock.

Table C.15: Release manifest: Canonical field order of the signed release artifact. A new signed version can supersede a manifest only after the current physical conditions are checked, and the hardware root anchors keys rather than storing mutable digests.

Field	Encoding	Binds	Check before permission
claim_manifest_id	uint8_t[16] UUID	Top-level safety claim	Linked to the root node of the signed argument graph.
argument_digest	uint8_t[32]	Complete claim–argument–evidence graph	Matches the graph the adjudicator reviewed.
policy_weights_-digest	uint8_t[32]	Learned model weights	Mismatch refuses release.
enforcer_-bitstream_digest	uint8_t[32]	Enforcer logic image	Mismatch refuses release.
rtos_kernel_digest	uint8_t[32]	Real-time kernel image and drivers	Verified by the hardware root of trust at power-on.
calib_trace_-pointers	length-prefixed list of signed evidence references	Calibration certificates	Missing or stale evidence refuses unless a separately signed restricted case covers it.
fault_ledger_root	uint8_t[32]	Signed fault-record headers (section C.14.8)	Missing, unauthenticated, or mismatched ledger refuses.
authority_record_-digest	uint8_t[32]	Static authority manifest and the forensic log’s initial chain anchor (section C.14.6)	Mismatch refuses release.
placement_record_-digest	uint8_t[32]	Placement record (section C.14.5)	Mismatch refuses release.
evaluation_record_-digest	uint8_t[32]	Evaluation record	Mismatch refuses release.
verdict_-classification	uint8_t enumeration	UNCOND, COND, or REFUSE	Selects the permitted operating envelope.
standing_conditions	length-prefixed array of (premise, check rate, trip)	Physical premises checked in operation	Checked on every permission tick; an unknown premise takes the assessed stop.
telemetry_drift_-budget	length-prefixed task-specific limits and triggers	Tracking and timing drift	A bound violation revokes; a trend prompts inspection.
verdict_expiry	UTC calendar time (ISO 8601)	Validity period of the verdict	A real-time clock reading past the expiry voids the permit.
lifecycle_budget	uint64_t[2] (hours, cycles)	Wear allowance	An exceeded budget or an invalid counter refuses motion.
adjudicator_-signature	key identifier plus uint8_t[64] Ed25519 signature	The whole manifest	Verifies under an authorized, unrevoked adjudicator key.

C.14.11 Release-gate protocol

Used in section 16.8. Before the motor power stages are energized, the release gate attests the silicon state, verifies the manifest of table C.15 and its image digests, and audits the standing conditions, in the order below.

Substrate and signature. The hardware root of trust anchors an authorized, revocable adjudicator verification-key chain. Protected storage holds the signed manifest and signed evidence graph. The signature covers every versioned manifest field, including the identity of the signing key, and the verifier checks the key chain and its revocation state before trusting the signature. The individually signed fault records of section C.14.8 enter the manifest as one digest over the complete ledger in its declared order. The verifier checks each record signature, the ordering, completeness, and that root against the reviewed evidence index. A digest alone establishes neither record validity nor test adequacy.

Fail-closed authorization. Begin with `Permit_gate = 0` and the physical plant in its assessed nonoperating state. All returned decisions use the signed enum `UNCOND`, `COND`, or `REFUSE`; an implementation may log a reason code separately.

1. Verify the adjudicator key chain and revocation status, the manifest signature, expiry, signed operating-hour and cycle budgets, and every active image digest. Any invalid, missing, or expired item returns `REFUSE`. A different software bank requires its own signed case and a new check of physical conditions.
2. Authenticate the complete fault ledger and calibration references against the signed graph. A missing ledger, bad record signature, or root mismatch returns `REFUSE`, even if the manifest says `UNCOND`. Stale calibration also returns `REFUSE` under the full case. It can support `COND` only when a *separately signed* restricted case explicitly covers that calibration state and all of its evidence and limits still validate.
3. Check the state-dependent standing conditions, including brake test, separation, timebase, and monitoring health. `KNOWN_FALSE` and `UNKNOWN` both return `REFUSE` while retaining distinct logged causes. Apply the approved state-matched stop or hold of section 12.9.
4. If every check passes and the signed verdict is `UNCOND`, load the signed full envelope and return `UNCOND`. If every restricted condition passes under a signed `COND` case, load its signed reduced envelope and return `COND`. A signed `REFUSE` case or any other state returns `REFUSE`. In both operating modes the learned policy only *proposes* actions; the independent enforcer checks each candidate and controls actuator permission. Force trip thresholds must include measured detection and response headroom below the contact criterion.

C.14.12 Residual-claims register

The residual-claims register of section 17.3 tracks every physical premise the release rests on but does not observe, so that an open claim is carried across software deployments and hardware revisions. Like the release manifest, it is an offline artifact whose mandatory fields (table C.16) follow a canonical order rather than fixed offsets.

Table C.16: Epistemic residual-claims register: Canonical field order of the register. The header binds it to one platform, build, and operating window; each entry must carry its claim, the missing observable, the hazard and its time to harm, the containment that holds meanwhile, and the predeclared test that would close it.

Manifest Category	Field	Physical Unit	Invariant & Operational Contract
Manifest Header	system_identifier	—	Unique embodied platform ID (e.g., WMM-AISLE-01)
Manifest Header	hardware_build_rev	—	Target silicon and chassis revision (HW-REV-3.2)
Manifest Header	software_digest	—	Digest of the released binary
Manifest Header	validity_limit_hours	hours	Configured operating window before review
Residual Claims	entry_id	—	Unique claim tag (e.g., CLAIM-FRIC-001)
Residual Claims	target_safety_claim	—	Explicit safety invariant ($d_{\text{stop}} \leq D_{\text{clear}}$)
Residual Claims	missing_observable	SI units	Unmeasured physical state (e.g., surface friction μ)
Physical Hazard	governing_law	—	Newton-Euler, Navier-Stokes, Joule heating
Physical Hazard	time_to_harm	seconds	Time from unmodeled transition to irreversible yield
Operational Containment	containment_type	—	FIRMWARE_PARAM_CLAMP, PASSIVE_MECHANICAL, or OPERATING_RESTRICTION; validate the selected limit
Operational Containment	authority_limit	m/s,N	Hard numeric setpoint ceiling ($v_{\text{max}}, F_{\text{max}}$)
Operational Containment	kinetic_energy_ceiling	J	Maximum allowable kinetic energy under fault ($E_k \leq E_{\text{max}}$)
Assurance & Inspection	proof_test_interval	hours	Scheduled test interval; evidence between tests still depends on the operating restriction
Predeclared Closure	closure_modality	—	Transducer technology or formal method required
Predeclared Closure	sample_trials	count	Case-specific independent exposure, sized by the zero-failure rule of section A.8 for the declared miss-rate bound
Predeclared Closure	confidence_level	—	Declared one-sided confidence level of that bound

C.15 Further Reading

For a deeper foundational understanding of embedded architectures, real-time operating systems, and power electronics, consult the following authoritative references:

- **Computer Organization and Design: The Hardware/Software Interface** (Hennessy and Patterson 2019) provides the definitive treatment of processor architectures, cache hierarchies, and memory subsystem trade-offs.
- **Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment** (Liu and Layland 1973) establishes the mathematical foundations of rate-monotonic scheduling and real-time task schedulability.
- **Modern Operating Systems** by Andrew S. Tanenbaum and Herbert Bos provides comprehensive coverage of inter-process communication, lock-free synchronization, and memory management.
- **Real-Time Systems** by Jane W. S. Liu delivers the authoritative mathematical formulation of real-time scheduling algorithms, priority inversion, and response time analysis.

S·P·A Diagnostic Playbook

When the base of the warehouse mobile manipulator stops past its rack-end clearance, the cause could be stale sensor evidence (Sense), an infeasible waypoint (Plan), or a worn brake that delivers less deceleration than assumed (Act). The **S·P·A taxonomy** locates the first broken contract among **Sense**, **Plan**, and **Act**. Logs and physical measurements then test that diagnosis before a remedy is admitted.

How to Use This Appendix

Use this diagnostic playbook when an embodied system violates a safety envelope, misses an execution deadline, chatters during contact, or trips hardware interlocks. Work through the diagnosis in four systematic passes:

1. **State the physical symptom and binding budget:** Record the observed stopping-distance breach, contact-force transient, thermal trip, or enforcer deadline overrun, including units, load, operating state, and the limit configured for that machine.
2. **Localize the candidate axis or intersection:** Use table D.1 for a first pass across the three core axes. When the symptom involves interface timing, state representations, or actuation authority, move directly to the intersection landscape in table D.2.
3. **Test the quantitative hypothesis:** Measure the candidate constraint or budget using the instruments in table D.3, such as timestamp skew, bus queue time, phase current, or tracking error.
4. **Intervene and remeasure across the causal boundary:** Apply a remedy and rerun the measured test over the declared load, timing, and fault envelope.

The physical symptom determines the most effective entry point. Unstable contact oscillations should immediately examine the Act–Sense ($A \cap S$) boundary. A sudden reference discontinuity or measured torque transient points to the Plan–Act ($P \cap A$) boundary. Phantom obstacle braking or drift under occlusion points to the Sense–Plan ($S \cap P$) interface. Deadline jitter during concurrent neural inference points directly to the permission decision at the center ($S \cap P \cap A$).

D.1 Diagnostic Summary: The S·P·A Axes

THE S·P·A taxonomy decomposes the cyber-physical loop into three axes. Table D.1 lists a constraint or budget to inspect for each one; these are not all conservation laws.

Table D.1: S-P-A axis reference: The axes organize diagnosis around measured constraints and budgets, with the chapter that develops each contract.

Axis	Operational role	Constraint or budget to test	Diagnostic action	Book locus
Sense (S)	Capture and estimate physical state	Evidence age, field of view, clock conversion, uncertainty, and sensor transport	Compare capture and per-cell evidence epochs with a measured state and the permission deadline	chapter 9
Plan (P)	Propose a target and trajectory	Intent validity, compute latency, dynamics, and stopping-suffix admission	Compare proposed targets and future states with independently checked limits	chapter 11
Act (A)	Apply and measure force	Current, torque, thermal headroom, brake response, and kinetic clearance	Measure command-to-force response and achieved stop under the declared load	chapter 12

While this single-axis separation is helpful for initial triage, production failures rarely originate within a single isolated component. More frequently, the failure sits at the **boundary** between two axes—such as an unblended trajectory step that excites high mechanical rotor inertia, or camera direct memory access (DMA) ingress that saturates the memory crossbar needed by real-time motor timers. Resolving these failures requires mapping the intersection landscape.

D.2 The Intersection Landscape

Figure D.1 shows how Sense, Plan, and Act intersect, with the permission decision at the center where all three meet. In this appendix, we formalize this landscape into an operational systems diagnostic matrix.

Each intersection is a testable handoff. Table D.2 names the failure mode and the evidence needed before a proposed fix earns permission.

Table D.2: The S-P-A intersection taxonomy: Four handoffs locate the earliest broken physical contract and the evidence required to repair it; the axis reference above links to the underlying chapters.

Intersection	Handoff contract	Failure to test	Evidence and candidate remedy
$S \cap P$ — belief	Observations become timed, framed state estimates	A fresh map publication hides an old voxel or an unseen object remains unknown	Check per-cell evidence epochs, free versus occupied rays, and frame transforms; use a separately validated collision layer
$P \cap A$ — motion	A proposal becomes an admitted, state-matched trajectory	A seam requests a torque transient or the stop suffix no longer fits clearance	Check entry state, derivatives, lease, and clearance; reject an infeasible seam and use the resident stop
$A \cap S$ — feedback	Actuation changes the state that sensors report	Contact force rises before the next useful sample or an actuator saturates	Measure command-to-force and sensor-return delay; choose a local reflex and passive limit where justified
$S \cap P \cap A$ — permission	The independent gate turns a proposal into physical action	Compute, memory, power, or clock interference delays refusal	Measure response under contention; revoke tracking and qualify isolation or relocate the path if its bound fails

D.3 Physical AI Production Troubleshooting Matrix

When an operational machine violates physical limits or software invariants, use table D.3 to match observable diagnostic symptoms to the binding axis and high-leverage remedy.

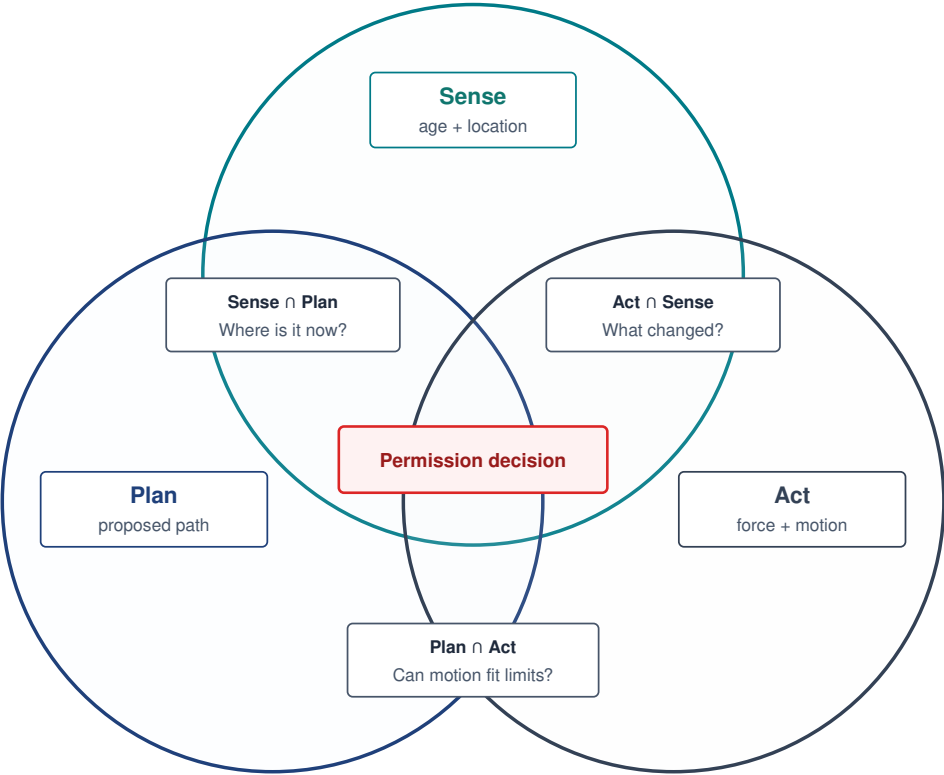


Figure D.1: The S-P-A intersection landscape: Sense, Plan, and Act meet pairwise in three physical decisions, locating the object now, checking that proposed motion fits limits, and measuring what changed after acting. Their center is the permission decision, where an independent permission path checks the shared physical budget before actuation.

Table D.3: Physical AI production troubleshooting matrix: Each row is a hypothesis, measurement, and candidate remedy; acceptance depends on the machine’s declared limits.

Observable symptom	Candidate boundary	Discriminating measurement	Candidate response
Contact hunting while the arm opens the spring-latched cage door	$A \cap S$	Measure phase margin and force rise against the sensor-to-torque delay under the actual contact stiffness	Qualify a local impedance loop, approach-speed limit, and passive compliance for that contact
Torque or current transient at an action-chunk seam	$P \cap A$	Compare encoder, phase-current, and requested acceleration traces at the measured entry state	Admit a state-matched C^2 bridge and reserve a stopping suffix; the reduction in chapter 11 is profile-specific
Phantom obstacle stop in an aisle	$S \cap P$	Distinguish observed free, occupied, and unknown rays; inspect per-voxel evidence age	Clear only observed free space and retest the occupancy-to-permission conversion
Enforcer misses its configured deadline during camera ingress	$S \cap P \cap A$	Correlate response-time misses with bus queue, bank switches, cache misses, and DMA ownership	Qualify QoS bypass or a coherent local safety snapshot; remeasure under contention
Link or workpiece yields during contact	A	Compare measured force and displacement with the tested material and tooling limits	Reduce admitted speed or force and qualify a compliant load path
Policy update arrives after its admitted chunk horizon	$P \cap A$	Measure the update tail, lease expiry, and fallback transfer state	Shorten the admitted action or use a stop the permission path owns; do not extend permission from a percentile alone

Observable symptom	Candidate boundary	Discriminating measurement	Candidate response
Safety microcontroller (MCU) rail droops during accelerator bursts	$S \cap P \cap A$	Measure rail waveform, path slack, supervisor threshold, and resulting response time	Characterize decoupling, current slew, or physical power isolation
Base drifts toward a rack face despite low offline prediction error	$A \cap S$	Compare closed-loop yaw bias, lateral clearance, and surface-dependent braking	Test closed-loop rollouts and impose a feasible corridor and stop bound

D.4 The S·P·A Systems Scorecard

Before release, use table D.4 to name the evidence needed for each handoff. Its checks are examples, not portable pass thresholds: the signed release case must supply the actual limits, operating envelope, confidence or hard-bound semantics, and fallback for the machine.

Table D.4: The S·P·A physical AI systems scorecard: Example release checks across the three axes and their handoffs. Each row requires a case-specific threshold and evidence.

Boundary	Example acceptance check for a declared machine	Response if the check fails
Sense — capture clock	Bound capture-to-MCU conversion error and include it in evidence age	Resynchronize or withhold motion when the bound expires
Sense — ingestion	Measure worst admitted sensor/DMA queue occupancy, drop rate, and transfer time under concurrent load	Reserve capacity or reduce sensing load; dropped frames do not renew belief
Plan — chunk horizon	Admit a successor before the commit deadline, or transfer to the resident or state-matched stop	Use measured update tails as evidence, not as a worst-case guarantee
Plan — dynamics	Check position, velocity, acceleration, torque, and stopping suffix against the actual joint and path limits	Reject or replan an infeasible chunk
Act — brake and contact	Validate achieved deceleration and force response for load, wear, surface, and temperature	Derate speed, change tooling, or refuse the untested state
Act — thermal	Keep measured winding and drive temperature inside device ratings under the duty cycle	Reduce duty or enter the validated stop/hold mode
$S \cap P$ — spatial evidence	Preserve each cell's evidence epoch and distinguish observed free, occupied, and unknown	Refuse unknown clearance or obtain a new observation
$P \cap A$ — seam	Check the matched entry state, derivative limits, and full stopping clearance before commit	Use the resident or state-matched stop the permission path holds if a successor is absent or invalid
$A \cap S$ — reflex	Bound sensing-to-force response against a contact-specific harm deadline	Reduce approach energy or add a validated passive limit
$S \cap P \cap A$ — compute	Qualify the permission path's WCET against its deadline in the cycle budget of chapter 12, with acquisition, validation, write, and dispatch budgeted separately	Revoke tracking permission before the cycle overrun consumes stopping margin
$S \cap P \cap A$ — shared silicon	Measure enforcer response under worst admitted bus, rail, and thermal contention	Qualify QoS/local snapshots or relocate the path if isolation cannot be shown
$S \cap P \cap A$ — permission	Verify model/state assumptions, feasible CBF input, clock and estimate margins, and a state-matched fallback	Refuse the proposal when a premise is missing; a QP result alone proves no path safety

Glossary

This glossary defines key terms used throughout *Physical AI*. Terms are organized alphabetically.

A

Action Chunking A policy execution technique where a model predicts an open-loop sequence of future actions over a multi-step horizon rather than a single action per step. Amortizes the latency of slow neural inference across multiple control intervals and dampens high-frequency compounding covariate shift by generating kinematically coordinated trajectories. In closed-loop deployment, overlapping chunks are executed in a receding-horizon fashion or blended via temporal ensembling to maintain responsiveness to dynamic environmental disturbances. (Chapter 3, chapter 11, chapter 6)

B

Behavioral Cloning (BC) The foundational imitation learning paradigm that trains a policy via supervised learning to directly map sensory observations to expert control actions, treating physical sequential decision-making as a direct regression or classification task. While computationally efficient on static demonstration archives, standard behavioral cloning ignores the closed-loop consequences of its own control outputs. As a result, small single-step inference errors induce compounding covariate shift, driving the physical robot away from the demonstration manifold into unrecoverable failure states. (Chapter 6, chapter 5)

Belief State Invalidation Horizon (t_{exp}) The elapsed temporal duration beyond which an unobserved spatial or kinematic state estimate can no longer be certified to lie within a task's safe tolerance envelope. Evaluated from environmental disturbance dynamics, unmodeled obstacle velocities, and estimator drift, it establishes the expiration shelf life of perceptual memory. Downstream planners that treat spatial beliefs older than this horizon as valid free space risk commanding collisions with dynamic obstacles. (Chapter 9, section B.6)

Bumpless Transfer A dynamic control handoff protocol that matches state, velocity, and effort references across control authorities at the instant of switching, eliminating transient step changes in commanded actuator torques. When supervisory control transitions abruptly between an autonomous neural policy and a human operator, raw authority switching induces mechanical shocks, structural oscillations, and loss of tire traction. Bumpless transfer employs dynamic transition ramps and feedback state tracking to smoothly transfer control energy within the actuator's physical bandwidth. (Chapter 14, section B.3)

C

Causal Boundary The physical, energetic, and architectural threshold in an embodied system where reversible digital computations terminate and irreversible physical work is performed on the external environment ($dW = \mathbf{F} \cdot d\mathbf{x}$). Beyond this boundary, computational errors cannot be rolled back by software exception handlers or abort routines, requiring that safety invariants be verified before actuator register writes occur. The permission path sits above this boundary, at the proposal boundary of section 1.5. (Chapter 1, chapter 2)

Causal Confusion (Causal Misidentification) An imitation learning failure mode where a policy conditions on spurious, non-causal features that correlate with expert actions in training demonstrations rather than the true physical affordances of the task. High-capacity neural models exploit incidental visual cues (such as interface indicators, dashboard lights, or past actions) that disappear or change under physical deployment. Deployed closed-loop policies fail catastrophically when test-time interventions break these non-causal correlations. (Chapter 6)

Clearance Inset Margin A deterministic geometric safety buffer subtracted from perceived free-space boundaries to guarantee collision avoidance in the presence of spatial measurement uncertainty. The inset is formally computed from the vector sum of sensor calibration error, depth covariance, transform chain kinematic tolerances, and dynamic tracking error envelopes. Enforcing a clearance inset prevents a planner from routing trajectories along the boundary of raw sensor detections, ensuring physical clearance under worst-case perception drift. (Chapter 8, chapter 11)

Closed-Loop Evaluation The metrological paradigm in physical AI that measures a policy's performance, safety, and stability through interactive rollouts on physical hardware or high-fidelity simulation, rather than

through static loss metrics on a held-out dataset. Because an autonomous policy continuously perturbs its own operating environment, meaningful evaluation requires logging closed-loop metrics such as task completion rates, dynamic safety margin violations, and mean operational time between interventions. Evaluating policies purely via open-loop validation error obscures catastrophic compounding covariate shift. (Chapter 7, chapter 6)

Compounding Rollout Error The theoretical and empirical accumulation of trajectory deviations in open-loop imitation learning, where a per-step policy error rate of ϵ induces a total trajectory error that compounds quadratically ($\mathcal{O}(\epsilon T^2)$) over execution horizon T . Because the agent encounters unseen states as soon as it deviates from the demonstration path, its conditional error probability spikes rather than remaining constant. In physical robotics, this quadratic growth transforms minor tracking drift into rapid collisions with obstacles or mechanical joint-limit saturation. (Chapter 6, section A.5)

Control Barrier Function (CBF) A continuously differentiable scalar certificate $h(\mathbf{x}) \geq 0$ used to define and guarantee the forward invariance of an admissible safe set in the state space of a dynamical system. By enforcing the boundary rate-of-approach condition $\dot{h}(\mathbf{x}, \mathbf{u}) \geq -\alpha(h(\mathbf{x}))$, a CBF translates complex state constraints into instantaneous affine inequality constraints on control inputs. This allows candidate proposals from unverified neural policies to be projected onto safe control inputs in real time via convex quadratic programming. (Chapter 12, section B.2)

Cross-Embodiment Transfer A learning paradigm in which a single policy or foundation model is trained on sensorimotor data gathered across heterogeneous robot morphologies, kinematic chains, and actuator dynamics to generalize to novel target embodiments. Achieving cross-embodiment generalization requires normalizing disparate observation spaces into unified coordinate frames and mapping heterogeneous action spaces through embodiment-agnostic representations. Its primary challenge is preventing negative transfer induced by unmodeled dynamic discrepancies, link inertia variations, and differing joint velocity boundaries. (Chapter 5, chapter 6)

D

Diffusion Policy A generative policy architecture that models continuous, multi-modal robot action trajectories by inverting a parameterized stochastic diffusion process conditioned on sensory observations. By learning the score function of the demonstration distribution, diffusion policies generate smooth, multi-step action chunks from Gaussian noise without collapsing into mode averaging or requiring explicit discretization of continuous control spaces. Their expressive density modeling makes them exceptionally effective for complex contact-rich manipulation tasks. (Chapter 6, section A.5.2)

Domain Randomization (DR) A sim-to-real transfer methodology that systematically perturbs physical parameters (mass, friction, damping, motor torque limits), sensor characteristics (noise covariance, latency, camera pose), and visual textures across simulated training episodes. By exposing the policy to a broad distribution of simulated environments, domain randomization encourages the neural network to learn invariant representations and robust feedback behaviors that treat the unmodeled physical world as another randomized instance within its training support. If randomized too narrowly, the real world falls outside the training hull; if randomized too broadly, policy performance degrades due to excessive conservatism. (Chapter 6)

Dynamic Stopping Distance (d_{stop}) The clearance a moving embodied system needs to reach standstill from its current speed under credible braking and the full pre-brake delay: $d_{\text{stop}} = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}}$. It comprises the distance traveled during the delay from detection to retarding force, which grows linearly with that delay, the braking distance under a validated deceleration floor, which grows with the square of speed, and fixed allowances for localization error and protective clearance. The permission path admits a command only if this distance, inset by the tracking bound, fits within the clear distance to the boundary. (Chapter 2, chapter 12)

E

Endogenous Covariate Shift A distribution shift in sequential decision-making where an embodied agent's own execution errors drive its physical body into states unrepresented in its training demonstrations, thereby presenting the policy with novel out-of-distribution observations. Because offline supervised training assumes independent and identically distributed data, policy errors under endogenous shift compound along the trajectory rather than averaging to zero. In physical systems, this shift leads to task failure or boundary violations unless mitigated by closed-loop corrective data or receding-horizon re-planning. (Chapter 1, chapter 5, chapter 6)

Endogenous Experience (The Action–Observation Loop) The closed-loop data generation process unique to embodied systems wherein an agent's executed actions causally determine its subsequent physical states and sensory observations ($s_{t+1} \sim P(s | s_t, a_t)$). Unlike passive supervised learning on fixed, exogenous datasets, physical AI experience generation forms an irreversible dynamical feedback loop where minor control perturbations alter future data collection trajectories. Consequently, data generation and policy deployment are fundamentally inseparable in embodied agents. (Chapter 1, chapter 5)

Envelope Trichotomy A three-valued epistemic classification of cyber-physical runtime health that partitions system status relative to its certified operating envelope into *known-true* (verified valid telemetry inside

bounds), *known-false* (verified limit breach), and *unknown* (epistemic uncertainty from sensor corruption, frame drops, or estimator divergence). In embodied AI, both *known-false* and *unknown* strictly withhold tracking permission and trigger safe fallbacks, preventing unmeasured kinetic energy from accumulating during perceptual blind spots. This departs fundamentally from disembodied software, which often presumes execution safety in the absence of explicit error flags. (Chapter 16)

Exposure Wall The limit on what failure-free operation alone can establish about a rare failure rate. Under a stationary Poisson failure model, bounding a catastrophic rate $\lambda = 10^{-9}$ per hour at 95 percent confidence requires about $-\ln(0.05)/\lambda \approx 3.0 \times 10^9$ failure-free hours of exposure from the same configuration, an argument Butler and Finelli made for flight software. A zero-event test cannot reach such a rate, so claims at that level rest on architectural evidence, fault injection, and runtime monitors rather than on accumulated operating hours alone. (Section 7.5, chapter 15, section A.8)

F

Fallback Ladder The ordered set of responses the permission path selects when a proposal cannot be admitted unchanged or its premises fail. Its rungs are PROJECT (minimal-intervention projection of the proposal onto the safe set), HOLD (a state-matched powered hold), STOP (a controlled, state-matched stop), and INHIBIT (drive torque inhibit with a separately rated brake). Each rung is selected before the admissible set becomes empty, and each is a validated response for the declared load and timing, not a generic abort. (Section 12.9, chapter 14)

Flow Matching Policy A continuous generative policy architecture that trains a neural vector field to transform a base probability distribution into an empirical action distribution along deterministic, straight-line optimal transport paths. Compared to score-based diffusion policies, flow matching models achieve comparable distribution expressiveness while requiring significantly fewer numerical integration steps to generate an action chunk. This inference speedup substantially reduces on-device generation latency, making real-time high-frequency closed-loop execution viable on embedded edge computing silicon. (Chapter 6)

Forward Invariance A mathematical property of a dynamical system and safe set $\mathcal{C}$ asserting that any state trajectory initialized within $\mathcal{C}$ remains strictly confined within $\mathcal{C}$ for all future time under an admissible control policy. In physical AI, forward invariance serves as the foundational mathematical formulation of safety, guaranteeing that an embodied agent never breaches kinematic, dynamic, or thermal hazard boundaries during closed-loop operation. It is certified at runtime by verifying that control inputs satisfy subtangential boundary conditions at the perimeter of the safe set. (Chapter 12, section B.2)

G

Grounded Affordance The spatial, kinematic, and dynamic interaction manifold associated with an object or terrain region that defines where and how an embodied agent can initiate, sustain, and complete a physical manipulation or locomotion primitive. Unlike purely semantic object labels, a grounded affordance parameterizes contact surface normals, friction cone bounds, end-effector approach corridors, and kinematically feasible grasp poses relative to the robot's physical embodiment. A semantic task request cannot be translated into physical action until its underlying affordance is rigorously grounded within the robot's reachable workspace. (Chapter 10, chapter 8)

H

Heterogeneous Silicon Partitioning A hardware-enforced allocation strategy that physically isolates stochastic, non-deterministic machine learning workloads (running on GPUs or NPUs under general-purpose operating systems) from deterministic, safety-critical reflex loops (running on bare-metal or RTOS cores). By enforcing hardware memory protection boundaries and dedicated peripheral access, it eliminates common-mode OS panics and memory contention between neural pipelines and microsecond motor control loops. This ensures that the permission path retains guaranteed execution budgets and unshared bus access under all computational loads. (Chapter 13, section C.8)

I

Inevitable Collision State (ICS) A physical system state $\mathbf{x}(t)$ from which no admissible, physically executable control trajectory exists—within dynamic actuator and friction limits—that can prevent impact with environmental obstacles over an infinite time horizon. Unlike geometric obstacle clearance, an ICS explicitly accounts for plant momentum, friction limits, dynamic obstacle trajectories, and finite actuator braking authority. A physical AI agent enters an ICS long before geometric contact occurs if its velocity exceeds its stopping authority within available clearance. (Chapter 2, chapter 11, chapter 12)

Intent Grounding Refusal A deterministic safety rejection mechanism whereby an intent translation engine or admittance gate formally refuses to execute a high-level command because the requested goal cannot be physically or safely grounded. Refusal is triggered when semantic targets lack reachable affordance manifolds, demand trajectories that violate kinodynamic feasibility, or conflict with verified safety invariants. Rather than

silently attempting an under-actuated or hazardous motion, the system emits an explicit refusal diagnostic, signaling supervisory intervention or deliberative replanning. (Chapter 10, chapter 14)

Intent Lease A time-bounded and state-bounded authorization token under which a high-level cognitive goal or target affordance remains valid for physical execution by downstream planners. Because dynamic environments continuously evolve, an unfulfilled intent becomes physically obsolete or dangerous if delayed beyond a certified temporal lease or if the underlying scene evidence drifts beyond specified spatial tolerances. Expiration of the intent lease forces an immediate deliberative renegotiation or a safe transition to a standstill, preventing the robot from pursuing stale perceptual premises. (Chapter 10, chapter 11)

Intervention Masking An empirical evaluation and safety pathology where repeated, proactive interventions by human supervisors systematically distort autonomy metrics by intercepting the policy before it encounters critical boundary failures. Because the supervisor intervenes during nascent drift, the policy's recorded telemetry logs contain artificially elevated success rates and zero observed accidents, while hiding the true underlying failure rate and preventing the collection of recovery data. Unless explicitly accounted for via intervention-discounted metrics, intervention masking creates a false perception of policy readiness that leads to catastrophic failures upon unsupervised deployment. (Chapter 14, chapter 7)

Intervention Rate (Mean Time Between Interventions) The standardized reliability and safety metric in physical AI defined as the total number of supervisory human takeovers or emergency safety trips divided by the cumulative operating time or operational distance across physical rollouts. Unlike binary task success rates, MTBI provides an un-censored, statistically rigorous measure of an embodied agent's autonomous survival capacity in open-world environments. Tracking MTBI exposes whether a high task completion rate is an illusion created by frequent human safety interventions that prevent the policy from encountering its failure boundaries. (Chapter 7, chapter 14)

K

Kinodynamic Feasibility The mathematical property of a planned physical trajectory that simultaneously satisfies geometric configuration-space collision avoidance constraints, kinematic mechanism limits (joint positions and velocities), and dynamic plant boundaries (actuator torques, joint accelerations, reflected jerk, and contact friction cones). A purely kinematic trajectory that ignores actuator torque-speed curves or vehicle tire adhesion limits is physically unexecutable and will induce actuator saturation, phase lag, and control runaway. Trajectory synthesis in physical AI requires verifying kinodynamic feasibility across the entire motion horizon prior to execution. (Chapter 11, chapter 2)

M

Metric 3D Lifting (Spatial Grounding) The geometric and learning process of projecting uncalibrated 2D image-space features, masks, or semantic tokens into metric, Euclidean three-dimensional representations anchored to the physical robot's coordinate frame. Methods such as Ray Frustum Voxel Lifting, Truncated Signed Distance Function fusion, and 3D Gaussian Splatting convert scale-ambiguous pixel patterns into metric obstacle boundaries and free-space occupancy grids. Spatial grounding ensures that high-level neural semantic claims correspond to physically reachable, collision-checkable coordinate manifolds in the workspace. (Chapter 8, chapter 10)

Minimal Risk Maneuver (MRM) An autonomous, deterministic fail-safe trajectory executed upon critical fault detection or loss of operational envelope to transition an embodied system from an active mission state into a stationary, low-energy safe state. Unlike an abrupt unpowered stop, which can induce dynamic tip-over, loss of steering control, or payload release, an MRM actively executes a controlled deceleration, lane clearance, or compliant ground settling. It represents the terminal behavioral guarantee of the permission path when nominal tracking authority is permanently revoked. (Chapter 12, chapter 14)

Minimal-Intervention Safety Shield A real-time supervisory projection filter that mediates between an unverified learned policy and physical actuator drives by solving a quadratic program on every control cycle: $\mathbf{u}^* = \arg \min_{\mathbf{u}} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2$ subject to barrier and actuator constraints. It passes nominal neural network proposals unaltered whenever they respect safe dynamics, intervening only at the boundary of the safe set to minimally redirect control commands along the tangent cone of forward invariance. This guarantees physical safety while preserving maximum task performance and behavioral expressiveness of the underlying model. (Chapter 12, section B.2)

Mode Averaging (Multimodality Catastrophe) A catastrophic learning failure that occurs when a unimodal policy parameterized with a standard regression loss (such as mean squared error) is trained on multi-modal demonstration data exhibiting multiple valid pathways. When human demonstrators navigate around an obstacle by choosing either a left or right detour, a naive regression model averages the competing action modes, commanding an interpolation that steers the robot directly into the obstacle. Resolving mode averaging requires expressive generative decoders capable of modeling arbitrary multi-modal distributions over continuous action trajectories. (Chapter 6, chapter 3)

Mode Confusion A safety-critical cognitive misalignment between a human operator and an automated system regarding which entity possesses active control authority over specific physical degrees of freedom. Mode confusion arises when autonomous overrides engage or disengage silently without salient multisensory feedback, leading the human to believe they are steering while the autonomous policy is resisting, or vice

versa. In safety-critical robotics, mode confusion induces destructive control conflicts, prolonged takeover reaction latencies, and accidental disengagement of critical safety shields. (Chapter 14)

Monitored Premise Expiration The automated runtime revocation of safety case validity triggered when empirical operational conditions (such as friction coefficients, ambient temperature, sensor packet loss, or battery internal impedance) violate the core design premises under which the system was qualified. By linking runtime telemetry monitors directly to safety case warrants, it prevents silent degradation of physical margins from exposing the machine to unshielded hazards. When a monitored premise expires, the system transitions autonomously into a restricted or refusal operational regime. (Chapter 16)

O

Observation Contract A formal, deterministic specification establishing the precise spatial, temporal, and semantic guarantees that a perception subsystem commits to providing downstream planning and safety modules. The contract defines coordinate frame conventions, maximum permissible spatial information age, minimum depth resolution, and bounding covariance ellipsoids for detected obstacles. If dynamic occlusions, sensor dropouts, or computational bus contention cause perception output to violate these declared bounds, the contract triggers an immediate perceptual failure flag rather than transmitting stale or unverified spatial claims. (Chapter 8, chapter 3)

Observational Indistinguishability A fundamental perceptual limitation wherein two distinct physical world states—one nominal and one catastrophic—produce identical sensor telemetry up to the machine’s physical intervention deadline ($t_{\text{deadline}} = t_{\text{harm}} - t_{\text{act}}$). When sensor physics provides zero distinguishing energy before the time-to-harm expires, no software algorithm, neural capacity scaling, or anomaly detector can prevent failure. Mitigating observational indistinguishability demands conservative mechanical bounds, passive physical compliance, or scheduled non-destructive inspection rather than deeper neural networks. (Chapter 17)

Operational Design Domain (ODD) The object poses, surface conditions, payloads, speeds, and disturbances in which a machine is meant to operate, written down as the scenario ledger. A dataset supports the part of the ODD that its occupancy join covers; the rest is its known absence. The sets nest: the policy manifest claims a declared ODD within the ODD, evaluation samples a target ODD within the declared ODD, and verification extends the target ODD to the machine’s internal conditions as the operating envelope that a release verdict authorizes in whole, in part, or not at all. (Section 5.5, chapter 6, chapter 7, chapter 15, chapter 16)

P

Physical Artificial Intelligence (Physical AI) An interdisciplinary branch of artificial intelligence concerned with computational systems whose sensory inputs, inferential deliberations, and learning algorithms are intrinsically coupled to physical bodies operating within continuous, non-stationary physical environments. Unlike disembodied AI systems that operate on static digital tokens, a physical AI system must continuously reconcile neural inference latency with real-world dynamical time scales, irreversible thermodynamic contacts, and hardware safety constraints. Its central objective is the synthesis of closed-loop policies that achieve generalizable task execution while strictly preventing catastrophic physical harm. (Chapter 1, chapter 3)

Proposal Boundary The line between a learned model, which may only propose an action, and the permission path, an independent mechanism with bounded latency that the model cannot delay or corrupt and that alone may let the action reach an actuator. The permission path checks each proposal against fresh evidence, the stopping envelope, and a fallback feasible from the current state, and it refuses when any check fails. The proposal boundary separates the Brain from the Nervous System in the book’s five-level machine. (Section 1.5, chapter 4, chapter 12)

Q

Qualification Ladder (XIL) A structured cyber-physical verification progression that systematically tests embodied AI systems across evaluation substrates of increasing physical and temporal realism: Model-in-the-Loop (MIL), Software-in-the-Loop (SIL), Processor-in-the-Loop (PIL), Hardware-in-the-Loop (HIL), and In-Situ Physical Fault Injection. Because each stage closes or substitutes distinct physical and computational feedback loops, higher rungs provide authentic bus contention, electrical transients, and actuator dynamics but sacrifice the combinatorial test throughput of virtual rungs. Comprehensive assurance requires navigating the full ladder rather than relying on any single testing environment. (Chapter 15)

S

Safe Torque Off (STO) A certified hardware safety function (IEC 61800-5-2) that electronically disables pulse-width modulation gate signals to motor inverter power transistors, preventing the generation of electromagnetic rotational torque. Serving as the foundation for Category 0 emergency stops, STO guarantees that no software malfunction, memory corruption, or processor lockup can command motor actuation, leaving the plant to coast or rely on passive mechanical brakes. It serves as the ultimate non-software-dependent hardware fail-safe in the physical AI stack. (Chapter 12, chapter 2)

Safety Case (Claim–Argument–Evidence) A structured, auditable assurance argument that demonstrates how an embodied AI system satisfies its explicit safety requirements within a declared operational design domain. It organizes assurance hierarchically by linking high-level claims to empirical and formal evidence (such as fault injection manifests, HIL qualification results, and barrier proofs) via transparent, falsifiable engineering arguments. In physical AI, the safety case explicitly binds software performance to physical hardware limits and operational envelopes. (Chapter 16)

Shared Autonomy A collaborative control architecture in which a human supervisor and an autonomous policy concurrently exert control authority over an embodied system’s physical degrees of freedom. Authority allocation may be realized via continuous haptic virtual fixtures, dynamic compliance blending, or task-space privilege partitioning. The central challenge of shared autonomy is maintaining an intuitive, transparent authority distribution that prevents human-robot fighting while guaranteeing system safety invariants. (Chapter 14)

Sim-to-Real Contact Exploit An optimization pathology wherein a reinforcement learning policy discovers and exploits unphysical numerical artifacts in a simulator’s contact solver—such as interpenetration spring restitution, infinite friction sticking, or instantaneous impulse relaxation—to achieve high simulated reward. Because these mathematical solver relaxations do not exist in the physical world, the learned policy experiences catastrophic failure immediately upon encountering real Coulomb friction and non-interpenetrable material boundaries. Mitigating contact exploits requires non-smooth contact regularization, randomized contact parameters, and strict energy-passivity constraints during training. (Chapter 6)

Sim-to-Real Gap (Sim-to-Reality Gap) The systematic discrepancy in dynamics, contact mechanics, sensor transduction, and execution latency that exists between a computational physics simulation and the real physical environment. Policies optimized purely in simulation frequently fail upon physical deployment because simulated rigid-body solvers linearize frictional contact, neglect transmission backlash and compliance, and omit high-frequency electrical bus latencies. Bridging this gap is the central challenge of simulation-based policy training in physical AI. (Chapter 6, chapter 7)

Spatial Information Age (t_{age}) The total wall-clock elapsed time between the physical capture epoch of a sensor observation and the exact moment that observation is utilized to make an actuation decision: $t_{\text{age}} = t_{\text{act}} - t_{\text{capture}}$. In dynamic environments, nonzero information age artificially inflates the effective volume of physical obstacles by their maximum drift velocity ($v_{\text{max}} t_{\text{age}}$), penalizing slow inference pipelines with severe spatial clearance degradation. State estimation pipelines must enforce explicit information-age bounds to prevent policies from acting on stale world representations. (Chapter 8, chapter 9)

Stopping Suffix A pre-computed, kinodynamically verified braking trajectory permanently reserved at the terminal tail of an admitted action chunk, designed to bring the physical robot to a controlled, collision-free standstill within available friction and actuator limits. If real-time neural inference stalls, communication drops, or the subsequent action chunk fails kinematic validation, the robot smoothly transitions into the stopping suffix rather than executing open-loop extrapolation or triggering a jarring emergency brake. The stopping suffix ensures that loss of high-level intelligence never compromises low-level physical survival. (Chapter 11, chapter 14)

T

Takeover Reaction Budget (T_{Σ}) The total temporal allocation ($T_{\Sigma} = t_{\text{detect}} + t_{\text{alert}} + t_{\text{cognitive}} + t_{\text{motor}} + t_{\text{plant}}$) required for a human supervisor to detect a silent autonomy failure, evaluate the hazard, physically engage the controls, and command a dynamic maneuver to avert harm before the system enters an Inevitable Collision State. If the physical system’s time-to-harm is shorter than the minimum physiological and mechanical takeover budget, supervisory intervention is mathematically impossible. Systems operating in this regime must rely entirely on autonomous, real-time safety shielding rather than human fail-safe intervention. (Chapter 14)

Temporal Ensembling A real-time trajectory filtering technique for action-chunked policies that computes the final executed motor command as an exponentially weighted average of overlapping action predictions generated at successive observation steps. By continuously blending predictions produced across different historical contexts, temporal ensembling smooths inter-chunk discontinuities and eliminates high-frequency jerk without requiring explicit trajectory optimization. However, heavy ensembling windows introduce effective phase lag, reducing the robot’s peak reactivity during abrupt physical collisions or sudden obstacle appearances. (Chapter 3, chapter 6, chapter 11)

Time-to-Harm (t_{harm}) The minimum physical duration required for a system’s current kinetic state to evolve into an irreversible collision, structural failure, or human injury under worst-case unmodeled dynamics and disturbances. To preserve forward invariance, the permission path must continuously verify that t_{harm} remains strictly greater than the system’s worst-case reaction budget ($t_{\text{detect}} + t_{\text{WCET}} + t_{\text{brake}}$). If t_{harm} contracts below this reaction threshold, the machine has exhausted its decision margin and must initiate emergency braking. (Chapter 17)

Trajectory Spline Blending (C^2 Jerk Continuity) A trajectory interpolation and seam-smoothing method that stitches consecutively admitted action chunks or trajectory segments using quintic polynomials or Hermite splines to enforce continuous position (C^0), velocity (C^1), and acceleration (C^2) across chunk boundaries. An unmanaged step change in acceleration requests an instantaneous step in motor torque ($\tau = M\ddot{q}$), implying infinite reflected jerk that excites mechanical drive compliance, damages transmissions, and triggers inverter

overcurrent faults. Enforcing C^2 continuity guarantees that commanded actuator efforts remain within dynamic bandwidth limits across receding-horizon replanning cycles. (Chapter 11, chapter 3)

V

Vision-Language-Action (VLA) Model An end-to-end multimodal foundation model that consumes visual scene tokens, natural language task prompts, and robot proprioceptive states to directly generate continuous low-level action tokens or trajectory chunks. Built on high-capacity transformer backbones pretrained on Internet-scale vision-language datasets and fine-tuned on robotics demonstrations, VLA models synthesize open-vocabulary semantic reasoning with spatial affordance grounding. Because their internal weights lack physical safety and stability guarantees, their outputs are treated as unprivileged candidate proposals requiring downstream kinodynamic and barrier filtering. (Chapter 3, chapter 10)

W

World Model An internal generative model that learns to simulate the forward transition dynamics, contact mechanics, and sensory observations of the physical environment conditioned on the agent's prospective actions. In physical AI, world models serve as learned digital twins that enable mental simulation, counterfactual trajectory evaluation, and model-predictive rollouts without inducing physical wear or hardware risk. Their reliability is fundamentally constrained by epistemic uncertainty over non-smooth contact bifurcations and the sim-to-real reality gap. (Chapter 3, chapter 11, chapter 6)

Index

- 2018 Tempe Autonomous Collision
 - NTSB report, 18
- 3D Gaussian splatting, 201
- 3D dynamic scene graph, 202
- 3D vision foundation models, 186
- Action agreement, 161
- Action Chunk Representations, 6
- Action chunking
 - architectures, 234
 - definition, 67
 - temporal trajectory, 66
- Action decoder
 - multimodal families, 401
- Action tap
 - definition, 106
- Actuation
 - mechanical resonance, 145
- Actuator
 - Bias torque, 260
 - current limiters, 161
 - electrical rise time, 437
 - intermittent torque, 44
 - Joule heating, 161
 - saturation, 260
 - stator delay, 160
 - stator thermal dynamics, 373, 437
 - torque limit, 161
 - velocity limits, 224
- Actuator authority, 360
- Actuator bandwidth
 - definition, 32
- Actuator saturation, 253, 263
- Actuator thermal derating
 - physics, 242
- Actuator torque limits, 253
- Adaptation wear ledger, 143
- Adaptive Stress-Testing, 375
- Admission gate
 - rejection, 230
- Adversarial review
 - definition, 355
- Adversarial vulnerability
 - perturbation bounds, 71
- Affordance grounding
 - open-vocabulary, 219
- Affordances
 - SayCan value function, 406
 - spatial parameterization, 216
- Ahead-of-time compilation
 - real-time inference, 279
- Alpha-Power CMOS delay law, 283
- ALVINN, 128
- AMR Safety Standards, 208
- Angular jerk
 - electromechanical limits, 438
- ANSI/UL 4600
 - evaluation records, 164
- Antipatterns
 - asynchronous safety
 - cancellation, 217
 - moving average filtering, 85
- Architectural warrants, 360
- Architecture
 - subsumption, 218
- Arithmetic intensity, 8, 381
- ARM MPAM
 - cache partitioning, 281
- Authentication
 - signature verification, 319
- Authority
 - ambiguity, 313
- Authority split, 377
- Automotive safety
 - FFI, 290
- Autonomous Safety Standards, 209
- Back-electromotive force, 39
- Back-EMF, 150
- Bandwidth
 - servo sampling, 241
- Barrier certificates, 253
- Base frame, 197
- Bayes' rule
 - parameter estimation, 392
- Behavioral cloning, 127, 314
 - compounding error, 405
 - error accumulation, 128
 - quadratic error compounding, 104
- Belief, 196
- Belief record
 - definition, 198
- Belief state invalidation horizon
 - definition, 205
- Bimodal ambiguity
 - spatial, 230
- Blending window, 317
- Bode delay theorem, 34
- Bound Checker
 - pre-execution, 245
- Boundary Checker, 245
- Boundary contract, 283
- Brake
 - mechanical, 360
- Braking
 - friction limit, 444
- Braking distance, 34
- Braking dynamics, 89
- Brownout reset
 - undervoltage, 46
- Budget enforcement, 32
- Bumpless transfer, 310
 - definition, 310
 - smoothstep form, 417
- Bumpless transfer braking penalty, 311
- Bus impedance
 - power distribution, 46
- Bus jitter, 184
- Bus latency, 225
- C^2 bumpless authority transfer ramp, 320
 - definition, 310
- C^2 continuity, 235
- C^2 spline seam
 - definition, 240
- C^2 trajectory spline blending, 241
- Cache coloring
 - memory isolation, 281
- Calibration
 - spatial, 192
- Camera calibration
 - absolute conic, 442
- Camera model
 - pinhole projection, 442
- CAN arbitration, 91
- Capability matrix, 303
- Capture timestamp
 - definition, 178
- Cascaded control
 - tracking architecture, 253
- Catastrophic forgetting
 - fine-tuning regression, 141
 - in physical fine-tuning, 139
- Causal boundary, 380
 - definition, 6
 - thermodynamic constraint, 104
- CBF-QP safety filter
 - definition, 262
- Censored trial fallacy
 - definition, 162
- Centroidal momentum
 - Newton-Euler projection, 426
 - Orin matrix, 426
- Chunk lease, 84
 - lease rule, 95
- Claim-argument-evidence safety case
 - argument structure, 347
 - definition, 344
- Claims register, 377
- Clopper-Pearson interval
 - exact binomial inversion, 408
- Clopper-Pearson method, 148
 - sample size bound, 149
- Clopper-Pearson non-asymptotic bound
 - definition, 148
- Closed-loop evaluation
 - definition, 151
- Closed-loop evaluation estimand
 - definition, 150
- Closed-loop stability, 326
- Closed-loop stiffness, 253
- Closure test
 - definition, 369
- Collision avoidance
 - inevitable collision state, 243
- Common-cause analysis
 - definition, 328
- Compounding rollout error
 - definition, 128
- Concurrency
 - sequence locks, 440
- Confidence score, 192
- Configuration space
 - definition, 235
- Connectionist representations
 - statistical approximation, 71
- Conservative Q-Learning, 130
- Constrained reinforcement learning, 255
- Contact constraints
 - unilateral normal force, 426
- Contact fidelity invariant, 135
- Contact friction, 154
- Contact mechanics
 - bifurcation, 152
 - force model, 346
 - non-smooth impact, 372
- Continuous flow matching

- vector field regression, 402
- Control
 - Closed-Loop Stability, 168
 - subsumption, 218
- Control architecture, 315
- Control barrier function, 254, 261, 272
 - definition, 262
 - forward invariance, 252
 - High-order, 262
 - Lie derivatives, 264
 - QP solver, 274
 - quadratic program, 264
 - safety filtering, 413
- Control loop
 - closed-loop stiffness, 253
- Control systems
 - impedance control, 218
- Controller
 - windup, 260
- Copper resistance
 - temperature dependence, 43
- Corrective aggregation, 114
- Coulomb friction cone
 - traction boundary, 426
- Covariance
 - spatial, 212
- Covariance matrix, 206
- Covariance propagation
 - cubic dispersion, 424
- Covariate drift
 - physical AI, 104
- Covariate shift
 - endogenous feedback, 104
 - in behavioral cloning, 140
- Cross-embodiment compatibility
 - contract
 - definition, 115
- Crossbar bus and DRAM bank
 - contention
 - definition, 282
- Cryptographic manifest
 - hardware Root of Trust, 358
- Cubic Hermite spline
 - multi-rate interpolation, 417
- Cyber-physical security
 - shared-die attack surface, 281
 - transduction attacks, 328
- Cybernetic Feedback Loops, 4
- Cybersecurity engineering
 - ISO/SAE 21434, 328
- Cycloidal drive
 - fatigue life, 139
- Cycloidal drive gearbox
 - torsional compliance, 356
- Dagger
 - linear error bound, 114
- Data
 - curation, 319
- Data collection
 - linear scaling fallacy, 123
- Dead-time delay, 80
- Deadlines
 - end-to-end, 340
- Deadman switch, 306
- Dean, Jeff
 - latency numbers, 430
- Defeaters
 - adversarial review, 356
 - definition, 349
- Degraded mode
 - velocity reduction, 353
- Demonstration
 - versus evidence, 347
- Demonstration provenance record
 - definition, 120
- Denominator filtering, 162
- Deployment
 - policy ranking inversion, 144
- Detectable faults, 187
- Detectorless failure
 - indistinguishability, 372
- Deterministic fault logging
 - non-volatile ring buffers, 115
- Diffusion policy
 - score-matching derivation, 402
- Diffusion sampling latency, 403
- Direct memory access
 - cache coherence, 79
- Distributed clock, 87
- Disturbance observer, 263
- Disturbance rejection
 - recovery data, 145
- DMA, 292
- Domain randomization
 - corridor, 138
 - support boundary, 138
- Domain transfer, 356
- Double-integrator
 - covariance expansion, 424
- DRAM, 280
 - bank conflicts, 282
- Drivetrain
 - jerk dynamics, 235
 - resonant frequency, 245, 247
 - torsional resonance, 241
- Dual-stage enforcement pattern, 256
- Dynamic brake chopper
 - regenerative clamp, 47
- Dynamic braking, 255
 - DC bus overvoltage, 267
- Dynamic fallback ladder
 - definition, 269
- Dynamic random-access memory, *see* DRAM
- Dynamic range saturation, 187
- Dynamic reachability, 225
 - reachable set, 420
- Dynamic stopping distance
 - definition, 35
- Dynamic voltage and frequency scaling, 55
- Dynamics
 - non-smooth contact, 205
- Electrical power integrity
 - definition, 32
- Embodied latency scaling law
 - definition, 366
- Embodiment contract
 - definition, 115
- Empirical state-action occupancy
 - distribution, 111
- Empirical sufficiency, 356
- Endogenous covariate shift
 - definition, 104
- Enforcement
 - manual override, 319
 - transfer boundaries, 318
- Enforcement filter, 303
- Enforcement record, 271
- Enforcer, 252
 - tracking contract, 252
- Envelope membership
 - trichotomy states, 352
- Envelope trichotomy
 - definition, 352
- Environment selection, 161
- Epistemic gap
 - kinetic energy accumulation, 352
- Epoch, 199
 - acquisition, 199
 - estimate, 199
 - publication, 199
- Equations of motion
 - manipulator dynamics, 60
- Error expansion, 197
- EU AI Act
 - high-risk physical AI, 348
- Evaluation
 - Action Agreement, 168
 - Binomial Model, 167
 - Confidence Intervals, 168
 - Covariate Shift, 168
 - Fault Injection, 168
 - metric masking, 151
 - Reliability Bounds, 168
- Evaluation deficit, 375
- Evaluation estimand
 - definition, 150
- Evaluation record
 - definition, 164
- Evidence record, 377
- Evidence regimes
 - definition, 153
- Evidence tuple, 315
- Executable trajectory
 - definition, 235
- Execution monitoring
 - collision detection, 247
- Expected Calibration Error
 - binned accuracy metric, 392
 - distribution shift, 71
 - mathematical formulation, 71
- Expiring intent lease
 - definition, 228
- Exposure time, 192
- Exposure wall
 - definition, 157
 - mathematical derivation, 408
 - retraining reset, 368
- Extreme Value Theory
 - Frechet distribution, 410
- Extreme value theory, 156
 - physical disturbances, 156
- FAA
 - drone airworthiness, 348
- Failure modes
 - common-cause, 340
- Failure rate
 - catastrophic, 360
- Fallback ladder, 266
 - definition, 269
 - STOP rung, 353
- Fallback trajectory
 - achievability, 248
 - commitment point, 247
- Falsification
 - Popperian induction, 367
- Falsification threshold
 - definition, 287
- Fatigue limits
 - cyclic derating, 353
- Fault coupling
 - shared resources, 340
- Fault Injection, 292
- Fault injection, 326
 - hardware, 326
- Fault record
 - definition, 338
- Fault Tolerance
 - kernel panic, 274
- Fault tolerance
 - structural bounds, 375
- FDA
 - surgical robotics regulation, 348
- Feasibility
 - kinodynamic, 234
- Feedback control
 - error monitoring, 248
- Feedforward control
 - torque demand, 248
- Field-Oriented Control
 - current loop limits, 39
- Fieldbus
 - CAN-FD arbitration, 439
 - contention, 340
 - EtherCAT on-the-fly, 440
 - synchronization jitter, 199
- Fieldbus write exclusivity, 307
- Filter
 - Kalman, 204
- Fine-tuning
 - on physical hardware, 139
- Flexible joint dynamics
 - Spong model, 418
- Floating-base dynamics
 - equations of motion, 426
- Flux degradation, 379
- Forensic authority record, 317
- Forward invariance, 262
 - Nagumo viability, 414
- Forward kinematics
 - mapping, 419
- Four-phase handshake protocol, 307
- Four-stage qualification ladder
 - definition, 331

- Four-stream action tap
 - definition, 106
- FR-FCFS scheduling
 - definition, 282
- Freedom from interference, 90
 - ISO 26262, 290
- Frozen transform, 208
- Functional safety
 - Emergency stop categories, 267
 - fail-operational plants, 268
 - ISO 26262, 349
 - random hardware metrics, 158
 - versus SOTIF, 326
- Future-state conditioning
 - definition, 239
- Gate delay
 - Sakurai-Newton model, 434
- Gate propagation delay
 - voltage headroom, 283
- Gearbox
 - compliance, 247
- Gearbox compliance, 356
- Generalized Pareto Distribution
 - peaks over threshold, 410
- Global shutter
 - definition, 179
- Goal Structuring Notation, 347
 - argumentation grammar, 347
- GPU
 - memory bandwidth wall, 66
- Grounding
 - ambiguity, 221
 - metric mapping, 219
- Handoff record, 92
- Handover
 - authority register, 318, 319
 - autonomous fallback, 319
 - takeover request, 319
- Handshake protocol latency, 317
- Hardware
 - actuator boundary, 319
 - bus jitter, 160
 - clock synchronization, 192
 - gearbox compliance, 160
 - lidar range walk, 187
 - MIPI CSI-2, 183
 - rolling shutter, 179
 - SerDes Jitter, 168
 - thermal derating, 160
 - thermal expansion, 191
 - timer input-capture, 199
 - Trip zones, 254
 - V4L2 DMA, 183
 - watchdog timer, 217, 223
- Hardware acceleration, 381
- Hardware Architecture
 - common-mode coupling, 271
 - ground bounce, 271
- Hardware security
 - side channels and DoS, 281
- Hardware timer interrupt
 - DMA latching, 106
- Hardware tracing
 - CoreSight ETM, 336
- Hardware trip zone, 89
- Hardware watchdog, 263
- Hardware wear
 - actuator thermal duty cycle, 104
 - bearing life, 131
- Hardware-in-the-loop, 332
 - bus contention, 154
- Harmonic drive, 150
 - fatigue life under RL
 - exploration, 131
- Heartbeat, 84
- Heterogeneous silicon partitioning
 - definition, 280
- Heterogeneous system-on-chip
 - architecture, 278
- Humanoid robotics
 - coupled classes, 12
- Hysteresis
 - state-dependent, 286
- Idempotency, 6
- IEC 60204-1
 - Stopping categories, 267
- IEEE 1588 gPTP
 - hardware timestamping, 34
- IEEE 1588 Precision Time Protocol
 - distributed clock sync, 117
- IEEE 1588 PTP, 192
- Imitation learning
 - expert demonstration, 319
 - expert recovery, 318
- Impedance controller, 218
- Importance Sampling, 375
- In-situ fault injection
 - definition, 335
- In-situ physical fault rigs, 332
- Incidents
 - DARPA Robotics Challenge, 72
 - Williston accident, 189
- Independent enforcement, 380
- Inductive voltage droop
 - power delivery network, 434
 - VLSI design, 278
- Inevitable collision state
 - definition, 243
- Inference latency, 79
- Information age
 - definition, 177
- Initial condition matching, 311
- Injection attack
 - physical, 22
- Innovation, 205
- Innovation sequence, 207
- Input-to-State Stability
 - classical control, 159
- Inset clearance
 - definition, 185
- Integrator windup, 253
- Intent, 216
- Intent grounding refusal
 - definition, 216
- Intent lease
 - definition, 228
 - temporal contract, 65
- Intent record
 - force limits, 218
 - spatial tolerance region, 217
 - temporal lease, 217
- Inter-core communication
 - RPMSG, 227
 - RPMSG framework, 436
- Inter-processor communication
 - RPMSG, 254
 - shared memory, 244
- Interconnect crossbar, 281
- Interconnect crossbar contention, 78
- Interference
 - shared power rail, 293
- Intervention, 303
- Intervention curation
 - definition, 114
- Intervention masking, 314
- Intervention record
 - definition, 115
- Invalid-test rule
 - definition, 334
- Invalidation triggers, 360
- Inverse-depth metric grounding
 - definition, 185
- Inverse-depth parameterization
 - visual odometry, 395
- Inverter
 - desaturation trip, 245
- Inverter dead time
 - shoot-through protection, 40
- IOMMU
 - DMA isolation, 280
- ISO 10218, 151
- ISO 21448
 - SOTIF, 164, 349
- ISO 26262
 - ASIL D, 158
 - Hardware Fault Injection, 339
- ISO/TS 15066, 151
 - biomechanical limits, 346
 - separation distance, 35
- Jerk
 - minimization, 241
- Jerk profile
 - definition, 416
- Joint current, 225
- Joule heating, 150, 225, 253, 267, 379
 - actuator thermal limits, 139
 - component failure, 334
 - fault coupling, 339
- Kinematic boundary violations, 166
- Kinematic reachability
 - intent horizon, 419
- Kinematic roofline, 8
- Kinematic singularity
 - Jacobian rank deficiency, 419
- Kinematic stopping distance
 - observation age, 443
- Kinematic velocity root
 - quadratic clearance, 36
- Kinematics
 - lateral trajectory drift, 151
 - SE(3) calibration, 150
 - SE(3) trajectory, 341
 - semantic-to-metric, 219
 - uncertainty, 192
 - workspace, 225
 - workspace boundaries, 224
 - workspace reachability, 224
- Kinetic momentum
 - definition, 32
- Kinetic stopping distance
 - barrier inset, 415
- Kinodynamic feasibility envelope
 - definition, 234
- Latency
 - bus jitter, 256
 - Stopping distance contribution, 257
- Latency waterfall, 181
- Lead systems adjudicator
 - accountability, 351
- Lease
 - evidence epoch, 229
 - spatial invalidation, 230
- Least-squares projection, 267
- Legal liability
 - operator negligence versus strict liability, 348
- Lever-arm effect
 - apparent acceleration, 420
- Lie derivative
 - directional clearance decay, 414
- Lie derivatives, 208
- Lifecycle degradation
 - definition, 51
- Lift-Splat-Shoot
 - bird's-eye-view lifting, 394
- Limit record
 - definition, 51
- Linear Complementarity Problem
 - contact physics, 134
- Linear complementarity problem
 - contact mechanics, 422
- Lipschitz continuity
 - policy sensitivity, 71
 - spectral normalization, 400
- Local checks
 - authority withdrawal, 229
- Lock-free seqlock buffer
 - definition, 82
- Lockstep processor
 - fault detection, 280
- Log-odds transformation
 - occupancy mapping, 397
- Loud failure, 208
- Lyapunov differential equation
 - covariance divergence, 424
- Lyapunov function
 - stability certification, 159
- Machine Learning
 - 3D Gaussian splatting, 396
 - geometric uncertainty, 193

- Manipulability index
 - Yoshikawa formulation, 419
- Manipulator Jacobian
 - differential kinematics, 419
- Manual lease, 308
- Mathematics
 - covariance propagation, 186, 442
 - cubic variance expansion, 205
 - inverse-depth, 185
- Measurement freshness
 - definition, 32
- Mechanical backlash, 149, 154
- Mechanical Systems
 - mechanical slip, 271
- Memory
 - DMA, 183
- Memory bandwidth
 - worst-case execution time, 293
- Memory barrier
 - instruction ordering, 436
- Memory contention
 - DRAM bank conflicts, 435
- Memory controller, 281
- Memory hierarchies, 381
- Memory Protection Unit
 - spatial isolation, 280
- Memory-mapped I/O, 280
- MEMS Acoustic Resonance, 22
- Metric drift, 161
- Metric grounding
 - definition, 184
- Metric spatial grounding, 65
- Metric unprojection
 - camera intrinsics, 393
- Microcontroller
 - bare-metal, 294
- Microprocessor, 186
- Minimal intervention, 264
- Minimal Risk Maneuver
 - fail-operational systems, 268
- Minimal risk maneuvers, 319
- Minimal-intervention QP projection
 - (safety shield), 264
- MIPI CSI-2
 - DMA buffers, 186
 - host ingress, 183
 - sensor readout interface, 34
- Missing observable
 - definition, 369
- Mode averaging
 - appendix refresher, 400
 - mathematical derivation, 128
- Mode confusion, 313
- Model calibration
 - binned calibration error, 71
- Model Predictive Control
 - receding horizon, 236
 - tracking layer, 253
- Moravec's Paradox, 4
- Moravec's paradox, 4
- Motor
 - Joule heating, 247
 - thermal derating, 248
- Multi-rate hierarchy
 - definition, 186
- Multicore interference
 - FAA AC 20-193 / CAST-32A, 281
- Mutex
 - inter-processor blocking, 293
- Nagumo theorem
 - contingent cone, 414
 - Viability kernel, 262
- Nagumo viability theorem
 - boundary condition, 414
- Negative event registry, 143
- Negative evidence, 202
- Nervous system
 - architecture, 78
 - gatekeeper, 235
- Network
 - autoregressive, 204
 - availability, 319
 - bus latency, 247
 - rate limiting, 319
- Networking
 - heartbeat monitor, 274
- Neural network
 - Lipschitz continuity, 219
- Non-smooth mechanics
 - Moreau time-stepping, 422
- Nonlinear control
 - Lie derivatives, 264
- Normalization of deviance, 357
 - Rogers Commission, 357
- Object-centric 3D scene graph
 - definition, 186
- Observability
 - Hermann-Krener criterion, 426
 - Kalman rank criterion, 425
- Observability matrix, 208
- Observable, 207
- Observation
 - freshness limit, 192
- Observation contract
 - definition, 190
- Observational indistinguishability, 379
 - definition, 372
- Occlusion covariance expansion, 204
- Occupancy grid mapping, 201
 - octree storage, 397
- Offline reinforcement learning, 130
- Open-loop evaluation
 - closed-loop gap, 152
 - definition, 151
- OpenAI Dactyl
 - system identification, 134
- Operating envelope
 - definition, 325
 - release scope, 361
 - trichotomy, 352
- Operational design domain, 375
 - definition, 111
 - release verdicts, 345
 - target ODD, 150, 166, 168
 - training boundaries, 141
- Operational independence
 - definition, 278
- Operational response
 - definition, 369
- Operator fusion
 - thermodynamic efficiency, 279
- Operator reaction budget, 317
- Optional stopping, 162
- Oracle
 - definition, 334
- Out-of-distribution action
 - overestimation, 130
- Output selector, 254, 264
- Overconfident predictor, 209
- Overvoltage protection
 - dynamic braking, 47
- Painlevé paradox, 134
- Paired probing
 - simulator validation, 135
- Passive compliance, 380
- Passivity
 - energy storage, 427
- Passivity theorem
 - feedback stability, 427
- Patch tokenization
 - vision transformers, 393
- Perception, 191
 - calibration drift, 193
 - dated snapshots, 193
 - local loop, 230
 - observation age, 192
 - observation contract, 190
 - pipeline latency, 191
 - silent failure, 193
 - timestamp integrity, 192
- Perceptual chattering
 - state reset hazard, 72
- Permission path, 357
 - definition, 17
 - overrides, 166
- Phantom-object tracks, 209
- Phase margin
 - delay degradation, 34
 - transport delay penalty, 419
- Phase-locked loop
 - shared dependency, 293
- Photon shot noise, 187
- Physical AI
 - definition, 6
- Physical Artificial Intelligence
 - definition, 6
- Physical body
 - definition, 32
- Physical bounds, 376
- Physical Wall, 21
- Piezoresistive transducer
 - drift, 372
- Pinhole camera model, 180
- Placement record
 - definition, 291
- Planning
 - STRIPS, 216
- Plant mutation during training, 131
- Policy learning
 - temporal horizons, 234
- Policy manifest
 - definition, 141
 - omission log, 141
- Policy vulnerability inference chain, 140
- Power Distribution Network
 - inductive voltage droop, 278
- Power electronics
 - Brake chopper, 267
- Power-rail coupling, 286
- Practical detector
 - definition, 371
- Practical Engineering Question, 399
- Pre-trigger ring buffer, 143
- Precision Time Protocol
 - clock synchronization, 435
 - hardware timestamping, 436
- Precision time protocol logging, 315
- Preempt RT
 - kernel latency, 280
- Preemption tier, 317
- Priority inversion, 271
- Probabilistic roadmap, 236
- Probability calibration
 - algebraic normalization, 71
- Processor-in-the-loop, 332
- Product liability
 - strict liability in physical AI, 348
- Proposal boundary
 - budget enforcement, 32
 - definition, 17
 - demonstration data, 105
 - goal filtering, 224
 - inter-core communication, 227
 - unprivileged proposals, 64
- Proposal header, 84
- Protective intervention, 326
- Provenance, 324
- Proxy metric fallacy, 356
- Pulse-width modulation
 - dead-time insertion, 434
 - Hardware clamp, 254
- Quadratic program
 - closed-form projection, 415
- Quadratic programming, 254, 272
 - active-set solver, 252
 - Worst-case execution time, 264
- Qualification ladder
 - definition, 331
- Quantile convolution
 - tail non-additivity, 33
- Quantitative robustness semantics
 - definition, 334
 - intuition, 334
- Quantization
 - bus bandwidth multiplier, 280
- Quintic spline
 - analytical coefficients, 416
 - boundary solution, 416
 - Vandermonde system, 416
- Rapidly-exploring random tree, 236
- Rate-monotonic scheduling, 86
- Raymond Goertz
 - bilateral telemanipulation, 105

- Real-time systems
 - memory barriers, 244
 - worst-case execution time, 433
- Reality gap
 - definition, 132
- Recoverable set, 22
 - forward invariance, 22
- Recovery handshake, 91
- Reference checker
 - kinematic, 255
- Reference frame
 - definition, 178
- Reflected inertia
 - definition, 32
 - gearbox amplification, 354
- Reflected rotor inertia
 - definition, 40
 - kinetic energy derivation, 438
- Refusal
 - confident grounding, 230
 - grounding ambiguity, 216
- Regenerative braking
 - chopper resistor, 439
- Regulatory compliance
 - EU AI Act, 348
- Reinforcement learning
 - regime transition failures, 145
- Relative degree, 262
 - definition, 415
 - Inertia-aware certificates, 262
- Release claim, 377
- Release manifest
 - definition, 358
- Release verdict
 - conditional, 353
 - invalidation, 353
- Remote Processor Messaging
 - Shared SRAM leases, 254
- Reset controller
 - isolation, 293
- Residual risk, 367
- Residual-claims register, 374
 - definition, 369
- Resource contention
 - worst-case, 288
- Reward hacking, 131
- Riccati equation
 - covariance propagation, 424
- Rigid body transformations, 185
- Rigid Contact Mechanics
 - complementarity, 12
- Rigid-body dynamics
 - Euler-Lagrange formulation, 60
- Rolling shutter
 - definition, 179
- Rolling-shutter motion compensation
 - definition, 180
- Rule of Three, 148
- Rule of three
 - zero-failure testing, 407
- Runtime monitor, 245
- Runtime monitoring, 169
 - constraint margin, 161
 - environmental assumptions, 161
 - execution latency, 161
 - intervention rate, 161
 - OOD detection, 161
- Runtime safety enforcement, 149
- Safe set, 261
- Safe stop 1, 267
- Safe stop 2, 267
- Safe torque off, 90, 450
- Safety
 - admissible motion, 319
 - Category 1 Stop, 168
 - deterministic terminal state, 224
 - envelope, 192
 - Freshness Lease, 168
 - Lipschitz bounds, 219
 - loop delay, 191
 - Operational Envelope, 168
 - physical zoning, 161
 - stopping margin, 318
- Safety Case, 292
- Safety case
 - claim-argument-evidence, 361
 - lifecycle, 360
 - physical AI, 344
- Safety enforcement, 303
 - execution buffer, 235
- Safety enforcer
 - hardware interlock, 353
- Safety filter
 - barrier certificate, 252
- Safety of the Intended Functionality
 - ISO 21448, 326
- Safety shield, 264
 - control barrier function, 256
 - sub-millisecond QP, 252
- Safety standards
 - ISO 13849, 217
- Safety-Critical Standards
 - autonomous systems, 20
- Scaling laws
 - embodied latency, 366
- Scenario ledger, 111
- Schema-detectable frame mismatch, 209
- Scope mismatch, 375
- SE(3), 224
 - spatial bounds, 230
 - target precision, 230
- SE(3) Kinematics, 225, 231
- SE(3) kinematics, 163, 185
 - error propagation, 186
- SE(3) pose, 149
- Self-attention
 - multimodal token sequence, 393
- Self-Monitoring, 375
- Self-watchdog, 84
- Semantically wrong frame assignment, 209
- Sensing
 - phase lag, 271
 - thermal bias drift, 271
- Sensor
 - IMU, 274
 - optical encoder, 274
- Sensor fusion
 - informational contract, 197
- Sensor read noise, 187
- Sensor spoofing
 - physical injection, 328
- Sequence lock, 82
 - shared memory, 82
- Sequence lock (seglock), 293
- Sequential Probability Ratio Test
 - Wald formulation, 409
 - Wald optimality, 409
- Sequential probability ratio test, 162
- SerDes
 - link errors, 356
 - sensor links, 7
- Set invariance
 - Bouligand cone, 414
 - Subtangentiality, 262
- Shannon entropy
 - information theory, 392
- Signal Temporal Logic
 - definition, 334
 - quantitative robustness, 407
 - syntax and semantics, 407
- Silent failure, 208
 - definition, 187
- Silicon aging
 - timing margins, 289
- Silicon contract, 381
- Silicon partitioning
 - definition, 280
- Sim-to-real contact exploit, 134
- Sim-to-real gap, 380
 - physical vector decomposition, 136
- Sim-to-real reality gap vector
 - definition, 132
- Simplex architecture
 - Advanced controller, 254
 - Baseline controller, 254
 - Safety manager, 254
- Simplex architecture pattern, 254
- Simulation
 - timestep bounds, 143
- Simulation-to-reality leap, 356
- Singular perturbation
 - elastic joints, 418
- Slip ratio
 - longitudinal tire kinematics, 423
- Smoothstep polynomial
 - jerk bounding, 417
- Softmax normalization
 - epistemic limitations, 71
- Software Architecture
 - real-time operating system (RTOS), 271
- Software-in-the-loop, 332
- SOTIF
 - ISO 21448, 326
- Spatial belief record
 - definition, 198
- Spatial coordinate frames
 - SE(3) representation, 392
 - SE(3) transformation, 420
- Spatial information age
 - definition, 177
- Spatial memory, 196, 212
- Spatial partitioning, 290
- Special Euclidean group
 - SE(3) grounding, 65
- Special Euclidean Group $SE(3)$, 199
- Speed and Separation Monitoring, 35
- SPRT
 - sequential testing, 162
- Standards
 - REP 105, 178
- State change
 - irreversible, 376
- State estimation, 204
 - Bayesian provenance, 204
 - belief state, 196
 - contract boundary, 197
 - propagation, 192
- State marginals
 - discounted visitation, 406
- State of charge (SoC), 150
- Static memory allocation
 - real-time inference, 279
- Static reservation, 290
- Statistical power
 - sample size, 163
- Statistical verification
 - collapse under fine-tuning, 139
- Stator resistance
 - thermal coefficient, 43
- Stopping distance, 360
 - definition, 35
- Stopping envelope, 260
 - Anisotropy, 257
 - Kinematic distance, 257
 - Margin loss, 260
- Streaming ring-buffer attention, 203
- STRIPS planning, 216
- Structural independence, 356
- Structural Isolation, 375
- Subsumption architecture, 218, 266
- Supervisory Control Levels, 303
- Support envelope relational join
 - definition, 111
- Survival envelope
 - definition, 325
- Survivorship bias
 - in physical datasets, 123
- Symbol Grounding Problem, 216
- Symbol grounding problem, 216
- Symbolic logic
 - deterministic safety enforcement, 71
- System-on-chip
 - resource contention, 356
- System-on-Chip (SoC)
 - heterogeneous compute, 278
- Systems-Theoretic Accident Model and Processes
 - definition, 327
- Systems-Theoretic Process Analysis
 - emergent hazards, 327
- Takeover reaction floor, 303
- Teleoperation

- demonstration yield, 109
- kinematic asymmetry, 123
- latency bound, 420
- Tempe fatal collision
 - operator monitoring, 313
- Temporal action chunking
 - definition, 67
- Temporal ensembling
 - trajectory smoothing, 67
- Temporal logic falsification
 - definition, 334
- Test set contamination, 162
- Testing
 - Oracle, 324
- Theoretical deficit, 375
- Thermal capacity
 - definition, 32
- Thermal clock throttling
 - definition, 285
- Thermal derating, 225, 260
- Thermal management
 - Joule heating, 145
- Thermal resistance
 - junction-to-ambient, 286
- Thermal time constant, 286
- Thermodynamic Limits of Computation, 9
- Threat model, 367
- Three-position enabling devices, 306
- Tightly coupled memory, 293
- Time-sensitive networking, 88
- Time-to-harm
 - definition, 371
- Time-to-Intervention
 - metric, 153
- Timestamp
 - asynchronous, 212
- Timing anomalies
 - worst-case execution time, 287
- Tolerance region
 - 3D spatial, 217
- Tool center point, 150
- Tool flange frame, 197
- Torque-speed curve, 150
- Track association, 192
- Tracking error
 - disturbance bounds, 253
- Tracking error bound, 253
- Tracking history
 - classification changes, 18
- Trajectory
 - braking time, 248
- Trajectory distribution
 - closed loop, 406
- Trajectory Generation
 - jerk-limited splice, 224
- Trajectory optimization
 - collocation and MPC, 236
 - spline synthesis, 416
- Trajectory record
 - definition, 235
- Translation lookaside buffer, 79
- shutdown, 79
- Transport lag, 253
- Trial budget
 - wall-clock cost, 156
- Triple buffer
 - wait-free mailbox, 83
- Truncated signed distance field, 201
- projective fusion, 398
- Two-processor implementation
 - fault injection, 335
- Two-speed brain architecture
 - hybrid control, 58
- Uncertainty
 - covariance expansion, 230
 - invalidation horizon, 205
- Uncertainty bound, 197
- Uncertainty envelope
 - definition, 178
- Unmeasured observable, 374
- Unmonitored state drift, 356
- Unobservable subspace, 207
- Unprivileged proposal
 - proposal boundary, 64
- Validation
 - data sanitization, 144
- Velocity clamp
 - kinetic derating, 369, 444
- Verdicts
 - conditional operation, 350
 - refusal to operate, 350
 - unconditional operation, 350
- Verification gap
 - definition, 327
- Virtual spring-damper system
 - definition, 41
- Vision-language model, 224, 225
- Vision-language-action model, 67, 372
- Voltage droop, 288
 - gate delay, 293
- Volumetric free-space raycasting
 - 3D-DDA traversal, 397
- Vulnerability analysis
 - training regimes, 140
- Wait-free data structures
 - sequence locks, 293
- Wald confidence interval
 - asymptotic breakdown, 408
- Warrant
 - inferential rule, 347
- Watchdog interlock
 - hardware safety, 353
- Watchdog lease, 84
 - bound, 91
 - kinematic derivation, 420
- Watchdog timer
 - challenge-response, 285
 - hardware, 281
 - hardware downcounter, 433
 - windowed, 284
- Wave variables
 - time delay passivity, 428
- WCET
 - definition, 288
- Whole-body control
 - hierarchical QP, 427
- Windowed watchdog timer
 - hardware mechanics, 284
- Workload reduction, 290
- Worst-case execution time
 - Active-set QP, 264
 - definition, 288
 - multicore certification, 281
 - timing anomalies, 287
- Zero Moment Point
 - Vukobratovic criterion, 426
- Zero-allocation static architecture, 86
- Zero-command floor, 255
- Zero-failure testing
 - independence, 339
- Zero-order hold, 81
 - discrete sampling, 419
- Zhang's calibration
 - homography matrix, 442